

**Alex's Anthology of Algorithms**  
**Common Code for Contests in Concise C++**

*v1.0, August 20, 2026*

Alex Li

© 2026. All rights reserved.

This page is intentionally left blank.

# Preface

Browse the latest version online at: [algorithms.alexli.ca](http://algorithms.alexli.ca)

Contributions are always welcome on GitHub: [github.com/alxli/algorithm-anthology](https://github.com/alxli/algorithm-anthology)

## Introduction

---

Welcome to a practical collection of concise C++ implementations of algorithms, data structures, and contest utilities. This is not a first textbook; it is a codebook about implementation details, interfaces, edge cases, and the choices that make familiar algorithms usable in real code. Use it:

- as a reference when you know an algorithm conceptually but want a clean implementation;
- as a printed codebook for contests that allow prepared material, including ICPC-style events;
- as a checklist for edge cases, complexity, overflow, indexing, and API choices;
- as a map of related techniques: slower versions, faster versions, variants, and common reductions sit near one another.

For learning, do not start by copying. Study the idea, prove the invariant or recurrence, implement it yourself, then compare; the differences reveal the practical choices. Each section gives only enough theory to orient the implementation: problem, interface, assumptions, and complexity. Source files are organized as documentation, reusable implementation, then example usage. The reusable part ends at the **Example Usage** marker; everything after exists only so the file compiles and demonstrates itself. When adapting a section, drop the example block and keep the implementation above it.

Guiding principles:

- *Clarity*: A reader familiar with the algorithm should follow the code without reverse-engineering it. Tricks, non-obvious invariants, overflow risks, and language assumptions are documented.
- *Concision*: Contest code is found, read, adapted, and typed under pressure. Short code helps when it removes noise, not when it hides the idea. So the code here is often more verbose than compact ICPC snippets; the extra lines buy clearer names, configurable types, safer defaults, reset functions, explicit handling of variants, and more.
- *Efficiency*: Expected asymptotic behavior and reasonable constants. Both simpler and more robust versions are often included, but educational-only sections are labeled so.
- *Adaptability*: Interfaces make common changes easy, such as swapping a numeric type, changing a combine function, exposing an edge ID, or choosing whether boundaries are included.
- *Consistency*: Similar structures expose similar operations, with repeated naming and example styles, so moving between chapters is easy.

- *Portability*: Targets modern C++17 and avoids unnecessary dependencies; the few contest-useful exceptions are noted at each use.

This began as a personal contest codebook, so it favors single-file snippets, predictable APIs, compact examples, and low constants. Use it actively: annotate it, delete what you dislike, rename functions to match your habits, and stress-test anything you intend to trust. Cheers!

## Portability Notes

---

Examples target a modern GCC or Clang environment with C++17. A typical compile command:

```
g++ -std=c++17 -O2 -Wall -pedantic
```

Most code assumes common contest-platform type sizes: 8-bit `char`, 32-bit `int` and `float`, 64-bit `double` and `long long`. `long double` width and precision are implementation-dependent; sections relying on it say so.

The code targets ISO C++17 unless noted. The most common exceptions:

- GCC/Clang integer extensions like `__int128` or `__uint128_t`, usually guarded with a portable fallback when overflow safety matters.
- Compiler builtins like `__builtin_popcount()` and `__builtin_clz()` — convenient and widely available, though C++20's `<bit>` header standardizes many of them. See <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html> for GCC's reference.
- GNU policy-based data structures like `__gnu_pbds::tree`, used in a few sections to show useful non-standard components.
- A few documented representation assumptions, such as `signbit_()` in the math utilities assuming an IEEE-like floating-point sign bit layout.

# Contents

|                                                    |          |
|----------------------------------------------------|----------|
| <b>Preface</b>                                     | <b>i</b> |
| <b>1 Elementary Algorithms</b>                     | <b>1</b> |
| 1.1 Array Transformations . . . . .                | 1        |
| 1.1.1 Sorting Algorithms . . . . .                 | 1        |
| 1.1.2 Partitioning and Selection . . . . .         | 9        |
| 1.1.3 Array Rotation . . . . .                     | 12       |
| 1.1.4 Coordinate Compression . . . . .             | 13       |
| 1.1.5 Counting Inversions . . . . .                | 16       |
| 1.2 1D Scan and Window Techniques . . . . .        | 18       |
| 1.2.1 Prefix Sums . . . . .                        | 18       |
| 1.2.2 Difference Array . . . . .                   | 20       |
| 1.2.3 Two Pointers and Sliding Window . . . . .    | 22       |
| 1.2.4 Monotonic Stack . . . . .                    | 26       |
| 1.2.5 Aggregate Queue . . . . .                    | 28       |
| 1.3 Dynamic Programming . . . . .                  | 31       |
| 1.3.1 Maximum Subarray Sum (Kadane) . . . . .      | 31       |
| 1.3.2 Longest Increasing Subsequence . . . . .     | 34       |
| 1.3.3 0-1 Knapsack . . . . .                       | 35       |
| 1.3.4 Unbounded Knapsack and Coin Change . . . . . | 36       |
| 1.3.5 Matrix Chain Parenthesization . . . . .      | 38       |
| 1.3.6 Digit DP . . . . .                           | 40       |
| 1.3.7 Egg Dropping . . . . .                       | 41       |
| 1.4 Grid Algorithms . . . . .                      | 42       |
| 1.4.1 Grid Transformations . . . . .               | 42       |
| 1.4.2 Grid Traversal . . . . .                     | 44       |
| 1.4.3 Grid DP . . . . .                            | 48       |
| 1.5 Greedy and Scheduling . . . . .                | 50       |
| 1.5.1 Fractional Knapsack . . . . .                | 50       |
| 1.5.2 Interval Scheduling . . . . .                | 51       |

|          |                                               |            |
|----------|-----------------------------------------------|------------|
| 1.5.3    | Weighted Interval Scheduling . . . . .        | 52         |
| 1.5.4    | Interval Merging and Covering . . . . .       | 54         |
| 1.5.5    | Interval Partitioning . . . . .               | 55         |
| 1.5.6    | Deadline Scheduling . . . . .                 | 57         |
| 1.5.7    | Minimum Late Jobs (Moore-Hodgson) . . . . .   | 58         |
| 1.5.8    | Weighted Completion Time . . . . .            | 59         |
| 1.6      | Streaming and Randomized Algorithms . . . . . | 61         |
| 1.6.1    | Fisher-Yates Shuffle . . . . .                | 61         |
| 1.6.2    | Reservoir Sampling . . . . .                  | 61         |
| 1.6.3    | Frequent Elements . . . . .                   | 64         |
| 1.6.4    | Online Statistics . . . . .                   | 65         |
| 1.6.5    | Online Median . . . . .                       | 69         |
| 1.6.6    | Probabilistic Sketches . . . . .              | 72         |
| 1.7      | Bit Manipulation . . . . .                    | 76         |
| 1.7.1    | Bit Operations and Masks . . . . .            | 76         |
| 1.7.2    | Subset and Submask Iteration . . . . .        | 78         |
| 1.7.3    | Bitmask DP . . . . .                          | 80         |
| 1.7.4    | Gray Code . . . . .                           | 83         |
| 1.7.5    | XOR Basis . . . . .                           | 84         |
| 1.7.6    | Binary Trie . . . . .                         | 86         |
| 1.7.7    | Bitwise Convolution (FWHT) . . . . .          | 89         |
| 1.8      | Classic Problems . . . . .                    | 93         |
| 1.8.1    | Minimum Excluded Value . . . . .              | 93         |
| 1.8.2    | Josephus Problem . . . . .                    | 95         |
| 1.8.3    | Tower of Hanoi . . . . .                      | 97         |
| 1.8.4    | Cycle Detection (Floyd) . . . . .             | 98         |
| 1.8.5    | Subset Sum . . . . .                          | 100        |
| 1.8.6    | N-Queens . . . . .                            | 103        |
| 1.8.7    | Knights Tour . . . . .                        | 104        |
| 1.8.8    | Exact Cover (Dancing Links) . . . . .         | 105        |
| <b>2</b> | <b>Data Structures</b>                        | <b>110</b> |
| 2.1      | Linked Lists . . . . .                        | 110        |
| 2.1.1    | Singly Linked List . . . . .                  | 110        |
| 2.1.2    | Doubly Linked List . . . . .                  | 112        |
| 2.1.3    | LRU Cache . . . . .                           | 115        |
| 2.1.4    | LFU Cache . . . . .                           | 116        |
| 2.2      | Binary Trees . . . . .                        | 119        |
| 2.2.1    | Binary Tree Traversal . . . . .               | 119        |

|        |                                                 |     |
|--------|-------------------------------------------------|-----|
| 2.2.2  | Binary Tree Encodings . . . . .                 | 122 |
| 2.2.3  | Binary Tree Queries . . . . .                   | 125 |
| 2.3    | Heaps and Priority Queues . . . . .             | 130 |
| 2.3.1  | Binary Heap . . . . .                           | 130 |
| 2.3.2  | Leftist Heap . . . . .                          | 132 |
| 2.3.3  | Skew Heap . . . . .                             | 135 |
| 2.3.4  | Pairing Heap . . . . .                          | 137 |
| 2.4    | Dictionaries and Ordered Sets . . . . .         | 141 |
| 2.4.1  | Treap . . . . .                                 | 141 |
| 2.4.2  | AVL Tree . . . . .                              | 145 |
| 2.4.3  | Red-Black Tree . . . . .                        | 149 |
| 2.4.4  | Splay Tree . . . . .                            | 154 |
| 2.4.5  | Size Balanced Tree . . . . .                    | 158 |
| 2.4.6  | Interval Treap . . . . .                        | 162 |
| 2.4.7  | B-Tree . . . . .                                | 167 |
| 2.4.8  | Skip List . . . . .                             | 172 |
| 2.4.9  | Hash Table (Chaining) . . . . .                 | 175 |
| 2.4.10 | Hash Table (Open Addressing) . . . . .          | 179 |
| 2.5    | Range Queries in One Dimension . . . . .        | 184 |
| 2.5.1  | Sparse Table . . . . .                          | 184 |
| 2.5.2  | Disjoint Sparse Table . . . . .                 | 186 |
| 2.5.3  | Square Root Decomposition . . . . .             | 187 |
| 2.5.4  | Mo's Algorithm . . . . .                        | 189 |
| 2.5.5  | Segment Tree (Point Update) . . . . .           | 191 |
| 2.5.6  | Segment Tree (Range Update) . . . . .           | 194 |
| 2.5.7  | Iterative Segment Tree (Point Update) . . . . . | 198 |
| 2.5.8  | Iterative Segment Tree (Range Update) . . . . . | 201 |
| 2.5.9  | Segment Tree Beats . . . . .                    | 205 |
| 2.5.10 | Sparse Segment Tree . . . . .                   | 209 |
| 2.5.11 | Persistent Segment Tree . . . . .               | 213 |
| 2.5.12 | Merge Sort Tree . . . . .                       | 216 |
| 2.5.13 | Wavelet Tree . . . . .                          | 218 |
| 2.5.14 | Cartesian Tree . . . . .                        | 220 |
| 2.5.15 | Implicit Treap . . . . .                        | 221 |
| 2.6    | Range Queries in Two Dimensions . . . . .       | 227 |
| 2.6.1  | 2D Segment Tree . . . . .                       | 227 |
| 2.6.2  | Sparse 2D Segment Tree . . . . .                | 229 |
| 2.6.3  | 2D Range Tree . . . . .                         | 233 |

|          |                                          |            |
|----------|------------------------------------------|------------|
| 2.6.4    | K-d Tree (2D Range Reporting)            | 235        |
| 2.6.5    | K-d Tree (Nearest Neighbor)              | 237        |
| 2.7      | Fenwick Trees                            | 239        |
| 2.7.1    | Fenwick Tree (Point Update, Range Query) | 239        |
| 2.7.2    | Fenwick Tree (Range Update, Point Query) | 241        |
| 2.7.3    | Fenwick Tree (Range Update, Range Query) | 242        |
| 2.7.4    | Sparse Fenwick Tree                      | 244        |
| 2.7.5    | 2D Fenwick Tree                          | 247        |
| 2.7.6    | Sparse 2D Fenwick Tree                   | 248        |
| 2.7.7    | Offline 2D Fenwick Tree                  | 250        |
| 2.8      | Disjoint Sets and Tree Structures        | 253        |
| 2.8.1    | Disjoint Set Union                       | 253        |
| 2.8.2    | Sparse Disjoint Set Union                | 254        |
| 2.8.3    | Rollback Disjoint Set Union              | 256        |
| 2.8.4    | Lowest Common Ancestor (Binary Lifting)  | 258        |
| 2.8.5    | Lowest Common Ancestor (Euler Tour)      | 260        |
| 2.8.6    | Virtual Tree                             | 263        |
| 2.8.7    | Weighted Tree Path Queries               | 266        |
| 2.8.8    | Heavy-Light Decomposition                | 269        |
| 2.8.9    | Euler Tour Tree                          | 273        |
| 2.8.10   | Link-Cut Tree                            | 278        |
| 2.8.11   | Top Tree                                 | 283        |
| <b>3</b> | <b>Strings</b>                           | <b>293</b> |
| 3.1      | String Utilities                         | 293        |
| 3.2      | Hashing                                  | 300        |
| 3.2.1    | Hash Functions                           | 300        |
| 3.2.2    | Rolling Hash                             | 305        |
| 3.2.3    | Rolling Hash Deque                       | 308        |
| 3.2.4    | Dynamic Rolling Hash                     | 311        |
| 3.2.5    | Zobrist Hashing                          | 314        |
| 3.3      | Pattern Matching                         | 315        |
| 3.3.1    | String Searching (KMP)                   | 315        |
| 3.3.2    | String Searching (Z Algorithm)           | 316        |
| 3.3.3    | String Searching (Boyer-Moore)           | 318        |
| 3.3.4    | String Searching (Aho-Corasick)          | 320        |
| 3.3.5    | Palindrome Searching (Manacher)          | 322        |
| 3.3.6    | Lyndon Factorization (Duval)             | 324        |
| 3.3.7    | De Bruijn Sequence                       | 326        |



|          |                                                |            |
|----------|------------------------------------------------|------------|
| 3.4      | Suffix Arrays and LCP . . . . .                | 327        |
| 3.4.1    | Suffix Array and LCP (Manber-Myers) . . . . .  | 327        |
| 3.4.2    | Suffix Array and LCP (Counting Sort) . . . . . | 329        |
| 3.4.3    | Suffix Array and LCP (DC3) . . . . .           | 331        |
| 3.5      | String Data Structures . . . . .               | 334        |
| 3.5.1    | Trie . . . . .                                 | 334        |
| 3.5.2    | Radix Tree . . . . .                           | 338        |
| 3.5.3    | Suffix Tree (Ukkonen) . . . . .                | 343        |
| 3.5.4    | Suffix Automaton . . . . .                     | 346        |
| 3.5.5    | Eertree . . . . .                              | 349        |
| 3.6      | Sequence Dynamic Programming . . . . .         | 351        |
| 3.6.1    | Longest Common Substring . . . . .             | 351        |
| 3.6.2    | Longest Common Subsequence . . . . .           | 352        |
| 3.6.3    | Edit Distance and Sequence Alignment . . . . . | 354        |
| 3.6.4    | Wildcard Matching . . . . .                    | 357        |
| 3.6.5    | Palindrome DP . . . . .                        | 359        |
| 3.7      | Expression Parsing . . . . .                   | 360        |
| 3.7.1    | Recursive Descent Parsing . . . . .            | 360        |
| 3.7.2    | Shunting Yard and Postfix Evaluation . . . . . | 361        |
| 3.7.3    | Pratt Parsing . . . . .                        | 365        |
| 3.8      | Encoding and Compression . . . . .             | 367        |
| 3.8.1    | Entropy and Frequency Tables . . . . .         | 367        |
| 3.8.2    | Run-Length Encoding . . . . .                  | 370        |
| 3.8.3    | Base64 Encoding . . . . .                      | 371        |
| 3.8.4    | Huffman Coding . . . . .                       | 372        |
| 3.8.5    | Burrows-Wheeler Transform . . . . .            | 374        |
| <b>4</b> | <b>Graphs</b>                                  | <b>377</b> |
| 4.1      | DFS and Trees . . . . .                        | 377        |
| 4.1.1    | Adjacency Lists and DFS . . . . .              | 377        |
| 4.1.2    | Topological Sort . . . . .                     | 379        |
| 4.1.3    | Cycle Finding . . . . .                        | 381        |
| 4.1.4    | Tree Centers, Centroid, and Diameter . . . . . | 386        |
| 4.1.5    | Subtree Small-to-Large Merging . . . . .       | 388        |
| 4.1.6    | Tree Rerooting DP . . . . .                    | 389        |
| 4.1.7    | Centroid Decomposition . . . . .               | 392        |
| 4.1.8    | Prufer Code . . . . .                          | 393        |
| 4.1.9    | Eulerian Trails (Hierholzer) . . . . .         | 395        |
| 4.2      | Connectivity . . . . .                         | 400        |

|        |                                                                         |     |
|--------|-------------------------------------------------------------------------|-----|
| 4.2.1  | Connected Components . . . . .                                          | 400 |
| 4.2.2  | Strongly Connected Components (Kosaraju) . . . . .                      | 402 |
| 4.2.3  | Strongly Connected Components (Tarjan) . . . . .                        | 404 |
| 4.2.4  | Dominator Tree (Lengauer-Tarjan) . . . . .                              | 406 |
| 4.2.5  | Articulation Points, Biconnected Components, Block-Cut Forest . . . . . | 407 |
| 4.2.6  | Bridges, 2-Edge-Connected Components, Bridge Forest . . . . .           | 410 |
| 4.2.7  | Online Bridge Finding . . . . .                                         | 412 |
| 4.2.8  | 2-SAT . . . . .                                                         | 415 |
| 4.2.9  | Functional Graph . . . . .                                              | 417 |
| 4.2.10 | Offline Dynamic Connectivity . . . . .                                  | 420 |
| 4.3    | Shortest Paths . . . . .                                                | 423 |
| 4.3.1  | Unweighted Shortest Path (BFS) . . . . .                                | 423 |
| 4.3.2  | 0-1 BFS . . . . .                                                       | 424 |
| 4.3.3  | Shortest Path (Dijkstra) . . . . .                                      | 426 |
| 4.3.4  | Shortest Path (A-Star) . . . . .                                        | 428 |
| 4.3.5  | Shortest Path (Bellman-Ford) . . . . .                                  | 430 |
| 4.3.6  | Shortest Path (SPFA) . . . . .                                          | 432 |
| 4.3.7  | DAG Shortest Path (Topological DFS) . . . . .                           | 434 |
| 4.3.8  | All-Pairs Shortest Path (Floyd-Warshall) . . . . .                      | 435 |
| 4.3.9  | All-Pairs Shortest Path (Johnson) . . . . .                             | 437 |
| 4.4    | Spanning Trees . . . . .                                                | 440 |
| 4.4.1  | Minimum Spanning Tree (Prim) . . . . .                                  | 440 |
| 4.4.2  | Minimum Spanning Tree (Kruskal) . . . . .                               | 441 |
| 4.4.3  | Kruskal Reconstruction Tree . . . . .                                   | 443 |
| 4.4.4  | Minimum Spanning Tree (Boruvka) . . . . .                               | 445 |
| 4.4.5  | Minimum Spanning Arborescence (Edmonds) . . . . .                       | 447 |
| 4.5    | Flows and Cuts . . . . .                                                | 449 |
| 4.5.1  | Maximum Flow (Ford-Fulkerson) . . . . .                                 | 449 |
| 4.5.2  | Maximum Flow (Edmonds-Karp) . . . . .                                   | 450 |
| 4.5.3  | Maximum Flow and Decomposition (Dinic) . . . . .                        | 452 |
| 4.5.4  | Maximum Flow (Push-Relabel) . . . . .                                   | 456 |
| 4.5.5  | Flow with Lower Bounds . . . . .                                        | 459 |
| 4.5.6  | Minimum-Cost Maximum-Flow (SPFA) . . . . .                              | 463 |
| 4.5.7  | Minimum-Cost Maximum-Flow (Potentials) . . . . .                        | 465 |
| 4.5.8  | Gomory-Hu Tree . . . . .                                                | 468 |
| 4.5.9  | Global Minimum Cut (Stoer-Wagner) . . . . .                             | 472 |
| 4.6    | Matching and Assignment . . . . .                                       | 473 |
| 4.6.1  | Bipartite Check (2-Coloring) . . . . .                                  | 473 |

|          |                                                 |            |
|----------|-------------------------------------------------|------------|
| 4.6.2    | Maximum Bipartite Matching (Kuhn)               | 475        |
| 4.6.3    | Maximum Bipartite Matching (Hopcroft-Karp)      | 477        |
| 4.6.4    | Minimum Vertex Cover (Konig)                    | 478        |
| 4.6.5    | Maximum Matching (Blossom)                      | 481        |
| 4.6.6    | Maximum-Weight Matching (Weighted Blossom)      | 483        |
| 4.6.7    | Assignment (Hungarian Algorithm)                | 490        |
| 4.6.8    | Stable Marriage (Gale-Shapley)                  | 492        |
| 4.7      | Exponential Graph Problems                      | 493        |
| 4.7.1    | Maximum Clique (Bron-Kerbosch)                  | 493        |
| 4.7.2    | Graph Coloring (Backtracking)                   | 495        |
| 4.7.3    | Edge Coloring (Misra-Gries)                     | 497        |
| 4.7.4    | Shortest Hamiltonian Path and Cycle (Held-Karp) | 499        |
| <b>5</b> | <b>Optimization</b>                             | <b>502</b> |
| 5.1      | Searching Sorted Data                           | 502        |
| 5.1.1    | Binary Search                                   | 502        |
| 5.1.2    | Exponential Search                              | 504        |
| 5.1.3    | Interpolation Search                            | 505        |
| 5.1.4    | Sorted Matrix Search                            | 506        |
| 5.1.5    | Local Minimum                                   | 507        |
| 5.2      | Continuous Optimization                         | 509        |
| 5.2.1    | Ternary and Golden Section Search               | 509        |
| 5.2.2    | 2D Hill Climbing                                | 513        |
| 5.2.3    | Simulated Annealing                             | 514        |
| 5.2.4    | Gradient Descent                                | 516        |
| 5.3      | Dynamic Programming Tricks                      | 518        |
| 5.3.1    | Divide-and-Conquer DP Optimization              | 518        |
| 5.3.2    | SMAWK Row Minima                                | 520        |
| 5.3.3    | Knuth DP Optimization                           | 522        |
| 5.3.4    | Lagrangian Relaxation (Aliens Trick)            | 523        |
| 5.3.5    | Convex Hull Trick (Semi-Dynamic)                | 524        |
| 5.3.6    | Convex Hull Trick (Fully-Dynamic)               | 526        |
| 5.3.7    | Li Chao Tree                                    | 528        |
| 5.4      | Numerical Methods                               | 530        |
| 5.4.1    | Root Finding (Bracketing)                       | 530        |
| 5.4.2    | Root Finding (Iteration)                        | 533        |
| 5.4.3    | Polynomial Root Finding (Differentiation)       | 534        |
| 5.4.4    | Polynomial Root Finding (Laguerre)              | 537        |
| 5.4.5    | Polynomial Root Finding (Ehrlich-Aberth)        | 539        |

|          |                                           |            |
|----------|-------------------------------------------|------------|
| 5.4.6    | Integration (Simpson)                     | 543        |
| 5.4.7    | Ordinary Differential Equations           | 544        |
| 5.4.8    | Linear Programming (Simplex)              | 546        |
| <b>6</b> | <b>Mathematics</b>                        | <b>550</b> |
| 6.1      | Math Utilities                            | 550        |
| 6.1.1    | Floating-Point Comparison                 | 550        |
| 6.1.2    | Rounding                                  | 552        |
| 6.1.3    | Base Conversion                           | 555        |
| 6.1.4    | Date and Time                             | 557        |
| 6.2      | Combinatorics                             | 560        |
| 6.2.1    | Combinatorial Calculations                | 560        |
| 6.2.2    | Enumerating Arrangements                  | 565        |
| 6.2.3    | Enumerating Permutations                  | 567        |
| 6.2.4    | Enumerating Combinations                  | 571        |
| 6.2.5    | Enumerating Partitions                    | 576        |
| 6.2.6    | Generic Combinatorial Enumeration         | 578        |
| 6.3      | Number Theory                             | 583        |
| 6.3.1    | Mod Helpers and Binary Exponentiation     | 583        |
| 6.3.2    | GCD, LCM, Mod Inverse, Chinese Remainder  | 584        |
| 6.3.3    | Modular Integer and Combinatorics         | 589        |
| 6.3.4    | Primitive Roots                           | 594        |
| 6.3.5    | Discrete Logarithm (Baby-Step Giant-Step) | 596        |
| 6.3.6    | Modular Square Root (Tonelli-Shanks)      | 597        |
| 6.3.7    | Jacobi Symbol                             | 599        |
| 6.3.8    | Prime Generation                          | 600        |
| 6.3.9    | Prime Tests and Factorization             | 603        |
| 6.3.10   | Euler's Totient Function                  | 611        |
| 6.3.11   | Divisor Functions                         | 613        |
| 6.3.12   | Mobius Function and Inclusion-Exclusion   | 615        |
| 6.4      | Arbitrary Precision Arithmetic            | 617        |
| 6.4.1    | Big Integer (Simple)                      | 617        |
| 6.4.2    | Big Integer                               | 620        |
| 6.4.3    | Big Decimal                               | 632        |
| 6.4.4    | Rational Numbers                          | 637        |
| 6.5      | Linear Algebra                            | 640        |
| 6.5.1    | Matrix Operations                         | 640        |
| 6.5.2    | Row Reduction                             | 645        |
| 6.5.3    | Determinant and Inverse                   | 648        |

|          |                                                |            |
|----------|------------------------------------------------|------------|
| 6.5.4    | LU Decomposition . . . . .                     | 651        |
| 6.5.5    | QR Decomposition and Least Squares . . . . .   | 655        |
| 6.5.6    | Characteristic Polynomial . . . . .            | 658        |
| 6.5.7    | Eigenvalues (Jacobi) . . . . .                 | 660        |
| 6.5.8    | Sparse Matrix . . . . .                        | 662        |
| 6.5.9    | Matrix-Tree Theorem . . . . .                  | 669        |
| 6.6      | Polynomials and Linear Recurrences . . . . .   | 671        |
| 6.6.1    | Number Theoretic Transform . . . . .           | 671        |
| 6.6.2    | Fast Fourier Transform . . . . .               | 673        |
| 6.6.3    | Polynomial Operations . . . . .                | 675        |
| 6.6.4    | Polynomial Interpolation . . . . .             | 681        |
| 6.6.5    | Linear Recurrence (Berlekamp-Massey) . . . . . | 683        |
| 6.6.6    | Linear Recurrence (Kitamasa) . . . . .         | 685        |
| 6.7      | Probability and Statistics . . . . .           | 686        |
| 6.7.1    | Probability Distributions . . . . .            | 686        |
| 6.7.2    | Expected Value DP . . . . .                    | 690        |
| 6.7.3    | Absorbing Markov Chains . . . . .              | 692        |
| 6.7.4    | Stationary Distribution . . . . .              | 694        |
| 6.7.5    | Statistical Learning . . . . .                 | 696        |
| 6.8      | Game Theory . . . . .                          | 702        |
| 6.8.1    | Nim . . . . .                                  | 702        |
| 6.8.2    | Nim Product . . . . .                          | 703        |
| 6.8.3    | Sprague-Grundy Theorem . . . . .               | 705        |
| 6.8.4    | Grundy Numbers on DAG . . . . .                | 706        |
| 6.8.5    | Minimax and Alpha-Beta Pruning . . . . .       | 707        |
| <b>7</b> | <b>Geometry</b>                                | <b>709</b> |
| 7.1      | Geometry Primitives . . . . .                  | 709        |
| 7.1.1    | Point . . . . .                                | 709        |
| 7.1.2    | Line . . . . .                                 | 713        |
| 7.1.3    | Circle . . . . .                               | 717        |
| 7.1.4    | Triangle . . . . .                             | 720        |
| 7.1.5    | Rectangle . . . . .                            | 722        |
| 7.2      | Angles, Distances, and Intersections . . . . . | 725        |
| 7.2.1    | Angles . . . . .                               | 725        |
| 7.2.2    | Distances . . . . .                            | 727        |
| 7.2.3    | Line Intersection . . . . .                    | 730        |
| 7.2.4    | Circle Intersection . . . . .                  | 734        |
| 7.2.5    | Circle Tangents . . . . .                      | 738        |

|          |                                                  |            |
|----------|--------------------------------------------------|------------|
| 7.3      | Polygons and Point Sets . . . . .                | 740        |
| 7.3.1    | Polygon Sorting and Area . . . . .               | 740        |
| 7.3.2    | Point-in-Polygon . . . . .                       | 743        |
| 7.3.3    | Convex Hull and Diametral Pair . . . . .         | 744        |
| 7.3.4    | Convex Polygon Queries . . . . .                 | 747        |
| 7.3.5    | Minimum Enclosing Circle . . . . .               | 750        |
| 7.3.6    | Closest Pair . . . . .                           | 752        |
| 7.3.7    | Maximum Collinear Points . . . . .               | 754        |
| 7.3.8    | Rectangle Coverage . . . . .                     | 756        |
| 7.3.9    | Rectangle Union . . . . .                        | 758        |
| 7.3.10   | Multiple Segment Intersection . . . . .          | 761        |
| 7.3.11   | Manhattan MST . . . . .                          | 764        |
| 7.4      | Advanced Planar Geometry . . . . .               | 766        |
| 7.4.1    | Convex Polygon Cut . . . . .                     | 766        |
| 7.4.2    | Half-Plane Intersection . . . . .                | 768        |
| 7.4.3    | Minkowski Sum . . . . .                          | 770        |
| 7.4.4    | Polygon Intersection and Union . . . . .         | 773        |
| 7.4.5    | Delaunay Triangulation (Simple) . . . . .        | 777        |
| 7.4.6    | Delaunay Triangulation (Guibas-Stolfi) . . . . . | 779        |
| 7.5      | 3D Geometry . . . . .                            | 784        |
| 7.5.1    | 3D Point . . . . .                               | 784        |
| 7.5.2    | Polyhedron Volume . . . . .                      | 785        |
| 7.5.3    | 3D Convex Hull . . . . .                         | 786        |
| <b>8</b> | <b>Miscellany</b>                                | <b>789</b> |
| 8.1      | Contest Utilities . . . . .                      | 789        |
| 8.2      | Fast IO . . . . .                                | 793        |
| 8.3      | Debugging . . . . .                              | 798        |
| 8.4      | Stress-Testing . . . . .                         | 802        |
| 8.5      | Benchmarking . . . . .                           | 808        |
| 8.6      | GNU Policy-Based Data Structures . . . . .       | 809        |
| 8.7      | Multithreading . . . . .                         | 812        |
| 8.8      | Bump Allocation . . . . .                        | 813        |

# Chapter 1

## Elementary Algorithms

### 1.1 Array Transformations

---

#### 1.1.1 Sorting Algorithms

The following functions are equivalent to `std::sort()`, taking random-access iterators as a half-open range `[lo, hi)` to be sorted. The range is sorted into ascending order after the function call. Optionally, a comparison function object specifying a strict weak ordering may be specified to replace the default `operator<`.

- `quicksort(lo, hi, comp = std::less<>)` sorts the range using quicksort.
- `mergesort(lo, hi, comp = std::less<>)` sorts the range using merge sort, which is stable.
- `heapsort(lo, hi, comp = std::less<>)` sorts the range using heapsort.
- `combsort(lo, hi, comp = std::less<>)` sorts the range using comb sort.
- `insertion_sort(lo, hi, comp = std::less<>)` sorts the range using insertion sort, which is stable.
- `shellsort(lo, hi, comp = std::less<>)` sorts the range using shell sort.
- `counting_sort(lo, hi, keys, key)` sorts the range by the index that `key(element)` returns in `[0, keys)`, which is stable; unlike the shared interface above, it takes no comparator.
- `radix_sort(lo, hi)` sorts an integer range using least-significant-byte radix sort; unlike the shared interface above, it takes no comparator.
- `bitonic_sort(lo, hi, comp = std::less<>)` sorts a range of any length using a compare-exchange sequence generated from that length.

These functions are not meant to compete with standard library implementations in terms of speed. Instead, they demonstrate how common sorting algorithms can be concisely implemented in C++.

```
#include <algorithm>
#include <climits>
#include <functional>
#include <iterator>
#include <type_traits>
#include <vector>
```

Quicksort repeatedly selects a pivot and partitions the range so that elements comparing less than the pivot precede it, elements comparing equal stay in the middle, and elements comparing greater follow it. Divide and conquer is then applied to the two outer ranges until the original range is sorted. The swaps used during partitioning make quicksort unstable. Despite having a worst case of  $O(n^2)$ , quicksort is often faster in practice than merge sort and heapsort, which both have a worst case time complexity of  $O(n \log n)$ .

The pivot chosen in this implementation is always a middle element of the range to be sorted. Shuffling the input first or choosing each pivot randomly makes consistently unbalanced partitions unlikely. Median-of-three is another practical choice, while median-of-medians can guarantee a balanced pivot at greater constant-factor cost.

**Time Complexity:**  $O(n)$  best (all equal),  $O(n \log n)$  average, and  $O(n^2)$  worst.

**Space Complexity:**  $O(\log n)$  auxiliary stack space.

```
template<typename It, typename Compare = std::less<>>
void quicksort(It lo, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    while (hi - lo >= 2) {
        T pivot = *(lo + (hi - lo) / 2);
        It lt = lo, mid = lo, gt = hi;
        while (mid != gt) {
            if (comp(*mid, pivot)) {
                std::iter_swap(lt++, mid++);
            } else if (comp(pivot, *mid)) {
                std::iter_swap(mid, --gt);
            } else {
                ++mid;
            }
        }
        if (lt - lo < hi - gt) {
            quicksort(lo, lt, comp);
            lo = gt;
        } else {
            quicksort(gt, hi, comp);
            hi = lt;
        }
    }
}
```

Merge sort first divides a list into  $n$  sublists of one element each, then recursively merges the sublists into sorted order until only a single sorted sublist remains. Merge sort is a stable sort, meaning that it preserves the relative order of elements which compare equal by `operator<` or the custom comparator given.

An analogous function in the C++ standard library is `std::stable_sort()`, except that the implementation here requires sufficient memory to be available. When  $O(n)$  auxiliary memory is not available, `std::stable_sort()` falls back to a time complexity of  $O(n \log^2 n)$  whereas the implementation here will simply fail.

**Time Complexity:**  $O(n \log n)$  in all cases.

**Space Complexity:**  $O(\log n)$  auxiliary stack space and  $O(n)$  auxiliary heap space.

```
template<typename It, typename Compare = std::less<>>
void mergesort(It lo, It hi, Compare comp = Compare{}) {
    if (hi - lo < 2) {
        return;
    }
    It mid = lo + (hi - lo - 1) / 2, a = lo, c = mid + 1;
    mergesort(lo, mid + 1, comp);
    mergesort(mid + 1, hi, comp);
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> merged;
    merged.reserve(hi - lo);
    while (a <= mid && c < hi) {
        merged.push_back(comp(*c, *a) ? *c++ : *a++);
    }
    merged.insert(merged.end(), a, mid + 1);
    merged.insert(merged.end(), c, hi);
    std::copy(merged.begin(), merged.end(), lo);
}
```



Heapsort first rearranges an array to satisfy the max-heap property. Then, it repeatedly pops the max element of the heap (the left, unsorted subrange), moving it to the beginning of the right, sorted subrange until the entire range is sorted. Heapsort has a better worst case time complexity than quicksort and also a better space complexity than merge sort, but its swaps make it unstable.

The C++ standard library equivalent is calling `std::make_heap(lo, hi)`, followed by `std::sort_heap(lo, hi)`.

`sift_down()` restores the heap property below one node. The first loop applies it to each non-leaf from right to left, which builds the heap in  $O(n)$  time. The second loop repeatedly moves the maximum to the back of the unsorted range and restores the heap among the remaining elements.

**Time Complexity:**  $O(n \log n)$  in all cases.

**Space Complexity:**  $O(1)$  auxiliary.

```
template<typename It, typename Compare = std::less<>>
void heapsort(It lo, It hi, Compare comp = Compare{}) {
    if (hi - lo < 2) {
        return;
    }
    using T = typename std::iterator_traits<It>::value_type;
    auto sift_down = [&](It root, It end) {
        T value = *root;
        auto parent = root - lo, n = end - lo;
        while (2 * parent + 1 < n) {
            auto child = 2 * parent + 1;
            if (child + 1 < n && comp(*(lo + child), *(lo + child + 1))) {
                child++;
            }
            if (!comp(value, *(lo + child))) {
                break;
            }
            *(lo + parent) = *(lo + child);
            parent = child;
        }
        *(lo + parent) = value;
    };
    for (It root = lo + (hi - lo) / 2; root != lo;) {
        sift_down(--root, hi);
    }
    for (It end = hi; --end != lo;) {
        std::iter_swap(lo, end);
        sift_down(lo, end);
    }
}
```

Comb sort is an improved bubble sort. While bubble sort compares only adjacent elements, comb sort compares elements separated by a fixed gap, decreasing that gap after every pass. Large gaps move small values near the end toward the front quickly, avoiding the slow movement of such values in bubble sort. These nonadjacent swaps make comb sort unstable. The shrink factor 1.3 is a common empirical choice.

**Time Complexity:**  $O(n \log n)$  best and  $O(n^2)$  worst.

**Space Complexity:**  $O(1)$  auxiliary.

```
template<typename It, typename Compare = std::less<>>
void combsort(It lo, It hi, Compare comp = Compare{}) {
    int gap = static_cast<int>(hi - lo);
    bool swapped = true;
    while (gap > 1 || swapped) {
        if (gap > 1) {
            gap = gap * 10 / 13;
        }
        swapped = false;
        for (It it = lo; it + gap < hi; ++it) {
```

```

        if (comp(*(it + gap), *it)) {
            std::iter_swap(it, it + gap);
            swapped = true;
        }
    }
}
}

```

Insertion sort builds the sorted range one element at a time. It scans left to right, and for each element shifts the larger elements of the already-sorted prefix one position to the right to open a slot where the element belongs. Although its average and worst cases are quadratic, it is simple, stable, in-place, and online (it can sort a stream as elements arrive).

Its strength is being adaptive: on an already-sorted or nearly-sorted range, each element travels only a short distance, giving  $O(n)$  best-case time and  $O(n + d)$  in general for  $d$  total inversions. This is why it outperforms the asymptotically faster sorts on small or nearly-sorted ranges, and why hybrid sorts such as introsort and Timsort fall back to it once a subrange is small enough.

The comparison `comp(key, *(j - 1))` is strict, so an element never moves past an earlier element it compares equal to, which keeps the sort stable.

**Time Complexity:**  $O(n + d)$ , i.e.  $O(n)$  best and  $O(n^2)$  average/worst.

**Space Complexity:**  $O(1)$  auxiliary.

```

template<typename It, typename Compare = std::less<>>
void insertion_sort(It lo, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    for (It i = lo; i != hi; ++i) {
        T key = *i;
        It j = i;
        while (j != lo && comp(key, *(j - 1))) {
            *j = *(j - 1);
            --j;
        }
        *j = key;
    }
}

```

Shell sort is insertion sort performed on subsequences of elements a fixed gap apart, repeating with smaller gaps until the final pass uses a gap of one and is an ordinary insertion sort. The earlier passes are what make that final pass cheap: each one leaves the range closer to sorted, and insertion sort runs in time proportional to the number of remaining inversions. Comparing elements a gap apart also moves a value many positions in one step, which is what insertion sort alone cannot do. Like comb sort above, the nonadjacent swaps make it unstable.

The gap sequence determines the bound, and choosing it well is the whole difficulty. This implementation uses Ciura's empirical sequence, extended upward by a factor of 2.25, which is the fastest known for small and medium ranges though it has no proven bound. The theoretical choices are Sedgewick's sequence at  $O(n^{4/3})$  and Pratt's at  $O(n \log^2 n)$ , the latter being slower in practice despite the better bound.

**Time Complexity:**  $O(n \log n)$  best, and  $O(n^{4/3})$  or better in practice for the gaps used here.

**Space Complexity:**  $O(1)$  auxiliary.

```

template<typename It, typename Compare = std::less<>>
void shellsort(It lo, It hi, Compare comp = Compare{}) {
    static const int GAPS[] = {1, 4, 10, 23, 57, 132, 301, 701, 1577, 3548, 7983, 17962};
    int n = static_cast<int>(hi - lo), count = sizeof(GAPS) / sizeof(GAPS[0]);
    for (int g = count - 1; g >= 0; g--) {
        int gap = GAPS[g];
        if (gap >= n) {
            continue;
        }
    }
}

```

```

    }
    for (It it = lo + gap; it < hi; ++it) {
        typename std::iterator_traits<It>::value_type key = *it;
        It j = it;
        while (j - lo >= gap && comp(key, *(j - gap))) {
            *j = *(j - gap);
            j -= gap;
        }
        *j = key;
    }
}
}

```

Counting sort sorts values from a small integer range by counting how many times each value occurs, converting those counts into starting offsets with a prefix sum, and then placing each element at the offset for its key. It never compares two elements, so the  $O(n \log n)$  lower bound for comparison sorts does not apply, and it runs in linear time whenever the range of keys is proportional to the number of elements.

Placing elements in a second pass over the input from right to left keeps the sort stable, which is what makes it usable as the inner pass of the radix sort below. The `key` function maps an element to an index in  $[0, \text{keys})$ , so records can be sorted by one field while carrying the rest.

**Time Complexity:**  $O(n + k)$  for  $n$  elements with  $k$  possible keys.

**Space Complexity:**  $O(n + k)$  auxiliary.

```

template<typename It, typename KeyFn>
void counting_sort(It lo, It hi, int keys, KeyFn key) {
    if (hi - lo < 2) {
        return;
    }
    std::vector<int> count(keys);
    for (It it = lo; it != hi; ++it) {
        count[key(*it)]++;
    }
    for (int i = 1; i < keys; ++i) {
        count[i] += count[i - 1];
    }
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> res(hi - lo);
    for (It it = hi; it != lo; --it) {
        res[--count[key(*it)]] = *it;
    }
    std::copy(res.begin(), res.end(), lo);
}

```

Radix sort is used to sort integer elements with a constant number of bits in linear time. This implementation works on ranges pointing to any signed or unsigned integer primitive. Signed values are handled by sorting on a key that flips the sign bit, which maps the two's-complement order onto the unsigned order so the most negative value sorts first.

Digits are processed from least significant to most significant, and each counting-sort pass is stable. After one pass, the values are sorted by the processed digit; stability ensures that a later pass preserves the ordering already established by all less-significant digits. Inductively, after the final pass the values are sorted by the entire key.

This implementation uses one byte per digit, so digits can be extracted with shifts and masks and the counting table has  $2^8 = 256$  entries.

**Time Complexity:**  $O(n \cdot w)$  for  $n$  integers of  $w$  bits each.

**Space Complexity:**  $O(n + 2^b)$  auxiliary for a radix of  $b$  bits, i.e.  $O(n)$  for constant  $b$ .

```

template<typename It>
void radix_sort(It lo, It hi) {
    if (hi - lo < 2) {
        return;
    }
    const int radix_bits = 8;
    const int radix_base = 1 << radix_bits; // e.g. 2^8 = 256
    const int radix_mask = radix_base - 1; // e.g. 2^8 - 1 = 0xFF
    using T = typename std::iterator_traits<It>::value_type;
    static_assert(std::is_integral_v<T> && !std::is_same_v<T, bool>);
    using U = typename std::make_unsigned_t<T>;
    const int num_bits = sizeof(T) * CHAR_BIT;
    // Sort on an unsigned key. For signed types, flipping the sign bit sends the most negative value
    // to 0, mapping the signed order onto the unsigned order; logical shifts then extract each digit.
    auto key = [](T x) -> U {
        U u = static_cast<U>(x);
        return std::is_signed<T>::value ? (u ^ (U{1} << (num_bits - 1))) : u;
    };
    std::vector<T> buf(hi - lo);
    for (int pos = 0; pos < num_bits; pos += radix_bits) {
        std::vector<int> count(radix_base);
        for (It it = lo; it != hi; ++it) {
            count[(key(*it) >> pos) & radix_mask]++;
        }
        std::vector<T *> bucket(radix_base);
        T *curr = buf.data();
        for (int i = 0; i < radix_base; curr += count[i++]) {
            bucket[i] = curr;
        }
        for (It it = lo; it != hi; ++it) {
            *bucket[(key(*it) >> pos) & radix_mask]++ = *it;
        }
        std::copy(buf.begin(), buf.end(), lo);
    }
}

```

Bitonic sort is a sorting network: a fixed sequence of compare-exchanges on fixed positions, each swapping its pair when out of order. The sequence depends only on how many elements there are, never on what they are, so no comparison branches on data. It sorts the first half ascending and the second half descending, leaving a bitonic sequence that rises and then falls, which a fixed pattern of halving compare-exchanges merges. Splitting each merge at the largest power of two below its length is what admits lengths that are not powers of two, which the textbook formulation requires.

Its  $O(n \log^2 n)$  comparators exceed the  $O(n \log n)$  comparisons of a comparison sort, so run one at a time it loses to every sort above, as the benchmark below shows. What it buys instead is that each group of comparators is independent and may run at once, which is why networks are the standard choice across SIMD lanes and on a GPU. The same independence pays off at a small size fixed at compile time: unrolled into branchless minimum and maximum, a network sorts eight integers about an order of magnitude faster than a general sort. Driving that same comparator sequence from a lookup table forfeits it, since each comparator then costs an indexed load and an unpredictable branch.

**Time Complexity:**  $O(n \log^2 n)$  in all cases.

**Space Complexity:**  $O(\log n)$  auxiliary stack space.

```

template<typename It, typename Compare = std::less<>>
void bitonic_sort(It lo, It hi, Compare comp = Compare{}) {
    auto merge = [&](auto &&merge, It lo, int n, bool up) {
        if (n < 2) {
            return;
        }
        int m = 1;
        while (2 * m < n) {
            m *= 2;
        }
    };
}

```

```

    }
    for (int i = 0; i < n - m; i++) {
        if (comp(*(lo + i + m), *(lo + i)) == up) {
            std::iter_swap(lo + i, lo + i + m);
        }
    }
    merge(merge, lo, m, up);
    merge(merge, lo + m, n - m, up);
};
auto rec = [&](auto &&rec, It lo, int n, bool up) {
    if (n < 2) {
        return;
    }
    rec(rec, lo, n / 2, !up);
    rec(rec, lo + n / 2, n - n / 2, up);
    merge(merge, lo, n, up);
};
rec(rec, lo, static_cast<int>(hi - lo), true);
}

```

## Example Usage

```

#include <cassert>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <random>
#include <utility>
#include <vector>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    { // Can be used to sort arrays like std::sort().
        int a[] {32, 71, 12, 45, 26, 80, 53, 33};
        quicksort(begin(a), end(a));
        assert(is_sorted(begin(a), end(a)));
    }
    { // STL containers work too.
        vector<int> a {32, 71, 12, 45, 26, 80, 53, 33};
        quicksort(a.begin(), a.end());
        assert(is_sorted(a.begin(), a.end()));
    }
    { // Reverse iterators work as expected.
        vector<int> a {32, 71, 12, 45, 26, 80, 53, 33};
        heapsort(a.rbegin(), a.rend());
        assert(is_sorted(a.rbegin(), a.rend()));
    }
    { // Insertion sort is adaptive: an already-sorted range is left untouched in O(n).
        vector<int> a {12, 26, 32, 33, 45, 53, 71, 80};
        insertion_sort(a.begin(), a.end());
        assert(is_sorted(a.begin(), a.end()));
    }
    { // We can sort doubles just as well.
        vector<double> a {1.1, -5.0, 6.23, 4.123, 155.2};
        combsort(a.begin(), a.end());
        assert(is_sorted(a.begin(), a.end()));
    }
    { // Shell sort moves values far in one step, so it is not adaptive but is nearly in-place.

```

```

vector<int> a{32, 71, 12, 45, 26, 80, 53, 33};
shellsort(a.begin(), a.end());
assert(is_sorted(a.begin(), a.end()));
}
{ // Counting sort keys records by one field, and equal keys keep their input order.
vector<pair<int, char>> a{{2, 'a'}, {0, 'b'}, {2, 'c'}, {1, 'd'}, {0, 'e'}};
counting_sort(a.begin(), a.end(), 3, [](const pair<int, char> &x) { return x.first; });
vector<pair<int, char>> expected{{0, 'b'}, {0, 'e'}, {1, 'd'}, {2, 'a'}, {2, 'c'}};
assert(a == expected);
}
{ // radix_sort() handles signed integers (including negatives), unlike a plain counting sort.
vector<int> a{32, -71, 12, -45, 26, -80, 53, 33};
radix_sort(a.begin(), a.end());
assert(is_sorted(a.begin(), a.end()));
}
{ // Bitonic sort is oblivious too, and takes any length rather than only a power of two.
vector<int> a{32, 71, 12, 45, 26, 80, 53, 33, 19};
bitonic_sort(a.begin(), a.end());
assert(is_sorted(a.begin(), a.end()));
}
{ // Empty and singleton ranges are valid no-ops.
vector<int> empty, single{42};
quicksort(empty.begin(), empty.end());
mergesort(empty.begin(), empty.end());
heapsort(empty.begin(), empty.end());
insertion_sort(empty.begin(), empty.end());
combsort(empty.begin(), empty.end());
radix_sort(empty.begin(), empty.end());
mergesort(single.begin(), single.end());
assert(empty.empty() && single[0] == 42);
}

// Stable sort.
const vector<double> a{3.14, 1.41, 2.72, 4.67, 1.73, 1.32, 1.62, 2.58};
{
vector<double> v(a);
cout << "mergesort() with default comparisons: ";
mergesort(v.begin(), v.end());
print_range(v.begin(), v.end());
}
{
vector<double> v(a);
cout << "mergesort() with integer comparisons: ";
mergesort(v.begin(), v.end(), [](double i, double j) {
return static_cast<int>(i) < static_cast<int>(j);
});
print_range(v.begin(), v.end());
}
cout << "-----" << endl;

mt19937 rng(1234567); // Fixed seed for reproducibility.
vector<int> data(5000000);
const int maxval = 10000000;
for (int &x : data) {
x = static_cast<int>(rng() % maxval); // limit magnitude for counting sort
}
cout << "Sorting five million integers..." << endl;
cout.precision(3);
auto benchmark = [&](const string &name, auto sort) {
vector<int> v = data;
clock_t start = clock();
sort(v.begin(), v.end());
double t = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
cout << setw(18) << left << name + "(): " << fixed << t << "s" << endl;
assert(is_sorted(v.begin(), v.end()));
};
benchmark("std::sort", [](auto lo, auto hi) { sort(lo, hi); });
benchmark("quicksort", [](auto lo, auto hi) { quicksort(lo, hi); });

```

```

benchmark("mergesort", [](auto lo, auto hi) { mergesort(lo, hi); });
benchmark("heapsort", [](auto lo, auto hi) { heapsort(lo, hi); });
benchmark("combsort", [](auto lo, auto hi) { combsort(lo, hi); });
benchmark("shellsort", [](auto lo, auto hi) { shellsort(lo, hi); });
benchmark("counting_sort", [](auto lo, auto hi) {
    counting_sort(lo, hi, maxval, [](int x) { return x; });
});
benchmark("radix_sort", [](auto lo, auto hi) { radix_sort(lo, hi); });
benchmark("bitonic_sort", [](auto lo, auto hi) { bitonic_sort(lo, hi); });
return 0;
}

```

### Example Output

```

mergesort() with default comparisons: 1.32 1.41 1.62 1.73 2.58 2.72 3.14 4.67
mergesort() with integer comparisons: 1.41 1.73 1.32 1.62 2.72 2.58 3.14 4.67
-----
Sorting five million integers...
std::sort():      0.285s
quicksort():      0.325s
mergesort():      0.517s
heapsort():       0.509s
combsort():       0.469s
shellsort():      0.574s
counting_sort():  0.044s
radix_sort():     0.024s
bitonic_sort():   1.040s

```

## 1.1.2 Partitioning and Selection

Partition a range around a pivot, or select the element with a given sorted rank without fully sorting the range. A comparator `comp` defines the ordering (defaults to `std::less<>`). Three-way partitioning rearranges the elements into consecutive groups that precede, are equivalent to, and follow the pivot. `std::partition()` can be used to place elements satisfying a unary predicate before those that do not, but forms only two groups and does not preserve relative order. Forming three groups can be done by first partitioning by `comp(x, pivot)`, then partitioning the remaining suffix by `!comp(pivot, x)`; using `std::stable_partition()` for both calls preserves relative order within each group.

`partition_three_way()` implements the Dutch National Flag algorithm, forming the three groups in one pass with  $O(1)$  auxiliary space. At every iteration, `[lo, lt)` contains elements preceding the pivot, `[lt, cur)` contains equivalent elements, `[cur, gt)` remains unclassified, and `[gt, hi)` contains elements following the pivot. The current element `*cur` is compared with the pivot. If it precedes the pivot, it is swapped with `*lt` and both `lt` and `cur` advance. If it is equivalent, only `cur` advances. Otherwise, it is swapped with `*-gt` without advancing `cur`, since the incoming element remains unclassified.

Quickselect repeatedly applies this partition and continues only into the group containing the desired rank. This is the same task as `std::nth_element()`: after the call, `*nth` is the value that would appear there in comparator order, no earlier value follows it, and no later value precedes it. Choosing pivots uniformly at random gives expected linear time, while the three-way split avoids unnecessary work on duplicate-heavy inputs.

- `partition_three_way(lo, hi, pivot, comp = std::less<>())` rearranges `[lo, hi)` in-place and returns a pair of iterators `(mid1, mid2)`. The three resulting ranges contain elements that precede, are equivalent to, and follow `pivot` according to `comp`, respectively. The resulting partition is not stable.
- `quickselect(lo, nth, hi, comp = std::less<>())` rearranges `[lo, hi)` in-place around the 0-based rank represented by iterator `nth` according to `comp`. This requires random-access iterators.

Random pivots leave an adversary free to force  $O(n^2)$ , which the median-of-medians rule removes. Split the range into groups of five, sort each, and pivot on the median of the group medians: at least three elements in half of the groups precede it, so every partition discards at least three tenths of the range. Finding that median is itself a selection, solved by recursing on the  $n/5$  medians gathered to the front, and  $T(n) = T(n/5) + T(7n/10) + O(n)$  is

linear because the two fractions sum to less than one, which five is the smallest group size to achieve. Each group is sorted directly, since at five elements `std::sort` is an inlined insertion sort. This guarantee is not free, but on random input the cost over a random pivot is modest.

- `quickselect_linear(lo, nth, hi, comp = std::less<>())` does the same in linear time even in the worst case, choosing each pivot by the median-of-medians rule instead of at random.

#### Time Complexity:

- $O(n)$  per call to `partition_three_way()`, where  $n$  is the range length.
- $O(n)$  expected per call to `quickselect()`, and  $O(n^2)$  in the worst case.
- $O(n)$  per call to `quickselect_linear()` in all cases.

#### Space Complexity:

- $O(1)$  auxiliary for `partition_three_way()` and `quickselect()`.
- $O(\log n)$  auxiliary stack space for `quickselect_linear()`.

```
#include <algorithm>
#include <chrono>
#include <functional>
#include <iterator>
#include <random>
#include <utility>

template<typename It, typename T, typename Compare = std::less<>>
std::pair<It, It> partition_three_way(It lo, It hi, const T &pivot, Compare comp = Compare{}) {
    It lt = lo, cur = lo, gt = hi;
    while (cur != gt) {
        if (comp(*cur, pivot)) {
            std::iter_swap(lt++, cur++);
        } else if (comp(pivot, *cur)) {
            std::iter_swap(cur, --gt);
        } else {
            ++cur;
        }
    }
    return {lt, gt};
}

template<typename It, typename Compare = std::less<>>
void quickselect(It lo, It nth, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    while (hi - lo > 1) {
        std::uniform_int_distribution<int> dist(0, hi - lo - 1);
        T pivot = *(lo + dist(rng));
        auto [lt, gt] = partition_three_way(lo, hi, pivot, comp);
        if (nth < lt) {
            hi = lt;
        } else if (nth >= gt) {
            lo = gt;
        } else {
            return;
        }
    }
}

template<typename It, typename Compare = std::less<>>
void quickselect_linear(It lo, It nth, It hi, Compare comp = Compare{}) {
    while (hi - lo > 5) {
        int groups = 0;
        for (It g = lo; g < hi; g += 5, groups++) {
            It ge = hi - g < 5 ? hi : g + 5;
            std::sort(g, ge, comp); // At most 5 elements, so this is an O(1) step.
            std::iter_swap(lo + groups, g + (ge - g) / 2);
        }
    }
}
```



```

    It mid = lo + groups / 2;
    quickselect_linear(lo, mid, lo + groups, comp);
    using T = typename std::iterator_traits<It>::value_type;
    T pivot = *mid;
    std::pair<It, It> parts = partition_three_way(lo, hi, pivot, comp);
    if (nth < parts.first) {
        hi = parts.first;
    } else if (nth < parts.second) {
        return;
    } else {
        lo = parts.second;
    }
}
std::sort(lo, hi, comp);
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> values{2, 0, 1, 2, 1, 0, 1};
    // On a range containing only 0, 1, and 2, partitioning around 1 sorts the range.
    partition_three_way(values.begin(), values.end(), 1);
    assert(is_sorted(values.begin(), values.end()));

    vector<int> b{4, 2, 5, 3, 3, 1};
    auto [mid1, mid2] = partition_three_way(b.begin(), b.end(), 3);
    for (auto it = b.begin(); it != mid1; ++it) {
        assert(*it < 3);
    }
    for (auto it = mid1; it != mid2; ++it) {
        assert(*it == 3);
    }
    for (auto it = mid2; it != b.end(); ++it) {
        assert(*it > 3);
    }

    vector<int> a{5, 6, 4, 3, 2, 6, 7, 9, 3};
    int n = static_cast<int>(a.size());
    quickselect(a.begin(), a.begin() + n / 2, a.end());
    assert(a[n / 2] == 5);
    // Values left of the median are <=, and values right are >= (the exact order within each side is
    // randomized, since the pivot is chosen at random).
    for (int i = 0; i < n / 2; i++) {
        assert(a[i] <= a[n / 2]);
    }
    for (int i = n / 2 + 1; i < n; i++) {
        assert(a[i] >= a[n / 2]);
    }

    // The median-of-medians variant gives the same answer with a worst case that is still linear.
    vector<int> c{5, 6, 4, 3, 2, 6, 7, 9, 3};
    quickselect_linear(c.begin(), c.begin() + n / 2, c.end());
    assert(c[n / 2] == 5);

    vector<int> desc{1, 4, 2, 3};
    quickselect(desc.begin(), desc.begin() + 1, desc.end(), greater<int>());
    assert(desc[1] == 3);
    return 0;
}

```

### 1.1.3 Array Rotation

Rotates the half-open iterator range  $[lo, hi)$  left around the split point  $mid$ , where  $lo \leq mid \leq hi$ . Writing the adjacent subranges  $[lo, mid)$  and  $[mid, hi)$  as  $A$  and  $B$ , respectively, the operation transforms  $A B$  into  $B A$  while preserving the relative order within each subrange. The standard library function `std::rotate(lo, mid, hi)` performs the same rearrangement.

All three versions below operate in place, using different algorithms and iterator capabilities.

- `rotate1(lo, mid, hi)` requires ForwardIterators and repeatedly swaps the next elements of the two unfinished subranges. When the right iterator reaches  $hi$ , it wraps to the current  $mid$ ; when the left iterator reaches  $mid$ , that boundary advances to the right iterator.
- `rotate2(lo, mid, hi)` requires BidirectionalIterators, applying a trick with three reversals. Writing the input subranges as  $A B$ , two initial reversals yield `reverse(A) reverse(B)`; then the whole range is reversed to yield  $B A$ , restoring the order within each subrange.
- `rotate3(lo, mid, hi)` requires random-access iterators, applying a juggling algorithm which first divides the range into `gcd(hi - lo, mid - lo)` sets and then rotates the corresponding elements in each set. The method follows the permutation that sends each position to its rotated destination. These form that many disjoint cycles, so walking each cycle once moves every element to its final position.

**Time Complexity:**  $O(n)$  per call to all versions, where  $n$  is the distance between  $lo$  and  $hi$ .

**Space Complexity:**  $O(1)$  auxiliary for all versions.

```
#include <algorithm>
#include <numeric>

template<typename It>
void rotate1(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    It next = mid;
    while (lo != next) {
        std::iter_swap(lo++, next++);
        if (next == hi) {
            next = mid;
        } else if (lo == mid) {
            mid = next;
        }
    }
}

template<typename It>
void rotate2(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    std::reverse(lo, mid);
    std::reverse(mid, hi);
    std::reverse(lo, hi);
}

template<typename It>
void rotate3(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    int n = static_cast<int>(hi - lo);
    int jump = static_cast<int>(mid - lo);
    int g = std::gcd(jump, n), cycle = n / g;
    for (int i = 0; i < g; i++) {
        int curr = i, next;
        for (int j = 0; j < cycle - 1; j++) {
            next = curr + jump;
```

```

        if (next >= n) {
            next -= n;
        }
        std::iter_swap(lo + curr, lo + next);
        curr = next;
    }
}
}

```

## Example Usage

```

#include <algorithm>
#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a0(10000), a1, a2, a3;
    iota(a0.begin(), a0.end(), 0);
    a1 = a2 = a3 = a0;
    int mid = 5678;
    std::rotate(a0.begin(), a0.begin() + mid, a0.end());
    rotate1(a1.begin(), a1.begin() + mid, a1.end());
    rotate2(a2.begin(), a2.begin() + mid, a2.end());
    rotate3(a3.begin(), a3.begin() + mid, a3.end());
    assert(a0 == a1 && a0 == a2 && a0 == a3);

    vector<int> no_op{1, 2, 3};
    rotate1(no_op.begin(), no_op.begin(), no_op.end());
    assert((no_op == vector<int>{1, 2, 3}));
    rotate2(no_op.begin(), no_op.end(), no_op.end());
    assert((no_op == vector<int>{1, 2, 3}));
    rotate3(no_op.begin(), no_op.begin(), no_op.end());
    assert((no_op == vector<int>{1, 2, 3}));

    vector<int> a{2, 4, 2, 0, 5, 10, 7, 3, 7, 1};

    // Insertion sort.
    for (auto i = a.begin(); i != a.end(); ++i) {
        rotate1(upper_bound(a.begin(), i, *i), i, i + 1);
    }
    assert((a == vector<int>{0, 1, 2, 2, 3, 4, 5, 7, 7, 10}));

    // Simple rotation to the left.
    rotate2(a.begin(), a.begin() + 1, a.end());
    assert((a == vector<int>{1, 2, 2, 3, 4, 5, 7, 7, 10, 0}));

    // Simple rotation to the right.
    rotate3(a.rbegin(), a.rbegin() + 1, a.rend());
    assert((a == vector<int>{0, 1, 2, 2, 3, 4, 5, 7, 7, 10}));
    return 0;
}

```

### 1.1.4 Coordinate Compression

Given a range of  $n$  numerical elements, reassign each element to an integer in the domain  $[0, k)$ , where  $k$  is the number of distinct elements in the original range, while preserving the initial relative ordering of elements. That is, if  $a$  is the array of original values and  $b$  is the array of compressed values, then every pair of indices  $i, j$  in  $[0, n)$  shall satisfy  $a[i] < a[j]$  if and only if  $b[i] < b[j]$ .

The two functions below take a ForwardIterator range, rewrite it in place, and discard the mapping. The comparator `comp` defines the value ordering: `comp(a, b)` is true when `a` precedes `b`.

- `compress1(lo, hi, comp = std::less<>())` performs the compression by sorting the array, removing duplicates, and binary searching for the position of each original value.
- `compress2(lo, hi, comp = std::less<>())` achieves the same result by inserting all values into an `std::map`, which automatically removes duplicate values and supports efficient lookups of the compressed values.

**Time Complexity:**  $O(n \log n)$  per call to `compress1(lo, hi)` and `compress2(lo, hi)`, where  $n$  is the distance between `lo` and `hi`.

**Space Complexity:**  $O(n)$  auxiliary for `compress1()` and `compress2()`.

```
#include <algorithm>
#include <cassert>
#include <functional>
#include <iterator>
#include <map>
#include <vector>

template<typename It, typename Compare = std::less<>>
void compress1(It lo, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> v(lo, hi);
    std::sort(v.begin(), v.end(), comp);
    v.erase(
        std::unique(
            v.begin(), v.end(), [&](const T &a, const T &b) { return !comp(a, b) && !comp(b, a); }
        ),
        v.end()
    );
    for (It it = lo; it != hi; ++it) {
        *it = static_cast<int>(std::lower_bound(v.begin(), v.end(), *it, comp) - v.begin());
    }
}

template<typename It, typename Compare = std::less<>>
void compress2(It lo, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    std::map<T, int, Compare> m(comp);
    for (It it = lo; it != hi; ++it) {
        m[*it] = 0;
    }
    int id = 0;
    for (auto &[key, val] : m) {
        val = id++;
    }
    for (It it = lo; it != hi; ++it) {
        *it = m[*it];
    }
}
```

`Compressor` retains the sorted table of distinct values so that arbitrary values can be mapped to and from compressed ranks long after construction, such as for offline queries arriving separately from the array being compressed. It uses `std::less<T>` by default; to customize the ordering, instantiate `Compressor<T, Compare>` and pass the comparator to either constructor.

- `Compressor<T>()` constructs an empty compressor. Register values with `add(x)`, then call `build()` once before the first query.
- `Compressor<T>(lo, hi)` constructs a compressor from the half-open iterator range `[lo, hi)`.
- `size()` returns the number of distinct registered values  $k$ .
- `value(r)` returns the original value with rank  $r$ , inverting `rank()`.
- `contains(x)` returns whether  $x$  is a registered value.
- `rank(x)` returns the compressed value (rank) of  $x$  in  $[0, k)$ .  $x$  must have been registered.

**Time Complexity:**

- $O(n \log n)$  per call to `Compressor(lo, hi)`, where  $n$  is the range length.
- $O(1)$  amortized per call to `add()`, and  $O(m \log m)$  per call to `build()`, where  $m$  is the total number of values registered.
- $O(\log k)$  per call to `rank()` and `contains()`, where  $k$  is the number of distinct values.
- $O(1)$  per call to `size()` and `value()`.

**Space Complexity:**  $O(k)$  storage, where  $k$  is the number of distinct registered values.

```
template<typename T, typename Compare = std::less<T>>
class Compressor {
    Compare comp;
    std::vector<T> v;

public:
    explicit Compressor(Compare comp = Compare{}) : comp(std::move(comp)) {}

    template<typename It>
    Compressor(It lo, It hi, Compare comp = Compare{}) : comp(std::move(comp)), v(lo, hi) {
        build();
    }

    void build() {
        std::sort(v.begin(), v.end(), comp);
        v.erase(
            std::unique(
                v.begin(), v.end(), [&](const T &a, const T &b) { return !comp(a, b) && !comp(b, a); }
            ),
            v.end()
        );
    }

    void add(const T &x) { v.push_back(x); }
    int size() const { return static_cast<int>(v.size()); }
    const T &value(int r) const { return v[r]; }
    bool contains(const T &x) const { return std::binary_search(v.begin(), v.end(), x, comp); }

    int rank(const T &x) const {
        int r = static_cast<int>(std::lower_bound(v.begin(), v.end(), x, comp) - v.begin());
        assert(r < size() && !comp(x, v[r])); // x must be registered.
        return r;
    }
};
```

**Example Usage**

```
#include <cassert>
using namespace std;

int main() {
    {
        vector<int> a{1, 30, 30, 7, 9, 8, 99, 99};
        compress1(a.begin(), a.end());
        assert((a == vector<int>{0, 4, 4, 1, 3, 2, 5, 5}));
    }
    {
        vector<int> a{1, 30, 30, 7, 9, 8, 99, 99};
        compress2(a.begin(), a.end());
        assert((a == vector<int>{0, 4, 4, 1, 3, 2, 5, 5}));
    }
    { // Non-integral types work too, as long as ints can be assigned to them.
        vector<double> a{0.5, -1.0, 3, -1.0, 20, 0.5};
        compress1(a.begin(), a.end());
        assert((a == vector<double>{1, 0, 2, 0, 3, 1}));
    }
}
```

```

}
{
    vector<int> a{10, 5, 30, 5, 20};
    Compressor<int> cc(a.begin(), a.end());
    assert(cc.size() == 4);
    assert(cc.rank(5) == 0);
    assert(cc.rank(30) == 3);
    assert(cc.value(2) == 20);
    assert(cc.contains(10) && !cc.contains(7));
}
{ // Register values incrementally, then build once before querying.
    Compressor<double> cc;
    cc.add(0.5);
    cc.add(-1.0);
    cc.add(0.5);
    cc.build();
    assert(cc.size() == 2);
    assert(cc.rank(0.5) == 1);
    assert(cc.value(0) == -1.0);
}
{
    vector<int> a{10, 30, 20, 10};
    Compressor<int, greater<int>> cc(a.begin(), a.end());
    assert(cc.rank(30) == 0 && cc.rank(10) == 2);
    assert(cc.value(1) == 20);
    compress1(a.begin(), a.end(), greater<int>());
    assert((a == vector<int>{2, 0, 1, 2}));
}
return 0;
}

```

### 1.1.5 Counting Inversions

The number of inversions for a sequence  $a$  is the number of ordered pairs  $(i, j)$  such that  $i < j$  and  $a[i] > a[j]$ . This is roughly how “close” an array is to being sorted, but is *not* the minimum number of swaps required to sort the array. If the array is sorted, then the inversion count is 0. If the array is sorted in decreasing order, then the inversion count is maximal. The following two functions are each techniques to efficiently count inversions. In the merge sort approach, whenever the merge step emits an element from the right half, that element jumps ahead of every unmerged left-half element, and exactly that many inversions are added to the count.

- `inversions(lo, hi, comp = std::less<>)` uses merge sort to return the number of inversions given two random-access iterators as a range  $[lo, hi)$ . The input range will be sorted after the function call. Optionally, a comparison function object specifying a strict weak ordering may be specified to replace the default `operator<`.
- `inversions(a)` uses coordinate compression and a Fenwick tree to return the number of inversions in an integer vector without modifying it.

#### Time Complexity:

- $O(n \log n)$  per call to `inversions(lo, hi)`, where  $n$  is the distance between `lo` and `hi`.
- $O(n \log n)$  per call to `inversions(a)`.

#### Space Complexity:

- $O(\log n)$  auxiliary stack space and  $O(n)$  auxiliary heap space for `inversions(lo, hi)`.
- $O(n)$  auxiliary heap space for `inversions(a)`.

```

#include <algorithm>
#include <cstdint>
#include <functional>
#include <iterator>
#include <vector>

```

```

template<typename It, typename Compare = std::less<>>
int64_t inversions(It lo, It hi, Compare comp = Compare{}) {
    int n = static_cast<int>(hi - lo);
    if (n < 2) {
        return 0;
    }
    It mid = lo + (n - 1) / 2, a = lo, c = mid + 1;
    int64_t res = inversions(lo, mid + 1, comp) + inversions(mid + 1, hi, comp);
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> merged;
    merged.reserve(n);
    while (a <= mid && c < hi) {
        if (comp(*c, *a)) {
            merged.push_back(*(c++));
            res += (mid - a) + 1;
        } else {
            merged.push_back(*(a++));
        }
    }
    if (a > mid) {
        for (It it = c; it != hi; ++it) {
            merged.push_back(*it);
        }
    } else {
        for (It it = a; it <= mid; ++it) {
            merged.push_back(*it);
        }
    }
    for (It it = lo; it != hi; ++it) {
        *it = merged[it - lo];
    }
    return res;
}

int64_t inversions(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> values(a);
    std::sort(values.begin(), values.end());
    values.resize(std::unique(values.begin(), values.end()) - values.begin());
    std::vector<int> bit(values.size() + 1);
    int64_t res = 0;
    for (int i = n - 1; i >= 0; i--) {
        int id =
            static_cast<int>(std::lower_bound(values.begin(), values.end(), a[i]) - values.begin()) + 1;
        for (int j = id - 1; j > 0; j -= j & -j) {
            res += bit[j];
        }
        for (int j = id; j < static_cast<int>(bit.size()); j += j & -j) {
            bit[j]++;
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        vector<int> a{6, 9, 1, 14, 8, 12, 3, 2};
        assert(inversions(a.begin(), a.end()) == 16);
        assert(is_sorted(a.begin(), a.end()));
    }
}

```

```

{
    vector<int> a{6, 9, 1, 14, 8, 12, 3, 2};
    assert(inversions(a) == 16);
    assert((a == vector<int>{6, 9, 1, 14, 8, 12, 3, 2}));
}
{
    vector<int> a{2, 2, 1};
    assert(inversions(a) == 2); // Equal elements are not inversions.
}
return 0;
}

```

## 1.2 1D Scan and Window Techniques

---

### 1.2.1 Prefix Sums

Precomputes prefix sums so range-sum queries can be answered in constant time. This is one of the most common array preprocessing tools, and also appears as a building block for subarray sums, difference arrays, and two-dimensional grids. Each table entry stores the sum of all elements before it, so any range sum is the difference of two entries; in two dimensions, rectangle sums combine four entries by inclusion-exclusion.

Prefix sums can also be combined with a frequency table to count subarrays having a target sum. When the current prefix sum is  $s$ , every earlier prefix sum equal to  $s - t$  identifies a subarray with sum  $t$ . Recording frequencies rather than only whether a prefix exists counts duplicate sums correctly.

- `prefix_sums(a)` returns the prefix sum array `pref` of length  $n + 1$ , with `pref[0] = 0` and `pref[i + 1]` equal to the sum of `a[0]` through `a[i]`.
- `range_sum(pref, lo, hi)` returns the sum of range `[lo, hi]`.
- `prefix_sums_2d(a)` returns a two-dimensional prefix sum table for matrix `a`.
- `rect_sum(pref, r1, c1, r2, c2)` returns the sum of the rectangle with rows `[r1, r2]` and columns `[c1, c2]`.
- `count_subarrays_with_sum(a, target)` returns the number of contiguous subarrays whose sum is `target`.

Overflow warning: All prefix sums, target differences, inclusion-exclusion results, and returned counts must fit in `int64_t`.

#### Time Complexity:

- $O(n)$  per call to `prefix_sums(a)`, where  $n$  is the array size.
- $O(R \cdot C)$  per call to `prefix_sums_2d(a)`, where  $R$  and  $C$  are the number of rows and columns of `a`, respectively.
- $O(1)$  per range or rectangle query.
- $O(n)$  expected per call to `count_subarrays_with_sum(a, target)`.

#### Space Complexity:

- $O(n)$  for the array returned by `prefix_sums(a)`.
- $O(R \cdot C)$  for the table returned by `prefix_sums_2d(a)`.
- $O(1)$  auxiliary for `range_sum()` and `rect_sum()`.
- $O(n)$  auxiliary for `count_subarrays_with_sum(a, target)`.

```

#include <cassert>
#include <cstdint>
#include <unordered_map>
#include <vector>

std::vector<int64_t> prefix_sums(const std::vector<int> &a) {
    std::vector<int64_t> pref(a.size() + 1);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        pref[i + 1] = pref[i] + a[i];
    }
}

```



```

    }
    return pref;
}

int64_t range_sum(const std::vector<int64_t> &pref, int lo, int hi) {
    assert(0 <= lo && lo <= hi && hi < static_cast<int>(pref.size()) - 1);
    return pref[hi + 1] - pref[lo];
}

std::vector<std::vector<int64_t>> prefix_sums_2d(const std::vector<std::vector<int>> &a) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    std::vector<std::vector<int64_t>> pref(rows + 1, std::vector<int64_t>(cols + 1));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1] + pref[i + 1][j] - pref[i][j];
        }
    }
    return pref;
}

int64_t rect_sum(const std::vector<std::vector<int64_t>> &pref, int r1, int c1, int r2, int c2) {
    assert(
        !pref.empty() && !pref[0].empty() && 0 <= r1 && r1 <= r2 &&
        r2 < static_cast<int>(pref.size()) - 1 && 0 <= c1 && c1 <= c2 &&
        c2 < static_cast<int>(pref[0].size()) - 1
    );
    return pref[r2 + 1][c2 + 1] - pref[r1][c2 + 1] - pref[r2 + 1][c1] + pref[r1][c1];
}

int64_t count_subarrays_with_sum(const std::vector<int> &a, int64_t target) {
    std::unordered_map<int64_t, int64_t> count{{0, 1}};
    int64_t sum = 0, result = 0;
    for (int x : a) {
        sum += x;
        int64_t needed = sum - target;
        if (auto it = count.find(needed); it != count.end()) {
            result += it->second;
        }
        count[sum]++;
    }
    return result;
}

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{3, -1, 4, 1, 5};
    auto pref = prefix_sums(a);
    assert(range_sum(pref, 0, 4) == 12); // Whole array.
    assert(range_sum(pref, 1, 3) == 4); // -1 + 4 + 1.
    assert(range_sum(pref, 2, 2) == 4); // Single element.
    assert(count_subarrays_with_sum(a, 4) == 2);
    assert(count_subarrays_with_sum(vector<int>{0, 0, 0}, 0) == 6);

    vector<vector<int>> grid{
        {1, 2, 3},
        {4, 5, 6},
    };
    auto pre2 = prefix_sums_2d(grid);
    assert(rect_sum(pre2, 0, 1, 1, 2) == 16); // Rows 0-1 and columns 1-2.
    assert(rect_sum(pre2, 0, 0, 1, 2) == 21); // Whole grid.
    assert(rect_sum(pre2, 1, 1, 1, 1) == 5); // Single cell.
    return 0;
}

```

```

}

```

### 1.2.2 Difference Array

Applies many range-add updates to an array in linear total time using a difference array. Instead of immediately adding `delta` to every element of range `[lo, hi]`, we can add `delta` at `diff[lo]` and subtract it at `diff[hi + 1]`; a final prefix sum reconstructs the updated values. This idea extends to two dimensions: a rectangle update touches just the four corner cells of a difference grid by inclusion-exclusion, and a final two-dimensional prefix sum reconstructs the grid.

- `DifferenceArray(n)` constructs an initially zero array of size `n`.
- `add(lo, hi, delta)` adds `delta` to every position in range `[lo, hi]`.
- `build()` returns the final array after all range updates.
- `DifferenceArray2D(rows, cols)` constructs an initially zero grid with `rows` rows and `cols` columns.
- `add(r1, c1, r2, c2, delta)` adds `delta` to every cell of the rectangle with rows `[r1, r2]` and columns `[c1, c2]`.
- `build()` returns the final grid after all rectangle updates.

Overflow warning: All stored differences and reconstructed values must fit in `int64_t`.

#### Time Complexity:

- $O(1)$  per call to `add()` of either version.
- $O(n)$  per call to `build()` of `DifferenceArray`, where  $n$  is the array size.
- $O(R \cdot C)$  per call to `build()` of `DifferenceArray2D`, where  $R$  and  $C$  are the number of rows and columns, respectively.

#### Space Complexity:

- $O(n)$  for storage and  $O(n)$  for the array returned by `DifferenceArray::build()`.
- $O(R \cdot C)$  for storage and  $O(R \cdot C)$  for the grid returned by `DifferenceArray2D::build()`.

```

#include <cassert>
#include <cstdint>
#include <vector>

class DifferenceArray {
    std::vector<int64_t> diff;

public:
    explicit DifferenceArray(int n) {
        assert(n >= 0);
        diff.assign(n + 1, 0);
    }

    void add(int lo, int hi, int64_t delta) {
        assert(0 <= lo && lo <= hi && hi + 1 < static_cast<int>(diff.size()));
        diff[lo] += delta; // Overflow warning.
        diff[hi + 1] -= delta;
    }

    std::vector<int64_t> build() const {
        std::vector<int64_t> res(static_cast<int>(diff.size()) - 1);
        int64_t cur = 0;
        for (int i = 0; i < static_cast<int>(res.size()); i++) {
            cur += diff[i]; // Overflow warning.
            res[i] = cur;
        }
        return res;
    }
};

class DifferenceArray2D {

```

```

std::vector<std::vector<int64_t>> diff;

public:
DifferenceArray2D(int rows, int cols) {
    assert(rows >= 0 && cols >= 0);
    diff.assign(rows + 1, std::vector<int64_t>(cols + 1));
}

void add(int r1, int c1, int r2, int c2, int64_t delta) {
    int rows = static_cast<int>(diff.size()) - 1;
    int cols = static_cast<int>(diff[0].size()) - 1;
    assert(0 <= r1 && r1 <= r2 && r2 < rows);
    assert(0 <= c1 && c1 <= c2 && c2 < cols);
    diff[r1][c1] += delta; // Overflow warning.
    diff[r1][c2 + 1] -= delta;
    diff[r2 + 1][c1] -= delta;
    diff[r2 + 1][c2 + 1] += delta;
}

std::vector<std::vector<int64_t>> build() const {
    int rows = static_cast<int>(diff.size()) - 1;
    int cols = static_cast<int>(diff[0].size()) - 1;
    std::vector<std::vector<int64_t>> res(rows, std::vector<int64_t>(cols));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            int64_t up = i > 0 ? res[i - 1][j] : 0;
            int64_t left = j > 0 ? res[i][j - 1] : 0;
            int64_t diag = i > 0 && j > 0 ? res[i - 1][j - 1] : 0;
            res[i][j] = diff[i][j] + up + left - diag; // Overflow warning.
        }
    }
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    DifferenceArray d(5);
    d.add(0, 2, 2); // Add 2 to indices 0-2.
    d.add(1, 4, 4); // Add 4 to indices 1-4.
    d.add(2, 3, -1); // Subtract 1 from indices 2-3.
    d.add(4, 4, 3); // Single index update.
    assert((d.build() == vector<int64_t>{2, 6, 5, 3, 7}));

    DifferenceArray2D d2(3, 4);
    d2.add(0, 0, 1, 2, 1); // Add 1 to rows 0-1 and columns 0-2.
    d2.add(1, 1, 2, 3, 2); // Add 2 to rows 1-2 and columns 1-3.
    d2.add(2, 0, 2, 0, 5); // Single cell update.
    vector<vector<int64_t>> expected2{
        {1, 1, 1, 0},
        {1, 3, 3, 2},
        {5, 2, 2, 2},
    };
    assert(d2.build() == expected2);
    return 0;
}

```

### 1.2.3 Two Pointers and Sliding Window

Uses two moving indices to avoid trying all pairs of positions. This technique has two common forms: opposite-end pointers over sorted arrays, and a sliding window over contiguous subarrays. For sorted 2-sum, if the current sum is too small, pairing the left value with any smaller right value also fails, so the left pointer can safely advance; if the sum is too large, the symmetric argument lets the right pointer retreat. Each step therefore discards an entire row or column of candidate pairs.

A sliding window applies the same idea when extending and shrinking a range changes its state monotonically. With nonnegative values, extending a window cannot decrease its sum and removing its leftmost value cannot increase it, so no shorter feasible window is skipped. For a bound such as at most  $k$  distinct values, the left pointer advances only until the window becomes valid again. Since neither endpoint moves backward, each element enters and leaves the window at most once.

For fixed-length windows, a monotonic deque of indices can report every minimum or maximum in linear total time. It stores only candidates that could still become the answer, ordered from the current extreme at the front to the least useful value at the back. A new element removes every older value that it dominates, while expired indices are removed from the front. Each index is again pushed and popped at most once.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <deque>
#include <functional>
#include <tuple>
#include <unordered_map>
#include <utility>
#include <vector>
```

The examples below cover sorted 2-sum/3-sum variants, finding the shortest subarray with sum at least a target when all values are nonnegative, and finding the longest subarray with at most  $k$  distinct values. They also cover the minimum or maximum of every fixed-length window and an online dynamic programming recurrence.

- `two_sum_sorted(a, target)` returns two indices whose values sum to `target`, or `(-1, -1)` if no such pair exists. The input array must be sorted.
- `three_sum(a, target)` returns a tuple of three original indices whose values sum to `target`, or `(-1, -1, -1)` if none exist.
- `min_subarray_at_least(a, target)` returns a tuple `(length, lo, hi)`, the minimum length and inclusive endpoints of a contiguous subarray with sum at least `target`, or length `-1` if none exists. Values in `a` must be nonnegative. If `target` is nonpositive, it returns the empty subarray.
- `max_subarray_at_most_k_distinct(a, k)` returns a tuple `(length, lo, hi)`, the maximum length and inclusive endpoints of a contiguous subarray containing at most `k` distinct values. If `k` is nonpositive, it returns the empty subarray.
- `sliding_window_extrema(a, k, comp = std::less<>)` returns the extreme value in each window of length `k`. With the default `less<>` comparator it returns minimums; passing `greater<>` returns maximums.

#### Time Complexity:

- $O(n)$  per call to `two_sum_sorted()` and `min_subarray_at_least()`, where  $n$  is the array size.
- $O(n^2)$  per call to `three_sum()`.
- $O(n)$  expected per call to `max_subarray_at_most_k_distinct()`.
- $O(n)$  per call to `sliding_window_extrema()`.

#### Space Complexity:

- $O(1)$  auxiliary for `two_sum_sorted(a, target)`.
- $O(n)$  auxiliary for `three_sum(a, target)`.
- $O(1)$  auxiliary for `min_subarray_at_least(a, target)`.
- $O(k)$  auxiliary for `max_subarray_at_most_k_distinct(a, k)`.

- $O(k)$  auxiliary and  $O(n)$  for the result of `sliding_window_extrema()`.

```
std::pair<int, int> two_sum_sorted(const std::vector<int> &a, int64_t target) {
    for (int lo = 0, hi = static_cast<int>(a.size()) - 1; lo < hi;) {
        int64_t sum = static_cast<int64_t>(a[lo]) + a[hi];
        if (sum == target) {
            return {lo, hi};
        } else if (sum < target) {
            lo++;
        } else {
            hi--;
        }
    }
    return {-1, -1};
}

std::tuple<int, int, int> three_sum(const std::vector<int> &a, int64_t target) {
    int n = static_cast<int>(a.size());
    std::vector<std::pair<int, int>> sorted;
    sorted.reserve(a.size());
    for (int i = 0; i < n; i++) {
        sorted.emplace_back(a[i], i);
    }
    std::sort(sorted.begin(), sorted.end());
    for (int i = 0; i < n; i++) {
        int lo = i + 1, hi = n - 1;
        while (lo < hi) {
            int64_t sum = static_cast<int64_t>(sorted[i].first) + sorted[lo].first + sorted[hi].first;
            if (sum == target) {
                return {sorted[i].second, sorted[lo].second, sorted[hi].second};
            } else if (sum < target) {
                lo++;
            } else {
                hi--;
            }
        }
    }
    return {-1, -1, -1};
}

std::tuple<int, int, int> min_subarray_at_least(const std::vector<int> &a, int64_t target) {
    if (target <= 0) {
        return {0, 0, -1};
    }
    int best = static_cast<int>(a.size()) + 1, best_lo = 0, best_hi = -1;
    int64_t sum = 0;
    for (int lo = 0, hi = 0; hi < static_cast<int>(a.size()); hi++) {
        sum += a[hi]; // Overflow warning.
        while (sum >= target) {
            if (hi - lo + 1 < best) {
                best = hi - lo + 1;
                best_lo = lo;
                best_hi = hi;
            }
            sum -= a[lo++];
        }
    }
    if (best == static_cast<int>(a.size()) + 1) {
        return {-1, 0, -1};
    }
    return {best, best_lo, best_hi};
}

std::tuple<int, int, int> max_subarray_at_most_k_distinct(const std::vector<int> &a, int k) {
    if (k <= 0) {
        return {0, 0, -1};
    }
    std::unordered_map<int, int> count;
```

```

int best = 0, best_lo = 0, best_hi = -1;
for (int lo = 0, hi = 0; hi < static_cast<int>(a.size()); hi++) {
    count[a[hi]]++;
    while (static_cast<int>(count.size()) > k) {
        if (--count[a[lo]] == 0) {
            count.erase(a[lo]);
        }
        lo++;
    }
    if (best < hi - lo + 1) {
        best = hi - lo + 1;
        best_lo = lo;
        best_hi = hi;
    }
}
return {best, best_lo, best_hi};
}

template<typename T, typename Compare = std::less<>>
std::vector<T> sliding_window_extrema(const std::vector<T> &a, int k, Compare comp = Compare{}) {
    int n = static_cast<int>(a.size());
    assert(1 <= k && k <= n);
    std::deque<int> window;
    std::vector<T> res;
    res.reserve(n - k + 1);
    for (int i = 0; i < n; i++) {
        while (!window.empty() && !comp(a[window.back()], a[i])) {
            window.pop_back();
        }
        window.push_back(i);
        if (window.front() <= i - k) {
            window.pop_front();
        }
        if (i >= k - 1) {
            res.push_back(a[window.front()]);
        }
    }
    return res;
}

```

A monotone queue can be used to maintain the minimum or maximum value in an online sliding window. This is useful for dynamic programming recurrences where each transition may only come from one of the last  $w$  states, such as  $dp(i) = a_i + \min_{j \in [i-w, i-1]} dp(j)$ .

The queue stores candidate (`index`, `value`) pairs in monotone order. Expired indices are removed from the front, and dominated values are removed from the back before inserting a new candidate. An index is an ordered `int` timestamp used only for expiration: indices need not be contiguous or nonnegative, but they must be pushed in strictly increasing order, and expiration thresholds must not decrease.

- `MonotoneQueue<T>()` constructs an empty queue for minimum queries. Instantiate `MonotoneQueue<T, std::greater<T>>` for maximum queries.
- `empty()` returns whether the queue has no active candidates.
- `push(index, value)` inserts candidate `value` with timestamp `index`, removing dominated candidates from the back.
- `expire(first_valid)` removes candidates whose indices are less than `first_valid`.
- `top()` returns the best active (`index`, `value`) pair. The queue must be nonempty.

#### Time Complexity:

- $O(n)$  for any sequence of  $n$  calls to `push(index, value)` and `expire(first_valid)`.
- $O(1)$  amortized per call to `push(index, value)` and `expire(first_valid)`.
- $O(1)$  worst-case per call to `top()` and `empty()`.

**Space Complexity:**  $O(w)$  for storage of a sliding window of width  $w$ .

```

template<typename T, typename Compare = std::less<T>>
class MonotoneQueue {
    std::deque<std::pair<int, T>> q;
    Compare better;

public:
    bool empty() const { return q.empty(); }

    void push(int index, const T &value) {
        while (!q.empty() && !better(q.back().second, value)) {
            q.pop_back();
        }
        q.emplace_back(index, value);
    }

    void expire(int first_valid) {
        while (!q.empty() && q.front().first < first_valid) {
            q.pop_front();
        }
    }

    std::pair<int, T> top() const {
        assert(!q.empty());
        return q.front();
    }
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> c{-3, -1, 2, 4, 8, 11};
    assert(two_sum_sorted(c, 10) == make_pair(1, 5)); // -1 + 11.
    assert(two_sum_sorted(c, 7) == make_pair(1, 4));  // -1 + 8.
    assert(two_sum_sorted(c, 100) == make_pair(-1, -1));

    vector<int> d{4, -1, 8, 2, -3, 11};
    auto [i, j, k] = three_sum(d, 9);
    assert(d[i] + d[j] + d[k] == 9);
    assert((three_sum(d, 30) == make_tuple(-1, -1, -1)));

    vector<int> a{2, 3, 1, 2, 4, 3};
    auto [length, lo, hi] = min_subarray_at_least(a, 7);
    assert(length == 2 && lo == 4 && hi == 5); // [4, 3].
    auto [empty_length, empty_lo, empty_hi] = min_subarray_at_least(a, 0);
    assert(empty_length == 0 && empty_lo == 0 && empty_hi == -1);

    vector<int> b{1, 2, 1, 3, 4, 3, 5};
    auto [longest, longest_lo, longest_hi] = max_subarray_at_most_k_distinct(b, 2);
    assert(longest == 3 && longest_lo == 0 && longest_hi == 2); // [1, 2, 1].

    vector<int> e{1, 3, -1, -3, 5, 3, 6, 7};
    assert((sliding_window_extrema(e, 3) == vector<int>{-1, -3, -3, -3, 3, 3}));
    assert((sliding_window_extrema(e, 3, greater<>()) == vector<int>{3, 3, 5, 5, 6, 7}));

    // Toy DP: dp[i] = f[i] + min(dp[j]) for j in [i - w, i - 1].
    // Scanning the window costs O(w) per state; the monotone queue makes it O(1) amortized.
    vector<int> f{4, 2, 7, 1, 3, 6};
    int w = 3;
    vector<int> dp(f.size());
    MonotoneQueue<int> mq;
    dp[0] = f[0];
    mq.push(0, dp[0]);
    for (int i = 1; i < static_cast<int>(f.size()); i++) {
        mq.expire(i - w);
    }
}

```

```

    dp[i] = f[i] + mq.top().second;
    mq.push(i, dp[i]);
}
assert((dp == vector<int>{4, 6, 11, 5, 8, 11}));
return 0;
}

```

### 1.2.4 Monotonic Stack

A monotonic stack maintains its elements in sorted order as values are pushed, popping away any entries that would violate the ordering. Scanning an array once while keeping such a stack of indices answers, for every position, where the nearest smaller or larger neighbor lies. These “nearest smaller/greater” relationships underlie many array problems, the most famous being the largest rectangle in a histogram. Each index is pushed and popped at most once, so a full scan runs in linear time. A largest all-zero submatrix can then be found by sweeping rows, turning each row into a histogram of consecutive zeros ending at that row.

For any histogram bar, the nearest strictly lower bars to its left and right are the first positions that prevent a rectangle at the current bar’s height from extending farther. The widest such rectangle therefore spans the positions strictly between those boundaries. For a binary matrix, the height at each column counts consecutive zeros ending in the current row; every all-zero rectangle has some bottom row, so solving one histogram per row considers every possible rectangle.

All functions below take a vector `a` of  $n$  comparable values and return a vector of  $n$  indices. A “less” query uses a strictly smaller neighbor and a “greater” query uses a strictly larger one; changing the comparison from strict to non-strict (e.g. `>=` to `>`) toggles how ties are handled.

- `prev_less(a)` returns, for each  $i$ , the largest index  $j < i$  with  $a[j] < a[i]$ , or  $-1$  if there’s no such index.
- `next_less(a)` returns, for each  $i$ , the smallest index  $j > i$  with  $a[j] < a[i]$ , or  $n$  if there’s no such index.
- `prev_greater(a)` returns, for each  $i$ , the largest index  $j < i$  with  $a[j] > a[i]$ , or  $-1$  if there’s no such index.
- `next_greater(a)` returns, for each  $i$ , the smallest index  $j > i$  with  $a[j] > a[i]$ , or  $n$  if there’s no such index.
- `largest_histogram_rectangle(heights)` returns the maximum area and its inclusive left and right endpoints in a histogram where bars have width 1 and nonnegative heights given by `heights`.
- `largest_zero_submatrix(a)` returns the maximum area and its inclusive top, left, bottom, and right boundaries among all-zero rectangular submatrices of the 0/1 matrix `a`.

Overflow warning: Histogram areas must fit in the height value type.

#### Time Complexity:

- $O(n)$  per call to all one-dimensional functions, where  $n$  is the size of the input.
- $O(R \cdot C)$  per call to `largest_zero_submatrix()`, where  $R$  and  $C$  are the matrix dimensions.

#### Space Complexity:

- $O(n)$  auxiliary and  $O(n)$  for the returned indices from each neighbor query.
- $O(n)$  auxiliary for `largest_histogram_rectangle()`.
- $O(C)$  auxiliary for `largest_zero_submatrix()`.

```

#include <cassert>
#include <stack>
#include <tuple>
#include <vector>

template<typename T>
std::vector<int> prev_less(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && a[s.top()] >= a[i]) {

```



```

        s.pop();
    }
    res[i] = s.empty() ? -1 : s.top();
    s.push(i);
}
return res;
}

template<typename T>
std::vector<int> next_less(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = n - 1; i >= 0; i--) {
        while (!s.empty() && a[s.top()] >= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? n : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
std::vector<int> prev_greater(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && a[s.top()] <= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? -1 : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
std::vector<int> next_greater(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = n - 1; i >= 0; i--) {
        while (!s.empty() && a[s.top()] <= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? n : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
std::tuple<T, int, int> largest_histogram_rectangle(const std::vector<T> &heights) {
    int n = static_cast<int>(heights.size());
    std::vector<int> left = prev_less(heights), right = next_less(heights);
    T best{};
    int lo = 0, hi = -1;
    for (int i = 0; i < n; i++) {
        T area = heights[i] * (right[i] - left[i] - 1); // Overflow warning.
        if (area > best) {
            best = area;
            lo = left[i] + 1;
            hi = right[i] - 1;
        }
    }
    return {best, lo, hi};
}

```

```

}

std::tuple<int, int, int, int, int> largest_zero_submatrix(
    const std::vector<std::vector<char>> &a
) {
    if (a.empty()) {
        return {0, 0, 0, -1, -1};
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    int best = 0, rlo = 0, clo = 0, rhi = -1, chi = -1;
    std::vector<int> height(cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            height[j] = a[i][j] ? 0 : height[j] + 1;
        }
        auto [area, lo, hi] = largest_histogram_rectangle(height);
        if (area > best) {
            best = area;
            rlo = i - area / (hi - lo + 1) + 1;
            clo = lo;
            rhi = i;
            chi = hi;
        }
    }
    return {best, rlo, clo, rhi, chi};
}

```

### Example Usage

```

using namespace std;

int main() {
    vector<int> a{2, 1, 4, 3};
    assert((prev_less(a) == vector<int>{-1, -1, 1, 1}));
    assert((next_less(a) == vector<int>{1, 4, 3, 4}));
    assert((prev_greater(a) == vector<int>{-1, 0, -1, 2}));
    assert((next_greater(a) == vector<int>{2, 2, 4, 4}));

    vector<int> hist{2, 1, 5, 6, 2, 3};
    // The best histogram rectangle uses bars 2 and 3: min height 5 times width 2.
    assert((largest_histogram_rectangle(hist) == make_tuple(10, 2, 3)));
    assert((largest_histogram_rectangle(vector<int>{}) == make_tuple(0, 0, -1)));

    vector<vector<char>> grid{
        {1, 0, 1, 1, 0, 0},
        {1, 0, 0, 1, 0, 0},
        {0, 0, 0, 0, 0, 1},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 1, 0, 0, 1}
    };
    // The largest zero rectangle has 3 rows and 2 columns.
    assert((largest_zero_submatrix(grid) == make_tuple(6, 1, 1, 3, 2)));
    assert((largest_zero_submatrix({}) == make_tuple(0, 0, 0, -1, -1)));
    return 0;
}

```

## 1.2.5 Aggregate Queue

Maintain a first-in, first-out queue that additionally answers, in  $O(1)$  amortized time, the fold of an associative operation over all elements currently in the queue. This is the “sliding window aggregation” (SWAG) technique: by pushing new elements and popping old ones as a window advances, it reports an aggregate of every window in linear total time.

Unlike the monotonic deque used for sliding-window extrema (see 1.2.3), the operation need only be associative, not order-based, so the same structure handles min, max, sum, gcd, matrix products, and other monoids. It is sometimes called a “monotonic queue,” but that term usually denotes the monotonic deque of 1.2.3 (which handles only min/max); the two are different structures. The queue is built from two stacks: a back stack receives pushes while a front stack serves pops, and each stack stores running folds so the two halves combine in  $O(1)$ . Whenever the front stack empties, the back stack is flipped onto it; this costs  $O(1)$  amortized per element, since each element is moved between stacks at most once.

The fold is defined by an associative `combine(a, b)` function. The default code below returns the “min” of the elements; for “sum”, `combine(a, b)` should return `a + b`. The fold is taken in queue order (front to back), so non-commutative operations such as matrix products are also supported.

- `AggregateQueue<T>()` constructs an empty queue.
- `empty()` returns whether the queue is empty.
- `size()` returns the number of elements in the queue.
- `push(x)` appends `x` to the back of the queue.
- `pop()` removes the element at the front of the queue, which must be non-empty.
- `front()` returns the element at the front of the queue, which must be non-empty.
- `fold()` returns `combine()` applied to all elements in queue order; the queue must be non-empty.

#### Time Complexity:

- $O(1)$  per call to the constructor, `empty()`, `size()`, `front()`, and `fold()`.
- $O(1)$  amortized per call to `push()` and `pop()`.

#### Space Complexity:

- $O(n)$  for storage of the queue elements, where  $n$  is the number of elements.
- $O(1)$  auxiliary per operation, amortized (a `pop()` may flip the back stack onto the front stack).

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class AggregateQueue {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

    // Each stack entry is (value, fold), where fold combines the value with all entries between it
    // and its end of the queue (the front for front_stack, the back for back_stack).
    std::vector<std::pair<T, T>> front_stack, back_stack;

public:
    bool empty() const { return front_stack.empty() && back_stack.empty(); }
    int size() const { return static_cast<int>(front_stack.size() + back_stack.size()); }

    void push(const T &x) {
        T acc = back_stack.empty() ? x : combine(back_stack.back().second, x);
        back_stack.emplace_back(x, acc);
    }

    void pop() {
        assert(!empty());
        if (front_stack.empty()) {
            // Flip the back stack onto the front stack, reversing order so the oldest element ends up on
            // top, and recompute the front folds in queue order.
            while (!back_stack.empty()) {
                T x = back_stack.back().first;
                back_stack.pop_back();
                T acc = front_stack.empty() ? x : combine(x, front_stack.back().second);
                front_stack.emplace_back(x, acc);
            }
        }
        front_stack.pop_back();
    }
};
```

```

}

T front() const {
    assert(!empty());
    return front_stack.empty() ? back_stack.front().first : front_stack.back().first;
}

T fold() const {
    assert(!empty());
    if (front_stack.empty()) {
        return back_stack.back().second;
    }
    if (back_stack.empty()) {
        return front_stack.back().second;
    }
    return combine(front_stack.back().second, back_stack.back().second);
}
};

```

## Example Usage

```

using namespace std;

int main() {
    // Used directly as a FIFO queue with an O(1) min query.
    AggregateQueue<int> q;
    assert(q.empty());
    q.push(5);
    q.push(3);
    q.push(8);
    assert(!q.empty());
    assert(q.front() == 5 && q.size() == 3);
    assert(q.fold() == 3); // min(5, 3, 8)
    q.pop(); // Removes 5.
    assert(q.fold() == 3); // min(3, 8)
    q.pop(); // Removes 3.
    assert(q.fold() == 8);
    q.pop(); // Removes 8.
    assert(q.empty());

    // Canonical use: the minimum of every width-3 window, advancing the window by push/pop.
    vector<int> a{4, 2, 5, 1, 3, 6};
    AggregateQueue<int> w;
    vector<int> mins;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        w.push(a[i]);
        if (w.size() > 3) {
            w.pop();
        }
        if (w.size() == 3) {
            mins.push_back(w.fold());
        }
    }
    assert((mins == vector<int>{2, 1, 1, 1})); // min of [4,2,5], [2,5,1], [5,1,3], [1,3,6]
    return 0;
}

```

## 1.3 Dynamic Programming

### 1.3.1 Maximum Subarray Sum (Kadane)

Given an array of numbers, determine the maximum possible sum of any contiguous subarray. Kadane's algorithm scans the array while maintaining a nonnegative running sum. At each position, it adds the current value, records the largest sum seen, and resets the running sum to zero if it becomes negative. Discarding a negative sum is safe because it could only reduce every subarray extending it. This can be adapted to compute the maximal submatrix sum as well.

Two related scans maintain slightly different states. For a circular array, the best wrapped subarray is the whole-array sum minus the minimum-sum subarray. For a maximum-product subarray, a negative value swaps the roles of the minimum and maximum products ending at the previous position, so both products must be tracked.

- `max_subarray_sum(a)` returns the sum and inclusive endpoints (`sum`, `begin`, `end`) for the maximal-sum subarray of vector `a`. This implementation requires operators `+` and `<` on the value type. By convention, the empty subarray is allowed, so an input containing only negative values returns sum 0 and endpoints `[0, -1]`.
- `max_circular_subarray_sum(a)` returns a tuple (`sum`, `begin`, `end`) containing the maximal circular-subarray sum and its inclusive endpoints. If `begin ≤ end`, the result is an ordinary range; if `begin > end`, it wraps from `begin` through the end of `a` and continues through `end`. The empty subarray is allowed under the same convention as `max_subarray_sum()`.
- `max_product_subarray(a)` returns a tuple (`product`, `begin`, `end`) containing the product and inclusive endpoints of a nonempty maximal-product subarray, or product 0 and endpoints `[0, -1]` for an empty input.
- `max_submatrix_sum(a)` returns the sum and inclusive boundaries (`sum`, `r1`, `c1`, `r2`, `c2`) for the largest rectangular submatrix of a matrix `a`. This implementation requires operators `+` and `<` on the matrix value type. By convention, the empty submatrix is allowed, so a matrix containing only negative values returns sum 0 with empty row and column intervals.

Overflow warning: All resulting sums and products must fit in the value type.

#### Time Complexity:

- $O(n)$  per call to `max_subarray_sum()`, `max_circular_subarray_sum()`, and `max_product_subarray()`, where  $n$  is the size of `a`.
- $O(R \cdot C^2)$  per call to `max_submatrix_sum()`, where  $R$  is the number of rows and  $C$  is the number of columns in the matrix.

#### Space Complexity:

- $O(1)$  auxiliary for each subarray function.
- $O(R)$  auxiliary for `max_submatrix_sum()`.

```
#include <tuple>
#include <utility>
#include <vector>

template<typename T>
std::tuple<T, int, int> max_subarray_sum(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return {T{}, 0, -1};
    }
    int curr_begin = 0, begin = 0, end = -1;
    T sum = 0, max_sum = 0;
    for (int i = 0; i < n; i++) {
        sum += a[i]; // Overflow warning.
        if (sum < 0) {
            sum = 0;
            curr_begin = i + 1;
        } else if (max_sum < sum) {
```

```

        max_sum = sum;
        begin = curr_begin;
        end = i;
    }
}
return {max_sum, begin, end};
}

template<typename T>
std::tuple<T, int, int> max_circular_subarray_sum(const std::vector<T> &a) {
    auto [max_sum, begin, end] = max_subarray_sum(a);
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return {max_sum, begin, end};
    }
    int curr_begin = 0, min_begin = 0, min_end = -1;
    T total = 0, min_sum = 0, curr_sum = 0;
    for (int i = 0; i < n; i++) {
        total += a[i]; // Overflow warning.
        curr_sum += a[i]; // Overflow warning.
        if (0 < curr_sum) {
            curr_sum = 0;
            curr_begin = i + 1;
        } else if (curr_sum < min_sum) {
            min_sum = curr_sum;
            min_begin = curr_begin;
            min_end = i;
        }
    }
    T wrapped_sum = total - min_sum;
    if (max_sum < wrapped_sum) {
        max_sum = wrapped_sum;
        begin = (min_end + 1) % n;
        end = (min_begin + n - 1) % n;
    }
    return {max_sum, begin, end};
}

template<typename T>
std::tuple<T, int, int> max_product_subarray(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return {T{}, 0, -1};
    }
    T max_end = a[0], min_end = a[0], best = a[0];
    int max_begin = 0, min_begin = 0, begin = 0, end = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] < 0) {
            std::swap(max_end, min_end);
            std::swap(max_begin, min_begin);
        }
        T next_max = max_end * a[i]; // Overflow warning.
        T next_min = min_end * a[i]; // Overflow warning.
        if (next_max < a[i]) {
            max_end = a[i];
            max_begin = i;
        } else {
            max_end = next_max;
        }
        if (a[i] < next_min) {
            min_end = a[i];
            min_begin = i;
        } else {
            min_end = next_min;
        }
        if (best < max_end) {
            best = max_end;
            begin = max_begin;
        }
    }
}

```

```

        end = i;
    }
}
return {best, begin, end};
}

template<typename T>
std::tuple<T, int, int, int, int> max_submatrix_sum(const std::vector<std::vector<T>> &a) {
    if (a.empty() || a[0].empty()) {
        return {T{}, 0, 0, -1, -1};
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    int r1 = 0, c1 = 0, r2 = -1, c2 = -1;
    T max_sum = 0;
    for (int clo = 0; clo < cols; clo++) {
        std::vector<T> sums(rows);
        for (int chi = clo; chi < cols; chi++) {
            for (int i = 0; i < rows; i++) {
                sums[i] += a[i][chi]; // Overflow warning.
            }
            auto [sum, rlo, rhi] = max_subarray_sum(sums);
            if (max_sum < sum) {
                max_sum = sum;
                r1 = rlo;
                c1 = clo;
                r2 = rhi;
                c2 = chi;
            }
        }
    }
    return {max_sum, r1, c1, r2, c2};
}

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    {
        // All negative values, so the empty subarray is maximal.
        auto [sum, begin, end] = max_subarray_sum(vector<int>{-2, -1, -3});
        assert(sum == 0 && begin == 0 && end == -1);
    }
    {
        vector<int> a{-2, -1, -3, 4, -1, 2, 1, -5, 4};
        auto [sum, begin, end] = max_subarray_sum(a);
        assert(sum == 6);
        cout << "Maximal sum subarray:" << endl;
        for (int i = begin; i <= end; i++) {
            cout << a[i] << " ";
        }
        cout << endl;
    }
    {
        vector<int> a{5, -3, 5};
        auto [sum, begin, end] = max_circular_subarray_sum(a);
        assert(sum == 10 && begin == 2 && end == 0);
    }
    {
        vector<int> a{2, 3, -2, 4};
        auto [product, begin, end] = max_product_subarray(a);
        assert(product == 6 && begin == 0 && end == 1);
    }
}

```

```

vector<vector<int>>> a{
    {0, -2, -7, 0, 5},
    {9, 2, -6, 2, -4},
    {-4, 1, -4, 1, 0},
    {-1, 8, 0, -2, 3},
};
auto [sum, r1, c1, r2, c2] = max_submatrix_sum(a);
assert(sum == 15);
cout << "\nMaximal sum submatrix:" << endl;
for (int i = r1; i <= r2; i++) {
    for (int j = c1; j <= c2; j++) {
        cout << a[i][j] << " ";
    }
    cout << endl;
}
return 0;
}

```

### Example Output

Maximal sum subarray:  
4 -1 2 1

Maximal sum submatrix:  
9 2  
-4 1  
-1 8

## 1.3.2 Longest Increasing Subsequence

Given an array of elements, determine a longest subsequence where all elements are in strictly increasing order. The subsequence is not necessarily contiguous or unique, so only one such answer will be found. The answer is computed by scanning left to right while maintaining, for each length, the smallest possible tail value of an increasing subsequence of that length. Each element extends or improves one of these tails, located in  $O(\log n)$  by binary search; predecessor links then reconstruct the subsequence. The comparator `comp` defines the ordering and defaults to `std::less<>`.

- `longest_increasing_subsequence(a, comp = std::less<>())` returns the indices of one longest strictly increasing subsequence of `a` according to `comp`, in order.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the size of `a`.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned indices.

```

#include <algorithm>
#include <functional>
#include <vector>

template<typename T, typename Compare = std::less<>>
std::vector<int> longest_increasing_subsequence(const std::vector<T> &a, Compare comp = Compare{}) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return {};
    }
    int len = 0;
    // prev[i] = index of the predecessor of element i in its LIS (or -1 for the first LIS element).
    // tail[i] = index of the last element of the best known LIS of length i + 1.
    std::vector<int> prev(n), tail(n);
    for (int i = 0; i < n; i++) {
        // Find the tail to extend or improve. Comparing by value through the stored indices keeps tail
        // index-based for reconstruction. Switch to an upper-bound search for a non-decreasing result.
        int pos = static_cast<int>(
            std::lower_bound(

```



```

        tail.begin(), tail.begin() + len, i, [&](int t, int x) { return comp(a[t], a[x]); }
    ) -
    tail.begin()
);
if (pos == len) {
    len++;
}
prev[i] = pos > 0 ? tail[pos - 1] : -1;
tail[pos] = i;
}
// Optional: reconstruct one longest increasing subsequence.
std::vector<int> res(len);
for (int i = tail[len - 1]; i != -1; i = prev[i]) {
    res[--len] = i;
}
return res;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{-2, -5, 1, 9, 10, 8, 11, 10, 13, 11};
    assert((longest_increasing_subsequence(a) == vector<int>{1, 2, 3, 4, 6, 8}));

    vector<int> b{1, 4, 3, 2};
    assert((longest_increasing_subsequence(b, greater<int>()) == vector<int>{1, 2, 3}));
    return 0;
}

```

### 1.3.3 0-1 Knapsack

Solves the 0-1 knapsack problem: given items with nonnegative integer weights and values, choose each item at most once to maximize total value without exceeding a capacity limit.

The one-dimensional dynamic programming array `dp[w]` stores the best value that can be achieved with total weight at most `w` after processing some prefix of the items. Iterating weights in decreasing order is what enforces the 0-1 constraint; it prevents the same item from being reused during the current item update.

- `knapsack_01(weight, value, capacity)` returns a pair `(best_value, items)` containing the maximum value and the selected item indices in increasing order, where item `i` has weight `weight[i]` and value `value[i]`. All weights must be nonnegative integers. Items with weight 0 are allowed and can each be taken once. The take/skip decision for every item and capacity is stored to reconstruct the optimal subset.

**Time Complexity:**  $O(n \cdot W)$  per call, where  $n$  is the number of items and  $W$  is the capacity.

**Space Complexity:**  $O(n \cdot W)$  auxiliary.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>> knapsack_01(
    const std::vector<int> &weight, const std::vector<int64_t> &value, int capacity
) {
    int n = static_cast<int>(weight.size());
    assert(static_cast<int>(value.size()) == n && capacity >= 0);
    for (int i = 0; i < n; i++) {

```

```

    assert(weight[i] >= 0 && value[i] >= 0);
}
std::vector<int64_t> dp(capacity + 1);
std::vector<std::vector<char>> take(n, std::vector<char>(capacity + 1));
for (int i = 0; i < n; i++) {
    for (int w = capacity; w >= weight[i]; w--) {
        int64_t candidate = dp[w - weight[i]] + value[i]; // Overflow warning.
        if (dp[w] < candidate) {
            dp[w] = candidate;
            take[i][w] = true;
        }
    }
}
// Optional: reconstruct one optimal subset.
std::vector<int> items;
for (int i = n - 1, w = capacity; i >= 0; i--) {
    if (take[i][w]) {
        items.push_back(i);
        w -= weight[i];
    }
}
std::reverse(items.begin(), items.end());
return {dp[capacity], items};
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> weight{3, 4, 5, 9};
    vector<int64_t> value{4, 5, 7, 10};
    auto [best_value, items] = knapsack_01(weight, value, 8);
    assert(best_value == 11 && (items == vector<int>{0, 2}));
    return 0;
}

```

### 1.3.4 Unbounded Knapsack and Coin Change

Solves unbounded knapsack and two related coin change problems, where each item or coin may be used any number of times. In the maximum-value version, each item has a weight and value, and the goal is to maximize total value subject to a capacity limit. In coin change variants, the goal is either to count unordered ways to form a target sum or to minimize the number of coins used.

For unbounded maximum-value knapsack, capacities are processed in increasing order. Thus, when computing `dp[w]`, every smaller-capacity state `dp[w - weight[i]]` is already final and may itself contain item `i`, allowing unlimited reuse. For unordered coin change, coins are the outer loop so each multiset of coins is counted once, independent of ordering. For minimum coins, each target sum chooses the best previous sum `sum - coin`, and the predecessor coin reconstructs one optimal multiset.

- `unbounded_knapsack(weight, value, capacity)` returns a pair `(best_value, count)` containing the maximum value achievable with total weight at most `capacity` and `count[i]`, the number of copies selected of each item `i`.
- `count_coin_change(coins, target)` returns the number of unordered ways to make sum `target` using the given coin denominations.
- `min_coin_change(coins, target)` returns a pair `(count, used)`, where `count` is the minimum number of coins needed to make sum `target`, and `used[i]` is the number of copies of `coins[i]` in one optimal multiset. If `target` cannot be formed, `count` is `-1` and `used` is empty.

`capacity` and `target` must be nonnegative integers. Weights and coin denominations must be positive. Coin denominations must also be distinct when counting ways; duplicate denominations are treated as different coin types and overcount identical multisets.

#### Time Complexity:

- $O(n \cdot W)$  per call to `unbounded_knapsack()`, where  $n$  is the number of items and  $W$  is `capacity`.
- $O(c \cdot t)$  per call to `count_coin_change()` and `min_coin_change()`, where  $c$  is the number of coin denominations and  $t$  is the target.

#### Space Complexity:

- $O(W)$  auxiliary and  $O(n)$  for the returned `count` from `unbounded_knapsack()`.
- $O(t)$  auxiliary for `count_coin_change()`.
- $O(t)$  auxiliary and  $O(c)$  for the returned `used` from `min_coin_change()`.

```
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>> unbounded_knapsack(
    const std::vector<int> &weight, const std::vector<int64_t> &value, int capacity
) {
    int n = static_cast<int>(weight.size());
    assert(static_cast<int>(value.size()) == n && capacity >= 0);
    for (int w : weight) {
        assert(w > 0);
    }
    std::vector<int64_t> dp(capacity + 1);
    std::vector<int> choice(capacity + 1, -1);
    for (int w = 1; w <= capacity; w++) {
        for (int i = 0; i < n; i++) {
            if (weight[i] <= w) {
                int64_t candidate = dp[w - weight[i]] + value[i]; // Overflow warning.
                if (dp[w] < candidate) {
                    dp[w] = candidate;
                    choice[w] = i;
                }
            }
        }
    }
    // Optional: reconstruct one optimal multiset of items.
    std::vector<int> count(n);
    for (int w = capacity; choice[w] != -1; w -= weight[choice[w]]) {
        count[choice[w]]++;
    }
    return {dp[capacity], count};
}

int64_t count_coin_change(const std::vector<int> &coins, int target) {
    assert(target >= 0);
    std::vector<int64_t> dp(target + 1);
    dp[0] = 1;
    for (int coin : coins) {
        assert(coin > 0);
        for (int sum = coin; sum <= target; sum++) {
            dp[sum] += dp[sum - coin]; // Overflow warning.
        }
    }
    return dp[target];
}

std::pair<int, std::vector<int>> min_coin_change(const std::vector<int> &coins, int target) {
    assert(target >= 0);
    for (int coin : coins) {
        assert(coin > 0);
    }
}
```

```

}
int inf = target + 1;
std::vector<int> dp(target + 1, inf), choice(target + 1, -1);
dp[0] = 0;
for (int sum = 1; sum <= target; sum++) {
    for (int i = 0; i < static_cast<int>(coins.size()); i++) {
        if (coins[i] <= sum && dp[sum] > dp[sum - coins[i]] + 1) {
            dp[sum] = dp[sum - coins[i]] + 1;
            choice[sum] = i;
        }
    }
}
if (dp[target] == inf) {
    return {-1, {}};
}
// Optional: reconstruct one minimum-size multiset of coins.
std::vector<int> used(coins.size());
for (int sum = target; sum > 0; sum -= coins[choice[sum]]) {
    used[choice[sum]]++;
}
return {dp[target], used};
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> weight{3, 4, 5};
    vector<int64_t> value{4, 5, 7};
    auto [best_value, count] = unbounded_knapsack(weight, value, 10);
    assert(best_value == 14 && (count == vector<int>{0, 0, 2}));

    vector<int> coins{1, 2, 5};
    assert(count_coin_change(coins, 5) == 4); // 5, 2+2+1, 2+1+1+1, 1*5.

    auto [coin_count, used] = min_coin_change(coins, 11);
    assert(coin_count == 3 && (used == vector<int>{1, 0, 2}));

    auto [impossible, empty_used] = min_coin_change(vector<int>{4, 7}, 6);
    assert(impossible == -1 && empty_used.empty());
    return 0;
}

```

### 1.3.5 Matrix Chain Parenthesization

Given a fixed-order chain of compatible matrices, finds a parenthesization that minimizes the number of scalar multiplications needed to compute their product. The algorithm examines only the matrix dimensions; it does not multiply matrix entries. Matrix multiplication is associative, but multiplying an  $a$ -by- $b$  matrix with a  $b$ -by- $c$  matrix costs  $abc$  scalar multiplications, so different parenthesizations can have very different costs. This is a canonical example of interval dynamic programming.

Write  $d_i$  for `dimensions[i]`, so matrix  $i$  has  $d_i$  rows and  $d_{i+1}$  columns. For a chain of matrices,  $dp(l, r)$  is the minimum cost of multiplying matrices  $l$  through  $r$ . Trying every final split  $k$  gives the recurrence  $dp(l, r) = \min_{l \leq k < r} (dp(l, k) + dp(k + 1, r) + d_l d_{k+1} d_{r+1})$ . The final product multiplies a  $d_l$ -by- $d_{k+1}$  matrix with a  $d_{k+1}$ -by- $d_{r+1}$  matrix. Processing intervals in increasing order of length ensures that both subintervals have already been solved. The same pattern applies whenever a contiguous interval is formed by combining two smaller intervals.

- `matrix_chain_order(dimensions)` returns a pair (`operations`, `parenthesization`) containing the minimum number of scalar multiplications and one optimal parenthesization. Matrix `i` has dimensions `dimensions[i]` by `dimensions[i + 1]`; all dimensions must be positive.

Overflow warning: The minimum cost and all intermediate products must fit in `int64_t`.

**Time Complexity:**  $O(n^3)$  per call, where  $n$  is the number of matrices. The dimension values affect the returned cost but not the running time.

**Space Complexity:**  $O(n^2)$  auxiliary for the dynamic programming and split tables.

```
#include <cassert>
#include <cstdint>
#include <string>
#include <utility>
#include <vector>

std::pair<int64_t, std::string> matrix_chain_order(const std::vector<int> &dimensions) {
    assert(dimensions.size() >= 2);
    for (int d : dimensions) {
        assert(d > 0);
    }
    int n = static_cast<int>(dimensions.size()) - 1;
    std::vector<std::vector<int64_t>> dp(n, std::vector<int64_t>(n));
    std::vector<std::vector<int>> split(n, std::vector<int>(n));
    for (int length = 2; length <= n; length++) {
        for (int lo = 0; lo + length <= n; lo++) {
            int hi = lo + length - 1;
            for (int k = lo; k < hi; k++) {
                int64_t candidate = dp[lo][k] + dp[k + 1][hi] +
                                   static_cast<int64_t>(dimensions[lo]) * dimensions[k + 1] *
                                   dimensions[hi + 1]; // Overflow warning.
                if (k == lo || candidate < dp[lo][hi]) {
                    dp[lo][hi] = candidate;
                    split[lo][hi] = k;
                }
            }
        }
    }
    // Optional: reconstruct one optimal parenthesization.
    auto rec = [&](auto &&rec, int lo, int hi) -> std::string {
        if (lo == hi) {
            return "A" + std::to_string(lo);
        }
        int k = split[lo][hi];
        return "(" + rec(rec, lo, k) + " * " + rec(rec, k + 1, hi) + ")";
    };
    return {dp[0][n - 1], rec(rec, 0, n - 1)};
}
```

## Example Usage

```
using namespace std;

int main() {
    auto [ops, parenthesization] = matrix_chain_order({40, 20, 30, 10, 30});
    assert(ops == 26000);
    assert(parenthesization == "((A0 * (A1 * A2)) * A3)");

    auto [single_ops, single_parenthesization] = matrix_chain_order({3, 5});
    assert(single_ops == 0 && single_parenthesization == "A0");
    return 0;
}
```

### 1.3.6 Digit DP

Digit dynamic programming counts integers in a large bounded range whose decimal representations satisfy a digit-based property. The concrete routine below counts integers with a specified digit sum and provides the standard template to adapt for other finite-state properties.

To count valid integers in  $[0, x]$ , process the digits of  $x$  from left to right. The state records the current position, accumulated digit sum, and whether the chosen prefix still equals the prefix of  $x$ . Once that “tight” condition is false, the remaining choices depend only on the position and sum, so those states can be memoized. Subtracting the prefix counts for `hi` and `lo - 1` gives the answer for the requested range.

Leading zeros do not affect a digit sum, so every integer can be treated as a zero-padded string of the same length as the bound. For properties where leading zeros matter, add a `started` flag to the state. More generally, the sum can be replaced by any small state with a transition for appending a digit.

- `count_digit_sum(lo, hi, target)` returns the number of integers in the inclusive range  $[lo, hi]$  whose decimal digits sum to `target`. The bounds and `target` must be nonnegative.

**Time Complexity:**  $O(d \cdot s)$  per call, where  $d$  is the number of digits in `hi` and  $s$  is `target`.

**Space Complexity:**  $O(d \cdot s)$  auxiliary heap space and  $O(d)$  call stack space.

```
#include <cassert>
#include <cstdint>
#include <string>
#include <vector>

int64_t count_digit_sum(int64_t lo, int64_t hi, int target) {
    assert(0 <= lo && lo <= hi && target >= 0);
    auto count_up_to = [&](int64_t bound) {
        if (bound < 0) {
            return 0LL;
        }
        std::string digits = std::to_string(bound);
        int n = static_cast<int>(digits.size());
        if (target > 9 * n) {
            return 0LL;
        }
        std::vector<std::vector<int64_t>> memo(n, std::vector<int64_t>(target + 1, -1));
        auto rec = [&](auto &&rec, int pos, int sum, bool tight) -> int64_t {
            if (sum > target || sum + 9 * (n - pos) < target) {
                return 0;
            }
            if (pos == n) {
                return sum == target;
            }
            if (!tight && memo[pos][sum] != -1) {
                return memo[pos][sum];
            }
            int bound_digit = digits[pos] - '0';
            int limit = tight ? bound_digit : 9;
            int64_t res = 0;
            for (int digit = 0; digit <= limit; digit++) {
                res += rec(rec, pos + 1, sum + digit, tight && digit == bound_digit);
            }
            if (!tight) {
                memo[pos][sum] = res;
            }
            return res;
        };
        return rec(rec, 0, 0, true);
    };
    return count_up_to(hi) - count_up_to(lo - 1);
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(count_digit_sum(0, 20, 2) == 3); // 2, 11, and 20.
    assert(count_digit_sum(10, 30, 3) == 3); // 12, 21, and 30.
    assert(count_digit_sum(0, 99, 9) == 10);
    assert(count_digit_sum(0, 0, 0) == 1);
    assert(count_digit_sum(0, 999, 28) == 0);
    return 0;
}
```

### 1.3.7 Egg Dropping

Given  $e$  identical eggs and a building of  $f$  floors, find the highest floor from which an egg survives a drop, using as few drops as possible in the worst case. An egg that survives can be reused; an egg that breaks is gone. With one egg, the only safe strategy is to climb one floor at a time, needing  $f$  drops. With unlimited eggs, binary search needs  $\lceil \log_2(f+1) \rceil$  drops. The problem is what happens in between.

The natural recurrence minimizes over the drop floor and maximizes over the two outcomes, costing  $O(e \cdot f^2)$ . Removing that minimization is a technique worth knowing beyond this puzzle: when the quantity being minimized is small and bounded, promote it to a state dimension and solve for the old one. So ask how many floors  $d$  drops with  $e$  eggs can cover, not how many drops  $f$  floors need. A drop splits the searchable floors into those below it, the drop floor, and those above, giving  $dp(d, e) = dp(d-1, e-1) + dp(d-1, e) + 1$  with nothing left to minimize. Each entry is then  $O(1)$ , and the answer is the smallest  $d$  whose reach is at least  $f$ .

That table is  $dp(d, e) = \sum_{i=1}^e \binom{d}{i}$  in the binomial coefficients of section 6.2.1, which is what keeps  $d$  small: enough eggs sum to  $2^d - 1$ , while one egg leaves only  $\binom{d}{1} = d$ . Two eggs reach 91 floors in 13 drops and 105 in 14, the classic answer. The same inversion fits any question of how many tests distinguish  $n$  cases, such as finding one defective item with a limited number of destructive tests.

- `min_drops(eggs, floors)` returns the number of drops needed in the worst case, where `eggs` is positive and `floors` is nonnegative.
- `drop_floor(eggs, floors)` returns the floor to drop from first under an optimal strategy, which is  $dp(d-1, e-1) + 1$  floors above the bottom of the current range, or 0 when no drop is needed.

**Time Complexity:**  $O(e \cdot d)$  per call, where  $e$  is `eggs` and  $d$  is the returned number of drops, which is at most  $f$  and is  $O(\log f)$  once  $e \geq \log_2(f)$ .

**Space Complexity:**  $O(e)$  auxiliary.

```
#include <cassert>
#include <cstdint>
#include <vector>

int min_drops(int eggs, int floors) {
    assert(eggs > 0 && floors >= 0);
    // reach[e] is the number of floors distinguishable by the current number of drops with e eggs.
    std::vector<int64_t> reach(eggs + 1);
    int drops = 0;
    while (reach[eggs] < floors) {
        drops++;
        for (int e = eggs; e > 0; e--) { // Descending, so reach[e - 1] is still the previous row.
            reach[e] += reach[e - 1] + 1;
        }
    }
    return drops;
}
```

```

int drop_floor(int eggs, int floors) {
    assert(eggs > 0 && floors >= 0);
    int drops = min_drops(eggs, floors);
    if (drops == 0) {
        return 0;
    }
    std::vector<int64_t> reach(eggs + 1);
    for (int i = 0; i < drops - 1; i++) { // Rebuild the table one drop short of the answer.
        for (int e = eggs; e > 0; e--) {
            reach[e] += reach[e - 1] + 1;
        }
    }
    return static_cast<int>(reach[eggs - 1]) + 1;
}

```

## Example Usage

```

#include <cassert>

int main() {
    assert(min_drops(1, 0) == 0);
    assert(min_drops(1, 10) == 10); // One egg forces a linear scan.
    assert(min_drops(2, 10) == 4);
    assert(min_drops(2, 100) == 14); // The classic two-egg, hundred-floor puzzle.
    assert(min_drops(3, 100) == 9);
    assert(min_drops(50, 1000) == 10); // Enough eggs to binary search.

    // With two eggs and a hundred floors, the first drop is from floor 14.
    assert(drop_floor(2, 100) == 14);
    assert(drop_floor(1, 10) == 1);
    assert(drop_floor(2, 0) == 0);
    return 0;
}

```

## 1.4 Grid Algorithms

---

### 1.4.1 Grid Transformations

Transforms a rectangular grid by transposing, rotating, or reflecting its cells. Transposition exchanges rows and columns. Rotation is clockwise by default, while a negative angle rotates counter-clockwise; the angle must be a multiple of 90 degrees. Together, rotations and reflections generate all eight symmetries of a square grid.

- `transpose(a)` returns the transpose of grid `a`, so each output cell  $(c, r)$  equals input cell  $(r, c)$ .
- `transpose_in_place(a)` transposes the square grid `a` in place and returns a reference to it.
- `rotate(a, degrees = 90)` returns `a` rotated clockwise by `degrees`.
- `rotate_in_place(a, degrees = 90)` rotates `a` in place and returns a reference to it. The grid must be square for rotations by 90 or 270 degrees.
- `reflect_horizontal(a)` returns `a` reflected across its horizontal axis, reversing the row order.
- `reflect_vertical(a)` returns `a` reflected across its vertical axis, reversing every row.

**Time Complexity:**  $O(R \cdot C)$  per call, where  $R$  and  $C$  are the grid dimensions.

**Space Complexity:**

- $O(R \cdot C)$  for the returned grid from the non-in-place operations.
- $O(1)$  auxiliary for the in-place operations.

```

#include <algorithm>
#include <cassert>

```



```

#include <utility>
#include <vector>

using Grid = std::vector<std::vector<int>>>;

Grid transpose(const Grid &a) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    Grid res(cols, std::vector<int>(rows));
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            res[c][r] = a[r][c];
        }
    }
    return res;
}

Grid &transpose_in_place(Grid &a) {
    assert(a.empty() || a.size() == a[0].size());
    for (int r = 0; r < static_cast<int>(a.size()); r++) {
        for (int c = r + 1; c < static_cast<int>(a.size()); c++) {
            std::swap(a[r][c], a[c][r]);
        }
    }
    return a;
}

Grid rotate(const Grid &a, int degrees = 90) {
    assert(degrees % 90 == 0);
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    degrees = (degrees % 360 + 360) % 360;
    Grid res(degrees % 180 == 0 ? rows : cols, std::vector<int>(degrees % 180 == 0 ? cols : rows));
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (degrees == 90) {
                res[c][rows - r - 1] = a[r][c];
            } else if (degrees == 180) {
                res[rows - r - 1][cols - c - 1] = a[r][c];
            } else if (degrees == 270) {
                res[cols - c - 1][r] = a[r][c];
            } else {
                res[r][c] = a[r][c];
            }
        }
    }
    return res;
}

Grid &rotate_in_place(Grid &a, int degrees = 90) {
    assert(degrees % 90 == 0);
    degrees = (degrees % 360 + 360) % 360;
    assert(degrees % 180 == 0 || a.empty() || a.size() == a[0].size());
    if (degrees == 90) {
        transpose_in_place(a);
        for (auto &row : a) {
            std::reverse(row.begin(), row.end());
        }
    } else if (degrees == 180) {
        std::reverse(a.begin(), a.end());
        for (auto &row : a) {
            std::reverse(row.begin(), row.end());
        }
    } else if (degrees == 270) {
        transpose_in_place(a);
        std::reverse(a.begin(), a.end());
    }
    return a;
}

```

```

}

Grid reflect_horizontal(Grid a) {
    std::reverse(a.begin(), a.end());
    return a;
}

Grid reflect_vertical(Grid a) {
    for (auto &row : a) {
        std::reverse(row.begin(), row.end());
    }
    return a;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Grid a{{1, 2, 3}, {4, 5, 6}};
    assert((transpose(a) == Grid{{1, 4}, {2, 5}, {3, 6}}));
    assert((rotate(a) == Grid{{4, 1}, {5, 2}, {6, 3}}));
    assert((rotate(a, -90) == Grid{{3, 6}, {2, 5}, {1, 4}}));
    assert((rotate(a, 180) == Grid{{6, 5, 4}, {3, 2, 1}}));
    assert((reflect_horizontal(a) == Grid{{4, 5, 6}, {1, 2, 3}}));
    assert((reflect_vertical(a) == Grid{{3, 2, 1}, {6, 5, 4}}));

    Grid square{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    for (int degrees = -360; degrees <= 360; degrees += 90) {
        Grid rotated = square;
        assert(rotate_in_place(rotated, degrees) == rotate(square, degrees));
    }
    Grid transposed = square;
    assert(transpose_in_place(transposed) == transpose(square));

    Grid rectangular = a;
    assert(rotate_in_place(rectangular, 180) == rotate(a, 180));
    return 0;
}

```

### 1.4.2 Grid Traversal

Traverses a rectangular grid as an implicit unweighted graph: each unblocked cell is a node joined to its unblocked (four-way by default) orthogonal neighbors. Depth-first flood fill finds connected regions, while breadth-first search finds shortest distances because every move has unit length. Seeding the BFS with several cells computes the distance to the nearest source in one pass.

- `inside(r, c, rows, cols)` returns whether  $0 \leq r < \text{rows}$  and  $0 \leq c < \text{cols}$ .
- `adj4(r, c)` returns the four orthogonally adjacent coordinates without filtering those outside the grid. For eight-way movement, also include the four diagonal neighbors.
- `component_area_perimeter(blocked, r, c)` uses flood fill to return the area and perimeter of the component containing `(r, c)`. If the starting cell is blocked, both values are 0.
- `grid_components(blocked)` uses repeated flood fills to return the connected components of cells marked zero, with each cell represented by a pair `(r, c)`. Movement is allowed in the four orthogonal directions.
- `grid_bfs(blocked, sources)` returns the pair `(dist, pred)` from every cell in `sources`. Each source must be an unblocked cell represented by a pair `(r, c)`. Passing one source gives ordinary single-source BFS. Blocked and unreachable cells have distance `-1`.
- `get_path(pred, tr, tc)` uses the predecessor grid returned by `grid_bfs()` to return the path from a nearest source to `(tr, tc)`, including both endpoints, or an empty vector if the target is blocked or unreachable.

- `spiral_order(a)` returns the cells of grid `a` in clockwise spiral order, starting at the top-left corner and walking inward.

#### Time Complexity:

- $O(R \cdot C)$  per call to `spiral_order()`.
- $O(R \cdot C)$  per call to `component_area_perimeter()`, `grid_components()`, or `grid_bfs()`, where  $R$  and  $C$  are the grid dimensions.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of cells in the returned path.

#### Space Complexity:

- $O(R \cdot C)$  auxiliary, including the recursion stack used by `component_area_perimeter()` and `grid_components()`.
- $O(R \cdot C)$  for the returned `dist` and `pred`, and  $O(p)$  for the path returned by `get_path()`.

```
#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <queue>
#include <utility>
#include <vector>

bool inside(int r, int c, int rows, int cols) {
    return 0 <= r && r < rows && 0 <= c && c < cols;
}

std::array<std::pair<int, int>, 4> adj4(int r, int c) {
    return {{r - 1, c}, {r + 1, c}, {r, c - 1}, {r, c + 1}};
}

template<typename Fn>
void grid_dfs(
    const std::vector<std::vector<char>> &blocked, int r, int c,
    std::vector<std::vector<char>> &visit, const Fn &f
) {
    int rows = static_cast<int>(blocked.size());
    int cols = static_cast<int>(blocked[0].size());
    visit[r][c] = true;
    f(r, c);
    for (auto [r2, c2] : adj4(r, c)) {
        if (inside(r2, c2, rows, cols) && !blocked[r2][c2] && !visit[r2][c2]) {
            grid_dfs(blocked, r2, c2, visit, f);
        }
    }
}

std::pair<int, int> component_area_perimeter(
    const std::vector<std::vector<char>> &blocked, int r, int c
) {
    int rows = static_cast<int>(blocked.size());
    int cols = blocked.empty() ? 0 : static_cast<int>(blocked[0].size());
    assert(inside(r, c, rows, cols));
    if (blocked[r][c]) {
        return {0, 0};
    }
    int area = 0, perimeter = 0;
    std::vector<std::vector<char>> visit(rows, std::vector<char>(cols));
    grid_dfs(blocked, r, c, visit, [&](int r, int c) {
        area++;
        for (auto [r2, c2] : adj4(r, c)) {
            perimeter += !inside(r2, c2, rows, cols) || blocked[r2][c2];
        }
    });
    return {area, perimeter};
}
```

```

std::vector<std::vector<std::pair<int, int>>> grid_components(
    const std::vector<std::vector<char>> &blocked
) {
    int rows = static_cast<int>(blocked.size());
    int cols = blocked.empty() ? 0 : static_cast<int>(blocked[0].size());
    std::vector<std::vector<std::pair<int, int>>> components;
    std::vector<std::vector<char>> visit(rows, std::vector<char>(cols));
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (blocked[r][c] || visit[r][c]) {
                continue;
            }
            components.emplace_back();
            grid_dfs(blocked, r, c, visit, [&](int r, int c) { components.back().emplace_back(r, c); });
        }
    }
    return components;
}

auto grid_bfs(
    const std::vector<std::vector<char>> &blocked, const std::vector<std::pair<int, int>> &sources
) {
    int rows = static_cast<int>(blocked.size());
    int cols = blocked.empty() ? 0 : static_cast<int>(blocked[0].size());
    std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));
    std::vector<std::vector<std::pair<int, int>>> pred(
        rows, std::vector<std::pair<int, int>>(cols, {-1, -1})
    );
    std::queue<std::pair<int, int>> q;
    for (auto [r, c] : sources) {
        assert(inside(r, c, rows, cols) && !blocked[r][c]);
        if (dist[r][c] == -1) {
            dist[r][c] = 0;
            pred[r][c] = {r, c};
            q.emplace(r, c);
        }
    }
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (auto [r2, c2] : adj4(r, c)) {
            if (inside(r2, c2, rows, cols) && !blocked[r2][c2] && dist[r2][c2] == -1) {
                dist[r2][c2] = dist[r][c] + 1;
                pred[r2][c2] = {r, c};
                q.emplace(r2, c2);
            }
        }
    }
    return std::pair{std::move(dist), std::move(pred)};
}

std::vector<std::pair<int, int>> get_path(
    const std::vector<std::vector<std::pair<int, int>>> &pred, int tr, int tc
) {
    int rows = static_cast<int>(pred.size());
    int cols = pred.empty() ? 0 : static_cast<int>(pred[0].size());
    assert(inside(tr, tc, rows, cols));
    if (pred[tr][tc].first == -1) {
        return {};
    }
    std::vector<std::pair<int, int>> path;
    std::pair cur{tr, tc};
    while (pred[cur.first][cur.second] != cur) {
        path.push_back(cur);
        cur = pred[cur.first][cur.second];
    }
    path.push_back(cur);
    std::reverse(path.begin(), path.end());
}

```

```

    return path;
}

template<typename T>
std::vector<T> spiral_order(const std::vector<std::vector<T>> &a) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    std::vector<T> res;
    res.reserve(static_cast<size_t>(rows) * cols);
    // Peel one ring at a time. Guard the last 2 passes so single remaining rows/cols don't repeat.
    for (int top = 0, bottom = rows - 1, left = 0, right = cols - 1; top <= bottom && left <= right;
        top++, bottom--, left++, right--) {
        for (int c = left; c <= right; c++) {
            res.push_back(a[top][c]);
        }
        for (int r = top + 1; r <= bottom; r++) {
            res.push_back(a[r][right]);
        }
        if (top < bottom) {
            for (int c = right - 1; c >= left; c--) {
                res.push_back(a[bottom][c]);
            }
        }
        if (left < right) {
            for (int r = bottom - 1; r > top; r--) {
                res.push_back(a[r][left]);
            }
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<char>> blocked{
        {0, 0, 1, 0},
        {1, 0, 1, 0},
        {0, 0, 1, 0},
    };
    auto components = grid_components(blocked);
    assert(components.size() == 2);
    assert(components[0].size() == 5 && components[1].size() == 3);
    assert((component_area_perimeter(blocked, 0, 0) == pair{5, 12}));
    assert((component_area_perimeter(blocked, 0, 2) == pair{0, 0}));

    auto [dist, pred] = grid_bfs(blocked, {{0, 0}, {2, 3}});
    assert((dist == vector<vector<int>>{{0, 1, -1, 2}, {-1, 2, -1, 1}, {4, 3, -1, 0}}));
    assert((get_path(pred, 2, 1) == vector<pair<int, int>>{{0, 0}, {0, 1}, {1, 1}, {2, 1}}));
    assert((get_path(pred, 0, 3) == vector<pair<int, int>>{{2, 3}, {1, 3}, {0, 3}}));
    assert(get_path(pred, 0, 2).empty());

    vector<vector<int>> g{
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12},
    };
    assert((spiral_order(g) == vector<int>{1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7}));
    assert((spiral_order(vector<vector<int>>{{1, 2, 3}}) == vector<int>{1, 2, 3}));
    assert((spiral_order(vector<vector<int>>{{1}, {2}, {3}}) == vector<int>{1, 2, 3}));
    return 0;
}

```

### 1.4.3 Grid DP

Solves basic dynamic programming problems on a rectangular grid. In grid DP, each cell represents a state and transitions usually come from already-computed neighboring cells. Processing rows from top to bottom and columns from left to right makes predecessors above and to the left available when a state is computed.

- `count_grid_paths(blocked)` returns the number of paths from the upper-left cell to the lower-right cell, moving only down or right and avoiding blocked cells marked nonzero. The state  $dp(r, c)$  counts paths to cell  $(r, c)$  from the counts above and left. Only the current row is retained.
- `min_cost_grid_path(cost)` returns a pair  $(\text{min\_cost}, \text{path})$  containing the minimum cost of a down/right path from the upper-left cell to the lower-right cell and the cells of one optimal path in order. The state  $dp(r, c)$  stores the cheapest cost to cell  $(r, c)$ . Only the current row of costs is retained, while predecessor directions reconstruct one optimal path. If the grid is empty, `min_cost` is 0 and `path` is empty.
- `largest_nonzero_square(a)` returns a tuple  $(\text{side}, \text{top}, \text{left})$  describing a largest all-nonzero square in `a`. The state  $dp(c)$  is the largest square ending at the current cell in column  $c$ ; a nonzero cell extends the minimum of the states above, left, and diagonally above-left. If no such square exists, the function returns  $(0, -1, -1)$ . Overflow warning: Path counts and path costs must fit in `int64_t`. Time Complexity:
- $O(R \cdot C)$  per call, where  $R$  and  $C$  are the grid dimensions.

#### Space Complexity:

- $O(C)$  auxiliary for `count_grid_paths(blocked)`.
- $O(R \cdot C)$  auxiliary for predecessor directions,  $O(C)$  for path costs, and  $O(R + C)$  for the returned path from `min_cost_grid_path(cost)`.
- $O(C)$  auxiliary for `largest_nonzero_square(a)`.

```
#include <algorithm>
#include <cstdint>
#include <tuple>
#include <utility>
#include <vector>

int64_t count_grid_paths(const std::vector<std::vector<char>> &blocked) {
    int rows = static_cast<int>(blocked.size());
    int cols = blocked.empty() ? 0 : static_cast<int>(blocked[0].size());
    if (rows == 0 || cols == 0 || blocked[0][0] || blocked[rows - 1][cols - 1]) {
        return 0;
    }
    std::vector<int64_t> dp(cols);
    dp[0] = 1;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (blocked[r][c]) {
                dp[c] = 0;
            } else if (c > 0) {
                dp[c] += dp[c - 1]; // Overflow warning.
            }
        }
    }
    return dp[cols - 1];
}

std::pair<int64_t, std::vector<std::pair<int, int>>> min_cost_grid_path(
    const std::vector<std::vector<int>> &cost
) {
    int rows = static_cast<int>(cost.size());
    int cols = cost.empty() ? 0 : static_cast<int>(cost[0].size());
    if (rows == 0 || cols == 0) {
        return {0, {}};
    }
    std::vector<int64_t> dp(cols);
    std::vector<std::vector<char>> pred(rows, std::vector<char>(cols));
    dp[0] = cost[0][0];
    for (int r = 0; r < rows; r++) {
```

```

    for (int c = 0; c < cols; c++) {
        if (r == 0 && c == 0) {
            continue;
        }
        if (r > 0 && (c == 0 || dp[c] <= dp[c - 1])) {
            dp[c] += cost[r][c]; // Overflow warning.
            pred[r][c] = 'U';
        } else {
            dp[c] = dp[c - 1] + cost[r][c];
            pred[r][c] = 'L';
        }
    }
}

// Optional: reconstruct one optimal path.
std::vector<std::pair<int, int>> path;
for (int r = rows - 1, c = cols - 1;;) {
    path.emplace_back(r, c);
    if (r == 0 && c == 0) {
        break;
    }
    if (pred[r][c] == 'U') {
        r--;
    } else {
        c--;
    }
}
std::reverse(path.begin(), path.end());
return {dp.back(), path};
}

std::tuple<int, int, int> largest_nonzero_square(const std::vector<std::vector<char>> &a) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    std::vector<int> dp(cols);
    int best = 0, top = -1, left = -1;
    for (int r = 0; r < rows; r++) {
        int diagonal = 0;
        for (int c = 0; c < cols; c++) {
            int above = dp[c];
            dp[c] = a[r][c] ? 1 + std::min({above, c == 0 ? 0 : dp[c - 1], diagonal}) : 0;
            diagonal = above;
            if (dp[c] > best) {
                best = dp[c];
                top = r - best + 1;
                left = c - best + 1;
            }
        }
    }
    return {best, top, left};
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<char>> blocked{
        {0, 0, 0},
        {0, 1, 0},
        {0, 0, 0},
    };
    assert(count_grid_paths(blocked) == 2);

    vector<vector<int>> cost{
        {1, 3, 1},
    };
}

```

```

    {1, 5, 1},
    {4, 2, 1},
};
auto [min_cost, path] = min_cost_grid_path(cost);
assert(min_cost == 7 && (path == vector<pair<int, int>>{{0, 0}, {0, 1}, {0, 2}, {1, 2}, {2, 2}}));

vector<vector<char>> binary{
    {1, 0, 1, 0, 0},
    {1, 0, 1, 1, 1},
    {1, 1, 1, 1, 1},
    {1, 0, 1, 1, 1},
};
assert((largest_nonzero_square(binary) == tuple{3, 1, 2}));
assert((largest_nonzero_square({{0, 0}}) == tuple{0, -1, -1}));
return 0;
}

```

## 1.5 Greedy and Scheduling

---

### 1.5.1 Fractional Knapsack

Solves the fractional knapsack problem: given items with positive integer weights and nonnegative integer values, choose any fraction of each item to maximize total value without exceeding a capacity limit.

The greedy algorithm processes items in decreasing order of value per unit weight, taking each item in full until only part of the next item fits. This is optimal because replacing any weight taken from a lower-density item with the same weight from a higher-density item cannot decrease the total value. Unlike 0-1 knapsack, allowing fractions makes each such exchange feasible.

- `fractional_knapsack(weight, value, capacity)` returns a pair `(best_value, fraction)` containing the maximum value and the fraction of each input item taken. Item `i` has weight `weight[i]` and value `value[i]`, `capacity` is the maximum total weight, and `fraction[i]` is in `[0, 1]`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting, where  $n$  is the number of items.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned fractions.

```

#include <algorithm>
#include <cassert>
#include <cstdlib>
#include <numeric>
#include <utility>
#include <vector>

std::pair<double, std::vector<double>> fractional_knapsack(
    const std::vector<int> &weight, const std::vector<int> &value, int capacity
) {
    int n = static_cast<int>(weight.size());
    assert(static_cast<int>(value.size()) == n && capacity >= 0);
    for (int i = 0; i < n; i++) {
        assert(weight[i] > 0 && value[i] >= 0);
    }
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int i, int j) {
        int64_t lhs = static_cast<int64_t>(value[i]) * weight[j];
        int64_t rhs = static_cast<int64_t>(value[j]) * weight[i];
        return lhs != rhs ? lhs > rhs : i < j;
    });
    double best_value = 0;
    std::vector<double> fraction(n);

```



```

for (int i : order) {
    int taken = std::min(capacity, weight[i]);
    fraction[i] = static_cast<double>(taken) / weight[i];
    best_value += fraction[i] * value[i];
    capacity -= taken;
    if (capacity == 0) {
        break;
    }
}
return {best_value, fraction};
}

```

## Example Usage

```

#include <cmath>
using namespace std;

int main() {
    vector<int> weight{10, 20, 30};
    vector<int> value{60, 100, 120};
    auto [best_value, fraction] = fractional_knapsack(weight, value, 50);
    assert(fabs(best_value - 240) < 1e-9 && (fraction == vector<double>{1, 1, 2.0 / 3}));
    return 0;
}

```

### 1.5.2 Interval Scheduling

Selects the maximum number of non-overlapping intervals using the classic earliest-finish-time greedy algorithm. This is the unweighted interval scheduling problem, also known as activity selection: every interval has the same value, so the objective is to choose as many compatible intervals as possible.

The greedy choice is safe because among all intervals that could be chosen first, taking one with the earliest finish time leaves the most room for the remaining intervals. In any optimal solution, the first chosen interval can be exchanged for an earliest-finishing compatible interval without reducing the number of intervals selected.

Repeating this argument after each choice proves the greedy algorithm optimal. The weighted version in 1.5.3 needs dynamic programming instead.

Intervals are represented as half-open ranges  $[\text{start}, \text{finish})$ , so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `select_intervals(intervals)` returns one maximum-size compatible subset of intervals, in the order they are selected, as original indices into an input vector of `Interval` with fields `start` and `finish`. Every interval must satisfy `start < finish`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting, where  $n$  is the number of intervals.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned indices.

```

#include <algorithm>
#include <cassert>
#include <climits>
#include <numeric>
#include <vector>

struct Interval {
    int start, finish;
};

std::vector<int> select_intervals(const std::vector<Interval> &intervals) {
    for (const auto &iv : intervals) {
        assert(iv.start < iv.finish);
    }
}

```

```

}
std::vector<int> order(intervals.size());
std::iota(order.begin(), order.end(), 0);
std::sort(order.begin(), order.end(), [&](int i, int j) {
    return intervals[i].finish != intervals[j].finish ? intervals[i].finish < intervals[j].finish
        : intervals[i].start < intervals[j].start;
});
std::vector<int> selected;
int last_finish = INT_MIN;
for (int i : order) {
    const auto &iv = intervals[i];
    if (iv.start >= last_finish) {
        selected.push_back(i);
        last_finish = iv.finish;
    }
}
return selected;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Interval> intervals{{1, 4}, {3, 5}, {0, 6}, {5, 7}, {8, 9}};
    // Earliest-finish greedy chooses original indices 0, 3, 4.
    assert((select_intervals(intervals) == vector<int>{0, 3, 4}));

    vector<Interval> touching{{0, 2}, {2, 4}, {4, 5}};
    assert((select_intervals(touching) == vector<int>{0, 1, 2}));
    return 0;
}

```

### 1.5.3 Weighted Interval Scheduling

Selects a maximum-weight subset of non-overlapping intervals using dynamic programming and binary search. Unlike unweighted interval scheduling, the earliest finish-time greedy choice is not sufficient when intervals have weights. With intervals sorted by finish time, the best total for the first  $i$  intervals either skips interval  $i$  or adds its weight to the best total over intervals finishing no later than its start, with that predecessor located by binary search.

Intervals are represented as half-open ranges  $[\text{start}, \text{finish})$ , so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `select_weighted_intervals(intervals)` returns a pair  $(\text{weight}, \text{selected})$  containing that maximum weight and the selected intervals as original input indices in execution order, from an input vector of `WeightedInterval` with fields `start`, `finish`, and `weight`. `dp[i]` stores the best answer using the first  $i$  intervals after sorting by finish time. Each interval must satisfy `start < finish`. The empty subset is allowed, so nonpositive weights need not be selected.

Overflow warning: Accumulated weights must fit in `int64_t`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting and binary searching compatible intervals.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned indices.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>

```

```

#include <utility>
#include <vector>

struct WeightedInterval {
    int start, finish;
    int64_t weight;
};

std::pair<int64_t, std::vector<int>> select_weighted_intervals(
    const std::vector<WeightedInterval> &intervals
) {
    for (const auto &iv : intervals) {
        assert(iv.start < iv.finish);
    }
    int n = static_cast<int>(intervals.size());
    std::vector<int> order(n), finish(n), prev(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int i, int j) {
        return intervals[i].finish != intervals[j].finish ? intervals[i].finish < intervals[j].finish
            : intervals[i].start < intervals[j].start;
    });
    for (int i = 0; i < n; i++) {
        finish[i] = intervals[order[i]].finish;
    }
    std::vector<int64_t> dp(n + 1);
    std::vector<char> take(n + 1);
    for (int i = 1; i <= n; i++) {
        int j =
            std::upper_bound(finish.begin(), finish.begin() + i - 1, intervals[order[i - 1]].start) -
            finish.begin();
        prev[i - 1] = j;
        int64_t candidate = dp[j] + intervals[order[i - 1]].weight; // Overflow warning.
        if (dp[i - 1] < candidate) {
            dp[i] = candidate;
            take[i] = true;
        } else {
            dp[i] = dp[i - 1];
        }
    }
    // Optional: reconstruct one optimal subset of intervals.
    std::vector<int> selected;
    for (int i = n; i > 0; i--) {
        if (take[i]) {
            selected.push_back(order[i - 1]);
            i = prev[i - 1];
        } else {
            i--;
        }
    }
    std::reverse(selected.begin(), selected.end());
    return {dp[n], selected};
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<WeightedInterval> intervals{{1, 3, 5}, {2, 5, 6}, {4, 6, 5}, {6, 7, 4}, {5, 8, 11}};
    auto [weight, selected] = select_weighted_intervals(intervals);
    // Taking ids 1 and 4 beats the earliest-finish unweighted-looking choices.
    assert(weight == 17 && (selected == vector<int>{1, 4}));

    vector<WeightedInterval> touching{{0, 2, 5}, {2, 4, 6}, {1, 3, 7}};
    auto [touching_weight, touching_selected] = select_weighted_intervals(touching);
}

```

```

    assert(touching_weight == 11 && (touching_selected == vector<int>{0, 1}));
    return 0;
}

```

### 1.5.4 Interval Merging and Covering

Two fundamental interval problems ask either for the union of a collection of intervals or for a minimum-size subcollection that covers a target interval. For the union, sorting by start time makes all intervals that overlap the current merged interval consecutive, so one scan can extend its finish or begin the next disjoint interval.

For minimum covering, maintain the rightmost point covered so far. Among every interval starting at or before that frontier, greedily select one with the farthest finish. Any feasible cover must next use one of those intervals, and replacing that choice with the farthest-reaching one cannot make the remaining target harder to cover. This is different from interval scheduling, which selects the compatible interval with the earliest finish.

Intervals are represented as half-open ranges  $[start, finish)$ . Touching intervals may be merged because their union is another half-open interval with no gap.

- `merge_intervals(intervals)` returns the union as disjoint intervals sorted by start. Every input interval must satisfy `start < finish`.
- `cover_interval(intervals, lo, hi)` returns a minimum-size list of original input indices whose intervals cover  $[lo, hi)$ , in the order selected, or `std::nullopt` if coverage is impossible. Every input interval must satisfy `start < finish`, and `lo` must not exceed `hi`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting, where  $n$  is the number of intervals.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned intervals or indices.

```

#include <algorithm>
#include <cassert>
#include <numeric>
#include <optional>
#include <vector>

struct Interval {
    int start, finish;
};

std::vector<Interval> merge_intervals(std::vector<Interval> intervals) {
    for (const auto &iv : intervals) {
        assert(iv.start < iv.finish);
    }
    std::sort(intervals.begin(), intervals.end(), [](const Interval &a, const Interval &b) {
        return a.start != b.start ? a.start < b.start : a.finish < b.finish;
    });
    std::vector<Interval> merged;
    for (const auto &iv : intervals) {
        if (merged.empty() || merged.back().finish < iv.start) {
            merged.push_back(iv);
        } else {
            merged.back().finish = std::max(merged.back().finish, iv.finish);
        }
    }
    return merged;
}

std::optional<std::vector<int>> cover_interval(
    const std::vector<Interval> &intervals, int lo, int hi
) {
    assert(lo <= hi);
    for (const auto &iv : intervals) {
        assert(iv.start < iv.finish);
    }
}

```

```

int n = static_cast<int>(intervals.size());
std::vector<int> order(n);
std::iota(order.begin(), order.end(), 0);
std::sort(order.begin(), order.end(), [&](int i, int j) {
    if (intervals[i].start != intervals[j].start) {
        return intervals[i].start < intervals[j].start;
    }
    return intervals[i].finish != intervals[j].finish ? intervals[i].finish > intervals[j].finish
        : i < j;
});
std::vector<int> selected;
int pos = 0, frontier = lo;
while (frontier < hi) {
    int best = -1, best_finish = frontier;
    while (pos < n && intervals[order[pos]].start <= frontier) {
        int i = order[pos++];
        if (best_finish < intervals[i].finish ||
            (best_finish == intervals[i].finish && best != -1 && i < best)) {
            best = i;
            best_finish = intervals[i].finish;
        }
    }
    if (best == -1) {
        return std::nullopt;
    }
    selected.push_back(best);
    frontier = best_finish;
}
return selected;
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<Interval> intervals{{8, 10}, {1, 3}, {2, 6}, {15, 18}, {10, 12}};
    auto merged = merge_intervals(intervals);
    assert(merged.size() == 3);
    assert(merged[0].start == 1 && merged[0].finish == 6);
    assert(merged[1].start == 8 && merged[1].finish == 12);
    assert(merged[2].start == 15 && merged[2].finish == 18);

    vector<Interval> cover{{0, 2}, {1, 5}, {4, 7}, {6, 10}, {2, 6}, {8, 11}};
    assert(cover_interval(cover, 0, 10) == vector<int>({0, 4, 3}));
    assert(!cover_interval({{0, 2}, {3, 5}}, 0, 5));
    assert(cover_interval({}, 4, 4) == vector<int>());
    return 0;
}

```

### 1.5.5 Interval Partitioning

Assigns intervals to the minimum number of groups so that no two intervals in the same group overlap. This is the classic interval partitioning problem, also known as finding the minimum number of rooms needed for meetings.

The greedy algorithm sorts intervals by start time and reuses the group whose current finish time is earliest whenever possible. Otherwise, it creates a new group. This is optimal because the number of groups must be at least the maximum number of intervals overlapping at any time. When the earliest-finishing active group still overlaps the next interval, every active group overlaps it, so creating a new group is unavoidable. If that group is free, reusing it preserves as many later options as possible.

Intervals are treated as half-open ranges  $[\text{start}, \text{finish})$ , so one interval may reuse a room that another interval vacates at the same time.

- `partition_intervals(intervals)` returns a pair `(rooms, room)`, where `rooms` is the minimum number of rooms and `room[i]` is the assigned room for input interval `i`. Each `Interval` has fields `start` and `finish`, which must satisfy `start < finish`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting and priority queue operations.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned assignment.

```
#include <algorithm>
#include <cassert>
#include <functional>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

struct Interval {
    int start, finish;
};

std::pair<int, std::vector<int>>> partition_intervals(const std::vector<Interval> &intervals) {
    for (const auto &iv : intervals) {
        assert(iv.start < iv.finish);
    }
    int n = static_cast<int>(intervals.size());
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int i, int j) {
        return intervals[i].start != intervals[j].start ? intervals[i].start < intervals[j].start
            : intervals[i].finish < intervals[j].finish;
    });
    std::vector<int> room(n);
    std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>, std::greater<>> pq;
    int rooms = 0;
    for (int i : order) {
        if (!pq.empty() && pq.top().first <= intervals[i].start) {
            int r = pq.top().second;
            pq.pop();
            room[i] = r;
        } else {
            room[i] = rooms++;
        }
        pq.emplace(intervals[i].finish, room[i]);
    }
    return {rooms, room};
}
```

## Example Usage

```
#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    auto [rooms, room] = partition_intervals(vector<Interval>{{0, 30}, {5, 10}, {15, 20}});
    // The long interval overlaps both short intervals, but the short intervals can share a room.
    assert(rooms == 2);
    assert(room[1] == room[2]);
    assert(room[0] != room[1]);

    vector<Interval> touching{{0, 2}, {2, 4}, {4, 5}};
    auto [touching_rooms, touching_room] = partition_intervals(touching);
    assert(touching_rooms == 1 && *max_element(touching_room.begin(), touching_room.end()) == 0);
}
```

```

    assert(partition_intervals({}).first == 0);
    return 0;
}

```

### 1.5.6 Deadline Scheduling

Schedules unit-time jobs with deadlines and profits to maximize total profit. Each job takes one time slot and must be scheduled at or before its deadline. The greedy algorithm processes jobs in decreasing profit order and places each chosen job in the latest available slot before its deadline.

The latest-slot choice is safe because it uses the current high-profit job while leaving earlier slots available for jobs with tighter deadlines. A disjoint-set forest over time slots makes this efficient: `find(t)` returns the latest still-free slot at or before `t`, and occupying slot `t` links it to the next candidate slot `t - 1`.

- `select_deadline_jobs(jobs)` returns a pair `(profit, slot)` containing that maximum profit and `slot[i]`, the assigned time slot of input job `i`, or `-1` if that job is not selected, for a vector of `Job` with fields `deadline` and `profit`. Deadlines are positive integers, and assigned slots are numbered starting from 1. Jobs with nonpositive profit are not selected.

Overflow warning: The total profit must fit in `int64_t`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting, with near-constant disjoint-set operations.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned slot assignment.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <utility>
#include <vector>

struct Job {
    int deadline;
    int64_t profit;
};

class SlotDSU {
    std::vector<int> root;

public:
    explicit SlotDSU(int n) : root(n + 1) { std::iota(root.begin(), root.end(), 0); }

    int find(int u) { return root[u] == u ? u : root[u] = find(root[u]); }
    void occupy(int u) { root[u] = find(u - 1); }
};

std::pair<int64_t, std::vector<int>> select_deadline_jobs(const std::vector<Job> &jobs) {
    for (const auto &job : jobs) {
        assert(job.deadline > 0);
    }
    int n = static_cast<int>(jobs.size());
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int i, int j) {
        return jobs[i].profit != jobs[j].profit ? jobs[i].profit > jobs[j].profit
            : jobs[i].deadline < jobs[j].deadline;
    });
    int max_deadline = 0;
    for (const auto &j : jobs) {
        max_deadline = std::max(max_deadline, j.deadline);
    }
    max_deadline = std::min(max_deadline, n);
    SlotDSU slots(max_deadline);
}

```

```

std::vector<int> assigned(n, -1);
int64_t res = 0;
for (int i : order) {
    if (jobs[i].profit <= 0) {
        continue;
    }
    int slot = slots.find(std::min(jobs[i].deadline, max_deadline));
    if (slot > 0) {
        res += jobs[i].profit; // Overflow warning.
        assigned[i] = slot;
        slots.occupy(slot);
    }
}
return {res, assigned};
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Job> jobs{{2, 100}, {1, 19}, {2, 27}, {1, 25}, {3, 15}};
    auto [profit, slot] = select_deadline_jobs(jobs);
    // Jobs 0, 2, 4 fill slots 2, 1, 3 for total profit 100 + 27 + 15.
    assert(profit == 142 && (slot == vector<int>{2, -1, 1, -1, 3}));
    return 0;
}

```

### 1.5.7 Minimum Late Jobs (Moore-Hodgson)

Selects a maximum-size subset of jobs that can all be completed by their deadlines. Each job has a processing time and deadline, and all jobs are available at time 0. The Moore-Hodgson algorithm sorts by deadline and keeps the accepted jobs in a max-heap by processing time; whenever the schedule becomes late, it removes the longest accepted job.

After considering jobs up to some deadline, if the accepted set no longer fits, at least one accepted job must be removed. Removing the longest one leaves the most remaining time while keeping the same number of removed jobs, so it is never worse than removing a shorter accepted job. Repeating this repair after each deadline leaves a largest feasible accepted set.

- `max_on_time_jobs(jobs)` returns the selected jobs as original input indices in a feasible earliest-deadline-first execution order, for an input vector of `TimedJob` with fields `duration` and `deadline`. Durations and deadlines must be nonnegative integers.

Overflow warning: The total duration must fit in `int64_t`.

**Time Complexity:**  $O(n \log n)$  per call due to sorting and heap operations.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned indices.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

struct TimedJob {
    int duration, deadline;
};

```



```

};

std::vector<int> max_on_time_jobs(const std::vector<TimedJob> &jobs) {
    for (const auto &job : jobs) {
        assert(job.duration >= 0 && job.deadline >= 0);
    }
    int n = static_cast<int>(jobs.size());
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int i, int j) {
        return jobs[i].deadline != jobs[j].deadline ? jobs[i].deadline < jobs[j].deadline
            : jobs[i].duration < jobs[j].duration;
    });
    std::priority_queue<std::pair<int, int>> accepted;
    std::vector<bool> selected(n);
    int64_t time = 0;
    for (int i : order) {
        time += jobs[i].duration; // Overflow warning.
        accepted.emplace(jobs[i].duration, i);
        selected[i] = true;
        if (time > jobs[i].deadline) {
            time -= accepted.top().first;
            selected[accepted.top().second] = false;
            accepted.pop();
        }
    }
    std::vector<int> res;
    for (int i : order) {
        if (selected[i]) {
            res.push_back(i);
        }
    }
    return res;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<TimedJob> jobs{{3, 4}, {2, 3}, {1, 2}, {2, 7}};
    assert((max_on_time_jobs(jobs) == vector<int>{2, 1, 3}));
    return 0;
}

```

## 1.5.8 Weighted Completion Time

Given jobs that must run one at a time on a single machine, each with a processing time and a weight, order them to minimize the weighted sum of completion times  $\sum w_j C_j$ , where  $C_j$  is the moment job  $j$  finishes. Unlike the deadline problems of sections 1.5.6 and 1.5.7, nothing is rejected and no deadline is missed; every order is feasible, and only the total cost differs.

Smith's rule sorts by the ratio of processing time to weight, running the job with the smallest  $p_j/w_j$  first. An exchange argument proves it optimal. Consider any schedule that runs job  $b$  immediately before job  $a$  with  $p_a/w_a < p_b/w_b$ , and swap the two. Every other job finishes at the same moment, since the pair occupies the same total time, so the change in cost is  $p_a w_b - p_b w_a$ , which the ratio inequality makes negative. Any schedule violating the ratio order therefore improves under a swap, so an optimal schedule must be sorted by it. With equal weights the rule reduces to shortest processing time first, the classic way to minimize the average completion time.

- `min_weighted_completion_time(jobs)` returns the pair `(cost, order)`, where `cost` is the minimum weighted sum of completion times and `order` lists the job indices in an optimal sequence. Each job is a pair `(time, weight)` with a positive weight and a nonnegative time.

Ties break by index, and the comparison cross-multiplies rather than divides so that it stays exact on integers. Parallel machines are a different problem, NP-hard for general weights.

Overflow warning: the products in the comparator and the running completion time are accumulated in `int64_t`, so the total processing time multiplied by the total weight must fit in that type.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of jobs.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned order.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>> min_weighted_completion_time(
    const std::vector<std::pair<int, int>> &jobs
) {
    for (auto [time, weight] : jobs) {
        assert(time >= 0 && weight > 0);
    }
    int n = static_cast<int>(jobs.size());
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int a, int b) {
        // Sort by time / weight ascending, cross-multiplied to stay exact.
        int64_t lhs = static_cast<int64_t>(jobs[a].first) * jobs[b].second;
        int64_t rhs = static_cast<int64_t>(jobs[b].first) * jobs[a].second;
        return lhs != rhs ? lhs < rhs : a < b;
    });
    int64_t clock = 0, cost = 0;
    for (int id : order) {
        clock += jobs[id].first;
        cost += clock * jobs[id].second; // Overflow warning.
    }
    return {cost, order};
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // (time, weight) pairs. Ratios are 3, 1, and 2, so job 1 runs first, then job 2, then job 0.
    auto [cost, order] = min_weighted_completion_time({{3, 1}, {2, 2}, {4, 2}});
    assert((order == vector<int>{1, 2, 0}));
    assert(cost == 2 * 2 + 6 * 2 + 9 * 1); // Completion times 2, 6, and 9.

    // Equal weights reduce the rule to shortest processing time first.
    auto [spt_cost, spt_order] = min_weighted_completion_time({{5, 1}, {1, 1}, {3, 1}});
    assert((spt_order == vector<int>{1, 2, 0}));
    assert(spt_cost == 1 + 4 + 9);

    assert(min_weighted_completion_time({}).first == 0);
    return 0;
}
```

## 1.6 Streaming and Randomized Algorithms

---

### 1.6.1 Fisher-Yates Shuffle

Randomly permutes a range in-place using the Fisher-Yates shuffle. Each possible permutation is produced with equal probability, assuming the random number source is uniform.

The algorithm scans from right to left. At each position  $i$ , it chooses a random position  $j$  from  $[0, i]$  and swaps the two elements. This fixes one uniformly random remaining element at a time.

- `fisher_yates_shuffle(lo, hi)` randomly shuffles the range  $[lo, hi)$  using an internal random engine. The range must provide random-access iterators.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the distance between `lo` and `hi`.

**Space Complexity:**  $O(1)$  auxiliary.

```
#include <algorithm>
#include <chrono>
#include <random>

template<typename It>
void fisher_yates_shuffle(It lo, It hi) {
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    int n = static_cast<int>(hi - lo);
    for (int i = n - 1; i > 0; i--) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(rng);
        std::iter_swap(lo + i, lo + j);
    }
}
```

#### Example Usage

```
#include <algorithm>
#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a{1, 2, 3, 4, 5}, original(a);
    fisher_yates_shuffle(a.begin(), a.end());
    sort(a.begin(), a.end());
    assert(a == original);
    return 0;
}
```

### 1.6.2 Reservoir Sampling

Selects uniformly random samples from a stream whose length may be unknown in advance. This is useful when values arrive online or when storing the full dataset is unnecessary.

Reservoir sampling keeps only the chosen sample in memory while processing each element once. The first  $k$  arrivals fill the reservoir, and the  $i$ -th element thereafter is accepted with probability  $k/i$ , evicting a uniformly chosen occupant. One draw performs both choices: an integer uniform on  $[0, i)$  is below  $k$  with probability  $k/i$ , and is then uniform on the  $k$  occupants.

The invariant is that after  $n \geq k$  arrivals the reservoir is a uniformly random  $k$ -subset, so each of the  $\binom{n}{k}$  possibilities is equally likely. This is certain at  $n = k$ . Assuming it after  $n - 1$ , fix a  $k$ -subset  $A$  of the first  $n$ . If

$n \notin A$ , the reservoir must already hold  $A$  and reject the arrival, with probability  $\binom{n-1}{k}^{-1}(n-k)/n$ . If  $n \in A$ , it must hold  $A$  with  $n$  replaced by one of the  $n-k$  outsiders, each then evicted with probability  $(k/n)(1/k) = 1/n$ , giving  $\binom{n-1}{k}^{-1}(n-k)/n$  again. Both equal  $\binom{n}{k}^{-1}$ , since  $\binom{n}{k} = \binom{n-1}{k} \cdot n/(n-k)$ . Taking marginals, every element seen is in the sample with probability  $k/n$ , and `ReservoirSampleOne` is the case  $k = 1$ .

Each class maintains its reservoir incrementally; call `add(x)` once per stream element in any order, then call `sample()` to retrieve the result.

- `ReservoirSampleOne<T>()` constructs a single-element sampler.
- `ReservoirSampleK<T>(k)` constructs a  $k$ -element sampler, allocating the reservoir up front so that no later `add()` reallocates.
- `add(x)` incorporates one more stream element.
- `sample()` returns the current sample. For `ReservoirSampleOne`, `sample()` requires at least one `add()` call; for `ReservoirSampleK`, it returns the full reservoir, which may contain fewer than  $k$  elements if the stream was shorter.
- `count()` returns the number of stream elements seen so far.

### Time Complexity:

- $O(1)$  per call to `ReservoirSampleOne::add()` and `ReservoirSampleK::add()`.
- $O(k)$  per call to the `ReservoirSampleK` constructor.
- $O(n)$  to process a range of  $n$  elements.

### Space Complexity:

- $O(1)$  storage for `ReservoirSampleOne`.
- $O(k)$  storage for `ReservoirSampleK`, allocated on construction.

```
#include <cassert>
#include <chrono>
#include <optional>
#include <random>
#include <vector>

int rand_int(int lo, int hi) {
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    return std::uniform_int_distribution<int>(lo, hi)(rng);
}

template<typename T>
class ReservoirSampleOne {
    std::optional<T> value;
    int seen = 0;

public:
    void add(const T &x) {
        seen++;
        if (rand_int(1, seen) == 1) {
            value = x;
        }
    }

    const T &sample() const {
        assert(value.has_value());
        return *value;
    }

    int count() const { return seen; }
};

template<typename T>
class ReservoirSampleK {
    int k, seen = 0;
    std::vector<T> reservoir;
```

```

public:
    explicit ReservoirSampleK(int k) : k(k) {
        assert(k >= 0);
        reservoir.reserve(k);
    }

    void add(const T &x) {
        ++seen;
        if (k == 0) {
            return;
        }
        if (static_cast<int>(reservoir.size()) < k) {
            reservoir.push_back(x);
        } else {
            int j = rand_int(0, seen - 1);
            if (j < k) {
                reservoir[j] = x;
            }
        }
    }

    const std::vector<T> &sample() const { return reservoir; }
    int count() const { return seen; }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{10, 20, 30, 40, 50};

    // Streaming interface: feed elements one at a time.
    ReservoirSampleOne<int> s1;
    for (int x : a) {
        s1.add(x);
    }
    assert(s1.count() == 5);
    int one = s1.sample();
    assert(one == 10 || one == 20 || one == 30 || one == 40 || one == 50);

    ReservoirSampleK<int> sk(3);
    for (int x : a) {
        sk.add(x);
    }
    assert(sk.count() == 5);
    assert(sk.sample().size() == 3);

    ReservoirSampleK<int> large(10);
    for (int x : a) {
        large.add(x);
    }
    assert(large.count() == 5);
    assert(large.sample().size() == 5);
    return 0;
}

```

### 1.6.3 Frequent Elements

Finds frequent elements in a range using cancellation algorithms that retain only a small set of candidates.

Boyer-Moore finds a possible majority element, while Misra-Gries generalizes the same idea to frequencies above any threshold  $n/k$ .

The Boyer-Moore voting algorithm maintains one candidate and counter, incrementing on a match and decrementing on a mismatch. Each decrement cancels two different values. Removing such pairs cannot eliminate an element occurring more than  $\lfloor n/2 \rfloor$  times, so a true majority survives as the final candidate. A second pass verifies that candidate.

Misra-Gries keeps at most  $k - 1$  candidates. A tracked value increments its counter, an untracked value claims a free counter, and otherwise every counter is decremented. Each full decrement cancels  $k$  distinct occurrences, so any value occurring more than  $\lfloor n/k \rfloor$  times must remain a candidate. These candidates require a second pass for exact verification because the algorithm does not retain the stream.

- `majority_element(lo, hi)` returns an iterator to the first occurrence of the majority element of  $[lo, hi)$ , or `hi` if no majority element exists. The range must provide ForwardIterators, and its value type must support equality.
- `frequent_candidates(lo, hi, k)` returns a hash table of candidate values to their residual counters. `k` must be at least 2, and the value type must support equality and `std::hash`. Use `k = 2` for the unverified Boyer-Moore candidate phase.

#### Time Complexity:

- $O(n)$  per call to `majority_element(lo, hi)`, where  $n$  is the range length.
- $O(n)$  expected per call to `frequent_candidates(lo, hi, k)`: a full-table decrement cancels  $k$  occurrences, so all decrement sweeps take  $O(n)$  total.
- $O(n \cdot k)$  per call to `frequent_candidates()` in the collision-heavy worst case.

#### Space Complexity:

- $O(1)$  auxiliary for `majority_element()`.
- $O(k)$  for the candidates returned by `frequent_candidates()`.

```
#include <cassert>
#include <iterator>
#include <unordered_map>

template<typename It>
It majority_element(It lo, It hi) {
    int count = 0;
    It candidate = lo;
    for (It it = lo; it != hi; ++it) {
        if (count == 0) {
            candidate = it;
            count = 1;
        } else if (*it == *candidate) {
            count++;
        } else {
            count--;
        }
    }
}

// Second pass to verify. If a majority is guaranteed, skip and return candidate directly instead.
int n = 0;
count = 0;
It first = hi;
for (It it = lo; it != hi; ++it) {
    n++;
    if (*it == *candidate) {
        if (first == hi) {
            first = it;
        }
        count++;
    }
}
```

```

    }
}
if (count <= n / 2) {
    return hi;
}
return first;
}

template<typename It>
auto frequent_candidates(It lo, It hi, int k) {
    using T = typename std::iterator_traits<It>::value_type;
    assert(k >= 2);
    std::unordered_map<T, int> count;
    for (It it = lo; it != hi; ++it) {
        if (auto found = count.find(*it); found != count.end()) {
            found->second++;
        } else if (static_cast<int>(count.size()) < k - 1) {
            count[*it] = 1;
        } else {
            for (auto jt = count.begin(); jt != count.end(); ) {
                if (--jt->second == 0) {
                    jt = count.erase(jt);
                } else {
                    ++jt;
                }
            }
        }
    }
    return count;
}

```

### Example Usage

```

#include <vector>
using namespace std;

int main() {
    vector<int> a{3, 2, 3, 1, 3};
    assert(majority_element(a.begin(), a.end()) == a.begin());
    vector<int> b{2, 3, 3, 3, 2, 1};
    assert(majority_element(b.begin(), b.end()) == b.end());

    vector<int> c{1, 2, 1, 3, 1, 2, 1, 4, 2, 2, 2};
    auto candidates = frequent_candidates(c.begin(), c.end(), 3);
    assert(candidates.count(1));
    assert(candidates.count(2));
    assert(!candidates.count(3));
    return 0;
}

```

#### 1.6.4 Online Statistics

Online stream summaries update useful statistics as values arrive without storing the full stream. This section uses Welford’s algorithm to maintain the mean and variance of a stream in constant space, then merges Welford summaries inside a queue to maintain the same statistics over a sliding window. For the median of a stream or of a trailing window, see section 1.6.5.

The naive variance formula  $\text{Var}(X) = E[X^2] - E[X]^2$  can lose precision when the two terms are large and close together. Welford instead maintains the current mean  $\bar{x}_n$  (stored as `avg`) and the unnormalized second central moment  $M_{2,n} = \sum_{i=1}^n (x_i - \bar{x}_n)^2$  (stored as `m2`). Each new value shifts the running mean by its deviation divided by the new count, then adds the product of its deviations from the old and new means to `m2`.

- `OnlineStatistics()` constructs an empty summary.
- `add(x)` incorporates one more value `x` into the summary.
- `count()` returns the number of values seen so far.
- `mean()` returns the current arithmetic mean, or 0 if the summary is empty.
- `var_pop()` returns population variance, dividing by  $n$ , or 0 if the summary is empty.
- `var_samp()` returns sample variance, dividing by  $n - 1$ , or 0 if fewer than two values have been added.

**Time Complexity:**  $O(1)$  per call to `add()` and each query.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <utility>
#include <vector>

class OnlineStatistics {
    int n;
    double avg, m2;

public:
    OnlineStatistics() : n(0), avg(0), m2(0) {}

    void add(double x) {
        n++;
        double delta = x - avg;
        avg += delta / n;
        double delta2 = x - avg;
        m2 += delta * delta2;
    }

    int count() const { return n; }
    double mean() const { return avg; }
    double var_pop() const { return n == 0 ? 0 : m2 / n; }
    double var_samp() const { return n <= 1 ? 0 : m2 / (n - 1); }
};
```

To additionally support FIFO removals, Welford's update cannot simply be run backwards to drop the oldest value: reversing it turns `m2` into a difference, which loses the very property that made the forward pass stable, and the resulting error has no way to correct itself as the window advances.

Summaries are mergeable instead. Two summaries of disjoint groups combine by shifting the mean toward the larger group and adding the spread between the two group means, so that  $M_2 = M_{2,A} + M_{2,B} + \delta^2 n_A n_B / n$ , where  $\delta$  is the gap between the group means. That merge is associative, which makes summaries a monoid and puts them within reach of the sliding window aggregation (SWAG) technique of section 1.2.5: a queue of two stacks, each entry storing the fold of everything between it and its end of the queue, reports the fold of the entire queue in  $O(1)$  amortized time. The class below is that structure specialized to this summary, so section 1.2.5 is the place to look for the general version over any associative operation, and for why flipping the back stack onto the front stack costs only  $O(1)$  amortized per value.

Every value still enters through a forward Welford update of a one-element summary, and the folds only ever add nonnegative terms, so the statistics of a window are as stable as those of a whole stream.

- `StatisticsQueue()` constructs an empty summary.
- `add(x)` incorporates `x` as the newest value of the queue.
- `remove()` drops the oldest value of the queue, which must be nonempty.
- `count()` returns the number of values currently in the queue.
- `mean()`, `var_pop()`, and `var_samp()` return the same statistics as above, over the values currently in the queue.

The window is left to the caller rather than fixed by the constructor. A trailing window of width  $w$  is one call to `add()` followed by a call to `remove()` whenever `count()` exceeds  $w$ , while a variable-width window from the two-pointer patterns of section 1.2.3 instead removes values until its own condition holds again.



**Time Complexity:**  $O(1)$  amortized per call to `add()` and `remove()`, and  $O(1)$  per query.

**Space Complexity:**  $O(n)$  storage, where  $n$  is the number of values currently in the queue.

```
class StatisticsQueue {
    struct Summary {
        int n;
        double mean, m2;
    };

    // Chan's parallel merge of two disjoint groups, which never subtracts a value out of m2.
    static Summary merge(const Summary &a, const Summary &b) {
        int n = a.n + b.n;
        double delta = b.mean - a.mean;
        return {n, a.mean + delta * b.n / n, a.m2 + b.m2 + delta * delta * a.n * b.n / n};
    }

    // Each stack entry is (summary of one value, fold), where the fold merges that value with all
    // entries between it and its end of the queue (the front for front_stack, back for back_stack).
    std::vector<std::pair<Summary, Summary>> front_stack, back_stack;

    Summary fold() const {
        if (front_stack.empty()) {
            return back_stack.empty() ? Summary{} : back_stack.back().second;
        }
        if (back_stack.empty()) {
            return front_stack.back().second;
        }
        return merge(front_stack.back().second, back_stack.back().second);
    }

public:
    void add(double x) {
        Summary value{1, x, 0};
        back_stack.emplace_back(
            value, back_stack.empty() ? value : merge(back_stack.back().second, value)
        );
    }

    void remove() {
        assert(count() > 0);
        if (front_stack.empty()) {
            // Flip the back stack onto the front stack, reversing order so the oldest value ends up on
            // top, and recompute the front folds in queue order.
            while (!back_stack.empty()) {
                Summary oldest = back_stack.back().first;
                back_stack.pop_back();
                front_stack.emplace_back(
                    oldest, front_stack.empty() ? oldest : merge(oldest, front_stack.back().second)
                );
            }
            front_stack.pop_back();
        }

        int count() const { return static_cast<int>(front_stack.size() + back_stack.size()); }
        double mean() const { return fold().mean; }

        double var_pop() const {
            Summary s = fold();
            return s.n == 0 ? 0 : s.m2 / s.n;
        }

        double var_samp() const {
            Summary s = fold();
            return s.n <= 1 ? 0 : s.m2 / (s.n - 1);
        }
};
```

## Example Usage

```

#include <cassert>
#include <cmath>
#include <vector>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    OnlineStatistics empty;
    assert(empty.count() == 0 && EQ(empty.mean(), 0.0));
    assert(EQ(empty.var_pop(), 0.0) && EQ(empty.var_samp(), 0.0));

    OnlineStatistics singleton;
    singleton.add(7);
    assert(singleton.count() == 1 && EQ(singleton.mean(), 7.0));
    assert(EQ(singleton.var_pop(), 0.0) && EQ(singleton.var_samp(), 0.0));

    OnlineStatistics stats;
    for (int x = 1; x <= 5; x++) {
        stats.add(x);
    }
    assert(stats.count() == 5 && EQ(stats.mean(), 3.0));
    assert(EQ(stats.var_pop(), 2.0) && EQ(stats.var_samp(), 2.5));

    OnlineStatistics shifted;
    shifted.add(1e12 + 1);
    shifted.add(1e12 + 2);
    shifted.add(1e12 + 3);
    assert(EQ(shifted.mean(), 1e12 + 2));
    assert(EQ(shifted.var_pop(), 2.0 / 3) && EQ(shifted.var_samp(), 1.0));

    StatisticsQueue empty_queue;
    assert(empty_queue.count() == 0 && EQ(empty_queue.mean(), 0.0));
    assert(EQ(empty_queue.var_pop(), 0.0) && EQ(empty_queue.var_samp(), 0.0));

    // A trailing window of width 3, maintained by the caller.
    StatisticsQueue window;
    for (int x = 1; x <= 5; x++) {
        window.add(x);
        if (window.count() > 3) {
            window.remove();
        }
    }
    assert(window.count() == 3); // The window holds 3, 4, and 5.
    assert(EQ(window.mean(), 4.0));
    assert(EQ(window.var_pop(), 2.0 / 3) && EQ(window.var_samp(), 1.0));

    // The same shifted values as above, now with an extra value removed from the front first.
    StatisticsQueue shifted_window;
    shifted_window.add(1e12);
    shifted_window.add(1e12 + 1);
    shifted_window.add(1e12 + 2);
    shifted_window.add(1e12 + 3);
    shifted_window.remove();
    assert(EQ(shifted_window.mean(), 1e12 + 2));
    assert(EQ(shifted_window.var_pop(), 2.0 / 3) && EQ(shifted_window.var_samp(), 1.0));

    // A variable-width window, which a fixed size cannot express: extend while the population
    // variance stays at most 1, shrinking from the front whenever it does not.
    StatisticsQueue spread;
    int longest = 0;
    for (double x : {1.0, 1.0, 2.0, 8.0, 8.0, 9.0}) {
        spread.add(x);
        while (spread.var_pop() > 1) {

```

```

        spread.remove();
    }
    if (spread.count() > longest) {
        longest = spread.count();
    }
}
assert(longest == 3); // The values 1, 1, and 2 have population variance 2/9.

return 0;
}

```

### 1.6.5 Online Median

The median of a stream can be maintained as values arrive, without ever sorting the stream. This section keeps the median of a growing stream in two heaps, and the median of a changing collection in a multiset, the difference between them being whether values ever have to leave again. For the mean and variance of a stream, see section 1.6.4.

Two heaps hold the values in halves, using linear storage. A max-heap stores the lower half and a min-heap stores the upper half, balanced so that the lower heap has either the same number of values as the upper heap or one extra value. Their roots are therefore the middle value or values of the stream.

- `OnlineMedian<T>()` constructs an empty median tracker for an ordered numeric type `T`.
- `empty()` returns whether the tracker has no values.
- `count()` returns the number of values in the tracker.
- `add(x)` inserts `x` into the tracker.
- `lower_median()` and `upper_median()` return the lower and the upper of the two middle values, or the middle value itself for an odd count. The tracker must be nonempty.
- `median()` returns the middle value for an odd count or the arithmetic mean of the two middle values for an even count. The tracker must be nonempty.

Since `median()` averages in `double`, an even count over a type whose values exceed the exactly representable range of `double` should be averaged by the caller from `lower_median()` and `upper_median()` instead.

**Time Complexity:**  $O(\log n)$  per call to `add()` and  $O(1)$  per query, where  $n$  is the number of values.

**Space Complexity:**  $O(n)$  storage.

```

#include <cassert>
#include <functional>
#include <iterator>
#include <queue>
#include <set>
#include <vector>

template<typename T>
class OnlineMedian {
    std::priority_queue<T> lower;
    std::priority_queue<T, std::vector<T>, std::greater<T>> upper;

public:
    bool empty() const { return lower.empty(); }
    int count() const { return static_cast<int>(lower.size() + upper.size()); }

    void add(const T &x) {
        if (lower.empty() || x <= lower.top()) {
            lower.push(x);
        } else {
            upper.push(x);
        }
        if (lower.size() < upper.size()) {
            lower.push(upper.top());
        }
    }
};

```

```

    upper.pop();
} else if (lower.size() > upper.size() + 1) {
    upper.push(lower.top());
    lower.pop();
}
}

const T &lower_median() const {
    assert(!empty());
    return lower.top();
}

const T &upper_median() const {
    assert(!empty());
    return lower.size() != upper.size() ? lower.top() : upper.top();
}

double median() const {
    return static_cast<double>(lower_median()) / 2 + static_cast<double>(upper_median()) / 2;
}
};

```

To additionally support arbitrary removals, a multiset (balanced binary search tree) holds the values in sorted order while an iterator tracks the lower median. Each insertion or erasure shifts the middle of the multiset by at most one position, so the iterator only ever steps one element in either direction: whether it moves depends on which side of the median the value lies and on the parity of the resulting size. Unlike the two heaps above, and unlike the statistics queue of section 1.6.4 that can only drop its oldest value, a multiset erases any value it holds, so removals may come in any order.

- `DeletableMedian<T>()` constructs an empty median tracker for an ordered numeric type `T`.
- `add(x)` inserts `x` into the median tracker.
- `remove(x)` erases one copy of `x`, which must currently be present.
- `count()` and `empty()` return the number of values and whether the tracker is empty.
- `lower_median()`, `upper_median()`, and `median()` answer as above, over the values currently held. The tracker must be nonempty.

A trailing window of width  $w$  over an array is one call to `add()` followed by a call to `remove()` on the value leaving the window, and a two-pointer scan from section 1.2.3 removes by its own condition instead. Because removals are unordered, the same structure also serves the add and remove operations of Mo's algorithm in section 2.5.4, which answers median queries over arbitrary ranges offline. Replacing the parked iterator with an order-statistic tree, such as the `__gnu_pbds` tree of section 8.6, generalizes further to any rank rather than just the middle.

**Time Complexity:**  $O(\log n)$  per call to `add()` and `remove()`, and  $O(1)$  per query, where  $n$  is the number of values currently held.

**Space Complexity:**  $O(n)$  storage.

```

template<typename T>
class DeletableMedian {
    std::multiset<T> sorted;
    typename std::multiset<T>::iterator mid;

public:
    bool empty() const { return sorted.empty(); }
    int count() const { return static_cast<int>(sorted.size()); }

    void add(const T &x) {
        if (sorted.empty()) {
            mid = sorted.insert(x);
            return;
        }
        bool below = x < *mid;
        sorted.insert(x);

```

```

    if (below) {
        if (sorted.size() % 2 == 0) {
            --mid;
        }
    } else if (sorted.size() % 2 == 1) {
        ++mid;
    }
}

void remove(const T &x) {
    assert(!empty());
    if (sorted.size() == 1) {
        assert(x == *mid);
        sorted.clear(); // The iterator is meaningless while empty, and add() resets it.
        return;
    }
    // Erasing the median's own node requires stepping off it first. Any other node holding x
    // lies strictly on one side of the median, so erasing it can never invalidate the iterator.
    auto target = (x == *mid) ? mid : sorted.find(x);
    assert(target != sorted.end());
    if (x <= *mid && sorted.size() % 2 == 0) {
        ++mid;
    } else if (x >= *mid && sorted.size() % 2 == 1) {
        --mid;
    }
    sorted.erase(target);
}

const T &lower_median() const {
    assert(!empty());
    return *mid;
}

const T &upper_median() const {
    assert(!empty());
    return sorted.size() % 2 == 1 ? *mid : *std::next(mid);
}

double median() const {
    return static_cast<double>(lower_median()) / 2 + static_cast<double>(upper_median()) / 2;
}
};

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    OnlineMedian<int> medians;
    assert(medians.empty() && medians.count() == 0);
    medians.add(5);
    assert(EQ(medians.median(), 5));
    medians.add(1);
    assert(medians.lower_median() == 1 && medians.upper_median() == 5);
    assert(EQ(medians.median(), 3));
    medians.add(9);
    assert(medians.lower_median() == 5 && medians.upper_median() == 5);
    medians.add(3);
    assert(medians.lower_median() == 3 && medians.upper_median() == 5);
    assert(EQ(medians.median(), 4));
}

```

```

// A trailing window of width 3 over an array, maintained by the caller.
vector<int> a{5, 1, 9, 3, 3};
DeletableMedian<int> window;
assert(window.empty() && window.count() == 0);
for (int hi = 0, lo = 0; hi < static_cast<int>(a.size()); hi++) {
    window.add(a[hi]);
    if (window.count() > 3) {
        window.remove(a[lo++]);
    }
}
assert(window.count() == 3); // The window holds 9, 3, and 3.
assert(window.lower_median() == 3 && EQ(window.median(), 3));

// Removals need not follow arrival order, and may target the median's own value.
DeletableMedian<int> tracker;
for (int x : {1, 2, 3, 4, 5}) {
    tracker.add(x);
}
assert(EQ(tracker.median(), 3));
tracker.remove(3); // Erasing the median itself steps the iterator off its own node first.
assert(tracker.lower_median() == 2 && tracker.upper_median() == 4);
tracker.remove(5);
assert(EQ(tracker.median(), 2));

// Duplicates of the median are erased one copy at a time.
DeletableMedian<int> repeats;
for (int x : {7, 7, 7}) {
    repeats.add(x);
}
repeats.remove(7);
assert(repeats.count() == 2 && EQ(repeats.median(), 7));
repeats.remove(7);
assert(repeats.count() == 1 && repeats.lower_median() == 7);
return 0;
}

```

### 1.6.6 Probabilistic Sketches

A sketch answers questions about a stream using far less memory than the stream itself, in exchange for a bounded, one-sided error. This section implements a Bloom filter, which tests set membership, and a count-min sketch, which estimates how often each value occurred. Both spread every value over several independent positions of a fixed-size table, and both derive those positions from one pair of base hashes: the Kirsch-Mitzenmacher technique uses  $h_i(x) = h_1(x) + i \cdot h_2(x)$  instead of computing an unrelated hash for every position, which matches the false-positive behavior of truly independent hashes while paying for only two. Both base hashes come from the SplitMix64 mixer of section 3.2.1, and the second is forced odd so that repeated steps never stand still.

A Bloom filter represents a set as a bit array. Inserting a value sets the  $k$  bits it hashes to, and a membership test reports that a value is present only if all  $k$  of its bits are set. Bits are never cleared, so a value that was inserted always tests positive: the error is one-sided, and only a false positive is possible, produced when unrelated insertions happen to have set all  $k$  bits. For  $n$  values in  $m$  bits, the false-positive probability is minimized by  $k = (m/n) \ln 2$ , which yields  $m = -n \ln p / (\ln 2)^2$  bits for a target rate  $p$ .

- `BloomFilter<T>(capacity, false_positive_rate)` constructs an empty filter sized for `capacity` insertions at the given target rate, where `capacity` is positive and the rate lies in  $(0, 1)$ .
- `BloomFilter<T, Hash>(capacity, false_positive_rate, hash)` instead stores the supplied hasher.
- `insert(x)` adds `x` to the filter.
- `contains(x)` returns whether `x` may be in the filter. It never returns `false` for a value that was inserted, but may return `true` for one that was not.
- `count()` returns the number of insertions performed, counting repeated values separately.
- `bit_count()` and `hash_count()` return the derived table size  $m$  and number of positions  $k$ .

- `false_positive_rate()` returns the current estimated rate  $(1 - e^{-kn/m})^k$ , which rises as the filter fills past its planned capacity.

Deletion is not supported, since clearing a bit could hide an unrelated value that also set it. Use a counting Bloom filter, which replaces each bit with a small counter, when values must leave.

#### Time Complexity:

- $O(1)$  per call to `count()`, `bit_count()`, `hash_count()`, and `false_positive_rate()`.
- $O(k)$  per call to `insert()` and `contains()`, where  $k$  is `hash_count()`, plus the cost of one call to the hasher.
- $O(m)$  per call to the constructor, where  $m$  is `bit_count()`.

#### Space Complexity:

- $O(m)$  for storage of the filter, where  $m$  is `bit_count()`.
- $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdint>
#include <functional>
#include <utility>
#include <vector>

uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<typename T, typename Hash>
std::pair<uint64_t, uint64_t> hash_pair(const Hash &hash, const T &x) {
    uint64_t h1 = splitmix64(static_cast<uint64_t>(hash(x)));
    return {h1, splitmix64(h1 + 1) | 1};
}

template<typename T, typename Hash = std::hash<T>>
class BloomFilter {
    std::vector<uint64_t> words;
    uint64_t nbits;
    int hashes, inserted;
    Hash hash;

public:
    BloomFilter(int capacity, double false_positive_rate, Hash hash = Hash{})
        : nbits(0), hashes(0), inserted(0), hash(hash) {
        assert(capacity > 0 && false_positive_rate > 0 && false_positive_rate < 1);
        double ln2 = std::log(2.0);
        nbits =
            static_cast<uint64_t>(std::ceil(-capacity * std::log(false_positive_rate) / (ln2 * ln2)));
        hashes = std::max(1, static_cast<int>(std::lround(nbits * ln2 / capacity)));
        words.resize((nbits + 63) / 64);
    }

    void insert(const T &x) {
        auto [h1, h2] = hash_pair(hash, x);
        for (int i = 0; i < hashes; i++) {
            uint64_t bit = (h1 + static_cast<uint64_t>(i) * h2) % nbits;
            words[bit >> 6] |= 1ULL << (bit & 63);
        }
        inserted++;
    }

    bool contains(const T &x) const {
        auto [h1, h2] = hash_pair(hash, x);
```

```

for (int i = 0; i < hashes; i++) {
    uint64_t bit = (h1 + static_cast<uint64_t>(i) * h2) % nbits;
    if (((words[bit >> 6] >> (bit & 63)) & 1) == 0) {
        return false;
    }
}
return true;
}

int count() const { return inserted; }
uint64_t bit_count() const { return nbits; }
int hash_count() const { return hashes; }

double false_positive_rate() const {
    double filled = 1 - std::exp(-1.0 * hashes * inserted / nbits);
    return std::pow(filled, hashes);
}
};

```

A count-min sketch estimates the total weight added under each value. It stores an  $R$  by  $C$  table of counters and, for each row, adds the weight to the one counter that the value hashes to in that row. Every counter that a value touches therefore holds that value's true total plus the totals of whatever else collided there, so with nonnegative weights each row is an overestimate and the minimum over the rows is the sharpest one available. Choosing  $C = \lceil e/\epsilon \rceil$  and  $R = \lceil \ln(1/\delta) \rceil$  bounds the overestimate by  $\epsilon N$  with probability at least  $1 - \delta$ , where  $N$  is the total weight added.

- `CountMinSketch<T>(eps, delta)` constructs an empty sketch whose estimates exceed the true count by at most `eps` times the total weight, with probability at least  $1 - \text{delta}$ . Both parameters must lie in  $(0, 1)$ .
- `CountMinSketch<T, Hash>(eps, delta, hash)` instead stores the supplied hasher.
- `add(x, c = 1)` adds weight `c` to the count of `x`.
- `count(x)` returns the estimated total weight of `x`, which is never an underestimate.
- `total()` returns the total weight added over all values.
- `num_rows()` and `num_cols()` return the derived table dimensions.

The error bound is relative to the total weight of the whole stream, so a sketch estimates heavy values well and light ones poorly. That makes it a natural companion to the deterministic heavy-hitter candidates of section 1.6.3: run both, then use the sketch to rank the candidates. Negative weights break the guarantee, since a row is then no longer an overestimate; deletions require the count-mean-min variant or a conservative sketch.

#### Time Complexity:

- $O(R)$  per call to `add()` and `count()`, plus the cost of one call to the hasher.
- $O(R \cdot C)$  per call to the constructor.
- $O(1)$  per call to `total()`, `num_rows()`, and `num_cols()`.

#### Space Complexity:

- $O(w \cdot d)$  for storage of the sketch.
- $O(1)$  auxiliary for all operations.

```

template<typename T, typename Hash = std::hash<T>>
class CountMinSketch {
    std::vector<std::vector<int64_t>>> table;
    int rows, cols;
    int64_t weight;
    Hash hash;

public:
    CountMinSketch(double eps, double delta, Hash hash = Hash{}) : weight(0), hash(hash) {
        assert(eps > 0 && eps < 1 && delta > 0 && delta < 1);
        rows = std::max(1, static_cast<int>(std::ceil(std::log(1 / delta))));
        cols = static_cast<int>(std::ceil(std::exp(1.0) / eps));
        table.assign(rows, std::vector<int64_t>(cols));
    }
};

```



```

}

void add(const T &x, int64_t c = 1) {
    auto [h1, h2] = hash_pair(hash, x);
    for (int r = 0; r < rows; r++) {
        table[r][(h1 + static_cast<uint64_t>(r) * h2) % cols] += c;
    }
    weight += c; // Overflow warning.
}

int64_t count(const T &x) const {
    auto [h1, h2] = hash_pair(hash, x);
    int64_t res = INT64_MAX;
    for (int r = 0; r < rows; r++) {
        res = std::min(res, table[r][(h1 + static_cast<uint64_t>(r) * h2) % cols]);
    }
    return res;
}

int64_t total() const { return weight; }
int num_rows() const { return rows; }
int num_cols() const { return cols; }
};

```

## Example Usage

```

#include <string>
using namespace std;

int main() {
    BloomFilter<string> filter(1000, 0.01);
    assert(filter.bit_count() == 9586 && filter.hash_count() == 7);
    for (int i = 0; i < 1000; i++) {
        filter.insert("key" + to_string(i));
    }
    assert(filter.count() == 1000);
    for (int i = 0; i < 1000; i++) {
        assert(filter.contains("key" + to_string(i))); // Inserted values never test negative.
    }
    int false_positives = 0;
    for (int i = 1000; i < 2000; i++) {
        false_positives += filter.contains("key" + to_string(i));
    }
    assert(false_positives < 40); // Absent values may test positive, about 10 of these 1000.
    assert(filter.false_positive_rate() < 0.02);

    CountMinSketch<string> sketch(0.01, 0.01);
    sketch.add("apple", 3);
    sketch.add("banana");
    assert(sketch.total() == 4);
    assert(sketch.count("apple") == 3 && sketch.count("banana") == 1);
    // A value that was never added reads 0 unless it collides in every row, and an estimate is never
    // lower than the truth.
    assert(sketch.count("cherry") <= sketch.total());
    return 0;
}

```

## 1.7 Bit Manipulation

### 1.7.1 Bit Operations and Masks

Integers are all fixed-size sets of bits, where the  $i$ -th bit (counting from the least significant bit where  $i = 0$ ) is “present” when set to 1. Treating an `int` as a bitmask makes set membership, insertion, deletion, and iteration into single machine instructions, which is why bitmasks are the backbone of subset dynamic programming and many low-level tricks. The helpers below operate on `Mask`, which is `uint32_t` by default; change its alias to `uint64_t` to use 64-bit masks. The loop implementations below exist for education only; production code should prefer the matching GCC built-ins when available, or on C++20 systems the generic `constexpr` equivalents from the `<bit>` header, which are additionally well-defined at 0.

- `test_bit(x, i)` returns whether the  $i$ -th bit of  $x$  is set, where  $0 \leq i < \text{MASK\_BITS}$ .
- `set_bit(x, i)`, `clear_bit(x, i)`, and `toggle_bit(x, i)` return  $x$  with the  $i$ -th respectively forced to 1, forced to 0, or flipped, where  $0 \leq i < \text{MASK\_BITS}$ .
- `lowest_set_bit(x)` returns the value of the lowest set bit of  $x$  (a power of two), or 0 if  $x$  is 0. `clear_lowest_set_bit(x)` returns  $x$  with its lowest set bit removed.
- `popcount(x)` returns the number of set bits, analogous to C++20’s `std::popcount()`.
- `parity(x)` returns 1 if the number of set bits is odd and 0 otherwise.
- `ctz(x)` returns the number of trailing 0-bits for  $x > 0$ , analogous to C++20’s `std::countr_zero()`.
- `ffs(x)` returns one plus the position of the lowest 1-bit of  $x$ , or 0 if  $x$  is 0.
- `clz(x)` returns the number of leading 0-bits for  $x > 0$ , analogous to C++20’s `std::countl_zero()`.
- `is_pow2(x)` returns whether  $x$  has exactly one set bit, analogous to C++20’s `std::has_single_bit()`.
- `floor_pow2(x)` returns the largest power of two that is  $\leq x$  (for  $x > 0$ ), analogous to C++20’s `std::bit_floor()`.
- `ceil_pow2(x)` returns the smallest power of two that is  $\geq x$ , for  $0 < x \leq 2^{b-1}$  where  $b$  is `MASK_BITS`, analogous to C++20’s `std::bit_ceil()`.
- `for_each_set_bit(x, f)` calls `f(i)` once for each set bit position  $i$  of  $x$ , in increasing order.

**Time Complexity:**

- $O(b)$  worst case per call to `popcount()`, `parity()`, `ctz()`, `ffs()`, `clz()`, and `for_each_set_bit()`, where  $b$  is `MASK_BITS`.
- $O(1)$  per call to every other function.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <climits>
#include <cstdint>

using Mask = uint32_t;
const int MASK_BITS = sizeof(Mask) * CHAR_BIT;

bool test_bit(Mask x, int i) { return (x >> i) & 1; }
Mask set_bit(Mask x, int i) { return x | (Mask{1} << i); }
Mask clear_bit(Mask x, int i) { return x & ~(Mask{1} << i); }
Mask toggle_bit(Mask x, int i) { return x ^ (Mask{1} << i); }
Mask lowest_set_bit(Mask x) { return x & -x; }
Mask clear_lowest_set_bit(Mask x) { return x & (x - 1); }

// popcount(), parity(), ctz(), ffs(), and clz() are spelled out here for educational purposes.

int popcount(Mask x) { // std::popcount(x) in C++20.
    int count = 0;
    for (; x != 0; x = clear_lowest_set_bit(x)) {
        count++;
    }
    return count;
}
```

```

int parity(Mask x) {
    return popcount(x) & 1;
}

int ctz(Mask x) { // std::countr_zero(x) in C++20.
    assert(x != 0);
    int count = 0;
    for (; (x & 1) == 0; x >>= 1) {
        count++;
    }
    return count;
}

int ffs(Mask x) {
    return x == 0 ? 0 : ctz(x) + 1;
}

int clz(Mask x) { // std::countl_zero(x) in C++20.
    assert(x != 0);
    int count = 0;
    for (Mask mask = Mask{1} << (MASK_BITS - 1); (x & mask) == 0; mask >>= 1) {
        count++;
    }
    return count;
}

bool is_pow2(Mask x) { // std::has_single_bit(x) in C++20.
    return x != 0 && (x & (x - 1)) == 0;
}

Mask floor_pow2(Mask x) { // std::bit_floor(x) in C++20.
    assert(x != 0);
    return Mask{1} << (MASK_BITS - 1 - clz(x));
}

Mask ceil_pow2(Mask x) { // std::bit_ceil(x) in C++20.
    assert(0 < x && x <= (Mask{1} << (MASK_BITS - 1)));
    return is_pow2(x) ? x : Mask{1} << (MASK_BITS - clz(x));
}

template<typename Fn>
void for_each_set_bit(Mask x, Fn f) {
    while (x != 0) {
        f(ctz(x));
        x = clear_lowest_set_bit(x);
    }
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    Mask x = 0b101100;
    assert(test_bit(x, 2) == true);
    assert(test_bit(x, 0) == false);
    assert(set_bit(x, 0) == 0b101101U);
    assert(clear_bit(x, 2) == 0b101000U);
    assert(toggle_bit(x, 3) == 0b100100U);

    assert(lowest_set_bit(x) == 0b100U);
    assert(clear_lowest_set_bit(x) == 0b101000U);
    assert(lowest_set_bit(0) == 0U);
}

```

```

assert(clear_lowest_set_bit(0) == 0U);
assert(popcount(x) == 3);
assert(popcount(0) == 0);
assert(parity(x) == 1);
assert(parity(0b101101U) == 0);
assert(parity(0) == 0);
assert(ctz(x) == 2);
assert(ffs(x) == 3);
assert(ffs(0) == 0);
assert(clz(x) == MASK_BITS - 6);

assert(is_pow2(16) == true);
assert(is_pow2(24) == false);
assert(floor_pow2(20) == 16U);
assert(ceil_pow2(20) == 32U);
assert(ceil_pow2(16) == 16U);
assert(ceil_pow2((Mask{1} << (MASK_BITS - 1)) - 1) == (Mask{1} << (MASK_BITS - 1)));

vector<int> bits;
for_each_set_bit(x, [&](int b) { bits.push_back(b); });
assert((bits == vector<int>{2, 3, 5}));
return 0;
}

```

## 1.7.2 Subset and Submask Iteration

Many bitmask algorithms must visit related masks in a structured order: every submask of a fixed mask, every mask with a fixed number of set bits, or every mask while accumulating contributions from its submasks. Each pattern has a short, well-known idiom.

The classic submask loop `for (int s = m; s > 0; s = (s - 1) & m)` walks every nonzero submask of `m` in decreasing order; subtracting 1 borrows through the zero bits of `m`, and the `& m` masks them back off. Summed over all masks `m` of `n` bits, the total work is  $\sum_k \binom{n}{k} 2^k = 3^n$ , since each of the `n` bit positions is independently absent, present in `m` only, or present in both `m` and `s`.

Gosper’s hack advances to the next mask with the same popcount. It isolates the lowest set bit and adds it to move the lowest movable 1 one position left, clearing the block of 1-bits beneath it. The remaining expression counts those cleared bits and packs them into the least-significant positions, producing the smallest larger integer with the same number of set bits.

- `submasks(m)` returns all submasks of `m`, including `m` itself and 0, in decreasing order.
- `next_same_popcount(x)` returns the smallest integer greater than `x` with the same number of set bits (Gosper’s hack), for `x > 0` when that next mask is representable in `Mask`.
- `masks_with_popcount(n, k)` returns all `k`-bit subsets of an `n`-bit universe as masks, in increasing order, where  $0 \leq k \leq n < \text{MASK\_BITS}$ .
- `subset_sum_transform(f)` overwrites `f` (indexed by mask over `n` bits) so that `f[m]` becomes the sum of the original `f[s]` over all submasks `s` of `m`. This “sum over subsets” (SOS) DP is the bitmask analog of a prefix sum, accumulating one bit dimension at a time. The value type must support `+=`.

Overflow warning: All intermediate sums must be representable in the value type.

### Time Complexity:

- $O(2^p)$  per call to `submasks(m)`, where  $p$  is `popcount(m)`.
- $O(1)$  per call to `next_same_popcount()`.
- $O(\binom{n}{k})$  per call to `masks_with_popcount(n, k)`.
- $O(n \cdot 2^n)$  per call to `subset_sum_transform(f)`, where `f` has  $2^n$  entries.

### Space Complexity:

- $O(1)$  auxiliary for `next_same_popcount()` and `subset_sum_transform()`.

- $O(1)$  auxiliary and  $O(s)$  for the returned masks from `submasks(m)` and `masks_with_popcount(n, k)`, where  $s$  is the result size.

```
#include <cassert>
#include <climits>
#include <cstdint>
#include <vector>

using Mask = uint32_t;
const int MASK_BITS = sizeof(Mask) * CHAR_BIT;

std::vector<Mask> submasks(Mask m) {
    std::vector<Mask> res;
    Mask s = m;
    while (true) {
        res.push_back(s);
        if (s == 0) {
            break;
        }
        s = (s - 1) & m;
    }
    return res;
}

Mask next_same_popcount(Mask x) {
    assert(x != 0);
    Mask c = x & (Mask{0} - x), r = x + c;
    assert(r != 0);
    return r | ((x ^ r) >> 2) / c;
}

std::vector<Mask> masks_with_popcount(int n, int k) {
    assert(0 <= k && k <= n && n < MASK_BITS);
    std::vector<Mask> res;
    if (k == 0) {
        return {0};
    }
    Mask limit = Mask{1} << n;
    for (Mask x = (Mask{1} << k) - 1; x < limit; x = next_same_popcount(x)) {
        res.push_back(x);
    }
    return res;
}

template<typename T>
void subset_sum_transform(std::vector<T> &f) {
    int size = static_cast<int>(f.size());
    assert(size > 0 && (size & (size - 1)) == 0);
    int n = __builtin_ctz(static_cast<unsigned>(size)); // size = 2^n.
    for (int b = 0; b < n; b++) {
        for (int m = 0; m < size; m++) {
            if (m & (1U << b)) {
                f[m] += f[m ^ (1U << b)];
            }
        }
    }
}
```

## Example Usage

```
using namespace std;

int main() {
    assert((submasks(0b101) == vector<Mask>{0b101, 0b100, 0b001, 0b000}));

    assert(next_same_popcount(0b0110) == 0b1001U);
}
```

```

assert(
    (masks_with_popcount(4, 2) == vector<Mask>{0b0011, 0b0101, 0b0110, 0b1001, 0b1010, 0b1100})
);

// f[m] = 1 for every mask; after the transform f[m] counts submasks of m = 2^popcount(m).
vector<int> f(1 << 3, 1);
subset_sum_transform(f);
assert(f[0b000] == 1);
assert(f[0b101] == 4);
assert(f[0b111] == 8);
return 0;
}

```

### 1.7.3 Bitmask DP

Bitmask dynamic programming uses a bitmask to represent a small set or frontier, then treats that mask as the DP state. This is useful when one dimension is small enough for  $2^n$  states.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

```

The assignment problem matches workers one-to-one with jobs while minimizing the total cost. The workers are processed in a fixed order: `dp[mask]` is the minimum cost after assigning the first `popcount(mask)` workers to the selected jobs, and the next worker tries every unused job.

- `min_cost_assignment(cost)` returns a pair `(sum, job)`, where `sum` is the minimum cost of assigning each worker to a distinct job, and `job[i]` is the job assigned to worker `i`. The input is a square matrix with one row per worker and one column per job.

Overflow warning: The accumulated cost must fit in `int64_t`.

**Time Complexity:**  $O(n \cdot 2^n)$  per call to `min_cost_assignment()`, where  $n$  is the number of workers and jobs.

**Space Complexity:**  $O(2^n)$  auxiliary and  $O(n)$  for the returned assignment from `min_cost_assignment()`.

```

std::pair<int64_t, std::vector<int>> min_cost_assignment(
    const std::vector<std::vector<int64_t>> &cost
) {
    int n = static_cast<int>(cost.size());
    assert(n < 31);
    int states = 1 << n;
    std::vector<int64_t> dp(states);
    std::vector<int> parent(states, -1);
    for (int mask = 0; mask < states; mask++) {
        int worker = __builtin_popcount(static_cast<unsigned>(mask));
        if (worker == n) {
            continue;
        }
        for (int job = 0; job < n; job++) {
            if ((mask & (1 << job)) == 0) {
                int next = mask | (1 << job);
                int64_t candidate = dp[mask] + cost[worker][job]; // Overflow warning.
                if (parent[next] == -1 || dp[next] > candidate) {
                    dp[next] = candidate;
                    parent[next] = job;
                }
            }
        }
    }
}

```

```

// Optional: reconstruct one optimal assignment.
std::vector<int> job(n);
for (int mask = states - 1, worker = n - 1; worker >= 0; worker--) {
    job[worker] = parent[mask];
    mask ^= 1 << job[worker];
}
return {dp[states - 1], job};
}

```

The minimum set cover problem asks for the fewest input sets whose union contains every element of a fixed universe. Both input sets and covered subsets are represented as masks: `dp[mask]` is the minimum number of chosen sets needed to cover exactly the elements in `mask`, and adding one set moves to the union mask.

- `set_cover(sets, universe_size)` returns a pair `(count, chosen)`, where `count` is the minimum number of sets needed to cover all elements in  $[0, u)$ , where  $u$  is `universe_size`, and `chosen` contains one optimal list of set indices. If no cover exists, `count` is `-1` and `chosen` is empty. Each input set is represented as a bitmask.

**Time Complexity:**  $O(s \cdot 2^u)$  per call, where  $s$  is the number of sets and  $u$  is `universe_size`.

**Space Complexity:**  $O(2^u)$  auxiliary and  $O(s)$  for the returned indices.

```

std::pair<int, std::vector<int>> set_cover(const std::vector<int> &sets, int universe_size) {
    assert(0 <= universe_size && universe_size < 31);
    int states = 1 << universe_size;
    int full = states - 1;
    for (int subset : sets) {
        assert((subset & ~full) == 0);
    }
    int n = static_cast<int>(sets.size());
    int inf = n + 1;
    std::vector<int> dp(states, inf), parent(states, -1), prev(states, -1);
    dp[0] = 0;
    for (int mask = 0; mask < states; mask++) {
        if (dp[mask] == inf) {
            continue;
        }
        for (int i = 0; i < n; i++) {
            int next = mask | sets[i];
            if (dp[next] > dp[mask] + 1) {
                dp[next] = dp[mask] + 1;
                parent[next] = i;
                prev[next] = mask;
            }
        }
    }
    if (dp[full] == inf) {
        return {-1, {}};
    }
    // Optional: reconstruct one minimum set cover.
    std::vector<int> chosen;
    for (int mask = full; mask != 0; mask = prev[mask]) {
        int i = parent[mask];
        chosen.push_back(i);
    }
    std::reverse(chosen.begin(), chosen.end());
    return {dp[full], chosen};
}

```

The minimum-cost set partition problem divides all elements into disjoint nonempty groups when the cost of every possible group is known. For each set `mask`, choosing a submask as one group leaves `mask ^ sub` to be partitioned, giving the recurrence `dp[mask] = min(dp[mask ^ sub] + cost[sub])`. Over all masks, this has  $O(3^n)$  transitions.

- `min_cost_partition(group_cost)` returns the minimum total cost to partition all elements into disjoint nonempty groups, where `group_cost[mask]` is the cost of taking `mask` as one group.

Overflow warning: The accumulated cost must fit in `int64_t`.

**Time Complexity:**  $O(3^n)$  per call, where  $n$  is the number of elements.

**Space Complexity:**  $O(2^n)$  auxiliary.

```
int64_t min_cost_partition(const std::vector<int64_t> &group_cost) {
    int states = static_cast<int>(group_cost.size());
    assert(states > 0 && (states & (states - 1)) == 0);
    std::vector<int64_t> dp(states);
    for (int mask = 1; mask < states; mask++) {
        dp[mask] = group_cost[mask];
        for (int sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) & mask) {
            int64_t candidate = dp[mask ^ sub] + group_cost[sub]; // Overflow warning.
            dp[mask] = std::min(dp[mask], candidate);
        }
    }
    return dp[states - 1];
}
```

Domino tiling asks for the number of ways to cover a rectangle with horizontal ( $1 \times 2$ ) and vertical ( $2 \times 1$ ) dominoes. Processing cells in row-major order, a profile mask records which cells on the current frontier are already occupied. An occupied current cell is cleared; otherwise, placing a horizontal domino marks the next column, while placing a vertical one carries the current column into the next row. This technique is often called DP on a broken profile or plug DP.

- `count_domino_tilings(rows, cols)` returns the number of ways to tile a `rows` by `cols` rectangle with  $1 \times 2$  and  $2 \times 1$  dominoes.

Overflow warning: The tiling count must fit in `int64_t`.

**Time Complexity:**  $O(R \cdot C \cdot 2^C)$  per call, where  $R$  and  $C$  are the number of rows and columns, respectively.

**Space Complexity:**  $O(2^C)$  auxiliary.

```
int64_t count_domino_tilings(int rows, int cols) {
    assert(rows >= 0 && cols >= 0);
    if (rows == 0 || cols == 0) {
        return 1;
    }
    if (rows < cols) {
        std::swap(rows, cols);
    }
    assert(cols < 31);
    int states = 1 << cols;
    std::vector<int64_t> dp(states), next(states);
    dp[0] = 1;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            next.assign(states, 0);
            for (int mask = 0; mask < states; mask++) {
                if (dp[mask] == 0) {
                    continue;
                }
                if (mask & (1 << c)) {
                    next[mask ^ (1 << c)] += dp[mask]; // Overflow warning.
                } else {
                    if (c + 1 < cols && (mask & (1 << (c + 1))) == 0) {
                        next[mask | (1 << (c + 1))] += dp[mask];
                    }
                    if (r + 1 < rows) {
                        next[mask | (1 << c)] += dp[mask];
                    }
                }
            }
        }
        dp.swap(next);
    }
}
```



```

    }
}
return dp[0];
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int64_t>> cost{{9, 2, 7}, {6, 4, 3}, {5, 8, 1}};
    auto [assignment_cost, job] = min_cost_assignment(cost);
    assert(assignment_cost == 9 && (job == vector<int>{1, 0, 2}));

    auto [cover_count, chosen] = set_cover(vector<int>{0b0011, 0b0110, 0b1100, 0b1000}, 4);
    assert(cover_count == 2 && (chosen == vector<int>{0, 2}));

    auto [impossible_count, impossible_chosen] = set_cover(vector<int>{0b001, 0b010}, 3);
    assert(impossible_count == -1 && impossible_chosen.empty());

    vector<int64_t> group_cost{
        0,          // Empty group is unused.
        4, 6, 7,    // Singletons.
        6, 2, 3,    // Pairs.
        9,          // All three together.
    };
    assert(min_cost_partition(group_cost) == 7); // Groups {0} and {1, 2}.

    assert(count_domino_tilings(0, 1000000000) == 1);
    assert(count_domino_tilings(2, 3) == 3);
    assert(count_domino_tilings(3, 3) == 0);
    assert(count_domino_tilings(4, 4) == 36);
    return 0;
}

```

### 1.7.4 Gray Code

The binary reflected Gray code is an ordering of all  $2^n$  bitmasks in which consecutive masks differ in exactly one bit. It is useful whenever the cost of moving between adjacent states depends on a single bit flip: enumerating subsets while incrementally maintaining a value, hardware where only one bit may toggle at a time, and constructions like Hamiltonian paths on the hypercube.

The Gray code at 0-based rank  $k$  is obtained from the ordinary binary value  $k$  by  $k \oplus (k \gg 1)$ . Incrementing  $k$  flips one binary bit and every lower bit, but XORing each bit with its higher neighbor cancels all but the highest of those changes, so consecutive Gray codes differ in exactly one bit. The map is a bijection: each binary bit is the prefix XOR of the Gray bits at and above it. The inverse loop computes those prefix XORs in doubling steps to recover the original rank.

- `gray_code(k)` returns the mask at 0-based rank  $k$  in Gray code order.
- `inverse_gray_code(g)` returns the 0-based rank  $k$  such that `gray_code(k)` equals  $g$ .
- `gray_sequence(n)` returns all  $2^n$  masks in Gray code order; consecutive entries (and the last with the first when  $n > 0$ ) differ in exactly one bit, where  $0 \leq n < \text{MASK\_BITS}$ .

#### Time Complexity:

- $O(1)$  per call to `gray_code()`.
- $O(\log b)$  per call to `inverse_gray_code()`, where  $b$  is `MASK_BITS`.
- $O(2^n)$  per call to `gray_sequence(n)`.

**Space Complexity:**

- $O(1)$  auxiliary for `gray_code()` and `inverse_gray_code()`.
- $O(1)$  auxiliary and  $O(2^n)$  for the returned sequence from `gray_sequence(n)`.

```
#include <cassert>
#include <climits>
#include <cstdint>
#include <vector>

using Mask = uint32_t;
const int MASK_BITS = sizeof(Mask) * CHAR_BIT;

Mask gray_code(Mask k) {
    return k ^ (k >> 1);
}

Mask inverse_gray_code(Mask g) {
    for (int shift = 1; shift < MASK_BITS && (g >> shift) != 0; shift <= 1) {
        g ^= g >> shift;
    }
    return g;
}

std::vector<Mask> gray_sequence(int n) {
    assert(0 <= n && n < MASK_BITS);
    std::vector<Mask> res(Mask{1} << n);
    for (Mask k = 0; k < static_cast<Mask>(res.size()); k++) {
        res[k] = gray_code(k);
    }
    return res;
}
```

**Example Usage**

```
#include <cassert>
using namespace std;

int main() {
    assert(gray_code(0) == 0b000U);
    assert(gray_code(1) == 0b001U);
    assert(gray_code(2) == 0b011U);
    assert(gray_code(3) == 0b010U);

    for (Mask k = 0; k < 256; k++) {
        assert(inverse_gray_code(gray_code(k)) == k);
    }
    assert(inverse_gray_code(gray_code(0xdeadbeefU)) == 0xdeadbeefU);

    vector<Mask> seq = gray_sequence(3);
    assert((seq == vector<Mask>{0b000, 0b001, 0b011, 0b010, 0b110, 0b111, 0b101, 0b100}));
    for (int i = 0; i < static_cast<int>(seq.size()); i++) {
        Mask diff = seq[i] ^ seq[(i + 1) % seq.size()];
        assert((diff & (diff - 1)) == 0); // Cyclically, one bit changes per step.
    }
    return 0;
}
```

**1.7.5 XOR Basis**

An XOR basis (also called a linear basis over the field  $\text{GF}(2)$ ) is the analog of a vector space basis for integers under the bitwise XOR operation. Each integer is treated as a vector of bits, where XOR plays the role of vector

addition. Inserting a set of integers one at a time and keeping only those that are linearly independent yields a basis whose XOR-combinations (subset XORs) span exactly the same set of values as the original integers. From a basis of size  $r$ , exactly  $2^r$  distinct values are reachable, and queries such as the maximum attainable XOR become a simple greedy scan.

The basis is kept in row-echelon form: the  $i$ -th basis element is either 0 or a value whose highest set bit is bit  $i$ , and no two basis elements share the same highest bit. Reducing a value means XORing it against the basis elements from the highest bit down, eliminating each set bit that a basis element covers.

- `XorBasis()` constructs an empty basis over `Mask`, which is `uint64_t` by default.
- `insert(x)` adds `x` to the basis, returning `true` if `x` was linearly independent of the current basis (increasing its size), or `false` if `x` was already representable as a subset XOR.
- `contains(x)` returns whether `x` is representable as the XOR of some subset of the inserted values, i.e. whether it reduces to 0 against the basis.
- `max_xor(base = 0)` returns the maximum value of `base` XORed with some subset XOR of the basis. With the default `base` of 0, this is the maximum subset XOR.
- `size()` returns the number of elements in the basis (its rank).

#### Time Complexity:

- $O(b)$  per call to `insert()`, `contains()`, and `max_xor()`, where  $b$  is the number of bits.
- $O(1)$  per call to `size()`.

#### Space Complexity:

- $O(b)$  for storage of the basis.
- $O(1)$  auxiliary for all operations.

```
#include <array>
#include <climits>
#include <cstdint>

using Mask = uint64_t;
const int MASK_BITS = sizeof(Mask) * CHAR_BIT;

class XorBasis {
    std::array<Mask, MASK_BITS> basis{}; // basis[i] != 0 has its highest set bit at bit i.
    int basis_size = 0;

public:
    bool insert(Mask x) {
        for (int i = MASK_BITS - 1; i >= 0; i--) {
            if (((x >> i) & 1)) {
                continue;
            }
            if (basis[i] == 0) {
                basis[i] = x;
                basis_size++;
                return true;
            }
            x ^= basis[i];
        }
        return false;
    }

    bool contains(Mask x) const {
        for (int i = MASK_BITS - 1; i >= 0; i--) {
            if (((x >> i) & 1) && basis[i] != 0) {
                x ^= basis[i];
            }
        }
        return x == 0;
    }

    Mask max_xor(Mask base = 0) const {
```

```

    for (int i = MASK_BITS - 1; i >= 0; i--) {
        if ((base ^ basis[i]) > base) {
            base ^= basis[i];
        }
    }
    return base;
}

int size() const { return basis_size; }
};

```

## Example Usage

```

#include <cassert>

int main() {
    XorBasis b;
    assert(b.insert(1)); // 001
    assert(b.insert(2)); // 010
    assert(!b.insert(3)); // 011 = 1 ^ 2, already representable.
    assert(b.size() == 2);

    assert(b.contains(3));
    assert(b.contains(0));
    assert(!b.contains(4));

    assert(b.max_xor() == 3); // Best subset XOR over {0, 1, 2, 3}.
    assert(b.max_xor(4) == 7); // 4 ^ 3.
    return 0;
}

```

### 1.7.6 Binary Trie

Maintain a multiset of nonnegative  $b$ -bit integers as a binary trie, storing each value as a root-to-leaf path of its bits from the most significant (bit  $b - 1$ ) down to the least significant. Every node keeps a count of the stored values passing through it, which supports deletion and counting queries, and lets the trie answer XOR-extremal queries by a greedy walk down the bits.

The classic application is the maximum-XOR query: to maximize  $\mathbf{x} \oplus \mathbf{y}$  over all stored  $\mathbf{y}$ , walk from the most significant bit. At each step, descend toward the child whose bit differs from that of  $\mathbf{x}$  whenever such a branch exists, since setting a higher bit of the result always dominates any combination of lower bits. The minimum-XOR query is the mirror image, preferring the matching bit.

This is a multiset (a value may be inserted more than once), and is distinct from the XOR basis: the basis spans subset XORs of a fixed set, whereas this answers XOR queries against the stored elements themselves with support for insertion and deletion.

Values must lie in  $[0, 2^b)$  for bit width  $b$ ; the template parameters select the unsigned word type `U` and the bit width `BITS`, which must be less than the width of `U` (for example, 30 with `uint32_t` or 62 with `uint64_t`). Nodes are kept in a pool indexed by integer, where index 0 is the root and also serves as the null child.

- `BinaryTrie<U, BITS>()` constructs an empty multiset.
- `size()` returns the number of stored values, and `empty()` returns whether there are none.
- `insert(x)` adds one occurrence of  $\mathbf{x}$ .
- `count(x)` returns the number of occurrences of  $\mathbf{x}$ .
- `erase(x)` removes one occurrence of  $\mathbf{x}$ , returning whether one was present.
- `max_xor(x)` and `min_xor(x)` return the maximum and minimum of  $\mathbf{x} \oplus \mathbf{y}$  over all stored  $\mathbf{y}$ . The trie must be non-empty.

- `count_xor_less(x, k)` returns the number of stored `y` (with multiplicity) such that `|x ^ y|` is strictly less than `k`.

**Time Complexity:**

- $O(b)$  per call to `insert()`, `erase()`, `count()`, `max_xor()`, `min_xor()`, and `count_xor_less()`, where  $b$  is the bit width `BITS`.
- $O(1)$  per call to `size()` and `empty()`.

**Space Complexity:**

- $O(n \cdot b)$  for storage, where  $n$  is the number of distinct values inserted so far.
- $O(1)$  auxiliary per operation.

```
#include <array>
#include <cassert>
#include <climits>
#include <cstdint>
#include <vector>

template<typename U = uint32_t, int BITS = 30>
class BinaryTrie {
    static_assert(0 < BITS && BITS < static_cast<int>(sizeof(U) * CHAR_BIT));

    std::vector<std::array<int, 2>> child; // child[node][bit], where 0 means no child.
    std::vector<int> cnt;                // Number of stored values passing through each node.

    static void assert_fits(U x) { assert((x >> BITS) == 0); }

    int new_node() {
        child.push_back({0, 0});
        cnt.push_back(0);
        return static_cast<int>(cnt.size()) - 1;
    }

public:
    BinaryTrie() { new_node(); } // Node 0 is the root (and doubles as the null child).

    int size() const { return cnt[0]; }
    bool empty() const { return cnt[0] == 0; }

    void insert(U x) {
        assert_fits(x);
        int node = 0;
        cnt[node]++;
        for (int b = BITS - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (child[node][bit] == 0) {
                int next = new_node();
                child[node][bit] = next;
            }
            node = child[node][bit];
            cnt[node]++;
        }
    }

    int count(U x) const {
        assert_fits(x);
        int node = 0;
        for (int b = BITS - 1; b >= 0; b--) {
            int next = child[node][(x >> b) & 1];
            if (next == 0) {
                return 0;
            }
            node = next;
        }
        return cnt[node];
    }
};
```

```

}

bool erase(U x) {
    if (count(x) == 0) {
        return false;
    }
    int node = 0;
    cnt[node]--;
    for (int b = BITS - 1; b >= 0; b--) {
        node = child[node][(x >> b) & 1];
        cnt[node]--;
    }
    return true;
}

U max_xor(U x) const {
    assert(!empty());
    assert_fits(x);
    int node = 0;
    U res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int bit = (x >> b) & 1;
        int opp = child[node][bit ^ 1];
        if (opp != 0 && cnt[opp] > 0) {
            res |= U{1} << b;
            node = opp;
        } else {
            node = child[node][bit];
        }
    }
    return res;
}

U min_xor(U x) const {
    assert(!empty());
    assert_fits(x);
    int node = 0;
    U res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int bit = (x >> b) & 1;
        int same = child[node][bit];
        if (same != 0 && cnt[same] > 0) {
            node = same;
        } else {
            res |= U{1} << b;
            node = child[node][bit ^ 1];
        }
    }
    return res;
}

int count_xor_less(U x, U k) const {
    assert_fits(x);
    if ((k >> BITS) != 0) {
        return cnt[0]; // Every XOR result is below 2^BITS, which is at most k.
    }
    int node = 0, res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int xb = (x >> b) & 1, kb = (k >> b) & 1;
        if (kb == 1) {
            // Stored values matching x at bit b give XOR bit 0 here, hence are already below k.
            int below = child[node][xb];
            if (below != 0) {
                res += cnt[below];
            }
            node = child[node][xb ^ 1]; // Stay tied with k by taking XOR bit 1.
        } else {
            node = child[node][xb]; // XOR bit must be 0 to remain below k.
        }
    }
}

```

```

    }
    if (node == 0) {
        break;
    }
}
return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    BinaryTrie<> trie;
    assert(trie.empty());
    for (int x : {3, 10, 5, 25, 2}) {
        trie.insert(x);
    }
    assert(!trie.empty());
    assert(trie.size() == 5);

    // x XOR y over the stored y: {26, 19, 28, 0, 27} for x = 25.
    assert(trie.max_xor(25) == 28);           // 25 XOR 5.
    assert(trie.min_xor(25) == 0);           // 25 XOR 25.
    assert(trie.count_xor_less(25, 20) == 2); // XOR values 0 and 19 are below 20.
    assert(trie.count_xor_less(25, 1U << 30) == 5); // Bound at/above 2^BITS counts all.

    assert(trie.count(10) == 1);
    trie.insert(10);
    assert(trie.count(10) == 2); // Multiset: two copies of 10.
    assert(trie.erase(10) && trie.count(10) == 1);
    assert(trie.erase(25) && trie.count(25) == 0);
    assert(!trie.erase(25));      // Already gone.
    assert(trie.max_xor(0) == 10); // Largest remaining value is 10.
    return 0;
}

```

### 1.7.7 Bitwise Convolution (FWHT)

Bitwise convolution combines two arrays indexed by masks, grouping pairs of masks by a bitwise operation instead of by ordinary addition. For arrays `a` and `b` over an  $n$ -bit universe, the XOR convolution `c[m]` is the sum of `a[x]*b[y]` over all `x ^ y = m`. OR and AND convolution replace `x ^ y` with `x | y` or `x & y`. These appear in subset DP, counting pairs by bitwise result, and algebra over the Boolean hypercube.

The fast Walsh-Hadamard transform (FWHT) diagonalizes XOR convolution in the same way that the FFT diagonalizes polynomial convolution: transform both arrays, multiply pointwise, and transform back. The OR and AND versions are Zeta/Mobius transforms over the subset lattice, but they have the same butterfly-shaped implementation and the same  $O(n \log n)$  runtime for arrays of length  $n = 2^k$ .

- `xor_transform(f, invert)` applies the XOR FWHT to `f` in place. The inverse transform uses `invert = true` and divides every coefficient by `f.size()`.
- `or_transform(f, invert)` applies the OR Zeta transform in place; the inverse is Mobius inversion.
- `and_transform(f, invert)` applies the AND Zeta transform in place; the inverse is Mobius inversion.
- `xor_convolve(a, b)` returns the XOR convolution of `a` and `b`.
- `or_convolve(a, b)` returns the OR convolution of `a` and `b`.
- `and_convolve(a, b)` returns the AND convolution of `a` and `b`.

Subset convolution restricts OR convolution to pairs of disjoint masks, so  $c[m]$  must count only  $x \mid y = m$  with  $x \& y = 0$ . Rank each entry by its number of set bits, transform every rank layer, and multiply the layers as polynomials in the rank. Mobius inversion then recovers pairs whose union is exactly  $m$ ; among these pairs, the ranks sum to the number of set bits in  $m$  exactly when the pair is disjoint.

- `subset_convolve(a, b)` returns the subset convolution of `a` and `b`, where  $c[m]$  sums  $a[x]*b[y]$  over the pairs of disjoint masks satisfying  $x \mid y = m$ .

The in-place transforms require a nonempty array whose length is a power of two. A convolution returns empty if either input is empty; otherwise, both inputs are padded with zeros to the smallest power of two at least their maximum length, which is also the output length. For exact integer XOR convolution, the inverse divisions are exact; for modular arithmetic, replace the division by multiplication with the modular inverse of the transform length.

Overflow warning: All intermediate sums, differences, and products must be representable in the value type.

### Time Complexity:

- $O(n \log n)$  per transform or bitwise convolution, where  $n$  is the padded power-of-two length.
- $O(n \log^2 n)$  per call to `subset_convolve()`.

### Space Complexity:

- $O(1)$  auxiliary for the in-place transforms.
- $O(n)$  auxiliary and  $O(n)$  for the returned array from each bitwise convolution.
- $O(n \log n)$  auxiliary for `subset_convolve()`, from the rank layers.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

template<typename T>
void xor_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert(n > 0 && (n & (n - 1)) == 0);
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += len <= 1) {
            for (int j = 0; j < len; j++) {
                T u = f[i + j], v = f[i + j + len];
                f[i + j] = u + v; // Overflow warning.
                f[i + j + len] = u - v;
            }
        }
    }
    if (invert) {
        for (T &x : f) {
            x /= n;
        }
    }
}

template<typename T>
void or_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert(n > 0 && (n & (n - 1)) == 0);
    for (int bit = 1; bit < n; bit <= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & bit) {
                if (invert) {
                    f[mask] -= f[mask ^ bit]; // Overflow warning.
                } else {
                    f[mask] += f[mask ^ bit];
                }
            }
        }
    }
}
```



```

    }
}

template<typename T>
void and_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert(n > 0 && (n & (n - 1)) == 0);
    for (int bit = 1; bit < n; bit <= 1) {
        for (int mask = 0; mask < n; mask++) {
            if ((mask & bit) == 0) {
                if (invert) {
                    f[mask] -= f[mask ^ bit]; // Overflow warning.
                } else {
                    f[mask] += f[mask ^ bit];
                }
            }
        }
    }
}

template<typename T, typename Transform>
std::vector<T> bitwise_convolve(std::vector<T> a, std::vector<T> b, Transform transform) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int needed = static_cast<int>(std::max(a.size(), b.size()));
    int n = 1;
    while (n < needed) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    transform(a, false);
    transform(b, false);
    for (int i = 0; i < n; i++) {
        a[i] *= b[i]; // Overflow warning.
    }
    transform(a, true);
    return a;
}

template<typename T>
std::vector<T> xor_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(std::move(a), std::move(b), xor_transform<T>);
}

template<typename T>
std::vector<T> or_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(std::move(a), std::move(b), or_transform<T>);
}

template<typename T>
std::vector<T> and_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(std::move(a), std::move(b), and_transform<T>);
}

template<typename T>
std::vector<T> subset_convolve(std::vector<T> a, std::vector<T> b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int needed = static_cast<int>(std::max(a.size(), b.size()));
    int n = 1, bits = 0;
    while (n < needed) {
        n <= 1;
        bits++;
    }
}

```

```

a.resize(n);
b.resize(n);
std::vector<int> ranks(n);
for (int mask = 1; mask < n; mask++) {
    ranks[mask] = ranks[mask >> 1] + (mask & 1);
}
// Split each input into layers by rank, so that layer k holds only the masks with k set bits.
std::vector<std::vector<T>> fa(bits + 1, std::vector<T>(n)), fb = fa, fc = fa;
for (int mask = 0; mask < n; mask++) {
    fa[ranks[mask]][mask] = a[mask];
    fb[ranks[mask]][mask] = b[mask];
}
for (int k = 0; k <= bits; k++) {
    or_transform(fa[k], false);
    or_transform(fb[k], false);
}
// Multiplying layers groups pairs by total rank. Once Mobius inversion recovers each pair's exact
// union, total rank ranks[mask] selects precisely the disjoint pairs covering mask.
for (int i = 0; i <= bits; i++) {
    for (int j = 0; i + j <= bits; j++) {
        for (int mask = 0; mask < n; mask++) {
            fc[i + j][mask] += fa[i][mask] * fb[j][mask]; // Overflow warning.
        }
    }
}
std::vector<T> res(n);
for (int k = 0; k <= bits; k++) {
    or_transform(fc[k], true);
}
for (int mask = 0; mask < n; mask++) {
    res[mask] = fc[ranks[mask]][mask];
}
return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        vector<int64_t> f{1, 2, 3, 4}, original(f);
        xor_transform(f, false);
        xor_transform(f, true);
        assert(f == original);
    }
    {
        vector<int64_t> f{1, 2, 3, 4}, original(f);
        or_transform(f, false);
        or_transform(f, true);
        assert(f == original);
    }
    {
        vector<int64_t> f{1, 2, 3, 4}, original(f);
        and_transform(f, false);
        and_transform(f, true);
        assert(f == original);
    }
}

vector<int64_t> a{1, 2, 3, 4}, b{5, 6, 7, 8};

// XOR: c[m] sums a[x] * b[y] over pairs with x ^ y == m.
assert((xor_convolve(a, b) == vector<int64_t>{70, 68, 62, 60}));

// OR: c[m] sums a[x] * b[y] over pairs with x | y == m.

```

```

assert((or_convolve(a, b) == vector<int64_t>{5, 28, 43, 184}));

// AND: c[m] sums a[x] * b[y] over pairs with x & y == m.
assert((and_convolve(a, b) == vector<int64_t>{103, 52, 73, 32}));

// Subset: c[m] sums a[x] * b[y] over disjoint pairs with x | y == m. Every entry is at most the
// corresponding OR convolution, which also counts the overlapping pairs.
assert((subset_convolve(a, b) == vector<int64_t>{5, 16, 22, 60}));

vector<int64_t> one_mask{0, 1, 0, 0};
vector<int64_t> two_mask{0, 0, 1};
assert((xor_convolve(one_mask, two_mask) == vector<int64_t>{0, 0, 0, 1}));
assert((or_convolve(one_mask, two_mask) == vector<int64_t>{0, 0, 0, 1}));
assert((and_convolve(one_mask, two_mask) == vector<int64_t>{1, 0, 0, 0}));
assert((subset_convolve(one_mask, two_mask) == vector<int64_t>{0, 0, 0, 1}));

// Overlapping masks contribute nothing at all to a subset convolution.
assert((subset_convolve(one_mask, one_mask) == vector<int64_t>{0, 0, 0, 0}));
return 0;
}

```

## 1.8 Classic Problems

---

### 1.8.1 Minimum Excluded Value

Given a multiset of integers, find the minimum excluded nonnegative value (MEX), i.e. the smallest integer  $x \geq 0$  not present in the multiset. MEX often appears in array problems, game theory with Grundy numbers, and dynamic programming states. Since the MEX of  $n$  values never exceeds  $n$ , the one-shot version tracks seen values in  $[0, n]$  before scanning to find the first missing.

- `mex(lo, hi)` returns the MEX of the values in  $[lo, hi)$ .

The MEX of a dynamic multiset can be computed using counts of present values and maintaining an ordered set of currently missing candidates.

- `DynamicMex()` maintains the MEX of a multiset of nonnegative integers.
- `add(x)` inserts one copy of value `x` if `x`  $\geq 0$ .
- `remove(x)` removes one copy of value `x` if present.
- `mex()` returns the current MEX.

**Time Complexity:**

- $O(n)$  per call to `mex(lo, hi)`, where  $n$  is the number of values.
- $O(\log n)$  expected per call to `add(x)` and `remove(x)` for `DynamicMex`, and  $O(n)$  in the collision-heavy worst case for the hash table.
- $O(1)$  per call to `mex()` for `DynamicMex`.

**Space Complexity:**

- $O(n)$  auxiliary for `mex(lo, hi)`.
- $O(n)$  storage for `DynamicMex`.

```

#include <iterator>
#include <set>
#include <unordered_map>
#include <vector>

template<typename It>
int mex(It lo, It hi) {
    int n = std::distance(lo, hi);
    std::vector<char> seen(n + 1);

```

```

    for (It it = lo; it != hi; ++it) {
        if (0 <= *it && *it <= n) {
            seen[*it] = true;
        }
    }
    for (int x = 0; x <= n; x++) {
        if (!seen[x]) {
            return x;
        }
    }
    return n + 1;
}

class DynamicMex {
    std::unordered_map<int, int> count;
    std::set<int> missing; // Missing candidates in [0, size].
    int size = 0;

public:
    DynamicMex() { missing.insert(0); }

    void add(int x) {
        if (x < 0) {
            return;
        }
        if (!count.count(size + 1)) {
            missing.insert(size + 1);
        }
        if (count[x]++ == 0) {
            missing.erase(x);
        }
        size++;
    }

    void remove(int x) {
        auto it = count.find(x);
        if (it == count.end()) {
            return;
        }
        bool removed_last = --it->second == 0;
        if (removed_last) {
            count.erase(it);
        }
        size--;
        missing.erase(size + 1);
        if (removed_last && x <= size) {
            missing.insert(x);
        }
    }

    int mex() const { return *missing.begin(); }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{0, 1, 4, 2, 1};
    assert(mex(a.begin(), a.end()) == 3);
    vector<int> b{-1, 0, 1, 2};
    assert(mex(b.begin(), b.end()) == 3); // Negative values are ignored.

    DynamicMex m;
    m.add(0);
}

```

```

m.add(1);
m.add(1);
m.add(3);
assert(m.mex() == 2);
m.add(2);
assert(m.mex() == 4);
m.remove(1);
assert(m.mex() == 4); // One copy of 1 remains.
m.remove(1);
assert(m.mex() == 1);
m.remove(100);
assert(m.mex() == 1);
return 0;
}

```

### 1.8.2 Josephus Problem

The Josephus problem arranges  $n$  people, numbered  $[0, n)$ , in a circle and repeatedly removes every  $k$ -th remaining person until one survives. Counting starts at person 0, which counts as the first person. After the first removal, relabeling the smaller circle from the next person gives the same problem on  $n - 1$  people. Mapping its survivor back to the original labels yields the recurrence  $J(1, k) = 0$  and  $J(n, k) = (J(n - 1, k) + k) \bmod n$ .

When  $k$  is small, several removals can be processed at once. One complete pass removes  $\lfloor n/k \rfloor$  people; recursively solving the compressed circle and undoing the resulting index shift reduces the running time to  $O(k \log n)$ . To recover the full elimination order, a Fenwick tree stores which labels remain and selects each next victim by rank.

- `josephus(n, k)` returns the 0-based label of the survivor, where `n` and `k` must be positive.
- `josephus_small_k(n, k)` returns the same survivor using the batched recurrence. It is preferable when  $k$  is small relative to  $n$ .
- `josephus_order(n, k)` returns all labels in elimination order, ending with the survivor.

**Time Complexity:**

- $O(n)$  per call to `josephus()`.
- $O(\min(n, k \log n))$  per call to `josephus_small_k()`.
- $O(n \log n)$  per call to `josephus_order()`.

**Space Complexity:**

- $O(1)$  auxiliary for `josephus()`.
- $O(\min(n, k \log n))$  call stack space for `josephus_small_k()`.
- $O(n)$  auxiliary and  $O(n)$  for the returned elimination order from `josephus_order()`.

```

#include <cassert>
#include <cstdint>
#include <vector>

int josephus(int n, int k) {
    assert(n > 0 && k > 0);
    int survivor = 0;
    for (int size = 2; size <= n; size++) {
        survivor = static_cast<int>((static_cast<int64_t>(survivor) + k) % size);
    }
    return survivor;
}

int josephus_small_k(int n, int k) {
    assert(n > 0 && k > 0);
    if (n == 1) {
        return 0;
    }
    if (k == 1) {

```

```

    return n - 1;
}
if (k > n) {
    return josephus(n, k);
}
int removed = n / k;
int survivor = josephus_small_k(n - removed, k);
survivor -= n % k;
if (survivor < 0) {
    survivor += n;
} else {
    survivor += survivor / (k - 1);
}
return survivor;
}

std::vector<int> josephus_order(int n, int k) {
    assert(n > 0 && k > 0);
    std::vector<int> bit(n + 1, order);
    order.reserve(n);
    for (int i = 1; i <= n; i++) {
        bit[i] = i & -i; // Initially every label is alive.
    }
    int max_step = 1;
    while (max_step <= n / 2) {
        max_step *= 2;
    }
    auto find_by_rank = [&](int rank) {
        int pos = 0;
        for (int step = max_step; step > 0; step /= 2) {
            int next = pos + step;
            if (next <= n && bit[next] <= rank) {
                pos = next;
                rank -= bit[next];
            }
        }
        return pos;
    };
    int rank = 0;
    for (int remaining = n; remaining > 0; remaining--) {
        rank = static_cast<int>(((static_cast<int64_t>(rank) + k - 1) % remaining));
        int removed = find_by_rank(rank);
        order.push_back(removed);
        for (int i = removed + 1; i <= n; i += i & -i) {
            bit[i]--;
        }
        if (remaining > 1) {
            rank %= remaining - 1;
        }
    }
    return order;
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    assert(josephus(1, 5) == 0);
    assert(josephus(7, 3) == 3);
    assert(josephus(10, 1) == 9);
    for (int n = 1; n <= 30; n++) {
        for (int k = 1; k <= 30; k++) {
            assert(josephus_small_k(n, k) == josephus(n, k));
        }
    }
}

```

```

    assert(josephus_order(n, k).back() == josephus(n, k));
}
}
assert(josephus_order(7, 3) == vector<int>({2, 5, 1, 6, 4, 0, 3}));
return 0;
}

```

### 1.8.3 Tower of Hanoi

Move a stack of  $n$  disks of distinct sizes from one peg to another using a third as scratch space, moving one disk at a time and never placing a larger disk on a smaller one. The recursive solution is forced by the largest disk: it can only move once the other  $n - 1$  disks are stacked on the spare peg, so the puzzle is solved by moving  $n - 1$  disks aside, moving the largest disk across, and moving the  $n - 1$  disks back on top of it. That gives  $T(n) = 2T(n - 1) + 1$  with  $T(0) = 0$ , so exactly  $2^n - 1$  moves are needed, and no shorter solution exists because the largest disk cannot move any earlier.

The sequence also has a closed form, for when one move is wanted rather than all of them: numbering moves from 1, move  $k$  transfers the disk given by the number of trailing zero bits of  $k$ , and that disk always steps in one rotational direction fixed by the parity of  $n$ . The smallest disk therefore moves every other turn, which is the rule an iterative solver follows.

- `hanoi_moves(n, from = 0, to = 2)` returns the moves solving the puzzle for `n` disks, as pairs (`from_peg`, `to_peg`). Pegs are numbered `[0, 3)` and the spare peg is whichever of the three is neither `from` nor `to`. The number of disks must be nonnegative.
- `hanoi_move_count(n)` returns  $2^n - 1$ , the number of moves, without generating them. It requires  $n \leq 63$ , the largest count that fits in `int64_t`.

With four or more pegs the problem becomes Frame-Stewart: move a prefix aside using every peg, transfer the rest with one peg fewer, bring the prefix back, and minimize over the split. That this family of strategies is optimal was only proved in 2014.

#### Time Complexity:

- $O(2^n)$  per call to `hanoi_moves()`, which is optimal since that is the size of the output.
- $O(1)$  per call to `hanoi_move_count()`.

#### Space Complexity:

- $O(n)$  auxiliary stack space and  $O(2^n)$  for the returned moves from `hanoi_moves()`.
- $O(1)$  auxiliary for `hanoi_move_count()`.

```

#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

std::vector<std::pair<int, int>> hanoi_moves(int n, int from = 0, int to = 2) {
    assert(n >= 0 && from >= 0 && from < 3 && to >= 0 && to < 3 && from != to);
    std::vector<std::pair<int, int>> moves;
    auto rec = [&](auto &&rec, int disks, int src, int dest) {
        if (disks == 0) {
            return;
        }
        int spare = 3 - src - dest;
        rec(rec, disks - 1, src, spare);
        moves.emplace_back(src, dest);
        rec(rec, disks - 1, spare, dest);
    };
    rec(rec, n, from, to);
    return moves;
}

```

```
int64_t hanoi_move_count(int n) {
    assert(n >= 0 && n <= 63);
    return static_cast<int64_t>((uint64_t{1} << n) - 1); // Unsigned, so n = 63 does not overflow.
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(hanoi_moves(0).empty());
    assert((hanoi_moves(1) == vector<pair<int, int>>{{0, 2}}));
    assert((hanoi_moves(2) == vector<pair<int, int>>{{0, 1}, {0, 2}, {1, 2}}));
    assert(hanoi_move_count(10) == 1023);

    // Replay the moves of a larger puzzle to confirm no disk ever lands on a smaller one.
    const int n = 12;
    vector<vector<int>> pgs(3);
    for (int disk = n; disk >= 1; disk--) {
        pgs[0].push_back(disk);
    }
    auto moves = hanoi_moves(n);
    assert(static_cast<int64_t>(moves.size()) == hanoi_move_count(n));
    for (auto [src, dest] : moves) {
        assert(!pgs[src].empty());
        assert(pgs[dest].empty() || pgs[dest].back() > pgs[src].back());
        pgs[dest].push_back(pgs[src].back());
        pgs[src].pop_back();
    }
    assert(pgs[0].empty() && pgs[1].empty() && static_cast<int>(pgs[2].size()) == n);
    return 0;
}
```

### 1.8.4 Cycle Detection (Floyd)

Given a function  $f$  mapping a finite set to itself and a starting value  $x_0$ , return the entry value, position, and length of the cycle reached by repeatedly applying  $f$ . Formally, the sequence  $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots, x_n = f(x_{n-1}), \dots$  must eventually repeat some value. If  $x_i = x_j$  for  $i < j$ , then the sequence from  $x_i$  to  $x_{j-1}$  repeats forever. This is useful for detecting cycles in functional graphs (see 4.2.9), pseudo-random generators, Pollard rho style algorithms, degenerate linked lists, and arrays where each index points to the next index. For example, if an array  $a_0, \dots, a_{n+1}$  contains only values in  $[1, n]$ , with exactly one value occurring more than once, then  $f(i) = a_i$  forms a functional graph; the duplicate value is the entry point of the cycle reached from 0.

Floyd’s cycle-finding algorithm, a.k.a. the “tortoise and the hare algorithm”, is a space-efficient default choice: it moves two pointers through the sequence at different speeds, finds a meeting point inside the cycle, then locates the cycle start. Brent’s algorithm is a compact variant that usually calls  $f$  fewer times by only resetting the tortoise at powers of two; prefer it when evaluating  $f$  is expensive and constant factors matter. In the detection phase, Brent advances one pointer with one call to  $f$  per step, while Floyd advances the tortoise once and the hare twice, using three calls to  $f$  per step.

For Floyd’s algorithm, suppose the tail has length  $m$  and the cycle has length  $n$ . At a meeting, the hare has traveled twice as far as the tortoise, so the tortoise’s distance is a multiple of  $n$ . Consequently, the distance from the meeting point around the cycle to its entry is congruent to  $m$  modulo  $n$ . Resetting one pointer to  $x_0$  and advancing both one step at a time makes them meet at the cycle entry after  $m$  steps. Brent first determines  $n$ , then advances one pointer  $n$  steps; moving both together from that fixed separation likewise makes them meet at the entry.



- `find_cycle_floyd(f, x0)` repeatedly applies `f` starting from `x0` and returns the reached cycle as a tuple `(entry, start, length)`, where `entry` is the first cycle value, `start` is the number of applications of `f` needed to reach it from `x0`, and `length` is the number of distinct values in the cycle.
- `find_cycle_brent(f, x0)` does the same with fewer calls to `f` in many cases.

**Time Complexity:**  $O(m + n)$  per call, where  $m$  is the first index in the cycle and  $n$  is the cycle length.

**Space Complexity:**  $O(1)$  auxiliary.

```
#include <tuple>

template<typename Fn, typename T>
std::tuple<T, int, int> find_cycle_floyd(Fn f, T x0) {
    // Find a meeting point inside the cycle.
    T tortoise = f(x0), hare = f(f(x0));
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(f(hare));
    }
    // Find the cycle entry.
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(hare);
        start++;
    }
    // Measure the cycle length.
    int length = 1;
    hare = f(tortoise);
    while (tortoise != hare) {
        hare = f(hare);
        length++;
    }
    return {tortoise, start, length};
}

template<typename Fn, typename T>
std::tuple<T, int, int> find_cycle_brent(Fn f, T x0) {
    // Find the cycle length by doubling the search block.
    int power = 1, length = 1;
    T tortoise = x0, hare = f(x0);
    while (tortoise != hare) {
        if (power == length) {
            tortoise = hare;
            power *= 2;
            length = 0;
        }
        hare = f(hare);
        length++;
    }
    // Separate the pointers by one cycle length.
    hare = x0;
    for (int i = 0; i < length; i++) {
        hare = f(hare);
    }
    // Find the cycle entry.
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(hare);
        start++;
    }
    return {tortoise, start, length};
}
```

## Example Usage

```

#include <cassert>
#include <set>
#include <vector>
using namespace std;

int f(int x) {
    return (123 * x * x + 4567890) % 1337;
}

void verify(int x0, int start, int length) {
    set<int> s;
    int x = x0;
    for (int i = 0; i < start; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    int startx = x;
    s.clear();
    for (int i = 0; i < length; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    assert(startx == x);
}

// Given n + 1 integers within [1, n], where one value in [1, n] is present 2 or more times (but
// other values in [1, n] are present at most once), find the duplicate without modifying the array.
// Interpret each value as a pointer to the next index; the duplicate is the cycle entry.
int find_duplicate_integer(const vector<int> &a) {
    return get<0>(find_cycle_floyd([&](int i) { return a[i]; }, 0));
}

int main() {
    int x0 = 0;
    auto [floyd_entry, floyd_start, floyd_len] = find_cycle_floyd(f, x0);
    assert(floyd_start == 4 && floyd_len == 2);
    verify(x0, floyd_start, floyd_len);

    auto [brent_entry, brent_start, brent_len] = find_cycle_brent(f, x0);
    assert(brent_entry == floyd_entry && brent_start == floyd_start && brent_len == floyd_len);

    assert(find_duplicate_integer({1, 3, 4, 2, 2}) == 2);
    assert(find_duplicate_integer({3, 1, 3, 4, 2}) == 3);
    return 0;
}

```

### 1.8.5 Subset Sum

Given a collection of integers and a target, the subset sum problem asks for the maximum sum of any subset that does not exceed the target. Two algorithms below solve this problem in complementary regimes, and which one wins depends on whether the bottleneck is the number of items or the size of the weights.

The meet-in-the-middle method splits the collection in half, enumerates every subset sum for each half, sorts one side, and binary searches for the best compatible partner. It makes no assumption about the sign or size of the values, so it handles negative inputs, and it is the method of choice when the number of items  $n$  is small (up to roughly 40).

Pisinger's algorithm targets the opposite regime: nonnegative weights whose maximum value is small, with possibly many items. Let  $W$  be the largest weight. It greedily takes a maximal prefix with sum  $s$  that fits the

target; because the next weight is at most  $W$  and does not fit,  $s$  is greater than `target`  $- W$ . The prefix itself is feasible, so the optimum lies between  $s$  and `target` and only that width- $W$  window matters. A balanced sweep explores exchanges that add later items to sums below the target and remove earlier items from sums above it. For each sum, it retains the farthest prefix boundary reached, which dominates smaller boundaries because it permits at least the same future exchanges. This gives a running time linear in the number of items times  $W$ , independent of the target beyond that window. This routine is sometimes called “fast knapsack,” a slight misnomer: it maximizes a subset sum rather than packing items with distinct values, so it is really the bounded-weight case of subset sum rather than the 0-1 value knapsack.

- `max_subset_sum_at_most(values, target)` returns a pair (`sum`, `items`) containing that maximum sum and the selected item indices. Values may be negative, and `target` must be nonnegative.
- `max_subset_sum_bounded(weights, target)` returns the maximum sum of any subset of `weights` that does not exceed `target`. All weights and `target` must be nonnegative integers. This is faster than meet-in-the-middle when the largest weight is small relative to  $2^{n/2}$ .

Overflow warning: All subset sums and intermediate differences in `max_subset_sum_at_most()` must fit in `int64_t`.

#### Time Complexity:

- $O(n \cdot 2^{n/2})$  per call to `max_subset_sum_at_most()`, where  $n$  is the number of values.
- $O(n \cdot W)$  per call to `max_subset_sum_bounded()`, where  $n$  is the number of items and  $W$  is the largest weight.

#### Space Complexity:

- $O(2^{n/2})$  auxiliary for `max_subset_sum_at_most()`.
- $O(W)$  auxiliary for `max_subset_sum_bounded()`.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>> max_subset_sum_at_most(
    const std::vector<int> &values, int64_t target
) {
    assert(target >= 0);
    int n = static_cast<int>(values.size());
    assert(n / 2 < 63 && n - n / 2 < 63);
    int64_t llen = 1LL << (n / 2), rlen = 1LL << (n - n / 2);
    std::vector<int64_t> lsum(llen);
    std::vector<std::pair<int64_t, int64_t>> rsum(rlen);
    for (int64_t mask = 1; mask < llen; mask++) {
        int bit = __builtin_ctzll(mask);
        lsum[mask] = lsum[mask ^ (1LL << bit)] + values[bit]; // Overflow warning.
    }
    for (int64_t mask = 1; mask < rlen; mask++){
        int bit = __builtin_ctzll(mask);
        rsum[mask].first = rsum[mask ^ (1LL << bit)].first + values[n / 2 + bit];
        rsum[mask].second = mask;
    }
    std::sort(rsum.begin(), rsum.end());
    int64_t best = INT64_MIN, lmask = 0, rmask = 0;
    for (int64_t mask = 0; mask < llen; mask++) {
        int64_t limit = target - lsum[mask]; // Overflow warning.
        auto it = std::upper_bound(rsum.begin(), rsum.end(), std::make_pair(limit, INT64_MAX));
        if (it != rsum.begin()) {
            --it;
            int64_t candidate = lsum[mask] + it->first;
            if (best < candidate) {
                best = candidate;
                lmask = mask;
                rmask = it->second;
            }
        }
    }
}
```

```

    }
    // Optional: reconstruct one optimal subset.
    std::vector<int> items;
    for (int i = 0; i < n / 2; i++) {
        if ((lmask >> i) & 1) {
            items.push_back(i);
        }
    }
    for (int i = 0; i < n - n / 2; i++) {
        if ((rmask >> i) & 1) {
            items.push_back(n / 2 + i);
        }
    }
    return {best, items};
}

int max_subset_sum_bounded(const std::vector<int> &weights, int target) {
    assert(target >= 0);
    for (int w : weights) {
        assert(w >= 0);
    }
    int n = static_cast<int>(weights.size());
    // Greedily take a prefix while it still fits; the optimum then lies within one weight of target.
    int sum = 0, b = 0;
    while (b < n && weights[b] <= target - sum) {
        sum += weights[b++];
    }
    if (b == n) {
        return sum; // Every item fits, so the whole collection is the best subset.
    }
    int m = 0;
    for (int w : weights) {
        m = std::max(m, w);
    }
    // reach[s - target + m] = largest prefix boundary that can realize a balanced subset of sum s.
    // Only sums within m of target are tracked, which is where the optimum must lie.
    std::vector<int> reach(2 * m, -1);
    reach[sum - target + m] = b;
    for (int i = b; i < n; i++) {
        std::vector<int> prev = reach;
        for (int x = 0; x < m; x++) { // Add item i to sums currently below target.
            reach[x + weights[i]] = std::max(reach[x + weights[i]], prev[x]);
        }
        for (int x = 2 * m - 1; x > m; x--) { // Drop an earlier item from sums above target.
            for (int j = std::max(0, prev[x]); j < reach[x]; j++) {
                reach[x - weights[j]] = std::max(reach[x - weights[j]], j);
            }
        }
    }
    int s = target;
    while (reach[s - target + m] < 0) {
        s--;
    }
    return s;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{9, 1, 5, 0, 1, 11, 5};
    auto [sum, items] = max_subset_sum_at_most(a, 8);
    int64_t selected_sum = 0;
    for (int i : items) {

```

```

    selected_sum += a[i];
}
assert(sum == 7 && selected_sum == sum);
vector<int> b{-7, -3, -2, 5, 8};
assert(max_subset_sum_at_most(b, 0).first == 0);

// The bounded-weight method agrees with meet-in-the-middle on nonnegative inputs.
assert(max_subset_sum_bounded(a, 8) == 7);
vector<int> c{3, 34, 4, 12, 5, 2};
assert(max_subset_sum_bounded(c, 9) == 9);    // 4 + 5.
assert(max_subset_sum_bounded(c, 10) == 10);   // 3 + 5 + 2.
assert(max_subset_sum_bounded(c, 100) == 60);  // All items fit.
return 0;
}

```

### 1.8.6 N-Queens

The N-queens problem asks for a placement of  $n$  queens on an  $n$ -by- $n$  chessboard such that no two share a row, column, or diagonal. Backtracking places one queen in each successive row and tracks which columns and diagonals are occupied. A conflicting placement is rejected immediately; after a failed recursive branch, its column and diagonals are released before trying the next column.

- `n_queens(n)` returns a placement as a vector `cols` of length `n`, where `cols[row]` is the queen's column in that row. It returns an empty vector if no placement exists.

**Time Complexity:**  $O(n!)$  per call.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned placement.

```

#include <cassert>
#include <vector>

std::vector<int> n_queens(int n) {
    assert(n > 0);
    std::vector<int> cols(n);
    std::vector<char> used_col(n), used_diag1(2 * n - 1), used_diag2(2 * n - 1);
    auto rec = [&](auto &&rec, int row) -> bool {
        if (row == n) {
            return true;
        }
        for (int col = 0; col < n; col++) {
            int diag1 = row - col + n - 1, diag2 = row + col;
            if (used_col[col] || used_diag1[diag1] || used_diag2[diag2]) {
                continue;
            }
            cols[row] = col;
            used_col[col] = used_diag1[diag1] = used_diag2[diag2] = true;
            if (rec(rec, row + 1)) {
                return true;
            }
            used_col[col] = used_diag1[diag1] = used_diag2[diag2] = false;
            cols[row] = -1;
        }
        return false;
    };
    return rec(rec, 0) ? cols : std::vector<int>{};
}

```

#### Example Usage

```

#include <cassert>
#include <vector>

```

```
using namespace std;

int main() {
    assert(n_queens(4) == vector<int>({1, 3, 0, 2}));
    assert(n_queens(2).empty());
    assert(n_queens(1) == vector<int>({0}));
    return 0;
}
```

## 1.8.7 Knights Tour

A knight's tour visits every square of an  $n$ -by- $n$  board exactly once, moving as a chess knight. Plain backtracking over the eight moves is hopeless past a small board, since the search tree has branching factor up to eight and depth  $n^2$ .

Warnsdorff's rule fixes the move order: from each square, try the onward squares in increasing order of how many unvisited moves they themselves have. The intuition is that squares with few remaining exits, such as corners, become unreachable if they are not used early, so the rule visits the most constrained squares first and leaves the flexible ones for later. This is the same minimum-remaining-values idea that drives the exact cover search of section 1.8.8.

Warnsdorff's rule alone is a heuristic and does fail on some boards, so the search below keeps backtracking as a fallback and merely orders the moves by the rule. That ordering is strong enough that a tour is normally found along the first path tried, with backtracking almost never invoked.

- `knight_tour(n, sr = 0, sc = 0)` returns an  $n$ -by- $n$  grid whose entry is the step at which the knight visits that square, numbered  $[0, n^2)$ , starting from square  $(sr, sc)$ . It returns an empty grid when no tour exists, which is the case for  $n \in \{2, 3, 4\}$ . The board size must be positive and the starting square must lie on the board.

A tour is closed when the knight can leap from its final square back to its start, which is possible only for even  $n$  at least 6. Requiring one means rejecting a completed tour whose last square is not a knight's move from the first, so the same search finds it by adding that test at the end.

**Time Complexity:**  $O(n^2)$  per call in practice, since the move ordering finds a tour without backtracking on the boards where one exists. The worst case remains exponential.

**Space Complexity:**  $O(n^2)$  auxiliary stack space and  $O(n^2)$  for the returned grid.

```
#include <algorithm>
#include <array>
#include <cassert>
#include <utility>
#include <vector>

const std::array<std::pair<int, int>, 8> KNIGHT_MOVES = {
    {{-2, -1}, {-2, 1}, {-1, -2}, {-1, 2}, {1, -2}, {1, 2}, {2, -1}, {2, 1}}
};

std::vector<std::vector<int>> knight_tour(int n, int sr = 0, int sc = 0) {
    assert(n > 0 && sr >= 0 && sr < n && sc >= 0 && sc < n);
    std::vector<std::vector<int>> visit(n, std::vector<int>(n, -1));
    auto onward_moves = [&](int r, int c) {
        int count = 0;
        for (auto [dr, dc] : KNIGHT_MOVES) {
            int nr = r + dr, nc = c + dc;
            if (nr >= 0 && nr < n && nc >= 0 && nc < n && visit[nr][nc] == -1) {
                count++;
            }
        }
    };
    return count;
};
```

```

auto rec = [&](auto &&rec, int r, int c, int step) -> bool {
    visit[r][c] = step;
    if (step == n * n - 1) {
        return true;
    }
    // Order the onward squares by how few exits they have left, which is Warnsdorff's rule.
    std::vector<std::pair<int, std::pair<int, int>>> candidates;
    for (auto [dr, dc] : KNIGHT_MOVES) {
        int r2 = r + dr, c2 = c + dc;
        if (r2 >= 0 && r2 < n && c2 >= 0 && c2 < n && visit[r2][c2] == -1) {
            candidates.emplace_back(onward_moves(r2, c2), std::make_pair(r2, c2));
        }
    }
    std::sort(candidates.begin(), candidates.end());
    for (auto [degree, square] : candidates) {
        if (rec(rec, square.first, square.second, step + 1)) {
            return true;
        }
    }
    visit[r][c] = -1;
    return false;
};
return rec(rec, sr, sc, 0) ? visit : std::vector<std::vector<int>>{};
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(knight_tour(1) == vector<vector<int>>{{0}});
    assert(knight_tour(3).empty()); // No tour exists on boards of size 2, 3, or 4.
    assert(knight_tour(4).empty());

    for (int n : {5, 6, 8}) {
        auto visit = knight_tour(n, n / 2, n / 2);
        assert(!visit.empty());
        vector<pair<int, int>> at(n * n);
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                assert(visit[r][c] >= 0 && visit[r][c] < n * n);
                at[visit[r][c]] = {r, c}; // Every step number is used exactly once.
            }
        }
        assert(at[0] == make_pair(n / 2, n / 2));
        for (int step = 1; step < n * n; step++) {
            int dr = abs(at[step].first - at[step - 1].first);
            int dc = abs(at[step].second - at[step - 1].second);
            assert(dr * dc == 2); // Each consecutive pair differs by a knight's move.
        }
    }
    return 0;
}

```

### 1.8.8 Exact Cover (Dancing Links)

Given a binary matrix, the exact cover problem asks for a set of rows in which every column contains exactly one 1. Many search problems reduce to it: a sudoku grid becomes a matrix whose columns demand that each cell hold one digit and that each digit appear once per row, column, and box, while a tiling becomes a matrix whose columns demand that each cell be covered by exactly one piece placement.

Knuth's Algorithm X solves it by repeatedly choosing an uncovered column, trying each row that covers it, and deleting from the matrix every column that row satisfies along with every other row conflicting with it. Dancing links is the representation that makes the deletions cheap. Each 1 of the matrix is a node in two circular doubly linked lists, one along its row and one along its column, and every column has a header node holding the count of its remaining nodes. Removing a node `x` from a list is `left[right[x]] = left[x]` and `right[left[x]] = right[x]`, which leaves `x` itself still pointing at its old neighbors; restoring it on backtrack is therefore just `left[right[x]] = x` and `right[left[x]] = x`, with no allocation and no search. Undoing the covering operations in the exact reverse of the order that performed them restores the matrix exactly, which is what makes the whole search run in place on one static set of nodes.

The column to branch on is the one with the fewest remaining rows. This is the minimum-remaining-values heuristic, and it fails as early as possible: a column with no remaining rows ends the branch immediately, and a column with one forces the choice.

Secondary columns spare a problem with optional requirements from padding its matrix with one slack row per optional column. In the N-queens reduction below, every board row and column must hold exactly one queen and so is primary, while a diagonal may hold none and so is secondary. Asking for two solutions is the usual test of uniqueness, as a sudoku generator does when validating a puzzle.

- `ExactCover(primary, secondary = 0)` constructs an empty matrix with `primary + secondary` columns, both counts nonnegative. Columns in `[0, primary)` must be covered exactly once, while the optional columns in `[primary, primary + secondary)` may be covered at most once.
- `add_row(cols)` appends a row containing a 1 in each column of `cols`, which must all be in `[0, primary + secondary)`, and returns the row's ID. The IDs of the first  $r$  rows added are  $[0, r)$ , in the order they were added.
- `solve(limit = 1)` returns up to `limit` solutions, where `limit` is positive. Each solution is the sorted list of the IDs of its rows, and an empty result means no exact cover exists. The matrix is left unchanged, so it may be queried again.

Solutions come in the order the search finds them, which follows the branching heuristic rather than row IDs.

### Time Complexity:

- $O(k)$  per call to `add_row(cols)`, where  $k$  is the size of `cols`.
- Exponential per call to `solve()`. Exact cover is NP-complete, and dancing links prunes the search rather than avoiding it: the cost is proportional to the number of nodes the search unlinks and relinks, which the branching heuristic reduces but does not bound polynomially.

### Space Complexity:

- $O(n + c)$  for storage, where  $n$  is the number of 1s in the matrix and  $c$  is the number of columns.
- $O(d)$  auxiliary stack space per call to `solve()`, where  $d$  is the number of rows in the deepest partial solution reached, plus  $O(s \cdot d)$  for the returned solutions, where  $s$  is the number found.

```
#include <algorithm>
#include <cassert>
#include <vector>

class ExactCover {
    int columns, rows;
    std::vector<int> left, right, up, down, col_of, row_of, col_size;
    std::vector<int> partial;

    int add_node(int c, int r) {
        left.push_back(0);
        right.push_back(0);
        up.push_back(0);
        down.push_back(0);
        col_of.push_back(c);
        row_of.push_back(r);
        return static_cast<int>(col_of.size()) - 1;
    }
}
```



```

// Unlinks column c and every row meeting it, so both vanish from all remaining traversals.
void cover(int c) {
    right[left[c]] = right[c];
    left[right[c]] = left[c];
    for (int i = down[c]; i != c; i = down[i]) {
        for (int j = right[i]; j != i; j = right[j]) {
            up[down[j]] = up[j];
            down[up[j]] = down[j];
            col_size[col_of[j]]--;
        }
    }
}

// Relinks in the mirror order of cover(), which is what restores the matrix exactly.
void uncover(int c) {
    for (int i = up[c]; i != c; i = up[i]) {
        for (int j = left[i]; j != i; j = left[j]) {
            col_size[col_of[j]]++;
            up[down[j]] = j;
            down[up[j]] = j;
        }
    }
    right[left[c]] = c;
    left[right[c]] = c;
}

bool search(int limit, std::vector<std::vector<int>> &found) {
    if (right[0] == 0) { // Every primary column is covered.
        found.push_back(partial);
        std::sort(found.back().begin(), found.back().end());
        return static_cast<int>(found.size()) >= limit;
    }
    int best = right[0];
    for (int c = right[0]; c != 0; c = right[c]) {
        if (col_size[c] < col_size[best]) {
            best = c;
        }
    }
    if (col_size[best] == 0) {
        return false; // No row can cover this column, so this branch is dead.
    }
    cover(best);
    for (int r = down[best]; r != best; r = down[r]) {
        partial.push_back(row_of[r]);
        for (int j = right[r]; j != r; j = right[j]) {
            cover(col_of[j]);
        }
        bool done = search(limit, found);
        for (int j = left[r]; j != r; j = left[j]) {
            uncover(col_of[j]);
        }
        partial.pop_back();
        if (done) {
            break;
        }
    }
    uncover(best);
    return static_cast<int>(found.size()) >= limit;
}

public:
explicit ExactCover(int primary, int secondary = 0) : columns(primary + secondary), rows(0) {
    assert(primary >= 0 && secondary >= 0);
    col_size.assign(columns + 1, 0);
    for (int c = 0; c <= columns; c++) { // Node 0 is the root, and node c + 1 heads column c.
        int header = add_node(c, -1);
        up[header] = down[header] = header;
    }
}

```

```

    left[header] = right[header] = header;
}
int prev = 0; // Secondary headers stay outside the root ring, so they are never branched on.
for (int c = 1; c <= primary; c++) {
    right[prev] = c;
    left[c] = prev;
    prev = c;
}
right[prev] = 0;
left[0] = prev;
}

int add_row(const std::vector<int> &cols) {
    for (int c : cols) {
        assert(c >= 0 && c < columns);
    }
    int row = rows++, first = -1;
    for (int c : cols) {
        int header = c + 1, node = add_node(header, row);
        up[node] = up[header];
        down[node] = header;
        down[up[header]] = node;
        up[header] = node;
        col_size[header]++;
        if (first == -1) {
            first = node;
            left[node] = right[node] = node;
        } else {
            left[node] = left[first];
            right[node] = first;
            right[left[first]] = node;
            left[first] = node;
        }
    }
    return row;
}

std::vector<std::vector<int>> solve(int limit = 1) {
    assert(limit > 0);
    std::vector<std::vector<int>> found;
    partial.clear();
    search(limit, found);
    return found;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Knuth's example matrix over 7 columns, whose unique cover is rows 1, 3, and 5.
    //   col: 0123456
    //   row 0: 1001001
    //   row 1: 1001000
    //   row 2: 0001101
    //   row 3: 0010110
    //   row 4: 0110011
    //   row 5: 0100001
    ExactCover matrix(7);
    matrix.add_row(vector<int>{0, 3, 6});
    matrix.add_row(vector<int>{0, 3});
    matrix.add_row(vector<int>{3, 4, 6});
    matrix.add_row(vector<int>{2, 4, 5});
    matrix.add_row(vector<int>{1, 2, 5, 6});
}

```

```

matrix.add_row(vector<int>{1, 6});
assert((matrix.solve() == vector<vector<int>>{{1, 3, 5}}));
assert(matrix.solve(2).size() == 1); // Asking for a second solution proves uniqueness.

ExactCover impossible(2);
impossible.add_row(vector<int>{0});
assert(impossible.solve().empty());

// The 8-queens problem. Each board row and each board column must hold exactly one queen, while
// each diagonal may hold at most one, so the 2*n board lines are primary and the 2*(2*n - 1)
// diagonals are secondary. One matrix row per square places a queen there.
const int n = 8;
ExactCover queens(2 * n, 2 * (2 * n - 1));
for (int r = 0; r < n; r++) {
    for (int c = 0; c < n; c++) {
        queens.add_row(vector<int>{r, n + c, 2 * n + r + c, 2 * n + (2 * n - 1) + r - c + n - 1});
    }
}
assert(queens.solve().size() == 1);
assert(queens.solve(1000).size() == 92);
return 0;
}

```

## Chapter 2

# Data Structures

### 2.1 Linked Lists

---

#### 2.1.1 Singly Linked List

Implements common singly linked list operations using raw nodes. Manual linked list implementations are rarely the best practical choice over arrays, vectors, or `std::list`, but the pointer patterns are useful for interview-style problems and understanding constant-time splicing.

For production C++, prefer containers such as `std::list`, `std::forward_list`, or `std::vector` when they fit the problem. Use the raw-node style below when the problem statement already gives nodes, or when manual pointer manipulation is the point of the exercise.

- `reverse_list(head)` reverses a singly linked list and returns the new head.
- `split_half(head, &second)` cuts a list into two halves, returning the first half and storing the second half in `second`.
- `merge_sorted_lists(a, b)` merges two sorted lists using a dummy node and returns the merged head.
- `splice_after(pos, before)` moves the node after `before` so it appears after `pos`.
- `splice_range_after(pos, before_first, last)` moves the half-open range (`before_first`, `last`) so it appears after `pos`; `pos` must not lie inside the moved range.

The splicing helpers use the same “after” convention as `std::forward_list::splice_after()`. They work naturally with a dummy head node, which makes insertions and removals at the beginning of a list match all other positions. For doubly linked list splicing, use the next section.

For cycle detection, see the variants on iterated functions in 1.8.4, which can be directly adapted for linked lists.

**Time Complexity:**

- $O(n)$  per call to `reverse_list()` and `split_half()`.
- $O(n + m)$  per call to `merge_sorted_lists()`.
- $O(1)$  per call to `splice_after()`.
- $O(k)$  per call to `splice_range_after()`, where  $k$  is the number of moved nodes. The actual pointer splicing is  $O(1)$ , but this compact version walks to the end of the moved range.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
struct ListNode {
    int value;
    ListNode *next;

    explicit ListNode(int value = 0) : value(value), next(nullptr) {}
};
```

```

ListNode *reverse_list(ListNode *head) {
    ListNode *prev = nullptr;
    while (head != nullptr) {
        ListNode *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

ListNode *split_half(ListNode *head, ListNode **second) {
    if (head == nullptr || head->next == nullptr) {
        *second = nullptr;
        return head;
    }
    ListNode *slow = head, *fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    *second = slow->next;
    slow->next = nullptr;
    return head;
}

ListNode *merge_sorted_lists(ListNode *a, ListNode *b) {
    ListNode dummy;
    ListNode *tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (b->value < a->value) {
            tail->next = b;
            b = b->next;
        } else {
            tail->next = a;
            a = a->next;
        }
        tail = tail->next;
    }
    tail->next = a != nullptr ? a : b;
    return dummy.next;
}

void splice_after(ListNode *pos, ListNode *before) {
    ListNode *node = before->next;
    if (node == nullptr || pos == before || pos == node) {
        return;
    }
    before->next = node->next;
    node->next = pos->next;
    pos->next = node;
}

void splice_range_after(ListNode *pos, ListNode *before_first, ListNode *last) {
    ListNode *first = before_first->next;
    if (first == last) {
        return;
    }
    ListNode *tail = first;
    while (tail->next != last) {
        tail = tail->next;
    }
    before_first->next = last;
    tail->next = pos->next;
    pos->next = first;
}

```

## Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

vector<int> values(ListNode *head) {
    vector<int> res;
    for (; head != nullptr; head = head->next) {
        res.push_back(head->value);
    }
    return res;
}

int main() {
    vector<ListNode> a{ListNode(1), ListNode(3), ListNode(5)};
    vector<ListNode> b{ListNode(2), ListNode(4), ListNode(6)};
    for (int i = 0; i + 1 < static_cast<int>(a.size()); i++) {
        a[i].next = &a[i + 1];
        b[i].next = &b[i + 1];
    }
    ListNode *merged = merge_sorted_lists(&a[0], &b[0]);
    assert((values(merged) == vector<int>{1, 2, 3, 4, 5, 6}));

    ListNode *second = nullptr;
    ListNode *first = split_half(merged, &second);
    assert((values(first) == vector<int>{1, 2, 3}));
    assert((values(second) == vector<int>{4, 5, 6}));
    assert((values(reverse_list(first)) == vector<int>{3, 2, 1}));

    ListNode dummy(0), w(1), x(2), y(3), z(4);
    dummy.next = &w;
    w.next = &x;
    x.next = &y;
    y.next = &z;
    splice_after(&dummy, &y); // Move 4 to the front: 4, 1, 2, 3.
    assert((values(dummy.next) == vector<int>{4, 1, 2, 3}));
    splice_range_after(&z, &w, nullptr); // Move 2, 3 after 4: 4, 2, 3, 1.
    assert((values(dummy.next) == vector<int>{4, 2, 3, 1}));
    return 0;
}
```

### 2.1.2 Doubly Linked List

Implements common doubly linked list operations using raw intrusive nodes. A node is intrusive when the `prev` and `next` pointers live inside the stored object itself, rather than inside a separate container allocation. This is useful when a problem already gives node pointers, when values must not be copied, or when a map needs to jump directly to a list node in  $O(1)$  time, as in an LRU cache.

The functions below use a circular sentinel node. When the list is empty, `sentinel.next` and `sentinel.prev` both point back to `sentinel`. This removes special cases for inserting or erasing at the front and back of the list.

- `init_list(sentinel)` initializes an empty circular list around `sentinel`.
- `empty(sentinel)` returns whether the list has no data nodes.
- `insert_after(pos, node)` inserts detached `node` immediately after `pos`.
- `insert_before(pos, node)` inserts detached `node` immediately before `pos`.
- `erase(node)` removes `node` from its current list and leaves it detached.
- `push_front(sentinel, node)` inserts detached `node` at the front of the list.
- `push_back(sentinel, node)` inserts detached `node` at the back of the list.
- `move_to_front(sentinel, node)` moves an already-linked `node` to the front of the list.
- `splice(pos, node)` moves an already-linked `node` so it appears immediately before `pos`.

- `splice_range(pos, first, last)` moves the half-open range `[first, last)` so it appears immediately before `pos`.

The data nodes passed to insertion functions must be detached, meaning their `prev` and `next` pointers are both null. The node passed to `erase()` or `move_to_front()` must already be linked into a list and must not be the sentinel. For `splice_range(pos, first, last)`, the `pos` node must not lie inside the moved range.

**Time Complexity:**  $O(1)$  per call to all operations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
struct DListNode {
    int value;
    DListNode *prev, *next;

    explicit DListNode(int value = 0) : value(value), prev(nullptr), next(nullptr) {}
};

void init_list(DListNode *sentinel) {
    sentinel->prev = sentinel;
    sentinel->next = sentinel;
}

bool empty(DListNode *sentinel) {
    return sentinel->next == sentinel;
}

void insert_after(DListNode *pos, DListNode *node) {
    node->prev = pos;
    node->next = pos->next;
    pos->next->prev = node;
    pos->next = node;
}

void insert_before(DListNode *pos, DListNode *node) {
    insert_after(pos->prev, node);
}

DListNode *erase(DListNode *node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
    node->prev = nullptr;
    node->next = nullptr;
    return node;
}

void push_front(DListNode *sentinel, DListNode *node) {
    insert_after(sentinel, node);
}

void push_back(DListNode *sentinel, DListNode *node) {
    insert_before(sentinel, node);
}

void move_to_front(DListNode *sentinel, DListNode *node) {
    push_front(sentinel, erase(node));
}

void splice(DListNode *pos, DListNode *node) {
    if (pos == node || pos == node->next) {
        return;
    }
    insert_before(pos, erase(node));
}

void splice_range(DListNode *pos, DListNode *first, DListNode *last) {
    if (first == last || pos == first) {
        return;
    }
}
```

```

DListNode *before_first = first->prev;
DListNode *tail = last->prev;
// Detach [first, last) from its current list.
before_first->next = last;
last->prev = before_first;
// Insert the detached range immediately before pos.
DListNode *before_pos = pos->prev;
before_pos->next = first;
first->prev = before_pos;
tail->next = pos;
pos->prev = tail;
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

// Returns the data values from front to back (walks the circular list until it loops to sentinel).
vector<int> values(DListNode *sentinel) {
    vector<int> res;
    for (DListNode *p = sentinel->next; p != sentinel; p = p->next) {
        res.push_back(p->value);
    }
    return res;
}

int main() {
    // dummy is the sentinel; a, b, c, d start detached. push_back/push_front link at back/front.
    DListNode dummy, a(1), b(2), c(3), d(4);
    init_list(&dummy);
    assert(empty(&dummy));

    push_back(&dummy, &a);
    push_back(&dummy, &b);
    push_back(&dummy, &c);
    push_front(&dummy, &d);
    assert((values(&dummy) == vector<int>{4, 1, 2, 3}));

    move_to_front(&dummy, &b); // list: 2 4 1 3
    assert((values(&dummy) == vector<int>{2, 4, 1, 3}));

    erase(&d); // detach d; list: 2 1 3
    assert((values(&dummy) == vector<int>{2, 1, 3}));

    insert_before(&dummy, &d); // before the sentinel == the back; list: 2 1 3 4
    assert((values(&dummy) == vector<int>{2, 1, 3, 4}));

    splice(&d, &a); // move a to just before d; list: 2 3 1 4
    assert((values(&dummy) == vector<int>{2, 3, 1, 4}));

    splice_range(&dummy, &a, &dummy); // range {a, d} already at the back: no-op
    assert((values(&dummy) == vector<int>{2, 3, 1, 4}));

    splice_range(&b, &a, &dummy); // move {a, d} to before b; list: 1 4 2 3
    assert((values(&dummy) == vector<int>{1, 4, 2, 3}));
    return 0;
}

```



### 2.1.3 LRU Cache

Maintains a least-recently-used cache with fixed capacity. The cache supports lookups and updates by key, evicting the least recently used entry whenever an insertion would exceed capacity.

This implementation combines a doubly linked list (`std::list`) with a map from keys to list iterators. The front of the list stores the most recently used item. The operation `items.splice(items.begin(), items, it)` is the standard-library way to move an existing list node to the front in  $O(1)$  time. The key type `K` must support `operator==`, and `std::hash<K>` must be defined.

To implement the same cache without `std::list`, use the intrusive doubly linked list operations from section 2.1.2. Store a `Node*` in the map instead of a list iterator, move hits to the front with `move_to_front(&sentinel, node)`, and evict `sentinel.prev`.

- `LRUCache<K, V>(capacity)` constructs an empty cache holding at most `capacity` keys.
- `size()` returns the number of keys currently stored.
- `get(key, &value)` returns whether `key` is present. On success, it copies the associated value into `value` and marks the key as most recently used.
- `put(key, value)` inserts or updates `key`, marking it as most recently used and evicting the least recently used key if needed.

**Time Complexity:**  $O(1)$  expected amortized per call to `get()` and `put()`, and  $O(n)$  in the collision-heavy worst case for the hash table.

**Space Complexity:**  $O(n)$ , where  $n$  is the cache capacity.

```
#include <cassert>
#include <list>
#include <unordered_map>
#include <utility>

// Replacing std::list with a manual doubly-linked list would look something like:
// struct DListNode { K key; V value; DListNode *prev, *next; };
// std::unordered_map<K, DListNode*> where;
//
// On get:
//   move_to_front(&sentinel, where[key]);
//
// On eviction:
//   DListNode *old = sentinel.prev;
//   erase(old);
//   where.erase(old->key).

template<typename K, typename V>
class LRUCache {
    int cap;
    std::list<std::pair<K, V>> items;
    std::unordered_map<K, decltype(items.begin())> where;

public:
    explicit LRUCache(int capacity) : cap(capacity) { assert(capacity >= 0); }

    LRUCache(const LRUCache &) = delete;
    LRUCache &operator=(const LRUCache &) = delete;
    int size() const { return static_cast<int>(items.size()); }

    bool get(const K &key, V *value) {
        assert(value != nullptr);
        auto it = where.find(key);
        if (it == where.end()) {
            return false;
        }
        items.splice(items.begin(), items, it->second);
        it->second = items.begin();
        *value = it->second->second;
    }
};
```

```

    return true;
}

void put(const K &key, const V &value) {
    auto it = where.find(key);
    if (it != where.end()) {
        it->second->second = value;
        items.splice(items.begin(), items, it->second);
        it->second = items.begin();
        return;
    }
    if (cap == 0) {
        return;
    }
    if (static_cast<int>(items.size()) == cap) {
        where.erase(items.back().first);
        items.pop_back();
    }
    items.emplace_front(key, value);
    where[key] = items.begin();
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    LRUCache<int, int> cache(2);
    int value = 0;
    cache.put(1, 10);
    cache.put(2, 20);
    assert(cache.get(1, &value) && value == 10);
    cache.put(3, 30); // Evicts key 2.
    assert(!cache.get(2, &value));
    assert(cache.get(3, &value) && value == 30);
    cache.put(1, 11);
    assert(cache.get(1, &value) && value == 11);

    LRUCache<int, int> disabled(0);
    disabled.put(1, 10);
    assert(disabled.size() == 0);
    return 0;
}

```

### 2.1.4 LFU Cache

Maintains a least-frequently-used cache with fixed capacity. The cache supports lookups and updates by key, evicting the entry with the fewest accesses whenever an insertion would exceed capacity, and breaking ties among equally frequent entries by evicting the least recently used of them.

The difficulty compared to the least-recently-used cache of section 2.1.3 is that frequencies form an unbounded set of buckets rather than a single ordering, so a naive implementation scans for the minimum on every eviction. The fix is to bucket the keys by frequency, keeping one list of keys per frequency in most-recently-used order, and to track the smallest occupied frequency. Every access moves a key from bucket  $f$  to bucket  $f + 1$ , which is  $O(1)$  with an iterator stored per key.

The tracked minimum needs no search either. An insertion always creates a key of frequency 1, so the minimum becomes 1; an access raises exactly one key from the minimum bucket, so the minimum can only rise by one, and

it does so only when that bucket empties. Both updates are  $O(1)$ , which is what makes the whole structure constant time.

- `LFUCache<K, V>(capacity)` constructs an empty cache holding at most `capacity` keys, which must be positive. The key type `K` must support `operator==` and have `std::hash<K>` defined.
- `size()` returns the number of keys currently stored.
- `freq(key)` returns the number of accesses counted for `key`, or 0 if it is absent.
- `get(key, &value)` returns whether `key` is present. On success, it copies the associated value into `value` and counts an access.
- `put(key, value)` inserts or updates `key`, counts an access, and evicts the least frequently used key if the insertion would exceed capacity.

Frequencies grow without bound, so a long-lived cache can be held hostage by keys popular only once; real implementations periodically halve every count, ageing the statistics without reordering them.

#### Time Complexity:

- $O(1)$  expected amortized per call to `get()`, `put()`, and `freq()`, and  $O(n)$  in the collision-heavy worst case for the hash table.
- $O(1)$  per call to `size()`.

**Space Complexity:**  $O(n)$ , where  $n$  is the cache capacity.

```
#include <cassert>
#include <list>
#include <unordered_map>

template<typename K, typename V>
class LFUCache {
    struct Entry {
        V value;
        int freq;
        typename std::list<K>::iterator pos;
    };

    int cap, min_freq;
    std::unordered_map<K, Entry> entries;
    std::unordered_map<int, std::list<K>> buckets; // Keys of each frequency, newest at the front.

    // Moves an existing key from its bucket to the next one up, counting one access.
    void touch(const K &key) {
        Entry &entry = entries[key];
        std::list<K> &bucket = buckets[entry.freq];
        bucket.erase(entry.pos);
        if (bucket.empty()) {
            buckets.erase(entry.freq);
            if (min_freq == entry.freq) {
                min_freq++;
            }
        }
        entry.freq++;
        bucket.push_front(key);
        entry.pos = bucket.begin();
    }

public:
    explicit LFUCache(int capacity) : cap(capacity), min_freq(0) { assert(cap > 0); }

    int size() const { return static_cast<int>(entries.size()); }

    int freq(const K &key) const {
        auto it = entries.find(key);
        return it == entries.end() ? 0 : it->second.freq;
    }

    bool get(const K &key, V *value) {
```

```

    auto it = entries.find(key);
    if (it == entries.end()) {
        return false;
    }
    if (value != nullptr) {
        *value = it->second.value;
    }
    touch(key);
    return true;
}

void put(const K &key, const V &value) {
    if (entries.count(key) > 0) {
        entries[key].value = value;
        touch(key);
        return;
    }
    if (size() == cap) {
        // The back of the minimum bucket is the least recently used of the least frequent keys.
        entries.erase(buckets[min_freq].back());
        buckets[min_freq].pop_back();
        if (buckets[min_freq].empty()) {
            buckets.erase(min_freq);
        }
    }
    min_freq = 1;
    buckets[1].push_front(key);
    entries[key] = Entry{value, 1, buckets[1].begin()};
}
};

```

## Example Usage

```

#include <string>
using namespace std;

int main() {
    LFUCache<int, string> cache(2);
    string value;
    assert(!cache.get(1, &value));

    cache.put(1, "one");
    cache.put(2, "two");
    assert(cache.size() == 2);
    assert(cache.get(1, &value) && value == "one");
    assert(cache.freq(1) == 2 && cache.freq(2) == 1);

    cache.put(3, "three"); // Key 2 is the least frequently used, so it is evicted.
    assert(!cache.get(2, &value));
    assert(cache.get(3, &value) && value == "three");

    // Keys 1 and 3 now both have frequency 2, so the least recently used of the two goes next.
    assert(cache.freq(1) == 2 && cache.freq(3) == 2);
    cache.put(4, "four");
    assert(!cache.get(1, &value));
    assert(cache.get(3, &value) && value == "three");

    // Updating a stored key counts as an access rather than an insertion.
    cache.put(3, "tres");
    assert(cache.get(3, &value) && value == "tres" && cache.size() == 2);
    return 0;
}

```

## 2.2 Binary Trees

### 2.2.1 Binary Tree Traversal

The three depth-first orders of a binary tree differ only in when a node is reported relative to its subtrees: preorder before both, inorder between them, postorder after both. Recursion expresses this directly but costs stack space proportional to the height, which overflows on a degenerate tree of a hundred thousand nodes, so the versions below replace the call stack with an explicit one. Postorder is the one that resists a direct translation, since a node must be reported only after returning from its right child; visiting root, right, then left is a mirrored preorder, and reversing that sequence yields postorder.

- `preorder(root)`, `inorder(root)`, and `postorder(root)` return the values of the tree in each depth-first order, using an explicit stack.
- `level_order(root)` returns the values one depth at a time, left to right, using a queue.

Morris traversal removes the stack entirely by threading. Before descending into a left subtree, it finds that subtree's rightmost node, which is the inorder predecessor of the current node, and points that node's empty right pointer back at the current node. On reaching the end of the subtree the walk follows the thread back up, and seeing it a second time signals that the left subtree is finished, so the node is reported and the borrowed pointer restored. Each edge is traversed at most three times, keeping the walk linear. Flattening uses the same predecessor search but keeps the rewiring: splicing the right subtree onto that node and moving the left subtree across leaves a right-leaning chain in preorder.

- `morris_inorder(root)` returns the values in inorder using  $O(1)$  auxiliary space, temporarily rewriting the tree's null pointers and restoring them before returning.
- `flatten(root)` rewrites the tree in place into a right-leaning chain whose order is the tree's preorder, leaving every left pointer null and destroying the original shape.

Since `morris_inorder()` mutates the tree as it runs, it is unsafe on a tree shared between threads and leaves threads behind if an exception escapes. Computing over a tree rather than listing it is dynamic programming on a tree; see sections 4.1.4 through 4.1.7 for general rooted trees.

**Time Complexity:**  $O(n)$  per call to every traversal and to `flatten()`, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(n)$  for each returned vector.
- $O(h)$  auxiliary for the stack-based and queue-based traversals, where  $h$  is the height of the tree, which is  $O(n)$  in the worst case.
- $O(1)$  auxiliary for `morris_inorder()` and `flatten()`.

```
#include <algorithm>
#include <queue>
#include <vector>

template<typename T>
struct TreeNode {
    T value;
    TreeNode *left, *right;

    explicit TreeNode(const T &value) : value(value), left(nullptr), right(nullptr) {}
};

template<typename T>
std::vector<T> preorder(TreeNode<T> *root) {
    std::vector<T> res;
    std::vector<TreeNode<T> *> st;
    if (root != nullptr) {
        st.push_back(root);
    }
    while (!st.empty()) {
        TreeNode<T> *node = st.back();
```

```

    st.pop_back();
    res.push_back(node->value);
    if (node->right != nullptr) { // Pushed first so that the left child is popped first.
        st.push_back(node->right);
    }
    if (node->left != nullptr) {
        st.push_back(node->left);
    }
}
}
return res;
}

```

```

template<typename T>
std::vector<T> inorder(TreeNode<T> *root) {
    std::vector<T> res;
    std::vector<TreeNode<T> *> st;
    for (TreeNode<T> *node = root; node != nullptr || !st.empty(); ) {
        for (; node != nullptr; node = node->left) {
            st.push_back(node);
        }
        node = st.back();
        st.pop_back();
        res.push_back(node->value);
        node = node->right;
    }
    return res;
}

```

```

template<typename T>
std::vector<T> postorder(TreeNode<T> *root) {
    std::vector<T> res;
    std::vector<TreeNode<T> *> st;
    if (root != nullptr) {
        st.push_back(root);
    }
    while (!st.empty()) { // Root, right, left, which is postorder reversed.
        TreeNode<T> *node = st.back();
        st.pop_back();
        res.push_back(node->value);
        if (node->left != nullptr) {
            st.push_back(node->left);
        }
        if (node->right != nullptr) {
            st.push_back(node->right);
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

```

```

template<typename T>
std::vector<T> level_order(TreeNode<T> *root) {
    std::vector<T> res;
    std::queue<TreeNode<T> *> q;
    if (root != nullptr) {
        q.push(root);
    }
    while (!q.empty()) {
        TreeNode<T> *node = q.front();
        q.pop();
        res.push_back(node->value);
        if (node->left != nullptr) {
            q.push(node->left);
        }
        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

```

```

    return res;
}

template<typename T>
std::vector<T> morris_inorder(TreeNode<T> *root) {
    std::vector<T> res;
    for (TreeNode<T> *node = root; node != nullptr;) {
        if (node->left == nullptr) {
            res.push_back(node->value);
            node = node->right;
            continue;
        }
        TreeNode<T> *pred = node->left;
        while (pred->right != nullptr && pred->right != node) {
            pred = pred->right;
        }
        if (pred->right == nullptr) { // First visit: thread the predecessor back to this node.
            pred->right = node;
            node = node->left;
        } else { // Second visit: the left subtree is done, so restore and report.
            pred->right = nullptr;
            res.push_back(node->value);
            node = node->right;
        }
    }
    return res;
}

template<typename T>
void flatten(TreeNode<T> *root) {
    for (TreeNode<T> *node = root; node != nullptr; node = node->right) {
        if (node->left != nullptr) {
            TreeNode<T> *pred = node->left;
            while (pred->right != nullptr) {
                pred = pred->right;
            }
            pred->right = node->right; // Splice the right subtree below the left subtree's last node.
            node->right = node->left;
            node->left = nullptr;
        }
    }
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      4
    //     / \
    //    2   6
    //   / \  \
    //  1  3  7
    TreeNode<int> n1(1), n2(2), n3(3), n4(4), n6(6), n7(7);
    n4.left = &n2;
    n4.right = &n6;
    n2.left = &n1;
    n2.right = &n3;
    n6.right = &n7;

    assert((preorder(&n4) == vector<int>{4, 2, 1, 3, 6, 7}));
    assert((inorder(&n4) == vector<int>{1, 2, 3, 4, 6, 7}));
    assert((postorder(&n4) == vector<int>{1, 3, 2, 7, 6, 4}));
    assert((level_order(&n4) == vector<int>{4, 2, 6, 1, 3, 7}));
    assert((morris_inorder(&n4) == vector<int>{1, 2, 3, 4, 6, 7}));
}

```

```

// The threads are undone, so the tree is unchanged and a second walk agrees.
assert(n3.right == nullptr && n1.right == nullptr);
assert(morris_inorder(&n4) == inorder(&n4));
assert(inorder<int>(nullptr).empty() && morris_inorder<int>(nullptr).empty());

// A left-leaning chain, the shape that makes a recursive walk overflow the stack.
TreeNode<int> a(1), b(2), c(3);
c.left = &b;
b.left = &a;
assert((morris_inorder(&c) == vector<int>{1, 2, 3}));
assert((postorder(&c) == vector<int>{1, 2, 3}));

// Flattening leaves a right-leaning chain in preorder, so a walk of it is that preorder.
vector<int> before = preorder(&n4);
flatten(&n4);
assert(n4.left == nullptr && preorder(&n4) == before);
for (TreeNode<int> *node = &n4; node != nullptr; node = node->right) {
    assert(node->left == nullptr);
}
return 0;
}

```

## 2.2.2 Binary Tree Encodings

An encoding of a binary tree is a sequence the tree can be rebuilt from, and a single traversal is not one, since many shapes share the same preorder. A pair of traversals usually is: an inorder together with either a preorder or a postorder pins the shape down, because the inorder splits the values into a left group and a right group once the root is known, and the other traversal is what identifies that root. Preorder puts the root first, postorder puts it last, and each recursion then knows exactly how many values belong to each side.

Written naively, locating the root inside the inorder range costs a linear scan and the construction degenerates to  $O(n^2)$  on a skewed tree. Hashing each value to its inorder position first makes every lookup  $O(1)$  expected, so the whole build is linear expected time. This is why the values must be distinct: with duplicates, the position of a root in the inorder sequence is ambiguous and no traversal pair determines a unique tree.

- `build_from_preorder_inorder(pre, in)` returns the root of the tree with the given preorder and inorder traversals, or `nullptr` when they are empty. Both sequences must be the same length and contain the same distinct values.
- `build_from_postorder_inorder(post, in)` does the same from a postorder and an inorder traversal.

The construction routines allocate each node with `new` and return raw pointers. Thus the caller owns the resulting tree and should eventually delete its nodes in long-running programs. Contest programs commonly omit this cleanup when the tree is needed until program termination, since the operating system reclaims the process's memory on exit.

A third encoding needs only one traversal, because recording the null children removes the ambiguity that made a pair necessary. A preorder in which every absent child becomes a sentinel token reconstructs the tree by reading the tokens back in order, and unlike a traversal pair it tolerates duplicate values, the shape now being explicit rather than inferred.

- `serialize(root)` returns the tree as a whitespace-separated preorder string, writing `#` for each null child. The value type must support stream output.
- `deserialize(s)` rebuilds the tree that `serialize()` produced. The value type must support stream input. Its returned nodes are allocated with `new` and have the same ownership convention as the construction routines above.

Serialization is what lets a subtree be used as a key: two subtrees are identical exactly when their serializations match, which turns finding duplicate subtrees, or memoizing over tree shapes, into hashing strings. Prefer hashing



a canonical encoding this way over comparing trees pairwise. Note that this encoding is specific to binary trees; section 4.1.8 encodes labeled general trees instead.

#### Time Complexity:

- $O(n)$  expected per call to both construction routines, where  $n$  is the number of nodes, from the hash lookups of inorder positions.
- $O(n)$  per call to `serialize()` and `deserialize()`.

#### Space Complexity:

- $O(n)$  for the allocated tree nodes.
- $O(n)$  auxiliary for the position map, and  $O(h)$  auxiliary stack space, where  $h$  is the height of the tree.
- $O(n)$  for the string returned by `serialize()`.

```
#include <cassert>
#include <sstream>
#include <string>
#include <unordered_map>
#include <vector>

template<typename T>
struct TreeNode {
    T value;
    TreeNode *left, *right;

    explicit TreeNode(const T &value) : value(value), left(nullptr), right(nullptr) {}
};

template<typename T>
TreeNode<T> *build_from_preorder_inorder(const std::vector<T> &pre, const std::vector<T> &in) {
    assert(pre.size() == in.size());
    int n = static_cast<int>(pre.size());
    std::unordered_map<T, int> pos;
    for (int i = 0; i < n; i++) {
        pos[in[i]] = i;
    }
    assert(static_cast<int>(pos.size()) == n);
    int next = 0;
    auto rec = [&](auto &&rec, int lo, int hi) -> TreeNode<T> * {
        if (lo > hi) {
            return nullptr;
        }
        const T &value = pre[next++];
        int mid = pos[value];
        TreeNode<T> *node = new TreeNode<T>(value);
        node->left = rec(rec, lo, mid - 1);
        node->right = rec(rec, mid + 1, hi);
        return node;
    };
    return rec(rec, 0, n - 1);
}

template<typename T>
TreeNode<T> *build_from_postorder_inorder(const std::vector<T> &post, const std::vector<T> &in) {
    assert(post.size() == in.size());
    int n = static_cast<int>(post.size());
    std::unordered_map<T, int> pos;
    for (int i = 0; i < n; i++) {
        pos[in[i]] = i;
    }
    assert(static_cast<int>(pos.size()) == n);
    int next = n - 1;
    auto rec = [&](auto &&rec, int lo, int hi) -> TreeNode<T> * {
        if (lo > hi) {
            return nullptr;
        }
    }
```

```

    const T &value = post[next--];
    TreeNode<T> *node = new TreeNode<T>(value);
    int mid = pos[value];
    node->right = rec(rec, mid + 1, hi); // Postorder ends with the root, so the right side first.
    node->left = rec(rec, lo, mid - 1);
    return node;
};
return rec(rec, 0, n - 1);
}

template<typename T>
std::string serialize(TreeNode<T> *root) {
    std::ostringstream out;
    auto rec = [&](auto &&rec, TreeNode<T> *node) {
        if (node == nullptr) {
            out << "# ";
            return;
        }
        out << node->value << " ";
        rec(rec, node->left);
        rec(rec, node->right);
    };
    rec(rec, root);
    return out.str();
}

template<typename T>
TreeNode<T> *deserialize(const std::string &s) {
    std::istringstream in(s);
    auto rec = [&](auto &&rec) -> TreeNode<T> * {
        if (!(in >> std::ws) || in.peek() == '#') {
            in.get(); // Consume the sentinel, or nothing at all once the stream is spent.
            return nullptr;
        }
        T value;
        in >> value;
        TreeNode<T> *node = new TreeNode<T>(value);
        node->left = rec(rec);
        node->right = rec(rec);
        return node;
    };
    return rec(rec);
}

```

## Example Usage

```

#include <cassert>
using namespace std;

template<typename T>
vector<T> inorder(TreeNode<T> *root) {
    vector<T> res;
    auto rec = [&](auto &&rec, TreeNode<T> *node) -> void {
        if (node != nullptr) {
            rec(rec, node->left);
            res.push_back(node->value);
            rec(rec, node->right);
        }
    };
    rec(rec, root);
    return res;
}

int main() {
    //      4
    //     / \

```

```

//      2      6
//    /  \    \.
//   1    3    7
vector<int> pre{4, 2, 1, 3, 6, 7}, in{1, 2, 3, 4, 6, 7}, post{1, 3, 2, 7, 6, 4};

TreeNode<int> *root = build_from_preorder_inorder(pre, in);
assert(root->value == 4 && root->left->value == 2 && root->right->value == 6);
assert(root->right->left == nullptr && root->right->right->value == 7);
assert(inorder(root) == in);

TreeNode<int> *same = build_from_postorder_inorder(post, in);
assert(serialize(same) == serialize(root));

// Preorder with a sentinel for every absent child, so the shape is written down rather than
// inferred: 4 descends left to 2, whose children 1 and 3 are leaves, then right to 6, whose
// missing left child is the third "#" after it.
assert(serialize(root) == "4 2 1 # # 3 # # 6 # 7 # # ");
assert(serialize(root->left) == "2 1 # # 3 # # ");
assert(serialize(root->left->left) == "1 # # ");

// Two sentinels in a row close off a leaf, so a left chain ends with one per level.
TreeNode<int> *chain = deserialize<int>("1 2 3 # # # # ");
assert(chain->left->left->value == 3 && chain->right == nullptr);
assert(serialize(chain) == "1 2 3 # # # # ");

// The serialized form round-trips, and matching strings mean identical subtrees.
assert(inorder(deserialize<int>(serialize(root))) == in);
assert(serialize(root->left) != serialize(root->right));

assert(build_from_preorder_inorder(vector<int>{}, vector<int>{}) == nullptr);
assert(serialize<int>(nullptr) == "# ");
assert(deserialize<int>("# ") == nullptr);
return 0;
}

```

### 2.2.3 Binary Tree Queries

Nearly every question about a binary tree is answered by one recursion over it, and the questions differ only in what that recursion carries. Three shapes cover almost all of them. The first passes information upward, each node combining what its children return, as `height()` and `size()` do. The second passes information downward, each node extending what its parent handed it, as a running path sum does. The third does both at once and is the shape most often gotten wrong, because the value a node returns is not the answer recorded there: in `diameter()` a node returns the longest downward path through it but records the sum of its two longest, which is a path no parent can extend.

- `height(root)` returns the number of nodes on the longest root-to-leaf path, so an empty tree has height 0 and a leaf has height 1.
- `size(root)` returns the number of nodes.
- `is_balanced(root)` returns whether every node's two subtrees differ in height by at most 1.

Balance is a condition on every node, not only the root. Checking it by calling `height()` at each node costs  $O(n^2)$  on a skewed tree, so the recursion below returns a height instead and reserves  $-1$  for “already unbalanced”, answering in one pass.

Validating a search tree is where carrying information downward is not optional: comparing a node only against its own children accepts trees that are not search trees at all, since a value deep in a left subtree can exceed the root while every local comparison passes. The recursion carries the open interval a subtree may occupy, using a null pointer for an absent bound so that no sentinel value of `T` is needed. Walking the tree in order and checking that values increase, using the traversal of section 2.2.1, is the alternative.

- `has_path_sum(root, target)` returns whether some root-to-leaf path has values summing to `target`. An empty tree has no such path, whatever the target.
- `root_to_leaf_paths(root)` returns every root-to-leaf path, each as the values along it.
- `depth(root, target)` returns the number of edges from `root` down to the node `target`, or  $-1$  if that node is not in the tree.
- `is_bst(root)` returns whether the tree satisfies the binary search tree property, using strict ordering so that equal values are rejected.

Counting downward paths by trying every start node is  $O(n^2)$ . Instead, keep the running sum from the root and note that a path ending here with sum `target` belongs to an ancestor whose own running sum was smaller by exactly `target`: the prefix-sum trick of section 1.2.1, lifted from an array to a root-to-node path. A map counts the ancestors' running sums, each node undoing its own entry on the way back out so that only genuine ancestors match. It is ordered here for genericity, costing a logarithmic factor that an `unordered_map` removes for hashable `T`. Storing another key instead counts paths whose sum is divisible by `k`, or whose values exclusive-or to a target.

- `diameter(root)` returns the number of edges on the longest path between any two nodes, which is 0 for a single node and for an empty tree.
- `max_path_sum(root)` returns the largest sum along any downward-then-upward path between two nodes, for a nonempty tree whose values may be negative. A child subtree contributes only when its best downward sum is positive, so an all-negative tree returns its largest single value.
- `count_paths_with_sum(root, target)` returns the number of downward paths, not necessarily starting at the root, whose values sum to `target`.

Comparing against the two targets while unwinding is all `lca()` needs, with no parent pointers and no preprocessing: a node whose subtrees each report a find is the meeting point, and a node reporting one find passes it upward. That is  $O(n)$  per query rather than the  $O(1)$  after preprocessing of sections 2.8.4 and 2.8.5, so prefer it for a handful of queries. A tree is symmetric exactly when `same_tree()` holds between one half and the mirror of the other, so no separate routine is needed.

- `invert(root)` mirrors the tree in place by swapping every node's children.
- `same_tree(a, b)` returns whether two trees have the same shape and the same values.
- `lca(root, a, b)` returns the lowest node having both `a` and `b` in its subtree, counting a node as its own ancestor. Both nodes must belong to the tree; if only one does, that one is returned.
- `distance(root, a, b)` returns the number of edges on the path between the two nodes, which is the depth of each below their lowest common ancestor.

For a tree held as an adjacency list, or one of arbitrary degree, these computations become the tree dynamic programming of sections 4.1.4 through 4.1.7.

### Time Complexity:

- $O(n)$  per call to every operation except `count_paths_with_sum()`, where  $n$  is the number of nodes.
- $O(n \log n)$  per call to `count_paths_with_sum()` from the ordered map, or  $O(n)$  expected with an unordered one.

### Space Complexity:

- $O(h)$  auxiliary stack space, where  $h$  is the height of the tree, which is  $O(n)$  in the worst case.
- $O(h)$  auxiliary for the running sums of `count_paths_with_sum()`, one per ancestor.
- $O(n \cdot h)$  for the paths returned by `root_to_leaf_paths()`.

```
#include <algorithm>
#include <cstdlib>
#include <map>
#include <utility>
#include <vector>

template<typename T>
struct TreeNode {
    T value;
    TreeNode *left, *right;

    explicit TreeNode(const T &value) : value(value), left(nullptr), right(nullptr) {}
};
```

```

};

template<typename T>
int height(TreeNode<T> *root) {
    return root == nullptr ? 0 : 1 + std::max(height(root->left), height(root->right));
}

template<typename T>
int size(TreeNode<T> *root) {
    return root == nullptr ? 0 : 1 + size(root->left) + size(root->right);
}

template<typename T>
bool is_balanced(TreeNode<T> *root) {
    // Returns the height, or -1 once any subtree below has been found unbalanced.
    auto rec = [&](auto &&rec, TreeNode<T> *node) -> int {
        if (node == nullptr) {
            return 0;
        }
        int left = rec(rec, node->left), right = rec(rec, node->right);
        if (left < 0 || right < 0 || std::abs(left - right) > 1) {
            return -1;
        }
        return 1 + std::max(left, right);
    };
    return rec(rec, root) >= 0;
}

template<typename T>
bool has_path_sum(TreeNode<T> *root, const T &target) {
    if (root == nullptr) {
        return false;
    }
    if (root->left == nullptr && root->right == nullptr) {
        return root->value == target;
    }
    T rest = target - root->value;
    return has_path_sum(root->left, rest) || has_path_sum(root->right, rest);
}

template<typename T>
std::vector<std::vector<T>> root_to_leaf_paths(TreeNode<T> *root) {
    std::vector<std::vector<T>> res;
    std::vector<T> path;
    auto rec = [&](auto &&rec, TreeNode<T> *node) {
        if (node == nullptr) {
            return;
        }
        path.push_back(node->value);
        if (node->left == nullptr && node->right == nullptr) {
            res.push_back(path);
        } else {
            rec(rec, node->left);
            rec(rec, node->right);
        }
        path.pop_back();
    };
    rec(rec, root);
    return res;
}

template<typename T>
int depth(TreeNode<T> *root, TreeNode<T> *target) {
    if (root == nullptr || root == target) {
        return root == nullptr ? -1 : 0;
    }
    int left = depth(root->left, target);
    if (left >= 0) {

```

```

    return left + 1;
}
int right = depth(root->right, target);
return right >= 0 ? right + 1 : -1;
}

template<typename T>
bool is_bst(TreeNode<T> *root) {
    // A null bound means unconstrained, so no sentinel value of T is required.
    auto rec = [&](auto &&rec, TreeNode<T> *node, const T *lo, const T *hi) -> bool {
        if (node == nullptr) {
            return true;
        }
        if ((lo != nullptr && !(*lo < node->value)) || (hi != nullptr && !(node->value < *hi))) {
            return false;
        }
        return rec(rec, node->left, lo, &node->value) && rec(rec, node->right, &node->value, hi);
    };
    return rec(rec, root, nullptr, nullptr);
}

template<typename T>
int diameter(TreeNode<T> *root) {
    int best = 0;
    // Returns the edge count of the longest downward path, while recording the longest path through.
    auto rec = [&](auto &&rec, TreeNode<T> *node) -> int {
        if (node == nullptr) {
            return -1;
        }
        int left = rec(rec, node->left) + 1, right = rec(rec, node->right) + 1;
        best = std::max(best, left + right);
        return std::max(left, right);
    };
    rec(rec, root);
    return best;
}

template<typename T>
T max_path_sum(TreeNode<T> *root) {
    T best = root->value;
    auto rec = [&](auto &&rec, TreeNode<T> *node) -> T {
        if (node == nullptr) {
            return T{};
        }
        // A subtree joins the path only when it adds something, so a negative branch is dropped.
        T left = std::max(T{}, rec(rec, node->left)), right = std::max(T{}, rec(rec, node->right));
        best = std::max(best, node->value + left + right);
        return node->value + std::max(left, right);
    };
    rec(rec, root);
    return best;
}

template<typename T>
int64_t count_paths_with_sum(TreeNode<T> *root, const T &target) {
    std::map<T, int> seen{{T{}, 1}}; // The empty prefix, so paths starting at the root are counted.
    int64_t res = 0;
    auto rec = [&](auto &&rec, TreeNode<T> *node, T prefix) {
        if (node == nullptr) {
            return;
        }
        prefix = prefix + node->value;
        auto it = seen.find(prefix - target);
        if (it != seen.end()) {
            res += it->second;
        }
        seen[prefix]++;
        rec(rec, node->left, prefix);
    };
    rec(rec, root, T{});
}

```

```

    rec(rec, node->right, prefix);
    if (--seen[prefix] == 0) { // Undone on the way out, leaving only true ancestors behind.
        seen.erase(prefix);
    }
};
rec(rec, root, T{});
return res;
}

template<typename T>
void invert(TreeNode<T> *root) {
    if (root != nullptr) {
        std::swap(root->left, root->right);
        invert(root->left);
        invert(root->right);
    }
}

template<typename T>
bool same_tree(TreeNode<T> *a, TreeNode<T> *b) {
    if (a == nullptr || b == nullptr) {
        return a == b;
    }
    return a->value == b->value && same_tree(a->left, b->left) && same_tree(a->right, b->right);
}

template<typename T>
TreeNode<T> *lca(TreeNode<T> *root, TreeNode<T> *a, TreeNode<T> *b) {
    if (root == nullptr || root == a || root == b) {
        return root;
    }
    TreeNode<T> *left = lca(root->left, a, b), *right = lca(root->right, a, b);
    if (left != nullptr && right != nullptr) {
        return root; // Targets found on opposite sides, so this node is where they meet.
    }
    return left != nullptr ? left : right;
}

template<typename T>
int distance(TreeNode<T> *root, TreeNode<T> *a, TreeNode<T> *b) {
    TreeNode<T> *meet = lca(root, a, b);
    return depth(meet, a) + depth(meet, b);
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      4
    //     / \
    //    2   6
    //   / \  \
    //  1  3  7
    TreeNode<int> n1(1), n2(2), n3(3), n4(4), n6(6), n7(7);
    n4.left = &n2;
    n4.right = &n6;
    n2.left = &n1;
    n2.right = &n3;
    n6.right = &n7;

    assert(height(&n4) == 3 && size(&n4) == 6);
    assert(height<int>(nullptr) == 0 && size<int>(nullptr) == 0);
    assert(is_balanced(&n4));
}

```

```

assert(has_path_sum(&n4, 7)); // 4 + 2 + 1
assert(has_path_sum(&n4, 17)); // 4 + 6 + 7
assert(!has_path_sum(&n4, 6)); // 4 + 2 stops at an internal node.
assert((root_to_leaf_paths(&n4) == vector<vector<int>>{{4, 2, 1}, {4, 2, 3}, {4, 6, 7}}));

assert(diameter(&n4) == 4); // The path 1, 2, 4, 6, 7 has four edges.
assert(max_path_sum(&n4) == 22); // The path 3, 2, 4, 6, 7.

assert(lca(&n4, &n1, &n3) == &n2);
assert(lca(&n4, &n1, &n7) == &n4);
assert(lca(&n4, &n6, &n7) == &n6); // A node is its own ancestor.
assert(depth(&n4, &n7) == 2 && depth(&n4, &n4) == 0);
assert(distance(&n4, &n1, &n7) == 4 && distance(&n4, &n1, &n3) == 2);

// The fixture happens to be a search tree, since its inorder is 1, 2, 3, 4, 6, 7.
assert(is_bst(&n4));

// Every local comparison passes here, yet 4 sits in the right subtree of 5.
//      5
//     / \
//    1   6
//   / \
//  4   7
TreeNode<int> b5(5), b1(1), b6(6), b4(4), b7(7);
b5.left = &b1;
b5.right = &b6;
b6.left = &b4;
b6.right = &b7;
assert(!is_bst(&b5));

// Downward paths summing to 7: the whole left spine 4, 2, 1 and the single node 7.
assert(count_paths_with_sum(&n4, 7) == 2);
assert(count_paths_with_sum(&n4, 100) == 0);

// A left-leaning chain is as unbalanced as a tree gets, and every value is on one path.
TreeNode<int> a(1), b(2), c(3);
c.left = &b;
b.left = &a;
assert(!is_balanced(&c) && height(&c) == 3 && diameter(&c) == 2);

// Negative values: only branches that add something join the best path.
TreeNode<int> neg(-5), worse(-9);
neg.left = &worse;
assert(max_path_sum(&neg) == -5);

TreeNode<int> m1(1), m2(2), m3(3);
m2.left = &m1;
m2.right = &m3;
assert(same_tree(&m2, &n2)); // A separate copy of the subtree rooted at 2.
invert(&m2);
assert(m2.left == &m3 && m2.right == &m1 && !same_tree(&m2, &n2));
return 0;
}

```

## 2.3 Heaps and Priority Queues

### 2.3.1 Binary Heap

Maintains a priority queue, returning the minimum element by default. The comparator `comp` defines priority: `comp(a, b)` is true when `a` has higher priority than `b`, so `std::greater<T>` instead creates a max-priority queue. A binary heap stores a heap-ordered binary tree implicitly in an array, where the children of the node at index  $i$  live at indices  $2i + 1$  and  $2i + 2$ . Insertion appends the new element and bubbles it up toward the root, while



extraction moves the last element to the root and sifts it down. To customize the ordering, instantiate `BinaryHeap<T, Compare>` and pass the comparator to either constructor.

- `BinaryHeap<T>()` constructs an empty priority queue.
- `BinaryHeap<T>(lo, hi)` constructs a priority queue from the elements in the half-open iterator range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the highest-priority element from the priority queue.
- `top()` returns the highest-priority element in the priority queue.

#### Time Complexity:

- $O(1)$  per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$  per call to `push()` and `pop()`, where  $n$  is the number of elements in the priority queue.
- $O(n)$  per call to the second constructor on the distance between `lo` and `hi` (bottom-up heapify).

#### Space Complexity:

- $O(n)$  for storage of the priority queue elements.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <functional>
#include <utility>
#include <vector>

template<typename T, typename Compare = std::less<T>>
class BinaryHeap {
    std::vector<T> heap;
    Compare comp;

    void sift_down(int i) {
        while (true) {
            int child = i * 2 + 1;
            if (child >= static_cast<int>(heap.size())) {
                break;
            }
            if (child + 1 < static_cast<int>(heap.size()) && comp(heap[child + 1], heap[child])) {
                child++;
            }
            if (comp(heap[child], heap[i])) {
                std::swap(heap[i], heap[child]);
                i = child;
            } else {
                break;
            }
        }
    }

public:
    explicit BinaryHeap(Compare comp = Compare{}) : comp(std::move(comp)) {}

    template<typename It>
    BinaryHeap(It lo, It hi, Compare comp = Compare{}) : heap(lo, hi), comp(std::move(comp)) {
        for (int i = static_cast<int>(heap.size()) / 2 - 1; i >= 0; i--) {
            sift_down(i);
        }
    }

    int size() const { return static_cast<int>(heap.size()); }
    bool empty() const { return heap.empty(); }

    void push(const T &v) {
        heap.push_back(v);
        int i = static_cast<int>(heap.size()) - 1;

```

```

while (i > 0) {
    int parent = (i - 1) / 2;
    if (!comp(heap[i], heap[parent])) {
        break;
    }
    std::swap(heap[i], heap[parent]);
    i = parent;
}

void pop() {
    assert(!heap.empty());
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) {
        sift_down(0);
    }
}

const T &top() const {
    assert(!heap.empty());
    return heap[0];
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{0, 5, -1, 12};
    BinaryHeap<int> h(a.begin(), a.end());
    assert(h.size() == 4);
    assert(h.top() == -1);
    h.push(10);
    vector<int> popped;
    while (!h.empty()) {
        popped.push_back(h.top());
        h.pop();
    }
    assert((popped == vector<int>{-1, 0, 5, 10, 12}));
    assert(h.empty());

    BinaryHeap<int, greater<int>> max_heap;
    max_heap.push(1);
    max_heap.push(3);
    assert(max_heap.top() == 3);
    return 0;
}

```

### 2.3.2 Leftist Heap

Maintains a mergeable priority queue, returning the minimum element by default. The comparator `comp` defines priority: `comp(a, b)` is true when `a` has higher priority than `b`, so `std::greater<T>` instead creates a max-priority queue. A leftist heap stores at each node a rank (the length of the shortest path to a missing descendant) and keeps every left child's rank at least that of its right child, so the right spine has length  $O(\log n)$ . Merging walks only this right spine, swapping children where needed to restore the invariant. Every other operation reduces to this merge: insertion merges a one-node heap, and extraction merges the root's two subtrees. Unlike the self-adjusting skew heap, the explicit rank bound makes every operation  $O(\log n)$  in the worst case rather than amortized. To customize the ordering, instantiate `LeftistHeap<T, Compare>` and pass the comparator to either constructor.

- `LeftistHeap<T>()` constructs an empty priority queue.
- `LeftistHeap<T>(lo, hi)` constructs a priority queue from the elements in the half-open iterator range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the highest-priority element from the priority queue.
- `top()` returns the highest-priority element in the priority queue.
- `absorb(h)` inserts every value from the distinct heap `h` and sets `h` to the empty priority queue. Both heaps must use equivalent comparators.

#### Time Complexity:

- $O(1)$  per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$  per call to `push()`, `pop()`, and `absorb()`, where  $n$  is the number of elements in the priority queue.
- $O(n \log n)$  per call to the second constructor, where  $n$  is the distance between `lo` and `hi`.

#### Space Complexity:

- $O(n)$  for storage of the priority queue elements.
- $O(\log n)$  auxiliary stack space for `push()`, `pop()`, and `absorb()`.
- $O(n)$  auxiliary stack space in the worst case for destruction.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <functional>
#include <utility>

template<typename T, typename Compare = std::less<T>>
class LeftistHeap {
    struct Node {
        T value;
        int rank;
        Node *left, *right;

        explicit Node(const T &v) : value(v), rank(1), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;
    Compare comp;

    static int rank(Node *n) { return (n == nullptr) ? 0 : n->rank; }

    Node *merge(Node *a, Node *b) {
        if (a == nullptr) {
            return b;
        }
        if (b == nullptr) {
            return a;
        }
        if (comp(b->value, a->value)) {
            std::swap(a, b);
        }
        a->right = merge(a->right, b);
        if (rank(a->left) < rank(a->right)) {
            std::swap(a->left, a->right);
        }
        a->rank = rank(a->right) + 1;
        return a;
    }

    static void clean_up(Node *n) {
        if (n != nullptr) {
            clean_up(n->left);
            clean_up(n->right);
            delete n;
        }
    }
};
```

```

    }
}

public:
    explicit LeftistHeap(Compare comp = Compare{})
        : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

    template<typename It>
    LeftistHeap(It lo, It hi, Compare comp = Compare{})
        : root(nullptr), num_nodes(0), comp(std::move(comp)) {
        while (lo != hi) {
            push(*(lo++));
        }
    }

    ~LeftistHeap() { clean_up(root); }
    LeftistHeap(const LeftistHeap &) = delete;
    LeftistHeap &operator=(const LeftistHeap &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }

    void push(const T &v) {
        root = merge(root, new Node(v));
        num_nodes++;
    }

    void pop() {
        assert(!empty());
        Node *tmp = root;
        root = merge(root->left, root->right);
        delete tmp;
        num_nodes--;
    }

    const T &top() const {
        assert(!empty());
        return root->value;
    }

    void absorb(LeftistHeap &h) {
        assert(this != &h);
        root = merge(root, h.root);
        num_nodes += h.num_nodes;
        h.root = nullptr;
        h.num_nodes = 0;
    }
};

```

## Example Usage

```

#include <vector>
using namespace std;

int main() {
    LeftistHeap<int> h, h2;
    assert(h.empty());
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    assert(h.size() == 5);
    assert(h2.empty());
    vector<int> popped;
    while (!h.empty()) {

```

```

    popped.push_back(h.top());
    h.pop();
}
assert((popped == vector<int>{-1, 0, 5, 10, 12}));

LeftistHeap<int, greater<int>> max_heap;
max_heap.push(1);
max_heap.push(3);
assert(max_heap.top() == 3);
return 0;
}

```

### 2.3.3 Skew Heap

Maintains a mergeable priority queue, returning the minimum element by default. The comparator `comp` defines priority: `comp(a, b)` is true when `a` has higher priority than `b`, so `std::greater<T>` instead creates a max-priority queue. A skew heap attempts to maintain balance by unconditionally swapping all nodes in the merge path when merging. Every other operation reduces to this merge: insertion merges a one-node heap, and extraction merges the root's two subtrees. To customize the ordering, instantiate `SkewHeap<T, Compare>` and pass the comparator to either constructor.

- `SkewHeap<T>()` constructs an empty priority queue.
- `SkewHeap<T>(lo, hi)` constructs a priority queue from the elements in the half-open iterator range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the highest-priority element from the priority queue.
- `top()` returns the highest-priority element in the priority queue.
- `absorb(h)` inserts every value from the distinct heap `h` and sets `h` to the empty priority queue. Both heaps must use equivalent comparators.

#### Time Complexity:

- $O(1)$  per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$  amortized per call to `push()`, `pop()`, and `absorb()`, where  $n$  is the number of elements in the priority queue.
- $O(n \log n)$  amortized per call to the second constructor, where  $n$  is the distance between `lo` and `hi`.

#### Space Complexity:

- $O(n)$  for storage of the priority queue elements.
- $O(n)$  auxiliary stack space in the worst case for `push()`, `pop()`, `absorb()`, and destruction.
- $O(1)$  auxiliary for all other operations.

```

#include <cassert>
#include <functional>
#include <utility>

template<typename T, typename Compare = std::less<T>>
class SkewHeap {
    struct Node {
        T value;
        Node *left, *right;

        explicit Node(const T &v) : value(v), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;
    Compare comp;

    Node *merge(Node *a, Node *b) {

```

```

    if (a == nullptr) {
        return b;
    }
    if (b == nullptr) {
        return a;
    }
    if (comp(b->value, a->value)) {
        std::swap(a, b);
    }
    std::swap(a->left, a->right);
    a->left = merge(b, a->left);
    return a;
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
explicit SkewHeap(Compare comp = Compare{})
    : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

template<typename It>
SkewHeap(It lo, It hi, Compare comp = Compare{})
    : root(nullptr), num_nodes(0), comp(std::move(comp)) {
    while (lo != hi) {
        push(*(lo++));
    }
}

~SkewHeap() { clean_up(root); }
SkewHeap(const SkewHeap &) = delete;
SkewHeap &operator=(const SkewHeap &) = delete;
int size() const { return num_nodes; }
bool empty() const { return root == nullptr; }

void push(const T &v) {
    root = merge(root, new Node(v));
    num_nodes++;
}

void pop() {
    assert(!empty());
    Node *tmp = root;
    root = merge(root->left, root->right);
    delete tmp;
    num_nodes--;
}

const T &top() const {
    assert(!empty());
    return root->value;
}

void absorb(SkewHeap &h) {
    assert(this != &h);
    root = merge(root, h.root);
    num_nodes += h.num_nodes;
    h.root = nullptr;
    h.num_nodes = 0;
}
};

```

## Example Usage

```
#include <vector>
using namespace std;

int main() {
    SkewHeap<int> h, h2;
    assert(h.empty());
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    assert(h.size() == 5);
    assert(h2.empty());
    vector<int> popped;
    while (!h.empty()) {
        popped.push_back(h.top());
        h.pop();
    }
    assert((popped == vector<int>{-1, 0, 5, 10, 12}));

    SkewHeap<int, greater<int>> max_heap;
    max_heap.push(1);
    max_heap.push(3);
    assert(max_heap.top() == 3);
    return 0;
}
```

### 2.3.4 Pairing Heap

Maintains a mergeable priority queue, returning the minimum element by default. The comparator `comp` defines priority: `comp(a, b)` is true when `a` has higher priority than `b`, so `std::greater<T>` instead creates a max-priority queue. A pairing heap is a heap-ordered multi-way tree, using a two-pass merge to self-adjust during each deletion. Unlike the other heaps in this section, it also supports `improve_key()` (the usual decrease-key operation for a min-heap) and `erase()` via handles returned by `push()`, the feature pairing heaps are best known for.

To customize the ordering, instantiate `PairingHeap<T, Compare>` and pass the comparator to either constructor.

- `PairingHeap<T>()` constructs an empty priority queue.
- `PairingHeap<T>(lo, hi)` constructs a priority queue from the elements in the half-open iterator range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` and returns a handle to the new element. The handle remains valid until that element is removed by `pop()` or `erase()`.
- `pop()` removes the highest-priority element from the priority queue.
- `top()` returns the highest-priority element in the priority queue.
- `absorb(h)` inserts every value from the distinct heap `h` and sets `h` to the empty priority queue. Both heaps must use equivalent comparators.
- `improve_key(n, v)` replaces the value at handle `n` with a value `v` of equal or higher priority.
- `erase(n)` removes the element at handle `n` from the priority queue.

In practice, `improve_key()` runs in near-constant time, making the pairing heap the usual practical substitute for a Fibonacci heap. When handles are not needed, the lazy-deletion idiom (push updated entries and skip stale ones on `pop()`, as the Dijkstra section does) is simpler and often faster. On GNU C++ judges, `__gnu_pbds::priority_queue` with `pairing_heap_tag` provides a shorter handled and meldable heap; see 8.6 for the contest wrapper.

**Time Complexity:**

- $O(1)$  per call to the first constructor, `size()`, `empty()`, `push()`, `top()`, and `absorb()`.
- $O(\log n)$  amortized per call to `pop()` and `erase()`.
- $O(\log n)$  amortized per call to `improve_key()`, a safe bound to budget for. The true amortized cost is sub-logarithmic, though the long-conjectured  $O(1)$  bound is provably impossible.
- $O(n)$  per call to the second constructor on the distance between `lo` and `hi`.

**Space Complexity:**

- $O(n)$  for storage of the priority queue elements.
- $O(n)$  auxiliary stack space in the worst case for `pop()`, `erase()`, and destruction.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <functional>
#include <utility>

template<typename T, typename Compare = std::less<T>>
class PairingHeap {
    struct Node {
        T value;
        Node *left, *next, *prev;

        explicit Node(const T &v) : value(v), left(nullptr), next(nullptr), prev(nullptr) {}

        // prev points to the previous sibling, or to the parent when this is the first child, so any
        // node can be cut from its child list in O(1).
        void add_child(Node *n) {
            n->prev = this;
            n->next = left;
            if (left != nullptr) {
                left->prev = n;
            }
            left = n;
        }
    } *root;

    int num_nodes;
    Compare comp;

    Node *merge(Node *a, Node *b) {
        if (a == nullptr) {
            return b;
        }
        if (b == nullptr) {
            return a;
        }
        if (comp(a->value, b->value)) {
            a->add_child(b);
            a->prev = a->next = nullptr;
            return a;
        }
        b->add_child(a);
        b->prev = b->next = nullptr;
        return b;
    }

    // Detaches non-root node n from its parent's child list.
    static void cut(Node *n) {
        if (n->prev->left == n) {
            n->prev->left = n->next;
        } else {
            n->prev->next = n->next;
        }
        if (n->next != nullptr) {
            n->next->prev = n->prev;
        }
    }
};
```



```

    }
    n->prev = n->next = nullptr;
}

Node *merge_pairs(Node *n) {
    if (n == nullptr || n->next == nullptr) {
        return n;
    }
    Node *a = n, *b = n->next, *c = n->next->next;
    a->next = b->next = nullptr;
    return merge(merge(a, b), merge_pairs(c));
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->next);
        delete n;
    }
}

public:
    explicit PairingHeap(Compare comp = Compare{})
        : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

    template<typename It>
    PairingHeap(It lo, It hi, Compare comp = Compare{})
        : root(nullptr), num_nodes(0), comp(std::move(comp)) {
        while (lo != hi) {
            push(*(lo++));
        }
    }

    ~PairingHeap() { clean_up(root); }
    PairingHeap(const PairingHeap &) = delete;
    PairingHeap &operator=(const PairingHeap &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }

    Node *push(const T &v) {
        Node *n = new Node(v);
        root = merge(root, n);
        num_nodes++;
        return n;
    }

    void pop() {
        assert(!empty());
        Node *tmp = root;
        root = merge_pairs(root->left);
        if (root != nullptr) {
            root->prev = root->next = nullptr;
        }
        delete tmp;
        num_nodes--;
    }

    const T &top() const {
        assert(!empty());
        return root->value;
    }

    void absorb(PairingHeap &h) {
        assert(this != &h);
        root = merge(root, h.root);
        num_nodes += h.num_nodes;
        h.root = nullptr;
        h.num_nodes = 0;
    }

```

```

}

void improve_key(Node *n, const T &v) {
    assert(!comp(n->value, v));
    n->value = v;
    if (n != root) {
        cut(n);
        root = merge(root, n);
    }
}

void erase(Node *n) {
    if (n == root) {
        pop();
        return;
    }
    cut(n);
    root = merge(root, merge_pairs(n->left));
    delete n;
    num_nodes--;
}
};

```

### Example Usage

```

#include <vector>
using namespace std;

int main() {
    PairingHeap<int> h, h2;
    assert(h.empty());
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    assert(h.size() == 5);
    assert(h2.empty());

    auto handle = h.push(100);
    h.improve_key(handle, -5); // 100 -> -5, the new minimum.
    assert(h.top() == -5);
    h.erase(handle); // Remove it again, restoring the original five elements.
    assert(h.size() == 5);

    vector<int> popped;
    while (!h.empty()) {
        popped.push_back(h.top());
        h.pop();
    }
    assert((popped == vector<int>{-1, 0, 5, 10, 12}));

    PairingHeap<int, greater<int>> max_heap;
    auto max_handle = max_heap.push(1);
    max_heap.improve_key(max_handle, 3);
    assert(max_heap.top() == 3);
    return 0;
}

```

## 2.4 Dictionaries and Ordered Sets

### 2.4.1 Treap

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A binary search tree (a.k.a. BST) implements this map by inserting and deleting keys into a binary tree while upholding the BST property: for each node in the BST, every node in its left subtree has a lesser key and every node in its right subtree has a greater key.

A treap is a BST where each node also holds a randomly assigned priority, simultaneously maintaining both the BST property on keys and the min-heap property on priorities (lower priority value is closer to root). Since priorities are random, this keeps the tree balanced with high probability, making insertions and deletions run in  $O(\log n)$  on average.

The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. It defaults to `std::less<K>`; to customize the ordering, instantiate `Treap<K, V, Compare>` and pass the comparator to the constructor.

- `Treap<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in comparator order.
- `min()` and `max()` return the first and last entries in comparator order, or `std::nullopt` if the map is empty.
- `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` return the matching entry according to the comparator, or `std::nullopt` if no such entry exists. These navigation routines depend only on the BST property and can be adapted to the other BST variants later in this chapter.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  on average per call to `insert()`, `erase()`, `find()`, `min()`, `max()`, `lower_bound()`, `upper_bound()`, `prev()`, and `next()`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(\log n)$  auxiliary stack space on average for `insert()`, `erase()`, `entries()`, and destruction, and  $O(n)$  in the worst case.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```
#include <chrono>
#include <stdint>
#include <functional>
#include <optional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class Treap {
    struct Node {
        static uint32_t rand32() {
            static uint32_t x =
                static_cast<uint32_t>(std::chrono::steady_clock::now().time_since_epoch().count()) | 1U;
```

```

    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return x;
}

K key;
V value;
uint32_t priority;
Node *left, *right;

Node(const K &k, const V &v)
    : key(k), value(v), priority(rand32()), left(nullptr), right(nullptr) {}
} *root;

int num_nodes;
Compare comp;

static void rotate_left(Node *&n) {
    Node *tmp = n;
    n = n->right;
    tmp->right = n->left;
    n->left = tmp;
}

static void rotate_right(Node *&n) {
    Node *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
}

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    if (comp(k, n->key) && insert(n->left, k, v)) {
        if (n->left->priority < n->priority) {
            rotate_right(n);
        }
        return true;
    }
    if (comp(n->key, k) && insert(n->right, k, v)) {
        if (n->right->priority < n->priority) {
            rotate_left(n);
        }
        return true;
    }
    return false;
}

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    if (comp(k, n->key)) {
        return erase(n->left, k);
    } else if (comp(n->key, k)) {
        return erase(n->right, k);
    }
    if (n->left != nullptr && n->right != nullptr) {
        if (n->left->priority < n->right->priority) {
            rotate_right(n);
            return erase(n->right, k);
        }
        rotate_left(n);
    }
}

```

```

    return erase(n->left, k);
}
Node *tmp = (n->left != nullptr) ? n->left : n->right;
delete n;
n = tmp;
num_nodes--;
return true;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit Treap(Compare comp = Compare{}) : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

    ~Treap() { clean_up(root); }
    Treap(const Treap &) = delete;
    Treap &operator=(const Treap &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (comp(k, n->key)) {
                n = n->left;
            } else if (comp(n->key, k)) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }

    std::optional<std::pair<K, V>> min() const {
        Node *n = root;
        if (n == nullptr) {
            return std::nullopt;
        }
        while (n->left != nullptr) {
            n = n->left;
        }
        return std::pair<K, V>{n->key, n->value};
    }
}

```

```

std::optional<std::pair<K, V>> max() const {
    Node *n = root;
    if (n == nullptr) {
        return std::nullopt;
    }
    while (n->right != nullptr) {
        n = n->right;
    }
    return std::pair<K, V>{n->key, n->value};
}

std::optional<std::pair<K, V>> lower_bound(const K &k) const {
    Node *n = root, *best = nullptr;
    while (n != nullptr) {
        if (!comp(n->key, k)) {
            best = n;
            n = n->left;
        } else {
            n = n->right;
        }
    }
    return best ? std::make_optional(std::pair{best->key, best->value}) : std::nullopt;
}

std::optional<std::pair<K, V>> upper_bound(const K &k) const {
    Node *n = root, *best = nullptr;
    while (n != nullptr) {
        if (comp(k, n->key)) {
            best = n;
            n = n->left;
        } else {
            n = n->right;
        }
    }
    return best ? std::make_optional(std::pair{best->key, best->value}) : std::nullopt;
}

std::optional<std::pair<K, V>> prev(const K &k) const {
    Node *n = root, *best = nullptr;
    while (n != nullptr) {
        if (comp(n->key, k)) {
            best = n;
            n = n->right;
        } else {
            n = n->left;
        }
    }
    return best ? std::make_optional(std::pair{best->key, best->value}) : std::nullopt;
}

std::optional<std::pair<K, V>> next(const K &k) const { return upper_bound(k); }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Treap<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
}

```

```

assert(!t.insert(4, 'd'));
assert(t.min()->first == 1 && t.min()->second == 'a');
assert(t.max()->first == 5 && t.max()->second == 'e');
assert(t.lower_bound(4)->first == 4);
assert(t.upper_bound(4)->first == 5);
assert(t.prev(4)->first == 3);
assert(t.next(4)->first == 5);
assert(!t.prev(1));
assert(!t.next(5));
assert(
    (t.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
);
assert(t.erase(1));
assert(!t.erase(1));
assert(t.find(1) == nullptr);
assert((t.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));

Treap<int, char, greater<int>> descending;
for (int key : {2, 1, 3}) {
    descending.insert(key, '0' + key);
}
assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
assert(descending.min()->first == 3 && descending.max()->first == 1);
assert(descending.lower_bound(2)->first == 2);
assert(descending.upper_bound(2)->first == 1);
assert(descending.prev(2)->first == 3 && descending.next(2)->first == 1);
assert(descending.erase(2) && descending.find(2) == nullptr);
return 0;
}

```

## 2.4.2 AVL Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. An AVL tree is a binary search tree balanced by height, guaranteeing  $O(\log n)$  worst-case running time in insertions and deletions by making sure that the heights of the left and right subtrees at every node differ by at most 1. Whenever an insertion or deletion breaks this invariant, it is repaired with one or two rotations at each affected node along the search path.

The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. It defaults to `std::less<K>`; to customize the ordering, instantiate `AVLTree<K, V, Compare>` and pass the comparator to the constructor.

- `AVLTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in comparator order.

The comparator-aware navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.4.1 depend only on the BST property and may be adapted here as needed.

### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  per call to `insert()`, `erase()`, and `find()`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

**Space Complexity:**

- $O(n)$  for storage of the map elements.
- $O(\log n)$  auxiliary stack space for `insert()`, `erase()`, `entries()`, and destruction.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```

#include <algorithm>
#include <functional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class AVLTree {
    struct Node {
        K key;
        V value;
        int height;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), height(1), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;
    Compare comp;

    static int height(Node *n) { return (n != nullptr) ? n->height : 0; }

    static void update_height(Node *n) {
        if (n != nullptr) {
            n->height = 1 + std::max(height(n->left), height(n->right));
        }
    }

    static void rotate_left(Node *&n) {
        Node *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
        update_height(tmp);
        update_height(n);
    }

    static void rotate_right(Node *&n) {
        Node *tmp = n;
        n = n->left;
        tmp->left = n->right;
        n->right = tmp;
        update_height(tmp);
        update_height(n);
    }

    static int balance_factor(Node *n) {
        return (n != nullptr) ? (height(n->left) - height(n->right)) : 0;
    }

    static void rebalance(Node *&n) {
        if (n == nullptr) {
            return;
        }
        update_height(n);
        int bf = balance_factor(n);
        if (bf > 1 && balance_factor(n->left) >= 0) {
            rotate_right(n);
        } else if (bf > 1 && balance_factor(n->left) < 0) {
            rotate_left(n->left);
            rotate_right(n);
        }
    }
}

```



```

    } else if (bf < -1 && balance_factor(n->right) <= 0) {
        rotate_left(n);
    } else if (bf < -1 && balance_factor(n->right) > 0) {
        rotate_right(n->right);
        rotate_left(n);
    }
}

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    if ((comp(k, n->key) && insert(n->left, k, v)) || (comp(n->key, k) && insert(n->right, k, v))) {
        rebalance(n);
        return true;
    }
    return false;
}

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    if (!(comp(k, n->key) || comp(n->key, k))) {
        if (n->left != nullptr && n->right != nullptr) {
            Node *tmp = n->right;
            while (tmp->left != nullptr) {
                tmp = tmp->left;
            }
            K successor_key = tmp->key;
            n->key = successor_key;
            n->value = tmp->value;
            if (!erase(n->right, successor_key)) {
                return false;
            }
        } else {
            Node *tmp = (n->left != nullptr) ? n->left : n->right;
            delete n;
            n = tmp;
            num_nodes--;
        }
        rebalance(n);
        return true;
    }
    if ((comp(k, n->key) && erase(n->left, k)) || (comp(n->key, k) && erase(n->right, k))) {
        rebalance(n);
        return true;
    }
    return false;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

```

```

public:
    explicit AVLTree(Compare comp = Compare{}) : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

    ~AVLTree() { clean_up(root); }
    AVLTree(const AVLTree &) = delete;
    AVLTree &operator=(const AVLTree &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (comp(k, n->key)) {
                n = n->left;
            } else if (comp(n->key, k)) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    AVLTree<int, char> t;
    assert(t.empty());
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(!t.empty() && t.size() == 5);
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    assert(t.size() == 5);
    assert(
        (t.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
    );
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    assert(t.size() == 4);
    assert((t.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));

    AVLTree<int, int> deep_successor;
    for (int key : {20, 10, 30, 25, 40, 22}) {
        deep_successor.insert(key, key);
    }
    assert(deep_successor.erase(20));
    assert(

```

```

    (deep_successor.entries() ==
     vector<pair<int, int>>{{10, 10}, {22, 22}, {25, 25}, {30, 30}, {40, 40}})
);

AVLTree<int, char, greater<int>> descending;
for (int key : {2, 1, 3}) {
    descending.insert(key, '0' + key);
}
assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
assert(*descending.find(2) == '2');
assert(descending.erase(2) && descending.find(2) == nullptr);
return 0;
}

```

### 2.4.3 Red-Black Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A red-black tree is a binary search tree balanced by coloring its nodes red or black, then constraining node colors on any simple path from the root to a leaf. Specifically, a red node may never have a red child, and every path from the root to a null leaf must pass through the same number of black nodes, which together bound the tree's height to  $O(\log n)$ . Insertions and deletions repair these invariants by recoloring nodes and performing rotations.

The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. The comparator defaults to `std::less<K>`; to customize the ordering, instantiate `RedBlackTree<K, V, Compare>` and pass the comparator to the constructor. The sentinel node also requires `K` and `V` to be default constructible.

- `RedBlackTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in comparator order.

The comparator-aware navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.4.1 depend only on the BST property and may be adapted here as needed.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  per call to `insert()`, `erase()`, and `find()`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(\log n)$  auxiliary stack space for `entries()` and destruction.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```

#include <functional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class RedBlackTree {

```

```

enum Color { RED, BLACK };
struct Node {
    K key;
    V value;
    Color color;
    Node *left, *right, *parent;

    Node(const K &k, const V &v, Color c)
        : key(k), value(v), color(c), left(nullptr), right(nullptr), parent(nullptr) {}
} *root, *LEAF_NIL;

int num_nodes;
Compare comp;

void rotate_left(Node *n) {
    Node *tmp = n->right;
    if ((n->right = tmp->left) != LEAF_NIL) {
        n->right->parent = n;
    }
    if ((tmp->parent = n->parent) == LEAF_NIL) {
        root = tmp;
    } else if (n->parent->left == n) {
        n->parent->left = tmp;
    } else {
        n->parent->right = tmp;
    }
    tmp->left = n;
    n->parent = tmp;
}

void rotate_right(Node *n) {
    Node *tmp = n->left;
    if ((n->left = tmp->right) != LEAF_NIL) {
        n->left->parent = n;
    }
    if ((tmp->parent = n->parent) == LEAF_NIL) {
        root = tmp;
    } else if (n->parent->right == n) {
        n->parent->right = tmp;
    } else {
        n->parent->left = tmp;
    }
    tmp->right = n;
    n->parent = tmp;
}

void insert_fix(Node *n) {
    while (n->parent->color == RED) {
        Node *parent = n->parent;
        Node *grandparent = n->parent->parent;
        if (parent == grandparent->left) {
            Node *uncle = grandparent->right;
            if (uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                n = grandparent;
            } else {
                if (n == parent->right) {
                    rotate_left(parent);
                    n = parent;
                    parent = n->parent;
                }
                rotate_right(grandparent);
                std::swap(parent->color, grandparent->color);
                break;
            }
        } else if (parent == grandparent->right) {

```

```

    Node *uncle = grandparent->left;
    if (uncle->color == RED) {
        grandparent->color = RED;
        parent->color = BLACK;
        uncle->color = BLACK;
        n = grandparent;
    } else {
        if (n == parent->left) {
            rotate_right(parent);
            n = parent;
            parent = n->parent;
        }
        rotate_left(grandparent);
        std::swap(parent->color, grandparent->color);
        break;
    }
}
}
root->color = BLACK;
}

void replace(Node *n, Node *replacement) {
    if (n->parent == LEAF_NIL) {
        root = replacement;
    } else if (n == n->parent->left) {
        n->parent->left = replacement;
    } else {
        n->parent->right = replacement;
    }
    replacement->parent = n->parent;
}

void erase_fix(Node *n) {
    while (n != root && n->color == BLACK) {
        Node *parent = n->parent;
        if (n == parent->left) {
            Node *sibling = parent->right;
            if (sibling->color == RED) {
                sibling->color = BLACK;
                parent->color = RED;
                rotate_left(parent);
                sibling = parent->right;
            }
            if (sibling->left->color == BLACK && sibling->right->color == BLACK) {
                sibling->color = RED;
                n = parent;
            } else {
                if (sibling->right->color == BLACK) {
                    sibling->left->color = BLACK;
                    sibling->color = RED;
                    rotate_right(sibling);
                    sibling = parent->right;
                }
                sibling->color = parent->color;
                parent->color = BLACK;
                sibling->right->color = BLACK;
                rotate_left(parent);
                n = root;
            }
        } else {
            Node *sibling = parent->left;
            if (sibling->color == RED) {
                sibling->color = BLACK;
                parent->color = RED;
                rotate_right(parent);
                sibling = parent->left;
            }
            if (sibling->left->color == BLACK && sibling->right->color == BLACK) {

```

```

        sibling->color = RED;
        n = parent;
    } else {
        if (sibling->left->color == BLACK) {
            sibling->right->color = BLACK;
            sibling->color = RED;
            rotate_left(sibling);
            sibling = parent->left;
        }
        sibling->color = parent->color;
        parent->color = BLACK;
        sibling->left->color = BLACK;
        rotate_right(parent);
        n = root;
    }
}
}
n->color = BLACK;
}

void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) const {
    if (n != LEAF_NIL) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

void clean_up(Node *n) {
    if (n != LEAF_NIL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
explicit RedBlackTree(Compare comp = Compare{}) : num_nodes(0), comp(std::move(comp)) {
    root = LEAF_NIL = new Node(K{}, V{}, BLACK);
    LEAF_NIL->left = LEAF_NIL->right = LEAF_NIL;
}

~RedBlackTree() {
    clean_up(root);
    delete LEAF_NIL;
}

RedBlackTree(const RedBlackTree &) = delete;
RedBlackTree &operator=(const RedBlackTree &) = delete;
int size() const { return num_nodes; }
bool empty() const { return num_nodes == 0; }

bool insert(const K &k, const V &v) {
    Node *curr = root, *prev = LEAF_NIL;
    while (curr != LEAF_NIL) {
        prev = curr;
        if (comp(k, curr->key)) {
            curr = curr->left;
        } else if (comp(curr->key, k)) {
            curr = curr->right;
        } else {
            return false;
        }
    }
    Node *n = new Node(k, v, RED);
    n->parent = prev;
    if (prev == LEAF_NIL) {
        root = n;
    }
}

```

```

    } else if (comp(k, prev->key)) {
        prev->left = n;
    } else {
        prev->right = n;
    }
    n->left = n->right = LEAF_NIL;
    insert_fix(n);
    num_nodes++;
    return true;
}

bool erase(const K &k) {
    Node *n = root;
    while (n != LEAF_NIL) {
        if (comp(k, n->key)) {
            n = n->left;
        } else if (comp(n->key, k)) {
            n = n->right;
        } else {
            break;
        }
    }
    if (n == LEAF_NIL) {
        return false;
    }
    Color color = n->color;
    Node *replacement;
    if (n->left == LEAF_NIL) {
        replacement = n->right;
        replace(n, n->right);
    } else if (n->right == LEAF_NIL) {
        replacement = n->left;
        replace(n, n->left);
    } else {
        Node *tmp = n->right;
        while (tmp->left != LEAF_NIL) {
            tmp = tmp->left;
        }
        color = tmp->color;
        replacement = tmp->right;
        if (tmp->parent == n) {
            replacement->parent = tmp;
        } else {
            replace(tmp, tmp->right);
            tmp->right = n->right;
            tmp->right->parent = tmp;
        }
        replace(n, tmp);
        tmp->left = n->left;
        tmp->left->parent = tmp;
        tmp->color = n->color;
    }
    delete n;
    if (color == BLACK) {
        erase_fix(replacement);
    }
    num_nodes--;
    return true;
}

const V *find(const K &k) const {
    Node *n = root;
    while (n != LEAF_NIL) {
        if (comp(k, n->key)) {
            n = n->left;
        } else if (comp(n->key, k)) {
            n = n->right;
        } else {

```

```

        return &(n->value);
    }
}
return nullptr;
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_nodes);
    collect_entries(root, res);
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    RedBlackTree<int, char> t;
    assert(t.empty());
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(!t.empty() && t.size() == 5);
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    assert(t.size() == 5);
    assert(
        (t.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
    );
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    assert(t.size() == 4);
    assert((t.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));
    for (int key : {2, 3, 4, 5}) {
        assert(t.erase(key));
    }
    assert(t.empty() && t.size() == 0);

    RedBlackTree<int, char, greater<int>> descending;
    for (int key : {2, 1, 3}) {
        descending.insert(key, '0' + key);
    }
    assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
    assert(*descending.find(2) == '2');
    assert(descending.erase(2) && descending.find(2) == nullptr);
    return 0;
}

```

### 2.4.4 Splay Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A splay tree is a balanced binary search tree with the additional property that recently accessed elements are quick to access again. Every operation “splays” its target node up to the root through a series of rotations, reshaping the tree so that frequently accessed keys settle near the top. A single operation may degrade to  $O(n)$ , but any sequence of operations averages out to  $O(\log n)$  amortized each.



The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. It defaults to `std::less<K>`; to customize the ordering, instantiate `SplayTree<K, V, Compare>` and pass the comparator to the constructor.

- `SplayTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in comparator order.

The comparator-aware navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.4.1 depend only on the BST property and may be adapted here as needed.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  amortized per call to `insert()`, `erase()`, and `find()`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(n)$  auxiliary stack space in the worst case for `insert()`, `erase()`, `find()`, `entries()`, and destruction.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for `size()` and `empty()`.

```
#include <functional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class SplayTree {
    struct Node {
        K key;
        V value;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;
    Compare comp;

    static void rotate_left(Node *&n) {
        Node *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
    }

    static void rotate_right(Node *&n) {
        Node *tmp = n;
        n = n->left;
        tmp->left = n->right;
        n->right = tmp;
    }

    void splay(Node *&n, const K &k) {
        if (n == nullptr) {
```

```

    return;
}
if (comp(k, n->key) && n->left != nullptr) {
    if (comp(k, n->left->key)) {
        splay(n->left->left, k);
        rotate_right(n);
    } else if (comp(n->left->key, k)) {
        splay(n->left->right, k);
        if (n->left->right != nullptr) {
            rotate_left(n->left);
        }
    }
}
if (n->left != nullptr) {
    rotate_right(n);
}
} else if (comp(n->key, k) && n->right != nullptr) {
    if (comp(k, n->right->key)) {
        splay(n->right->left, k);
        if (n->right->left != nullptr) {
            rotate_right(n->right);
        }
    } else if (comp(n->right->key, k)) {
        splay(n->right->right, k);
        rotate_left(n);
    }
    if (n->right != nullptr) {
        rotate_left(n);
    }
}
}
}

```

```

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    splay(n, k);
    if (comp(k, n->key)) {
        Node *tmp = new Node(k, v);
        tmp->left = n->left;
        tmp->right = n;
        n->left = nullptr;
        n = tmp;
    } else if (comp(n->key, k)) {
        Node *tmp = new Node(k, v);
        tmp->left = n;
        tmp->right = n->right;
        n->right = nullptr;
        n = tmp;
    } else {
        return false;
    }
    num_nodes++;
    return true;
}

```

```

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    splay(n, k);
    if (comp(k, n->key) || comp(n->key, k)) {
        return false;
    }
    Node *tmp = n;
    if (n->left == nullptr) {
        n = n->right;
    }
}

```

```

    } else {
        splay(n->left, k);
        n = n->left;
        n->right = tmp->right;
    }
    delete tmp;
    num_nodes--;
    return true;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SplayTree(Compare comp = Compare{})
        : root(nullptr), num_nodes(0), comp(std::move(comp)) {}

    ~SplayTree() { clean_up(root); }
    SplayTree(const SplayTree &) = delete;
    SplayTree &operator=(const SplayTree &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) {
        splay(root, k);
        if (root == nullptr) {
            return nullptr;
        }
        return (comp(k, root->key) || comp(root->key, k)) ? nullptr : &(root->value);
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SplayTree<int, char> t;
    assert(t.empty());
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
}

```

```

assert(t.insert(4, 'd'));
assert(!t.empty() && t.size() == 5);
assert(*t.find(4) == 'd');
assert(!t.insert(4, 'd'));
assert(
    (t.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
);
assert(t.erase(1));
assert(!t.erase(1));
assert(t.find(1) == nullptr);
assert(t.size() == 4);
assert((t.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));

SplayTree<int, char, greater<int>> descending;
for (int key : {2, 1, 3}) {
    descending.insert(key, '0' + key);
}
assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
assert(*descending.find(2) == '2');
assert(descending.erase(2) && descending.find(2) == nullptr);
return 0;
}

```

### 2.4.5 Size Balanced Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. In addition, support queries for keys given their ranks as well as queries for the ranks of given keys. A size balanced tree augments each node with the size of its subtree, using it to maintain balance and compute order statistics. After each update, rotations restore the invariant that every subtree is at least as large as each of its sibling's child subtrees, keeping the height logarithmic.

The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. It defaults to `std::less<K>`; to customize the ordering, instantiate `SBTree<K, V, Compare>` and pass the comparator to the constructor.

- `SBTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `find_by_order(k)` returns a key-value pair of the node with a key of 0-based rank `k`, which must lie in the range `[0, size())`.
- `order_of_key(x)` returns the number of keys that precede `x` in comparator order. The key does not need to be present in the map.
- `entries()` returns all key-value entries in comparator order.

The comparator-aware navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.4.1 depend only on the BST property and may be adapted here as needed. For contest use on GNU C++ judges, PBDS ordered trees provide the same order-statistic operations with much less code; see 8.6. This implementation is useful when PBDS is unavailable or when customizing tree internals.

The order-statistic API matches GNU PBDS naming and 0-based rank conventions.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.

- $O(\log n)$  per call to `insert()`, `erase()`, `find()`, `find_by_order()`, and `order_of_key()`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(\log n)$  auxiliary stack space for `insert()`, `erase()`, `find_by_order()`, `order_of_key()`, `entries()`, and destruction.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <functional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class SBTTree {
    struct Node {
        K key;
        V value;
        int size;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), size(1), left(nullptr), right(nullptr) {}

        inline Node *&child(int c) { return (c == 0) ? left : right; }

        void update() {
            size = 1;
            if (left != nullptr) {
                size += left->size;
            }
            if (right != nullptr) {
                size += right->size;
            }
        }
    } *root;

    Compare comp;

    static inline int size(Node *n) { return (n == nullptr) ? 0 : n->size; }

    static void rotate(Node *&n, int c) {
        Node *tmp = n->child(c);
        n->child(c) = tmp->child(!c);
        tmp->child(!c) = n;
        n->update();
        tmp->update();
        n = tmp;
    }

    static void maintain(Node *&n, int c) {
        if (n == nullptr || n->child(c) == nullptr) {
            return;
        }
        Node *&tmp = n->child(c);
        if (size(tmp->child(c)) > size(n->child(!c))) {
            rotate(n, c);
        } else if (size(tmp->child(!c)) > size(n->child(!c))) {
            rotate(tmp, !c);
            rotate(n, c);
        } else {
            return;
        }
    }
}
```

```

    maintain(n->left, 0);
    maintain(n->right, 1);
    maintain(n, 0);
    maintain(n, 1);
}

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        return true;
    }
    bool found;
    if (comp(k, n->key)) {
        found = insert(n->left, k, v);
        maintain(n, 0);
    } else if (comp(n->key, k)) {
        found = insert(n->right, k, v);
        maintain(n, 1);
    } else {
        return false;
    }
    n->update();
    return found;
}

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    bool found;
    int c = comp(k, n->key);
    if (comp(k, n->key)) {
        found = erase(n->left, k);
    } else if (comp(n->key, k)) {
        found = erase(n->right, k);
    } else {
        if (n->right == nullptr || n->left == nullptr) {
            Node *tmp = n;
            n = (n->right == nullptr) ? n->left : n->right;
            delete tmp;
            return true;
        }
        Node *p = n->right;
        while (p->left != nullptr) {
            p = p->left;
        }
        K successor_key = p->key;
        n->key = successor_key;
        n->value = p->value;
        found = erase(n->right, successor_key);
    }
    maintain(n, c);
    n->update();
    return found;
}

static std::pair<K, V> find_by_order(Node *n, int k) {
    int left_size = size(n->left);
    if (k < left_size) {
        return find_by_order(n->left, k);
    } else if (k > left_size) {
        return find_by_order(n->right, k - left_size - 1);
    }
    return {n->key, n->value};
}

int order_of_key(Node *n, const K &x) const {
    if (n == nullptr) {

```

```

    return 0;
}
if (comp(x, n->key)) {
    return order_of_key(n->left, x);
} else if (comp(n->key, x)) {
    return order_of_key(n->right, x) + size(n->left) + 1;
}
return size(n->left);
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SBTTree(Compare comp = Compare{}) : root(nullptr), comp(std::move(comp)) {}

    ~SBTTree() { clean_up(root); }
    SBTTree(const SBTTree &) = delete;
    SBTTree &operator=(const SBTTree &) = delete;
    int size() const { return size(root); }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (comp(k, n->key)) {
                n = n->left;
            } else if (comp(n->key, k)) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::pair<K, V> find_by_order(int k) const {
        assert(0 <= k && k < size(root));
        return find_by_order(root, k);
    }

    int order_of_key(const K &x) const { return order_of_key(root, x); }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(size(root));
        collect_entries(root, res);
        return res;
    }
};

```

## Example Usage

```
using namespace std;

int main() {
    SBTTree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    assert(
        (t.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
    );
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    assert((t.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));
    assert(t.order_of_key(2) == 0);
    assert(t.order_of_key(3) == 1);
    assert(t.order_of_key(5) == 3);
    assert(t.order_of_key(6) == 4);
    assert(t.find_by_order(0).first == 2);
    assert(t.find_by_order(1).first == 3);
    assert(t.find_by_order(2).first == 4);

    SBTTree<int, char> replacement;
    replacement.insert(2, 'b');
    replacement.insert(1, 'a');
    replacement.insert(3, 'c');
    assert(replacement.erase(2));
    assert(*replacement.find(3) == 'c');

    SBTTree<int, char, greater<int>> descending;
    for (int key : {2, 1, 3}) {
        descending.insert(key, '0' + key);
    }
    assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
    assert(descending.order_of_key(2) == 1);
    assert(descending.find_by_order(0).first == 3);
    assert(descending.erase(2) && descending.find(2) == nullptr);
    return 0;
}
```

## 2.4.6 Interval Treap

Maintain an ordered map from closed, one-dimensional intervals to values while supporting efficient reporting of any or all entries that intersect with a given query interval. A treap is used to process the entries, where keys are compared lexicographically as pairs. Each node is additionally augmented with the maximum upper endpoint among the intervals in its subtree, letting intersection queries skip any subtree whose maximum falls below the query interval's lower bound.

This implementation uses `std::pair` to represent intervals and requires `operator<` on the numeric key type `K`. Every interval must satisfy `!(hi < lo)`.

- `IntervalTreap<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.



- `insert(lo, hi, v)` adds an entry with key `[lo, hi]` and value `v` to the map, returning `true` if a new interval was added or `false` if the interval already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(lo, hi)` removes the entry with key `[lo, hi]` from the map, returning `true` if the removal was successful or `false` if the interval was not found.
- `find_key(lo, hi)` returns a pointer to a const `std::pair` representing the key of some interval in the map which intersects with `[lo, hi]`, or `nullptr` if no such entry was found.
- `find_value(lo, hi)` returns a pointer to a const value of some entry in the map with a key that intersects with `[lo, hi]`, or `nullptr` if no such entry was found.
- `find_all(lo, hi)` returns all entries that overlap `[lo, hi]` in lexicographically ascending order of intervals.
- `entries()` returns all intervals and their values in lexicographically ascending order of intervals.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  on average per call to `insert()`, `erase()`, `find_key()`, and `find_value()`, where  $n$  is the number of intervals currently in the set.
- $O(\log n + m)$  on average per call to `find_all()`, where  $m$  is the number of intersecting intervals that are reported.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(1)$  auxiliary for `size()` and `empty()`.
- $O(\log n)$  auxiliary stack space on average for all other operations and destruction, and  $O(n)$  in the worst case.
- $O(n)$  for the vector returned by `entries()`.
- $O(m)$  for the vector returned by `find_all()`, where  $m$  is the number of intersecting intervals.

```
#include <cassert>
#include <chrono>
#include <cstdint>
#include <tuple>
#include <utility>
#include <vector>

template<typename K, typename V>
class IntervalTreap {
    using Interval = std::pair<K, K>;

    struct Node {
        static uint32_t rand32() {
            static uint32_t x =
                static_cast<uint32_t>(std::chrono::steady_clock::now().time_since_epoch().count()) | 1U;
            x ^= x << 13;
            x ^= x >> 17;
            x ^= x << 5;
            return x;
        }
    };

    Interval interval;
    V value;
    K max;
    uint32_t priority;
    Node *left, *right;

    Node(const Interval &i, const V &v)
        : interval(i), value(v), max(i.second), priority(rand32()), left(nullptr), right(nullptr) {}

    void update() {
        max = interval.second;
        if (left != nullptr && max < left->max) {
```

```

        max = left->max;
    }
    if (right != nullptr && max < right->max) {
        max = right->max;
    }
}
} *root;

int num_nodes;

static void rotate_left(Node *&n) {
    Node *tmp = n;
    n = n->right;
    tmp->right = n->left;
    n->left = tmp;
    tmp->update();
}

static void rotate_right(Node *&n) {
    Node *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
    tmp->update();
}

bool insert(Node *&n, const Interval &i, const V &v) {
    if (n == nullptr) {
        n = new Node(i, v);
        num_nodes++;
        return true;
    }
    if (i < n->interval && insert(n->left, i, v)) {
        if (n->left->priority < n->priority) {
            rotate_right(n);
        }
        n->update();
        return true;
    }
    if (n->interval < i && insert(n->right, i, v)) {
        if (n->right->priority < n->priority) {
            rotate_left(n);
        }
        n->update();
        return true;
    }
    return false;
}

bool erase(Node *&n, const Interval &i) {
    if (n == nullptr) {
        return false;
    }
    if (i < n->interval) {
        bool res = erase(n->left, i);
        if (res) {
            n->update();
        }
        return res;
    }
    if (n->interval < i) {
        bool res = erase(n->right, i);
        if (res) {
            n->update();
        }
        return res;
    }
    if (n->left != nullptr && n->right != nullptr) {

```

```

    bool res;
    if (n->left->priority < n->right->priority) {
        rotate_right(n);
        res = erase(n->right, i);
    } else {
        rotate_left(n);
        res = erase(n->left, i);
    }
    n->update();
    return res;
}

Node *tmp = (n->left != nullptr) ? n->left : n->right;
delete n;
n = tmp;
num_nodes--;
return true;
}

static Node *find_any(Node *n, const Interval &i) {
    if (n == nullptr) {
        return nullptr;
    }
    if (!(i.second < n->interval.first) && !(n->interval.second < i.first)) {
        return n;
    }
    if (n->left != nullptr && !(n->left->max < i.first)) {
        return find_any(n->left, i);
    }
    return find_any(n->right, i);
}

static void find_all(Node *n, const Interval &i, std::vector<std::tuple<K, K, V>> &res) {
    if (n == nullptr || n->max < i.first) {
        return;
    }
    find_all(n->left, i, res);
    if (!(i.second < n->interval.first) && !(n->interval.second < i.first)) {
        res.emplace_back(n->interval.first, n->interval.second, n->value);
    }
    if (!(i.second < n->interval.first)) {
        find_all(n->right, i, res);
    }
}

static void collect_entries(Node *n, std::vector<std::tuple<K, K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->interval.first, n->interval.second, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
IntervalTreap() : root(nullptr), num_nodes(0) {}

~IntervalTreap() { clean_up(root); }
IntervalTreap(const IntervalTreap &) = delete;
IntervalTreap &operator=(const IntervalTreap &) = delete;
int size() const { return num_nodes; }
bool empty() const { return root == nullptr; }

```

```

bool insert(const K &lo, const K &hi, const V &v) {
    assert(!(hi < lo));
    return insert(root, Interval{lo, hi}, v);
}

bool erase(const K &lo, const K &hi) {
    assert(!(hi < lo));
    return erase(root, Interval{lo, hi});
}

const Interval *find_key(const K &lo, const K &hi) const {
    assert(!(hi < lo));
    Node *n = find_any(root, Interval{lo, hi});
    return (n == nullptr) ? nullptr : &(n->interval);
}

const V *find_value(const K &lo, const K &hi) const {
    assert(!(hi < lo));
    Node *n = find_any(root, Interval{lo, hi});
    return (n == nullptr) ? nullptr : &(n->value);
}

std::vector<std::tuple<K, K, V>> find_all(const K &lo, const K &hi) const {
    assert(!(hi < lo));
    std::vector<std::tuple<K, K, V>> res;
    find_all(root, Interval{lo, hi}, res);
    return res;
}

std::vector<std::tuple<K, K, V>> entries() const {
    std::vector<std::tuple<K, K, V>> res;
    res.reserve(num_nodes);
    collect_entries(root, res);
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    IntervalTreap<int, char> t;
    t.insert(15, 20, 'a');
    t.insert(10, 30, 'b');
    t.insert(17, 19, 'c');
    t.insert(5, 20, 'd');
    t.insert(12, 15, 'e');
    t.insert(10, 40, 'f');
    assert(t.size() == 6);
    assert(!t.insert(5, 20, 'x'));
    t.erase(17, 19);
    assert(t.size() == 5);
    assert(*t.find_key(3, 9) == (pair<int, int>{5, 20}));
    assert(*t.find_value(3, 9) == 'd');
    assert(
        (t.find_all(16, 20) ==
         vector<tuple<int, int, char>>{{5, 20, 'd'}, {10, 30, 'b'}, {10, 40, 'f'}, {15, 20, 'a'}})
    );
    assert(
        (t.entries() == vector<tuple<int, int, char>>{
            {5, 20, 'd'}, {10, 30, 'b'}, {10, 40, 'f'}, {12, 15, 'e'}, {15, 20, 'a'}
        })
    );
};

```

```

return 0;
}

```

### 2.4.7 B-Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A B-tree is a balanced search tree whose nodes hold many entries instead of one. Every node except the root stores between  $d - 1$  and  $2d - 1$  entries for a chosen minimum degree  $d$ , an internal node with  $k$  entries has exactly  $k + 1$  children, and every leaf sits at the same depth. The height is  $O(\log_d(n))$ , and a search examines only that many nodes.

The point is the unit of cost. When reading a node is far more expensive than comparing keys inside one (as with disk blocks or cache lines), the height is what matters rather than the comparison count, and packing hundreds of entries per node makes a tree of billions of keys three or four nodes deep. This is why B-trees hold filesystem and database indexes while BSTs hold in-memory indexes.

Balance is maintained without any rebalancing pass. Descending to insert, the search splits every full node it meets, so a node always has room for the median that a split of its child pushes up, and the split cannot cascade back upward. Descending to erase, it does the reverse: before entering a child with only  $d - 1$  entries, it borrows one from an adjacent sibling, or merges the child with one, so that a deletion never leaves a node underfull. Each pass therefore visits each level once.

- `BTree<K, V, MIN_DEGREE>()` constructs an empty map, where the minimum degree must be at least 2 and defaults to 3.
- `size()` and `empty()` return the number of entries and whether the map is empty.
- `height()` returns the number of levels, which is 0 for an empty map.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `insert(k, v)` adds an entry with key `k` and value `v`, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k`, returning whether it was present.
- `entries()` returns all key-value entries in sorted order.

Only `operator<` is required of `K`. The tree grows in height solely by splitting a full root, and shrinks solely when the root's last two children merge, which is what keeps every leaf at one depth.

For an in-memory ordered dictionary the treap of section 2.4.1 or the standard `std::map` is smaller, simpler, and faster; reach for this shape when node reads dominate, or when a problem statement describes a tree of exactly this kind.

#### Time Complexity:

- $O(d \log_d(n))$  per call to `insert()`, `erase()`, and `find()`, where  $n$  is the number of entries. The searches within a node are linear in  $d$ , which is negligible against the cost of reaching the node in the setting the structure is designed for.
- $O(1)$  per call to `size()`, `empty()`, and `height()`.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the entries.
- $O(\log_d(n))$  auxiliary stack space per operation, and  $O(n)$  for the vector returned by `entries()`.

```

#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

```

```

template<typename K, typename V, int MIN_DEGREE = 3>

```

```

class BTree {
    static_assert(MIN_DEGREE >= 2, "a B-tree node must hold at least one entry");

    using Entry = std::pair<K, V>;

    struct Node {
        std::vector<Entry> entries;
        std::vector<Node*> children;

        bool leaf() const { return children.empty(); }
        bool full() const { return static_cast<int>(entries.size()) == 2 * MIN_DEGREE - 1; }
        bool minimal() const { return static_cast<int>(entries.size()) < MIN_DEGREE; }
    };

    Node *root;
    int num_nodes, levels;

    // Returns the first position whose key is not less than the target.
    static int lower_position(const Node *n, const K &key) {
        auto by_key = [](const Entry &e, const K &k) { return e.first < k; };
        auto it = std::lower_bound(n->entries.begin(), n->entries.end(), key, by_key);
        return static_cast<int>(it - n->entries.begin());
    }

    // Splits the full child at i, lifting its median entry into the parent, which must not be full.
    static void split_child(Node *parent, int i) {
        Node *left = parent->children[i], *right = new Node();
        right->entries.assign(left->entries.begin() + MIN_DEGREE, left->entries.end());
        if (!left->leaf()) {
            right->children.assign(left->children.begin() + MIN_DEGREE, left->children.end());
            left->children.resize(MIN_DEGREE);
        }
        parent->entries.insert(parent->entries.begin() + i, left->entries[MIN_DEGREE - 1]);
        parent->children.insert(parent->children.begin() + i + 1, right);
        left->entries.resize(MIN_DEGREE - 1);
    }

    static void insert_nonfull(Node *n, const Entry &entry) {
        int i = lower_position(n, entry.first);
        if (n->leaf()) {
            n->entries.insert(n->entries.begin() + i, entry);
            return;
        }
        if (n->children[i]->full()) {
            split_child(n, i);
            if (n->entries[i].first < entry.first) { // The lifted median now sits before the key.
                i++;
            }
        }
        insert_nonfull(n->children[i], entry);
    }

    // Restores at least MIN_DEGREE entries in child i by borrowing from a sibling or merging with
    // one, then returns the child to descend into, which merging may shift left.
    static Node *ensure_not_minimal(Node *parent, int i) {
        Node *child = parent->children[i];
        if (!child->minimal()) {
            return child;
        }
        Node *left = i > 0 ? parent->children[i - 1] : nullptr;
        Node *right =
            i + 1 < static_cast<int>(parent->children.size()) ? parent->children[i + 1] : nullptr;
        if (left != nullptr && !left->minimal()) { // Rotate an entry across through the parent.
            child->entries.insert(child->entries.begin(), parent->entries[i - 1]);
            parent->entries[i - 1] = left->entries.back();
            left->entries.pop_back();
            if (!left->leaf()) {
                child->children.insert(child->children.begin(), left->children.back());
            }
        }
    }
}

```

```

    left->children.pop_back();
}
return child;
}
if (right != nullptr && !right->minimal()) {
    child->entries.push_back(parent->entries[i]);
    parent->entries[i] = right->entries.front();
    right->entries.erase(right->entries.begin());
    if (!right->leaf()) {
        child->children.push_back(right->children.front());
        right->children.erase(right->children.begin());
    }
    return child;
}
// Both neighbors are minimal, so fold the separating entry and one sibling into the child.
if (left != nullptr) {
    left->entries.push_back(parent->entries[i - 1]);
    left->entries.insert(left->entries.end(), child->entries.begin(), child->entries.end());
    left->children.insert(left->children.end(), child->children.begin(), child->children.end());
    parent->entries.erase(parent->entries.begin() + i - 1);
    parent->children.erase(parent->children.begin() + i);
    delete child;
    return left;
}
child->entries.push_back(parent->entries[i]);
child->entries.insert(child->entries.end(), right->entries.begin(), right->entries.end());
child->children.insert(child->children.end(), right->children.begin(), right->children.end());
parent->entries.erase(parent->entries.begin() + i);
parent->children.erase(parent->children.begin() + i + 1);
delete right;
return child;
}

// The key is taken by value: callers pass a key stored in the tree, and a merge below may erase
// from the very vector a reference would point into.
static bool erase(Node *n, K key) {
    int i = lower_position(n, key);
    bool here = i < static_cast<int>(n->entries.size()) && !(key < n->entries[i].first);
    if (n->leaf()) {
        if (!here) {
            return false;
        }
        n->entries.erase(n->entries.begin() + i);
        return true;
    }
    if (!here) {
        return erase(ensure_not_minimal(n, i), key);
    }
    if (!n->children[i]->minimal()) { // Replace with the predecessor and delete that instead.
        Node *walk = n->children[i];
        while (!walk->leaf()) {
            walk = walk->children.back();
        }
        n->entries[i] = walk->entries.back();
        return erase(n->children[i], n->entries[i].first);
    }
    if (!n->children[i + 1]->minimal()) { // Or the successor.
        Node *walk = n->children[i + 1];
        while (!walk->leaf()) {
            walk = walk->children.front();
        }
        n->entries[i] = walk->entries.front();
        return erase(n->children[i + 1], n->entries[i].first);
    }
    // Both children are minimal, so fold them and the entry between them into one node, which is
    // where the entry now lives. Borrowing would rotate a different entry down and lose this one.
    Node *left = n->children[i], *right = n->children[i + 1];
    left->entries.push_back(n->entries[i]);

```

```

    left->entries.insert(left->entries.end(), right->entries.begin(), right->entries.end());
    left->children.insert(left->children.end(), right->children.begin(), right->children.end());
    n->entries.erase(n->entries.begin() + i);
    n->children.erase(n->children.begin() + i + 1);
    delete right;
    return erase(left, key);
}

static void collect(const Node *n, std::vector<Entry> *out) {
    for (int i = 0; i < static_cast<int>(n->entries.size()); i++) {
        if (!n->leaf()) {
            collect(n->children[i], out);
        }
        out->push_back(n->entries[i]);
    }
    if (!n->leaf()) {
        collect(n->children.back(), out);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        for (Node *child : n->children) {
            clean_up(child);
        }
        delete n;
    }
}

public:
    BTree() : root(new Node()), num_nodes(0), levels(1) {}

    BTree(const BTree &) = delete;
    BTree &operator=(const BTree &) = delete;
    ~BTree() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return num_nodes == 0; }
    int height() const { return num_nodes == 0 ? 0 : levels; }

    const V *find(const K &k) const {
        for (const Node *n = root; n != nullptr;) {
            int i = lower_position(n, k);
            if (i < static_cast<int>(n->entries.size()) && !(k < n->entries[i].first)) {
                return &n->entries[i].second;
            }
            n = n->leaf() ? nullptr : n->children[i];
        }
        return nullptr;
    }

    bool insert(const K &k, const V &v) {
        if (find(k) != nullptr) {
            return false;
        }
        if (root->full()) { // The only way the tree grows taller.
            Node *grown = new Node();
            grown->children.push_back(root);
            split_child(grown, 0);
            root = grown;
            levels++;
        }
        insert_nonfull(root, Entry(k, v));
        num_nodes++;
        return true;
    }

    bool erase(const K &k) {
        // The descent merges nodes on the way down whether or not the key turns out to be present,

```



```

    // so an emptied root must be collapsed even when nothing was removed.
    bool removed = erase(root, k);
    if (root->entries.empty() && !root->leaf()) { // The only way the tree grows shorter.
        Node *old = root;
        root = root->children[0];
        old->children.clear();
        delete old;
        levels--;
    }
    num_nodes -= removed;
    return removed;
}

std::vector<Entry> entries() const {
    std::vector<Entry> res;
    res.reserve(num_nodes);
    collect(root, &res);
    return res;
}
};

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    BTree<int, int> tree; // Minimum degree 3, so a node holds 2 to 5 entries.
    assert(tree.empty() && tree.height() == 0 && tree.find(1) == nullptr);

    for (int x : {10, 20, 5, 6, 12, 30, 7, 17}) {
        assert(tree.insert(x, x * x));
    }
    assert(!tree.insert(10, 0)); // Duplicate keys are rejected and the old value preserved.
    assert(tree.size() == 8 && *tree.find(10) == 100 && tree.find(18) == nullptr);

    vector<int> keys;
    for (const auto &entry : tree.entries()) {
        keys.push_back(entry.first);
    }
    assert((keys == vector<int>{5, 6, 7, 10, 12, 17, 20, 30}));

    // Enough entries to split the root, which is the only way the height changes.
    BTree<int, int> deep;
    for (int x = 0; x < 100; x++) {
        deep.insert(x, -x);
    }
    assert(deep.size() == 100 && deep.height() == 4);
    assert(deep.entries().front().first == 0 && deep.entries().back().first == 99);

    // Erasing from an internal node pulls up a neighbouring entry rather than leaving a gap.
    assert(tree.erase(10) && !tree.erase(10));
    assert(tree.size() == 7 && tree.find(10) == nullptr && *tree.find(12) == 144);
    for (int x : {5, 6, 7, 12, 17, 20, 30}) {
        assert(tree.erase(x));
    }
    assert(tree.empty() && tree.entries().empty());

    // Deleting every entry in order collapses the height back to one level.
    for (int x = 0; x < 100; x++) {
        assert(deep.erase(x));
    }
    assert(deep.empty() && deep.height() == 0);
    return 0;
}

```

### 2.4.8 Skip List

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. The comparator `comp` defines the key ordering: `comp(a, b)` is true when `a` precedes `b`. A skip list maintains a linked hierarchy of sorted subsequences with each successive subsequence skipping over fewer elements than the previous one. Each new node joins a number of levels decided by repeated coin flips, and a search starts at the sparsest level, moving forward until the next key would overshoot and then dropping down a level, which makes operations take  $O(\log n)$  with high probability. The comparator defaults to `std::less<K>`; to customize the ordering, instantiate `SkipList<K, V, Compare>` and pass the comparator to the constructor.

- `SkipList<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the const value associated with key `k`, or `nullptr` if the key was not found.
- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `entries()` returns all key-value entries in comparator order.

The sentinel node requires `K` and `V` to be default constructible.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$  on average per call to `insert()`, `erase()`, `find()`, and `operator[]`, where  $n$  is the number of entries currently in the map.
- $O(n)$  per call to `entries()`.

#### Space Complexity:

- $O(n)$  for storage of the map elements.
- $O(1)$  auxiliary for all operations because the maximum number of levels is fixed at 32.
- $O(n)$  for the vector returned by `entries()`.

```
#include <algorithm>
#include <chrono>
#include <functional>
#include <random>
#include <utility>
#include <vector>

template<typename K, typename V, typename Compare = std::less<K>>
class SkipList {
    static const int MAX_LEVELS = 32; // log2(max possible keys)

    struct Node {
        K key;
        V value;
        std::vector<Node*> next;

        Node(const K& k, const V& v, int levels) : key(k), value(v), next(levels, nullptr) {}
    } *head;

    int num_nodes;
    Compare comp;

    static int rand_level() {
        static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
        static std::uniform_int_distribution<int> coin(0, 1);
```

```

    int level = 1;
    while (coin(rng) && level < MAX_LEVELS) {
        level++;
    }
    return level;
}

static int node_level(const std::vector<Node *> &v) {
    int i = 0;
    while (i < static_cast<int>(v.size()) && v[i] != nullptr) {
        i++;
    }
    return std::max(1, i);
}

Node *find_node(const K &k) const {
    Node *n = head;
    for (int i = node_level(n->next); i-- > 0;) {
        while (n->next[i] != nullptr && comp(n->next[i]->key, k)) {
            n = n->next[i];
        }
    }
    n = n->next[0];
    return (n != nullptr && !comp(k, n->key) && !comp(n->key, k)) ? n : nullptr;
}

public:
explicit SkipList(Compare comp = Compare{})
    : head(new Node(K{}, V{}, MAX_LEVELS)), num_nodes(0), comp(std::move(comp)) {
    for (auto &ptr : head->next) {
        ptr = nullptr;
    }
}

~SkipList() {
    Node *n = head;
    while (n != nullptr) {
        Node *next = n->next[0];
        delete n;
        n = next;
    }
}

SkipList(const SkipList &) = delete;
SkipList &operator=(const SkipList &) = delete;
int size() const { return num_nodes; }
bool empty() const { return num_nodes == 0; }

bool insert(const K &k, const V &v) {
    std::vector<Node *> update(head->next);
    int curr_level = node_level(update);
    Node *n = head;
    for (int i = curr_level; i-- > 0;) {
        while (n->next[i] != nullptr && comp(n->next[i]->key, k)) {
            n = n->next[i];
        }
    }
    update[i] = n;
}
n = n->next[0];
if (n != nullptr && !comp(k, n->key) && !comp(n->key, k)) {
    return false;
}
int new_level = rand_level();
if (new_level > curr_level) {
    for (int i = curr_level; i < new_level; i++) {
        update[i] = head;
    }
}
}

```

```

    n = new Node(k, v, new_level);
    for (int i = 0; i < new_level; i++) {
        n->next[i] = update[i]->next[i];
        update[i]->next[i] = n;
    }
    num_nodes++;
    return true;
}

bool erase(const K &k) {
    std::vector<Node *> update(head->next);
    Node *n = head;
    for (int i = node_level(update); i-- > 0;) {
        while (n->next[i] != nullptr && comp(n->next[i]->key, k)) {
            n = n->next[i];
        }
        update[i] = n;
    }
    n = n->next[0];
    if (n != nullptr && !comp(k, n->key) && !comp(n->key, k)) {
        for (int i = 0; i < static_cast<int>(n->next.size()); i++) {
            update[i]->next[i] = n->next[i];
        }
        delete n;
        num_nodes--;
        return true;
    }
    return false;
}

const V *find(const K &k) const {
    Node *n = find_node(k);
    return n == nullptr ? nullptr : &(n->value);
}

V &operator[] (const K &k) {
    Node *n = find_node(k);
    if (n != nullptr) {
        return n->value;
    }
    insert(k, V{});
    return find_node(k)->value;
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_nodes);
    Node *n = head->next[0];
    while (n != nullptr) {
        res.emplace_back(n->key, n->value);
        n = n->next[0];
    }
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Skiplist<int, char> l;
    assert(l.empty());
    l.insert(2, 'b');
    l.insert(1, 'a');
}

```

```

l.insert(3, 'c');
l.insert(5, 'e');
assert(l.insert(4, 'd'));
assert(!l.empty() && l.size() == 5);
assert(*l.find(4) == 'd');
assert(!l.insert(4, 'd'));
assert(l.size() == 5);
assert(
    (l.entries() == vector<pair<int, char>>{{1, 'a'}, {2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}})
);
assert(l.erase(1));
assert(!l.erase(1));
assert(l.find(1) == nullptr);
assert(l.size() == 4);
assert((l.entries() == vector<pair<int, char>>{{2, 'b'}, {3, 'c'}, {4, 'd'}, {5, 'e'}}));

Skiplist<int, char, greater<int>> descending;
for (int key : {2, 1, 3}) {
    descending.insert(key, '0' + key);
}
assert((descending.entries() == vector<pair<int, char>>{{3, '3'}, {2, '2'}, {1, '1'}}));
assert(*descending.find(2) == '2');
assert(descending.erase(2) && descending.find(2) == nullptr);
return 0;
}

```

### 2.4.9 Hash Table (Chaining)

Maintain an unordered map: a collection of key-value pairs in which each key appears at most once. Keys are hashed into buckets, and this implementation resolves collisions by chaining the entries in each bucket into a linked list. The hash and key-equality policies follow the conventions of `std::unordered_map`.

Compared with the open-addressing version (2.4.10), chaining keeps each entry at a stable address: a rehash never moves existing nodes, so pointers from `find()` and references from `operator[]` stay valid, and load factors above 1 are tolerated gracefully. The costs are a separate allocation per entry and cache-unfriendly pointer chasing during traversal.

- `ChainingHashMap<K, V>(buckets = 128)` constructs an empty map with the positive number of buckets given by `buckets`, using `std::hash<K>` and `std::equal_to<K>`.
- `ChainingHashMap<K, V, Hash, KeyEqual>(buckets, hash, equal)` instead stores the supplied hash and equality policies.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the value associated with key `k`, or `nullptr` if the key was not found.
- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `entries()` returns all key-value entries in no guaranteed order.

This is an educational implementation; in practice prefer one of the standard options:

- `std::unordered_map` is the portable standard-library hash map. It is also chaining-based, so this class is essentially a transparent version of it. Pass a custom hash (see 3.2.1) to harden it against adversarial inputs, as its default integer hash is effectively the identity.

- `__gnu_pbds::gp_hash_table` (from GCC's policy-based library) is an open-addressing table that is typically several times faster than `std::unordered_map`, but is a non-portable GNU extension. Prefer it on GCC-only judges when hashing is the bottleneck; see 8.6 for `HashMap` / `HashSet` wrappers with a randomized integer hash.

#### Time Complexity:

- $O(b)$  per call to the constructor, where  $b$  is the initial number of buckets.
- $O(1)$  per call to `size()` and `empty()`.
- $O(1)$  expected amortized per call to `insert()`, `erase()`, `find()`, and `operator[]`, and  $O(n)$  in the collision-heavy worst case.
- $O(n + b)$  per call to `entries()`, where  $n$  is the number of entries and  $b$  is the number of buckets.

#### Space Complexity:

- $O(n + b)$  for storage of the map elements and buckets.
- $O(n + b)$  auxiliary during a rehash.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <cstddef>
#include <cstdint>
#include <functional>
#include <list>
#include <utility>
#include <vector>

template<typename K, typename V, typename Hash = std::hash<K>, typename KeyEqual = std::equal_to<K>>
class ChainingHashMap {
    struct HashNode {
        K key;
        V value;
        HashNode(const K &k, const V &v) : key(k), value(v) {}
    };

    std::vector<std::list<HashNode>> table;
    int table_size, num_entries;
    Hash hash;
    KeyEqual equal;

    int bucket(const K &k) const {
        return static_cast<int>(hash(k) % static_cast<std::size_t>(table_size));
    }

    void grow_and_rehash() {
        auto old = std::move(table);
        table_size = 2 * table_size;
        table.assign(table_size, std::list<HashNode>{});
        for (auto &chain : old) {
            while (!chain.empty()) {
                auto it = chain.begin();
                int i = bucket(it->key);
                table[i].splice(table[i].end(), chain, it);
            }
        }
    }

public:
    explicit ChainingHashMap(int buckets = 128, Hash hash = Hash{}, KeyEqual equal = KeyEqual{})
        : table_size(buckets), num_entries(0), hash(std::move(hash)), equal(std::move(equal)) {
        assert(buckets > 0);
        table.resize(buckets);
    }

    int size() const { return num_entries; }
    bool empty() const { return num_entries == 0; }
```

```

bool insert(const K &k, const V &v) {
    if (find(k) != nullptr) {
        return false;
    }
    if (num_entries >= table_size) {
        grow_and_rehash();
    }
    int i = bucket(k);
    table[i].emplace_back(k, v);
    num_entries++;
    return true;
}

bool erase(const K &k) {
    int i = bucket(k);
    auto it = table[i].begin();
    while (it != table[i].end() && !equal(it->key, k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return false;
    }
    table[i].erase(it);
    num_entries--;
    return true;
}

V *find(const K &k) {
    int i = bucket(k);
    auto it = table[i].begin();
    while (it != table[i].end() && !equal(it->key, k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return nullptr;
    }
    return &(it->value);
}

V &operator[](const K &k) {
    if (V *ptr = find(k); ptr != nullptr) {
        return *ptr;
    }
    if (num_entries >= table_size) {
        grow_and_rehash();
    }
    int i = bucket(k);
    table[i].emplace_back(k, V{});
    num_entries++;
    return table[i].back().value;
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_entries);
    for (int i = 0; i < table_size; i++) {
        for (const auto &[k, v] : table[i]) {
            res.emplace_back(k, v);
        }
    }
    return res;
}
};

```

## Example Usage

```

#include <algorithm>
#include <cassert>
#include <chrono>
#include <string>
using namespace std;

// Example key hashers. For more hash algorithms and overloads, see 3.2.1. To defend against
// adversarially crafted collisions in open-hacking environments, a random seed is added to input
// keys before mixing (as 3.2.1's IntHasher does).
struct Hasher {
    inline static const uint64_t RAND_SEED = chrono::steady_clock::now().time_since_epoch().count();

    // Signed -> unsigned delegates.
    uint32_t operator()(int k) const { return Hasher{}(static_cast<uint32_t>(k)); }
    uint32_t operator()(int64_t k) const { return Hasher{}(static_cast<uint64_t>(k)); }

    // Knuth's multiplicative method. Fast, but affine: an additive RAND_SEED only shifts all buckets
    // uniformly and won't stop crafted collisions (unlike the non-linear hashers below). To harden
    // it, randomize the odd multiplier and take the high bits instead.
    uint32_t operator()(uint32_t k) const {
        return k * 2654435761U; // Or just return k.
    }

    // SplitMix64.
    uint32_t operator()(uint64_t k) const {
        k += RAND_SEED;
        k = (k ^ (k >> 30)) * 0xbf58476d1ce4e5b9ULL;
        k = (k ^ (k >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<uint32_t>(k ^ (k >> 31));
    }

    // Jenkins's one-at-a-time hash.
    uint32_t operator()(const string &k) const {
        uint32_t hash = RAND_SEED;
        for (char c : k) {
            hash += ((hash + static_cast<unsigned char>(c)) << 10);
            hash ^= (hash >> 6);
        }
        hash += (hash << 3);
        hash ^= (hash >> 11);
        return hash + (hash << 15);
    }
};

struct AbsHash {
    int salt;
    explicit AbsHash(int salt = 0) : salt(salt) {}
    size_t operator()(int x) const { return (x < 0 ? -static_cast<int64_t>(x) : x) + salt; }
};

struct AbsEqual {
    bool operator()(int a, int b) const { return AbsHash{}(a) == AbsHash{}(b); }
};

int main() {
    ChainingHashMap<string, char, Hasher> m;
    assert(m.empty());
    m["foo"] = 'a';
    assert(m.insert("bar", 'b'));
    assert(!m.insert("bar", 'z'));
    assert(!m.empty() && m.size() == 2);
    assert(m["foo"] == 'a');
    assert(m["bar"] == 'b');
    assert(m.find("foo") != nullptr && *m.find("foo") == 'a');
    assert(m.find("qux") == nullptr);
    assert(m["baz"] == '\0');
}

```



```

m["baz"] = 'c';

string vals;
for (const auto &[k, v] : m.entries()) {
    vals += v;
}
sort(vals.begin(), vals.end());
assert(vals == "abc");
assert(m.erase("foo"));
assert(m.size() == 2);
assert(m["foo"] == '\0');
assert(m.size() == 3);

ChainingHashMap<string, char, Hasher> stable(1);
stable["x"] = 'x';
char *ptr = stable.find("x");
stable["y"] = 'y'; // Rehashes without relocating the existing list node.
assert(ptr == stable.find("x") && *ptr == 'x');

ChainingHashMap<string, int> defaults;
defaults["answer"] = 42;
assert(*defaults.find("answer") == 42);

ChainingHashMap<int, char, AbsHash, AbsEqual> absolute(128, AbsHash(7), AbsEqual{});
assert(absolute.insert(-2, 'x'));
assert(!absolute.insert(2, 'y') && *absolute.find(2) == 'x');
return 0;
}

```

### 2.4.10 Hash Table (Open Addressing)

Maintain an unordered map: a collection of key-value pairs in which each key appears at most once. Keys are hashed into table slots, and this implementation resolves collisions using open addressing with linear probing. Deletions leave tombstones behind so that probe chains remain searchable. The hash and key-equality policies follow the conventions of `std::unordered_map`.

Linear probing checks slots  $i$ ,  $i + 1$ ,  $i + 2$ , ... (modulo the table size) until the desired key or an empty slot is found. Other probe-step schemes include quadratic probing and double hashing, but these require additional table-sizing and coprimality conditions to guarantee correct coverage of the table. The `next_bucket()` helper includes a commented quadratic-probing alternative for power-of-two table sizes.

Compared with the chaining version (2.4.9), open addressing stores entries contiguously, so it is more cache-friendly and needs no per-entry allocation. The costs: deletions must leave tombstones, the load factor must stay well below 1, and, unlike chaining or `std::unordered_map`, a pointer from `find()` or reference from `operator[]` is invalidated as soon as a later insertion rehashes and relocates the entries.

- `ProbingHashMap<K, V>(buckets = 128)` constructs an empty map with the positive number of slots given by `buckets`, using `std::hash<K>` and `std::equal_to<K>`.
- `ProbingHashMap<K, V, Hash, KeyEqual>(buckets, hash, equal)` instead stores the supplied hash and equality policies.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the value associated with key `k`, or `nullptr` if the key was not found.
- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.

- `entries()` returns all key-value entries in no guaranteed order.

This is an educational implementation; in practice prefer one of the standard options:

- `std::unordered_map` is the portable standard-library hash map. Unlike this class, it is chaining-based rather than open-addressed. Pass a custom hash (see 3.2.1) to harden it against adversarial inputs, as its default integer hash is effectively the identity.
- `__gnu_pbds::gp_hash_table` (from GCC's policy-based library) is an open-addressing table that is typically several times faster than `std::unordered_map`, but is a non-portable GNU extension. Prefer it on GCC-only judges when hashing is the bottleneck; see 8.6 for `HashMap`/`HashSet` wrappers with a randomized integer hash.

### Time Complexity:

- $O(b)$  per call to the constructor, where  $b$  is the initial number of slots.
- $O(1)$  per call to `size()` and `empty()`.
- $O(1)$  expected amortized per call to `insert()`, `erase()`, `find()`, and `operator[]`, and  $O(n)$  in the collision-heavy worst case.
- $O(n + b)$  per call to `entries()`, where  $n$  is the number of entries and  $b$  is the number of slots.

### Space Complexity:

- $O(n + b)$  for storage of the map elements and slots.
- $O(n + b)$  auxiliary during a rehash.
- $O(n)$  for the vector returned by `entries()`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <cstddef>
#include <cstdint>
#include <functional>
#include <optional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Hash = std::hash<K>, typename KeyEqual = std::equal_to<K>>
class ProbingHashMap {
    enum class State { EMPTY, OCCUPIED, DELETED };

    struct HashNode {
        K key;
        V value;
        HashNode(const K &k, const V &v) : key(k), value(v) {}
    };

    std::vector<std::optional<HashNode>> table;
    std::vector<State> state;
    int table_size, num_entries, num_tombstones;
    Hash hash;
    KeyEqual equal;

    int bucket(const K &k) const {
        return static_cast<int>(hash(k) % static_cast<std::size_t>(table_size));
    }

    int next_bucket(int i, int probes) const {
        return (i + 1) % table_size; // Linear probing.
        // Or use quadratic probing (cumulative offsets 1, 3, 6, 10, ...) for power-of-2 table sizes.
        // return (i + probes + 1) & (table_size - 1);
    }

    void rehash(int new_size) {
        auto old_table = std::move(table);
        auto old_state = std::move(state);
        table_size = new_size;
        table.assign(table_size, std::nullopt);
        state.assign(table_size, State::EMPTY);
    }
};
```

```

    num_entries = 0;
    num_tombstones = 0;
    for (int i = 0; i < static_cast<int>(old_table.size()); i++) {
        if (old_state[i] == State::OCCUPIED) {
            insert(old_table[i]->key, old_table[i]->value);
        }
    }
}

void grow_if_needed() {
    // Keep total non-empty slots below 50%. Tombstones count because they lengthen probe chains.
    if (2 * (num_entries + num_tombstones) >= table_size) {
        rehash(2 * num_entries >= table_size ? 2 * table_size : table_size);
    }
}

public:
    explicit ProbingHashMap(int buckets = 128, Hash hash = Hash{}, KeyEqual equal = KeyEqual{})
        : table_size(buckets),
          num_entries(0),
          num_tombstones(0),
          hash(std::move(hash)),
          equal(std::move(equal)) {
        assert(buckets > 0);
        // Also require this when using the quadratic alternative in next_bucket():
        // assert((buckets & (buckets - 1)) == 0);
        table.resize(buckets);
        state.assign(buckets, State::EMPTY);
    }

    int size() const { return num_entries; }
    bool empty() const { return num_entries == 0; }

    bool insert(const K &k, const V &v) {
        if (2 * (num_entries + num_tombstones) >= table_size && find(k) != nullptr) {
            return false;
        }
        grow_if_needed();
        int first_deleted = table_size;
        int i = bucket(k);
        for (int probes = 0; probes < table_size; probes++) {
            if (state[i] == State::OCCUPIED) {
                if (equal(table[i]->key, k)) {
                    return false;
                }
            }
            else if (state[i] == State::DELETED) {
                if (first_deleted == table_size) {
                    first_deleted = i;
                }
            }
            else {
                int dest = first_deleted == table_size ? i : first_deleted;
                table[dest].emplace(k, v);
                if (state[dest] == State::DELETED) {
                    num_tombstones--;
                }
                state[dest] = State::OCCUPIED;
                num_entries++;
                return true;
            }
        }
        i = next_bucket(i, probes);
    }
    // Should be rare because grow_if_needed keeps slack, but safe.
    rehash(2 * table_size);
    return insert(k, v);
}

bool erase(const K &k) {
    int i = bucket(k);

```

```

for (int probes = 0; probes < table_size; probes++) {
    if (state[i] == State::EMPTY) {
        return false;
    }
    if (state[i] == State::OCCUPIED && equal(table[i]->key, k)) {
        table[i].reset();
        state[i] = State::DELETED;
        num_entries--;
        num_tombstones++;
        return true;
    }
    i = next_bucket(i, probes);
}
return false;
}

V *find(const K &k) {
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::EMPTY) {
            return nullptr;
        }
        if (state[i] == State::OCCUPIED && equal(table[i]->key, k)) {
            return &table[i]->value;
        }
        i = next_bucket(i, probes);
    }
    return nullptr;
}

V &operator[](const K &k) {
    if (V *ptr = find(k); ptr != nullptr) {
        return *ptr;
    }
    grow_if_needed();
    int first_deleted = table_size;
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::DELETED) {
            if (first_deleted == table_size) {
                first_deleted = i;
            }
        }
        else if (state[i] == State::EMPTY) {
            int dest = first_deleted == table_size ? i : first_deleted;
            table[dest].emplace(k, V{});
            if (state[dest] == State::DELETED) {
                num_tombstones--;
            }
            state[dest] = State::OCCUPIED;
            num_entries++;
            return table[dest]->value;
        }
        i = next_bucket(i, probes);
    }
    rehash(2 * table_size);
    return (*this)[k];
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_entries);
    for (int i = 0; i < table_size; i++) {
        if (state[i] == State::OCCUPIED) {
            res.emplace_back(table[i]->key, table[i]->value);
        }
    }
    return res;
}

```

```
};
```

## Example Usage

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <string>
using namespace std;

// Example key hashers. For more hash algorithms and overloads, see 3.2.1. To defend against
// adversarially crafted collisions in open-hacking environments, a random seed is added to input
// keys before mixing (as 3.2.1's IntHasher does).
struct Hasher {
    inline static const uint64_t RAND_SEED = chrono::steady_clock::now().time_since_epoch().count();

    // Signed -> unsigned delegates.
    uint32_t operator()(int k) const { return Hasher{}(static_cast<uint32_t>(k)); }
    uint32_t operator()(int64_t k) const { return Hasher{}(static_cast<uint64_t>(k)); }

    // Knuth's multiplicative method. Fast, but affine: an additive RAND_SEED only shifts all buckets
    // uniformly and won't stop crafted collisions (unlike the non-linear hashers below). To harden
    // it, randomize the odd multiplier and take the high bits instead.
    uint32_t operator()(uint32_t k) const {
        return k * 2654435761U; // Or just return k.
    }

    // SplitMix64.
    uint32_t operator()(uint64_t k) const {
        k += RAND_SEED;
        k = (k ^ (k >> 30)) * 0xbf58476d1ce4e5b9ULL;
        k = (k ^ (k >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<uint32_t>(k ^ (k >> 31));
    }

    // Jenkins's one-at-a-time hash.
    uint32_t operator()(const string &k) const {
        uint32_t hash = RAND_SEED;
        for (char c : k) {
            hash += ((hash + static_cast<unsigned char>(c)) << 10);
            hash ^= (hash >> 6);
        }
        hash += (hash << 3);
        hash ^= (hash >> 11);
        return hash + (hash << 15);
    }
};

struct AbsHash {
    int salt;
    explicit AbsHash(int salt = 0) : salt(salt) {}
    size_t operator()(int x) const { return (x < 0 ? -static_cast<int64_t>(x) : x) + salt; }
};

struct AbsEqual {
    bool operator()(int a, int b) const { return AbsHash{}(a) == AbsHash{}(b); }
};

int main() {
    ProbingHashMap<string, char, Hasher> m;
    assert(m.empty());
    m["foo"] = 'a';
    assert(m.insert("bar", 'b'));
    assert(!m.insert("bar", 'z'));
    assert(!m.empty() && m.size() == 2);
    assert(m["foo"] == 'a');
```

```

assert(m["bar"] == 'b');
assert(m.find("foo") != nullptr && *m.find("foo") == 'a');
assert(m.find("qux") == nullptr);
assert(m["baz"] == '\0');
m["baz"] = 'c';

string vals;
for (const auto &[k, v] : m.entries()) {
    vals += v;
}
sort(vals.begin(), vals.end());
assert(vals == "abc");
assert(m.erase("foo"));
assert(m.size() == 2);
assert(m["foo"] == '\0');
assert(m.size() == 3);

ProbingHashMap<string, char, Hasher> duplicate(1);
duplicate.insert("x", 'x');
char *ptr = duplicate.find("x");
assert(!duplicate.insert("x", 'z'));
assert(ptr == duplicate.find("x")); // A rejected duplicate does not trigger a rehash.

ProbingHashMap<string, int> defaults;
defaults["answer"] = 42;
assert(*defaults.find("answer") == 42);

ProbingHashMap<int, char, AbsHash, AbsEqual> absolute(128, AbsHash(7), AbsEqual{});
assert(absolute.insert(-2, 'x'));
assert(!absolute.insert(2, 'y') && *absolute.find(2) == 'x');
return 0;
}

```

## 2.5 Range Queries in One Dimension

### 2.5.1 Sparse Table

Given a static array, precompute a table that answers range minimum queries in constant time. More generally, the sparse table works for any range query whose combining operation is both associative and idempotent (i.e. `combine(x, x)` must equal `x`). The canonical examples are “min”, “max”, “gcd”, and the bitwise “and” and “or”.

For each level  $j$ , the state  $dp(j, i)$  holds the result of `combine()` over the half-open range  $[i, i + 2^j)$ , built by combining two ranges of length  $2^{j-1}$ . A query over  $[lo, hi]$  is answered by combining the two overlapping ranges of length  $2^j$  that are anchored at `lo` and at `hi`, where  $2^j$  is the largest power of two not exceeding the length of the range. Because these two ranges overlap whenever the query length is not a power of two, the elements in the overlap are folded in twice; this is harmless only when `combine()` is idempotent. To support a non-idempotent but associative operation such as “sum” or “XOR” in constant time, use the disjoint sparse table in the next section instead, whose ranges meet at a center and never overlap.

The query operation is defined by an associative and idempotent function `combine(a, b)`. The default code below returns the “min” of the range; for “gcd”, `combine(a, b)` should return `gcd(a, b)`.

- `SparseTable<T>(lo, hi)` builds the table over the half-open iterator range  $[lo, hi)$ .
- `size()` returns the size of the array.
- `query(lo, hi)` returns the aggregate of the values at indices in  $[lo, hi]$ .

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `query()`.

**Space Complexity:**

- $O(n \log n)$  for storage of the table.
- $O(1)$  auxiliary for `query()`.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class SparseTable {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

    int len;
    std::vector<int> log2; // Can also use 31 - __builtin_clz(n) instead of precomputing log2[n].
    std::vector<std::vector<T>> dp; // dp[j][i] = combine() over [i, i + 2^j).

public:
    template<typename It>
    SparseTable(It lo, It hi) {
        std::vector<T> a(lo, hi);
        len = static_cast<int>(a.size());
        log2.assign(len + 1, 0);
        for (int i = 2; i <= len; i++) {
            log2[i] = log2[i >> 1] + 1;
        }
        if (len == 0) {
            return;
        }
        dp.assign(log2[len] + 1, std::vector<T>(len, a[0]));
        dp[0] = std::move(a);
        for (int j = 1; (1 << j) <= len; j++) {
            for (int i = 0; i + (1 << j) <= len; i++) {
                dp[j][i] = combine(dp[j - 1][i], dp[j - 1][i + (1 << (j - 1))]);
            }
        }
    }

    int size() const { return len; }

    T query(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi < len);
        int j = log2[hi - lo + 1];
        return combine(dp[j][lo], dp[j][hi - (1 << j) + 1]);
    }
};
```

**Example Usage**

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SparseTable<int> t(a.begin(), a.end());
    assert(t.size() == 5);
    assert(t.query(0, 3) == -2);
    assert(t.query(2, 4) == 1);
    assert(t.query(3, 3) == 8);
    return 0;
}
```

## 2.5.2 Disjoint Sparse Table

Given a static array, precompute a table that answers range queries for any associative operation in constant time. Unlike the ordinary sparse table for range minimum queries, the disjoint sparse table does not require the operation to be idempotent, so it also handles sums, products, and matrix products where overlapping the two query halves would double-count.

The array is divided into blocks whose size doubles at each of the  $O(\log n)$  levels. At the level whose block size is  $2^{k+1}$ , every block is split at its center, and the table stores, for each position, the fold of the contiguous run from that position up to (or down to) the center. To answer a query  $[lo, hi]$ , find the level at which  $lo$  and  $hi$  first fall on opposite sides of a center: this is the position of the highest set bit of  $lo \sim hi$ . At that level,  $lo$  lies in the block's left half and  $hi$  in its right half, so the answer is the stored suffix fold ending at the center combined with the stored prefix fold beginning at the center.

The query operation is defined by an associative function `combine(a, b)`. The default code below returns the “min” of the range; for “sum”, `combine(a, b)` should return `a + b`.

- `DisjointSparseTable<T>(lo, hi)` builds the table over the half-open iterator range  $[lo, hi)$ .
- `size()` returns the size of the array.
- `query(lo, hi)` returns the aggregate of the values at indices in  $[lo, hi)$ .

### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `query()`.

### Space Complexity:

- $O(n \log n)$  for storage of the table.
- $O(1)$  auxiliary for `query()`.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

template<typename T>
class DisjointSparseTable {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

    int len;
    std::vector<T> a;
    std::vector<std::vector<T>> fold;

public:
    template<typename It>
    DisjointSparseTable(It lo, It hi) : a(lo, hi) {
        len = static_cast<int>(a.size());
        if (len == 0) {
            return;
        }
        int levels = 1;
        while ((1 << levels) < len) {
            levels++;
        }
        fold.assign(levels, std::vector<T>(len, a[0]));
        for (int level = 0; level < levels; level++) {
            int range = 1 << (level + 1), half = 1 << level;
            for (int center = half; center < len + half; center += range) {
                int left = center - half, right = std::min(len, center + half);
                int top = std::min(center, len) - 1; // Suffix folds over [i, center).
                fold[level][top] = a[top];
                for (int i = top - 1; i >= left; i--) {
                    fold[level][i] = combine(a[i], fold[level][i + 1]);
                }
            }
        }
    }
};
```



```

        if (center < len) { // Prefix folds over [center, i].
            fold[level][center] = a[center];
            for (int i = center + 1; i < right; i++) {
                fold[level][i] = combine(fold[level][i - 1], a[i]);
            }
        }
    }
}

int size() const { return len; }

T query(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi < len);
    if (lo == hi) {
        return a[lo];
    }
    int level = 31 - __builtin_clz(static_cast<uint32_t>(lo ^ hi));
    return combine(fold[level][lo], fold[level][hi]);
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10, -5, 3};
    DisjointSparseTable<int> t(a.begin(), a.end());
    assert(t.size() == 7);
    assert(t.query(0, 6) == -5);
    assert(t.query(1, 3) == -2);
    assert(t.query(4, 4) == 10);
    assert(t.query(2, 4) == 1);

    vector<int> empty;
    DisjointSparseTable<int> empty_table(empty.begin(), empty.end());
    assert(empty_table.size() == 0);
    return 0;
}

```

### 2.5.3 Square Root Decomposition

Maintain a fixed-size array while supporting point updates and contiguous range aggregate queries. Square root decomposition partitions the array into contiguous blocks of about  $\sqrt{n}$  elements, each caching the aggregate of its block. A range query combines the cached aggregates of the whole blocks contained in the range with the individual elements of the partial blocks at either end, while a point update refreshes only the single block containing the changed index.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default definition below assumes a numerical array type, supporting queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `SqrtDecomposition<T>(n, v = T{})` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `SqrtDecomposition<T>(lo, hi)` constructs an array from the half-open random-access iterator range `[lo, hi)`.

- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in `[lo, hi]`.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.

The supported operations are identical to those of the point-update segment tree in this section.

#### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `at()`.
- $O(\sqrt{n})$  per call to `query()` and `update()`.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <vector>

template<typename T>
class SqrtDecomposition {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int len, blocklen;
    std::vector<T> value, block;

    void init() {
        blocklen = std::max(1, static_cast<int>(std::sqrt(len)));
        int nblocks = (len + blocklen - 1) / blocklen;
        for (int i = 0; i < nblocks; i++) {
            T blockval = value[i * blocklen];
            int blockhi = std::min(len, (i + 1) * blocklen);
            for (int j = i * blocklen + 1; j < blockhi; j++) {
                blockval = combine(blockval, value[j]);
            }
            block.push_back(blockval);
        }
    }

public:
    explicit SqrtDecomposition(int n, const T &v = T{}) : len(n), value(n, v) { init(); }

    template<typename It>
    SqrtDecomposition(It lo, It hi) : len(hi - lo), value(lo, hi) {
        init();
    }

    int size() const { return len; }

    T at(int i) const {
        assert(0 <= i && i < len);
        return value[i];
    }

    T query(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi < len);
        int blocklo = (lo + blocklen - 1) / blocklen, blockhi = (hi + 1) / blocklen - 1;
        if (blocklo > blockhi) {
            T res = value[lo];
            for (int i = lo + 1; i <= hi; i++) {
                res = combine(res, value[i]);
            }
        }
    }
};
```

```

    return res;
}
T res = block[blocklo];
for (int i = blocklo + 1; i <= blockhi; i++) {
    res = combine(res, block[i]);
}
for (int i = lo; i < blocklo * blocklen; i++) {
    res = combine(res, value[i]);
}
for (int i = (blockhi + 1) * blocklen; i <= hi; i++) {
    res = combine(res, value[i]);
}
return res;
}
};

void update(int i, const T &d) {
    assert(0 <= i && i < len);
    value[i] = apply_delta(value[i], d);
    int b = i / blocklen;
    int blockhi = std::min(len, (b + 1) * blocklen);
    block[b] = value[b * blocklen];
    for (int j = b * blocklen + 1; j < blockhi; j++) {
        block[b] = combine(block[b], value[j]);
    }
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SqrtDecomposition<int> sd(a.begin(), a.end());
    sd.update(2, 4);
    vector<int> expected{6, -2, 4, 8, 10};
    for (int i = 0; i < sd.size(); i++) {
        assert(sd.at(i) == expected[i]);
    }
    assert(sd.query(0, 3) == -2);
    return 0;
}

```

## 2.5.4 Mo's Algorithm

Mo's algorithm answers a batch of range queries offline by reordering them so that the current window  $[\text{cur}_l, \text{cur}_r]$  can be transformed into the next query's range with as few single-element insertions and deletions as possible. It applies whenever there is no fast direct formula, but extending or shrinking the window by one element updates the answer cheaply, as with the number of distinct values in a range or the sum of squares of element frequencies.

The trick is the query order. Sort queries by the block of their left endpoint, where each block spans about  $n/\sqrt{q}$  indices, breaking ties by right endpoint. The left pointer then travels  $O(n/\sqrt{q})$  per query, for  $O(n\sqrt{q})$  moves in all, while within each block the right pointer sweeps monotonically across the array, contributing  $O(n)$  per block over the  $\sqrt{q}$  blocks, again  $O(n\sqrt{q})$ . So the pointers move  $O(n\sqrt{q})$  times in total. Sorting the right endpoint in alternating directions between consecutive blocks (a “boustrophedon” order) avoids resetting the right pointer at every block boundary.

- `mos_algorithm(n, queries, add, remove, current)` answers the given inclusive 0-based ranges over an array of size `n`. The caller supplies three callbacks that maintain an arbitrary window state: `add(i)` and `remove(i)`

insert or delete index `i` from the current window, and `current()` returns the answer for the window in its present state. The result is a vector of answers in the original query order, with element type matching the return type of `current()`.

**Time Complexity:**  $O(n\sqrt{q})$  calls to `add()` and `remove()` in total, plus  $O(q \log q)$  to sort, where  $n$  is the array size and  $q$  is the number of queries.

**Space Complexity:**  $O(q)$  auxiliary and  $O(q)$  for the returned answers, excluding whatever window state the callbacks maintain.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <numeric>
#include <utility>
#include <vector>

template<typename AddFn, typename RemoveFn, typename CurrentFn>
std::vector<decltype(std::declval<CurrentFn>())> mos_algorithm(
    int n, const std::vector<std::pair<int, int>> &queries, AddFn add, RemoveFn remove,
    CurrentFn current
) {
    assert(n >= 0);
    for (auto [lo, hi] : queries) {
        assert(0 <= lo && lo <= hi && hi < n);
    }
    int q = static_cast<int>(queries.size());
    int block = std::max(1, static_cast<int>(n / std::max(1.0, std::sqrt(q))));
    std::vector<int> order(q);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int x, int y) {
        int bx = queries[x].first / block, by = queries[y].first / block;
        if (bx != by) {
            return bx < by;
        }
        return (bx & 1) ? queries[x].second > queries[y].second : queries[x].second < queries[y].second;
    });
    std::vector<decltype(current())> res(q);
    int cur_l = 0, cur_r = -1; // Window [cur_l, cur_r] is initially empty.
    for (int idx : order) {
        int l = queries[idx].first, r = queries[idx].second;
        while (cur_r < r) add(++cur_r);
        while (cur_l > l) add(--cur_l);
        while (cur_r > r) remove(cur_r--);
        while (cur_l < l) remove(cur_l++);
        res[idx] = current();
    }
    return res;
}
```

## Example Usage

```
using namespace std;

int main() {
    // Answer "number of distinct values in [l, r]" for several ranges.
    vector<int> a{1, 1, 2, 1, 3, 2, 3};
    vector<int> freq(4);
    int distinct = 0;
    auto add = [&](int i) {
        if (freq[a[i]]++ == 0) distinct++;
    };
    auto remove = [&](int i) {
        if (--freq[a[i]] == 0) distinct--;
    };
}
```

```

auto current = [&]() { return distinct; };

int n = static_cast<int>(a.size());
vector<pair<int, int>> queries{{0, 6}, {0, 2}, {3, 5}, {1, 1}, {2, 4}};
assert((mos_algorithm(n, queries, add, remove, current) == vector<int>{3, 2, 3, 1, 3}));
return 0;
}

```

### 2.5.5 Segment Tree (Point Update)

Maintain a fixed-size array while supporting point updates and range aggregate queries. A segment tree stores one aggregate value for each recursively split interval, so an update only rebuilds the  $O(\log n)$  intervals on the path from one leaf to the root.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `SegTree<T>(n, v = T{})` constructs an array of size `n` with indices  $[0, n)$ , and all values initialized to `v`.
- `SegTree<T>(lo, hi)` constructs an array from the half-open random-access iterator range  $[lo, hi)$ .
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in  $[lo, hi]$ . If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range  $[lo, hi)$  satisfies `pred()`. As `hi` increases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and `size()` is returned if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range  $[lo, hi)$  satisfies `pred()`. As `lo` decreases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and 0 is returned if the predicate remains true to the beginning.

For the boundary-search functions, `pred()` takes aggregate `T` values. For `combine = min`, `pred(mn) = (mn > x)` makes `max_right()` stop at the first value `<= x` when extending right, while `min_left()` stops just after the first such value when extending left. With nonnegative values and `combine = sum`, `pred(sum) = (sum <= x)` finds the longest extension with sum at most `x`.

#### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to `at()`, `query()`, `update()`, `max_right()`, and `min_left()`.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(\log n)$  auxiliary stack space for `query()`, `update()`, `max_right()`, and `min_left()`.
- $O(1)$  auxiliary for `size()`.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <vector>

template<typename T>

```

```

class SegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int len;
    std::vector<T> value;

    template<typename Gen>
    void build(int i, int lo, int hi, const Gen &gen) {
        if (lo == hi) {
            value[i] = gen(lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, gen);
        build(i * 2 + 2, mid + 1, hi, gen);
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }

    T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) const {
        if (lo == tgt_lo && hi == tgt_hi) {
            return value[i];
        }
        int mid = lo + (hi - lo) / 2;
        if (tgt_lo <= mid && mid < tgt_hi) {
            return combine(
                query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
                query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
            );
        }
        if (tgt_lo <= mid) {
            return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
        }
        return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
    }

    void update(int i, int lo, int hi, int target, const T &d) {
        if (lo == hi) {
            value[i] = apply_delta(value[i], d);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        if (target <= mid) {
            update(i * 2 + 1, lo, mid, target, d);
        } else {
            update(i * 2 + 2, mid + 1, hi, target, d);
        }
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }

    template<typename Pred>
    int max_right(int i, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) const {
        if (hi < tgt_lo) {
            return -1;
        }
        if (tgt_lo <= lo) {
            T next = acc ? combine(*acc, value[i]) : value[i];
            if (pred(next)) {
                acc = next;
                return -1;
            }
            if (lo == hi) {
                return lo;
            }
        }
        int mid = lo + (hi - lo) / 2;
        int res = max_right(i * 2 + 1, lo, mid, tgt_lo, pred, acc);
        return res != -1 ? res : max_right(i * 2 + 2, mid + 1, hi, tgt_lo, pred, acc);
    }

```

```

}

template<typename Pred>
int min_left(int i, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) const {
    if (tgt_hi <= lo) {
        return -1;
    }
    if (hi < tgt_hi) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo + 1;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = min_left(i * 2 + 2, mid + 1, hi, tgt_hi, pred, acc);
    return res != -1 ? res : min_left(i * 2 + 1, lo, mid, tgt_hi, pred, acc);
}

public:
explicit SegTree(int n, const T &v = T{}) : len(n) {
    assert(len > 0);
    value.assign(4 * len, v);
    build(0, 0, len - 1, [&](int) { return v; });
}

template<typename It>
SegTree(It lo, It hi) : len(hi - lo) {
    assert(len > 0);
    value.assign(4 * len, *lo);
    build(0, 0, len - 1, [&](int i) { return *(lo + i); });
}

int size() const { return len; }
T at(int i) const { return query(i, i); }

T query(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi < len);
    return query(0, 0, len - 1, lo, hi);
}

void update(int i, const T &d) {
    assert(0 <= i && i < len);
    update(0, 0, len - 1, i, d);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) const {
    assert(0 <= lo && lo <= len);
    std::optional<T> acc;
    int res = max_right(0, 0, len - 1, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) const {
    assert(0 <= hi && hi <= len);
    std::optional<T> acc;
    int res = min_left(0, 0, len - 1, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

## Example Usage

```
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    vector<int> expected{6, -2, 4, 8, 10};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == -2);

    // Boundary search by accumulated aggregate: stop before including the -2 at index 1.
    assert(t.max_right(0, [](int mn) { return mn > -2; }) == 1);
    assert(t.min_left(5, [](int mn) { return mn > -2; }) == 2);
    assert(t.max_right(2, [](int mn) { return mn >= 4; }) == t.size());
    assert(t.min_left(2, [](int mn) { return mn >= 0; }) == 2);
    return 0;
}
```

## 2.5.6 Segment Tree (Range Update)

Maintain a fixed-size array while supporting range updates and range aggregate queries. Lazy propagation lets a segment tree defer work below intervals covered by an update: each node stores an aggregate value and a pending update that is pushed to its children only when needed.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `LazySegTree<T>(n, v = T{})` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `LazySegTree<T>(lo, hi)` constructs an array from the half-open random-access iterator range `[lo, hi)`.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in `[lo, hi]`. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` modifies the value at index `i` by applying the delta `d`.
- `update(lo, hi, d)` modifies the value at each array index in `[lo, hi]` by applying the delta `d` to each value.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `hi` increases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and `size()` is returned if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `lo` decreases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and 0 is returned if the predicate remains true to the beginning.

For the boundary-search functions, `pred()` takes aggregate `T` values. For `combine = min`, `pred(mn) = (mn > x)` makes `max_right()` stop at the first value `<= x` when extending right, while `min_left()` stops just after the first



such value when extending left. With nonnegative values and `combine = sum`, `pred(sum) = (sum <= x)` finds the longest extension with sum at most `x`.

#### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to `at()`, `query()`, `update()`, `max_right()`, and `min_left()`.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(\log n)$  auxiliary stack space for `query()`, `update()`, `max_right()`, and `min_left()`.
- $O(1)$  auxiliary for `size()`.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <optional>
#include <vector>

template<typename T>
class LazySegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    int len;
    std::vector<T> value, delta;
    std::vector<char> pending;

    template<typename Gen>
    void build(int i, int lo, int hi, const Gen &gen) {
        if (lo == hi) {
            value[i] = gen(lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, gen);
        build(i * 2 + 2, mid + 1, hi, gen);
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }

    void push_delta(int i, int lo, int hi) {
        if (pending[i]) {
            value[i] = apply_delta(value[i], delta[i], hi - lo + 1);
            if (lo != hi) {
                int l = i * 2 + 1, r = i * 2 + 2;
                delta[l] = pending[l] ? compose_deltas(delta[l], delta[i]) : delta[i];
                delta[r] = pending[r] ? compose_deltas(delta[r], delta[i]) : delta[i];
                pending[l] = pending[r] = true;
            }
            pending[i] = false;
        }
    }

    T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) {
        push_delta(i, lo, hi);
        if (lo == tgt_lo && hi == tgt_hi) {
            return value[i];
        }
        int mid = lo + (hi - lo) / 2;
        if (tgt_lo <= mid && mid < tgt_hi) {
            return combine(
                query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
                query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
            );
        }
    }
};
```

```

    }
    if (tgt_lo <= mid) {
        return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
    }
    return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
}

void update(int i, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
    push_delta(i, lo, hi);
    if (hi < tgt_lo || lo > tgt_hi) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        delta[i] = d;
        pending[i] = true;
        push_delta(i, lo, hi);
        return;
    }
    update(i * 2 + 1, lo, lo + (hi - lo) / 2, tgt_lo, tgt_hi, d);
    update(i * 2 + 2, lo + (hi - lo) / 2 + 1, hi, tgt_lo, tgt_hi, d);
    value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
}

template<typename Pred>
int max_right(int i, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) {
    push_delta(i, lo, hi);
    if (hi < tgt_lo) {
        return -1;
    }
    if (tgt_lo <= lo) {
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = max_right(i * 2 + 1, lo, mid, tgt_lo, pred, acc);
    return res != -1 ? res : max_right(i * 2 + 2, mid + 1, hi, tgt_lo, pred, acc);
}

template<typename Pred>
int min_left(int i, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) {
    push_delta(i, lo, hi);
    if (tgt_hi <= lo) {
        return -1;
    }
    if (hi < tgt_hi) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo + 1;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = min_left(i * 2 + 2, mid + 1, hi, tgt_hi, pred, acc);
    return res != -1 ? res : min_left(i * 2 + 1, lo, mid, tgt_hi, pred, acc);
}

public:
    explicit LazySegTree(int n, const T &v = T{}) : len(n), pending(4 * len, false) {
        assert(len > 0);

```

```

    value.assign(4 * len, v);
    delta.assign(4 * len, v);
    build(0, 0, len - 1, [&](int) { return v; });
}

template<typename It>
LazySegTree(It lo, It hi) : len(hi - lo), pending(4 * len, false) {
    assert(len > 0);
    value.assign(4 * len, *lo);
    delta.assign(4 * len, *lo);
    build(0, 0, len - 1, [&](int i) { return *(lo + i); });
}

int size() const { return len; }
T at(int i) { return query(i, i); }

T query(int lo, int hi) {
    assert(0 <= lo && lo <= hi && hi < len);
    return query(0, 0, len - 1, lo, hi);
}

void update(int i, const T &d) { update(i, i, d); }

void update(int lo, int hi, const T &d) {
    assert(0 <= lo && lo <= hi && hi < len);
    update(0, 0, len - 1, lo, hi, d);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) {
    assert(0 <= lo && lo <= len);
    std::optional<T> acc;
    int res = max_right(0, 0, len - 1, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) {
    assert(0 <= hi && hi <= len);
    std::optional<T> acc;
    int res = min_left(0, 0, len - 1, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    LazySegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    vector<int> expected{6, -2, 4, 8, 10};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    expected = {5, 5, 5, 1, 5};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == 1);
}

```

```
// Boundary search works through lazy updates too; values are now {5, 5, 5, 1, 5}.
assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
return 0;
}
```

### 2.5.7 Iterative Segment Tree (Point Update)

Maintain a fixed-size array while supporting point updates and range aggregate queries using an iterative segment tree. The leaves begin at a power-of-two offset in a flat array, and each internal node stores the aggregate of its two children. Point updates rebuild ancestors by walking upward, while queries scan a decomposition of the requested range. Compared with the recursive segment tree, these flat loops avoid recursive stack usage and can reduce constant factors.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `IterativeSegTree<T>(n, v = T{})` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `IterativeSegTree<T>(lo, hi)` constructs an array from the half-open random-access iterator range `[lo, hi)`.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in `[lo, hi]`. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `hi` increases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and `size()` is returned if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `lo` decreases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and 0 is returned if the predicate remains true to the beginning.

For the boundary-search functions, `pred()` takes aggregate `T` values. For `combine = min`, `pred(mn) = (mn > x)` makes `max_right()` stop at the first value `<= x` when extending right, while `min_left()` stops just after the first such value when extending left. With nonnegative values and `combine = sum`, `pred(sum) = (sum <= x)` finds the longest extension with sum at most `x`.

Both boundary searches are iterative to demonstrate how the flat-array layout can avoid recursion. They decompose the search range into  $O(\log n)$  canonical nodes, scan those nodes in order while accumulating their aggregates, then descend inside the first node that makes `pred` false. A recursive descent like the one in 2.5.5 is equally valid and may be easier to adapt.

#### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `at()`.
- $O(\log n)$  per call to `query()`, `update()`, `max_right()`, and `min_left()`.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(1)$  auxiliary for all operations.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <vector>

template<typename T>
class IterativeSegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int len, base;
    std::vector<T> value;

    template<typename Gen>
    void build(const Gen &gen) {
        base = 1;
        while (base < len) {
            base <= 1;
        }
        value.assign(2 * base, gen(0));
        for (int i = 1; i < len; i++) {
            value[base + i] = gen(i);
        }
        for (int i = base - 1; i > 0; i--) {
            value[i] = combine(value[i << 1], value[i << 1 | 1]);
        }
    }

public:
    explicit IterativeSegTree(int n, const T &v = T{}) : len(n) {
        assert(len > 0);
        build([&](int) { return v; });
    }

    template<typename It>
    IterativeSegTree(It lo, It hi) : len(hi - lo) {
        assert(len > 0);
        build([&](int i) { return *(lo + i); });
    }

    int size() const { return len; }

    T at(int i) const {
        assert(0 <= i && i < len);
        return value[base + i];
    }

    T query(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi < len);
        std::optional<T> left, right;
        for (lo += base, hi += base + 1; lo < hi; lo >>= 1, hi >>= 1) {
            if (lo & 1) {
                left = left ? combine(*left, value[lo++]) : value[lo++];
            }
            if (hi & 1) {
                right = right ? combine(value[--hi], *right) : value[--hi];
            }
        }
        return !left ? *right : (!right ? *left : combine(*left, *right));
    }

    void update(int i, const T &d) {
        assert(0 <= i && i < len);
        i += base;
        value[i] = apply_delta(value[i], d);
        for (i >>= 1; i > 0; i >>= 1) {
            value[i] = combine(value[i << 1], value[i << 1 | 1]);
        }
    }
}

```

```

}

template<typename Pred>
int max_right(int lo, const Pred &pred) const {
    assert(0 <= lo && lo <= len);
    std::optional<T> acc;
    for (int pos = lo; pos < len;) {
        int width = pos == 0 ? base : pos & -pos;
        while (width > len - pos) {
            width >>= 1;
        }
        int i = (base + pos) / width;
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            pos += width;
            continue;
        }
        while (i < base) {
            i <<= 1;
            next = acc ? combine(*acc, value[i]) : value[i];
            if (pred(next)) {
                acc = next;
                i++;
            }
        }
        return i - base;
    }
    return len;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) const {
    assert(0 <= hi && hi <= len);
    std::optional<T> acc;
    for (int pos = hi; pos > 0;) {
        int width = pos & -pos;
        int i = (base + pos) / width - 1;
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            pos -= width;
            continue;
        }
        while (i < base) {
            i = i << 1 | 1;
            next = acc ? combine(value[i], *acc) : value[i];
            if (pred(next)) {
                acc = next;
                i--;
            }
        }
        return i - base + 1;
    }
    return 0;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    IterativeSegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
}

```

```

vector<int> expected{6, -2, 4, 8, 10};
for (int i = 0; i < t.size(); i++) {
    assert(t.at(i) == expected[i]);
}
assert(t.query(0, 3) == -2);

assert(t.max_right(0, [](int mn) { return mn > -2; }) == 1);
assert(t.min_left(5, [](int mn) { return mn > -2; }) == 2);
return 0;
}

```

## 2.5.8 Iterative Segment Tree (Range Update)

Maintain a fixed-size array while supporting range updates and range aggregate queries using an iterative segment tree with lazy propagation. The leaves begin at a power-of-two offset in a flat array, and each internal node stores an aggregate value plus a pending update. Range updates and queries push pending updates only along paths that inspect children, then scan a decomposition of the requested range. Compared with the recursive lazy segment tree, these flat loops avoid recursive stack usage and can reduce constant factors, though the recursive version is usually easier to adapt when the update or search logic becomes unusual.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `IterativeLazySegTree<T>(n, v = T{})` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `IterativeLazySegTree<T>(lo, hi)` constructs an array from the half-open random-access iterator range `[lo, hi)`.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in `[lo, hi]`. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value at index `i` by applying the delta `d`.
- `update(lo, hi, d)` modifies the value at each array index in `[lo, hi]` by applying the delta `d` to each value.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `hi` increases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and `size()` is returned if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred()`. As `lo` decreases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and 0 is returned if the predicate remains true to the beginning.

For the boundary-search functions, `pred()` takes aggregate `T` values. For `combine = min`, `pred(mn) = (mn > x)` makes `max_right()` stop at the first value `<= x` when extending right, while `min_left()` stops just after the first such value when extending left. With nonnegative values and `combine = sum`, `pred(sum) = (sum <= x)` finds the longest extension with sum at most `x`.

Both boundary searches are iterative to demonstrate how the flat-array layout can avoid recursion. They push lazy deltas along the endpoints, decompose the search range into  $O(\log n)$  canonical nodes, scan those nodes in

order while accumulating their aggregates, then push and descend inside the first node that makes `pred` false. A recursive descent like the one in 2.5.6 is equally valid and may be easier to adapt.

### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to `at()`, `query()`, `update()`, `max_right()`, and `min_left()`.

### Space Complexity:

- $O(n)$  for storage of the array elements and lazy deltas.
- $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <optional>
#include <vector>

template<typename T>
class IterativeLazySegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    int len, base, height;
    std::vector<T> value, delta;
    std::vector<int64_t> seg_len;
    std::vector<char> pending;

    template<typename Gen>
    void build(const Gen &gen) {
        base = 1;
        height = 0;
        while (base < len) {
            base <<= 1;
            height++;
        }
        T init = gen(0);
        value.assign(2 * base, init);
        delta.assign(2 * base, init);
        pending.assign(2 * base, false);
        seg_len.assign(2 * base, 1);
        for (int i = base - 1; i > 0; i--) {
            seg_len[i] = seg_len[i << 1] + seg_len[i << 1 | 1];
        }
        for (int i = 1; i < len; i++) {
            value[base + i] = gen(i);
        }
        for (int i = base - 1; i > 0; i--) {
            pull(i);
        }
    }

    void pull(int i) { value[i] = combine(value[i << 1], value[i << 1 | 1]); }

    void apply(int i, const T &d) {
        value[i] = apply_delta(value[i], d, seg_len[i]);
        if (i < base) {
            delta[i] = pending[i] ? compose_deltas(delta[i], d) : d;
            pending[i] = true;
        }
    }

    void push(int i) {
        if (pending[i]) {
```



```

    apply(i << 1, delta[i]);
    apply(i << 1 | 1, delta[i]);
    pending[i] = false;
}
}

void push_path(int i) {
    for (int h = height; h > 0; h--) {
        push(i >> h);
    }
}

public:
    explicit IterativeLazySegTree(int n, const T &v = T{}) : len(n) {
        assert(len > 0);
        build([&](int) { return v; });
    }

    template<typename It>
    IterativeLazySegTree(It lo, It hi) : len(hi - lo) {
        assert(len > 0);
        build([&](int i) { return *(lo + i); });
    }

    int size() const { return len; }

    T at(int i) {
        assert(0 <= i && i < len);
        i += base;
        push_path(i);
        return value[i];
    }

    T query(int lo, int hi) {
        assert(0 <= lo && lo <= hi && hi < len);
        int l = lo + base, r = hi + 1 + base;
        push_path(l);
        push_path(r - 1);
        std::optional<T> left, right;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) {
                left = left ? combine(*left, value[l++]) : value[l++];
            }
            if (r & 1) {
                right = right ? combine(value[--r], *right) : value[--r];
            }
        }
        return !left ? *right : (!right ? *left : combine(*left, *right));
    }

    void update(int lo, int hi, const T &d) {
        assert(0 <= lo && lo <= hi && hi < len);
        int l = lo + base, r = hi + 1 + base;
        push_path(l);
        push_path(r - 1);
        int l0 = l, r0 = r;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) {
                apply(l++, d);
            }
            if (r & 1) {
                apply(--r, d);
            }
        }
        // Rebuild only ancestors whose segments were not fully covered by the update.
        for (int h = 1; h <= height; h++) {
            if (((l0 >> h) << h) != 10) {
                pull(l0 >> h);
            }
        }
    }

```

```

    }
    if (((r0 >> h) << h) != r0) {
        pull((r0 - 1) >> h);
    }
}

void update(int i, const T &d) { update(i, i, d); }

template<typename Pred>
int max_right(int lo, const Pred &pred) {
    assert(0 <= lo && lo <= len);
    if (lo == len) {
        return len;
    }
    push_path(lo + base);
    push_path(len - 1 + base);
    std::optional<T> acc;
    for (int pos = lo; pos < len;) {
        int width = pos == 0 ? base : pos & -pos;
        while (width > len - pos) {
            width >>= 1;
        }
        int i = (base + pos) / width;
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            pos += width;
            continue;
        }
        while (i < base) {
            push(i);
            i <<= 1;
            next = acc ? combine(*acc, value[i]) : value[i];
            if (pred(next)) {
                acc = next;
                i++;
            }
        }
        return i - base;
    }
    return len;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) {
    assert(0 <= hi && hi <= len);
    if (hi == 0) {
        return 0;
    }
    push_path(base);
    push_path(hi - 1 + base);
    std::optional<T> acc;
    for (int pos = hi; pos > 0;) {
        int width = pos & -pos;
        int i = (base + pos) / width - 1;
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            pos -= width;
            continue;
        }
    }
    while (i < base) {
        push(i);
        i = i << 1 | 1;
        next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
        }
    }
}

```

```

        i--;
    }
    }
    return i - base + 1;
}
return 0;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    IterativeLazySegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    vector<int> expected{6, -2, 4, 8, 10};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    expected = {5, 5, 5, 1, 5};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == 1);

    // Boundary search works through lazy updates too; values are now {5, 5, 5, 1, 5}.
    assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
    assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
    return 0;
}

```

## 2.5.9 Segment Tree Beats

Maintain a fixed-size array under the range “chmin” update  $a[i] = \min(a[i], t)$  for all  $i$  in a range, while answering range-sum and range-maximum queries. An ordinary lazy segment tree cannot represent a clamp like this with a single composable tag, because clamping affects only the entries that currently exceed  $t$ . Segment tree beats (a.k.a. the Ji Driver tree) resolves this by storing, for each node, the strict maximum, the second-largest distinct value, and a count of entries equal to the maximum. A clamp then has three cases at each node: if  $t \geq \text{max1}$ , the clamp changes nothing and the recursion stops (the prune case); if  $\text{max2} < t < \text{max1}$ , every entry equal to  $\text{max1}$  drops to  $t$  and all others are untouched, so the node updates in  $O(1)$ , the sum falls by  $(\text{max1} - t) * \text{count}$ , and  $\text{max1}$  becomes  $t$  (the break, or tag, case); otherwise ( $t \leq \text{max2}$ ) the node is too coarse to update directly, so the clamp recurses into both children and re-pulls. The lazy tag pushes down to a child only when the child’s maximum is larger.

The break condition is what makes this efficient: a potential-function argument on the number of distinct values along root-to-leaf paths shows the total work over any sequence of clamps is amortized  $O(\log^2 n)$  each, even though a single clamp can momentarily descend deep. The same scheme extends to range “chmax” by tracking the symmetric minimum trio, and to mixed clamp-and-add workloads by carrying an additional add tag; only the chmin / sum / max core is shown here. Unlike the generic lazy segment tree, this structure cannot be reduced to a pluggable `combine` and `apply`, because a clamp acts on only the maximal entries of a node rather than uniformly on all of them, so each operation set must be hand-written and its amortized bound argued separately.

- `SegTreeBeats<T>(n, v = T{})` constructs an array of size  $n$ , indices  $[0, n)$ , all equal to  $v$ .

- `SegTreeBeats<T>(lo, hi)` constructs an array from the half-open random-access iterator range `[lo, hi)`.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query_sum(lo, hi)` returns the sum of `a[i]` over `i` in `[lo, hi]`.
- `query_max(lo, hi)` returns the maximum of `a[i]` over `i` in `[lo, hi]`.
- `chmin(lo, hi, t)` replaces `a[i]` with `min(a[i], t)` for every `i` in `[lo, hi]`.

#### Time Complexity:

- $O(n)$  per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log^2 n)$  amortized per call to `chmin()`.
- $O(\log n)$  per call to `query_sum()`, `query_max()`, and `at()`.

#### Space Complexity:

- $O(n)$  for storage of the tree.
- $O(\log n)$  auxiliary stack space per `chmin()` and query call.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <vector>

template<typename T>
class SegTreeBeats {
    static constexpr T NEG_INF = std::numeric_limits<T>::lowest();

    int len;
    std::vector<T> max1, max2, sum;
    std::vector<int> cnt; // Number of entries in the node equal to max1.

    void pull(int i) {
        int l = i * 2 + 1, r = i * 2 + 2;
        sum[i] = sum[l] + sum[r];
        if (max1[l] == max1[r]) {
            max1[i] = max1[l];
            cnt[i] = cnt[l] + cnt[r];
            max2[i] = std::max(max2[l], max2[r]);
        } else if (max1[l] > max1[r]) {
            max1[i] = max1[l];
            cnt[i] = cnt[l];
            max2[i] = std::max(max2[l], max1[r]);
        } else {
            max1[i] = max1[r];
            cnt[i] = cnt[r];
            max2[i] = std::max(max1[l], max2[r]);
        }
    }

    // Lower the node's maximum to t. Valid only when max2 < t < max1, so exactly the cnt entries
    // equal to max1 change and the second maximum is left untouched.
    void apply_chmin(int i, const T &t) {
        sum[i] -= (max1[i] - t) * static_cast<T>(cnt[i]); // Overflow warning.
        max1[i] = t;
    }

    void push(int i) {
        for (int c : {i * 2 + 1, i * 2 + 2}) {
            if (max1[c] > max1[i]) {
                apply_chmin(c, max1[i]);
            }
        }
    }
}
```

```

template<typename Gen>
void build(int i, int lo, int hi, const Gen &gen) {
    if (lo == hi) {
        max1[i] = sum[i] = gen(lo);
        max2[i] = NEG_INF;
        cnt[i] = 1;
        return;
    }
    int mid = lo + (hi - lo) / 2;
    build(i * 2 + 1, lo, mid, gen);
    build(i * 2 + 2, mid + 1, hi, gen);
    pull(i);
}

void chmin(int i, int lo, int hi, int tgt_lo, int tgt_hi, const T &t) {
    if (tgt_hi < lo || hi < tgt_lo || max1[i] <= t) {
        return; // Prune: outside the range, or every entry is already at most t.
    }
    if (tgt_lo <= lo && hi <= tgt_hi && (lo == hi || max2[i] < t)) {
        apply_chmin(i, t); // Break: only the maximal entries are affected.
        return;
    }
    push(i);
    int mid = lo + (hi - lo) / 2;
    chmin(i * 2 + 1, lo, mid, tgt_lo, tgt_hi, t);
    chmin(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi, t);
    pull(i);
}

T query_sum(int i, int lo, int hi, int tgt_lo, int tgt_hi) {
    if (tgt_hi < lo || hi < tgt_lo) {
        return 0;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        return sum[i];
    }
    push(i);
    int mid = lo + (hi - lo) / 2;
    return query_sum(i * 2 + 1, lo, mid, tgt_lo, tgt_hi) +
           query_sum(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi);
}

T query_max(int i, int lo, int hi, int tgt_lo, int tgt_hi) {
    if (tgt_hi < lo || hi < tgt_lo) {
        return NEG_INF;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        return max1[i];
    }
    push(i);
    int mid = lo + (hi - lo) / 2;
    return std::max(
        query_max(i * 2 + 1, lo, mid, tgt_lo, tgt_hi),
        query_max(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi)
    );
}

public:
explicit SegTreeBeats(int n, const T &v = T{})
    : len(n), max1(4 * n), max2(4 * n), sum(4 * n), cnt(4 * n) {
    assert(len > 0);
    build(0, 0, len - 1, [&](int) { return v; });
}

template<typename It>
SegTreeBeats(It lo, It hi)
    : len(static_cast<int>(hi - lo)), max1(4 * len), max2(4 * len), sum(4 * len), cnt(4 * len) {
    assert(len > 0);

```

```

    build(0, 0, len - 1, [&](int i) { return *(lo + i); });
}

int size() const { return len; }
T at(int i) { return query_sum(i, i); }

T query_sum(int lo, int hi) {
    assert(0 <= lo && lo <= hi && hi < len);
    return query_sum(0, 0, len - 1, lo, hi);
}

T query_max(int lo, int hi) {
    assert(0 <= lo && lo <= hi && hi < len);
    return query_max(0, 0, len - 1, lo, hi);
}

void chmin(int lo, int hi, const T &t) {
    assert(0 <= lo && lo <= hi && hi < len);
    chmin(0, 0, len - 1, lo, hi, t);
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int64_t> a{5, 4, 3, 2, 1};
    SegTreeBeats<int64_t> t(a.begin(), a.end());
    assert(t.query_sum(0, 4) == 15);
    assert(t.query_max(0, 4) == 5);

    t.chmin(0, 2, 3); // {3, 3, 3, 2, 1}: only the 5 and 4 are clamped down to 3.
    assert(t.at(0) == 3);
    assert(t.at(1) == 3);
    assert(t.query_sum(0, 4) == 12);
    assert(t.query_max(0, 4) == 3);

    t.chmin(0, 4, 2); // {2, 2, 2, 2, 1}.
    assert(t.query_sum(0, 4) == 9);
    assert(t.query_max(0, 3) == 2);

    SegTreeBeats<int> lowest(1, numeric_limits<int>::lowest() + 1);
    lowest.chmin(0, 0, numeric_limits<int>::lowest());
    assert(lowest.at(0) == numeric_limits<int>::lowest());

    // Cross-check against a brute-force array over random clamps and queries.
    vector<int64_t> b{7, 1, 4, 9, 2, 6, 5, 8, 3, 0};
    SegTreeBeats<int64_t> s(b.begin(), b.end());
    int clamps[][3] = {{0, 9, 6}, {2, 5, 3}, {4, 8, 7}, {0, 4, 2}, {3, 9, 5}};
    for (auto &c : clamps) {
        for (int i = c[0]; i <= c[1]; i++) {
            b[i] = min(b[i], static_cast<int64_t>(c[2]));
        }
        s.chmin(c[0], c[1], c[2]);
        for (int lo = 0; lo < 10; lo++) {
            int64_t want_sum = 0, want_max = INT64_MIN;
            for (int hi = lo; hi < 10; hi++) {
                want_sum += b[hi];
                want_max = max(want_max, b[hi]);
                assert(s.query_sum(lo, hi) == want_sum);
                assert(s.query_max(lo, hi) == want_max);
            }
        }
    }
    return 0;
}

```

```

}

```

### 2.5.10 Sparse Segment Tree

A sparse segment tree (also commonly called a dynamic or implicit segment tree) maintains an array over a large index range while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. This implementation uses lazy initialization of nodes to conserve memory: only the nodes covering touched indices are ever allocated, so a huge index range is supported without preallocating the whole tree.

The query operation is defined by an associative aggregate function `combine(a, b)`. Since untouched nodes are implicit, `combine_n(v, len)` must return the aggregate summary of `len` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For range-sum queries, `combine(a, b)` should return `a + b` and `combine_n(v, len)` should return `v * len`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `SparseSegTree<T, N>(v = T{})` constructs an array over indices  $[0, N)$ , with every value implicitly initialized to `v`. Nodes are allocated lazily as indices are touched.
- `at(i)` returns the value at index `i`, where `i` must be in  $[0, N)$ .
- `query(lo, hi)` returns the aggregate of the values at indices in  $[lo, hi]$ . If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `update(lo, hi, d)` modifies the value at each array index in  $[lo, hi]$  by applying the delta `d` to each value.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range  $[lo, hi)$  satisfies `pred()`. As `hi` increases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and `N` is returned if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range  $[lo, hi)$  satisfies `pred()`. As `lo` decreases, `pred()` applied to this aggregate may change only from true to false. The empty range is valid, and 0 is returned if the predicate remains true to the beginning.

For the boundary-search functions, `pred()` takes aggregate `T` values. For `combine = min`, `pred(mn) = (mn > x)` makes `max_right()` stop at the first value `<= x` when extending right, while `min_left()` stops just after the first such value when extending left. With nonnegative values and `combine = sum`, `pred(sum) = (sum <= x)` finds the longest extension with sum at most `x`.

#### Time Complexity:

- $O(1)$  per call to the constructor.
- $O(\log N)$  per call to `at()`, `query()`, `update()`, `max_right()`, and `min_left()`.

#### Space Complexity:

- $O(q \log N)$  for storage after  $q$  updates.
- $O(\log N)$  auxiliary stack space for `query()`, `update()`, `max_right()`, and `min_left()`.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <optional>
#include <vector>

template<typename T, int N = 1000000001>

```

```

class SparseSegTree {
    static_assert(N > 0);

    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T combine_n(const T &v, int64_t len) { return v; }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    struct Node {
        T value, delta;
        bool pending;
        Node *left, *right;

        explicit Node(const T &v) : value(v), pending(false), left(nullptr), right(nullptr) {}
    } *root;

    T init;

    void update_delta(Node *&n, const T &d, int64_t len) {
        if (n == nullptr) {
            n = new Node(combine_n(init, len));
        }
        n->delta = n->pending ? compose_deltas(n->delta, d) : d;
        n->pending = true;
    }

    void push_delta(Node *n, int lo, int hi) {
        if (n == nullptr) {
            return;
        }
        if (n->pending) {
            n->value = apply_delta(n->value, n->delta, hi - lo + 1);
            if (lo != hi) {
                int mid = lo + (hi - lo) / 2;
                update_delta(n->left, n->delta, mid - lo + 1);
                update_delta(n->right, n->delta, hi - mid);
            }
        }
        n->pending = false;
    }

    T query(Node *n, int lo, int hi, int tgt_lo, int tgt_hi) {
        if (n == nullptr) {
            return combine_n(init, tgt_hi - tgt_lo + 1);
        }
        push_delta(n, lo, hi);
        if (lo == tgt_lo && hi == tgt_hi) {
            return n->value;
        }
        int mid = lo + (hi - lo) / 2;
        if (tgt_lo <= mid && mid < tgt_hi) {
            return combine(
                query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
                query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
            );
        }
        if (tgt_lo <= mid) {
            return query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid));
        }
        return query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
    }

    void update(Node *&n, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
        if (n == nullptr) {
            if (hi < tgt_lo || lo > tgt_hi) {
                return;
            }
            n = new Node(combine_n(init, hi - lo + 1));
        }
    }
}

```



```

    } else {
        push_delta(n, lo, hi);
    }
    if (hi < tgt_lo || lo > tgt_hi) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        n->delta = d;
        n->pending = true;
        push_delta(n, lo, hi);
        return;
    }
    int mid = lo + (hi - lo) / 2;
    update(n->left, lo, mid, tgt_lo, tgt_hi, d);
    update(n->right, mid + 1, hi, tgt_lo, tgt_hi, d);
    T left_value = (n->left != nullptr) ? n->left->value : combine_n(init, mid - lo + 1);
    T right_value = (n->right != nullptr) ? n->right->value : combine_n(init, hi - mid);
    n->value = combine(left_value, right_value);
}

template<typename Pred>
int max_right(Node *n, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) {
    if (hi < tgt_lo) {
        return -1;
    }
    if (n != nullptr) {
        push_delta(n, lo, hi);
    }
    T node_value = n != nullptr ? n->value : combine_n(init, hi - lo + 1);
    if (tgt_lo <= lo) {
        T next = acc ? combine(*acc, node_value) : node_value;
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = max_right(n == nullptr ? nullptr : n->left, lo, mid, tgt_lo, pred, acc);
    return res != -1 ? res
        : max_right(n == nullptr ? nullptr : n->right, mid + 1, hi, tgt_lo, pred, acc);
}

template<typename Pred>
int min_left(Node *n, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) {
    if (tgt_hi <= lo) {
        return -1;
    }
    if (n != nullptr) {
        push_delta(n, lo, hi);
    }
    T node_value = n != nullptr ? n->value : combine_n(init, hi - lo + 1);
    if (hi < tgt_hi) {
        T next = acc ? combine(node_value, *acc) : node_value;
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo + 1;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = min_left(n == nullptr ? nullptr : n->right, mid + 1, hi, tgt_hi, pred, acc);
    return res != -1 ? res : min_left(n == nullptr ? nullptr : n->left, lo, mid, tgt_hi, pred, acc);
}

```

```

void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SparseSegTree(const T &v = T{}) : root(nullptr), init(v) {}

    ~SparseSegTree() { clean_up(root); }
    SparseSegTree(const SparseSegTree &) = delete;
    SparseSegTree &operator=(const SparseSegTree &) = delete;

    T at(int i) { return query(i, i); }

    T query(int lo, int hi) {
        assert(0 <= lo && lo <= hi && hi < N);
        return query(root, 0, N - 1, lo, hi);
    }

    void update(int i, const T &d) { update(i, i, d); }

    void update(int lo, int hi, const T &d) {
        assert(0 <= lo && lo <= hi && hi < N);
        update(root, 0, N - 1, lo, hi, d);
    }

    template<typename Pred>
    int max_right(int lo, const Pred &pred) {
        assert(0 <= lo && lo <= N);
        std::optional<T> acc;
        int res = max_right(root, 0, N - 1, lo, pred, acc);
        return res == -1 ? N : res;
    }

    template<typename Pred>
    int min_left(int hi, const Pred &pred) {
        assert(0 <= hi && hi <= N);
        std::optional<T> acc;
        int res = min_left(root, 0, N - 1, hi, pred, acc);
        return res == -1 ? 0 : res;
    }
};

```

## Example Usage

```

using namespace std;

int main() {
    SparseSegTree<int> t(0);
    t.update(0, 6);
    t.update(1, -2);
    t.update(2, 4);
    t.update(3, 8);
    t.update(4, 10);
    vector<int> expected{6, -2, 4, 8, 10};
    for (int i = 0; i < 5; i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
}

```

```

expected = {5, 5, 5, 1, 5};
for (int i = 0; i < 5; i++) {
    assert(t.at(i) == expected[i]);
}
assert(t.query(0, 3) == 1);

// Boundary search over the sparse domain; values at indices 0..4 are now {5, 5, 5, 1, 5}.
assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
return 0;
}

```

### 2.5.11 Persistent Segment Tree

Maintain immutable versions of a fixed-size array while supporting range updates and range aggregate queries. Each update creates a new version by copying only the  $O(\log n)$  nodes on the path it descends, sharing every unchanged subtree with the version it came from. Earlier versions are never disturbed, and an update may branch from any of them, so versions form a tree rather than a line.

Lazy propagation normally works by pushing a pending update down into both children before descending, which rewrites nodes that older versions still point at and so would destroy them here. The fix is to never push. A node keeps its pending delta permanently, and a query accumulates the deltas it passes on the way down, adding each one's contribution to the span it covers. Since a node is only ever read together with its ancestors, an update that stops at a node is still seen by every query that reaches below it, and the nodes above are the only ones that need copying.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “sum” of the target range. Another possible query operation is “min”, in which case `combine(a, b)` should return `min(a, b)`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range increment for range-sum queries. For range-min/range-max queries, `apply_delta(v, d, len)` should return `v + d`.

Deltas here must also commute, which they need not for usual lazy propagation. Namely, for every value and length, `apply_delta(apply_delta(v, d1, len), d2, len)` must equal `apply_delta(apply_delta(v, d2, len), d1, len)`, so that `compose_deltas(d1, d2)` and `compose_deltas(d2, d1)` act identically on every aggregate. The reason is that an update stopping high in the tree leaves the older deltas below it untouched, so a query composes the deltas it passes in order of depth rather than in the order they were applied. Range increment satisfies this and range assignment does not, which is why the latter is the default there and is unavailable here. Pushing each delta into fresh copies of both children instead of leaving it in place lifts the restriction, at the cost of copying two nodes per level rather than one.

- `PersistentSegTree<T>(n, v = T{})` constructs version 0 of an array of size `n` with indices  $[0, n)$ , and all values initialized to `v`.
- `PersistentSegTree<T>(lo, hi)` constructs version 0 from the half-open random-access iterator range  $[lo, hi)$ .
- `size()` returns the size of every version of the array, and `versions()` the number of stored versions, which occupy  $[0, versions())$ .
- `query(version, lo, hi)` returns the aggregate of the values at indices in  $[lo, hi]$  in the given version. If `lo == hi`, then the single specified value is returned.
- `update(version, lo, hi, d)` creates and returns a new version in which the delta `d` has been applied to the value at each array index in  $[lo, hi]$ . The given version is unchanged.

**Time Complexity:**

- $O(n)$  per call to either constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `versions()`.
- $O(\log n)$  per call to `query()` and `update()`.

**Space Complexity:**

- $O(n + q \log n)$  for storage after  $q$  updates.
- $O(\log n)$  auxiliary stack space per call to `query()` and `update()`.

```
#include <cassert>
#include <optional>
#include <vector>

template<typename T>
class PersistentSegTree {
    static T combine(const T &a, const T &b) { return a + b; }
    static T apply_delta(const T &v, const T &d, int len) { return v + d * len; }
    static T compose_deltas(const T &d1, const T &d2) { return d1 + d2; }

    struct Node {
        T value;
        std::optional<T> delta; // Empty when no update is pending on this node's descendants.
        int left, right;

        Node(const T &value, const std::optional<T> &delta, int left = -1, int right = -1)
            : value(value), delta(delta), left(left), right(right) {}
    };

    int len;
    std::vector<Node> nodes;
    std::vector<int> roots;

    int make(const T &value, const std::optional<T> &delta, int left, int right) {
        nodes.emplace_back(value, delta, left, right);
        return static_cast<int>(nodes.size()) - 1;
    }

    template<typename Gen>
    int build(int lo, int hi, const Gen &gen) {
        if (lo == hi) {
            return make(gen(lo), std::nullopt, -1, -1);
        }
        int mid = lo + (hi - lo) / 2;
        int left = build(lo, mid, gen), right = build(mid + 1, hi, gen);
        return make(combine(nodes[left].value, nodes[right].value), std::nullopt, left, right);
    }

    // The carried argument accumulates every ancestor's delta, since none are pushed down.
    T query(int i, int lo, int hi, int tgt_lo, int tgt_hi, const std::optional<T> &carried) const {
        if (tgt_lo <= lo && hi <= tgt_hi) {
            return carried ? apply_delta(nodes[i].value, *carried, hi - lo + 1) : nodes[i].value;
        }
        int mid = lo + (hi - lo) / 2;
        std::optional<T> below = carried;
        if (nodes[i].delta) {
            below = below ? compose_deltas(*below, *nodes[i].delta) : *nodes[i].delta;
        }
        if (tgt_hi <= mid) {
            return query(nodes[i].left, lo, mid, tgt_lo, tgt_hi, below);
        }
        if (tgt_lo > mid) {
            return query(nodes[i].right, mid + 1, hi, tgt_lo, tgt_hi, below);
        }
        return combine(
            query(nodes[i].left, lo, mid, tgt_lo, mid, below),
            query(nodes[i].right, mid + 1, hi, mid + 1, tgt_hi, below),
        );
    }
};
```

```

        query(nodes[i].right, mid + 1, hi, mid + 1, tgt_hi, below)
    );
}

int update(int i, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
    if (tgt_lo <= lo && hi <= tgt_hi) {
        // The delta stays here; every query below this node will pick it up on the way past.
        return make(
            apply_delta(nodes[i].value, d, hi - lo + 1),
            nodes[i].delta ? compose_deltas(*nodes[i].delta, d) : d, nodes[i].left, nodes[i].right
        );
    }
    int mid = lo + (hi - lo) / 2;
    int left = nodes[i].left, right = nodes[i].right;
    if (tgt_lo <= mid) {
        left = update(left, lo, mid, tgt_lo, tgt_hi, d);
    }
    if (tgt_hi > mid) {
        right = update(right, mid + 1, hi, tgt_lo, tgt_hi, d);
    }
    T merged = combine(nodes[left].value, nodes[right].value);
    return make(
        nodes[i].delta ? apply_delta(merged, *nodes[i].delta, hi - lo + 1) : merged, nodes[i].delta,
        left, right
    );
}

public:
    explicit PersistentSegTree(int n, const T &v = T{}) : len(n) {
        assert(n > 0);
        roots.push_back(build(0, n - 1, [&](int) { return v; }));
    }

    template<typename It>
    PersistentSegTree(It lo, It hi) : len(static_cast<int>(hi - lo)) {
        assert(len > 0);
        roots.push_back(build(0, len - 1, [&](int i) { return *(lo + i); }));
    }

    int size() const { return len; }
    int versions() const { return static_cast<int>(roots.size()); }

    T query(int version, int lo, int hi) const {
        assert(version >= 0 && version < versions() && lo >= 0 && lo <= hi && hi < len);
        return query(roots[version], 0, len - 1, lo, hi, std::nullopt);
    }

    int update(int version, int lo, int hi, const T &d) {
        assert(version >= 0 && version < versions() && lo >= 0 && lo <= hi && hi < len);
        roots.push_back(update(roots[version], 0, len - 1, lo, hi, d));
        return versions() - 1;
    }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 2, 3, 4, 5};
    PersistentSegTree<int> t(a.begin(), a.end());
    assert(t.size() == 5 && t.versions() == 1);
    assert(t.query(0, 0, 4) == 15 && t.query(0, 1, 3) == 9);

    int v1 = t.update(0, 1, 3, 10); // Adds 10 to indices 1, 2, and 3.
}

```

```

assert(v1 == 1);
assert(t.query(1, 1, 3) == 39 && t.query(1, 0, 4) == 45);
assert(t.query(1, 0, 0) == 1 && t.query(1, 4, 4) == 5);
assert(t.query(0, 1, 3) == 9); // Version 0 is untouched.

// Updates compose, and any version may be branched from more than once.
int v2 = t.update(1, 0, 4, 1);
assert(t.query(2, 0, 4) == 50);
int v3 = t.update(1, 0, 1, 100);
assert(v3 == 3 && t.query(3, 0, 1) == 213 && t.query(2, 0, 1) == 15);

// A partially overlapping update reaches the same leaves through different nodes.
int v4 = t.update(3, 1, 2, 5);
assert(t.query(4, 1, 1) == 117 && t.query(4, 2, 2) == 18 && t.query(4, 3, 3) == 14);
assert(t.versions() == 5 && v2 == 2 && v4 == 4);
return 0;
}

```

## 2.5.12 Merge Sort Tree

A merge sort tree is a static segment tree in which every node stores the sorted multiset of the values in its range. The nodes are exactly the ranges visited by a merge sort, and each node's sorted list is the merge of its two children's lists. This supports queries that count, within a range of indices, how many values fall below a threshold or inside a value interval: decompose the index range into  $O(\log n)$  canonical nodes and binary search each node's sorted list.

The structure is built once and not modified afterward. All index ranges are inclusive  $[l_o, h_i]$  with 0-based indices, and values may be of any comparable type.

- `MergeSortTree<T>(a)` builds the tree over the array `a`.
- `size()` returns the size of the array.
- `count_leq(lo, hi, x)` returns the number of indices `i` in  $[l_o, h_i]$  with `a[i] ≤ x`.
- `count_in(lo, hi, x, y)` returns the number of indices `i` in  $[l_o, h_i]$  such that `a[i] ∈ [x, y]`.

### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log^2 n)$  per call to `count_leq()` and `count_in()`.

### Space Complexity:

- $O(n \log n)$  for storage of the tree.
- $O(\log n)$  auxiliary stack space per query.

```

#include <algorithm>
#include <cassert>
#include <iterator>
#include <vector>

template<typename T>
class MergeSortTree {
    int len;
    std::vector<std::vector<T>> tree;

    void build(int i, int lo, int hi, const std::vector<T> &a) {
        if (lo == hi) {
            tree[i] = {a[lo]};
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2, lo, mid, a);
        build(i * 2 + 1, mid + 1, hi, a);
    }
};

```

```

    std::merge(
        tree[i * 2].begin(), tree[i * 2].end(), tree[i * 2 + 1].begin(), tree[i * 2 + 1].end(),
        std::back_inserter(tree[i])
    );
}

template<typename Fn>
int query(int i, int lo, int hi, int tgt_lo, int tgt_hi, const Fn &count_node) const {
    if (tgt_hi < lo || hi < tgt_lo) {
        return 0;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        return count_node(tree[i]);
    }
    int mid = lo + (hi - lo) / 2;
    return query(i * 2, lo, mid, tgt_lo, tgt_hi, count_node) +
        query(i * 2 + 1, mid + 1, hi, tgt_lo, tgt_hi, count_node);
}

public:
explicit MergeSortTree(const std::vector<T> &a) : len(a.size()), tree(4 * a.size()) {
    assert(len > 0);
    build(1, 0, len - 1, a);
}

int size() const { return len; }

int count_leq(int lo, int hi, const T &x) const {
    assert(0 <= lo && lo <= hi && hi < len);
    return query(1, 0, len - 1, lo, hi, [&](const std::vector<T> &v) {
        return static_cast<int>(std::upper_bound(v.begin(), v.end(), x) - v.begin());
    });
}

int count_in(int lo, int hi, const T &x, const T &y) const {
    assert(0 <= lo && lo <= hi && hi < len);
    assert(!(y < x));
    return query(1, 0, len - 1, lo, hi, [&](const std::vector<T> &v) {
        return static_cast<int>(
            std::upper_bound(v.begin(), v.end(), y) - std::lower_bound(v.begin(), v.end(), x)
        );
    });
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{5, 2, 8, 6, 1, 9, 3};
    MergeSortTree<int> t(a);

    assert(t.size() == 7);
    assert(t.count_leq(0, 6, 5) == 4);    // 5, 2, 1, 3
    assert(t.count_leq(2, 4, 6) == 2);    // 6, 1
    assert(t.count_in(0, 6, 3, 8) == 4);    // 5, 8, 6, 3
    assert(t.count_in(1, 5, 2, 6) == 2);    // 2, 6 (8, 1, 9 excluded)
    return 0;
}

```

### 2.5.13 Wavelet Tree

A wavelet tree is a static structure over an integer array with values in a known range  $[\text{min\_val}, \text{max\_val}]$ . The root splits the value range at its midpoint, recording for every prefix of the array how many of its elements belong to the lower value half, then stably partitions the array so that those elements form the left child and the rest form the right child. Recursing on each half builds a balanced tree of height  $O(\log s)$ , where  $s$  is the size of the value range.

The recorded prefix counts let a query at any node translate a range of array positions into the corresponding range of positions in either child, so a single root-to-leaf descent answers order statistics and rank queries. This is strictly more powerful than a merge sort tree: besides counting values below a threshold, it reports the  $k$ -th smallest value of a range in  $O(\log s)$  time.

All position ranges use 0-based indices and closed intervals  $[\text{lo}, \text{hi}]$ . If the values are large or sparse, compress them to a small contiguous range first.

- `WaveletTree(a, min_val, max_val)` builds the tree over the array `a`, whose values must all lie in  $[\text{min\_val}, \text{max\_val}]$ .
- `size()` returns the size of the array.
- `kth_smallest(lo, hi, k)` returns the  $k$ -th smallest value among positions  $[\text{lo}, \text{hi}]$ , where  $k$  is 1-based (so  $k == 1$  returns the minimum).
- `count_leq(lo, hi, x)` returns the number of positions  $i \in [\text{lo}, \text{hi}]$  such that `a[i] ≤ x`.
- `count_in(lo, hi, x, y)` returns the number of positions  $i \in [\text{lo}, \text{hi}]$  such that `a[i] ∈ [x, y]`.

#### Time Complexity:

- $O(n \log s)$  per call to the constructor, where  $n$  is the size of the array and  $s$  is the size of the value range.
- $O(1)$  per call to `size()`.
- $O(\log s)$  per call to `kth_smallest()`, `count_leq()`, and `count_in()`.

#### Space Complexity:

- $O(n \log s)$  for storage of the tree.
- $O(\log s)$  auxiliary stack space per query.

```
#include <algorithm>
#include <cassert>
#include <climits>
#include <cstdint>
#include <memory>
#include <vector>

class WaveletTree {
    int min_val, max_val; // The value range this node covers.
    std::unique_ptr<WaveletTree> left, right;
    std::vector<int> b; // b[i] = number of the first i elements that go to the left child.

    WaveletTree(std::vector<int>::iterator lo, std::vector<int>::iterator hi, int mn, int mx)
        : min_val(mn), max_val(mx) {
        assert(min_val <= max_val);
        b.reserve((hi - lo) + 1);
        b.push_back(0);
        if (lo >= hi) {
            return;
        }
        int mid = min_val + static_cast<int>((static_cast<int64_t>(max_val) - min_val) / 2);
        auto go_left = [mid](int v) { return v <= mid; };
        for (auto it = lo; it != hi; ++it) {
            assert(min_val <= *it && *it <= max_val);
            b.push_back(b.back() + (go_left(*it) ? 1 : 0));
        }
        if (min_val == max_val) {
            return;
        }
    }
};
```



```

    auto pivot = std::stable_partition(lo, hi, go_left);
    left.reset(new WaveletTree(lo, pivot, min_val, mid));
    right.reset(new WaveletTree(pivot, hi, mid + 1, max_val));
}

// 1-based positions [lo, hi] for the recursion below.
int kth(int lo, int hi, int k) const {
    if (min_val == max_val) {
        return min_val;
    }
    int in_left = b[hi] - b[lo - 1];
    if (k <= in_left) {
        return left->kth(b[lo - 1] + 1, b[hi], k);
    }
    return right->kth(lo - b[lo - 1], hi - b[hi], k - in_left);
}

int leq(int lo, int hi, int x) const {
    if (lo > hi || x < min_val) {
        return 0;
    }
    if (max_val <= x) {
        return hi - lo + 1;
    }
    return left->leq(b[lo - 1] + 1, b[hi], x) + right->leq(lo - b[lo - 1], hi - b[hi], x);
}

public:
WaveletTree(std::vector<int> a, int min_val, int max_val)
    : WaveletTree(a.begin(), a.end(), min_val, max_val) {}

int size() const { return static_cast<int>(b.size()) - 1; }

int kth_smallest(int lo, int hi, int k) const {
    assert(0 <= lo && lo <= hi && hi < size());
    assert(1 <= k && k <= hi - lo + 1);
    return kth(lo + 1, hi + 1, k);
}

int count_leq(int lo, int hi, int x) const {
    assert(0 <= lo && lo <= hi && hi < size());
    return leq(lo + 1, hi + 1, x);
}

int count_in(int lo, int hi, int x, int y) const {
    assert(0 <= lo && lo <= hi && hi < size());
    assert(x <= y);
    return leq(lo + 1, hi + 1, y) - (x <= min_val ? 0 : leq(lo + 1, hi + 1, x - 1));
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{5, 2, 8, 6, 1, 9, 3};
    WaveletTree t(a, 1, 9);

    assert(t.size() == 7);
    assert(t.kth_smallest(0, 6, 1) == 1); // Minimum of the whole array.
    assert(t.kth_smallest(0, 6, 4) == 5); // Median of the whole array.
    assert(t.kth_smallest(2, 4, 2) == 6); // {8, 6, 1} -> 6 is 2nd smallest.

    assert(t.count_leq(0, 6, 5) == 4); // 5, 2, 1, 3
}

```

```

assert(t.count_in(0, 6, 3, 8) == 4); // 5, 8, 6, 3
assert(t.count_in(1, 5, 2, 6) == 2); // 2, 6

vector<int> extremes{INT_MIN, 0, INT_MAX};
WaveletTree extreme_tree(extremes, INT_MIN, INT_MAX);
assert(extreme_tree.count_in(0, 2, INT_MIN, 0) == 2);
return 0;
}

```

### 2.5.14 Cartesian Tree

Build the min-Cartesian tree of an array in linear time. A Cartesian tree is a binary tree whose inorder traversal is the original array order and whose heap property is determined by array values. The comparator `comp` defines the value ordering and defaults to `std::less<>`, producing a min-Cartesian tree in which each parent precedes or is equivalent to its children. Each subtree therefore represents a contiguous array interval whose first value in comparator order is at the subtree root. Equivalent values are broken by position, so the earlier one becomes an ancestor of the later one.

With the default comparator, this gives a direct connection to range minimum queries: the minimum index in  $[lo, hi]$  is the lowest common ancestor of indices `lo` and `hi`. The included search finds that node without extra LCA preprocessing. Starting at the root, it moves right while the current index is left of the range and left while it is right of the range; the first index inside the range is its minimum. This takes  $O(h)$  time for tree height  $h$ , which can be  $O(n)$  for a skewed tree. For many worst-case-fast queries, preprocess the tree for LCA or use the sparse table in 2.5.1 instead. The explicit tree is also useful for interval divide-and-conquer and tree DP; the largest-rectangle application is implemented directly with a monotone stack in 1.2.4.

- `CartesianTree(a, comp = std::less<>())` constructs the tree for array `a`.
- `root` is the root index, or `-1` if the array is empty. The arrays `parent`, `left`, and `right` store the neighboring node indices, using `-1` when a neighbor is absent.
- `range_min_index(lo, hi)` returns the index of the first value in comparator order within  $[lo, hi]$ .

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the array size.
- $O(h)$  per call to `range_min_index()`, where  $h$  is the tree height and may be  $O(n)$ .

**Space Complexity:**  $O(n)$  for tree arrays and the monotone stack.

```

#include <cassert>
#include <functional>
#include <vector>

struct CartesianTree {
    int root;
    std::vector<int> parent, left, right;

    template<typename T, typename Compare = std::less<>>
    explicit CartesianTree(const std::vector<T> &a, Compare comp = Compare{})
        : root(-1), parent(a.size(), -1), left(a.size(), -1), right(a.size(), -1) {
        std::vector<int> st;
        for (int i = 0, n = static_cast<int>(a.size()); i < n; i++) {
            int last = -1;
            while (!st.empty() && comp(a[i], a[st.back()])) {
                last = st.back();
                st.pop_back();
            }
            if (!st.empty()) {
                right[st.back()] = i;
                parent[i] = st.back();
            }
            if (last != -1) {

```

```

        left[i] = last;
        parent[last] = i;
    }
    st.push_back(i);
}
root = st.empty() ? -1 : st[0];
}

int size() const { return static_cast<int>(parent.size()); }

int range_min_index(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi < static_cast<int>(parent.size()));
    int i = root;
    while (i < lo || hi < i) {
        i = i < lo ? right[i] : left[i];
    }
    return i;
}
};

```

### Example Usage

```

using namespace std;

int main() {
    vector<int> a{3, 2, 6, 1, 9};
    CartesianTree t(a);
    assert(t.size() == 5);
    assert(t.root == 3); // The global minimum is a[3] = 1.
    assert(t.parent[1] == 3);
    assert(t.left[3] == 1);
    assert(t.right[3] == 4);
    assert(t.range_min_index(0, 2) == 1); // min({3, 2, 6}) is a[1].
    assert(t.range_min_index(2, 4) == 3); // min({6, 1, 9}) is a[3].

    vector<int> equal{4, 2, 2, 3};
    CartesianTree equal_tree(equal);
    assert(equal_tree.range_min_index(1, 3) == 1); // Earlier equal minima win ties.

    CartesianTree max_tree(a, greater<int>());
    assert(max_tree.root == 4);
    assert(max_tree.range_min_index(0, 2) == 2); // First under greater<int>: the numeric maximum.
    return 0;
}

```

#### 2.5.15 Implicit Treap

Maintain a dynamically-sized array using a balanced binary search tree while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. A treap maintains a balanced binary tree structure by preserving the heap property on the randomly generated priority values of nodes, thereby making insertions and deletions run in  $O(\log n)$  with high probability.

Array indices are implicit rather than stored as keys: subtree sizes determine each node’s current position, so inserting or erasing a value does not require renumbering later elements. The internal `split(t, left, right, i)` operation separates the first `i` values from the rest, and `merge()` concatenates two such sequences. Range operations isolate their target with two splits, modify or inspect its root, and merge the three pieces back together.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

Range reversal is also propagated lazily. Since `combine()` may be order-sensitive, each node stores the aggregate of its subtree in both forward and reverse order. Reversing a subtree then swaps its children and these two aggregates before marking its descendants for later reversal.

- `ImplicitTreap<T>(n = 0, v = T{})` constructs an array of size `n` with indices `[0, n)`, with all values initialized to `v`.
- `ImplicitTreap<T>(lo, hi)` constructs an array from the half-open iterator range `[lo, hi)`.
- `size()` returns the size of the array.
- `empty()` returns whether the array is empty.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the aggregate of the values at indices in `[lo, hi]`.
- `update(lo, hi, d)` applies the delta `d` to every index in `[lo, hi]`.
- `insert(i, v)` inserts a new value `v` before index `i`, shifting later elements one position right.
- `erase(i)` removes the element at index `i`, shifting later elements one position left.
- `update(i, d)` applies the delta `d` to the single index `i`.
- `push_back(v)` appends a new value `v` to the end of the array.
- `pop_back()` removes the last element of the array.
- `reverse(lo, hi)` reverses the order of the values in `[lo, hi]`.

The query and update operations match those of the point- and range-update segment trees in this section; `insert()`, `erase()`, `push_back()`, and `pop_back()` are additionally analogous to those of `std::vector` (here, `insert()` and `erase()` take an index instead of an iterator).

### Time Complexity:

- $O(n \log n)$  on average per call to both constructors, where  $n$  is the size of the array.
- $O(1)$  per call to `size()` and `empty()`.
- $O(\log n)$  on average per call to all other operations.

### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(1)$  auxiliary for `size()` and `empty()`.
- $O(\log n)$  auxiliary stack space for all other operations.

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <cstdint>
#include <utility>

template<typename T>
class ImplicitTreap {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    struct Node {
        static uint32_t rand32() {
            static uint32_t x =
                static_cast<uint32_t>(std::chrono::steady_clock::now().time_since_epoch().count()) | 1U;
            x ^= x << 13;
            x ^= x >> 17;
            x ^= x << 5;
        }
    };
};
```

```

    return x;
}

T value, subtree_value, reverse_value, delta;
bool pending, reversed;
int size;
uint32_t priority;
Node *left, *right;

explicit Node(const T &v)
: value(v),
  subtree_value(v),
  reverse_value(v),
  pending(false),
  reversed(false),
  size(1),
  priority(rand32()),
  left(nullptr),
  right(nullptr) {}

} *root;

static int size(Node *n) { return (n == nullptr) ? 0 : n->size; }

static void update_value(Node *n) {
    if (n == nullptr) {
        return;
    }
    n->subtree_value = n->left != nullptr ? combine(n->left->subtree_value, n->value) : n->value;
    if (n->right != nullptr) {
        n->subtree_value = combine(n->subtree_value, n->right->subtree_value);
    }
    n->reverse_value = n->right != nullptr ? combine(n->right->reverse_value, n->value) : n->value;
    if (n->left != nullptr) {
        n->reverse_value = combine(n->reverse_value, n->left->reverse_value);
    }
    n->size = 1 + size(n->left) + size(n->right);
}

static void update_delta(Node *n, const T &d) {
    if (n != nullptr) {
        n->value = apply_delta(n->value, d, 1);
        n->subtree_value = apply_delta(n->subtree_value, d, n->size);
        n->reverse_value = apply_delta(n->reverse_value, d, n->size);
        n->delta = n->pending ? compose_deltas(n->delta, d) : d;
        n->pending = true;
    }
}

static void reverse_subtree(Node *n) {
    if (n != nullptr) {
        std::swap(n->left, n->right);
        std::swap(n->subtree_value, n->reverse_value);
        n->reversed = !n->reversed;
    }
}

static void push(Node *n) {
    if (n == nullptr) {
        return;
    }
    if (n->pending) {
        update_delta(n->left, n->delta);
        update_delta(n->right, n->delta);
        n->pending = false;
    }
    if (n->reversed) {
        reverse_subtree(n->left);
        reverse_subtree(n->right);
    }
}

```

```

    n->reversed = false;
}
}

static void merge(Node *&n, Node *left, Node *right) {
    push(left);
    push(right);
    if (left == nullptr) {
        n = right;
    } else if (right == nullptr) {
        n = left;
    } else if (left->priority < right->priority) {
        merge(left->right, left->right, right);
        n = left;
    } else {
        merge(right->left, left, right->left);
        n = right;
    }
    update_value(n);
}

static void split(Node *n, Node *&left, Node *&right, int i) {
    push(n);
    if (n == nullptr) {
        left = right = nullptr;
    } else if (i <= size(n->left)) {
        split(n->left, left, n->left, i);
        right = n;
    } else {
        split(n->right, n->right, right, i - size(n->left) - 1);
        left = n;
    }
    update_value(n);
}

static void insert(Node *&n, Node *new_node, int i) {
    push(n);
    if (n == nullptr) {
        n = new_node;
    } else if (new_node->priority < n->priority) {
        split(n, new_node->left, new_node->right, i);
        n = new_node;
    } else if (i <= size(n->left)) {
        insert(n->left, new_node, i);
    } else {
        insert(n->right, new_node, i - size(n->left) - 1);
    }
    update_value(n);
}

static void erase(Node *&n, int i) {
    assert(n != nullptr);
    push(n);
    if (i == size(n->left)) {
        Node *left = n->left, *right = n->right;
        delete n;
        merge(n, left, right);
    } else if (i < size(n->left)) {
        erase(n->left, i);
    } else {
        erase(n->right, i - size(n->left) - 1);
    }
    update_value(n);
}

static Node *select(Node *n, int i) {
    assert(n != nullptr);
    push(n);

```

```

    if (i < size(n->left)) {
        return select(n->left, i);
    }
    if (i > size(n->left)) {
        return select(n->right, i - size(n->left) - 1);
    }
    return n;
}

void clean_up(Node *&n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit ImplicitTreap(int n = 0, const T &v = T{}) : root(nullptr) {
        assert(n >= 0);
        for (int i = 0; i < n; i++) {
            push_back(v);
        }
    }

    template<typename It>
    ImplicitTreap(It lo, It hi) : root(nullptr) {
        for (; lo != hi; ++lo) {
            push_back(*lo);
        }
    }

    ~ImplicitTreap() { clean_up(root); }
    ImplicitTreap(const ImplicitTreap &) = delete;
    ImplicitTreap &operator=(const ImplicitTreap &) = delete;
    int size() const { return size(root); }
    bool empty() const { return root == nullptr; }

    T at(int i) const {
        assert(0 <= i && i < size());
        return select(root, i)->value;
    }

    T query(int lo, int hi) {
        assert(0 <= lo && lo <= hi && hi < size());
        Node *l1, *r1, *l2, *r2, *t;
        split(root, l1, r1, hi + 1);
        split(l1, l2, r2, lo);
        T res = r2->subtree_value;
        merge(t, l2, r2);
        merge(root, t, r1);
        return res;
    }

    void update(int lo, int hi, const T &d) {
        assert(0 <= lo && lo <= hi && hi < size());
        Node *l1, *r1, *l2, *r2, *t;
        split(root, l1, r1, hi + 1);
        split(l1, l2, r2, lo);
        update_delta(r2, d);
        merge(t, l2, r2);
        merge(root, t, r1);
    }

    void insert(int i, const T &v) {
        assert(0 <= i && i <= size());
        insert(root, new Node(v), i);
    }

```

```

void erase(int i) {
    assert(0 <= i && i < size());
    erase(root, i);
}

void update(int i, const T &d) { update(i, i, d); }
void push_back(const T &v) { insert(size(), v); }
void pop_back() { erase(size() - 1); }

void reverse(int lo, int hi) {
    assert(0 <= lo && lo <= hi && hi < size());
    Node *l1, *r1, *l2, *r2, *t;
    split(root, l1, r1, hi + 1);
    split(l1, l2, r2, lo);
    reverse_subtree(r2);
    merge(t, l2, r2);
    merge(root, t, r1);
}
};

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

vector<int> values(ImplicitTreap<int> &t) {
    vector<int> result;
    for (int i = 0; i < t.size(); i++) {
        result.push_back(t.at(i));
    }
    return result;
}

int main() {
    vector<int> a{99, -2, 1, 8, 10};
    ImplicitTreap<int> t(a.begin(), a.end());

    // Append 11, then append and remove 12.
    t.push_back(11);
    t.push_back(12);
    t.pop_back();
    assert(t.size() == 6);
    assert(t.at(5) == 11);
    assert(t.query(0, t.size() - 1) == -2);
    assert((values(t) == vector<int>{99, -2, 1, 8, 10, 11}));

    // Replace the first value by inserting 90 before it and erasing the old 99.
    t.insert(0, 90);
    t.erase(1);
    assert(t.at(0) == 90);
    assert(t.at(1) == -2);
    assert((values(t) == vector<int>{90, -2, 1, 8, 10, 11}));

    // Assign 2 to the first two values.
    t.update(0, 1, 2);
    assert(t.at(0) == 2);
    assert(t.at(1) == 2);
    assert(t.query(0, t.size() - 1) == 1);
    assert((values(t) == vector<int>{2, 2, 1, 8, 10, 11}));

    // Reverse the middle range: {2, 2, 1, 8, 10, 11} becomes {2, 10, 8, 1, 2, 11}.
    t.reverse(1, 4);
    assert(t.at(0) == 2);
    assert(t.at(1) == 10);
}

```



```

assert(t.at(4) == 2);
assert(t.at(5) == 11);
assert(t.query(0, t.size() - 1) == 1);
assert((values(t) == vector<int>{2, 10, 8, 1, 2, 11}));
return 0;
}

```

## 2.6 Range Queries in Two Dimensions

### 2.6.1 2D Segment Tree

Maintain a dense two-dimensional array while supporting point assignments and rectangle queries. A 2D segment tree applies the ordinary segment-tree decomposition independently to rows and columns: each row-tree node contains a complete segment tree over its columns, so a rectangle decomposes into  $O(\log(R) \cdot \log(C))$  canonical blocks.

The query operation is defined by a commutative associative `combine(a, b)`. The default below computes the minimum value in a rectangle. For rectangle maximum queries, use `std::max`; for rectangle sums, use addition.

Point updates are defined by `apply_delta(v, d)`, which applies delta `d` to the current value `v`. The default below assigns `d`; for point increments, return `v + d` instead.

Use this when the grid is small enough for  $O(R \cdot C)$  storage and queries need an aggregate other than addition. For additive rectangle sums, a 2D Fenwick tree is simpler and has smaller constants. Use the sparse 2D segment tree in the next section when the coordinate range is too large to allocate dense storage.

- `SegTree2D<T>(rows, cols, v = T{})` constructs a `rows` by `cols` array with 0-based indices, with all entries initialized to `v`.
- `SegTree2D<T>(a)` constructs the tree from the matrix `a`.
- `num_rows()` and `num_cols()` return the dimensions of the array.
- `at(r, c)` returns the value at index `(r, c)`.
- `query(r1, c1, r2, c2)` returns the aggregate over the rectangle with rows in `[r1, r2]` and columns in `[c1, c2]`.
- `update(r, c, d)` applies delta `d` to the entry at index `(r, c)`.

#### Time Complexity:

- $O(R \cdot C)$  per call to either constructor, where  $R$  and  $C$  are the matrix dimensions.
- $O(\log(R) \cdot \log(C))$  per call to `update()` and `query()`.
- $O(1)$  per call to `num_rows()`, `num_cols()`, and `at()`.

#### Space Complexity:

- $O(R \cdot C)$  for storage of the segment tree.
- $O(1)$  auxiliary for all operations.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <vector>

template<typename T>
class SegTree2D {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int rows, cols;
    std::vector<std::vector<T>> tree;

    template<typename Gen>
    void build(const Gen &gen) {

```

```

tree.assign(2 * rows, std::vector<T>(2 * cols, gen(0, 0)));
for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        tree[r + rows][c + cols] = gen(r, c);
    }
}
for (int r = rows; r < 2 * rows; r++) {
    for (int c = cols - 1; c > 0; c--) {
        tree[r][c] = combine(tree[r][2 * c], tree[r][2 * c + 1]);
    }
}
for (int r = rows - 1; r > 0; r--) {
    for (int c = 1; c < 2 * cols; c++) {
        tree[r][c] = combine(tree[2 * r][c], tree[2 * r + 1][c]);
    }
}
}

T query_columns(int r, int c1, int c2) const {
    std::optional<T> res;
    for (c1 += cols, c2 += cols + 1; c1 < c2; c1 /= 2, c2 /= 2) {
        if ((c1 & 1) != 0) {
            res = res ? combine(*res, tree[r][c1++]) : tree[r][c1++];
        }
        if ((c2 & 1) != 0) {
            res = res ? combine(*res, tree[r][--c2]) : tree[r][--c2];
        }
    }
    return *res;
}

public:
SegTree2D(int rows, int cols, const T &v = T{}) : rows(rows), cols(cols) {
    assert(rows > 0 && cols > 0);
    build([&](int, int) { return v; });
}

explicit SegTree2D(const std::vector<std::vector<T>> &a)
    : rows(static_cast<int>(a.size())), cols(a.empty() ? 0 : static_cast<int>(a[0].size())) {
    assert(rows > 0 && cols > 0);
    build([&](int r, int c) { return a[r][c]; });
}

int num_rows() const { return rows; }
int num_cols() const { return cols; }

T at(int r, int c) const {
    assert(0 <= r && r < rows && 0 <= c && c < cols);
    return tree[r + rows][c + cols];
}

T query(int r1, int c1, int r2, int c2) const {
    assert(0 <= r1 && r1 <= r2 && r2 < rows);
    assert(0 <= c1 && c1 <= c2 && c2 < cols);
    std::optional<T> res;
    for (r1 += rows, r2 += rows + 1; r1 < r2; r1 /= 2, r2 /= 2) {
        if ((r1 & 1) != 0) {
            T value = query_columns(r1++, c1, c2);
            res = res ? combine(*res, value) : value;
        }
        if ((r2 & 1) != 0) {
            T value = query_columns(--r2, c1, c2);
            res = res ? combine(*res, value) : value;
        }
    }
    return *res;
}

```

```

void update(int r, int c, const T &d) {
    assert(0 <= r && r < rows && 0 <= c && c < cols);
    int row = r + rows, col = c + cols;
    tree[row][col] = apply_delta(tree[row][col], d);
    for (int j = col; j > 1; j /= 2) {
        tree[row][j / 2] = combine(tree[row][j], tree[row][j ^ 1]);
    }
    for (row /= 2; row > 0; row /= 2) {
        tree[row][col] = combine(tree[2 * row][col], tree[2 * row + 1][col]);
        for (int j = col; j > 1; j /= 2) {
            tree[row][j / 2] = combine(tree[row][j], tree[row][j ^ 1]);
        }
    }
}
};

```

## Example Usage

```

using namespace std;

int main() {
    SegTree2D<int> t(3, 4, 100);
    t.update(0, 1, 8);
    t.update(1, 2, 3);
    t.update(2, 0, 5);
    assert(t.num_rows() == 3 && t.num_cols() == 4);
    assert(t.at(0, 1) == 8);
    assert(t.query(0, 0, 1, 3) == 3);
    assert(t.query(2, 0, 2, 3) == 5);

    SegTree2D<int> from_matrix(vector<vector<int>>>{{7, 2, 6}, {4, 9, 1}});
    assert(from_matrix.query(0, 0, 1, 1) == 2);
    assert(from_matrix.query(0, 1, 1, 2) == 1);
    return 0;
}

```

### 2.6.2 Sparse 2D Segment Tree

Maintain a two-dimensional array over a huge grid while supporting dynamic queries of rectangular subarrays and dynamic updates of individual indices. This is a sparse (a.k.a. dynamic or implicit) 2D segment tree: row and column nodes are allocated lazily as cells are touched, so large coordinate bounds are supported without allocating the full grid.

The query operation is defined by a commutative associative aggregate function `combine(a, b)`. Because untouched regions are implicit, `combine_n(v, area)` must return the aggregate summary of `area` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For rectangle-sum queries, `combine(a, b)` should return `a + b` and `combine_n(v, area)` should return `v * area`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at one updated cell. The default code below defines updates that “set” the chosen cell to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

Rows and columns are split independently, so every rectangle query decomposes into  $O(\log(R) \cdot \log(C))$  canonical rectangles regardless of whether it is thin, off-center, or otherwise adversarially placed.

Use the dense 2D segment tree in the previous section when the grid is modest enough for  $O(R \cdot C)$  storage; it is simpler and faster. Use this sparse version when the coordinate range is huge but only a small fraction of cells are updated. For dense additive rectangle sums, prefer the 2D Fenwick tree in 2.7.5.

- `SparseSegTree2D<T, R, C>(v = T{})` constructs a two-dimensional array over rows  $[0, R)$  and columns  $[0, C)$ . All array values are implicitly initialized to `v`. Nodes are allocated lazily as indices are touched.

- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `combine()` applied to every value in the rectangular region consisting of rows in `[r1, r2]` and columns in `[c1, c2]`.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `apply_delta(v, d)`.

#### Time Complexity:

- $O(1)$  per call to the constructor.
- $O(\log(R) \cdot \log(C))$  per call to `at()`, `query()`, and `update()`.

#### Space Complexity:

- $O(n \cdot \log(R) \cdot \log(C))$  for storage after  $n$  point updates.
- $O(\log(R) + \log(C))$  auxiliary stack space for `at()`, `query()`, and `update()`.

```
#include <algorithm>
#include <cassert>
#include <stdint>
#include <optional>

template<typename T, int R = 1000000001, int C = 1000000001>
class SparseSegTree2D {
    static_assert(R > 0 && C > 0);

    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T combine_n(const T &v, int64_t area) { return v; }
    static T apply_delta(const T &v, const T &d) { return d; }

    struct InnerNode {
        T value;
        int lo, hi;
        InnerNode *left, *right;

        InnerNode(int lo, int hi, const T &v)
            : value(v), lo(lo), hi(hi), left(nullptr), right(nullptr) {}
    };

    struct OuterNode {
        InnerNode inner;
        int lo, hi;
        OuterNode *left, *right;

        OuterNode(int lo, int hi, const T &v)
            : inner(0, C - 1, v), lo(lo), hi(hi), left(nullptr), right(nullptr) {}
    } *root;

    T init;

    static int64_t length(int lo, int hi) { return hi - lo + 1LL; }
    static void append_result(std::optional<T> &res, const T &v) { res = res ? combine(*res, v) : v; }

    template<typename Node, typename Get>
    T query_nodes(const Node *n, int qlo, int qhi, int64_t span, const Get &get) const {
        if (n == nullptr) {
            return combine_n(init, span * length(qlo, qhi));
        }
        int lo = n->lo, hi = n->hi, mid = lo + (hi - lo) / 2;
        std::optional<T> res;
        if (qlo < lo) {
            append_result(res, combine_n(init, span * length(qlo, std::min(qhi, lo - 1))));
        }
        int ql = std::max(qlo, lo), qr = std::min(qhi, hi);
        if (ql <= qr) {
            if (ql == lo && qr == hi) {
                append_result(res, get(n));
            } else {
                if (ql <= mid) {

```

```

        append_result(res, query_nodes(n->left, ql, std::min(qr, mid), span, get));
    }
    if (mid < qr) {
        append_result(res, query_nodes(n->right, std::max(ql, mid + 1), qr, span, get));
    }
}
}
if (hi < qhi) {
    append_result(res, combine_n(init, span * length(std::max(qlo, hi + 1), qhi)));
}
return *res;
}

T query_inner(const InnerNode *n, int c1, int c2, int64_t rows) const {
    return query_nodes(n, c1, c2, rows, [](const InnerNode *node) { return node->value; });
}

T query_outer(const OuterNode *n, int r1, int r2, int c1, int c2) const {
    return query_nodes(n, r1, r2, length(c1, c2), [&](const OuterNode *node) {
        return query_inner(&node->inner, c1, c2, length(node->lo, node->hi));
    });
}

template<typename Apply>
void update_inner(InnerNode *n, int c, int64_t rows, const Apply &apply) {
    int lo = n->lo, hi = n->hi, mid = lo + (hi - lo) / 2;
    if (lo == hi) {
        n->value = apply(n->value);
        return;
    }
    InnerNode *&target = (c <= mid) ? n->left : n->right;
    if (target == nullptr) {
        target = new InnerNode(c, c, combine_n(init, rows));
    }
    if (target->lo <= c && c <= target->hi) {
        update_inner(target, c, rows, apply);
    } else {
        int split_lo = lo, split_hi = hi, split_mid = mid;
        do {
            if (c <= split_mid) {
                split_hi = split_mid;
            } else {
                split_lo = split_mid + 1;
            }
            split_mid = split_lo + (split_hi - split_lo) / 2;
        } while ((c <= split_mid) == (target->lo <= split_mid));
        InnerNode *tmp =
            new InnerNode(split_lo, split_hi, combine_n(init, rows * length(split_lo, split_hi)));
        if (target->lo <= split_mid) {
            tmp->left = target;
        } else {
            tmp->right = target;
        }
        target = tmp;
        update_inner(tmp, c, rows, apply);
    }
}

T lval = (n->left != nullptr) ? n->left->value : combine_n(init, rows * length(lo, mid));
T rval = (n->right != nullptr) ? n->right->value : combine_n(init, rows * length(mid + 1, hi));
n->value = combine(lval, rval);
}

void update(OuterNode *n, int r, int c, const T &d) {
    int lo = n->lo, hi = n->hi, mid = lo + (hi - lo) / 2;
    int64_t rows = length(lo, hi);
    if (lo == hi) {
        update_inner(&n->inner, c, 1, [&](const T &v) { return apply_delta(v, d); });
        return;
    }
}

```

```

    if (r <= mid) {
        if (n->left == nullptr) {
            n->left = new OuterNode(lo, mid, combine_n(init, length(lo, mid) * C));
        }
        update(n->left, r, c, d);
    } else {
        if (n->right == nullptr) {
            n->right = new OuterNode(mid + 1, hi, combine_n(init, length(mid + 1, hi) * C));
        }
        update(n->right, r, c, d);
    }
    T value = combine_n(init, rows);
    if (n->left != nullptr || n->right != nullptr) {
        T lval = (n->left != nullptr) ? query_inner(&n->left->inner, c, c, length(lo, mid))
            : combine_n(init, length(lo, mid));
        T rval = (n->right != nullptr) ? query_inner(&n->right->inner, c, c, length(mid + 1, hi))
            : combine_n(init, length(mid + 1, hi));
        value = combine(lval, rval);
    }
    update_inner(&n->inner, c, rows, [&](const T &) { return value; });
}

static void clean_up(InnerNode *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

static void clean_up(OuterNode *n) {
    if (n != nullptr) {
        clean_up(n->inner.left);
        clean_up(n->inner.right);
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SparseSegTree2D(const T &v = T{})
        : root(new OuterNode(0, R - 1, combine_n(v, static_cast<int64_t>(R) * C))), init(v) {}

    ~SparseSegTree2D() { clean_up(root); }
    SparseSegTree2D(const SparseSegTree2D &) = delete;
    SparseSegTree2D &operator=(const SparseSegTree2D &) = delete;

    T at(int r, int c) const { return query(r, c, r, c); }

    T query(int r1, int c1, int r2, int c2) const {
        assert(0 <= r1 && r1 <= r2 && r2 < R);
        assert(0 <= c1 && c1 <= c2 && c2 < C);
        return query_outer(root, r1, r2, c1, c2);
    }

    void update(int r, int c, const T &d) {
        assert(0 <= r && r < R && 0 <= c && c < C);
        update(root, r, c, d);
    }
};

```

## Example Usage

```

#include <iostream>
using namespace std;

```

```

int main() {
    SparseSegTree2D<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.query(0, 0, 0, 1) == 6);
    assert(t.query(0, 0, 1, 0) == 5);
    assert(t.query(1, 1, 2, 2) == 0);
    assert(t.query(0, 0, 1000000000, 1000000000) == 0);
    t.update(500000000, 500000000, -100);
    assert(t.query(0, 0, 1000000000, 1000000000) == -100);

    SparseSegTree2D<int, 1, 2> rectangular(0);
    rectangular.update(0, 0, 5);
    rectangular.update(0, 1, 6);
    assert(rectangular.query(0, 0, 0, 1) == 5);
    return 0;
}

```

#### Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

### 2.6.3 2D Range Tree

Maintain a set of two-dimensional points while supporting queries for all points that fall inside given rectangular regions. This implementation uses `std::pair` to represent points, requiring operators `<`, `<=`, and `==` to be defined on the numeric template type. A balanced tree over the points sorted by  $x$  stores at each node its subrange of points sorted by  $y$ , merged from its children's lists as in a merge sort tree. A query decomposes the  $x$ -range into  $O(\log n)$  nodes and binary searches each node's list for the matching  $y$ -range.

Use this for static point-reporting queries when guaranteed worst-case bounds are more important than memory. Compared with a range k-d tree, it uses more space but gives  $O(\log^2 n + m)$  query time regardless of point distribution; the k-d tree is lighter and often faster on typical inputs, but its pruning is more distribution-dependent.

- `RangeTree<T>(lo, hi)` constructs a set of `std::pair` points from the half-open forward-iterator range  $[lo, hi)$ .
- `query(x1, y1, x2, y2)` returns  $(i, x, y)$  tuples for all points in the closed rectangle  $[x1, x2] \times [y1, y2]$ , where  $i$  is the point's 0-based index in the original range.

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the number of points.
- $O(\log^2 n + m)$  per call to `query()`, where  $m$  is the number of points that are reported by the query.

#### Space Complexity:

- $O(n \log n)$  for storage of the points.
- $O(\log n)$  auxiliary stack space for `query()`.
- $O(m)$  for the vector returned by `query()`, where  $m$  is the number of reported points.

```

#include <algorithm>
#include <cassert>
#include <iterator>
#include <tuple>
#include <utility>
#include <vector>

template<typename T>
class RangeTree {
    struct IndexedPoint {
        T x, y;
        int original_index;
    };

    std::vector<IndexedPoint> points;
    std::vector<std::vector<std::pair<int, T>>> cols;

    void build(int n, int lo, int hi) {
        if (points[lo].x == points[hi].x) {
            for (int i = lo; i <= hi; i++) {
                cols[n].emplace_back(i, points[i].y);
            }
            return;
        }
        int l = n * 2 + 1, r = n * 2 + 2, mid = lo + (hi - lo) / 2;
        build(l, lo, mid);
        build(r, mid + 1, hi);
        cols[n].resize(cols[l].size() + cols[r].size());
        std::merge(
            cols[l].begin(), cols[l].end(), cols[r].begin(), cols[r].end(), cols[n].begin(),
            [](const auto &a, const auto &b) { return a.second < b.second; }
        );
    }

    void query(
        int n, int lo, int hi, const T &x1, const T &y1, const T &x2, const T &y2,
        std::vector<std::tuple<int, T, T>> &res
    ) {
        if (points[hi].x < x1 || x2 < points[lo].x) {
            return;
        }
        if (!(points[lo].x < x1 || x2 < points[hi].x)) {
            if (!cols[n].empty() && !(y2 < y1)) {
                auto it =
                    std::lower_bound(cols[n].begin(), cols[n].end(), y1, [](const auto &a, const T &value) {
                        return a.second < value;
                    });
                for (; it != cols[n].end() && it->second <= y2; ++it) {
                    const IndexedPoint &p = points[it->first];
                    res.emplace_back(p.original_index, p.x, p.y);
                }
            }
        } else if (lo != hi) {
            int mid = lo + (hi - lo) / 2;
            query(n * 2 + 1, lo, mid, x1, y1, x2, y2, res);
            query(n * 2 + 2, mid + 1, hi, x1, y1, x2, y2, res);
        }
    }

public:
    template<typename It>
    RangeTree(It lo, It hi) {
        int n = std::distance(lo, hi);
        assert(n > 0);
        points.reserve(n);
        int index = 0;
        for (It it = lo; it != hi; ++it) {
            points.push_back({it->first, it->second, index++});
        }
    }

```



```

    }
    cols.resize(4 * n + 1);
    std::sort(points.begin(), points.end(), [](const IndexedPoint &a, const IndexedPoint &b) {
        return a.x != b.x ? a.x < b.x : a.y < b.y;
    });
    build(0, 0, n - 1);
}

std::vector<std::tuple<int, T, T>> query(const T &x1, const T &y1, const T &x2, const T &y2) {
    assert(!(x2 < x1) && !(y2 < y1));
    std::vector<std::tuple<int, T, T>> res;
    query(0, 0, static_cast<int>(points.size()) - 1, x1, y1, x2, y2, res);
    return res;
}
};

```

## Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    vector<pair<int, int>> v{{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                           {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    RangeTree<int> t(v.begin(), v.end());
    auto got = t.query(-1, -1, 2, 5);
    sort(got.begin(), got.end());
    assert((got == vector<tuple<int, int, int>>{{0, 1, 4}, {2, 2, 2}, {8, -1, -1}, {9, 2, -1}}));
    got = t.query(1, 1, 4, 8);
    sort(got.begin(), got.end());
    assert((got == vector<tuple<int, int, int>>{{0, 1, 4}, {2, 2, 2}, {3, 3, 1}}));
    return 0;
}

```

### 2.6.4 K-d Tree (2D Range Reporting)

Maintain a static set of two-dimensional points while supporting rectangle reporting queries. A k-d tree recursively splits points by alternating coordinates and stores a bounding box for each subtree, allowing whole subtrees to be accepted or pruned during a query.

This implementation uses `std::pair` to represent points, requiring `operator<` to be defined on the numeric template type.

Use this for static point-reporting queries when  $O(n)$  space and good average performance are more important than a strict worst-case guarantee. Use the 2D range tree instead when adversarial point sets or query rectangles are expected and the extra  $O(n \log n)$  space is acceptable.

- `RangeKDTree<T>(lo, hi)` constructs a set of `std::pair` points from the half-open forward-iterator range `[lo, hi)`.
- `query(x1, y1, x2, y2)` returns all points in the closed rectangle  $[x1, x2] \times [y1, y2]$  as `std::pair` values.

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the number of points.
- $O(\sqrt{n} + m)$  per call to `query()`, where  $m$  is the number of points that are reported by the query.

#### Space Complexity:

- $O(n)$  for storage of the points.
- $O(\log n)$  auxiliary stack space for `query()`.
- $O(m)$  for the vector returned by `query()`, where  $m$  is the number of reported points.

```

#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class RangeKDTree {
    std::vector<std::pair<T, T>> tree, minp, maxp;
    std::vector<int> l_index, h_index;

    void build(int lo, int hi, bool div_x) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        std::nth_element(
            tree.begin() + lo, tree.begin() + mid, tree.begin() + hi,
            [div_x](const auto &a, const auto &b) {
                return div_x ? a.first < b.first : a.second < b.second;
            }
        );
        l_index[mid] = lo;
        h_index[mid] = hi;
        minp[mid].first = maxp[mid].first = tree[lo].first;
        minp[mid].second = maxp[mid].second = tree[lo].second;
        for (int i = lo + 1; i < hi; i++) {
            minp[mid].first = std::min(minp[mid].first, tree[i].first);
            minp[mid].second = std::min(minp[mid].second, tree[i].second);
            maxp[mid].first = std::max(maxp[mid].first, tree[i].first);
            maxp[mid].second = std::max(maxp[mid].second, tree[i].second);
        }
        build(lo, mid, !div_x);
        build(mid + 1, hi, !div_x);
    }

    void query(
        int lo, int hi, const T &x1, const T &y1, const T &x2, const T &y2,
        std::vector<std::pair<T, T>> &res
    ) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T ax = minp[mid].first, ay = minp[mid].second;
        T bx = maxp[mid].first, by = maxp[mid].second;
        if (x2 < ax || bx < x1 || y2 < ay || by < y1) {
            return;
        }
        if (!(ax < x1 || x2 < bx || ay < y1 || y2 < by)) {
            res.insert(res.end(), tree.begin() + l_index[mid], tree.begin() + h_index[mid]);
            return;
        }
        query(lo, mid, x1, y1, x2, y2, res);
        query(mid + 1, hi, x1, y1, x2, y2, res);
        if (tree[mid].first < x1 || x2 < tree[mid].first || tree[mid].second < y1 ||
            y2 < tree[mid].second) {
            return;
        }
        res.push_back(tree[mid]);
    }
}

public:
    template<typename It>
    RangeKDTree(It lo, It hi)
        : tree(lo, hi),
          minp(tree.size()),
          maxp(tree.size()),
          l_index(tree.size()),

```

```

    h_index(tree.size()) {
        build(0, static_cast<int>(tree.size()), true);
    }

    std::vector<std::pair<T, T>> query(const T &x1, const T &y1, const T &x2, const T &y2) {
        assert(!(x2 < x1) && !(y2 < y1));
        std::vector<std::pair<T, T>> res;
        query(0, static_cast<int>(tree.size()), x1, y1, x2, y2, res);
        return res;
    }
};

```

## Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    vector<pair<int, int>> v{{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                           {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    RangeKDTree<int> t(v.begin(), v.end());
    auto got = t.query(-1, -1, 2, 5);
    sort(got.begin(), got.end());
    assert((got == vector<pair<int, int>>{{-1, -1}, {1, 4}, {2, -1}, {2, 2}}));
    got = t.query(1, 1, 4, 8);
    sort(got.begin(), got.end());
    assert((got == vector<pair<int, int>>{{1, 4}, {2, 2}, {3, 1}}));
    return 0;
}

```

### 2.6.5 K-d Tree (Nearest Neighbor)

Maintain a static set of two-dimensional points while supporting nearest-neighbor queries. A k-d tree recursively splits points by the coordinate with larger spread, searches the more promising side first, and only explores the other side when its splitting plane can still improve the answer.

This implementation uses `std::pair` to represent points. The numeric template type must support `operator<` and conversion to `int64_t` for integral types or `long double` otherwise.

Use this for static nearest-neighbor queries on point sets in low dimensions. It is a geometric search tree rather than an aggregate structure: for rectangle reporting use the range k-d tree or 2D range tree, and for grid cell updates or rectangle sums/minima use a Fenwick tree or segment tree.

- `NearestKDTree<T>(lo, hi)` constructs a set of `std::pair` points from the half-open forward-iterator range `[lo, hi)`.
- `nearest(x, y, can_equal = true)` returns a point in the set that is closest to `(x, y)` by Euclidean distance. This may be equal to `(x, y)` only if `can_equal` is `true`; at least one eligible point must exist.

Overflow warning: For integral coordinate types, every coordinate difference, squared difference, and sum of squared differences must fit in `int64_t`.

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the number of points.
- $O(\log n)$  on average per call to `nearest()`.

#### Space Complexity:

- $O(n)$  for storage of the points.
- $O(\log n)$  auxiliary stack space for `nearest()`.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <type_traits>
#include <utility>
#include <vector>

template<typename T>
class NearestKDTree {
    using Dist = std::conditional_t<std::is_integral_v<T>, int64_t, long double>;

    std::vector<std::pair<T, T>> tree;
    std::vector<char> div_x;

    void build(int lo, int hi) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T minx, maxx, miny, maxy;
        minx = maxx = tree[lo].first;
        miny = maxy = tree[lo].second;
        for (int i = lo + 1; i < hi; i++) {
            minx = std::min(minx, tree[i].first);
            miny = std::min(miny, tree[i].second);
            maxx = std::max(maxx, tree[i].first);
            maxy = std::max(maxy, tree[i].second);
        }
        Dist x_span = static_cast<Dist>(maxx) - static_cast<Dist>(minx);
        Dist y_span = static_cast<Dist>(maxy) - static_cast<Dist>(miny);
        div_x[mid] = !(x_span < y_span);
        std::nth_element(
            tree.begin() + lo, tree.begin() + mid, tree.begin() + hi,
            [&](const auto &a, const auto &b) {
                return div_x[mid] ? a.first < b.first : a.second < b.second;
            }
        );
        if (lo + 1 == hi) {
            return;
        }
        build(lo, mid);
        build(mid + 1, hi);
    }

public:
    template<typename It>
    NearestKDTree(It lo, It hi) : tree(lo, hi), div_x(tree.size()) {
        assert(!tree.empty());
        build(0, static_cast<int>(tree.size()));
    }

    std::pair<T, T> nearest(const T &x, const T &y, bool can_equal = true) const {
        Dist best_dist = std::numeric_limits<Dist>::max();
        int best = -1;
        auto rec = [&](auto &&rec, int lo, int hi) {
            if (lo >= hi) {
                return;
            }
            int mid = lo + (hi - lo) / 2;
            Dist dx = static_cast<Dist>(x) - static_cast<Dist>(tree[mid].first);
            Dist dy = static_cast<Dist>(y) - static_cast<Dist>(tree[mid].second);
            Dist dist = dx * dx + dy * dy; // Overflow warning.
            if (dist < best_dist && (can_equal || dist != 0)) {
                best_dist = dist;
                best = mid;
            }
        };
        if (lo + 1 == hi) {

```

```

        return;
    }
    Dist axis_delta = div_x[mid] ? dx : dy;
    int l1 = lo, r1 = mid, l2 = mid + 1, r2 = hi;
    if (axis_delta > 0) {
        std::swap(l1, l2);
        std::swap(r1, r2);
    }
    rec(rec, l1, r1);
    if (axis_delta * axis_delta < best_dist) {
        rec(rec, l2, r2);
    }
};
rec(rec, 0, static_cast<int>(tree.size()));
assert(best != -1);
return tree[best];
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<pair<int, int>> p{{0, 2}, {0, 3}, {-1, 0}};
    NearestKDTree<int> t(p.begin(), p.end());
    assert(t.nearest(0, 2, true) == (pair<int, int>{0, 2}));
    assert(t.nearest(0, 2, false) == (pair<int, int>{0, 3}));
    assert(t.nearest(0, 0) == (pair<int, int>{-1, 0}));
    assert(t.nearest(-10000, 0) == (pair<int, int>{-1, 0}));

    vector<pair<int, int>> large{{-1000000000, 1}, {1000000000, 0}};
    NearestKDTree<int> large_tree(large.begin(), large.end());
    assert(large_tree.nearest(0, 0) == (pair<int, int>{1000000000, 0}));
    return 0;
}

```

## 2.7 Fenwick Trees

### 2.7.1 Fenwick Tree (Point Update, Range Query)

Maintain a numerical array while supporting point increments and range-sum queries. A Fenwick tree (also called a binary indexed tree) stores partial prefix sums: each internal entry `tree[i]` holds the sum of a block of indices whose length is the lowest set bit of `i`. A point update touches  $O(\log n)$  covering blocks, and a prefix-sum query combines  $O(\log n)$  disjoint blocks.

When prefix sums are nondecreasing, `max_right(c)` answers prefix-boundary queries by binary lifting: it walks the implicit power-of-two block structure to find the longest prefix with sum at most `c`. This lets the same structure double as a dynamic multiset over a bounded integer domain, useful for coordinate-compressed  $k$ -th element lookups and online rank queries.

- `Fenwick<T>(n)` constructs an array with 0-based indices  $[0, n)$ , with all values initialized to 0.
- `size()` returns the size of the array.
- `add(i, x)` adds `x` to the value at index `i`.
- `set(i, x)` assigns the value at index `i` to `x`.
- `at(i)` returns the value at index `i`.
- `sum(hi)` returns the sum of all values at indices  $[0, hi]$ .
- `sum(lo, hi)` returns the sum of all values at indices  $[lo, hi]$ .

- `max_right(c)` returns the largest boundary `hi` such that `sum(0, hi - 1) ≤ c`, assuming prefix sums are nondecreasing and `c` is nonnegative. It may return any boundary in  $[0, n]$ .

The value type `T` must represent 0 and support addition and subtraction.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to `add()`, both `sum()` functions, `set()`, `at()`, and `max_right()`.

#### Space Complexity:

- $O(n)$  for storage of the Fenwick tree.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <vector>

template<typename T>
class Fenwick {
    int len;
    std::vector<T> tree;

public:
    explicit Fenwick(int n) : len(n), tree(n + 1) {}

    int size() const { return len; }

    void add(int i, const T &x) {
        assert(0 <= i && i < len);
        for (++i; i <= len; i += i & -i) {
            tree[i] += x;
        }
    }

    void set(int i, const T &x) { add(i, x - at(i)); }
    T at(int i) const { return sum(i, i); }

    T sum(int hi) const {
        assert(-1 <= hi && hi < len);
        T res = 0;
        for (++hi; hi > 0; hi -= hi & -hi) {
            res += tree[hi];
        }
        return res;
    }

    T sum(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi < len);
        return sum(hi) - sum(lo - 1);
    }

    int max_right(T c) const {
        int pos = 0, pw = 1;
        while (pw * 2 <= len) {
            pw *= 2;
        }
        for (; pw > 0; pw >>= 1) {
            if (pos + pw <= len && tree[pos + pw] <= c) {
                c -= tree[pos + pw];
                pos += pw;
            }
        }
        return pos;
    }
};
```

## Example Usage

```
using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    Fenwick<int> t(5);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        t.set(i, a[i]);
    }
    t.add(0, -5); // a is now {5, 1, 2, 3, 4}.
    assert(t.at(0) == 5);
    assert(t.sum(1, 3) == 6); // 1 + 2 + 3
    assert(t.sum(4) == 15); // 5 + 1 + 2 + 3 + 4

    // With nonnegative values, the tree doubles as a frequency table for order-statistic queries.
    Fenwick<int> freq(8);
    freq.add(1, 1); // One element of value 1.
    freq.add(3, 3); // Three elements of value 3.
    freq.add(6, 1); // One element of value 6.
    assert(freq.max_right(0) == 1); // The longest prefix with count at most 0 is [0, 1).
    assert(freq.max_right(3) == 3); // Prefix [0, 3) has count 1; [0, 4) has count 4.
    assert(freq.max_right(4) == 6); // Prefix [0, 6) has count 4; [0, 7) has count 5.
    return 0;
}
```

### 2.7.2 Fenwick Tree (Range Update, Point Query)

Maintain a numerical array while supporting range increments and point queries. This is done by storing the difference array in a Fenwick tree: adding  $x$  to  $[lo, hi]$  adds  $+x$  at  $lo$  and  $-x$  after  $hi$ , and the value at index  $i$  is the prefix sum of those differences through  $i$ .

- `FenwickRUPQ<T>(n)` constructs an array with 0-based indices  $[0, n)$ , with all values initialized to 0.
- `size()` returns the size of the array.
- `add(i, x)` adds  $x$  to the value at index  $i$ .
- `add(lo, hi, x)` adds  $x$  to the values at all indices in  $[lo, hi]$ .
- `at(i)` returns the value at index  $i$ .

The value type `T` must represent 0 and support addition and subtraction.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to `at()` and both `add()` functions.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <vector>

template<typename T>
class FenwickRUPQ {
    int len;
    std::vector<T> tree;

    // Adds x to the value at every index from i onward.
    void add_suffix(int i, const T &x) {
        for (i++; i <= len + 1; i += i & -i) {
            tree[i] += x;
        }
    }
};
```

```

    }
}

public:
    explicit FenwickRUPQ(int n) : len(n), tree(n + 2) {}

    int size() const { return len; }
    void add(int i, const T &x) { add(i, i, x); }

    void add(int lo, int hi, const T &x) {
        assert(0 <= lo && lo <= hi && hi < len);
        add_suffix(lo, x);
        add_suffix(hi + 1, -x);
    }

    T at(int i) const {
        assert(0 <= i && i < len);
        T res = 0;
        for (i++; i > 0; i -= i & -i) {
            res += tree[i];
        }
        return res;
    }
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    FenwickRUPQ<int> t(5);
    t.add(0, 1, 5);
    t.add(1, 2, 5);
    t.add(2, 4, 10);
    assert(t.size() == 5);
    assert(t.at(0) == 5);
    assert(t.at(1) == 10);
    assert(t.at(2) == 15);
    assert(t.at(4) == 10);
    t.add(3, -4);
    assert(t.at(3) == 6);
    assert(t.at(4) == 10);
    return 0;
}

```

### 2.7.3 Fenwick Tree (Range Update, Range Query)

Maintain a numerical array while supporting range increments and range-sum queries. This uses two Fenwick trees to recover prefix sums after difference-array updates: if the difference array stores range additions, then the prefix sum through `hi` can be written as `hi*sum(t1, hi) - sum(t2, hi)`.

- `FenwickRURQ<T>(n)` constructs an array with 0-based indices  $[0, n)$ , with all values initialized to 0.
- `size()` returns the size of the array.
- `add(i, x)` adds `x` to the value at index `i`.
- `add(lo, hi, x)` adds `x` to the values at all indices in  $[lo, hi]$ .
- `set(i, x)` assigns the value at index `i` to `x`.
- `at(i)` returns the value at index `i`.
- `sum(hi)` returns the sum of all values at indices  $[0, hi]$ .
- `sum(lo, hi)` returns the sum of all values at indices  $[lo, hi]$ .



- `max_right(c)` returns the largest boundary `hi` such that `sum(0, hi - 1) ≤ c`, assuming prefix sums are nondecreasing and `c` is nonnegative. It may return any boundary in  $[0, n]$ .

The value type `T` must represent 0 and support addition, subtraction, and multiplication by an integer index.

Overflow warning: all index-weighted products and resulting sums must fit in `T`.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the size of the array.
- $O(1)$  per call to `size()`.
- $O(\log n)$  per call to all other operations.

#### Space Complexity:

- $O(n)$  for storage of the array elements.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <vector>

template<typename T>
class FenwickRURQ {
    int len;
    std::vector<T> t1, t2;

    T sum(const std::vector<T> &tree, int i) const {
        assert(0 <= i && i <= len);
        T res = 0;
        for (; i != 0; i -= i & -i) {
            res += tree[i];
        }
        return res;
    }

    void add(std::vector<T> &tree, int i, const T &x) {
        assert(1 <= i && i <= len + 1);
        for (; i <= len + 1; i += i & -i) {
            tree[i] += x;
        }
    }

public:
    explicit FenwickRURQ(int n) : len(n), t1(n + 2), t2(n + 2) {}

    int size() const { return len; }
    void add(int i, const T &x) { add(t1, i, x); }

    void add(int lo, int hi, const T &x) {
        assert(0 <= lo && lo <= hi && hi < len);
        lo++;
        hi++;
        add(t1, lo, x);
        add(t1, hi + 1, -x);
        add(t2, lo, x * (lo - 1)); // Overflow warning.
        add(t2, hi + 1, -x * hi);
    }

    void set(int i, const T &x) { add(i, x - at(i)); }
    T at(int i) const { return sum(t1, i); }

    T sum(int hi) const {
        assert(-1 <= hi && hi < len);
        hi++;
        return hi * sum(t1, hi) - sum(t2, hi);
    }

    T sum(int lo, int hi) const {
```

```

    assert(0 <= lo && lo <= hi && hi < len);
    return sum(hi) - sum(lo - 1);
}

int max_right(T c) const {
    T s1 = 0, s2 = 0;
    int pos = 0, pw = 1;
    while (pw * 2 <= len) {
        pw *= 2;
    }
    for (; pw > 0; pw >>= 1) {
        int next = pos + pw;
        if (next <= len) {
            T ns1 = s1 + t1[next], ns2 = s2 + t2[next];
            if (next * ns1 - ns2 <= c) {
                s1 = ns1;
                s2 = ns2;
                pos = next;
            }
        }
    }
    return pos;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    FenwickRURQ<int> t(5);
    for (int i = 0; i < t.size(); i++) {
        t.set(i, a[i]);
    }
    t.add(0, 2, 5);
    t.set(3, -5);
    vector<int> expected{15, 6, 7, -5, 4};
    for (int i = 0; i < t.size(); i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.sum(0, 4) == 27);
    FenwickRURQ<int> freq(8);
    freq.add(1, 1, 1);
    freq.add(3, 3, 3);
    freq.add(6, 6, 1);
    assert(freq.max_right(0) == 1);
    assert(freq.max_right(3) == 3);
    assert(freq.max_right(4) == 6);
    return 0;
}

```

### 2.7.4 Sparse Fenwick Tree

Maintain a numerical array over a huge index range while supporting range increments and range-sum queries. This is the sparse version of the two-tree Fenwick range-update/range-query trick: only Fenwick nodes reached by previous operations are stored, using `std::unordered_map` instead of dense vectors.

- `SparseFenwick<T, N>()` constructs an array with 0-based indices  $[0, N)$ , implicitly initialized to 0.
- `add(i, x)` adds `x` to the value at index `i`.
- `add(lo, hi, x)` adds `x` to the values at all indices in  $[lo, hi]$ .
- `set(i, x)` assigns the value at index `i` to `x`.

- `at(i)` returns the value at index `i`.
- `sum(hi)` returns the sum of all values at indices  $[0, hi]$ .
- `sum(lo, hi)` returns the sum of all values at indices  $[lo, hi]$ .
- `max_right(c)` returns the largest boundary `hi` such that  $sum(0, hi - 1) \leq c$ , assuming prefix sums are nondecreasing and `c` is nonnegative. It may return any boundary in  $[0, N]$ .

The value type `T` must represent 0 and support addition, subtraction, and multiplication by an integer index.

Overflow warning: all products of values by indices up to `N`, and all resulting sums, must fit in `T`.

**Time Complexity:**  $O(\log N)$  expected per call to all member functions.

**Space Complexity:**

- $O(q \log N)$  expected for storage after  $q$  updates.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <unordered_map>

template<typename T, int N = 1000000001>
class SparseFenwick {
    static_assert(0 < N && N < (1 << 30));

    std::unordered_map<int, T> tmul, tadd;

    T get(const std::unordered_map<int, T> &tree, int i) const {
        if (auto it = tree.find(i); it != tree.end()) {
            return it->second;
        }
        return 0;
    }

    // Adds the linear correction mul*i + add to every index from at onward.
    void add_suffix(int at, const T &mul, const T &add) {
        assert(1 <= at && at <= N + 1);
        for (int i = at; i <= N; i += i & -i) {
            tmul[i] += mul;
            tadd[i] += add;
        }
    }

public:
    void add(int i, const T &x) { add(i, i, x); }

    void add(int lo, int hi, const T &x) {
        assert(0 <= lo && lo <= hi && hi < N);
        lo++;
        hi++;
        add_suffix(lo, x, x * (lo - 1)); // Overflow warning.
        add_suffix(hi + 1, -x, -x * hi);
    }

    void set(int i, const T &x) { add(i, x - at(i)); }
    T at(int i) const { return sum(i, i); }

    T sum(int hi) const {
        assert(-1 <= hi && hi < N);
        T mul = 0, add = 0;
        hi++;
        for (int i = hi; i > 0; i -= i & -i) {
            mul += get(tmul, i);
            add += get(tadd, i);
        }
        return mul * hi - add;
    }
}
```

```

T sum(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi < N);
    return sum(hi) - sum(lo - 1);
}

int max_right(T c) const {
    T mul = 0, add = 0;
    int pos = 0, pw = 1;
    while (pw * 2 <= N) {
        pw *= 2;
    }
    for (; pw > 0; pw >>= 1) {
        int next = pos + pw;
        if (next <= N) {
            T nmul = mul + get(tmul, next), nadd = add + get(tadd, next);
            if (next * nmul - nadd <= c) {
                mul = nmul;
                add = nadd;
                pos = next;
            }
        }
    }
    return pos;
}
};

```

## Example Usage

```

#include <vector>
using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    SparseFenwick<long long> t;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        t.set(i, a[i]);
    }
    t.add(0, 2, 5);
    t.set(3, -5);
    vector<int> expected{15, 6, 7, -5, 4};
    for (int i = 0; i < 5; i++) {
        assert(t.at(i) == expected[i]);
    }
    assert(t.sum(0, 4) == 27);
    t.add(500000001, 500000010, 3);
    t.add(500000011, 500000015, 5);
    t.set(500000000, 10);
    assert(t.sum(0, 1000000000) == 92);

    SparseFenwick<long long> freq;
    freq.add(1, 1, 1);
    freq.add(3, 3, 3);
    freq.add(6, 6, 1);
    assert(freq.max_right(0) == 1);
    assert(freq.max_right(3) == 3);
    assert(freq.max_right(4) == 6);
    return 0;
}

```

### 2.7.5 2D Fenwick Tree

Maintain a two-dimensional numerical array while supporting point increments and rectangle-sum queries. This is the two-dimensional form of the standard Fenwick tree: each internal entry stores a rectangular block sum, and a prefix query combines  $O(\log(R) \cdot \log(C))$  disjoint blocks.

Choose among the three 2D Fenwick trees by coordinate range and update style. Use this dense version when the rows and columns are small enough to allocate the full grid: it has the simplest code and the best constants, supporting point updates and rectangle sums. Use the sparse 2D Fenwick tree (next section) when the coordinates are too large to store densely and the updates arrive online; it relies on hashing and is the only one of the three that also supports rectangle updates. Use the offline 2D Fenwick tree when the coordinates are large but every updated cell is known before the queries begin, which lets it coordinate-compress instead of hashing for markedly better constants than the sparse version. For non-additive aggregates such as min/max with custom updates, use a 2D segment tree instead, since Fenwick-tree algebra relies on addition and subtraction.

- `Fenwick2D<T>(rows, cols)` constructs a `rows` by `cols` array with 0-based indices, with all values initialized to 0.
- `num_rows()` and `num_cols()` return the number of rows and columns.
- `add(r, c, x)` adds `x` to the value at index `(r, c)`.
- `set(r, c, x)` assigns `x` to the value at index `(r, c)`.
- `at(r, c)` returns the value at index `(r, c)`.
- `sum(r, c)` returns the sum of the rectangle with rows `[0, r]` and columns `[0, c]`.
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with rows `[r1, r2]` and columns `[c1, c2]`.

The value type `T` must represent 0 and support addition and subtraction.

#### Time Complexity:

- $O(R \cdot C)$  per call to the constructor, where  $R$  and  $C$  are the number of rows and columns.
- $O(1)$  per call to `num_rows()` and `num_cols()`.
- $O(\log(R) \cdot \log(C))$  per call to all other operations.

#### Space Complexity:

- $O(R \cdot C)$  for storage of the Fenwick tree.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <vector>

template<typename T>
class Fenwick2D {
    int rows, cols;
    std::vector<std::vector<T>> tree;

public:
    Fenwick2D(int rows, int cols)
        : rows(rows), cols(cols), tree(rows + 1, std::vector<T>(cols + 1)) {}

    int num_rows() const { return rows; }
    int num_cols() const { return cols; }

    void add(int r, int c, const T &x) {
        assert(0 <= r && r < rows && 0 <= c && c < cols);
        for (int i = r + 1; i <= rows; i += i & -i) {
            for (int j = c + 1; j <= cols; j += j & -j) {
                tree[i][j] += x;
            }
        }
    }

    void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }
    T at(int r, int c) const { return sum(r, c, r, c); }
```

```

T sum(int r, int c) const {
    assert(-1 <= r && r < rows && -1 <= c && c < cols);
    T res = 0;
    for (int i = r + 1; i > 0; i -= i & -i) {
        for (int j = c + 1; j > 0; j -= j & -j) {
            res += tree[i][j];
        }
    }
    return res;
}

T sum(int r1, int c1, int r2, int c2) const {
    assert(0 <= r1 && r1 <= r2 && r2 < rows);
    assert(0 <= c1 && c1 <= c2 && c2 < cols);
    return sum(r2, c2) - sum(r1 - 1, c2) - sum(r2, c1 - 1) + sum(r1 - 1, c1 - 1);
}
};

```

## Example Usage

```

using namespace std;

int main() {
    Fenwick2D<int> t(3, 3);
    t.set(0, 0, 5);
    t.set(0, 1, 6);
    t.set(1, 0, 7);
    t.add(2, 2, 9);
    t.add(1, 0, -4); // Value at (1, 0) is now 3.
    assert(t.at(1, 0) == 3);
    assert(t.sum(0, 0, 0, 1) == 11); // 5 + 6
    assert(t.sum(0, 0, 1, 0) == 8); // 5 + 3
    assert(t.sum(0, 0, 2, 2) == 23); // 5 + 6 + 3 + 9
    return 0;
}

```

### 2.7.6 Sparse 2D Fenwick Tree

Maintain a 2D numerical array over a huge index range while supporting rectangle increments and rectangle-sum queries. This is the sparse two-dimensional analogue of range-update/range-query Fenwick trees: four sparse trees store the inclusion-exclusion coefficients needed to recover any prefix rectangle sum.

Choose this for huge sparse grids when the operation is additive: point or rectangle increments and rectangle sums. It avoids allocating the dense  $R \times C$  table of the dense 2D Fenwick tree, and unlike the dense and offline variants it supports rectangle updates, not just point updates. Prefer it over the offline 2D Fenwick tree only when updates arrive online; if every updated cell is known in advance, that version compresses coordinates instead of hashing and runs with better constants. It is less general than the sparse 2D segment tree because Fenwick-tree algebra relies on addition and subtraction.

- `SparseFenwick2D<T, R, C>()` constructs a 2D array over rows  $[0, R)$  and columns  $[0, C)$ . All values are implicitly initialized to 0; nodes are allocated lazily as indices are touched.
- `sum(r, c)` returns the sum of the rectangle with rows  $[0, r]$  and columns  $[0, c]$ .
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with rows in  $[r1, r2]$  and columns in  $[c1, c2]$ .
- `add(r1, c1, r2, c2, x)` adds `x` to all indices in the rectangle with rows  $[r1, r2]$  and columns  $[c1, c2]$ .
- `at(r, c)` returns the value at index  $(r, c)$ .
- `add(r, c, x)` adds `x` to the value at index  $(r, c)$ .
- `set(r, c, x)` assigns `x` to the value at index  $(r, c)$ .

The value type `T` must represent 0 and support addition, subtraction, and multiplication by integer coordinates.

Overflow warning: all coordinate-weighted products and resulting sums must fit in  $\mathbb{T}$ . In particular, `x * r * c` can overflow even `int64_t` when both coordinates and values are large.

**Time Complexity:**  $O(\log(R) \cdot \log(C))$  expected per call to all member functions.

**Space Complexity:**

- $O(q \cdot \log(R) \cdot \log(C))$  expected for storage after  $q$  updates.
- $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <cstdint>
#include <unordered_map>

template<typename T, int R = 1000000001, int C = 1000000001>
class SparseFenwick2D {
    static_assert(0 < R && R < (1 << 30) && 0 < C && C < (1 << 30));

    std::unordered_map<int64_t, T> t1, t2, t3, t4;

    static T get(const std::unordered_map<int64_t, T> &tree, int r, int c) {
        auto it = tree.find(static_cast<int64_t>(r) * (C + 1) + c);
        return it == tree.end() ? T{} : it->second;
    }

    static void add(std::unordered_map<int64_t, T> &tree, int r, int c, const T &x) {
        assert(0 <= r && r <= R && 0 <= c && c <= C);
        for (int i = r + 1; i <= R; i += i & -i) {
            for (int j = c + 1; j <= C; j += j & -j) {
                tree[static_cast<int64_t>(i) * (C + 1) + j] += x;
            }
        }
    }

    // Adds x to every cell with row at least r and column at least c.
    void add_quadrant(int r, int c, const T &x) {
        add(t1, 0, 0, x);
        add(t1, 0, c, -x);
        add(t2, 0, c, x * c);
        add(t1, r, 0, -x);
        add(t3, r, 0, x * r);
        add(t1, r, c, x);
        add(t2, r, c, -x * c);
        add(t3, r, c, -x * r);
        add(t4, r, c, x * r * c); // Overflow warning.
    }

public:
    T sum(int r, int c) const {
        assert(-1 <= r && r < R && -1 <= c && c < C);
        r++;
        c++;
        T s1 = 0, s2 = 0, s3 = 0, s4 = 0;
        for (int i = r; i > 0; i -= i & -i) {
            for (int j = c; j > 0; j -= j & -j) {
                s1 += get(t1, i, j);
                s2 += get(t2, i, j);
                s3 += get(t3, i, j);
                s4 += get(t4, i, j);
            }
        }
        return s1 * r * c + s2 * r + s3 * c + s4;
    }

    T sum(int r1, int c1, int r2, int c2) const {
        assert(0 <= r1 && r1 <= r2 && r2 < R);
        assert(0 <= c1 && c1 <= c2 && c2 < C);
        return sum(r2, c2) + sum(r1 - 1, c1 - 1) - sum(r1 - 1, c2) - sum(r2, c1 - 1);
    }
};
```

```

}

void add(int r1, int c1, int r2, int c2, const T &x) {
    assert(0 <= r1 && r1 <= r2 && r2 < R);
    assert(0 <= c1 && c1 <= c2 && c2 < C);
    add_quadrant(r2 + 1, c2 + 1, x);
    add_quadrant(r1, c2 + 1, -x);
    add_quadrant(r2 + 1, c1, -x);
    add_quadrant(r1, c1, x);
}

T at(int r, int c) const { return sum(r, c, r, c); }
void add(int r, int c, const T &x) { add(r, c, r, c, x); }
void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }
};

```

## Example Usage

```

#include <iostream>
using namespace std;

int main() {
    SparseFenwick2D<long long> t;
    t.set(0, 0, 5);
    t.set(0, 1, 6);
    t.set(1, 0, 7);
    t.add(2, 2, 9);
    t.add(1, 0, -4);
    t.add(1, 1, 2, 2, 5);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.sum(0, 0, 0, 1) == 11);
    assert(t.sum(0, 0, 1, 0) == 8);
    assert(t.sum(1, 1, 2, 2) == 29);
    t.set(500000000, 500000000, 1);
    assert(t.sum(0, 0, 999999999, 999999999) == 44);
    return 0;
}

```

### Example Output

```

Values:
5 6 0
3 5 5
0 5 14

```

## 2.7.7 Offline 2D Fenwick Tree

Maintain a 2D numerical array over an arbitrary, possibly huge index range while supporting point increments and rectangle-sum queries, when every coordinate that will ever be updated is known in advance. This offline restriction is what makes the structure cheap: both coordinates are coordinate-compressed against the update points, so no hashing is required and the storage is exactly the set of cells each Fenwick node can ever touch.

The outer dimension is an ordinary Fenwick tree over the sorted distinct update rows. Each outer node owns an inner Fenwick tree over only those columns that reach it along an update path, again stored in a sorted vector. A prefix query maps its row argument to the number of update rows at or below it, walks the outer tree, and within each visited node maps its column argument to a rank by binary search. Compared with the online sparse 2D



Fenwick tree backed by hash tables, this is deterministic and has markedly better constants, at the cost of requiring all update locations up front. Prefer the dense 2D Fenwick tree when the coordinates are small enough to allocate directly, and the sparse 2D Fenwick tree when update locations are not known ahead of time or rectangle updates are needed.

Usage is two-phase. First declare every cell that will be updated with `register_point()`; then call `build()` once; then issue `add()` updates and `sum()` / `at()` queries freely. Updates may only target registered cells. All row and column coordinates must exceed `INT_MIN`, which is reserved so inclusive rectangle queries can form predecessor bounds.

- `OfflineFenwick2D<T>()` constructs an empty structure in the declaration phase.
- `register_point(r, c)` declares that the cell  $(r, c)$  may later be updated. Call before `build()`.
- `build()` finalizes the coordinate compression. Call exactly once, after all `register_point()` calls.
- `add(r, c, x)` adds `x` to the value at index  $(r, c)$ , which must have been registered.
- `set(r, c, x)` assigns `x` to the value at index  $(r, c)$ , which must have been registered.
- `at(r, c)` returns the value at index  $(r, c)$ .
- `sum(r, c)` returns the sum over all registered rows at most `r` and columns at most `c`.
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with rows  $[r1, r2]$  and columns  $[c1, c2]$ .

The value type `T` must represent 0 and support addition and subtraction.

#### Time Complexity:

- $O(n \log^2 n)$  per call to `build()`, where  $n$  is the number of registered cells.
- $O(\log^2 n)$  per call to `add()`, `set()`, `sum()`, and `at()`.

#### Space Complexity:

- $O(n \log n)$  for storage, where  $n$  is the number of reserved cells.
- $O(1)$  auxiliary for all operations after `build()`.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class OfflineFenwick2D {
    std::vector<int> xs;           // Sorted distinct update rows.
    std::vector<std::vector<int>> ys; // ys[p]: sorted distinct columns at outer node p.
    std::vector<std::vector<T>> tree; // tree[p]: 1-indexed inner Fenwick over ys[p].
    std::vector<std::pair<int, int>> pending; // Registered (row, column) cells.
    bool built = false;

    // Number of update rows at most r; the outer Fenwick prefix bound for a query at row r.
    int row_rank(int r) const {
        return static_cast<int>(std::upper_bound(xs.begin(), xs.end(), r) - xs.begin());
    }

    // 1-indexed outer position of an update row r, which is guaranteed to be present in xs.
    int row_pos(int r) const {
        return static_cast<int>(std::lower_bound(xs.begin(), xs.end(), r) - xs.begin()) + 1;
    }

    // Number of columns at most c registered at outer node p; the inner prefix bound.
    int col_rank(int p, int c) const {
        return static_cast<int>(std::upper_bound(ys[p].begin(), ys[p].end(), c) - ys[p].begin());
    }

    bool is_registered(int r, int c) const {
        int p = row_pos(r);
        return p < static_cast<int>(tree.size()) && p > 0 && xs[p - 1] == r &&
            std::binary_search(ys[p].begin(), ys[p].end(), c);
    }
}
```

```

public:
    void register_point(int r, int c) {
        assert(!built);
        pending.emplace_back(r, c);
    }

    void build() {
        assert(!built);
        built = true;
        for (const auto &u : pending) {
            xs.push_back(u.first);
        }
        std::sort(xs.begin(), xs.end());
        xs.erase(std::unique(xs.begin(), xs.end()), xs.end());
        int m = static_cast<int>(xs.size());
        ys.assign(m + 1, {});
        // Register each update's column at every outer node on its update path.
        for (const auto &u : pending) {
            for (int x = row_pos(u.first); x <= m; x += x & -x) {
                ys[x].push_back(u.second);
            }
        }
        tree.assign(m + 1, {});
        for (int x = 1; x <= m; x++) {
            std::sort(ys[x].begin(), ys[x].end());
            ys[x].erase(std::unique(ys[x].begin(), ys[x].end()), ys[x].end());
            tree[x].assign(ys[x].size() + 1, T{});
        }
        pending.clear();
    }

    void add(int r, int c, const T &x) {
        assert(built && is_registered(r, c));
        for (int i = row_pos(r); i < static_cast<int>(tree.size()); i += i & -i) {
            for (int j = col_rank(i, c); j <= static_cast<int>(ys[i].size()); j += j & -j) {
                tree[i][j] += x;
            }
        }
    }

    void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }
    T at(int r, int c) const { return sum(r, c, r, c); }

    T sum(int r, int c) const {
        assert(built);
        T res{};
        for (int i = row_rank(r); i > 0; i -= i & -i) {
            for (int j = col_rank(i, c); j > 0; j -= j & -j) {
                res += tree[i][j];
            }
        }
        return res;
    }

    T sum(int r1, int c1, int r2, int c2) const {
        assert(r1 <= r2 && c1 <= c2);
        return sum(r2, c2) - sum(r1 - 1, c2) - sum(r2, c1 - 1) + sum(r1 - 1, c1 - 1);
    }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {

```

```

OfflineFenwick2D<int> t;
// Declare every cell that will be updated, then finalize.
t.register_point(0, 0);
t.register_point(0, 1000000);
t.register_point(1000000, 0);
t.register_point(1000000, 1000000);
t.register_point(500000, 500000);
t.build();

t.set(0, 0, 5);
t.set(0, 1000000, 6);
t.set(1000000, 0, 7);
t.add(1000000, 1000000, 9);
t.add(500000, 500000, 4);

assert(t.at(1000000, 1000000) == 9);
assert(t.sum(0, 0, 0, 1000000) == 11);           // 5 + 6.
assert(t.sum(0, 0, 1000000, 0) == 12);           // 5 + 7.
assert(t.sum(0, 0, 1000000, 1000000) == 31);     // 5 + 6 + 7 + 9 + 4.
assert(t.sum(400000, 400000, 600000, 600000) == 4); // Only the center cell.
return 0;
}

```

## 2.8 Disjoint Sets and Tree Structures

---

### 2.8.1 Disjoint Set Union

Maintain a set of elements partitioned into non-overlapping subsets. Each partition is identified by a unique representative called its root. Each subset is stored as a tree whose nodes point toward its root: finding a representative follows parent pointers, redirecting visited nodes straight to the root along the way (path compression), while unions attach the smaller tree beneath the root of the larger one (union-by-size). Together these keep the trees nearly flat. Union-by-size is interchangeable with the union-by-rank strategy used by Sparse Disjoint Set Union and yields the same complexity bounds, but union-by-size additionally supports `set_size()` queries.

This version is simplified to work only on the fixed set of integer elements  $[0, n)$ , chosen at construction. For arbitrary element types, see Sparse Disjoint Set Union.

- `DSU(n)` constructs  $n$  singleton partitions over elements  $[0, n)$ .
- `find(u)` returns the unique representative of the partition containing `u`.
- `sets()` returns the current number of partitions.
- `set_size(u)` returns the number of elements in the partition containing `u`.
- `is_united(u, v)` returns whether elements `u` and `v` belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions, returning `true` if a merge occurred or `false` if `u` and `v` already belonged to the same partition.

#### Time Complexity:

- $O(n)$  per call to the constructor, and  $O(1)$  per call to `sets()`.
- $O(\alpha(n))$  per call to `find()`, `set_size()`, `is_united()`, and `unite()`, where  $n$  is the number of elements, and  $\alpha(n)$  is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of  $n$ ).

#### Space Complexity:

- $O(n)$  for storage of the disjoint set forest elements.
- $O(\log n)$  auxiliary stack space for operations that call `find()`.
- $O(1)$  auxiliary for `sets()`.

```

#include <numeric>
#include <utility>
#include <vector>

class DSU {
    std::vector<int> root, size;
    int num_sets;

public:
    explicit DSU(int n) : root(n), size(n, 1), num_sets(n) { std::iota(root.begin(), root.end(), 0); }

    int find(int u) { return root[u] == u ? u : root[u] = find(root[u]); }
    int sets() const { return num_sets; }
    int set_size(int u) { return size[find(u)]; }
    bool is_united(int u, int v) { return find(u) == find(v); }

    bool unite(int u, int v) {
        u = find(u);
        v = find(v);
        if (u == v) {
            return false;
        }
        num_sets--;
        if (size[u] < size[v]) {
            std::swap(u, v);
        }
        root[v] = u;
        size[u] += size[v];
        return true;
    }
};

```

### Example Usage

```

#include <cassert>

int main() {
    DSU dsu(7);
    assert(dsu.unite(0, 1));
    assert(dsu.unite(1, 5));
    assert(dsu.unite(3, 4));
    assert(dsu.unite(3, 6));
    assert(!dsu.unite(0, 5)); // Already united.
    assert(dsu.sets() == 3);
    assert(dsu.set_size(1) == 3);
    assert(dsu.set_size(2) == 1);
    assert(dsu.is_united(0, 1));
    assert(!dsu.is_united(0, 2));
    assert(!dsu.is_united(1, 6));
    assert(dsu.is_united(4, 6));
    return 0;
}

```

## 2.8.2 Sparse Disjoint Set Union

Maintain a set of elements partitioned into non-overlapping subsets using a collection of trees. Each partition is identified by a unique representative called its root. Each subset is stored as a tree whose nodes point toward its root: finding a representative follows parent pointers, redirecting visited nodes straight to the root along the way (path compression), while unions attach the shallower tree beneath the root of the deeper one (union-by-rank). Union-by-rank is interchangeable with union-by-size as used by the plain Disjoint Set Union, yielding the same complexity bounds; rank stores only the tree-depth estimate, so it cannot answer set-size queries.

This version uses an `std::unordered_map` for storage and coordinate compression (thus, element types must meet the requirements of key types for `std::unordered_map`). The order of sets returned by `get_all_sets()` is unspecified.

- `SparseDSU<T>()` constructs an empty set.
- `make_set(u)` adds `u` as a singleton partition, or does nothing if `u` is already present.
- `size()` returns the number of elements that have been added.
- `sets()` returns the current number of disjoint sets.
- `is_united(u, v)` returns whether elements `u` and `v`, which must have been added with `make_set()`, belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions, returning `true` if a merge occurred or `false` if `u` and `v` already belonged to the same partition. Both elements must have been added with `make_set()`.
- `get_all_sets()` returns all current partitions as a vector of vectors.

#### Time Complexity:

- $O(1)$  per call to the constructor.
- $O(1)$  per call to `size()` and `sets()`.
- $O(1)$  on average per call to `make_set()`.
- $O(\alpha(n))$  on average per call to `is_united()` and `unite()`, where  $n$  is the number of elements that have been added via `make_set()` so far, and  $\alpha(n)$  is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of  $n$ ).
- $O(n)$  per call to `get_all_sets()`.

#### Space Complexity:

- $O(n)$  for storage of the disjoint set forest elements.
- $O(n)$  auxiliary for `get_all_sets()`.
- $O(\log n)$  auxiliary stack space for `is_united()` and `unite()`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <unordered_map>
#include <utility>
#include <vector>

template<typename T>
class SparseDSU {
    int num_elements, num_sets;
    std::unordered_map<T, int> id;
    std::vector<int> root, rank;

    int find(int u) { return root[u] == u ? u : root[u] = find(root[u]); }

    int get_id(const T &u) const {
        auto it = id.find(u);
        assert(it != id.end());
        return it->second;
    }

public:
    SparseDSU() : num_elements(0), num_sets(0) {}

    void make_set(const T &u) {
        if (id.find(u) != id.end()) {
            return;
        }
        id[u] = num_elements;
        root.push_back(num_elements++);
        rank.push_back(0);
        num_sets++;
    }

    int size() const { return num_elements; }
```

```

int sets() const { return num_sets; }
bool is_united(const T &u, const T &v) { return find(get_id(u)) == find(get_id(v)); }

bool unite(const T &u, const T &v) {
    int ru = find(get_id(u)), rv = find(get_id(v));
    if (ru == rv) {
        return false;
    }
    num_sets--;
    if (rank[ru] < rank[rv]) {
        std::swap(ru, rv);
    }
    root[rv] = ru;
    if (rank[ru] == rank[rv]) {
        rank[ru]++;
    }
    return true;
}

std::vector<std::vector<T>> get_all_sets() {
    std::unordered_map<int, std::vector<T>> tmp;
    for (auto &[key, val] : id) {
        tmp[find(val)].push_back(key);
    }
    std::vector<std::vector<T>> res;
    for (auto &[key, val] : tmp) {
        res.push_back(val);
    }
    return res;
}
};

```

## Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    SparseDSU<char> dsu;
    for (char c = 'a'; c <= 'g'; c++) {
        dsu.make_set(c);
    }
    assert(dsu.unite('a', 'b'));
    assert(dsu.unite('b', 'f'));
    assert(dsu.unite('d', 'e'));
    assert(dsu.unite('d', 'g'));
    assert(!dsu.unite('a', 'f')); // Already united.
    assert(dsu.size() == 7);
    assert(dsu.sets() == 3);
    auto s = dsu.get_all_sets();
    for (auto &group : s) {
        sort(group.begin(), group.end());
    }
    sort(s.begin(), s.end());
    assert((s == vector<vector<char>>{{'a', 'b', 'f'}, {'c'}, {'d', 'e', 'g'}}));
    return 0;
}

```

### 2.8.3 Rollback Disjoint Set Union

Maintains disjoint sets while supporting rollback to previous snapshots. Rollback DSU is useful for offline dynamic connectivity, divide-and-conquer over time, and backtracking search where unions must be undone.

Path compression is intentionally not used, because it is hard to undo. Union by size keeps tree height logarithmic, and every successful union records enough information to restore the previous state. Specifically, a union attaches the smaller root below the larger one and records the attached root, its new parent, and that parent's old size. A snapshot is just the current number of history records. Rolling back pops records until reaching that position, detaching each child, restoring the parent size, and increasing the set count. Unions of elements already in the same set change nothing and therefore add no history record.

- `RollbackDSU(n)` constructs `n` singleton sets over elements  $[0, n)$ .
- `find(u)` returns the representative of the set containing `u`.
- `sets()` returns the current number of disjoint sets.
- `is_united(u, v)` returns whether `u` and `v` are in the same set.
- `unite(u, v)` merges two sets and returns whether a merge occurred.
- `snapshot()` returns a token representing the current history size.
- `rollback(snapshot)` undoes all changes made after `snapshot`.

#### Time Complexity:

- $O(n)$  per call to the constructor.
- $O(1)$  per call to `sets()`.
- $O(\log n)$  worst-case per call to `find()`, `is_united()`, and `unite()`.
- $O(1)$  per undone union during `rollback()`.

**Space Complexity:**  $O(n + q)$  for  $q$  successful unions stored in the rollback history.

```
#include <cassert>
#include <numeric>
#include <tuple>
#include <utility>
#include <vector>

class RollbackDSU {
    std::vector<int> root, size;
    std::vector<std::tuple<int, int, int>> history; // (child, parent, old parent size)
    int num_sets;

public:
    explicit RollbackDSU(int n) : root(n), size(n, 1), num_sets(n) {
        std::iota(root.begin(), root.end(), 0);
    }

    int find(int u) const {
        assert(0 <= u && u < static_cast<int>(root.size()));
        while (root[u] != u) {
            u = root[u];
        }
        return u;
    }

    int sets() const { return num_sets; }
    bool is_united(int u, int v) const { return find(u) == find(v); }

    bool unite(int u, int v) {
        u = find(u);
        v = find(v);
        if (u == v) {
            return false;
        }
        if (size[u] < size[v]) {
            std::swap(u, v);
        }
        history.emplace_back(v, u, size[u]);
        root[v] = u;
        size[u] += size[v];
        num_sets--;
        return true;
    }
};
```

```

}

int snapshot() const { return static_cast<int>(history.size()); }

void rollback(int snapshot) {
    assert(0 <= snapshot && snapshot <= static_cast<int>(history.size()));
    while (static_cast<int>(history.size()) > snapshot) {
        auto [child, parent, parent_size] = history.back();
        history.pop_back();
        root[child] = child;
        size[parent] = parent_size;
        num_sets++;
    }
}
};

```

### Example Usage

```

int main() {
    RollbackDSU dsu(5);
    dsu.unite(0, 1);
    int s = dsu.snapshot();
    dsu.unite(1, 2);
    dsu.unite(3, 4);
    assert(dsu.is_united(0, 2));
    assert(dsu.sets() == 2);
    dsu.rollback(s);
    assert(dsu.is_united(0, 1));
    assert(!dsu.is_united(0, 2));
    assert(!dsu.is_united(3, 4));
    assert(dsu.sets() == 4);
    return 0;
}

```

## 2.8.4 Lowest Common Ancestor (Binary Lifting)

Given a rooted tree or forest, determine the lowest common ancestor of any two nodes in the same tree. The lowest common ancestor of two nodes  $u$  and  $v$  is the node that has the greatest depth while having both  $u$  and  $v$  as descendants. A node is considered to be a descendant of itself.

This implementation preprocesses binary ancestor jumps. During depth-first search, it records entry and exit times so ancestry can be tested in  $O(1)$ , records each node's depth and root, and stores in `up[u][i]` the ancestor  $2^i$  edges above node `u`. To answer `lca(u, v)`, it first handles the case where one node is already an ancestor of the other, then jumps `u` upward by decreasing powers of two until its parent is the lowest common ancestor.

- `BinaryLiftingLCA(adj)` builds the structure over a forest represented by the bidirectional adjacency list `adj`, whose indices represent the nodes.
- `go_up(u, k)` returns the ancestor `k` edges above node `u`. When `k == 0`, it returns `u`; values larger than `u`'s depth stop at that tree's root.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`, or `-1` if they are in different trees.
- `dist(u, v)` returns the number of edges on the path between nodes `u` and `v`, or `-1` if they are in different trees.

#### Time Complexity:

- $O(n \log n)$  for construction, where  $n$  is the number of nodes.
- $O(\log n)$  per call to `go_up()`, `lca()`, and `dist()`.

#### Space Complexity:

- $O(n \log n)$  to store the binary ancestor table.



- $O(n)$  auxiliary stack space for construction.
- $O(1)$  auxiliary for `go_up()`, `lca()`, and `dist()`.

```

#include <algorithm>
#include <cassert>
#include <vector>

class BinaryLiftingLCA {
    std::vector<std::vector<int>>> up;
    std::vector<int> tin, tout, depth, root;
    int len;

    bool is_ancestor(int parent, int child) const {
        assert(0 <= parent && parent < static_cast<int>(root.size()));
        assert(0 <= child && child < static_cast<int>(root.size()));
        return root[parent] == root[child] && tin[parent] <= tin[child] && tout[child] <= tout[parent];
    }

public:
    explicit BinaryLiftingLCA(const std::vector<std::vector<int>>> &adj)
        : tin(adj.size()), tout(adj.size()), depth(adj.size()), root(adj.size(), -1) {
        int n = static_cast<int>(adj.size());
        len = 1;
        while ((1 << len) <= std::max(1, n)) {
            len++;
        }
        up.assign(n, std::vector<int>(len));
        int timer = 0;
        for (int r = 0; r < n; r++) {
            if (root[r] == -1) {
                auto dfs = [&](auto &&dfs, int u, int p, int d) -> void {
                    tin[u] = timer++;
                    depth[u] = d;
                    root[u] = r;
                    up[u][0] = p;
                    for (int i = 1; i < len; i++) {
                        up[u][i] = up[up[u][i - 1]][i - 1];
                    }
                    for (int v : adj[u]) {
                        if (v != p) {
                            dfs(dfs, v, u, d + 1);
                        }
                    }
                    tout[u] = timer++;
                };
                dfs(dfs, r, r, 0);
            }
        }
    }

    int go_up(int u, int k) const {
        assert(0 <= u && u < static_cast<int>(root.size()) && k >= 0);
        k = std::min(k, depth[u]);
        for (int i = 0; i < len; i++) {
            if ((k & (1 << i)) != 0) {
                u = up[u][i];
            }
        }
        return u;
    }

    int lca(int u, int v) const {
        assert(0 <= u && u < static_cast<int>(root.size()));
        assert(0 <= v && v < static_cast<int>(root.size()));
        if (root[u] != root[v]) {
            return -1;
        }
    }
}

```

```

    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}

int dist(int u, int v) const {
    int l = lca(u, v);
    return l == -1 ? -1 : depth[u] + depth[v] - 2 * depth[l];
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0---1---2    5---6
    // |   |
    // 4   3
    vector<vector<int>> adj(7);
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    add_edge(0, 1);
    add_edge(0, 4);
    add_edge(1, 2);
    add_edge(1, 3);
    add_edge(5, 6);
    BinaryLiftingLCA tree(adj);
    assert(tree.go_up(3, 1) == 1);
    assert(tree.go_up(3, 2) == 0);
    assert(tree.go_up(3, 10) == 0);
    assert(tree.lca(3, 2) == 1);
    assert(tree.lca(2, 4) == 0);
    assert(tree.lca(2, 6) == -1);
    assert(tree.lca(5, 6) == 5);
    assert(tree.dist(3, 2) == 2);    // 3-1-2.
    assert(tree.dist(2, 4) == 3);    // 2-1-0-4.
    assert(tree.dist(5, 6) == 1);    // 5-6.
    assert(tree.dist(2, 6) == -1);    // Different trees.
    return 0;
}

```

## 2.8.5 Lowest Common Ancestor (Euler Tour)

Given a rooted tree or forest, determine the lowest common ancestor of any two nodes in the same tree. The lowest common ancestor of two nodes  $u$  and  $v$  is the node that has the greatest depth while having both  $u$  and  $v$  as descendants. A node is considered to be a descendant of itself.

This reduces LCA to a range-minimum query. An Euler tour of the forest records, at each visit to a node, that node's depth; the lowest common ancestor of  $u$  and  $v$  in the same tree is the shallowest node visited between their

first occurrences in the tour, found by a range-minimum query over the depth sequence. This version answers those queries with a segment tree.

- `EulerTourLCA(adj)` builds the structure over a forest represented by the bidirectional adjacency list `adj`, whose indices represent the nodes.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`, or `-1` if they are in different trees.
- `dist(u, v)` returns the number of edges on the path between nodes `u` and `v`, or `-1` if they are in different trees.

#### Time Complexity:

- $O(n)$  for construction, where  $n$  is the number of nodes.
- $O(\log n)$  per call to `lca()` and `dist()`.

#### Space Complexity:

- $O(n)$  for storage of the segment tree.
- $O(n)$  auxiliary stack space for construction.
- $O(\log n)$  auxiliary stack space for `lca()`.

```
#include <algorithm>
#include <cassert>
#include <vector>

class EulerTourLCA {
    std::vector<int> root, depth, first, order, minpos;
    int len;

    int better(int a, int b) const { return depth[a] < depth[b] ? a : b; }

    void build(int i, int lo, int hi) {
        if (lo == hi) {
            minpos[i] = order[lo];
            return;
        }
        int mid = lo + (hi - lo) / 2;
        int l = i * 2 + 1, r = i * 2 + 2;
        build(l, lo, mid);
        build(r, mid + 1, hi);
        minpos[i] = better(minpos[l], minpos[r]);
    }

    int get_minpos(int a, int b, int i, int lo, int hi) const {
        if (a == lo && b == hi) {
            return minpos[i];
        }
        int mid = lo + (hi - lo) / 2;
        int l = i * 2 + 1, r = i * 2 + 2;
        if (b <= mid) {
            return get_minpos(a, b, l, lo, mid);
        }
        if (mid < a) {
            return get_minpos(a, b, r, mid + 1, hi);
        }
        return better(get_minpos(a, mid, l, lo, mid), get_minpos(mid + 1, b, r, mid + 1, hi));
    }

public:
    explicit EulerTourLCA(const std::vector<std::vector<int>> &adj)
        : root(adj.size(), -1), depth(adj.size(), -1), first(adj.size(), -1) {
        int n = static_cast<int>(adj.size());
        for (int r = 0; r < n; r++) {
            if (depth[r] == -1) {
                auto dfs = [&](auto &&dfs, int u, int r, int d) -> void {
                    root[u] = r;
                    depth[u] = d;
                    first[u] = static_cast<int>(order.size());
                };
                dfs(dfs, r, 0);
            }
        }
    }
};
```

```

        order.push_back(u);
        for (int v : adj[u]) {
            if (depth[v] == -1) {
                dfs(dfs, v, r, d + 1);
                order.push_back(u);
            }
        }
    };
    dfs(dfs, r, r, 0);
}

len = static_cast<int>(order.size());
minpos.assign(std::max(1, 4 * len), 0);
if (len > 0) {
    build(0, 0, len - 1);
}
}

int lca(int u, int v) const {
    assert(0 <= u && u < static_cast<int>(root.size()));
    assert(0 <= v && v < static_cast<int>(root.size()));
    if (root[u] != root[v]) {
        return -1;
    }
    return get_minpos(std::min(first[u], first[v]), std::max(first[u], first[v]), 0, 0, len - 1);
}

int dist(int u, int v) const {
    int l = lca(u, v);
    return l == -1 ? -1 : depth[u] + depth[v] - 2 * depth[l];
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0---1---2    5---6
    // |   |
    // 4   3
    vector<vector<int>> adj(7);
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    add_edge(0, 1);
    add_edge(0, 4);
    add_edge(1, 2);
    add_edge(1, 3);
    add_edge(5, 6);
    EulerTourLCA tree(adj);
    assert(tree.lca(3, 2) == 1);
    assert(tree.lca(2, 4) == 0);
    assert(tree.lca(2, 6) == -1);
    assert(tree.lca(5, 6) == 5);
    assert(tree.dist(3, 2) == 2);    // 3-1-2.
    assert(tree.dist(2, 4) == 3);    // 2-1-0-4.
    assert(tree.dist(5, 6) == 1);    // 5-6.
    assert(tree.dist(2, 6) == -1);    // Different trees.
    return 0;
}

```

### 2.8.6 Virtual Tree

Given a rooted tree and a subset of its nodes, build the virtual tree (also called the auxiliary or compressed tree): the smallest tree that contains every node of the subset together with the lowest common ancestor of each pair, with chains of irrelevant degree-two ancestors compressed into single edges. Many problems ask repeated queries each restricted to a handful of “important” nodes; the virtual tree has  $O(k)$  nodes for a subset of size  $k$ , so an algorithm can run over it instead of the whole tree, turning a per-query cost proportional to  $n$  into one proportional to the subset size.

The construction sorts the subset by depth-first entry time, which lists the nodes in the order a walk of the tree would meet them. The lowest common ancestor of any contiguous span of the subset is the minimum-depth node among the pairwise LCAs of adjacent entries, so it suffices to add the LCA of each adjacent pair, re-sort, and deduplicate. A single stack pass over the resulting nodes then links each one to its parent in the virtual tree: ancestors that are no longer on the path to the current node are popped, and the node on top becomes the parent. This implementation carries its own binary lifting so it is self-contained; given an existing lowest common ancestor structure, the same sort-and-stack logic applies directly.

- `VirtualTree(adj, root)` builds the structure over a tree rooted at `root`, represented by the bidirectional adjacency list `adj`, whose indices represent the nodes.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`.
- `depth_of(u)` returns the depth of `u` in the original tree (the root has depth 0). The original path length of a virtual-tree edge `p-c` is `depth_of(c) - depth_of(p)`.
- `build(nodes)` returns the virtual tree induced by `nodes`. The result has `root` set to the topmost node (the common ancestor of `nodes`), `nodes` listing every original node ID in the virtual tree sorted by entry time, and `edges` listing each `(p, c)` pair in original IDs. Both the node set and the edges use the node numbering of the original tree. An empty input yields an empty result with `root = -1`.

#### Time Complexity:

- $O(n \log n)$  for construction, where  $n$  is the number of nodes.
- $O(\log n)$  per call to `lca()`.
- $O(k \log n)$  per call to `build()`, where  $k$  is the size of the input subset.

#### Space Complexity:

- $O(n \log n)$  to store the binary ancestor table.
- $O(k)$  auxiliary plus result size for `build()`.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

class VirtualTree {
    std::vector<std::vector<int>>> up;
    std::vector<int> tin, tout, depth;
    int len;

public:
    struct Tree {
        int root = -1;
        std::vector<int> nodes;
        std::vector<std::pair<int, int>> edges; // (parent, child) in original node ids.
    };

    VirtualTree(const std::vector<std::vector<int>>> &adj, int root)
        : tin(adj.size()), tout(adj.size()), depth(adj.size()) {
        int n = static_cast<int>(adj.size());
        assert(0 <= root && root < n);
        len = 1;
        while ((1 << len) <= std::max(1, n)) {
            len++;
        }
    }
};
```

```

up.assign(n, std::vector<int>(len));
int timer = 0;
auto dfs = [&](auto &&dfs, int u, int p, int d) -> void {
    tin[u] = timer++;
    depth[u] = d;
    up[u][0] = p;
    for (int i = 1; i < len; i++) {
        up[u][i] = up[up[u][i - 1]][i - 1];
    }
    for (int v : adj[u]) {
        if (v != p) {
            dfs(dfs, v, u, d + 1);
        }
    }
    tout[u] = timer++;
};
dfs(dfs, root, root, 0);
}

bool is_ancestor(int parent, int child) const {
    assert(0 <= parent && parent < static_cast<int>(tin.size()));
    assert(0 <= child && child < static_cast<int>(tin.size()));
    return tin[parent] <= tin[child] && tout[child] <= tout[parent];
}

int lca(int u, int v) const {
    assert(0 <= u && u < static_cast<int>(tin.size()));
    assert(0 <= v && v < static_cast<int>(tin.size()));
    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}

// Depth of u in the original tree; depth(child) - depth(parent) is the length of the original
// path that a compressed virtual-tree edge stands for.
int depth_of(int u) const {
    assert(0 <= u && u < static_cast<int>(depth.size()));
    return depth[u];
}

Tree build(std::vector<int> nodes) const {
    for (int u : nodes) {
        assert(0 <= u && u < static_cast<int>(tin.size()));
    }
    Tree res;
    if (nodes.empty()) {
        return res;
    }
    auto by_tin = [&](int a, int b) { return tin[a] < tin[b]; };
    std::sort(nodes.begin(), nodes.end(), by_tin);
    nodes.erase(std::unique(nodes.begin(), nodes.end(), nodes.end()));
    int k = static_cast<int>(nodes.size());
    for (int i = 0; i + 1 < k; i++) {
        nodes.push_back(lca(nodes[i], nodes[i + 1]));
    }
    std::sort(nodes.begin(), nodes.end(), by_tin);
    nodes.erase(std::unique(nodes.begin(), nodes.end(), nodes.end()));
    std::vector<int> ancestors;
    for (int v : nodes) {

```

```

    while (!ancestors.empty() && !is_ancestor(ancestors.back(), v)) {
        ancestors.pop_back();
    }
    if (!ancestors.empty()) {
        res.edges.emplace_back(ancestors.back(), v);
    }
    ancestors.push_back(v);
}
res.root = nodes.front();
res.nodes = std::move(nodes);
return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      0
    //     / |
    //    1 2
    //   /| |
    //  6--3 4 5--7
    vector<vector<int>>> adj(8);
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    add_edge(0, 1);
    add_edge(0, 2);
    add_edge(1, 3);
    add_edge(1, 4);
    add_edge(2, 5);
    add_edge(3, 6);
    add_edge(5, 7);
    VirtualTree vt(adj, 0);
    assert(vt.lca(6, 4) == 1);
    assert(vt.lca(6, 7) == 0);
    assert(vt.depth_of(0) == 0 && vt.depth_of(6) == 3);
    // The compressed edge 0 -> 7 stands for an original path of length 3 (0-2-5-7).
    assert(vt.depth_of(7) - vt.depth_of(0) == 3);

    // Virtual tree of {6, 7}: their LCA is 0, so the tree is 0 -> {6, 7} with edges compressed.
    //      0
    //     / |
    //    6 7
    VirtualTree::Tree t = vt.build({6, 7});
    assert(t.root == 0);
    assert((t.nodes == vector<int>{0, 6, 7}));
    assert(t.edges.size() == 2);
    for (auto &e : t.edges) {
        assert(e.first == 0 && (e.second == 6 || e.second == 7));
    }

    // Virtual tree of {6, 4, 7}: node 1 becomes a branch point (LCA of 6 and 4).
    //      0
    //     / |
    //    1 7
    //   / |
    //  6 4
    VirtualTree::Tree u = vt.build({6, 4, 7});
    assert(u.root == 0);
    assert((u.nodes == vector<int>{0, 1, 6, 4, 7}));
    // Expected edges: 0->1, 1->6, 1->4, 0->7.

```

```

vector<pair<int, int>> want{{0, 1}, {1, 6}, {1, 4}, {0, 7}};
auto got = u.edges;
sort(got.begin(), got.end());
sort(want.begin(), want.end());
assert(got == want);

// A single node is its own virtual tree with no edges.
VirtualTree::Tree s = vt.build({4});
assert(s.root == 4 && s.edges.empty() && s.nodes == vector<int>{4});
return 0;
}

```

## 2.8.7 Weighted Tree Path Queries

Given a tree or forest with weighted edges, aggregate the edge weights along the path between any two nodes in the same tree. The input is undirected; construction roots each component at the first unvisited node in index order. This builds on the same binary lifting table as LCA: for every valid  $2^i$ -edge jump from  $u$ , it stores  $\text{agg}[u][i]$ , the combined weight of the edges traversed by that jump. A path query splits the path  $u-v$  at their lowest common ancestor and merges the two upward climbs.

The aggregate is defined by a commutative, associative `combine()`. The default computes the maximum edge weight on the path. For minimum queries, use `std::min`; for sums, use `a + b`. Commutativity is required because the two half-paths are merged without regard to orientation. For an invertible aggregate such as the sum, an alternative that avoids the `agg` table is to store each node's weighted depth `dw[u]` and return `dw[u] + dw[v] - 2*dw[lca]`.

- `TreePathQueries<W>(adj)` builds the structure over a forest given by a weighted, bidirectional adjacency list `adj`, whose indices represent the nodes and where `adj[u]` holds pairs  $(v, w)$  for each edge  $u-v$  of weight  $w$ . Each connected component is rooted at its first node reached by the constructor's outer loop.
- `lca(u, v)` returns the lowest common ancestor of  $u$  and  $v$ , or  $-1$  if they lie in different trees.
- `kth_ancestor(u, k)` returns the ancestor  $k$  edges above  $u$  ( $k = 0$  returns  $u$ ), stopping at that tree's root if  $k$  exceeds  $u$ 's depth.
- `path_query(u, v)` returns the combined weight of the edges on the path from  $u$  to  $v$ . The nodes must be distinct and lie in the same tree.

### Time Complexity:

- $O(n \log n)$  for construction, where  $n$  is the number of nodes.
- $O(\log n)$  per call to `lca()`, `kth_ancestor()`, and `path_query()`.

### Space Complexity:

- $O(n \log n)$  to store the binary ancestor and aggregate tables.
- $O(n)$  auxiliary stack space for construction.
- $O(1)$  auxiliary for `lca()`, `kth_ancestor()`, and `path_query()`.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <utility>
#include <vector>

template<typename W>
class TreePathQueries {
    static W combine(const W &a, const W &b) { return std::max(a, b); }

    std::vector<std::vector<int>> up;
    std::vector<std::vector<W>> agg; // agg[u][i] = combined weight of the 2^i edges above u.
    std::vector<int> tin, tout, depth, root;
    int len;

```



```

bool is_ancestor(int parent, int child) const {
    assert(0 <= parent && parent < static_cast<int>(root.size()));
    assert(0 <= child && child < static_cast<int>(root.size()));
    return root[parent] == root[child] && tin[parent] <= tin[child] && tout[child] <= tout[parent];
}

```

```

public:

```

```

    explicit TreePathQueries(const std::vector<std::vector<std::pair<int, W>>> &adj)
        : agg(adj.size()),
          tin(adj.size()),
          tout(adj.size()),
          depth(adj.size()),
          root(adj.size(), -1) {
        int n = static_cast<int>(adj.size());
        len = 1;
        while ((1 << len) <= std::max(1, n)) {
            len++;
        }
        up.assign(n, std::vector<int>(len));
        int timer = 0;
        for (int r = 0; r < n; r++) {
            if (root[r] == -1) {
                auto dfs = [&](auto &&dfs, int u, int p, int d) -> void {
                    tin[u] = timer++;
                    depth[u] = d;
                    root[u] = r;
                    up[u][0] = p;
                    for (int i = 1; i < len; i++) {
                        up[u][i] = up[up[u][i - 1]][i - 1];
                        if ((1 << i) <= d) {
                            agg[u].push_back(combine(agg[u][i - 1], agg[up[u][i - 1]][i - 1]));
                        }
                    }
                    for (const auto &[v, w] : adj[u]) {
                        if (v != p) {
                            agg[v].push_back(w); // The edge into a child is stored on the child.
                            dfs(dfs, v, u, d + 1);
                        }
                    }
                    tout[u] = timer++;
                };
                dfs(dfs, r, r, 0);
            }
        }
    }
}

```

```

int lca(int u, int v) const {
    assert(0 <= u && u < static_cast<int>(root.size()));
    assert(0 <= v && v < static_cast<int>(root.size()));
    if (root[u] != root[v]) {
        return -1;
    }
    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}

```

```

int kth_ancestor(int u, int k) const {
    assert(0 <= u && u < static_cast<int>(root.size()) && k >= 0);

```

```

    k = std::min(k, depth[u]);
    for (int i = 0; i < len; i++) {
        if ((k & (1 << i)) != 0) {
            u = up[u][i];
        }
    }
    return u;
}

W path_query(int u, int v) const {
    int l = lca(u, v);
    assert(l != -1 && u != v);
    // Combined weight of the k edges on the upward path of k steps from a node.
    auto climb = [&](int from, int k) -> std::optional<W> {
        std::optional<W> res;
        for (int i = 0; i < len; i++) {
            if ((k & (1 << i)) != 0) {
                res = res ? combine(*res, agg[from][i]) : agg[from][i];
                from = up[from][i];
            }
        }
        return res;
    };
    std::optional<W> left = climb(u, depth[u] - depth[l]);
    std::optional<W> right = climb(v, depth[v] - depth[l]);
    return !left ? *right : (!right ? *left : combine(*left, *right));
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //          0
    //      w=4 /      \ w=2
    //      1         2
    //  w=7 / \w=1   w=5/ \ w=3
    //   3  4       5  6
    vector<vector<pair<int, int>>>> adj(7);
    auto add_edge = [&](int u, int v, int w) {
        adj[u].emplace_back(v, w);
        adj[v].emplace_back(u, w);
    };
    add_edge(0, 1, 4);
    add_edge(0, 2, 2);
    add_edge(1, 3, 7);
    add_edge(1, 4, 1);
    add_edge(2, 5, 5);
    add_edge(2, 6, 3);
    TreePathQueries<int> t(adj);
    assert(t.lca(3, 4) == 1);
    assert(t.lca(5, 6) == 2);
    assert(t.lca(3, 6) == 0);
    assert(t.kth_ancestor(3, 1) == 1);
    assert(t.kth_ancestor(3, 2) == 0);
    assert(t.kth_ancestor(3, 5) == 0); // Clamped to the root.
    // Default combine() reports the maximum edge weight on the path.
    assert(t.path_query(3, 4) == 7); // 3-1 (7), 1-4 (1).
    assert(t.path_query(5, 6) == 5); // 5-2 (5), 2-6 (3).
    assert(t.path_query(4, 5) == 5); // 4-1 (1), 1-0 (4), 0-2 (2), 2-5 (5).
    assert(t.path_query(0, 3) == 7); // 0-1 (4), 1-3 (7).

    vector<vector<pair<int, int>>>> forest(2);
    TreePathQueries<int> disconnected(forest);
}

```

```

    assert(disconnected.lca(0, 1) == -1);
    return 0;
}

```

### 2.8.8 Heavy-Light Decomposition

Maintain a tree or forest with values associated with either edges or nodes, while supporting both dynamic queries and dynamic updates of all values on a given path between two nodes in the tree. Heavy-light decomposition chooses at most one heavy child for each node and partitions the tree into maximal paths of heavy edges. This implementation keeps a child on its parent’s path when its subtree contains at least half of its parent’s nodes. Every remaining edge is light and at least halves the remaining subtree size, so walking from any node to the root crosses  $O(\log n)$  heavy paths. Each path stores its values in its own lazy segment tree, decomposing any path query or update into  $O(\log n)$  contiguous range operations.

The query operation is defined by an associative `combine()` function. The default code below assumes a numerical tree type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`. For direction-independent path queries, `combine()` should also be commutative; otherwise, store enough information in each aggregate to combine paths in the required order.

The update operation is defined by `apply_delta()` and `compose_deltas()`. A delta must act on an aggregate summary of a path of length `len`: `apply_delta(v, d, len)` returns the aggregate after applying update `d` to every element represented by aggregate `v`. Pending deltas are combined in chronological order by `compose_deltas(old, d)`, meaning “apply `old`, then apply `d`”. These hooks do not support arbitrary query/update pairings; the delta operation must distribute over `combine()`, and composed deltas must have the same effect as applying their updates sequentially. The default code below defines updates that “set” a path’s edges or nodes to a new value. For range increment updates, `apply_delta(v, d, len)` would return `v + d` for min/max queries, or `v + d * len` for sum queries, and `compose_deltas(old, d)` would return `old + d`.

- `HeavyLight<T, VALUES_ON_EDGES = true>(adj, v)` constructs a new heavy-light decomposition on a forest defined by the adjacency list `adj`, with all values initialized to `v`. Its entries must be valid indices into `adj`, and no duplicate edges should exist. Set `VALUES_ON_EDGES` to false to store values on nodes instead.
- `for_each_path(u, v, include_lca, f)` decomposes the path from node `u` to node `v` into heavy path ranges and calls `f(path, lo, hi, up)` on each range, where `up` says the path segment is traversed upward from `u` toward the LCA. If `VALUES_ON_EDGES = true`, pass `include_lca = false` to skip the LCA’s node slot. Returns false if `u` and `v` are in different trees.
- `query(u, v)` returns the result of `combine()` applied to all values on the path from node `u` to node `v`. The nodes must be in the same tree and, when values are on edges, must be distinct.
- `update(u, v, d)` modifies all values on the path from node `u` to node `v` by applying the delta `d`. The nodes must be in the same tree.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the number of nodes.
- $O(\log n)$  calls to `f()` per call to `for_each_path()`.
- $O(\log^2 n)$  per call to `query()` and `update()`.

#### Space Complexity:

- $O(n)$  for storage of the decomposition.
- $O(n)$  auxiliary stack space for the constructor.
- $O(\log n)$  auxiliary for `for_each_path()`, `query()`, and `update()`.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <tuple>
#include <vector>

```

```

template<typename T, bool VALUES_ON_EDGES = true>
class HeavyLight {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int len) { return d; }
    static T compose_deltas(const T &old, const T &d) { return d; }

    std::vector<std::vector<T>> value;
    std::vector<std::vector<std::optional<T>>> delta;
    std::vector<std::vector<int>> len;
    std::vector<int> parent, root, tin, tout, path, pathpos, pathroot;

    inline T applied_value(int path_id, int i) {
        return delta[path_id][i] ? apply_delta(value[path_id][i], *delta[path_id][i], len[path_id][i])
            : value[path_id][i];
    }

    void push_delta(int path_id, int i) {
        int d = 0;
        while ((i >> d) > 0) {
            d++;
        }
        for (d -= 2; d >= 0; d--) {
            int l = (i >> d), r = (1 ^ 1), n = 1 / 2;
            if (delta[path_id][n]) {
                value[path_id][n] = applied_value(path_id, n);
                const T &d = *delta[path_id][n];
                delta[path_id][l] = delta[path_id][l] ? compose_deltas(*delta[path_id][l], d) : d;
                delta[path_id][r] = delta[path_id][r] ? compose_deltas(*delta[path_id][r], d) : d;
                delta[path_id][n].reset();
            }
        }
    }

    std::optional<T> query(int path_id, int u, int v) {
        push_delta(path_id, u += static_cast<int>(value[path_id].size() / 2));
        push_delta(path_id, v += static_cast<int>(value[path_id].size() / 2));
        std::optional<T> res;
        for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
            if ((u & 1) != 0) {
                T part_value = applied_value(path_id, u);
                res = res ? combine(*res, part_value) : part_value;
            }
            if ((v & 1) == 0) {
                T part_value = applied_value(path_id, v);
                res = res ? combine(*res, part_value) : part_value;
            }
        }
        return res;
    }

    void update(int path_id, int u, int v, const T &d) {
        push_delta(path_id, u += static_cast<int>(value[path_id].size() / 2));
        push_delta(path_id, v += static_cast<int>(value[path_id].size() / 2));
        int tu = -1, tv = -1;
        for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
            if ((u & 1) != 0) {
                delta[path_id][u] = delta[path_id][u] ? compose_deltas(*delta[path_id][u], d) : d;
                if (tu == -1) {
                    tu = u;
                }
            }
            if ((v & 1) == 0) {
                delta[path_id][v] = delta[path_id][v] ? compose_deltas(*delta[path_id][v], d) : d;
                if (tv == -1) {
                    tv = v;
                }
            }
        }
    }
}

```

```

    for (int i = tu; i > 1; i /= 2) {
        value[path_id][i / 2] = combine(applied_value(path_id, i), applied_value(path_id, i ^ 1));
    }
    for (int i = tv; i > 1; i /= 2) {
        value[path_id][i / 2] = combine(applied_value(path_id, i), applied_value(path_id, i ^ 1));
    }
}

inline bool is_ancestor(int ancestor, int node) {
    return (tin[ancestor] <= tin[node]) && (tout[node] <= tout[ancestor]);
}

public:
explicit HeavyLight(const std::vector<std::vector<int>> &adj, const T &v = T{})
    : parent(adj.size()),
      root(adj.size(), -1),
      tin(adj.size()),
      tout(adj.size()),
      path(adj.size()),
      pathpos(adj.size()),
      pathroot(adj.size()) {
    int n = static_cast<int>(adj.size()), counter = 0, paths = 0;
    std::vector<int> subtree(n), pathlen(n);
    auto dfs = [&](auto &&dfs, int u, int p, int r) -> void {
        tin[u] = counter++;
        parent[u] = p;
        root[u] = r;
        subtree[u] = 1;
        for (int w : adj[u]) {
            if (w != p) {
                dfs(dfs, w, u, r);
                subtree[u] += subtree[w];
            }
        }
        tout[u] = counter++;
    };
    auto new_path = [&](int u) {
        pathroot[paths] = u;
        return paths++;
    };
    // Label each node with its heavy path and its position along that path.
    auto label = [&](auto &&label, int u, int path_id) -> void {
        path[u] = path_id;
        pathpos[u] = pathlen[path_id]++;
        for (int w : adj[u]) {
            if (w != parent[u]) {
                label(label, w, (2 * subtree[w] >= subtree[u]) ? path_id : new_path(w));
            }
        }
    };
    for (int u = 0; u < n; u++) {
        if (root[u] == -1) {
            dfs(dfs, u, -1, u);
            label(label, u, new_path(u));
        }
    }
    value.resize(paths);
    delta.resize(paths);
    len.resize(paths);
    for (int i = 0; i < paths; i++) {
        int m = pathlen[i];
        value[i].assign(2 * m, v);
        delta[i].resize(2 * m);
        len[i].assign(2 * m, 1);
        for (int j = 2 * m - 1; j > 1; j -= 2) {
            value[i][j / 2] = combine(value[i][j], value[i][j ^ 1]);
            len[i][j / 2] = len[i][j] + len[i][j ^ 1];
        }
    }
}

```

```

    }
}

template<typename Fn>
bool for_each_path(int u, int v, bool include_lca, Fn f) {
    assert(0 <= u && u < static_cast<int>(root.size()));
    assert(0 <= v && v < static_cast<int>(root.size()));
    if (root[u] != root[v]) {
        return false;
    }
    int path_root;
    while (!is_ancestor(path_root = pathroot[path[u]], v)) {
        f(path[u], 0, pathpos[u], true);
        u = parent[path_root];
    }
    std::vector<std::tuple<int, int, int>> down_parts;
    while (!is_ancestor(path_root = pathroot[path[v]], u)) {
        down_parts.emplace_back(path[v], 0, pathpos[v]);
        v = parent[path_root];
    }
    int lo = std::min(pathpos[u], pathpos[v]) + static_cast<int>(!include_lca);
    int hi = std::max(pathpos[u], pathpos[v]);
    if (lo <= hi) {
        f(path[u], lo, hi, pathpos[u] > pathpos[v]);
    }
    for (int i = static_cast<int>(down_parts.size()) - 1; i >= 0; i--) {
        auto [down_path, down_lo, down_hi] = down_parts[i];
        f(down_path, down_lo, down_hi, false);
    }
    return true;
}

T query(int u, int v) {
    assert(0 <= u && u < static_cast<int>(root.size()));
    assert(0 <= v && v < static_cast<int>(root.size()));
    assert(root[u] == root[v] && (!VALUES_ON_EDGES || u != v));
    std::optional<T> res;
    for_each_path(u, v, !VALUES_ON_EDGES, [&](int path_id, int lo, int hi, bool) {
        if (auto part = query(path_id, lo, hi)) {
            res = res ? combine(*res, *part) : *part;
        }
    });
    assert(res);
    return *res;
}

void update(int u, int v, const T &d) {
    assert(0 <= u && u < static_cast<int>(root.size()));
    assert(0 <= v && v < static_cast<int>(root.size()));
    assert(root[u] == root[v]);
    if (VALUES_ON_EDGES && u == v) {
        return;
    }
    for_each_path(u, v, !VALUES_ON_EDGES, [&](int path_id, int lo, int hi, bool) {
        update(path_id, lo, hi, d);
    });
}
};

```

## Example Usage

```

using namespace std;

int main() {
    //   v=40   v=20   v=10
    // 0-----1-----2-----3   5-----6

```

```

//          |
//          +-----4
//          v=30
vector<vector<int>> adj(7);
auto add_edge = [&](int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
};
add_edge(0, 1);
add_edge(1, 2);
add_edge(2, 3);
add_edge(2, 4);
add_edge(5, 6);
HeavyLight<int> hld(adj, 0);
hld.update(0, 1, 40);
hld.update(1, 2, 20);
hld.update(2, 3, 10);
hld.update(2, 4, 30);
assert(hld.query(0, 3) == 10);
assert(hld.query(2, 4) == 30);

// Update path 3-2-4 to value 5:
//
//   v=40   v=20   v=5
// 0-----1-----2-----3   5-----6
//           |
//           +-----4
//           v=5
hld.update(3, 4, 5);
assert(hld.query(1, 4) == 5);

// The disconnected component can be updated independently:
//
// 5-----6
//   v=7
hld.update(5, 6, 7);
assert(hld.query(5, 6) == 7);
assert(!hld.for_each_path(0, 5, true, [](int, int, int, bool) {}));

// v=100 v=100 v=100 v=100   v=100 v=100
// 0-----1-----2-----3   5-----6
//           |
//           +-----4
//           v=100
HeavyLight<int, false> node_hld(adj, 100);

// Update nodes on path 1-2-4 to value 8:
//
// v=100 v=8 v=8 v=100 v=100 v=100
// 0-----1-----2-----3   5-----6
//           |
//           +-----4
//           v=8
node_hld.update(1, 4, 8);
assert(node_hld.query(0, 3) == 8); // Min over nodes 0, 1, 2, 3.
assert(node_hld.query(5, 5) == 100); // Single-node paths are allowed.
node_hld.update(5, 5, 7);
assert(node_hld.query(5, 5) == 7);
return 0;
}

```

### 2.8.9 Euler Tour Tree

Maintain an undirected forest dynamically using Euler Tour Trees. Each tree is represented by a sequence of directed edge occurrences in an implicit treap: an undirected edge  $(u, v)$  contributes two nodes,  $(u \rightarrow v)$  and

( $v \rightarrow u$ ). Rerooting a represented tree is a cyclic rotation of this sequence, while `link` concatenates two tours with the new edge occurrences and `cut` removes the two occurrences of an existing edge, splitting one tour into two.

This version supports dynamic connectivity, `link`, `cut`, rerooting, and listing the nodes in a connected component. It does not store node aggregates. To add component aggregates, a common extension is to insert one self-loop occurrence per node and maintain aggregate data in the treap.

- `EulerTourTree(n)` constructs a forest on nodes numbered  $[0, n)$ .
- `connected(u, v)` returns whether `u` and `v` are in the same tree.
- `link(u, v)` adds an edge between different trees and returns false if the edge would create a cycle or already exists.
- `cut(u, v)` removes an existing edge and returns false if it is not present.
- `reroot(u)` rotates the Euler tour of `u`'s tree so an occurrence leaving `u` is first.
- `component_nodes(u)` returns the distinct nodes in `u`'s tree in unspecified order.

#### Time Complexity:

- $O(\log m)$  expected per call to `connected()`, `link()`, `cut()`, and `reroot()`, where  $m$  is the number of edges currently in the forest.
- $O(k)$  expected per call to `component_nodes()`, where  $k$  is the number of directed edge occurrences in the component.

#### Space Complexity:

- $O(n + m)$  for the forest.
- $O(\log m)$  auxiliary recursion stack for treap operations.

```
#include <cassert>
#include <chrono>
#include <random>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>

class EulerTourTree {
    struct Node {
        int u, v, priority, size;
        Node *left, *right, *parent;

        Node(int u, int v, int priority)
            : u(u), v(v), priority(priority), size(1), left(nullptr), right(nullptr), parent(nullptr) {}
    };

    int n;
    std::mt19937 rng;
    std::vector<std::unordered_map<int, Node*>> adj;

    static int size(Node *t) { return t == nullptr ? 0 : t->size; }

    static void update(Node *t) {
        if (t != nullptr) {
            t->size = 1 + size(t->left) + size(t->right);
            if (t->left != nullptr) {
                t->left->parent = t;
            }
            if (t->right != nullptr) {
                t->right->parent = t;
            }
        }
    }

    static Node *root(Node *t) {
        if (t == nullptr) {
            return nullptr;
        }
    }
};
```



```

    }
    while (t->parent != nullptr) {
        t = t->parent;
    }
    return t;
}

static int index(Node *t) {
    int res = size(t->left);
    while (t->parent != nullptr) {
        if (t == t->parent->right) {
            res += 1 + size(t->parent->left);
        }
        t = t->parent;
    }
    return res;
}

static Node *merge(Node *a, Node *b) {
    if (a == nullptr) {
        return b;
    }
    if (b == nullptr) {
        return a;
    }
    if (a->priority > b->priority) {
        a->right = merge(a->right, b);
        update(a);
        a->parent = nullptr;
        return a;
    }
    b->left = merge(a, b->left);
    update(b);
    b->parent = nullptr;
    return b;
}

static std::pair<Node *, Node *> split(Node *t, int left_size) {
    if (t == nullptr) {
        return {nullptr, nullptr};
    }
    if (size(t->left) >= left_size) {
        auto [a, b] = split(t->left, left_size);
        t->left = b;
        update(t);
        t->parent = nullptr;
        return {a, t};
    }
    auto [a, b] = split(t->right, left_size - size(t->left) - 1);
    t->right = a;
    update(t);
    t->parent = nullptr;
    return {t, b};
}

static Node *make_first(Node *x) {
    auto [a, b] = split(root(x), index(x));
    return merge(b, a);
}

static Node *remove_first(Node *t) {
    auto [first, rest] = split(t, 1);
    if (first != nullptr) {
        first->left = first->right = first->parent = nullptr;
        first->size = 1;
    }
    return rest;
}

```

```

Node *new_node(int u, int v) { return new Node(u, v, std::uniform_int_distribution<int>(rng)); }
Node *any_occurrence(int u) const { return adj[u].empty() ? nullptr : adj[u].begin()->second; }

static void collect(Node *t, std::unordered_set<int> &out) {
    if (t == nullptr) {
        return;
    }
    collect(t->left, out);
    out.insert(t->u);
    collect(t->right, out);
}

public:
explicit EulerTourTree(int n)
    : n(n), rng(std::chrono::steady_clock::now().time_since_epoch().count()), adj(n) {}

~EulerTourTree() {
    for (const auto &neighbors : adj) {
        for (const auto &[_ , n] : neighbors) {
            delete n;
        }
    }
}

EulerTourTree(const EulerTourTree &) = delete;
EulerTourTree &operator=(const EulerTourTree &) = delete;

bool connected(int u, int v) const {
    assert(0 <= u && u < n && 0 <= v && v < n);
    if (u == v) {
        return true;
    }
    Node *a = any_occurrence(u), *b = any_occurrence(v);
    return a != nullptr && b != nullptr && root(a) == root(b);
}

bool link(int u, int v) {
    assert(0 <= u && u < n && 0 <= v && v < n);
    if (u == v || adj[u].count(v) || connected(u, v)) {
        return false;
    }
    Node *tu = any_occurrence(u), *tv = any_occurrence(v);
    if (tu != nullptr) {
        tu = make_first(tu);
    }
    if (tv != nullptr) {
        tv = make_first(tv);
    }
    Node *uv = new_node(u, v), *vu = new_node(v, u);
    adj[u][v] = uv;
    adj[v][u] = vu;
    merge(merge(tu, uv), merge(tv, vu));
    return true;
}

bool cut(int u, int v) {
    assert(0 <= u && u < n && 0 <= v && v < n);
    auto it = adj[u].find(v);
    if (it == adj[u].end()) {
        return false;
    }
    Node *uv = it->second, *vu = adj[v].at(u);
    make_first(uv);
    auto [left, right] = split(root(uv), index(vu));
    remove_first(left); // Remove uv.
    remove_first(right); // Remove vu.
    adj[u].erase(v);
}

```

```

    adj[v].erase(u);
    delete uv;
    delete vu;
    return true;
}

void reroot(int u) {
    assert(0 <= u && u < n);
    Node *a = any_occurrence(u);
    if (a != nullptr) {
        make_first(a);
    }
}

std::vector<int> component_nodes(int u) const {
    assert(0 <= u && u < n);
    Node *a = any_occurrence(u);
    if (a == nullptr) {
        return {};
    }
    std::unordered_set<int> nodes;
    collect(root(a), nodes);
    return std::vector<int>(nodes.begin(), nodes.end());
}
};

```

## Example Usage

```

#include <algorithm>
using namespace std;

int main() {
    EulerTourTree ett(6);
    assert(!ett.connected(0, 1));
    assert(ett.connected(0, 0));

    // 0---1---2    3---4    5
    assert(ett.link(0, 1));
    assert(ett.link(1, 2));
    assert(ett.link(3, 4));
    assert(ett.connected(0, 2));
    assert(!ett.connected(0, 4));
    assert(!ett.link(0, 2)); // Would create a cycle.

    auto c = ett.component_nodes(1);
    sort(c.begin(), c.end());
    assert((c == vector<int>{0, 1, 2}));
    ett.reroot(2);
    assert(ett.connected(0, 2));

    // 0---1    2    3---4    5
    assert(ett.cut(1, 2));
    assert(!ett.connected(0, 2));
    assert((ett.component_nodes(2) == vector<int>{2}));

    // 0---1    2---3---4    5
    assert(ett.link(2, 3));
    assert(ett.connected(0, 4) == false);
    assert(ett.connected(2, 4));
    assert(!ett.cut(0, 5));
    return 0;
}

```

### 2.8.10 Link-Cut Tree

Maintain a forest of trees with values associated with its nodes, while supporting both dynamic queries and dynamic updates of all values on any path between two nodes in a given tree. In addition, support testing of whether two nodes are connected in the forest, as well as the merging and splitting of trees by adding or removing specific edges. Link/cut forests divide each of its trees into node-disjoint paths, each represented by a splay tree.

The query operation is defined by an associative `combine()` function. The default code below assumes a numerical forest type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`. For direction-independent path queries, `combine()` should also be commutative; otherwise, store enough information in each aggregate to combine paths in the required order.

The update operation is defined by `apply_delta()` and `compose_deltas()`. A delta must act on an aggregate summary of a path of length `len`: `apply_delta(v, d, len)` returns the aggregate after applying update `d` to every element represented by aggregate `v`. Pending deltas are combined in chronological order by `compose_deltas(old, d)`, meaning “apply `old`, then apply `d`”. These hooks do not support arbitrary query/update pairings; the delta operation must distribute over `combine()`, and composed deltas must have the same effect as applying their updates sequentially. The default code below defines updates that “set” a path’s nodes to a new value. For range increment updates, `apply_delta(v, d, len)` would return `v + d` for min/max queries, or `v + d * len` for sum queries, and `compose_deltas(old, d)` would return `old + d`.

- `LinkCut<T>()` constructs an empty forest.
- `size()` returns the number of nodes in the forest.
- `trees()` returns the number of trees in the forest.
- `add_node(u, value = T{})` adds a new single-node tree to the forest, labeled with the integer `u` and with value initialized to `value`.
- `connected(u, v)` returns whether nodes `u` and `v` are connected.
- `link(u, v)` adds an edge between the nodes `u` and `v`, both of which must exist and not be connected.
- `cut(u, v)` removes the edge between the nodes `u` and `v`, both of which must exist and be connected.
- `query(u, v)` returns the result of `combine()` applied to all values on the path from node `u` to node `v`.
- `update(u, v, d)` modifies all the values on the path from node `u` to node `v` by applying the delta `d`.

The forest is unrooted: a tree’s root is whichever node was most recently established as such. `reroot(u)` sets it explicitly, while `link`, `query`, and `update` do so implicitly; the latter two use their first argument. The root-sensitive operations below are relative to that current root, so call `reroot(r)` first when a specific root `r` is intended.

- `reroot(u)` makes node `u` the root of its tree.
- `find_root(u)` returns the label of the root of the tree containing node `u`.
- `lca(u, v)` returns the lowest common ancestor of `u` and `v` relative to the tree’s current root.

#### Time Complexity:

- $O(1)$  per call to the constructor, `size()`, and `trees()`.
- $O(\log n)$  amortized per call to all other operations, where  $n$  is the number of nodes.

#### Space Complexity:

- $O(n)$  for storage of the forest, where  $n$  is the number of nodes.
- $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cassert>
#include <unordered_map>
#include <utility>

template<typename T>
class LinkCut {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int len) { return d; }
    static T compose_deltas(const T &old, const T &d) { return d; }
```

```

struct Node {
    int id;
    T value, subtree_value, delta;
    int size;
    bool rev, pending;
    Node *left, *right, *parent;

    Node(int id, const T &value)
        : id(id),
          value(value),
          subtree_value(value),
          size(1),
          rev(false),
          pending(false),
          left(nullptr),
          right(nullptr),
          parent(nullptr) {}

    inline bool is_root() const {
        return parent == nullptr || (parent->left != this && parent->right != this);
    }

    inline T get_subtree_value() const {
        return pending ? apply_delta(subtree_value, delta, size) : subtree_value;
    }

    void push() {
        if (rev) {
            rev = false;
            std::swap(left, right);
            if (left != nullptr) {
                left->rev = !left->rev;
            }
            if (right != nullptr) {
                right->rev = !right->rev;
            }
        }
        if (pending) {
            value = apply_delta(value, delta, 1);
            subtree_value = apply_delta(subtree_value, delta, size);
            if (left != nullptr) {
                left->delta = left->pending ? compose_deltas(left->delta, delta) : delta;
                left->pending = true;
            }
            if (right != nullptr) {
                right->delta = right->pending ? compose_deltas(right->delta, delta) : delta;
                right->pending = true;
            }
            pending = false;
        }
    }

    void update() {
        size = 1;
        subtree_value = value;
        if (left != nullptr) {
            // Combine in in-order (left, value, right) so non-commutative aggregates stay correct.
            subtree_value = combine(left->get_subtree_value(), subtree_value);
            size += left->size;
        }
        if (right != nullptr) {
            subtree_value = combine(subtree_value, right->get_subtree_value());
            size += right->size;
        }
    }
};

```

```

int num_trees;
std::unordered_map<int, Node *> nodes;

static void connect(Node *child, Node *parent, bool is_left) {
    if (child != nullptr) {
        child->parent = parent;
    }
    if (is_left) {
        parent->left = child;
    } else {
        parent->right = child;
    }
}

static void rotate(Node *n) {
    Node *parent = n->parent, *grandparent = parent->parent;
    bool parent_is_root = parent->is_root(), is_left = (n == parent->left);
    connect(is_left ? n->right : n->left, parent, is_left);
    connect(parent, n, !is_left);
    if (parent_is_root) {
        if (n != nullptr) {
            n->parent = grandparent;
        }
    } else {
        connect(n, grandparent, parent == grandparent->left);
    }
    parent->update();
}

static void splay(Node *n) {
    while (!n->is_root()) {
        Node *parent = n->parent, *grandparent = parent->parent;
        if (!parent->is_root()) {
            grandparent->push();
        }
        parent->push();
        n->push();
        if (!parent->is_root()) {
            if ((n == parent->left) == (parent == grandparent->left)) {
                rotate(parent);
            } else {
                rotate(n);
            }
        }
        rotate(n);
    }
    n->push();
    n->update();
}

// Convention: a node's left child leads toward the tree's root, so an exposed path is ordered by
// depth with the root as its leftmost node (matching the standard link/cut tree presentation).
// cut() and find_root() rely on this orientation.
static Node *expose(Node *n) {
    Node *prev = nullptr;
    for (Node *curr = n; curr != nullptr; curr = curr->parent) {
        splay(curr);
        curr->right = prev;
        curr->update();
        prev = curr;
    }
    splay(n);
    return prev;
}

Node *find_node(int u) const {
    auto it = nodes.find(u);
    assert(it != nodes.end());
}

```

```

    return it->second;
}

public:
LinkCut() : num_trees(0) {}

~LinkCut() {
    for (auto &[key, node] : nodes) {
        delete node;
    }
}

LinkCut(const LinkCut &) = delete;
LinkCut &operator=(const LinkCut &) = delete;
int size() const { return static_cast<int>(nodes.size()); }
int trees() const { return num_trees; }

void add_node(int u, const T &value = T{}) {
    assert(nodes.find(u) == nodes.end());
    Node *n = new Node(u, value);
    expose(n);
    n->rev = !n->rev;
    nodes[u] = n;
    num_trees++;
}

bool connected(int u, int v) {
    Node *nu = find_node(u), *nv = find_node(v);
    if (u == v) {
        return true;
    }
    expose(nu);
    expose(nv);
    return nu->parent != nullptr;
}

void link(int u, int v) {
    assert(!connected(u, v));
    Node *nu = find_node(u), *nv = find_node(v);
    expose(nu);
    nu->rev = !nu->rev;
    nu->parent = nv;
    num_trees--;
}

void cut(int u, int v) {
    Node *nu = find_node(u), *nv = find_node(v);
    expose(nu);
    nu->rev = !nu->rev;
    expose(nv);
    assert(nv->left == nu && nu->right == nullptr);
    nv->left->parent = nullptr;
    nv->left = nullptr;
    num_trees++;
}

T query(int u, int v) {
    assert(connected(u, v));
    Node *nu = find_node(u), *nv = find_node(v);
    expose(nu);
    nu->rev = !nu->rev;
    expose(nv);
    return nv->get_subtree_value();
}

void update(int u, int v, const T &d) {
    assert(connected(u, v));
    Node *nu = find_node(u), *nv = find_node(v);

```

```

    expose(nu);
    nu->rev = !nu->rev;
    expose(nv);
    nv->delta = nv->pending ? compose_deltas(nv->delta, d) : d;
    nv->pending = true;
}

void reroot(int u) {
    Node *n = find_node(u);
    expose(n);
    n->rev = !n->rev;
}

int find_root(int u) {
    Node *n = find_node(u);
    expose(n);
    while (n->left != nullptr) { // The leftmost node of the exposed path is the tree's root.
        n = n->left;
        n->push();
    }
    splay(n);
    return n->id;
}

int lca(int u, int v) {
    Node *nu = find_node(u), *nv = find_node(v);
    if (u == v) {
        return u;
    }
    expose(nu);
    expose(nv);
    assert(nu->parent != nullptr);
    splay(nu);
    // nu->parent is now where nu's path rejoins nv's exposed path, i.e. their LCA.
    return nu->parent != nullptr ? nu->parent->id : nu->id;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    // v=10      v=40      v=20      v=10
    // 0-----1-----2-----3
    //           |
    //           +-----4
    //                   v=30
    LinkCut<int> lcf;
    lcf.add_node(0, 10);
    lcf.add_node(1, 40);
    lcf.add_node(2, 20);
    lcf.add_node(3, 10);
    lcf.add_node(4, 30);
    assert(lcf.size() == 5);
    assert(lcf.trees() == 5);
    lcf.link(0, 1);
    lcf.link(1, 2);
    lcf.link(2, 3);
    lcf.link(2, 4);
    assert(lcf.trees() == 1);
    assert(lcf.query(1, 4) == 20);

    // find_root and lca are relative to the current root, set explicitly via reroot.
    lcf.reroot(0);
    assert(lcf.find_root(3) == 0 && lcf.find_root(4) == 0);
}

```



```

assert(lcf.lca(3, 4) == 2); // Paths 3-2 and 4-2 meet at 2 under root 0.
assert(lcf.lca(1, 4) == 1); // 1 is an ancestor of 4 under root 0.
lcf.reroot(4);
assert(lcf.lca(0, 3) == 2); // Rerooting at 4 changes the LCA of 0 and 3.

// v=10    v=100    v=100    v=10
// 0-----1-----2-----3
//           |
//           +-----4
//                   v=100
lcf.update(1, 1, 100);
lcf.update(2, 4, 100);
assert(lcf.query(4, 4) == 100);
assert(lcf.query(0, 4) == 10);
assert(lcf.query(3, 4) == 10);

// v=10    v=100    v=100    v=0
// 0-----1          2-----3
//           |
//           +-----4
//                   v=100
lcf.cut(1, 2);
assert(lcf.trees() == 2);
assert(!lcf.connected(1, 2));
assert(!lcf.connected(0, 4));
assert(lcf.connected(2, 3));
return 0;
}

```

### 2.8.11 Top Tree

Maintain a dynamic forest with values on both nodes and edges, supporting connectivity, linking and cutting edges, rerooting, and path and rooted-subtree aggregate queries. A top tree represents each tree as a hierarchy of clusters, each with at most two boundary nodes. Path clusters summarize contiguous paths, while non-path clusters rake side subtrees into a boundary node. Changing the exposed path requires only a logarithmic number of local rotations and recomputations.

This is an advanced alternative to link/cut trees. Link/cut trees are usually simpler when only path queries are needed, but top trees handle when the dynamic forest also needs rooted-subtree queries.

The aggregate operation is defined by an associative `combine(a, b)`. The default code below computes sums over node and edge values. For minimum queries, use `std::min(a, b)`. For non-commutative aggregates, store enough information in `T` to support reversing a path, and update `flip_path()` accordingly.

- `TopTree(n, value = T{})` constructs a forest on nodes  $[0, n)$  with every node value initialized to `value`.
- `size()` returns the number of nodes in the forest.
- `edges()` returns the number of edge nodes that have been created.
- `connected(u, v)` returns whether nodes `u` and `v` are in the same tree.
- `link(u, v, value = T{})` adds an edge with value `value` between different trees, returning its edge id or `-1` if the edge would create a cycle.
- `cut(e)` removes edge id `e`. The edge id must currently be present.
- `set_node(u, value)` changes node `u`'s value to `value`.
- `set_edge(e, value)` changes edge `e`'s value to `value`. The edge id must currently be present.
- `path_query(u, v)` returns the aggregate over all node and edge values on the path from `u` to `v`. The nodes must be connected.
- `subtree_query(root, u)` reroots the represented tree at `root` and returns the aggregate over the rooted subtree of `u`.
- `reroot(u)` makes node `u` the root of its represented tree.
- `lca(u, v)` returns the lowest common ancestor of `u` and `v` relative to the current root. The nodes must be connected.

- `maybe_lca(u, v)` returns the lowest common ancestor of `u` and `v`, or `-1` if they are disconnected.

#### Time Complexity:

- $O(n)$  per call to the constructor, and  $O(1)$  per call to `size()` and `edges()`.
- $O(\log n)$  amortized per call to all other operations, where  $n$  is the number of nodes and edges in the represented tree.

#### Space Complexity:

- $O(n + m)$  for the internal nodes representing graph nodes and edges, where  $m$  is the number of edges ever added.
- $O(\log n)$  auxiliary stack space for all operations other than `size()` and `edges()`.

```
#include <array>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class TopTree {
    static T combine(const T &a, const T &b) { return a + b; }

    struct Node {
        Node *p;
        std::array<Node *, 3> c; // c[0] and c[1] are auxiliary children; c[2] is the raked path child.
        T value, path_value, subtree_value;
        int graph_node_id;
        bool is_path, is_graph_node, lazy_flip_path, alive;

        Node(const T &value, int graph_node_id)
            : p(nullptr),
              c{nullptr, nullptr, nullptr},
              value(value),
              path_value(value),
              subtree_value(value),
              graph_node_id(graph_node_id),
              is_path(graph_node_id != -1),
              is_graph_node(graph_node_id != -1),
              lazy_flip_path(false),
              alive(graph_node_id != -1) {}

        int dir() const {
            assert(p != nullptr);
            for (int i = 0; i < 3; i++) {
                if (this == p->c[i]) {
                    return i;
                }
            }
            assert(false);
            return -1;
        }

        Node *&parent_child() const { return p->c[dir()]; }
        bool root_of_auxiliary_tree() const { return p == nullptr || p->is_path != is_path; }

        void set_child(int i, Node *child) {
            c[i] = child;
            if (child != nullptr) {
                child->p = this;
            }
        }

        static void replace_in_parent(Node *old, Node *replacement) {
            assert(replacement->p == old);
            if (old->p != nullptr) {
                old->parent_child() = replacement;
            }
        }
    };
};
```

```

    }
    replacement->p = old->p;
    old->p = nullptr;
}

void flip_path() {
    assert(is_path);
    std::swap(c[0], c[1]);
    lazy_flip_path = !lazy_flip_path;
}

void push() {
    if (lazy_flip_path) {
        assert(is_path);
        if (!is_graph_node) {
            c[0]->flip_path();
            c[1]->flip_path();
        }
        lazy_flip_path = false;
    }
}

void push_all() {
    if (p != nullptr) {
        p->push_all();
    }
    push();
}

Node *pull_all() {
    Node *cur = this;
    cur->pull();
    while (cur->p != nullptr) {
        cur = cur->p;
        cur->pull();
    }
    return cur;
}

void prepend_subtree(Node *n) {
    if (n != nullptr) {
        subtree_value = combine(n->subtree_value, subtree_value);
    }
}

void append_subtree(Node *n) {
    if (n != nullptr) {
        subtree_value = combine(subtree_value, n->subtree_value);
    }
}

void pull() {
    subtree_value = value;
    if (is_graph_node) {
        path_value = value;
        append_subtree(c[0]);
        append_subtree(c[1]);
    } else if (is_path) {
        path_value = value;
        if (c[0] != nullptr) {
            path_value = combine(c[0]->path_value, path_value);
            prepend_subtree(c[0]);
        }
        if (c[1] != nullptr) {
            path_value = combine(path_value, c[1]->path_value);
            append_subtree(c[1]);
        }
    } else {

```

```

    prepend_subtree(c[0]);
    append_subtree(c[2]);
    append_subtree(c[1]);
}
}

void rotate() {
    assert(!is_graph_node);
    assert(!root_of_auxiliary_tree());
    Node *parent = p;
    int x = dir();
    assert(x == 0 || x == 1);
    Node *child = c[!x];
    replace_in_parent(parent, this);
    parent->set_child(x, child);
    set_child(!x, parent);
    parent->pull();
}

void rotate2(int child_dir) {
    assert(!is_graph_node);
    assert(!root_of_auxiliary_tree());
    assert(c[child_dir] != nullptr);
    assert(!c[child_dir]->is_graph_node);
    if (dir() == child_dir) {
        rotate();
        return;
    }
    Node *parent = p;
    int x = dir();
    assert(x == 0 || x == 1);
    assert(child_dir == !x);
    Node *middle = c[child_dir];
    Node *child = middle->c[!x];
    replace_in_parent(parent, this);
    parent->set_child(x, child);
    middle->set_child(!x, parent);
    parent->pull();
}

void splay_dir(int x) {
    while (!root_of_auxiliary_tree() && dir() == x) {
        if (!p->root_of_auxiliary_tree() && p->dir() == x) {
            p->rotate();
        }
        rotate();
    }
}

void splay2(int child_dir) {
    assert(!is_graph_node && is_path);
    assert(c[child_dir] != nullptr && !c[child_dir]->is_graph_node);
    while (!root_of_auxiliary_tree()) {
        if (!p->root_of_auxiliary_tree()) {
            if (p->dir() == dir()) {
                p->rotate();
            } else {
                rotate2(child_dir);
            }
        }
        rotate2(child_dir);
    }
}

void splay2() {
    assert(!is_graph_node && is_path);
    assert(!root_of_auxiliary_tree());
    p->splay2(dir());
}

```

```

}

void splay_graph_node() {
    assert(is_graph_node);
    if (root_of_auxiliary_tree()) {
        return;
    }
    p->splay_dir(dir());
    if (p->root_of_auxiliary_tree()) {
        return;
    }
    assert(p->dir() != dir());
    if (dir() == 1) {
        p->rotate();
    }
    assert(dir() == 0);
    p->splay2();
    assert(dir() == 0);
    assert(p->dir() == 1);
    assert(p->p->root_of_auxiliary_tree());
}

void splay() {
    assert(!is_graph_node);
    while (!root_of_auxiliary_tree()) {
        if (!p->root_of_auxiliary_tree()) {
            if (p->dir() == dir()) {
                p->rotate();
            } else {
                rotate();
            }
        }
        rotate();
    }
}

void cut_right() {
    assert(is_graph_node && is_path);
    splay_graph_node();
    if (root_of_auxiliary_tree() || dir() == 1) {
        assert(root_of_auxiliary_tree() || (dir() == 1 && p->root_of_auxiliary_tree()));
        assert(c[0] == nullptr);
        return;
    }
    Node *parent = p;
    assert(
        parent->root_of_auxiliary_tree() ||
        (parent->dir() == 1 && parent->p->root_of_auxiliary_tree())
    );
    assert(!parent->is_graph_node);
    assert(parent->is_path);
    assert(parent->c[0] == this);
    assert(parent->c[2] == nullptr);
    replace_in_parent(parent, this);
    parent->is_path = false;
    parent->set_child(2, parent->c[1]);
    parent->set_child(0, c[0]);
    parent->set_child(1, c[1]);
    set_child(0, nullptr);
    set_child(1, parent);
    assert(c[2] == nullptr);
    assert(c[0] == nullptr);
    parent->pull();
}

Node *splice_non_path() {
    assert(!is_path);
    assert(!is_graph_node);

```

```

    splay();
    assert(p != nullptr && p->is_graph_node && p->is_path);
    p->cut_right();
    if (!p->is_path) {
        rotate();
    }
    assert(p != nullptr && p->is_graph_node && p->is_path);
    assert(p->root_of_auxiliary_tree() || (p->dir() == 1 && p->p->root_of_auxiliary_tree()));
    assert(p->c[dir()] == this && p->c[!dir()] == nullptr);
    Node *parent = p;
    replace_in_parent(parent, this);
    parent->set_child(0, c[0]);
    parent->set_child(1, c[1]);
    assert(c[2] != nullptr && c[2]->is_path);
    set_child(1, c[2]);
    set_child(0, parent);
    set_child(2, nullptr);
    is_path = true;
    parent->pull();
    return parent;
}

Node *splice_all() {
    Node *res = this;
    for (Node *cur = this; cur != nullptr; cur = cur->p) {
        if (!cur->is_path) {
            res = cur->splice_non_path();
        }
        assert(cur->is_path);
    }
    return res;
}

Node *expose() {
    assert(is_graph_node);
    push_all();
    Node *res = splice_all();
    cut_right();
    pull_all();
    return res;
}

void expose_edge() {
    assert(!is_graph_node);
    push_all();
    Node *v = is_path ? c[1] : c[2];
    v->push();
    while (!v->is_graph_node) {
        v = v->c[0];
        v->push();
    }
    v->splice_all();
    v->cut_right();
    v->pull_all();
    assert(p == nullptr);
    assert(v == c[1]);
}

Node *meld_path_end() {
    assert(p == nullptr);
    Node *rt = this;
    while (true) {
        rt->push();
        if (rt->is_graph_node) {
            break;
        }
        rt = rt->c[1];
    }
}

```

```

    assert(rt->is_graph_node);
    rt->splay_graph_node();
    if (rt->c[0] != nullptr && rt->c[1] != nullptr) {
        Node *child = rt->c[1];
        while (true) {
            child->push();
            if (child->c[0] == nullptr) {
                break;
            }
            child = child->c[0];
        }
        child->splay();
        assert(child->c[0] == nullptr);
        child->set_child(0, rt->c[0]);
        rt->set_child(0, nullptr);
        child->pull();
    } else if (rt->c[0] != nullptr) {
        rt->set_child(1, rt->c[0]);
        rt->set_child(0, nullptr);
    }
    assert(rt->c[0] == nullptr);
    return rt->pull_all();
}

void make_root() {
    expose();
    Node *rt = this;
    while (rt->p != nullptr) {
        assert(rt->dir() == 1);
        rt = rt->p;
    }
    rt->flip_path();
    rt->meld_path_end();
    expose();
    assert(p == nullptr);
}

};

std::vector<Node *> graph_nodes, edge_nodes;

static void link_nodes(Node *edge, Node *u, Node *v) {
    assert(edge != nullptr && u != nullptr && v != nullptr);
    assert(edge->c[0] == nullptr && edge->c[1] == nullptr && edge->c[2] == nullptr);
    u->expose();
    while (u->p != nullptr) {
        u = u->p;
    }
    v->make_root();
    assert(u->p == nullptr);
    assert(v->p == nullptr);
    edge->is_path = true;
    edge->is_graph_node = false;
    edge->alive = true;
    edge->set_child(0, u);
    edge->set_child(1, v);
    edge->pull();
}

static void cut_node(Node *edge) {
    assert(edge != nullptr && edge->alive && !edge->is_graph_node);
    edge->expose_edge();
    assert(edge->p == nullptr);
    assert(edge->is_path);
    Node *l = edge->c[0], *r = edge->c[1];
    assert(l != nullptr && r != nullptr);
    edge->c.fill(nullptr);
    l->p = r->p = nullptr;
    l->meld_path_end();
}

```

```

    edge->is_path = false;
    edge->alive = false;
    edge->pull();
}

static Node *get_path(Node *u, Node *v) {
    assert(u->is_graph_node && v->is_graph_node);
    u->make_root();
    v->expose();
    if (u == v) {
        assert(v->p == nullptr);
        return v;
    }
    assert(v->p->p == nullptr);
    return v->p;
}

static Node *get_subtree(Node *root, Node *u) {
    root->make_root();
    u->expose();
    return u;
}

static Node *maybe_lca_node(Node *u, Node *v) {
    u->expose();
    Node *up = u->p;
    assert(up == nullptr || up->p == nullptr);
    Node *res = v->expose();
    assert(v->p == nullptr || v->p->p == nullptr);
    if (u != v && up == u->p && (up == nullptr || up->p == nullptr)) {
        return nullptr;
    }
    return res;
}

public:
explicit TopTree(int n, const T &value = T{}) {
    assert(n >= 0);
    graph_nodes.reserve(n);
    for (int i = 0; i < n; i++) {
        graph_nodes.push_back(new Node(value, i));
    }
}

~TopTree() {
    for (Node *v : graph_nodes) {
        delete v;
    }
    for (Node *e : edge_nodes) {
        delete e;
    }
}

TopTree(const TopTree &) = delete;
TopTree &operator=(const TopTree &) = delete;
int size() const { return static_cast<int>(graph_nodes.size()); }
int edges() const { return static_cast<int>(edge_nodes.size()); }

bool connected(int u, int v) {
    assert(0 <= u && u < size());
    assert(0 <= v && v < size());
    return u == v || maybe_lca_node(graph_nodes[u], graph_nodes[v]) != nullptr;
}

int link(int u, int v, const T &value = T{}) {
    assert(0 <= u && u < size());
    assert(0 <= v && v < size());
    if (connected(u, v)) {

```



```

    return -1;
}
int id = edges();
edge_nodes.push_back(new Node(value, -1));
link_nodes(edge_nodes.back(), graph_nodes[u], graph_nodes[v]);
return id;
}

void cut(int e) {
    assert(0 <= e && e < edges());
    cut_node(edge_nodes[e]);
}

void set_node(int u, const T &value) {
    assert(0 <= u && u < size());
    graph_nodes[u]->expose();
    graph_nodes[u]->value = value;
    graph_nodes[u]->pull_all();
}

void set_edge(int e, const T &value) {
    assert(0 <= e && e < edges());
    assert(edge_nodes[e]->alive);
    edge_nodes[e]->expose_edge();
    edge_nodes[e]->value = value;
    edge_nodes[e]->pull_all();
}

T path_query(int u, int v) {
    assert(connected(u, v));
    return get_path(graph_nodes[u], graph_nodes[v])->path_value;
}

T subtree_query(int root, int u) {
    assert(connected(root, u));
    return get_subtree(graph_nodes[root], graph_nodes[u])->subtree_value;
}

void reroot(int u) {
    assert(0 <= u && u < size());
    graph_nodes[u]->make_root();
}

int lca(int u, int v) {
    assert(connected(u, v));
    graph_nodes[u]->expose();
    return graph_nodes[v]->expose()->graph_node_id;
}

int maybe_lca(int u, int v) {
    assert(0 <= u && u < size());
    assert(0 <= v && v < size());
    Node *res = maybe_lca_node(graph_nodes[u], graph_nodes[v]);
    return res == nullptr ? -1 : res->graph_node_id;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    TopTree<int> t(5);
    for (int i = 0; i < 5; i++) {
        t.set_node(i, i + 1);
    }
}

```

```

int e01 = t.link(0, 1, 10);
int e12 = t.link(1, 2, 20);
int e13 = t.link(1, 3, 30);
int e34 = t.link(3, 4, 40);
assert(e01 == 0 && e12 == 1 && e13 == 2 && e34 == 3);
assert(t.link(0, 4, 100) == -1);

//      0
//  v=10 |
//      1
// v=20 / \ v=30
//      2   3
//          | v=40
//          4
assert(t.path_query(2, 4) == 104);
assert(t.subtree_query(0, 3) == 49);
assert(t.lca(2, 4) == 1);

// Cut edge 1-3:
//
//      0
//  v=10 |
//      1       3
// v=20 /       | v=40
//      2       4
t.cut(e13);
assert(!t.connected(2, 4));
assert(t.maybe_lca(2, 4) == -1);

// Re-link 2-4 with value 30:
//
//      0
//  v=10 |
//      1       3
// v=20 /       | v=40
//      2-----4
//          v=30
e13 = t.link(2, 4, 30);
assert(e13 == 4);
assert(t.path_query(0, 3) == 115);

// Change edge 3-4 to value 4 and node 4 to value 50:
//
//      0
//  v=10 |
//      1       3
// v=20 /       | v=4
//      2-----4
//          v=30   v=50
t.set_edge(e34, 4);
t.set_node(4, 50);
assert(t.subtree_query(2, 4) == 58);
assert(t.path_query(0, 3) == 124);
return 0;
}

```

# Chapter 3

## Strings

### 3.1 String Utilities

---

Common string functions, many of which have standard library counterparts. The implementations below are educational rather than heavily optimized, and often depend on `std::string` operations with unspecified complexity.

**Time Complexity:**  $O(n)$  per call to most operations, where  $n$  is the length of the input string or total length of all input strings. Exceptions are noted with the individual functions below.

**Space Complexity:**

- $O(n)$  auxiliary for operations that return a new string or vector of strings.
- $O(1)$  auxiliary for in-place operations.

```
#include <cctype>
#include <cstdint>
#include <regex>
#include <sstream>
#include <string>
#include <vector>
using std::string;
```

Integer Conversion:

- `to_str(i)` returns the string representation of integer `i`, much like `std::to_string()`.
- `to_int(s)` returns the integer representation of string `s`, much like `std::atoi()`, except here we handle special cases of overflow by throwing an exception.
- `itoa(value, str, base = 10)` implements the non-standard C function which converts `value` into a C string, storing it into pointer `str` in the given `base`. For more generalized base conversion, see the math utilities section.
- `is_integer(s)` returns whether `s` is a valid base-10 integer literal with an optional sign.
- `is_number(s)` returns whether `s` is a decimal number literal with optional sign, decimal point, and exponent.

```
template<typename Int>
string to_str(Int i) {
    std::ostringstream oss;
    oss << i;
    return oss.str();
}

int to_int(const string &s) {
```

```

std::istringstream iss(s);
int res;
if (!(iss >> res) || !(iss >> std::ws).eof()) {
    throw std::runtime_error("to_int failed");
}
return res;
}

char *itoa(int value, char *str, int base = 10) {
    if (base < 2 || base > 36) {
        *str = '\0';
        return str;
    }
    char *ptr = str, *ptr1 = str, tmp_c;
    int tmp_v;
    do {
        tmp_v = value;
        value /= base;
        *ptr++ =
            "zyxwvutsrqponmlkjihgfedcba9876543210123456789"
            "abcdefghijklmnopqrstuvwxyz"[35 + (tmp_v - value * base)];
    } while (value);
    if (tmp_v < 0) {
        *ptr++ = '-';
    }
    for (*ptr-- = '\0'; ptr1 < ptr; *ptr1++ = tmp_c) {
        tmp_c = *ptr;
        *ptr-- = *ptr1;
    }
    return str;
}

bool is_integer(const string &s) {
    // return std::regex_match(s, std::regex(R"([+-]?\d+)"));
    int i = (s.empty() || (s[0] != '-' && s[0] != '+')) ? 0 : 1;
    if (i == static_cast<int>(s.size())) {
        return false;
    }
    for (; i < static_cast<int>(s.size()); i++) {
        if (!isdigit(static_cast<unsigned char>(s[i]))) {
            return false;
        }
    }
    return true;
}

bool is_number(const string &s) {
    // return std::regex_match(s, std::regex(R"([+-]?(\d+(\.\d*)?|\.\d+)([eE][+-]?\d+)?)"));
    int i = (s.empty() || (s[0] != '-' && s[0] != '+')) ? 0 : 1;
    bool seen_digit = false, seen_dot = false;
    for (; i < static_cast<int>(s.size()); i++) {
        if (isdigit(static_cast<unsigned char>(s[i]))) {
            seen_digit = true;
        } else if (s[i] == '.' && !seen_dot) {
            seen_dot = true;
        } else {
            break;
        }
    }
    if (!seen_digit) {
        return false;
    }
    if (i < static_cast<int>(s.size()) && (s[i] == 'e' || s[i] == 'E')) {
        i++;
        if (i < static_cast<int>(s.size()) && (s[i] == '-' || s[i] == '+')) {
            i++;
        }
        bool seen_exp_digit = false;
    }

```

```

for (; i < static_cast<int>(s.size()); i++) {
    if (!isdigit(static_cast<unsigned char>(s[i]))) {
        return false;
    }
    seen_exp_digit = true;
}
return seen_exp_digit;
}
return i == static_cast<int>(s.size());
}

```

Case Conversion:

- `to_upper(s)` returns `s` with all alphabetical characters converted to uppercase.
- `to_lower(s)` returns `s` with all alphabetical characters converted to lowercase.
- `to_title(s)` returns the title case representation of string `s`, where the first letter of every word (consecutive alphabetical characters) is capitalized.

```

string to_upper(const string &s) {
    string res;
    res.reserve(s.size());
    for (unsigned char c : s) {
        res.push_back(static_cast<char>(toupper(c)));
    }
    return res;
}

```

```

string to_lower(const string &s) {
    string res;
    res.reserve(s.size());
    for (unsigned char c : s) {
        res.push_back(static_cast<char>(tolower(c)));
    }
    return res;
}

```

```

string to_title(const string &s) {
    string res;
    unsigned char prev = '\0';
    for (unsigned char c : s) {
        res.push_back(isalpha(prev) ? static_cast<char>(tolower(c)) : static_cast<char>(toupper(c)));
        prev = res.back();
    }
    return res;
}

```

Stripping:

- `lstrip(s, delim = WHITESPACE)` strips the left side of `s` in-place (that is, the input is modified) using the given delimiters and returns a reference to the stripped string.
- `rstrip(s, delim = WHITESPACE)` strips the right side of `s` in-place using the given delimiters and returns a reference to the stripped string.
- `strip(s, delim = WHITESPACE)` strips both sides of `s` in-place and returns a reference to the stripped string.
- `ltrimmed(s, delim = WHITESPACE)`, `rtrimmed(s, delim = WHITESPACE)`, and `trimmed(s, delim = WHITESPACE)` do not modify `s` and return stripped copies.

```

const string WHITESPACE = " \n\t\v\f\r";

string &lstrip(string &s, const string &delim = WHITESPACE) {
    std::size_t pos = s.find_first_not_of(delim);
    if (pos != string::npos) {
        s.erase(0, pos);
    } else {

```

```

    s.clear();
}
return s;
}

string &rstrip(string &s, const string &delim = WHITESPACE) {
    std::size_t pos = s.find_last_not_of(delim);
    if (pos != string::npos) {
        s.erase(pos + 1);
    } else {
        s.clear();
    }
    return s;
}

string &lstrip(string &s, const string &delim = WHITESPACE) {
    return lstrip(rstrip(s, delim), delim);
}

string &ltrimmed(string s, const string &delim = WHITESPACE) {
    return lstrip(s, delim);
}

string &rtrimmed(string s, const string &delim = WHITESPACE) {
    return rstrip(s, delim);
}

string &trimmed(string s, const string &delim = WHITESPACE) {
    return strip(s, delim);
}

```

#### Find and Replace:

- `starts_with(s, prefix)` returns whether `s` begins with `prefix`.
- `ends_with(s, suffix)` returns whether `s` ends with `suffix`.
- `contains(s, needle)` returns whether `needle` occurs in `s`.
- `find_all(haystack, needle)` returns a vector of all positions where the nonempty string `needle` appears in the string `haystack`. Matches may overlap; e.g. `find_all("aaa", "aa")` returns `{0, 1}`. An empty `needle` produces no matches.
- `count(haystack, needle)` returns the number of non-overlapping occurrences of `needle` in `haystack`, like Python's `str.count`. This differs from `find_all().size()`, which counts overlapping matches; e.g. `count("aaa", "aa")` returns 1, but `find_all("aaa", "aa")` finds 2.
- `replace(s, old, replacement)` returns a copy of `s` with all occurrences of the string `old` replaced with the given `replacement`.
- `regex_find_all(s, pattern)` returns every substring of `s` matched by `pattern`.
- `regex_groups(s, pattern)` returns the capture groups of the first match of `pattern` in `s`, excluding the full match. If there is no match, returns an empty vector.
- `regex_replace_all(s, pattern, replacement)` returns a copy of `s` with every regex match replaced by `replacement`, which may refer to capture groups as `$1`, `$2`, and so on.

```

bool starts_with(const string &s, const string &prefix) {
    return s.size() >= prefix.size() && s.compare(0, prefix.size(), prefix) == 0;
}

bool ends_with(const string &s, const string &suffix) {
    return s.size() >= suffix.size() &&
        s.compare(s.size() - suffix.size(), suffix.size(), suffix) == 0;
}

bool contains(const string &s, const string &needle) {
    return s.find(needle) != string::npos;
}

```

```

std::vector<int> find_all(const string &haystack, const string &needle) {
    std::vector<int> res;
    if (needle.empty()) {
        return res;
    }
    std::size_t pos = haystack.find(needle, 0);
    while (pos != string::npos) {
        res.push_back(static_cast<int>(pos));
        pos = haystack.find(needle, pos + 1);
    }
    return res;
}

int count(const string &haystack, const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    int res = 0;
    std::size_t pos = haystack.find(needle, 0);
    while (pos != string::npos) {
        res++;
        pos = haystack.find(needle, pos + needle.size());
    }
    return res;
}

string replace(const string &s, const string &old, const string &replacement) {
    if (old.empty()) {
        return s;
    }
    string res(s);
    std::size_t pos = 0;
    while ((pos = res.find(old, pos)) != string::npos) {
        res.replace(pos, old.length(), replacement);
        pos += replacement.length();
    }
    return res;
}

std::vector<string> regex_find_all(const string &s, const string &pattern) {
    std::regex re(pattern);
    return {std::sregex_token_iterator(s.begin(), s.end(), re, 0), std::sregex_token_iterator()};
}

std::vector<string> regex_groups(const string &s, const string &pattern) {
    std::smatch match;
    if (!std::regex_search(s, match, std::regex(pattern))) {
        return {};
    }
    std::vector<string> res;
    for (int i = 1; i < static_cast<int>(match.size()); i++) {
        res.push_back(match[i].str());
    }
    return res;
}

string regex_replace_all(const string &s, const string &pattern, const string &replacement) {
    return std::regex_replace(s, std::regex(pattern), replacement);
}

```

#### Joining and Splitting:

- `join(v, delim = " ")` returns the strings in vector `v` concatenated, separated by the given delimiter.
- `repeat(s, n)` returns `s` concatenated with itself `n` times, like Python's `s * n`, or an empty string if `n` is nonpositive. For a single repeated character, prefer `std::string(n, ch)`.

- `ljust(s, width, ch = ' ')` returns `s` left-justified (padded on the right with `ch`) to at least `width` characters, like Python's `str.ljust`.
- `rjust(s, width, ch = ' ')` returns `s` right-justified (padded on the left with `ch`) to at least `width` characters, like Python's `str.rjust`.
- `center(s, width, ch = ' ')` returns `s` centered in a field of at least `width` characters by padding both sides with `ch`, like Python's `str.center`. If the padding is uneven, the extra character goes on the right.
- `split(s, char delim)` returns a vector of tokens of `s`, split on a single character delimiter. Empty tokens are skipped, so `split("a::b", ':')` returns `{"a", "b"}`, not `{"a", "", "b"}`.
- `split(s, string delim = WHITESPACE)` returns a vector of tokens of `s`, split on a set of many possible single character delimiters. All characters of `delim` will be removed from `s`, and the remaining token(s) of `s` will be added sequentially to a vector and returned. As with the first version, empty tokens are skipped. For example, `split("a::b", ":", " ")` returns `{"a", "b"}`, not `{"a", "", "b"}`.
- `explode(s, delim)` returns a vector of tokens of `s`, split on the entire delimiter string `delim`. Unlike the `split()` functions above, `delim` is treated as a contiguous boundary string, not merely a set of possible boundary characters. This will not skip empty tokens. For example, `explode("a:::b", ":", " ")` yields `{"a", "", "b"}`, not `{"a", "b"}`. An empty delimiter returns `{s}`.
- `explode(s, char delim)` is the single-character delimiter version that also preserves empty tokens.

```
string join(const std::vector<string> &v, const string &delim = " ") {
    if (v.empty()) {
        return "";
    }
    string res;
    std::size_t total = delim.size() * (v.size() - 1);
    for (const string &s : v) {
        total += s.size();
    }
    res.reserve(total);
    for (int i = 0; i < static_cast<int>(v.size()); i++) {
        res += (i > 0 ? delim : "") + v[i];
    }
    return res;
}

string repeat(const string &s, int n) {
    if (n <= 0) {
        return "";
    }
    string res;
    res.reserve(s.size() * n);
    for (int i = 0; i < n; i++) {
        res += s;
    }
    return res;
}

string ljust(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    return pad > 0 ? s + string(pad, ch) : s;
}

string rjust(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    return pad > 0 ? string(pad, ch) + s : s;
}

string center(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    if (pad <= 0) {
        return s;
    }
    int left = pad / 2;
    return string(left, ch) + s + string(pad - left, ch);
}
```



```

}

std::vector<string> split(const string &s, char delim) {
    std::vector<string> res;
    std::stringstream ss(s);
    string curr;
    while (std::getline(ss, curr, delim)) {
        if (!curr.empty()) {
            res.push_back(curr);
        }
    }
    return res;
}

std::vector<string> split(const string &s, const string &delim = WHITESPACE) {
    std::vector<string> res;
    string curr;
    for (char c : s) {
        if (delim.find(c) == string::npos) {
            curr += c;
        } else if (!curr.empty()) {
            res.push_back(curr);
            curr = "";
        }
    }
    if (!curr.empty()) {
        res.push_back(curr);
    }
    return res;
}

std::vector<string> explode(const string &s, const string &delim) {
    if (delim.empty()) {
        return {s};
    }
    std::vector<string> res;
    std::size_t last = 0, next = 0;
    while ((next = s.find(delim, last)) != string::npos) {
        res.push_back(s.substr(last, next - last));
        last = next + delim.size();
    }
    res.push_back(s.substr(last));
    return res;
}

std::vector<string> explode(const string &s, char delim) {
    std::vector<string> res;
    std::size_t last = 0, next = 0;
    while ((next = s.find(delim, last)) != string::npos) {
        res.push_back(s.substr(last, next - last));
        last = next + 1;
    }
    res.push_back(s.substr(last));
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(to_str(123) + "4" == "1234");
    assert(to_int("1234") == 1234);
    bool threw = false;
    try {

```

```

    to_int("123x");
} catch (const runtime_error &) {
    throw = true;
}
assert(throw);
vector<char> buffer(50);
assert(string(itoa(1750, buffer.data(), 10)) == "1750");
assert(string(itoa(1750, buffer.data(), 16)) == "6d6");
assert(string(itoa(1750, buffer.data(), 2)) == "11011010110");
assert(is_integer("-123") && !is_integer("12.3"));
assert(is_number("-.5e+2") && is_number("123.") && !is_number("1e"));

assert(to_upper("Hello world") == "HELLO WORLD");
assert(to_lower("Hello World") == "hello world");
assert(to_title("hello world") == "Hello World");

string s("  abc \n");
string t = s;
assert(lstrip(s) == "abc \n");
assert(rstrip(s) == strip(t));
assert(trimmed(" \t abc \n") == "abc");
assert(trimmed("xxx", "x").empty());

vector<int> pos;
pos.push_back(0);
pos.push_back(7);
assert(starts_with("abracadabra", "abra"));
assert(ends_with("abracadabra", "dabra"));
assert(contains("abracadabra", "cad"));
assert(find_all("abracadabra", "ab") == pos);
assert(count("abracadabra", "ab") == 2);
assert(count("aaa", "aa") == 1);
assert((find_all("aaa", "aa") == vector<int>{0, 1}));
assert(find_all("abc", "").empty());
assert(replace("abcdabba", "ab", "00") == "00cd00ba");
assert(join(regex_find_all("a12b345", "[0-9]+"), "|") == "12|345");
assert(join(regex_groups("abc-123", "([a-z]+)-([0-9]+)", "|") == "abc|123");
assert(regex_replace_all("a12b345", "[0-9]+", "#") == "a#b#");
assert(regex_replace_all("2026-08-04", R"((\d{4})-(\d{2})-(\d{2}))", "$2/$3/$1") == "08/04/2026");

assert(repeat("ab", 3) == "ababab");
assert(repeat("ab", -1).empty());
assert(rjust("42", 5, '0') == "00042");
assert(ljust("hi", 4, '.') == "hi..");
assert(center("ab", 5, '*') == "*ab*");
assert(join(split("a\nb\ncde\nf", '\n'), "|") == "a|b|cde|f"); // split v1
assert(join(split("a::b", ':'), "|") == "a|b"); // split v1 skips empty tokens
assert(join(split("a::b,cde:,f", ":", ","), "|") == "a|b|cde|f"); // split v2
assert(join(explode("a..b.cde...f", "."), "|") == "a|b.cde|f");
assert((explode("abc", "") == vector<string>{"abc"}));
assert(join(explode("a::b:", ':'), "|") == "a||b|");
return 0;
}

```

## 3.2 Hashing

### 3.2.1 Hash Functions

Provides a small library of non-cryptographic hash functions and hash functors for common contest keys. These functions are useful for fingerprinting values, combining keys, seeding randomized algorithms, and defining `std::unordered_map` keys for pairs, tuples, and vectors. They are not suitable for passwords, signatures, or adversarial security.

The standalone functions below are deterministic.

FNV-1a (Fowler-Noll-Vo) is a tiny hash for variable-length byte sequences. Starting from a fixed offset basis, it XORs each byte into the accumulator and then multiplies by a fixed prime; the `1a` variant XORs before multiplying, which mixes slightly better than the original FNV-1. Reach for it when the key is a string or byte buffer, whereas `fmix32` and `splitmix64` are mixers for a single fixed-width integer. Its appeal is simplicity: a few lines, no lookup tables, and no seed, with distribution good enough for the short keys common in contests. For very long inputs a block hash such as MurmurHash or xxHash is faster and mixes more thoroughly, and because FNV-1a is unseeded it offers no protection against adversarial inputs.

- `fmix32(x)` mixes a 32-bit unsigned integer `x`, using MurmurHash3's finalizer.
- `splitmix64(x)` mixes a 64-bit unsigned integer `x`.
- `hash32(x)` and `hash64(x)` hash any integer type that can be converted to `uint32_t` or `uint64_t`, respectively.
- `hash_combine32(h, x)` and `hash_combine64(h, x)` fold another already-hashed value into an existing hash accumulator.
- `hash_float(x)` and `hash_double(x)` hash floating-point bit patterns, normalizing `-0.0` to `0.0`. Different NaN representations may still hash differently.
- `fnv1a32(s)` and `fnv1a64(s)` compute FNV-1a hashes of string `s`.
- `hash_range32(lo, hi)` and `hash_range64(lo, hi)` hash a sequence of integer-like values in the half-open iterator range `[lo, hi)`.

#### Time Complexity:

- $O(1)$  per call to the integer, combiner, and floating-point hash functions.
- $O(n)$  per call to string and range hashing functions, where  $n$  is the number of elements processed.

Space Complexity:  $O(1)$  auxiliary.

```
#include <chrono>
#include <cstdint>
#include <cstdint>
#include <cstring>
#include <functional>
#include <string>
#include <tuple>
#include <type_traits>
#include <unordered_map>
#include <utility>
#include <vector>

uint32_t fmix32(uint32_t x) {
    x ^= x >> 16;
    x *= 0x85ebca6bU;
    x ^= x >> 13;
    x *= 0xc2b2ae35U;
    return x ^ (x >> 16);
}

uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<typename Int>
uint32_t hash32(Int x) {
    return fmix32(static_cast<uint32_t>(x));
}

template<typename Int>
uint64_t hash64(Int x) {
    return splitmix64(static_cast<uint64_t>(x));
}
```

```

uint32_t hash_combine32(uint32_t h, uint32_t x) {
    return fmix32(h ^ (x + 0x9e3779b9U + (h << 6) + (h >> 2)));
}

uint64_t hash_combine64(uint64_t h, uint64_t x) {
    return splitmix64(h ^ (x + 0x9e3779b97f4a7c15ULL + (h << 6) + (h >> 2)));
}

uint32_t hash_float(float x) {
    if (x == 0.0F) {
        x = 0.0F; // Normalize -0.0.
    }
    uint32_t bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return fmix32(bits);
}

uint64_t hash_double(double x) {
    if (x == 0.0) {
        x = 0.0; // Normalize -0.0.
    }
    uint64_t bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return splitmix64(bits);
}

uint32_t fnv1a32(const std::string &s) {
    uint32_t h = 2166136261U;
    for (unsigned char c : s) {
        h ^= c;
        h *= 16777619U;
    }
    return h;
}

uint64_t fnv1a64(const std::string &s) {
    uint64_t h = 14695981039346656037ULL;
    for (unsigned char c : s) {
        h ^= c;
        h *= 1099511628211ULL;
    }
    return h;
}

template<typename It>
uint32_t hash_range32(It lo, It hi) {
    uint32_t h = 0;
    for (It it = lo; it != hi; ++it) {
        h = hash_combine32(h, hash32(*it));
    }
    return h;
}

template<typename It>
uint64_t hash_range64(It lo, It hi) {
    uint64_t h = 0;
    for (It it = lo; it != hi; ++it) {
        h = hash_combine64(h, hash64(*it));
    }
    return h;
}

```

Hash functors adapt common composite keys for unordered containers. `IntHasher` adds a per-run random seed before mixing, which makes integer-keyed hash tables harder to hack in open-test contests. The composite hashers

separate the mixing primitive (`splitmix64` / `hash_combine64`) from each type's traversal of its parts, so swapping in another mixer only requires editing one place.

- `PairIntHasher` is a self-contained hasher for `std::pair<int, int>` keys, written without any dependency on the templates below so that it can be copy-pasted on its own.
- `IntHasher<Int>`, `PairHasher<A, B>`, `TupleHasher<Ts...>`, and `VectorHasher<T>` are hash functors passed as the third template argument of `std::unordered_map` or `std::unordered_set`.
- `GenericHasher<T>` recursively handles integer, pair, tuple, and vector keys, and falls back to `std::hash<T>` for other hashable types.

**Time Complexity:**  $O(1)$  per scalar or pair hash and  $O(n)$  per vector or tuple hash, where  $n$  is the number of elements processed.

**Space Complexity:**  $O(1)$  auxiliary.

```
template<typename Int>
struct IntHasher {
    std::size_t operator()(Int x) const {
        static const uint64_t RAND_SEED = std::chrono::steady_clock::now().time_since_epoch().count();
        return static_cast<std::size_t>(splitmix64(static_cast<uint64_t>(x) + RAND_SEED));
    }
};

template<typename T>
struct GenericHasher;

// Use the seeded hasher for integers, and std::hash by default for everything else.
template<typename T>
struct ScalarHasher {
    std::size_t operator()(const T &x) const {
        if constexpr (std::is_integral_v<T>) {
            return IntHasher<T>{}(x);
        }
        return std::hash<T>{}(x);
    }
};

// A self-contained hasher for std::pair<int, int> keys: copy just this struct when you do not need
// the rest of this file. The two 32-bit halves pack losslessly into one 64-bit word, which the
// an inlined copy of splitmix64() above then scrambles. Add a random seed to key before
// mixing, as IntHasher does, if you need anti-hacking protection.
struct PairIntHasher {
    std::size_t operator()(const std::pair<int, int> &p) const {
        uint64_t key = (static_cast<uint64_t>(static_cast<uint32_t>(p.first)) << 32) |
            static_cast<uint32_t>(p.second);
        key += 0x9e3779b97f4a7c15ULL;
        key = (key ^ (key >> 30)) * 0xbf58476d1ce4e5b9ULL;
        key = (key ^ (key >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<std::size_t>(key ^ (key >> 31));
    }
};

template<typename A, typename B>
struct PairHasher {
    std::size_t operator()(const std::pair<A, B> &p) const {
        uint64_t h = 0;
        h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<A>{}(p.first)));
        h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<B>{}(p.second)));
        return static_cast<std::size_t>(h);
    }
};

template<typename T>
struct VectorHasher {
    std::size_t operator()(const std::vector<T> &v) const {
        uint64_t h = 0;
```

```

    for (const auto &x : v) {
        h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<T>{}(x)));
    }
    h = hash_combine64(h, v.size()); // Mix in the length so different sizes (e.g. empty) separate.
    return static_cast<std::size_t>(h);
}
};

template<typename... Ts>
struct TupleHasher {
    std::size_t operator()(const std::tuple<Ts...> &t) const {
        uint64_t h = 0;
        std::apply(
            [&](const Ts &...xs) {
                ((h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<Ts>{}(xs)))), ...);
            },
            t
        );
        return static_cast<std::size_t>(h);
    }
};

template<typename T>
struct GenericHasher : ScalarHasher<T> {};

template<typename A, typename B>
struct GenericHasher<std::pair<A, B>> : PairHasher<A, B> {};

template<typename... Ts>
struct GenericHasher<std::tuple<Ts...>> : TupleHasher<Ts...> {};

template<typename T>
struct GenericHasher<std::vector<T>> : VectorHasher<T> {};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(fmix32(123U) == fmix32(123U));
    assert(fmix32(123U) != fmix32(124U));
    assert(hash32(-1) == hash32(-1));
    assert(hash64(123) == splitmix64(123ULL));
    assert(hash_float(-0.0F) == hash_float(0.0F));
    assert(hash_double(-0.0) == hash_double(0.0));
    assert(fnv1a32("abc") == fnv1a32("abc"));
    assert(fnv1a64("abc") != fnv1a64("acb"));

    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    assert(hash_range32(v.begin(), v.end()) == hash_range32(v.begin(), v.end()));
    assert(hash_range64(v.begin(), v.end()) == hash_range64(v.begin(), v.end()));

    unordered_map<int64_t, int, IntHasher<int64_t>> count;
    count[1000000000000LL] = 7;
    assert(count[1000000000000LL] == 7);

    using Point = pair<int, int>;
    unordered_map<Point, int, PairHasher<int, int>> dist;
    dist[Point(3, 4)] = 5;
    assert(dist[Point(3, 4)] == 5);

    // The standalone PairIntHasher is the quick choice for pair<int, int> keys.

```

```

unordered_map<Point, int, PairIntHasher> dist_fast;
dist_fast[Point(3, 4)] = 5;
assert(dist_fast[Point(3, 4)] == 5);

unordered_map<vector<int>, int, VectorHasher<int>> seen;
seen[v] = 1;
assert(seen[v] == 1);

using State = pair<vector<int>, double>;
unordered_map<State, int, GenericHasher<State>> dp;
dp[State(v, 4.5)] = 9;
assert(dp[State(v, 4.5)] == 9);

// Tuple keys via the explicit functor.
using Triple = tuple<int, char, double>;
unordered_map<Triple, int, GenericHasher<Triple>> grid;
grid[Tuple(1, 'a', 3.14)] = 6;
assert(grid[Tuple(1, 'a', 3.14)] == 6);
return 0;
}

```

### 3.2.2 Rolling Hash

Computes polynomial rolling hashes for sequences of an arbitrary hashable type `T`, supporting  $O(1)$  contiguous subsequence hash queries after preprocessing. This is useful for probabilistic string matching, substring equality checks, repeated subarray detection, and binary-searching over candidate lengths.

Given a sequence  $a_0, a_1, \dots, a_{n-1}$  and a value hash  $h(a_i)$  for each element, the hash is the polynomial  $H(a) = \sum_{i=0}^{n-1} h(a_i) B^{n-1-i} \pmod{M}$ , where  $B$  is `HASH_BASE` and  $M$  is `HASH_MOD`. Let  $p(i)$  be the hash of the first  $i$  values. It is built left-to-right by the prefix hash recurrence  $p(0) = 0$  and  $p(i+1) = p(i)B + h(a_i) \pmod{M}$ . The hash of a half-open subsequence  $[l, r)$  is then computed as  $p(r) - p(l)B^{r-l} \pmod{M}$ .

The implementation works modulo the Mersenne prime  $2^{61} - 1$ , using `__uint128_t` for the multiplication where available and falling back to a double-and-add modular multiply otherwise. The base `HASH_BASE` should be changed or chosen randomly for open-hacking environments. With a fixed base, hashing remains fast and practical, but it is still probabilistic and should not be used as proof of equality when exact verification is required.

By default, each sequence value is cast to `uint64_t` and mixed. For non-integer element types, instantiate `RollingHash<T, Hash>` and pass a custom value hasher as the final constructor or `hash()` argument. It must map each element to a stable nonzero value in  $[1, \text{HASH\_MOD})$ .

- `RollingHash<T>()` constructs an empty hash sequence.
- `RollingHash<T>(lo, hi)` constructs prefix hashes from the values in the half-open iterator range  $[lo, hi)$ .
- `RollingHash<T>(v)` constructs prefix hashes for vector `v`.
- `h.hash(lo, hi)` returns the hash of the half-open subsequence  $[lo, hi)$ .
- `RollingHash<T>::hash(lo, hi)` returns the hash of the half-open iterator range  $[lo, hi)$ .
- `RollingHash<T>::hash(v)` returns the hash of vector `v`.
- `concat(left, right, right_len)` returns the hash of the concatenation of a sequence with hash `left` and a sequence with hash `right` and length `right_len`.

#### Time Complexity:

- $O(n)$  per call to the constructor or static `hash()`, where  $n$  is the sequence length.
- $O(1)$  per call to the member `hash()` and `concat()`.

#### Space Complexity:

- $O(n)$  for storage of prefix hashes and powers, where  $n$  is the maximum sequence length processed so far.
- $O(1)$  auxiliary per query.

```
#include <cassert>
```

```

#include <stdint>
#include <string>
#include <utility>
#include <vector>

const uint64_t HASH_MOD = (1ULL << 61) - 1;
const uint64_t HASH_BASE = 911382323ULL; // Change or randomize.

uint64_t hash_add(uint64_t a, uint64_t b) {
    uint64_t c = a + b;
    return c >= HASH_MOD ? c - HASH_MOD : c;
}

uint64_t hash_sub(uint64_t a, uint64_t b) {
    return a >= b ? a - b : a + HASH_MOD - b;
}

uint64_t hash_mul(uint64_t a, uint64_t b) {
#if defined(__SIZEOF_INT128__)
    __uint128_t p = static_cast<__uint128_t>(a) * b;
    return hash_add(static_cast<uint64_t>(p) & HASH_MOD, static_cast<uint64_t>(p >> 61));
#else
    // Portable fallback when no 128-bit type exists: double-and-add modular multiply.
    uint64_t res = 0;
    for (a %= HASH_MOD; b > 0; b >>= 1) {
        if (b & 1) {
            res = hash_add(res, a);
        }
        a = hash_add(a, a);
    }
    return res;
#endif
}

uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<typename T>
struct ValueHasher {
    // +1 ensures the result is in [1, HASH_MOD) and never zero; a zero element in a polynomial
    // hash is invisible and enables trivial collisions.
    uint64_t operator()(const T &x) const {
        return splitmix64(static_cast<uint64_t>(x)) % (HASH_MOD - 1) + 1;
    }
};

template<typename T, typename Hash = ValueHasher<T>>
class RollingHash {
    static std::vector<uint64_t> pow_base;
    std::vector<uint64_t> pref;
    Hash hasher;

    static void ensure_powers(int n) {
        while (static_cast<int>(pow_base.size()) <= n) {
            pow_base.push_back(hash_mul(pow_base.back(), HASH_BASE));
        }
    }

    template<typename It>
    void build(It lo, It hi) {
        pref.clear();
        pref.push_back(0);
        for (; lo != hi; ++lo) {
            pref.push_back(hash_add(hash_mul(pref.back(), HASH_BASE), hasher(*lo)));
        }
    }
};

```



```

    }
    ensure_powers(static_cast<int>(pref.size()) - 1);
}

public:
explicit RollingHash(const Hash &hasher = Hash{}) : hasher(hasher) {
    ensure_powers(0);
    pref.push_back(0);
}

template<typename It>
RollingHash(It lo, It hi, const Hash &hasher = Hash{}) : hasher(hasher) {
    build(lo, hi);
}

explicit RollingHash(const std::vector<T> &v, const Hash &hasher = Hash{})
    : RollingHash(v.begin(), v.end(), hasher) {}

int size() const { return static_cast<int>(pref.size()) - 1; }

uint64_t hash(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi <= size());
    return hash_sub(pref[hi], hash_mul(pref[lo], pow_base[hi - lo]));
}

template<typename It>
static uint64_t hash(It lo, It hi, const Hash &hasher = Hash{}) {
    RollingHash<T, Hash> h(lo, hi, hasher);
    return h.hash(0, h.size());
}

static uint64_t hash(const std::vector<T> &v, const Hash &hasher = Hash{}) {
    RollingHash<T, Hash> h(v, hasher);
    return h.hash(0, h.size());
}

static uint64_t concat(uint64_t left, uint64_t right, int right_len) {
    assert(right_len >= 0);
    ensure_powers(right_len);
    return hash_add(hash_mul(left, pow_base[right_len]), right);
}
};

template<typename T, typename Hash>
std::vector<uint64_t> RollingHash<T, Hash>::pow_base(1, 1);

```

## Example Usage

```

using namespace std;

using PointI = pair<int, int>;

struct PointHasher {
    uint64_t operator()(const PointI &p) const {
        // Pack the two 32-bit coordinates losslessly into one 64-bit key before mixing, so distinct
        // points never collide pre-mix (a small-multiplier combine like a * C + b would).
        uint64_t key = (static_cast<uint64_t>(static_cast<uint32_t>(p.first)) << 32) |
            static_cast<uint32_t>(p.second);
        return splitmix64(key) % (HASH_MOD - 1) + 1;
    }
};

int main() {
    string s = "abracadabra";
    RollingHash<char> hs(s.begin(), s.end());
    assert(hs.hash(0, 4) == hs.hash(7, 11)); // "abra" == "abra"
}

```

```

assert(hs.hash(0, 4) != hs.hash(3, 7)); // "abra" != "acad"

string a = "abc", b = "def";
uint64_t ab = RollingHash<char>::concat(
    RollingHash<char>::hash(a.begin(), a.end()), RollingHash<char>::hash(b.begin(), b.end()),
    static_cast<int>(b.size())
);
string c = a + b;
assert(ab == RollingHash<char>::hash(c.begin(), c.end()));

vector<int> v;
v.push_back(1);
v.push_back(2);
v.push_back(3);
v.push_back(1);
v.push_back(2);
RollingHash<int> hv(v);
assert(hv.hash(0, 2) == hv.hash(3, 5)); // Both ranges are [1, 2].
assert(hv.hash(0, 3) == RollingHash<int>::hash(v.begin(), v.begin() + 3));

vector<PointI> poly{{1, 2}, {3, 4}};
RollingHash<PointI, PointHasher> hp(poly);
assert((hp.hash(0, 2) == RollingHash<PointI, PointHasher>::hash(poly)));
return 0;
}

```

### 3.2.3 Rolling Hash Deque

Maintains a polynomial rolling hash for a sequence of an arbitrary hashable type `T`, supporting insertion and deletion at either end in  $O(1)$  time per operation. Each element is first reduced to a nonnegative integer “digit” by a user-supplied hasher, so characters, integers, whole words, and tuples can all be hashed. This is the deque generalization of the classic Rabin-Karp sliding window: pushing at the back and popping from the front is exactly a moving window, and this structure additionally supports pushing at the front and popping from the back.

The sequence of digits is hashed with the front element as the most significant digit, so a sequence of length  $n$  has hash  $\sum_{i=0}^{n-1} a_i B^{n-1-i}$  (the front element  $a_0$  carries the highest power). This makes the four end operations pure  $O(1)$  updates of a running accumulator:

- `push_back(c)`:  $H \rightarrow H \cdot B + c$  (Horner’s step; the existing weights all shift up).
- `pop_front()`:  $H \rightarrow H - a_0 \cdot B^{n-1}$  (drop the most significant term).
- `push_front(c)`:  $H \rightarrow c \cdot B^n + H$  (the new front becomes the most significant term).
- `pop_back()`:  $H \rightarrow (H - a_{n-1}) \cdot B^{-1}$  (drop the constant term, then renormalize).

Only `pop_back()` needs a modular inverse of the base, which is precomputed once. The weight of the front element,  $B^{n-1}$ , is maintained incrementally (multiplied by  $B$  on a push, by  $B^{-1}$  on a pop), so no power table or length-dependent exponentiation is needed. Each element’s digit is stored when pushed, so popping never re-invokes the hasher. Because the hash depends only on the current contents, two deques with equal sequences hash equally regardless of how they were built, and a sliding window can be compared against a fixed target in  $O(1)$  per step.

`MOD1` and `MOD2` are two distinct primes, each kept below  $2^{31}$  so that the product of any two residues fits in a 64-bit integer; hashing modulo both and comparing the resulting pair makes a collision astronomically unlikely. `BASE1` and `BASE2` are the corresponding polynomial bases, which must be nonzero modulo their corresponding primes and may be randomized per run to resist adversarial inputs. For a small raw alphabet, choosing bases larger than every digit also prevents carry-style collisions before modular reduction, but arbitrary hasher outputs need not satisfy this bound.

The hasher’s output must be nonnegative. As with any polynomial hash, comparing sequences of different lengths is only safe when digits are positive (for example, map an alphabet to  $[1, B)$ ) so that a leading zero cannot make a shorter sequence collide with a longer one.

The element type defaults to `int`, and integer-like values use `IdentityDigit<T>`. To customize the mapping, instantiate `HashDeque<T, Hash>` and pass the hasher to the constructor.

- `HashDeque<T>()` constructs an empty deque. `HashDeque<char>` and `HashDeque<int>` need no custom hasher.
- `size()` returns the number of elements, and `empty()` whether there are none.
- `push_back(x)` and `push_front(x)` append `x` at the back or front.
- `pop_back()` and `pop_front()` remove the back or front element; the deque must be nonempty.
- `hash()` returns the pair of hashes of the current sequence.
- Operators `==` and `!=` compare whether two deques are equal using the lengths and hash pairs. Equal sequences always compare equal; equality indicates equal sequences with high probability.

**Time Complexity:**  $O(1)$  per call to every operation, plus one hasher evaluation per `push_back()` / `push_front()`.

**Space Complexity:**

- $O(n)$  for storage, where  $n$  is the current number of elements.
- $O(1)$  auxiliary per operation.

```
#include <cassert>
#include <stdint>
#include <deque>
#include <utility>

template<typename T>
struct IdentityDigit {
    uint64_t operator()(const T &x) const { return static_cast<uint64_t>(x); }
};

template<typename T = int, typename Hash = IdentityDigit<T>>
class HashDeque {
    static const uint64_t MOD1 = 1000000007, MOD2 = 1000000009; // Distinct primes below 2^31.
    static const uint64_t BASE1 = 131, BASE2 = 137;           // Nonzero modulo MOD1 and MOD2.

    // Products and sums of residues below 2^30 stay under 2^60, so 64-bit arithmetic never overflows.
    static uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
        uint64_t res = 1 % m;
        for (x %= m; n > 0; n >>= 1) {
            if (n & 1) {
                res = res * x % m;
            }
            x = x * x % m;
        }
        return res;
    }

    inline static const uint64_t INV1 = powmod(BASE1, MOD1 - 2, MOD1);
    inline static const uint64_t INV2 = powmod(BASE2, MOD2 - 2, MOD2);

    Hash hasher; // Maps an element to its nonnegative integer digit.
    std::deque<uint64_t> digits; // Per-element digit values, retained so pops never re-hash.
    uint64_t h1 = 0, h2 = 0; // Hash of the current sequence modulo MOD1, MOD2.
    uint64_t top1 = INV1, top2 = INV2; // B^(len-1), or B^-1 while empty.

public:
    explicit HashDeque(Hash hasher = Hash{}) : hasher(hasher) {}

    int size() const { return static_cast<int>(digits.size()); }
    bool empty() const { return digits.empty(); }
    std::pair<uint64_t, uint64_t> hash() const { return {h1, h2}; }

    void push_back(const T &x) {
        uint64_t d = hasher(x), c1 = d % MOD1, c2 = d % MOD2;
        h1 = (h1 * BASE1 + c1) % MOD1;
        h2 = (h2 * BASE2 + c2) % MOD2;
        top1 = top1 * BASE1 % MOD1;
        top2 = top2 * BASE2 % MOD2;
    }
};
```

```

    digits.push_back(d);
}

void push_front(const T &x) {
    uint64_t d = hasher(x), c1 = d % MOD1, c2 = d % MOD2;
    top1 = top1 * BASE1 % MOD1; // Now B^len: the weight the new front element receives.
    top2 = top2 * BASE2 % MOD2;
    h1 = (c1 * top1 + h1) % MOD1;
    h2 = (c2 * top2 + h2) % MOD2;
    digits.push_front(d);
}

void pop_front() {
    assert(!digits.empty());
    uint64_t d = digits.front(), c1 = d % MOD1, c2 = d % MOD2;
    h1 = (h1 + MOD1 - c1 * top1 % MOD1) % MOD1;
    h2 = (h2 + MOD2 - c2 * top2 % MOD2) % MOD2;
    top1 = top1 * INV1 % MOD1;
    top2 = top2 * INV2 % MOD2;
    digits.pop_front();
}

void pop_back() {
    assert(!digits.empty());
    uint64_t d = digits.back(), c1 = d % MOD1, c2 = d % MOD2;
    h1 = (h1 + MOD1 - c1) % MOD1 * INV1 % MOD1;
    h2 = (h2 + MOD2 - c2) % MOD2 * INV2 % MOD2;
    top1 = top1 * INV1 % MOD1;
    top2 = top2 * INV2 % MOD2;
    digits.pop_back();
}

// Compares lengths first, so sequences of different lengths never compare equal.
friend bool operator==(const HashDeque &a, const HashDeque &b) {
    return a.digits.size() == b.digits.size() && a.h1 == b.h1 && a.h2 == b.h2;
}

friend bool operator!=(const HashDeque &a, const HashDeque &b) { return !(a == b); }
};

```

## Example Usage

```

#include <functional>
#include <string>
using namespace std;

int main() {
    // Default hasher: characters hash by their code point.
    HashDeque<char> hd, hrev;
    for (char c : string("abc")) {
        hd.push_back(c);
    }
    for (char c : string("cba")) {
        hrev.push_front(c); // Prepending c, b, a also yields "abc".
    }
    assert(hd.size() == 3 && hd == hrev);

    // Slide a width-3 window across the text like Rabin-Karp: push_back + pop_front.
    string text = "Xabcab";
    auto target = hd.hash(); // Hash of "abc".
    HashDeque<char> window;
    for (int i = 0; i < 3; i++) {
        window.push_back(text[i]); // "Xab".
    }
    bool found = (window.hash() == target);
    for (int i = 3; i < static_cast<int>(text.size()); i++) {

```

```

    window.push_back(text[i]);
    window.pop_front();
    found = found || (window.hash() == target);
}
assert(found); // "abc" occurs in "Xabcab".

// pop_back undoes a push_back, restoring an earlier state's hash.
HashDeque<char> d;
d.push_back('a');
d.push_back('b');
auto ab = d.hash();
d.push_back('c');
d.pop_back();
assert(d.hash() == ab); // Back to "ab".

// An arbitrary element type via a custom hasher: a deque of whole words.
auto whash = [](const string &w) -> uint64_t { return hash<string>{}(w); };
HashDeque<string, decltype(whash)> sentence(whash), reversed(whash);
for (auto w : {"the", "quick", "fox"}) {
    sentence.push_back(w);
}
for (auto w : {"fox", "quick", "the"}) {
    reversed.push_front(w); // Same word sequence, assembled from the front.
}
assert(sentence == reversed);

sentence.pop_back(); // Drop "fox", leaving the words "the quick".
HashDeque<string, decltype(whash)> prefix(whash);
prefix.push_back("the");
prefix.push_back("quick");
assert(sentence == prefix);
return 0;
}

```

### 3.2.4 Dynamic Rolling Hash

Maintains a polynomial rolling hash for a sequence of nonnegative integers, supporting point updates and  $O(\log n)$  contiguous subsequence hash queries. Unlike the static rolling hash, which fixes its prefix hashes at construction, this structure allows any element to change between queries, which is useful for problems that compare substrings while characters are edited.

The key idea is to weight each element by a power of the base tied to its absolute position: storing  $a_i B^i$  at index [1](#) turns the hash of any prefix into an ordinary prefix sum, which a Fenwick tree maintains under point updates. The raw sum over a half-open range  $[l, h)$  is  $\sum_i a_i B^i$ , and multiplying by  $B^{-l}$  shifts it to the position-independent value  $\sum_i a_i B^{i-l}$ , so two ranges with equal contents receive equal hashes (the leftmost element carries the lowest power). A segment tree could replace the Fenwick tree if lazy range updates are needed.

[MOD1](#) and [MOD2](#) are two distinct primes, each kept below  $2^{31}$  so that the product of any two residues fits in a 64-bit integer; hashing modulo both and comparing the resulting pair makes a collision astronomically unlikely. [BASE1](#) and [BASE2](#) are the corresponding polynomial bases, which must be nonzero modulo their corresponding primes and may be randomized per run to resist adversarial inputs. For a small raw alphabet, choosing bases larger than every value also prevents carry-style collisions before modular reduction, but general integer values need not satisfy this bound.

Element values must be nonnegative. As with any polynomial hash, comparing subsequences of different lengths is only safe when values are positive (for example, map an alphabet to  $[1, B)$ ) so that trailing zeros cannot make a shorter sequence collide with a longer one.

- `DynamicRollingHash(a)` builds prefix hashes over the integer sequence `a`.
- `size()` returns the length of the sequence.
- `at(i)` returns the value at index `i`.

- `set(i, x)` assigns the value at index `i` to `x`.
- `hash(lo, hi)` returns the hash pair of the half-open subsequence `[lo, hi)`. Equal ranges always have equal hash pairs, while equal hash pairs indicate equal ranges with high probability.

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the length of the sequence.
- $O(1)$  per call to `size()` and `at()`.
- $O(\log n)$  per call to `set()` and `hash()`.

#### Space Complexity:

- $O(n)$  for storage.
- $O(1)$  auxiliary per query.

```
#include <cassert>
#include <stdint>
#include <utility>
#include <vector>

class DynamicRollingHash {
    static const uint64_t MOD1 = 1000000007, MOD2 = 1000000009; // Distinct primes below 2^31.
    static const uint64_t BASE1 = 131, BASE2 = 137; // Nonzero modulo MOD1 and MOD2.

    int len;
    std::vector<int> cur;
    std::vector<uint64_t> pw1, pw2, ip1, ip2; // Base powers and inverse base powers.
    std::vector<uint64_t> ft1, ft2; // Fenwick trees, 1-indexed of size len + 1.

    // Products and sums of residues below 2^30 stay under 2^60, so 64-bit arithmetic never overflows.
    static uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
        uint64_t res = 1 % m;
        for (x %= m; n > 0; n >>= 1) {
            if (n & 1) {
                res = res * x % m;
            }
            x = x * x % m;
        }
        return res;
    }

    static void add(std::vector<uint64_t> &ft, int i, uint64_t delta, uint64_t m) {
        for (i++; i < static_cast<int>(ft.size()); i += i & -i) {
            ft[i] = (ft[i] + delta) % m;
        }
    }

    static uint64_t sum(const std::vector<uint64_t> &ft, int i, uint64_t m) {
        uint64_t res = 0;
        for (i++; i > 0; i -= i & -i) {
            res = (res + ft[i]) % m;
        }
        return res;
    }

public:
    explicit DynamicRollingHash(const std::vector<int> &a)
        : len(static_cast<int>(a.size())),
          cur(len, 0),
          pw1(len + 1),
          pw2(len + 1),
          ip1(len + 1),
          ip2(len + 1),
          ft1(len + 1),
          ft2(len + 1) {
        pw1[0] = pw2[0] = ip1[0] = ip2[0] = 1;
        uint64_t invb1 = powmod(BASE1, MOD1 - 2, MOD1), invb2 = powmod(BASE2, MOD2 - 2, MOD2);
```

```

    for (int i = 1; i <= len; i++) {
        pw1[i] = pw1[i - 1] * BASE1 % MOD1;
        pw2[i] = pw2[i - 1] * BASE2 % MOD2;
        ip1[i] = ip1[i - 1] * invb1 % MOD1;
        ip2[i] = ip2[i - 1] * invb2 % MOD2;
    }
    for (int i = 0; i < len; i++) {
        set(i, a[i]);
    }
}

int size() const { return len; }

int at(int i) const {
    assert(0 <= i && i < len);
    return cur[i];
}

void set(int i, int x) {
    assert(0 <= i && i < len && x >= 0);
    uint64_t old1 = static_cast<uint64_t>(cur[i]) % MOD1 * pw1[i] % MOD1;
    uint64_t old2 = static_cast<uint64_t>(cur[i]) % MOD2 * pw2[i] % MOD2;
    uint64_t new1 = static_cast<uint64_t>(x) % MOD1 * pw1[i] % MOD1;
    uint64_t new2 = static_cast<uint64_t>(x) % MOD2 * pw2[i] % MOD2;
    add(ft1, i, (new1 + MOD1 - old1) % MOD1, MOD1);
    add(ft2, i, (new2 + MOD2 - old2) % MOD2, MOD2);
    cur[i] = x;
}

std::pair<uint64_t, uint64_t> hash(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi <= len);
    uint64_t raw1 = (sum(ft1, hi - 1, MOD1) + MOD1 - sum(ft1, lo - 1, MOD1)) % MOD1;
    uint64_t raw2 = (sum(ft2, hi - 1, MOD2) + MOD2 - sum(ft2, lo - 1, MOD2)) % MOD2;
    return {raw1 * ip1[lo] % MOD1, raw2 * ip2[lo] % MOD2};
}
};

```

## Example Usage

```

#include <string>
using namespace std;

int main() {
    string s = "abcabc";
    vector<int> a(s.begin(), s.end());
    DynamicRollingHash h(a);
    assert(h.size() == 6);
    assert(h.hash(0, 3) == h.hash(3, 6)); // "abc" == "abc".

    h.set(0, 'x'); // "xabcabc".
    assert(!(h.hash(0, 3) == h.hash(3, 6)));
    h.set(3, 'x'); // "xbcxabc".
    assert(h.hash(0, 3) == h.hash(3, 6)); // "xbc" == "xbc".

    // A single element's hash is invariant under where an equal value sits.
    assert(h.hash(1, 2) == h.hash(4, 5)); // both 'b'.
    assert(h.at(0) == 'x' && h.at(1) == 'b');
    return 0;
}

```

### 3.2.5 Zobrist Hashing

Computes randomized XOR fingerprints for keys and sets of keys using Zobrist hashing. Each distinct key is assigned a pseudorandom 64-bit token, and a set is represented by the XOR of its members' tokens. This makes insertion and deletion the same operation: XOR the key's token once.

This technique is useful for probabilistic equality checks on sets, distinct-prefix queries, board-game states, and other unordered collections. It is not cryptographic, and collisions are possible with probability roughly  $q^2/2^{64}$  over  $q$  compared fingerprints. For multisets, plain XOR only records element parity; use a summed hash or pair it with counts if multiplicity matters.

- `ZobristHash<T>(seed = 0)` constructs a token generator with initial value `seed`.
- `token(x)` returns the stable token assigned to key `x`, creating it if needed.
- `toggle(h, x)` returns the hash obtained by inserting `x` into set hash `h` if absent, or removing `x` from `h` if present.
- `distinct_prefix_hashes(lo, hi)` returns `n + 1` hashes. Element `i` hashes the first `i` values in `[lo, hi)` with each distinct key contributing only on its first occurrence.

#### Time Complexity:

- $O(1)$  expected per call to `token()` and `toggle()`.
- $O(n)$  expected per call to `distinct_prefix_hashes(lo, hi)`, where  $n$  is the distance between `lo` and `hi`.

#### Space Complexity:

- $O(n)$  for stored key tokens, where  $n$  is the number of distinct keys seen so far.
- $O(n)$  auxiliary for `distinct_prefix_hashes()`.

```
#include <stdint>
#include <unordered_map>
#include <unordered_set>
#include <vector>

template<typename T>
class ZobristHash {
    std::unordered_map<T, uint64_t> tokens;
    uint64_t state;

    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

public:
    explicit ZobristHash(uint64_t seed = 0) : state(seed) {}

    uint64_t token(const T &x) {
        if (auto it = tokens.find(x); it != tokens.end()) {
            return it->second;
        }
        state = splitmix64(state);
        tokens[x] = state;
        return state;
    }

    uint64_t toggle(uint64_t h, const T &x) { return h ^ token(x); }

    template<typename It>
    std::vector<uint64_t> distinct_prefix_hashes(It lo, It hi) {
        std::unordered_set<T> seen;
        std::vector<uint64_t> res(1);
        uint64_t h = 0;
        for (It it = lo; it != hi; ++it) {
```



```

        if (seen.insert(*it).second) {
            h ^= token(*it);
        }
        res.push_back(h);
    }
    return res;
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 2, 1, 3};
    vector<int> b{2, 1, 3};
    ZobristHash<int> zh(1234567); // Fixed seed for reproducibility.
    vector<uint64_t> pa = zh.distinct_prefix_hashes(a.begin(), a.end());
    vector<uint64_t> pb = zh.distinct_prefix_hashes(b.begin(), b.end());

    assert(pa[2] == pb[2]); // Distinct sets {1, 2} and {2, 1}.
    assert(pa[4] == pb[3]); // Distinct sets {1, 2, 3} and {2, 1, 3}.

    uint64_t h = 0;
    h = zh.toggle(h, 5);
    assert(h != 0);
    h = zh.toggle(h, 5);
    assert(h == 0);
    return 0;
}

```

## 3.3 Pattern Matching

---

### 3.3.1 String Searching (KMP)

Given a single string (needle) and subsequent queries of texts (haystacks) to be searched, determine the first positions in which the needle occurs within the given haystacks in linear time using the Knuth-Morris-Pratt algorithm. Unlike `std::string::find`, KMP guarantees a linear worst-case bound. KMP precomputes, for every prefix of the needle, the length of its longest proper border (a prefix that is also a suffix). On a mismatch, the needle falls back to that border instead of restarting, so the position in the haystack never moves backward.

- `KMP(needle)` constructs the partial match table for a string `needle` that is to be searched for subsequently in `haystack` queries.
- `find_in(haystack)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. Note that the function can be modified to return all matches by simply letting the loop run and storing the results instead of returning early.

#### Time Complexity:

- $O(m)$  per call to the constructor, where  $m$  is the length of `needle`.
- $O(n)$  per call to `find_in(haystack)`, where  $n$  is the length of `haystack`.

#### Space Complexity:

- $O(m)$  for storage of the partial match table, where  $m$  is the length of `needle`.
- $O(1)$  auxiliary for `find_in()`.

```

#include <cstdint>

```

```

#include <string>
#include <vector>
using std::string;

class KMP {
    string needle;
    std::vector<int> table;

public:
    explicit KMP(const string &needle) : needle(needle), table(needle.size()) {
        for (int i = 1, j = 0; i < static_cast<int>(needle.size()); i++) {
            while (j > 0 && needle[i] != needle[j]) {
                j = table[j - 1];
            }
            if (needle[i] == needle[j]) {
                j++;
            }
            table[i] = j;
        }
    }

    std::size_t find_in(const string &haystack) const {
        int m = static_cast<int>(needle.size());
        if (m == 0) {
            return 0;
        }
        for (int i = 0, j = 0; i < static_cast<int>(haystack.size()); i++) {
            while (j > 0 && needle[j] != haystack[i]) {
                j = table[j - 1];
            }
            if (needle[j] == haystack[i]) {
                j++;
            }
            if (j == m) {
                return i + 1 - m;
            }
        }
        return string::npos;
    }
};

```

### Example Usage

```

#include <cassert>

int main() {
    KMP kmp("ABCDABD");
    assert(kmp.find_in("ABC ABCDAB ABCDABCDABDE") == 15);
    assert(kmp.find_in("ABC ABCDAB ABCDABCDAB") == string::npos);
    assert(KMP("aa").find_in("aaa") == 0); // First overlapping occurrence.
    assert(KMP("").find_in("abc") == 0);
    return 0;
}

```

### 3.3.2 String Searching (Z Algorithm)

Given a single string (needle) and a single text (haystack) to be searched, determine the first position in which the needle occurs within the haystack in linear time using the Z algorithm. In comparison, `std::string::find` does not guarantee a linear worst-case bound.

The Z array stores, for each position of a string, the length of the longest substring starting there that matches a prefix of the whole string. It is computed in linear time by maintaining the rightmost matched window and seeding each position from its mirror earlier in that window.

The `find_substring_z()` function below calls the Z algorithm on the concatenation of `needle` and `haystack`, separated by a sentinel value that is guaranteed not to collide with any byte in either input. Any position whose Z value reaches the needle's length marks an occurrence.

- `z_array(s)` constructs the Z array of sequence `s`. Each `z[i]` is the length of the longest prefix of `s` that also occurs starting at index `i`.
- `find_substring_z(haystack, needle)` returns the first position where `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. To return all matches, let the loop run and store the results instead of returning early.

#### Time Complexity:

- $O(n)$  per call to `z_array(s)`, where  $n$  is the length of `s`.
- $O(n + m)$  per call to `find_substring_z(haystack, needle)`, where  $n$  and  $m$  are the lengths of `haystack` and `needle`.

#### Space Complexity:

- $O(n)$  auxiliary for `z_array(s)`, where  $n$  is the length of `s`.
- $O(n + m)$  auxiliary for `find_substring_z(haystack, needle)`, where  $n$  is the length of `haystack` and  $m$  is the length of `needle`.

```
#include <algorithm>
#include <cstdint>
#include <string>
#include <vector>
using std::string;

template<typename Seq>
std::vector<int> z_array(const Seq &s) {
    std::vector<int> z(s.size());
    for (int i = 1, l = 0, r = 0; i < static_cast<int>(z.size()); i++) {
        if (i <= r) {
            z[i] = std::min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < static_cast<int>(z.size()) && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (r < i + z[i] - 1) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

std::size_t find_substring_z(const string &haystack, const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    std::vector<int> s;
    s.reserve(needle.size() + haystack.size() + 1);
    for (unsigned char c : needle) {
        s.push_back(c + 1);
    }
    s.push_back(0);
    for (unsigned char c : haystack) {
        s.push_back(c + 1);
    }
    auto z = z_array(s);
    int m = static_cast<int>(needle.size());
    for (int i = m + 1; i < static_cast<int>(z.size()); i++) {
```

```

    if (z[i] == m) {
        return i - m - 1;
    }
}
return string::npos;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert((z_array(string("aaaaa")) == vector<int>{0, 4, 3, 2, 1}));
    assert(find_substring_z("ABC ABCDAB ABCDABCDABDE", "ABCDABD") == 15);
    assert(find_substring_z("ABC ABCDAB ABCDABCDAB", "ABCDABD") == string::npos);
    assert(find_substring_z("aaa", "aa") == 0); // First overlapping occurrence.
    assert(find_substring_z("abc", "") == 0);
    return 0;
}

```

### 3.3.3 String Searching (Boyer-Moore)

Given a single string (needle) and subsequent queries of texts (haystacks) to be searched, determine the positions in which the needle occurs within the given haystacks using the Boyer-Moore algorithm. Unlike KMP and the Z algorithm, which compare left to right and inspect every character of the haystack, Boyer-Moore aligns the needle and compares right to left, which lets a single mismatch prove that many alignments are impossible. Most alignments are then skipped entirely, so a long needle over a large alphabet is typically matched in sublinear time.

Two precomputed rules propose a shift, and the algorithm takes the larger of the two.

The bad character rule looks at the haystack character that caused the mismatch. If it occurs in the needle, slide the needle so that its last occurrence lines up with that position; if it does not occur at all, slide past the position entirely. This rule is what makes a large alphabet cheap, since a character absent from the needle skips a full needle length.

The good suffix rule looks at the suffix of the needle that did match before the mismatch. The needle can only be reused where that suffix appears again, so slide to the next occurrence of the matched suffix within the needle, or, when none remains, to the longest prefix of the needle that is also a suffix of the matched part. Both cases come from one border table, computed exactly as in KMP but on the reversed needle.

Dropping the good suffix rule and keying the bad character shift on the window's last character rather than the mismatched one gives the Boyer-Moore-Horspool variant: only the  $O(A)$  table, shorter to write under pressure, and nearly as fast in practice. Library routines such as `std::string::find` and `memmem()` are usually built on one of these.

- `BoyerMoore(needle)` constructs the two shift tables for a nonempty string `needle` that is to be searched for subsequently in `haystack` queries.
- `find_in(haystack)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found.
- `find_all_in(haystack)` returns the positions of every occurrence of `needle` in `haystack`, in increasing order.

Overlapping occurrences are all reported, since after a match the needle only advances by its period.

Speed here is average-case, not worst-case: reporting every occurrence of a periodic needle such as `"aaaa"` in a run of one character rescans the matched part each time, giving  $O(n \cdot m)$  until the Galil rule, which remembers how much of the needle a shift by the period already matched, restores  $O(n + m)$ . Prefer KMP in section 3.3.1 when the guarantee matters more than the average.

**Time Complexity:**

- $O(m + A)$  per call to the constructor, where  $m$  is the length of `needle` and  $A$  is the alphabet size.
- $O(n/m)$  per call to `find_in()` in the best case and  $O(n \cdot m)$  in the worst case, where  $n$  is the length of `haystack`.
- $O(n \cdot m)$  per call to `find_all_in()` in the worst case, plus  $O(z)$  for reporting  $z$  matches.

**Space Complexity:**

- $O(m + A)$  for storage of the shift tables.
- $O(1)$  auxiliary for `find_in()`, and  $O(z)$  for the vector returned by `find_all_in()`.

```

#include <algorithm>
#include <cassert>
#include <climits>
#include <cstdint>
#include <string>
#include <vector>
using std::string;

const int ALPHABET = 1 << CHAR_BIT;

class BoyerMoore {
    string needle;
    std::vector<int> last, good_suffix;

    // Returns the first occurrence at or after start, or -1 if there is none.
    int find_from(const string &haystack, int start) const {
        int n = static_cast<int>(haystack.size()), m = static_cast<int>(needle.size());
        for (int i = start; i + m <= n; i++) {
            int j = m - 1;
            while (j >= 0 && needle[j] == haystack[i + j]) {
                j--;
            }
            if (j < 0) {
                return i;
            }
            i += std::max(good_suffix[j + 1], j - last[static_cast<unsigned char>(haystack[i + j])]);
        }
        return -1;
    }

public:
    explicit BoyerMoore(const string &needle) : needle(needle), last(ALPHABET, -1) {
        assert(!needle.empty());
        int m = static_cast<int>(needle.size());
        for (int i = 0; i < m; i++) {
            last[static_cast<unsigned char>(needle[i])] = i;
        }
        // border[i] is one past the start of the widest border of the suffix beginning at index i.
        good_suffix.assign(m + 1, 0);
        std::vector<int> border(m + 1);
        border[m] = m + 1;
        for (int i = m, j = m + 1; i > 0; i--) {
            while (j <= m && needle[i - 1] != needle[j - 1]) {
                if (good_suffix[j] == 0) { // The first shift proposed for a suffix is the smallest one.
                    good_suffix[j] = j - i;
                }
                j = border[j];
            }
            border[--i] = --j;
        }
        // Suffixes with no other occurrence fall back to the widest prefix that is also a suffix.
        for (int i = 0, j = border[0]; i <= m; i++) {
            if (good_suffix[i] == 0) {
                good_suffix[i] = j;
            }
        }
    }

```

```

        if (i == j) {
            j = border[j];
        }
    }
}

size_t find_in(const string &haystack) const {
    int pos = find_from(haystack, 0);
    return pos < 0 ? string::npos : static_cast<size_t>(pos);
}

std::vector<int> find_all_in(const string &haystack) const {
    std::vector<int> res;
    for (int pos = find_from(haystack, 0); pos >= 0;
         pos = find_from(haystack, pos + good_suffix[0])) {
        res.push_back(pos);
    }
    return res;
}
};

```

### Example Usage

```

using namespace std;

int main() {
    BoyerMoore needle("needle");
    assert(needle.find_in("haystack with a needle in it") == 16);
    assert(needle.find_in("haystack without one") == string::npos);
    assert(needle.find_in("needle") == 0);
    assert(needle.find_in("need") == string::npos); // A haystack shorter than the needle.

    // A character absent from the needle lets the search skip a whole needle length.
    assert(BoyerMoore("abcd").find_in("xxxxxxxxabcd") == 8);

    // Overlapping occurrences are all reported.
    assert((BoyerMoore("aa").find_all_in("aaaa") == vector<int>{0, 1, 2}));
    assert((BoyerMoore("aba").find_all_in("abababa") == vector<int>{0, 2, 4}));
    assert(BoyerMoore("ab").find_all_in("cccc").empty());

    return 0;
}

```

### 3.3.4 String Searching (Aho-Corasick)

Given a set of strings (needles) and subsequent queries of texts (haystacks) to be searched, determine all positions in which needles occur within the given haystacks in linear time using the Aho-Corasick algorithm. The needles are arranged in a trie whose nodes carry failure links to the longest proper suffix that is also a trie node. Scanning the haystack once while following transitions and failure links then reports every match through each node's output list.

This implementation stores transitions in hash tables, which keeps lookups expected constant time without assuming a fixed alphabet size. Output lists store both needles ending at each trie node and needles inherited through its failure link.

- `AhoCorasick(needles)` constructs the finite-state automaton for a set of needle strings that are to be searched for subsequently in haystack queries. Empty needles are ignored.
- `find_all_in(haystack)` returns a vector of `(needle_idx, pos)` pairs for all matches, where `needle_idx` is the 0-based index of a matched needle and `pos` is its starting position in `haystack`. Matches are ordered by increasing ending position in `haystack`.

**Time Complexity:**

- $O(L + z)$  expected per call to the constructor, where  $L$  is the total needle length and  $z$  is the total number of inherited output entries.
- $O(n + z)$  expected per call to `find_all_in(haystack)`, where  $n$  is the length of `haystack` and  $z$  is the number of matches.

**Space Complexity:**

- $O(L + z)$  for storage of the automaton and inherited output entries.
- $O(1)$  auxiliary and  $O(z)$  for the vector returned by `find_all_in()`.

```
#include <queue>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

class AhoCorasick {
    std::vector<string> needles;
    std::vector<int> fail;
    std::vector<std::unordered_map<char, int>> adj;
    std::vector<std::vector<int>> out;

    int next_state(int curr, char c) const {
        int next = curr;
        while (next != 0 && adj[next].find(c) == adj[next].end()) {
            next = fail[next];
        }
        auto it = adj[next].find(c);
        return (it != adj[next].end()) ? it->second : 0;
    }

public:
    explicit AhoCorasick(const std::vector<string> &needles) : needles(needles) {
        int total_len = 0;
        for (const auto &needle : needles) {
            total_len += static_cast<int>(needle.size());
        }
        // The trie has at most total_len + 1 states: the root plus one per character when no prefixes
        // are shared. Sizing to total_len alone overflows for low-sharing needle sets (e.g. a single
        // needle, which still needs the root plus one node).
        int max_states = total_len + 1;
        fail.resize(max_states, -1);
        adj.resize(max_states);
        out.resize(max_states);
        int states = 1;
        for (int i = 0; i < static_cast<int>(needles.size()); i++) {
            if (needles[i].empty()) {
                continue;
            }
            int curr = 0;
            for (char c : needles[i]) {
                if (auto it = adj[curr].find(c); it != adj[curr].end()) {
                    curr = it->second;
                } else {
                    curr = adj[curr][c] = states++;
                }
            }
            out[curr].push_back(i);
        }
        std::queue<int> q;
        for (auto &[c, v] : adj[0]) {
            if (v != 0) {
                fail[v] = 0;
                q.push(v);
            }
        }
    }
};
```

```

    }
}
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (auto &[c, v] : adj[u]) {
        int f = fail[u];
        while (f != 0 && adj[f].find(c) == adj[f].end()) {
            f = fail[f];
        }
        auto fit = adj[f].find(c);
        f = (fit != adj[f].end()) ? fit->second : 0;
        fail[v] = f;
        out[v].insert(out[v].end(), out[f].begin(), out[f].end());
        q.push(v);
    }
}
}

std::vector<std::pair<int, int>> find_all_in(const string &haystack) const {
    std::vector<std::pair<int, int>> matches;
    int state = 0;
    for (int i = 0; i < static_cast<int>(haystack.size()); i++) {
        state = next_state(state, haystack[i]);
        for (int idx : out[state]) {
            matches.emplace_back(idx, i - static_cast<int>(needles[idx].size()) + 1);
        }
    }
    return matches;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<string> needles;
    needles.push_back("a");
    needles.push_back("ab");
    needles.push_back("bab");
    needles.push_back("bc");
    needles.push_back("bca");
    needles.push_back("c");
    needles.push_back("caa");
    needles.push_back("abccab");

    AhoCorasick ac(needles);
    assert(
        (ac.find_all_in("abccab") ==
         vector<pair<int, int>>{{0, 0}, {1, 0}, {3, 1}, {5, 2}, {5, 3}, {0, 4}, {7, 0}, {1, 4}})
    );
    assert(ac.find_all_in("zzzz").empty());
    return 0;
}

```

### 3.3.5 Palindrome Searching (Manacher)

Computes all odd- and even-length palindromic radii of a string in linear time using Manacher's algorithm. These radii can be used to test whether any substring is a palindrome in  $O(1)$ , find the longest palindromic substring, or count all palindromic substrings. The algorithm tracks the rightmost known palindrome and seeds each new



center with the radius of its mirror inside that window, so characters are only ever compared when a palindrome is extended past the window.

For odd palindromes, `odd[i]` is the radius centered at character `s[i]`, including the center character. Thus the palindrome spans `[i - odd[i] + 1, i + odd[i]]`. For even palindromes, `even[i]` is the radius centered between `s[i - 1]` and `s[i]`. Thus the palindrome spans `[i - even[i], i + even[i]]`.

- `Manacher(s)` constructs the palindromic radii arrays for string `s`.
- `is_palindrome(lo, hi)` returns whether substring `[lo, hi)` is a palindrome.
- `longest_palindrome()` returns one longest palindromic substring of `s`.
- `count_palindromes()` returns the total number of palindromic substrings of `s`, counted with multiplicity by position.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the length of `s`.
- $O(1)$  per call to `is_palindrome()`.
- $O(n)$  per call to `longest_palindrome()` and `count_palindromes()`.

#### Space Complexity:

- $O(n)$  for storage of the radii arrays.
- $O(1)$  auxiliary per query.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <string>
#include <vector>
using std::string;

class Manacher {
    string s;
    std::vector<int> odd, even;

public:
    explicit Manacher(const string &s) : s(s), odd(s.size()), even(s.size()) {
        int n = static_cast<int>(s.size());
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 1 : std::min(odd[l + r - i], r - i + 1);
            while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
                k++;
            }
            odd[i] = k;
            if (i + k - 1 > r) {
                l = i - k + 1;
                r = i + k - 1;
            }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 0 : std::min(even[l + r - i + 1], r - i + 1);
            while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
                k++;
            }
            even[i] = k;
            if (i + k - 1 > r) {
                l = i - k;
                r = i + k - 1;
            }
        }
    }

    bool is_palindrome(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi <= static_cast<int>(s.size()));
        int len = hi - lo;
        if (len <= 0) {
```

```

    return true;
}
int mid = lo + (hi - lo) / 2;
return (len % 2) ? odd[mid] >= len / 2 + 1 : even[mid] >= len / 2;
}

string longest_palindrome() const {
    int best_l = 0, best_len = 0;
    int n = static_cast<int>(s.size());
    for (int i = 0; i < n; i++) {
        int len = 2 * odd[i] - 1;
        if (len > best_len) {
            best_len = len;
            best_l = i - odd[i] + 1;
        }
        len = 2 * even[i];
        if (len > best_len) {
            best_len = len;
            best_l = i - even[i];
        }
    }
    return s.substr(best_l, best_len);
}

int64_t count_palindromes() const {
    int64_t res = 0;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        res += odd[i] + even[i];
    }
    return res;
}
};

```

### Example Usage

```

int main() {
    Manacher m("abacaba");
    assert(m.is_palindrome(0, 7));
    assert(m.is_palindrome(1, 6));
    assert(!m.is_palindrome(0, 6));
    assert(m.longest_palindrome() == "abacaba");
    assert(m.count_palindromes() == 12);

    Manacher e("cabbad");
    assert(e.is_palindrome(1, 5));
    assert(e.longest_palindrome() == "abba");
    return 0;
}

```

### 3.3.6 Lyndon Factorization (Duval)

A Lyndon word is a nonempty string that is strictly smaller than all of its proper suffixes, or equivalently, strictly smaller than all of its nontrivial cyclic rotations. By the Chen-Fox-Lyndon theorem, every string has a unique factorization into a sequence of Lyndon words whose values are non-increasing:  $s = w_1 w_2 \cdots w_k$  with  $w_1 \geq w_2 \geq \cdots \geq w_k$ . Duval's algorithm computes this factorization in linear time and constant auxiliary space by scanning the string while tracking the current candidate Lyndon prefix and the position it last matched.

The same scan, applied to the string viewed as a cycle, yields the starting position of its lexicographically least rotation. This is the standard way to canonicalize a necklace, for example to test whether two strings are cyclic rotations of one another.

Both functions below operate on any random-access sequence whose elements support `operator<`.

- `lyndon_factorization(s)` returns a vector of the starting indices of the Lyndon factors of `s`, in order. The factor at index `i` of the result spans `[res[i], res[i + 1])`, or `[res[i], s.size())` for the final factor.
- `min_rotation(s)` returns the 0-based position `p` such that the rotation beginning at `p`, that is, `s[p]`, `s[(p + 1) % n]`, ..., `s[(p + n - 1) % n]`, is lexicographically smallest. If several rotations tie, the smallest such `p` is returned.

**Time Complexity:**  $O(n)$  per call to `lyndon_factorization(s)` and `min_rotation(s)`, where  $n$  is the length of `s`.

**Space Complexity:**

- $O(k)$  auxiliary for `lyndon_factorization()`, where  $k$  is the number of factors.
- $O(1)$  auxiliary for `min_rotation()`.

```
#include <vector>

template<typename Seq>
std::vector<int> lyndon_factorization(const Seq &s) {
    int n = static_cast<int>(s.size());
    std::vector<int> starts;
    int i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && !(s[j] < s[k])) {
            k = (s[k] < s[j]) ? i : k + 1;
            j++;
        }
        while (i <= k) {
            starts.push_back(i);
            i += j - k;
        }
    }
    return starts;
}

template<typename Seq>
int min_rotation(const Seq &s) {
    int n = static_cast<int>(s.size());
    if (n == 0) {
        return 0;
    }
    int i = 0, res = 0;
    while (i < n) {
        res = i;
        int j = i + 1, k = i;
        while (j < 2 * n && !(s[j % n] < s[k % n])) {
            k = (s[k % n] < s[j % n]) ? i : k + 1;
            j++;
        }
        while (i <= k) {
            i += j - k;
        }
    }
    return res;
}
```

### Example Usage

```
#include <cassert>
#include <string>
using namespace std;

int main() {
    // "banana" factors as "b" >= "an" >= "an" >= "a".
    assert((lyndon_factorization(string("banana")) == vector<int>{0, 1, 3, 5}));
}
```

```
// Among the rotations of "baca", "abac" (starting at index 3) is smallest.
assert(min_rotation(string("baca")) == 3);

// Both routines work on any comparable sequence, not just strings.
assert(min_rotation(vector<int>{3, 1, 2, 1, 3}) == 1);
return 0;
}
```

### 3.3.7 De Bruijn Sequence

A de Bruijn sequence  $B(k, n)$  is a cyclic string over the alphabet  $[0, k)$  of length  $k^n$  in which every one of the  $k^n$  possible length- $n$  strings appears exactly once as a (cyclic) substring. Reading a window of  $n$  symbols and advancing one position at a time therefore cycles through all length- $n$  strings without repetition, which makes the sequence a compact way to probe every short pattern: cracking a rotating lock, addressing a shift register, or building the perfect-hash table that maps an isolated set bit to its index.

The construction here is the Fredricksen-Kessler-Maiorana (FKM) algorithm. It concatenates, in lexicographic order, the Lyndon words (see the Lyndon factorization section) whose length divides  $n$ . The result is the lexicographically smallest de Bruijn sequence. The recursion enumerates these necklace representatives directly, appending the first  $p$  symbols of each candidate whenever its period  $p$  divides  $n$ .

- `de_bruijn(k, n)` returns a vector of length  $k^n$  holding one period of  $B(k, n)$ . Each entry lies in  $[0, k)$ . Both  $k$  and  $n$  must be at least 1. Treat the result as cyclic: the length- $n$  windows that wrap past the end are exactly the ones still needed to cover all strings.

**Time Complexity:**  $O(k^n)$  per call, which is optimal since the output itself has length  $k^n$ .

**Space Complexity:**  $O(k^n)$  for the returned sequence, plus  $O(n)$  auxiliary stack space for the recursion.

```
#include <cassert>
#include <vector>

std::vector<int> de_bruijn(int k, int n) {
    assert(k >= 1 && n >= 1);
    std::vector<int> word(n + 1), seq;
    auto dfs = [&](auto &&dfs, int t, int p) {
        if (t > n) {
            if (n % p == 0) {
                for (int i = 1; i <= p; i++) {
                    seq.push_back(word[i]);
                }
            }
            return;
        }
        word[t] = word[t - p];
        dfs(dfs, t + 1, p);
        for (int j = word[t - p] + 1; j < k; j++) {
            word[t] = j;
            dfs(dfs, t + 1, t);
        }
    };
    dfs(dfs, 1, 1);
    return seq;
}
```

#### Example Usage

```
#include <set>
#include <vector>
using namespace std;
```

```

int main() {
    // B(2, 3) has length 8 and every 3-bit string appears once cyclically.
    assert((de_bruijn(2, 3) == vector<int>{0, 0, 0, 1, 0, 1, 1, 1}));

    for (int k = 2; k <= 4; k++) {
        for (int n = 1; n <= 4; n++) {
            vector<int> s = de_bruijn(k, n);
            int len = 1;
            for (int i = 0; i < n; i++) {
                len *= k;
            }
            assert(static_cast<int>(s.size()) == len);

            // Collect every length-n window, wrapping cyclically, and confirm all k^n are distinct.
            set<vector<int>> windows;
            for (int i = 0; i < len; i++) {
                vector<int> w(n);
                for (int j = 0; j < n; j++) {
                    w[j] = s[(i + j) % len];
                }
                windows.insert(w);
            }
            assert(static_cast<int>(windows.size()) == len);
        }
    }
    return 0;
}

```

## 3.4 Suffix Arrays and LCP

### 3.4.1 Suffix Array and LCP (Manber-Myers)

Given a string  $s$ , a suffix array stores the starting positions of the suffixes of  $s$  in sorted order. That is, the  $i$ -th position of the suffix array stores the starting position of the  $i$ -th lexicographically smallest suffix of  $s$ . For example,  $s = \text{"cab"}$  has the suffixes  $\text{"cab"}$ ,  $\text{"ab"}$ , and  $\text{"b"}$ . In sorted order they are  $\text{"ab"}$ ,  $\text{"b"}$ , and  $\text{"cab"}$ , so their starting indices form the suffix array  $[1, 2, 0]$ .

For a string  $s$  of length  $n$ , the longest common prefix (LCP) array of length  $n - 1$  stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in  $s$ . For example,  $\text{"baa"}$  has the sorted suffixes  $\text{"a"}$ ,  $\text{"aa"}$ , and  $\text{"baa"}$ , with an LCP array of  $[1, 0]$ .

The original Manber-Myers doubling algorithm comparison-sorts the suffixes by their first  $2^k$  characters for increasing  $k$ : each round orders suffixes by their pair of ranks from the previous round, so a comparison costs  $O(1)$  and the full order emerges after  $O(\log n)$  rounds.

- `SuffixArrayManberMyers(s)` constructs a suffix array from string `s`.
- `suffix_array()` returns the constructed suffix array.
- `lcp_array()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length  $m$ , this implementation uses an  $O(m \log n)$  binary search, but can be optimized to  $O(m + \log n)$  by first computing the LCP-LR array using the LCP array.

#### Time Complexity:

- $O(n \log^2 n)$  per call to the constructor, where  $n$  is the length of `s`.
- $O(1)$  per call to `suffix_array()`.
- $O(n)$  per call to `lcp_array()`, where  $n$  is the length of `s`.
- $O(m \log n)$  per call to `find(needle)`, where  $m$  is the length of `needle` and  $n$  is the length of `s`.

**Space Complexity:**

- $O(n)$  object storage for the input string and suffix array.
- $O(n)$  auxiliary for the constructor and `lcp_array()`, plus  $O(n)$  for the returned LCP array.
- $O(1)$  auxiliary for `suffix_array()` and `find()`.

```

#include <algorithm>
#include <cstdint>
#include <numeric>
#include <string>
#include <utility>
#include <vector>
using std::string;

class SuffixArrayManberMyers {
    string s;
    std::vector<int> sa;

public:
    explicit SuffixArrayManberMyers(const string &s) : s(s), sa(s.size()) {
        int n = static_cast<int>(s.size());
        std::vector<int> rank(n);
        std::iota(sa.begin(), sa.end(), 0);
        for (int i = 0; i < n; i++) {
            rank[i] = static_cast<unsigned char>(s[i]);
        }
        std::vector<std::pair<int, int>> rank2(n);
        for (int gap = 1; gap < n; gap *= 2) {
            for (int i = 0; i < n; i++) {
                rank2[i] = {rank[i], i + gap < n ? rank[i + gap] + 1 : 0};
            }
            std::sort(sa.begin(), sa.end(), [&](int i, int j) { return rank2[i] < rank2[j]; });
            for (int i = 0; i < n; i++) {
                rank[sa[i]] = (i > 0 && rank2[sa[i - 1]] == rank2[sa[i]]) ? rank[sa[i - 1]] : i;
            }
        }
    }

    const std::vector<int> &suffix_array() const { return sa; }

    std::vector<int> lcp_array() const {
        int n = static_cast<int>(s.size());
        if (n == 0) {
            return {};
        }
        std::vector<int> rank(n), lcp(n - 1);
        for (int i = 0; i < n; i++) {
            rank[sa[i]] = i;
        }
        for (int i = 0, k = 0; i < n; i++) {
            if (rank[i] < n - 1) {
                int j = sa[rank[i] + 1];
                while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                    k++;
                }
                lcp[rank[i]] = k;
                if (k > 0) {
                    k--;
                }
            }
        }
        return lcp;
    }

    std::size_t find(const string &needle) const {
        if (needle.empty()) {
            return 0;
        }
    }

```

```

}
int lo = 0, hi = static_cast<int>(s.size()) - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    int cmp = s.compare(sa[mid], needle.size(), needle);
    if (cmp < 0) {
        lo = mid + 1;
    } else if (cmp > 0) {
        hi = mid - 1;
    } else {
        return sa[mid];
    }
}
return string::npos;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayManberMyers sa("banana");
    assert((sa.suffix_array() == vector<int>{5, 3, 1, 0, 4, 2}));
    assert((sa.lcp_array() == vector<int>{1, 3, 0, 0, 2}));
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

### 3.4.2 Suffix Array and LCP (Counting Sort)

Given a string  $s$ , a suffix array stores the starting positions of the suffixes of  $s$  in sorted order. That is, the  $i$ -th position of the suffix array stores the starting position of the  $i$ -th lexicographically smallest suffix of  $s$ . For example,  $s = \text{"cab"}$  has the suffixes  $\text{"cab"}$ ,  $\text{"ab"}$ , and  $\text{"b"}$ . In sorted order they are  $\text{"ab"}$ ,  $\text{"b"}$ , and  $\text{"cab"}$ , so their starting indices form the suffix array  $[1, 2, 0]$ .

For a string  $s$  of length  $n$ , the longest common prefix (LCP) array of length  $n - 1$  stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in  $s$ . For example,  $\text{"baa"}$  has the sorted suffixes  $\text{"a"}$ ,  $\text{"aa"}$ , and  $\text{"baa"}$ , with an LCP array of  $[1, 0]$ .

The original Manber-Myers doubling algorithm comparison-sorts the suffixes by their first  $2^k$  characters for increasing  $k$ : each round orders suffixes by their pair of ranks from the previous round, so a comparison costs  $O(1)$  and the full order emerges after  $O(\log n)$  rounds. Replacing the comparison sort of each round with counting-sort-style rank updates removes a logarithmic factor.

- `SuffixArrayCountingSort(s)` constructs a suffix array from string `s`.
- `suffix_array()` returns the constructed suffix array.
- `lcp_array()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length  $m$ , this implementation uses an  $O(m \log n)$  binary search, but can be optimized to  $O(m + \log n)$  by first computing the LCP-LR array using the LCP array.

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the length of `s`.
- $O(1)$  per call to `suffix_array()`.
- $O(n)$  per call to `lcp_array()`, where  $n$  is the length of `s`.
- $O(m \log n)$  per call to `find(needle)`, where  $m$  is the length of `needle` and  $n$  is the length of `s`.

**Space Complexity:**

- $O(n)$  object storage for the input string and suffix array.
- $O(n)$  auxiliary for the constructor and `lcp_array()`, plus  $O(n)$  for the returned LCP array.
- $O(1)$  auxiliary for `suffix_array()` and `find()`.

```

#include <algorithm>
#include <cstdint>
#include <numeric>
#include <string>
#include <vector>
using std::string;

class SuffixArrayCountingSort {
    string s;
    std::vector<int> sa;

public:
    explicit SuffixArrayCountingSort(const string &s) : s(s), sa(s.size()) {
        int n = static_cast<int>(s.size());
        std::vector<int> rank(n);
        std::iota(sa.rbegin(), sa.rend(), 0);
        for (int i = 0; i < n; i++) {
            rank[i] = static_cast<unsigned char>(s[i]);
        }
        std::stable_sort(sa.begin(), sa.end(), [&](int i, int j) {
            return static_cast<unsigned char>(s[i]) < static_cast<unsigned char>(s[j]);
        });
        for (int gap = 1; gap < n; gap *= 2) {
            std::vector<int> prev_rank(rank), prev_sa(sa), cnt(n);
            std::iota(cnt.begin(), cnt.end(), 0);
            for (int i = 0; i < n; i++) {
                rank[sa[i]] = (i > 0 && prev_rank[sa[i - 1]] == prev_rank[sa[i]] && sa[i - 1] + gap < n &&
                    prev_rank[sa[i - 1] + gap / 2] == prev_rank[sa[i] + gap / 2])
                    ? rank[sa[i - 1]]
                    : i;
            }
            for (int i = 0; i < n; i++) {
                int s1 = prev_sa[i] - gap;
                if (s1 >= 0) {
                    sa[cnt[rank[s1]]++] = s1;
                }
            }
        }
    }

    const std::vector<int> &suffix_array() const { return sa; }

    std::vector<int> lcp_array() const {
        int n = static_cast<int>(s.size());
        if (n == 0) {
            return {};
        }
        std::vector<int> rank(n), lcp(n - 1);
        for (int i = 0; i < n; i++) {
            rank[sa[i]] = i;
        }
        for (int i = 0, k = 0; i < n; i++) {
            if (rank[i] < n - 1) {
                int j = sa[rank[i] + 1];
                while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                    k++;
                }
                lcp[rank[i]] = k;
                if (k > 0) {
                    k--;
                }
            }
        }
    }
}

```



```

    }
}
return lcp;
}

std::size_t find(const string &needle) const {
    if (needle.empty()) {
        return 0;
    }
    int lo = 0, hi = static_cast<int>(s.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayCountingSort sa("banana");
    assert((sa.suffix_array() == vector<int>{5, 3, 1, 0, 4, 2}));
    assert((sa.lcp_array() == vector<int>{1, 3, 0, 0, 2}));
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);

    SuffixArrayCountingSort repeated("bbba");
    assert((repeated.lcp_array() == vector<int>{0, 1, 2}));
    return 0;
}

```

### 3.4.3 Suffix Array and LCP (DC3)

Given a string  $s$ , a suffix array stores the starting positions of the suffixes of  $s$  in sorted order. That is, the  $i$ -th position of the suffix array stores the starting position of the  $i$ -th lexicographically smallest suffix of  $s$ . For example,  $s = \text{"cab"}$  has the suffixes **"cab"**, **"ab"**, and **"b"**. In sorted order they are **"ab"**, **"b"**, and **"cab"**, so their starting indices form the suffix array  $[1, 2, 0]$ .

For a string  $s$  of length  $n$ , the longest common prefix (LCP) array of length  $n - 1$  stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in  $s$ . For example, **"baa"** has the sorted suffixes **"a"**, **"aa"**, and **"baa"**, with an LCP array of  $[1, 0]$ .

The DC3/skew algorithm recursively constructs the suffix array of the suffixes starting at positions not divisible by 3, uses that order to sort the remaining suffixes with radix sort, and merges the two sorted groups in linear time.

- `SuffixArrayDC3(s)` constructs a suffix array from string `s`.
- `suffix_array()` returns the constructed suffix array.
- `lcp_array()` returns the corresponding LCP array for the suffix array.

- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length  $m$ , this implementation uses an  $O(m \log n)$  binary search, but can be optimized to  $O(m + \log n)$  by first computing the LCP-LR array using the LCP array.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the length of `s`.
- $O(1)$  per call to `suffix_array()`.
- $O(n)$  per call to `lcp_array()`, where  $n$  is the length of `s`.
- $O(m \log n)$  per call to `find(needle)`, where  $m$  is the length of `needle` and  $n$  is the length of `s`.

#### Space Complexity:

- $O(n)$  object storage for the input string and suffix array.
- $O(n)$  auxiliary for the constructor and `lcp_array()`, plus  $O(n)$  for the returned LCP array.
- $O(1)$  auxiliary for `suffix_array()` and `find()`.

```
#include <algorithm>
#include <cstdint>
#include <string>
#include <vector>
using std::string;

class SuffixArrayDC3 {
    static bool leq(int a1, int a2, int b1, int b2) { return (a1 < b1) || (a1 == b1 && a2 <= b2); }

    static bool leq(int a1, int a2, int a3, int b1, int b2, int b3) {
        return (a1 < b1) || (a1 == b1 && leq(a2, a3, b2, b3));
    }
}

template<typename It>
static void radix_pass(It a, It b, It r, int n, int alphabet) {
    std::vector<int> cnt(alphabet + 1);
    for (int i = 0; i < n; i++) {
        cnt[r[a[i]]]++;
    }
    for (int i = 1; i <= alphabet; i++) {
        cnt[i] += cnt[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        b[--cnt[r[a[i]]]] = a[i];
    }
}

template<typename It>
static void suffix_array_dc3(It s, It sa, int n, int alphabet) {
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3, n2 = n / 3, n02 = n0 + n2;
    std::vector<int> s12(n02 + 3), sa12(n02 + 3), s0(n0), sa0(n0);
    s12[n02] = s12[n02 + 1] = s12[n02 + 2] = 0;
    sa12[n02] = sa12[n02 + 1] = sa12[n02 + 2] = 0;
    for (int i = 0, j = 0; i < n + n0 - n1; i++) {
        if (i % 3 != 0) {
            s12[j++] = i;
        }
    }
    radix_pass(s12.begin(), sa12.begin(), s + 2, n02, alphabet);
    radix_pass(sa12.begin(), s12.begin(), s + 1, n02, alphabet);
    radix_pass(s12.begin(), sa12.begin(), s, n02, alphabet);
    int name = 0, c0 = -1, c1 = -1, c2 = -1;
    for (int i = 0; i < n02; i++) {
        if (s[sa12[i]] != c0 || s[sa12[i] + 1] != c1 || s[sa12[i] + 2] != c2) {
            name++;
            c0 = s[sa12[i]];
            c1 = s[sa12[i] + 1];
            c2 = s[sa12[i] + 2];
        }
        (sa12[i] % 3 == 1 ? s12[sa12[i] / 3] : s12[sa12[i] / 3 + n0]) = name;
    }
}
```

```

}
if (name < n02) {
    suffix_array_dc3(s12.begin(), sa12.begin(), n02, name);
    for (int i = 0; i < n02; i++) {
        s12[sa12[i]] = i + 1;
    }
} else {
    for (int i = 0; i < n02; i++) {
        sa12[s12[i] - 1] = i;
    }
}
for (int i = 0, j = 0; i < n02; i++) {
    if (sa12[i] < n0) {
        s0[j++] = 3 * sa12[i];
    }
}
radix_pass(s0.begin(), sa0.begin(), s, n0, alphabet);
for (int p = 0, t = n0 - n1, k = 0; k < n; k++) {
    int i = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2, j = sa0[p];
    if (sa12[t] < n0
        ? leq(s[i], s12[sa12[t] + n0], s[j], s12[j / 3])
        : leq(s[i], s[i + 1], s12[sa12[t] - n0 + 1], s[j], s[j + 1], s12[j / 3 + n0])) {
        sa[k] = i;
        if (++t == n02) {
            for (k++; p < n0; p++, k++) {
                sa[k] = sa0[p];
            }
        }
    } else {
        sa[k] = j;
        if (++p == n0) {
            for (k++; t < n02; t++, k++) {
                sa[k] = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2;
            }
        }
    }
}
}

string s;
std::vector<int> sa;

public:
explicit SuffixArrayDC3(const string &s) : s(s), sa(s.size() + 1) {
    int n = static_cast<int>(s.size());
    std::vector<int> scopy(n);
    for (int i = 0; i < n; i++) {
        scopy[i] = static_cast<unsigned char>(s[i]) + 1;
    }
    scopy.resize(n + 4);
    suffix_array_dc3(scopy.begin(), sa.begin(), n + 1, 256);
    sa.erase(sa.begin());
}

const std::vector<int> &suffix_array() const { return sa; }

std::vector<int> lcp_array() const {
    int n = static_cast<int>(s.size());
    if (n == 0) {
        return {};
    }
    std::vector<int> rank(n), lcp(n - 1);
    for (int i = 0; i < n; i++) {
        rank[sa[i]] = i;
    }
    for (int i = 0, k = 0; i < n; i++) {
        if (rank[i] < n - 1) {
            int j = sa[rank[i] + 1];

```

```

        while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
            k++;
        }
        lcp[rank[i]] = k;
        if (k > 0) {
            k--;
        }
    }
}
return lcp;
}

std::size_t find(const string &needle) const {
    if (needle.empty()) {
        return 0;
    }
    int lo = 0, hi = static_cast<int>(s.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayDC3 sa("banana");
    assert((sa.suffix_array() == vector<int>{5, 3, 1, 0, 4, 2}));
    assert((sa.lcp_array() == vector<int>{1, 3, 0, 0, 2}));
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

## 3.5 String Data Structures

---

### 3.5.1 Trie

Maintain a map of strings to values using a tree data structure. Each node corresponds to a character, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node. Children are stored in hash tables, and `entries()` sorts child labels as needed to preserve lexicographic traversal.

- `Trie<V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.

- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `nullptr` if the key was not found.
- `count_prefix(s)` returns the number of keys currently in the map that have `s` as a prefix (a key equal to `s` counts as well). `count_prefix("")` therefore equals `size()`.
- `entries()` returns all string-value pairs in lexicographically ascending order of string keys.

For a small, fixed alphabet, the `std::unordered_map` children can be replaced by a fixed-size array of child pointers indexed by character. This removes the per-step hashing and lets `entries()` traverse in sorted order without sorting each node's labels, at the cost of restricting the alphabet and spending more memory per node.

#### Time Complexity:

- $O(n)$  expected per call to `insert(s, v)`, `erase(s)`, `find(s)`, and `count_prefix(s)`, where  $n$  is the length of `s`.
- $O(l)$  per call to `entries()`, where  $l$  is the total length of string keys that are currently in the map, treating the character alphabet as constant.
- $O(1)$  per call to all other operations.

#### Space Complexity:

- $O(l)$  for storage of the Trie, where  $l$  is the total length of string keys that are currently in the map.
- $O(l)$  for the vector returned by `entries()`.
- $O(n)$  auxiliary stack space for destruction and `entries()`, where  $n$  is the maximum length of any string that has been inserted so far.
- $O(n)$  auxiliary stack space for `erase(s)`, where  $n$  is the length of `s`.
- $O(1)$  auxiliary for all other operations.

```
#include <algorithm>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

template<typename V>
class Trie {
    struct Node {
        V value;
        bool is_terminal;
        int cnt; // Optional: maintain count of terminal keys in this subtree for count_prefix queries.
        std::unordered_map<char, Node*> children;

        Node() : is_terminal(false), cnt(0) {}
    } *root;

    static bool erase(Node *n, const string &s, int i) {
        if (i == static_cast<int>(s.size())) {
            if (!n->is_terminal) {
                return false;
            }
            n->is_terminal = false;
            n->cnt--;
            return true;
        }
        auto it = n->children.find(s[i]);
        if (it == n->children.end() || !erase(it->second, s, i + 1)) {
            return false;
        }
        n->cnt--;
        // Prune the child only if it is now a non-terminal leaf; a childless terminal is still a key.
        if (it->second->children.empty() && !it->second->is_terminal) {
```

```

        delete it->second;
        n->children.erase(it);
    }
    return true;
}

static void collect_entries(Node *n, string &s, std::vector<std::pair<string, V>> &res) {
    if (n->is_terminal) {
        res.emplace_back(s, n->value);
    }
    std::vector<std::pair<char, Node *>> children(n->children.begin(), n->children.end());
    std::sort(children.begin(), children.end());
    for (auto &[c, child] : children) {
        s += c;
        collect_entries(child, s, res);
        s.pop_back();
    }
}

static void clean_up(Node *n) {
    for (auto &[c, child] : n->children) {
        clean_up(child);
    }
    delete n;
}

int num_terminals;

public:
Trie() : root(new Node()), num_terminals(0) {}

~Trie() { clean_up(root); }
Trie(const Trie &) = delete;
Trie &operator=(const Trie &) = delete;
int size() const { return num_terminals; }
bool empty() const { return num_terminals == 0; }

bool insert(const string &s, const V &v) {
    Node *n = root;
    for (char c : s) {
        auto [it, inserted] = n->children.try_emplace(c, nullptr);
        if (inserted) {
            it->second = new Node();
        }
        n = it->second;
    }
    if (n->is_terminal) {
        return false;
    }
    num_terminals++;
    n->is_terminal = true;
    n->value = v;
    // Re-walk from the root to bump the subtree count along the inserted path.
    n = root;
    n->cnt++;
    for (char c : s) {
        n = n->children[c];
        n->cnt++;
    }
    return true;
}

bool erase(const string &s) {
    if (erase(root, s, 0)) {
        num_terminals--;
        return true;
    }
    return false;
}

```

```

}

const V *find(const string &s) const {
    Node *n = root;
    for (char c : s) {
        if (auto it = n->children.find(c); it != n->children.end()) {
            n = it->second;
        } else {
            return nullptr;
        }
    }
    return n->is_terminal ? &(n->value) : nullptr;
}

int count_prefix(const string &s) const {
    Node *n = root;
    for (char c : s) {
        auto it = n->children.find(c);
        if (it == n->children.end()) {
            return 0;
        }
        n = it->second;
    }
    return n->cnt;
}

std::vector<std::pair<string, V>> entries() const {
    std::vector<std::pair<string, V>> res;
    res.reserve(num_terminals);
    string s;
    collect_entries(root, s, res);
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<string> s{"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
    Trie<int> t;
    assert(t.empty());
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        assert(t.insert(s[i], i));
    }
    assert(
        (t.entries() == vector<pair<string, int>>{
            {"", 0},
            {"a", 1},
            {"i", 6},
            {"in", 7},
            {"inn", 8},
            {"tea", 3},
            {"ted", 4},
            {"ten", 5},
            {"to", 2}
        })
    );
    assert(!t.empty());
    assert(t.size() == 9);
    assert(!t.insert(s[0], 2));
    assert(t.size() == 9);
    assert(*t.find("") == 0);
    assert(*t.find("ten") == 5);
}

```

```

assert(t.count_prefix("") == 9);    // Every key has the empty prefix.
assert(t.count_prefix("te") == 3); // "tea", "ted", "ten".
assert(t.count_prefix("in") == 2); // "in", "inn".
assert(t.count_prefix("z") == 0);
assert(t.erase("tea"));
assert(t.size() == 8);
assert(t.find("tea") == nullptr);
assert(t.count_prefix("te") == 2); // "ted", "ten" remain.
assert(t.erase(""));
assert(t.find("") == nullptr);
assert(t.count_prefix("") == 7);

// Erasing a key must not delete a shorter key sharing its path.
Trie<int> u;
u.insert("bc", 1);
u.insert("bca", 2);
assert(u.erase("bca"));
assert(u.find("bc") != nullptr && *u.find("bc") == 1);
assert(u.size() == 1 && u.count_prefix("bc") == 1);
return 0;
}

```

### 3.5.2 Radix Tree

Maintain a map of strings to values using a compressed tree data structure. Each node corresponds to a substring of an inserted string, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node. Contrary to a regular trie, a radix tree is more space efficient as it combines chains of nodes with only a single child. Children are stored in hash tables, and `entries()` sorts child labels as needed to preserve lexicographic traversal.

- `RadixTree<V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `nullptr` if the key was not found.
- `count_prefix(s)` returns the number of keys currently in the map that have `s` as a prefix (a key equal to `s` counts as well), including the case where `s` ends partway along a compressed edge. `count_prefix("")` therefore equals `size()`.
- `entries()` returns all string-value pairs in lexicographically ascending order of string keys.

#### Time Complexity:

- $O(n)$  expected per call to `insert(s, v)`, `erase(s)`, `find(s)`, and `count_prefix(s)`, where  $n$  is the length of `s`, assuming the number of outgoing compressed edges checked at each node is small.
- $O(l \log l)$  per call to `entries()`, where  $l$  is the total length of string keys that are currently in the map.
- $O(1)$  per call to all other operations.

#### Space Complexity:

- $O(l)$  for storage of the radix tree, where  $l$  is the total length of string keys that are currently in the map.
- $O(l)$  for the vector returned by `entries()`.
- $O(n)$  auxiliary stack space for `insert()`, destruction, and `entries()`, where  $n$  is the maximum length of any string that has been inserted so far.
- $O(n)$  auxiliary stack space for `erase(s)`, where  $n$  is the length of `s`.
- $O(1)$  auxiliary for all other operations.



```

#include <algorithm>
#include <optional>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

template<typename V>
class RadixTree {
    struct Node {
        std::optional<V> value;
        int cnt; // Optional: maintain count of terminal keys in this subtree for count_prefix queries.
        std::unordered_map<string, Node*> children;

        Node() : cnt(0) {}
        explicit Node(const V &value) : value(value), cnt(0) {}
    } *root;

    static int lcp_len(const string &s1, const string &s2, int s2start) {
        int i = 0;
        for (int j = s2start; i < static_cast<int>(s1.size()) && j < static_cast<int>(s2.size());
             i++, j++) {
            if (s1[i] != s2[j]) {
                break;
            }
        }
        return i;
    }

    static bool insert(Node *n, const string &s, int i, const V &v) {
        if (i == static_cast<int>(s.size())) {
            if (n->value) {
                return false;
            }
            n->value.emplace(v);
            n->cnt++;
            return true;
        }
        for (auto it = n->children.begin(); it != n->children.end(); ++it) {
            int len = lcp_len(it->first, s, i);
            if (len == 0) {
                continue;
            }
            if (len == static_cast<int>(it->first.size())) {
                if (!insert(it->second, s, i + len, v)) {
                    return false;
                }
                n->cnt++;
                return true;
            }
            string left = it->first.substr(0, len);
            string right = it->first.substr(len);
            Node *tmp = new Node();
            tmp->cnt = it->second->cnt; // The new split node heads the same subtree as the old child.
            tmp->children[right] = it->second;
            n->children.erase(it);
            n->children[left] = tmp;
            if (len == static_cast<int>(s.size()) - i) {
                tmp->value.emplace(v);
                tmp->cnt++;
            } else {
                insert(tmp, s, i + len, v);
            }
            n->cnt++;
            return true;
        }
        Node *leaf = new Node(v);

```

```

    leaf->cnt = 1;
    n->children[s.substr(i)] = leaf;
    n->cnt++;
    return true;
}

static bool erase(Node *n, const string &s, int i) {
    if (i == static_cast<int>(s.size())) {
        if (!n->value) {
            return false;
        }
        n->value.reset();
        n->cnt--;
        return true;
    }
    for (auto it = n->children.begin(); it != n->children.end(); ++it) {
        int len = lcp_len(it->first, s, i);
        if (len == 0) {
            continue;
        }
        // The entire edge label must be consumed; a partial match means the key is not present, so
        // erasing must fail rather than descend (otherwise erase("te") would remove "tea").
        if (len < static_cast<int>(it->first.size())) {
            return false;
        }
        Node *child = it->second;
        if (!erase(child, s, i + len)) {
            return false;
        }
        n->cnt--;
        // The merge below leaves counts intact: a non-terminal node with one child already has the
        // same subtree count as that child, so reusing the node keeps its cnt correct.
        if (child->children.empty() && !child->value) {
            delete child;
            n->children.erase(it);
        } else if (child->children.size() == 1) {
            Node *grandchild = child->children.begin()->second;
            if (!child->value) {
                string merged_key(it->first + child->children.begin()->first);
                if (grandchild->value) {
                    child->value.emplace(*grandchild->value);
                }
                child->children = grandchild->children;
                delete grandchild;
                n->children.erase(it);
                n->children[merged_key] = child;
            }
        }
        return true;
    }
    return false;
}

static void collect_entries(Node *n, string &s, std::vector<std::pair<string, V>> &res) {
    if (n->value) {
        res.emplace_back(s, *n->value);
    }
    std::vector<std::pair<string, Node *>> children(n->children.begin(), n->children.end());
    std::sort(children.begin(), children.end());
    for (auto &[key, child] : children) {
        s += key;
        collect_entries(child, s, res);
        s.erase(s.size() - key.size());
    }
}

static void clean_up(Node *n) {
    for (auto &[key, child] : n->children) {

```

```

        clean_up(child);
    }
    delete n;
}

int num_terminals;

public:
RadixTree() : root(new Node()), num_terminals(0) {}

~RadixTree() { clean_up(root); }
RadixTree(const RadixTree &) = delete;
RadixTree &operator=(const RadixTree &) = delete;
int size() const { return num_terminals; }
bool empty() const { return num_terminals == 0; }

bool insert(const string &s, const V &v) {
    if (insert(root, s, 0, v)) {
        num_terminals++;
        return true;
    }
    return false;
}

bool erase(const string &s) {
    if (erase(root, s, 0)) {
        num_terminals--;
        return true;
    }
    return false;
}

const V *find(const string &s) const {
    Node *n = root;
    int i = 0;
    while (i < static_cast<int>(s.size())) {
        bool found = false;
        for (auto &[key, child] : n->children) {
            if (key[0] == s[i]) {
                // The entire edge label must be consumed; a partial match (s ends mid-edge or diverges
                // from it) means the key is not present.
                if (lcp_len(key, s, i) < static_cast<int>(key.size())) {
                    return nullptr;
                }
                i += static_cast<int>(key.size());
                n = child;
                found = true;
                break;
            }
        }
        if (!found) {
            return nullptr;
        }
    }
    return n->value ? &*n->value : nullptr;
}

int count_prefix(const string &s) const {
    Node *n = root;
    int i = 0;
    while (i < static_cast<int>(s.size())) {
        bool found = false;
        for (auto &[key, child] : n->children) {
            if (key[0] == s[i]) {
                int len = lcp_len(key, s, i);
                if (i + len == static_cast<int>(s.size())) {
                    // s ends within (or exactly at the end of) this edge: every key below qualifies.
                    return child->cnt;
                }
            }
        }
    }
    return 0;
}

```

```

    }
    if (len < static_cast<int>(key.size())) {
        return 0; // s diverges partway along the edge.
    }
    i += len;
    n = child;
    found = true;
    break;
}
}
if (!found) {
    return 0;
}
}
return n->cnt;
}

std::vector<std::pair<string, V>> entries() const {
    std::vector<std::pair<string, V>> res;
    res.reserve(num_terminals);
    string s;
    collect_entries(root, s, res);
    return res;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<string> s{"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
    RadixTree<int> t;
    assert(t.empty());
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        assert(t.insert(s[i], i));
    }
    assert(
        (t.entries() == vector<pair<string, int>>{
            {"", 0},
            {"a", 1},
            {"i", 6},
            {"in", 7},
            {"inn", 8},
            {"tea", 3},
            {"ted", 4},
            {"ten", 5},
            {"to", 2}
        })
    );
    assert(!t.empty());
    assert(t.size() == 9);
    assert(!t.insert(s[0], 2));
    assert(t.size() == 9);
    assert(t.find("") && *t.find("") == 0);
    assert(*t.find("ten") == 5);
    assert(t.count_prefix("") == 9); // Every key has the empty prefix.
    assert(t.count_prefix("te") == 3); // "tea", "ted", "ten" (prefix ends mid-edge).
    assert(t.count_prefix("in") == 2); // "in", "inn".
    assert(t.count_prefix("z") == 0);
    assert(t.erase("tea"));
    assert(t.size() == 8);
    assert(t.find("tea") == nullptr);
    assert(t.count_prefix("te") == 2); // "ted", "ten" remain.
    assert(t.erase(""));
}

```

```

assert(t.find("") == nullptr);
assert(t.count_prefix("") == 7);

// Erasing a key must not delete a shorter key sharing its path.
RadixTree<int> u;
u.insert("bc", 1);
u.insert("bca", 2);
assert(u.erase("bca"));
assert(u.find("bc") != nullptr && *u.find("bc") == 1);
assert(u.size() == 1 && u.count_prefix("bc") == 1);
return 0;
}

```

### 3.5.3 Suffix Tree (Ukkonen)

A suffix tree is the compressed trie of every suffix of a string: a rooted tree in which each edge is labeled with a substring, no two edges leaving a node start with the same character, and the concatenation of labels from the root to each leaf spells one suffix. Paths from the root therefore spell exactly the distinct substrings of the string, which makes the tree a powerful index for substring queries, repeated-substring problems, and many other tasks. It is the explicit-pointer counterpart of the suffix automaton; the two store the same information and either can be converted into the other.

This implementation is Ukkonen’s algorithm, which builds the tree online in linear time by feeding one character at a time. It maintains an “active point” (a node, an edge, and a length along that edge) describing where the next suffix must be inserted, together with suffix links that let it hop between successive insertion points in amortized constant time. Edges to leaves are stored with an open-ended length that grows automatically as the string lengthens, so each phase touches only the suffixes that genuinely change. A unique terminal sentinel smaller than every real character is appended internally so that no suffix is a prefix of another and every suffix ends at its own leaf. The input must therefore not contain the character `'\1'`.

- `SuffixTree(s)` builds the suffix tree of string `s`.
- `contains(t)` returns whether string `t` occurs as a substring of `s`.
- `count_distinct_substrings()` returns the number of distinct nonempty substrings of `s`.
- `longest_repeated_substring()` returns one longest substring of `s` that occurs at least twice, or the empty string if every character is distinct.

#### Time Complexity:

- $O(n)$  per call to the constructor for a string of length  $n$  (using a per-node ordered map,  $O(n \log s)$  where  $s$  is the alphabet size).
- $O(m \log s)$  per call to `contains(t)`, where  $m$  is the length of `t`.
- $O(n)$  per call to `count_distinct_substrings()` and `longest_repeated_substring()`.

#### Space Complexity:

- $O(n)$  nodes and edges.
- $O(n)$  auxiliary stack space for the traversals.

```

#include <stdint>
#include <map>
#include <string>
#include <vector>

class SuffixTree {
    static const int OPEN = -1; // Marks a leaf edge whose right end follows the growing string.

    struct Node {
        int start, end; // Edge into this node labels str[start..end]; end == OPEN means leaf_end.
        int link;
        std::map<char, int> next;
        Node(int start, int end) : start(start), end(end), link(0) {}
    };
};

```

```

};

std::string str;
std::vector<Node> tree;
int leaf_end;
// Active point and bookkeeping for Ukkonen's algorithm.
int active_node, active_edge, active_length, remaining, last_new;

int edge_end(int v) const { return tree[v].end == OPEN ? leaf_end : tree[v].end; }
int edge_len(int v) const { return edge_end(v) - tree[v].start + 1; }

int make_node(int start, int end) {
    tree.emplace_back(start, end);
    return static_cast<int>(tree.size()) - 1;
}

// If the active length runs past the edge into v, descend and report that we moved.
bool walk_down(int v) {
    if (active_length >= edge_len(v)) {
        active_edge += edge_len(v);
        active_length -= edge_len(v);
        active_node = v;
        return true;
    }
    return false;
}

void extend(int pos) {
    leaf_end = pos;
    remaining++;
    last_new = -1;
    while (remaining > 0) {
        if (active_length == 0) {
            active_edge = pos;
        }
        char c = str[active_edge];
        auto it = tree[active_node].next.find(c);
        if (it == tree[active_node].next.end()) {
            // Rule 2: no edge starts with c, so grow a new leaf from the active node.
            tree[active_node].next[c] = make_node(pos, OPEN);
            if (last_new != -1) {
                tree[last_new].link = active_node;
                last_new = -1;
            }
        } else {
            int v = it->second;
            if (walk_down(v)) {
                continue; // Re-evaluate the same suffix from the deeper node.
            }
            if (str[tree[v].start + active_length] == str[pos]) {
                // Rule 3: the character is already present; stop and lengthen the active point.
                if (last_new != -1) {
                    tree[last_new].link = active_node;
                }
                active_length++;
                break;
            }
            // Rule 2 with a split: break edge v in two and attach a new leaf at the split point.
            int split = make_node(tree[v].start, tree[v].start + active_length - 1);
            tree[active_node].next[c] = split;
            tree[split].next[str[pos]] = make_node(pos, OPEN);
            tree[v].start += active_length;
            tree[split].next[str[tree[v].start]] = v;
            if (last_new != -1) {
                tree[last_new].link = split;
            }
            last_new = split;
        }
    }
}

```

```

    remaining--;
    if (active_node == 0 && active_length > 0) {
        active_length--;
        active_edge = pos - remaining + 1;
    } else if (active_node != 0) {
        active_node = tree[active_node].link;
    }
}
}

// String depth and right end (one past) of the deepest internal node, for repeated substrings.
void deepest_internal(int v, int depth, int &best_depth, int &best_end) const {
    bool internal = false;
    for (const auto &kv : tree[v].next) {
        internal = true;
        int w = kv.second;
        deepest_internal(w, depth + edge_len(w), best_depth, best_end);
    }
    if (internal && v != 0 && depth > best_depth) {
        best_depth = depth;
        best_end = edge_end(v) + 1;
    }
}

public:
explicit SuffixTree(const std::string &s) : str(s) {
    str.push_back('\1'); // Unique terminal sentinel, smaller than every real character.
    make_node(-1, -1); // Root, at index 0; its incoming edge is never read.
    active_node = 0;
    active_edge = 0;
    active_length = 0;
    remaining = 0;
    leaf_end = -1;
    last_new = -1;
    for (int i = 0; i < static_cast<int>(str.size()); i++) {
        extend(i);
    }
}

bool contains(const std::string &t) const {
    int v = 0, i = 0, m = static_cast<int>(t.size());
    while (i < m) {
        auto it = tree[v].next.find(t[i]);
        if (it == tree[v].next.end()) {
            return false;
        }
        v = it->second;
        int len = edge_len(v);
        for (int j = 0; j < len && i < m; j++, i++) {
            if (str[tree[v].start + j] != t[i]) {
                return false;
            }
        }
    }
    return true;
}

int64_t count_distinct_substrings() const {
    int64_t total = 0;
    for (int v = 1; v < static_cast<int>(tree.size()); v++) {
        total += edge_len(v);
    }
    // Every leaf edge ends in the sentinel. The str.size() suffixes of the sentineled string are
    // the only substrings containing it, so removing them leaves the original string's count.
    return total - static_cast<int64_t>(str.size());
}

std::string longest_repeated_substring() const {

```

```

    int best_depth = 0, best_end = 0;
    deepest_internal(0, 0, best_depth, best_end);
    return str.substr(best_end - best_depth, best_depth);
}
};

```

## Example Usage

```

#include <cassert>
#include <set>
using namespace std;

int main() {
    SuffixTree st("banana");
    assert(st.contains("ana"));
    assert(st.contains("banana"));
    assert(st.contains(""));
    assert(!st.contains("anana_"));
    assert(!st.contains("\n"));
    assert(st.longest_repeated_substring() == "ana");

    // "banana" has 15 distinct substrings:
    // a, an, ana, anan, anana, b, ba, ban, bana, banan, banana, n, na, nan, nana.
    assert(st.count_distinct_substrings() == 15);

    // Cross-check both queries against brute force on a small string.
    string text = "mississippi";
    SuffixTree t(text);
    set<string> subs;
    int n = static_cast<int>(text.size());
    for (int i = 0; i < n; i++) {
        for (int len = 1; i + len <= n; len++) {
            subs.insert(text.substr(i, len));
        }
    }
    assert(t.count_distinct_substrings() == static_cast<int>(subs.size()));
    for (const string &sub : subs) {
        assert(t.contains(sub));
    }
    assert(!t.contains("issa"));
    assert(t.longest_repeated_substring() == "issi"); // "issi" appears at indices 1 and 4.
    return 0;
}

```

### 3.5.4 Suffix Automaton

Maintains a compact deterministic automaton representing all substrings of a string. A suffix automaton is useful for substring membership queries, counting distinct substrings, finding longest common substrings, and many other string dynamic programming tasks.

Each state represents an equivalence class of substrings with the same set of ending positions. The value `st[v].len` is the maximum length represented by state `v`, while `st[v].link` points to the state representing the longest proper suffix of that class.

More precisely, state `v` contributes one distinct substring for each length greater than `st[st[v].link].len` and at most `st[v].len`. When a character is appended, `append()` creates a state for the new full string and walks suffix links from the previous `last`, adding missing transitions to it. If an existing transition leads to a state whose maximum length is too large, a shorter clone of that state is inserted and the affected transitions and suffix links are redirected to it, splitting the ending-position classes without changing the recognized substrings. Summing these length ranges gives the number of distinct substrings. Membership queries follow transitions, while longest common substring queries retreat through suffix links when a match fails.



- `SuffixAutomaton()` constructs an empty automaton.
- `SuffixAutomaton(s)` constructs the automaton for string `s`.
- `append(c)` appends character `c` to the current string.
- `contains(t)` returns whether string `t` occurs as a substring.
- `first_occurrence(t)` returns the starting index of the first occurrence of `t`, or `-1` if absent.
- `count_distinct_substrings()` returns the number of distinct nonempty substrings.
- `longest_common_substring(t)` returns one longest substring common to the built string and string `t`.

#### Time Complexity:

- $O(n)$  expected per call to the constructor for a string of length  $n$ .
- $O(1)$  expected amortized per call to `append()`.
- $O(m)$  expected per call to `contains(t)`, `first_occurrence(t)`, or `longest_common_substring(t)`, where  $m$  is the length of `t`.
- $O(n)$  per call to `count_distinct_substrings()`.

#### Space Complexity:

- $O(n)$  transition storage and  $O(n)$  states.
- $O(1)$  auxiliary per character processed.

```
#include <cstdlib>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

class SuffixAutomaton {
    struct State {
        int len, link, first_pos;
        std::unordered_map<char, int> next;

        State() : len(0), link(-1), first_pos(-1) {}
    };

    std::vector<State> st;
    int last;

public:
    SuffixAutomaton() : st(1), last(0) {}

    explicit SuffixAutomaton(const string &s) : SuffixAutomaton() {
        for (char c : s) {
            append(c);
        }
    }

    void append(char c) {
        int cur = static_cast<int>(st.size());
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].first_pos = st[cur].len - 1;
        int p = last;
        while (p != -1 && st[p].next.find(c) == st[p].next.end()) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = static_cast<int>(st.size());
                st.push_back(st[q]);
```

```

        st[clone].len = st[p].len + 1;
        while (p != -1 && st[p].next[c] == q) {
            st[p].next[c] = clone;
            p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}

bool contains(const string &t) const {
    int v = 0;
    for (char c : t) {
        if (auto it = st[v].next.find(c); it != st[v].next.end()) {
            v = it->second;
        } else {
            return false;
        }
    }
    return true;
}

// Returns the starting index of the first (leftmost) occurrence of t as a substring, or -1 if t
// does not occur. first_pos stores the end index of the first occurrence of each state's class,
// so subtracting the length of t recovers its start.
int first_occurrence(const string &t) const {
    int v = 0;
    for (char c : t) {
        auto it = st[v].next.find(c);
        if (it == st[v].next.end()) {
            return -1;
        }
        v = it->second;
    }
    return st[v].first_pos - static_cast<int>(t.size()) + 1;
}

int64_t count_distinct_substrings() const {
    int64_t res = 0;
    for (int v = 1; v < static_cast<int>(st.size()); v++) {
        res += st[v].len - st[st[v].link].len;
    }
    return res;
}

string longest_common_substring(const string &t) const {
    int v = 0, len = 0, best = 0, best_pos = -1;
    for (int i = 0; i < static_cast<int>(t.size()); i++) {
        while (v && !st[v].next.count(t[i])) {
            v = st[v].link;
            len = st[v].len;
        }
        if (auto it = st[v].next.find(t[i]); it != st[v].next.end()) {
            v = it->second;
            len++;
        } else {
            v = 0;
            len = 0;
        }
        if (len > best) {
            best = len;
            best_pos = i;
        }
    }
    return best == 0 ? "" : t.substr(best_pos - best + 1, best);
}
};

```

## Example Usage

```
#include <cassert>

int main() {
    SuffixAutomaton sa("ababa");
    assert(sa.contains("aba"));
    assert(sa.contains("bab"));
    assert(!sa.contains("abba"));
    // "ababa" has 9 distinct nonempty substrings.
    assert(sa.count_distinct_substrings() == 9);
    // The longest substring shared by "ababa" and "zzbabab" is "baba".
    assert(sa.longest_common_substring("zzbabab") == "baba");
    assert(sa.first_occurrence("aba") == 0);
    assert(sa.first_occurrence("bab") == 1);
    assert(sa.first_occurrence("abba") == -1);

    SuffixAutomaton online;
    online.append('a');
    online.append('b');
    online.append('c');
    assert(online.contains("bc"));
    return 0;
}
```

### 3.5.5 Eertree

Maintains all distinct palindromic substrings of a string online using an Eertree, also known as a palindromic tree. Each non-root node represents one distinct palindrome, and suffix links connect each palindrome to its longest proper palindromic suffix.

The structure has two roots: one of length  $-1$ , which simplifies boundary cases, and one of length  $0$ , representing the empty palindrome. After each appended character, `last` points to the node for the longest palindromic suffix of the current string.

To append a character, `append()` follows suffix links from `last` until finding the longest suffix palindrome that can be enclosed by the new character. If the corresponding transition already exists, that node becomes `last`; otherwise, a node two characters longer is created and linked to its longest proper palindromic suffix, found by the same search. Exactly one distinct palindrome can be created at each position, so the number of non-root nodes is the distinct count. Each node first counts the times it is the longest palindromic suffix; propagating those counts through suffix links then includes occurrences where it appears as a suffix of a longer palindrome.

- `Eertree()` constructs an empty palindromic tree.
- `Eertree(s)` constructs the tree for string `s`.
- `append(c)` appends character `c` and returns `true` if this creates a new distinct palindrome.
- `count_distinct_palindromes()` returns the number of distinct nonempty palindromic substrings.
- `longest_suffix_length()` returns the length of the current longest palindromic suffix.
- `count_occurrences()` propagates occurrence counts through suffix links and returns `occ[v]` for each node `v`.

#### Time Complexity:

- $O(n)$  expected per call to the constructor for a string of length  $n$ .
- $O(1)$  expected amortized per call to `append()`.
- $O(n)$  per call to `count_occurrences()`.

#### Space Complexity:

- $O(n)$  transition storage and  $O(n)$  nodes.
- $O(1)$  auxiliary per appended character.

```
#include <string>
```

```

#include <unordered_map>
#include <vector>
using std::string;

class Eertree {
    struct Node {
        int len, link, occ;
        std::unordered_map<char, int> next;

        Node(int len = 0) : len(len), link(0), occ(0) {}
    };

    std::vector<Node> tree;
    string str;
    int last;

    int get_suffix(int v, int pos, char c) const {
        while (pos - 1 - tree[v].len < 0 || str[pos - 1 - tree[v].len] != c) {
            v = tree[v].link;
        }
        return v;
    }

public:
    Eertree() : last(1) {
        tree.emplace_back(-1);
        tree.emplace_back(0);
        tree[0].link = 0;
        tree[1].link = 0;
    }

    explicit Eertree(const string &s) : Eertree() {
        for (char c : s) {
            append(c);
        }
    }

    bool append(char c) {
        str += c;
        int pos = static_cast<int>(str.size()) - 1;
        int cur = get_suffix(last, pos, c);
        auto it = tree[cur].next.find(c);
        if (it != tree[cur].next.end()) {
            last = it->second;
            tree[last].occ++;
            return false;
        }
        last = static_cast<int>(tree.size());
        tree.emplace_back(tree[cur].len + 2);
        tree[last].occ = 1;
        tree[cur].next[c] = last;
        if (tree[last].len == 1) {
            tree[last].link = 1;
            return true;
        }
        int link_parent = get_suffix(tree[cur].link, pos, c);
        tree[last].link = tree[link_parent].next[c];
        return true;
    }

    int count_distinct_palindromes() const { return static_cast<int>(tree.size()) - 2; }
    int longest_suffix_length() const { return tree[last].len; }

    std::vector<int> count_occurrences() const {
        std::vector<int> occ(tree.size());
        for (int i = 0; i < static_cast<int>(tree.size()); i++) {
            occ[i] = tree[i].occ;
        }
    }
};

```

```

    for (int i = static_cast<int>(tree.size()) - 1; i >= 2; i--) {
        occ[tree[i].link] += occ[i];
    }
    return occ;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Eertree t("abacaba");
    // Distinct palindromes: a, b, c, aba, aca, bacab, abacaba.
    assert(t.count_distinct_palindromes() == 7);
    assert(t.longest_suffix_length() == 7);

    Eertree online;
    assert(online.append('a'));
    assert(online.append('a'));
    assert(online.append('b'));
    assert(online.append('a'));
    assert(online.count_distinct_palindromes() == 4);
    assert(online.longest_suffix_length() == 3);

    Eertree duplicate;
    assert(duplicate.append('a'));
    assert(duplicate.append('b'));
    assert(duplicate.append('c'));
    // The second 'a' creates substring "abca", but palindrome "a" already exists.
    assert(!duplicate.append('a'));

    // Nodes 2-8 represent a, b, aba, c, aca, bacab, and abacaba in creation order.
    assert((t.count_occurrences() == vector<int>{0, 7, 4, 2, 2, 1, 1, 1, 1}));
    return 0;
}

```

## 3.6 Sequence Dynamic Programming

---

### 3.6.1 Longest Common Substring

Given two strings, determine their longest common substring: a contiguous run of characters that appears in both. A dynamic program computes, for each pair of prefixes, the length of their longest common suffix; the largest value over all pairs locates a longest common substring. Only the previous row of the table is retained, so the auxiliary space is linear in the shorter string.

- `longest_common_substring(s1, s2)` returns a longest common substring of `s1` and `s2` (one such substring if several are tied in length), or the empty string if they have none in common.

**Time Complexity:**  $O(n \cdot m)$  per call, where  $n$  and  $m$  are the lengths of strings `s1` and `s2`, respectively.

**Space Complexity:**  $O(\min(n, m))$  auxiliary, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.

```

#include <string>
#include <vector>
using std::string;

string longest_common_substring(const string &s1, const string &s2) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());

```

```

if (n == 0 || m == 0) {
    return "";
}
if (n < m) {
    return longest_common_substring(s2, s1);
}
std::vector<int> curr(m), prev(m);
int pos = 0, len = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (s1[i] == s2[j]) {
            curr[j] = (i > 0 && j > 0) ? prev[j - 1] + 1 : 1;
            if (len < curr[j]) {
                len = curr[j];
                pos = i - curr[j] + 1;
            }
        } else {
            curr[j] = 0;
        }
    }
    curr.swap(prev);
}
return s1.substr(pos, len);
}

```

### Example Usage

```

#include <cassert>

int main() {
    assert(longest_common_substring("bbbabca", "aababcd") == "babc");
    assert(longest_common_substring("abc", "def") == "");
    assert(longest_common_substring("", "abc") == "");
    assert(longest_common_substring("xyabc", "zzabczz") == "abc");
    return 0;
}

```

## 3.6.2 Longest Common Subsequence

Given two strings, determine their longest common subsequence. A subsequence is a string that can be derived from the original string by deleting some elements without changing the order of the remaining elements (e.g. "ACE" is a subsequence of "ABCDE", but "BAE" is not).

The dynamic program compares prefixes: if the last characters match, they can extend an optimal subsequence of the smaller prefixes; otherwise, one of the two last characters must be skipped. Following the filled table backward reconstructs one optimal subsequence. Hirschberg's algorithm keeps only one DP row at a time, using forward and backward length rows to choose where an optimal answer crosses the midpoint of the longer string.

- `longest_common_subsequence(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using a classic dynamic programming approach. It computes  $dp(i, j)$ , the longest common subsequence length for the length- $i$  prefix of `s1` and length- $j$  prefix of `s2`, before following the table backward to construct the answer.
- `hirschberg_lcs(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

**Time Complexity:**  $O(n \cdot m)$  per call to `longest_common_subsequence(s1, s2)` as well as `hirschberg_lcs(s1, s2)`, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.

**Space Complexity:**

- $O(n \cdot m)$  auxiliary heap space for `longest_common_subsequence(s1, s2)`, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.

- $O(\log(\max(n, m)))$  auxiliary stack space and  $O(\min(n, m))$  auxiliary heap space for `hirschberg_lcs(s1, s2)`, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.

```
#include <algorithm>
#include <iterator>
#include <string>
#include <vector>
using std::string;

string longest_common_subsequence(const string &s1, const string &s2) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    std::vector<std::vector<int>>> dp(n + 1, std::vector<int>(m + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = std::max(dp[i][j - 1], dp[i - 1][j]);
            }
        }
    }
    // Optional: reconstruct one longest common subsequence.
    string res;
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (s1[i - 1] == s2[j - 1]) {
            res += s1[i - 1];
            i--;
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

template<typename It>
void hirschberg_rec(It lo1, It hi1, It lo2, It hi2, string *res) {
    if (lo1 == hi1) {
        return;
    }
    if (lo1 + 1 == hi1) {
        if (std::find(lo2, hi2, *lo1) != hi2) {
            *res += *lo1;
        }
        return;
    }
    It mid1 = lo1 + (hi1 - lo1) / 2;
    std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
    auto lcs_len = [](auto lo1, auto hi1, auto lo2, auto hi2) {
        std::vector<int> res(std::distance(lo2, hi2) + 1, prev(res));
        for (auto it1 = lo1; it1 != hi1; ++it1) {
            res.swap(prev);
            int i = 0;
            for (auto it2 = lo2; it2 != hi2; ++it2) {
                res[i + 1] = (*it1 == *it2) ? prev[i] + 1 : std::max(res[i], prev[i + 1]);
                i++;
            }
        }
        return res;
    };
    return res;
};

std::vector<int> fwd = lcs_len(lo1, mid1, lo2, hi2);
std::vector<int> rev = lcs_len(rlo1, rmid1, rlo2, rhi2);
It mid2 = lo2;
int maxlen = -1;
```

```

for (int i = 0, j = static_cast<int>(rev.size()) - 1; i < static_cast<int>(fwd.size());
    i++, j--) {
    if (fwd[i] + rev[j] > maxlen) {
        maxlen = fwd[i] + rev[j];
        mid2 = lo2 + i;
    }
}
hirschberg_rec(lo1, mid1, lo2, mid2, res);
hirschberg_rec(mid1, hi1, mid2, hi2, res);
}

string hirschberg_lcs(const string &s1, const string &s2) {
    if (s1.size() < s2.size()) {
        return hirschberg_lcs(s2, s1);
    }
    string res;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res);
    return res;
}

```

### Example Usage

```

#include <cassert>

int main() {
    assert(longest_common_subsequence("xmjyauz", "mzjawxu") == "mjau");
    assert(hirschberg_lcs("xmjyauz", "mzjawxu") == "mjau");
    assert(longest_common_subsequence("abc", "def") == "");
    assert(hirschberg_lcs("abc", "def") == "");
    assert(longest_common_subsequence("", "abc") == "");
    assert(hirschberg_lcs("", "abc") == "");
    return 0;
}

```

### 3.6.3 Edit Distance and Sequence Alignment

Given two strings, determine their edit distance or a minimum-cost alignment. The Levenshtein edit distance is the minimum number of insertions, deletions, and substitutions needed to transform one string into the other. More generally, an alignment inserts gap characters `'_'` into both strings so that the final lengths match; its cost is `gap_cost` times the number of inserted gaps plus `sub_cost` times the number of indices where the aligned characters differ.

Dynamic programming compares prefixes and chooses the cheapest last operation: align the final characters together, leave a gap in the first string, or leave a gap in the second string. Following the filled table backward reconstructs one minimum-cost alignment. This is the Needleman-Wunsch sequence-alignment recurrence. Hirschberg's algorithm uses the same recurrence with only one DP row at a time, splitting the problem where an optimal alignment crosses the longer string's midpoint. Both `gap_cost` and `sub_cost` must be nonnegative.

- `edit_distance(s1, s2)` returns the Levenshtein edit distance between strings `s1` and `s2`.
- `align_sequences(s1, s2, gap_cost, sub_cost)` returns a pair of aligned strings for strings `s1` and `s2`, using a classic dynamic programming approach. It first computes  $dp(i, j)$ , the cost of aligning the length- $i$  prefix of `s1` with the length- $j$  prefix of `s2`, before following the table backward to construct the answer. For `gap_cost = sub_cost = 1`,  $dp(n, m)$  is the Levenshtein edit distance, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.
- `hirschberg_align(s1, s2, gap_cost, sub_cost)` returns the sequence alignment of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

**Time Complexity:**  $O(n \cdot m)$  per call to `edit_distance(s1, s2)`, `align_sequences(s1, s2)`, and `hirschberg_align(s1, s2)`, where  $n$  and  $m$  are the lengths of `s1` and `s2`, respectively.



**Space Complexity:**

- $O(\min(n, m))$  auxiliary heap space for `edit_distance(s1, s2)`.
- $O(n \cdot m)$  auxiliary heap space for `align_sequences(s1, s2)`.
- $O(\log(\max(n, m)))$  auxiliary stack space and  $O(\min(n, m))$  auxiliary heap space for `hirschberg_align(s1, s2)`.

```

#include <algorithm>
#include <iterator>
#include <string>
#include <utility>
#include <vector>
using std::string;

int edit_distance(const string &s1, const string &s2) {
    if (s1.size() < s2.size()) {
        return edit_distance(s2, s1);
    }
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    std::vector<int> dp(m + 1), prev(m + 1);
    for (int j = 0; j <= m; j++) {
        dp[j] = j;
    }
    for (int i = 1; i <= n; i++) {
        dp.swap(prev);
        dp[0] = i;
        for (int j = 1; j <= m; j++) {
            dp[j] =
                (s1[i - 1] == s2[j - 1]) ? prev[j - 1] : std::min({prev[j - 1], prev[j], dp[j - 1]}) + 1;
        }
    }
    return dp[m];
}

std::pair<string, string> align_sequences(
    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1));
    for (int i = 0; i <= n; i++) {
        dp[i][0] = i * gap_cost;
    }
    for (int j = 0; j <= m; j++) {
        dp[0][j] = j * gap_cost;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            dp[i][j] = (s1[i - 1] == s2[j - 1]) ? dp[i - 1][j - 1]
                : std::min(
                    dp[i - 1][j - 1] + sub_cost,
                    std::min(dp[i - 1][j], dp[i][j - 1]) + gap_cost
                );
        }
    }
    // Optional: reconstruct one minimum-cost alignment.
    string res1, res2;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1] || dp[i][j] == dp[i - 1][j - 1] + sub_cost) {
            res1 += s1[--i];
            res2 += s2[--j];
        } else if (dp[i][j] == dp[i - 1][j] + gap_cost) {
            res1 += s1[--i];
            res2 += '_';
        } else if (dp[i][j] == dp[i][j - 1] + gap_cost) {
            res1 += '_';
            res2 += s2[--j];
        }
    }
    reverse(res1.begin(), res1.end());
    reverse(res2.begin(), res2.end());
    return {res1, res2};
}

```

```

    }
}
while (i > 0 || j > 0) {
    res1 += (i > 0) ? s1[--i] : '_';
    res2 += (j > 0) ? s2[--j] : '_';
}
std::reverse(res1.begin(), res1.end());
std::reverse(res2.begin(), res2.end());
return {res1, res2};
}

template<typename It>
std::vector<int> row_cost(It lo1, It hi1, It lo2, It hi2, int gap_cost, int sub_cost) {
    std::vector<int> res(std::distance(lo2, hi2) + 1);
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        res[i] = i * gap_cost;
    }
    std::vector<int> prev(res);
    int row = 0;
    for (It it1 = lo1; it1 != hi1; ++it1) {
        res.swap(prev);
        res[0] = ++row * gap_cost;
        int i = 0;
        for (It it2 = lo2; it2 != hi2; ++it2) {
            res[i + 1] = (*it1 == *it2)
                ? prev[i]
                : std::min({prev[i] + sub_cost, prev[i + 1] + gap_cost, res[i] + gap_cost});
            i++;
        }
    }
    return res;
}

template<typename It>
void hirschberg_rec(
    It lo1, It hi1, It lo2, It hi2, string *res1, string *res2, int gap_cost, int sub_cost
) {
    if (lo1 == hi1) {
        for (It it2 = lo2; it2 != hi2; ++it2) {
            *res1 += '_';
            *res2 += *it2;
        }
        return;
    }
    if (lo1 + 1 == hi1) {
        It pos = std::find(lo2, hi2, *lo1);
        bool use_two_gaps = (pos == hi2) && (2 * gap_cost < sub_cost);
        if (lo2 == hi2 || use_two_gaps) {
            *res1 += *lo1;
            *res2 += '_';
        }
        for (It it2 = lo2; it2 != hi2; ++it2) {
            *res1 += (pos == it2 || (pos == hi2 && !use_two_gaps && it2 == lo2)) ? *lo1 : '_';
            *res2 += *it2;
        }
        return;
    }
    It mid1 = lo1 + (hi1 - lo1) / 2;
    std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
    std::vector<int> fwd = row_cost(lo1, mid1, lo2, hi2, gap_cost, sub_cost);
    std::vector<int> rev = row_cost(rlo1, rmid1, rlo2, rhi2, gap_cost, sub_cost);
    It mid2 = lo2;
    int mincost = -1;
    for (int i = 0, j = static_cast<int>(rev.size()) - 1; i < static_cast<int>(fwd.size());
        i++, j--) {
        if (mincost < 0 || fwd[i] + rev[j] < mincost) {
            mincost = fwd[i] + rev[j];
            mid2 = lo2 + i;
        }
    }
}

```

```

    }
}
hirschberg_rec(lo1, mid1, lo2, mid2, res1, res2, gap_cost, sub_cost);
hirschberg_rec(mid1, hi1, mid2, hi2, res1, res2, gap_cost, sub_cost);
}

std::pair<string, string> hirschberg_align(
    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    if (s1.size() < s2.size()) {
        auto res = hirschberg_align(s2, s1, gap_cost, sub_cost);
        return {res.second, res.first};
    }
    string res1, res2;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res1, &res2, gap_cost, sub_cost);
    return {res1, res2};
}

```

### Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    assert(edit_distance("kitten", "sitting") == 3);
    assert(edit_distance("", "abc") == 3);

    auto alignment = align_sequences("AGGGCT", "AGGCA", 2, 3);
    assert(alignment == (pair<string, string>{"AGGGCT", "A_GGCA"}));
    assert(hirschberg_align("AGGGCT", "AGGCA", 2, 3) == alignment);

    assert((hirschberg_align("ab", "zabz") == pair<string, string>{"_ab_", "zabz"}));
    assert(
        (hirschberg_align("a", "bbbbbbbbbb", 1, 5) ==
         pair<string, string>{"_____a", "bbbbbbbbbb"})
    );
    assert((hirschberg_align("aa", "ba", 1, 2) == pair<string, string>{"a_a", "_ba"}));

    cout << "Aligned strings:" << endl;
    cout << alignment.first << endl << alignment.second << endl;
    return 0;
}

```

#### Example Output

```

Aligned strings:
AGGGCT
A_GGCA

```

### 3.6.4 Wildcard Matching

Decide whether a pattern of literal characters, single-character wildcards, and multi-character wildcards matches an entire text. The wildcard `?` matches exactly one character, and `*` matches any run of characters including the empty one. This is glob matching rather than regular expression matching, so there is no alternation and no repetition of a group.

Let  $m(i, j)$  mean that the first  $i$  characters of the text are matched by the first  $j$  characters of the pattern. A literal or `?` consumes one character from each side, so it depends only on  $m(i - 1, j - 1)$ . A `*` either matches nothing, leaving  $m(i, j - 1)$ , or absorbs one more text character, leaving  $m(i - 1, j)$ ; every longer run is covered by repeating that second case, which is what keeps the table two-dimensional instead of requiring a loop over run lengths. Only the previous row is ever read, so one row of storage suffices.

- `wildcard_match(text, pattern)` returns whether `pattern` matches all of `text`, where `?` matches any single character and `*` matches any sequence of characters.

A greedy walk needs only  $O(1)$  space: remember the most recent `*` and the text position it was reached at, then on a mismatch advance the text and resume just after that `*`. Backtracking to the latest `*` suffices, since an earlier one absorbs whatever a later gives up. It runs in  $O(n + m)$  on ordinary inputs but stays  $O(n \cdot m)$  in the worst case, since each mismatch can rescan the pattern from the last `*`. The table is what generalizes, to counting matches or allowing bounded mismatches.

**Time Complexity:**  $O(n \cdot m)$  per call, where  $n$  and  $m$  are the lengths of `text` and `pattern`.

**Space Complexity:**  $O(m)$  auxiliary.

```
#include <string>
#include <vector>

bool wildcard_match(const std::string &text, const std::string &pattern) {
    int n = static_cast<int>(text.size()), m = static_cast<int>(pattern.size());
    std::vector<char> prev(m + 1), curr(m + 1);
    prev[0] = true;
    for (int j = 1; j <= m; j++) { // An all-star prefix is the only way to match an empty text.
        prev[j] = prev[j - 1] && pattern[j - 1] == '*';
    }
    for (int i = 1; i <= n; i++) {
        curr[0] = false;
        for (int j = 1; j <= m; j++) {
            if (pattern[j - 1] == '*') {
                curr[j] = curr[j - 1] || prev[j]; // Match nothing, or absorb one more character.
            } else if (pattern[j - 1] == '?' || pattern[j - 1] == text[i - 1]) {
                curr[j] = prev[j - 1];
            } else {
                curr[j] = false;
            }
        }
        prev.swap(curr);
    }
    return prev[m];
}
```

## Example Usage

```
#include <cassert>

int main() {
    assert(wildcard_match("", ""));
    assert(wildcard_match("", "*"));
    assert(wildcard_match("", "***"));
    assert(!wildcard_match("", "?"));
    assert(!wildcard_match("abc", ""));

    assert(wildcard_match("abc", "abc"));
    assert(wildcard_match("abc", "a?c"));
    assert(wildcard_match("abc", "a*"));
    assert(wildcard_match("abc", "*c"));
    assert(wildcard_match("abcde", "a*d?"));
    assert(!wildcard_match("abc", "a?")); // The pattern must cover the whole text.
    assert(!wildcard_match("abc", "*d*"));

    // The wildcard must not consume characters the literals still need.
    assert(wildcard_match("aaa", "*a"));
    assert(!wildcard_match("aab", "*a"));
    assert(wildcard_match("mississippi", "m*i*s*p?"));
    assert(!wildcard_match("mississippi", "m*i*s*x*"));
    return 0;
}
```

### 3.6.5 Palindrome DP

Solves basic palindrome dynamic programming problems on a string. The core interval state considers substring  $[l, r]$ : if the two endpoints match, they can wrap an optimal answer for the inside substring; otherwise, at least one endpoint must be skipped or inserted around the other side. Since each state depends on shorter substrings, intervals are processed by increasing length.

The longest palindromic subsequence (LPS) is a maximum-length subsequence that reads the same forward and backward. The minimum number of insertions needed to make a string a palindrome is the number of characters outside an LPS, so it equals  $n$  minus the LPS length.

- `longest_palindromic_subsequence(s)` returns one longest palindromic subsequence of string `s`.
- `min_insertions_palindrome(s)` returns the minimum number of characters that must be inserted anywhere in string `s` to make it a palindrome.

**Time Complexity:**  $O(n^2)$  per call to `longest_palindromic_subsequence(s)` and `min_insertions_palindrome(s)`, where  $n$  is the length of `s`.

**Space Complexity:**

- $O(n^2)$  auxiliary for `longest_palindromic_subsequence()`.
- $O(n)$  auxiliary for `min_insertions_palindrome()`.

```
#include <algorithm>
#include <cassert>
#include <string>
#include <vector>
using std::string;

string longest_palindromic_subsequence(const string &s) {
    int n = static_cast<int>(s.size());
    if (n == 0) {
        return "";
    }
    std::vector<std::vector<int>> dp(n, std::vector<int>(n));
    for (int len = 1; len <= n; len++) {
        for (int l = 0, r = len - 1; r < n; l++, r++) {
            if (l == r) {
                dp[l][r] = 1;
            } else if (s[l] == s[r]) {
                dp[l][r] = 2 + (l + 1 <= r - 1 ? dp[l + 1][r - 1] : 0);
            } else {
                dp[l][r] = std::max(dp[l + 1][r], dp[l][r - 1]);
            }
        }
    }
    // Optional: reconstruct one longest palindromic subsequence.
    string left, right;
    for (int l = 0, r = n - 1; l <= r;) {
        if (l == r) {
            left += s[l];
            break;
        }
        if (s[l] == s[r] && dp[l][r] == 2 + (l + 1 <= r - 1 ? dp[l + 1][r - 1] : 0)) {
            left += s[l++];
            right += s[r--];
        } else if (dp[l + 1][r] >= dp[l][r - 1]) {
            l++;
        } else {
            r--;
        }
    }
    std::reverse(right.begin(), right.end());
    return left + right;
}
```

```
int min_insertions_palindrome(const string &s) {
    int n = static_cast<int>(s.size());
    std::vector<int> dp(n);
    for (int l = n - 1; l >= 0; l--) {
        int inside = 0;
        for (int r = l + 1; r < n; r++) {
            int old = dp[r];
            if (s[l] == s[r]) {
                dp[r] = inside;
            } else {
                dp[r] = 1 + std::min(dp[r], dp[r - 1]);
            }
            inside = old;
        }
    }
    return n == 0 ? 0 : dp[n - 1];
}
```

### Example Usage

```
#include <cassert>

int main() {
    assert(longest_palindromic_subsequence("bbbab") == "bbbb");
    assert(longest_palindromic_subsequence("character") == "carac");
    assert(min_insertions_palindrome("abcda") == 2);
    assert(min_insertions_palindrome("race") == 3);
    assert(min_insertions_palindrome("") == 0);
    return 0;
}
```

## 3.7 Expression Parsing

---

### 3.7.1 Recursive Descent Parsing

Evaluate integer arithmetic expressions containing parentheses, unary plus and minus, multiplication and division, and addition and subtraction. The parser represents these as three precedence levels. At the tightest level, it consumes a chain of unary signs followed by either a decimal literal or a parenthesized expression.

At each binary-operator level, the parser first evaluates one operand at the next tighter level, then repeatedly consumes operators of the current level and their right operands. These loops make binary operators left-associative, while recursively restarting at the loosest level inside parentheses enforces grouping.

Literals must be nonnegative decimal integers, although unary signs may precede them. The input must be valid and contain no whitespace. Division truncates toward zero, and every divisor must be nonzero.

- `eval(s)` returns an evaluation of the arithmetic expression `s`.

Overflow warning: All intermediate results must fit in `int`.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the length of `s`.

**Space Complexity:**  $O(n)$  auxiliary stack space, where  $n$  is the length of `s`.

```
#include <string>

int eval(const std::string &s) {
    std::string expr = s + '\0';
    auto it = expr.begin();
    auto rec = [&](auto &&rec, int prec) -> int {
```

```

if (prec == 0) {
    int sign = 1, res = 0;
    for (; *it == '-' || *it == '+'; it++) {
        if (*it == '-') {
            sign *= -1;
        }
    }
    if (*it == '(') {
        it++;
        res = rec(rec, 2);
        it++;
    } else {
        while (*it >= '0' && *it <= '9') {
            res = 10 * res + (*(it++) - '0');
        }
    }
    return sign * res;
}
int num = rec(rec, prec - 1);
while (!((prec == 2 && *it != '+' && *it != '-') || (prec == 1 && *it != '*' && *it != '/'))) {
    switch (*(it++)) {
        case '+':
            num += rec(rec, prec - 1);
            break;
        case '-':
            num -= rec(rec, prec - 1);
            break;
        case '*':
            num *= rec(rec, prec - 1);
            break;
        case '/':
            num /= rec(rec, prec - 1);
            break;
    }
}
return num;
};
return rec(rec, 2);
}

```

## Example Usage

```

#include <cassert>

int main() {
    assert(eval("+1") == 1);
    assert(eval("1+-2") == -1);
    assert(eval("1++1") == 2);
    assert(eval("1+2*3*4+3*(2+2)-100") == -63);
    return 0;
}

```

### 3.7.2 Shunting Yard and Postfix Evaluation

Convert an infix expression to postfix notation, also called reverse Polish notation (RPN), and evaluate the result. In postfix notation each operator follows its operands, so evaluation needs no parentheses or precedence rules: operands are pushed onto a value stack, and each operator replaces its operands with their result.

The shunting yard algorithm produces postfix notation by sending operands directly to the output and holding operators on a stack. An operator is emitted before a lower-precedence operator, or before a left-associative operator of equal precedence. Parentheses delimit independent portions of the operator stack. Whether  $\oplus$  or  $\ominus$  is

unary follows from context: a prefix operator is expected at the beginning, after another operator, or after an opening parenthesis.

Customize the operand type and syntax by changing `Operand` and `eval_operand(token)`. The unary and binary operator tables define their functions and precedences; binary rules additionally specify right-associativity. Prefix unary operators are emitted with a `u` prefix, so unary `-` becomes the distinct postfix token `u-`. Operators may contain multiple non-operand characters, and the longest matching operator is chosen. Operands are maximal strings of letters, digits, underscores, and periods.

- `tokenize(s)` splits expression `s` into operands, operators, and parentheses. It throws `runtime_error` for unknown tokens.
- `to_postfix(tokens)` converts infix `tokens` into postfix notation. It throws `runtime_error` for malformed expressions or mismatched parentheses.
- `eval_postfix(tokens)` evaluates a tokenized postfix expression. It throws `runtime_error` if an operator lacks operands or the expression does not produce one value.
- `eval_expression(s)` converts and evaluates infix expression `s`.

#### Time Complexity:

- $O(m \cdot k \cdot \log m + n \cdot m \cdot k)$  expected per call to `tokenize()` and `eval_expression()`, where  $n$  is the length of `s`,  $m$  is the total number of operators, and  $k$  is their maximum length.
- $O(n)$  expected per call to `to_postfix()` and `eval_postfix()`, where  $n$  is the total length of the tokens.

#### Space Complexity:

- $O(n + m \cdot k)$  auxiliary space for `tokenize()` and `eval_expression()`.
- $O(n)$  auxiliary space for `to_postfix()` and `eval_postfix()`.

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

using Operand = double;

struct UnaryRule {
    std::function<Operand(Operand)> op;
    int precedence;
};

struct BinaryRule {
    std::function<Operand(Operand, Operand)> op;
    int precedence;
    bool right_associative = false;
};

std::unordered_map<string, UnaryRule> unary_ops;
std::unordered_map<string, BinaryRule> binary_ops;

Operand eval_operand(const string &token) {
    return std::stod(token);
}

void require(bool condition, const string &message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

std::vector<string> tokenize(const string &s) {
    std::vector<string> op_tokens;
```



```

for (const auto &[op, rule] : unary_ops) {
    op_tokens.push_back(op);
}
for (const auto &[op, rule] : binary_ops) {
    op_tokens.push_back(op);
}
std::sort(op_tokens.begin(), op_tokens.end(), [](const string &a, const string &b) {
    return a.size() != b.size() ? a.size() > b.size() : a < b;
});
op_tokens.erase(std::unique(op_tokens.begin(), op_tokens.end(), op_tokens.end()));
auto is_operand_char = [](unsigned char c) { return std::isalnum(c) || c == '_' || c == '.'; };
std::vector<string> tokens;
int n = static_cast<int>(s.size());
for (int i = 0; i < n; i++) {
    unsigned char c = static_cast<unsigned char>(s[i]);
    if (std::isspace(c)) {
        i++;
    } else if (is_operand_char(c)) {
        int j = i + 1;
        while (j < n && is_operand_char(static_cast<unsigned char>(s[j]))) {
            j++;
        }
        tokens.push_back(s.substr(i, j - i));
        i = j;
    } else if (s[i] == '(' || s[i] == ')') {
        tokens.emplace_back(1, s[i]);
        i++;
    } else {
        string op;
        for (const string &candidate : op_tokens) {
            if (s.compare(i, candidate.size(), candidate) == 0) {
                op = candidate;
                break;
            }
        }
        require(!op.empty(), "Unknown token at position " + std::to_string(i) + ".");
        tokens.push_back(op);
        i += static_cast<int>(op.size());
    }
}
return tokens;
}

std::vector<string> to_postfix(const std::vector<string> &tokens) {
    struct PendingOp {
        string token;
        int precedence;
    };
    std::vector<string> output;
    std::vector<PendingOp> op_stack;
    bool expect_operand = true;
    for (const string &token : tokens) {
        bool is_operator = unary_ops.count(token) || binary_ops.count(token);
        if (!is_operator && token != "(" && token != ")") {
            require(expect_operand, "Expected an operator before token '" + token + "'.");
            output.push_back(token);
            expect_operand = false;
        } else if (token == "(") {
            require(expect_operand, "Expected an operator before opening parenthesis.");
            op_stack.push_back({ "(", -1 });
        } else if (token == ")") {
            require(!expect_operand, "Unexpected closing parenthesis.");
            while (!op_stack.empty() && op_stack.back().token != "(") {
                output.push_back(op_stack.back().token);
                op_stack.pop_back();
            }
            require(!op_stack.empty(), "Mismatched parentheses.");
            op_stack.pop_back();
        }
    }
}

```

```

    expect_operand = false;
} else if (expect_operand) {
    auto it = unary_ops.find(token);
    require(it != unary_ops.end(), "Expected an operand before token '" + token + "'.");
    op_stack.push_back({"u" + token, it->second.precedence});
} else {
    auto it = binary_ops.find(token);
    require(it != binary_ops.end(), "Expected a binary operator, got '" + token + "'.");
    const BinaryRule &rule = it->second;
    while (!op_stack.empty() && op_stack.back().token != "(" &&
           (op_stack.back().precedence > rule.precedence ||
            (op_stack.back().precedence == rule.precedence && !rule.right_associative))) {
        output.push_back(op_stack.back().token);
        op_stack.pop_back();
    }
    op_stack.push_back({token, rule.precedence});
    expect_operand = true;
}
}
require(!expect_operand, "Expression ends before an operand.");
while (!op_stack.empty()) {
    require(op_stack.back().token != "(", "Mismatched parentheses.");
    output.push_back(op_stack.back().token);
    op_stack.pop_back();
}
return output;
}

Operand eval_postfix(const std::vector<string> &tokens) {
    std::vector<Operand> values;
    for (const string &token : tokens) {
        auto binary = binary_ops.find(token);
        auto unary =
            token.size() > 1 && token[0] == 'u' ? unary_ops.find(token.substr(1)) : unary_ops.end();
        if (binary != binary_ops.end()) {
            require(values.size() >= 2, "Binary operator lacks two operands: " + token);
            Operand b = values.back();
            values.pop_back();
            values.back() = binary->second.op(values.back(), b);
        } else if (unary != unary_ops.end()) {
            require(!values.empty(), "Unary operator lacks an operand: " + token);
            values.back() = unary->second.op(values.back());
        } else {
            values.push_back(eval_operand(token));
        }
    }
    require(values.size() == 1, "Invalid postfix expression.");
    return values.back();
}

Operand eval_expression(const string &s) {
    return eval_postfix(to_postfix(tokenize(s)));
}

```

## Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return a == b || fabs(a - b) <= 1e-9;
}

int main() {
    unary_ops = {

```

```

    {"+", {[] (Operand x) { return x; }, 3}},
    {"-", {[] (Operand x) { return -x; }, 3}},
};
binary_ops = {
    {"<=", {[] (Operand a, Operand b) { return a <= b; }, 0}},
    {"+", {[] (Operand a, Operand b) { return a + b; }, 1}},
    {"-", {[] (Operand a, Operand b) { return a - b; }, 1}},
    {"*", {[] (Operand a, Operand b) { return a * b; }, 2}},
    {"/", {[] (Operand a, Operand b) { return a / b; }, 2}},
    {"^", {[] (Operand a, Operand b) { return pow(a, b); }, 4, true}},
};
assert((tokenize("left <= -12.5") == vector<string>{"left", "<=", "-", "12.5"}));
assert((to_postfix(tokenize("-(2+3)*4")) == vector<string>{"2", "3", "+", "u-", "4", "*"}));
assert((to_postfix(tokenize("2^-3^2")) == vector<string>{"2", "3", "2", "^", "u-", "^"}));
assert(EQ(eval_postfix({"2", "3", "4", "*", "+"}), 14));
assert(EQ(eval_expression("-2^2"), -4));
assert(EQ(eval_expression("2^-3"), 0.125));
assert(EQ(eval_expression("1<=1"), 1));
assert(EQ(eval_expression("5*(3+2)/-1*-2"), 50));
return 0;
}

```

### 3.7.3 Pratt Parsing

Evaluate an expression with configurable prefix unary and infix binary operators using Pratt parsing, also called top-down operator-precedence parsing. Unlike recursive descent with one function per precedence level, a Pratt parser uses numeric binding powers to decide whether the next operator belongs to the current expression or to its caller.

Parsing begins with an operand, parenthesized expression, or prefix operator. It then repeatedly consumes binary operators whose precedence is high enough for the current call. A left-associative operator raises the minimum precedence for its right operand, preventing another equal-precedence operator from joining that operand; a right-associative operator leaves the threshold unchanged.

Customize the operand type and syntax by changing `Operand` and `eval_operand()`. Operators may contain multiple non-operand characters, and the longest matching operator is chosen. Operands are maximal strings of letters, digits, underscores, and periods. Multiplication by juxtaposition and postfix operators are not supported.

- `eval_pratt(s)` evaluates expression `s` using the global operator tables. It throws `runtime_error` for malformed expressions, unknown tokens, or mismatched parentheses.

**Time Complexity:**  $O(m \cdot k \cdot \log m + n \cdot m \cdot k)$  expected per call, where  $n$  is the length of `s`,  $m$  is the total number of operators, and  $k$  is their maximum length.

**Space Complexity:**  $O(n + m \cdot k)$  auxiliary space, including the recursive stack.

```

#include <algorithm>
#include <cctype>
#include <functional>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

using Operand = double;

struct UnaryRule {
    std::function<Operand(Operand)> op;
    int precedence;
};

struct BinaryRule {

```

```

    std::function<Operand(Operand, Operand)> op;
    int precedence;
    bool right_associative = false;
};

std::unordered_map<string, UnaryRule> unary_ops;
std::unordered_map<string, BinaryRule> binary_ops;

Operand eval_operand(const string &token) {
    return std::stod(token);
}

void require(bool condition, const string &message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

Operand eval_pratt(const string &s) {
    std::vector<string> op_tokens;
    for (const auto &[op, rule] : unary_ops) {
        op_tokens.push_back(op);
    }
    for (const auto &[op, rule] : binary_ops) {
        op_tokens.push_back(op);
    }
    std::sort(op_tokens.begin(), op_tokens.end(), [](const string &a, const string &b) {
        return a.size() != b.size() ? a.size() > b.size() : a < b;
    });
    op_tokens.erase(std::unique(op_tokens.begin(), op_tokens.end()), op_tokens.end());
    int pos = 0, n = static_cast<int>(s.size());
    auto skip_spaces = [&] {
        while (pos < n && std::isspace(static_cast<unsigned char>(s[pos]))) {
            pos++;
        }
    };
    auto match_operator = [&](bool unary) {
        skip_spaces();
        for (const string &op : op_tokens) {
            const bool allowed = unary ? unary_ops.count(op) : binary_ops.count(op);
            if (allowed && s.compare(pos, op.size(), op) == 0) {
                return op;
            }
        }
        return string();
    };
    auto is_operand_char = [](unsigned char c) { return std::isalnum(c) || c == '_' || c == '.'; };
    auto parse = [&](auto &&parse, int min_precedence) -> Operand {
        skip_spaces();
        require(pos != n, "Expression ends before an operand.");
        Operand lhs;
        if (s[pos] == '(') {
            pos++;
            lhs = parse(parse, 0);
            skip_spaces();
            require(pos != n && s[pos] == ')', "Mismatched parentheses.");
            pos++;
        } else if (string op = match_operator(true); !op.empty()) {
            pos += static_cast<int>(op.size());
            const UnaryRule &rule = unary_ops.at(op);
            lhs = rule.op(parse(parse, rule.precedence));
        } else {
            require(
                is_operand_char(static_cast<unsigned char>(s[pos])),
                "Expected operand at position " + std::to_string(pos) + "."
            );
            int start = pos++;
            while (pos < n && is_operand_char(static_cast<unsigned char>(s[pos]))) {

```

```

        pos++;
    }
    lhs = eval_operand(s.substr(start, pos - start));
}
while (true) {
    string op = match_operator(false);
    if (op.empty()) {
        return lhs;
    }
    const BinaryRule &rule = binary_ops.at(op);
    if (rule.precedence < min_precedence) {
        return lhs;
    }
    pos += static_cast<int>(op.size());
    lhs = rule.op(lhs, parse(parse, rule.precedence + !rule.right_associative));
}
};
Operand value = parse(parse, 0);
skip_spaces();
require(pos == n, "Unexpected token at position " + std::to_string(pos) + ".");
return value;
}

```

### Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return a == b || fabs(a - b) <= 1e-9;
}

int main() {
    unary_ops = {
        {"+", {[] (Operand x) { return x; }, 3}},
        {"-", {[] (Operand x) { return -x; }, 3}},
    };
    binary_ops = {
        {"+", {[] (Operand a, Operand b) { return a + b; }, 1}},
        {"-", {[] (Operand a, Operand b) { return a - b; }, 1}},
        {"*", {[] (Operand a, Operand b) { return a * b; }, 2}},
        {"/", {[] (Operand a, Operand b) { return a / b; }, 2}},
        {"^", {[] (Operand a, Operand b) { return pow(a, b); }, 4, true}},
    };
    assert(EQ(eval_pratt("-2^2"), -4));
    assert(EQ(eval_pratt("2^-3"), 0.125));
    assert(EQ(eval_pratt("2^3^2"), 512));
    assert(EQ(eval_pratt("5*(3+2)/-1*-2"), 50));
    return 0;
}

```

## 3.8 Encoding and Compression

---

### 3.8.1 Entropy and Frequency Tables

Computes byte frequency tables and empirical information-theoretic quantities for strings. These primitives are useful when analyzing compression, building coding schemes such as Huffman coding, or comparing symbol distributions.

Shannon entropy measures the average uncertainty of a random symbol, in bits. It is called empirical\* here because the probabilities are estimated from the string itself rather than supplied by a known source. The *order* of the estimate is how much preceding context it conditions on: an order-0 model treats symbols as independent and looks only at how often each byte occurs, while an order- $k$  model conditions each symbol on the  $k$  bytes before it.

For symbol probabilities  $p_i$ , the order-0 empirical entropy is  $H = -\sum_i p_i \log_2 p_i$  bits per symbol, a lower bound on the average number of bits per symbol achievable by any uniquely decodable code under that independent model. Because it ignores ordering, "abab" and "aabb" have identical order-0 entropy. Real data has structure across positions, so conditioning lowers the estimate: order- $k$  entropy never exceeds order- $(k-1)$  entropy, which is why practical compressors beat the order-0 bound.

- `byte_frequencies(s)` returns a length-256 table of byte counts in string `s`.
- `entropy(freq)` returns order-0 empirical entropy in bits per symbol for a frequency table.
- `entropy(s)` returns order-0 empirical entropy in bits per symbol for string `s`.
- `conditional_entropy(s, order = 1)` returns the order-`order` empirical entropy of `s` in bits per symbol, that is, the average uncertainty of each byte given the `order` bytes preceding it. `conditional_entropy(s, 0)` equals `entropy(s)`, and raising `order` never increases the result. A string of at most `order` bytes has no complete context, so the result is 0.
- `expected_code_length(freq, length)` returns the average encoded bits per symbol for code lengths `length[c]`. The two input vectors must have the same size.

#### Time Complexity:

- $O(n + m)$  per call to `entropy(s)`, where  $n$  is the string length and  $m = 256$ .
- $O(m)$  per call to `entropy(freq)` and `expected_code_length(freq, length)`.
- $O(n \cdot k)$  expected per call to `conditional_entropy(s, order)`, where  $k$  is `order`, for hashing the context of each position.

#### Space Complexity:

- $O(m)$  for the table returned by `byte_frequencies()` and  $O(m)$  auxiliary for `entropy(s)`.
- $O(n \cdot k)$  auxiliary for `conditional_entropy(s, order)`, since at most  $n$  distinct contexts are stored.
- $O(1)$  auxiliary for the other operations.

```
#include <cassert>
#include <cmath>
#include <cstdint>
#include <numeric>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

std::vector<int> byte_frequencies(const string &s) {
    std::vector<int> freq(256);
    for (unsigned char c : s) {
        freq[c]++;
    }
    return freq;
}

double entropy(const std::vector<int> &freq) {
    int64_t total = std::accumulate(freq.begin(), freq.end(), 0LL);
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int count : freq) {
        if (count == 0) {
            continue;
        }
        double p = static_cast<double>(count) / total;
        res -= p * std::log2(p);
    }
}
```

```

    return res;
}

double entropy(const string &s) {
    return entropy(byte_frequencies(s));
}

double conditional_entropy(const string &s, int order = 1) {
    assert(order >= 0);
    int n = static_cast<int>(s.size());
    if (n <= order) {
        return 0;
    }
    // Contexts are keyed by their bytes rather than indexed by an alphabet-sized table, so the space
    // used depends on how many distinct contexts occur instead of on the alphabet size raised to the
    // order. Every context is exactly that many bytes, so splitting an event key is unambiguous.
    std::unordered_map<string, int> context_count, event_count;
    for (int i = order; i < n; i++) {
        string context = s.substr(i - order, order);
        context_count[context]++;
        event_count[context + s[i]]++;
    }
    double total = n - order;
    double res = 0;
    for (const auto &[event, count] : event_count) {
        double conditional = count / static_cast<double>(context_count[event.substr(0, order)]);
        res -= (count / total) * std::log2(conditional);
    }
    return res;
}

double expected_code_length(const std::vector<int> &freq, const std::vector<int> &length) {
    assert(freq.size() == length.size());
    int64_t total = std::accumulate(freq.begin(), freq.end(), 0LL);
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int i = 0; i < static_cast<int>(freq.size()); i++) {
        res += static_cast<double>(freq[i]) * length[i] / total;
    }
    return res;
}

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    vector<int> freq = byte_frequencies("aabb");
    assert(freq['a'] == 2);
    assert(freq['b'] == 2);
    assert(EQ(entropy(freq), 1.0));
    assert(EQ(entropy("aaaa"), 0.0));

    vector<int> length(256);
    length['a'] = 1;
    length['b'] = 1;
    assert(EQ(expected_code_length(freq, length), 1.0));

    // Order 0 conditions on nothing, so it reproduces the plain entropy.

```

```

assert(EQ(conditional_entropy("aabb", 0), entropy("aabb")));

// Order-0 entropy ignores ordering; higher orders do not. In "abab" the previous symbol
// determines the next one exactly, so no uncertainty remains.
assert(EQ(entropy("abab"), entropy("aabb")));
assert(EQ(conditional_entropy("abab"), 0.0));
assert(EQ(conditional_entropy("aabb"), 2.0 / 3));
assert(conditional_entropy("aabb") < entropy("aabb"));

// Raising the order never increases the estimate, and too little text yields no context at all.
assert(conditional_entropy("abcabcabc", 2) <= conditional_entropy("abcabcabc", 1));
assert(EQ(conditional_entropy("a"), 0.0));
assert(EQ(conditional_entropy("abc", 3), 0.0));
return 0;
}

```

### 3.8.2 Run-Length Encoding

Encodes consecutive equal characters as (`character`, `count`) runs. Run-length encoding is a simple lossless compression technique that works well when strings contain long repeated blocks and poorly when repetitions are rare.

The representation below stores runs in a vector of pairs rather than formatting them as a string. This avoids ambiguity when the original text contains digits or separator characters.

- `run_length_encode(s)` returns the sequence of runs in string `s`.
- `run_length_decode(runs)` reconstructs the original string from a sequence of runs.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the input or output length.

**Space Complexity:**

- $O(r)$  for encoded output, where  $r$  is the number of runs.
- $O(n)$  for decoded output.

```

#include <string>
#include <utility>
#include <vector>
using std::string;

std::vector<std::pair<char, int>> run_length_encode(const string &s) {
    std::vector<std::pair<char, int>> res;
    for (char c : s) {
        if (res.empty() || res.back().first != c) {
            res.emplace_back(c, 1);
        } else {
            res.back().second++;
        }
    }
    return res;
}

string run_length_decode(const std::vector<std::pair<char, int>> &runs) {
    string res;
    for (auto [c, count] : runs) {
        res.append(count, c);
    }
    return res;
}

```



## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    string s = "aaabccccdd";
    vector<pair<char, int>> runs = run_length_encode(s);
    assert((runs == vector<pair<char, int>>{{'a', 3}, {'b', 1}, {'c', 4}, {'d', 2}}));
    assert(run_length_decode(runs) == s);
    assert(run_length_encode("").empty());
    assert(run_length_decode({}) == "");
    assert((run_length_encode("z") == vector<pair<char, int>>{{'z', 1}}));
    return 0;
}
```

### 3.8.3 Base64 Encoding

Encodes binary data as printable ASCII using Base64, and decodes Base64 text back to bytes. Base64 is an encoding, not encryption or compression: it preserves all input data exactly, but usually increases size by about one third. Each group of three input bytes is split into four 6-bit values, each selecting one character of a 64-symbol alphabet, with `=` padding marking a final group of only one or two bytes.

This implementation uses the standard alphabet with `+`, `/`, and `=` padding. Whitespace is not skipped by the decoder; clean the input first if line breaks or spaces may appear.

- `base64_encode(bytes)` returns the Base64 representation of string `bytes`.
- `base64_decode(text)` returns the decoded byte string. It assumes `text` is valid padded Base64.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the input length.

**Space Complexity:**  $O(n)$  for the output string.

```
#include <cstdint>
#include <string>
#include <vector>
using std::string;

const char *BASE64_ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

string base64_encode(const string &bytes) {
    string res;
    uint32_t val = 0;
    int bits = -6;
    for (unsigned char byte : bytes) {
        val = (val << 8) + byte;
        bits += 8;
        while (bits >= 0) {
            res.push_back(BASE64_ALPHABET[(val >> bits) & 63]);
            bits -= 6;
        }
    }
    if (bits > -6) {
        res.push_back(BASE64_ALPHABET[((val << 8) >> (bits + 8)) & 63]);
    }
    while (res.size() % 4) {
        res.push_back('=');
    }
    return res;
}

string base64_decode(const string &text) {
    std::vector<int> table(256, -1);
```

```

for (int i = 0; i < 64; i++) {
    table[static_cast<unsigned char>(BASE64_ALPHABET[i])] = i;
}
string res;
uint32_t val = 0;
int bits = -8;
for (unsigned char c : text) {
    if (c == '=') {
        break;
    }
    int x = table[c];
    if (x < 0) {
        break;
    }
    val = (val << 6) + x;
    bits += 6;
    if (bits >= 0) {
        res.push_back(static_cast<char>(((val >> bits) & 255)));
        bits -= 8;
    }
}
return res;
}

```

### Example Usage

```

#include <cassert>

int main() {
    assert(base64_encode("") == "");
    assert(base64_decode("") == "");
    assert(base64_encode("f") == "Zg==");
    assert(base64_encode("fo") == "Zm8=");
    assert(base64_encode("hello") == "aGVsbG8=");
    assert(base64_decode("aGVsbG8=") == "hello");
    assert(base64_decode(base64_encode("abc123!?")) == "abc123!?");
    return 0;
}

```

## 3.8.4 Huffman Coding

Builds a Huffman code for a string and uses it to encode and decode that string. Huffman coding is a lossless prefix-code compression algorithm: more frequent characters receive shorter bit strings, and no code is a prefix of another code. The code tree is built greedily with a priority queue by repeatedly merging the two lowest-frequency subtrees until one tree remains; each character's bit string is then read off its root-to-leaf path.

The implementation below stores encoded bits as a string of `'0'` and `'1'` characters for clarity. For real compression, pack those bits into bytes. The tree is also needed to decode the bit string, so compressed data normally stores enough metadata to reconstruct the same tree.

- `HuffmanTree(text)` constructs a Huffman tree from the character frequencies in `text`.
- `codes()` returns a table mapping each byte value to its bit string.
- `encode(text)` returns the encoded bit string. Every character in `text` must occur in the string used to construct the tree.
- `decode(bits)` returns the decoded text for a bit string produced by this tree.

#### Time Complexity:

- $O(n + m \log m)$  per call to the constructor, where  $n$  is the input length and  $m$  is the number of distinct characters.
- $O(n + b)$  per call to `encode()` for  $n$  characters producing  $b$  bits, and  $O(b)$  per call to `decode()`.

**Space Complexity:**

- $O(m)$  for the tree and code table, where  $m$  is the number of distinct characters.
- $O(n)$  for the encoded or decoded output.

```

#include <functional>
#include <queue>
#include <string>
#include <utility>
#include <vector>
using std::string;

class HuffmanTree {
    struct Node {
        int freq;
        unsigned char ch;
        int left, right;

        Node(int freq = 0, unsigned char ch = 0, int left = -1, int right = -1)
            : freq(freq), ch(ch), left(left), right(right) {}
    };

    int root;
    std::vector<Node> nodes;
    std::vector<string> code;

    void build_codes(int u, const string &path) {
        if (nodes[u].left == -1 && nodes[u].right == -1) {
            code[nodes[u].ch] = path.empty() ? "0" : path;
            return;
        }
        build_codes(nodes[u].left, path + '0');
        build_codes(nodes[u].right, path + '1');
    }

public:
    explicit HuffmanTree(const string &text) : root(-1), code(256) {
        std::vector<int> freq(256);
        for (unsigned char c : text) {
            freq[c]++;
        }
        std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>, std::greater<>> pq;
        for (int c = 0; c < 256; c++) {
            if (freq[c] > 0) {
                nodes.emplace_back(freq[c], static_cast<unsigned char>(c));
                pq.emplace(freq[c], static_cast<int>(nodes.size()) - 1);
            }
        }
        if (pq.empty()) {
            return;
        }
        while (static_cast<int>(pq.size()) > 1) {
            int a = pq.top().second;
            pq.pop();
            int b = pq.top().second;
            pq.pop();
            nodes.emplace_back(nodes[a].freq + nodes[b].freq, 0, a, b);
            pq.emplace(nodes.back().freq, static_cast<int>(nodes.size()) - 1);
        }
        root = pq.top().second;
        build_codes(root, "");
    }

    std::vector<string> codes() const { return code; }

    string encode(const string &text) const {
        string bits;
        for (unsigned char c : text) {

```

```

        bits += code[c];
    }
    return bits;
}

string decode(const string &bits) const {
    string text;
    if (root == -1) {
        return text;
    }
    // Degenerate tree: the input had a single distinct character, so the root is itself a leaf and
    // each symbol was encoded as one bit. Walking children would dereference node -1.
    if (nodes[root].left == -1 && nodes[root].right == -1) {
        return string(bits.size(), static_cast<char>(nodes[root].ch));
    }
    int u = root;
    for (char bit : bits) {
        u = bit == '0' ? nodes[u].left : nodes[u].right;
        if (nodes[u].left == -1 && nodes[u].right == -1) {
            text.push_back(static_cast<char>(nodes[u].ch));
            u = root;
        }
    }
    return text;
}
};

```

## Example Usage

```

#include <cassert>

int main() {
    string text = "banana bandana";
    HuffmanTree h(text);
    string bits = h.encode(text);
    assert(h.decode(bits) == text);
    assert(bits.size() < text.size() * 8);

    HuffmanTree single("aaaa");
    assert(single.encode("aaaa") == "0000");
    assert(single.decode(single.encode("aaaa")) == "aaaa");

    HuffmanTree empty("");
    assert(empty.encode("") == "");
    assert(empty.decode("") == "");
    return 0;
}

```

### 3.8.5 Burrows-Wheeler Transform

The Burrows-Wheeler transform (BWT) permutes a string into one that tends to be far more compressible while remaining perfectly invertible. It writes down every cyclic rotation of the string, sorts them lexicographically, and reads off the last character of each sorted rotation. Characters that occur in similar contexts are brought together, producing long runs that run-length and move-to-front coders exploit; it is the heart of the bzip2 compressor and a building block of compressed-text indexes.

To make the permutation invertible, a unique sentinel character smaller than every real character is appended before transforming, so the rotations are all distinct and one of them is distinguished as the original. Sorting the rotations of the sentineled string is the same as sorting its suffixes, so the forward transform is computed with a prefix-doubling rotation sort. The inverse rebuilds the string with the last-to-first (LF) mapping: stably ordering

the transformed characters yields, for each position of the sorted first column, the row that supplies the next character, and following that mapping from the sentinel row walks the original string out one character at a time.

The input must not contain the sentinel character `'\0'`.

- `bwt(s)` returns the BWT of `s`. The result has length `s.size() + 1` and contains the sentinel `'\0'` exactly once.
- `inverse_bwt(code)` returns the original string, inverting `bwt`. The argument must be a string produced by that function.

#### Time Complexity:

- $O(n \log^2 n)$  per call to `bwt(s)`, where  $n$  is the length of `s`.
- $O(n \log n)$  per call to `inverse_bwt(code)`.

**Space Complexity:**  $O(n)$  auxiliary for both functions.

```
#include <algorithm>
#include <numeric>
#include <string>
#include <tuple>
#include <vector>

std::string bwt(std::string s) {
    s.push_back('\0'); // Unique sentinel, smaller than every real character.
    int n = static_cast<int>(s.size());
    // Sort the indices of the cyclic rotations by prefix doubling; with the sentinel this matches
    // sorting the suffixes.
    std::vector<int> order(n), rank(n), tmp(n);
    for (int i = 0; i < n; i++) {
        order[i] = i;
        rank[i] = static_cast<unsigned char>(s[i]);
    }
    for (int k = 1; k < n; k <= 1) {
        auto cmp = [&](int a, int b) {
            return std::tie(rank[a], rank[(a + k) % n]) < std::tie(rank[b], rank[(b + k) % n]);
        };
        std::sort(order.begin(), order.end(), cmp);
        tmp[order[0]] = 0;
        for (int i = 1; i < n; i++) {
            tmp[order[i]] = tmp[order[i - 1]] + (cmp(order[i - 1], order[i]) ? 1 : 0);
        }
        rank = tmp;
        if (rank[order[n - 1]] == n - 1) {
            break;
        }
    }
    std::string code(n, '\0');
    for (int i = 0; i < n; i++) {
        code[i] = s[(order[i] + n - 1) % n]; // Last character of the i-th sorted rotation.
    }
    return code;
}

std::string inverse_bwt(const std::string &code) {
    int n = static_cast<int>(code.size());
    // first[k] is the row supplying the first column's k-th character; following it from row 0 (the
    // sentinel row, sorted first) reconstructs the sentinel followed by the original string.
    std::vector<int> first(n);
    std::iota(first.begin(), first.end(), 0);
    std::stable_sort(first.begin(), first.end(), [&](int a, int b) {
        return static_cast<unsigned char>(code[a]) < static_cast<unsigned char>(code[b]);
    });
    std::string res;
    res.reserve(n);
    for (int i = 0, row = 0; i < n; i++) {
        row = first[row];
```

```

    res.push_back(code[row]);
}
return res.substr(1); // Drop the leading sentinel.
}

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

string escaped(const string &s) {
    string res;
    for (char c : s) {
        res += (c == '\\0') ? "\\0" : string(1, c);
    }
    return res;
}

int count_runs(const string &s) {
    int runs = 0;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        runs += i == 0 || s[i] != s[i - 1];
    }
    return runs;
}

int main() {
    string code = bwt("banana");
    assert(escaped(code) == "annb\\0aa");
    assert(inverse_bwt(code) == "banana");
    cout << "BWT(banana): " << escaped(code) << endl;
    cout << "inverse: " << inverse_bwt(code) << endl;

    // BWT does not compress by itself, but can cluster characters for a subsequent run-length coder.
    string text = "abracadabra abracadabra";
    string transformed = bwt(text);
    assert(inverse_bwt(transformed) == text);
    assert(count_runs(text) == 23);
    assert(count_runs(transformed) == 9);

    // Round-trips on assorted strings, including repeats and a single character.
    for (const string &s :
        {string(""), string("a"), string("aaaa"), string("abracadabra"),
         string("the quick brown fox"), string("\\1\\2abc", 5)}) {
        assert(inverse_bwt(bwt(s)) == s);
    }
    return 0;
}

```

### Example Output

```

BWT(banana): annb\0aa
inverse: banana

```

# Chapter 4

## Graphs

### 4.1 DFS and Trees

---

#### 4.1.1 Adjacency Lists and DFS

Stores a directed or undirected graph as adjacency lists. The list for node `u` contains its out-neighbors; an undirected edge is stored in both endpoints' lists. Nodes are nonnegative integer indices in  $[0, \text{size}())$ , and `add_edge()` grows the graph when either endpoint is new. The class also provides depth-first search, cycle detection, and forest/DAG checks.

Parallel edges and self-loops are stored as given, and the cycle checks read them as a multigraph would: a self-loop is a cycle, and a repeated undirected edge is a cycle of length two. The undirected search skips edges back to the parent by node index, so a duplicate is caught when the parent scans it rather than when the child looks back.

- `Graph(directed = true)` constructs an empty graph, directed if `directed` is true and undirected otherwise.
- `size()` returns the current number of nodes.
- `operator[u]` returns a mutable or const reference to the adjacency list (`std::vector<int>`) of node `u`.
- `add_edge(u, v)` adds an edge from `u` to `v`, plus the reverse edge if the graph is undirected, growing the node count to accommodate the larger index if necessary.
- `dfs(start, f)` runs a depth-first search from node `start`, calling function `f` on each node in the order it is first visited.
- `has_cycle()` returns whether the graph contains a cycle.
- `is_directed()` returns whether the graph is directed.
- `is_forest()` returns whether the graph is undirected and acyclic.
- `is_dag()` returns whether the graph is directed and acyclic.

#### Time Complexity:

- $O(1)$  amortized per call to `add_edge()` when no resize is required; up to  $O(n)$  for a call that grows the graph to  $n$  nodes.
- $O(\max(n, m))$  per call for `dfs()`, `has_cycle()`, `is_forest()`, or `is_dag()`, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(1)$  per call to all other public member functions.

#### Space Complexity:

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary stack space for `dfs()`, `has_cycle()`, `is_forest()`, and `is_dag()`.
- $O(1)$  auxiliary for all other public member functions.

```
#include <algorithm>
```

```

#include <vector>

class Graph {
    std::vector<std::vector<int>>> adj;
    bool directed;

public:
    explicit Graph(bool directed = true) : directed(directed) {}

    int size() const { return static_cast<int>(adj.size()); }
    std::vector<int> &operator[](int u) { return adj[u]; }
    const std::vector<int> &operator[](int u) const { return adj[u]; }

    void add_edge(int u, int v) {
        int n = static_cast<int>(adj.size());
        if (u >= n || v >= n) {
            adj.resize(std::max(u, v) + 1);
        }
        adj[u].push_back(v);
        if (!directed) {
            adj[v].push_back(u);
        }
    }

    template<typename Fn>
    void dfs(int start, Fn f) const {
        std::vector<char> visit(adj.size());
        auto dfs = [&](auto &&dfs, int u) -> void {
            f(u);
            visit[u] = true;
            for (int v : adj[u]) {
                if (!visit[v]) {
                    dfs(dfs, v);
                }
            }
        };
        dfs(dfs, start);
    }

    bool has_cycle() const {
        int n = static_cast<int>(adj.size());
        std::vector<char> visit(n), onstack(n);
        auto dfs = [&](auto &&dfs, int u, int p) -> bool {
            visit[u] = true;
            onstack[u] = true;
            for (int v : adj[u]) {
                if ((directed && onstack[v]) ||           // back edge in directed graph
                    (!directed && visit[v] && v != p) || // back edge in undirected graph
                    (!visit[v] && dfs(dfs, v, u))) {      // tree edge to unvisited node
                    return true;
                }
            }
            onstack[u] = false;
            return false;
        };
        for (int i = 0; i < n; i++) {
            if (!visit[i] && dfs(dfs, i, -1)) {
                return true;
            }
        }
        return false;
    }

    bool is_directed() const { return directed; }
    bool is_forest() const { return !directed && !has_cycle(); }
    bool is_dag() const { return directed && !has_cycle(); }
};

```



## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    //      0
    //    / / |
    //   v v v
    //  1 6 7
    // / |   / |
    // v v   v v
    // 2 5   8 11
    // / |   / |
    // v v   v v
    // 3 4   9 10
    Graph g;
    g.add_edge(0, 1);
    g.add_edge(0, 6);
    g.add_edge(0, 7);
    g.add_edge(1, 2);
    g.add_edge(1, 5);
    g.add_edge(2, 3);
    g.add_edge(2, 4);
    g.add_edge(7, 8);
    g.add_edge(7, 11);
    g.add_edge(8, 9);
    g.add_edge(8, 10);
    vector<int> order;
    g.dfs(0, [&](int u) { order.push_back(u); });
    assert((order == vector<int>{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}));
    assert(g.size() == 12 && g[0].size() == 3);
    assert(g.is_dag());
    assert(!g.has_cycle());

    //      0
    //    / |
    //   1 2
    // / |
    // 3 4
    Graph tree(false);
    tree.add_edge(0, 1);
    tree.add_edge(0, 2);
    tree.add_edge(1, 3);
    tree.add_edge(1, 4);
    assert(tree.is_forest());
    assert(!tree.is_dag());
    tree.add_edge(2, 3);
    assert(!tree.is_forest());
    return 0;
}
```

### 4.1.2 Topological Sort

Given a directed acyclic graph (DAG), find one of possibly many orderings of the nodes such that for every edge from node  $u$  to  $v$ ,  $u$  comes before  $v$  in the ordering.

The DFS version uses the fact that every outgoing dependency of a node is finished before the node itself is appended, so reversing DFS post-order places each prerequisite before everything it can reach. Kahn's algorithm instead repeatedly removes nodes with indegree zero: these are exactly the nodes whose remaining prerequisites have all already been placed. If at some point not every node can be removed, the leftover nodes must lie on a directed cycle.

- `toposort_dfs()` returns a valid topological ordering using DFS post-order, or an empty vector if the graph contains a cycle.
- `toposort_kahn()` returns a valid topological ordering using indegrees and a queue, or returns an empty vector if the graph contains a cycle.

**Time Complexity:**  $O(\max(n, m))$  per call to either function, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary stack and heap space for `toposort_dfs()`.
- $O(n)$  auxiliary heap space for the indegree array and queue in `toposort_kahn()`.

```
#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

std::vector<std::vector<int>> adj;

std::vector<int> toposort_dfs() {
    int n = static_cast<int>(adj.size());
    std::vector<char> state(n);
    std::vector<int> res;
    auto dfs = [&](auto &&dfs, int u) -> bool {
        if (state[u] == 1) {
            return false;
        }
        if (state[u] == 2) {
            return true;
        }
        state[u] = 1;
        for (int v : adj[u]) {
            if (!dfs(dfs, v)) {
                return false;
            }
        }
        state[u] = 2;
        res.push_back(u);
        return true;
    };
    for (int i = 0; i < n; i++) {
        if (state[i] == 0 && !dfs(dfs, i)) {
            return {};
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

std::vector<int> toposort_kahn() {
    int n = static_cast<int>(adj.size());
    std::vector<int> indeg(n), res;
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            indeg[v]++;
        }
    }
    std::queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) {
            q.push(i);
        }
    }
    while (!q.empty()) {
        int u = q.front();
```

```

    q.pop();
    res.push_back(u);
    for (int v : adj[u]) {
        if (--indeg[v] == 0) {
            q.push(v);
        }
    }
}
}
if (static_cast<int>(res.size()) != n) {
    return {};
}
return res;
}

```

## Example Usage

```

using namespace std;

int main() {
    //      1
    //      |
    //      v
    // 0 ----> 3 ----> 5
    // |      / |
    // v      v v
    // 4 -> 6 7 <--- 2
    // ^-----|
    adj.assign(8, {});
    adj[0].push_back(3);
    adj[0].push_back(4);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[2].push_back(7);
    adj[3].push_back(5);
    adj[3].push_back(6);
    adj[3].push_back(7);
    adj[4].push_back(6);
    vector<int> dfs_res = toposort_dfs();
    assert((dfs_res == vector<int>{2, 1, 0, 4, 3, 7, 6, 5}));
    vector<int> kahn = toposort_kahn();
    assert(dfs_res.size() == adj.size());
    assert(kahn.size() == adj.size());
    vector<int> position(kahn.size());
    for (int i = 0; i < static_cast<int>(kahn.size()); i++) {
        position[kahn[i]] = i;
    }
    for (int u = 0; u < static_cast<int>(adj.size()); u++) {
        for (int v : adj[u]) {
            assert(position[u] < position[v]);
        }
    }
    adj = {{1}, {0}};
    assert(toposort_dfs().empty());
    assert(toposort_kahn().empty());
    return 0;
}

```

### 4.1.3 Cycle Finding

Given a directed or undirected graph, find simple cycles encountered by a depth-first search. The free functions return one cycle of a simple graph as a list of nodes. For undirected graphs, an edge to a visited non-parent node

closes a cycle. For directed graphs, only an edge to a node still on the recursion stack closes a cycle; edges to already-finished nodes do not.

Both functions remember the DFS parent of each node. When a closing edge `u → cycle_start` is found, the answer is reconstructed by walking parents backward from `u` to `cycle_start`. In the directed version, `state[v]` is 0 for unvisited nodes, 1 for nodes currently on the recursion stack, and 2 for finished nodes.

- `find_cycle_undirected(adj)` returns one node cycle in a simple undirected graph, or an empty vector if no cycle exists.
- `find_cycle_directed(adj)` returns one node cycle in a simple directed graph, or an empty vector if no cycle exists.

**Time Complexity:**  $O(\max(n, m))$  per call to `find_cycle_undirected()` and `find_cycle_directed()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph.
- $O(n)$  auxiliary stack and heap space for the DFS, plus  $O(n)$  for the returned cycle.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

std::vector<int> find_cycle_undirected(const std::vector<std::vector<int>>& adj) {
    int n = static_cast<int>(adj.size());
    if (n == 0) {
        return {};
    }
    std::vector<char> visit(n);
    std::vector<int> parent(n, -1);
    int cycle_start = -1, cycle_end = -1;
    auto dfs = [&](auto &&dfs, int u, int p) -> bool {
        visit[u] = true;
        for (int v : adj[u]) {
            if (v == p) {
                continue;
            }
            if (visit[v]) {
                cycle_start = v;
                cycle_end = u;
                return true;
            }
            parent[v] = u;
            if (dfs(dfs, v, u)) {
                return true;
            }
        }
        return false;
    };
    for (int u = 0; u < n && cycle_start == -1; u++) {
        if (!visit[u]) {
            dfs(dfs, u, -1);
        }
    }
    if (cycle_start == -1) {
        return {};
    }
    std::vector<int> cycle{cycle_start};
    for (int u = cycle_end; u != cycle_start; u = parent[u]) {
        cycle.push_back(u);
    }
    std::reverse(cycle.begin() + 1, cycle.end());
    return cycle;
}
```

```

std::vector<int> find_cycle_directed(const std::vector<std::vector<int>>& adj) {
    int n = static_cast<int>(adj.size());
    if (n == 0) {
        return {};
    }
    std::vector<char> state(n); // 0 = unvisited, 1 = currently on DFS stack, 2 = finished.
    std::vector<int> parent(n, -1);
    int cycle_start = -1, cycle_end = -1;
    auto dfs = [&](auto &&dfs, int u) -> bool {
        state[u] = 1;
        for (int v : adj[u]) {
            if (state[v] == 0) {
                parent[v] = u;
                if (dfs(dfs, v)) {
                    return true;
                }
            } else if (state[v] == 1) {
                cycle_start = v;
                cycle_end = u;
                return true;
            }
        }
        state[u] = 2;
        return false;
    };
    for (int u = 0; u < n && cycle_start == -1; u++) {
        if (state[u] == 0) {
            dfs(dfs, u);
        }
    }
    if (cycle_start == -1) {
        return {};
    }
    std::vector<int> cycle{cycle_start};
    for (int u = cycle_end; u != cycle_start; u = parent[u]) {
        cycle.push_back(u);
    }
    std::reverse(cycle.begin() + 1, cycle.end());
    return cycle;
}

```

The `CycleFinder` class is the more general multigraph variant. Its cycles are returned as edge IDs rather than node lists, which makes the representation robust for parallel edges. In an undirected graph, the cycles found this way form a DFS cycle basis: one cycle for each non-tree edge. In a directed graph, this finds cycles created by back edges to nodes still on the recursion stack. Here `state[v]` is `-1` before discovery, `-2` after finishing, and otherwise the index where the node first appeared in the current edge stack; that index marks which stack suffix forms a cycle.

- `CycleFinder(n, directed)` constructs a graph of `n` nodes numbered `[0, n)`. The graph is directed if `directed` is true, or undirected otherwise.
- `add_edge(u, v)` adds an edge and returns its edge ID.
- `find_cycles(max_cycles, max_total_size)` returns simple cycles as vectors of edge IDs, stopping once either optional limit is reached.
- `cycle_nodes(edge_cycle)` converts one edge-ID cycle into a corresponding cyclic list of nodes.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the number of nodes.
- $O(1)$  per call to `add_edge()`.
- $O(\max(n, m + s))$  per call to `find_cycles()`, where  $n$  is the number of nodes,  $m$  is the number of edges, and  $s$  is the total size of the returned cycles.
- $O(k)$  per call to `cycle_nodes()`, where  $k$  is the number of edges in the cycle.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph.
- $O(n)$  auxiliary stack and heap space for `find_cycles()`, plus  $O(s)$  for the returned cycles, where  $s$  is their total size.
- $O(k)$  for the vector returned by `cycle_nodes()`, where  $k$  is the number of edges in the cycle.

```

class CycleFinder {
    std::vector<std::pair<int, int>>> edges;
    std::vector<std::vector<int>>> adj;
    bool directed;

public:
    CycleFinder(int n, bool directed) : adj(n), directed(directed) {}

    int add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        edges.emplace_back(u, v);
        adj[u].push_back(id);
        if (!directed) {
            adj[v].push_back(id);
        }
        return id;
    }

    std::vector<std::vector<int>>> find_cycles(
        int max_cycles = 1 << 30, int max_total_size = 1 << 30
    ) const {
        int n = static_cast<int>(adj.size());
        std::vector<int> state(n, -1), edge_stack;
        std::vector<std::vector<int>>> cycles;
        int total_size = 0;
        auto dfs = [&](auto &&dfs, int u, int parent_edge) {
            if (static_cast<int>(cycles.size()) >= max_cycles || total_size >= max_total_size) {
                return;
            }
            state[u] = static_cast<int>(edge_stack.size());
            for (int id : adj[u]) {
                if (!directed && id == parent_edge) {
                    continue;
                }
                const auto &[eu, ev] = edges[id];
                int v = directed ? ev : eu ^ ev ^ u;
                if (state[v] >= 0) {
                    std::vector<int> cycle{id};
                    for (int i = state[v]; i < static_cast<int>(edge_stack.size()); i++) {
                        cycle.push_back(edge_stack[i]);
                    }
                    cycles.push_back(cycle);
                    total_size += static_cast<int>(cycle.size());
                    if (static_cast<int>(cycles.size()) >= max_cycles || total_size >= max_total_size) {
                        return;
                    }
                }
                else if (state[v] == -1) {
                    edge_stack.push_back(id);
                    dfs(dfs, v, id);
                    edge_stack.pop_back();
                }
            }
            state[u] = -2;
        };
        for (int u = 0; u < n; u++) {
            if (state[u] == -1) {
                dfs(dfs, u, -1);
            }
        }
        return cycles;
    }
};

```

```

}

std::vector<int> cycle_nodes(const std::vector<int> &edge_cycle) const {
    int size = static_cast<int>(edge_cycle.size());
    std::vector<int> nodes;
    if (size == 0) {
        return nodes;
    }
    if (size <= 2) {
        nodes.push_back(edges[edge_cycle[0]].first);
        if (size == 2) {
            nodes.push_back(edges[edge_cycle[0]].second);
        }
        return nodes;
    }
    for (int i = 0; i < size; i++) {
        const auto &[au, av] = edges[edge_cycle[i]];
        const auto &[bu, bv] = edges[edge_cycle[(i + 1) % size]];
        nodes.push_back((bu == au || bv == au) ? av : au);
    }
    return nodes;
}
};

```

## Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    {
        // 0---1    3---4
        // |  /
        // | /
        // 2
        vector<vector<int>> adj(5);
        auto add_edge = [&](int u, int v) {
            adj[u].push_back(v);
            adj[v].push_back(u);
        };
        add_edge(0, 1);
        add_edge(1, 2);
        add_edge(2, 0);
        add_edge(3, 4);
        // DFS encounters the triangle component before the acyclic 3-4 component.
        assert((find_cycle_undirected(adj) == vector<int>{0, 1, 2}));
    }
    {
        // 0 --> 1 --> 2    3 --> 4 --> 5
        //                ^         |
        //                |_____|
        vector<vector<int>> adj(6);
        adj[0].push_back(1);
        adj[1].push_back(2);
        adj[3].push_back(4);
        adj[4].push_back(5);
        adj[5].push_back(3);
        assert((find_cycle_directed(adj) == vector<int>{3, 4, 5}));
        assert(find_cycle_directed({{1}, {2}, {}}).empty());
    }
    {
        // 0---a---1
        // |      /
        // c    b
        // |  /

```

```

// 2-----3
CycleFinder g(4, false);
int a = g.add_edge(0, 1);
int b = g.add_edge(1, 2);
int c = g.add_edge(2, 0);
g.add_edge(2, 3);
vector<vector<int>> cycles = g.find_cycles();
assert(cycles.size() == 1);
sort(cycles[0].begin(), cycles[0].end());
assert((cycles[0] == vector<int>{a, b, c}));
assert((g.cycle_nodes(cycles[0]) == vector<int>{0, 1, 2}));
}
{
// 0---a---1
// 0---b---1
CycleFinder g(2, false);
int a = g.add_edge(0, 1);
int b = g.add_edge(0, 1);
// Parallel undirected edges form a 2-edge cycle; edge IDs make this representable.
vector<vector<int>> cycles = g.find_cycles();
assert(cycles.size() == 1);
sort(cycles[0].begin(), cycles[0].end());
assert((cycles[0] == vector<int>{a, b}));
}
{
// 0 ---> 1
// ^      /
// |      /
// 2 <-+
CycleFinder g(3, true);
g.add_edge(0, 1);
g.add_edge(1, 2);
g.add_edge(2, 0);
assert(g.find_cycles(1).size() == 1);
}
return 0;
}

```

#### 4.1.4 Tree Centers, Centroid, and Diameter

An unweighted tree has two useful notions of its middle and one of its extent. Jordan centers optimize distances, centroids balance component sizes, and a diameter identifies the farthest pair of nodes; a tree's centers and centroids need not coincide.

Repeatedly removing all leaves reveals the centers. Subtree sizes locate a centroid. A diameter is found by first finding the farthest node from any start, then traversing again from that endpoint to find the other.

- `find_centers()` returns the one or two Jordan centers: the nodes minimizing the maximum distance to every other node.
- `find_centroid()` returns a centroid: a node whose removal leaves every connected component with at most  $n/2$  nodes.
- `diameter()` returns `(length, u, v)`, where `length` is the maximum distance between any two nodes and `u` and `v` are endpoints attaining it.

The functions use a global, bidirectionally pre-populated adjacency list `adj`, which must form a nonempty tree with nodes numbered  $[0, n)$ , where  $n$  is `adj.size()`.

**Time Complexity:**  $O(n)$  per call to `find_centers()`, `find_centroid()`, and `diameter()`.

**Space Complexity:**

- $O(n)$  auxiliary heap space for `find_centers()`.
- $O(n)$  auxiliary stack space for `find_centroid()` and `diameter()`.



```

#include <algorithm>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::vector<int>>> adj;

std::vector<int> find_centers() {
    int n = static_cast<int>(adj.size());
    std::vector<int> leaves, degree(n);
    for (int i = 0; i < n; i++) {
        degree[i] = static_cast<int>(adj[i].size());
        if (degree[i] <= 1) {
            leaves.push_back(i);
        }
    }
    int removed = static_cast<int>(leaves.size());
    while (removed < n) {
        std::vector<int> nleaves;
        for (int u : leaves) {
            for (int v : adj[u]) {
                if (--degree[v] == 1) {
                    nleaves.push_back(v);
                }
            }
        }
        leaves = nleaves;
        removed += static_cast<int>(leaves.size());
    }
    return leaves;
}

// Returns the centroid node index if found in this subtree, or -(subtree size) to propagate the
// size up to the parent so it can check the complementary component's size.
int find_centroid(int u = 0, int p = -1) {
    int n = static_cast<int>(adj.size());
    int count = 1;
    bool good_center = true;
    for (int v : adj[u]) {
        if (v == p) {
            continue;
        }
        int res = find_centroid(v, u);
        if (res >= 0) {
            return res;
        }
        int size = -res;
        good_center &= (size <= n / 2);
        count += size;
    }
    good_center &= (n - count <= n / 2);
    return good_center ? u : -count;
}

std::tuple<int, int, int> diameter() {
    auto dfs = [&](auto &&dfs, int u, int p = -1, int depth = 0) -> std::pair<int, int> {
        std::pair<int, int> res{depth, u};
        for (int v : adj[u]) {
            if (v != p) {
                res = std::max(res, dfs(dfs, v, u, depth + 1));
            }
        }
        return res;
    };
    int u = dfs(dfs, 0).second;
    auto [length, v] = dfs(dfs, u);
    return {length, u, v};
}

```

## Example Usage

```
#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    // 0---1---4---3
    //   |   |
    //   2   5
    adj.assign(6, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(1, 4);
    add_edge(3, 4);
    add_edge(4, 5);
    assert((find_centers() == vector<int>{1, 4}));
    assert(find_centroid() == 4);
    auto [length, u, v] = diameter();
    assert(length == 3 && u == 5 && v == 2);
    return 0;
}
```

### 4.1.5 Subtree Small-to-Large Merging

Given a rooted tree, compute a mutable summary for every subtree by merging child summaries into one kept summary. Small-to-large merging always keeps the largest child summary and absorbs smaller summaries into it. Each stored item can move only into summaries at least twice as large as before, so the total number of moved items is logarithmic per item.

This pattern is often called DSU on tree, although it is unrelated to the disjoint-set union data structure used by Kruskal’s algorithm. It is useful for subtree queries whose answer can be maintained by inserting all items from child summaries into one mutable summary, such as distinct colors, frequency modes, or other multiset-style statistics.

- `subtree_small_to_large_merge(adj, root, init, merge, answer)` visits the tree rooted at `root`, given by the bidirectional adjacency list `adj`, which must form a valid tree. The callback `init(u)` returns the initial summary for node `u`; `merge(big, small)` absorbs one child summary into another; and `answer(u, summary)` records the final subtree summary for `u`.

Summary objects must support `.size()`, and `merge(big, small)` should iterate over `small` and insert into `big`.

**Time Complexity:**  $O(n \log n)$  expected per call when `merge()` does expected  $O(1)$  work per moved item, where  $n$  is the number of nodes.

**Space Complexity:**  $O(n)$  auxiliary stack space and  $O(n \log n)$  auxiliary heap space in the worst case, since absorbed summaries remain stored.

```
#include <cassert>
#include <unordered_map>
#include <vector>

template<typename Init, typename Merge, typename Answer>
void subtree_small_to_large_merge(
    const std::vector<std::vector<int>> &adj, int root, Init init, Merge merge, Answer answer
) {
    int n = static_cast<int>(adj.size());
    assert(0 <= root && root < n);
```

```

std::vector<decltype(init(0))> summary(n);
auto dfs = [&](auto &&dfs, int u, int p) -> int {
    int keep = u;
    summary[u] = init(u);
    for (int v : adj[u]) {
        if (v == p) {
            continue;
        }
        int child = dfs(dfs, v, u);
        if (summary[keep].size() < summary[child].size()) {
            std::swap(keep, child);
        }
        merge(summary[keep], summary[child]);
    }
    answer(u, summary[keep]);
    return keep;
};
dfs(dfs, root, -1);
}

```

### Example Usage

```

using namespace std;

int main() {
    //          0:A
    //         /  |
    //        1:B  2:A
    //       / |  |
    //      3:C 4:B 5:C
    vector<vector<int>> adj(6);
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    add_edge(0, 1);
    add_edge(0, 2);
    add_edge(1, 3);
    add_edge(1, 4);
    add_edge(2, 5);
    vector<int> color{0, 1, 0, 2, 1, 2};
    vector<int> ans(6);
    using Summary = unordered_map<int, int>;
    auto init = [&](int u) {
        Summary freq;
        freq[color[u]] = 1;
        return freq;
    };
    auto merge = [](Summary &big, const Summary &small) {
        for (const auto &[c, cnt] : small) {
            big[c] += cnt;
        }
    };
    auto answer = [&](int u, const Summary &freq) { ans[u] = static_cast<int>(freq.size()); };
    subtree_small_to_large_merge(adj, 0, init, merge, answer);
    assert((ans == vector<int>{3, 2, 2, 1, 1, 1}));
    return 0;
}

```

### 4.1.6 Tree Rerooting DP

Given a DP on a rooted tree, compute its result for every choice of root. For each direction  $u \rightarrow v$  of an edge  $u-v$ , removing the edge leaves a component containing  $u$ ; rerooting DP summarizes that component as a contribution

from  $u$  to  $v$ . After rooting the tree arbitrarily, a bottom-up pass computes contributions toward the initial root, and a top-down pass computes contributions in the opposite direction. Prefix/suffix aggregation combines all contributions except the one from a chosen neighbor in  $O(1)$  time. This requires `combine` to be associative and gives an  $O(n)$  algorithm; a segment-tree/exclusive-combine variant can instead support a left-fold style interface in  $O(n \log n)$ .

- `rerooting_dp(identity, combine, finalize, lift)` returns the finalized aggregate for every node when chosen as the root. The global, bidirectionally pre-populated adjacency list `adj` must form a nonempty tree whose indices represent its nodes.

The aggregation scheme for `rerooting_dp()` is defined by the following pieces:

- `Summary` is the type used for component aggregates and neighbor contributions. A contribution from  $u$  to  $v$  summarizes the component containing  $u$  after removing edge  $u-v$ , transformed so it can be combined at  $v$ .
- `identity` is the neutral value for an empty component, which for all aggregate values `a` must satisfy `combine(a, identity) = combine(identity, a) = a`.
- `combine(a, b)` merges two independent neighbor contributions expressed at the same node.
- `finalize(acc, u)` incorporates node `u` after its available neighbor contributions have been merged.
- `lift(acc, u, v)` transfers a finalized aggregate at node `u` across edge `u-v`, producing a contribution expressed at adjacent node `v`.

This covers problems such as subtree sizes, sum/max of distances from every root, and rerooted polynomial/hash-like aggregates, as long as sibling contributions can be combined associatively. The usage example shows how to compute the sum of distances from every root: `Summary` stores `sum`, the total distance from the current boundary node to all nodes in the summarized component, and `nodes`, the number of nodes in that component. The node count is needed because `lift(acc, u, v)` crosses one edge, so every represented node becomes 1 farther from `v` and `sum` increases by `nodes`.

**Time Complexity:**  $O(n)$  per call to `rerooting_dp()`, assuming  $O(1)$  callback functions.

**Space Complexity:**

- $O(n)$  for the tree and auxiliary arrays.
- $O(n)$  auxiliary stack space for the recursive search.

```
#include <cstdint>
#include <vector>

std::vector<std::vector<int>>> adj;

template<typename Summary, typename Combine, typename Finalize, typename Lift>
std::vector<Summary> rerooting_dp(
    const Summary &identity, Combine combine, Finalize finalize, Lift lift)
{
    int n = static_cast<int>(adj.size());
    std::vector<int> parent(n, -1), order;
    auto dfs = [&](auto &&dfs, int u, int p) -> void {
        parent[u] = p;
        order.push_back(u);
        for (int v : adj[u]) {
            if (v != p) {
                dfs(dfs, v, u);
            }
        }
    };
    dfs(dfs, 0, -1);
    std::vector<Summary> down(n, identity), outside(n, identity), answer(n, identity);
    for (int i = n - 1; i >= 0; i--) {
        int u = order[i];
        Summary acc = identity;
        for (int v : adj[u]) {
            if (parent[v] == u) {
                acc = combine(acc, lift(down[v], v, u));
            }
        }
    }
}
```

```

    }
    down[u] = finalize(acc, u);
}
for (int u : order) {
    std::vector<Summary> vals;
    for (int v : adj[u]) {
        if (parent[v] == u) {
            vals.push_back(lift(down[v], v, u));
        } else {
            vals.push_back(outside[u]);
        }
    }
    std::vector<Summary> suffix(vals.size() + 1, identity);
    for (int i = static_cast<int>(vals.size()) - 1; i >= 0; i--) {
        suffix[i] = combine(vals[i], suffix[i + 1]);
    }
    Summary prefix = identity;
    for (int i = 0; i < static_cast<int>(adj[u].size()); i++) {
        int v = adj[u][i];
        if (parent[v] == u) {
            Summary without_child = finalize(combine(prefix, suffix[i + 1]), u);
            outside[v] = lift(without_child, u, v);
        }
        prefix = combine(prefix, vals[i]);
    }
    answer[u] = finalize(prefix, u);
}
return answer;
}

```

## Example Usage

```

#include <cassert>
#include <string>
using namespace std;

vector<int64_t> sum_distances_all_roots() {
    struct DistanceSummary {
        int64_t sum = 0;
        int nodes = 0;
    };
    auto combine = [](DistanceSummary a, DistanceSummary b) {
        return DistanceSummary{a.sum + b.sum, a.nodes + b.nodes};
    };
    auto finalize = [](DistanceSummary a, int) {
        a.nodes++;
        return a;
    };
    auto lift = [](DistanceSummary a, int, int) {
        a.sum += a.nodes;
        return a;
    };
    vector<DistanceSummary> dp = rerooting_dp(DistanceSummary{}, combine, finalize, lift);
    vector<int64_t> answer(dp.size());
    for (int i = 0; i < static_cast<int>(dp.size()); i++) {
        answer[i] = dp[i].sum;
    }
    return answer;
}

int main() {
    // 0---1---2
    //   |
    //   3---4
    adj = {{1}, {0, 2, 3}, {1}, {1, 4}, {3}};
    assert((sum_distances_all_roots() == vector<int64_t>{8, 5, 8, 6, 9}));
}

```

```
// Associative contributions retain adjacency-list order even when combine is not commutative.
adj = {{1}, {2, 0}, {1}}; // adj[1] is [2, 0], while 0 is its initial parent.
auto combine = [](const string &a, const string &b) { return a + b; };
auto finalize = [](const string &a, int u) { return a + to_string(u); };
auto lift = [](const string &a, int, int) { return a; };
assert(rerooting_dp(string{}, combine, finalize, lift)[1] == "201");
return 0;
}
```

### 4.1.7 Centroid Decomposition

Given a tree, build its centroid decomposition. A centroid is a node whose removal splits the current subtree into connected components of size at most half of that subtree. Recursively choosing centroids creates a decomposition tree of height  $O(\log n)$ , which is useful for queries based on distances to marked nodes, nearest special node, and other “path through a centroid” problems.

- `centroid_decomposition()` uses the tree in the global bidirectional adjacency list `adj` and returns a vector `parent`, where `parent[u]` is the parent of node `u` in the centroid tree, or  $-1$  if `u` is the root centroid.

**Time Complexity:**  $O(n \log n)$  for construction, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(n)$  for storage of the centroid tree.
- $O(n)$  auxiliary stack space for the recursive searches.

```
#include <vector>

std::vector<std::vector<int>> adj;

std::vector<int> centroid_decomposition() {
    int n = static_cast<int>(adj.size());
    std::vector<int> subtree_size(n), parent(n, -1);
    std::vector<char> removed(n);
    auto get_size = [&](auto &&get_size, int u, int p) -> int {
        subtree_size[u] = 1;
        for (int v : adj[u]) {
            if (v != p && !removed[v]) {
                subtree_size[u] += get_size(get_size, v, u);
            }
        }
        return subtree_size[u];
    };
    auto get_centroid = [&](auto &&get_centroid, int u, int p, int total) -> int {
        for (int v : adj[u]) {
            if (v != p && !removed[v] && subtree_size[v] > total / 2) {
                return get_centroid(get_centroid, v, u, total);
            }
        }
        return u;
    };
    auto decompose = [&](auto &&decompose, int entry, int p) -> void {
        int total = get_size(get_size, entry, -1);
        int centroid = get_centroid(get_centroid, entry, -1, total);
        parent[centroid] = p;
        removed[centroid] = true;
        for (int v : adj[centroid]) {
            if (!removed[v]) {
                decompose(decompose, v, centroid);
            }
        }
    };
    if (n > 0) {
```

```

    decompose(decompose, 0, -1);
}
return parent;
}

```

## Example Usage

```

#include <algorithm>
#include <cassert>
#include <climits>
using namespace std;

// Example-only distance helper; use LCA preprocessing for O(1) distances in a full solution.
int tree_dist(int u, int target, int p = -1) {
    if (u == target) {
        return 0;
    }
    for (int v : adj[u]) {
        if (v != p) {
            int d = tree_dist(v, target, u);
            if (d != -1) {
                return d + 1;
            }
        }
    }
    return -1;
}

int main() {
    // 0---1---3---5---6
    //   |   |
    //   2   4
    adj = {{1}, {0, 2, 3}, {1}, {1, 4, 5}, {3}, {3, 6}, {5}};
    vector<int> parent = centroid_decomposition();
    assert((parent == vector<int>{1, 3, 1, -1, 3, 3, 5}));

    const int INF = INT_MAX / 2;
    vector<int> best(7, INF);
    auto mark = [&](int u) {
        for (int c = u; c != -1; c = parent[c]) {
            best[c] = min(best[c], tree_dist(u, c));
        }
    };
    auto nearest_marked = [&](int u) {
        int answer = INF;
        for (int c = u; c != -1; c = parent[c]) {
            answer = min(answer, best[c] + tree_dist(u, c));
        }
        return answer;
    };
    mark(0);
    mark(6);
    assert(nearest_marked(2) == 2);
    assert(nearest_marked(4) == 3);
    assert(nearest_marked(5) == 1);
    return 0;
}

```

### 4.1.8 Prufer Code

Encode and decode labeled trees using Prufer codes. A Prufer code is a sequence of length  $n - 2$  that uniquely represents a labeled tree on nodes  $0, 1, \dots, n - 1$ . It is useful for counting labeled trees, generating test cases, and

converting between trees and compact sequences. Encoding repeatedly removes a leaf and records its neighbor until two nodes remain; decoding reverses this, deducing each removed leaf from the node degrees implied by the code. Both functions choose the smallest available leaf at each step. This makes the implementation deterministic and matches the usual textbook convention.

- `encode_prufer()` returns the Prufer code for the global, bidirectionally pre-populated adjacency list `adj`, which must form a valid tree on at least two nodes and whose indices represent the nodes. The empty code represents the unique tree on two nodes.
- `decode_prufer()` takes a Prufer code and returns the corresponding tree edges.

**Time Complexity:**  $O(n \log n)$  per call to `encode_prufer()` or `decode_prufer()`, where  $n$  is the number of nodes in the tree.

**Space Complexity:**  $O(n)$  auxiliary for `encode_prufer()` and `decode_prufer()`.

```
#include <set>
#include <utility>
#include <vector>

std::vector<int> encode_prufer(const std::vector<std::vector<int>> &adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> degree(n), parent(n, -1), code;
    std::set<int> leaves;
    if (n <= 2) {
        return code;
    }
    // Root at node n - 1, which is guaranteed to survive smallest-leaf removal (it is never the
    // smallest leaf while other nodes remain). Rooting elsewhere risks recording parent[root] == -2
    // if the chosen root itself gets stripped to a leaf (e.g. when n - 1 happens to be a leaf).
    int root = n - 1;
    for (int u = 0; u < n; u++) {
        degree[u] = static_cast<int>(adj[u].size());
        if (degree[u] == 1) {
            leaves.insert(u);
        }
    }
    std::vector<int> st(1, root);
    parent[root] = -2;
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : adj[u]) {
            if (parent[v] == -1) {
                parent[v] = u;
                st.push_back(v);
            }
        }
    }
    for (int i = 0; i < n - 2; i++) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());
        int p = parent[leaf];
        code.push_back(p);
        if (--degree[p] == 1) {
            leaves.insert(p);
        }
    }
    return code;
}

std::vector<std::pair<int, int>> decode_prufer(const std::vector<int> &code) {
    int n = static_cast<int>(code.size()) + 2;
    std::vector<int> degree(n, 1);
    std::set<int> leaves;
    std::vector<std::pair<int, int>> edges;
    for (int c : code) {

```



```

    degree[c]++;
}
for (int u = 0; u < n; u++) {
    if (degree[u] == 1) {
        leaves.insert(u);
    }
}
for (int u : code) {
    int leaf = *leaves.begin();
    leaves.erase(leaves.begin());
    edges.emplace_back(leaf, u);
    if (--degree[u] == 1) {
        leaves.insert(u);
    }
}
edges.emplace_back(*leaves.begin(), *leaves.rbegin());
return edges;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      0
    //      |
    // 1---3---2
    //      |
    //      4
    vector<vector<int>> adj(5);
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    add_edge(0, 3);
    add_edge(1, 3);
    add_edge(2, 3);
    add_edge(3, 4);
    vector<int> code = encode_prufer(adj);
    assert((code == vector<int>{3, 3, 3}));
    assert((decode_prufer(code) == vector<pair<int, int>>{{0, 3}, {1, 3}, {2, 3}, {3, 4}}));
    return 0;
}

```

### 4.1.9 Eulerian Trails (Hierholzer)

Provides compact directed-graph functions and a general edge-ID class for finding Eulerian trails in directed or undirected multigraphs. An Eulerian trail is a path which contains every edge exactly once; it is an Eulerian cycle or circuit when it begins and ends on the same node.

A directed graph has an Eulerian trail when all nonzero-degree nodes belong to one connected part of the underlying graph, and either every node has equal in-degree and out-degree or exactly one node has one extra outgoing edge and exactly one node has one extra incoming edge.

Hierholzer's algorithm walks unused edges until stuck, then backtracks to splice each closed detour into the final trail. For a directed graph known to have a trail from `start`, the core algorithm can simply consume outgoing edges by popping them from a local copy of the adjacency list.

- `known_eulerian_path_directed(adj, start)` returns a node trail using every directed edge in `adj` exactly once, assuming such a trail exists and begins at `start`.

- `eulerian_path_directed(adj, start = -1)` returns a node trail using every directed edge in `adj` exactly once, or an empty vector if no such trail exists. If `start = -1`, a valid start is chosen automatically; otherwise the trail must begin at `start`.

Parallel directed edges are supported by storing duplicate neighbors in `adj`. Since these functions return only nodes, use `EulerianGraph` below when edge IDs are needed.

**Time Complexity:**  $O(\max(n, m))$  per call to `known_eulerian_path_directed()` and `eulerian_path_directed()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(\max(n, m))$  auxiliary for `known_eulerian_path_directed()` and `eulerian_path_directed()`.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

std::vector<int> known_eulerian_path_directed(std::vector<std::vector<int>> adj, int start) {
    assert(0 <= start && start < static_cast<int>(adj.size()));
    std::vector<int> st{start}, path;
    while (!st.empty()) {
        int u = st.back();
        if (adj[u].empty()) {
            path.push_back(u);
            st.pop_back();
        } else {
            st.push_back(adj[u].back());
            adj[u].pop_back();
        }
    }
    std::reverse(path.begin(), path.end());
    return path;
}

std::vector<int> eulerian_path_directed(const std::vector<std::vector<int>> &adj, int start = -1) {
    int n = static_cast<int>(adj.size()), m = 0;
    if (n == 0) {
        return {};
    }
    assert(-1 <= start && start < n);
    std::vector<int> indeg(n), outdeg(n);
    for (int u = 0; u < n; u++) {
        outdeg[u] = static_cast<int>(adj[u].size());
        m += outdeg[u];
        for (int v : adj[u]) {
            indeg[v]++;
        }
    }
    if (m == 0) {
        return {start == -1 ? 0 : start};
    }
    int source = -1, sources = 0, sinks = 0;
    for (int u = 0; u < n; u++) {
        int diff = outdeg[u] - indeg[u];
        if (diff == 1) {
            source = u;
            sources++;
        } else if (diff == -1) {
            sinks++;
        } else if (diff != 0) {
            return {};
        }
    }
    if (sources != sinks || sources > 1) {
        return {};
    }
    if (start == -1) {
```

```

    if (source != -1) {
        start = source;
    } else {
        for (int u = 0; u < n && start == -1; u++) {
            if (outdeg[u] > 0) {
                start = u;
            }
        }
    }
} else if ((source != -1 && start != source) || (source == -1 && outdeg[start] == 0 && m > 0)) {
    return {};
}
std::vector<int> path = known_eulerian_path_directed(adj, start);
return static_cast<int>(path.size()) == m + 1 ? path : std::vector<int>();
}

```

The `EulerianGraph` class is the more general edge-ID variant. An undirected graph has an Eulerian trail when all nonzero-degree nodes are connected and either zero or two nodes have odd degree. This class stores and returns edge IDs, which supports directed graphs, undirected graphs, and multigraphs: parallel edges are distinct because each edge receives its own ID.

- `EulerianGraph(n, directed)` constructs a graph of  $n$  nodes numbered  $[0, n)$ . The graph is directed if `directed` is true, or undirected otherwise.
- `add_edge(u, v)` adds an edge and returns its edge ID.
- `eulerian_path(start = -1)` returns an `EulerianTrail` containing every edge exactly once, or an invalid result with `start = -1` if no such trail exists. In a valid result, `start` is the first node, `edges` lists edge IDs in traversal order, and `nodes` lists the visited nodes in order. If the input `start = -1`, a valid start is chosen automatically. For a valid result, `is_cycle()` returns whether the valid trail uses at least one edge and begins and ends on the same node.

**Time Complexity:**  $O(\max(n, m))$  per call to `eulerian_path()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(\max(n, m))$  for graph storage and auxiliary arrays.

```

class EulerianGraph {
    std::vector<std::pair<int, int>> edges;
    std::vector<std::vector<int>> adj;
    bool directed;

    int select_start(int start) const {
        int n = static_cast<int>(adj.size());
        if (directed) {
            std::vector<int> indeg(n), outdeg(n);
            for (const auto &[eu, ev] : edges) {
                outdeg[eu]++;
                indeg[ev]++;
            }
            int source = -1, sources = 0, sinks = 0, first = -1;
            for (int u = 0; u < n; u++) {
                if (outdeg[u] > 0 && first == -1) {
                    first = u;
                }
                int diff = outdeg[u] - indeg[u];
                if (diff == 1) {
                    source = u;
                    sources++;
                } else if (diff == -1) {
                    sinks++;
                } else if (diff != 0) {
                    return -1;
                }
            }
            if (sources != sinks || sources > 1) {

```

```

        return -1;
    }
    if (start != -1) {
        return source == -1 ? (outdeg[start] > 0 ? start : -1) : (start == source ? start : -1);
    }
    return source == -1 ? first : source;
}

std::vector<int> degree(n);
for (const auto &[eu, ev] : edges) {
    degree[eu]++;
    degree[ev]++;
}

std::vector<int> odd;
int first = -1;
for (int u = 0; u < n; u++) {
    if (degree[u] > 0 && first == -1) {
        first = u;
    }
    if (degree[u] % 2 == 1) {
        odd.push_back(u);
    }
}

if (!odd.empty() && odd.size() != 2) {
    return -1;
}

if (start != -1) {
    if (odd.empty()) {
        return degree[start] > 0 ? start : -1;
    }
    return start == odd[0] || start == odd[1] ? start : -1;
}
return odd.empty() ? first : odd[0];
}

std::vector<int> build_nodes(int start, const std::vector<int> &trail_edges) const {
    std::vector<int> nodes{start};
    int u = start;
    for (int id : trail_edges) {
        const auto &[eu, ev] = edges[id];
        u = directed ? ev : eu ^ ev ^ u;
        nodes.push_back(u);
    }
    return nodes;
}

public:
    struct EulerianTrail {
        int start = -1;
        std::vector<int> edges, nodes;

        bool is_cycle() const {
            return start != -1 && nodes.size() > 1 && nodes.front() == nodes.back();
        }
    };

    EulerianGraph(int n, bool directed) : adj(n), directed(directed) {}

    int add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        edges.emplace_back(u, v);
        adj[u].push_back(id);
        if (!directed) {
            adj[v].push_back(id);
        }
        return id;
    }

    EulerianTrail eulerian_path(int start = -1) const {

```

```

int n = static_cast<int>(adj.size());
int m = static_cast<int>(edges.size());
if (n == 0) {
    return EulerianTrail{};
}
assert(-1 <= start && start < n);
if (m == 0) {
    int s = (start == -1 ? 0 : start);
    return EulerianTrail{s, {}, {s}};
}
start = select_start(start);
if (start == -1) {
    return EulerianTrail{};
}
std::vector<char> used(m);
std::vector<int> ptr(n), node_stack{start}, edge_stack{-1}, trail_edges;
while (!node_stack.empty()) {
    int u = node_stack.back();
    while (ptr[u] < static_cast<int>(adj[u].size()) && used[adj[u][ptr[u]]]) {
        ptr[u]++;
    }
    if (ptr[u] == static_cast<int>(adj[u].size())) {
        if (edge_stack.back() != -1) {
            trail_edges.push_back(edge_stack.back());
        }
        node_stack.pop_back();
        edge_stack.pop_back();
    } else {
        int id = adj[u][ptr[u]++];
        used[id] = true;
        const auto &[eu, ev] = edges[id];
        int v = directed ? ev : eu ^ ev ^ u;
        node_stack.push_back(v);
        edge_stack.push_back(id);
    }
}
if (static_cast<int>(trail_edges.size()) != m) {
    return EulerianTrail{};
}
std::reverse(trail_edges.begin(), trail_edges.end());
return EulerianTrail{start, trail_edges, build_nodes(start, trail_edges)};
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        // 3 <-- 0 --> 1
        //      ^   |
        //      \   v
        //      +--2
        vector<vector<int>> adj(4);
        adj[0].push_back(1);
        adj[1].push_back(2);
        adj[2].push_back(0);
        adj[0].push_back(3);
        assert((known_eulerian_path_directed(adj, 0) == vector<int>{0, 1, 2, 0, 3}));
        assert((eulerian_path_directed(adj) == vector<int>{0, 1, 2, 0, 3}));
        assert(eulerian_path_directed(adj, 1).empty());
    }
    {
        // 0 <--- 2
    }
}

```

```

// |      ^
// |      /
// v /
// 1 ----> 3 ----> 4
// ^-----|
EulerianGraph g(5, true);
g.add_edge(0, 1);
g.add_edge(1, 2);
g.add_edge(2, 0);
g.add_edge(1, 3);
g.add_edge(3, 4);
g.add_edge(4, 1);
auto trail = g.eulerian_path(0);
assert((trail.nodes == vector<int>{0, 1, 3, 4, 1, 2, 0}) && trail.is_cycle());
}
{
// 0 ---- 2
// |      /
// |      /
// | /
// 1 ---- 3 ---- 4
// |-----|
EulerianGraph g(5, false);
g.add_edge(0, 1);
g.add_edge(1, 2);
g.add_edge(2, 0);
g.add_edge(1, 3);
g.add_edge(3, 4);
g.add_edge(4, 1);
auto trail = g.eulerian_path();
vector<int> used_edges = trail.edges;
sort(used_edges.begin(), used_edges.end());
assert((used_edges == vector<int>{0, 1, 2, 3, 4, 5}) && trail.is_cycle());
}
{
EulerianGraph g(2, false);
int a = g.add_edge(0, 1);
int b = g.add_edge(0, 1);
auto trail = g.eulerian_path(0);
assert(trail.is_cycle() && (trail.nodes == vector<int>{0, 1, 0}));
assert((trail.edges == vector<int>{a, b} || trail.edges == vector<int>{b, a}));
}
{
EulerianGraph g(0, false);
assert(g.eulerian_path().start == -1);
}
{
EulerianGraph g(3, true);
g.add_edge(0, 1);
g.add_edge(1, 2);
auto trail = g.eulerian_path();
assert(trail.start == 0 && (trail.nodes == vector<int>{0, 1, 2}) && !trail.is_cycle());
}
return 0;
}

```

## 4.2 Connectivity

---

### 4.2.1 Connected Components

Given an undirected graph, label every node with its connected component and collect the nodes in each component. This is the standard graph reachability primitive behind many larger routines: run one depth-first

search from every unseen node, and all nodes visited by that search belong to the same component. For directed graphs, use a strongly connected components algorithm instead.

- `connected_components()` populates the global arrays `comp_id` (component ID for each node) and `components` (nodes for each component ID) given a global, bidirectionally pre-populated adjacency list `adj` whose indices represent the nodes.

**Time Complexity:**  $O(\max(n, m))$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary stack space.

```
#include <vector>

std::vector<std::vector<int>>> adj;
std::vector<std::vector<int>>> components;
std::vector<int> comp_id;

void dfs(int u, int id) {
    comp_id[u] = id;
    components[id].push_back(u);
    for (int v : adj[u]) {
        if (comp_id[v] == -1) {
            dfs(v, id);
        }
    }
}

void connected_components() {
    int n = static_cast<int>(adj.size());
    components.clear();
    comp_id.assign(n, -1);
    for (int u = 0; u < n; u++) {
        if (comp_id[u] == -1) {
            components.emplace_back();
            dfs(u, static_cast<int>(components.size() - 1));
        }
    }
}
```

## Example Usage

```
#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    // 0---1---2    3---4
    adj.assign(6, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(3, 4);
    connected_components();
    assert((components == vector<vector<int>>>{{0, 1, 2}, {3, 4}, {5}}));
    assert(comp_id[0] == comp_id[2]);
    assert(comp_id[0] != comp_id[3]);
    assert(comp_id[5] == 2);
    return 0;
}
```

### 4.2.2 Strongly Connected Components (Kosaraju)

Given a directed graph, determine the strongly connected components (SCCs) using the Kosaraju-Sharir algorithm. A strongly connected component is a maximal set of nodes where every node can reach every other node. Condensing each SCC into one node produces a directed acyclic graph. The algorithm runs two passes of depth-first search: the first records the order in which nodes finish, and the second explores the transposed graph in reverse finish order, with each search collecting exactly one component.

- `KosarajuSCC(n = 0)` constructs a directed graph of `n` nodes numbered  $[0, n)$ .
- `add_edge(u, v)` adds the directed edge from `u` to `v`.
- `build_sccs()` computes the strongly connected components.
- `sccs()` returns the strongly connected components from the last `build_sccs()` call.
- `comp_id(v)` returns the component ID containing node `v`. Component IDs are in topological order: for every edge from component `a` to a different component `b`,  $a < b$ .

**Time Complexity:**  $O(\max(n, m))$  per call to `build_sccs()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, reverse graph, SCCs, and component IDs.
- $O(n)$  auxiliary stack space.

```
#include <algorithm>
#include <vector>

class KosarajuSCC {
    std::vector<std::vector<int>>> adj, rev, scc;
    std::vector<int> comp;
    std::vector<char> visit;

    void dfs_order(int u, std::vector<int> &order) {
        visit[u] = true;
        for (int v : adj[u]) {
            if (!visit[v]) {
                dfs_order(v, order);
            }
        }
        order.push_back(u);
    }

    void dfs_component(int u) {
        visit[u] = true;
        comp[u] = static_cast<int>(scc.size()) - 1;
        scc.back().push_back(u);
        for (int v : rev[u]) {
            if (!visit[v]) {
                dfs_component(v);
            }
        }
    }

public:
    explicit KosarajuSCC(int n = 0) : adj(n), rev(n) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        rev[v].push_back(u);
    }

    void build_sccs() {
        int n = static_cast<int>(adj.size());
        visit.assign(n, false);
        std::vector<int> order;
        for (int i = 0; i < n; i++) {
```



```

        if (!visit[i]) {
            dfs_order(i, order);
        }
    }
    std::reverse(order.begin(), order.end());
    visit.assign(n, false);
    comp.assign(n, -1);
    scc.clear();
    for (int u : order) {
        if (!visit[u]) {
            scc.emplace_back();
            dfs_component(u);
        }
    }
}

const std::vector<std::vector<int>> &sccs() const { return scc; }
int comp_id(int v) const { return comp[v]; }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0 ----> 1 ----> 2 <----> 3
    // ^      / |      |      ^
    // |      / |      |      |
    // |      / |      |      |
    // | v      v      v      v
    // 4 ----> 5 <----> 6 <----> 7
    KosarajuSCC g(8);
    g.add_edge(0, 1);
    g.add_edge(1, 2);
    g.add_edge(1, 4);
    g.add_edge(1, 5);
    g.add_edge(2, 3);
    g.add_edge(2, 6);
    g.add_edge(3, 2);
    g.add_edge(3, 7);
    g.add_edge(4, 0);
    g.add_edge(4, 5);
    g.add_edge(5, 6);
    g.add_edge(6, 5);
    g.add_edge(7, 3);
    g.add_edge(7, 6);
    g.build_sccs();
    // SCC condensation DAG:
    // {0,1,4} -> {2,3,7} -> {5,6}
    //      \-----^
    vector<vector<int>> sccs = g.sccs();
    for (auto &scc : sccs) {
        sort(scc.begin(), scc.end());
    }
    sort(sccs.begin(), sccs.end());
    assert((sccs == vector<vector<int>>{{0, 1, 4}, {2, 3, 7}, {5, 6}}));
    assert(g.comp_id(0) == g.comp_id(1) && g.comp_id(1) == g.comp_id(4));
    assert(g.comp_id(2) == g.comp_id(3) && g.comp_id(3) == g.comp_id(7));
    assert(g.comp_id(5) == g.comp_id(6));
    assert(g.comp_id(0) != g.comp_id(2) && g.comp_id(2) != g.comp_id(5));
    return 0;
}

```

### 4.2.3 Strongly Connected Components (Tarjan)

Given a directed graph, determine the strongly connected components (SCCs) using Tarjan's algorithm. A strongly connected component is a maximal set of nodes where every node can reach every other node. Condensing each SCC into one node produces a directed acyclic graph. A single depth-first search keeps visit nodes on a stack and tracks each node's low-link, the smallest entry time reachable from its subtree; a node whose low-link equals its own entry time roots a component, which is popped off the stack in one piece.

- `TarjanSCC(n = 0)` constructs a directed graph of `n` nodes numbered  $[0, n)$ .
- `add_edge(u, v)` adds the directed edge from `u` to `v`.
- `build_sccs()` computes the strongly connected components.
- `sccs()` returns the strongly connected components from the last `build_sccs()` call.
- `comp_id(v)` returns the component ID containing node `v`. Component IDs are in reverse topological order: for every edge from component `a` to a different component `b`,  $a > b$ .

**Time Complexity:**  $O(\max(n, m))$  per call to `build_sccs()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, SCCs, and component IDs.
- $O(n)$  auxiliary stack space.

```
#include <algorithm>
#include <climits>
#include <vector>

class TarjanSCC {
    static const int INF = INT_MAX / 2;
    std::vector<std::vector<int>>> adj, scc;
    std::vector<int> component, active, lowlink;
    std::vector<char> visit;
    int timer;

    void dfs(int u) {
        lowlink[u] = timer++;
        visit[u] = true;
        active.push_back(u);
        bool is_component_root = true;
        for (int v : adj[u]) {
            if (!visit[v]) {
                dfs(v);
            }
            if (lowlink[u] > lowlink[v]) {
                lowlink[u] = lowlink[v];
                is_component_root = false;
            }
        }
        if (!is_component_root) {
            return;
        }
        std::vector<int> comp_nodes;
        int id = static_cast<int>(scc.size());
        int v;
        do {
            v = active.back();
            active.pop_back();
            lowlink[v] = INF; // marks v as removed from the active stack
            component[v] = id;
            comp_nodes.push_back(v);
        } while (u != v);
        scc.push_back(comp_nodes);
    }

public:
```

```

explicit TarjanSCC(int n = 0) : adj(n) {}

void add_edge(int u, int v) { adj[u].push_back(v); }

void build_sccs() {
    int n = static_cast<int>(adj.size());
    scc.clear();
    component.assign(n, -1);
    active.clear();
    lowlink.assign(n, 0);
    visit.assign(n, false);
    timer = 0;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            dfs(i);
        }
    }
}

const std::vector<std::vector<int>> &sccs() const { return scc; }
int comp_id(int v) const { return component[v]; }
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0 ---> 1 ----> 2 <---> 3
    // ^      / |      |      ^
    // |      / |      |      |
    // |      / |      |      |
    // | v    v      v      v
    // 4 ---> 5 <---> 6 <---> 7
    TarjanSCC g(8);
    g.add_edge(0, 1);
    g.add_edge(1, 2);
    g.add_edge(1, 4);
    g.add_edge(1, 5);
    g.add_edge(2, 3);
    g.add_edge(2, 6);
    g.add_edge(3, 2);
    g.add_edge(3, 7);
    g.add_edge(4, 0);
    g.add_edge(4, 5);
    g.add_edge(5, 6);
    g.add_edge(6, 5);
    g.add_edge(7, 3);
    g.add_edge(7, 6);
    g.build_sccs();
    // SCC condensation DAG:
    // {0,1,4} -> {2,3,7} -> {5,6}
    //      \-----^
    vector<vector<int>> sccs = g.sccs();
    for (auto &scc : sccs) {
        sort(scc.begin(), scc.end());
    }
    sort(sccs.begin(), sccs.end());
    assert((sccs == vector<vector<int>>{{0, 1, 4}, {2, 3, 7}, {5, 6}}));
    assert(g.comp_id(0) == g.comp_id(1) && g.comp_id(1) == g.comp_id(4));
    assert(g.comp_id(2) == g.comp_id(3) && g.comp_id(3) == g.comp_id(7));
    assert(g.comp_id(5) == g.comp_id(6));
    assert(g.comp_id(0) != g.comp_id(2) && g.comp_id(2) != g.comp_id(5));
    return 0;
}

```

### 4.2.4 Dominator Tree (Lengauer-Tarjan)

Given a directed graph and a start node, find the immediate dominator of every node reachable from the start. A node  $u$  dominates node  $v$  if every path from the start to  $v$  passes through  $u$ . The immediate dominator  $\text{idom}[v]$  is the closest strict dominator of  $v$ ; these parent links form the dominator tree rooted at the start. Dominator trees are useful in control-flow graphs, program analysis, and reachability problems with unavoidable checkpoints.

Lengauer-Tarjan's algorithm numbers nodes by DFS order, computes each reachable node's semidominator using a disjoint-set structure with path compression, and then resolves immediate dominators from semidominator buckets.

- `immediate_dominators(start)` uses the directed graph in the global adjacency list `adj` and returns a vector `idom` where `idom[start] = start`, `idom[v]` is the immediate dominator of reachable node  $v$ , and `idom[v] = -1` if  $v$  is unreachable.

**Time Complexity:**  $O(\max(n, m) \log n)$  per call to `immediate_dominators()` in this path-compressed implementation, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for the graph, result, and auxiliary heap arrays.
- $O(n)$  auxiliary stack space for the DFS.

```
#include <algorithm>
#include <vector>

std::vector<std::vector<int>> adj;

std::vector<int> immediate_dominators(int start) {
    int n = static_cast<int>(adj.size());
    std::vector<std::vector<int>> pred(n + 1), bucket(n + 1);
    std::vector<int> time(n), node_at_time(n + 1), parent(n + 1), sdom(n + 1);
    std::vector<int> idom_index(n + 1), dsu_root(n + 1), best(n + 1);
    int timer = 0;
    auto dfs = [&](auto &&dfs, int u) -> void {
        time[u] = ++timer;
        node_at_time[time[u]] = u;
        sdom[time[u]] = dsu_root[time[u]] = best[time[u]] = timer;
        for (int v : adj[u]) {
            if (time[v] == 0) {
                dfs(dfs, v);
                parent[time[v]] = time[u];
            }
            if (time[v] != 0) {
                pred[time[v]].push_back(time[u]);
            }
        }
    };
    dfs(dfs, start);
    auto find_best = [&](auto &&find_best, int u, int depth) -> int {
        if (u == dsu_root[u]) {
            return depth == 0 ? u : -1;
        }
        int root = find_best(find_best, dsu_root[u], depth + 1);
        if (root == -1) {
            return u;
        }
        if (sdom[best[dsu_root[u]]] < sdom[best[u]]) {
            best[u] = best[dsu_root[u]];
        }
        dsu_root[u] = root;
        return depth == 0 ? best[u] : root; // The outer call returns the label, recursion the root.
    };
    for (int i = timer; i >= 1; i--) {
        for (int v : pred[i]) {
            sdom[i] = std::min(sdom[i], sdom[find_best(find_best, v, 0)]);
        }
    }
}
```

```

    }
    if (i > 1) {
        bucket[sdom[i]].push_back(i);
    }
    for (int v : bucket[i]) {
        int u = find_best(find_best, v, 0);
        idom_index[v] = (sdom[u] == sdom[v]) ? sdom[v] : u;
    }
    if (i > 1) {
        dsu_root[i] = parent[i];
    }
}
std::vector<int> idom(n, -1);
idom[start] = start;
for (int i = 2; i <= timer; i++) {
    if (idom_index[i] != sdom[i]) {
        idom_index[i] = idom_index[idom_index[i]];
    }
    idom[node_at_time[i]] = node_at_time[idom_index[i]];
}
return idom;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0 ---> 1 ---+ +-----> 4
    // |           | /         |
    // v           v /         v
    // 2 -----> 3 ---> 5 ---> 6
    adj = {{1, 2}, {3}, {3}, {4, 5}, {6}, {6}, {}};
    assert((immediate_dominators(0) == vector<int>{0, 0, 0, 0, 3, 3, 3}));

    // Node 3 is reachable through either 1 or 2, so neither one dominates it.
    // 4 ---> 1 -----+
    // |      ^|       |
    // | /    |       |
    // v /    v       v
    // 0 ---> 2 ---> 3
    adj = {{1, 2}, {2, 3}, {3}, {}, {0, 1}};
    assert((immediate_dominators(4) == vector<int>{4, 4, 4, 4, 4}));
    return 0;
}

```

### 4.2.5 Articulation Points, Biconnected Components, Block-Cut Forest

Given an undirected graph, compute articulation points, vertex-biconnected components (BCCs), and the block-cut forest using Tarjan's algorithm. An articulation point (a.k.a. cut vertex) is a node whose removal increases the number of connected components in the graph. A vertex-biconnected component is a maximal subgraph that cannot be disconnected by removing one node, with single-edge and isolated-node components handled as degenerate cases. A single depth-first search tracks each node's low-link, the earliest entry time reachable from its subtree: a node is an articulation point exactly when some child's subtree cannot reach above it, and the edges of each BCC are collected on a stack popped at that moment.

The block-cut forest is a bipartite forest with one node for each BCC and one node for each articulation point, with an edge whenever an articulation point belongs to a BCC. Note that this differs from a bridge forest: the block-cut forest describes vertex connectivity, while a bridge forest describes edge connectivity after compressing 2-edge-connected components.

- `BiconnectedComponents(n = 0)` constructs an undirected graph of `n` nodes numbered  $[0, n)$ .
- `add_edge(u, v)` adds the undirected edge `u-v`. Parallel edges are supported.
- `build_bccs()` computes the articulation points and vertex-biconnected components.
- `articulation_points()` and `bccs()` return those results.
- `build_block_cut_forest()` computes the block-cut forest using the results of the previous `build_bccs()` call.
- `block_cut_forest()` returns that forest. Its component nodes are numbered in the range  $[0, \text{bccs().size}())$ .
- `block_cut_id(v)` returns the block-cut forest node representing articulation point `v`, or `-1` if `v` is not an articulation point.

**Time Complexity:**  $O(\max(n, m))$  per call to `build_bccs()` and `build_block_cut_forest()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, edge stack, BCCs, and block-cut forest.
- $O(n)$  auxiliary stack space for `build_bccs()`.

```
#include <algorithm>
#include <utility>
#include <vector>

class BiconnectedComponents {
    std::vector<std::vector<int>>> adj, bcc, forest;
    std::vector<int> lowlink, tin, forest_id, articulation;
    std::vector<char> visit, is_articulation;
    std::vector<std::pair<int, int>> edges;
    std::vector<int> edge_stack;
    int timer;

    int other(int id, int u) const { return edges[id].first ^ edges[id].second ^ u; }

    void add_component_until(int stop_id) {
        std::vector<int> comp;
        while (true) {
            int id = edge_stack.back();
            edge_stack.pop_back();
            comp.push_back(edges[id].first);
            comp.push_back(edges[id].second);
            if (id == stop_id) {
                break;
            }
        }
        std::sort(comp.begin(), comp.end());
        comp.erase(std::unique(comp.begin(), comp.end()), comp.end());
        bcc.push_back(comp);
    }

    void dfs(int u, int p) {
        visit[u] = true;
        lowlink[u] = tin[u] = timer++;
        int children = 0;
        for (int id : adj[u]) {
            if (id == p) {
                continue;
            }
            int v = other(id, u);
            if (visit[v]) {
                if (tin[v] < tin[u]) {
                    edge_stack.push_back(id);
                    lowlink[u] = std::min(lowlink[u], tin[v]);
                }
            } else {
                edge_stack.push_back(id);
                dfs(v, id);
                lowlink[u] = std::min(lowlink[u], lowlink[v]);
                children++;
            }
        }
    }
};
```

```

        if (lowlink[v] >= tin[u]) {
            if (p != -1 || children > 1) {
                is_articulation[u] = true;
            }
            add_component_until(id);
        }
    }
}

if (p == -1 && children == 0) {
    bcc.push_back({u});
}
}

public:
explicit BiconnectedComponents(int n = 0) : adj(n) {}

void add_edge(int u, int v) {
    int id = static_cast<int>(edges.size());
    adj[u].push_back(id);
    adj[v].push_back(id);
    edges.emplace_back(u, v);
}

void build_bccs() {
    int n = static_cast<int>(adj.size());
    articulation.clear();
    bcc.clear();
    edge_stack.clear();
    lowlink.assign(n, 0);
    tin.assign(n, 0);
    visit.assign(n, false);
    is_articulation.assign(n, false);
    timer = 0;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            dfs(i, -1);
        }
    }
    for (int i = 0; i < n; i++) {
        if (is_articulation[i]) {
            articulation.push_back(i);
        }
    }
}

void build_block_cut_forest() {
    int n = static_cast<int>(adj.size());
    int blocks = static_cast<int>(bcc.size());
    forest_id.assign(n, -1);
    int total = blocks;
    for (int v : articulation) {
        forest_id[v] = total++;
    }
    forest.assign(total, {});
    for (int b = 0; b < blocks; b++) {
        for (int v : bcc[b]) {
            if (forest_id[v] != -1) {
                forest[b].push_back(forest_id[v]);
                forest[forest_id[v]].push_back(b);
            }
        }
    }
}

const std::vector<int> &articulation_points() const { return articulation; }
const std::vector<std::vector<int>> &bccs() const { return bcc; }
const std::vector<std::vector<int>> &block_cut_forest() const { return forest; }
int block_cut_id(int v) const { return forest_id[v]; }

```

```
};
```

### Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // 0---1---2    3---7
    //   \ |
    //     5---4
    BiconnectedComponents g(8);
    g.add_edge(0, 1);
    g.add_edge(0, 5);
    g.add_edge(1, 2);
    g.add_edge(1, 5);
    g.add_edge(3, 7);
    g.add_edge(4, 5);
    g.build_bccs();
    assert((g.articulation_points() == vector<int>{1, 5}));
    assert((g.bccs() == vector<vector<int>>{{1, 2}, {4, 5}, {0, 1, 5}, {3, 7}, {6}}));
    g.build_block_cut_forest();
    assert((g.block_cut_forest() == vector<vector<int>>{{5}, {6}, {5, 6}, {}, {}, {0, 2}, {1, 2}}));
    return 0;
}
```

## 4.2.6 Bridges, 2-Edge-Connected Components, Bridge Forest

Given an undirected graph, compute bridges, 2-edge-connected components, and the bridge forest using Tarjan's algorithm. A bridge is an edge whose removal increases the number of connected components in the graph. A 2-edge-connected component is a maximal set of nodes connected without crossing a bridge. A single depth-first search tracks each node's low-link, the earliest entry time reachable from its subtree: a tree edge is a bridge exactly when the child's subtree cannot reach back to the parent's side by any other route.

After each 2-edge-connected component is condensed into one node, the original bridges connect those nodes into a bridge tree, or a bridge forest when the original graph is disconnected. This differs from a block-cut forest: the bridge forest describes edge connectivity, while the block-cut forest describes vertex connectivity using articulation points and vertex-biconnected components.

- `BridgeDecomposition(n = 0)` constructs an undirected graph of `n` nodes numbered  $[0, n)$ .
- `add_edge(u, v)` adds the undirected edge `u-v`. Parallel edges are supported.
- `build_bridges()` computes the bridges, returned by `bridges()`.
- `build_bridge_forest()` computes the 2-edge-connected components and bridge forest using the results of the previous `build_bridges()` call.
- `two_edge_comps()` and `bridge_forest()` return those results.
- `comp_id(u)` returns the 2-edge-connected component ID containing node `u`.

**Time Complexity:**  $O(\max(n, m))$  per call to `build_bridges()` followed by `build_bridge_forest()`, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, bridges, 2-edge-connected components, and bridge forest.
- $O(n)$  auxiliary stack space for the DFS calls.

```
#include <algorithm>
#include <utility>
#include <vector>

class BridgeDecomposition {
```



```

std::vector<std::vector<int>>> adj, two_edge_comp, forest;
std::vector<int> lowlink, tin, comp;
std::vector<char> visit, is_bridge_edge;
std::vector<std::pair<int, int>> edges, bridge_edges;
int timer;

int other(int id, int u) const { return edges[id].first ^ edges[id].second ^ u; }

void dfs_bridges(int u, int p) {
    visit[u] = true;
    lowlink[u] = tin[u] = timer++;
    for (int id : adj[u]) {
        if (id == p) {
            continue;
        }
        int v = other(id, u);
        if (visit[v]) {
            lowlink[u] = std::min(lowlink[u], tin[v]);
        } else {
            dfs_bridges(v, id);
            lowlink[u] = std::min(lowlink[u], lowlink[v]);
            if (lowlink[v] > tin[u]) {
                bridge_edges.emplace_back(u, v);
                is_bridge_edge[id] = true;
            }
        }
    }
}

void dfs_component(int u, int id) {
    comp[u] = id;
    two_edge_comp[id].push_back(u);
    for (int edge_id : adj[u]) {
        int v = other(edge_id, u);
        if (comp[v] == -1 && !is_bridge_edge[edge_id]) {
            dfs_component(v, id);
        }
    }
}

public:
explicit BridgeDecomposition(int n = 0) : adj(n) {}

void add_edge(int u, int v) {
    int id = static_cast<int>(edges.size());
    adj[u].push_back(id);
    adj[v].push_back(id);
    edges.emplace_back(u, v);
}

void build_bridges() {
    int n = static_cast<int>(adj.size());
    bridge_edges.clear();
    lowlink.assign(n, 0);
    tin.assign(n, 0);
    visit.assign(n, false);
    is_bridge_edge.assign(edges.size(), false);
    timer = 0;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            dfs_bridges(i, -1);
        }
    }
}

void build_bridge_forest() {
    int n = static_cast<int>(adj.size());
    comp.assign(n, -1);

```

```

    two_edge_comp.clear();
    for (int i = 0; i < n; i++) {
        if (comp[i] == -1) {
            two_edge_comp.emplace_back();
            dfs_component(i, static_cast<int>(two_edge_comp.size()) - 1);
        }
    }
    forest.assign(two_edge_comp.size(), {});
    for (auto &[u, v] : bridge_edges) {
        int cu = comp[u], cv = comp[v];
        forest[cu].push_back(cv);
        forest[cv].push_back(cu);
    }
}

const std::vector<std::pair<int, int>> &bridges() const { return bridge_edges; }
const std::vector<std::vector<int>> &two_edge_comps() const { return two_edge_comp; }
const std::vector<std::vector<int>> &bridge_forest() const { return forest; }
int comp_id(int u) const { return comp[u]; }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0---1---2    3---7
    //   \ |
    //     5---4
    BridgeDecomposition g(8);
    g.add_edge(0, 1);
    g.add_edge(0, 5);
    g.add_edge(1, 2);
    g.add_edge(1, 5);
    g.add_edge(3, 7);
    g.add_edge(4, 5);
    g.build_bridges();
    assert((g.bridges() == vector<pair<int, int>>{{1, 2}, {5, 4}, {3, 7}}));
    g.build_bridge_forest();
    assert((g.two_edge_comps() == vector<vector<int>>{{0, 1, 5}, {2}, {3}, {4}, {6}, {7}}));
    assert((g.bridge_forest() == vector<vector<int>>{{1, 3}, {0}, {5}, {0}, {0}, {2}}));
    return 0;
}

```

### 4.2.7 Online Bridge Finding

Maintain the number of bridges in an undirected graph while edges are inserted one at a time, along with the 2-edge-connected components and ordinary connected components. When an inserted edge joins two different connected components, it creates a new bridge. When it joins two nodes already in the same connected component, it creates a cycle and every bridge on the path between the two endpoints stops being a bridge.

- `OnlineBridges(n = 0)` constructs a graph with `n` isolated nodes numbered  $[0, n)$ .
- `add_edge(u, v)` adds the undirected edge `u-v` and updates the 2-edge-connected and ordinary connected components.
- `bridge_count()` returns the current number of bridges.
- `find_2ecc(u)` returns node `u`'s representative in the 2-edge-connected components partition.
- `find_cc(u)` returns node `u`'s representative in the ordinary connected components partition.

Parallel edges are supported. Adding a second edge between two nodes that are already in the same connected component dissolves every bridge on the path between them, merging those nodes into the same 2-edge-connected component.

**Time Complexity:**  $O(\log n)$  amortized per call to `add_edge()` in this implementation, where  $n$  is the number of nodes.

**Space Complexity:**  $O(n)$  for the disjoint-set arrays and auxiliary parent links.

```
#include <utility>
#include <vector>

class OnlineBridges {
    std::vector<int> dsu_2ecc, dsu_cc, dsu_cc_size, parent, last_visit;
    int lca_iteration, num_bridges;

    void make_root(int u) {
        u = find_2ecc(u);
        int root = u, child = -1;
        while (u != -1) {
            int p = find_2ecc(parent[u]);
            parent[u] = child;
            dsu_cc[u] = root;
            child = u;
            u = p;
        }
        dsu_cc_size[root] = dsu_cc_size[child];
    }

    void merge_path(int u, int v) {
        lca_iteration++;
        std::vector<int> upath, vpath;
        int lca = -1;
        while (lca == -1) {
            if (u != -1) {
                u = find_2ecc(u);
                upath.push_back(u);
                if (last_visit[u] == lca_iteration) {
                    lca = u;
                    break;
                }
                last_visit[u] = lca_iteration;
                u = parent[u];
            }
            if (v != -1) {
                v = find_2ecc(v);
                vpath.push_back(v);
                if (last_visit[v] == lca_iteration) {
                    lca = v;
                    break;
                }
                last_visit[v] = lca_iteration;
                v = parent[v];
            }
        }
        for (int a : upath) {
            dsu_2ecc[a] = lca;
            if (a == lca) {
                break;
            }
        }
        num_bridges--;
        for (int b : vpath) {
            dsu_2ecc[b] = lca;
            if (b == lca) {
                break;
            }
        }
    }
};
```

```

        num_bridges--;
    }
}

public:
explicit OnlineBridges(int n = 0)
: dsu_2ecc(n),
  dsu_cc(n),
  dsu_cc_size(n, 1),
  parent(n, -1),
  last_visit(n, 0),
  lca_iteration(0),
  num_bridges(0) {
    for (int i = 0; i < n; i++) {
        dsu_2ecc[i] = dsu_cc[i] = i;
    }
}

void add_edge(int u, int v) {
    u = find_2ecc(u);
    v = find_2ecc(v);
    if (u == v) {
        return;
    }
    int cu = find_cc(u), cv = find_cc(v);
    if (cu != cv) {
        num_bridges++;
        if (dsu_cc_size[cu] > dsu_cc_size[cv]) {
            std::swap(u, v);
            std::swap(cu, cv);
        }
        make_root(u);
        parent[u] = v;
        dsu_cc[u] = v;
        dsu_cc_size[cv] += dsu_cc_size[u];
    } else {
        merge_path(u, v);
    }
}

int bridge_count() const { return num_bridges; }

int find_2ecc(int u) {
    if (u == -1) {
        return -1;
    }
    return dsu_2ecc[u] == u ? u : dsu_2ecc[u] = find_2ecc(dsu_2ecc[u]);
}

int find_cc(int u) {
    u = find_2ecc(u);
    return dsu_cc[u] == u ? u : dsu_cc[u] = find_cc(dsu_cc[u]);
}
};

```

## Example Usage

```

#include <cassert>

int main() {
    // 0---1
    //   \ |
    //     2---3
    OnlineBridges g(4);
    g.add_edge(0, 1);
    assert(g.bridge_count() == 1);
}

```

```

g.add_edge(1, 2);
assert(g.bridge_count() == 2);
g.add_edge(2, 0);
assert(g.bridge_count() == 0);
g.add_edge(2, 3);
assert(g.bridge_count() == 1);
return 0;
}

```

### 4.2.8 2-SAT

Solve a Boolean formula in 2-CNF, where each clause contains at most two literals. A clause like  $(a \vee b)$  is represented by the implications  $\neg a \rightarrow b$  and  $\neg b \rightarrow a$ . The formula is satisfiable if and only if no variable and its negation belong to the same strongly connected component of the implication graph.

Variables are numbered  $[0, n)$ . A literal is represented by `literal(variable, value)`, where `value == true` means the variable itself and `value == false` means its negation.

- `TwoSAT(n = 0)` constructs an empty formula over `n` variables.
- `literal(variable, value)` returns the integer ID for a variable or its negation.
- `add_implication(a, b)` adds the implication  $a \rightarrow b$  together with its contrapositive  $\neg b \rightarrow \neg a$ . Both edges are required for the strongly connected component test to detect contradictions, so callers should not add the contrapositive separately.
- `add_or(a, b)` adds the clause  $(a \vee b)$ .
- `add_true(a)` forces literal `a` to be true.
- `add_false(a)` forces literal `a` to be false.
- `add(variable, value)` forces a variable to equal `value`.
- `add(x, xval, y, yval)` adds the clause  $(x = xval \vee y = yval)$ .
- `satisfiable()` returns whether all added clauses can be satisfied.
- `assignment()` returns the valid assignment found by the last successful `satisfiable()` call.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the number of variables.
- $O(1)$  per call to `literal()`, `add_implication()`, `add_or()`, `add_true()`, `add_false()`, and `add()`.
- $O(\max(n, m))$  per call to `satisfiable()`, where  $n$  is the number of variables and  $m$  is the number of edges in the implication graph.

**Space Complexity:**  $O(\max(n, m))$  for the implication graph and DFS stacks.

```

#include <algorithm>
#include <vector>

class TwoSAT {
    int variables;
    std::vector<std::vector<int>>> adj, rev;
    std::vector<int> order, component, solution;

    static int neg(int x) { return x ^ 1; }

    void add_edge(int a, int b) {
        adj[a].push_back(b);
        rev[b].push_back(a);
    }

    void dfs_order(int u, std::vector<char> &visit) {
        visit[u] = true;
        for (int v : adj[u]) {
            if (!visit[v]) {
                dfs_order(v, visit);
            }
        }
    }
}

```

```

    }
    order.push_back(u);
}

void dfs_component(int u, int id) {
    component[u] = id;
    for (int v : rev[u]) {
        if (component[v] == -1) {
            dfs_component(v, id);
        }
    }
}

public:
explicit TwoSAT(int n = 0) : variables(n), adj(2 * n), rev(2 * n), solution(n, false) {}

int literal(int variable, bool value) const { return 2 * variable + (value ? 0 : 1); }

void add_implication(int a, int b) {
    add_edge(a, b);
    add_edge(neg(b), neg(a));
}

void add_or(int a, int b) { add_implication(neg(a), b); }
void add_true(int a) { add_edge(neg(a), a); }
void add_false(int a) { add_edge(a, neg(a)); }
void add(int variable, bool value) { add_true(literal(variable, value)); }
void add(int x, bool xval, int y, bool yval) { add_or(literal(x, xval), literal(y, yval)); }

bool satisfiable() {
    order.clear();
    component.assign(2 * variables, -1);
    std::vector<char> visit(2 * variables);
    for (int i = 0; i < 2 * variables; i++) {
        if (!visit[i]) {
            dfs_order(i, visit);
        }
    }
    std::reverse(order.begin(), order.end());
    int id = 0;
    for (int u : order) {
        if (component[u] == -1) {
            dfs_component(u, id++);
        }
    }
    for (int i = 0; i < variables; i++) {
        if (component[2 * i] == component[2 * i + 1]) {
            return false;
        }
    }
    for (int i = 0; i < variables; i++) {
        solution[i] = component[2 * i] > component[2 * i + 1];
    }
    return true;
}

const std::vector<int> &assignment() const { return solution; }
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    TwoSAT solver(3);

```

```

int x0 = solver.literal(0, true);
int not_x0 = solver.literal(0, false);
int x1 = solver.literal(1, true);
int x2 = solver.literal(2, true);
solver.add_or(x0, x1);
solver.add_or(not_x0, x2);
solver.add_true(x0);
solver.add(1, true, 2, false);
assert(solver.satisfiable());
assert((solver.assignment() == vector<int>{true, true, true}));

TwoSAT stateful(2);
assert(stateful.satisfiable());
vector<int> last_assignment = stateful.assignment();
stateful.add(0, false);
stateful.add(1, true);
stateful.add(1, false);
assert(!stateful.satisfiable());
assert(stateful.assignment() == last_assignment);

TwoSAT implication(2);
int x = implication.literal(0, true);
int y = implication.literal(1, true);
implication.add_true(x);
implication.add_false(y);
implication.add_implication(x, y);
assert(!implication.satisfiable());
return 0;
}

```

### 4.2.9 Functional Graph

A functional graph is a directed graph in which every node has exactly one outgoing edge, given by a successor function  $f$ , where  $f(i)$  is the node reached in one step from node  $i$ . Following the edges from any node eventually enters a cycle, so each weakly connected component consists of one directed cycle with trees of nodes feeding into it. This structure underlies problems about iterating a function: where a sequence  $i, f(i), f(f(i)), \dots$  ends up, and how far it is from repeating.

Building the structure peels away nodes of in-degree zero to expose the cycles (the survivors), then labels each cycle and walks the remaining tail nodes to record their distance to the cycle. A binary lifting table stores the function applied  $2^k$  times, so any number of steps can be taken in logarithmic time. Together these answer, for any node, which cycle it eventually reaches, how many steps until it gets there, and exactly where it lands after a huge number of steps.

- `FunctionalGraph(f)` builds the structure for the successor function `f`, whose indices represent the nodes.
- `on_cycle(i)` returns whether node `i` lies on a cycle.
- `dist_to_cycle(i)` returns the number of steps from `i` to the first cycle node it reaches (0 if `i` is itself on a cycle).
- `cycle_id(i)` returns an identifier for the cycle that `i` eventually reaches; two nodes share an identifier exactly when they funnel into the same cycle.
- `cycle_length(id)` returns the length of the cycle with the given identifier.
- `num_cycles()` returns the number of distinct cycles.
- `successor(i, steps)` returns the node reached from `i` after applying `f` exactly `steps` times, for any nonnegative `steps` (which may be astronomically large).

#### Time Complexity:

- $O(n \log n)$  per call to the constructor, where  $n$  is the number of nodes.
- $O(\log n)$  per call to `successor()`.
- $O(1)$  per call to all other member functions.

**Space Complexity:**  $O(n \log n)$  for storage of the binary lifting table.

```
#include <stdint>
#include <vector>

class FunctionalGraph {
    std::vector<std::vector<int>>> up;
    std::vector<char> on_cyc;
    std::vector<int> to_cyc, comp, clen;

public:
    explicit FunctionalGraph(const std::vector<int> &f) {
        int n = static_cast<int>(f.size()), levels = 1;
        while ((1LL << levels) < n) {
            levels++;
        }
        up.assign(levels + 1, std::vector<int>(n));
        up[0] = f;
        for (int k = 1; k <= levels; k++) {
            for (int i = 0; i < n; i++) {
                up[k][i] = up[k - 1][up[k - 1][i]];
            }
        }
        // Peel nodes of in-degree zero; the survivors are exactly the cycle nodes.
        std::vector<int> indeg(n);
        for (int i = 0; i < n; i++) {
            indeg[f[i]]++;
        }
        std::vector<char> removed(n);
        std::vector<int> st;
        for (int i = 0; i < n; i++) {
            if (indeg[i] == 0) {
                st.push_back(i);
            }
        }
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            removed[u] = true;
            if (--indeg[f[u]] == 0) {
                st.push_back(f[u]);
            }
        }
        on_cyc.assign(n, false);
        for (int i = 0; i < n; i++) {
            on_cyc[i] = !removed[i];
        }
        // Label each cycle and record its length.
        comp.assign(n, -1);
        to_cyc.assign(n, 0);
        for (int i = 0; i < n; i++) {
            if (on_cyc[i] && comp[i] == -1) {
                int len = 0;
                for (int u = i; comp[u] == -1; u = f[u]) {
                    comp[u] = static_cast<int>(clen.size());
                    len++;
                }
                clen.push_back(len);
            }
        }
        // Walk each tail to the cycle, assigning its component and distance.
        for (int i = 0; i < n; i++) {
            if (comp[i] != -1) {
                continue;
            }
            std::vector<int> path;
            int u = i;
            while (comp[u] == -1) {

```



```

        path.push_back(u);
        u = f[u];
    }
    for (int j = static_cast<int>(path.size()) - 1, step = 1; j >= 0; j--, step++) {
        comp[path[j]] = comp[u];
        to_cyc[path[j]] = to_cyc[u] + step;
    }
}

bool on_cycle(int i) const { return on_cyc[i]; }
int dist_to_cycle(int i) const { return to_cyc[i]; }
int cycle_id(int i) const { return comp[i]; }
int cycle_length(int id) const { return clen[id]; }
int num_cycles() const { return static_cast<int>(clen.size()); }

int successor(int i, int64_t steps) const {
    // Advances i by following f exactly dist times.
    auto advance = [&](int64_t dist) {
        for (int k = 0; dist > 0; k++, dist >>= 1) {
            if (dist & 1) {
                i = up[k][i];
            }
        }
    };
    int enter = to_cyc[i];
    if (steps > enter) {
        steps -= enter;
        advance(enter); // Advance to the cycle entry node.
        steps %= clen[comp[i]];
    }
    advance(steps);
    return i;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0 --> 1 <-- 3 <-- 4 <-- 5
    //   ^   /
    //   \ v
    //   2 <-- 6
    FunctionalGraph g({1, 2, 0, 1, 3, 4, 2});
    assert(g.on_cycle(0) && g.on_cycle(1) && g.on_cycle(2));
    assert(!g.on_cycle(3) && !g.on_cycle(5));
    assert(g.num_cycles() == 1);
    assert(g.cycle_length(0) == 3);

    assert(g.dist_to_cycle(2) == 0);
    assert(g.dist_to_cycle(5) == 3);
    assert(g.dist_to_cycle(6) == 1);
    assert(g.cycle_id(5) == g.cycle_id(0));

    assert(g.successor(5, 1) == 4);
    assert(g.successor(5, 3) == 1);    // 5 -> 4 -> 3 -> 1.
    assert(g.successor(5, 4) == 2);    // One more step around into the cycle.
    assert(g.successor(0, 100) == 1);  // 100 mod 3 == 1 step around the cycle.
    return 0;
}

```

### 4.2.10 Offline Dynamic Connectivity

Maintains the number of connected components of an undirected graph as edges are added and removed over time, answering queries offline once the entire sequence of operations is known. Each edge is alive over a contiguous interval of time, from when it is added until it is removed. These intervals are placed into a segment tree over the time axis, so that for  $T$  operations, each edge appears in  $O(\log T)$  nodes whose ranges it spans. A depth-first traversal of this segment tree unites the edges stored at each node on the way down and undoes them on the way back up, so at each leaf the disjoint set forest reflects exactly the edges alive at that moment.

This offline method supports arbitrary undirected graphs. When updates must be processed online and the graph remains a forest, see the Euler Tour Tree in 2.8.9.

Undoing unions requires a disjoint set forest with rollback (see the disjoint sets section): it joins by size or rank without path compression, recording each change on a stack so it can be reversed. Path compression is incompatible with rollback, so finding a representative costs  $O(\log n)$  rather than near-constant time.

- `OfflineDynamicConnectivity(n)` creates a structure of `n` nodes numbered  $[0, n)$ , initially with no edges.
- `add_edge(u, v)` and `remove_edge(u, v)` record adding or removing the undirected edge `u-v` at the current time, then advance the time by one step. Each edge may be present at most once at a time: `add_edge` must not name an edge that is already present, and `remove_edge` must name one that is. Both preconditions are checked with `assert` during `solve()`.
- `query_components()` records a query at the current time and advances the time by one step.
- `solve()` processes all recorded operations and returns a vector holding, for each `query_components()` query in the order issued, the number of connected components at that time.

#### Time Complexity:

- $O(T \log T + m \log T \log n)$  per call to `solve()`, where  $T$  is the total number of operations (`add_edge()`, `remove_edge()`, and `query_components()` combined),  $m$  is the number of `add_edge()` operations, and  $n$  is the number of nodes.
- $O(1)$  amortized per call to `add_edge()`, `remove_edge()`, and `query_components()`.

**Space Complexity:**  $O(n + T + m \log T)$  auxiliary.

```
#include <algorithm>
#include <cassert>
#include <map>
#include <numeric>
#include <utility>
#include <vector>

class OfflineDynamicConnectivity {
    static const int ADD = 0, REMOVE = 1, QUERY = 2;

    struct Op {
        int type, u, v;
    };

    int n, num_sets;
    std::vector<Op> ops;
    std::vector<int> root, rank, answers;
    std::vector<std::pair<int, int>> history; // (child root, whether its parent's rank increased)
    std::vector<std::vector<std::pair<int, int>>> seg;

    int find(int u) const {
        while (root[u] != u) {
            u = root[u];
        }
        return u;
    }

    void unite(int u, int v) {
        u = find(u);
        v = find(v);
```

```

    if (u == v) {
        history.emplace_back(-1, 0);
        return;
    }
    if (rank[u] < rank[v]) {
        std::swap(u, v);
    }
    history.emplace_back(v, rank[u] == rank[v] ? 1 : 0);
    root[v] = u;
    if (rank[u] == rank[v]) {
        rank[u]++;
    }
    num_sets--;
}

void rollback() {
    auto [child, raised] = history.back();
    history.pop_back();
    if (child == -1) {
        return;
    }
    rank[root[child]] -= raised;
    root[child] = child;
    num_sets++;
}

void add_interval(
    int node, int lo, int hi, int tgt_lo, int tgt_hi, const std::pair<int, int> &e
) {
    if (tgt_hi < lo || hi < tgt_lo) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        seg[node].push_back(e);
        return;
    }
    int mid = lo + (hi - lo) / 2;
    add_interval(2 * node, lo, mid, tgt_lo, tgt_hi, e);
    add_interval(2 * node + 1, mid + 1, hi, tgt_lo, tgt_hi, e);
}

void dfs(int node, int lo, int hi) {
    int applied = 0;
    for (const auto &[eu, ev] : seg[node]) {
        unite(eu, ev);
        applied++;
    }
    if (lo == hi) {
        if (ops[lo].type == QUERY) {
            answers.push_back(num_sets);
        }
    } else {
        int mid = lo + (hi - lo) / 2;
        dfs(2 * node, lo, mid);
        dfs(2 * node + 1, mid + 1, hi);
    }
    while (applied-- > 0) {
        rollback();
    }
}

public:
explicit OfflineDynamicConnectivity(int n) : n(n) {}

void add_edge(int u, int v) { ops.push_back({ADD, std::min(u, v), std::max(u, v)}); }
void remove_edge(int u, int v) { ops.push_back({REMOVE, std::min(u, v), std::max(u, v)}); }
void query_components() { ops.push_back({QUERY, 0, 0}); }

```

```

std::vector<int> solve() {
    int t = static_cast<int>(ops.size());
    answers.clear();
    if (t == 0) {
        return answers;
    }
    seg.assign(4 * t, {});
    std::map<std::pair<int, int>, int> active;
    for (int i = 0; i < t; i++) {
        std::pair<int, int> key(ops[i].u, ops[i].v);
        if (ops[i].type == ADD) {
            assert(active.find(key) == active.end()); // The edge must not already be present.
            active[key] = i;
        } else if (ops[i].type == REMOVE) {
            auto it = active.find(key);
            assert(it != active.end()); // The edge must currently be present.
            add_interval(1, 0, t - 1, it->second, i - 1, {ops[i].u, ops[i].v});
            active.erase(it);
        }
    }
    for (const auto &[key, start] : active) {
        add_interval(1, 0, t - 1, start, t - 1, {key.first, key.second});
    }
    root.resize(n);
    std::iota(root.begin(), root.end(), 0);
    rank.assign(n, 0);
    num_sets = n;
    history.clear();
    dfs(1, 0, t - 1);
    return answers;
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    OfflineDynamicConnectivity dc(4);
    // 0 1 2 3
    dc.query_components(); // 4 isolated nodes.
    dc.add_edge(0, 1);
    dc.add_edge(2, 3);
    // 0---1 2---3
    dc.query_components(); // 2 components.
    dc.add_edge(1, 2);
    // 0---1---2---3
    dc.query_components(); // All connected => 1 component.
    dc.remove_edge(1, 2);
    // 0---1 2---3
    dc.query_components(); // Back to 2 components.
    assert((dc.solve() == vector<int>{4, 2, 1, 2}));
    return 0;
}

```

## 4.3 Shortest Paths

### 4.3.1 Unweighted Shortest Path (BFS)

Given one or more starting nodes in an unweighted, directed graph, visit every reachable node and determine its minimum distance from any start. Breadth-first search explores nodes in order of increasing distance using a queue, so the first time a node is reached is via a shortest path. Initializing the queue with several distance-0 sources gives multi-source BFS without changing the traversal. Optionally, reconstruct a shortest path using the predecessor vector `pred`.

- `bfs(starts)` populates `dist` and `pred` from every node in `starts`, assigning each node its minimum distance from any start. The global, pre-populated adjacency list `adj` uses its indices as nodes and must contain only valid node indices. Pass a singleton vector for single-source BFS.
- `get_path(dest)` returns the path from a nearest starting node to `dest`, or an empty vector if `dest` is unreachable, using the state left by the most recent call to `bfs()`.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from a nearest start to `v`, or `-1` if `v` is a starting node or unreachable. Follow `pred` backward from the destination to its starting node, then reverse that sequence to recover the path.

#### Time Complexity:

- $O(\max(n, m, s))$  per call, where  $n$  is the number of nodes,  $m$  is the number of edges, and  $s$  is the number of supplied starting nodes. For single-source BFS,  $s = 1$ .
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

#### Space Complexity:

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```
#include <algorithm>
#include <climits>
#include <queue>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<int>>> adj;
std::vector<int> dist, pred;

void bfs(const std::vector<int> &starts) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::queue<int> q;
    for (int start : starts) {
        if (dist[start] > 0) {
            dist[start] = 0;
            q.push(start);
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (dist[v] == INF) {
                dist[v] = dist[u] + 1;
                pred[v] = u;
                q.push(v);
            }
        }
    }
}
```

```

}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 0 --> 1 --> 2
    //      |   |
    //      v   |
    //      3 <---+
    adj.assign(4, {});
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(3);
    bfs(vector<int>{0});
    assert((dist == vector<int>{0, 1, 2, 2}) && (pred == vector<int>{-1, 0, 1, 1}));
    assert((get_path(3) == vector<int>{0, 1, 3}));

    bfs(vector<int>{0, 2});
    assert((dist == vector<int>{0, 1, 0, 1}) && (pred == vector<int>{-1, 0, -1, 2}));
    assert((get_path(3) == vector<int>{2, 3}));
    return 0;
}

```

### 4.3.2 0-1 BFS

Given one or more starting nodes in a weighted graph whose edge weights are only 0 or 1, compute the shortest distance from any start to every reachable node. Optionally, reconstruct the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred`.

0-1 BFS is a specialized version of Dijkstra's algorithm. Because every relaxation changes the distance by either 0 or 1, a deque maintains nodes in nondecreasing distance order: push weight-0 relaxations to the front and weight-1 relaxations to the back.

- `bfs01(starts)` populates `dist` and `pred` for shortest paths from the nodes in `starts`, assigning each node its minimum distance from any start. The global, pre-populated adjacency list `adj` uses its indices as nodes. Each edge is stored as `(neighbor, weight)`, where `weight` is either 0 or 1. Pass a singleton vector for a single source.
- `get_path(dest)` returns the path from a nearest starting node to `dest`, or an empty vector if `dest` is unreachable, using the state left by the most recent call to `bfs01()`.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from a nearest start to `v`, or `-1` if `v` is a starting node or unreachable. Follow `pred` backward from the destination to its starting node, then reverse that sequence to recover the path.

**Time Complexity:**

- $O(\max(n, m, s))$  per call, where  $n$  is the number of nodes,  $m$  is the number of edges, and  $s$  is the number of supplied starting nodes. For a single source,  $s = 1$ .
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary deque space.
- $O(p)$  for the path returned by `get_path()`.

```
#include <algorithm>
#include <climits>
#include <deque>
#include <utility>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int> dist, pred;

void bfs01(const std::vector<int> &starts) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::deque<int> dq;
    for (int start : starts) {
        if (dist[start] > 0) {
            dist[start] = 0;
            dq.push_back(start);
        }
    }
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pred[v] = u;
                if (w == 0) {
                    dq.push_front(v);
                } else {
                    dq.push_back(v);
                }
            }
        }
    }
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}
```

### Example Usage

```
#include <cassert>
using namespace std;

int main() {
    //      w=1
    //      0 -----> 1
    //      |         ^ |
    // w=0 |         w=0 | w=1
    //      v /         v
    //      2 -----> 3
    //      w=1
    adj.assign(4, {});
    adj[0].emplace_back(1, 1);
    adj[0].emplace_back(2, 0);
    adj[2].emplace_back(1, 0);
    adj[1].emplace_back(3, 1);
    adj[2].emplace_back(3, 1);
    bfs01(vector<int>{0});
    assert((dist == vector<int>{0, 0, 0, 1}) && (pred == vector<int>{-1, 2, 0, 2}));
    assert((get_path(3) == vector<int>{0, 2, 3}));

    bfs01(vector<int>{1, 0});
    assert((dist == vector<int>{0, 0, 0, 1}) && (pred == vector<int>{-1, -1, 0, 1}));
    assert((get_path(3) == vector<int>{1, 3}));
    return 0;
}
```

### 4.3.3 Shortest Path (Dijkstra)

Given a starting node in a weighted, directed graph with nonnegative weights only, visit every reachable node and determine the minimum distance to each such node. Optionally, reconstruct the shortest path to a destination node using the shortest-path tree from the predecessor array `pred`.

Dijkstra's algorithm repeatedly selects the unvisited node of smallest tentative distance using a priority queue and relaxes its outgoing edges. Dijkstra's algorithm requires nonnegative edge weights. Use Bellman-Ford or SPFA instead when negative edges are present. Because the weights are nonnegative, a node's distance is final the first time it is removed from the queue.

- `dijkstra(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` whose indices represent the nodes. Each edge is stored as `(neighbor, weight)`, where `weight` is nonnegative.
- `get_path(dest)` returns the path from `start` to `dest`, or an empty vector if `dest` is unreachable, using the state left by the most recent call to `dijkstra()`.

When only one destination matters, two searches beat one: run a second Dijkstra backwards from the destination over reversed edges, expanding whichever frontier holds the smaller key. The trap is the stopping rule. Track  $\mu$ , the smallest  $d_f(v) + d_r(v)$  over nodes both searches have reached; with  $t_f$  and  $t_r$  the smallest keys in the two queues, stop once  $t_f + t_r \geq \mu$ . Halting at the first node settled by both is the usual bug, since it need not lie on a shortest path.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

#### Time Complexity:

- $O(n + m \log n)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.



**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(\max(n, m))$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```

#include <algorithm>
#include <cassert>
#include <stdint>
#include <functional>
#include <queue>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

void dijkstra(int start) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    using qnode = std::pair<int64_t, int>; // (distance, node)
    std::priority_queue<qnode, std::vector<qnode>, std::greater<>> pq;
    pq.emplace(0, start);
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != dist[u]) {
            continue;
        }
        for (auto [v, w] : adj[u]) {
            if (dist[v] > dist[u] + w) { // Overflow warning.
                dist[v] = dist[u] + w;
                pred[v] = u;
                pq.emplace(dist[v], v);
            }
        }
    }
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}

```

**Example Usage**

```

#include <cassert>
using namespace std;

int main() {
    //      w=2      w=2
    //  0 ----> 1 ----> 2
    //   \      |      /
    // w=8\  w=4|  /w=1
    //      \   v   /

```

```

//      --> 3 <--
adj.assign(4, {});
adj[0].emplace_back(1, 2);
adj[0].emplace_back(3, 8);
adj[1].emplace_back(2, 2);
adj[1].emplace_back(3, 4);
adj[2].emplace_back(3, 1);
dijkstra(0);
assert((dist == vector<int64_t>{0, 2, 4, 5}) && (pred == vector<int>{-1, 0, 1, 2}));
assert((get_path(3) == vector<int>{0, 1, 2, 3}));
return 0;
}

```

#### 4.3.4 Shortest Path (A-Star)

Given a starting node and a destination node in a weighted, directed graph with nonnegative weights only, determine the minimum distance between them, using a heuristic estimate of the remaining distance to steer the search toward the destination. Optionally, reconstruct the shortest path using the predecessor array `pred`.

A\* is Dijkstra's algorithm with the priority queue keyed by  $f(v) = g(v) + h(v)$ , where  $g(v)$  is the best known distance from `start` to `v` and  $h(v)$  estimates the remaining distance to `dest`. Nodes whose estimated total already exceeds the answer are never expanded, so an informed heuristic visits far fewer nodes than Dijkstra's algorithm while returning the same distance. The heuristic must be admissible, never exceeding the true remaining distance, which guarantees that no shortest path is discarded and that the first removal of `dest` from the queue yields its final distance.

A heuristic is consistent when  $h(u) \leq w(u, v) + h(v)$  for every edge  $(u, v)$  and  $h(\text{dest}) = 0$ . Consistency implies admissibility and makes every distance final on first removal, exactly as in Dijkstra's algorithm; an admissible but inconsistent heuristic may improve a node after it has been expanded, which the stale-entry check below handles by re-expanding it. Setting  $h(v) = 0$  satisfies both and reduces this to Dijkstra's algorithm, so the heuristic is what buys the speedup: use the Manhattan distance on a unit-weight grid, the Euclidean distance scaled by the minimum edge weight in a geometric graph, or the number of misplaced tiles in a sliding puzzle.

- `astar(start, dest, h)` returns the minimum distance from `start` to `dest`, or `INF` if `dest` is unreachable, for a global, pre-populated adjacency list `adj` whose indices represent the nodes. Each edge is stored as `(neighbor, weight)`, where `weight` is nonnegative. The heuristic `h(v)` must return an admissible estimate of the remaining distance from node `v` to `dest`.
- `get_path(dest)` returns the path from `start` to `dest`, or an empty vector if `dest` is unreachable, using the state left by the most recent call to `astar()`.

Since the search stops as soon as `dest` is expanded, only `dist[dest]` and the `pred` chain back to `start` are final; other entries may remain tentative, so use Dijkstra's algorithm in section 4.3.3 when every distance is needed.

##### Time Complexity:

- $O(n + m \log n)$  per call in the worst case, where  $n$  is the number of nodes and  $m$  is the number of edges, plus one call to `h()` per queue insertion and removal. A more informed heuristic lowers the constant by expanding fewer nodes, but does not improve the worst case.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

##### Space Complexity:

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(\max(n, m))$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```

#include <algorithm>
#include <cstdint>
#include <functional>
#include <queue>

```

```

#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

template<typename Fn>
int64_t astar(int start, int dest, Fn h) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    using qnode = std::pair<int64_t, int>; // (estimated total distance, node)
    std::priority_queue<qnode, std::vector<qnode>, std::greater<>> pq;
    pq.emplace(h(start), start);
    while (!pq.empty()) {
        auto [f, u] = pq.top();
        pq.pop();
        if (f != dist[u] + h(u)) {
            continue; // Stale entry, left behind by a later improvement to dist[u].
        }
        if (u == dest) {
            return dist[u];
        }
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pred[v] = u;
                pq.emplace(dist[v] + h(v), v);
            }
        }
    }
    return INF;
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}

```

### Example Usage

```

#include <cassert>
#include <cstdlib>
using namespace std;

int main() {
    // A 3-by-3 grid of unit-weight edges with the center cell walled off, plus an
    // isolated node 9. Cell (r, c) is node 3*r + c.
    //  0 - 1 - 2
    //  |     |
    //  3     5
    //  |     |
    //  6 - 7 - 8
    const int R = 3, C = 3;
    adj.assign(R * C + 1, {});
    auto walled = [](int r, int c) { return r == 1 && c == 1; };

```

```

auto add_edge = [](int u, int v) {
    adj[u].emplace_back(v, 1);
    adj[v].emplace_back(u, 1);
};
for (int r = 0; r < R; r++) {
    for (int c = 0; c < C; c++) {
        if (walled(r, c)) {
            continue;
        }
        if (r + 1 < R && !walled(r + 1, c)) {
            add_edge(r * C + c, (r + 1) * C + c);
        }
        if (c + 1 < C && !walled(r, c + 1)) {
            add_edge(r * C + c, r * C + c + 1);
        }
    }
}

int start = 0, dest = R * C - 1;
auto manhattan = [=](int u) { return abs(u / C - (R - 1)) + abs(u % C - (C - 1)); };
assert(atar(start, dest, manhattan) == 4);
assert((get_path(dest) == vector<int>{0, 1, 2, 5, 8}));

// A zero heuristic is admissible and consistent, making this Dijkstra's algorithm.
auto zero = [](int) { return 0; };
assert(atar(start, dest, zero) == 4);
assert((get_path(dest) == vector<int>{0, 1, 2, 5, 8}));

assert(atar(start, R * C, zero) == INF);
assert(get_path(R * C).empty());
return 0;
}

```

### 4.3.5 Shortest Path (Bellman-Ford)

Given a starting node in a weighted, directed graph with possibly negative weights, visit every reachable node and determine the minimum distance to each such node. Optionally, reconstruct the shortest path to a destination node using the shortest-path tree from the predecessor array `pred`.

Bellman-Ford relaxes every edge in the graph up to  $n - 1$  times, stopping early if a pass makes no changes. Since any shortest path uses at most  $n - 1$  edges, all distances are correct after these passes unless a negative-weight cycle keeps reducing them, which a further pass detects. Bellman-Ford will also detect whether the graph contains a negative-weight cycle reachable from the start node, in which case the affected shortest paths are undefined. (To detect a negative cycle anywhere in the graph, add a virtual source with zero-weight edges to every node and start from it.)

- `bellman_ford(n, start)` populates `dist` and `pred` for a global, pre-populated edge list `edges` whose endpoints must be numbered  $[0, n)$ , and returns whether no reachable negative-weight cycle was found.
- `get_path(dest)` returns the path from `start` to `dest`, or an empty vector if `dest` is unreachable, provided the most recent call to `bellman_ford()` returned true. If it returned false, a reachable negative-weight cycle leaves the distances and paths undefined.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

#### Time Complexity:

- $O(n \cdot m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```
#include <algorithm>
#include <cstdint>
#include <tuple>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::tuple<int, int, int>> edges; // (u, v, w)
std::vector<int64_t> dist;
std::vector<int> pred;

bool bellman_ford(int n, int start) {
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    for (int i = 0; i < n - 1; i++) {
        bool changed = false;
        for (auto [u, v, w] : edges) {
            // The dist[u] != INF guard avoids relaxing out of unreachable nodes: a negative edge from an
            // unreachable u would otherwise give v a bogus finite distance (INF + w < INF).
            if (dist[u] != INF && dist[v] > dist[u] + w) { // Overflow warning.
                dist[v] = dist[u] + w;
                pred[v] = u;
                changed = true;
            }
        }
        if (!changed) break;
    }
    // Check for a negative-weight cycle reachable from the start node.
    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[v] > dist[u] + w) { // Overflow warning.
            return false;
        }
    }
    return true;
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}
```

**Example Usage**

```
#include <cassert>
using namespace std;

int main() {
    //      w=1      w=2
    // 0 -----> 1 -----> 2
    // |               ^
    // +-----+-----+
    //              w=5
    edges.emplace_back(0, 1, 1);
```

```

edges.emplace_back(1, 2, 2);
edges.emplace_back(0, 2, 5);
assert(bellman_ford(3, 0));
assert((dist == vector<int64_t>{0, 1, 3}) && (pred == vector<int>{-1, 0, 1}));
assert((get_path(2) == vector<int>{0, 1, 2}));

edges = {{0, 1, -1}, {1, 0, -1}};
assert(!bellman_ford(2, 0));
return 0;
}

```

### 4.3.6 Shortest Path (SPFA)

Given a starting node in a weighted directed graph, compute shortest paths even when some edge weights are negative. The Shortest Path Faster Algorithm (SPFA) is a queue-based optimization of Bellman-Ford: instead of relaxing every edge in every round, it keeps a queue of nodes whose distances have improved and relaxes only their outgoing edges. It is often fast on benign inputs, but it still has Bellman-Ford's worst-case behavior and can be forced to run in  $O(n \cdot m)$ . Prefer Dijkstra for nonnegative weights, and use SPFA mainly when negative edges are present and the input is not adversarial.

- `spfa(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` which uses its indices as nodes. Each edge is stored as `(neighbor, weight)`. The function returns `false` if it detects a reachable negative cycle, and returns `true` otherwise.
- `get_path(dest)` returns the path from `start` to `dest`, or an empty vector if `dest` is unreachable, provided the most recent call to `spfa()` returned true. If it returned false, a reachable negative-weight cycle leaves the distances and paths undefined.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

#### Time Complexity:

- $O(n \cdot m)$  per call in the worst case, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

#### Space Complexity:

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary queue space.
- $O(p)$  for the path returned by `get_path()`.

```

#include <algorithm>
#include <cstdint>
#include <queue>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

bool spfa(int start) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::vector<int> path_edges(n);
    std::vector<char> in_queue(n);
    std::queue<int> q;
    dist[start] = 0;
    q.push(start);

```

```

in_queue[start] = true;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    in_queue[u] = false;
    for (auto [v, w] : adj[u]) {
        if (dist[v] > dist[u] + w) { // Overflow warning.
            dist[v] = dist[u] + w;
            pred[v] = u;
            path_edges[v] = path_edges[u] + 1;
            if (path_edges[v] >= n) {
                return false;
            }
            if (!in_queue[v]) {
                q.push(v);
                in_queue[v] = true;
            }
        }
    }
}
return true;
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //          w=4
    //    0 ----> 1
    //    |      /
    // w=5 |    / w=-2
    //    | /
    //    v v   w=3
    //    2 -----> 3
    adj.assign(4, {});
    adj[0].emplace_back(1, 4);
    adj[0].emplace_back(2, 5);
    adj[1].emplace_back(2, -2);
    adj[2].emplace_back(3, 3);
    spfa(0);
    assert((dist == vector<int64_t>{0, 4, 2, 5}) && (pred == vector<int>{-1, 0, 1, 2}));
    assert((get_path(3) == vector<int>{0, 1, 2, 3}));

    adj[3].emplace_back(1, -2); // The cycle 1 -> 2 -> 3 -> 1 now has total weight -1.
    assert(!spfa(0));
    return 0;
}

```

### 4.3.7 DAG Shortest Path (Topological DFS)

Given a starting node in a weighted, directed acyclic graph (DAG), determine the minimum distance to every reachable node. Because the graph is acyclic, its nodes admit a topological ordering in which every edge points forward. Relaxing edges in this order settles each node's distance in a single pass, with no priority queue and no restriction on edge signs. This makes it both faster than Dijkstra's algorithm and able to handle negative edge weights, as long as the graph has no cycles. The same pass computes longest paths if the relaxation comparison is reversed, which is the standard way to find the critical path in a schedule of dependent tasks.

- `dag_shortest_path(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` whose indices represent the nodes. Each edge is stored as `(neighbor, weight)` and may have any sign. `dist[v]` is set to `INF` for nodes not reachable from `start`, and `pred` stores the shortest-path tree for path reconstruction.
- `get_path(dest)` returns the path from `start` to `dest`, or an empty vector if `dest` is unreachable, using the state left by the most recent call to `dag_shortest_path()`.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

#### Time Complexity:

- $O(n + m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

#### Space Complexity:

- $O(\max(n, m))$  for storage of the graph.
- $O(n)$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```
#include <algorithm>
#include <cstdint>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

void dag_shortest_path(int start) {
    int n = static_cast<int>(adj.size());
    std::vector<char> visit(n);
    std::vector<int> order;
    auto dfs = [&](auto &&dfs, int u) -> void {
        visit[u] = true;
        for (auto [v, w] : adj[u]) {
            if (!visit[v]) {
                dfs(dfs, v);
            }
        }
        order.push_back(u);
    };
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            dfs(dfs, i);
        }
    }
    std::reverse(order.begin(), order.end());
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    for (int u : order) {
        if (dist[u] == INF) {
            continue;
        }
    }
}
```



```

    }
    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) { // Overflow warning.
            dist[v] = dist[u] + w;
            pred[v] = u;
        }
    }
}
}
}

std::vector<int> get_path(int dest) {
    if (dist[dest] == INF) {
        return {};
    }
    std::vector<int> path;
    for (int v = dest; v != -1; v = pred[v]) {
        path.push_back(v);
    }
    std::reverse(path.begin(), path.end());
    return path;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=1    w=-2    w=3
    //      0 ----> 1 ----> 3 ----> 4
    //      |      /      ^
    // w=5 |    / w=1    /
    //      v v      /w=1
    //      2 -----+
    adj.assign(5, {});
    adj[0].emplace_back(1, 1);
    adj[0].emplace_back(2, 5);
    adj[1].emplace_back(2, 1);
    adj[1].emplace_back(3, -2); // A negative edge, which Dijkstra could not handle.
    adj[2].emplace_back(3, 1);
    adj[3].emplace_back(4, 3);
    dag_shortest_path(0);
    assert((dist == vector<int64_t>{0, 1, 2, -1, 2}) && (pred == vector<int>{-1, 0, 1, 1, 3}));
    assert((get_path(4) == vector<int>{0, 1, 3, 4}));
    return 0;
}

```

### 4.3.8 All-Pairs Shortest Path (Floyd-Warshall)

Given a weighted, directed graph with possibly negative weights, determine the minimum distance between all pairs of start and destination nodes in the graph. Optionally, reconstruct a shortest path between two nodes using the precomputed next-hop matrix `next_node`.

Floyd-Warshall's algorithm is a dynamic program over intermediate nodes: for each node  $k$  in turn, every pair  $(i, j)$  is relaxed by considering a path through  $k$ , so once all  $k$  have been processed the matrix holds the all-pairs shortest distances.

- `init_floyd(n)` initializes `dist` and `next_node` for a graph of  $n$  nodes numbered  $[0, n)$ .
- `floyd_warshall()` updates the global adjacency matrix `dist` so `dist[u][v]` stores the shortest-path distance from  $u$  to  $v$ , updates `next_node` for path reconstruction, and returns whether the graph contains no negative-weight cycle.

- `get_path(start, dest)` returns the shortest path from `start` to `dest`, or an empty vector if `dest` is unreachable from `start`, provided the most recent call to `floyd_warshall()` returned true. If it returned false, a reachable negative-weight cycle leaves the distances and paths undefined.

For path reconstruction, `next_node[i][j]` stores the next node to visit after `i` on a current shortest path from `i` to `j`. It is initialized to `j` for every pair and, when a shorter route `i`  $\rightarrow$  `k`  $\rightarrow$  `j` is found, becomes `next_node[i][k]`. Repeatedly setting `i` to `next_node[i][j]` therefore walks the path from source to destination. This value is meaningful only when `dist[i][j] != INF`.

Overflow warning: All finite path distances and intermediate sums must fit in `int64_t`.

#### Time Complexity:

- $O(n^2)$  per call to `init_floyd()`, where  $n$  is the number of nodes.
- $O(n^3)$  per call to `floyd_warshall()`.
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

#### Space Complexity:

- $O(n^2)$  for storage of the graph, where  $n$  is the number of nodes.
- $O(1)$  auxiliary.
- $O(p)$  for the path returned by `get_path()`.

```
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<int64_t>> dist;
std::vector<std::vector<int>> next_node;

void init_floyd(int n) {
    dist.assign(n, std::vector<int64_t>(n));
    next_node.assign(n, std::vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0 : INF;
            next_node[i][j] = j;
        }
    }
}

bool floyd_warshall() {
    int n = static_cast<int>(dist.size());
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                // The INF guards avoid relaxing through an unreachable intermediate: with a negative edge,
                // INF + w < INF would otherwise give an unreachable pair a bogus finite distance.
                if (dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    next_node[i][j] = next_node[i][k];
                }
            }
        }
    }
    // Check for negative-weight cycles.
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < 0) {
            return false;
        }
    }
    return true;
}

std::vector<int> get_path(int start, int dest) {
    if (dist[start][dest] == INF) {
        return {};
    }
}
```

```

}
std::vector<int> path{start};
while (start != dest) {
    start = next_node[start][dest];
    path.push_back(start);
}
return path;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=1      w=2
    // 0 ----> 1 ----> 2
    // |           ^
    // +-----+
    //      w=5
    init_floyd(3);
    dist[0][1] = 1;
    dist[1][2] = 2;
    dist[0][2] = 5;
    assert(floyd_warshall());
    assert((dist == vector<vector<int64_t>>{{0, 1, 3}, {INF, 0, 2}, {INF, INF, 0}}));
    assert((get_path(0, 2) == vector<int>{0, 1, 2}));

    init_floyd(2);
    dist[0][1] = dist[1][0] = -1;
    assert(!floyd_warshall());
    return 0;
}

```

### 4.3.9 All-Pairs Shortest Path (Johnson)

Given a sparse, weighted directed graph with possibly negative edge weights, determine the minimum distance between every pair of nodes. Johnson's algorithm first computes a potential  $h(v)$  using Bellman-Ford, then replaces each edge weight  $w(u, v)$  with  $w'(u, v) = w(u, v) + h(u) - h(v)$ . Every reduced weight is nonnegative, so Dijkstra's algorithm can run from every node. Restoring the original distance afterward preserves all shortest paths because the potential terms telescope along each path.

Initializing every potential to 0 is equivalent to adding a virtual source with zero-weight edges to all nodes. Bellman-Ford therefore detects a negative cycle anywhere in the graph, not only one reachable from a particular original node. If such a cycle exists, shortest paths are undefined.

- `johnson_all_pairs(n)` populates `dist` and `next_node` for a global, pre-populated edge list `edges` whose nodes are numbered  $[0, n)$ . Each edge is stored as  $(u, v, weight)$ . Afterward, `dist[u][v]` is the shortest distance from  $u$  to  $v$ , or `INF` if  $v$  is unreachable from  $u$ . The function returns whether the graph contains no negative-weight cycle.
- `get_path(start, dest)` returns the shortest path from `start` to `dest`, or an empty vector if `dest` is unreachable from `start`, provided the most recent call to `johnson_all_pairs()` returned true. If it returned false, a reachable negative-weight cycle leaves the distances and paths undefined.

For path reconstruction, `next_node[u][v]` stores the first node after  $u$  on a shortest path to  $v$ , or  $-1$  if  $v$  is unreachable from  $u$ . Repeatedly advance  $u$  to `next_node[u][v]` until it equals  $v$ .

Overflow warning: All potentials, reduced weights, and path distances must fit in `int64_t`.

**Time Complexity:**

- $O(n \cdot m + n \cdot (n + m) \cdot \log n)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges. For a sparse connected graph, this is  $O(n \cdot m \cdot \log n)$ .
- $O(p)$  per call to `get_path()`, where  $p$  is the number of nodes in the returned path.

**Space Complexity:**

- $O(n^2 + \max(n, m))$  for the graph, distance matrix, and auxiliary storage.
- $O(p)$  for the path returned by `get_path()`.

```
#include <cstdint>
#include <functional>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::tuple<int, int, int>>> edges; // (u, v, weight)
std::vector<std::vector<int64_t>>> dist;
std::vector<std::vector<int>>> next_node;

bool johnson_all_pairs(int n) {
    std::vector<int64_t> potential(n);
    for (int i = 0; i < n; i++) {
        bool changed = false;
        for (auto [u, v, w] : edges) {
            int64_t pv = potential[u] + w; // Overflow warning.
            if (pv < potential[v]) {
                if (i == n - 1) {
                    return false;
                }
                potential[v] = pv;
                changed = true;
            }
        }
    }
    if (!changed) {
        break;
    }
}

std::vector<std::vector<std::pair<int, int64_t>>>> adj(n);
for (auto [u, v, w] : edges) {
    int64_t reduced = static_cast<int64_t>(w) + potential[u] - potential[v]; // Overflow warning.
    adj[u].emplace_back(v, reduced);
}

dist.assign(n, std::vector<int64_t>(n, INF));
next_node.assign(n, std::vector<int>(n, -1));
for (int start = 0; start < n; start++) {
    using qnode = std::pair<int64_t, int>; // (distance, node)
    std::priority_queue<qnode, std::vector<qnode>, std::greater<>> pq;
    dist[start][start] = 0;
    next_node[start][start] = start;
    pq.emplace(0, start);
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != dist[start][u]) {
            continue;
        }
        for (auto [v, w] : adj[u]) {
            int64_t dv = du + w; // Overflow warning.
            if (dv < dist[start][v]) {
                dist[start][v] = dv;
                next_node[start][v] = (u == start) ? v : next_node[start][u];
                pq.emplace(dv, v);
            }
        }
    }
}
```

```

    }
}
for (int v = 0; v < n; v++) {
    if (dist[start][v] != INF) {
        dist[start][v] += potential[v] - potential[start]; // Overflow warning.
    }
}
}
return true;
}

std::vector<int> get_path(int start, int dest) {
    if (next_node[start][dest] == -1) {
        return {};
    }
    std::vector<int> path{start};
    while (start != dest) {
        start = next_node[start][dest];
        path.push_back(start);
    }
    return path;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //
    //           w=3
    //           +-----+
    //           |         |
    //      w=1  v  w=-2    w=2  |
    // 0 -----> 1 -----> 2 -----> 3
    // |         ^
    // +-----+
    //      w=4
    edges = {{0, 1, 1}, {0, 2, 4}, {1, 2, -2}, {2, 3, 2}, {3, 1, 3}};
    assert(johnson_all_pairs(5)); // Node 4 is isolated.
    assert(
        (dist == vector<vector<int64_t>>{
            {0, 1, -1, 1, INF},
            {INF, 0, -2, 0, INF},
            {INF, 5, 0, 2, INF},
            {INF, 3, 1, 0, INF},
            {INF, INF, INF, INF, 0},
        })
    );
    assert((get_path(0, 3) == vector<int>{0, 1, 2, 3}));
    assert(get_path(0, 4).empty());

    edges = {{0, 1, -1}, {1, 0, -1}};
    assert(!johnson_all_pairs(2));
    return 0;
}

```

## 4.4 Spanning Trees

### 4.4.1 Minimum Spanning Tree (Prim)

For a connected, undirected, weighted graph with possibly negative weights, a minimum spanning tree (MST) connects all nodes using a subset of edges with minimum possible total weight. On disconnected input, this implementation instead finds the minimum spanning forest.

Prim's algorithm grows the tree from an arbitrary start node, repeatedly adding the minimum-weight edge that joins a new node to the current tree, with a priority queue supplying the cheapest such edge at each step.

- `prim_mst()` populates `mst` with the edges in the minimum spanning forest and returns the total MST weight for a global, bidirectionally pre-populated adjacency list `adj`, whose indices represent the nodes. Adjacency entries are stored as `(neighbor, weight)`, while each MST edge is stored as `(from, to, weight)`.

The priority queue stores candidate edges as `(weight, from, to)` and uses `std::greater` to make it a min-heap. To find a maximum spanning tree instead, use the default max-heap ordering. Multigraphs are supported; parallel edges are stored as separate adjacency entries and the algorithm automatically selects the minimum-weight one to each unvisited node.

**Time Complexity:**  $O(n + m \log m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(\max(n, m))$  auxiliary.

```
#include <cstdint>
#include <functional>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::vector<std::pair<int, int>>> adj; // adj[u] = {(v, weight), ...}
std::vector<std::tuple<int, int, int>> mst;      // (u, v, weight)

int64_t prim_mst() {
    int n = static_cast<int>(adj.size());
    mst.clear();
    std::vector<char> visit(n);
    int64_t total_weight = 0;
    for (int i = 0; i < n; i++) {
        if (visit[i]) {
            continue;
        }
        visit[i] = true;
        using qnode = std::tuple<int, int, int>; // (weight, u, v)
        std::priority_queue<qnode, std::vector<qnode>, std::greater<>> pq;
        for (auto [v, w] : adj[i]) {
            pq.emplace(w, i, v);
        }
        while (!pq.empty()) {
            auto [w, u, v] = pq.top();
            pq.pop();
            if (visit[v]) {
                continue;
            }
            visit[v] = true;
            mst.emplace_back(u, v, w);
            total_weight += w; // Overflow warning.
            for (auto [to, ew] : adj[v]) {
                pq.emplace(ew, v, to);
            }
        }
    }
}
```

```

    }
}
return total_weight;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].emplace_back(v, w);
    adj[v].emplace_back(u, w);
}

int main() {
    // Two connected components; MST routine returns a minimum spanning forest.
    //      w=4      w=1
    //  0 ----- 1      3 ----- 4
    //   \       /      / |
    //  w=3 \   / w=6   w=2 / | w=4
    //       \ /       / w=3 |
    //        2       5 ----- 6
    adj.assign(7, {});
    add_edge(0, 1, 4);
    add_edge(1, 2, 6);
    add_edge(2, 0, 3);
    add_edge(3, 4, 1);
    add_edge(4, 5, 2);
    add_edge(5, 6, 3);
    add_edge(6, 4, 4);
    assert(prim_mst() == 13);
    assert(
        (mst == vector<tuple<int, int, int>>{{0, 2, 3}, {0, 1, 4}, {3, 4, 1}, {4, 5, 2}, {5, 6, 3}})
    );
    return 0;
}

```

### 4.4.2 Minimum Spanning Tree (Kruskal)

For a connected, undirected, weighted graph with possibly negative weights, a minimum spanning tree (MST) connects all nodes using a subset of edges with minimum possible total weight. On disconnected input, this implementation instead finds the minimum spanning forest.

Kruskal's algorithm scans the edges in nondecreasing weight order, adding each edge whose endpoints lie in different components (tracked with a disjoint-set structure) and skipping any edge that would form a cycle.

- `kruskal_mst(n)` populates `mst` with the edge IDs in the minimum spanning forest and returns the total MST weight for a global, pre-populated edge list `edges` whose endpoints must be numbered  $[0, n)$ . Each edge is stored as a tuple  $(\text{weight}, u, v)$ .

Multigraphs are supported; parallel edges appear as separate entries in `edges` and any redundant ones are skipped once the cheaper edge has already connected the two components.

**Time Complexity:**  $O(n + m \log m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges. The internal DSU uses path compression and union by size, making each DSU operation  $O(\alpha(n))$  amortized.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(\max(n, m))$  auxiliary heap space and  $O(\log m)$  auxiliary stack space.

```

#include <algorithm>
#include <cstdint>
#include <numeric>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::tuple<int, int, int>> edges; // (weight, u, v)
std::vector<int> dsu_root, dsu_size, mst;

int find(int u) {
    return dsu_root[u] == u ? u : dsu_root[u] = find(dsu_root[u]);
}

int64_t kruskal_mst(int n) {
    mst.clear();
    std::vector<int> order(edges.size());
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [](int a, int b) {
        return std::get<0>(edges[a]) < std::get<0>(edges[b]);
    });
    dsu_root.assign(n, 0);
    dsu_size.assign(n, 1);
    std::iota(dsu_root.begin(), dsu_root.end(), 0);
    int64_t total_weight = 0;
    for (int id : order) {
        auto [w, u, v] = edges[id];
        u = find(u);
        v = find(v);
        if (u != v) {
            if (dsu_size[u] < dsu_size[v]) {
                std::swap(u, v);
            }
            dsu_root[v] = u;
            dsu_size[u] += dsu_size[v];
            mst.push_back(id);
            total_weight += w; // Overflow warning.
        }
    }
    return total_weight;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Two connected components; MST routine returns a minimum spanning forest.
    //      w=4          w=1
    //  0 ----- 1      3 ----- 4
    //   \      /        / |
    //  w=3 \    / w=6    w=2 / | w=4
    //       \  /        / w=3 |
    //        2          5 ----- 6
    edges.emplace_back(4, 0, 1); // (weight, u, v)
    edges.emplace_back(6, 1, 2);
    edges.emplace_back(3, 2, 0);
    edges.emplace_back(1, 3, 4);
    edges.emplace_back(2, 4, 5);
    edges.emplace_back(3, 5, 6);
    edges.emplace_back(4, 6, 4);
    assert(kruskal_mst(7) == 13);
    assert((mst == vector<int>{3, 4, 2, 5, 0}));
    return 0;
}

```



### 4.4.3 Kruskal Reconstruction Tree

Given a connected, undirected, weighted graph, build the Kruskal reconstruction tree of its minimum spanning tree. Kruskal's algorithm adds MST edges in nondecreasing weight order; whenever it joins two components, create a new internal node whose children are the two component roots and whose value is the joining edge weight. The original graph nodes become leaves.

The lowest common ancestor of two leaves is exactly the merge step that first connects their MST components, so its value is the maximum edge weight on the MST path between those two nodes. This turns bottleneck path queries into ordinary LCA queries on the reconstruction tree.

- `KruskalReconstructionTree(n, edges)` builds the tree from a connected graph whose weighted edges are stored as `(weight, u, v)`, where the `n` original nodes are numbered `[0, n)`. Parallel edges are supported.
- `max_edge_on_path(u, v)` returns the maximum edge weight on the MST path between original nodes `u` and `v`, which must be distinct.

**Time Complexity:**

- $O(m \log m + n \log n)$  for construction, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(\log n)$  per call to `max_edge_on_path()`.

**Space Complexity:**  $O(n \log n)$  object storage for the LCA table and node values, plus  $O(n + m)$  auxiliary during construction.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <tuple>
#include <vector>

class KruskalReconstructionTree {
    std::vector<int> tin, tout;
    std::vector<std::vector<int>>> up;
    std::vector<int64_t> value;

    bool is_ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const {
        if (is_ancestor(u, v)) {
            return u;
        }
        if (is_ancestor(v, u)) {
            return v;
        }
        for (int k = static_cast<int>(up.size()) - 1; k >= 0; k--) {
            if (!is_ancestor(up[k][u], v)) {
                u = up[k][u];
            }
        }
        return up[0][u];
    }
};

public:
KruskalReconstructionTree(int n, std::vector<std::tuple<int64_t, int, int>> edges) {
    assert(n > 0);
    int total_nodes = 2 * n - 1;
    std::vector<int> dsu_root(total_nodes), dsu_tree_root(total_nodes);
    std::vector<std::vector<int>>> tree(total_nodes);
    value.resize(total_nodes);
    std::iota(dsu_root.begin(), dsu_root.begin() + n, 0);
    std::iota(dsu_tree_root.begin(), dsu_tree_root.begin() + n, 0);
    auto find = [&](auto &&find, int u) -> int {
        return dsu_root[u] == u ? u : dsu_root[u] = find(find, dsu_root[u]);
    };
    std::sort(edges.begin(), edges.end());
```

```

int nodes = n;
for (auto [w, u, v] : edges) {
    u = find(find, u);
    v = find(find, v);
    if (u == v) {
        continue;
    }
    value[nodes] = w;
    tree[nodes].push_back(dsu_tree_root[u]);
    tree[nodes].push_back(dsu_tree_root[v]);
    dsu_root[u] = nodes;
    dsu_root[v] = nodes;
    dsu_root[nodes] = nodes;
    dsu_tree_root[nodes] = nodes;
    nodes++;
}
assert(nodes == total_nodes);
int lg = 1;
while ((1 << lg) <= nodes) {
    lg++;
}
tin.resize(nodes);
tout.resize(nodes);
up.assign(lg, std::vector<int>(nodes));
int timer = 0;
auto dfs = [&](auto &&dfs, int u, int p) -> void {
    tin[u] = timer++;
    up[0][u] = p;
    for (int k = 1; k < lg; k++) {
        up[k][u] = up[k - 1][up[k - 1][u]];
    }
    for (int v : tree[u]) {
        dfs(dfs, v, u);
    }
    tout[u] = timer;
};
dfs(dfs, nodes - 1, nodes - 1);
}

int64_t max_edge_on_path(int u, int v) const {
    assert(u != v);
    return value[lca(u, v)];
}
};

```

## Example Usage

```

using namespace std;

int main() {
    //
    //      0
    //    w=2 / | w=8
    //      / |
    //    1-----2
    //      | w=4 /
    // w=5 | /
    //    | / w=9
    //    3
    vector<tuple<int64_t, int, int>> edges{{2, 0, 1}, {4, 1, 2}, {5, 1, 3}, {8, 0, 2}, {9, 2, 3}};
    KruskalReconstructionTree tree(4, edges);

    // Reconstruction tree; internal nodes are labeled with their joining edge weight.
    //
    //      6 (5)
    //      / |
    //    5 (4) 3
    //    / |

```

```

// 4 (2) 2
// / |
// 0 1
assert(tree.max_edge_on_path(0, 2) == 4);
assert(tree.max_edge_on_path(0, 3) == 5);
assert(tree.max_edge_on_path(2, 3) == 5);
return 0;
}

```

#### 4.4.4 Minimum Spanning Tree (Boruvka)

For a connected, undirected, weighted graph with possibly negative weights, a minimum spanning tree (MST) connects all nodes using a subset of edges with minimum possible total weight. On disconnected input, this implementation instead finds the minimum spanning forest.

Boruvka's algorithm grows all components at once. In each round, every current component selects the cheapest edge leaving it, and all selected edges are added to the forest. The cut property makes each such edge safe, because it is the minimum-weight edge crossing the cut that separates its component from the rest of the graph. Every component merges with at least one other per round, so the number of components at least halves and only  $O(\log n)$  rounds are needed. Ties are broken by edge ID, giving each component a unique cheapest outgoing edge and making the forest deterministic; selecting one edge from both endpoints, or closing a cycle among mutually chosen components, is rejected by the disjoint-set check before the edge joins.

- `boruvka_mst(n)` populates `mst` with the edge IDs in the minimum spanning forest and returns the total MST weight for a global, pre-populated edge list `edges` whose endpoints must be numbered  $[0, n)$ . Each edge is stored as a tuple `(weight, u, v)`. Multigraphs are supported, with redundant parallel edges skipped once the cheaper one has connected the two components.

Unlike Prim's algorithm in section 4.4.1 and Kruskal's in section 4.4.2, no global ordering or priority queue is needed, since each round only compares edges within a component. That independence makes this the starting point for parallel MST computation, for the randomized linear-time algorithm of Karger, Klein, and Tarjan, and for geometric spanning trees where a spatial query replaces the edge scan, as in the Manhattan MST of section 7.3.11.

**Time Complexity:**  $O(n + m \log n)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges. Each of the  $O(\log n)$  rounds scans every edge once. The internal DSU uses path compression and union by size, making each DSU operation  $O(\alpha(n))$  amortized.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary.

```

#include <cstdint>
#include <numeric>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::tuple<int, int, int>> edges; // (weight, u, v)
std::vector<int> dsu_root, dsu_size, mst;

int find(int u) {
    return dsu_root[u] == u ? u : dsu_root[u] = find(dsu_root[u]);
}

int64_t boruvka_mst(int n) {
    mst.clear();
    dsu_root.assign(n, 0);
    dsu_size.assign(n, 1);
    std::iota(dsu_root.begin(), dsu_root.end(), 0);
}

```

```

int m = static_cast<int>(edges.size());
std::vector<int> cheapest(n);
int64_t total_weight = 0;
for (int components = n; components > 1;) {
    cheapest.assign(n, -1);
    for (int id = 0; id < m; id++) {
        auto [w, u, v] = edges[id];
        int ru = find(u), rv = find(v);
        if (ru == rv) {
            continue;
        }
        // Edges are scanned by increasing ID, so a strict comparison keeps the lowest ID on ties.
        if (cheapest[ru] == -1 || w < std::get<0>(edges[cheapest[ru]])) {
            cheapest[ru] = id;
        }
        if (cheapest[rv] == -1 || w < std::get<0>(edges[cheapest[rv]])) {
            cheapest[rv] = id;
        }
    }
    int merges = 0;
    for (int r = 0; r < n; r++) {
        if (cheapest[r] == -1) {
            continue;
        }
        auto [w, u, v] = edges[cheapest[r]];
        int ru = find(u), rv = find(v);
        if (ru == rv) {
            continue; // Already merged earlier in this round, by this edge or a cycle of selections.
        }
        if (dsu_size[ru] < dsu_size[rv]) {
            std::swap(ru, rv);
        }
        dsu_root[rv] = ru;
        dsu_size[ru] += dsu_size[rv];
        mst.push_back(cheapest[r]);
        total_weight += w; // Overflow warning.
        merges++;
    }
    if (merges == 0) {
        break; // Disconnected: no edge crosses between the remaining components.
    }
    components -= merges;
}
return total_weight;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Two connected components; MST routine returns a minimum spanning forest.
    //      w=4          w=1
    //  0 ----- 1      3 ----- 4
    //   \      /        /  |
    //  w=3 \    / w=6    w=2 /  | w=4
    //       \  /         / w=3 |
    //        2          5 ----- 6
    edges.emplace_back(4, 0, 1); // (weight, u, v)
    edges.emplace_back(6, 1, 2);
    edges.emplace_back(3, 2, 0);
    edges.emplace_back(1, 3, 4);
    edges.emplace_back(2, 4, 5);
    edges.emplace_back(3, 5, 6);
    edges.emplace_back(4, 6, 4);
}

```

```

assert(boruvka_mst(7) == 13);
// The same forest that section 4.4.2 finds, discovered by component instead of by weight.
assert((mst == vector<int>{2, 0, 3, 4, 5}));
return 0;
}

```

### 4.4.5 Minimum Spanning Arborescence (Edmonds)

Given a weighted, directed graph and a root node, a spanning arborescence is a set of edges forming a tree in which every other node is reachable from the root along directed edges. Equivalently, every node except the root has exactly one incoming edge. The minimum spanning arborescence minimizes the total edge weight and is the directed analog of the minimum spanning tree.

Edmonds' (a.k.a. the Chu-Liu/Edmonds) algorithm computes the arborescence by first selecting, for each non-root node, its cheapest incoming edge. If these choices form no cycle, they already constitute the answer. Otherwise each cycle is contracted into a single supernode, and every edge entering the cycle has its weight reduced by the cost of the edge it would replace inside the cycle. Solving the problem on the contracted graph and expanding the cycles back yields the optimum. The implementation below repeats this contraction in rounds, accumulating the selected weights, until no cycle remains.

- `directed_mst(n, edges, root)` returns the total weight of the minimum spanning arborescence rooted at `root` over `n` nodes numbered `[0, n)`, or `std::nullopt` if no arborescence exists. Edges are given as `(from, to, weight)` triples; parallel edges and self-loops are allowed, with self-loops simply ignored.

**Time Complexity:**  $O(n \cdot m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(n + m)$  auxiliary.

```

#include <cstdint>
#include <optional>
#include <tuple>
#include <vector>

using Edge = std::tuple<int, int, int64_t>; // (u, v, weight)

std::optional<int64_t> directed_mst(int n, std::vector<Edge> edges, int root) {
    const int64_t INF = INT64_MAX / 4;
    int64_t total_dist = 0;
    while (true) {
        std::vector<int64_t> min_in(n, INF);
        std::vector<int> pre(n, -1);
        for (const auto &[u, v, w] : edges) {
            if (u != v && w < min_in[v]) {
                min_in[v] = w;
                pre[v] = u;
            }
        }
        for (int v = 0; v < n; v++) {
            if (v != root && min_in[v] == INF) {
                return std::nullopt;
            }
        }
        // Identify cycles formed by the chosen incoming edges, labeling each with an ID in comp.
        int cycles = 0;
        std::vector<int> comp(n, -1), seen(n, -1);
        min_in[root] = 0;
        for (int v = 0; v < n; v++) {
            total_dist += min_in[v]; // Overflow warning.
            int u = v;
            while (seen[u] != v && comp[u] == -1 && u != root) {
                seen[u] = v;
                u = pre[u];
            }
        }
    }
}

```

```

    if (u != root && comp[u] == -1) {
        for (int w = pre[u]; w != u; w = pre[w]) {
            comp[w] = cycles;
        }
        comp[u] = cycles++;
    }
}
if (cycles == 0) {
    break; // No cycle remains; the selected edges form the arborescence.
}
// Contract each cycle into a supernode and reduce the weights of edges entering it.
for (int v = 0; v < n; v++) {
    if (comp[v] == -1) {
        comp[v] = cycles++;
    }
}
for (auto &[u, v, w] : edges) {
    int cu = comp[u], cv = comp[v];
    if (cu != cv) {
        w -= min_in[v]; // Overflow warning.
    }
    u = cu;
    v = cv;
}
n = cycles;
root = comp[root];
}
return total_dist;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //          w=1
    // root=0 -----> 1 <-----+
    //          |          / |          |
    //          |          w=2 |          |
    // w=5| /          |w=2   |w=4
    //    v v          v      |
    //    2 -----> 3 -----+
    //          w=3
    vector<Edge> edges{
        {0, 1, 1}, {0, 2, 5}, {1, 2, 2}, {2, 3, 3}, {3, 1, 4},
    };
    assert(directed_mst(4, edges, 0) == 6); // Cheapest edges 0->1->2->3 form a tree of weight 6.

    //          w=1    w=2
    // root=0 ---> 1 ---> 2      3
    vector<Edge> disconnected{{0, 1, 1}, {1, 2, 1}};
    assert(!directed_mst(4, disconnected, 0)); // Node 3 is unreachable from root 0.

    vector<Edge> negative{{0, 1, -1}};
    assert(directed_mst(2, negative, 0) == -1); // A weight of -1 is a valid result.
    return 0;
}

```

## 4.5 Flows and Cuts

---

### 4.5.1 Maximum Flow (Ford-Fulkerson)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

The Ford-Fulkerson method repeatedly finds any augmenting path from source to sink in the residual graph (here by depth-first search) and pushes as much flow as the path allows, until no augmenting path remains. This algorithm should only be used on graphs with integer capacities, because some real-valued inputs can make it fail to terminate. The Edmonds-Karp algorithm instead uses breadth-first search and always terminates in polynomial time.

- `max_flow(source, sink)` modifies the global residual capacity matrix `cap` and returns the maximum flow. Nodes are numbered  $[0, n)$ , where  $n$  is `cap.size()`.

**Time Complexity:**  $O(n^2 \cdot f)$  per call, where  $n$  is the number of nodes and  $f$  is the maximum flow.

**Space Complexity:**

- $O(n^2)$  for storage of the flow network, where  $n$  is the number of nodes.
- $O(n)$  auxiliary stack space.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<int64_t>> cap;
std::vector<char> visit;

int64_t dfs(int u, int64_t f, int sink) {
    int n = static_cast<int>(cap.size());
    if (u == sink) {
        return f;
    }
    visit[u] = true;
    for (int v = 0; v < n; v++) {
        if (!visit[v] && cap[u][v] > 0) {
            int64_t flow = dfs(v, std::min(f, cap[u][v]), sink);
            if (flow > 0) {
                cap[u][v] -= flow;
                cap[v][u] += flow;
                return flow;
            }
        }
    }
    return 0;
}

int64_t max_flow(int source, int sink) {
    assert(source != sink);
    int n = static_cast<int>(cap.size());
    int64_t total = 0;
    while (true) {
        visit.assign(n, false);
        int64_t flow = dfs(source, INF, sink);
        if (flow == 0) {
            break;
        }
        total += flow;
    }
}
```

```
    return total;
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \       |
    // 3/4 /   \ 1/1   | 2/2
    //    /     v       v
    //   0       4 ----> 5
    //    \     ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //       2
    int nodes = 6;
    cap.assign(nodes, vector<int64_t>(nodes));
    cap[0][1] = 4;
    cap[0][2] = 3;
    cap[1][3] = 2;
    cap[1][4] = 1;
    cap[2][4] = 2;
    cap[3][5] = 2;
    cap[4][5] = 3;
    assert(max_flow(0, 5) == 5);
    return 0;
}
```

### 4.5.2 Maximum Flow (Edmonds-Karp)

Given a flow network with integer or floating-point capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Edmonds-Karp is the Ford-Fulkerson method with the augmenting path always chosen as a shortest one (fewest edges) via breadth-first search. This bounds the number of augmentations at  $O(n \cdot m)$ , independent of the capacity magnitudes.

- `EdmondsKarp<T>(n)` constructs an empty flow network with nodes numbered  $[0, n)$ .
- `add_edge(u, v, cap)` adds a directed edge from `u` to `v` with capacity `cap`.
- `max_flow(source, sink)` modifies the residual network and returns the maximum flow.
- `clear_flow()` resets all edge flows to zero, allowing a fresh recomputation on the same graph.
- `min_cut(source)` returns the source side of a minimum cut after `max_flow()` has been called.

Repeated calls to `max_flow()` continue augmenting from the current flow. This is useful after adding new edges or increasing capacities; call `clear_flow()` first to recompute from zero. The capacity type `T` should be signed, since reverse residual edges store negative flow.

#### Time Complexity:

- $O(n \cdot m^2)$  per call to `max_flow()`, where  $n$  is the number of nodes and  $m$  is the number of edges. For integer capacities, this is also  $O(m \cdot f)$ , where  $f$  is the maximum flow.
- $O(n + m)$  per call to `min_cut()`.

**Space Complexity:**  $O(\max(n, m))$  for storage of the flow network, where  $n$  is the number of nodes and  $m$  is the number of edges.



```

#include <algorithm>
#include <cassert>
#include <limits>
#include <queue>
#include <vector>

template<typename T>
class EdmondsKarp {
    struct Edge {
        int u, v, rev;
        T cap, flow;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    T flow;
    std::vector<std::vector<Edge>> adj;

public:
    explicit EdmondsKarp(int n) : nodes(n), flow(0), adj(n) {}

    void add_edge(int u, int v, T cap) {
        adj[u].push_back(Edge{u, v, static_cast<int>(adj[v].size()), cap, 0});
        adj[v].push_back(Edge{v, u, static_cast<int>(adj[u].size()) - 1, 0, 0});
    }

    T max_flow(int source, int sink) {
        assert(source != sink);
        while (true) {
            std::vector<Edge*> pred(nodes);
            std::queue<int> q;
            q.push(source);
            while (!q.empty() && pred[sink] == nullptr) {
                int u = q.front();
                q.pop();
                for (Edge &e : adj[u]) {
                    if (e.v != source && pred[e.v] == nullptr && residual(e)) {
                        pred[e.v] = &e;
                        q.push(e.v);
                    }
                }
            }
            if (pred[sink] == nullptr) {
                break;
            }
            T aug = std::numeric_limits<T>::max();
            for (int u = sink; u != source; u = pred[u]->u) {
                aug = std::min(aug, pred[u]->cap - pred[u]->flow);
            }
            for (int u = sink; u != source; u = pred[u]->u) {
                pred[u]->flow += aug;
                adj[pred[u]->v][pred[u]->rev].flow -= aug;
            }
            flow += aug;
        }
        return flow;
    }

    void clear_flow() {
        for (auto &edges : adj) {
            for (Edge &e : edges) {
                e.flow = 0;
            }
        }
        flow = 0;
    }
};

```

```

std::vector<char> min_cut(int source) const {
    std::vector<char> reachable(nodes);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const Edge &e : adj[u]) {
            if (!reachable[e.v] && residual(e)) {
                reachable[e.v] = true;
                q.push(e.v);
            }
        }
    }
    return reachable;
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \      |
    // 3/4 /   \ 1/1   | 2/2
    //   /       v      v
    //  0        4 ----> 5
    //   \       ^      3/3
    // 2/3 \   / 2/2
    //       v /
    //       2
    EdmondsKarp<int> g(6);
    g.add_edge(0, 1, 4);
    g.add_edge(0, 2, 3);
    g.add_edge(1, 3, 2);
    g.add_edge(1, 4, 1);
    g.add_edge(2, 4, 2);
    g.add_edge(3, 5, 2);
    g.add_edge(4, 5, 3);
    int flow = g.max_flow(0, 5);
    assert(flow == 5);
    assert(g.max_flow(0, 5) == 5);
    assert((g.min_cut(0) == vector<char>{true, true, true, false, false, false}));
    g.clear_flow();
    assert(g.max_flow(0, 5) == 5);
    return 0;
}

```

### 4.5.3 Maximum Flow and Decomposition (Dinic)

Given a flow network with integer or floating-point capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Dinic's algorithm proceeds in phases. Each phase runs a breadth-first search to build a level graph of shortest residual distances from the source, then finds a blocking flow that saturates that level graph using depth-first

search before repeating on the new residual graph. Residual edges are stored in a flat array so callers can inspect edge flows and decompose the final flow into source-to-sink paths.

- `Dinic<T>(n)` constructs an empty flow network with nodes numbered  $[0, n)$ .
- `add_edge(u, v, cap)` adds a directed edge from `u` to `v` with capacity `cap` and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.
- `max_flow(source, sink)` modifies the residual network and returns the maximum flow.
- `clear_flow()` resets all edge flows to zero, allowing a fresh recomputation on the same graph.
- `min_cut(source)` returns the source side of a minimum cut after `max_flow()` has been called.
- `decompose(source, sink)` decomposes the current flow into paths from `source` to `sink`.

Repeated calls to `max_flow()` continue augmenting from the current flow. This is useful after adding new edges or increasing capacities; call `clear_flow()` first to recompute from zero. The capacity type `T` should be signed, since reverse residual edges store negative flow.

#### Time Complexity:

- $O(n^2 \cdot m)$  per call to `max_flow()`, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n + m)$  per call to `min_cut()`.
- $O(p \cdot m)$  per call to `decompose()`, where  $p$  is the number of paths returned.

#### Space Complexity:

- $O(\max(n, m))$  for storage of the flow network.
- $O(n)$  auxiliary stack space and  $O(n)$  auxiliary heap space for `max_flow()`.
- $O(n + m)$  auxiliary heap space for `decompose()`, not counting the returned paths.

```
#include <algorithm>
#include <cassert>
#include <limits>
#include <queue>
#include <vector>

template<typename T>
class Dinic {
    struct Edge {
        int u, v;
        T cap, flow;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    T flow;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;
    std::vector<int> dist, ptr;

    bool bfs(int source, int sink) {
        dist.assign(nodes, -1);
        dist[source] = 0;
        std::queue<int> q;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (dist[e.v] < 0 && residual(e)) {
                    dist[e.v] = dist[u] + 1;
                    q.push(e.v);
                }
            }
        }
    }
};
```

```

    return dist[sink] >= 0;
}

T dfs(int u, T pushed, int sink) {
    if (u == sink || pushed <= EPS) {
        return pushed;
    }
    for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
        int id = adj[u][ptr[u]];
        Edge &e = edges[id];
        if (dist[e.v] == dist[u] + 1 && residual(e)) {
            T next = dfs(e.v, std::min(pushed, e.cap - e.flow), sink);
            if (next > EPS) {
                e.flow += next;
                edges[id ^ 1].flow -= next;
                return next;
            }
        }
    }
    return 0;
}

public:
    struct FlowPath {
        T flow;
        std::vector<int> nodes, edges;
    };

    explicit Dinic(int n) : nodes(n), flow(0), adj(n), dist(n), ptr(n) {}

    int add_edge(int u, int v, T cap) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        edges.push_back(Edge{u, v, cap, 0});
        adj[v].push_back(id + 1);
        edges.push_back(Edge{v, u, 0, 0});
        return id;
    }

    T edge_flow(int id) const { return edges[id].flow; }

    T max_flow(int source, int sink) {
        assert(source != sink);
        while (bfs(source, sink)) {
            ptr.assign(nodes, 0);
            while (true) {
                T pushed = dfs(source, std::numeric_limits<T>::max(), sink);
                if (pushed <= EPS) {
                    break;
                }
                flow += pushed;
            }
        }
        return flow;
    }

    void clear_flow() {
        for (Edge &e : edges) {
            e.flow = 0;
        }
        flow = 0;
    }

    std::vector<char> min_cut(int source) const {
        std::vector<char> reachable(nodes);
        std::queue<int> q;
        reachable[source] = true;
        q.push(source);
    }

```

```

while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int id : adj[u]) {
        const Edge &e = edges[id];
        if (!reachable[e.v] && residual(e)) {
            reachable[e.v] = true;
            q.push(e.v);
        }
    }
}
return reachable;
}

std::vector<FlowPath> decompose(int source, int sink) const {
    std::vector<T> rem(edges.size());
    for (int id = 0; id < static_cast<int>(edges.size()); id++) {
        rem[id] = edges[id].flow;
    }
    std::vector<FlowPath> res;
    while (true) {
        FlowPath path{std::numeric_limits<T>::max(), {source}, {}};
        std::vector<char> seen(nodes);
        auto dfs = [&](auto &&dfs, int u, int sink) -> bool {
            if (u == sink) {
                return true;
            }
            seen[u] = true;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (rem[id] > EPS && !seen[e.v]) {
                    path.edges.push_back(id);
                    path.nodes.push_back(e.v);
                    if (dfs(dfs, e.v, sink)) {
                        return true;
                    }
                    path.nodes.pop_back();
                    path.edges.pop_back();
                }
            }
            return false;
        };
        if (!dfs(dfs, source, sink)) {
            break;
        }
        for (int id : path.edges) {
            path.flow = std::min(path.flow, rem[id]);
        }
        for (int id : path.edges) {
            rem[id] -= path.flow;
        }
        res.push_back(path);
    }
    return res;
}
};

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //      2/2

```

```

//      1 -----> 3
//      / \      |
// 3/4 /   \ 1/1 | 2/2
//      /     v   v
// 0      4 -----> 5
//      \     ^   3/3
// 2/3 \   / 2/2
//      v /
//      2

Dinic<int> g(6);
int id01 = g.add_edge(0, 1, 4);
int id02 = g.add_edge(0, 2, 3);
int id13 = g.add_edge(1, 3, 2);
int id14 = g.add_edge(1, 4, 1);
int id24 = g.add_edge(2, 4, 2);
int id35 = g.add_edge(3, 5, 2);
int id45 = g.add_edge(4, 5, 3);
int flow = g.max_flow(0, 5);
cout << "Max flow from 0 to 5: " << flow << endl;
assert(flow == 5);
assert(g.max_flow(0, 5) == 5);
assert(g.edge_flow(id01) == 3);
assert(g.edge_flow(id02) == 2);
assert(g.edge_flow(id13) == 2);
assert(g.edge_flow(id14) == 1);
assert(g.edge_flow(id24) == 2);
assert(g.edge_flow(id35) == 2);
assert(g.edge_flow(id45) == 3);
vector<char> cut = g.min_cut(0);
assert(cut[0] && !cut[5]);
auto paths = g.decompose(0, 5);
cout << "Flow decomposition:" << endl;
int total = 0;
for (const auto &path : paths) {
    cout << path.flow << ": ";
    for (int i = 0; i < static_cast<int>(path.nodes.size()); i++) {
        if (i > 0) {
            cout << " -> ";
        }
        cout << path.nodes[i];
    }
    cout << endl;
    assert(path.nodes.front() == 0 && path.nodes.back() == 5);
    assert(path.edges.size() + 1 == path.nodes.size());
    total += path.flow;
}
assert(total == 5);
g.clear_flow();
assert(g.max_flow(0, 5) == 5);
return 0;
}

```

#### Example Output

```

Max flow from 0 to 5: 5
Flow decomposition:
2: 0 -> 1 -> 3 -> 5
1: 0 -> 1 -> 4 -> 5
2: 0 -> 2 -> 4 -> 5

```

### 4.5.4 Maximum Flow (Push-Relabel)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Rather than augmenting along whole paths, the push-relabel algorithm maintains a preflow and a height label on each node, repeatedly pushing excess flow to lower-labeled neighbors and relabeling (raising) a node whose excess cannot yet be pushed, until no node other than the source and sink holds excess.

- `push_relabel(source, sink)` returns the maximum flow for a global capacity matrix `cap`, whose row and column indices represent the nodes.
- `min_cut(source)` returns the source side of a minimum cut after `push_relabel()` has been called.

Unlike Ford-Fulkerson, the  $O(n^3)$  bound does not improve when the integral maximum flow  $f$  is small; the matrix-based Ford-Fulkerson implementation runs in  $O(n^2 \cdot f)$ .

#### Time Complexity:

- $O(n^3)$  per call to `push_relabel()`, where  $n$  is the number of nodes.
- $O(n^2)$  per call to `min_cut()`.

#### Space Complexity:

- $O(n^2)$  for storage of the flow network, where  $n$  is the number of nodes.
- $O(n)$  auxiliary for `push_relabel()`.

```
#include <algorithm>
#include <cassert>
#include <climits>
#include <cstdint>
#include <queue>
#include <vector>

std::vector<std::vector<int64_t>> cap, f;

int64_t push_relabel(int source, int sink) {
    assert(source != sink);
    int n = static_cast<int>(cap.size());
    f.assign(n, std::vector<int64_t>(n));
    std::vector<int64_t> excess(n);
    std::vector<int> height(n, max_height(n));
    height[source] = n - 1;
    for (int i = 0; i < n; i++) {
        f[source][i] = cap[source][i];
        f[i][source] = -f[source][i];
        excess[i] = cap[source][i];
    }
    int size = 0;
    while (true) {
        if (size == 0) {
            for (int i = 0; i < n; i++) {
                if (i != source && i != sink && excess[i] > 0) {
                    if (size != 0 && height[i] > height[max_height[0]]) {
                        size = 0;
                    }
                    max_height[size++] = i;
                }
            }
        }
        if (size == 0) {
            break;
        }
        while (size != 0) {
            int i = max_height[size - 1];
            bool pushed = false;
            for (int j = 0; j < n && excess[i] != 0; j++) {
                if (height[i] == height[j] + 1 && cap[i][j] - f[i][j] > 0) {
                    int64_t df = std::min(cap[i][j] - f[i][j], excess[i]);
                    f[i][j] += df;
                    f[j][i] -= df;
                    excess[i] -= df;
                    excess[j] += df;
                }
            }
        }
    }
}
```

```

        if (excess[i] == 0) {
            size--;
        }
        pushed = true;
    }
}
if (pushed) {
    continue;
}
height[i] = INT_MAX / 2;
for (int j = 0; j < n; j++) {
    if (height[i] > height[j] + 1 && cap[i][j] - f[i][j] > 0) {
        height[i] = height[j] + 1;
    }
}
if (height[i] > height[max_height[0]]) {
    size = 0;
    break;
}
}
}
int64_t max_flow = 0;
for (int i = 0; i < n; i++) {
    max_flow += f[source][i];
}
return max_flow;
}

std::vector<char> min_cut(int source) {
    int n = static_cast<int>(cap.size());
    std::vector<char> reachable(n);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v = 0; v < n; v++) {
            if (!reachable[v] && cap[u][v] - f[u][v] > 0) {
                reachable[v] = true;
                q.push(v);
            }
        }
    }
    return reachable;
}
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //       1 -----> 3
    //      / \      |
    // 3/4 /   \ 1/1 | 2/2
    //   /     v     v
    //  0      4 ----> 5
    //   \     ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //       2
    int nodes = 6;
    cap.assign(nodes, vector<int64_t>(nodes));
}

```



```

cap[0][1] = 4;
cap[0][2] = 3;
cap[1][3] = 2;
cap[1][4] = 1;
cap[2][4] = 2;
cap[3][5] = 2;
cap[4][5] = 3;
assert(push_relabel(0, 5) == 5);
vector<char> cut = min_cut(0);
assert(cut[0] && !cut[5]);
return 0;
}

```

### 4.5.5 Flow with Lower Bounds

Given a directed network where each edge has lower and upper capacity bounds, find whether the lower bounds can be satisfied, and optionally optimize an  $s$ - $t$  flow value. Reserve each edge's lower bound, leaving the difference between its bounds as residual capacity, then record the forced imbalance: for an edge from  $u$  to  $v$ , its lower-bound flow must leave  $u$  and enter  $v$ . A super-source supplies every node with net demand, and every node with net surplus sends that surplus to a super-sink. The bounds are feasible exactly when all such auxiliary edges can be saturated.

For  $s$ - $t$  flow, add infinite auxiliary edges in both directions between  $s$  and  $t$  before checking feasibility. The difference between their flows is one feasible signed value. After the feasibility check, augmenting from  $s$  to  $t$  maximizes the value; augmenting from  $t$  to  $s$  minimizes it.

- `BoundedFlow(n)` constructs a directed lower-bound flow network with nodes in  $[0, n)$ .
- `add_edge(u, v, lo, hi)` adds an edge with lower capacity `lo` and upper capacity `hi`.
- `feasible_circulation()` returns whether all edge bounds can be satisfied with flow conserved at every node.
- `max_flow(source, sink)` returns the maximum feasible flow from `source` to `sink`, or `std::nullopt` if no feasible flow exists.
- `min_flow(source, sink)` returns the minimum feasible flow from `source` to `sink`, or `std::nullopt` if no feasible flow exists.
- `edge_flows()` returns one feasible flow value for each original edge after a successful call.

The flow value is the net flow leaving `source` and may be negative when the bounds force flow in the opposite direction. All capacities should be nonnegative integers and `lo`  $\leq$  `hi`. Choose `INF` larger than the absolute value of any possible finite flow. Each feasibility or optimization call starts from the original network, so different variants may be solved successively on the same instance.

**Time Complexity:**  $O(n^2 \cdot (n + m))$  per feasibility check or optimized flow call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(\max(n, m))$  for network storage and auxiliary arrays.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <optional>
#include <queue>
#include <vector>

class BoundedFlow {
    struct Edge {
        int v, rev;
        int64_t cap;
    };

    struct OriginalEdge {
        int u, v, index;
        int64_t lo, hi;
    };

```

```

};

static constexpr int64_t INF = (1LL << 60);
int nodes;
std::vector<std::vector<Edge>> adj;
std::vector<OriginalEdge> original;
std::vector<int> level, ptr;

void add_residual_edge(int u, int v, int64_t cap) {
    adj[u].push_back(Edge{v, static_cast<int>(adj[v].size()), cap});
    adj[v].push_back(Edge{u, static_cast<int>(adj[u].size()) - 1, 0});
}

bool bfs(int source, int sink) {
    level.assign(static_cast<int>(adj.size()), -1);
    level[source] = 0;
    std::queue<int> q;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const Edge &e : adj[u]) {
            if (e.cap > 0 && level[e.v] == -1) {
                level[e.v] = level[u] + 1;
                q.push(e.v);
            }
        }
    }
    return level[sink] != -1;
}

int64_t dfs(int u, int sink, int64_t pushed) {
    if (u == sink || pushed == 0) {
        return pushed;
    }
    for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
        Edge &e = adj[u][ptr[u]];
        if (e.cap > 0 && level[e.v] == level[u] + 1) {
            int64_t next = dfs(e.v, sink, std::min(pushed, e.cap));
            if (next > 0) {
                e.cap -= next;
                adj[e.v][e.rev].cap += next;
                return next;
            }
        }
    }
    return 0;
}

int64_t dinic(int source, int sink) {
    int64_t flow = 0;
    while (bfs(source, sink)) {
        ptr.assign(static_cast<int>(adj.size()), 0);
        while (int64_t pushed = dfs(source, sink, INF)) {
            flow += pushed;
        }
    }
    return flow;
}

bool satisfy_demands(
    int source, int sink, int &backward_index, int &forward_index, int64_t &forced_flow
) {
    int super_source = static_cast<int>(adj.size());
    int super_sink = super_source + 1;
    adj.resize(super_sink + 1);
    level.resize(super_sink + 1);
    ptr.resize(super_sink + 1);

```

```

std::vector<int64_t> balance(nodes);
for (const OriginalEdge &e : original) {
    balance[e.u] -= e.lo;
    balance[e.v] += e.lo;
}
if (source != -1) {
    backward_index = static_cast<int>(adj[sink].size());
    add_residual_edge(sink, source, INF);
    forward_index = static_cast<int>(adj[source].size());
    add_residual_edge(source, sink, INF);
}
int64_t need = 0;
for (int u = 0; u < nodes; u++) {
    if (balance[u] > 0) {
        add_residual_edge(super_source, u, balance[u]);
        need += balance[u];
    } else if (balance[u] < 0) {
        add_residual_edge(u, super_sink, -balance[u]);
    }
}
bool ok = dinic(super_source, super_sink) == need;
forced_flow = 0;
if (source != -1) {
    forced_flow = adj[source][adj[sink][backward_index].rev].cap -
        adj[sink][adj[source][forward_index].rev].cap;
}
return ok;
}

std::optional<int64_t> optimize_flow(int source, int sink, bool maximize) {
    int backward_index = -1, forward_index = -1;
    int64_t value = 0;
    if (!satisfy_demands(source, sink, backward_index, forward_index, value)) {
        return std::nullopt;
    }
    Edge &backward = adj[sink][backward_index];
    Edge &backward_reverse = adj[source][backward.rev];
    Edge &forward = adj[source][forward_index];
    Edge &forward_reverse = adj[sink][forward.rev];
    backward.cap = backward_reverse.cap = 0;
    forward.cap = forward_reverse.cap = 0;
    return maximize ? value + dinic(source, sink) : value - dinic(sink, source);
}

void reset() {
    adj.assign(nodes, {});
    level.assign(nodes, 0);
    ptr.assign(nodes, 0);
    for (OriginalEdge &e : original) {
        e.index = static_cast<int>(adj[e.u].size());
        add_residual_edge(e.u, e.v, e.hi - e.lo);
    }
}

public:
explicit BoundedFlow(int n) : nodes(n), adj(n), level(n), ptr(n) {}

void add_edge(int u, int v, int64_t lo, int64_t hi) {
    assert(0 <= u && u < nodes && 0 <= v && v < nodes);
    assert(0 <= lo && lo <= hi);
    original.push_back(OriginalEdge{u, v, static_cast<int>(adj[u].size()), lo, hi});
    add_residual_edge(u, v, hi - lo);
}

bool feasible_circulation() {
    reset();
    int backward_index = -1, forward_index = -1;
    int64_t value = 0;

```

```

    return satisfy_demands(-1, -1, backward_index, forward_index, value);
}

std::optional<int64_t> max_flow(int source, int sink) {
    assert(0 <= source && source < nodes && 0 <= sink && sink < nodes && source != sink);
    reset();
    return optimize_flow(source, sink, true);
}

std::optional<int64_t> min_flow(int source, int sink) {
    assert(0 <= source && source < nodes && 0 <= sink && sink < nodes && source != sink);
    reset();
    return optimize_flow(source, sink, false);
}

std::vector<int64_t> edge_flows() const {
    std::vector<int64_t> flow;
    flow.reserve(original.size());
    for (const OriginalEdge &e : original) {
        const Edge &forward = adj[e.u][e.index];
        flow.push_back(e.hi - forward.cap);
    }
    return flow;
}
};

```

## Example Usage

```

using namespace std;

int main() {
    //           [1,4]       [1,3]
    //      0 -----> 1 -----> 3
    //      |               ^
    // [0,2] |               | [0,2]
    //      v               |
    //      2 -----+-----+
    BoundedFlow g(4);
    g.add_edge(0, 1, 1, 4);
    g.add_edge(0, 2, 0, 2);
    g.add_edge(1, 3, 1, 3);
    g.add_edge(2, 3, 0, 2);
    assert(g.max_flow(0, 3) == 5);
    vector<int64_t> max_flow = g.edge_flows();
    assert(max_flow[0] + max_flow[1] == 5);

    assert(g.min_flow(0, 3) == 1);

    BoundedFlow rev(2);
    rev.add_edge(1, 0, 1, 1);
    assert(rev.max_flow(0, 1) == -1);
    assert(rev.min_flow(0, 1) == -1);
    assert((rev.edge_flows() == vector<int64_t>{1}));

    //           [2,3]
    //      0 -----> 1
    //      ^               |
    // [2,3] |               | [2,3]
    //      |               |
    //      2 <-----+
    BoundedFlow circulation(3);
    circulation.add_edge(0, 1, 2, 3);
    circulation.add_edge(1, 2, 2, 3);
    circulation.add_edge(2, 0, 2, 3);
    assert(circulation.feasible_circulation());
    assert((circulation.edge_flows() == vector<int64_t>{2, 2, 2}));
}

```

```

return 0;
}

```

### 4.5.6 Minimum-Cost Maximum-Flow (SPFA)

Given a flow network with capacities and edge costs, find a minimum-cost flow from a source node to a sink node. This is the successive shortest path method: repeatedly find a minimum-cost augmenting path from source to sink in the residual graph and push as much flow as it allows, until the flow target is met or no path remains. The path search uses SPFA (queue-based Bellman-Ford), since residual edges carry negated costs.

- `MinCostMaxFlow<T, C>(n)` constructs an empty residual network with nodes numbered  $[0, n)$ .
- `add_edge(u, v, cap, cost)` adds a directed edge from `u` to `v` with capacity `cap` and per-unit cost `cost`, and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.
- `clear_flow()` resets all edge flows to zero.
- `min_cost_flow(source, sink, target_flow)` sends up to `target_flow` additional units of flow and returns a pair  $(\text{flow}, \text{cost})$  for the flow sent by that call. If the returned flow is smaller than `target_flow`, the residual network cannot carry the requested amount.

This implementation uses shortest augmenting paths with SPFA, so it supports negative edge costs as long as the residual graph has no reachable negative-cost cycle. For dense or adversarial inputs, a potential-based Dijkstra version is usually faster when reduced costs are nonnegative. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Overflow warning: all path costs, flow-cost products, and their sums must fit in the cost type `C`.

#### Time Complexity:

- $O(n)$  per call to the constructor, where  $n$  is the number of nodes.
- $O(1)$  per call to `add_edge()`.
- $O(a \cdot n \cdot m)$  per call to `min_cost_flow()`, where  $a$  is the number of augmenting paths,  $n$  is the number of nodes, and  $m$  is the number of edges. For integer capacities,  $a$  is at most the flow sent.

**Space Complexity:**  $O(\max(n, m))$  for storage of the residual network and auxiliary arrays.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

template<typename T, typename C = int64_t>
class MinCostMaxFlow {
    struct Edge {
        int u, v;
        T cap, flow;
        C cost;
    };

    static constexpr T EPS = static_cast<T>(1e-9);

    int nodes;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;

public:
    explicit MinCostMaxFlow(int n) : nodes(n), adj(n) {}

    int add_edge(int u, int v, T cap, C cost) {

```

```

    int id = static_cast<int>(edges.size());
    adj[u].push_back(id);
    edges.push_back(Edge{u, v, cap, 0, cost});
    adj[v].push_back(id + 1);
    edges.push_back(Edge{v, u, 0, 0, -cost});
    return id;
}

T edge_flow(int id) const { return edges[id].flow; }

void clear_flow() {
    for (Edge &e : edges) {
        e.flow = 0;
    }
}

std::pair<T, C> min_cost_flow(int source, int sink, T target_flow) {
    assert(source != sink);
    static const C INF_COST = std::numeric_limits<C>::max() / 4;
    T flow = 0;
    C cost = 0;
    std::vector<C> dist(nodes);
    std::vector<int> parent_edge(nodes);
    std::vector<char> in_queue(nodes);
    while (flow < target_flow) {
        dist.assign(nodes, INF_COST);
        in_queue.assign(nodes, false);
        std::queue<int> q;
        dist[source] = 0;
        q.push(source);
        in_queue[source] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            in_queue[u] = false;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (e.cap - e.flow > EPS && dist[u] + e.cost < dist[e.v]) {
                    dist[e.v] = dist[u] + e.cost;
                    parent_edge[e.v] = id;
                    if (!in_queue[e.v]) {
                        q.push(e.v);
                        in_queue[e.v] = true;
                    }
                }
            }
        }
        if (dist[sink] == INF_COST) {
            break;
        }
        T aug = target_flow - flow;
        for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
            const Edge &e = edges[parent_edge[v]];
            aug = std::min(aug, e.cap - e.flow);
        }
        for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
            int id = parent_edge[v];
            Edge &e = edges[id];
            e.flow += aug;
            edges[id ^ 1].flow -= aug;
            cost += aug * e.cost; // Overflow warning.
        }
        flow += aug;
    }
    return {flow, cost};
}
};

```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \       |
    // 3/4 /   \ 1/1   | 2/2
    //    /     v       v
    // 0     4 -----> 5
    //    \   ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    MinCostMaxFlow<int> g(6);
    int id01 = g.add_edge(0, 1, 4, 1);
    g.add_edge(0, 2, 3, 1);
    g.add_edge(1, 3, 2, 1);
    g.add_edge(1, 4, 1, 2);
    g.add_edge(2, 4, 2, 1);
    g.add_edge(3, 5, 2, 1);
    g.add_edge(4, 5, 3, 1);
    auto [flow, cost] = g.min_cost_flow(0, 5, 5);
    assert(flow == 5 && cost == 16);
    assert(g.edge_flow(id01) == 3);
    g.clear_flow();
    auto [flow2, cost2] = g.min_cost_flow(0, 5, 5);
    assert(flow2 == 5 && cost2 == 16);
    return 0;
}
```

### 4.5.7 Minimum-Cost Maximum-Flow (Potentials)

Given a flow network with capacities and edge costs, find a minimum-cost flow from a source node to a sink node. This is the successive shortest path method with Johnson potentials. A node potential  $h[v]$  transforms each edge cost  $c(u, v)$  into a reduced cost  $c(u, v) + h[u] - h[v]$ ; when all reduced costs are nonnegative, Dijkstra's algorithm can find the shortest augmenting path safely. After each augmentation,  $h$  is updated with the new shortest-path distances so that reduced costs remain nonnegative on the updated residual graph.

- `MinCostMaxFlow<T, C>(n)` constructs an empty residual network with nodes numbered  $[0, n)$ .
- `add_edge(u, v, cap, cost)` adds a directed edge from `u` to `v` with capacity `cap` and per-unit cost `cost`, and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.
- `clear_flow()` resets all edge flows to zero.
- `min_cost_flow(source, sink, target_flow)` sends up to `target_flow` additional units of flow and returns a pair  $(\text{flow}, \text{cost})$  for the flow sent by that call. If the returned flow is smaller than `target_flow`, the residual network cannot carry the requested amount.

If all initial edge costs are nonnegative, the initial potentials can be zero and the first shortest path search is already Dijkstra-safe. If negative costs are present, this implementation first runs SPFA to compute valid initial potentials; it assumes there is no reachable negative-cost cycle. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Overflow warning: all path costs, reduced costs, potentials, flow-cost products, and their sums must fit in the cost type `C`.

**Time Complexity:**

- $O(n)$  per call to the constructor, where  $n$  is the number of nodes.
- $O(1)$  per call to `add_edge()`.
- $O(m)$  to check whether initial potentials are needed, plus  $O(n \cdot m)$  to compute them when negative residual costs are reachable.
- $O(a \cdot m \cdot \log n)$  for the augmenting paths, where  $a$  is the number of augmenting paths,  $n$  is the number of nodes, and  $m$  is the number of edges. For integer capacities,  $a$  is at most the flow sent.

**Space Complexity:**  $O(\max(n, m))$  for storage of the residual network and auxiliary arrays.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <functional>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

template<typename T, typename C = int64_t>
class MinCostMaxFlow {
    struct Edge {
        int u, v;
        T cap, flow;
        C cost;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;
    std::vector<C> potential;

    void init_potentials(int source) {
        static const C INF_COST = std::numeric_limits<C>::max() / 4;
        potential.assign(nodes, 0);
        bool has_negative_cost = false;
        for (const Edge &e : edges) {
            has_negative_cost |= residual(e) && e.cost < 0;
        }
        if (!has_negative_cost) {
            return;
        }
        potential.assign(nodes, INF_COST);
        std::vector<char> in_queue(nodes);
        std::queue<int> q;
        potential[source] = 0;
        q.push(source);
        in_queue[source] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            in_queue[u] = false;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (residual(e) && potential[u] + e.cost < potential[e.v]) {
                    potential[e.v] = potential[u] + e.cost;
                    if (!in_queue[e.v]) {
                        q.push(e.v);
                        in_queue[e.v] = true;
                    }
                }
            }
        }
    }
};
```



```

    for (C &p : potential) {
        if (p == INF_COST) {
            p = 0;
        }
    }
}

public:
    explicit MinCostMaxFlow(int n) : nodes(n), adj(n), potential(n, 0) {}

    int add_edge(int u, int v, T cap, C cost) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        edges.push_back(Edge{u, v, cap, 0, cost});
        adj[v].push_back(id + 1);
        edges.push_back(Edge{v, u, 0, 0, -cost});
        return id;
    }

    T edge_flow(int id) const { return edges[id].flow; }

    void clear_flow() {
        for (Edge &e : edges) {
            e.flow = 0;
        }
        potential.assign(nodes, 0);
    }

    std::pair<T, C> min_cost_flow(int source, int sink, T target_flow) {
        assert(source != sink);
        static const C INF_COST = std::numeric_limits<C>::max() / 4;
        init_potentials(source);
        T flow = 0;
        C cost = 0;
        std::vector<C> dist(nodes);
        std::vector<int> parent_edge(nodes);
        using qnode = std::pair<C, int>; // (distance, node)
        while (flow < target_flow) {
            std::priority_queue<qnode, std::vector<qnode>, std::greater<>> pq;
            dist.assign(nodes, INF_COST);
            dist[source] = 0;
            pq.emplace(0, source);
            while (!pq.empty()) {
                auto [du, u] = pq.top();
                pq.pop();
                if (du != dist[u]) {
                    continue;
                }
                for (int id : adj[u]) {
                    const Edge &e = edges[id];
                    C reduced_cost = e.cost + potential[u] - potential[e.v];
                    if (residual(e) && dist[u] + reduced_cost < dist[e.v]) {
                        dist[e.v] = dist[u] + reduced_cost;
                        parent_edge[e.v] = id;
                        pq.emplace(dist[e.v], e.v);
                    }
                }
            }
        }
        if (dist[sink] == INF_COST) {
            break;
        }
        for (int i = 0; i < nodes; i++) {
            if (dist[i] < INF_COST) {
                potential[i] += dist[i];
            }
        }
        T aug = target_flow - flow;
        for (int v = sink; v != source; v = edges[parent_edge[v]].u) {

```

```

    const Edge &e = edges[parent_edge[v]];
    aug = std::min(aug, e.cap - e.flow);
}
for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
    int id = parent_edge[v];
    Edge &e = edges[id];
    e.flow += aug;
    edges[id ^ 1].flow -= aug;
    cost += aug * e.cost; // Overflow warning.
}
flow += aug;
}
return {flow, cost};
}
};

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Example graph after max flow, with each edge labeled flow/capacity:
    //
    //           1 -----> 3
    //          / \         |
    // 3/4 /   \ 1/1   | 2/2
    //   /     v     v
    // 0   \   ^ 4 ----> 5
    //   \   ^ 3/3
    // 2/3 \ / 2/2
    //      v /
    //      2
    MinCostMaxFlow<int> g(6);
    int id01 = g.add_edge(0, 1, 4, 1);
    g.add_edge(0, 2, 3, 1);
    g.add_edge(1, 3, 2, 1);
    g.add_edge(1, 4, 1, 2);
    g.add_edge(2, 4, 2, 1);
    g.add_edge(3, 5, 2, 1);
    g.add_edge(4, 5, 3, 1);
    auto [flow, cost] = g.min_cost_flow(0, 5, 5);
    assert(flow == 5 && cost == 16);
    assert(g.edge_flow(id01) == 3);
    g.clear_flow();
    auto [flow2, cost2] = g.min_cost_flow(0, 5, 5);
    assert(flow2 == 5 && cost2 == 16);
    return 0;
}

```

### 4.5.8 Gomory-Hu Tree

Given an undirected graph with nonnegative edge weights, build a Gomory-Hu tree: a weighted tree on the same nodes that represents every pairwise minimum  $s$ - $t$  cut value. For any two nodes  $u$  and  $v$ , the minimum cut value between them in the original graph is the minimum edge weight on the path from  $u$  to  $v$  in the Gomory-Hu tree.

The algorithm starts with every node attached to node 0, then performs  $n - 1$  max-flow/min-cut computations. After computing the minimum cut between node  $s$  and its current parent, nodes on the  $s$  side of that cut are reparented under  $s$ , gradually refining the cut-equivalent tree.

- `gomory_hu(n, edges)` returns the  $n - 1$  edges of a Gomory-Hu tree as tuples  $(u, v, \text{weight})$  for an undirected, weighted graph with  $n$  nodes and `edges` of the same shape.

- `min_cut_value(tree, source, sink)` returns the minimum cut value between nodes `source` and `sink` using the tree returned by `gomory_hu()`. For many pairwise cut queries on the same tree, it is more efficient to prebuild the tree adjacency once and answer minimum edge-on-path queries with LCA/binary lifting instead of calling `min_cut_value()` each time.

#### Time Complexity:

- $O(n)$  calls to maximum flow. With the included Dinic implementation, this is  $O(n^3 \cdot m)$  in the worst case, where  $n$  is the number of nodes and  $m$  is the number of undirected edges.
- $O(n)$  per call to `min_cut_value()`.

**Space Complexity:**  $O(\max(n, m))$  for the residual network and auxiliary arrays.

```
#include <algorithm>
#include <cassert>
#include <limits>
#include <optional>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

template<typename T>
class Dinic {
    int nodes;
    std::vector<std::vector<std::tuple<int, int, T>>> adj; // (v, rev, cap)
    std::vector<int> dist, ptr;

    bool bfs(int source, int sink) {
        dist.assign(nodes, -1);
        dist[source] = 0;
        std::queue<int> q;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto &[v, rev, cap] : adj[u]) {
                if (dist[v] < 0 && cap > 0) {
                    dist[v] = dist[u] + 1;
                    q.push(v);
                }
            }
        }
        return dist[sink] >= 0;
    }

    T dfs(int u, int sink, T pushed) {
        if (u == sink || pushed == 0) {
            return pushed;
        }
        for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
            auto &[v, rev, cap] = adj[u][ptr[u]];
            if (dist[v] == dist[u] + 1 && cap > 0) {
                T pushed_next = dfs(v, sink, std::min(pushed, cap));
                if (pushed_next > 0) {
                    cap -= pushed_next;
                    std::get<2>(adj[v][rev]) += pushed_next;
                    return pushed_next;
                }
            }
        }
        return 0;
    }

public:
    explicit Dinic(int n) : nodes(n), adj(n), dist(n), ptr(n) {}
};
```

```

void add_edge(int u, int v, T cap) {
    adj[u].emplace_back(v, static_cast<int>(adj[v].size()), cap);
    adj[v].emplace_back(u, static_cast<int>(adj[u].size()) - 1, 0);
}

T max_flow(int source, int sink) {
    T flow = 0;
    while (bfs(source, sink)) {
        ptr.assign(nodes, 0);
        while (true) {
            T pushed = dfs(source, sink, std::numeric_limits<T>::max());
            if (pushed == 0) {
                break;
            }
            flow += pushed;
        }
    }
    return flow;
}

std::vector<char> min_cut(int source) const {
    std::vector<char> reachable(nodes);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto &[v, rev, cap] : adj[u]) {
            if (!reachable[v] && cap > 0) {
                reachable[v] = true;
                q.push(v);
            }
        }
    }
    return reachable;
}

};

template<typename T>
using Edge = std::tuple<int, int, T>;

template<typename T>
std::vector<Edge<T>> gomory_hu(int n, const std::vector<Edge<T>> &edges) {
    assert(n >= 1);
    std::vector<int> parent(n);
    std::vector<T> cut_value(n);
    for (int s = 1; s < n; s++) {
        int t = parent[s];
        Dinic<T> mf(n);
        for (const auto &[u, v, w] : edges) {
            mf.add_edge(u, v, w);
            mf.add_edge(v, u, w);
        }
        T flow = mf.max_flow(s, t);
        std::vector<char> cut = mf.min_cut(s);
        for (int v = s + 1; v < n; v++) {
            if (parent[v] == t && cut[v] == cut[s]) {
                parent[v] = s;
            }
        }
        if (cut[parent[t]]) {
            parent[s] = parent[t];
            parent[t] = s;
            cut_value[s] = cut_value[t];
            cut_value[t] = flow;
        } else {
            cut_value[s] = flow;
        }
    }
}

```

```

    }
}
std::vector<Edge<T>> tree;
for (int v = 1; v < n; v++) {
    tree.emplace_back(v, parent[v], cut_value[v]);
}
return tree;
}

template<typename T>
T min_cut_value(const std::vector<Edge<T>> &tree, int source, int sink) {
    int n = static_cast<int>(tree.size()) + 1;
    assert(0 <= source && source < n && 0 <= sink && sink < n && source != sink);
    // For many queries, prebuild this adjacency and add LCA/binary lifting for min edge on path.
    std::vector<std::vector<std::pair<int, T>>> adj(n);
    for (const auto &[u, v, w] : tree) {
        adj[u].emplace_back(v, w);
        adj[v].emplace_back(u, w);
    }
    std::vector<char> seen(n);
    auto dfs = [&](auto &&dfs, int u, T best) -> std::optional<T> {
        if (u == sink) {
            return best;
        }
        seen[u] = true;
        for (auto [v, w] : adj[u]) {
            if (!seen[v]) {
                auto res = dfs(dfs, v, std::min(best, w));
                if (res) {
                    return res;
                }
            }
        }
        return std::nullopt;
    };
    return *dfs(dfs, source, std::numeric_limits<T>::max());
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=3      w=2
    //  0 -----> 1 -----> 3
    //   \      w=1|      ^
    // w=2 \      v      / w=4
    //      +----> 2 ----+
    vector<Edge<int>> edges{
        {0, 1, 3}, {0, 2, 2}, {1, 2, 1}, {1, 3, 2}, {2, 3, 4},
    };
    auto tree = gomory_hu(4, edges);
    assert((tree == vector<Edge<int>>{{1, 0, 5}, {2, 1, 5}, {3, 2, 6}}));
    assert(min_cut_value(tree, 0, 1) == 5);
    assert(min_cut_value(tree, 0, 2) == 5);
    assert(min_cut_value(tree, 0, 3) == 5);
    assert(min_cut_value(tree, 2, 3) == 6);
    vector<Edge<uint64_t>> wide_tree{{0, 1, numeric_limits<uint64_t>::max()}};
    assert(min_cut_value(wide_tree, 0, 1) == numeric_limits<uint64_t>::max());
    return 0;
}

```

### 4.5.9 Global Minimum Cut (Stoer-Wagner)

Given an undirected graph with nonnegative edge weights, the global minimum cut is the partition of the nodes into two nonempty sets that minimizes the total weight of edges crossing between them. Unlike a minimum  $s$ - $t$  cut, no source or sink is fixed; the goal is the cheapest way to split the graph in two. The Stoer-Wagner algorithm finds it without any maximum flow computation by repeatedly running a “minimum cut phase”.

A phase grows a set, starting from an arbitrary node, by repeatedly adding the most tightly connected outside node (the one with the greatest total weight to the set so far), exactly as in Prim’s algorithm. The last node added, together with its connection weight, yields one candidate cut, the “cut of the phase”. The last two nodes added are then merged into a single supernode and the next phase begins on the smaller graph. After  $n - 1$  phases every candidate has been considered, and the smallest is the global minimum cut.

- `global_min_cut(cap)` returns a pair (`weight`, `side`) for a graph as a symmetric capacity matrix `cap` (with 0 for the diagonal and absent edges), where `weight` is the total weight of the global minimum cut and `side` lists the nodes on one side of that cut. The graph must have at least two nodes.

Parallel edges should be pre-summed into the capacity matrix before calling: `cap[u][v]` holds a single combined weight, so the sum of all parallel edge weights between `u` and `v` should be stored there.

**Time Complexity:**  $O(n^3)$  per call, where  $n$  is the number of nodes.

**Space Complexity:**  $O(n^2)$  auxiliary.

```
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>>> global_min_cut(std::vector<std::vector<int64_t>>> cap) {
    int n = static_cast<int>(cap.size());
    assert(n >= 2);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            assert(cap[i][j] >= 0);
            assert(cap[i][j] == cap[j][i]);
        }
    }
    std::pair<int64_t, std::vector<int>>> best = {INT64_MAX, {}};
    std::vector<std::vector<int>>> groups(n);
    std::vector<int64_t> weight(n);
    std::vector<char> exists(n, true), added(n);
    for (int i = 0; i < n; i++) {
        groups[i] = {i};
    }
    for (int phase = 1; phase < n; phase++) {
        weight.assign(n, 0);
        added.assign(n, false);
        int s = -1;
        for (int it = 0; it <= n - phase; it++) {
            int sel = -1;
            for (int i = 0; i < n; i++) {
                if (exists[i] && !added[i] && (sel == -1 || weight[i] > weight[sel])) {
                    sel = i;
                }
            }
            if (it == n - phase) {
                if (weight[sel] < best.first) {
                    best = {weight[sel], groups[sel]};
                }
                groups[s].insert(groups[s].end(), groups[sel].begin(), groups[sel].end());
                for (int i = 0; i < n; i++) {
                    cap[s][i] += cap[sel][i]; // Overflow warning.
                    cap[i][s] = cap[s][i];
                }
                exists[sel] = false;
            }
        }
    }
}
```

```

        break;
    }
    added[sel] = true;
    for (int i = 0; i < n; i++) {
        weight[i] += cap[sel][i]; // Overflow warning.
    }
    s = sel;
}
}
return best;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=2
    //  0 ----- 1
    //   \       /
    // w=1 \   / w=3
    //      2
    vector<vector<int64_t>> cap{
        {0, 2, 1},
        {2, 0, 3},
        {1, 3, 0},
    };
    // Cuts: {0}|{1,2} = 3, {1}|{0,2} = 5, {2}|{0,1} = 4. Minimum is 3.
    auto [weight, side] = global_min_cut(cap);
    assert(weight == 3);
    assert((side == vector<int>{1, 2})); // The shore {1, 2}, opposite the single node 0.

    vector<vector<int64_t>> disconnected{{0, 0}, {0, 0}};
    assert(global_min_cut(disconnected).first == 0);
    return 0;
}

```

## 4.6 Matching and Assignment

---

### 4.6.1 Bipartite Check (2-Coloring)

Given an undirected graph, determine whether its nodes can be split into two sets such that every edge joins a node of one set to a node of the other. A graph admitting such a split is bipartite, and the split is exactly a proper coloring with two colors. Both routines below give each node the color opposite to the node they reached it from, so an edge joining two nodes of equal color closes a cycle of odd length. No bipartition can accommodate an odd cycle, so the first such edge decides the answer.

Only the depth-first version reports the cycle it found. A depth-first conflict is always a back edge to an ancestor, so the cycle is one walk up the parent chain, while a breadth-first conflict may join two nodes in unrelated subtrees and would cost a depth array and a climb from both ends.

- `bipartite_dfs()` returns whether the graph is bipartite for a global, bidirectionally pre-populated adjacency list `adj` whose indices represent the nodes. On success it populates the global array `side` with 0 or 1 for every node and leaves `odd_cycle` empty. On failure it leaves `side` partially filled and populates `odd_cycle` with the nodes of an odd cycle, in cycle order.
- `bipartite_bfs()` returns the same answer and coloring using a queue instead of recursion, which suits graphs deep enough to overflow the call stack. It does not compute `odd_cycle` on failure.

Each connected component is colored independently, so a disconnected graph receives one arbitrary orientation of the two colors per component. The sections that follow take the two sides as separate node numberings: number the nodes with `side[u]` of 0 and those with `side[u]` of 1 each consecutively from 0, then store only the neighbors of the first set.

**Time Complexity:**  $O(\max(n, m))$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary heap space, plus  $O(n)$  auxiliary stack space for `bipartite_dfs()`.

```
#include <algorithm>
#include <queue>
#include <vector>

std::vector<std::vector<int>> adj;
std::vector<int> side, odd_cycle;

bool bipartite_dfs() {
    int n = static_cast<int>(adj.size());
    side.assign(n, -1);
    std::vector<int> parent(n, -1);
    odd_cycle.clear();
    // A back edge to an ancestor of equal color closes an odd cycle along the tree path to it.
    auto dfs = [&](auto &&dfs, int u) -> bool {
        for (int v : adj[u]) {
            if (side[v] == -1) {
                side[v] = side[u] ^ 1;
                parent[v] = u;
                if (!dfs(dfs, v)) {
                    return false;
                }
            } else if (side[v] == side[u]) {
                for (int x = u; x != v; x = parent[x]) {
                    odd_cycle.push_back(x);
                }
                odd_cycle.push_back(v);
                std::reverse(odd_cycle.begin(), odd_cycle.end());
                return false;
            }
        }
        return true;
    };
    for (int u = 0; u < n; u++) {
        if (side[u] == -1) {
            side[u] = 0;
            if (!dfs(dfs, u)) {
                return false;
            }
        }
    }
    return true;
}

bool bipartite_bfs() {
    int n = static_cast<int>(adj.size());
    side.assign(n, -1);
    for (int s = 0; s < n; s++) {
        if (side[s] != -1) {
            continue;
        }
        side[s] = 0;
        std::queue<int> q;
        for (q.push(s); !q.empty(); q.pop()) {
            int u = q.front();
            for (int v : adj[u]) {

```



```

        if (side[v] == -1) {
            side[v] = side[u] ^ 1;
            q.push(v);
        } else if (side[v] == side[u]) {
            return false;
        }
    }
}
}
return true;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    // 0---1
    // |   |   4
    // 3---2
    adj.assign(5, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 3);
    add_edge(3, 0);
    assert(bipartite_dfs() && odd_cycle.empty());
    assert(side[0] == side[2] && side[1] == side[3] && side[0] != side[1]);
    assert(bipartite_bfs());
    assert(side[0] == side[2] && side[1] == side[3] && side[0] != side[1]);

    // 0---1---3
    // \   /
    //   2
    adj.assign(4, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 0);
    add_edge(1, 3);
    assert(!bipartite_dfs());
    assert((odd_cycle == vector<int>{0, 1, 2}));
    assert(!bipartite_bfs());
    return 0;
}

```

### 4.6.2 Maximum Bipartite Matching (Kuhn)

Given two sets of nodes  $A = \{0, 1, \dots, n_1 - 1\}$  and  $B = \{0, 1, \dots, n_2 - 1\}$ , as well as a set of edges  $E$  mapping nodes from set  $A$  to set  $B$ , find the largest possible subset of  $E$  containing no edges that share the same node.

Kuhn's algorithm builds the matching one left node at a time. For each, a depth-first search looks for an augmenting path: an alternating sequence of unmatched and matched edges that reaches a free right node. Flipping the edges along such a path enlarges the matching by one.

- `match_bipartite(n2)` populates `match_left` and `match_right`, then returns maximum matching size for a global, pre-populated adjacency list `adj`, whose indices represent left-side nodes and whose entries contain right-side neighbors numbered  $[0, n_2)$ . The two sides are given as separate node numberings. Use the

bipartite check of section 4.6.1 to produce them for a graph that does not already arrive presented as two sets. Time Complexity:

- $O(n_1 \cdot m)$  per call, where  $m$  is the number of edges.

#### Space Complexity:

- $O(\max(n_1 + n_2, m))$  for storage of the graph, where  $m$  is the number of edges.
- $O(n_1 + n_2)$  auxiliary heap space and  $O(n_1)$  auxiliary stack space.

```
#include <vector>

std::vector<std::vector<int>>> adj;
std::vector<int> match_left, match_right, visit;
int timer;

bool dfs(int u) {
    visit[u] = timer;
    for (int nb : adj[u]) {
        if (match_right[nb] == -1) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    for (int nb : adj[u]) {
        int v = match_right[nb];
        if (visit[v] != timer && dfs(v)) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    return false;
}

int match_bipartite(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_left.assign(n1, -1);
    match_right.assign(n2, -1);
    visit.assign(n1, 0);
    timer = 0;
    int matches = 0;
    for (int i = 0; i < n1; i++) {
        timer++;
        if (dfs(i)) {
            matches++;
        }
    }
    return matches;
}
```

#### Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Left nodes a0..a2, right nodes b0..b3:
    //  a0 -- b1
    //  a1 -- b0, b1, b2
    //  a2 -- b2, b3
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
}
```

```

adj[1].push_back(2);
adj[2].push_back(2);
adj[2].push_back(3);
assert(match_bipartite(n2) == 3);
assert((match_right == vector<int>{1, 0, 2, -1}));
return 0;
}

```

### 4.6.3 Maximum Bipartite Matching (Hopcroft-Karp)

Given two sets of nodes  $A = \{0, 1, \dots, n_1 - 1\}$  and  $B = \{0, 1, \dots, n_2 - 1\}$ , as well as a set of edges  $E$  mapping nodes from set  $A$  to set  $B$ , find the largest possible subset of  $E$  containing no edges that share the same node.

Hopcroft-Karp augments along many shortest augmenting paths per phase. Each phase first runs a breadth-first search to layer the graph by distance, then a depth-first search to find a maximal set of node-disjoint shortest augmenting paths to flip at once, for  $O(m \cdot \sqrt{n_1 + n_2})$  running time overall.

- `match_bipartite_hk(n2)` populates `match_left` and `match_right`, then returns maximum matching size for a global, pre-populated adjacency list `adj`, whose indices represent left-side nodes and whose entries contain right-side neighbors numbered  $[0, n_2)$ . Time Complexity:
- $O(m \cdot \sqrt{n_1 + n_2})$  per call, where  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n_1 + n_2)$  auxiliary stack space and  $O(n_1 + n_2)$  auxiliary heap space.

```

#include <queue>
#include <vector>

std::vector<std::vector<int>> adj;
std::vector<int> match_left, match_right, dist;
std::vector<char> used, visit;

void bfs() {
    int n1 = static_cast<int>(adj.size());
    dist.assign(n1, -1);
    std::queue<int> q;
    for (int u = 0; u < n1; u++) {
        if (!used[u]) {
            q.push(u);
            dist[u] = 0;
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int nb : adj[u]) {
            int v = match_right[nb];
            if (v >= 0 && dist[v] < 0) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

bool dfs(int u) {
    visit[u] = true;
    for (int nb : adj[u]) {
        int v = match_right[nb];
        if (v < 0 || (!visit[v] && dist[v] == dist[u] + 1 && dfs(v))) {
            match_left[u] = nb;

```

```

        match_right[nb] = u;
        used[u] = true;
        return true;
    }
}
return false;
}

int match_bipartite_hk(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_left.assign(n1, -1);
    match_right.assign(n2, -1);
    used.assign(n1, false);
    int res = 0;
    while (true) {
        bfs();
        visit.assign(n1, false);
        int f = 0;
        for (int u = 0; u < n1; u++) {
            if (!used[u] && dfs(u)) {
                f++;
            }
        }
        if (f == 0) {
            return res;
        }
        res += f;
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Left nodes a0..a2, right nodes b0..b3:
    //  a0 -- b1
    //  a1 -- b0, b1, b2
    //  a2 -- b2, b3
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    assert(match_bipartite_hk(n2) == 3);
    assert((match_right == vector<int>{1, 0, 2, -1}));
    return 0;
}

```

### 4.6.4 Minimum Vertex Cover (Konig)

Given a bipartite graph with left nodes  $A = \{0, \dots, n_1 - 1\}$  and right nodes  $B = \{0, \dots, n_2 - 1\}$ , find a minimum vertex cover: a smallest set of nodes such that every edge has at least one endpoint in the set. By Konig's theorem, in a bipartite graph the size of a minimum vertex cover equals the size of a maximum matching, and the nodes left uncovered form a maximum independent set.

The construction first computes a maximum matching with Kuhn's algorithm. It then marks every left node reachable from an unmatched left node by an alternating path (following unmatched edges from left to right and matched edges from right to left). König's theorem says the minimum vertex cover is exactly the left nodes that are not reachable together with the right nodes that are reachable; each matched edge contributes exactly one endpoint, so the cover size equals the matching size.

- `min_vertex_cover(n2)` returns the nodes of a minimum vertex cover for the global adjacency list `adj` (left nodes  $[0, n_1)$ , right neighbors  $[0, n_2)$ ), where `n1` is `adj.size()`. Each returned value is a left node `i` in  $[0, n_1)$ , or a right node `j` encoded as `n1 + j`.

**Time Complexity:**  $O(n_1 \cdot m)$  per call, dominated by the matching, where  $m$  is the number of edges.

**Space Complexity:**

- $O(\max(n_1 + n_2, m))$  for storage of the graph.
- $O(n_1 + n_2)$  auxiliary.

```
#include <vector>

std::vector<std::vector<int>>> adj;
std::vector<int> match_left, match_right, visit;
int timer;

bool dfs(int u) {
    visit[u] = timer;
    for (int nb : adj[u]) {
        if (match_right[nb] == -1) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    for (int nb : adj[u]) {
        int v = match_right[nb];
        if (visit[v] != timer && dfs(v)) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    return false;
}

int match_bipartite(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_left.assign(n1, -1);
    match_right.assign(n2, -1);
    visit.assign(n1, 0);
    timer = 0;
    int matches = 0;
    for (int i = 0; i < n1; i++) {
        timer++;
        if (dfs(i)) {
            matches++;
        }
    }
    return matches;
}

std::vector<int> min_vertex_cover(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_bipartite(n2);
    std::vector<char> reachable_left(n1), reachable_right(n2);
    std::vector<int> st;
    for (int u = 0; u < n1; u++) {
        if (match_left[u] == -1) {
            reachable_left[u] = true;
        }
    }
    for (int v = 0; v < n2; v++) {
        if (match_right[v] != -1) {
            reachable_right[v] = true;
        }
    }
    st.reserve(n1 + n2);
    for (int i = 0; i < n1; i++) {
        if (reachable_left[i]) {
            st.push_back(i);
        }
    }
    for (int j = 0; j < n2; j++) {
        if (reachable_right[j]) {
            st.push_back(n1 + j);
        }
    }
    return st;
}
```

```

        st.push_back(u);
    }
}
while (!st.empty()) {
    int u = st.back();
    st.pop_back();
    for (int v : adj[u]) {
        if (v != match_left[u] && !reachable_right[v]) {
            reachable_right[v] = true;
            int next = match_right[v];
            if (next != -1 && !reachable_left[next]) {
                reachable_left[next] = true;
                st.push_back(next);
            }
        }
    }
}
}
std::vector<int> cover;
for (int u = 0; u < n1; u++) {
    if (!reachable_left[u]) {
        cover.push_back(u);
    }
}
for (int v = 0; v < n2; v++) {
    if (reachable_right[v]) {
        cover.push_back(n1 + v);
    }
}
return cover;
}

```

## Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    // Left nodes a0..a2, right nodes b0..b3:
    //  a0 -- b1
    //  a1 -- b0, b1, b2
    //  a2 -- b2, b3
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    vector<int> cover = min_vertex_cover(n2);
    assert(cover.size() == 3); // Equals the maximum matching size, by Konig's theorem.
    // The cover is valid: every edge has at least one endpoint in it.
    for (int u = 0; u < n1; u++) {
        for (int v : adj[u]) {
            bool left_in = find(cover.begin(), cover.end(), u) != cover.end();
            bool right_in = find(cover.begin(), cover.end(), n1 + v) != cover.end();
            assert(left_in || right_in);
        }
    }
    return 0;
}

```

### 4.6.5 Maximum Matching (Blossom)

Given an undirected, unweighted simple graph, determine a maximum matching: a maximum subset of its edges such that no node is shared between different edges in the resulting subset.

Edmonds' blossom algorithm extends augmenting-path search to general (non-bipartite) graphs, where odd-length cycles called blossoms can conceal augmenting paths. Each blossom is contracted into a single node so the search can proceed, and is expanded afterward to recover the matching. For maximum-weight matching in bipartite graphs, use Hungarian or min-cost max-flow; for maximum-weight matching in general graphs, use the weighted blossom algorithm in the next section.

The graph must be simple: self-loops and parallel edges are not supported.

- `max_matching(adj)` returns a matching for a bidirectionally pre-populated adjacency list `adj`, whose indices represent the nodes. The returned vector `match` has `match[u] == v` and `match[v] == u` when `u` and `v` are matched, or `match[u]` is `-1` when `u` is unmatched.

**Time Complexity:**  $O(n^3)$  per call, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(\max(n, m))$  for storage of the graph, where  $n$  is the number of nodes and  $m$  is the number of edges.
- $O(n)$  auxiliary.

```
#include <algorithm>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

std::vector<int> max_matching(const std::vector<std::vector<int>>& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> match(n, -1), label(n), parent(n), base(n), aux(n, -1);
    std::queue<int> q;
    int aux_time = -1;
    auto lca = [&](int u, int v) {
        aux_time++;
        while (true) {
            if (u != -1) {
                u = base[u];
                if (aux[u] == aux_time) {
                    return u;
                }
                aux[u] = aux_time;
                u = (match[u] == -1) ? -1 : parent[match[u]];
            }
            std::swap(u, v);
        }
    };
    auto mark_blossom = [&](int u, int v, int b) {
        while (base[u] != b) {
            parent[u] = v;
            v = match[u];
            if (label[v] == 1) {
                label[v] = 0;
                q.push(v);
            }
            base[u] = base[v] = b;
            u = parent[v];
        }
    };
    auto augment = [&](int u) {
        while (u != -1) {
            int p = parent[u];
            int next = match[p];
            match[u] = p;
```

```

        match[p] = u;
        u = next;
    }
};

auto find_path = [&](int root) {
    label.assign(n, -1);
    parent.assign(n, -1);
    std::iota(base.begin(), base.end(), 0);
    while (!q.empty()) {
        q.pop();
    }
    label[root] = 0;
    q.push(root);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (base[u] == base[v] || match[u] == v) {
                continue;
            }
            if (label[v] == -1) {
                label[v] = 1;
                parent[v] = u;
                if (match[v] == -1) {
                    augment(v);
                    return true;
                }
                label[match[v]] = 0;
                q.push(match[v]);
            } else if (label[v] == 0) {
                int b = lca(u, v);
                mark_blossom(u, v, b);
                mark_blossom(v, u, b);
            }
        }
    }
    return false;
};

for (int i = 0; i < n; i++) {
    if (match[i] == -1) {
        for (int v : adj[i]) {
            if (i != v && match[v] == -1) {
                match[i] = v;
                match[v] = i;
                break;
            }
        }
    }
}

for (int i = 0; i < n; i++) {
    if (match[i] == -1) {
        find_path(i);
    }
}

return match;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

vector<vector<int>> adj;

void add_edge(int u, int v) {
    adj[u].push_back(v);
}

```



```

    adj[v].push_back(u);
}

int matching_size(const vector<int> &match) {
    return count_if(match.begin(), match.end(), [](int v) { return v != -1; }) / 2;
}

int main() {
    {
        // 0---1
        // |   |
        // 3---2
        int nodes = 4;
        adj.assign(nodes, {});
        add_edge(0, 1);
        add_edge(1, 2);
        add_edge(2, 3);
        add_edge(3, 0);
        vector<int> match = max_matching(adj);
        assert(matching_size(match) == 2);
        assert((match == vector<int>{1, 0, 3, 2}));
    }
    {
        // 3---0---1---4
        //       \ /
        //        2
        int nodes = 5;
        adj.assign(nodes, {});
        add_edge(0, 1);
        add_edge(1, 2);
        add_edge(2, 0);
        add_edge(0, 3);
        add_edge(1, 4);
        assert(matching_size(max_matching(adj)) == 2);
    }
    return 0;
}

```

#### 4.6.6 Maximum-Weight Matching (Weighted Blossom)

Given an undirected graph with positive edge weights, determine a maximum-weight matching: a subset of edges, no two sharing a node, whose total weight is as large as possible. Unlike the bipartite case (which Hungarian or min-cost max-flow solves), general graphs may contain odd cycles, so this is the weighted generalization of Edmonds' blossom algorithm.

The algorithm runs a primal-dual method. Each node carries a dual label, and an edge is “tight” when its label sum equals twice its weight; the search uses only tight edges, exactly as unweighted blossom matching does, contracting odd cycles into blossoms. When no tight augmenting structure remains, the labels are adjusted by the largest step that keeps every dual constraint satisfied, which makes a new edge tight or lets a blossom expand, and the search resumes. Because the matching is over the implicit complete graph with absent edges weighted 0, a node may be left effectively unmatched whenever staying unmatched is at least as good as any incident edge.

All edge weights must be positive; an absent edge is treated as weight 0. Self-loops are not supported. Parallel edges are supported by retaining only the maximum weight added between each pair of nodes. The node count should be kept modest, as the algorithm uses dense storage.

- `MaxWeightMatching(n)` creates a graph of `n` nodes numbered `[0, n)`.
- `add_edge(u, v, w)` adds an undirected edge of positive weight `w` between `u` and `v`. If several edges join the same pair, only the maximum weight is retained.
- `solve()` computes a maximum-weight matching and returns its total weight.
- `partner(u)` returns the node matched with `u`, or `-1` if `u` is unmatched. Valid after `solve()`.

Overflow warning: labels and the returned matching weight are stored in `int64_t`; all doubled weights, label sums, and the total matching weight must fit.

**Time Complexity:**  $O(n^3)$  per call to `solve()`, where  $n$  is the number of nodes.

**Space Complexity:**  $O(n^2)$  for the dense edge and blossom storage.

```
#include <algorithm>
#include <cstdint>
#include <queue>
#include <utility>
#include <vector>

class MaxWeightMatching {
    struct Edge {
        int u, v;
        int64_t w;
    };

    int n, nx; // n original nodes (1-based internally); nx includes blossom nodes.
    std::vector<std::vector<Edge>> g;
    std::vector<std::vector<int>> flo, flo_from;
    std::vector<int64_t> lab;
    std::vector<int> match_, slack, st, par, s_, aux_;
    std::queue<int> q;
    int aux_clock;

    int64_t e_delta(const Edge &e) const {
        return lab[e.u] + lab[e.v] - 2 * e.w; // Overflow warning.
    }

    void update_slack(int u, int x) {
        if (slack[x] == 0 || e_delta(g[u][x]) < e_delta(g[slack[x]][x])) {
            slack[x] = u;
        }
    }

    void set_slack(int x) {
        slack[x] = 0;
        for (int u = 1; u <= n; u++) {
            if (g[u][x].w > 0 && st[u] != x && s_[st[u]] == 0) {
                update_slack(u, x);
            }
        }
    }

    void queue_push(int x) {
        if (x <= n) {
            q.push(x);
        } else {
            for (int t : flo[x]) {
                queue_push(t);
            }
        }
    }

    void set_st(int x, int b) {
        st[x] = b;
        if (x > n) {
            for (int t : flo[x]) {
                set_st(t, b);
            }
        }
    }

    int get_pr(int b, int xr) {
        int pr = static_cast<int>(std::find(flo[b].begin(), flo[b].end(), xr) - flo[b].begin());
        if (pr & 1) {
```

```

        std::reverse(flo[b].begin() + 1, flo[b].end());
        return static_cast<int>(flo[b].size()) - pr;
    }
    return pr;
}

void set_match(int u, int v) {
    match_[u] = g[u][v].v;
    if (u <= n) {
        return;
    }
    Edge e = g[u][v];
    int xr = flo_from[u][e.u], pr = get_pr(u, xr);
    for (int i = 0; i < pr; i++) {
        set_match(flo[u][i], flo[u][i ^ 1]);
    }
    set_match(xr, v);
    std::rotate(flo[u].begin(), flo[u].begin() + pr, flo[u].end());
}

void augment(int u, int v) {
    while (true) {
        int xnv = st[match_[u]];
        set_match(u, v);
        if (xnv == 0) {
            return;
        }
        set_match(xnv, st[par[xnv]]);
        u = st[par[xnv]];
        v = xnv;
    }
}

int lca(int u, int v) {
    aux_clock++;
    while (u != 0 || v != 0) {
        if (u == 0) {
            std::swap(u, v);
            continue;
        }
        if (aux_[u] == aux_clock) {
            return u;
        }
        aux_[u] = aux_clock;
        u = st[match_[u]];
        if (u != 0) {
            u = st[par[u]];
        }
        std::swap(u, v);
    }
    return 0;
}

void add_blossom(int u, int anc, int v) {
    int b = n + 1;
    while (b <= nx && st[b] != 0) {
        b++;
    }
    if (b > nx) {
        nx++;
    }
    lab[b] = 0;
    s_[b] = 0;
    match_[b] = match_[anc];
    flo[b] = {anc};
    auto build = [&](int x) {
        for (int y; x != anc; x = st[par[y]]) {
            flo[b].push_back(x);
        }
    };
}

```

```

        y = st[match_[x]];
        flo[b].push_back(y);
        queue_push(y);
    }
};
build(u);
std::reverse(flo[b].begin() + 1, flo[b].end());
build(v);
set_st(b, b);
for (int x = 1; x <= nx; x++) {
    g[b][x].w = g[x][b].w = 0;
}
for (int x = 1; x <= n; x++) {
    flo_from[b][x] = 0;
}
for (int xs : flo[b]) {
    for (int x = 1; x <= nx; x++) {
        if (g[b][x].w == 0 || e_delta(g[xs][x]) < e_delta(g[b][x])) {
            g[b][x] = g[xs][x];
            g[x][b] = g[x][xs];
        }
    }
    for (int x = 1; x <= n; x++) {
        if (flo_from[xs][x] != 0) {
            flo_from[b][x] = xs;
        }
    }
}
set_slack(b);
}

void expand_blossom(int b) {
    for (int t : flo[b]) {
        set_st(t, t);
    }
    int xr = flo_from[b][g[b][par[b]].u], pr = get_pr(b, xr);
    for (int i = 0; i < pr; i += 2) {
        int xs = flo[b][i], xns = flo[b][i + 1];
        par[xs] = g[xns][xs].u;
        s_[xs] = 1;
        s_[xns] = 0;
        slack[xs] = 0;
        set_slack(xns);
        queue_push(xns);
    }
    s_[xr] = 1;
    par[xr] = par[b];
    for (int i = pr + 1; i < static_cast<int>(flo[b].size()); i++) {
        int xs = flo[b][i];
        s_[xs] = -1;
        set_slack(xs);
    }
    st[b] = 0;
}

bool on_found_edge(const Edge &e) {
    int u = st[e.u], v = st[e.v];
    if (s_[v] == -1) {
        par[v] = e.u;
        s_[v] = 1;
        int nu = st[match_[v]];
        slack[v] = slack[nu] = 0;
        s_[nu] = 0;
        queue_push(nu);
    } else if (s_[v] == 0) {
        int anc = lca(u, v);
        if (anc == 0) {
            augment(u, v);
        }
    }
}

```

```

    augment(v, u);
    return true;
}
add_blossom(u, anc, v);
}
return false;
}

bool matching() {
    std::fill(s_.begin(), s_.begin() + nx + 1, -1);
    std::fill(slack.begin(), slack.begin() + nx + 1, 0);
    q = std::queue<int>();
    for (int x = 1; x <= nx; x++) {
        if (st[x] == x && match_[x] == 0) {
            par[x] = s_[x] = 0;
            queue_push(x);
        }
    }
    if (q.empty()) {
        return false;
    }
    while (true) {
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (s_[st[u]] == 1) {
                continue;
            }
            for (int v = 1; v <= n; v++) {
                if (g[u][v].w > 0 && st[u] != st[v]) {
                    if (e_delta(g[u][v]) == 0) {
                        if (on_found_edge(g[u][v])) {
                            return true;
                        }
                    } else {
                        update_slack(u, st[v]);
                    }
                }
            }
        }
    }
    int64_t d = INT64_MAX;
    for (int b = n + 1; b <= nx; b++) {
        if (st[b] == b && s_[b] == 1) {
            d = std::min(d, lab[b] / 2);
        }
    }
    for (int x = 1; x <= nx; x++) {
        if (st[x] == x && slack[x] != 0) {
            if (s_[x] == -1) {
                d = std::min(d, e_delta(g[slack[x]][x]));
            } else if (s_[x] == 0) {
                d = std::min(d, e_delta(g[slack[x]][x]) / 2);
            }
        }
    }
    if (d == INT64_MAX) {
        return false;
    }
    for (int u = 1; u <= n; u++) {
        if (s_[st[u]] == 0) {
            if (lab[u] <= d) {
                return false;
            }
        }
        lab[u] -= d;
    }
    for (int u = 1; u <= n; u++) {
        if (s_[st[u]] == 1) {
            lab[u] += d;
        }
    }
}

```

```

    for (int b = n + 1; b <= nx; b++) {
        if (st[b] == b && s_[b] != -1) {
            lab[b] += (s_[b] == 0 ? 1 : -1) * d * 2;
        }
    }
    q = std::queue<int>();
    for (int x = 1; x <= nx; x++) {
        if (st[x] == x && slack[x] != 0 && st[slack[x]] != x && e_delta(g[slack[x]][x]) == 0) {
            if (on_found_edge(g[slack[x]][x])) {
                return true;
            }
        }
    }
    for (int b = n + 1; b <= nx; b++) {
        if (st[b] == b && s_[b] == 1 && lab[b] == 0) {
            expand_blossom(b);
        }
    }
}

public:
explicit MaxWeightMatching(int n)
: n(n),
  nx(n),
  g(2 * n + 1, std::vector<Edge>(2 * n + 1)),
  flo(2 * n + 1),
  flo_from(2 * n + 1, std::vector<int>(n + 1)),
  lab(2 * n + 1, 0),
  match_(2 * n + 1, 0),
  slack(2 * n + 1, 0),
  st(2 * n + 1, 0),
  par(2 * n + 1, 0),
  s_(2 * n + 1, 0),
  aux_(2 * n + 1, 0),
  aux_clock(0) {
    for (int u = 1; u <= n; u++) {
        for (int v = 1; v <= n; v++) {
            g[u][v] = {u, v, 0};
        }
    }
}

void add_edge(int u, int v, int64_t w) {
    w = std::max(w, g[u + 1][v + 1].w);
    g[u + 1][v + 1].w = g[v + 1][u + 1].w = w;
}

int64_t solve() {
    nx = n;
    for (int u = 1; u <= n; u++) {
        st[u] = u;
        match_[u] = 0;
        flo[u].clear();
    }
    int64_t w_max = 0;
    for (int u = 1; u <= n; u++) {
        for (int v = 1; v <= n; v++) {
            flo_from[u][v] = (u == v ? u : 0);
            w_max = std::max(w_max, g[u][v].w);
        }
    }
    for (int u = 1; u <= n; u++) {
        lab[u] = w_max;
    }
    while (matching()) {
    }
    int64_t weight = 0;

```

```

    for (int u = 1; u <= n; u++) {
        if (match_[u] != 0 && match_[u] < u) {
            weight += g[u][match_[u]].w; // Overflow warning.
        }
    }
    return weight;
}

int partner(int u) const {
    int v = match_[u + 1];
    // A node "matched" through an absent (weight 0) edge is really unmatched.
    if (v == 0 || g[u + 1][v].w == 0) {
        return -1;
    }
    return v - 1;
}
};

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=5   w=6   w=5   w=6
    // 0-----1-----2-----3-----4
    //  \-----/
    //           w=5
    MaxWeightMatching g(5);
    g.add_edge(0, 1, 5);
    g.add_edge(1, 2, 6);
    g.add_edge(2, 3, 5);
    g.add_edge(3, 4, 6);
    g.add_edge(4, 0, 5);
    assert(g.solve() == 12); // Edges 1-2 and 3-4 with weights 6 + 6.
    assert(g.partner(1) == 2 && g.partner(2) == 1);
    assert(g.partner(3) == 4 && g.partner(4) == 3);
    assert(g.partner(0) == -1); // Node 0 is left unmatched.

    //      w=3   w=3   w=5
    // 0-----1-----2-----3
    //  \-----/
    //           w=3
    MaxWeightMatching t(4);
    t.add_edge(0, 1, 3);
    t.add_edge(1, 2, 3);
    t.add_edge(0, 2, 3);
    t.add_edge(2, 3, 5);
    t.add_edge(2, 3, 1); // A lighter parallel edge does not replace the heavier one.
    assert(t.solve() == 8); // Edges 0-1 and 2-3 with weights 3 + 5.

    MaxWeightMatching star(4);
    star.add_edge(0, 1, 12);
    star.add_edge(0, 2, 29);
    star.add_edge(0, 3, 29);
    assert(star.solve() == 29); // Once one heavy edge is matched, no finite dual step remains.
    return 0;
}

```

### 4.6.7 Assignment (Hungarian Algorithm)

Solves the minimum-cost or maximum-value assignment problem for a rectangular matrix. Given  $n$  workers and  $m$  jobs (with  $n \leq m$ ), choose a distinct job for each worker so that the total cost is minimized or the total value is maximized.

The classical Hungarian algorithm using potentials transforms a cost matrix using the principle that adding or subtracting constants from rows or columns does not change the optimal assignment. Workers are assigned one row at a time: each new row triggers a search for a least-cost augmenting path over zero-slack edges, with the potentials adjusted along the way to expose new zero-slack edges until the path reaches a free job. Note that while the input matrix is 0-based here, the internal calculations are 1-based.

Maximum-value assignment reduces to minimum-cost assignment by replacing each value  $x$  with  $M - x$ , where  $M$  is the largest matrix entry. Every assignment contains exactly  $n$  entries, so minimizing the transformed total  $nM - \sum x$  maximizes the original total.

- `min_assignment(cost)` returns a pair `(min_cost, assignment)` for a 0-based `cost` matrix, where `assignment` contains  $n$  values and `assignment[i]` is the chosen job for worker `i`, using 0-based job numbers.
- `max_assignment(value)` returns the analogous pair `(max_value, assignment)` maximizing a 0-based `value` matrix.

**Time Complexity:**  $O(n^2 \cdot m)$  per call, where the input matrix has  $n$  rows and  $m$  columns.

**Space Complexity:**

- $O(n + m)$  auxiliary, not counting the input matrix and returned assignment.
- $O(n \cdot m)$  auxiliary for `max_assignment()` due to the transformed matrix.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <utility>
#include <vector>

template<typename T>
std::pair<T, std::vector<int>> min_assignment(const std::vector<std::vector<T>> &cost) {
    static const T INF = std::numeric_limits<T>::max() / 4;
    int n = static_cast<int>(cost.size());
    int m = cost.empty() ? 0 : static_cast<int>(cost[0].size());
    assert(n <= m);
    std::vector<T> u(n + 1), v(m + 1), minv(m + 1);
    std::vector<int> p(m + 1), way(m + 1);
    std::vector<char> used(m + 1);
    for (int i = 1; i <= n; i++) {
        minv.assign(m + 1, INF);
        used.assign(m + 1, false);
        p[0] = i;
        int j0 = 0;
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            T delta = INF;
            for (int j = 1; j <= m; j++) {
                if (used[j]) {
                    continue;
                }
                T cur = cost[i0 - 1][j - 1] - u[i0] - v[j]; // Overflow warning.
                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }
                if (minv[j] < delta) {
                    delta = minv[j];
                    j1 = j;
                }
            }
            j0 = j1;
        } while (j0 < m);
        p[j0 + 1] = i0;
        for (int j = j0; j > 0; j = way[j]) {
            p[j] = p[j + 1];
        }
    }
    T min_cost = 0;
    for (int i = 1; i <= n; i++) {
        min_cost += cost[i - 1][p[i] - 1];
    }
    return {min_cost, p};
}
```



```

    }
}
for (int j = 0; j <= m; j++) {
    if (used[j]) {
        u[p[j]] += delta;
        v[j] -= delta;
    } else {
        minv[j] -= delta;
    }
}
j0 = j1;
} while (p[j0] != 0);
do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0 != 0);
}
// Optional: reconstruct one optimal assignment.
std::vector<int> assignment(n, -1);
for (int j = 1; j <= m; j++) {
    if (p[j] != 0) {
        assignment[p[j] - 1] = j - 1;
    }
}
return {-v[0], assignment}; // Overflow warning.
}

template<typename T>
std::pair<T, std::vector<int>> max_assignment(std::vector<std::vector<T>> value) {
    if (value.empty()) {
        return {0, {}};
    }
    T max_value = value[0][0];
    for (const auto &row : value) {
        max_value = std::max(max_value, *std::max_element(row.begin(), row.end()));
    }
    for (auto &row : value) {
        for (T &x : row) {
            x = max_value - x;
        }
    }
    auto [min_cost, assignment] = min_assignment(value);
    return {max_value * static_cast<T>(value.size()) - min_cost, assignment}; // Overflow warning.
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int64_t>> cost{
        {9, 2, 7, 8},
        {6, 4, 3, 7},
        {5, 8, 1, 8},
    };
    auto [min_cost, assignment] = min_assignment(cost);
    assert(min_cost == 9 && (assignment == vector<int>{1, 0, 2}));

    auto [max_value, max_jobs] = max_assignment(cost);
    assert(max_value == 24 && (max_jobs == vector<int>{0, 3, 1}));
    return 0;
}

```

### 4.6.8 Stable Marriage (Gale-Shapley)

The stable marriage problem takes  $n$  men and  $n$  women, with every person ranking all members of the opposite group in strict order of preference, and seeks a bijection between the two groups. An unmatched pair  $(m, w)$  is a blocking pair if man  $m$  prefers woman  $w$  to his assigned partner and woman  $w$  prefers man  $m$  to hers. A matching is stable if it has no blocking pair. Unlike maximum matching, the goal is therefore not to maximize the number of pairs, but to find a complete pairing that satisfies this stability condition.

The Gale-Shapley algorithm produces a stable matching, and a stable matching always exists. Each free man proposes to the most preferred woman who has not yet rejected him. A woman provisionally accepts her best proposal so far and rejects the rest, possibly breaking a previous engagement when a man she prefers proposes later. Because every man descends his list at most once and each proposal is constant work after precomputing the women's rankings, the algorithm runs in quadratic time. The result is man-optimal: every man receives the best partner he could have in any stable matching.

- `stable_marriage(men_pref, women_pref)` returns a vector `wife` where `wife[m]` is the woman matched to man `m` in the man-optimal stable matching, given the input preference lists each as  $n$  by  $n$  matrices. `men_pref[m]` lists the women in `m`'s order of preference, most preferred first, and `women_pref[w]` lists the men in `w`'s order of preference.

**Time Complexity:**  $O(n^2)$  per call, where  $n$  is the number of men (equivalently, women).

**Space Complexity:**  $O(n^2)$  auxiliary for the precomputed rankings.

```
#include <cassert>
#include <queue>
#include <vector>

std::vector<int> stable_marriage(
    const std::vector<std::vector<int>>& men_pref, const std::vector<std::vector<int>>& women_pref
) {
    int n = static_cast<int>(men_pref.size());
    assert(static_cast<int>(women_pref.size()) == n);
    std::vector<std::vector<int>> rank(n, std::vector<int>(n));
    for (int w = 0; w < n; w++) {
        for (int i = 0; i < n; i++) {
            rank[w][women_pref[w][i]] = i; // How woman w ranks each man (smaller is better).
        }
    }
    std::vector<int> next_proposal(n), husband(n, -1), wife(n, -1);
    std::queue<int> free_men;
    for (int m = 0; m < n; m++) {
        free_men.push(m);
    }
    while (!free_men.empty()) {
        int m = free_men.front();
        free_men.pop();
        int w = men_pref[m][next_proposal[m]++];
        if (husband[w] == -1) {
            husband[w] = m;
            wife[m] = w;
        } else if (rank[w][m] < rank[w][husband[w]]) {
            wife[husband[w]] = -1;
            free_men.push(husband[w]);
            husband[w] = m;
            wife[m] = w;
        } else {
            free_men.push(m); // Rejected; m remains free and will propose to the next woman.
        }
    }
    return wife;
}
```

## Example Usage

```
using namespace std;

int main() {
    // All three men prefer w0 > w1 > w2; all three women prefer m2 > m1 > m0.
    vector<vector<int>> men_pref{{0, 1, 2}, {0, 1, 2}, {0, 1, 2}};
    vector<vector<int>> women_pref{{2, 1, 0}, {2, 1, 0}, {2, 1, 0}};

    // Cascading proposals leave m2 with w0, m1 with w1, m0 with w2.
    assert((stable_marriage(men_pref, women_pref) == vector<int>{2, 1, 0}));
    return 0;
}
```

## 4.7 Exponential Graph Problems

---

### 4.7.1 Maximum Clique (Bron-Kerbosch)

Given a simple undirected graph, determine a maximum clique: a largest subset of nodes in which every pair is connected by an edge. A weighted variant instead seeks the clique of maximum total node weight, given a weight for each node.

The Bron-Kerbosch algorithm recursively extends a growing clique, tracking the set of candidate nodes that may still be added and the set of nodes already excluded. Choosing a pivot node to avoid branching on its neighbors prunes large parts of the search, keeping it efficient on most graphs.

- `max_clique()` returns the maximum clique size for a global, bidirectionally pre-populated adjacency matrix `adj`, whose row and column indices represent the nodes.
- `max_clique_weighted()` additionally uses the global weight array `w` and returns the maximum clique weight.

These implementations use `Mask`, which is `uint64_t` by default, so the number of nodes must be less than `MASK_BITS`. Node weights must be nonnegative.

**Time Complexity:**  $O(3^{n/3})$  per call to `max_clique()` and `max_clique_weighted()`, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(n^2)$  for storage of the graph, where  $n$  is the number of nodes.
- $O(n)$  auxiliary stack space for `max_clique()` and `max_clique_weighted()`.

```
#include <algorithm>
#include <cassert>
#include <climits>
#include <cstdint>
#include <vector>

using Mask = uint64_t;
const int MASK_BITS = sizeof(Mask) * CHAR_BIT;

std::vector<std::vector<char>> adj;
std::vector<int64_t> w;

std::vector<Mask> build_mask_graph() {
    int n = static_cast<int>(adj.size());
    assert(n < MASK_BITS);
    std::vector<Mask> g(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j]) {
                g[i] |= Mask{1} << j;
            }
        }
    }
}
```

```

    }
}
return g;
}

int max_clique_rec(const std::vector<Mask> &g, Mask curr, Mask pool, Mask excl) {
    if (pool == 0) {
        return __builtin_popcountll(curr);
    }
    int res = 0;
    int pivot = __builtin_ctzll(pool | excl);
    Mask candidates = pool & ~g[pivot];
    while (candidates != 0) {
        int u = __builtin_ctzll(candidates);
        res = std::max(res, max_clique_rec(g, curr | (Mask{1} << u), pool & g[u], excl & g[u]));
        pool ^= Mask{1} << u;
        excl |= Mask{1} << u;
        candidates &= candidates - 1;
    }
    return res;
}

int max_clique() {
    int n = static_cast<int>(adj.size());
    std::vector<Mask> g = build_mask_graph();
    return max_clique_rec(g, 0, (Mask{1} << n) - 1, 0);
}

int64_t max_clique_weighted_rec(const std::vector<Mask> &g, int64_t curr, Mask pool, Mask excl) {
    if (pool == 0) {
        return curr;
    }
    int64_t res = -1;
    int pivot = __builtin_ctzll(pool | excl);
    Mask z = pool & ~g[pivot];
    while (z != 0) {
        int u = __builtin_ctzll(z);
        int64_t next_weight = curr + w[u]; // Overflow warning.
        res = std::max(res, max_clique_weighted_rec(g, next_weight, pool & g[u], excl & g[u]));
        pool ^= Mask{1} << u;
        excl |= Mask{1} << u;
        z &= z - 1;
    }
    return res;
}

int64_t max_clique_weighted() {
    int n = static_cast<int>(adj.size());
    assert(static_cast<int>(w.size()) == n);
    std::vector<Mask> g = build_mask_graph();
    return max_clique_weighted_rec(g, 0, (Mask{1} << n) - 1, 0);
}

```

## Example Usage

```

using namespace std;

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

int main() {
    //      0(10)
    //      // |
    //      /  / |

```

```

//      /   /   |
// 1(20)---+---3(40)---4(50)
//      \   |   /
//      \   |   /
//      \   |   /
//      2(30)-----+
int nodes = 5;
adj.assign(nodes, vector<char>(nodes));
w.assign(nodes, 0);
add_edge(0, 1);
add_edge(0, 2);
add_edge(0, 3);
add_edge(1, 2);
add_edge(1, 3);
add_edge(2, 3);
add_edge(3, 4);
add_edge(4, 2);
w = {10, 20, 30, 40, 50};
assert(max_clique() == 4);
assert(max_clique_weighted() == 120);
return 0;
}

```

### 4.7.2 Graph Coloring (Backtracking)

Given a simple undirected graph, assign a color to every node such that no pair of adjacent nodes have the same color, and that the total number of colors used is minimized.

This implementation finds the chromatic number by backtracking: nodes (ordered by degree to prune sooner) are colored one at a time, each tried with every color already in use plus one new color, while the fewest colors seen in a complete coloring so far bounds the search and cuts off any branch at least as costly.

- `min_coloring()` populates `color` and returns minimum color count for a global, bidirectionally pre-populated adjacency matrix `adj`, whose row and column indices represent the nodes.

**Time Complexity:**  $O(n^n)$  per call in the worst case, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(n^2)$  for storage of the graph, where  $n$  is the number of nodes.
- $O(n)$  auxiliary stack space and  $O(n)$  auxiliary heap space.

```

#include <algorithm>
#include <numeric>
#include <vector>

std::vector<std::vector<char>> adj;
std::vector<int> color, curr, order;
int min_colors;

void color_rec(int pos, int used_colors) {
    if (used_colors >= min_colors) {
        return;
    }
    if (pos == static_cast<int>(order.size())) {
        color = curr;
        min_colors = used_colors;
        return;
    }
    int u = order[pos];
    for (int c = 0; c < used_colors; c++) {
        bool valid = true;
        for (int i = 0; i < pos; i++) {
            int v = order[i];

```

```

        if (adj[u][v] && curr[v] == c) {
            valid = false;
            break;
        }
    }
    if (valid) {
        curr[u] = c;
        color_rec(pos + 1, used_colors);
        curr[u] = -1;
    }
}
curr[u] = used_colors;
color_rec(pos + 1, used_colors + 1);
curr[u] = -1;
}

int min_coloring() {
    int n = static_cast<int>(adj.size());
    if (n == 0) {
        color.clear();
        return 0;
    }
    std::vector<int> degree(n);
    for (int u = 0; u < n; u++) {
        degree[u] = std::accumulate(adj[u].begin(), adj[u].end(), 0);
    }
    order.resize(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int u, int v) {
        if (degree[u] != degree[v]) {
            return degree[u] > degree[v];
        }
        return u < v;
    });
    min_colors = n + 1;
    color.assign(n, -1);
    curr.assign(n, -1);
    color_rec(0, 0);
    std::vector<int> remap(min_colors, -1);
    int colors = 0;
    for (int &c : color) {
        if (remap[c] == -1) {
            remap[c] = colors++;
        }
        c = remap[c];
    }
    return min_colors;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

int main() {
    //      0
    //    / |
    // 1---4---2
    //  \ | /
    //    3
    int nodes = 5;
}

```

```

adj.assign(nodes, vector<char>(nodes));
add_edge(0, 1);
add_edge(0, 4);
add_edge(1, 3);
add_edge(1, 4);
add_edge(2, 3);
add_edge(2, 4);
add_edge(3, 4);
assert(min_coloring() == 3);
for (int u = 0; u < nodes; u++) {
    for (int v = u + 1; v < nodes; v++) {
        if (adj[u][v]) {
            assert(color[u] != color[v]);
        }
    }
}
assert((color == vector<int>{0, 1, 1, 0, 2}));
return 0;
}

```

### 4.7.3 Edge Coloring (Misra-Gries)

Given a simple undirected graph with maximum degree  $D$ , assign a color to every edge so that no two edges sharing an endpoint receive the same color. Vizing's theorem guarantees that  $D + 1$  colors always suffice for a simple graph (while  $D$  colors are clearly necessary), and the Misra-Gries algorithm constructs such a coloring. Deciding between  $D$  and  $D + 1$  colors is NP-hard in general, though bipartite graphs can always be edge-colored with exactly  $D$  colors.

The algorithm colors edges one at a time. To color an edge  $(u, v)$  it grows a maximal “fan” of  $u$ : a sequence of neighbors starting at  $v$  in which each successive edge's color is free at the previous fan node. It then picks a color  $c$  free at  $u$  and a color  $d$  free at the fan's last node, and flips the colors along the maximal alternating  $c/d$  path leaving  $u$ . This frees  $d$  at  $u$ , after which rotating the fan shifts colors over by one and assigns  $d$  to the new edge. Each step keeps the partial coloring proper and never introduces a  $(D + 2)$ -th color.

The graph must be simple: the implementation stores one color per ordered node pair in a dense matrix, so self-loops and parallel edges between the same pair cannot be represented.

- `edge_coloring(n, edges)` returns a vector of colors, one per entry of `edges`, using colors in the range  $[0, D]$  where  $D$  is the maximum degree. Nodes are numbered  $[0, n)$ , and `edges[i]` is an undirected edge  $u-v$ .

**Time Complexity:**  $O(n^3 \cdot m)$  per call in the worst case, where  $n$  is the number of nodes and  $m$  is the number of edges; substantially faster in practice.

**Space Complexity:**

- $O(n^2)$  to store the per-edge colors.
- $O(n)$  auxiliary heap and stack space.

```

#include <utility>
#include <vector>

std::vector<int> edge_coloring(int n, const std::vector<std::pair<int, int>> &edges) {
    std::vector<std::vector<int>> color(n, std::vector<int>(n));
    auto is_free = [&](int x, int c) {
        for (int w = 0; w < n; w++) {
            if (color[x][w] == c) {
                return false;
            }
        }
        return true;
    };
    auto free_color = [&](int x) {

```

```

    int c = 1;
    while (!is_free(x, c)) {
        c++;
    }
    return c;
};

auto set_color = [&](int u, int v, int c) {
    color[u][v] = c;
    color[v][u] = c;
};

// Flip colors c and d along the maximal alternating path of those colors starting at x.
auto flip_path = [&](auto &&self, int x, int d, int c) -> void {
    for (int w = 0; w < n; w++) {
        if (color[x][w] == d) {
            set_color(x, w, 0);
            self(self, w, c, d);
            set_color(x, w, c);
            return;
        }
    }
};

for (auto [u, v] : edges) {
    // Build a maximal fan of u starting at v: fan[0] = v, and color(u, fan[i+1]) is free at fan[i].
    std::vector<int> fan{v};
    std::vector<char> used(n);
    used[v] = true;
    while (true) {
        int last = fan.back(), next = -1;
        for (int w = 0; w < n; w++) {
            if (!used[w] && color[u][w] != 0 && is_free(last, color[u][w])) {
                next = w;
                break;
            }
        }
        if (next == -1) {
            break;
        }
        fan.push_back(next);
        used[next] = true;
    }
    int c = free_color(u), d = free_color(fan.back());
    flip_path(flip_path, u, d, c);
    // Find the prefix of the fan to rotate: the first node on which d is now free.
    int k = 0;
    while (k < static_cast<int>(fan.size()) && color[u][fan[k]] != d && !is_free(fan[k], d)) {
        k++;
    }
    for (int j = 0; j < k; j++) {
        set_color(u, fan[j], color[u][fan[j + 1]]);
    }
    set_color(u, fan[k], d);
}

std::vector<int> res;
res.reserve(edges.size());
for (auto [u, v] : edges) {
    res.push_back(color[u][v] - 1); // Shift internal 1..D+1 colors to 0..D.
}
return res;
}

```

## Example Usage

```

#include <cassert>
#include <set>
using namespace std;

```



```

int main() {
    // A 5-cycle has maximum degree 2 but is odd, so it needs 3 colors (D + 1).
    vector<pair<int, int>> cycle{{0, 1}, {1, 2}, {2, 3}, {3, 4}, {4, 0}};
    vector<int> col = edge_coloring(5, cycle);
    set<int> palette(col.begin(), col.end());
    assert(palette.size() == 3);

    // Verify the coloring is proper: edges sharing an endpoint differ in color.
    int m = static_cast<int>(cycle.size());
    for (int i = 0; i < m; i++) {
        for (int j = i + 1; j < m; j++) {
            bool share = cycle[i].first == cycle[j].first || cycle[i].first == cycle[j].second ||
                         cycle[i].second == cycle[j].first || cycle[i].second == cycle[j].second;
            if (share) {
                assert(col[i] != col[j]);
            }
        }
    }

    // A bipartite graph (here K_{2,3}) is edge-colored with exactly D colors.
    vector<pair<int, int>> bip{{0, 2}, {0, 3}, {0, 4}, {1, 2}, {1, 3}, {1, 4}};
    vector<int> bcol = edge_coloring(5, bip);
    set<int> bpal(bcol.begin(), bcol.end());
    assert(static_cast<int>(bpal.size()) == 3); // Max degree is 3.
    return 0;
}

```

#### 4.7.4 Shortest Hamiltonian Path and Cycle (Held-Karp)

Given a complete, weighted, directed graph, find a minimum-length Hamiltonian path or cycle. A Hamiltonian path visits each node exactly once. A Hamiltonian cycle additionally returns to its starting node; this cycle version is the traveling salesman problem (TSP).

Both functions use the Held-Karp subset dynamic program. Let  $dp(S, i)$  be the shortest path that visits exactly the nodes in  $S$  and ends at  $i$ . Its final edge must come from some  $j \in S \setminus \{i\}$ , giving the recurrence  $dp(S, i) = \min_j (dp(S \setminus \{i\}, j) + w(j, i))$ . Paths initialize every singleton set; cycles initialize only node 0 and consider only subsets containing it. After processing the full set, the algorithm chooses the best final node, including its return edge to node 0 for a cycle. Reconstruction repeats the same minimization backward instead of storing a separate parent table.

- `shortest_hamiltonian_path()` populates `path` and returns the minimum Hamiltonian path length for a global, pre-populated adjacency matrix `adj`.
- `shortest_hamiltonian_cycle()` fixes the start at node 0, populates `path`, and returns the minimum Hamiltonian cycle length for a global, pre-populated adjacency matrix `adj`. The path lists every node once; the closing edge back to node 0 is implicit.

Since this implementation uses bitmasks with signed 32-bit integers, the maximum number of nodes must be less than 31.

**Time Complexity:**  $O(2^n \cdot n^2)$  per call, where  $n$  is the number of nodes.

**Space Complexity:**

- $O(n^2)$  for storage of the graph, where  $n$  is the number of nodes.
- $O(2^n \cdot n)$  auxiliary.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;

```

```

std::vector<std::vector<int64_t>> adj;
std::vector<int> path;

int64_t shortest_hamiltonian_path() {
    int n = static_cast<int>(adj.size());
    assert(1 <= n && n < 31);
    int max_mask = (1 << n) - 1;
    std::vector<std::vector<int64_t>> dp(max_mask + 1, std::vector<int64_t>(n, INF));
    path.assign(n, 0);
    for (int i = 0; i < n; i++) {
        dp[1 << i][i] = 0;
    }
    for (int mask = 1; mask <= max_mask; mask++) {
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                for (int j = 0; j < n; j++) {
                    if ((mask & (1 << j)) != 0 && dp[mask ^ (1 << i)][j] != INF) {
                        int64_t candidate = dp[mask ^ (1 << i)][j] + adj[j][i]; // Overflow warning.
                        dp[mask][i] = std::min(dp[mask][i], candidate);
                    }
                }
            }
        }
    }
    int64_t res = INF;
    for (int i = 0; i < n; i++) {
        res = std::min(res, dp[max_mask][i]);
    }
    // Optional: reconstruct one shortest Hamiltonian path.
    int mask = max_mask, old = -1;
    for (int i = n - 1; i >= 0; i--) {
        int best = -1;
        for (int j = 0; j < n; j++) {
            if ((mask & (1 << j)) != 0 &&
                (best == -1 || dp[mask][best] + (old == -1 ? 0 : adj[best][old]) >
                 dp[mask][j] + (old == -1 ? 0 : adj[j][old]))) {
                best = j;
            }
        }
        path[i] = best;
        mask ^= 1 << best;
        old = best;
    }
    return res;
}

int64_t shortest_hamiltonian_cycle() {
    int n = static_cast<int>(adj.size());
    assert(1 <= n && n < 31);
    if (n == 1) {
        path = {0};
        return 0;
    }
    int max_mask = (1 << n) - 1;
    std::vector<std::vector<int64_t>> dp(max_mask + 1, std::vector<int64_t>(n, INF));
    path.assign(n, 0);
    dp[1][0] = 0;
    for (int mask = 1; mask <= max_mask; mask += 2) {
        for (int i = 1; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                for (int j = 0; j < n; j++) {
                    if ((mask & (1 << j)) != 0 && dp[mask ^ (1 << i)][j] != INF) {
                        int64_t candidate = dp[mask ^ (1 << i)][j] + adj[j][i]; // Overflow warning.
                        dp[mask][i] = std::min(dp[mask][i], candidate);
                    }
                }
            }
        }
    }
}

```

```

}
int64_t res = INF;
for (int i = 1; i < n; i++) {
    res = std::min(res, dp[max_mask][i] + adj[i][0]); // Overflow warning.
}
// Optional: reconstruct one shortest Hamiltonian cycle.
int mask = max_mask, old = 0;
for (int i = n - 1; i >= 1; i--) {
    int best = -1;
    for (int j = 1; j < n; j++) {
        if ((mask & (1 << j)) != 0 &&
            (best == -1 || dp[mask][best] + adj[best][old] > dp[mask][j] + adj[j][old])) {
            best = j;
        }
    }
    path[i] = best;
    mask ^= 1 << best;
    old = best;
}
return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Complete directed graph where 0->1->2->0 costs 1 + 2 + 3.
    // The reverse cycle is intentionally more expensive.
    //   +--> 0 <-----+
    //   |      | \      |
    // w=7| w=1|  \w=1 |w=3
    //   |      | \      |
    //   |      v  w=2 v |
    //   +--- 1 -----> 2
    //           ^-----/
    //           w=5
    int nodes = 3;
    adj.assign(nodes, vector<int64_t>(nodes));
    adj[0][1] = 1;
    adj[0][2] = 1;
    adj[1][0] = 7;
    adj[1][2] = 2;
    adj[2][0] = 3;
    adj[2][1] = 5;
    assert(shortest_hamiltonian_path() == 3);
    assert((path == vector<int>{0, 1, 2}));
    assert(shortest_hamiltonian_cycle() == 6);
    assert((path == vector<int>{0, 1, 2}));
    return 0;
}

```

## Chapter 5

# Optimization

### 5.1 Searching Sorted Data

---

#### 5.1.1 Binary Search

Binary search locates a target in sorted data, or more generally the transition point of a monotonic Boolean predicate that changes from false to true or true to false at most once. Each step evaluates the midpoint of the current interval and keeps the half that still contains the transition, halving the search space until it closes on the answer. The same technique applies to non-increasing or non-decreasing numeric functions. Unlike searching through an array, discrete binary search is not restricted by available memory, making it useful for implicit domains that are too large to materialize, including broad integer or real intervals.

- `first_true(lo, hi, pred)` takes signed integer boundaries for the search space  $[lo, hi)$  (i.e. including `lo`, but excluding `hi`) and returns the smallest integer `k` in  $[lo, hi)$  for which the predicate `pred(k)` tests true. If `pred(k)` tests false for the entire input range, then the original `hi` is returned. The caller must ensure `pred` is monotonically ascending on the input range, i.e. returning all false for some (possibly empty) prefix, followed by all true in some (possibly empty) suffix. E.g., patterns `01`, `00`, and `11` are allowed, but `10` is disallowed.
- `last_true(lo, hi, pred)` takes signed integer boundaries for the search space  $[lo, hi)$  (i.e. including `lo`, but excluding `hi`) and returns the largest integer `k` in  $[lo, hi)$  for which the predicate `pred(k)` tests true. If `pred(k)` tests false for the entire input range, then the original `lo - 1` is returned, so `lo` must exceed the minimum representable value. The caller must ensure `pred` is monotonically descending on the input range, i.e. returning all true for some (possibly empty) prefix, followed by all false in some (possibly empty) suffix. E.g., patterns `10`, `00`, and `11` are allowed, but `01` is disallowed.
- `first_true_real(lo, hi, pred)` is the equivalent of `first_true()` on floating point predicates. Since any interval of real numbers is dense, the exact target cannot be found due to floating point error. Instead, a value that is very close to the border between false and true is returned. The precision of the answer depends on the number of repetitions the function performs. Since each repetition bisects the search space, the absolute error of the answer is  $1/(2^n)$  times the distance between `lo` and `hi` after  $n$  repetitions. Although the error can be controlled by looping until the distance shrinks to an arbitrary epsilon, it is simpler to let the loop run for a desired number of iterations until floating point arithmetic breaks down. 100 iterations is usually sufficient, since the search space will be reduced to  $2^{-100}$  (roughly  $10^{-30}$ ) times its original size. This implementation can be modified to find the “last true” point by simply interchanging the assignments of `lo` and `hi` in the if-else statements.

Overflow warning: For both integer searches, `hi - lo` must be representable by `Int`. For example, `[INT_MIN, INT_MAX)` will overflow if passed as signed 32-bit integer boundaries.

**Time Complexity:**

- $O(\log n)$  calls to `pred()` per call to `first_true()` and `last_true()`, where  $n$  is the distance between `lo` and `hi`.
- $O(n)$  calls to `pred()` per call to `first_true_real()`, where  $n$  is the number of iterations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
template<typename Int, typename Pred>
Int first_true(Int lo, Int hi, Pred pred) { // 000[1]11
    while (lo < hi) {
        Int mid = lo + (hi - lo) / 2;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo; // hi if all false
}

template<typename Int, typename Pred>
Int last_true(Int lo, Int hi, Pred pred) { // 11[1]000
    while (lo < hi) {
        Int mid = lo + (hi - lo) / 2;
        if (pred(mid)) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo - 1;
}

template<typename Pred>
double first_true_real(double lo, double hi, Pred pred) { // 000[1]11
    double mid;
    for (int i = 0; i < 100; i++) {
        mid = (lo + hi) / 2.0;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    return lo;
}
```

**Example Usage**

```
#include <cassert>
#include <cmath>
#include <vector>
using namespace std;

int main() {
    assert(first_true(0, 7, [](int x) { return x >= 3; }) == 3);
    assert(first_true(0, 7, [](int x) { return true; }) == 0);
    assert(first_true(0, 7, [](int x) { return false; }) == 7);
    assert(first_true(4, 4, [](int x) { return x >= 4; }) == 4);

    assert(last_true(0, 7, [](int x) { return x <= 5; }) == 5);
    assert(last_true(0, 7, [](int x) { return true; }) == 6);
    assert(last_true(0, 7, [](int x) { return false; }) == -1);
    assert(last_true(4, 4, [](int x) { return x <= 4; }) == 3);
}
```

```

vector<int> a{1, 2, 2, 4, 7};
int n = static_cast<int>(a.size());
assert(first_true(0, n, [&](int i) { return a[i] >= 2; }) == 1); // lower_bound(2)
assert(first_true(0, n, [&](int i) { return a[i] > 2; }) == 3); // upper_bound(2)

double res = first_true_real(-10.0, 10.0, [](double x) { return x >= 1.2345; });
assert(fabs(res - 1.2345) < 1e-15);
return 0;
}

```

### 5.1.2 Exponential Search

Finds the first value where a monotone Boolean predicate becomes true when an upper bound is not known in advance. Exponential search first grows the search range by powers of two until it brackets the transition point, then finishes with ordinary binary search. This is useful for answer-search problems over unbounded or very large integer domains.

- `exponential_first_true(lo, pred)` returns the smallest integer `x` greater than or equal to `lo` such that `pred(x)` is true.

Overflow warning: The predicate must become true before the exponentially growing probe overflows `Int`; otherwise, add an explicit limit.

**Time Complexity:**  $O(\log n)$  calls to `pred()` per call, where  $n$  is the distance from `lo` to the first true value.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cstdint>

template<typename Int, typename Pred>
Int exponential_first_true(Int lo, Pred pred) { // 000[1]11
    if (pred(lo)) {
        return lo;
    }
    Int step = 1;
    while (!pred(lo + step)) {
        step *= 2;
    }
    Int hi = lo + step;
    lo += step / 2;
    while (lo < hi) {
        Int mid = lo + (hi - lo) / 2;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

```

#### Example Usage

```

#include <cassert>

int main() {
    auto at_least_1000 = [](int x) { return x >= 1000; };
    assert(exponential_first_true(1000, at_least_1000) == 1000);
    assert(exponential_first_true(0, at_least_1000) == 1000);
    assert(exponential_first_true(5, at_least_1000) == 1000);
    auto square_large_enough = [](int64_t x) { return x * x >= 123456789LL; };
    assert(exponential_first_true(0LL, square_large_enough) == 11112);
}

```

```
    return 0;
}
```

### 5.1.3 Interpolation Search

Given a sorted range of numeric values, find a target by guessing where it should be rather than always probing the middle. Binary search halves the range because it only asks whether the target is smaller or larger than the probe; interpolation search additionally uses how much smaller or larger. Assuming the values rise roughly linearly across the range, the target's position is estimated by linear interpolation between the two endpoints, the same way a reader opens a dictionary near the back to look up a word starting with “s”.

Each probe reduces the expected distance to the target from  $n$  to  $\sqrt{n}$  when the values are drawn from a uniform distribution, which gives  $O(\log \log n)$  probes: for a billion sorted integers that is about five probes rather than thirty. The bound depends entirely on the distribution. Clustered values push the estimate to one end repeatedly and degrade the search to  $O(n)$  probes, so this is the rare algorithm whose worst case is worse than the method it replaces. Use it when the data is known to be near-uniform, such as generated IDs, timestamps at a fixed rate, or hash values, and use binary search otherwise.

- `interpolation_search(lo, hi, key)` returns an iterator to an occurrence of `key` in the sorted random-access range `[lo, hi)`, or `hi` if `key` does not occur. The value type must support subtraction and conversion to `double`.

**Time Complexity:**  $O(\log \log n)$  per call on uniformly distributed values and  $O(n)$  in the worst case, where  $n$  is the range length.

**Space Complexity:**  $O(1)$  auxiliary.

```
#include <iterator>

template<typename It, typename T>
It interpolation_search(It lo, It hi, const T &key) {
    It last = hi;
    while (lo < hi && key >= *lo && key <= *(hi - 1)) {
        if (*(hi - 1) == *lo) { // A constant span holds the key only if it equals that value.
            return *lo == key ? lo : last;
        }
        // Estimate the offset by where the key falls between the endpoint values.
        double fraction = static_cast<double>(key - *lo) / static_cast<double>(*(hi - 1) - *lo);
        auto span = std::distance(lo, hi) - 1;
        It probe =
            lo + static_cast<typename std::iterator_traits<It>::difference_type>(fraction * span);
        if (*probe == key) {
            return probe;
        }
        if (*probe < key) {
            lo = probe + 1;
        } else {
            hi = probe;
        }
    }
    return last;
}
```

#### Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

int main() {
```

```

vector<int> a{2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
for (int i = 0; i < static_cast<int>(a.size()); i++) {
    assert(interpolation_search(a.begin(), a.end(), a[i]) - a.begin() == i);
}
assert(interpolation_search(a.begin(), a.end(), 5) == a.end());
assert(interpolation_search(a.begin(), a.end(), 1) == a.end()); // Below the range.
assert(interpolation_search(a.begin(), a.end(), 99) == a.end()); // Above the range.
assert(interpolation_search(a.begin(), a.begin(), 2) == a.begin());

// Heavily clustered values still return the right answer, only more slowly.
vector<int> skewed{1, 1, 1, 1, 1, 1000000};
assert(*interpolation_search(skewed.begin(), skewed.end(), 1000000) == 1000000);
assert(*interpolation_search(skewed.begin(), skewed.end(), 1) == 1);
assert(interpolation_search(skewed.begin(), skewed.end(), 7) == skewed.end());
return 0;
}

```

### 5.1.4 Sorted Matrix Search

Given a matrix whose every row is sorted left to right and every column sorted top to bottom, find a target value. Such a matrix is not sorted overall, since a cell says nothing about the cells diagonally adjacent to it, so a single binary search over the flattened grid does not apply.

Starting at the top-right corner makes each comparison decisive. That cell is the largest in its row and the smallest in its column, so if it exceeds the target then no cell in its column can match and the whole column is discarded, while if it is smaller then no cell in its row can match and the row is discarded. Either way one row or one column disappears per comparison, so the walk visits at most  $R + C$  cells while descending a staircase-shaped boundary. The bottom-left corner works the same way with the directions swapped; the other two corners are useless, since they are extreme in both directions at once and a comparison there rules out nothing.

- `search_sorted_matrix(a, key)` returns the position (`row`, `col`) of an occurrence of `key` in the row-wise and column-wise sorted matrix `a`, or `(-1, -1)` if `key` does not occur.
- `count_at_most(a, key)` returns how many entries of `a` are less than or equal to `key`, by measuring the same staircase boundary rather than stopping at a match. Combined with the binary search of section 5.1.1 over the value range, it yields the 1-based  $k$ -th smallest entry in  $O((R + C) \log(\max - \min))$  time without materializing the sorted order.

A matrix sorted in reading order, every row beginning after the previous ends, is the easier problem of binary searching one sorted sequence of length  $R \cdot C$ .

**Time Complexity:**  $O(R + C)$  per call, where  $R$  and  $C$  are the row and column counts.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cstdint>
#include <utility>
#include <vector>

template<typename T>
std::pair<int, int> search_sorted_matrix(const std::vector<std::vector<T>> &a, const T &key) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    for (int r = 0, c = cols - 1; r < rows && c >= 0; ) {
        if (a[r][c] < key) {
            r++; // Every cell left in this row is even smaller.
        } else if (key < a[r][c]) {
            c--; // Every cell below in this column is even larger.
        } else {
            return {r, c};
        }
    }
    return {-1, -1};
}

```



```

}

template<typename T>
int64_t count_at_most(const std::vector<std::vector<T>> &a, const T &key) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    int64_t res = 0;
    for (int r = 0, c = cols - 1; r < rows;) {
        if (c >= 0 && key < a[r][c]) {
            c--;
        } else {
            res += c + 1; // The row's first c + 1 entries are all at most key.
            r++;
        }
    }
    return res;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>> a{
        {1, 4, 7, 11},
        {2, 5, 8, 12},
        {3, 6, 9, 16},
        {10, 13, 14, 17},
    };
    assert(search_sorted_matrix(a, 5) == make_pair(1, 1));
    assert(search_sorted_matrix(a, 1) == make_pair(0, 0)); // Top-left corner.
    assert(search_sorted_matrix(a, 17) == make_pair(3, 3)); // Bottom-right corner.
    assert(search_sorted_matrix(a, 11) == make_pair(0, 3)); // The starting cell itself.
    assert(search_sorted_matrix(a, 15) == make_pair(-1, -1));

    assert(count_at_most(a, 0) == 0);
    assert(count_at_most(a, 5) == 5); // 1, 2, 3, 4, and 5.
    assert(count_at_most(a, 100) == 16);

    vector<vector<int>> single{{42}};
    assert(search_sorted_matrix(single, 42) == make_pair(0, 0));
    assert(search_sorted_matrix(single, 7) == make_pair(-1, -1));
    return 0;
}

```

### 5.1.5 Local Minimum

Finds a non-strict local minimum in an array, that is, an index  $i$  whose value is no greater than the values at its existing neighbors. Unlike ternary search, this does not require the array to be unimodal, but the result is only a local minimum and need not be the global minimum.

Compare the middle element with its right neighbor. If the sequence slopes downward, the right half contains a local minimum; otherwise, the middle element or the left half contains one. Each comparison therefore discards half of the remaining indices.

The same argument works in two dimensions, where a local minimum is a cell no greater than its four orthogonal neighbors. Scan the middle column for its smallest entry; if that entry is no greater than its left and right neighbors it is a local minimum, and otherwise the smaller of those two neighbors lies in a half that must contain one. The half discarded cannot be entered again, because crossing back would require passing through a cell larger

than the one just left, so recursing on the surviving half halves the columns each time. Scanning a column costs  $O(R)$ , giving  $O(R \log C)$ .

- `local_min_index(a)` returns the index of a non-strict local minimum in the nonempty array `a`. Either endpoint may be returned when it is no greater than its sole neighbor.
- `local_min_cell(a)` returns the position (`row`, `col`) of a non-strict local minimum in the nonempty grid `a`, whose rows must all have the same nonzero length.

#### Time Complexity:

- $O(\log n)$  comparisons per call to `local_min_index()`, where  $n$  is the size of `a`.
- $O(R \cdot \log C)$  comparisons per call to `local_min_cell()`, where  $R$  and  $C$  are the grid dimensions.

**Space Complexity:**  $O(1)$  auxiliary for both operations.

```
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
int local_min_index(const std::vector<T> &a) {
    assert(!a.empty());
    int lo = 0, hi = static_cast<int>(a.size()) - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[mid + 1]) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}

template<typename T>
std::pair<int, int> local_min_cell(const std::vector<std::vector<T>> &a) {
    int rows = static_cast<int>(a.size());
    assert(rows > 0 && !a[0].empty());
    int lo = 0, hi = static_cast<int>(a[0].size()) - 1;
    while (true) {
        int mid = lo + (hi - lo) / 2, best = 0;
        for (int r = 1; r < rows; r++) { // Smallest entry of the middle column.
            if (a[r][mid] < a[best][mid]) {
                best = r;
            }
        }
        bool descends_left = mid > lo && a[best][mid - 1] < a[best][mid];
        bool descends_right = mid < hi && a[best][mid + 1] < a[best][mid];
        if (!descends_left && !descends_right) {
            return {best, mid}; // No neighbor is smaller, so this cell is a local minimum.
        }
        if (descends_left && (!descends_right || a[best][mid - 1] <= a[best][mid + 1])) {
            hi = mid - 1;
        } else {
            lo = mid + 1;
        }
    }
}
```

#### Example Usage

```
using namespace std;

int main() {
    assert(local_min_index(vector<int>{9, 7, 3, 4, 8}) == 2);
}
```

```

assert(local_min_index(vector<int>{1, 2, 3, 4}) == 0);

vector<int> duplicates{4, 2, 2, 3};
int i = local_min_index(duplicates);
assert((i == 1 || i == 2) && duplicates[i] == 2);

vector<vector<int>> grid{
    {9, 8, 7},
    {4, 1, 6},
    {5, 2, 3},
};
assert(local_min_cell(grid) == make_pair(1, 1)); // The value 1 beats all four neighbors.
assert(local_min_cell(vector<vector<int>>{{5}}) == make_pair(0, 0));

// Every cell of a monotone grid slopes toward the corner, which is the only local minimum.
vector<vector<int>> slope{
    {1, 2, 3},
    {2, 3, 4},
    {3, 4, 5},
};
assert(local_min_cell(slope) == make_pair(0, 0));
return 0;
}

```

## 5.2 Continuous Optimization

---

### 5.2.1 Ternary and Golden Section Search

Given a unimodal function  $f$ , find the position of its global minimum or maximum: a value  $x$  such that the function strictly decreases up to  $x$  and strictly increases after it (for a minimum), or the reverse (for a maximum). All three searches below exploit the same idea: evaluate two interior points, and the comparison reveals one side of the interval that cannot contain the optimum, so it can be discarded. They differ only in how the probes are placed and whether the domain is continuous or discrete. Each maximum routine is the minimum routine applied to the negated function, since the location of a maximum of  $f$  is the location of a minimum of  $-f$ .

Ternary search splits the interval into thirds. It is the simplest to reason about, but it recomputes both probe points each step, spending two function evaluations per iteration while the interval shrinks to two-thirds.

Golden-section search places the probes at the golden ratio so that one probe is reused as an interior point of the next, smaller interval. It therefore performs only one new evaluation per iteration while shrinking the interval by a factor of  $1/\varphi$  (about 0.618), and reaches a given accuracy in roughly 2.4 times fewer evaluations than ternary search. Prefer it when the function is expensive to evaluate.

Brent's method keeps the golden-section guarantee while converging much faster on the smooth functions that occur in practice. It tracks the three best points seen so far, fits a parabola through them, and jumps to that parabola's vertex, which is superlinear once the interval is small enough for the function to look quadratic. A proposed vertex is accepted only when it lands inside the current interval and moves less than half the step before last; otherwise the method takes a golden-section step instead, so it never loses the guaranteed interval reduction. Unlike the two searches above, it stops early once the interval is smaller than the tolerance rather than always running a fixed number of iterations.

The discrete version operates on an integer domain, narrowing by thirds until at most three candidates remain and then scanning them. It returns the index of the optimum rather than a real coordinate. As with ternary search on reals, the function must be strictly unimodal.

- `ternary_search_min(lo, hi, f, eps = 1e-12)` and `ternary_search_max(lo, hi, f, eps = 1e-12)` return a `double x` in  $[lo, hi]$  optimizing the continuous unimodal function `f`, to within the optional absolute error `eps`.

- `golden_section_min(lo, hi, f, eps = 1e-12)` and `golden_section_max(lo, hi, f, eps = 1e-12)` do the same with one function evaluation per iteration.
- `brent_min(lo, hi, f, eps = 1e-12)` and `brent_max(lo, hi, f, eps = 1e-12)` do the same, fitting a parabola through the three best points when that fit is well behaved and falling back to a golden-section step when it is not.
- `discrete_ternary_min(lo, hi, f)` and `discrete_ternary_max(lo, hi, f)` return the integer index in range `[lo, hi]` optimizing the unimodal function `f`, breaking ties toward the smaller index.

#### Time Complexity:

- $O(\log(n/\text{eps}))$  calls to `f()` per call to `brent_min()` and `brent_max()` in the worst case, and far fewer once the parabolic steps take over.
- $O(\log(n/\text{eps}))$  calls to `f()` per continuous-search call, where  $n$  is the distance between `lo` and `hi`. Golden-section makes about half as many per iteration as ternary search.
- $O(\log n)$  calls to `f()` per discrete-search call.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cmath>

template<typename Fn>
double ternary_search_min(double lo, double hi, Fn f, double eps = 1e-12) {
    while (hi - lo > eps) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) {
            hi = m2;
        } else {
            lo = m1;
        }
    }
    return (lo + hi) / 2;
}

template<typename Fn>
double ternary_search_max(double lo, double hi, Fn f, double eps = 1e-12) {
    return ternary_search_min(lo, hi, [&](double x) { return -f(x); }, eps);
}

template<typename Fn>
double golden_section_min(double lo, double hi, Fn f, double eps = 1e-12) {
    const double r = 0.6180339887498949; // 1 / phi = (sqrt(5) - 1) / 2.
    double m1 = hi - r * (hi - lo), m2 = lo + r * (hi - lo);
    double f1 = f(m1), f2 = f(m2);
    while (hi - lo > eps) {
        if (f1 < f2) {
            hi = m2;
            m2 = m1;
            f2 = f1;
            m1 = hi - r * (hi - lo);
            f1 = f(m1);
        } else {
            lo = m1;
            m1 = m2;
            f1 = f2;
            m2 = lo + r * (hi - lo);
            f2 = f(m2);
        }
    }
    return (lo + hi) / 2;
}

template<typename Fn>
double golden_section_max(double lo, double hi, Fn f, double eps = 1e-12) {
    return golden_section_min(lo, hi, [&](double x) { return -f(x); }, eps);
}
```

```

template<typename Fn>
double brent_min(double lo, double hi, Fn f, double eps = 1e-12) {
    const double GOLD = 0.3819660112501051; // (3 - sqrt(5)) / 2, the golden-section fraction.
    double x = lo + GOLD * (hi - lo), w = x, v = x;
    double fx = f(x), fw = fx, fv = fx;
    double step = 0, prev_step = 0;
    for (int i = 0; i < 200; i++) {
        double mid = (lo + hi) / 2, tol = eps * std::fabs(x) + eps;
        if (std::fabs(x - mid) <= 2 * tol - (hi - lo) / 2) {
            break;
        }
        double p = 0, q = 0;
        if (std::fabs(prev_step) > tol) {
            // Fit a parabola through the three best points and solve for its vertex.
            double r = (x - w) * (fx - fv), s = (x - v) * (fx - fw);
            p = (x - v) * s - (x - w) * r;
            q = 2 * (s - r);
            if (q > 0) {
                p = -p;
            } else {
                q = -q;
            }
        }
        double old_step = prev_step;
        prev_step = step;
        // Accept the vertex only if it stays inside the interval and at least halves the last step.
        if (std::fabs(p) < std::fabs(q * old_step / 2) && p > q * (lo - x) && p < q * (hi - x)) {
            step = p / q;
            if (x + step - lo < 2 * tol || hi - (x + step) < 2 * tol) {
                step = x < mid ? tol : -tol;
            }
        } else {
            prev_step = (x < mid ? hi : lo) - x;
            step = GOLD * prev_step;
        }
        double u = x + (std::fabs(step) >= tol ? step : (step > 0 ? tol : -tol));
        double fu = f(u);
        if (fu <= fx) {
            if (u < x) {
                hi = x;
            } else {
                lo = x;
            }
        }
        v = w;
        fw = fx;
        w = x;
        fx = fu;
        x = u;
        fv = fu;
    } else {
        if (u < x) {
            lo = u;
        } else {
            hi = u;
        }
        if (fu <= fw || w == x) {
            v = w;
            fw = fx;
            w = u;
            fx = fu;
        } else if (fu <= fv || v == x || v == w) {
            v = u;
            fv = fu;
        }
    }
}
return x;
}

```

```

template<typename Fn>
double brent_max(double lo, double hi, Fn f, double eps = 1e-12) {
    return brent_min(lo, hi, [&f](double x) { return -f(x); }, eps);
}

template<typename Int, typename Fn>
Int discrete_ternary_min(Int lo, Int hi, Fn f) {
    while (hi - lo > 2) {
        Int m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) {
            hi = m2 - 1;
        } else {
            lo = m1 + 1;
        }
    }
    Int best = lo;
    auto best_value = f(best);
    for (Int k = lo + 1; k <= hi; k++) {
        auto value = f(k);
        if (value < best_value) {
            best = k;
            best_value = value;
        }
    }
    return best;
}

template<typename Int, typename Fn>
Int discrete_ternary_max(Int lo, Int hi, Fn f) {
    return discrete_ternary_min(lo, hi, [&](Int x) { return -f(x); });
}

```

## Example Usage

```

#include <cassert>
#include <cmath>
#include <vector>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-6;
}

int main() {
    // Parabola opening up with vertex at x = -2.
    auto up = [](double x) { return 3 * x * x + 12 * x - 12; };
    // Parabola opening down with vertex at x = 2/19.
    auto down = [](double x) { return -5.7 * x * x + 1.2 * x + 88; };

    assert(EQ(ternary_search_min(-1000, 1000, up), -2));
    assert(EQ(golden_section_min(-1000, 1000, up), -2));
    assert(EQ(ternary_search_max(-1000, 1000, down), 2.0 / 19));
    assert(EQ(golden_section_max(-1000, 1000, down), 2.0 / 19));
    assert(EQ(brent_min(-1000, 1000, up), -2));
    assert(EQ(brent_max(-1000, 1000, down), 2.0 / 19));

    // Discrete unimodal arrays: a valley and a peak.
    vector<int> valley{5, 3, 1, 2, 4, 6};
    vector<int> peak{1, 3, 7, 6, 2};
    auto v = [&](int i) { return valley[i]; };
    auto p = [&](int i) { return peak[i]; };
    assert(discrete_ternary_min(0, static_cast<int>(valley.size()) - 1, v) == 2); // Value 1.
    assert(discrete_ternary_max(0, static_cast<int>(peak.size()) - 1, p) == 2); // Value 7.
    return 0;
}

```

### 5.2.2 2D Hill Climbing

Given a continuous geometric objective  $f(x, y)$  and a (possibly arbitrary) starting guess  $(x_0, y_0)$ , approximately minimize the function using two-dimensional hill climbing.

The heuristic starts at the guess and evaluates one step in each of eight evenly spaced directions. It moves to the best improving neighbor, checks all eight directions again, and reduces the step size when no neighbor improves the answer. This repeats until the step size falls below the chosen threshold. Success depends heavily on the behavior of  $f$  and the initial guess, so the result is not guaranteed to be the global minimum.

- `hill_climb_min(f, x0, y0, &best_x, &best_y, step_min = 1e-9, step_max = 1e6)` returns an approximate minimum value of function `f` reached by hill climbing an initial guess `(x0, y0)`. If either optional pointer `best_x` or `best_y` is supplied, the corresponding coordinate of the point attaining the returned value is stored through it. The search starts with step size `step_max` and stops below `step_min`.

**Time Complexity:**  $O(k + s)$  calls to `f()` per call, where  $k$  is the number of times the step size is reduced and  $s$  is the total number of improving moves accepted.

**Space Complexity:**  $O(1)$  auxiliary.

```
template<typename Fn>
double hill_climb_min(
    Fn f, double x0, double y0, double *best_x = nullptr, double *best_y = nullptr,
    double step_min = 1e-9, double step_max = 1e6
) {
    static const double inv_sqrt2 = 0.7071067811865476;
    static const double dx[] = {1, inv_sqrt2, 0, -inv_sqrt2, -1, -inv_sqrt2, 0, inv_sqrt2};
    static const double dy[] = {0, inv_sqrt2, 1, inv_sqrt2, 0, -inv_sqrt2, -1, -inv_sqrt2};
    double x = x0, y = y0, res = f(x0, y0);
    for (double step = step_max; step > step_min;) {
        double next_value = res, next_x = x, next_y = y;
        bool found = false;
        for (int i = 0; i < 8; i++) {
            double x2 = x + step * dx[i], y2 = y + step * dy[i];
            double value = f(x2, y2);
            if (value < next_value) {
                next_x = x2;
                next_y = y2;
                next_value = value;
                found = true;
            }
        }
        if (!found) {
            step /= 2.0;
        } else {
            x = next_x;
            y = next_y;
            res = next_value;
        }
    }
    if (best_x != nullptr) {
        *best_x = x;
    }
    if (best_y != nullptr) {
        *best_y = y;
    }
    return res;
}
```

#### Example Usage

```
#include <cassert>
#include <cmath>
```

```

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

// Paraboloid with global minimum at f(2, 3) = 0.
double f(double x, double y) {
    return (x - 2) * (x - 2) + (y - 3) * (y - 3);
}

int main() {
    double x, y;
    assert(EQ(hill_climb_min(f, 0, 0, &x, &y), 0) && EQ(x, 2) && EQ(y, 3));
    return 0;
}

```

### 5.2.3 Simulated Annealing

Approximately minimizes an energy function  $E(s)$  over an arbitrary state space using simulated annealing. This randomized heuristic is useful when the states may be discrete, the energy has many local minima, and gradients or stronger structural properties are unavailable.

Starting from an initial state  $s$ , each iteration generates a random neighboring state  $s'$ . An improving move is always accepted, while a worsening move is accepted with probability  $\exp(-(E(s') - E(s))/T)$  at temperature  $T$ . High temperatures encourage exploration across energy barriers; as the temperature decreases, the search increasingly favors improvements. The best state is tracked separately because the current state may later move to a worse one.

The geometric schedule multiplies the temperature by `cooling_rate` each round, performing  $k = \lceil \log(T_{\text{end}}/T_{\text{start}}) / \log(r) \rceil$  iterations. Temperature has the same scale as energy, so scaling every energy requires scaling both temperature bounds by the same factor to preserve acceptance probabilities.

- `anneal_min(initial, energy, rand_neighbor, rng, ...)` returns `(best_energy, best_state)` given an initial state `initial`, a callable `energy(state)` that returns the state's energy, and a callable `rand_neighbor(state, temp, rng)` that returns a randomly chosen nearby state. Optional parameters default to `temp_start = 1000`, `temp_end = 1e-6`, `cooling_rate = 0.995`, and `time_limit = 0`. `temp_end` must be positive and less than `temp_start`, while `cooling_rate` must be strictly between 0 and 1. A positive `time_limit`, in seconds, stops the search once that much wall-clock time has elapsed; the best state found so far is returned either way.

For continuous states, temperature can also control the neighbor step size. Slower cooling and independent restarts often improve results, but simulated annealing never proves optimality.

This generic interface copies each proposed state and recomputes its energy. For large permutation states, adapt the loop to mutate and revert moves in place and update the energy from the affected terms instead.

**Time Complexity:**  $O(k(C_E + C_N))$  per call, where  $k$  is the number of temperature levels and  $C_E$  and  $C_N$  are the costs of evaluating the energy and generating a neighboring state. A positive `time_limit` caps  $k$  at whatever the deadline allows.

**Space Complexity:**  $O(S)$  for the current, next, and best states, where  $S$  is the size of one state.

```

#include <cassert>
#include <chrono>
#include <cmath>
#include <random>
#include <utility>

template<typename State, typename Energy, typename Neighbor>
std::pair<double, State> anneal_min(
    State initial, Energy energy, Neighbor rand_neighbor, std::mt19937 &rng,
    double temp_start = 1000, double temp_end = 1e-6, double cooling_rate = 0.995,

```



```

    double time_limit = 0
) {
    assert(0 < temp_end && temp_end < temp_start);
    assert(0 < cooling_rate && cooling_rate < 1);
    std::uniform_real_distribution<double> unit(0.0, 1.0);
    State current = std::move(initial), best = current;
    double curr_energy = energy(current), best_energy = curr_energy;
    auto deadline = std::chrono::steady_clock::now() + std::chrono::duration<double>(time_limit);
    for (double temp = temp_start; temp > temp_end; temp *= cooling_rate) {
        if (time_limit > 0 && std::chrono::steady_clock::now() >= deadline) {
            break; // The schedule is abandoned mid-cooling; the best state so far is still returned.
        }
        State next = rand_neighbor(current, temp, rng);
        double next_energy = energy(next);
        if (next_energy <= curr_energy || unit(rng) < std::exp((curr_energy - next_energy) / temp)) {
            current = std::move(next);
            curr_energy = next_energy;
            if (curr_energy < best_energy) {
                best = current;
                best_energy = curr_energy;
            }
        }
    }
    return {best_energy, std::move(best)};
}

```

## Example Usage

```

#include <algorithm>
#include <vector>
using namespace std;

int main() {
    mt19937 rng(1234567); // Fixed seed for reproducibility.

    // Escape the local minimum near x = 1 and find the lower minimum near x = -1.
    auto energy_1d = [](double x) { return (x * x - 1) * (x * x - 1) + 0.2 * x; };
    auto neighbor_1d = [](double x, double temp, mt19937 &gen) {
        uniform_real_distribution<double> move(-1.0, 1.0);
        return x + move(gen) * sqrt(temp);
    };
    auto [best_energy, best_x] = anneal_min(1.0, energy_1d, neighbor_1d, rng, 2.0, 1e-8);
    assert(best_energy < -0.2 && best_x < -1);

    // Approximate TSP with a permutation state and random 2-opt reversals.
    vector<pair<double, double>> points{{0, 0}, {1, 0}, {2, 0}, {2, 1},
                                         {2, 2}, {1, 2}, {0, 2}, {0, 1}};
    auto tour_length = [&](const vector<int> &tour) {
        double length = 0;
        for (int i = 0; i < static_cast<int>(tour.size()); i++) {
            auto [x1, y1] = points[tour[i]];
            auto [x2, y2] = points[tour[(i + 1) % tour.size()]];
            length += hypot(x1 - x2, y1 - y2);
        }
        return length;
    };
    auto reverse_segment = [](vector<int> tour, double, mt19937 &gen) {
        uniform_int_distribution<int> pick(0, static_cast<int>(tour.size()) - 1);
        int lo = pick(gen), hi = pick(gen);
        if (lo > hi) {
            swap(lo, hi);
        }
        reverse(tour.begin() + lo, tour.begin() + hi + 1);
        return tour;
    };
    vector<int> initial_tour{0, 4, 2, 6, 1, 5, 3, 7};
}

```

```

auto [best_length, best_tour] =
    anneal_min(initial_tour, tour_length, reverse_segment, rng, 5.0, 1e-6, 0.995);
assert(fabs(best_length - 8) < 1e-9);
assert(fabs(tour_length(best_tour) - best_length) < 1e-9);
return 0;
}

```

## 5.2.4 Gradient Descent

Given a differentiable objective  $f$  over  $\mathbb{R}^n$  and a starting guess, approximately minimize the function by repeatedly stepping against the gradient. The gradient points in the direction of steepest increase, so moving the opposite way lowers  $f$  for a small enough step; the whole difficulty is choosing how far to move, since too large a step overshoots and diverges while too small a step never arrives.

Backtracking answers this by trying a full step and halving it until the Armijo condition  $f(x - t\nabla f) \leq f(x) - ct \|\nabla f\|^2$  holds, which accepts a step only when it delivers a fixed fraction of the decrease that the gradient predicts. It needs no tuning and is the default to reach for. Adam instead keeps exponentially decaying averages of the gradient and of its squared entries, then divides one by the square root of the other, so each coordinate gets a step scaled to its own recent gradient magnitude. That makes it far less sensitive to badly scaled variables and to noisy gradients, at the cost of a learning rate to choose.

- `numeric_gradient(f, x, h = 1e-6)` returns the gradient of `f` at `x` estimated by central differences, using a step scaled by the magnitude of each coordinate.
- `gradient_descent_min(f, grad, x0, ...)` returns `(value, point)` for an approximate minimum of `f` reached from the initial guess `x0`, taking backtracking steps against `grad(x)`. Optional parameters default to `eps = 1e-9`, `iterations = 10000`, and `step_max = 1`, where `eps` is the gradient norm below which the search stops and `step_max` is the first step length tried.
- `adam_min(f, grad, x0, ...)` returns `(value, point)` for an approximate minimum of `f` reached from `x0` using Adam. Optional parameters default to `rate = 0.01`, `iterations = 10000`, `eps = 1e-9`, `beta1 = 0.9`, and `beta2 = 0.999`, where `rate` is the learning rate and the two decay factors, both required to lie in  $[0, 1)$ , control the two running averages.

Both routines find a local minimum, and only a convex  $f$  makes that the global one, as with the logistic regression of section 6.7.5. Prefer the golden-section search of section 5.2.1 in one dimension, or the hill climbing of section 5.2.2 and annealing of section 5.2.3 when no gradient is available, since a numeric gradient costs  $2n$  evaluations per step and half the usable precision.

### Time Complexity:

- $O(n)$  calls to `f()` per call to `numeric_gradient()`, where  $n$  is the number of variables.
- $O(k(C_g + l \cdot C_f))$  per call to `gradient_descent_min()`, where  $k$  is the number of iterations performed,  $l$  is the number of trial steps per line search, and  $C_f$  and  $C_g$  are the costs of evaluating `f` and `grad`.
- $O(k(C_g + n))$  per call to `adam_min()`, where  $k$  is the number of iterations performed.

**Space Complexity:**  $O(n)$  auxiliary for all operations.

```

#include <cassert>
#include <cmath>
#include <cstdint>
#include <utility>
#include <vector>

template<typename Fn>
std::vector<double> numeric_gradient(Fn f, const std::vector<double> &x, double h = 1e-6) {
    std::vector<double> res(x.size(), p(x);
    for (size_t i = 0; i < x.size(); i++) {
        double step = h * (1 + std::fabs(x[i]));
        p[i] = x[i] + step;
        double hi = f(p);
        p[i] = x[i] - step;
    }
}

```

```

    double lo = f(p);
    p[i] = x[i];
    res[i] = (hi - lo) / (2 * step);
}
return res;
}

template<typename Fn, typename Grad>
std::pair<double, std::vector<double>> gradient_descent_min(
    Fn f, Grad grad, std::vector<double> x, double eps = 1e-9, int iterations = 10000,
    double step_max = 1
) {
    static const double ARMIJO = 1e-4, STEP_MIN = 1e-18;
    double res = f(x);
    std::vector<double> next(x.size());
    for (int it = 0; it < iterations; it++) {
        std::vector<double> g = grad(x);
        double norm2 = 0;
        for (double v : g) {
            norm2 += v * v;
        }
        if (std::sqrt(norm2) < eps) {
            break;
        }
        bool moved = false;
        for (double step = step_max; step > STEP_MIN; step /= 2) {
            for (size_t i = 0; i < x.size(); i++) {
                next[i] = x[i] - step * g[i];
            }
            double value = f(next);
            if (value <= res - ARMIJO * step * norm2) {
                x = next;
                res = value;
                moved = true;
                break;
            }
        }
        if (!moved) {
            break;
        }
    }
    return std::make_pair(res, x);
}

template<typename Fn, typename Grad>
std::pair<double, std::vector<double>> adam_min(
    Fn f, Grad grad, std::vector<double> x, double rate = 0.01, int iterations = 10000,
    double eps = 1e-9, double beta1 = 0.9, double beta2 = 0.999
) {
    static const double EPS = 1e-8;
    assert(0 <= beta1 && beta1 < 1 && 0 <= beta2 && beta2 < 1);
    std::vector<double> m(x.size()), v(x.size());
    for (int t = 1; t <= iterations; t++) {
        std::vector<double> g = grad(x);
        double norm2 = 0;
        for (double d : g) {
            norm2 += d * d;
        }
        if (std::sqrt(norm2) < eps) {
            break;
        }
        double c1 = 1 - std::pow(beta1, t), c2 = 1 - std::pow(beta2, t);
        for (size_t i = 0; i < x.size(); i++) {
            m[i] = beta1 * m[i] + (1 - beta1) * g[i];
            v[i] = beta2 * v[i] + (1 - beta2) * g[i] * g[i];
            x[i] -= rate * (m[i] / c1) / (std::sqrt(v[i] / c2) + EPS);
        }
    }
}

```

```
    return std::make_pair(f(x), x);
}
```

## Example Usage

```
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-6;
}

// Ill-conditioned paraboloid with global minimum at f(2, 3) = 0.
double f(const vector<double> &x) {
    return 100 * (x[0] - 2) * (x[0] - 2) + (x[1] - 3) * (x[1] - 3);
}

vector<double> grad(const vector<double> &x) {
    return {200 * (x[0] - 2), 2 * (x[1] - 3)};
}

int main() {
    vector<double> x0{0, 0};
    vector<double> g = numeric_gradient(f, x0);
    assert(EQ(g[0], -400) && EQ(g[1], -6));

    pair<double, vector<double>> res = gradient_descent_min(f, grad, x0);
    assert(EQ(res.first, 0) && EQ(res.second[0], 2) && EQ(res.second[1], 3));

    res = adam_min(f, grad, x0, 0.05, 100000);
    assert(EQ(res.first, 0) && EQ(res.second[0], 2) && EQ(res.second[1], 3));

    // A gradient is not required: central differences suffice, at 2n evaluations per step.
    res = gradient_descent_min(f, [](const vector<double> &x) { return numeric_gradient(f, x); }, x0);
    assert(EQ(res.first, 0));
    return 0;
}
```

## 5.3 Dynamic Programming Tricks

### 5.3.1 Divide-and-Conquer DP Optimization

Computes one layer  $t$  of a dynamic program of the form  $dp_t(i) = \min_{0 \leq k \leq i} (dp_{t-1}(k) + C(k, i))$ , where  $C(k, i)$  is the cost of moving from state  $k$  of the previous layer to state  $i$  of the current one. Let  $opt(i)$  be the smallest  $k$  attaining this minimum. Divide-and-conquer optimization applies when  $opt(i) \leq opt(i + 1)$ , so the best transition index moves only to the right as  $i$  increases.

To compute  $dp_t$  over a range of states, evaluate its midpoint and record the best transition index  $k^*$  found there. Monotonicity restricts every optimum in the left half to indices at most  $k^*$ , and every optimum in the right half to indices at least  $k^*$ . Recursively applying these bounds avoids checking every transition for every state. The caller must verify the required monotonicity property.

- `compute_dp_layer(dp_prev, dp_cur, lo, hi, opt_lo, opt_hi, cost)` fills `dp_cur[lo..hi]` using candidate transition indices in `[opt_lo, opt_hi]`, where `dp_prev` holds  $dp_{t-1}$  and `dp_cur` receives  $dp_t$ . The template parameter `cost` must be callable such that `cost(k, i)` returns  $C(k, i)$ , the transition cost from previous state  $k$  to current state  $i$ .

**Time Complexity:**  $O(n \log n)$  calls to `cost()` per call, when computing one layer whose states each have  $O(n)$  possible transitions.

**Space Complexity:**  $O(\log n)$  auxiliary stack space.

```
#include <algorithm>
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;

template<typename Cost>
void compute_dp_layer(
    const std::vector<int64_t> &dp_prev, std::vector<int64_t> &dp_cur, int lo, int hi, int opt_lo,
    int opt_hi, const Cost &cost
) {
    if (lo > hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;
    int64_t best = INF;
    int best_k = opt_lo;
    int upper = std::min(mid, opt_hi);
    for (int k = opt_lo; k <= upper; k++) {
        int64_t candidate = dp_prev[k] + cost(k, mid); // Overflow warning.
        if (candidate < best) {
            best = candidate;
            best_k = k;
        }
    }
    dp_cur[mid] = best;
    compute_dp_layer(dp_prev, dp_cur, lo, mid - 1, opt_lo, best_k, cost);
    compute_dp_layer(dp_prev, dp_cur, mid + 1, hi, best_k, opt_hi, cost);
}
```

### Example Usage

```
#include <cassert>
using namespace std;

struct SquareSegmentCost {
    vector<int64_t> prefix;

    explicit SquareSegmentCost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    int64_t operator()(int k, int i) const {
        // Cost of placing the half-open segment a[k..i) in one group.
        int64_t sum = prefix[i] - prefix[k];
        return sum * sum;
    }
};

int main() {
    vector<int> a{1, 2, 3, 4};
    int n = static_cast<int>(a.size());
    SquareSegmentCost cost(a);

    // Base layer for zero groups: only the empty prefix is reachable.
    vector<int64_t> dp_prev(n + 1, INF), dp_cur(n + 1, INF);
    dp_prev[0] = 0;

    // The first layer places each prefix a[0..j) into one group.
    compute_dp_layer(dp_prev, dp_cur, 1, n, 0, n - 1, cost);
    assert(dp_cur[4] == 100); // One group: (1 + 2 + 3 + 4)^2.
}
```

```

// The second layer chooses the best split between two groups.
vector<int64_t> dp_two(n + 1, INF);
compute_dp_layer(dp_cur, dp_two, 1, n, 0, n - 1, cost);
assert(dp_two[4] == 52); // (1 + 2 + 3)^2 + 4^2.
return 0;
}

```

### 5.3.2 SMAWK Row Minima

Finds the position of the minimum in every row of a totally monotone matrix in time linear in its dimensions, without ever materializing the matrix. A matrix is totally monotone (for row minima) when the column index of each row's leftmost minimum never decreases as the row index increases, and this holds not just for the whole matrix but for every submatrix. Monge matrices, where  $M_{i,j} + M_{i+1,j+1} \leq M_{i,j+1} + M_{i+1,j}$ , are the most common source of such matrices and arise throughout dynamic programming on intervals and partitions.

The SMAWK algorithm interleaves two steps. REDUCE discards columns that cannot hold the minimum of any remaining row, leaving at most as many candidate columns as active rows. INTERPOLATE recurses on the odd-indexed active rows, then recovers each even-indexed active row's minimum by scanning only the candidate-column range bounded by its already-solved neighbors; total monotonicity guarantees those scans advance monotonically and cost  $O(R + C)$  overall, where  $R$  is the number of rows and  $C$  is the number of columns in the original matrix. This removes the logarithmic factor of divide-and-conquer DP optimization, but unlike that technique it is purely offline: every matrix entry must be available on demand from the oracle, so it cannot be used when an entry depends on a row minimum computed earlier in the same pass.

- `smawk_row_minima(rows, cols, get)` returns a vector `arg` of length `rows`, where `arg[r]` is the column index minimizing `get(r, c)` over  $0 \leq c < \text{cols}$  (on ties, the smallest qualifying column is chosen). The input matrix is supplied as a totally monotone function `get(r, c)` with bounds `rows`  $\geq 0$  and `cols`  $\geq 1$ .

**Time Complexity:**  $O(R + C)$  evaluations of `get` per call, where  $R$  is `rows` and  $C$  is `cols`.

**Space Complexity:**  $O(R + C)$  auxiliary.

```

#include <cassert>
#include <numeric>
#include <vector>

template<typename Get>
std::vector<int> smawk_rec(
    const Get &get, const std::vector<int> &active_rows, const std::vector<int> &candidate_cols
) {
    int num_rows = static_cast<int>(active_rows.size());
    std::vector<int> arg(num_rows, -1);
    if (num_rows == 0) {
        return arg;
    }
    // Base case: a single row or column is cheap to scan directly.
    if (num_rows <= 2 || candidate_cols.size() <= 2) {
        for (int row_pos = 0; row_pos < num_rows; row_pos++) {
            for (int c : candidate_cols) {
                if (arg[row_pos] == -1 ||
                    get(active_rows[row_pos], c) < get(active_rows[row_pos], arg[row_pos])) {
                    arg[row_pos] = c;
                }
            }
        }
        return arg;
    }
    // REDUCE: keep only columns that can be the minimum of some row.
    std::vector<int> kept;
    if (static_cast<int>(candidate_cols.size()) > num_rows) {
        for (int c : candidate_cols) {
            while (!kept.empty()) {

```

```

        int r = active_rows[static_cast<int>(kept.size()) - 1];
        if (get(r, kept.back()) <= get(r, c)) {
            break;
        }
        kept.pop_back();
    }
    if (static_cast<int>(kept.size()) < num_rows) {
        kept.push_back(c);
    }
}
} else {
    kept = candidate_cols;
}
// INTERPOLATE: solve the odd rows recursively, then fill the even rows between known bounds.
std::vector<int> odd_rows;
for (int row_pos = 1; row_pos < num_rows; row_pos += 2) {
    odd_rows.push_back(active_rows[row_pos]);
}
std::vector<int> odd_arg = smawk_rec(get, odd_rows, kept);
for (int odd_pos = 0; odd_pos < static_cast<int>(odd_arg.size()); odd_pos++) {
    arg[2 * odd_pos + 1] = odd_arg[odd_pos];
}
int col_pos = 0;
for (int row_pos = 0; row_pos < num_rows; row_pos += 2) {
    int hi = (row_pos + 1 < num_rows) ? arg[row_pos + 1] : kept.back();
    int r = active_rows[row_pos];
    while (true) {
        int c = kept[col_pos];
        if (arg[row_pos] == -1 || get(r, c) < get(r, arg[row_pos])) {
            arg[row_pos] = c;
        }
        if (c == hi) {
            break; // Stop at the upper bound; the next even row starts scanning from here.
        }
        col_pos++;
    }
}
return arg;
}

template<typename Get>
std::vector<int> smawk_row_minima(int rows, int cols, const Get &get) {
    assert(rows >= 0 && cols >= 1);
    std::vector<int> row_ids(rows), col_ids(cols);
    std::iota(row_ids.begin(), row_ids.end(), 0);
    std::iota(col_ids.begin(), col_ids.end(), 0);
    return smawk_rec(get, row_ids, col_ids);
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    // An explicit totally monotone matrix; row minima sit at non-decreasing columns.
    vector<vector<int>> m{
        {10, 17, 13, 28, 23},
        {17, 22, 16, 29, 23},
        {24, 28, 22, 34, 24},
        {11, 13, 6, 17, 7},
        {45, 44, 32, 37, 23},
    };
    auto get = [&](int r, int c) { return m[r][c]; };
    assert((smawk_row_minima(5, 5, get) == vector<int>{0, 2, 2, 2, 4}));
}

```

```

    return 0;
}

```

### 5.3.3 Knuth DP Optimization

Computes minimum costs for every half-open interval  $[l, r)$  using the recurrence

$dp(l, r) = \min_{l < k < r} (dp(l, k) + dp(k, r)) + C(l, r)$ . Each state chooses a split  $k$  strictly inside the interval and pays  $C(l, r)$  after solving the two resulting subintervals.

A direct implementation checks every split and takes  $O(n^3)$ . Knuth optimization applies when the smallest optimal splits satisfy  $opt(l, r - 1) \leq opt(l, r) \leq opt(l + 1, r)$ . The two neighboring states therefore bound the splits that must be checked for  $dp(l, r)$ , reducing the total number of candidate checks to  $O(n^2)$ . Common applications include optimal binary search trees and some range merging problems. The caller must verify the required monotonicity property for the chosen  $C(l, r)$ . The common sufficient conditions are interval monotonicity and the quadrangle inequality  $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$  for  $a \leq b \leq c \leq d$ , which is the Monge condition of section 5.3.2 written for an interval cost.

- `knuth_interval_dp(n, cost, &opt_out)` computes minimum costs for all half-open intervals  $[l, r)$  over `n` items. The template parameter `cost` must be callable such that `cost(l, r)` returns  $C(l, r)$ , the interval cost added after choosing the best split. If the optional pointer `opt_out` is supplied, it is filled with the chosen split points.

**Time Complexity:**  $O(n^2)$  calls to `cost()` and  $O(n^2)$  candidate split checks per call.

**Space Complexity:**  $O(n^2)$  for the `dp` and `opt` tables.

```

#include <algorithm>
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;

template<typename Cost>
std::vector<std::vector<int64_t>> knuth_interval_dp(
    int n, Cost cost, std::vector<std::vector<int>>> *opt_out = nullptr
) {
    std::vector<std::vector<int64_t>> dp(n + 1, std::vector<int64_t>(n + 1));
    std::vector<std::vector<int>> opt(n + 1, std::vector<int>(n + 1));
    // Empty and one-item intervals have cost 0.
    for (int i = 0; i <= n; i++) {
        opt[i][i] = i;
        if (i < n) {
            opt[i][i + 1] = i + 1;
        }
    }
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len;
            dp[l][r] = INF;
            int split_lo = std::max(opt[l][r - 1], l + 1);
            int split_hi = std::min(opt[l + 1][r], r - 1);
            int64_t interval_cost = cost(l, r);
            for (int k = split_lo; k <= split_hi; k++) {
                int64_t candidate = dp[l][k] + dp[k][r] + interval_cost; // Overflow warning.
                if (candidate < dp[l][r]) {
                    dp[l][r] = candidate;
                    opt[l][r] = k;
                }
            }
        }
    }
}

if (opt_out != nullptr) {

```



```

    *opt_out = opt;
}
return dp;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

struct MergeCost {
    vector<int64_t> prefix;

    explicit MergeCost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    int64_t operator()(int l, int r) const {
        // Cost of merging the half-open interval a[l..r) into one group.
        return prefix[r] - prefix[l];
    }
};

int main() {
    vector<int> a{1, 2, 3, 4};
    vector<vector<int>> opt;
    vector<vector<int64_t>> dp = knuth_interval_dp(a.size(), MergeCost(a), &opt);
    assert(dp[0][4] == 19); // Merge 1 + 2, then 3 + 3, then 6 + 4.
    assert(opt[0][4] == 3); // The final merge splits [1, 2, 3] from [4].
    return 0;
}

```

### 5.3.4 Lagrangian Relaxation (Aliens Trick)

Given an optimization problem with a constraint on the number of chosen objects, Lagrangian relaxation moves that count into the objective by charging a penalty per object. If the relaxed problem can be solved quickly and the number chosen is monotone as the penalty changes, binary search recovers the best value for the desired count. This technique is often called the Aliens trick. Widen the penalty range when in doubt.

For maximization, solve the relaxed problem as `score - penalty*count`, breaking ties toward larger `count`. As `penalty` increases, the optimal count never increases. The largest penalty whose relaxed optimum still chooses at least `target_count` objects gives the exact constrained score after adding back `penalty*target_count`.

- `lagrangian_maximize(target_count, lo, hi, solve)` returns the maximum original score using exactly `target_count` objects. The callable `solve(penalty)` must return `relaxed_score` and `count` as a pair and must tie-break toward larger `count`. Penalties are searched over the half-open range `[lo, hi)`; `lo` must choose at least `target_count` objects, while `hi` must be greater than the last penalty that does so.

Overflow warning: The difference `hi - lo` must fit in `int64_t`.

**Time Complexity:**  $O(\log n)$  calls to `solve()` per call, where  $n$  is the distance between `lo` and `hi`.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

```

```

template<typename Solve>
int64_t lagrangian_maximize(int target_count, int64_t lo, int64_t hi, Solve solve) {
    // The chosen count is nonincreasing, so find the last penalty that still chooses enough objects.
    while (lo < hi) {
        int64_t mid = lo + (hi - lo) / 2;
        if (solve(mid).second >= target_count) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    int64_t penalty = lo - 1;
    auto [relaxed_score, count] = solve(penalty);
    assert(count >= target_count);
    return relaxed_score + penalty * target_count; // Overflow warning.
}

```

### Example Usage

```

using namespace std;

int main() {
    vector<int64_t> value{10, 7, 3, 2};
    auto choose_profitable = [&](int64_t penalty) {
        int64_t score = 0;
        int count = 0;
        for (int64_t x : value) {
            int64_t gain = x - penalty;
            if (gain >= 0) { // Include zero gains to break ties toward larger counts.
                score += gain;
                count++;
            }
        }
        return pair<int64_t, int>{score, count};
    };

    assert(lagrangian_maximize(2, 0, 20, choose_profitable) == 17);
    assert(lagrangian_maximize(3, 0, 20, choose_profitable) == 20);
    return 0;
}

```

### 5.3.5 Convex Hull Trick (Semi-Dynamic)

Given a set of pairs  $(m, b)$  specifying lines of the form  $y = mx + b$ , answer queries at specified  $x$ -coordinates, each asking for the minimum  $y$ -value over all given lines. This is useful for dynamic programming recurrences of the form  $dp(i) = \min_j (m_j x_i + b_j)$ . Only the lower envelope of the lines can ever answer a query, so each added line pops previously stored lines that it renders useless, and ascending queries advance a pointer along the envelope so every line is visited at most twice.

The following implementation is a concise, semi-dynamic version of the convex hull optimization technique. It supports an interlaced sequence of `add_line()` and `query()` calls, as long as the preconditions of descending `m` and ascending `x` are satisfied. As a result, it may be necessary to sort the lines and queries before calling the functions. In that case, the overall time complexity will be dominated by the sorting step.

- `SemiDynamicCHT()` constructs an empty hull.
- `add_line(m, b)` inserts the line  $y = mx + b$ . The caller must ensure that slope `m` is less than or equal to the slope of every line added so far.
- `query(x)` returns the minimum  $y$ -value among all inserted lines at coordinate `x`. The caller must ensure that query coordinates are nondecreasing across calls. At least one line must have been inserted.

Overflow warning: `add_line()` compares intersections by cross-multiplying slope and intercept differences, a product on the order of the squared coefficient magnitude. For large `m/b` (roughly beyond  $10^9$  with `int64_t`), cast that comparison to `__int128`. `query()` evaluates `m * x + b`, which also needs to fit in `int64_t`.

**Time Complexity:**  $O(n + q)$  for any interlaced sequence containing  $n$  calls to `add_line()` and  $q$  calls to `query()`. Each line is removed or passed by the query pointer at most once, so both operations take amortized  $O(1)$  time.

**Space Complexity:**

- $O(n)$  for storage of the lines.
- $O(1)$  auxiliary for `add_line()` and `query()`.

```
#include <cassert>
#include <cstdint>
#include <vector>

class SemiDynamicCHT {
    std::vector<int64_t> vm, vb;
    int ptr = 0;

public:
    void add_line(int64_t m, int64_t b) {
        int len = static_cast<int>(vm.size());
        if (len > 0 && vm.back() == m) {
            if (vb.back() <= b) {
                return;
            }
            vm.pop_back();
            vb.pop_back();
            len--;
            if (ptr > len) {
                ptr = len;
            }
        }
        while (len > 1 && (vb[len - 2] - vb[len - 1]) * (m - vm[len - 1]) >=
                    (vb[len - 1] - b) * (vm[len - 1] - vm[len - 2])) { // Overflow warning.
            len--;
        }
        vm.resize(len);
        vb.resize(len);
        vm.push_back(m);
        vb.push_back(b);
    }

    int64_t query(int64_t x) {
        assert(!vm.empty());
        if (ptr >= static_cast<int>(vm.size())) {
            ptr = static_cast<int>(vm.size()) - 1;
        }
        while (ptr + 1 < static_cast<int>(vm.size()) &&
                vm[ptr + 1] * x + vb[ptr + 1] <= vm[ptr] * x + vb[ptr]) { // Overflow warning.
            ptr++;
        }
        return vm[ptr] * x + vb[ptr];
    }
};
```

### Example Usage

```
#include <cassert>

int main() {
    SemiDynamicCHT cht;
    cht.add_line(3, 0);
    cht.add_line(2, 1);
```

```

cht.add_line(1, 2);
cht.add_line(0, 6);
assert(cht.query(0) == 0);
assert(cht.query(1) == 3);
assert(cht.query(2) == 4);
assert(cht.query(3) == 5);
cht.add_line(0, 7); // Same slope, worse intercept; ignored.
assert(cht.query(4) == 6);
cht.add_line(0, 5); // Same slope, better intercept; replaces the old constant line.
assert(cht.query(5) == 5);
return 0;
}

```

### 5.3.6 Convex Hull Trick (Fully-Dynamic)

Maintains a dynamic set of lines  $y = mx + b$  and answers minimum value queries at integer points. Lines and queries may arrive in arbitrary order, making this useful for dynamic programming recurrences of the form  $dp(i) = \min_j(m_j x_i + b_j)$  without monotone slopes or query coordinates. The same interface can answer maximum queries when constructed with `query_max = true`.

This fully dynamic convex hull stores the envelope in a self-balancing binary search tree (`std::set`). Inserting a line removes any neighbors it dominates, and each remaining line records the last coordinate where it is optimal, so a query is one tree lookup. Unlike a Li Chao tree, this structure requires no fixed coordinate domain and its complexity depends on the number of lines rather than the domain width. Prefer it when the domain is unknown or very large, or when maximum queries are needed; a Li Chao tree is often simpler when a manageable domain is known.

- `FullyDynamicCht(query_max = false)` constructs an empty hull. By default, `query(x)` minimizes; if `query_max` is true, `query(x)` maximizes.
- `add_line(m, b)` inserts line  $y = mx + b$ . Lines may be added in any order.
- `query(x)` returns the best  $y$ -value among all inserted lines at coordinate  $x$ . At least one line must have been inserted, and query coordinates may be supplied in any order.

Lines cannot be removed. Each insertion erases every line it dominates, and restoring them is the dynamic convex hull problem rather than a local repair, since one deletion can revive arbitrarily many. When deletions are needed and the operation order is known in advance, the Li Chao tree of the next section rolls insertions back instead.

Overflow warning: minimization negates the coefficients, border updates subtract slopes/intercepts, and `query()` evaluates `m * x + b`, so those intermediate values must fit in `int64_t`.

**Time Complexity:**  $O(\log n)$  amortized per call to `add_line()` and  $O(\log n)$  per call to `query()`, where  $n$  is the number of lines added.

**Space Complexity:**

- $O(n)$  for storage of the lines.
- $O(1)$  auxiliary for `add_line()` and `query()`.

```

#include <cassert>
#include <cstdint>
#include <set>

class FullyDynamicCht {
    struct Line {
        int64_t m, b;
        mutable int64_t xhi;
        bool is_query;

        Line(int64_t m, int64_t b, int64_t xhi = 0, bool is_query = false)
            : m(m), b(b), xhi(xhi), is_query(is_query) {}
    };
};

```

```

    bool operator<(const Line &l) const { return l.is_query ? xhi < l.xhi : m < l.m; }
};

std::multiset<Line> hull;
bool query_max;

template<class It>
bool update_border(It x, It y) {
    if (y == hull.end()) {
        x->xhi = INT64_MAX;
        return false;
    }
    if (x->m == y->m) {
        x->xhi = (x->b > y->b) ? INT64_MAX : INT64_MIN;
    } else {
        int64_t a = y->b - x->b, b = x->m - y->m; // Overflow warning.
        x->xhi = a / b - ((a ^ b) < 0 && a % b);
    }
    return x->xhi >= y->xhi;
}

public:
explicit FullyDynamicCHT(bool query_max = false) : query_max(query_max) {}

void add_line(int64_t m, int64_t b) {
    if (!query_max) {
        m = -m;
        b = -b;
    }
    auto z = hull.insert(Line(m, b));
    auto y = z++;
    auto x = y;
    while (update_border(y, z)) {
        z = hull.erase(z);
    }
    if (x != hull.begin() && update_border(--x, y)) {
        update_border(x, y = hull.erase(y));
    }
    while ((y = x) != hull.begin() && (--x)->xhi >= y->xhi) {
        update_border(x, hull.erase(y));
    }
}

int64_t query(int64_t x) const {
    assert(!hull.empty());
    Line q(0, 0, x, true);
    auto it = hull.lower_bound(q);
    int64_t res = it->m * x + it->b; // Overflow warning.
    return query_max ? res : -res;
}
};

```

## Example Usage

```

#include <cassert>

int main() {
    FullyDynamicCHT h;
    h.add_line(3, 0);
    h.add_line(0, 6);
    h.add_line(1, 2);
    h.add_line(2, 1);
    // Minimize among y = 3x, 6, x + 2, and 2x + 1.
    assert(h.query(0) == 0);
    assert(h.query(2) == 4);
    assert(h.query(1) == 3);
}

```

```

assert(h.query(3) == 5);

FullyDynamicCHT mx(true);
mx.add_line(3, 0);
mx.add_line(0, 6);
mx.add_line(1, 2);
// Same interface can maximize when constructed with query_max = true.
assert(mx.query(0) == 6);
assert(mx.query(3) == 9);
return 0;
}

```

### 5.3.7 Li Chao Tree

Maintains a dynamic set of lines  $y = mx + b$  and answers minimum value queries at integer points. Lines and queries may arrive in arbitrary order, making this useful for dynamic programming recurrences of the form  $dp(i) = \min_j(m_j x_i + b_j)$  without monotone slopes or query coordinates.

A Li Chao tree fixes an inclusive coordinate domain  $[lo, hi]$  and stores one line at each node of a dynamic segment tree. Inserting a line keeps the line that wins at the midpoint and recurses only into the half where the other line may still win; a query takes the best value along one root-to-leaf path. Prefer this structure when the domain is known and manageable. The fully dynamic convex hull in the previous section needs no fixed domain and also supports maximum queries, while a Li Chao tree has a simpler invariant and is easier to extend to lines active only on subintervals.

No line can be removed once inserted, since a node records only the winner at its midpoint and never what that line beat. Insertions can instead be undone in reverse order: `add_line()` replaces the line at  $O(\log d)$  nodes and creates at most one, so remembering each previous line restores the tree exactly. Pairing that with a segment tree over the time axis gives arbitrary deletion offline, the construction that section 4.2.10 applies to a rollback disjoint set.

- `LiChaoTree(lo, hi)` constructs an empty tree over integer domain  $[lo, hi]$ .
- `add_line(m, b)` inserts line  $y = mx + b$ . Lines may be added in any order.
- `query(x)` returns the minimum  $y$ -value among all inserted lines at coordinate  $x$ . At least one line must have been inserted, and query coordinates may be supplied in any order.
- `snapshot()` returns a token representing the current history size.
- `rollback(snapshot)` undoes all insertions made after `snapshot`.

Overflow warning: each comparison and query evaluates  $m * x + b$ , which must fit in `int64_t`. The difference `hi - lo` must also fit in `int64_t`.

#### Time Complexity:

- $O(\log d)$  per call to `add_line()` and `query()`, where  $d$  is the distance between `lo` and `hi`.
- $O(1)$  per call to `snapshot()`, and  $O(\log d)$  per line undone by `rollback()`.

#### Space Complexity:

- $O(n)$  node storage for  $n$  lines inserted, since each insertion creates at most one node.
- $O(\log d)$  history per inserted line that has not been rolled back.
- $O(\log d)$  auxiliary stack space per operation.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <memory>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;

```

```

class LiChaoTree {
    struct Line {
        int64_t m, b;
        Line(int64_t m = 0, int64_t b = INF) : m(m), b(b) {}
        int64_t eval(int64_t x) const { return m * x + b; } // Overflow warning.
    };

    struct Node {
        Line f;
        std::unique_ptr<Node> left, right;
        explicit Node(const Line &f) : f(f) {}
    };

    // One insertion either replaces a node's line or creates the node, so recording the previous
    // state of each node it touches is enough to restore the tree exactly.
    struct Undo {
        Node *node; // Node whose line was replaced, or null if a node was created.
        Line prev; // The line that node held before.
        std::unique_ptr<Node> *slot; // Owner of the created node, when node is null.
        Undo(Node *node, const Line &prev, std::unique_ptr<Node> *slot)
            : node(node), prev(prev), slot(slot) {}
    };

    std::unique_ptr<Node> root;
    int64_t lo, hi;
    std::vector<Undo> trail;

    void add_line(std::unique_ptr<Node> &n, int64_t lo, int64_t hi, Line f) {
        if (n == nullptr) {
            n = std::make_unique<Node>(f);
            trail.emplace_back(nullptr, Line(), &n);
            return;
        }
        int64_t mid = lo + (hi - lo) / 2;
        bool left_better = f.eval(lo) < n->f.eval(lo);
        bool mid_better = f.eval(mid) < n->f.eval(mid);
        if (mid_better) {
            trail.emplace_back(n.get(), n->f, nullptr);
            std::swap(f, n->f);
        }
        if (lo == hi) {
            return;
        }
        if (left_better != mid_better) {
            add_line(n->left, lo, mid, f);
        } else {
            add_line(n->right, mid + 1, hi, f);
        }
    }

    static int64_t query(Node *n, int64_t lo, int64_t hi, int64_t x) {
        if (n == nullptr) {
            return INF;
        }
        int64_t res = n->f.eval(x);
        if (lo == hi) {
            return res;
        }
        int64_t mid = lo + (hi - lo) / 2;
        if (x <= mid) {
            return std::min(res, query(n->left.get(), lo, mid, x));
        }
        return std::min(res, query(n->right.get(), mid + 1, hi, x));
    }

public:
    LiChaoTree(int64_t lo, int64_t hi) : lo(lo), hi(hi) { assert(lo <= hi); }

```

```

LiChaoTree(const LiChaoTree &) = delete;
LiChaoTree &operator=(const LiChaoTree &) = delete;
void add_line(int64_t m, int64_t b) { add_line(root, lo, hi, Line(m, b)); }

int64_t query(int64_t x) const {
    assert(root != nullptr);
    assert(lo <= x && x <= hi);
    return query(root.get(), lo, hi, x);
}

int snapshot() const { return static_cast<int>(trail.size()); }

void rollback(int snapshot) {
    assert(0 <= snapshot && snapshot <= static_cast<int>(trail.size()));
    while (static_cast<int>(trail.size()) > snapshot) {
        const Undo &u = trail.back();
        if (u.slot != nullptr) {
            u.slot->reset();
        } else {
            u.node->f = u.prev;
        }
        trail.pop_back();
    }
}
};

```

## Example Usage

```

#include <cassert>

int main() {
    LiChaoTree h(-10, 10);
    h.add_line(3, 0);
    h.add_line(0, 6);
    h.add_line(1, 2);
    h.add_line(2, 1);
    // Minimize among y = 3x, 6, x + 2, and 2x + 1.
    assert(h.query(0) == 0);
    assert(h.query(2) == 4);
    assert(h.query(1) == 3);
    assert(h.query(3) == 5);

    // Insertions undo in reverse order, which is what makes offline deletion possible.
    int token = h.snapshot();
    h.add_line(-1, -9);
    assert(h.query(3) == -12);
    h.rollback(token);
    assert(h.query(3) == 5);
    return 0;
}

```

## 5.4 Numerical Methods

---

### 5.4.1 Root Finding (Bracketing)

Finds an  $x$  in an interval  $[a, b]$  for a continuous function  $f$  such that  $f(x) = 0$ . By the intermediate value theorem, a root must exist in  $[a, b]$  if either endpoint is a root or the signs of  $f(a)$  and  $f(b)$  differ. Each bisection step evaluates the midpoint of the interval and keeps the half whose endpoints still differ in sign, so a root always remains bracketed. After  $n$  iterations, the interval width is  $(b - a)/2^n$ . Although it is possible to control the error by looping while  $b - a$  is greater than an arbitrary epsilon, it is simpler to let the loop run for a desired number of



iterations until floating point arithmetic breaks down. 100 iterations is usually sufficient, since the search space will be reduced to  $2^{-100}$  (roughly  $10^{-30}$ ) times its original size.

- `bisection_root(f, a, b, iterations = 100)` returns a root in an interval  $[a, b]$  for a continuous function `f` where either endpoint is a root or the endpoint values have opposite signs, using the bisection method.
- `falsi_illinois_root(f, a, b, iterations = 100)` returns a root in an interval  $[a, b]$  for a continuous function `f` under the same endpoint conditions, using the Illinois algorithm variant of the false position (a.k.a. regula falsi) method.
- `brent_root(f, a, b, eps = 1e-15, iterations = 100)` returns a root under the same endpoint conditions using Brent's method, stopping early once the bracket is narrower than `eps`.

Brent's method is the one to reach for when the number of evaluations matters. It proposes the root of the inverse quadratic through its three most recent points, which converges superlinearly on smooth functions, and it falls back to secant steps when two of those values coincide. Any proposal that leaves the bracket, or that fails to at least halve the step taken two iterations earlier, is discarded in favor of a bisection step. That guard is what keeps the guaranteed bracketing of bisection while retaining the speed of interpolation, so Brent's method is the default root finder in most numerical libraries.

**Time Complexity:**  $O(n)$  calls to `f()` per call to `bisection_root()`, `falsi_illinois_root()`, and `brent_root()`, where  $n$  is the number of iterations. Brent's method usually returns after far fewer than the iteration limit, since it stops once the bracket is narrower than `eps`.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <cmath>

template<typename Fn>
double bisection_root(Fn f, double a, double b, int iterations = 100) {
    double fa = f(a), fb = f(b);
    assert(a <= b && ((fa <= 0 && fb >= 0) || (fa >= 0 && fb <= 0)));
    if (fa == 0) {
        return a;
    }
    if (fb == 0) {
        return b;
    }
    double m = a;
    for (int i = 0; i < iterations; i++) {
        m = a + (b - a) / 2;
        double fm = f(m);
        if (fm == 0) {
            return m;
        }
        if ((fa < 0) == (fm < 0)) {
            a = m;
            fa = fm;
        } else {
            b = m;
        }
    }
    return m;
}

template<typename Fn>
double falsi_illinois_root(Fn f, double a, double b, int iterations = 100) {
    double fa = f(a), fb = f(b);
    assert(a <= b && ((fa <= 0 && fb >= 0) || (fa >= 0 && fb <= 0)));
    if (fa == 0) {
        return a;
    }
    if (fb == 0) {
        return b;
    }
    double m = a;
```

```

int side = 0;
for (int i = 0; i < iterations; i++) {
    m = (fa * b - fb * a) / (fa - fb);
    double fm = f(m);
    if (fm == 0) {
        break;
    }
    if ((fb < 0) == (fm < 0)) {
        b = m;
        fb = fm;
        if (side < 0) {
            fa /= 2;
        }
        side = -1;
    } else {
        a = m;
        fa = fm;
        if (side > 0) {
            fb /= 2;
        }
        side = 1;
    }
}
return m;
}

template<typename Fn>
double brent_root(Fn f, double a, double b, double eps = 1e-15, int iterations = 100) {
    double fa = f(a), fb = f(b);
    assert(a <= b && ((fa <= 0 && fb >= 0) || (fa >= 0 && fb <= 0)));
    if (fa == 0) {
        return a;
    }
    if (fb == 0) {
        return b;
    }
    if (std::fabs(fa) < std::fabs(fb)) { // Keep b as the better of the two endpoints.
        std::swap(a, b);
        std::swap(fa, fb);
    }
    double c = a, fc = fa, d = c;
    bool bisection = true;
    for (int i = 0; i < iterations && fb != 0 && std::fabs(b - a) > eps; i++) {
        double s;
        if (fa != fc && fb != fc) {
            s = a * fb * fc / ((fa - fb) * (fa - fc)) + b * fa * fc / ((fb - fa) * (fb - fc)) +
                c * fa * fb / ((fc - fa) * (fc - fb));
        } else {
            s = b - fb * (b - a) / (fb - fa);
        }
        // Reject an interpolated point that leaves the bracket or converges too slowly, so that the
        // step count stays within a constant factor of plain bisection.
        double lo = (3 * a + b) / 4, hi = b;
        if (lo > hi) {
            std::swap(lo, hi);
        }
        double last = bisection ? std::fabs(b - c) : std::fabs(c - d);
        if (!(lo < s && s < hi) || std::fabs(s - b) >= last / 2 || last < eps) {
            s = a + (b - a) / 2;
            bisection = true;
        } else {
            bisection = false;
        }
        double fs = f(s);
        d = c;
        c = b;
        fc = fb;
        if ((fa < 0) == (fs < 0)) {

```

```

    a = s;
    fa = fs;
} else {
    b = s;
    fb = fs;
}
if (std::fabs(fa) < std::fabs(fb)) {
    std::swap(a, b);
    std::swap(fa, fb);
}
}
return b;
}

```

## Example Usage

```

#include <cmath>

double f(double x) {
    return x * x - 4 * sin(x);
}

int main() {
    assert(fabs(f(bisection_root(f, 1, 3))) < 1e-10);
    assert(fabs(f(falsi_illinois_root(f, 1, 3))) < 1e-10);
    assert(fabs(f(brent_root(f, 1, 3))) < 1e-10);
    assert(bisection_root(f, 0, 1) == 0);
    assert(falsi_illinois_root(f, 0, 1) == 0);
    assert(brent_root(f, 0, 1) == 0);

    // A function whose flat tail defeats plain false position but not the bisection fallback.
    auto flat = [](double x) { return x * x * x - 2 * x - 5; };
    assert(fabs(flat(brent_root(flat, 2, 100))) < 1e-9);
    return 0;
}

```

### 5.4.2 Root Finding (Iteration)

Finds an  $x$  for a continuous function  $f$  such that  $f(x) = 0$  using iterative approximation by an initial guess that is close to the answer. Each step follows the tangent line at the current guess down to its  $x$ -intercept, that is,  $x \leftarrow x - f(x)/f'(x)$ . Newton's method requires an explicit definition of the function's derivative while the secant method starts with two initial guesses and approximates the derivative using the secant slope from the previous iteration. With smooth functions and good initial guesses, Newton's method has quadratic convergence, while the secant method has convergence order approximately 1.618.

- `newton_root(f, fprime, x0, eps = 1e-15, iterations = 100)` returns a root  $x$  for a function `f` with derivative `fprime` using an initial guess `x0` which should be relatively close to  $x$ .
- `secant_root(f, x0, x1, eps = 1e-15, iterations = 100)` returns a root  $x$  for a function `f` using two initial guesses `x0` and `x1` which should be relatively close to  $x$ .

**Time Complexity:**  $O(n)$  calls to `f()` and `fprime()` per call to `newton_root()`, and  $O(n)$  calls to `f()` per call to `secant_root()`, where  $n$  is the number of iterations.

**Space Complexity:**  $O(1)$  auxiliary for both operations.

```

#include <cmath>
#include <stdexcept>

template<typename Fn, typename Deriv>
double newton_root(Fn f, Deriv fprime, double x0, double eps = 1e-15, int iterations = 100) {
    double x = x0, error = eps + 1;

```

```

for (int i = 0; std::isfinite(error) && error > eps && i < iterations; i++) {
    double fx = f(x);
    if (fx == 0) {
        return x;
    }
    double xnew = x - fx / fprime(x);
    error = fabs(xnew - x);
    x = xnew;
}
if (!std::isfinite(error) || error > eps) {
    throw std::runtime_error("Newton's method failed to converge.");
}
return x;
}

template<typename Fn>
double secant_root(Fn f, double x0, double x1, double eps = 1e-15, int iterations = 100) {
    double xold = x0, fxold = f(x0), x = x1, error = eps + 1;
    for (int i = 0; std::isfinite(error) && error > eps && i < iterations; i++) {
        double fx = f(x);
        if (fx == 0) {
            return x;
        }
        double xnew = x - fx * ((x - xold) / (fx - fxold));
        error = fabs(xnew - x);
        xold = x;
        fxold = fx;
        x = xnew;
    }
    if (!std::isfinite(error) || error > eps) {
        throw std::runtime_error("Secant method failed to converge.");
    }
    return x;
}

```

## Example Usage

```

#include <cassert>

double f(double x) {
    return x * x - 4 * sin(x);
}

double fprime(double x) {
    return 2 * x - 4 * cos(x);
}

int main() {
    assert(fabs(f(newton_root(f, fprime, 3))) < 1e-10);
    assert(fabs(f(secant_root(f, 3, 2))) < 1e-10);
    auto g = [](double x) { return x * x - 2; };
    auto gprime = [](double x) { return 2 * x; };
    assert(fabs(g(newton_root(g, gprime, 1))) < 1e-10);
    return 0;
}

```

### 5.4.3 Polynomial Root Finding (Differentiation)

Finds every real root  $x$  of a polynomial  $p$  satisfying  $p(x) = 0$  by recursively finding the roots of its derivative. Each adjacent pair of local extrema is searched using the bisection method.

The key observation is that derivative roots split the real line into intervals where the polynomial is monotone. A monotone interval contains at most one root, and if an endpoint is a root or the endpoint values have opposite

signs, bisection isolates that root. Recursively solving the derivative therefore provides all interval boundaries needed to find every real root in the requested range, including roots of even multiplicity. Each recursive call first scales the coefficients by their largest magnitude so that root detection is unaffected by a common factor.

- `horner_eval(p, x)` evaluates the polynomial `p` of degree  $d$  (represented as a vector of size  $d + 1$  where `p[i]` stores the degree-`i` coefficient) at `x`, using Horner's method.
- `find_one_root(p, a, b, eps = 1e-15)` returns a root in the interval `[a, b]` for a polynomial `p` where either endpoint is a root or the endpoint values have opposite signs, using the bisection method. If this precondition is not satisfied, then `NaN` is returned. The root is found to a tolerance of `eps` in absolute or relative error (whichever is reached first).
- `find_all_roots(p, a = -1e20, b = 1e20, eps = 1e-15)` returns a vector of all roots in the interval `[a, b]` for a polynomial `p` using the bisection method. The roots are found to a tolerance of `eps` in absolute or relative error (whichever is reached first).

The coefficient vector must be nonempty and contain at least one nonzero coefficient.

#### Time Complexity:

- $O(n)$  per call to `horner_eval()`, where  $n$  is the degree of the polynomial.
- $O(nt)$  per call to `find_one_root()`, where  $n$  is the degree of the polynomial and  $t$  is the number of bisection iterations required to reach the requested tolerance.
- $O(n^3t)$  per call to `find_all_roots()`, where  $n$  is the degree of the polynomial and  $t$  is the number of bisection iterations required to reach the requested tolerance.

#### Space Complexity:

- $O(1)$  auxiliary for `horner_eval()` and `find_one_root()`.
- $O(n)$  auxiliary stack space and  $O(n^2)$  auxiliary heap space for `find_all_roots()`, where  $n$  is the degree of the polynomial.

```
#include <algorithm>
#include <cmath>
#include <limits>
#include <vector>

using dbl = long double;

dbl horner_eval(const std::vector<dbl> &p, dbl x) {
    dbl res = p.back();
    for (int i = static_cast<int>(p.size()) - 2; i >= 0; i--) {
        res = res * x + p[i];
    }
    return res;
}

dbl find_one_root(const std::vector<dbl> &p, dbl a, dbl b, dbl eps = 1e-15L) {
    dbl scale = 0;
    for (dbl c : p) {
        scale = std::max(scale, std::fabs(c));
    }
    dbl pa = horner_eval(p, a), pb = horner_eval(p, b);
    if (std::fabs(pa) <= eps * scale) {
        return a;
    }
    if (std::fabs(pb) <= eps * scale) {
        return b;
    }
    bool paneg = pa < 0, pbneg = pb < 0;
    if (paneg == pbneg) {
        return std::numeric_limits<dbl>::quiet_NaN();
    }
    while (b - a > eps && a * (1 + eps) < b && a < b * (1 + eps)) {
        dbl m = a + (b - a) / 2;
        if ((horner_eval(p, m) < 0) == paneg) {
            a = m;
        }
    }
}
```

```

    } else {
        b = m;
    }
}
return a;
}

std::vector<dbl> find_all_roots(
    std::vector<dbl> p, dbl a = -1e20L, dbl b = 1e20L, dbl eps = 1e-15L
) {
    while (p.size() > 1 && p.back() == 0) {
        p.pop_back();
    }
    if (p.size() <= 1) {
        return {};
    }
    dbl scale = 0;
    for (dbl c : p) {
        scale = std::max(scale, std::fabs(c));
    }
    for (dbl &c : p) {
        c /= scale;
    }
    std::vector<dbl> pprime;
    pprime.reserve(p.size() > 0 ? p.size() - 1 : 0);
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        pprime.push_back(p[i] * i);
    }
    std::vector<dbl> res, r = find_all_roots(pprime, a, b, eps);
    r.push_back(b);
    for (int i = 0; i < static_cast<int>(r.size()); i++) {
        dbl root = find_one_root(p, i == 0 ? a : r[i - 1], r[i], eps);
        dbl dedup_eps = eps * std::max(dbl{1}, std::fabs(root));
        if (!std::isnan(root) && (res.empty() || std::fabs(root - res.back()) > dedup_eps)) {
            res.push_back(root);
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    { // -1 + 2x - 6x^2 + 2x^3
        vector<dbl> p{-1, 2, -6, 2};
        vector<dbl> roots = find_all_roots(p);
        assert(roots.size() == 1 && fabs(horner_eval(p, roots[0])) < 1e-10L);
    }
    { // -20 + 4x + 3x^2
        vector<dbl> p{-20, 4, 3};
        vector<dbl> roots = find_all_roots(p);
        assert(roots.size() == 2);
        assert(fabs(horner_eval(p, roots[0])) < 1e-10L);
        assert(fabs(horner_eval(p, roots[1])) < 1e-10L);
    }
    { // (x - 1)^2
        vector<dbl> p{1, -2, 1};
        vector<dbl> roots = find_all_roots(p);
        assert(roots.size() == 1 && fabs(roots[0] - 1) < 1e-10L);
    }
    { // Scaling the coefficients does not duplicate or hide the repeated root (x - sqrt(2))^2.
        dbl r = sqrt(2.0L), scale = 1e24L;
        vector<dbl> p{2 * scale, -2 * r * scale, scale};
    }
}

```

```

vector<dbl> roots = find_all_roots(p, -10, 10);
assert(roots.size() == 1 && fabs(roots[0] - r) < 1e-10L);
assert(fabs(find_one_root(p, r, 10) - r) < 1e-10L);
}
{
    vector<dbl> roots = find_all_roots(vector<dbl>{-1, 1, 0}, -10, 10);
    assert(roots.size() == 1 && fabs(roots[0] - 1) < 1e-10L);
}
return 0;
}

```

#### 5.4.4 Polynomial Root Finding (Laguerre)

Finds every complex root  $x$  for a polynomial  $p$  with complex coefficients such that  $p(x) = 0$ . Laguerre's method is a root-finding iteration with reliable, near-cubic convergence from almost any starting point. Once a root is found it is removed by polynomial deflation, and the iteration repeats on the lower-degree quotient until every root has been extracted.

- `horner_eval(p, x)` evaluates a complex polynomial  $p$  of degree  $d$  (represented as a vector of size  $d + 1$  where `p[i]` stores the degree- $i$  coefficient) at  $x$ , using Horner's method, returning a pair where the first value is the final result  $p(x)$  and the second value is the quotient polynomial  $q$  from the identity  $p(t) = (t - x)q(t) + p(x)$ . The coefficient vector must be nonempty.
- `find_one_root(p, x0, eps = 1e-15, iterations = 10000)` returns a complex root  $x$  for polynomial  $p$  (represented as a vector of size  $d + 1$  where `p[i]` stores the degree- $i$  coefficient) using an initial guess `x0` which should be relatively close to  $x$ . The root is found to a tolerance of `eps` in absolute or relative error (whichever is reached first). The polynomial must have degree at least one and omit trailing zero coefficients; its coefficients are scaled by their largest magnitude so a common factor does not affect convergence.
- `find_all_roots(p, eps = 1e-15, iterations = 10000)` returns a vector of all complex roots for a complex polynomial  $p$ . The roots are found to a tolerance of `eps` in absolute or relative error (whichever is reached first). The coefficient vector must be nonempty and omit trailing zero coefficients.

##### Time Complexity:

- $O(n)$  per call to `horner_eval()`, where  $n$  is the degree of the polynomial.
- $O(nt)$  per call to `find_one_root()`, where  $n$  is the degree of the polynomial and  $t$  is the number of iterations performed.
- $O(n^2t)$  per call to `find_all_roots()`, where  $n$  is the degree of the polynomial and  $t$  is the number of iterations performed per root.

##### Space Complexity:

- $O(n)$  auxiliary for `horner_eval()` and `find_one_root()`, where  $n$  is the degree of the polynomial.
- $O(n)$  auxiliary for `find_all_roots()`, where  $n$  is the degree of the polynomial.

```

#include <algorithm>
#include <chrono>
#include <cmath>
#include <complex>
#include <random>
#include <utility>
#include <vector>

using dbl = long double;
using cdbl = std::complex<dbl>;
using cpoly = std::vector<cdbl>;

std::pair<cdbl, cpoly> horner_eval(const cpoly &p, const cdbl &x) {
    int n = static_cast<int>(p.size());
    cpoly b(std::max(1, n - 1));
    for (int i = n - 1; i > 0; i--) {

```

```

    b[i - 1] = p[i] + (i < n - 1 ? b[i] * x : 0);
}
return {p[0] + b[0] * x, b};
}

cpoly derivative(const cpoly &p) {
    int n = static_cast<int>(p.size());
    cpoly res(std::max(1, n - 1));
    for (int i = 1; i < n; i++) {
        res[i - 1] = p[i] * cdbl(i);
    }
    return res;
}

cdbl find_one_root(cpoly p, const cdbl &x0, dbl eps = 1e-15L, int iterations = 10000) {
    dbl scale = 0;
    for (const cdbl &c : p) {
        scale = std::max(scale, std::abs(c));
    }
    for (cdbl &c : p) {
        c /= scale;
    }
    cdbl x = x0;
    int n = static_cast<int>(p.size()) - 1;
    cpoly p1 = derivative(p), p2 = derivative(p1);
    for (int i = 0; i < iterations; i++) {
        cdbl y0 = horner_eval(p, x).first;
        if (std::abs(y0) <= eps) {
            break;
        }
        cdbl g = horner_eval(p1, x).first / y0;
        cdbl h = g * g - horner_eval(p2, x).first / y0;
        cdbl r = std::sqrt(cdbl(n - 1) * (h * cdbl(n) - g * g));
        cdbl d1 = g + r, d2 = g - r;
        cdbl a = cdbl(n) / (std::abs(d1) > std::abs(d2) ? d1 : d2);
        x -= a;
        if (std::abs(a) <= eps) {
            break;
        }
    }
    return x;
}

std::vector<cdbl> find_all_roots(const cpoly &p, dbl eps = 1e-15L, int iterations = 10000) {
    std::vector<cdbl> res;
    cpoly q = p;
    if (q.size() <= 1) {
        return res;
    }
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    std::uniform_real_distribution<dbl> unit(0.0L, 1.0L);
    while (q.size() > 2) {
        cdbl z(unit(rng), unit(rng));
        z = find_one_root(p, find_one_root(q, z, eps, iterations), eps, iterations);
        q = horner_eval(q, z).second;
        res.push_back(z);
    }
    res.push_back(-q[0] / q[1]);
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

```



```

void assert_roots(const vector<cdbl> &actual, const vector<cdbl> &expected) {
    assert(actual.size() == expected.size());
    vector<char> matched(actual.size());
    for (const cdbl &root : expected) {
        int found = -1;
        for (int i = 0; i < static_cast<int>(actual.size()); i++) {
            if (!matched[i] && abs(actual[i] - root) < 1e-8L) {
                found = i;
                break;
            }
        }
        assert(found != -1);
        matched[found] = true;
    }
}

int main() {
    { // 140 - 13x - 8x^2 + x^3 = (x + 4)(x - 5)(x - 7)
        cpoly p;
        p.push_back(140);
        p.push_back(-13);
        p.push_back(-8);
        p.push_back(1);
        assert_roots(find_all_roots(p), {-4, 5, 7});
        for (cdbl &c : p) {
            c *= 1e-30L;
        }
        assert_roots(find_all_roots(p), {-4, 5, 7});
    }
    { // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
      // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
        cpoly p;
        p.emplace_back(-24, 36);
        p.emplace_back(-26, 12);
        p.emplace_back(-30, 40);
        p.emplace_back(-26, 12);
        p.emplace_back(-6, 4);
        assert_roots(find_all_roots(p), {{-3, -2}, {-12.0L / 13, 18.0L / 13}, {0, -1}, {0, 1}});
    }
    return 0;
}

```

### 5.4.5 Polynomial Root Finding (Ehrlich-Aberth)

Finds every complex root  $x$  for a polynomial  $p$  such that  $p(x) = 0$ . The Ehrlich-Aberth method is a simultaneous iteration: every root estimate is updated using both the Newton correction and a repulsion term from the other estimates.

This routine is intended for well-scaled polynomials when all complex roots are wanted. If only real roots are needed, a real-root isolator such as derivative recursion with bisection is usually more reliable. Multiple or tightly clustered roots may converge slowly, and very large or very small root scales may require rescaling the input or using multiprecision arithmetic.

- `eval_with_derivative(p, x)` returns a pair  $(p(x), p'(x))$  for a polynomial  $p$  given as a vector `p` where `p[i]` stores the degree-`i` coefficient. The vector must be nonempty.
- `find_all_roots(p, eps = ROOT_EPS, iterations = 2000)` returns a vector of all complex roots for a complex polynomial given by the vector of coefficients `p`. A `vector<dbl>` overload is provided for polynomials with real coefficients. The roots are found to a tolerance of `eps` in absolute or relative error (whichever is reached first), and zero roots are removed exactly before the simultaneous iteration starts. The input must contain at least one nonzero coefficient; after normalization, trailing coefficients no larger than `ZERO_EPS` relative to the largest coefficient are discarded.

**Time Complexity:**

- $O(n)$  per call to `eval_with_derivative()`, where  $n$  is the degree of the polynomial.
- $O(n^2t)$  per call to `find_all_roots()`, where  $n$  is the degree of the polynomial and  $t$  is the number of iterations required to reach the desired precision.

**Space Complexity:**

- $O(1)$  auxiliary for `eval_with_derivative()`.
- $O(n)$  auxiliary for `find_all_roots()`, where  $n$  is the degree of the polynomial.

```
#include <algorithm>
#include <cmath>
#include <complex>
#include <utility>
#include <vector>

using dbl = long double;
using cdbl = std::complex<dbl>;
using cpoly = std::vector<cdbl>;

const dbl PI = std::acos(dbl{-1});
const dbl ZERO_EPS = 1e-30L; // Treat normalized coefficients and denominators this small as zero.
const dbl ROOT_EPS = 1e-15L; // Stop once root updates are this small relative to the root.
const dbl CHECK_EPS = 1e-12L; // Residual tolerance used by the example assertions.

bool is_zero(const cdbl &z) {
    return std::abs(z) <= ZERO_EPS;
}

bool is_finite(const cdbl &z) {
    return std::isfinite(z.real()) && std::isfinite(z.imag());
}

std::pair<cdbl, cdbl> eval_with_derivative(const cpoly &p, const cdbl &x) {
    cdbl value = p.back(), derivative = 0;
    for (int i = static_cast<int>(p.size()) - 2; i >= 0; i--) {
        derivative = derivative * x + value;
        value = value * x + p[i];
    }
    return {value, derivative};
}

dbl root_bound(const cpoly &p) {
    dbl res = 0;
    int n = static_cast<int>(p.size()) - 1;
    for (int i = 0; i < n; i++) {
        dbl ratio = std::abs(p[i] / p.back());
        if (ratio > 0) {
            res = std::max(res, std::pow(ratio, dbl{1} / (n - i)));
        }
    }
    return 2 * std::max(dbl{1}, res);
}

cpoly find_all_roots(cpoly p, dbl eps = ROOT_EPS, int iterations = 2000) {
    while (!p.empty() && p.back() == cdbl{0}) {
        p.pop_back();
    }
    if (p.size() <= 1) {
        return {};
    }
    dbl scale = 0;
    for (const cdbl &c : p) {
        scale = std::max(scale, std::abs(c));
    }
    for (cdbl &c : p) {
```

```

    c /= scale;
}
while (!p.empty() && is_zero(p.back())) {
    p.pop_back();
}
cpoly roots;
while (p.size() > 1 && is_zero(p[0])) {
    roots.push_back(0);
    p.erase(p.begin());
}
if (p.size() <= 1) {
    return roots;
}
int n = static_cast<int>(p.size()) - 1;
if (n == 1) {
    roots.push_back(-p[0] / p[1]);
    return roots;
}
cpoly z(n);
dbl radius = root_bound(p), offset = PI / (2 * n);
dbl max_radius = 2 * radius;
for (int i = 0; i < n; i++) {
    dbl angle = offset + 2 * PI * i / n;
    z[i] = radius * cdbl(std::cos(angle), std::sin(angle));
}
for (int it = 0; it < iterations; it++) {
    bool done = true;
    cpoly next = z;
    for (int i = 0; i < n; i++) {
        auto [fx, dfx] = eval_with_derivative(p, z[i]);
        if (std::abs(fx) <= eps) {
            continue;
        }
        cdbl repulsion = 0;
        for (int j = 0; j < n; j++) {
            if (i != j) {
                cdbl diff = z[i] - z[j];
                if (std::abs(diff) <= eps * eps) {
                    dbl angle = 2 * PI * (i + 1) / (n + 1);
                    diff = eps * (1 + std::abs(z[i])) * cdbl(std::cos(angle), std::sin(angle));
                }
                repulsion += cdbl{1} / diff;
            }
        }
        cdbl denom = dfx - fx * repulsion;
        if (is_zero(denom)) {
            done = false;
            continue;
        }
        cdbl step = fx / denom;
        if (!is_finite(step) && !is_zero(dfx)) {
            step = fx / dfx;
        }
        if (!is_finite(step)) {
            done = false;
            continue;
        }
        dbl limit = 2 * max_radius;
        if (std::abs(step) > limit) {
            step *= limit / std::abs(step);
        }
        next[i] = z[i] - step;
        if (!is_finite(next[i])) {
            next[i] = z[i];
            done = false;
            continue;
        }
        if (std::abs(next[i]) > max_radius) {

```

```

        next[i] *= max_radius / std::abs(next[i]);
    }
    if (std::abs(step) > eps * (1 + std::abs(next[i]))) {
        done = false;
    }
}
z = std::move(next);
if (done) {
    break;
}
}
for (int i = 0; i < n; i++) {
    roots.emplace_back(z[i]);
}
std::sort(roots.begin(), roots.end(), [](const cdbl &a, const cdbl &b) {
    return a.real() != b.real() ? a.real() < b.real() : a.imag() < b.imag();
});
return roots;
}

std::vector<cdbl> find_all_roots(
    const std::vector<dbl> &p, dbl eps = ROOT_EPS, int iterations = 2000
) {
    return find_all_roots(cpoly(p.begin(), p.end()), eps, iterations);
}

```

## Example Usage

```

#include <cassert>
using namespace std;

cdbl eval(const cpoly &p, const cdbl &x) {
    return eval_with_derivative(p, x).first;
}

void assert_roots(const vector<cdbl> &actual, const vector<cdbl> &expected) {
    assert(actual.size() == expected.size());
    vector<char> matched(actual.size());
    for (const cdbl &root : expected) {
        int found = -1;
        for (int i = 0; i < static_cast<int>(actual.size()); i++) {
            if (!matched[i] && abs(actual[i] - root) < CHECK_EPS) {
                found = i;
                break;
            }
        }
        assert(found != -1);
        matched[found] = true;
    }
}

int main() {
    { // -1 + 2x - 6x^2 + 2x^3
        vector<dbl> p{-1, 2, -6, 2};
        vector<cdbl> roots = find_all_roots(p);
        assert(roots.size() == 3);
        for (const auto &root : roots) {
            assert(abs(eval(cpoly(p.begin(), p.end()), root)) < CHECK_EPS);
        }
    }
    { // Scaling all coefficients does not change the roots.
        vector<dbl> p{-6e-40L, 1e-40L};
        assert_roots(find_all_roots(p), {6});
    }
    { // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
        // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
    }
}

```

```

cpoly p;
p.emplace_back(-24, 36);
p.emplace_back(-26, 12);
p.emplace_back(-30, 40);
p.emplace_back(-26, 12);
p.emplace_back(-6, 4);
vector<cdbl> roots = find_all_roots(p);
assert(roots.size() == 4);
for (const auto &root : roots) {
    assert(abs(eval(p, root)) < CHECK_EPS);
}
assert_roots(roots, {{-3, -2}, {-12.0L / 13, 18.0L / 13}, {0, -1}, {0, 1}});
}
return 0;
}

```

### 5.4.6 Integration (Simpson)

Computes the definite integral from  $a$  to  $b$  for a continuous function  $f$  using the composite Simpson's rule: the interval is split into  $n$  equal subintervals and each consecutive pair is approximated by a parabola, giving the approximation  $\int_a^b f(x) dx \approx h/3 \cdot [f(x_0) + 4f(x_1) + 2f(x_2) + \dots + 4f(x_{n-1}) + f(x_n)]$  with step  $h = (b - a)/n$ .

Unlike adaptive quadrature, the step size here is uniform, so a feature narrower than  $h$  may be under-resolved. The benefit is a simple, non-recursive routine with predictable cost that is not fooled by the premature-termination heuristic that adaptive Simpson's rule can suffer on functions whose coarse sample points happen to fit a parabola.

- `integrate(f, a, b, n = 1000000)` returns the definite integral of a function `f` from `a` to `b` using `n` subintervals, where `n` must be even. Larger `n` trades runtime for accuracy; the leading error scales with  $(b - a)h^4$  and the maximum magnitude of the fourth derivative of  $f$  over the interval.

**Time Complexity:**  $O(n)$  per call, requiring  $n + 1$  evaluations of  $f$ .

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cassert>

template<typename Fn>
double integrate(Fn f, double a, double b, int n = 1000000) {
    assert(n > 0 && n % 2 == 0);
    double h = (b - a) / n, s = f(a) + f(b);
    for (int i = 1; i < n; i++) {
        s += f(a + h * i) * (i & 1 ? 4 : 2);
    }
    return s * h / 3;
}

```

#### Example Usage

```

#include <cmath>
using namespace std;

double f(double x) {
    return sin(x);
}

int main() {
    const double PI = acos(-1.0);
    assert(fabs(integrate(f, 0.0, PI / 2) - 1) < 1e-10);
    assert(fabs(integrate(f, PI / 2, 0.0) + 1) < 1e-10);
    assert(fabs(integrate(f, 1.0, 1.0)) < 1e-12);
}

```

```
    return 0;
}
```

### 5.4.7 Ordinary Differential Equations

Given an initial value  $y(t_0) = y_0$  and a derivative  $y'(t) = f(t, y)$ , an initial value problem asks for  $y$  at some later time. Since only the slope is known, the solution is traced by stepping forward: estimate the slope over a short interval  $h$ , move along it, and repeat. The methods below differ only in how many slope samples each step averages, which is what determines how quickly the error vanishes as  $h$  shrinks.

The forward Euler method takes the slope at the current point and follows it for the whole step. Each step therefore misses the curvature of the solution, leaving an error per step proportional to  $h^2$  that accumulates to  $O(h)$  over a fixed interval. The midpoint method spends one extra evaluation to sample the slope halfway across the step and uses that instead, which cancels the leading error term and leaves  $O(h^2)$ . Classical Runge-Kutta combines four samples, one at the start, two at the middle, and one at the end, weighted so that every error term through  $h^4$  cancels, leaving  $O(h^4)$ . Halving  $h$  therefore buys 16 times the accuracy for twice the work, which is why this method is the usual default.

The state is a vector, so a system of equations is integrated exactly like a single one. A higher-order equation becomes such a system by naming its derivatives:  $y'' = g(t, y, y')$  turns into the pair  $y'_0 = y_1$  and  $y'_1 = g(t, y_0, y_1)$ .

- `euler(f, t0, y0, t1, steps)` returns the state at time `t1`, advancing from state `y0` at time `t0` in `steps` equal steps of the forward Euler method, where `steps` must be positive. The derivative `f(t, y)` returns a vector of the same length as `y`.
- `midpoint(f, t0, y0, t1, steps)` and `runge_kutta(f, t0, y0, t1, steps)` do the same using the midpoint method and classical fourth-order Runge-Kutta.

A `t1` below `t0` integrates backwards, since the step is then negative. None of these adapt their step, so doubling `steps` should shrink the change in the answer by roughly  $2^p$  for a method of order  $p$ ; when it does not, the problem is stiff and needs an implicit method.

**Time Complexity:**  $O(s)$  calls to `f()` per call to `euler()`,  $O(2s)$  per call to `midpoint()`, and  $O(4s)$  per call to `runge_kutta()`, where  $s$  is `steps`.

**Space Complexity:**  $O(n)$  auxiliary and  $O(n)$  for the returned state, where  $n$  is the number of equations.

```
#include <cassert>
#include <cstddef>
#include <vector>

using State = std::vector<double>;

template<typename Fn>
State euler(Fn f, double t0, const State &y0, double t1, int steps) {
    assert(steps > 0);
    double h = (t1 - t0) / steps;
    State y = y0;
    for (int i = 0; i < steps; i++) {
        State k = f(t0 + i * h, y);
        for (size_t j = 0; j < y.size(); j++) {
            y[j] += h * k[j];
        }
    }
    return y;
}

template<typename Fn>
State midpoint(Fn f, double t0, const State &y0, double t1, int steps) {
    assert(steps > 0);
    double h = (t1 - t0) / steps;
    State y = y0;
```

```

for (int i = 0; i < steps; i++) {
    double t = t0 + i * h;
    State k1 = f(t, y), half = y;
    for (size_t j = 0; j < y.size(); j++) {
        half[j] += h / 2 * k1[j];
    }
    State k2 = f(t + h / 2, half);
    for (size_t j = 0; j < y.size(); j++) {
        y[j] += h * k2[j];
    }
}
return y;
}

template<typename Fn>
State runge_kutta(Fn f, double t0, const State &y0, double t1, int steps) {
    assert(steps > 0);
    double h = (t1 - t0) / steps;
    State y = y0, probe = y0;
    for (int i = 0; i < steps; i++) {
        double t = t0 + i * h;
        State k1 = f(t, y);
        for (size_t j = 0; j < y.size(); j++) {
            probe[j] = y[j] + h / 2 * k1[j];
        }
        State k2 = f(t + h / 2, probe);
        for (size_t j = 0; j < y.size(); j++) {
            probe[j] = y[j] + h / 2 * k2[j];
        }
        State k3 = f(t + h / 2, probe);
        for (size_t j = 0; j < y.size(); j++) {
            probe[j] = y[j] + h * k3[j];
        }
        State k4 = f(t + h, probe);
        for (size_t j = 0; j < y.size(); j++) {
            y[j] += h / 6 * (k1[j] + 2 * k2[j] + 2 * k3[j] + k4[j]);
        }
    }
    return y;
}

```

## Example Usage

```

#include <cmath>
using namespace std;

const double PI = acos(-1.0), E = exp(1.0);

int main() {
    // y' = y, so y(1) = e starting from y(0) = 1.
    auto exponential = [](double, const State &y) { return State{y[0]}; };
    assert(fabs(euler(exponential, 0, State{1}, 1, 1000)[0] - E) < 2e-3);
    assert(fabs(midpoint(exponential, 0, State{1}, 1, 1000)[0] - E) < 1e-6);
    assert(fabs(runge_kutta(exponential, 0, State{1}, 1, 1000)[0] - E) < 1e-12);

    // Quartering the step size should cut the fourth-order error by about 256.
    double coarse = fabs(runge_kutta(exponential, 0, State{1}, 1, 10)[0] - E);
    double fine = fabs(runge_kutta(exponential, 0, State{1}, 1, 40)[0] - E);
    assert(coarse / fine > 100);

    // The harmonic oscillator y'' = -y as the system y0' = y1, y1' = -y0, whose orbit is a circle.
    auto oscillator = [](double, const State &y) { return State{y[1], -y[0]}; };
    State full_turn = runge_kutta(oscillator, 0, State{1, 0}, 2 * PI, 2000);
    assert(fabs(full_turn[0] - 1) < 1e-9 && fabs(full_turn[1]) < 1e-9);

    // Integrating backwards returns to the starting state.
}

```

```

State there = runge_kutta(oscillator, 0, State{1, 0}, 3.0, 500);
State back = runge_kutta(oscillator, 3.0, there, 0, 500);
assert(fabs(back[0] - 1) < 1e-9 && fabs(back[1]) < 1e-9);
return 0;
}

```

### 5.4.8 Linear Programming (Simplex)

Solves a linear programming problem using Dantzig's simplex algorithm. The canonical form of a linear programming problem is to maximize (or minimize) the dot product  $cx$ , subject to  $Ax \leq b$  and  $x \geq 0$ , where  $x$  is a vector of unknowns to be solved,  $c$  is a vector of coefficients,  $A$  is a matrix of constraint coefficients, and  $b$  is a vector of bounds.

The simplex algorithm walks between vertices of the feasible polytope, at each step pivoting to an adjacent vertex that improves the objective, until no improving move remains (an optimum is reached) or an unbounded improving direction is found.

The implementation uses a two-phase simplex tableau. Phase 1 introduces an artificial variable to find a feasible starting basis, which is essential when some constraint bounds are negative. Phase 2 then optimizes the original objective. The `basis` and `nonbasis` arrays track which variable each tableau row or column currently represents, making solution recovery independent of pivot order.

- `simplex_solve(a, b, c, &x, maximize = true, eps = 1e-10)` solves the linear programming problem for an  $m$  by  $n$  matrix `a` of real values, a length  $m$  vector `b`, and a length  $n$  vector `c`, returning 0 if an optimum was found,  $-1$  if there are no feasible solutions, or 1 if the objective is unbounded. Set `maximize = false` to minimize instead, and use `eps` as the pivot and feasibility tolerance. If an optimum is found, `x` is assigned a vector of length `n`, where `x[j]` is the value of the  $j$ -th variable.

**Time Complexity:**  $O(pmn)$  per call, where  $p$  is the total number of pivots across both phases. Each pivot takes  $O(mn)$ , and  $p$  can be exponential in the worst case, although it is usually much smaller in practice.

**Space Complexity:**  $O(m \cdot n)$  auxiliary.

```

#include <cassert>
#include <cmath>
#include <numeric>
#include <utility>
#include <vector>

int simplex_solve(
    const std::vector<std::vector<double>> &a, const std::vector<double> &b,
    const std::vector<double> &c, std::vector<double> *x, bool maximize = true, double eps = 1e-10
) {
    int m = static_cast<int>(a.size()), n = static_cast<int>(c.size());
    assert(x != nullptr && n > 0 && b.size() == a.size());
    std::vector<int> basis(m), nonbasis(n + 1);
    std::vector<std::vector<double>> tab(m + 2, std::vector<double>(n + 2));
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            tab[i][j] = a[i][j];
        }
        basis[i] = n + i;
        tab[i][n] = -1;
        tab[i][n + 1] = b[i];
    }
    for (int j = 0; j < n; j++) {
        nonbasis[j] = j;
        tab[m][j] = maximize ? -c[j] : c[j];
    }
    nonbasis[n] = -1;
    tab[m + 1][n] = 1;
    auto pivot = [&](int row, int col) {

```



```

double inv = 1.0 / tab[row][col];
for (int i = 0; i < m + 2; i++) {
    if (i == row || fabs(tab[i][col]) <= eps) {
        continue;
    }
    double factor = tab[i][col] * inv;
    for (int j = 0; j < n + 2; j++) {
        if (j != col) {
            tab[i][j] -= factor * tab[row][j];
        }
    }
    tab[i][col] = -factor;
}
for (int j = 0; j < n + 2; j++) {
    if (j != col) {
        tab[row][j] *= inv;
    }
}
tab[row][col] = inv;
std::swap(basis[row], nonbasis[col]);
};

auto simplex = [&](int phase) {
    int objective = (phase == 1 ? m + 1 : m);
    while (true) {
        int col = -1;
        for (int j = 0; j <= n; j++) {
            if (phase == 2 && nonbasis[j] == -1) {
                continue;
            }
            if (col == -1 || tab[objective][j] < tab[objective][col] - eps ||
                (fabs(tab[objective][j] - tab[objective][col]) <= eps && nonbasis[j] < nonbasis[col])) {
                col = j;
            }
        }
        if (tab[objective][col] >= -eps) {
            return true;
        }
        int row = -1;
        for (int i = 0; i < m; i++) {
            if (tab[i][col] <= eps) {
                continue;
            }
            double cur = tab[i][n + 1] / tab[i][col];
            double best = row == -1 ? 0 : tab[row][n + 1] / tab[row][col];
            if (row == -1 || cur < best - eps || (fabs(cur - best) <= eps && basis[i] < basis[row])) {
                row = i;
            }
        }
        if (row == -1) {
            return false;
        }
        pivot(row, col);
    }
};

int row = 0;
for (int i = 1; i < m; i++) {
    if (tab[i][n + 1] < tab[row][n + 1]) {
        row = i;
    }
}
if (m > 0 && tab[row][n + 1] < -eps) {
    pivot(row, n);
    if (!simplex(1) || fabs(tab[m + 1][n + 1]) > eps) {
        return -1;
    }
    for (int i = 0; i < m; i++) {
        if (basis[i] != -1) {
            continue;
        }
    }
}

```

```

    }
    int col = -1;
    for (int j = 0; j <= n; j++) {
        if (fabs(tab[i][j]) > eps && (col == -1 || nonbasis[j] < nonbasis[col])) {
            col = j;
        }
    }
    if (col != -1) {
        pivot(i, col);
    }
}
}
if (!simplex(2)) {
    return 1;
}
x->assign(n, 0);
for (int i = 0; i < m; i++) {
    if (basis[i] < n) {
        (*x)[basis[i]] = tab[i][n + 1];
    }
}
return 0;
}

```

## Example Usage

```

#include <iostream>
using namespace std;

int main() {
    // Solve [x, y] that maximizes 3x + 4y, subject to x, y >= 0 and:
    // -2x + 1y <= 0
    // 1x + 0.85y <= 9
    // 1x + 2y <= 14
    vector<vector<double>> va{{-2, 1}, {1, 0.85}, {1, 2}};
    vector<double> vb{0, 9, 14};
    vector<double> vc{3, 4};
    vector<double> x;
    assert(simplex_solve(va, vb, vc, &x) == 0);
    double maxval = inner_product(vc.begin(), vc.end(), x.begin(), 0.0);
    cout << "Solution = " << maxval << " at (" << x[0];
    for (int i = 1; i < static_cast<int>(x.size()); i++) {
        cout << ", " << x[i];
    }
    cout << ")." << endl;

    // Feasible even though the first RHS is negative: x >= 1 and x <= 3.
    vector<vector<double>> negative_rhs{{-1}, {1}};
    vector<double> negative_bounds{-1, 3};
    vector<double> one_var{1};
    assert(simplex_solve(negative_rhs, negative_bounds, one_var, &x) == 0);
    assert(fabs(x[0] - 3) < 1e-9);

    // Infeasible: x <= -1 contradicts x >= 0.
    vector<vector<double>> infeasible{{1}};
    vector<double> bad_bounds{-1};
    assert(simplex_solve(infeasible, bad_bounds, one_var, &x) == -1);

    // Unbounded: maximize x subject only to x >= 0.
    vector<vector<double>> unbounded{{-1}};
    vector<double> lower_bound{0};
    assert(simplex_solve(unbounded, lower_bound, one_var, &x) == 1);
    return 0;
}

```

**Example Output**

Solution = 33.3043 at (5.30435, 4.34783).

## Chapter 6

# Mathematics

### 6.1 Math Utilities

---

#### 6.1.1 Floating-Point Comparison

Floating-point results carry rounding error, so two values that should be equal in exact arithmetic usually differ in their last bits. Comparing them with `==` is therefore a bug in almost every numeric setting, and the helpers below compare within a tolerance instead. This section also collects the mathematical constants and the sign inspections that the rest of the anthology uses.

- `M_PI`, `M_E`, `M_PHI`, `M_INF`, and `M_NAN` are the usual constants, computed rather than assumed so that no platform-specific macro is required.
- `EQ()`, `NE()`, `LT()`, `GT()`, `LE()`, and `GE()` relationally compare two values  $x$  and  $y$ . Arguments may be of different types; the common type governs behavior. If the common type is a floating-point type, exactly equal values (including same-signed infinities) compare equal; otherwise absolute-error epsilon comparison is used. Values within `EPS` of each other are considered equal, and `LT`/`GT`/`LE`/`GE` shift the boundary by `EPS` accordingly. Otherwise exact comparison is used (`==`, `<`, etc.). The branch is selected with `if constexpr`, so exact types without floating-point arithmetic (e.g. integers, `Modular`, `Rational`) compose as well.
- `req(ref, val)` returns whether `val` equals reference `ref` within relative error `EPS`. The tolerance scales with `|ref|`, so `req(ref, val)` is NOT the same as `req(val, ref)`. Use this when one argument is a known exact value and the other is a computed approximation. Degenerates to exact comparison when `ref` is 0, since the tolerance collapses to 0; use `EQ` near zero.
- `req_sym(a, b)` is the symmetric (commutative) variant: tolerance scales with `max(|a|, |b|)`, so the result is the same regardless of argument order. Still degenerates near zero when both arguments are close to 0. For both relative comparisons, exactly equal infinities compare equal, while unequal infinities and comparisons between finite and non-finite values compare unequal. Non-floating common types use exact equality.
- `sgn(x)` returns `-1` (if  $x < 0$ ), `0` (if  $x = 0$ ), or `1` (if  $x > 0$ ). Unlike `std::signbit()` or `std::copysign()`, this does not handle the sign of `NaN`.
- `sign_bit(x)` is analogous to `std::signbit()`, returning whether the sign bit of the floating point number is set to true. If so, then `x` is considered “negative.” Note that this works as expected on `+0.0`, `-0.0`, `Inf`, `-Inf`, `NaN`, as well as `-NaN`. Warning: This assumes that the sign bit is the leading (most significant) bit in the internal IEEE representation and that bytes are stored in little-endian order.
- `copy_sign(x, y)` is analogous to `std::copysign()`, returning a number with the magnitude of `x` but the sign of `y`.

**Time Complexity:**  $O(1)$  per call to all operations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
```

```

#include <climits>
#include <cmath>
#include <limits>
#include <type_traits>

#ifdef M_PI
const double M_PI = std::acos(-1.0); // Or std::numbers::pi in C++20 and later.
#else
#ifdef M_E
const double M_E = std::exp(1.0); // or std::numbers::e and std::numbers::e_v<> in C++20 and later
#else
const double M_PHI = (1.0 + std::sqrt(5.0)) / 2.0;
const double M_INF = std::numeric_limits<double>::infinity();
const double M_NAN = std::numeric_limits<double>::quiet_NaN();

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U> bool NE(T a, U b) { return !EQ(a, b); }
template<typename T, typename U> bool GT(T a, U b) { return LT(b, a); }
template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool rEQ(T ref, U val) {
    C x = C(ref), y = C(val);
    if (x == y) return true;
    if constexpr (!std::is_floating_point_v<C>) {
        return false;
    }
    if (!std::isfinite(x) || !std::isfinite(y)) return false;
    return std::fabs(x - y) <= EPS * std::fabs(x);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool rEQ_sym(T a, U b) {
    C x = C(a), y = C(b);
    if (x == y) return true;
    if constexpr (!std::is_floating_point_v<C>) {
        return false;
    }
    if (!std::isfinite(x) || !std::isfinite(y)) return false;
    return std::fabs(x - y) <= EPS * std::max(std::fabs(x), std::fabs(y));
}

template<typename T>
int sgn(const T &x) {
    return (T{0} < x) - (x < T{0});
}

template<typename Dbl>
bool sign_bit(Dbl x) {
    return (((unsigned char *)&x)[sizeof(x) - 1] >> (CHAR_BIT - 1)) & 1;
}

template<typename Dbl>
Dbl copy_sign(Dbl x, Dbl y) {

```

```
    return sign_bit(y) ? -std::fabs(x) : std::fabs(x);
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(EQ(M_PI, 3.14159265359));
    assert(EQ(M_E, 2.718281828459) && EQ(M_PHI, 1.61803398875));
    assert(EQ(M_INF, M_INF) && rEQ(M_INF, M_INF) && rEQ_sym(M_INF, M_INF));
    assert(!rEQ(M_INF, 0.0) && !rEQ_sym(M_INF, -M_INF));
    assert(!rEQ(10000000000000LL, 100000000000001LL));

    double x = -12345.6789;
    assert((-M_INF < x) && (x < M_INF));
    assert((M_INF + x == M_INF) && (M_INF - x == M_INF));
    assert((M_NAN != x) && (M_NAN != M_INF) && (M_NAN != M_NAN));
    assert(!(M_NAN < x) && !(M_NAN > x) && !(M_NAN <= x) && !(M_NAN >= x));
    assert(isnan(0.0 * M_INF) && isnan(M_INF - M_INF));

    assert(sgn(x) == -1 && sgn(0.0) == 0 && sgn(5678) == 1);
    assert(sign_bit(x) && !sign_bit(0.0) && sign_bit(-0.0));
    assert(!sign_bit(M_INF) && sign_bit(-M_INF));
    assert(!sign_bit(M_NAN) && sign_bit(-M_NAN));
    assert(copy_sign(1.0, +2.0) == +1.0 && copy_sign(M_INF, -2.0) == -M_INF);
    assert(copy_sign(1.0, -2.0) == -1.0 && sign_bit(copy_sign(M_NAN, -2.0)));
    return 0;
}
```

### 6.1.2 Rounding

Rounding is only ambiguous at a tie, when a value sits exactly halfway between two integers, and the rule chosen for that case is what distinguishes the functions below. The choice matters more than it appears: always rounding ties away from zero biases a sum of many rounded values upward, which is why financial and statistical work usually rounds ties to even instead, spreading them evenly between the two neighbors.

- `floor0(x)` returns `x` rounded down, symmetrically towards zero. This function is analogous to `std::trunc()`.
- `ceil0(x)` returns `x` rounded up, symmetrically away from zero. It mirrors `floor0()` and has no `<cmath>` equivalent, since `std::ceil()` rounds towards positive infinity rather than outward.
- `round_half_up(x)` returns `x` rounded half up, towards positive infinity.
- `round_half_down(x)` returns `x` rounded half down, towards negative infinity.
- `round_half_to0(x)` returns `x` rounded half down, symmetrically towards zero.
- `round_half_from0(x)` returns `x` rounded half up, symmetrically away from zero. This function is analogous to `std::round()`.
- `round_half_even(x, eps = 1e-9)` returns `x` rounded half to even, using `eps` to detect ties for banker's rounding.
- `round_half_alterate(x)` returns `x` rounded, where ties are broken by alternating rounds towards positive and negative infinity.
- `round_half_alterate0(x)` returns `x` rounded, where ties are broken by alternating symmetric rounds towards and away from zero.
- `round_half_random(x)` returns `x` rounded, where ties are broken randomly.
- `round_n_places(x, n, round)` returns `x` rounded to `n` digits after the decimal, using the specified rounding function `round(x)`.

Every function propagates `+0.0`, `-0.0`, `Inf`, `-Inf`, `NaN`, and `-NaN` exactly with the same sign, as `std::round()` does. The four built on adding or subtracting 0.5 before flooring are otherwise exact except at  $0.5 - 2^{-54}$ , which

rounds up to 1 because the sum is already 1 before flooring; prefer `std::round()` or `round_half_even()` where that last bit matters.

**Time Complexity:**  $O(1)$  per call to all operations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <chrono>
#include <cmath>
#include <random>

template<typename Dbl>
Dbl floor0(const Dbl &x) {
    Dbl res = std::floor(std::fabs(x));
    return std::signbit(x) ? -res : res;
}

template<typename Dbl>
Dbl ceil0(const Dbl &x) {
    Dbl res = std::ceil(std::fabs(x));
    return std::signbit(x) ? -res : res;
}

template<typename Dbl>
Dbl round_half_up(const Dbl &x) {
    return std::copysign(std::floor(x + 0.5), x); // copysign() only ever fixes a zero result.
}

template<typename Dbl>
Dbl round_half_down(const Dbl &x) {
    return std::copysign(std::ceil(x - 0.5), x); // copysign() only ever fixes a zero result.
}

template<typename Dbl>
Dbl round_half_to0(const Dbl &x) {
    Dbl res = round_half_down(std::fabs(x));
    return std::signbit(x) ? -res : res;
}

template<typename Dbl>
Dbl round_half_from0(const Dbl &x) {
    Dbl res = round_half_up(std::fabs(x));
    return std::signbit(x) ? -res : res;
}

template<typename Dbl>
Dbl round_half_even(const Dbl &x, const Dbl &eps = 1e-9) {
    if (std::signbit(x)) {
        return -round_half_even(-x, eps);
    }
    Dbl ipart;
    std::modf(x, &ipart);
    if (std::fabs(x - (ipart + 0.5)) < eps) { // exactly halfway: break the tie towards even
        return (std::fmod(ipart, 2.0) < eps) ? ipart : ceil0(ipart + 0.5);
    }
    return round_half_from0(x);
}

template<typename Dbl>
Dbl round_half_alterate(const Dbl &x) {
    Dbl up = round_half_up(x), down = round_half_down(x);
    if (std::isnan(x) || up == down) { // A NaN must not consume a toggle: NaN == NaN is false.
        return up;
    }
    static bool round_up = false;
    return (round_up = !round_up) ? up : down;
}
```

```

template<typename Dbl>
Dbl round_half_alterate0(const Dbl &x) {
    Dbl away = round_half_from0(x), toward = round_half_to0(x);
    if (std::isnan(x) || away == toward) { // A NaN must not consume a toggle.
        return away;
    }
    static bool round_away = false;
    return (round_away = !round_away) ? away : toward;
}

template<typename Dbl>
Dbl round_half_random(const Dbl &x) {
    Dbl away = round_half_from0(x), toward = round_half_to0(x);
    if (std::isnan(x) || away == toward) {
        return away;
    }
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    return (rng() % 2 == 0) ? away : toward;
}

template<typename Dbl, typename RoundFn>
Dbl round_n_places(const Dbl &x, unsigned int n, RoundFn round) {
    Dbl scale = std::pow(Dbl{10}, n);
    return round(x * scale) / scale;
}

```

## Example Usage

```

#include <cassert>
#include <limits>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    assert(EQ(floor0(1.5), 1.0) && EQ(ceil0(1.5), 2.0));
    assert(EQ(floor0(-1.5), -1.0) && EQ(ceil0(-1.5), -2.0));
    // Both round outward at any fraction, unlike std::round() which rounds to the nearest.
    assert(EQ(floor0(1.8), 1.0) && EQ(ceil0(1.2), 2.0) && EQ(ceil0(-1.2), -2.0));

    // NaN, the infinities, and the sign of zero are all preserved, as std::round() does.
    const double M_INF = numeric_limits<double>::infinity();
    const double M_NAN = numeric_limits<double>::quiet_NaN();
    assert(isnan(round_half_even(M_NAN)) && isinf(ceil0(-M_INF)));
    assert(signbit(round_half_from0(-0.0)) && !signbit(round_half_down(0.0)));
    assert(EQ(round_half_up(+1.5), +2) && EQ(round_half_down(+1.5), +1));
    assert(EQ(round_half_up(-1.5), -1) && EQ(round_half_down(-1.5), -2));
    assert(EQ(round_half_to0(+1.5), +1) && EQ(round_half_from0(+1.5), +2));
    assert(EQ(round_half_to0(-1.5), -1) && EQ(round_half_from0(-1.5), -2));
    assert(EQ(round_half_even(+1.5), +2) && EQ(round_half_even(-1.5), -2));
    assert(EQ(round_half_even(3.1), 3) && EQ(round_half_even(3.4), 3)); // Non-ties round normally.

    double alt1 = round_half_alterate(+1.5);
    assert(EQ(round_half_alterate(+1.2), +1)); // Non-ties do not consume an alternating turn.
    double alt2 = round_half_alterate(+1.5);
    assert(!EQ(alt1, alt2));
    double alt01 = round_half_alterate0(-1.5);
    assert(EQ(round_half_alterate0(-1.2), -1));
    double alt02 = round_half_alterate0(-1.5);
    assert(!EQ(alt01, alt02));

    assert(EQ(round_n_places(-1.23456, 3, round_half_to0<double>), -1.235));
    return 0;
}

```



### 6.1.3 Base Conversion

A positional numeral is a sequence of digits weighted by powers of its base, so converting between bases is repeated division: divide by the target base, record the remainder as the next digit, and continue with the quotient. Doing that division on a digit vector rather than on a machine integer is what lets `convert_base()` handle values far larger than any built-in type. Roman numerals are the counterexample that shows what positional notation buys, being additive with a subtractive special case and having no digit for zero.

- `to_base(x, b = 10)` returns the digits of the unsigned integer `x` in base `b`, where index 0 of the result stores the least significant digit.
- `to_roman(x)` returns the Roman numeral representation of the unsigned integer `x` as a string.
- `from_roman(s)` returns the value of the Roman numeral `s`, which must consist of Roman digits. Subtractive pairs such as `"IX"` are handled, but the numeral is not otherwise validated, so a malformed one such as `"IC"` simply returns the value its symbols imply.
- `convert_base(d, a, b)` converts an integer in base `a` as a vector `d` of digits (where `d[0]` is the least significant digit) to base `b` as a vector of digits (again with index 0 holding the least significant digit). This uses repeated long division, so the value itself does not need to fit in a machine integer.

Overflow warning: Each intermediate `rem*a + digit` must fit in `uint64_t`.

#### Time Complexity:

- $O(\log_b(x + 1) + 1)$  per call to `to_base()`.
- $O(x/1000 + 1)$  per call to `to_roman()` due to the repeated `M` prefix.
- $O(n)$  per call to `from_roman()`, where  $n$  is the length of the numeral.
- $O(d \cdot e)$  per call to `convert_base(d, a, b)`, where  $d$  is the number of input digits and  $e$  is the number of output digits.

#### Space Complexity:

- $O(e)$  auxiliary for `to_base(x, b)`, where  $e$  is the number of output digits.
- $O(x/1000 + 1)$  auxiliary for `to_roman(x)`.
- $O(1)$  auxiliary for `from_roman()`.
- $O(d + e)$  auxiliary for `convert_base(d, a, b)`.

```
#include <cassert>
#include <cstdint>
#include <map>
#include <string>
#include <vector>

std::vector<int> to_base(unsigned int x, int b = 10) {
    assert(b >= 2);
    std::vector<int> res;
    do { // do-while so that a value of 0 yields the single digit {0}
        res.push_back(x % b);
        x /= b;
    } while (x != 0);
    return res;
}

std::string to_roman(unsigned int x) {
    static const std::string h[] = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
    static const std::string t[] = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
    static const std::string o[] = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
    std::string prefix(x / 1000, 'M');
    x %= 1000;
    return prefix + h[x / 100] + t[x / 10 % 10] + o[x % 10];
}

unsigned int from_roman(const std::string &s) {
    static const std::map<char, unsigned int> VALUES = {{ 'I', 1}, { 'V', 5}, { 'X', 10}, { 'L', 50},
                                                         { 'C', 100}, { 'D', 500}, { 'M', 1000}};
```

```

unsigned int res = 0;
for (int i = 0; i < static_cast<int>(s.size()); i++) {
    auto it = VALUES.find(s[i]);
    assert(it != VALUES.end());
    auto next = i + 1 < static_cast<int>(s.size()) ? VALUES.find(s[i + 1]) : VALUES.end();
    // A smaller symbol before a larger one is subtracted, as the IV in XIV.
    res += next != VALUES.end() && it->second < next->second ? -it->second : it->second;
}
return res;
}

std::vector<int> convert_base(const std::vector<int> &d, int a, int b) {
    assert(a >= 2 && b >= 2);
    std::vector<int> cur = d, res;
    auto trim = [](std::vector<int> &v) {
        while (v.size() > 1 && v.back() == 0) {
            v.pop_back();
        }
    };
    trim(cur);
    if (cur.empty() || (cur.size() == 1 && cur[0] == 0)) {
        return {0};
    }
    while (!(cur.size() == 1 && cur[0] == 0)) {
        std::vector<int> q(cur.size());
        uint64_t rem = 0;
        for (int i = static_cast<int>(cur.size()) - 1; i >= 0; i--) {
            uint64_t x = rem * static_cast<uint64_t>(a) + static_cast<uint64_t>(cur[i]);
            q[i] = static_cast<int>(x / static_cast<uint64_t>(b));
            rem = x % static_cast<uint64_t>(b);
        }
        res.push_back(static_cast<int>(rem));
        trim(q);
        cur = std::move(q);
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(to_roman(1234) == "MCCXXXIV");
    assert(to_roman(5678) == "MMMMDCLXXVIII");
    assert(from_roman("MCCXXXIV") == 1234);
    assert(from_roman("IX") == 9 && from_roman("MMXXVI") == 2026);

    vector<int> digits{6, 5, 4, 3, 2, 1};
    assert(convert_base(to_base(123456, 20), 20, 10) == digits);
    assert(convert_base(vector<int>{0, 0, 0}, 10, 2) == vector<int>{0});

    vector<int> big_decimal(30, 9); // 10^30 - 1, larger than uint64_t.
    assert(convert_base(convert_base(big_decimal, 10, 2), 2, 10) == big_decimal);
    return 0;
}

```

### 6.1.4 Date and Time

Date questions become ordinary arithmetic once dates are converted to a day count. These functions use the proleptic Gregorian calendar, which applies the modern leap rule to every year, including those before the calendar was adopted, and they count days from the Unix epoch of 1970-01-01.

The conversion follows Howard Hinnant's closed-form algorithm, which shifts the year to start in March so that the leap day falls at the end of the year and never interrupts the month lengths. The remaining months then have lengths that repeat in a pattern whose running total is exactly  $\lfloor (153m + 2)/5 \rfloor$  days, and whole 400-year eras of 146097 days handle the leap rule without a single division-by-100 special case. Both directions are branch-light formulas rather than loops over years, so a date thousands of years away costs the same as tomorrow.

- `is_leap_year(y)` returns whether year `y` has a February 29.
- `days_in_month(y, m)` returns the length of month `m` of year `y`, where `m` is in  $[1, 12]$ .
- `days_from_civil(y, m, d)` returns the number of days from 1970-01-01 to the date `y-m-d`, which is negative for earlier dates. The date must be valid.
- `civil_from_days(days)` returns the tuple `(year, month, day)` of the date that many days from 1970-01-01, inverting `days_from_civil()`.
- `day_of_week(y, m, d)` returns the weekday of the date, from 0 for Sunday to 6 for Saturday.
- `days_between(y1, m1, d1, y2, m2, d2)` returns the number of days from the first date to the second, which is negative when the second date is earlier.

Unix time counts seconds from the same epoch, and is exactly 86400 times the day count plus the seconds elapsed within the day. That decomposition is exact only because Unix time ignores leap seconds: it is not a count of elapsed SI seconds but a label that repeats one value during a positive leap second, which is what keeps every day 86400 seconds long.

- `seconds_from_hms(h, m, s)` returns the seconds elapsed since midnight, and `hms_from_seconds(seconds)` returns the tuple `(hour, minute, second)` inverting it.
- `timestamp_from_civil(y, m, d, hh = 0, mm = 0, ss = 0)` returns the Unix timestamp of that instant.
- `civil_from_timestamp(t)` returns the `DateTime` of a Unix timestamp, which may be negative for instants before the epoch.

ISO 8601 numbers weeks from the one containing the first Thursday of the year, and assigns a date to the year of its own week's Thursday. Early January therefore often belongs to the previous ISO year and late December to the next, which is the mistake this replaces: 2021-01-01 is ISO week 53 of 2020.

- `iso_week_date(y, m, d)` returns the tuple `(iso_year, week, weekday)` of the date, where `week` counts from 1 and `weekday` runs from 1 for Monday to 7 for Sunday.

Easter follows the anonymous Gregorian computus, which locates the paschal full moon from the year's place in the 19-year Metonic cycle, corrects for the century's leap and lunar drift, then advances to the following Sunday.

- `easter(y)` returns the `(month, day)` of Easter Sunday in the Gregorian year `y`, which must be at least 1583.

Dates are ordinary `int` values with no range checking, and the era arithmetic stays within `int` for any year within a few million of the epoch. The proleptic calendar disagrees with history before a country's adoption of the Gregorian one in or after 1582, where problems usually specify Julian rules instead: the simpler leap rule of every fourth year.

**Time Complexity:**  $O(1)$  per call to all operations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <cstdint>
#include <tuple>
#include <utility>

struct DateTime {
    int year, month, day, hour, minute, second;
};
```

```

bool is_leap_year(int y) {
    return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0);
}

int days_in_month(int y, int m) {
    assert(m >= 1 && m <= 12);
    static const int lengths[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return m == 2 && is_leap_year(y) ? 29 : lengths[m - 1];
}

int days_from_civil(int y, int m, int d) {
    y -= m <= 2; // Start the year in March, so that February 29 is the final day of a year.
    int era = (y >= 0 ? y : y - 399) / 400;
    int yoe = y - era * 400; // Year of era, in [0, 399].
    int doy =
        (153 * (m + (m > 2 ? -3 : 9)) + 2) / 5 + d - 1; // Day of March-based year, in [0, 365].
    int doe = yoe * 365 + yoe / 4 - yoe / 100 + doy; // Day of era, in [0, 146096].
    return era * 146097 + doe - 719468; // 719468 is the day of era of 1970-01-01.
}

std::tuple<int, int, int> civil_from_days(int days) {
    days += 719468;
    int era = (days >= 0 ? days : days - 146096) / 146097;
    int doe = days - era * 146097;
    int yoe = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
    int y = yoe + era * 400;
    int doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
    int mp = (5 * doy + 2) / 153; // Month index of the March-based year, in [0, 11].
    int d = doy - (153 * mp + 2) / 5 + 1;
    int m = mp + (mp < 10 ? 3 : -9);
    return {y + (m <= 2), m, d};
}

int day_of_week(int y, int m, int d) {
    // 1970-01-01 was a Thursday, which is weekday 4.
    return ((days_from_civil(y, m, d) + 4) % 7 + 7) % 7;
}

int days_between(int y1, int m1, int d1, int y2, int m2, int d2) {
    return days_from_civil(y2, m2, d2) - days_from_civil(y1, m1, d1);
}

int seconds_from_hms(int h, int m, int s) {
    assert(h >= 0 && h < 24 && m >= 0 && m < 60 && s >= 0 && s < 60);
    return (h * 60 + m) * 60 + s;
}

std::tuple<int, int, int> hms_from_seconds(int seconds) {
    assert(seconds >= 0 && seconds < 86400);
    return {seconds / 3600, seconds / 60 % 60, seconds % 60};
}

int64_t timestamp_from_civil(int y, int m, int d, int hh = 0, int mm = 0, int ss = 0) {
    return int64_t{86400} * days_from_civil(y, m, d) + seconds_from_hms(hh, mm, ss);
}

DateTime civil_from_timestamp(int64_t t) {
    int64_t days = t / 86400, rest = t % 86400;
    if (rest < 0) { // Truncating division rounds toward zero, but the day count must floor.
        rest += 86400;
        days--;
    }
    auto [y, m, d] = civil_from_days(static_cast<int>(days));
    auto [hh, mm, ss] = hms_from_seconds(static_cast<int>(rest));
    return {y, m, d, hh, mm, ss};
}

```

```

std::tuple<int, int, int> iso_week_date(int y, int m, int d) {
    int days = days_from_civil(y, m, d);
    int weekday = ((days + 3) % 7 + 7) % 7 + 1; // The epoch was a Thursday, ISO weekday 4.
    int thursday = days - weekday + 4; // This week's Thursday fixes the ISO year.
    int iso_year = std::get<0>(civil_from_days(thursday));
    return {iso_year, (thursday - days_from_civil(iso_year, 1, 1)) / 7 + 1, weekday};
}

std::pair<int, int> easter(int y) {
    assert(y >= 1583);
    int a = y % 19; // Position in the Metonic cycle.
    int b = y / 100, c = y % 100; // Century and year within the century.
    int d = b / 4, e = b % 4; // Leap corrections for the century.
    int f = (b + 8) / 25, g = (b - f + 1) / 3;
    int h = (19 * a + b - d - g + 15) % 30; // Days from March 21 to the paschal full moon.
    int i = c / 4, k = c % 4;
    int l = (32 + 2 * e + 2 * i - h - k) % 7; // Days from the full moon to the next Sunday.
    int mm = (a + 11 * h + 22 * l) / 451; // Correction for the two latest possible dates.
    return {(h + 1 - 7 * mm + 114) / 31, (h + 1 - 7 * mm + 114) % 31 + 1};
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(is_leap_year(2024) && is_leap_year(2000));
    assert(!is_leap_year(2023) && !is_leap_year(1900));
    assert(days_in_month(2024, 2) == 29 && days_in_month(2023, 2) == 28);
    assert(days_in_month(2023, 12) == 31 && days_in_month(2023, 4) == 30);

    assert(days_from_civil(1970, 1, 1) == 0);
    assert(days_from_civil(1969, 12, 31) == -1);
    assert(days_from_civil(2000, 1, 1) == 10957);
    assert(civil_from_days(0) == make_tuple(1970, 1, 1));
    assert(civil_from_days(-1) == make_tuple(1969, 12, 31));
    assert(civil_from_days(10957) == make_tuple(2000, 1, 1));

    assert(day_of_week(1970, 1, 1) == 4); // A Thursday.
    assert(day_of_week(2000, 1, 1) == 6); // A Saturday.
    assert(day_of_week(1969, 7, 20) == 0); // A Sunday.

    // 2024 was a leap year, so February contributed 29 days.
    assert(days_between(2024, 1, 1, 2025, 1, 1) == 366);
    assert(days_between(2023, 1, 1, 2024, 1, 1) == 365);
    assert(days_between(2024, 3, 1, 2024, 2, 1) == -29);

    assert(seconds_from_hms(23, 31, 30) == 84690);
    assert(hms_from_seconds(84690) == make_tuple(23, 31, 30));
    assert(timestamp_from_civil(1970, 1, 1) == 0);
    assert(timestamp_from_civil(2009, 2, 13, 23, 31, 30) == 1234567890);

    auto [y, mo, d, hh, mm, ss] = civil_from_timestamp(1234567890);
    assert(y == 2009 && mo == 2 && d == 13 && hh == 23 && mm == 31 && ss == 30);
    auto before = civil_from_timestamp(-1); // One second before the epoch.
    assert(before.year == 1969 && before.month == 12 && before.day == 31);
    assert(before.hour == 23 && before.minute == 59 && before.second == 59);

    assert(iso_week_date(1970, 1, 1) == make_tuple(1970, 1, 4));
    // An ISO year runs from the Monday of the week holding the first Thursday, so it straddles
    // the calendar year on both sides.
    assert(iso_week_date(2019, 12, 30) == make_tuple(2020, 1, 1));
    assert(iso_week_date(2021, 1, 1) == make_tuple(2020, 53, 5));
    assert(iso_week_date(2026, 8, 17) == make_tuple(2026, 34, 1));
}

```

```

assert(easter(2000) == make_pair(4, 23));
assert(easter(2024) == make_pair(3, 31));
assert(easter(2025) == make_pair(4, 20));
auto [month, day] = easter(2026);
assert(day_of_week(2026, month, day) == 0); // Easter always falls on a Sunday.

// The century rule, where 2000 is a leap year but 1900 is not.
assert(days_between(2000, 2, 28, 2000, 3, 1) == 2);
assert(days_between(1900, 2, 28, 1900, 3, 1) == 1);
assert(civil_from_days(days_from_civil(2000, 2, 29)) == make_tuple(2000, 2, 29));
assert(civil_from_days(days_from_civil(1, 1, 1)) == make_tuple(1, 1, 1));
assert(civil_from_days(days_from_civil(-44, 3, 15)) == make_tuple(-44, 3, 15));
return 0;
}

```

## 6.2 Combinatorics

### 6.2.1 Combinatorial Calculations

The following functions implement common operations in combinatorics. All size and count inputs must be nonnegative, and all moduli must be positive. All return values and table entries are computed as 64-bit integers modulo an input argument  $m$  or  $p$ .

- `factorial(n, m = MOD)` returns  $n! \bmod m$ .
- `factorial_without_p(n, p = MOD)` returns  $n!$  modulo the prime  $p$  after removing every factor of  $p$  from the factorial.
- `legendre(n, p)` returns the exponent of the prime  $p$  in the factorization of  $n!$ , where  $p$  must be at least 2.
- `binomial_table(n, m = MOD)` returns the first  $n + 1$  rows of Pascal's triangle as a two-dimensional vector  $t$  such that  $t[i][j] = \binom{i}{j} \bmod m$ .
- `permute(n, k, m = MOD)` returns  $(n \text{ permute } k) \bmod m$ .
- `choose(n, k, p = MOD)` returns  $\binom{n}{k} \bmod p$ , where  $p$  is prime and  $n < p$ .
- `choose_lucas(n, k, p = MOD)` returns  $\binom{n}{k} \bmod p$  for any 64-bit  $n$  and  $k$ , where  $p$  is prime. It is only practical for a small  $p$ , since each base- $p$  digit costs a call to `choose()`.
- `multichoose(n, k, p = MOD)` returns  $(n \text{ multichoose } k) \bmod p$ , where  $p$  is prime and  $n + k - 1 < p$  when  $n > 0$ .
- `catalan(n, p = MOD)` returns the  $n$ -th Catalan number mod  $p$ , where  $p$  is prime and  $2n < p$ .
- `partitions(n, m = MOD)` returns the number of partitions of  $n$ , mod  $m$ .
- `partitions(n, k, m = MOD)` returns the number of partitions of  $n$  into  $k$  parts, mod  $m$ .
- `stirling1(n, k, m = MOD)` returns the  $(n, k)$  unsigned Stirling number of the 1st kind mod  $m$ .
- `stirling2(n, k, m = MOD)` returns the  $(n, k)$  Stirling number of the 2nd kind mod  $m$ .
- `eulerian1(n, k, m = MOD)` returns the  $(n, k)$  Eulerian number of the 1st kind mod  $m$ , where  $n > k$ .
- `eulerian2(n, k, m = MOD)` returns the  $(n, k)$  Eulerian number of the 2nd kind mod  $m$ , where  $n > k$ .

Legendre's formula sums  $\lfloor n/p^i \rfloor$  over every  $i \geq 1$ , counting how many of  $1, \dots, n$  are divisible by each power of  $p$ , which totals the exponent of  $p$  in  $n!$ . It answers questions such as how many trailing zeros  $n!$  has in base  $p$ . Paired with Kummer's theorem, which equates the exponent of  $p$  in  $\binom{n}{k}$  with the number of carries when adding  $k$  and  $n - k$  in base  $p$ , it also decides when a binomial coefficient is divisible by  $p$ .

Lucas' theorem reduces a binomial coefficient modulo a prime to a product over base- $p$  digits:  $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$ , where  $n_i$  and  $k_i$  are the  $i$ -th digits of  $n$  and  $k$  in base  $p$ . The product is 0 as soon as some  $k_i$  exceeds  $n_i$ . For a composite modulus, factor it into prime powers, apply the generalized version of the theorem to each factor, and recombine the residues with the Chinese remainder theorem of section 6.3.2.

Every routine here recomputes its answer from scratch, which is what suits a handful of values or a modulus that varies between calls. When a solution instead needs thousands of binomial coefficients against one fixed modulus, precompute factorials and inverse factorials: the combinatorics tables of section 6.3.3 amortize  $O(n)$  of table growth and then answer each query in  $O(1)$ .

Overflow warning: Modular products use ordinary `int64_t` multiplication, so the square of the chosen modulus must fit in `int64_t`.

#### Time Complexity:

- $O(n)$  per call to `factorial()`.
- $O(p \log_p(n))$  per call to `factorial_without_p()`.
- $O(\log_p(n))$  per call to `legendre()`.
- $O(n^2)$  per call to `binomial_table()`.
- $O(k)$  per call to `permute()`.
- $O(\min(k, n - k))$  per call to `choose()`.
- $O(p \log_p(n))$  per call to `choose_lucas()`.
- $O(k)$  per call to `multichoose()`.
- $O(n)$  per call to `catalan()`.
- $O(n^2)$  per call to `partitions(n, m)`.
- $O(n \cdot k)$  per call to `partitions(n, k, m)`, `stirling1()`, `stirling2()`, `eulerian1()`, and `eulerian2()`.

#### Space Complexity:

- $O(n^2)$  auxiliary for `binomial_table(n, m)`.
- $O(n \cdot k)$  auxiliary for `partitions(n, k, m)`, `stirling1(n, k, m)`, `stirling2(n, k, m)`, `eulerian1(n, k, m)`, and `eulerian2(n, k, m)`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <stdint>
#include <vector>

const int64_t MOD = 1000000007;

int64_t factorial(int n, int64_t m = MOD) {
    int64_t res = 1;
    for (int i = 2; i <= n; i++) {
        res = (res * i) % m;
    }
    return res % m;
}

int64_t factorial_without_p(int64_t n, int64_t p = MOD) {
    int64_t res = 1;
    while (n > 1) {
        if (n / p % 2 == 1) {
            res = res * (p - 1) % p;
        }
        int max = n % p;
        for (int i = 2; i <= max; i++) {
            res = (res * i) % p;
        }
        n /= p;
    }
    return res % p;
}

int64_t legendre(int64_t n, int64_t p) {
    assert(p >= 2);
    int64_t res = 0;
    for (int64_t q = n / p; q > 0; q /= p) { // Dividing repeatedly avoids overflowing p^i.
        res += q;
    }
    return res;
}

std::vector<std::vector<int64_t>> binomial_table(int n, int64_t m = MOD) {
    std::vector<std::vector<int64_t>> t(n + 1);
    for (int i = 0; i <= n; i++) {
```

```

    for (int j = 0; j <= i; j++) {
        if (i < 2 || j == 0 || i == j) {
            t[i].push_back(1);
        } else {
            t[i].push_back((t[i - 1][j - 1] + t[i - 1][j]) % m);
        }
    }
}
return t;
}

int64_t permute(int n, int k, int64_t m = MOD) {
    if (n < k) {
        return 0;
    }
    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        res = res * (n - i) % m;
    }
    return res % m;
}

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

int64_t choose(int n, int k, int64_t p = MOD) {
    if (n < k) {
        return 0;
    }
    if (k > n - k) {
        k = n - k;
    }
    int64_t num = 1, den = 1;
    for (int i = 0; i < k; i++) {
        num = num * (n - i) % p;
    }
    for (int i = 1; i <= k; i++) {
        den = den * i % p;
    }
    return num * powmod(den, p - 2, p) % p;
}

int64_t choose_lucas(int64_t n, int64_t k, int64_t p = MOD) {
    if (k < 0 || k > n) {
        return 0;
    }
    int64_t res = 1;
    while (n > 0) { // Since k never exceeds n, k is also exhausted when n reaches 0.
        int64_t ni = n % p, ki = k % p;
        if (ki > ni) {
            return 0;
        }
        res = res * choose(static_cast<int>(ni), static_cast<int>(ki), p) % p;
        n /= p;
        k /= p;
    }
    return res;
}

int64_t multichoose(int n, int k, int64_t p = MOD) {

```



```

    if (n == 0) {
        return k == 0;
    }
    return choose(n + k - 1, k, p);
}

int64_t catalan(int n, int64_t p = MOD) {
    return choose(2 * n, n, p) * powmod(n + 1, p - 2, p) % p;
}

int64_t partitions(int n, int64_t m = MOD) {
    std::vector<int64_t> t(n + 1);
    t[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = i; j <= n; j++) {
            t[j] = (t[j] + t[j - i]) % m;
        }
    }
    return t[n] % m;
}

int64_t partitions(int n, int k, int64_t m = MOD) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1, h = k < i ? k : i; j <= h; j++) {
            t[i][j] = (t[i - 1][j - 1] + t[i - j][j]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t stirling1(int n, int k, int64_t m = MOD) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - 1) * t[i - 1][j] % m;
            t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t stirling2(int n, int k, int64_t m = MOD) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = j * t[i - 1][j] % m;
            t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t eulerian1(int n, int k, int64_t m = MOD) {
    if (k > n - 1 - k) {
        k = n - 1 - k;
    }
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 1));
    for (int j = 1; j <= k; j++) {
        t[0][j] = 0;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - j) * t[i - 1][j - 1] % m;
            t[i][j] = (t[i][j] + ((j + 1) * t[i - 1][j] % m)) % m;
        }
    }
}

```

```

    }
}
return t[n][k] % m;
}

int64_t eulerian2(int n, int k, int64_t m = MOD) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (i == j) {
                t[i][j] = 0;
            } else {
                t[i][j] = (j + 1) * t[i - 1][j] % m;
                t[i][j] = ((2 * i - 1 - j) * t[i - 1][j - 1] % m + t[i][j]) % m;
            }
        }
    }
    return t[n][k] % m;
}

```

## Example Usage

```

#include <cassert>

int main() {
    auto t = binomial_table(10);
    for (int i = 0; i < static_cast<int>(t.size()); i++) {
        for (int j = 0; j < static_cast<int>(t[i].size()); j++) {
            assert(t[i][j] == choose(i, j));
        }
    }
    assert(factorial(10) == 3628800);
    assert(factorial_without_p(123456) == 639390503);
    assert(factorial_without_p(7, 5) == 3); // 7! / 5 = 1008 = 3 (mod 5).
    assert(legendre(10, 2) == 8); // 10! = 2^8 * 3^4 * 5^2 * 7.
    assert(legendre(100, 5) == 24); // 100! ends in 24 zeros.
    assert(permute(10, 4) == 5040);
    assert(choose(20, 7) == 77520);
    assert(choose_lucas(20, 7) == 77520);
    auto mod7 = binomial_table(30, 7);
    for (int i = 0; i <= 30; i++) {
        for (int j = 0; j <= i; j++) {
            assert(choose_lucas(i, j, 7) == mod7[i][j]);
        }
    }
    // By Lucas' theorem mod 2, a binomial coefficient is odd exactly when k is a submask of n.
    assert(choose_lucas((1LL << 40) - 1, 12345, 2) == 1);
    assert(choose_lucas(1LL << 40, 1, 2) == 0);
    assert(multichoose(20, 7) == 657800);
    assert(catalan(10) == 16796);
    assert(partitions(4) == 5);
    assert(partitions(0, 0) == 1);
    assert(partitions(100, 5) == 38225);
    assert(stirling1(4, 2) == 11);
    assert(stirling2(4, 3) == 6);
    assert(eulerian1(9, 5) == 88234);
    assert(eulerian2(8, 3) == 195800);
    return 0;
}

```

### 6.2.2 Enumerating Arrangements

For the purposes of this section, we define a “size  $k$  arrangement of  $n$ ” to be a permutation of a size  $k$  subset of the integers in  $[0, n)$ , for  $0 \leq k \leq n$ . There are  $n$  permute  $k$  possible arrangements, but  $n^k$  possible arrangements if repeated values are allowed.

For arrangements without repeats, each prefix fixes some used values and leaves a shrinking set of choices for the remaining positions. Ranking counts how many unused smaller choices could have been placed at the current position, multiplied by the number of possible suffix arrangements; unranking performs the inverse search by subtracting those block sizes. With repeats allowed, the sequence is just a base- $n$  counter.

- `next_arrangement(n, a)` tries to rearrange `a` to the next lexicographically greater arrangement, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a` is unchanged). The input `a` must consist of distinct integers in the range  $[0, n)$ .
- `arrangement_by_rank(n, k, r)` returns the size  $k$  arrangement of  $n$  which is lexicographically ranked  $r$  out of all size  $k$  arrangements of  $n$ , where  $r$  is a 0-based rank in the range  $[0, n \text{ permute } k)$ .
- `rank_by_arrangement(n, a)` returns an integer representing the 0-based rank of arrangement `a`, which must consist of distinct integers in the range  $[0, n)$ .
- `next_arrangement_with_repeats(n, a)` tries to rearrange `a` to the next lexicographically greater arrangement with repeats, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a` is unchanged). The input `a` must consist of integers in the range  $[0, n)$ . If `a` were interpreted as a  $k$  digit integer in base  $n$ , this function could be thought of as incrementing the integer.

#### Time Complexity:

- $O(n \cdot k)$  per call to `next_arrangement()`, `arrangement_by_rank()`, and `rank_by_arrangement(n, a)`, where  $k$  is the size of `a`.
- $O(k)$  per call to `next_arrangement_with_repeats()`.

#### Space Complexity:

- $O(n)$  auxiliary for `next_arrangement()`, `arrangement_by_rank()`, and `rank_by_arrangement()`.
- $O(1)$  auxiliary for `next_arrangement_with_repeats()`.

```
#include <algorithm>
#include <cstdint>
#include <numeric>
#include <vector>

bool next_arrangement(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    std::vector<char> used(n);
    for (int i = 0; i < k; i++) {
        used[a[i]] = true;
    }
    for (int i = k - 1; i >= 0; i--) {
        used[a[i]] = false;
        for (int j = a[i] + 1; j < n; j++) {
            if (!used[j]) {
                a[i++] = j;
                used[j] = true;
                for (int x = 0; x < k; x++) {
                    if (!used[x]) {
                        a[i++] = x;
                    }
                }
                return true;
            }
        }
    }
    return false;
}

int64_t n_permute_k(int n, int k) {
```

```

    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        res *= n - i;
    }
    return res;
}

std::vector<int> arrangement_by_rank(int n, int k, int64_t r) {
    std::vector<int> values(n), res(k);
    std::iota(values.begin(), values.end(), 0);
    for (int i = 0; i < k; i++) {
        int64_t count = n_permute_k(n - 1 - i, k - 1 - i);
        int pos = r / count;
        res[i] = values[pos];
        values.erase(values.begin() + pos);
        r %= count;
    }
    return res;
}

int64_t rank_by_arrangement(int n, const std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    int64_t res = 0;
    std::vector<char> used(n);
    for (int i = 0; i < k; i++) {
        int count = 0;
        for (int j = 0; j < a[i]; j++) {
            if (!used[j]) {
                count++;
            }
        }
        res += count * n_permute_k(n - i - 1, k - i - 1);
        used[a[i]] = true;
    }
    return res;
}

bool next_arrangement_with_repeats(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            a[i]++;
            std::fill(a.begin() + i + 1, a.end(), 0);
            return true;
        }
    }
    return false;
}

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {

```

```

int n = 4, k = 3;
vector<int> a{0, 1, 2};
cout << n << " permute " << k << " arrangements:" << endl;
int count = 0;
do {
    print_range(a.begin(), a.end());
    assert(a == arrangement_by_rank(n, k, count));
    assert(rank_by_arrangement(n, a) == count);
    count++;
    if (count == 10 || count == 20) {
        cout << endl;
    }
} while (next_arrangement(n, a));
assert(count == 24);
cout << endl;
}
{
    int n = 4, k = 2;
    vector<int> a{0, 0};
    cout << endl << n << "^" << k << " arrangements with repeats:" << endl;
    int count = 0;
    do {
        print_range(a.begin(), a.end());
        count++;
        if (count == 13) {
            cout << endl;
        }
    } while (next_arrangement_with_repeats(n, a));
    assert(count == 16);
    cout << endl;
}
return 0;
}

```

### Example Output

```

4 permute 3 arrangements:
{0,1,2} {0,1,3} {0,2,1} {0,2,3} {0,3,1} {0,3,2} {1,0,2} {1,0,3} {1,2,0} {1,2,3}
{1,3,0} {1,3,2} {2,0,1} {2,0,3} {2,1,0} {2,1,3} {2,3,0} {2,3,1} {3,0,1} {3,0,2}
{3,1,0} {3,1,2} {3,2,0} {3,2,1}

4^2 arrangements with repeats:
{0,0} {0,1} {0,2} {0,3} {1,0} {1,1} {1,2} {1,3} {2,0} {2,1} {2,2} {2,3} {3,0}
{3,1} {3,2} {3,3}

```

### 6.2.3 Enumerating Permutations

These routines enumerate and analyze reorderings of a sequence of  $n$  elements; the values need not be distinct.

The lexicographic successor is found by locating the rightmost ascent, increasing that position by the smallest possible amount, then reversing the suffix into its minimum order. Ranking and unranking permutations use the factorial number system: each position contributes the number of unused smaller values before the chosen one, scaled by the number of possible suffix permutations. The cycle decomposition views a permutation as a function on indices and follows each unvisited orbit until it returns to its start.

- `next_permutation2(lo, hi, comp = std::less<>())` is analogous to `std::next_permutation()`, taking two BidirectionalIterators as a range `[lo, hi)` for which the function tries to rearrange to the next lexicographically greater permutation according to `comp`. The function returns true if such a permutation exists, or false if the range is already in reverse comparator order, in which case it is rearranged into comparator order.
- `next_permutation(a, comp = std::less<>())` is analogous to `next_permutation2()`, except that it takes a vector instead of a range.

- `next_permutation_mask(x)` returns the next integer having the same number of 1-bits. Treating each 1-bit as whether to take its corresponding item generates combinations of a set of  $n$  items. It returns 0 if no successor fits in `uint64_t`.
- `permutation_by_rank(n, r)` returns the permutation of the integers in the range  $[0, n)$  which is lexicographically ranked  $r$ , where  $r$  is a 0-based rank in the range  $[0, n!)$ .
- `rank_by_permutation(a)` returns an integer representing the 0-based rank of permutation `a`, which must be a permutation of the integers  $[0, n)$ .
- `permutation_cycles(a)` returns the orbits of the index mapping  $i \mapsto a[i]$ . For example,  $\{3, 1, 0, 2\}$  decomposes into cycles  $\{0, 3, 2\}$  and  $\{1\}$  because following the mapping from index 0 gives  $0 \rightarrow 3 \rightarrow 2 \rightarrow 0$ , while index 1 maps to itself.

#### Time Complexity:

- $O(n)$  per call to `next_permutation2()` and `next_permutation()`, where  $n$  is the input size.
- $O(n^2)$  per call to `permutation_by_rank()` and `rank_by_permutation()`.
- $O(1)$  per call to `next_permutation_mask()`.
- $O(n)$  per call to `permutation_cycles()`.

#### Space Complexity:

- $O(1)$  auxiliary for `next_permutation2()` and `next_permutation()`.
- $O(n)$  auxiliary for `permutation_by_rank()`, `rank_by_permutation()`, and `permutation_cycles()`.

```
#include <algorithm>
#include <cstdint>
#include <functional>
#include <numeric>
#include <utility>
#include <vector>

template<typename It, typename Compare = std::less<>>
bool next_permutation2(It lo, It hi, Compare comp = Compare{}) {
    if (lo == hi) {
        return false;
    }
    It i = lo;
    if (++i == hi) {
        return false;
    }
    i = hi;
    --i;
    while (true) {
        It j = i;
        if (comp(*--i, *j)) {
            It k = hi;
            while (!comp(*i, *--k)) {
            }
            std::iter_swap(i, k);
            std::reverse(j, hi);
            return true;
        }
        if (i == lo) {
            std::reverse(lo, hi);
            return false;
        }
    }
}

template<typename T, typename Compare = std::less<>>
bool next_permutation(std::vector<T> &a, Compare comp = Compare{}) {
    int n = static_cast<int>(a.size());
    for (int i = n - 2; i >= 0; i--) {
        if (comp(a[i], a[i + 1])) {
            for (int j = n - 1; j-- > i) {
                if (comp(a[i], a[j])) {

```

```

        std::swap(a[i++], a[j]);
        for (j = n - 1; i < j; i++, j--) {
            std::swap(a[i], a[j]);
        }
        return true;
    }
}
}
std::reverse(a.begin(), a.end());
return false;
}

uint64_t next_permutation_mask(uint64_t x) {
    if (x == 0) {
        return 0;
    }
    uint64_t s = x & -x, r = x + s;
    if (r == 0) {
        return 0;
    }
    return r | (((x ^ r) >> 2) / s);
}

std::vector<int> permutation_by_rank(int n, int64_t r) {
    if (n == 0) {
        return {};
    }
    std::vector<int64_t> factorial(n);
    std::vector<int> values(n), res(n);
    factorial[0] = 1;
    for (int i = 1; i < n; i++) {
        factorial[i] = i * factorial[i - 1];
    }
    std::iota(values.begin(), values.end(), 0);
    for (int i = 0; i < n; i++) {
        int pos = r / factorial[n - 1 - i];
        res[i] = values[pos];
        values.erase(values.begin() + pos);
        r %= factorial[n - 1 - i];
    }
    return res;
}

int64_t rank_by_permutation(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return 0;
    }
    std::vector<int64_t> factorial(n);
    factorial[0] = 1;
    for (int i = 1; i < n; i++) {
        factorial[i] = i * factorial[i - 1];
    }
    int64_t res = 0;
    for (int i = 0; i < n; i++) {
        int v = a[i];
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i]) {
                v--;
            }
        }
        res += v * factorial[n - 1 - i];
    }
    return res;
}

using Cycles = std::vector<std::vector<int>>>;

```

```

Cycles permutation_cycles(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    std::vector<char> visit(n);
    Cycles res;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            int j = i;
            std::vector<int> curr;
            do {
                curr.push_back(j);
                visit[j] = true;
                j = a[j];
            } while (j != i);
            res.push_back(std::move(curr));
        }
    }
    return res;
}

```

## Example Usage

```

#include <bitset>
#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {
        const int n = 4;
        vector<int> a{0, 1, 2, 3}, b = a, c = a;
        cout << "Permutations of [0, " << n << "):" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            assert(b == a);
            assert(c == a);
            assert(permutation_by_rank(n, count) == a);
            assert(rank_by_permutation(a) == count);
            count++;
            if (count == 8 || count == 16) {
                cout << endl;
            }
            std::next_permutation(b.begin(), b.end());
            next_permutation(c);
        } while (next_permutation(a));
        assert(count == 24);
        assert((a == vector<int>{0, 1, 2, 3}));
        cout << endl;
    }
    { // Permutations of binary digits.
        const int n = 5;
        cout << "\nPermutations of 2 zeros and 3 ones:" << endl;
        uint64_t lo = bitset<5>(string("00111")).to_ullong();
        uint64_t hi = bitset<6>(string("100011")).to_ullong();
        int count = 0;
        do {

```



```

    cout << bitset<n>(lo).to_string() << " ";
    count++;
} while ((lo = next_permutation_mask(lo)) != hi);
assert(count == 10);
assert(next_permutation_mask(1ULL << 63) == 0);
cout << endl;
}
{
    vector<int> a{3, 2, 1}, b = a;
    assert(next_permutation(a, greater<int>()));
    assert(next_permutation2(b.begin(), b.end(), greater<int>()));
    assert((a == vector<int>{3, 1, 2} && b == a));
}
{
    // Decomposition into cycles.
    vector<int> a{3, 1, 0, 2};
    cout << "\nDecomposition of {3,1,0,2} into cycles:" << endl;
    Cycles c = permutation_cycles(a);
    assert((c == Cycles{{0, 3, 2}, {1}}));
    for (const auto &cycle : c) {
        print_range(cycle.begin(), cycle.end());
    }
    cout << endl;
}
return 0;
}

```

### Example Output

```

Permutations of [0, 4):
{0,1,2,3} {0,1,3,2} {0,2,1,3} {0,2,3,1} {0,3,1,2} {0,3,2,1} {1,0,2,3} {1,0,3,2}
{1,2,0,3} {1,2,3,0} {1,3,0,2} {1,3,2,0} {2,0,1,3} {2,0,3,1} {2,1,0,3} {2,1,3,0}
{2,3,0,1} {2,3,1,0} {3,0,1,2} {3,0,2,1} {3,1,0,2} {3,1,2,0} {3,2,0,1} {3,2,1,0}

Permutations of 2 zeros and 3 ones:
00111 01011 01101 01110 10011 10101 10110 11001 11010 11100

Decomposition of {3,1,0,2} into cycles:
{0,3,2} {1}

```

## 6.2.4 Enumerating Combinations

A combination is a subset of size  $k$  chosen from a total of  $n$  (not necessarily distinct) elements, where order does not matter.

The lexicographic successor advances the rightmost chosen position that can still move right, then packs every later position as far left as possible. Ranking and unranking use the combinatorial number system: at each position, count how many combinations would be skipped by choosing a smaller next value, then either add that count to the rank or subtract it while searching for the requested rank. Bitmask successors use the same order as increasing integers with a fixed popcount.

- `next_combination(lo, mid, hi, comp = std::less<>())` takes random-access iterators `lo`, `mid`, and `hi` as a range  $[lo, hi)$  of  $n$  elements for which the function will rearrange such that the  $k$  elements in  $[lo, mid)$  become the next lexicographically greater combination. The function returns true if such a combination exists, or false if  $[lo, mid)$  already consists of the lexicographically greatest combination of the elements in  $[lo, hi)$ , in which case the range is reset to its first combination. The comparator `comp` defines the element ordering.
- `next_combination(n, a)` rearranges `a` to become the next lexicographically greater combination of distinct integers in the range  $[0, n)$ . The vector `a` must be sorted and contain distinct integers in the range  $[0, n)$ .
- `next_combination_mask(x)` interprets the bits of an integer `x` as a mask with 1-bits specifying the chosen items for a combination and returns the mask of the next lexicographically greater combination (that is, the lowest integer greater than `x` with the same number of 1 bits). Note that this does not generate

combinations in the same order as `next_combination()`, nor does it work if the corresponding  $n$  items are not distinct (in that case, duplicate combinations will be generated). It returns 0 if no successor fits in `uint64_t`.

- `combination_by_rank(n, k, r)` returns the combination of  $k$  distinct integers in the range  $[0, n)$  that is lexicographically ranked  $r$ , where  $r$  is a 0-based rank in the range  $[0, \binom{n}{k})$ .
- `rank_by_combination(n, a)` returns an integer representing the 0-based rank of combination `a`, which must contain sorted distinct integers in  $[0, n)$ .
- `next_combination_with_repeats(n, a)` rearranges `a` to become the next lexicographically greater combination of not necessarily distinct integers in the range  $[0, n)$ . The vector `a` must be sorted. Note that there is a total of  $n$  multichoose  $k$  combinations if repetition is allowed, where  $n$  multichoose  $k = \binom{n+k-1}{k}$ .

Overflow warning: Exact counts and ranks used by the ranking operations must fit in `int64_t`.

### Time Complexity:

- $O(n)$  per call to `next_combination(lo, mid, hi)`, where  $n$  is the distance between `lo` and `hi`.
- $O(k)$  per call to `next_combination(n, a)` and `next_combination_with_repeats(n, a)`, where  $k$  is the size of `a`.
- $O(1)$  per call to `next_combination_mask()`.
- $O(n \cdot k)$  per call to `combination_by_rank()` and `rank_by_combination()`.

### Space Complexity:

- $O(k)$  auxiliary for `combination_by_rank(n, k, r)`.
- $O(1)$  auxiliary for all other operations.

```
#include <algorithm>
#include <cstdint>
#include <functional>
#include <iterator>
#include <numeric>
#include <utility>
#include <vector>

template<typename It, typename Compare = std::less<>>
bool next_combination(It lo, It mid, It hi, Compare comp = Compare{}) {
    using T = typename std::iterator_traits<It>::value_type;
    if (lo == mid || mid == hi) {
        return false;
    }
    It l = mid - 1, h = hi - 1;
    int len1 = 1, len2 = 1;
    while (l != lo && !comp(*l, *h)) {
        --l;
        ++len1;
    }
    if (l == lo && !comp(*l, *h)) {
        std::rotate(lo, mid, hi);
        return false;
    }
    for (; mid < h; ++len2) {
        if (!comp(*l, *--h)) {
            ++h;
            break;
        }
    }
    if (len1 == 1 || len2 == 1) {
        std::iter_swap(l, h);
    } else if (len1 == len2) {
        std::swap_ranges(l, mid, h);
    } else {
        std::iter_swap(l++, h++);
        int total = (--len1) + (--len2), gcd = total;
        for (int i = len1; i != 0; i--) {
            std::swap(gcd %= i, i);
        }
        int skip = total / gcd - 1;
        for (int i = 0; i < gcd; i++) {
```

```

        It curr = (i < len1) ? (1 + i) : (h + (i - len1));
        int k = i;
        T prev = *curr;
        for (int j = 0; j < skip; j++) {
            k = (k + len1) % total;
            It next = (k < len1) ? (1 + k) : (h + (k - len1));
            *curr = *next;
            curr = next;
        }
        *curr = prev;
    }
}
return true;
}

bool next_combination(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i) {
            a[i]++;
            while (++i < k) {
                a[i] = a[i - 1] + 1;
            }
            return true;
        }
    }
    return false;
}

uint64_t next_combination_mask(uint64_t x) {
    if (x == 0) {
        return 0;
    }
    uint64_t s = x & -x, r = x + s;
    if (r == 0) {
        return 0;
    }
    return r | ((x ^ r) >> 2) / s;
}

int64_t n_choose_k(int64_t n, int64_t k) {
    if (k > n - k) {
        k = n - k;
    }
    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        int64_t num = n - i, den = i + 1;
        int64_t g = std::gcd(num, den);
        num /= g;
        den /= g;
        res /= den;
        res *= num; // Overflow warning.
    }
    return res;
}

std::vector<int> combination_by_rank(int n, int k, int64_t r) {
    std::vector<int> res(k);
    int count = n;
    for (int i = 0; i < k; i++) {
        int j = 1;
        for (;;) {
            int64_t am = n_choose_k(count - j, k - 1 - i);
            if (r < am) {
                break;
            }
            r -= am;
        }
    }
}

```

```

    res[i] = (i > 0) ? (res[i - 1] + j) : (j - 1);
    count -= j;
}
return res;
}

int64_t rank_by_combination(int n, const std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    int64_t res = 0;
    int prev = -1;
    for (int i = 0; i < k; i++) {
        for (int j = prev + 1; j < a[i]; j++) {
            res += n_choose_k(n - 1 - j, k - 1 - i);
        }
        prev = a[i];
    }
    return res;
}

bool next_combination_with_repeats(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            for (++a[i]; ++i < k; ) {
                a[i] = a[i - 1];
            }
            return true;
        }
    }
    return false;
}

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {
        int k = 3;
        string s = "11234";
        cout << "\n" << s << "\n choose " << k << " : " << endl;
        int count = 0;
        do {
            cout << s.substr(0, k) << " ";
            count++;
        } while (next_combination(s.begin(), s.begin() + k, s.end()));
        assert(count == 7); // Duplicate '1' values collapse equal output strings.
        cout << endl;
    }
    { // Unordered combinations using masks.
        int n = 5, k = 3;
        string char_set = "abcde"; // Must be distinct.
        cout << "\n\n" << char_set << "\n choose " << k << " with masks:" << endl;
        uint64_t mask = (1ULL << k) - 1, limit = 1ULL << n;
        int count = 0;
    }
}

```

```

do {
    for (int i = 0; i < n; i++) {
        if ((mask >> i) & 1) {
            cout << char_set[i];
        }
    }
    cout << " ";
    count++;
    mask = next_combination_mask(mask);
} while (mask < limit);
assert(count == 10);
assert(next_combination_mask(1ULL << 63) == 0);
cout << endl;
}
{
    string s = "4321";
    assert(next_combination(s.begin(), s.begin() + 2, s.end(), greater<char>()));
    assert(s.substr(0, 2) == "42");
}
{ // Combinations of distinct integers in [0, n).
    int n = 5, k = 3;
    vector<int> a{0, 1, 2};
    cout << endl << n << " choose " << k << ":" << endl;
    int count = 0;
    do {
        print_range(a.begin(), a.end());
        assert(a == combination_by_rank(n, k, count));
        assert(rank_by_combination(n, a) == count);
        count++;
    } while (next_combination(n, a));
    assert(count == 10);
    assert(n_choose_k(66, 33) == 7219428434016265740LL);
    cout << endl;
}
{ // Combinations with repeats.
    int n = 3, k = 2;
    vector<int> a{0, 0};
    cout << endl << n << " multichoose " << k << ":" << endl;
    int count = 0;
    do {
        print_range(a.begin(), a.end());
        count++;
    } while (next_combination_with_repeats(n, a));
    assert(count == 6);
    cout << endl;
}
return 0;
}

```

### Example Output

"11234" choose 3:

112 113 114 123 124 134 234

"abcde" choose 3 with masks:

abc abd acd bcd abe ace bce ade bde cde

5 choose 3:

{0,1,2} {0,1,3} {0,1,4} {0,2,3} {0,2,4} {0,3,4} {1,2,3} {1,2,4} {1,3,4} {2,3,4}

3 multichoose 2:

{0,0} {0,1} {0,2} {1,1} {1,2} {2,2}

## 6.2.5 Enumerating Partitions

A partition of a natural number  $n$  is a way to write  $n$  as a sum of positive integers where the order of the addends does not matter.

The lexicographic successor edits the suffix of a non-increasing partition: increase the rightmost part that can grow without breaking the order, then refill the remaining sum with ones. Ranking and unranking use a dynamic-programming table that counts partitions of a remaining sum by their next part. This lets the algorithms skip whole blocks of partitions sharing a common prefix.

- `next_partition(p)` takes a reference to a vector `p` of positive integers as a partition of  $n$  for which the function will re-assign to become the next lexicographically greater partition. The function returns true if such a partition exists, or false if `p` already consists of the lexicographically greatest partition (i.e. the single integer  $n$ ).
- `partition_by_rank(n, r)` returns the partition of  $n$  that is lexicographically ranked  $r$  if addends in each partition were sorted in non-increasing order, where  $r$  is a 0-based rank in the range  $[0, p(n))$  where  $p(n)$  is the number of partitions of  $n$ .
- `rank_by_partition(p)` returns an integer representing the 0-based rank of the partition specified by vector `p`, which must consist of positive integers sorted in non-increasing order.
- `generate_increasing_partitions(n, f)` calls the function `f(lo, hi)` on strictly increasing partitions of  $n$  in lexicographically increasing order of partition, where `lo` and `hi` are random-access iterators to a range  $[lo, hi)$  of integers. Note that non-strictly increasing partitions like  $\{1, 1, 1, 1\}$  are skipped.

**Time Complexity:**

- $O(n)$  per call to `next_partition()`.
- $O(n^2)$  per call to `partition_by_rank()` and `rank_by_partition()`.
- $O(p(n))$  per call to `generate_increasing_partitions()`, where  $p(n)$  is the number of partitions of  $n$ .

**Space Complexity:**

- $O(1)$  auxiliary for `next_partition()`.
- $O(n^2)$  auxiliary heap space for `partition_function()`, `partition_by_rank()`, and `rank_by_partition()`.
- $O(n)$  auxiliary stack space for `generate_increasing_partitions()`.

```
#include <stdint>
#include <numeric>
#include <vector>

bool next_partition(std::vector<int> &p) {
    int n = static_cast<int>(p.size());
    if (n <= 1) {
        return false;
    }
    int s = p[n - 1] - 1, i = n - 2;
    p.pop_back();
    for (; i > 0 && p[i] == p[i - 1]; i--) {
        s += p[i];
        p.pop_back();
    }
    for (p[i]++; s > 0; s--) {
        p.push_back(1);
    }
    return true;
}

int64_t partition_function(int a, int b) {
    static std::vector<std::vector<int64_t>> p(1, std::vector<int64_t>(1, 1));
    if (a >= static_cast<int>(p.size())) {
        int old = static_cast<int>(p.size());
        p.resize(a + 1);
        for (int i = 0; i < old; i++) {
            p[i].resize(a + 1);
        }
    }
}
```

```

    }
    for (int i = old; i <= a; i++) {
        p[i].resize(a + 1);
        for (int j = 1; j <= i; j++) {
            p[i][j] = p[i - 1][j - 1] + p[i - j][j];
        }
    }
}
return p[a][b];
}

std::vector<int> partition_by_rank(int n, int64_t r) {
    std::vector<int> res;
    for (int i = n, j; i > 0; i -= j) {
        for (j = 1;; j++) {
            int64_t count = partition_function(i, j);
            if (r < count) {
                break;
            }
            r -= count;
        }
        res.push_back(j);
    }
    return res;
}

int64_t rank_by_partition(const std::vector<int> &p) {
    int64_t res = 0;
    int sum = std::accumulate(p.begin(), p.end(), 0);
    for (int x : p) {
        for (int j = 0; j < x; j++) {
            res += partition_function(sum, j);
        }
        sum -= x;
    }
    return res;
}

template<typename Fn>
void generate_increasing_partitions(int left, int prev, int i, std::vector<int> &p, Fn f) {
    if (left == 0) {
        f(p.begin(), p.begin() + i);
        return;
    }
    for (p[i] = prev + 1; p[i] <= left; p[i]++) {
        generate_increasing_partitions(left - p[i], p[i], i + 1, p, f);
    }
}

template<typename Fn>
void generate_increasing_partitions(int n, Fn f) {
    std::vector<int> p(n);
    generate_increasing_partitions(n, 0, 0, p, f);
}

```

## Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : "," );
    }
}

```

```

    }
    cout << "} ";
}

int main() {
    // Exercise incremental growth of the cached dynamic-programming table.
    assert(partition_function(3, 1) == 1);
    assert(partition_function(8, 3) == 5);
    {
        int n = 4;
        vector<int> a(n, 1);
        cout << "Partitions of " << n << ":" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            assert(a == partition_by_rank(n, count));
            assert(rank_by_partition(a) == count);
            count++;
        } while (next_partition(a));
        assert(count == 5);
        cout << endl;
    }
    {
        int n = 8;
        cout << "\nIncreasing partitions of " << n << ":" << endl;
        int count = 0;
        generate_increasing_partitions(n, [&](auto lo, auto hi) {
            print_range(lo, hi);
            count++;
        });
        assert(count == 6);
        cout << endl;
    }
    return 0;
}

```

### Example Output

```

Partitions of 4:
{1,1,1,1} {2,1,1} {2,2} {3,1} {4}

Increasing partitions of 8:
{1,2,5} {1,3,4} {1,7} {2,6} {3,5} {8}

```

## 6.2.6 Generic Combinatorial Enumeration

Ranks, unrank, and enumerates combinatorial sequences in lexicographic order using only a prefix counting oracle. This is useful when the objects are too numerous to precompute, but there is a simple dynamic-programming formula for the number of valid completions after a fixed prefix.

`Enumerator(range, length, count)` considers values in  $[0, \text{range})$  at each position of a length-`length` sequence. The callback `count(prefix)` returns the number of valid full sequences beginning with `prefix`; it may capture a precomputed dynamic-programming table or other state. The factory functions below supply this oracle for the principal combinatorial families, while the class remains available directly for custom families. The range and length must be nonnegative; the factory arguments must satisfy  $0 \leq k \leq n$ .

- `arrangements(n, k)` returns an enumerator for the length- $k$  arrangements of  $[0, n)$ .
- `permutations(n)` returns an enumerator for the permutations of  $[0, n)$ .
- `combinations(n, k)` returns an enumerator for the size- $k$  subsets of  $[0, n)$ , represented as strictly increasing sequences.
- `partitions(n)` returns an enumerator for the additive partitions of  $n$ , represented as non-increasing length- $n$  sequences padded with zeros.
- `to_rank(a)` returns the 0-based rank of the valid combinatorial sequence `a`.



- `from_rank(r)` returns a combinatorial sequence of integers that is lexicographically ranked `r`, where `r` is a 0-based rank in the range `[0, total_count())`.
- `total_count()` returns the number of valid sequences.
- `enumerate(f)` calls the function `f(a)` on every specified combinatorial sequence `a` in lexicographically increasing order.

Overflow warning: All exact counts and ranks must fit in `int64_t`.

#### Time Complexity:

- $O(A \cdot L \cdot C)$  per call to `to_rank()` and `from_rank()`, where  $A$  is `range`,  $L$  is `length`, and  $C$  is the cost of `count()`.
- $O(T \cdot A \cdot L \cdot C)$  per call to `enumerate()`, excluding the cost of the callback `f`, where  $T$  is `total_count()`.
- $O(A \cdot (\min(L, A - L) + 1))$  per call to `combinations()` and  $O(L^2)$  per call to `partitions()`; the other factories take  $O(1)$  time per call.

#### Space Complexity:

- $O(L)$  auxiliary for `to_rank()`, `from_rank()`, and `enumerate()`, excluding output storage.
- $O(S)$  object storage, where  $S$  is the state captured by `count()`.
- $O(A \cdot (\min(L, A - L) + 1))$  object storage for `combinations()` and  $O(L^2)$  for `partitions()`.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <numeric>
#include <utility>
#include <vector>

template<typename Count>
class Enumerator {
    int range, length;
    Count count;

public:
    Enumerator(int range, int length, Count count)
        : range(range), length(length), count(std::move(count)) {
        assert(range >= 0 && length >= 0);
    }

    int64_t total_count() { return count({}); }

    int64_t to_rank(const std::vector<int> &a) {
        assert(static_cast<int>(a.size()) == length);
        assert(std::all_of(a.begin(), a.end(), [&](int x) { return 0 <= x && x < range; }));
        int64_t res = 0;
        std::vector<int> prefix;
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            prefix.push_back(0);
            for (; prefix.back() < a[i]; prefix.back()++) {
                res += count(prefix);
            }
        }
        return res;
    }

    std::vector<int> from_rank(int64_t r) {
        assert(0 <= r && r < total_count());
        std::vector<int> a;
        for (int i = 0; i < length; i++) {
            a.push_back(0);
            for (; a.back() < range; a.back()++) {
                int64_t curr = count(a);
                if (r < curr) {
                    break;
                }
            }
        }
    }
};
```

```

        r -= curr;
    }
}
return a;
}

// Accepts any callable f(a), including capturing lambdas and functors.
template<typename Fn>
void enumerate(Fn f) {
    int64_t total = total_count();
    for (int64_t i = 0; i < total; i++) {
        std::vector<int> curr = from_rank(i);
        f(curr);
    }
}
};

auto arrangements(int n, int k) {
    assert(n >= 0 && 0 <= k && k <= n);
    auto count = [=](const std::vector<int> &prefix) -> int64_t {
        int len = static_cast<int>(prefix.size());
        for (int i = 0; i < len - 1; i++) {
            if (prefix[i] == prefix[len - 1]) {
                return 0;
            }
        }
        int64_t res = 1;
        for (int i = 0; i < k - len; i++) {
            res *= n - len - i; // Overflow warning.
        }
        return res;
    };
    return Enumerator(n, k, count);
}

auto permutations(int n) {
    return arrangements(n, n);
}

auto combinations(int n, int k) {
    assert(n >= 0 && 0 <= k && k <= n);
    std::vector<std::vector<int64_t>> table(n + 1, std::vector<int64_t>(std::min(k, n - k) + 1));
    int max_col = static_cast<int>(table[0].size()) - 1;
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= std::min(i, max_col); j++) {
            table[i][j] = (j == 0) ? 1 : table[i - 1][j - 1] + table[i - 1][j]; // Overflow warning.
        }
    }
    auto count = [n, k, table = std::move(table)](const std::vector<int> &pref) -> int64_t {
        int len = static_cast<int>(pref.size());
        if (len >= 2 && pref[len - 1] <= pref[len - 2]) {
            return 0;
        }
        int rem = len == 0 ? n : n - pref.back() - 1;
        int choose = k - len;
        return (choose < 0 || choose > rem) ? 0 : table[rem][std::min(choose, rem - choose)];
    };
    return Enumerator(n, k, std::move(count));
}

auto partitions(int n) {
    assert(n >= 0);
    std::vector<std::vector<int64_t>> table(n + 1, std::vector<int64_t>(n + 1));
    std::fill(table[0].begin(), table[0].end(), 1);
    for (int sum = 1; sum <= n; sum++) {
        for (int max_part = 1; max_part <= n; max_part++) {
            table[sum][max_part] = table[sum][max_part - 1];
            if (max_part <= sum) {

```

```

        table[sum][max_part] += table[sum - max_part][max_part]; // Overflow warning.
    }
}
}
auto count = [n, table = std::move(table)](const std::vector<int> &pref) -> int64_t {
    int len = static_cast<int>(pref.size());
    int sum = std::accumulate(pref.begin(), pref.end(), 0);
    if (sum == n) {
        return 1;
    }
    if (sum > n || (len > 0 && pref.back() == 0) || (len >= 2 && pref[len - 1] > pref[len - 2])) {
        return 0;
    }
    return table[n - sum][len == 0 ? n : pref.back()];
};
return Enumerator(n + 1, n, std::move(count));
}

```

## Example Usage

```

#include <iostream>
using namespace std;

void print_sequence(const vector<int> &a) {
    cout << "{";
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        cout << a[i] << (i + 1 == static_cast<int>(a.size()) ? "" : "," );
    }
    cout << "} ";
}

int main() {
    cout << "3 permute 2 arrangements:" << endl;
    auto arr = arrangements(3, 2);
    int count = 0;
    arr.enumerate([&](const auto &a) {
        print_sequence(a);
        count++;
    });
    assert(count == 6);
    assert((arr.from_rank(5) == vector<int>{2, 1}));
    assert(arr.to_rank(vector<int>{2, 1}) == 5);

    cout << "\n\nPermutations of [0, 3):" << endl;
    auto perm = permutations(3);
    count = 0;
    perm.enumerate([&](const auto &a) {
        print_sequence(a);
        count++;
    });
    assert(count == 6);

    cout << "\n\n4 choose 3 combinations:" << endl;
    auto comb = combinations(4, 3);
    count = 0;
    comb.enumerate([&](const auto &a) {
        print_sequence(a);
        count++;
    });
    assert(count == 4);
    assert((comb.from_rank(3) == vector<int>{1, 2, 3}));
    assert(comb.to_rank(vector<int>{1, 2, 3}) == 3);
    assert(combinations(67, 1).total_count() == 67);
    assert(combinations(67, 67).total_count() == 1);
    assert(combinations(66, 33).total_count() == 7219428434016265740LL);
}

```

```

cout << "\n\nPartitions of 4:" << endl;
auto part = partitions(4);
count = 0;
part.enumerate([&](const auto &a) {
    print_sequence(a);
    count++;
});
assert(count == 5);

int length = 4;
auto count_completions = [length](const vector<int> &pref) -> int64_t {
    if (static_cast<int>(pref.size()) > length) {
        return 0;
    }
    for (int i = 0; i < static_cast<int>(pref.size()); i++) {
        if (pref[i] < 0 || pref[i] > 1 || (i > 0 && pref[i - 1] == 1 && pref[i] == 1)) {
            return 0;
        }
    }
    int64_t after_zero = 1, after_one = 1;
    for (int i = static_cast<int>(pref.size()); i < length; i++) {
        int64_t next_zero = after_zero + after_one; // Overflow warning.
        after_one = after_zero;
        after_zero = next_zero;
    }
    return pref.empty() || pref.back() == 0 ? after_zero : after_one;
};
Enumerator e(2, length, count_completions);
cout << "\n\nLength-4 binary strings without consecutive ones:" << endl;
count = 0;
e.enumerate([&](const auto &a) {
    print_sequence(a);
    count++;
});
assert(count == 8 && e.total_count() == 8);
assert((e.from_rank(7) == vector<int>{1, 0, 1, 0}));
assert(e.to_rank(vector<int>{1, 0, 1, 0}) == 7);
cout << endl;
return 0;
}

```

### Example Output

```

3 permute 2 arrangements:
{0,1} {0,2} {1,0} {1,2} {2,0} {2,1}

Permutations of [0, 3]:
{0,1,2} {0,2,1} {1,0,2} {1,2,0} {2,0,1} {2,1,0}

4 choose 3 combinations:
{0,1,2} {0,1,3} {0,2,3} {1,2,3}

Partitions of 4:
{1,1,1,1} {2,1,1,0} {2,2,0,0} {3,1,0,0} {4,0,0,0}

Length-4 binary strings without consecutive ones:
{0,0,0,0} {0,0,0,1} {0,0,1,0} {0,1,0,0} {0,1,0,1} {1,0,0,0} {1,0,0,1} {1,0,1,0}

```

## 6.3 Number Theory

### 6.3.1 Mod Helpers and Binary Exponentiation

Modular arithmetic replaces every value by its remainder modulo `m`, which keeps intermediate results bounded and is what makes counting problems with astronomically large answers tractable. The operations are straightforward except for two hazards, both handled below: a sum or product can overflow the underlying integer type before the remainder is taken, and a negative operand leaves a negative remainder under C++ truncating division. Binary exponentiation squares the base while walking the bits of the exponent, so a power costs  $O(\log n)$  multiplications rather than  $n$ ; the same loop computes powers in any monoid, which is how the matrix exponentiation of section 6.5.1 and the linear recurrence of section 6.6.6 are evaluated.

- `addmod(a, b, m)` and `submod(a, b, m)` respectively return addition and subtraction modulo `m`, each result in  $[0, m)$ . Both operands may be negative or unreduced; they are normalized before arithmetic to avoid signed overflow. The modulus `m` must be positive.
- `mulmod(a, b, m)` returns `a` multiplied by `b`, modulo `m`. This is done in a way to avoid overflow: on compilers with `__uint128_t` it uses one wide product, while the portable fallback uses double-and-add multiplication. The fallback is slower by a logarithmic factor, but avoids relying on nonstandard 128-bit integers. Unlike `addmod/submod`, this takes unsigned operands and supports a full 64-bit modulus.
- `powmod(x, n, m)` returns `x` raised to the power `n`, modulo `m`.

Paste these when a solution needs a handful of modular operations; when modular arithmetic pervades one instead, the `Modular` value type of section 6.3.3 overloads the operators so expressions read normally. Only `mulmod()` has no counterpart there, since a value type multiplying in `int64_t` overflows once the modulus passes about three billion, which is why it underlies the Miller-Rabin test and Pollard's rho of section 6.3.9.

#### Time Complexity:

- $O(1)$  per call to `addmod()` and `submod()`.
- $O(1)$  per call to `mulmod()` with `__uint128_t`, or  $O(\log b)$  with the portable fallback, where  $b$  is the second argument.
- $O(\log n)$  calls to `mulmod()` per call to `powmod(x, n, m)`.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cassert>
#include <cstdint>

int64_t addmod(int64_t a, int64_t b, int64_t m) {
    assert(m > 0);
    if ((a %= m) < 0) a += m;
    if ((b %= m) < 0) b += m;
    return a >= m - b ? a - (m - b) : a + b;
}

int64_t submod(int64_t a, int64_t b, int64_t m) {
    assert(m > 0);
    if ((a %= m) < 0) a += m;
    if ((b %= m) < 0) b += m;
    return a >= b ? a - b : m - (b - a);
}

uint64_t mulmod(uint64_t a, uint64_t b, uint64_t m) {
    assert(m > 0);
    #if defined(__SIZEOF_INT128__)
        return static_cast<uint64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        uint64_t res = 0, cur = a % m;
        for (; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - cur ? res - (m - cur) : res + cur;
            }
        }
    #endif
}
```

```

    cur = cur >= m - cur ? cur - (m - cur) : cur + cur;
}
return res;
#endif
}

uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
    assert(m > 0);
    uint64_t res = 1 % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, x, m);
        }
        x = mulmod(x, x, m);
    }
    return res;
}

```

### Example Usage

```

#include <cassert>

int main() {
    assert(addmod(7, 8, 10) == 5 && submod(2, 5, 10) == 7);
    // Negative and unreduced operands are normalized into [0, m).
    assert(addmod(-3, -4, 10) == 3 && submod(-3, 4, 10) == 3);
    assert(addmod(25, -7, 10) == 8);
    assert(addmod(INT64_MAX - 1, INT64_MAX - 1, INT64_MAX) == INT64_MAX - 2);

    assert(mulmod(INT64_MAX - 1, INT64_MAX - 1, INT64_MAX) == 1);
    assert(powmod(2, 10, 1000000007) == 1024);
    assert(powmod(2, 62, 1000000) == 387904);
    assert(powmod(10001, 10001, 100000) == 10001);
    return 0;
}

```

### 6.3.2 GCD, LCM, Mod Inverse, Chinese Remainder

Core integer and modular-arithmetic operations built around the Euclidean algorithm. Extended GCD produces Bezout coefficients, which solve linear Diophantine equations and modular inverses, while the Chinese remainder routines combine compatible congruences into one residue class.

- `gcd(a, b)` returns the greatest common divisor of `a` and `b` using the Euclidean algorithm. This is mainly for educational purposes, as `std::gcd(a, b)` from `<numeric>` is available as of C++17 (`__gcd(a, b)` from `<algorithm>` in C++14 and earlier).
- `lcm(a, b)` returns the least common multiple of `a` and `b`. This implementation is mainly for educational purposes, as `std::lcm(a, b)` from `<numeric>` is available as of C++17.
- `extended_gcd(a, b)` returns a pair  $(x, y)$  of integers such that  $\text{gcd}(a, b) = ax + by$ .
- `diophantine(a, b, c, &g, &x, &y)` solves the linear Diophantine equation  $ax + by = c$ , returning whether a solution exists (one does if and only if  $\text{gcd}(a, b)$  divides  $c$ ). `g` is always set to  $\text{gcd}(a, b)$ , while  $(x, y)$  is set only on success to a particular solution bounded by  $\max(|a|, |b|, |c|)$  in magnitude. Every other solution is  $(x + t(b/g), y - t(a/g))$  for an integer  $t$ . A 128-bit intermediate is used where available to keep the scaling step overflow-free.
- `mod(a, m)` returns the least nonnegative residue of `a` modulo `m`, that is, the unique value in  $[0, m)$  congruent to `a`, where `m` must be positive. Unlike the C++ remainder operator `%`, whose result follows the sign of `a`, the result is never negative.
- `mod_inverse(a, m)` returns an integer  $x$  such that  $ax \equiv 1 \pmod{m}$ , where the arguments must satisfy  $m > 0$  and  $\text{gcd}(a, m) = 1$ .

- `mod_inverse_table(p)` returns a vector `v` where `v[i]` is the modular inverse of `i` modulo the prime `p` for every valid index `i`.
- `crt(r1, m1, r2, m2, &r, &m)` merges the two congruences  $x \equiv r_1 \pmod{m_1}$  and  $x \equiv r_2 \pmod{m_2}$  for arbitrary moduli (not necessarily coprime). It returns whether the system is consistent, and on success sets `m` to `lcm(m1, m2)` and `r` to the unique solution in  $[0, m)$ . Both moduli must be positive, and their least common multiple must fit in `int64_t`. Fold it pairwise to merge more than two congruences.
- `garner_restore(a, p)` returns the smallest nonnegative solution whose residue modulo `p[i]` is `a[i]` at every valid index. The entries of `p` must be pairwise coprime (unlike `crt`, which allows shared factors) and positive, and `a` and `p` must have equal sizes. The exact solution is unique modulo the product of all moduli, so that product and the final answer must fit in `int64_t`.
- `garner_restore_mod(a, p, m)` returns that same CRT solution modulo `m`. This is the right variant when the product of the moduli is too large to fit in `int64_t`; `m` must be positive.

Overflow warning: Every intermediate and result of the templated signed-integer helpers must be representable by `Int`; in particular, the minimum value of `Int` cannot be negated or divided by `-1`.

#### Time Complexity:

- $O(\log M)$  per call to `gcd()`, `lcm()`, `extended_gcd()`, `diophantine()`, `mod_inverse()`, and `crt()`, where  $M$  is the largest relevant input magnitude.
- $O(1)$  per call to `mod()`.
- $O(p)$  per call to `mod_inverse_table()`.
- $O(n^2)$  per call to `garner_restore()` and `garner_restore_mod()`.

#### Space Complexity:

- $O(p)$  auxiliary for `mod_inverse_table()`.
- $O(n)$  auxiliary for `garner_restore()` and `garner_restore_mod()`.
- $O(1)$  auxiliary for all other operations.

```
#include <cassert>
#include <cstdint>
#include <type_traits>
#include <utility>
#include <vector>

template<typename Int>
Int gcd(Int a, Int b) {
    while (b != 0) {
        Int t = b;
        b = a % b;
        a = t;
    }
    return (a < 0 ? -a : a);
}

template<typename Int>
Int lcm(Int a, Int b) {
    if (a == 0 || b == 0) {
        return 0;
    }
    Int res = a / gcd(a, b) * b;
    return (res < 0 ? -res : res);
}

template<typename Int>
std::pair<Int, Int> extended_gcd(Int a, Int b) {
    Int x = 1, y = 0, x1 = 0, y1 = 1;
    while (b != 0) {
        Int q = a / b, prev_x1 = x1, prev_y1 = y1, prev_b = b;
        x1 = x - q * x1;
        y1 = y - q * y1;
        b = a - q * b;
        x = prev_x1;
```

```

    y = prev_y1;
    a = prev_b;
}
return (a > 0) ? std::pair<Int, Int>{x, y} : std::pair<Int, Int>{-x, -y};
}

template<typename Int>
bool diophantine(Int a, Int b, Int c, Int *g, Int *x, Int *y) {
    if (a == 0 && b == 0) {
        *g = *x = *y = 0;
        return c == 0;
    }
    if (a == 0) {
        *g = (b < 0) ? -b : b;
        if (c % b != 0) {
            return false;
        }
        *x = 0;
        *y = c / b;
        return true;
    }
    if (b == 0) {
        *g = (a < 0) ? -a : a;
        if (c % a != 0) {
            return false;
        }
        *x = c / a;
        *y = 0;
        return true;
    }
    *g = gcd(a, b);
    if (c % *g != 0) {
        return false;
    }
    // Absorb the bulk of c into a*dx + b*dy, then scale the base solution by the small remainder so
    // the reported (x, y) stay bounded by max(|a|, |b|, |c|). The scaled product is taken modulo b
    // (resp. a) through a 128-bit intermediate where available, to avoid overflow.
    std::pair<Int, Int> base = extended_gcd(a, b);
    Int dx = c / a;
    c -= dx * a;
    Int dy = c / b;
    c -= dy * b;
    Int f = c / *g;
#ifdef __SIZEOF_INT128__
    __extension__ typedef std::conditional_t<sizeof(Int) <= 4, int64_t, __int128> wide_t;
#else
    using wide_t = int64_t;
#endif
    *x = dx + static_cast<Int>(static_cast<wide_t>(base.first) * f % b);
    *y = dy + static_cast<Int>(static_cast<wide_t>(base.second) * f % a);
    return true;
}

template<typename Int>
Int mod(Int a, Int m) {
    assert(m > 0);
    Int r = a % m;
    return (r < 0) ? (r + m) : r;
}

template<typename Int>
Int mod_inverse(Int a, Int m) {
    return mod(extended_gcd(a, m).first, m);
}

std::vector<int> mod_inverse_table(int p) {
    std::vector<int> res(p);
    res[1] = 1;

```



```

    for (int i = 2; i < p; i++) {
        res[i] = (p - (p / i) * res[p % i] % p) % p;
    }
    return res;
}

bool crt(int64_t r1, int64_t m1, int64_t r2, int64_t m2, int64_t *r, int64_t *m) {
    r1 = mod(r1, m1);
    r2 = mod(r2, m2);
    int64_t g, x, y;
    if (!diophantine(m1, -m2, r2 - r1, &g, &x, &y)) {
        return false;
    }
    *m = m1 / g * m2;
#ifdef __SIZEOF_INT128__
    __extension__ typedef __int128 int128_t;
    *r = static_cast<int64_t>(
        (static_cast<int128_t>(r1) + static_cast<int128_t>(m1) * mod(x, m2 / g)) % *m
    );
#else
    *r = mod(r1 + m1 * mod(x, m2 / g), *m); // Overflow warning.
#endif
    return true;
}

std::vector<int64_t> garner_digits(const std::vector<int> &a, const std::vector<int> &p) {
    assert(a.size() == p.size());
    int n = static_cast<int>(a.size());
    std::vector<int64_t> x(n);
    for (int i = 0; i < n; i++) {
        assert(p[i] > 0);
        x[i] = mod(static_cast<int64_t>(a[i]), static_cast<int64_t>(p[i]));
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            // Reduce mod p[i] each step; otherwise x[i] compounds to ~p^i and overflows int64_t.
            x[i] = mod_inverse(static_cast<int64_t>(p[j]), static_cast<int64_t>(p[i])) * (x[i] - x[j]);
            x[i] = (x[i] % p[i] + p[i]) % p[i];
        }
    }
    return x;
}

int64_t garner_restore(const std::vector<int> &a, const std::vector<int> &p) {
    assert(a.size() == p.size());
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return 0;
    }
    std::vector<int64_t> x = garner_digits(a, p);
    int64_t res = x[0], m = 1;
    for (int i = 1; i < n; i++) {
        m *= p[i - 1];
        res += x[i] * m;
    }
    return res;
}

int64_t garner_restore_mod(const std::vector<int> &a, const std::vector<int> &p, int64_t m) {
    assert(a.size() == p.size() && m > 0);
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return 0;
    }
    std::vector<int64_t> x = garner_digits(a, p);
    int64_t res = 0;
    for (int i = n - 1; i >= 0; i--) {
#ifdef __SIZEOF_INT128__

```

```

__extension__ typedef __int128 int128_t;
res = static_cast<int64_t>((static_cast<int128_t>(res) * p[i] + x[i]) % m);
#else
res = (res * p[i] + x[i]) % m; // Requires the intermediate product to fit without int128.
#endif
}
return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(mod(-5, 3) == 1); // Negative dividends wrap up into [0, m).
    assert(mod(5, 3) == 2);
    assert(mod(-6, 3) == 0);
    {
        for (int a = -20; a <= 20; a++) {
            for (int b = -20; b <= 20; b++) {
                int g = gcd(a, b);
                auto [x, y] = extended_gcd(a, b);
                assert(g == a * x + b * y);
                if (g == 1 && b > 1) {
                    assert(mod(a * mod_inverse(a, b), b) == 1);
                }
            }
        }
    }
    {
        int p = 17;
        auto res = mod_inverse_table(p);
        for (int i = 0; i < p; i++) {
            if (i > 0) {
                assert(mod(i * res[i], p) == 1);
            }
        }
    }
    {
        vector<int> a{2, 3, 1}, m{3, 4, 5};
        int x = garner_restore(a, m);
        assert(x == garner_restore_mod(a, m, 1000000007));
        int n = static_cast<int>(a.size());
        for (int i = 0; i < n; i++) {
            assert(mod(x, m[i]) == a[i]);
        }
        assert(x == 11);
    }
    assert(garner_restore(vector<int>{-1}, vector<int>{5}) == 4);
#ifdef __SIZEOF_INT128__
    { // Garner modulo another number works even when the product of CRT moduli is too large.
        vector<int> p{1000000007, 1000000009, 1000000033};
        __extension__ typedef __int128 int128_t;
        int128_t x = (int128_t{1} << 80) + 123456789;
        vector<int> a;
        for (int m : p) {
            a.push_back(static_cast<int>(x % m));
        }
        int64_t m = 998244353;
        assert(garner_restore_mod(a, p, m) == static_cast<int64_t>(x % m));
    }
#endif
    { // Diophantine: exhaustively verify solvability and that solutions satisfy a*x + b*y == c.
        for (int a = -8; a <= 8; a++) {
            for (int b = -8; b <= 8; b++) {

```

```

    for (int c = -8; c <= 8; c++) {
        int g, x, y, gg = gcd(a, b);
        bool ok = diophantine(a, b, c, &g, &x, &y);
        assert(ok == (gg == 0 ? c == 0 : c % gg == 0));
        assert(g == gg); // g is set even when no solution exists.
        if (ok) {
            assert(a * x + b * y == c);
        }
    }
}
}
}
{ // CRT: merge two congruences, including non-coprime and inconsistent moduli.
    for (int m1 = 1; m1 <= 12; m1++) {
        for (int m2 = 1; m2 <= 12; m2++) {
            for (int r1 = 0; r1 < m1; r1++) {
                for (int r2 = 0; r2 < m2; r2++) {
                    int64_t r, m;
                    bool ok = crt(r1, m1, r2, m2, &r, &m);
                    int limit = lcm(m1, m2), want = -1;
                    for (int v = 0; v < limit && want < 0; v++) {
                        if (v % m1 == r1 && v % m2 == r2) {
                            want = v;
                        }
                    }
                    assert(ok == (want >= 0));
                    if (ok) {
                        assert(m == limit && r == want);
                    }
                }
            }
        }
    }
}
}
return 0;
}

```

### 6.3.3 Modular Integer and Combinatorics

Provides a modular integer value type together with lazy factorial-based combinatorics tables. Both are common contest helpers for dynamic programming, counting, polynomial operations, and any calculation whose answers are taken modulo some number such as  $10^9 + 7$ . Where the helpers of section 6.3.1 suit a few scattered operations, a value type is what makes an entire solution read in ordinary arithmetic notation while every operation reduces silently.

The modulus is a template parameter rather than a member, which is a deliberate speed choice: a compile-time constant lets the compiler replace each division by a multiplication and a shift, while a modulus only known at run time forces a hardware divide costing several times as much. Storing it in a `static` member instead makes the same class work with a runtime modulus at that price, and is worth doing when the modulus is read from the input. A modulus above roughly  $3 \times 10^9$  needs the `mulmod()` of section 6.3.1 in place of the plain products used here.

The `Modular<MOD>` value type wraps arithmetic modulo the positive compile-time constant `MOD`. Its signed `auto` argument determines the storage type: `Modular<1000000007>` uses 32-bit storage, while `Modular<(1LL << 61) - 1>` uses 64-bit storage. The implementation follows the conventional `Mint` wrapper: construction normalizes values, while hidden friend operators support mixed expressions such as `2 + Mint(3)` through implicit conversion.

- `Modular<MOD>(x = 0)` constructs the residue class of integer `x` modulo `MOD`.
- `value()` and `operator()()` return the stored representative in  $[0, \text{MOD})$ .
- Explicit casts to `int`, `long long`, `double`, and `long double` convert that stored representative to the corresponding primitive.

- `pow(n)` returns this value raised to nonnegative integer exponent `n`.
- `inv()` returns the multiplicative inverse, asserting the value is coprime to `MOD`.
- Operators `+`, `-`, `*`, `/`, comparison, increment, decrement, and stream I/O are overloaded. Division likewise requires the divisor to be coprime to `MOD`.

Overflow warning: Construction, multiplication, and inverses widen through `int64_t` for 32-bit storage or `__int128` for 64-bit storage. Thus  $2p$  must fit the storage type and  $(p-1)^2$  must fit the intermediate type. On compilers without `__int128`, only 32-bit moduli are supported. Estimating a 64-bit product with `long double` is not used because it silently loses the needed precision on platforms where `long double` is only 64 bits.

#### Time Complexity:

- $O(1)$  per addition, subtraction, multiplication, comparison, and stream output.
- $O(\log n)$  per call to `pow()`.
- $O(\log m)$  per call to `inv()` and division, where  $m = \text{MOD}$ .

**Space Complexity:**  $O(1)$  auxiliary for Modular arithmetic.

```
#include <cassert>
#include <stdint>
#include <iostream>
#include <limits>
#include <type_traits>
#include <utility>
#include <vector>

template<typename T>
T inverse(T a, T m) {
    T original_m = m;
    T u = 0, v = 1;
    while (a != 0) {
        T q = m / a;
        m -= q * a;
        std::swap(a, m);
        u -= q * v;
        std::swap(u, v);
    }
    assert(m == 1);
    u %= original_m;
    return u < 0 ? u + original_m : u;
}

template<auto MOD>
class Modular {
    using T = decltype(MOD);
    static_assert(std::is_integral_v<T> && std::is_signed_v<T>);
    static_assert(MOD > 0 && MOD <= std::numeric_limits<T>::max() / 2);

    // Widen through int64_t for 32-bit storage, or __int128 for 64-bit storage where it exists
    // (__extension__ silences -pedantic). Without __int128, only 32-bit moduli are supported.
    #if defined(__SIZEOF_INT128__)
        __extension__ typedef std::conditional_t<sizeof(T) <= 4, int64_t, __int128> wide_t;
    #else
        using wide_t = int64_t;
        static_assert(sizeof(T) <= 4, "64-bit moduli require __int128, which is unavailable");
    #endif

    T v;

    static T normalize(wide_t x) {
        if (-static_cast<wide_t>(mod()) <= x && x < static_cast<wide_t>(mod())) {
            T y = static_cast<T>(x);
            return y < 0 ? y + mod() : y;
        }
        x %= mod();
        if (x < 0) {
```

```

    x += mod();
}
return static_cast<T>(x);
}

public:
Modular(wide_t x = 0) : v(normalize(x)) {}

static T mod() { return MOD; }
T value() const { return v; }
T operator()() const { return v; }
explicit operator int() const { return static_cast<int>(v); }
explicit operator long long() const { return static_cast<long long>(v); }
explicit operator double() const { return static_cast<double>(v); }
explicit operator long double() const { return static_cast<long double>(v); }

Modular pow(int64_t n) const {
    assert(n >= 0);
    Modular base = *this, res = 1;
    while (n > 0) {
        if (n & 1) {
            res *= base;
        }
        base *= base;
        n >>= 1;
    }
    return res;
}

Modular inv() const {
    assert(v != 0);
    return Modular(inverse(static_cast<wide_t>(v), static_cast<wide_t>(mod())));
}

Modular &operator+=(const Modular &other) {
    v += other.v;
    if (v >= mod()) {
        v -= mod();
    }
    return *this;
}

Modular &operator-=(const Modular &other) {
    v -= other.v;
    if (v < 0) {
        v += mod();
    }
    return *this;
}

Modular &operator*=(const Modular &other) {
    v = normalize(static_cast<wide_t>(v) * other.v);
    return *this;
}

Modular operator-() const { return Modular(-v); }
Modular &operator++() { return *this += 1; }
Modular &operator--() { return *this -= 1; }
Modular &operator/=(const Modular &other) { return *this *= other.inv(); }
Modular operator++(int) { Modular res = *this; ++*this; return res; }
Modular operator--(int) { Modular res = *this; --*this; return res; }
friend Modular operator+(Modular a, const Modular &b) { return a += b; }
friend Modular operator-(Modular a, const Modular &b) { return a -= b; }
friend Modular operator*(Modular a, const Modular &b) { return a *= b; }
friend Modular operator/(Modular a, const Modular &b) { return a /= b; }
friend bool operator==(const Modular &a, const Modular &b) { return a.v == b.v; }
friend bool operator!=(const Modular &a, const Modular &b) { return !(a == b); }
friend bool operator<(const Modular &a, const Modular &b) { return a.v < b.v; }

```

```

friend std::ostream &operator<<(std::ostream &os, const Modular &x) { return os << x.v; }

friend std::istream &operator>>(std::istream &is, Modular &x) {
    int64_t value;
    is >> value;
    x = Modular(value);
    return is;
}
};

```

`ModCombinatorics<Mod>` maintains lazy factorial and inverse-factorial tables for a modular type `Mod`. It assumes the modulus is prime and all requested factorials are invertible. Growing the tables costs one modular inverse rather than one per entry: after extending the factorials by repeated multiplication, a single inversion of the largest factorial seeds a backward pass in which each inverse factorial follows from the next by one multiplication.

- `factorial(n)` returns  $n!$  using a lazy factorial table.
- `choose(n, k)` returns  $\binom{n}{k}$  using lazy factorial and inverse-factorial tables.
- `permute(n, k)` returns the number of ordered selections of  $k$  distinct elements from  $n$ .
- `multichoose(n, k)` returns the number of size- $k$  multisets drawn from  $n$  types.

Tables pay off once a solution asks for many values against one fixed modulus, which is the usual counting or dynamic programming case. For a handful of values, or for a modulus that changes between calls, the table-free routines of section 6.2.1 compute the same quantities in  $O(\min(k, n - k))$  per call and allocate nothing.

**Time Complexity:**  $O(n)$  total table growth to answer calls with arguments up to  $n$ , then  $O(1)$  per call.

**Space Complexity:**  $O(n)$  for the factorial and inverse-factorial tables.

```

template<typename Mod>
class ModCombinatorics {
    std::vector<Mod> fact, inv_fact;

    void ensure(int n) {
        auto old = static_cast<int>(fact.size());
        if (old > n) {
            return;
        }
        fact.resize(n + 1);
        inv_fact.resize(n + 1);
        for (int i = old; i <= n; i++) {
            fact[i] = fact[i - 1] * i;
        }
        inv_fact[n] = fact[n].inv();
        for (int i = n; i > old; i--) {
            inv_fact[i - 1] = inv_fact[i] * i;
        }
    }

public:
    ModCombinatorics() : fact(1, Mod{1}), inv_fact(1, Mod{1}) {}

    Mod factorial(int n) {
        ensure(n);
        return fact[n];
    }

    Mod choose(int n, int k) {
        if (k < 0 || k > n) {
            return 0;
        }
        ensure(n);
        return fact[n] * inv_fact[k] * inv_fact[n - k];
    }

    Mod permute(int n, int k) {

```

```

    if (k < 0 || k > n) {
        return 0;
    }
    ensure(n);
    return fact[n] * inv_fact[n - k];
}

Mint multichoose(int n, int k) { return n == 0 ? Mint(k == 0) : choose(n + k - 1, k); }
};

using Mint = Modular<1000000007>;

```

## Example Usage

```

#include <cassert>
#include <sstream>
using namespace std;

int main() {
    Mint a = 1000000008LL;
    Mint b = -2;
    assert(a.value() == 1);
    assert(b() == Mint::mod() - 2);
    assert(static_cast<int>(a) == 1);
    assert(static_cast<long long>(a) == 1LL);
    assert(static_cast<double>(a) == 1.0);
    assert(a + b == -1);
    assert(2 + Mint(3) == 5);
    assert(Mint(2) * 3 == 6);
    assert(Mint(2).pow(10) == 1024);
    assert(Mint(5) / 5 == 1);

    Mint x = 0;
    x++;
    ++x;
    assert(x == 2);
    stringstream ss("1000000008");
    ss >> x;
    assert(x == 1);

    ModCombinatorics<Mint> comb;
    assert(comb.factorial(5) == 120);
    assert(comb.choose(5, 2) == 10);
    assert(comb.permute(5, 2) == 20);
    assert(comb.multichoose(3, 2) == 6);
    assert(comb.multichoose(0, 0) == 1);

    // Each distinct modulus is just another instantiation.
    using Mint2 = Modular<998244353>;
    assert(Mint2(2).pow(10) == 1024);
    assert(Mint2(-1) == Mint2::mod() - 1);

    #if defined(__SIZEOF_INT128__)
    // A 64-bit modulus (deduced as long long) automatically widens through __int128.
    using Mint3 = Modular<(1LL << 61) - 1>;
    assert(Mint3(2).pow(62) == 2); // 2^62 mod (2^61 - 1)
    assert(Mint3(3).inv() * 3 == 1);
    #endif
    return 0;
}

```

### 6.3.4 Primitive Roots

A primitive root modulo  $m$  is a number  $g$  whose powers generate every invertible residue modulo  $m$ . Equivalently, the multiplicative order of  $g$  modulo  $m$  is exactly  $\phi(m)$ . Primitive roots are useful for constructing generators of finite multiplicative groups, choosing roots for number theoretic transforms, discrete logarithm setups, and randomized modular hashing or testing schemes that need a full-period multiplicative generator.

Primitive roots do not exist for every  $m$ . They exist exactly for  $m \in \{1, 2, 4, p^k, 2p^k\}$ , where  $p$  is an odd prime and  $k \geq 1$ . This implementation first checks that classification by factoring  $m$ , then factors  $\phi(m)$ . A candidate  $g$  is primitive exactly when  $g^{\phi(m)} \equiv 1 \pmod{m}$  and  $g^{\phi(m)/q} \not\equiv 1 \pmod{m}$  for every prime factor  $q$  of  $\phi(m)$ .

For a prime modulus  $p$ , this reduces to the common contest test: factor  $p - 1$ , then scan for the first  $g$  such that  $g^{(p-1)/q} \not\equiv 1 \pmod{p}$  for every prime factor  $q$  of  $p - 1$ .

- `primitive_root(m)` returns the smallest primitive root modulo `m`, or `-1` if no primitive root exists.

Modular multiplication uses `__uint128_t` when available for speed, with a slower portable double-and-add fallback to avoid overflow on compilers without 128-bit integers.

**Time Complexity:**  $O(\sqrt{m} + g \cdot k \cdot \log m)$  per call, where  $g$  is the answer candidate tested and  $k$  is the number of distinct prime factors of  $\phi(m)$ .

**Space Complexity:**  $O(k)$  auxiliary.

```
#include <cassert>
#include <cstdint>
#include <numeric>
#include <set>
#include <utility>
#include <vector>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    assert(a >= 0 && b >= 0 && m > 0);
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, x, m);
        }
        x = mulmod(x, x, m);
    }
    return res;
}

std::vector<std::pair<int64_t, int>> factorize(int64_t n) {
    std::vector<std::pair<int64_t, int>> res;
    for (int64_t p = 2; p <= n / p; p += (p == 2 ? 1 : 2)) {
        if (n % p != 0) {
            continue;
        }
        int count = 0;
        while (n % p == 0) {
```



```

        n /= p;
        count++;
    }
    res.emplace_back(p, count);
}
if (n > 1) {
    res.emplace_back(n, 1);
}
return res;
}

int64_t primitive_root(int64_t m) {
    if (m <= 0) {
        return -1;
    }
    if (m == 1 || m == 2 || m == 4) {
        return m - 1;
    }
    std::vector<std::pair<int64_t, int>> factors = factorize(m);
    bool has_root = (factors.size() == 1 && factors[0].first != 2) ||
        (factors.size() == 2 && factors[0].first == 2 && factors[0].second == 1);
    if (!has_root) {
        return -1;
    }
    int64_t phi = m;
    std::set<int64_t> phi_factors;
    for (auto [p, e] : factors) {
        phi = phi / p * (p - 1);
        if (e > 1) {
            phi_factors.insert(p);
        }
        for (auto [q, count] : factorize(p - 1)) {
            phi_factors.insert(q);
        }
    }
    for (int64_t g = 1; g < m; g++) {
        if (std::gcd(g, m) != 1 || powmod(g, phi, m) != 1) {
            continue;
        }
        bool ok = true;
        for (int64_t q : phi_factors) {
            if (powmod(g, phi / q, m) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) {
            return g;
        }
    }
    return -1;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(primitive_root(1) == 0);
    assert(primitive_root(2) == 1);
    assert(primitive_root(4) == 3);

    assert(primitive_root(17) == 3);           // 3 has order phi(17) = 16.
    assert(primitive_root(998244353) == 3);    // Common NTT prime.
    assert(primitive_root(9) == 2);             // Odd prime power.
}

```

```

assert(primitive_root(18) == 5);           // Twice an odd prime power.

assert(primitive_root(8) == -1);
assert(primitive_root(12) == -1);
return 0;
}

```

### 6.3.5 Discrete Logarithm (Baby-Step Giant-Step)

Given integers  $a$ ,  $b$ , and a modulus  $m$ , the discrete logarithm problem asks for the smallest nonnegative integer  $x$  with  $a^x$  congruent to  $b$  modulo  $m$ . This is the modular analogue of a logarithm and underlies Diffie-Hellman key exchange, order computations in the multiplicative group, and many number-theory reductions.

The baby-step giant-step method writes  $x = np - q$  for a block size  $n$  near the square root of the modulus. It tabulates the “baby steps”  $b \cdot a^q$  for  $q$  in  $[0, n]$ , then walks the “giant steps”  $a^{np}$  for increasing  $p$  until one lands in the table, yielding  $x = np - q$ . This trades the linear scan of all exponents for  $O(\sqrt{m})$  work and memory. A short reduction loop first strips common factors between  $a$  and  $m$ , so the routine works for any modulus, not only when  $a$  and  $m$  are coprime.

- `discrete_log(a, b, m)` returns the smallest nonnegative `x` with `ax` congruent to `b` modulo `m`, or `-1` if no such `x` exists. Requires `m ≥ 1`; `a` and `b` are reduced modulo `m` internally.

**Time Complexity:**  $O(\sqrt{m})$  expected per call with constant-time modular multiplication, and  $O(\sqrt{m} \cdot \log m)$  with the portable double-and-add fallback.

**Space Complexity:**  $O(\sqrt{m})$  auxiliary for the table.

```

#include <cassert>
#include <cmath>
#include <cstdint>
#include <numeric>
#include <unordered_map>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    assert(a >= 0 && b >= 0 && m > 0);
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t discrete_log(int64_t a, int64_t b, int64_t m) {
    a %= m;
    b %= m;
    if (a < 0) {
        a += m;
    }
    if (b < 0) {
        b += m;
    }
    // Strip common factors of a and m so that a becomes invertible; add counts the exponents removed.
    int64_t k = 1 % m, add = 0, g;
    while ((g = std::gcd(a, m)) > 1) {
        if (b == k) {
            return add;
        }
    }
}

```

```

    }
    if (b % g != 0) {
        return -1;
    }
    b /= g;
    m /= g;
    add++;
    k = mulmod(k, a / g, m);
}
int64_t n = static_cast<int64_t>(std::sqrt(static_cast<double>(m))) + 1;
int64_t an = 1;
for (int64_t i = 0; i < n; i++) {
    an = mulmod(an, a, m); // an = a^n, the giant step.
}
std::unordered_map<int64_t, int64_t> baby;
int64_t cur = b;
for (int64_t q = 0; q <= n; q++) {
    baby[cur] = q; // Keep the largest q for each value to make the answer minimal.
    cur = mulmod(cur, a, m);
}
cur = k;
for (int64_t p = 1; p <= n; p++) {
    cur = mulmod(cur, an, m);
    auto it = baby.find(cur);
    if (it != baby.end()) {
        return n * p - it->second + add;
    }
}
return -1;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 2^x = 9 (mod 11): 2^6 = 64 = 9, and 6 is the smallest such exponent.
    assert(discrete_log(2, 9, 11) == 6);

    assert(discrete_log(3, 1, 7) == 0); // a^0 = 1 always.
    assert(discrete_log(2, 3, 5) == 3); // 2^3 = 8 = 3 (mod 5).
    assert(discrete_log(2, 1, 6) == 0); // Works for composite, non-coprime moduli.
    assert(discrete_log(4, 6, 8) == -1); // No power of 4 is 6 (mod 8).
    assert(discrete_log(5, 33, 58) == 9); // 5^9 = 33 (mod 58).
    return 0;
}

```

### 6.3.6 Modular Square Root (Tonelli-Shanks)

Given an integer  $a$  and an odd prime  $p$ , find a modular square root: an  $x$  with  $x^2$  congruent to  $a$  modulo  $p$ . Exactly half of the nonzero residues modulo a prime are squares (quadratic residues), so a solution exists only for those; when it does, the two roots are  $x$  and  $p - x$ .

The Tonelli-Shanks algorithm handles the general case. Euler's criterion,  $a^{(p-1)/2}$  modulo  $p$ , first tests whether  $a$  is a residue at all. When  $p$  is congruent to 3 (mod 4), the root is simply  $a^{(p+1)/4}$ . Otherwise the algorithm factors  $p - 1$  as  $s \cdot 2^e$ , picks any quadratic non-residue to generate the 2-power part of the group, and repeatedly squares a running value to find the order of the current error term, correcting the candidate root until the error becomes 1.

- `mod_sqrt(a, p)` returns the smaller of the two square roots of  $a$  modulo the odd prime  $p$ , or  $-1$  if  $a$  is not a quadratic residue. The value 0 maps to 0. The argument  $a$  is reduced modulo  $p$  internally. The modulus  $p$  must be an odd prime.

**Time Complexity:**  $O(\log^2 p + z \log p)$  per call, where  $z$  is the smallest integer at least 2 that is a quadratic non-residue modulo  $p$ ; thus the unconditional worst-case bound is  $O(p \log p)$ .

**Space Complexity:**  $O(1)$  auxiliary.

```
#include <cassert>
#include <stdint>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    assert(a >= 0 && b >= 0 && m > 0);
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, x, m);
        }
        x = mulmod(x, x, m);
    }
    return res;
}

int64_t mod_sqrt(int64_t a, int64_t p) {
    a %= p;
    if (a < 0) {
        a += p;
    }
    if (a == 0) {
        return 0;
    }
    if (powmod(a, (p - 1) / 2, p) != 1) {
        return -1; // a is a quadratic non-residue.
    }
    if (p % 4 == 3) {
        int64_t x = powmod(a, p / 4 + 1, p);
        return x < p - x ? x : p - x;
    }
    // Write p - 1 = s * 2^e with s odd.
    int64_t s = p - 1, e = 0;
    while (s % 2 == 0) {
        s /= 2;
        e++;
    }
    // Any quadratic non-residue generates the 2-power part of the group.
    int64_t z = 2;
    while (powmod(z, (p - 1) / 2, p) != p - 1) {
        z++;
    }
    int64_t x = powmod(a, (s + 1) / 2, p); // Candidate root, off by a 2-power factor.
    int64_t b = powmod(a, s, p);        // Error term, a power of two in order.
    int64_t g = powmod(z, s, p);         // Generator of the relevant 2-group.
    int64_t r = e;
    while (true) {
        int64_t t = b, order = 0;
        while (order < r && t != 1) {
```

```

    t = mulmod(t, t, p);
    order++;
}
if (order == 0) {
    return x < p - x ? x : p - x;
}
int64_t gs = powmod(g, 1LL << (r - order - 1), p);
g = mulmod(gs, gs, p);
x = mulmod(x, gs, p);
b = mulmod(b, g, p);
r = order;
}
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int64_t r = mod_sqrt(2, 113);
    assert(r != -1 && mulmod(r, r, 113) == 2);
    assert(r == 51); // 51^2 = 2601 = 2 (mod 113); the other root is 62.

    assert(mod_sqrt(0, 7) == 0);
    assert(mod_sqrt(5, 41) != -1); // p = 1 (mod 4) exercises the full algorithm.
    assert(mod_sqrt(4, 7) == 2); // p = 3 (mod 4) fast path.
    assert(mod_sqrt(3, 7) == -1); // 3 is a non-residue modulo 7.

    // Every residue's reported root squares back to it.
    for (int64_t a = 0; a < 41; a++) {
        int64_t x = mod_sqrt(a, 41);
        if (x != -1) {
            assert(mulmod(x, x, 41) == a);
        }
    }
    return 0;
}

```

### 6.3.7 Jacobi Symbol

The Jacobi symbol  $(a/n)$ , defined for an odd positive integer  $n$ , generalizes the Legendre symbol. For an odd prime  $p$  the Legendre symbol  $(a/p)$  is 0 when  $p$  divides  $a$ , 1 when  $a$  is a nonzero quadratic residue modulo  $p$ , and  $-1$  when  $a$  is a non-residue. The Jacobi symbol extends this to composite  $n$  by multiplying the Legendre symbols of the prime factors of  $n$ , counted with multiplicity. It is computed without factoring  $n$ , using the law of quadratic reciprocity together with the supplementary rules for  $-1$  and  $2$ , in the style of the binary GCD.

The symbol is fully multiplicative in  $a$  and in  $n$ . For prime  $n$  it exactly decides quadratic residuosity, so it is the fast way to test whether a modular square root exists. For composite  $n$ , beware that  $(a/n) = 1$  does not guarantee that  $a$  is a quadratic residue; this asymmetry is what the Solovay-Strassen primality test exploits.

- `jacobi(a, n)` returns the Jacobi symbol  $(a/n)$  as one of  $-1$ ,  $0$ , or  $1$ . The modulus `n` must be a positive odd integer; `a` is reduced modulo `n` internally and may be negative. The result is  $0$  exactly when `a` and `n` share a common factor.

**Time Complexity:**  $O(\log n)$  arithmetic operations per call.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cassert>
#include <cstdint>

```

```

#include <utility>

int jacobi(int64_t a, int64_t n) {
    assert(n > 0 && n % 2 == 1);
    a %= n;
    if (a < 0) {
        a += n;
    }
    int result = 1;
    while (a != 0) {
        while (a % 2 == 0) {
            a /= 2;
            // (2 / n) is -1 when n is 3 or 5 modulo 8.
            if (n % 8 == 3 || n % 8 == 5) {
                result = -result;
            }
        }
        std::swap(a, n);
        // Quadratic reciprocity flips the sign when both are 3 modulo 4.
        if (a % 4 == 3 && n % 4 == 3) {
            result = -result;
        }
        a %= n;
    }
    return n == 1 ? result : 0;
}

```

### Example Usage

```

using namespace std;

int64_t legendre_via_euler(int64_t a, int64_t p) {
    int64_t r = 1 % p;
    for (int64_t b = ((a % p) + p) % p, e = (p - 1) / 2; e > 0; e >>= 1) {
        if (e & 1) {
            r = r * b % p;
        }
        b = b * b % p;
    }
    return r == p - 1 ? -1 : r; // r is 0, 1, or p - 1.
}

int main() {
    assert(jacobi(2, 15) == 1); // (2/15) = (2/3)(2/5) = (-1)(-1).
    assert(jacobi(7, 15) == -1);
    assert(jacobi(3, 15) == 0); // gcd(3, 15) = 3.
    assert(jacobi(1001, 9907) == -1);

    // For odd primes the Jacobi symbol equals the Legendre symbol from Euler's criterion.
    for (int64_t p : {3, 5, 7, 11, 13, 101, 9973}) {
        for (int64_t a = 0; a < 50; a++) {
            assert(jacobi(a, p) == legendre_via_euler(a, p));
        }
    }
    return 0;
}

```

### 6.3.8 Prime Generation

Generate prime numbers using the sieve of Eratosthenes, which repeatedly marks the multiples of each prime as composite. The linear sieve technique optimizes the classic sieve by finding the least prime factor of each integer. When processing `i`, it marks `i * p` for each known prime `p` no greater than `least[i]`. Every composite `x` is

therefore generated exactly once: writing  $x = i \cdot p$ , where  $p$  is its least prime factor, gives  $p \leq \text{least}[i]$ , while any representation using a larger prime is skipped. This single visit per composite gives the  $O(n)$  running time.

The segmented sieve technique computes primes in a high but narrow window without sieving everything below it. Since every composite at most  $hi$  has a prime factor at most  $\sqrt{hi}$ , it first sieves the primes up to  $\sqrt{hi}$ , then uses only those primes to mark composites within  $[lo, hi]$  (each prime  $p$  starting at the first of its multiples that is at least both  $p^2$  and  $lo$ ). This needs only  $O(w + \sqrt{hi})$  space, where  $w = hi - lo + 1$ , instead of the  $O(hi)$  space that sieving all of  $[2, hi]$  would require.

- `sieve(n)` returns a vector of all the primes less than or equal to  $n$ .
- `linear_sieve(n, &least_out)` returns a vector of all primes less than or equal to  $n$  in linear time. If `least_out` is not null, it is filled so that `least_out[x]` is the least prime factor of  $x$  for every  $x \geq 2$ .
- `segmented_sieve(lo, hi)` returns a vector of all the primes in the range  $[lo, hi]$ .

#### Time Complexity:

- $O(n \log \log n)$  per call to `sieve(n)`.
- $O(n)$  per call to `linear_sieve()`.
- $O((w + \sqrt{hi}) \cdot \log(\log(hi)))$  per call to `segmented_sieve(lo, hi)`, where  $w = hi - lo + 1$ .

#### Space Complexity:

- $O(n)$  auxiliary for `sieve(n)`.
- $O(n)$  auxiliary for `linear_sieve()`.
- $O(w + \sqrt{hi})$  auxiliary for `segmented_sieve(lo, hi)`.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <vector>

std::vector<int> sieve(int n) {
    if (n < 2) {
        return {};
    }
    std::vector<char> prime(n + 1, true);
    for (int i = 2; i <= n / i; i++) {
        if (prime[i]) {
            for (int64_t j = 1LL * i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
    std::vector<int> res;
    for (int i = 2; i <= n; i++) {
        if (prime[i]) {
            res.push_back(i);
        }
    }
    return res;
}

std::vector<int> linear_sieve(int n, std::vector<int> *least_out = nullptr) {
    if (n < 2) {
        if (least_out != nullptr) {
            least_out->assign(std::max(n + 1, 0), 0);
        }
        return {};
    }
    std::vector<int> least(n + 1), primes;
    for (int i = 2; i <= n; i++) {
        if (least[i] == 0) {
            least[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {

```

```

        if (p > least[i] || i > n / p) {
            break;
        }
        least[i * p] = p;
    }
}
if (least_out != nullptr) {
    *least_out = least;
}
return primes;
}

std::vector<int> segmented_sieve(int lo, int hi) {
    if (hi < 2 || lo > hi) {
        return {};
    }
    int sqrt_hi = std::sqrt(hi);
    while (1LL * (sqrt_hi + 1) * (sqrt_hi + 1) <= hi) {
        sqrt_hi++;
    }
    int fourth_root_hi = std::sqrt(sqrt_hi);
    while (1LL * (fourth_root_hi + 1) * (fourth_root_hi + 1) <= sqrt_hi) {
        fourth_root_hi++;
    }
    std::vector<char> prime1(sqrt_hi + 1, true), prime2(hi - lo + 1, true);
    for (int i = 2; i <= fourth_root_hi; i++) {
        if (prime1[i]) {
            for (int64_t j = 1LL * i * i; j <= sqrt_hi; j += i) {
                prime1[j] = false;
            }
        }
    }
    for (int i = 2; i <= sqrt_hi; i++) {
        if (prime1[i]) {
            int64_t first_multiple = ((static_cast<int64_t>(lo) + i - 1) / i) * i;
            int64_t start = std::max(static_cast<int64_t>(i) * i, first_multiple);
            for (int64_t j = start; j <= hi; j += i) {
                prime2[j - lo] = false;
            }
        }
    }
    std::vector<int> res;
    for (int i = (lo > 1) ? lo : 2; i <= hi; i++) {
        if (prime2[i - lo]) {
            res.push_back(i);
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
#include <ctime>
#include <iostream>
using namespace std;

int main() {
    int pmax = 10000000;
    time_t start;
    double delta;

    start = clock();
    auto p = sieve(pmax);
    delta = static_cast<double>((clock() - start)) / CLOCKS_PER_SEC;
    cout << "sieve(n=" << pmax << "): " << delta << "s" << endl;
}

```



```

assert((sieve(1) == vector<int>{}));
assert((sieve(10) == vector<int>{2, 3, 5, 7}));

vector<int> least;
assert(linear_sieve(pmax, &least) == p);
assert(least[999983] == 999983);
assert(least[999984] == 2);

int lo = 1000000000, hi = 1005000000;
start = clock();
p = segmented_sieve(lo, hi);
delta = static_cast<double>((clock() - start)) / CLOCKS_PER_SEC;
cout << "segmented_sieve([" << lo << ", " << hi << "]): " << delta << "s" << endl;
assert((segmented_sieve(14, 16) == vector<int>{}));
assert((segmented_sieve(17, 19) == vector<int>{17, 19}));
assert(!p.empty() && p.front() >= lo && p.back() <= hi);
return 0;
}

```

#### Example Output

```

sieve(n=100000000): 0.042641s
segmented_sieve([1000000000, 1005000000]): 0.01969s

```

### 6.3.9 Prime Tests and Factorization

Provides primality tests, prime factorization, and divisor generation. Alongside generic trial division, it includes deterministic Miller-Rabin and Pollard's rho for signed 64-bit integers. Prime factorizations are represented as sorted vectors of (`prime`, `exponent`) pairs. For 0 and 1, the prime factorization is empty.

Trial division tests primality and computes prime factorizations by trying possible divisors through the square root.

- `is_prime_slow(n)` returns whether the integer `n` is prime using trial division.
- `factorize_slow(n)` returns the prime factorization of `n` using trial division.

**Time Complexity:**  $O(\sqrt{n})$  per call to `is_prime_slow()` and `factorize_slow()`.

**Space Complexity:**  $O(1)$  auxiliary for `is_prime_slow()` and  $O(f)$  for the returned factorization, where  $f$  is the number of compressed prime factors.

```

#include <algorithm>
#include <cassert>
#include <chrono>
#include <cstdint>
#include <numeric>
#include <random>
#include <utility>
#include <vector>

template<typename Int>
bool is_prime_slow(Int n) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    for (Int i = 5, w = 4; i <= n / i; i += w) {
        if (n % i == 0) {
            return false;
        }
        w = 6 - w;
    }
}

```

```

    return true;
}

template<typename Int>
std::vector<std::pair<Int, int>> factorize_slow(Int n) {
    if (n <= 1) {
        return {};
    }
    std::vector<std::pair<Int, int>> res;
    for (Int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            int exponent = 0;
            do {
                n /= i;
                exponent++;
            } while (n % i == 0);
            res.emplace_back(i, exponent);
        }
    }
    if (n > 1) {
        res.emplace_back(n, 1);
    }
    return res;
}

```

The Miller-Rabin test writes  $n - 1 = d \cdot 2^r$  and repeatedly squares  $a^d$ ; a base that breaks the pattern required of a prime proves that  $n$  is composite.

- `is_probable_prime(n, k = 20)` returns whether `n` is probably prime using `k` random Miller-Rabin bases. The result is guaranteed correct when `n` is prime. For composite `n`, a true result has one-sided error probability at most  $(1/4)^k$ .
- `is_prime(n)` returns whether the signed 64-bit integer `n` is prime using deterministic Miller-Rabin bases sufficient up to and including  $2^{63} - 1$ .

Modular multiplication uses `__uint128_t` when available, with a slower portable double-and-add fallback that avoids overflow on compilers without 128-bit integers.

**Time Complexity:**  $O(k \cdot \log n)$  modular multiplications per call to `is_probable_prime()` and  $O(\log n)$  per call to `is_prime()`. The portable multiplication fallback adds another  $O(\log n)$  factor.

**Space Complexity:**  $O(1)$  auxiliary.

```

uint64_t mulmod(uint64_t a, uint64_t b, uint64_t m) {
    assert(m > 0);
#ifdef __SIZEOF_INT128__
    return static_cast<uint64_t>(static_cast<__uint128_t>(a) * b % m);
#else
    uint64_t res = 0, cur = a % m;
    for (; b > 0; b >>= 1) {
        if (b & 1) {
            res = res >= m - cur ? res - (m - cur) : res + cur;
        }
        cur = cur >= m - cur ? cur - (m - cur) : cur + cur;
    }
    return res;
#endif
}

uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
    assert(m > 0);
    uint64_t res = 1 % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, x, m);
        }
    }
}

```

```

    x = mulmod(x, x, m);
}
return res;
}

uint64_t rand64u() {
    static std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    return rng();
}

bool is_probable_prime(int64_t n, int k = 20) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    uint64_t s = n - 1, p = n - 1;
    while (!(s & 1)) {
        s >>= 1;
    }
    for (int i = 0; i < k; i++) {
        uint64_t x, r = powmod(rand64u() % (n - 3) + 2, s, n); // random witness base in [2, n - 2]
        for (x = s; x != p && r != 1 && r != p; x <<= 1) {
            r = mulmod(r, r, n);
        }
        if (r != p && !(x & 1)) {
            return false;
        }
    }
    return true;
}

bool is_prime(int64_t n) {
    if (n < 2) {
        return false;
    }
    static const int small_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
    for (int p : small_primes) {
        if (n % p == 0) {
            return n == p;
        }
    }
    if (n < 31 * 31) {
        return true;
    }
    uint64_t t;
    int s = 0;
    for (t = n - 1; !(t & 1); t >>= 1) {
        s++;
    }
    static const uint64_t bases[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    for (uint64_t a : bases) {
        if (a % n == 0) {
            continue;
        }
        uint64_t r = powmod(a, t, n);
        if (r == 1) {
            continue;
        }
        bool ok = false;
        for (int j = 0; j < s && !ok; j++) {
            ok |= (r == static_cast<uint64_t>(n) - 1);
            r = mulmod(r, r, n);
        }
        if (!ok) {
            return false;
        }
    }
}

```

```

    }
    return true;
}

```

Pollard's rho iterates a pseudorandom map modulo  $n$ ; two iterates that collide modulo a hidden prime factor reveal that factor through a GCD with their difference. The complete factorization routine first removes small prime factors with a cached linear sieve, then uses Miller-Rabin and Pollard's rho on the remaining cofactor. Factorizations remain sorted and compressed as `(prime, exponent)` pairs.

- `rho_factor(n)` returns a factor of `n` that is not necessarily prime using Pollard's rho with Brent's optimization. If `n` is prime, then `n` itself is returned. The algorithm may need to be retried to find a nontrivial factor, as done in `factorize_rho()`.
- `factorize_rho(n)` returns the prime factorization of a 64-bit integer using Miller-Rabin and Pollard's rho without initial trial division.
- `factorize(n, small_prime_limit = 1000000)` returns the prime factorization of a 64-bit integer `n`, first testing primes through `small_prime_limit` and then falling back to Pollard's rho. It supports integers up to and including  $2^{63} - 1$ .
- `divisors_from_factors(factors)` returns all divisors from a compressed factorization.
- `divisors(n)` returns all divisors of a 64-bit integer using `factorize()` followed by `divisors_from_factors()`.

#### Time Complexity:

- $O(\sqrt{p})$  expected modular multiplications per call to `rho_factor(n)`, where  $p$  is the smallest prime factor of `n`. For composite `n`, this is at most  $O(n^{1/4})$ .
- $O(n^{1/4})$  expected per call to `factorize_rho()`. `factorize()` additionally performs  $O(L/\log L)$  trial divisions, where  $L$  is `small_prime_limit`, and takes  $O(L)$  time if the cache is rebuilt.
- $O(d \log d)$  per call to `divisors_from_factors()`, where  $d$  is the number of divisors generated.
- $O(L/\log L + n^{1/4} + d \log d)$  expected per call to `divisors()`, plus  $O(L)$  if the cache is rebuilt.

#### Space Complexity:

- $O(c)$  cached space after sieving through  $c$ .
- $O(f + \log n)$  auxiliary for recursive factorization, where  $f$  is the number of compressed prime factors returned.
- $O(d)$  output space and  $O(\log d)$  auxiliary stack space for `divisors_from_factors()` and `divisors()`.

```

using Factors = std::vector<std::pair<int64_t, int>>>;

int64_t rho_factor(int64_t n) {
    if (n % 2 == 0) {
        return 2;
    }
    uint64_t y = rand64u() % (n - 1) + 1;
    uint64_t c = rand64u() % (n - 1) + 1;
    uint64_t m = rand64u() % (n - 1) + 1;
    uint64_t g = 1, r = 1, q = 1, ys = 0, x = 0;
    for (r = 1; g == 1; r <= 1) {
        x = y;
        for (uint64_t i = 0; i < r; i++) {
            y = (mulmod(y, y, n) + c) % n;
        }
        for (uint64_t k = 0; k < r && g == 1; k += m) {
            ys = y;
            int64_t lim = std::min(m, r - k);
            for (int j = 0; j < lim; j++) {
                y = (mulmod(y, y, n) + c) % n;
                q = mulmod(q, (x > y) ? (x - y) : (y - x), n);
            }
            g = std::gcd(q, n);
        }
    }
    if (g == static_cast<uint64_t>(n)) {

```

```

    do {
        ys = (mulmod(ys, ys, n) + c) % n;
        g = std::gcd((x > ys) ? (x - ys) : (ys - x), n);
    } while (g <= 1);
}
return g;
}

Factors factorize_rho(int64_t n) {
    std::vector<int64_t> factors;
    auto collect = [&](auto &&collect, int64_t value) {
        if (value <= 1) {
            return;
        }
        if (is_prime(value)) {
            factors.push_back(value);
            return;
        }
        int64_t p;
        do {
            p = rho_factor(value);
        } while (p == value);
        collect(collect, p);
        collect(collect, value / p);
    };
    collect(collect, n);
    std::sort(factors.begin(), factors.end());
    Factors res;
    for (int64_t p : factors) {
        if (res.empty() || res.back().first != p) {
            res.emplace_back(p, 1);
        } else {
            res.back().second++;
        }
    }
    return res;
}

static const auto &cached_sieve(int n) {
    struct Cache {
        int limit = 1;
        std::vector<int> least{0, 1}, primes;
    };
    static Cache cache;
    if (n <= cache.limit) {
        return cache;
    }
    cache.limit = std::max(1, n);
    cache.least.assign(cache.limit + 1, 0);
    cache.primes.clear();
    for (int i = 2; i <= cache.limit; i++) {
        if (cache.least[i] == 0) {
            cache.least[i] = i;
            cache.primes.push_back(i);
        }
        for (int p : cache.primes) {
            if (p > cache.least[i] || i > cache.limit / p) {
                break;
            }
            cache.least[i * p] = p;
        }
    }
    return cache;
}

Factors factorize(int64_t n, int small_prime_limit = 1000000) {
    assert(small_prime_limit >= 1);
    if (n <= 1) {

```

```

    return {};
}
const auto &sieve = cached_sieve(small_prime_limit);
if (n <= small_prime_limit) {
    Factors res;
    while (n > 1) {
        int p = sieve.least[n], cnt = 0;
        do {
            n /= p;
            cnt++;
        } while (n > 1 && sieve.least[n] == p);
        res.emplace_back(p, cnt);
    }
    return res;
}
Factors res;
const std::vector<int> &primes = sieve.primes;
for (int p : primes) {
    if (p > small_prime_limit || static_cast<int64_t>(p) > n / p) {
        break;
    }
    if (n % p == 0) {
        int cnt = 0;
        do {
            n /= p;
            cnt++;
        } while (n % p == 0);
        res.emplace_back(p, cnt);
    }
}
Factors tail = factorize_rho(n);
res.insert(res.end(), tail.begin(), tail.end());
return res;
}

std::vector<int64_t> divisors_from_factors(const Factors &factors) {
    std::vector<int64_t> res{1};
    for (const auto &factor : factors) {
        int old_size = static_cast<int>(res.size());
        int64_t power = 1;
        for (int e = 1; e <= factor.second; e++) {
            power *= factor.first;
            for (int i = 0; i < old_size; i++) {
                res.push_back(res[i] * power);
            }
        }
    }
    std::sort(res.begin(), res.end());
    return res;
}

std::vector<int64_t> divisors(int64_t n) {
    if (n <= 1) {
        return (n < 1) ? std::vector<int64_t>() : std::vector<int64_t>(1, 1);
    }
    return divisors_from_factors(factorize(n));
}

```

## Example Usage

```

#include <ctime>
#include <iomanip>
#include <iostream>
#include <set>
using namespace std;

```

```

void validate(int64_t n, const Factors &factors) {
    if (n <= 1) {
        assert(factors.empty());
        return;
    }
    if (is_prime(n)) {
        assert((factors == Factors{{n, 1}}));
        return;
    }
    int64_t prod = 1;
    for (const auto &factor : factors) {
        assert(is_prime(factor.first));
        for (int i = 0; i < factor.second; i++) {
            prod *= factor.first;
        }
    }
    assert(prod == n);
}

int main() {
    { // Small primality tests.
        vector<pair<int, bool>> cases{
            {-10, false}, {-1, false}, {0, false}, {1, false}, // non-primes
            {2, true}, {3, true}, {4, false}, {5, true},
            {9, false}, // divisible by 3
            {25, false}, // square of small prime
            {29, true}, // last small-prime table entry
            {31, true}, // just past small-prime table
            {49, false}, // square, not divisible by 2 or 3
            {121, false}, // 11^2
            {169, false}, // 13^2
            {961, false}, // 31^2, boundary for n < 31*31 shortcut
            {997, true}, // prime near 1000
            {1001, false}, // 7*11*13
            {2047, false}, // 23*89, pseudoprime to base 2
            {341, false}, // 11*31, Fermat pseudoprime to base 2
            {561, false}, // Carmichael number
        };
        for (const auto &[n, expected] : cases) {
            assert(is_prime_slow(n) == expected);
            assert(is_probable_prime(n) == expected);
            assert(is_prime(n) == expected);
        }
    }
    { // Primality test benchmark.
        vector<int64_t> nums{
            772023803LL, 2147483647LL, 5705234089LL, 6339503641LL, 999966000289LL,
        };
        cout << "Primality test:" << endl;
        cout << setw(15) << left << "num:";
        cout << setw(18) << left << "is_prime_slow():";
        cout << setw(20) << left << "is_probable_prime():";
        cout << setw(13) << right << "is_prime():" << endl;
        cout << fixed << setprecision(3);
        for (int64_t n : nums) {
            clock_t start = clock();
            bool p1 = is_prime_slow(n);
            double t1 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            start = clock();
            bool p2 = is_probable_prime(n);
            double t2 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            start = clock();
            bool p3 = is_prime(n);
            double t3 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            assert(p1 == p2 && p1 == p3);
            cout << setw(12) << left << n << right;
            cout << setw(14) << 1000 * t1 << "ms";
            cout << setw(16) << 1000 * t2 << "ms";
        }
    }
}

```

```

    cout << setw(16) << 1000 * t3 << "ms" << endl;
}
cout << endl;
}
{ // Small factorization tests.
for (int64_t i = 1; i <= 10000; i++) {
    auto v1 = factorize_slow(i);
    auto v2 = factorize(i);
    validate(i, v1);
    assert(v1 == v2);
    auto d = divisors(i);
    set<int> s(d.begin(), d.end());
    assert(d.size() == s.size());
    for (int j = 1; j <= i; j++) {
        if (i % j == 0) {
            assert(s.count(j));
        }
    }
}
}
{ // Compressed factors are convenient for building divisors.
Factors factors{{2, 3}, {3, 2}, {5, 1}};
vector<int64_t> divs = divisors_from_factors(factors);
assert(
    (divs == vector<int64_t>{
        1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18,
        20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360,
    })
);
assert(factorize(360) == factors);
assert(divisors(360) == divs);
}
{ // Large factorization tests.
const vector<int64_t> nums{
    (1LL << 62), // high power of two
    9223372036854775807LL, // 2^63 - 1, many small factors
    999983LL * 999983, // square of a large prime
    1900009LL * 1910009 * 2541547, // three medium-size prime factors
    50000017LL * 50001037, // 16-digit semiprime
};
cout << "Factorization test:" << endl;
cout << setw(24) << left << "num:";
cout << setw(20) << left << "factorize_slow():";
cout << "factorize():" << endl;
cout << fixed << setprecision(3);
for (int64_t n : nums) {
    clock_t start = clock();
    Factors factors1 = factorize_slow(n);
    double t1 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
    start = clock();
    Factors factors2 = factorize(n);
    double t2 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
    validate(n, factors1);
    validate(n, factors2);
    assert(factors1 == factors2);
    cout << setw(26) << left << n << right << setw(8) << 1000 * t1 << "ms";
    cout << setw(16) << 1000 * t2 << "ms" << endl;
    cout << left;
}
}
return 0;
}

```



**Example Output**

```

Primity test:
num:          is_prime_slow():  is_probable_prime():  is_prime():
772023803      0.006ms           0.005ms           0.001ms
2147483647      0.009ms           0.006ms           0.002ms
5705234089      0.014ms           0.000ms           0.001ms
6339503641      0.013ms           0.001ms           0.001ms
999966000289    0.171ms           0.001ms           0.001ms

Factorization test:
num:          factorize_slow():  factorize():
4611686018427387904  0.002ms           0.002ms
9223372036854775807  0.049ms           0.007ms
999966000289          0.503ms           0.043ms
9223361212852495307  0.952ms           0.096ms
2500052700017629     24.587ms          0.194ms

```

**6.3.10 Euler's Totient Function**

Euler's totient function  $\phi(n)$  returns the number of positive integers less than or equal to  $n$  that are relatively prime to  $n$ ; that is, the number of integers  $k$  in the range  $[1, n]$  for which  $\gcd(n, k) = 1$ .

The function is multiplicative, with  $\phi(p^k) = p^k - p^{k-1}$  for a prime power, which yields the product formula  $\phi(n) = n \prod_{p|n} (1 - 1/p)$  taken over the distinct primes  $p$  dividing  $n$ . `phi(n)` applies this formula directly, factoring  $n$  by trial division. `phi_table(n)` computes the whole range at once with a sieve of Eratosthenes: starting from `v[i] = i`, it sweeps each prime  $p$  across its multiples `j` and folds in the factor  $(1 - 1/p)$  by replacing `v[j]` with `v[j] - v[j]/p`. This is faster than the alternative that uses the divisor-sum identity  $\sum_{d|n} \phi(d) = n$  to subtract each `v[i]` from its multiples, which costs  $O(n \log n)$ .

The overload `phi_table(lo, hi)` computes the totients of a high but narrow range with a segmented sieve, analogous to the segmented prime sieve. It sieves the primes up to  $\sqrt{hi}$ , applies each one's  $(1 - 1/p)$  factor to its multiples within  $[lo, hi]$  while dividing that prime out of a parallel copy of every value, and finally folds in any single leftover prime factor greater than  $\sqrt{hi}$  (a number at most `hi` can have at most one such factor).

- `phi(n)` returns Euler's totient of `n`.
- `phi_table(n)` returns a vector `v` of length `n + 1` such that `v[i]` stores `phi(i)` for every `i` in the range  $[0, n]$ .
- `phi_table(lo, hi)` returns a vector `v` of length `hi - lo + 1` such that `v[i - lo]` stores `phi(i)` for every `i` in the range  $[lo, hi]$ . This overload assumes `lo ≥ 0`; if `hi < lo`, it returns an empty vector.

**Time Complexity:**

- $O(\sqrt{n})$  per call to `phi(n)`.
- $O(n \log \log n)$  per call to `phi_table(n)`.
- $O((w + \sqrt{hi}) \cdot \log(\log(hi)))$  per call to `phi_table(lo, hi)`, where  $w = hi - lo + 1$ .

**Space Complexity:**

- $O(1)$  auxiliary for `phi(n)`.
- $O(n)$  auxiliary for `phi_table(n)`.
- $O(w + \sqrt{hi})$  auxiliary for `phi_table(lo, hi)`.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <numeric>
#include <vector>

int phi(int n) {
    int res = n;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            while (n % i == 0) {

```

```

        n /= i;
    }
    res -= res / i;
}
}
if (n > 1) {
    res -= res / n;
}
return res;
}

std::vector<int> phi_table(int n) {
    std::vector<int> res(n + 1);
    std::iota(res.begin(), res.end(), 0);
    for (int i = 2; i <= n; i++) {
        if (res[i] == i) { // i is prime, so apply its (1 - 1/i) factor to every multiple.
            for (int j = i; j <= n; j += i) {
                res[j] -= res[j] / i;
            }
        }
    }
    return res;
}

std::vector<int> phi_table(int lo, int hi) {
    if (hi < lo) {
        return {};
    }
    int root = static_cast<int>(std::sqrt(hi));
    while (1LL * (root + 1) * (root + 1) <= hi) {
        root++;
    }
    // Sieve the primes up to sqrt(hi); any larger prime divisor is the one leftover factor.
    std::vector<char> composite(root + 1);
    std::vector<int> primes;
    for (int i = 2; i <= root; i++) {
        if (!composite[i]) {
            primes.push_back(i);
            for (int64_t j = 1LL * i * i; j <= root; j += i) {
                composite[j] = true;
            }
        }
    }
    std::vector<int> res(hi - lo + 1);
    std::vector<int64_t> remaining(hi - lo + 1);
    for (int i = lo; i <= hi; i++) {
        res[i - lo] = i;
        remaining[i - lo] = i;
    }
    for (int p : primes) {
        // Start at the first multiple of p at least lo, but never below p itself (this skips 0).
        int64_t start = std::max(1LL * p, ((1LL * lo + p - 1) / p) * p);
        for (int64_t j = start; j <= hi; j += p) {
            int idx = static_cast<int>(j - lo);
            res[idx] -= res[idx] / p;
            while (remaining[idx] % p == 0) {
                remaining[idx] /= p;
            }
        }
    }
    for (int i = lo; i <= hi; i++) {
        int idx = i - lo;
        if (remaining[idx] > 1) { // a single prime factor greater than sqrt(hi) is left over
            res[idx] -= static_cast<int>(res[idx] / remaining[idx]);
        }
    }
    return res;
}

```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(phi(1) == 1);
    assert(phi(9) == 6);
    assert(phi(1234567) == 1224720);
    const int n = 1000;
    vector<int> pt = phi_table(n);
    for (int i = 0; i <= n; i++) {
        assert(pt[i] == phi(i));
    }
    // The segmented overload agrees with phi(), including over a low range (with 0 and 1) and a
    // high, narrow window where many values have a prime factor exceeding sqrt(hi).
    vector<int> lphi = phi_table(0, 50);
    for (int i = 0; i <= 50; i++) {
        assert(lphi[i] == phi(i));
    }
    int lo = 1000000000, hi = 1000000100;
    vector<int> hphi = phi_table(lo, hi);
    for (int i = lo; i <= hi; i++) {
        assert(hphi[i - lo] == phi(i));
    }
    return 0;
}
```

### 6.3.11 Divisor Functions

The divisor-counting function  $d(n)$  counts the positive divisors of  $n$ , and the divisor-sum function  $\sigma(n)$  adds them up. Both are multiplicative, so each is determined by the prime factorization  $n = \prod p_i^{e_i}$ : choosing an exponent in  $[0, e_i]$  for every prime gives  $d(n) = \prod (e_i + 1)$ , and summing the resulting geometric series for each prime gives  $\sigma(n) = \prod (1 + p_i + \dots + p_i^{e_i})$ . A single value is therefore computed by trial division, factoring  $n$  as the divisors are accumulated.

- `divisor_count(n)` returns  $d(n)$ , the number of positive divisors of  $n$ , which must be positive.
- `divisor_sum(n)` returns  $\sigma(n)$ , the sum of the positive divisors of  $n$ , which must be positive.

For every value in a range, factoring each number separately is wasteful. Instead, sweep over each candidate divisor  $d$  and add its contribution to every multiple of  $d$ . The total work is the harmonic sum  $n/1 + n/2 + \dots + n/n$ , which is  $O(n \log n)$ . The same sweep computes any divisor-indexed accumulation, such as the divisor sums used by the Mobius inversion of section 6.3.12.

- `divisor_count_table(n)` returns a vector `v` of length `n + 1` such that `v[i]` is the number of divisors of `i` for every `i` in  $[1, n]$ , with `v[0]` left as 0.
- `divisor_sum_table(n)` returns a vector `v` of length `n + 1` such that `v[i]` is the sum of the divisors of `i` for every `i` in  $[1, n]$ , with `v[0]` left as 0.

The aliquot sum  $\sigma(n) - n$  adds the proper divisors, marking  $n$  perfect when it equals  $n$  and pairing amicable numbers when each is the other's. For an  $n$  too large for trial division, factor it with Pollard's rho in section 6.3.9 and apply the product formulas; to tabulate a high narrow window, sieve it with the primes up to  $\sqrt{hi}$  as in section 6.3.8.

Overflow warning:  $\sigma(n)$  can be several times larger than  $n$ , so `divisor_sum()` overflows `int64_t` somewhat before  $n$  itself does. Trial division binds first in practice, since  $\sqrt{n}$  divisions are already too slow well below that point; factor with section 6.3.9 instead.

#### Time Complexity:

- $O(\sqrt{n})$  per call to `divisor_count(n)` and `divisor_sum(n)`.

- $O(n \log n)$  per call to `divisor_count_table(n)` and `divisor_sum_table(n)`.

#### Space Complexity:

- $O(1)$  auxiliary for `divisor_count()` and `divisor_sum()`.
- $O(n)$  for the vectors returned by `divisor_count_table(n)` and `divisor_sum_table(n)`.

```
#include <cassert>
#include <cstdint>
#include <vector>

int64_t divisor_count(int64_t n) {
    assert(n > 0);
    int64_t res = 1;
    for (int64_t p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            int e = 0;
            for (; n % p == 0; n /= p) {
                e++;
            }
            res *= e + 1;
        }
    }
    if (n > 1) { // A prime cofactor larger than the square root is left over.
        res *= 2;
    }
    return res;
}

int64_t divisor_sum(int64_t n) {
    assert(n > 0);
    int64_t res = 1;
    for (int64_t p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            int64_t power = 1, series = 1;
            for (; n % p == 0; n /= p) {
                power *= p;
                series += power;
            }
            res *= series; // Overflow warning.
        }
    }
    if (n > 1) {
        res *= n + 1;
    }
    return res;
}

std::vector<int> divisor_count_table(int n) {
    std::vector<int> res(n + 1);
    for (int d = 1; d <= n; d++) {
        for (int i = d; i <= n; i += d) {
            res[i]++;
        }
    }
    return res;
}

std::vector<int64_t> divisor_sum_table(int n) {
    std::vector<int64_t> res(n + 1);
    for (int d = 1; d <= n; d++) {
        for (int i = d; i <= n; i += d) {
            res[i] += d;
        }
    }
    return res;
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(divisor_count(1) == 1);
    assert(divisor_count(12) == 6); // 1, 2, 3, 4, 6, and 12.
    assert(divisor_count(999999937) == 2);
    assert(divisor_sum(1) == 1);
    assert(divisor_sum(12) == 28);
    assert(divisor_sum(6) == 12); // 6 is perfect, since its aliquot sum is 6.

    // 220 and 284 are the smallest amicable pair.
    assert(divisor_sum(220) - 220 == 284);
    assert(divisor_sum(284) - 284 == 220);

    auto counts = divisor_count_table(100);
    auto sums = divisor_sum_table(100);
    assert(counts[0] == 0 && sums[0] == 0);
    for (int i = 1; i <= 100; i++) {
        assert(counts[i] == divisor_count(i));
        assert(sums[i] == divisor_sum(i));
    }
    return 0;
}
```

### 6.3.12 Mobius Function and Inclusion-Exclusion

The Mobius function  $\mu(n)$  is the multiplicative function defined as 1 if  $n = 1$ , 0 if  $n$  is divisible by the square of a prime, and  $(-1)^k$  if  $n$  is a product of  $k$  distinct primes. It is the coefficient that appears in Mobius inversion: if  $g(n) = \sum_{d|n} f(d)$  for all  $n$ , then  $f(n) = \sum_{d|n} \mu(d) g(n/d)$ . This formalizes the principle of inclusion-exclusion in the divisor lattice, letting one recover a summand function from its divisor sums.

Inclusion-exclusion itself counts the size of a union by alternately adding and subtracting the sizes of intersections. A canonical number-theoretic instance is counting integers in  $[1, n]$  that are coprime to a modulus  $m$ : subtract the multiples of each prime factor of  $m$ , add back those counted twice, and so on. The signed terms are precisely the values of  $\mu$  over the squarefree divisors of  $m$ , so `count_coprime(n, m)` equals  $\sum_{d|m} \mu(d) \lfloor n/d \rfloor$ .

- `mobius(n)` returns  $\mu(n)$  for a single positive `n` by trial division.
- `mobius_sieve(n)` returns a vector of length  $n + 1$  where index  $i$  holds  $\mu(i)$ , computed together with a linear sieve, where `n` must be nonnegative.
- `count_coprime(n, m)` returns the number of integers in  $[1, n]$  that are coprime to `m`, via inclusion-exclusion over the distinct prime factors of `m`, where `n` must be nonnegative and `m` must be positive.

#### Time Complexity:

- $O(\sqrt{n})$  per call to `mobius()`.
- $O(n)$  per call to `mobius_sieve()`.
- $O(\sqrt{m} + 2^k \cdot k)$  per call to `count_coprime()`, where  $k$  is the number of distinct prime factors of  $m$  (at most 15 for  $m$  within 64-bit range).

#### Space Complexity:

- $O(n)$  auxiliary for `mobius_sieve()`.
- $O(k)$  auxiliary for `count_coprime()`.
- $O(1)$  auxiliary for `mobius()`.

```
#include <cassert>
#include <cstdint>
#include <vector>
```

```

int mobius(int n) {
    assert(n > 0);
    if (n == 1) {
        return 1;
    }
    int res = 1;
    for (int p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            n /= p;
            if (n % p == 0) {
                return 0; // p^2 divides the original n.
            }
            res = -res;
        }
    }
    if (n > 1) {
        res = -res; // One remaining prime factor larger than its square root.
    }
    return res;
}

std::vector<int> mobius_sieve(int n) {
    assert(n >= 0);
    std::vector<int> mu(n + 1), primes;
    std::vector<char> composite(n + 1);
    if (n >= 1) {
        mu[1] = 1;
    }
    for (int i = 2; i <= n; i++) {
        if (!composite[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (p > n / i) {
                break;
            }
            composite[i * p] = true;
            if (i % p == 0) {
                mu[i * p] = 0; // p^2 divides i * p.
                break;
            }
            mu[i * p] = -mu[i];
        }
    }
    return mu;
}

int64_t count_coprime(int64_t n, int64_t m) {
    assert(n >= 0 && m > 0);
    std::vector<int64_t> primes;
    for (int64_t p = 2; p <= m / p; p++) {
        if (m % p == 0) {
            primes.push_back(p);
            while (m % p == 0) {
                m /= p;
            }
        }
    }
    if (m > 1) {
        primes.push_back(m);
    }
    int k = static_cast<int>(primes.size());
    int64_t total = 0;
    for (int mask = 0; mask < (1 << k); mask++) {
        int64_t product = 1;
        int bits = 0;

```

```

for (int i = 0; i < k; i++) {
    if ((mask >> i) & 1) {
        product *= primes[i];
        bits++;
    }
}
total += (bits & 1 ? -1 : 1) * (n / product);
}
return total;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(mobius(1) == 1);
    assert(mobius(2) == -1);
    assert(mobius(6) == 1); // 2 * 3, two distinct primes.
    assert(mobius(12) == 0); // Divisible by 2^2.
    assert(mobius(30) == -1); // 2 * 3 * 5, three distinct primes.

    assert((mobius_sieve(12) == vector<int>{0, 1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0}));

    assert(count_coprime(10, 6) == 3); // 1, 5, 7
    assert(count_coprime(100, 1) == 100);
    return 0;
}

```

## 6.4 Arbitrary Precision Arithmetic

---

### 6.4.1 Big Integer (Simple)

Perform simple arithmetic operations on arbitrary precision big integers whose digits are internally represented as an `std::string` in little-endian order.

This version is intentionally small and decimal-oriented. Addition and subtraction are the usual schoolbook digit scans with carry or borrow. Multiplication keeps a shifted copy of the left operand for each digit of the right operand, adds that row once per digit value, then shifts by one decimal place. Division is long division: it scans the dividend from most significant digit to least, maintains a running remainder, and repeatedly subtracts the divisor to discover each quotient digit.

- `BigInt(n = 0)` constructs a big integer from an integer `n`.
- `BigInt(s)` constructs a big integer from a string `s`, which must strictly consist of a sequence of numeric digits, optionally preceded by a minus sign.
- `to_string()` returns the string representation of the big integer.
- `comp(a, b)` returns `-1`, `0`, or `1` depending on whether the big integers `a` and `b` compare less, equal, or greater, respectively.
- `add(a, b)` returns the sum of big integers `a` and `b`.
- `sub(a, b)` returns the difference of big integers `a` and `b`.
- `mul(a, b)` returns the product of big integers `a` and `b`.
- `div(a, b)` returns the quotient of big integers `a` and `b`.

#### Time Complexity:

- $O(n)$  per call to the constructor, `to_string()`, `comp()`, `add()`, and `sub()`, where  $n$  is the total number of digits in the arguments and result for each operation.

- $O(n \cdot m)$  per call to `mul(a, b)` and `div(a, b)` where  $n$  is the number of digits in `a` and  $m$  is the number of digits in `b`.

### Space Complexity:

- $O(n)$  for storage of the big integer, where  $n$  is the number of digits.
- $O(n)$  auxiliary for `to_string()`, `add()`, `sub()`, `mul()`, and `div()`, where  $n$  is the total number of digits in the arguments and result for each operation.

```
#include <algorithm>
#include <cassert>
#include <cctype>
#include <cstdint>
#include <cstddef>
#include <cstdlib>
#include <string>
#include <string>

class BigInt {
    std::string digits;
    int sign;

    void normalize() {
        std::size_t pos = digits.find_last_not_of('0');
        if (pos != std::string::npos) {
            digits.erase(pos + 1);
        } else {
            digits = "0"; // An all-zero (or empty) magnitude collapses to a single "0".
        }
        if (digits.size() == 1 && digits[0] == '0') {
            sign = 1;
        }
    }

    static int comp(const std::string &a, const std::string &b, int asign, int bsign) {
        if (asign != bsign) {
            return asign < bsign ? -1 : 1;
        }
        if (a.size() != b.size()) {
            return a.size() < b.size() ? -asign : asign;
        }
        for (int i = static_cast<int>(a.size()) - 1; i >= 0; i--) {
            if (a[i] != b[i]) {
                return a[i] < b[i] ? -asign : asign;
            }
        }
        return 0;
    }

    static BigInt add(const std::string &a, const std::string &b, int asign, int bsign) {
        if (asign != bsign) {
            return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
        }
        BigInt res;
        res.sign = asign;
        res.digits.resize(std::max(a.size(), b.size()) + 1, '0');
        for (int i = 0, carry = 0; i < static_cast<int>(res.digits.size()); i++) {
            int d = carry;
            if (i < static_cast<int>(a.size())) {
                d += a[i] - '0';
            }
            if (i < static_cast<int>(b.size())) {
                d += b[i] - '0';
            }
            res.digits[i] = '0' + (d % 10);
            carry = d / 10;
        }
        res.normalize();
    }
};
```



```

    return res;
}

static BigInt sub(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {
        return add(a, b, asign, -bsign);
    }
    BigInt res;
    if (comp(a, b, asign, bsign) < 0) {
        res = sub(b, a, bsign, asign);
        res.sign = -1;
        return res;
    }
    res.digits.assign(a.size(), '0');
    for (int i = 0, borrow = 0; i < static_cast<int>(res.digits.size()); i++) {
        int d = (i < static_cast<int>(b.size()) ? a[i] - b[i] : a[i] - '0') - borrow;
        if (a[i] > '0') {
            borrow = 0;
        }
        if (d < 0) {
            d += 10;
            borrow = 1;
        }
        res.digits[i] = '0' + (d % 10);
    }
    res.normalize();
    return res;
}

public:
    BigInt(int64_t n = 0) {
        sign = (n < 0) ? -1 : 1;
        if (n == 0) {
            digits = "0";
            return;
        }
        for (; n != 0; n /= 10) {
            int digit = static_cast<int>(n % 10);
            digits += '0' + (digit < 0 ? -digit : digit);
        }
        normalize();
    }

    BigInt(const std::string &s) {
        if (s.empty() || (s[0] == '-' && s.size() == 1)) {
            throw std::runtime_error("Invalid string format to construct BigInt.");
        }
        digits.assign(s.rbegin(), s.rend());
        if (s[0] == '-') {
            sign = -1;
            digits.erase(digits.size() - 1);
        } else {
            sign = 1;
        }
        if (digits.find_first_not_of("0123456789") != std::string::npos) {
            throw std::runtime_error("Invalid string format to construct BigInt.");
        }
        normalize();
    }

    std::string to_string() const {
        return (sign < 0 ? "-" : "") + std::string(digits.rbegin(), digits.rend());
    }

    friend int comp(const BigInt &a, const BigInt &b) {
        return comp(a.digits, b.digits, a.sign, b.sign);
    }
}

```

```

friend BigInt add(const BigInt &a, const BigInt &b) {
    return add(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt sub(const BigInt &a, const BigInt &b) {
    return sub(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt mul(const BigInt &a, const BigInt &b) {
    BigInt res, row(a);
    for (char dc : b.digits) {
        for (int j = 0; j < (dc - '0'); j++) {
            res = add(res.digits, row.digits, res.sign, row.sign);
        }
        if (row.digits.size() > 1 || row.digits[0] != '0') {
            row.digits.insert(0, "0");
        }
    }
    res.sign = a.sign * b.sign;
    res.normalize();
    return res;
}

friend BigInt div(const BigInt &a, const BigInt &b) {
    assert(b.digits.size() != 1 || b.digits[0] != '0');
    BigInt res, row;
    res.digits.assign(a.digits.size(), '0');
    row.digits.clear(); // Running remainder; kept normalized and nonnegative below.
    for (int i = static_cast<int>(a.digits.size()) - 1; i >= 0; i--) {
        row.digits.insert(row.digits.begin(), a.digits[i]);
        row.normalize(); // Strip the spurious high-order zero left by the previous remainder.
        // Subtract while the remainder is >= b. A strict ">" undercounts the quotient digit when the
        // remainder equals b, and leaves the remainder too large, overflowing the next digit past 9.
        while (comp(row.digits, b.digits, 1, 1) >= 0) {
            res.digits[i]++;
            row = sub(row.digits, b.digits, 1, 1);
        }
    }
    res.sign = a.sign * b.sign;
    res.normalize();
    return res;
}
};

```

## Example Usage

```

int main() {
    BigInt a("-9899819294989142124"), b("12398124981294214");
    assert(add(a, b).to_string() == "-9887421170007847910");
    assert(sub(a, b).to_string() == "-9912217419970436338");
    assert(mul(a, b).to_string() == "-122739196911503356525379735104870536");
    assert(div(a, b).to_string() == "-798");
    assert(comp(a, b) == -1 && comp(a, a) == 0 && comp(b, a) == 1);
    assert(BigInt(INT64_MIN).to_string() == "-9223372036854775808");
    return 0;
}

```

## 6.4.2 Big Integer

Perform operations on arbitrary precision big integers internally represented as a vector of base-`BASE` digits in little-endian order. `BASE` defaults to  $10^9$  and must be a power of 10 no larger than  $10^9$ . Multiplication selects schoolbook, Karatsuba, or FFT from the operand size; `MUL_CUTOFF` is the padded transform length where FFT

takes over and normally needs no adjustment. Division likewise splits into a quadratic loop for small divisors and a recursive one past `DIV_CUTOFF` limbs, so it inherits whichever multiplication the size selects. Typical arithmetic operations involving mixed numeric primitives and strings are supported through implicit construction and hidden friend operators, as long as at least one operand is a `BigInt` at any given level of evaluation.

Addition, subtraction, and comparisons are performed using the standard linear algorithms. Multiplication first converts limbs from base `BASE` to the narrower base `MUL_BASE` so coefficient products fit safely, then multiplies with schoolbook, Karatsuba, or (if beyond `MUL_CUTOFF`) complex FFT convolution according to the operand size, converting the result back. Division and modulo are computed together by normalized long division: scale the operands so the divisor's leading limb is large, estimate each quotient limb from the top one or two limbs of the running remainder, correct the estimate by adding the divisor back if necessary, then unscale the remainder. Beyond `DIV_CUTOFF` limbs, that loop becomes the base case of Burnikel-Ziegler recursion, which halves both operands and recovers each half of the quotient from a division of the same shape, paying one multiplication per level rather than one estimate per limb.

- `BigInt()` constructs zero, and `BigInt(n)` constructs a big integer from an integer `n`.
- `BigInt(s)` constructs a big integer from a C string or an `std::string s`.
- `operator=` is defined to copy from another big integer or to assign from a 64-bit integer primitive.
- `size()` returns the number of digits in the base-10 representation.
- Operators `>>` and `<<` are defined to support stream-based input and output.
- `to_string()`, `to_llong()`, `to_double()`, and `to_ldouble()` return the big integer converted to an `std::string`, `int64_t`, `double`, and `long double` respectively. For the latter three data types, overflow behavior is based on that of inputting from `std::istream`. Equivalent conversions are also available through explicit casts to `int`, `long long`, `double`, and `long double`.
- `abs()` returns the absolute value.
- `comp(v)` returns `-1`, `0`, or `1` depending on whether the big integer compares less than, equal to, or greater than `v`, respectively.
- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on integer primitives.
- `div(v)` returns a pair consisting of the quotient and remainder after dividing by `v`.
- `pow(n)` returns the big integer raised to the nonnegative power `n` using binary exponentiation.
- `mul_pow10(n)` returns the big integer multiplied by  $10^n$  for nonnegative `n`, which costs a limb shift rather than a multiplication.
- `sqrt()` returns the integral part of the square root using a digit-by-digit algorithm in the internal base.
- `nth_root(n)` returns the integral part of the `n`-th root using Newton's method. Negative inputs require odd `n`.
- `rand(d)` returns a random, positive big integer with `d` digits.

#### Time Complexity:

- $O(1)$  per call to `size()`.
- $O(d)$  per call to the constructors, `to_string()`, `to_llong()`, `to_double()`, `to_ldouble()`, `abs()`, `comp()`, `rand()`, and all comparison and arithmetic operators except multiplication, division, and modulo, where  $d$  is the total number of digits in the arguments and result for each operation.
- $O(d \cdot \log(d) \cdot \log(\log(d)))$  per call to multiplication operations once operands reach `MUL_CUTOFF`, and  $O(d^{1.585})$  below it.
- $O((d/m) \cdot M(m) \cdot \log m)$  per call to division and modulo operations once the divisor reaches `DIV_CUTOFF`, and  $O(d \cdot m)$  below it, where  $d$  and  $m$  are the number of digits in the dividend and divisor respectively, and  $M(m)$  is the cost of one multiplication of two  $m$ -digit big integers. Both forms consume the dividend in  $d/m$  blocks of the divisor's size; only the per-block cost differs.
- $O(M(d) \log n)$  per call to `pow()`, where  $n$  is the exponent and  $d$  is the maximum digit length reached during exponentiation.
- $O(d + n)$  per call to `mul_pow10()`, where  $n$  is the power of ten.
- $O(d^2)$  per call to `sqrt()`.

- $O(M(d) \cdot \log(d) \cdot \log(d \cdot n))$  per call to `nth_root()`, where  $n$  is the root. Newton's iteration starts within a factor of ten of the answer and doubles the correct digits each step, so it converges in  $O(\log d)$  steps, each costing one exponentiation and one division.

### Space Complexity:

- $O(d)$  for storage of the big integer.
- $O(d)$  auxiliary for negation, addition, subtraction, multiplication, division, `abs()`, `sqr()`, `pow()`, and `nth_root()`.
- $O(1)$  auxiliary for all other operations.

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <climits>
#include <cmath>
#include <complex>
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <istream>
#include <iterator>
#include <ostream>
#include <random>
#include <sstream>
#include <string>
#include <utility>
#include <vector>

class BigInt {
    static const int BASE = 1000000000, BASE_DIGITS = 9; // BASE must equal 10^BASE_DIGITS.
    // Multiplication rebases to half as many decimal digits per limb, so that coefficient products
    // stay exact both in int64_t and within the FFT's double precision.
    static const int MUL_BASE = 10000, MUL_BASE_DIGITS = 4; // Keep consistent with BASE_DIGITS.
    static const int MUL_CUTOFF = 256; // Padded MUL_BASE length past which FFT beats Karatsuba.
    static const int DIV_CUTOFF = 64; // Divisor limb count past which recursion wins.
    static_assert(
        BASE >= 100 && BASE <= 1000000000,
        "BASE must be between 10^2 and 10^9; below that MUL_BASE_DIGITS would round down to zero"
    );
    static_assert(
        MUL_BASE_DIGITS == BASE_DIGITS / 2 && MUL_BASE <= 10000,
        "MUL_BASE must hold half of BASE's digits and stay exact under the FFT"
    );

    using vint = std::vector<int>;
    using vint64 = std::vector<int64_t>;
    using vcd = std::vector<std::complex<double>>;

    vint digits;
    int sign;

    void normalize() {
        while (!digits.empty() && digits.back() == 0) {
            digits.pop_back();
        }
        if (digits.empty()) {
            sign = 1;
        }
    }

    void read(int n, const char *s) {
        sign = 1;
        digits.clear();
        int pos = 0;
        while (pos < n && (s[pos] == '-' || s[pos] == '+')) {
            if (s[pos] == '-') {

```

```

        sign = -sign;
    }
    pos++;
}
for (int i = n - 1; i >= pos; i -= BASE_DIGITS) {
    int x = 0;
    for (int j = std::max(pos, i - BASE_DIGITS + 1); j <= i; j++) {
        x = x * 10 + s[j] - '0';
    }
    digits.push_back(x);
}
normalize();
}

static int comp(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return asign < bsign ? -1 : 1;
    }
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -asign : asign;
    }
    for (int i = static_cast<int>(a.size()) - 1; i >= 0; i--) {
        if (a[i] != b[i]) {
            return a[i] < b[i] ? -asign : asign;
        }
    }
    return 0;
}

static BigInt add(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
    }
    BigInt res;
    res.digits = a;
    res.sign = asign;
    int carry = 0, size = static_cast<int>(std::max(a.size(), b.size()));
    for (int i = 0; i < size || carry; i++) {
        if (i == static_cast<int>(res.digits.size())) {
            res.digits.push_back(0);
        }
        res.digits[i] += carry + (i < static_cast<int>(b.size()) ? b[i] : 0);
        carry = (res.digits[i] >= BASE) ? 1 : 0;
        if (carry) {
            res.digits[i] -= BASE;
        }
    }
    return res;
}

static BigInt sub(const vint &a, const vint &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {
        return add(a, b, asign, -bsign);
    }
    BigInt res;
    if (comp(a, b, asign, bsign) < 0) {
        res = sub(b, a, bsign, asign);
        res.sign = -1;
        return res;
    }
    res.digits = a;
    res.sign = asign;
    for (int i = 0, borrow = 0; i < static_cast<int>(a.size()) || borrow; i++) {
        res.digits[i] -= borrow + (i < static_cast<int>(b.size()) ? b[i] : 0);
        borrow = res.digits[i] < 0;
        if (borrow) {
            res.digits[i] += BASE;
        }
    }
}

```

```

    }
    res.normalize();
    return res;
}

static vint convert_base(const vint &digits, int l1, int l2) {
    vint64 p(std::max(l1, l2) + 1);
    p[0] = 1;
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        p[i] = p[i - 1] * 10;
    }
    vint res;
    int64_t curr = 0;
    for (int i = 0, curr_digits = 0; i < static_cast<int>(digits.size()); i++) {
        curr += digits[i] * p[curr_digits];
        curr_digits += l1;
        while (curr_digits >= l2) {
            res.push_back(static_cast<int>((curr % p[l2])));
            curr /= p[l2];
            curr_digits -= l2;
        }
    }
    res.push_back(static_cast<int>(curr));
    while (!res.empty() && res.back() == 0) {
        res.pop_back();
    }
    return res;
}

template<typename It>
static vint64 karatsuba(It alo, It ahi, It blo, It bhi) {
    int n = std::distance(alo, ahi), k = n / 2;
    vint64 res(n * 2);
    if (n <= 32) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                res[i + j] += alo[i] * blo[j];
            }
        }
        return res;
    }
    auto a1b1 = karatsuba(alo, alo + k, blo, blo + k);
    auto a2b2 = karatsuba(alo + k, ahi, blo + k, bhi);
    vint64 a2(alo + k, ahi), b2(blo + k, bhi);
    for (int i = 0; i < k; i++) {
        a2[i] += alo[i];
        b2[i] += blo[i];
    }
    auto r = karatsuba(a2.begin(), a2.end(), b2.begin(), b2.end());
    for (int i = 0; i < static_cast<int>(a1b1.size()); i++) {
        r[i] -= a1b1[i];
        res[i] += a1b1[i];
    }
    for (int i = 0; i < static_cast<int>(a2b2.size()); i++) {
        r[i] -= a2b2[i];
        res[i + n] += a2b2[i];
    }
    for (int i = 0; i < static_cast<int>(r.size()); i++) {
        res[i + k] += r[i];
    }
    return res;
}

template<typename It>
static vcd fft(It lo, It hi, bool invert = false) {
    int n = std::distance(lo, hi), k = 0, high1 = -1;
    while ((1 << k) < n) {
        k++;
    }

```

```

}
std::vector<int> rev(n);
for (int i = 1; i < n; i++) {
    if (!(i & (i - 1))) {
        high1++;
    }
    rev[i] = rev[i ^ (1 << high1)];
    rev[i] |= (1 << (k - high1 - 1));
}
vcd roots(n), res(n);
for (int i = 0; i < n; i++) {
    double alpha = 2 * 3.14159265358979323846 * i / n;
    roots[i] = std::complex<double>(std::cos(alpha), std::sin(alpha));
    res[i] = *(lo + rev[i]);
}
for (int len = 1; len < n; len <= 1) {
    vcd tmp(n);
    int rstep = static_cast<int>(roots.size()) / (len < 1);
    for (int pdest = 0; pdest < n; pdest += len) {
        int p = pdest;
        for (int i = 0; i < len; i++) {
            std::complex<double> c = roots[i * rstep] * res[p + len];
            tmp[pdest] = res[p] + c;
            tmp[pdest + len] = res[p] - c;
            pdest++;
            p++;
        }
    }
    res.swap(tmp);
}
if (invert) {
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        res[i] /= n;
    }
    std::reverse(res.begin() + 1, res.end());
}
return res;
}

// Schoolbook long division of nonnegative values, quadratic in the limb counts.
std::pair<BigInt, BigInt> div_schoolbook(const BigInt &v) const {
    int norm = BASE / (v.digits.back() + 1);
    BigInt an = *this * norm, bn = v * norm, q, r;
    q.digits.resize(an.digits.size());
    for (int i = static_cast<int>(an.digits.size()) - 1; i >= 0; i--) {
        r *= BASE;
        r += an.digits[i];
        int s1 = (r.digits.size() <= bn.digits.size()) ? 0 : r.digits[bn.digits.size()];
        int s2 = (r.digits.size() <= bn.digits.size() - 1) ? 0 : r.digits[bn.digits.size() - 1];
        int d = (static_cast<int64_t>(s1) * BASE + s2) / bn.digits.back();
        for (r -= bn * d; r < 0; r += bn) {
            d--;
        }
        q.digits[i] = d;
    }
    q.normalize();
    r.normalize();
    return {q, r / norm};
}

// Burnikel-Ziegler recursive division of nonnegative values. Balanced halves replace the
// quadratic inner loop, paying one multiplication per level instead of one estimate per limb, so
// division inherits whichever multiplication the operand size selects.
std::pair<BigInt, BigInt> div_recursive(const BigInt &v) const {
    auto shifted = [](const BigInt &x, int k) { // Multiplies by BASE^k.
        BigInt res(x);
        if (!res.digits.empty()) {
            res.digits.insert(res.digits.begin(), k, 0);
        }
    };

```

```

    }
    return res;
};

auto slice = [](const BigInt &x, int lo, int hi) { // Limbs [lo, hi), or 0 if empty range.
    BigInt res;
    lo = std::max(lo, 0);
    hi = std::min(hi, static_cast<int>(x.digits.size()));
    if (lo < hi) {
        res.digits.assign(x.digits.begin() + lo, x.digits.begin() + hi);
    }
    res.normalize();
    return res;
};

// Divides at most 2n limbs by exactly n limbs whose leading limb is at least BASE / 2, where
// the quotient is known to be below BASE^n. Each half of the quotient comes from one
// three-by-two block division, which itself divides two half-size blocks by one.
auto rec = [&](auto &&rec, const BigInt &a, const BigInt &b,
               int n) -> std::pair<BigInt, BigInt> {
    if (n % 2 != 0 || n < DIV_CUTOFF) {
        return a.div_schoolbook(b);
    }
    int k = n / 2;
    BigInt hi = slice(b, k, 2 * k), lo = slice(b, 0, k);
    auto block = [&](const BigInt &x) { // At most 3k limbs by b, with quotient below BASE^k.
        BigInt q, r;
        if (slice(x, 2 * k, 3 * k).comp(hi) < 0) {
            std::tie(q, r) = rec(rec, slice(x, k, 3 * k), hi, k);
        } else {
            // The estimate saturates: the leading limbs already reach the divisor, so take the
            // largest quotient the block can hold and recover the matching remainder.
            q = shifted(BigInt(1), k) - 1;
            r = slice(x, k, 3 * k) - hi * q;
        }
        r = shifted(r, k) + slice(x, 0, k) - q * lo;
        while (r < 0) { // At most twice, since the divisor's leading limb is at least BASE / 2.
            r += b;
            q -= 1;
        }
        return std::make_pair(q, r);
    };
    auto top = block(slice(a, k, 4 * k));
    auto bottom = block(shifted(top.second, k) + slice(a, 0, k));
    return {shifted(top.first, k) + bottom.first, bottom.second};
};

// Scale so the divisor's leading limb exceeds BASE / 2, then pad it to a multiple of a power
// of two so that every recursive split stays even until the blocks fall under DIV_CUTOFF, and
// consume the dividend one block at a time.
int scale = BASE / (v.digits.back() + 1);
BigInt a = *this * scale, b = v * scale;
int limbs = static_cast<int>(b.digits.size()), block = 1;
while (block * DIV_CUTOFF < limbs) {
    block *= 2;
}
int n = (limbs + block - 1) / block * block, sigma = n - limbs;
a = shifted(a, sigma);
b = shifted(b, sigma);
BigInt q, r;
for (int i = (static_cast<int>(a.digits.size()) + n - 1) / n - 1; i >= 0; i--) {
    auto step = rec(rec, shifted(r, n) + slice(a, i * n, (i + 1) * n), b, n);
    q = shifted(q, n) + step.first;
    r = step.second;
}
// Both scalings scale the remainder and leave the quotient alone.
return {q, slice(r, sigma, static_cast<int>(r.digits.size())) / scale};
}

public:
    BigInt() : sign(1) {}

```



```

BigInt(int v) { *this = static_cast<int64_t>(v); }
BigInt(int64_t v) { *this = v; }
BigInt(const char *s) { read(strlen(s), s); }
BigInt(const std::string &s) { read(s.size(), s.c_str()); }

BigInt &operator=(int64_t v) {
    sign = v < 0 ? -1 : 1;
    digits.clear();
    for (; v != 0; v /= BASE) {
        int limb = static_cast<int>(v % BASE);
        digits.push_back(limb < 0 ? -limb : limb);
    }
    return *this;
}

int size() const {
    return digits.empty() ? 1
        : static_cast<int>(std::to_string(digits.back()).size()) +
          BASE_DIGITS * (static_cast<int>(digits.size()) - 1);
}

friend std::istream &operator>>(std::istream &in, BigInt &v) {
    std::string s;
    in >> s;
    v.read(s.size(), s.c_str());
    return in;
}

friend std::ostream &operator<<(std::ostream &out, const BigInt &v) {
    return out << v.to_string();
}

std::string to_string() const {
    std::ostringstream oss;
    if (sign == -1) {
        oss << '-';
    }
    oss << (digits.empty() ? 0 : digits.back());
    for (int i = static_cast<int>(digits.size()) - 2; i >= 0; i--) {
        oss << std::setw(BASE_DIGITS) << std::setfill('0') << digits[i];
    }
    return oss.str();
}

template<typename T>
T to_arithmetic() const {
    std::stringstream ss(to_string());
    T res;
    ss >> res;
    return res;
}

int64_t to_llong() const { return to_arithmetic<int64_t>(); }
double to_double() const { return to_arithmetic<double>(); }
long double to_ldouble() const { return to_arithmetic<long double>(); }

explicit operator int() const { return static_cast<int>(to_llong()); }
explicit operator long long() const { return static_cast<long long>(to_llong()); }
explicit operator double() const { return to_double(); }
explicit operator long double() const { return to_ldouble(); }

int comp(const BigInt &v) const { return comp(digits, v.digits, sign, v.sign); }

// The comparison and binary arithmetic operators are hidden friends, so a raw integer operand on
// either side converts through the implicit constructor.
friend bool operator<(const BigInt &a, const BigInt &b) { return a.comp(b) < 0; }
friend bool operator>(const BigInt &a, const BigInt &b) { return a.comp(b) > 0; }
friend bool operator<=(const BigInt &a, const BigInt &b) { return a.comp(b) <= 0; }

```

```

friend bool operator>=(const BigInt &a, const BigInt &b) { return a.comp(b) >= 0; }
friend bool operator==(const BigInt &a, const BigInt &b) { return a.comp(b) == 0; }
friend bool operator!=(const BigInt &a, const BigInt &b) { return a.comp(b) != 0; }

BigInt abs() const {
    BigInt res(*this);
    res.sign = 1;
    return res;
}

BigInt operator-() const {
    BigInt res(*this);
    if (!digits.empty()) res.sign = -sign;
    return res;
}

friend BigInt operator+(const BigInt &a, const BigInt &b) {
    return add(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt operator-(const BigInt &a, const BigInt &b) {
    return sub(a.digits, b.digits, a.sign, b.sign);
}

BigInt &operator*=(int v) {
    int64_t factor = v;
    if (factor < 0) {
        sign = -sign;
        factor = -factor;
    }
    int64_t carry = 0;
    for (int i = 0; i < static_cast<int>(digits.size()) || carry; i++) {
        if (i == static_cast<int>(digits.size())) {
            digits.push_back(0);
        }
        int64_t curr = digits[i] * factor + carry;
        carry = curr / BASE;
        digits[i] = static_cast<int>(curr % BASE);
    }
    normalize();
    return *this;
}

BigInt operator*(int v) const {
    BigInt res(*this);
    res *= v;
    return res;
}

friend BigInt operator*(const BigInt &u, const BigInt &v) {
    vint a = convert_base(u.digits, BASE_DIGITS, MUL_BASE_DIGITS);
    vint b = convert_base(v.digits, BASE_DIGITS, MUL_BASE_DIGITS);
    int n = 1;
    while (n < 2 * static_cast<int>(std::max(a.size(), b.size()))) {
        n <= 1;
    }
    a.resize(n, 0);
    b.resize(n, 0);
    vint64 c;
    if (n < MUL_CUTOFF) {
        c = karatsuba(a.begin(), a.end(), b.begin(), b.end());
    } else {
        auto at = fft(a.begin(), a.end()), bt = fft(b.begin(), b.end());
        for (int i = 0; i < n; i++) {
            at[i] *= bt[i];
        }
        at = fft(at.begin(), at.end(), true);
        c.resize(n);
    }
}

```

```

    for (int i = 0; i < n; i++) {
        c[i] = at[i].real() + 0.5;
    }
}

BigInt res;
res.sign = u.sign * v.sign;
for (int i = 0, carry = 0; i < static_cast<int>(c.size()); i++) {
    int64_t d = c[i] + carry;
    res.digits.push_back(d % MUL_BASE);
    carry = d / MUL_BASE;
}
res.digits = convert_base(res.digits, MUL_BASE_DIGITS, BASE_DIGITS);
res.normalize();
return res;
}

BigInt &operator/=(int v) {
    assert(v != 0);
    int64_t divisor = v;
    if (divisor < 0) {
        sign = -sign;
        divisor = -divisor;
    }
    int64_t rem = 0;
    for (int i = static_cast<int>(digits.size()) - 1; i >= 0; i--) {
        int64_t curr = digits[i] + rem * static_cast<int64_t>(BASE);
        digits[i] = static_cast<int>((curr / divisor));
        rem = curr % divisor;
    }
    normalize();
    return *this;
}

BigInt operator/(int v) const {
    BigInt res(*this);
    res /= v;
    return res;
}

int operator%(int v) const {
    assert(v != 0);
    int64_t divisor = v;
    if (divisor < 0) {
        divisor = -divisor;
    }
    int64_t m = 0;
    for (int i = static_cast<int>(digits.size()) - 1; i >= 0; i--) {
        m = (digits[i] + m * static_cast<int64_t>(BASE)) % divisor;
    }
    return static_cast<int>(m * sign);
}

std::pair<BigInt, BigInt> div(const BigInt &v) const {
    assert(v != 0);
    if (comp(digits, v.digits, 1, 1) < 0) {
        return {BigInt(0), *this};
    }
    auto res = static_cast<int>(v.digits.size()) < DIV_CUTOFF ? abs().div_schoolbook(v.abs())
        : abs().div_recursive(v.abs());

    res.first.sign = sign * v.sign;
    res.second.sign = sign;
    res.first.normalize();
    res.second.normalize();
    return res;
}

friend BigInt operator/(const BigInt &a, const BigInt &b) { return a.div(b).first; }
friend BigInt operator%(const BigInt &a, const BigInt &b) { return a.div(b).second; }

```

```

BigInt operator++(int){ BigInt t(*this); operator++(); return t; }
BigInt operator--(int){ BigInt t(*this); operator--(); return t; }
BigInt &operator++() { *this = *this + BigInt(1); return *this; }
BigInt &operator--() { *this = *this - BigInt(1); return *this; }
BigInt &operator+=(const BigInt &v) { *this = *this + v; return *this; }
BigInt &operator-=(const BigInt &v) { *this = *this - v; return *this; }
BigInt &operator*=(const BigInt &v) { *this = *this * v; return *this; }
BigInt &operator/=(const BigInt &v) { *this = *this / v; return *this; }
BigInt &operator%=(const BigInt &v) { *this = *this % v; return *this; }

BigInt pow(int n) const {
    assert(n >= 0);
    if (n == 0) {
        return BigInt(1);
    }
    if (*this == 0) {
        return BigInt(0);
    }
    BigInt x(*this), res(1);
    for (; n != 0; n >>= 1) {
        if (n & 1) {
            res *= x;
        }
        x *= x;
    }
    return res;
}

// Multiplying by a power of ten is a limb shift, since BASE is a power of ten by construction.
BigInt mul_pow10(int n) const {
    assert(n >= 0);
    static const int POW10[] = {1, 10, 100, 1000, 10000, 100000, 1000000, 10000000, 100000000};
    BigInt res(*this);
    res.digits.insert(res.digits.begin(), n / BASE_DIGITS, 0);
    res *= POW10[n % BASE_DIGITS]; // Also normalizes, which is what clears a shifted zero.
    return res;
}

BigInt sqrt() const {
    assert(sign != -1);
    BigInt v(*this);
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    int n = static_cast<int>(v.digits.size());
    int ldig =
        static_cast<int> (::sqrt(static_cast<double>(v.digits[n - 1]) * BASE + v.digits[n - 2]));
    int norm = BASE / (ldig + 1);
    v *= norm;
    v *= norm;
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    BigInt r(static_cast<int64_t>(v.digits[n - 1]) * BASE + v.digits[n - 2]);
    int q = ldig =
        static_cast<int> (::sqrt(static_cast<double>(v.digits[n - 1]) * BASE + v.digits[n - 2]));
    BigInt res;
    for (int j = n / 2 - 1; j >= 0; j--) {
        for (; q--> 0) {
            BigInt r1 =
                (r - (res * 2 * BASE + q) * q) * BASE * BASE +
                (j > 0 ? static_cast<int64_t>(v.digits[2 * j - 1]) * BASE + v.digits[2 * j - 2] : 0);
            if (r1 >= 0) {
                r = r1;
                break;
            }
        }
    }
    res = res * BASE + q;
}

```

```

    if (j > 0) {
        int sz1 = static_cast<int>(res.digits.size());
        int sz2 = static_cast<int>(r.digits.size());
        int d1 = (sz1 + 2 < sz2) ? r.digits[sz1 + 2] : 0;
        int d2 = (sz1 + 1 < sz2) ? r.digits[sz1 + 1] : 0;
        int d3 = (sz1 < sz2) ? r.digits[sz1] : 0;
        q = (static_cast<int64_t>(d1) * BASE * BASE + static_cast<int64_t>(d2) * BASE + d3) /
            (ldig * 2);
    }
}
res.normalize();
return res / norm;
}

BigInt nth_root(int n) const {
    assert(n > 0 && (sign != -1 || n % 2 == 1));
    if (*this == 0) {
        return BigInt(0);
    }
    // Newton's iteration, started just above the answer and decreasing to it.
    BigInt magnitude = abs();
    BigInt x = BigInt(10).pow(static_cast<int>(std::ceil(static_cast<double>(size()) / n))), y;
    while (x.comp(y) = (x * (n - 1) + magnitude / x.pow(n - 1)) / n > 0) {
        x = y;
    }
    return sign == -1 ? -x : x;
}

static BigInt rand(int d) {
    if (d == 0) {
        return BigInt(0);
    }
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    std::uniform_int_distribution<int> first_digit(1, 9), digit(0, 9);
    std::string s(1, static_cast<char>('0' + first_digit(rng)));
    for (int i = 1; i < d; i++) {
        s += static_cast<char>('0' + digit(rng));
    }
    return BigInt(s);
}

friend int comp(const BigInt &a, const BigInt &b) { return a.comp(b); }
friend BigInt abs(const BigInt &v) { return v.abs(); }
friend BigInt pow(const BigInt &v, int n) { return v.pow(n); }
friend BigInt sqrt(const BigInt &v) { return v.sqrt(); }
friend BigInt nth_root(const BigInt &v, int n) { return v.nth_root(n); }
};

```

## Example Usage

```

using namespace std;

int main() {
    BigInt a("-9899819294989142124"), b("12398124981294214");
    assert(a + b == "-9887421170007847910");
    assert(a - b == "-9912217419970436338");
    assert(a * b == "-122739196911503356525379735104870536");
    assert(a / b == "-798");

    // Raw integers and strings work on either side of comparisons and arithmetic.
    assert("12398124981294214" == b);
    assert(5 + BigInt(7) == 12);
    assert(-100 < b && 5 % BigInt(3) == 2);

    assert(BigInt(20).pow(12345).size() == 16062);
    assert(BigInt("9812985918924981892491829").nth_root(4) == 1769906);
}

```

```

assert(BigInt(-8).nth_root(3) == -2);
for (int i = -100; i <= 100; i++) {
    if (i >= 0) {
        assert(BigInt(i).sqrt() == static_cast<int>(sqrt(i)));
    }
    for (int j = -100; j <= 100; j++) {
        assert(BigInt(i) + BigInt(j) == i + j);
        assert(BigInt(i) - BigInt(j) == i - j);
        assert(BigInt(i) * BigInt(j) == i * j);
        if (j != 0) {
            assert(BigInt(i) / BigInt(j) == i / j);
        }
        if (0 < i && i <= 10 && 0 < j && j <= 10) {
            assert(BigInt(i).nth_root(j) == static_cast<int64_t>((pow(i, 1.0 / j) + 1E-5)));
            int64_t p = 1;
            for (int k = 0; k < j; k++) {
                p *= i;
            }
            assert(BigInt(i).pow(j) == p);
        }
    }
}
}

mt19937 rng(1234567); // Fixed seed for reproducibility.
uniform_int_distribution<int> length_dist(1, 100);
for (int i = 0; i < 20; i++) {
    int d = length_dist(rng);
    BigInt value(BigInt::rand(d)), root(value.sqrt()), lower(root * root), upper(root + 1);
    upper *= upper;
    assert(lower <= value && value < upper);
    uniform_int_distribution<int> divisor_length_dist(1, d);
    BigInt divisor(BigInt::rand(divisor_length_dist(rng)) + 1), quotient(value / divisor);
    lower = quotient * divisor;
    upper = divisor * (quotient + 1);
    assert(value >= lower && value < upper);
}
BigInt x(-6);
assert(x.to_string() == "-6");
assert(x.to_llong() == -6LL);
assert(x.to_double() == -6.0);
assert(x.to_ldouble() == -6.0);
assert(static_cast<int>(x) == -6);
assert(static_cast<long long>(x) == -6LL);
assert(static_cast<double>(x) == -6.0);
assert(BigInt(INT64_MIN).to_string() == "-9223372036854775808");
assert(BigInt(INT64_MIN).to_llong() == INT64_MIN);
assert(-BigInt(0) == 0);
assert(BigInt(3) * INT_MIN == "-6442450944");
assert(BigInt("6442450944") / INT_MIN == -3);
assert(BigInt("6442450945") % INT_MIN == 1);
return 0;
}

```

### 6.4.3 Big Decimal

Performs exact decimal arithmetic using the full `BigInt` implementation from 6.4.2 as its coefficient type, together with a nonnegative scale. Unlike binary floating point, every parsed decimal is represented exactly: for example, `BigDecimal("1.20")` stores coefficient 120 with scale 2. Addition and subtraction preserve the larger operand scale, multiplication adds the scales, and formatting preserves trailing zeros unless `normalized()` is called. Only finite, well-formed decimal strings are supported; `NaN` and infinities are not represented.

Division and square root generally have nonterminating decimal expansions, so both operations require an explicit result scale. The rounding mode defaults to `HalfAway`, which rounds ties away from zero. The other modes are `TowardZero`, `AwayFromZero`, `Floor`, `Ceil`, and `HalfEven`.

- `BigDecimal(n = 0)` constructs a decimal from integer `n`.
- `BigDecimal(s)` parses decimal string `s`, optionally written in scientific notation, and preserves trailing zeros in its mantissa.
- `decimal_places()`, `precision()`, and `sign()` return the scale, the number of coefficient digits, and the sign as `-1`, `0`, or `1`, respectively.
- `unscaled_value()` returns the underlying `BigInt` coefficient.
- `abs()` and `normalized()` return the absolute value and a value with trailing fractional zeros removed.
- `move_point(places)` moves the decimal point right by `places` positions, or left if `places` is negative.
- `rescale(new_scale, mode = Rounding::HalfAway)` returns the value with exactly `new_scale` decimal places, rounding if the scale is reduced.
- `trunc()`, `floor()`, and `ceil()` return the corresponding integer as a `BigInt`.
- `divide(v, result_scale, mode = Rounding::HalfAway)` divides by nonzero `v` and rounds to exactly `result_scale` decimal places.
- `sqrt(result_scale, mode = Rounding::HalfAway)` returns the nonnegative square root rounded to exactly `result_scale` decimal places.
- `pow(n)` returns the value raised to nonnegative integer power `n`.
- `to_string()` returns ordinary decimal notation with the stored number of decimal places.
- Stream operators and comparisons are defined, as are `+`, `-`, `*`, `%`, and their compound assignments.

#### Time Complexity:

- $O(1)$  per call to `decimal_places()`, `precision()`, `sign()`, and `unscaled_value()`.
- $O(d)$  per call to parsing, formatting, normalization, addition, subtraction, comparisons, and decimal-point movement, where  $d$  is the number of relevant decimal digits.
- $O(M(d) \log n)$  per call to `pow(n)`, where  $M(d)$  is the multiplication cost for the largest intermediate coefficient.
- $O(d^2)$  per call to square root, matching the `BigInt` operation used here.
- Division, remainder, and rescaling to fewer places inherit the `BigInt` division cost of 6.4.2:  $O((d/m) \cdot M(m) \cdot \log m)$  once the divisor reaches its `DIV_CUTOFF` and  $O(d \cdot m)$  below it, where  $d$  and  $m$  are the number of digits in the dividend and divisor.

**Space Complexity:**  $O(d)$  for stored digits, returned values, and arithmetic results.

```
#define main bigint_example
#include "6.4.2_Big_Integer.cpp"
#undef main

class BigDecimal {
public:
    enum class Round { TowardZero, AwayFromZero, Floor, Ceil, HalfAway, HalfEven };

private:
    BigInt coeff;
    int scale;

    BigDecimal(BigInt coeff, int scale) : coeff(std::move(coeff)), scale(scale) {}
    BigInt scaled(int new_scale) const { return coeff.mul_pow10(new_scale - scale); }
    static BigInt pow10(int n) { return BigInt(10).pow(n); }

    static bool round_up(bool negative, int half_cmp, const BigInt &q, Round mode) {
        return mode == Round::AwayFromZero || (mode == Round::Floor && negative) ||
            (mode == Round::Ceil && !negative) || (mode == Round::HalfAway && half_cmp >= 0) ||
            (mode == Round::HalfEven && (half_cmp > 0 || (half_cmp == 0 && q % 2 != 0)));
    }

    static BigInt rounded_quotient(const BigInt &num, const BigInt &den, Round mode) {
        assert(den > 0);
        bool negative = num < 0;
        auto [q, r] = num.abs().div(den);
        bool increment = r != 0 && round_up(negative, (2 * r).comp(den), q, mode);
        return negative ? -(q + increment) : q + increment;
    }
}
```

```

public:
    BigDecimal(int64_t n = 0) : coeff(n), scale(0) {}

    explicit BigDecimal(const std::string &s) : coeff(0), scale(0) {
        size_t e = s.find_first_of("eE");
        std::string mantissa = s.substr(0, e);
        int exponent = e == std::string::npos ? 0 : std::stoi(s.substr(e + 1));
        int pos = 0;
        bool negative = false, point = false;
        if (pos < static_cast<int>(mantissa.size()) && (mantissa[pos] == '+' || mantissa[pos] == '-')) {
            negative = mantissa[pos++] == '-';
        }
        std::string digits;
        for (; pos < static_cast<int>(mantissa.size()); pos++) {
            if (mantissa[pos] == '.') {
                assert(!point);
                point = true;
            } else {
                assert('0' <= mantissa[pos] && mantissa[pos] <= '9');
                digits += mantissa[pos];
                scale += point;
            }
        }
        assert(!digits.empty());
        coeff = negative ? -BigInt(digits) : BigInt(digits);
        scale -= exponent;
        if (scale < 0) {
            coeff = coeff.mul_pow10(-scale);
            scale = 0;
        }
    }

    int decimal_places() const { return scale; }
    int precision() const { return coeff.size(); }
    int sign() const { return (coeff > 0) - (coeff < 0); }
    const BigInt &unscaled_value() const { return coeff; }
    BigDecimal abs() const { return BigDecimal(coeff.abs(), scale); }

    BigDecimal normalized() const {
        BigInt c = coeff;
        int s = scale;
        while (s > 0 && c % 10 == 0) {
            c /= 10;
            s--;
        }
        return BigDecimal(c, s);
    }

    BigDecimal move_point(int places) const {
        int new_scale = scale - places;
        return new_scale >= 0 ? BigDecimal(coeff, new_scale)
            : BigDecimal(coeff.mul_pow10(-new_scale), 0);
    }

    BigDecimal rescale(int new_scale, Round mode = Round::HalfAway) const {
        assert(new_scale >= 0);
        return new_scale >= scale
            ? BigDecimal(scaled(new_scale), new_scale)
            : BigDecimal(rounded_quotient(coeff, pow10(scale - new_scale), mode), new_scale);
    }

    BigInt trunc() const { return rescale(0, Round::TowardZero).coeff; }
    BigInt floor() const { return rescale(0, Round::Floor).coeff; }
    BigInt ceil() const { return rescale(0, Round::Ceil).coeff; }

    BigDecimal divide(const BigDecimal &v, int result_scale, Round mode = Round::HalfAway) const {
        assert(v.coeff != 0 && result_scale >= 0);
    }

```



```

    BigInt num = coeff.abs(), den = v.coeff.abs();
    int shift = v.scale + result_scale - scale;
    (shift >= 0 ? num : den) = (shift >= 0 ? num : den).mul_pow10(std::abs(shift));
    if ((coeff < 0) != (v.coeff < 0)) {
        num = -num;
    }
    return BigDecimal(rounded_quotient(num, den, mode), result_scale);
}

BigDecimal sqrt(int result_scale, Round mode = Round::HalfAway) const {
    assert(coeff >= 0 && result_scale >= 0);
    BigInt num = coeff, den = 1;
    int shift = 2 * result_scale - scale;
    (shift >= 0 ? num : den) = (shift >= 0 ? num : den).mul_pow10(std::abs(shift));
    BigInt q = (num / den).sqrt();
    bool exact = q * q * den == num;
    int half_cmp = (4 * num).comp(den * (4 * q * q + 4 * q + 1));
    return BigDecimal(q + (!exact && round_up(false, half_cmp, q, mode)), result_scale);
}

std::string to_string() const {
    std::string s = coeff.abs().to_string();
    if (static_cast<int>(s.size()) <= scale) {
        s.insert(0, scale + 1 - s.size(), '0');
    }
    if (scale > 0) {
        s.insert(s.size() - scale, 1, '.');
    }
    return coeff < 0 ? "-" + s : s;
}

friend std::istream &operator>>(std::istream &in, BigDecimal &v) {
    std::string s;
    if (in >> s) {
        v = BigDecimal(s);
    }
    return in;
}

friend std::ostream &operator<<(std::ostream &out, const BigDecimal &v) {
    return out << v.to_string();
}

friend bool operator<(const BigDecimal &a, const BigDecimal &b) {
    int s = std::max(a.scale, b.scale);
    return a.scaled(s) < b.scaled(s);
}

friend bool operator==(const BigDecimal &a, const BigDecimal &b) {
    int s = std::max(a.scale, b.scale);
    return a.scaled(s) == b.scaled(s);
}

friend bool operator>(const BigDecimal &a, const BigDecimal &b) { return b < a; }
friend bool operator<=(const BigDecimal &a, const BigDecimal &b) { return !(b < a); }
friend bool operator>=(const BigDecimal &a, const BigDecimal &b) { return !(a < b); }
friend bool operator!=(const BigDecimal &a, const BigDecimal &b) { return !(a == b); }

friend BigDecimal operator+(const BigDecimal &a, const BigDecimal &b) {
    int s = std::max(a.scale, b.scale);
    return BigDecimal(a.scaled(s) + b.scaled(s), s);
}

friend BigDecimal operator-(const BigDecimal &a, const BigDecimal &b) { return a + -b; }

friend BigDecimal operator*(const BigDecimal &a, const BigDecimal &b) {
    return BigDecimal(a.coeff * b.coeff, a.scale + b.scale);
}

```

```

friend BigDecimal operator%(const BigDecimal &a, const BigDecimal &b) {
    assert(b.coeff != 0);
    int s = std::max(a.scale, b.scale);
    return BigDecimal(a.scaled(s) % b.scaled(s), s);
}

BigDecimal operator-() const { return BigDecimal(-coeff, scale); }

BigDecimal pow(int n) const {
    assert(n >= 0);
    return BigDecimal(coeff.pow(n), scale * n);
}

BigDecimal &operator+=(const BigDecimal &v) { return *this = *this + v; }
BigDecimal &operator-=(const BigDecimal &v) { return *this = *this - v; }
BigDecimal &operator*=(const BigDecimal &v) { return *this = *this * v; }
BigDecimal &operator%=(const BigDecimal &v) { return *this = *this % v; }
};

```

## Example Usage

```

using namespace std;

int main() {
    using Round = BigDecimal::Round;

    assert(BigDecimal().to_string() == "0");
    assert(BigDecimal(-12).to_string() == "-12");
    assert(BigDecimal("+.50").to_string() == "0.50");
    assert(BigDecimal("1.20e3").to_string() == "1200");
    assert(BigDecimal("1.20e-2").to_string() == "0.0120");

    BigDecimal a("1.20"), b("3.4");
    assert(a.decimal_places() == 2 && a.precision() == 3);
    assert(a.unscaled_value() == 120);
    assert(a.sign() == 1 && BigDecimal(0).sign() == 0 && BigDecimal(-1).sign() == -1);
    assert(BigDecimal("-1.20").abs().to_string() == "1.20");
    assert(a.normalized().to_string() == "1.2");
    assert(a == BigDecimal("1.2") && a != b);
    assert(a < b && b > a && a <= BigDecimal("1.200") && b >= a);

    assert((a + b).to_string() == "4.60");
    assert((b - a).to_string() == "2.20");
    assert((a * b).to_string() == "4.080");
    assert((-a).to_string() == "-1.20");
    assert((BigDecimal("5.50") % 2).to_string() == "1.50");

    BigDecimal x = a;
    assert((x += b).to_string() == "4.60");
    assert((x -= b).to_string() == "1.20");
    assert((x *= b).to_string() == "4.080");
    assert((x /= 1.0).to_string() == "0.080");

    assert(a.move_point(2).to_string() == "120");
    assert(a.move_point(-1).to_string() == "0.120");
    assert(a.rescale(4).to_string() == "1.2000");
    assert(BigDecimal("1.29").rescale(1, Round::TowardZero).to_string() == "1.2");
    assert(BigDecimal("1.21").rescale(1, Round::AwayFromZero).to_string() == "1.3");
    assert(BigDecimal("-1.21").rescale(1, Round::Floor).to_string() == "-1.3");
    assert(BigDecimal("-1.21").rescale(1, Round::Ceil).to_string() == "-1.2");
    assert(BigDecimal("1.25").rescale(1, Round::HalfAway).to_string() == "1.3");
    assert(BigDecimal("2.685").rescale(2, Round::HalfEven).to_string() == "2.68");

    assert(BigDecimal(1).divide(8, 4).to_string() == "0.1250");
    assert(BigDecimal(2).divide(3, 4).to_string() == "0.6667");
}

```

```

assert(BigDecimal(-1).divide(8, 2).to_string() == "-0.13");
assert(BigDecimal(1).divide(8, 2, Round::HalfEven).to_string() == "0.12");

assert(BigDecimal("-1.2").trunc() == -1);
assert(BigDecimal("-1.2").floor() == -2);
assert(BigDecimal("-1.2").ceil() == -1);
assert(a.pow(3).to_string() == "1.728000");

assert(BigDecimal(2).sqrt(5).to_string() == "1.41421");
assert(BigDecimal("2.25").sqrt(2).to_string() == "1.50");
assert(BigDecimal("2.25").sqrt(0, Round::HalfAway).to_string() == "2");
assert(BigDecimal("6.25").sqrt(0, Round::HalfEven).to_string() == "2");
assert(BigDecimal(2).sqrt(0, Round::Ceil).to_string() == "2");
for (int i = 0; i <= 100; i++) {
    assert(BigDecimal(i * i).sqrt(0) == BigDecimal(i));
}

stringstream in("-12.340"), out;
BigDecimal streamed;
in >> streamed;
out << streamed;
assert(out.str() == "-12.340");

assert((BigDecimal("12345678901234567890.12") * 10).to_string() == "123456789012345678901.20");
return 0;
}

```

### 6.4.4 Rational Numbers

Perform operations on rational numbers internally represented as two integers: a numerator and a denominator. The template integer type must support streamed input/output, comparisons, and arithmetic operations.

- `Rational<Int>(n)` constructs a rational number with numerator `n` and denominator 1.
- `Rational<Int>(n, d)` constructs a rational number with numerator `n` and denominator `d`.
- `operator>>` inputs a rational number using the next integer from the stream as the numerator and 1 as the denominator.
- `operator<<` outputs the rational number as a string consisting of possibly a minus sign followed by the numerator, followed by a slash, followed by the denominator.
- `to_string()`, `to_llong()`, `to_double()`, and `to_ldouble()` return the rational converted to an `std::string`, `int64_t`, `double`, and `long double` respectively with potential truncation or approximation for the primitive types. Equivalent conversions are also available through explicit casts to `int`, `long long`, `double`, and `long double`.
- `to_arithmetic<T>()` is the general form behind the three numeric conversions above, returning the rational as any arithmetic type `T` readable from a stream. Integral `T` truncates toward zero.
- `abs()`, `floor()`, and `ceil()` return the absolute value, floor, and ceiling.
- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on numerical primitives. The comparisons and binary arithmetic are hidden friends, so a raw integer operand works on either side.

Overflow warning: Internal operations do not check for overflow. Comparisons and arithmetic cross-multiply numerators and denominators, so instantiate with a wider integer type (such as `__int128`, or even `BigInt`) if the values may grow large.

#### Time Complexity:

- $O(\log s)$  operations on `Int` per call to the constructor `Rational(n, d)`, where  $s = |n| + |d|$ , spent reducing to lowest terms.
- $O(1)$  operations on `Int` per call to everything else.

**Space Complexity:**

- Two `Int` values for storage of the rational number.
- $O(1)$  auxiliary `Int` values for all operations.

```

#include <cassert>
#include <cstdint>
#include <istream>
#include <ostream>
#include <sstream>
#include <string>

template<typename Int>
class Rational {
    Int num, den;

public:
    Rational() : num(0), den(1) {}
    Rational(const Int &n) : num(n), den(1) {}

    Rational(const Int &n, const Int &d) : num(n), den(d) {
        assert(den != 0);
        if (den < 0) {
            num = -num;
            den = -den;
        }
        Int a(num < 0 ? -num : num), b(den), tmp;
        while (a != 0 && b != 0) {
            tmp = a % b;
            a = b;
            b = tmp;
        }
        Int gcd = (b == 0) ? a : b;
        num /= gcd;
        den /= gcd;
    }

    friend std::istream &operator>>(std::istream &in, Rational &r) {
        in >> r.num;
        r.den = 1;
        return in;
    }

    friend std::ostream &operator<<(std::ostream &out, const Rational &r) {
        out << r.num << "/" << r.den;
        return out;
    }

    std::string to_string() const {
        std::stringstream ss;
        ss << num << "/" << den;
        return ss.str();
    }

    // Round-trips through a stringstream so that any Int supporting streamed I/O (e.g. a big-integer
    // type) converts, even without a direct cast to the target type. Integral T truncates.
    template<typename T>
    T to_arithmetic() const {
        std::stringstream ss;
        ss << num << " " << den;
        T n, d;
        ss >> n >> d;
        return n / d;
    }

    int64_t to_llong() const { return to_arithmetic<int64_t>(); }
    double to_double() const { return to_arithmetic<double>(); }
    long double to_ldouble() const { return to_arithmetic<long double>(); }

```

```

explicit operator int() const { return static_cast<int>(to_llong()); }
explicit operator long long() const { return static_cast<long long>(to_llong()); }
explicit operator double() const { return to_double(); }
explicit operator long double() const { return to_ldouble(); }

// The comparison and binary arithmetic operators are hidden friends, so a raw integer operand on
// either side converts through the implicit constructor.
friend bool operator<(const Rational &a, const Rational &b) {
    return a.num * b.den < b.num * a.den;
}

friend bool operator==(const Rational &a, const Rational &b) {
    return a.num == b.num && a.den == b.den;
}

friend bool operator>(const Rational &a, const Rational &b) { return b < a; }
friend bool operator<=(const Rational &a, const Rational &b) { return !(b < a); }
friend bool operator>=(const Rational &a, const Rational &b) { return !(a < b); }
friend bool operator!=(const Rational &a, const Rational &b) { return !(a == b); }

Rational abs() const { return Rational(num < 0 ? -num : num, den); }
friend Rational abs(const Rational &r) { return r.abs(); }
Int floor() const { return num < 0 ? -((-num + den - 1) / den) : num / den; }
Int ceil() const { return num < 0 ? -(-num / den) : (num + den - 1) / den; }

friend Rational operator+(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den + b.num * a.den, a.den * b.den);
}

friend Rational operator-(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den - b.num * a.den, a.den * b.den);
}

friend Rational operator*(const Rational &a, const Rational &b) {
    return Rational(a.num * b.num, a.den * b.den);
}

friend Rational operator/(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den, a.den * b.num);
}

friend Rational operator%(const Rational &a, const Rational &b) {
    return a - b * Rational(a.num * b.den / (b.num * a.den), 1);
}

Rational operator-() const { return Rational(-num, den); }
Rational operator++(int) { Rational t(*this); operator++(); return t; }
Rational operator--(int) { Rational t(*this); operator--(); return t; }
Rational &operator++() { *this = *this + 1; return *this; }
Rational &operator--() { *this = *this - 1; return *this; }
Rational &operator+=(const Rational &r) { *this = *this + r; return *this; }
Rational &operator-=(const Rational &r) { *this = *this - r; return *this; }
Rational &operator*=(const Rational &r) { *this = *this * r; return *this; }
Rational &operator/=(const Rational &r) { *this = *this / r; return *this; }
Rational &operator%=(const Rational &r) { *this = *this % r; return *this; }
};

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

```

```

int main() {
    using Rational = Rational<int64_t>;
    assert(Rational(-21, 1) % 2 == -1);
    Rational r(Rational(-53, 10) % Rational(-17, 10));
    assert(EQ(r.to_ldouble(), fmod(-5.3, -1.7)));
    assert(r.to_string() == "-1/5");
    assert(static_cast<int>(Rational(7, 2)) == 3);
    assert(static_cast<long long>(Rational(7, 2)) == 3LL);
    assert(EQ(static_cast<double>(Rational(1, 2)), 0.5));

    // Raw integers work on either side of comparisons and arithmetic.
    assert(2 + Rational(1, 2) == Rational(5, 2));
    assert(1 < Rational(3, 2) && Rational(3, 2) < 2);
    assert(2 % Rational(3, 4) == Rational(1, 2));

    // Construction reduces the fraction and moves any sign onto the numerator.
    assert(Rational(2, 4).to_string() == "1/2");
    assert(Rational(1, -2).to_string() == "-1/2");
    assert(Rational(5).to_string() == "5/1");

    // Arithmetic between rationals reduces the result too.
    assert(Rational(1, 2) + Rational(1, 3) == Rational(5, 6));
    assert(Rational(1, 2) - Rational(1, 3) == Rational(1, 6));
    assert(Rational(2, 3) * Rational(3, 4) == Rational(1, 2));
    assert(Rational(1, 2) / Rational(3, 4) == Rational(2, 3));

    // Rounding takes a different branch on each sign, and is exact on whole numbers.
    assert(Rational(7, 2).floor() == 3 && Rational(7, 2).ceil() == 4);
    assert(Rational(-7, 2).floor() == -4 && Rational(-7, 2).ceil() == -3);
    assert(Rational(4, 2).floor() == 2 && Rational(4, 2).ceil() == 2);
    assert(Rational(-7, 2).abs() == Rational(7, 2) && abs(Rational(-7, 2)) == Rational(7, 2));

    // The named conversions and the general form behind them.
    assert(Rational(7, 2).to_llong() == 3 && EQ(Rational(7, 2).to_double(), 3.5));
    assert(Rational(7, 2).to_arithmetic<int>() == 3);

    // Stream input reads one integer, and output prints the reduced fraction.
    Rational streamed;
    istreamstream in("7");
    in >> streamed;
    assert(streamed == 7);
    ostreamstream out;
    out << Rational(-3, 6);
    assert(out.str() == "-1/2");
    return 0;
}

```

## 6.5 Linear Algebra

---

### 6.5.1 Matrix Operations

Basic matrix operations defined on a two-dimensional vector of numeric values. Matrix arithmetic also supports modular value types such as `Modular<MOD>` from 6.3.3. The element type must be constructible from 0 and 1 and support the arithmetic operators used by the requested operation.

- `make_matrix<T>(m, n, v)` constructs and returns an  $m$  by  $n$  matrix with 0-based indices (row indices  $[0, m)$  and column indices  $[0, n)$ ), where every value is initialized to `v`.
- `identity_matrix<T>(n)` returns the  $n$  by  $n$  identity matrix, that is, a matrix where each `a[i][j]` equals 1 if `i == j`, or 0 otherwise.
- `rows(a)` returns the number of rows in matrix `a`.

- `cols(a)` returns the number of columns in matrix `a`.
- `a[i][j]` may be used to access or modify the specified entry of `a`.
- Operators `<`, `>`, `<=`, `>=`, `==`, and `!=` define lexicographical comparison based on that of `std::vector`.
- Operators `+`, `-`, `*`, `/`, `+=`, `-=`, `*=`, and `/=` define scalar addition, subtraction, multiplication, and division involving a matrix and a numeric scalar value.
- `a * v` returns the matrix-vector product as a vector.
- Operators `*` and `*=` define matrix multiplication.

Multiplication is the schoolbook triple loop, which is  $O(m \cdot n \cdot p)$  and is what contest constraints assume. Strassen's algorithm reaches  $O(n^{2.807})$  by splitting each matrix into quarters and combining them with seven recursive products instead of eight, and Toom-Cook style refinements go lower still, but the additions that replace the saved multiplication carry large constants and worsen numerical stability. The crossover sits in the hundreds of rows even in tuned libraries, so the plain loop below is the right default; reach for a blocked or cache-oblivious ordering before reaching for a subcubic algorithm.

Exponentiation uses iterative binary exponentiation: keep an accumulated result, square the base each round, and multiply it when the current exponent bit is set. The power sum uses the block identity that  $\begin{bmatrix} A & A \\ 0 & I \end{bmatrix}^p$  has

$A + A^2 + \dots + A^p$  in its upper-right block.

- Operators `^` and `^=` define iterative binary matrix exponentiation of a square matrix `a` by a `uint64_t` power `p`.
- `power_sum(a, p)` returns the power sum of a square matrix `a` up to a `uint64_t` power `p`, that is,  $a + a^2 + \dots + a^p$ .

### Time Complexity:

- $O(m \cdot n)$  per construction, output, comparison, or scalar-arithmetic operation on  $m$  by  $n$  matrices.
- $O(1)$  per call to `rows()` and `cols()`.
- $O(m \cdot n)$  per matrix-matrix addition or subtraction of  $m$  by  $n$  matrices.
- $O(n^3 \log p)$  per exponentiation of an  $n$  by  $n$  matrix to power  $p$ .
- $O(n^3 \log p)$  per call to `power_sum()` for an  $n$  by  $n$  matrix and power  $p$ .
- $O(m \cdot n)$  per multiplication of an  $m$  by  $n$  matrix by a vector of length  $n$ .
- $O(m \cdot n \cdot k)$  per multiplication of an  $m$  by  $n$  matrix by an  $n$  by  $k$  matrix.

### Space Complexity:

- $O(1)$  auxiliary for `rows()`, `cols()`, `a[i][j]` access, comparison and scalar compound operators.
- $O(n^2)$  auxiliary for exponentiation of an  $n$  by  $n$  matrix to power  $p$ .
- $O(n^2)$  auxiliary for `power_sum()` of an  $n$  by  $n$  matrix up to power  $p$ .
- $O(m)$  for the vector returned by matrix-vector multiplication.
- $O(m \cdot n)$  auxiliary for all non-in-place operations returning an  $m$  by  $n$  matrix.

```
#include <cassert>
#include <cstdint>
#include <iomanip>
#include <ostream>
#include <vector>

template<typename T>
using Matrix = std::vector<std::vector<T>>>;

template<typename T = int>
Matrix<T> make_matrix(int m, int n, const T &v = T{}) {
    return Matrix<T>(m, std::vector<T>(n, v));
}

template<typename T = int>
Matrix<T> identity_matrix(int n) {
    Matrix<T> res = make_matrix<T>(n, n);
    for (int i = 0; i < n; i++) {
        res[i][i] = 1;
    }
}
```

```

    }
    return res;
}

template<typename T>
int rows(const Matrix<T> &a) {
    return static_cast<int>(a.size());
}

template<typename T>
int cols(const Matrix<T> &a) {
    return a.empty() ? 0 : static_cast<int>(a[0].size());
}

template<typename T>
std::ostream &operator<<(std::ostream &out, const Matrix<T> &a) {
    auto flags = out.flags();
    auto precision = out.precision();
    out << std::fixed << std::setprecision(5);
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            out << std::setw(10) << a[i][j];
        }
        out << std::endl;
    }
    out.flags(flags);
    out.precision(precision);
    return out;
}

template<typename T, typename U>
Matrix<T> &operator+=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] += v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator-=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] -= v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator*=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] *= v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator/=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] /= v;
        }
    }
    return a;
}

```



```

}

template<typename T>
Matrix<T> &operator+=(Matrix<T> &a, const Matrix<T> &b) {
    assert(rows(a) == rows(b) && cols(a) == cols(b));
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] += b[i][j];
        }
    }
    return a;
}

template<typename T>
Matrix<T> &operator-=(Matrix<T> &a, const Matrix<T> &b) {
    assert(rows(a) == rows(b) && cols(a) == cols(b));
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            a[i][j] -= b[i][j];
        }
    }
    return a;
}

template<typename T>
Matrix<T> operator+(const Matrix<T> &a, const Matrix<T> &b) {
    Matrix<T> c(a);
    return c += b;
}

template<typename T>
Matrix<T> operator-(const Matrix<T> &a, const Matrix<T> &b) {
    Matrix<T> c(a);
    return c -= b;
}

template<typename T>
std::vector<T> operator*(const Matrix<T> &a, const std::vector<T> &v) {
    assert(cols(a) == static_cast<int>(v.size()));
    std::vector<T> res(rows(a));
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < cols(a); j++) {
            res[i] += a[i][j] * v[j];
        }
    }
    return res;
}

template<typename T>
Matrix<T> operator*(const Matrix<T> &a, const Matrix<T> &b) {
    assert(cols(a) == rows(b));
    Matrix<T> res = make_matrix<T>(rows(a), cols(b));
    // The k loop sits outside j so the inner loop walks res[i] and b[k] contiguously instead of
    // striding down a column of b. Each res[i][j] still accumulates in increasing k, so results are
    // unchanged, including for floating-point types.
    for (int i = 0; i < rows(a); i++) {
        for (int k = 0; k < rows(b); k++) {
            for (int j = 0; j < cols(b); j++) {
                res[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    return res;
}

template<typename T>
Matrix<T> &operator*=(Matrix<T> &a, const Matrix<T> &b) { return a = a * b; }

```

```

template<typename T, typename U>
Matrix<T> operator+(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m += v; }

template<typename T, typename U>
Matrix<T> operator-(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m -= v; }

template<typename T, typename U>
Matrix<T> operator*(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m *= v; }

template<typename T, typename U>
Matrix<T> operator/(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m /= v; }

template<typename T, typename U>
Matrix<T> operator+(const U &v, const Matrix<T> &a) { return a + v; }

template<typename T, typename U>
Matrix<T> operator*(const U &v, const Matrix<T> &a) { return a * v; }

template<typename T, typename U>
Matrix<T> operator-(const U &v, const Matrix<T> &a) {
    Matrix<T> m = make_matrix<T>(rows(a), cols(a), v);
    return m -= a;
}

template<typename T>
Matrix<T> operator^(Matrix<T> a, uint64_t p) {
    assert(rows(a) == cols(a));
    Matrix<T> res = identity_matrix<T>(rows(a));
    while (p > 0) {
        if (p & 1) {
            res *= a;
        }
        p >>= 1;
        if (p > 0) {
            a *= a;
        }
    }
    return res;
}

template<typename T>
Matrix<T> &operator^=(Matrix<T> &a, uint64_t p) {
    return a = a ^ p;
}

template<typename T>
Matrix<T> power_sum(const Matrix<T> &a, uint64_t p) {
    assert(rows(a) == cols(a));
    int n = rows(a);
    if (p == 0) {
        return make_matrix<T>(n, n);
    }
    // The upper-right block of  $\begin{bmatrix} A & A \\ 0 & I \end{bmatrix}^p$  is  $A + A^2 + \dots + A^p$ .
    Matrix<T> block = make_matrix<T>(2 * n, 2 * n);
    for (int i = 0; i < n; i++) {
        block[i + n][i + n] = 1;
        for (int j = 0; j < n; j++) {
            block[i][j] = a[i][j];
            block[i][j + n] = a[i][j];
        }
    }
    block = block ^ p;
    Matrix<T> res = make_matrix<T>(n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res[i][j] = block[i][j + n];
        }
    }
}

```

```
    return res;
}
```

## Example Usage

```
using namespace std;

int main() {
    using matrix = Matrix<int>;
    matrix m = make_matrix(5, 5, 10) + 10;
    vector<int> v{1, 2, 3, 4, 5};
    assert((m * v == vector<int>{300, 300, 300, 300, 300}));

    m[0][0] += 5;
    assert(m[0][0] == 25 && m[1][1] == 20);
    assert((m ^ 0) == identity_matrix(5));
    assert((identity_matrix<int>(3) ^ 5) == identity_matrix<int>(3));
    assert(power_sum(m, 3) == m + m * m + (m ^ 3));

    Matrix<double> d = make_matrix<double>(2, 2, 0.5);
    assert(rows(d) == 2 && cols(d) == 2);
    assert((d + 0.25)[0][0] == 0.75);
    return 0;
}
```

### 6.5.2 Row Reduction

Converts a matrix to reduced row echelon form using Gaussian elimination to solve a system of linear equations and compute rank. Each round finds a row with a nonzero entry in the current leading column, normalizes that row, and subtracts multiples of it from every other row to clear the column. Floating-point matrices use the largest-magnitude candidate for partial pivoting, though elimination can still be prone to rounding error; use LU decomposition for greater numerical stability. Exact field types such as `Modular<MOD>` instead use any nonzero pivot and perform exact arithmetic. Such types must support construction from 0 and 1, equality, addition, subtraction, multiplication, and division.

- `row_reduce(a)` assigns the matrix `a` to its reduced row echelon form, returning a reference to the modified argument itself.
- `matrix_rank(a)` returns the rank of matrix `a`, i.e. the number of nonzero rows after row reduction.
- `solve_system(a, b, &x)` solves the system of linear equations  $ax = b$  given an  $m$  by  $n$  matrix `a` over the real numbers or an exact field, and a length  $m$  vector `b`, returning 0 if there is one solution,  $-1$  if there are zero solutions, or  $-2$  if there are infinite solutions. If there is exactly one solution, then the output vector `x` is populated with the solution of length  $n$ .

#### Time Complexity:

- $O(m \cdot n \cdot \min(m, n))$  per call to `row_reduce()` and `matrix_rank()`, where  $m$  and  $n$  are the numbers of rows and columns of `a`, respectively.
- $O(m \cdot n \cdot \min(m, n))$  per call to `solve_system()`.

#### Space Complexity:

- $O(1)$  auxiliary for `row_reduce()`.
- $O(m \cdot n)$  auxiliary for `matrix_rank()` and `solve_system()`.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <type_traits>
#include <utility>
#include <vector>
```

```

const double EPS = 1e-9;

template<typename T>
bool rref_is_zero(const T &v) {
    if constexpr (std::is_floating_point_v<T>) {
        return std::fabs(v) < EPS;
    }
    return v == T{0};
}

template<typename Matrix>
Matrix &row_reduce(Matrix &a) {
    if (a.empty()) {
        return a;
    }
    using T = std::decay_t<decltype(a[0][0])>;
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    for (int r = 0, lead = 0; r < rows && lead < cols; lead++) {
        int pivot = r;
        if constexpr (std::is_floating_point_v<T>) {
            for (int i = r + 1; i < rows; i++) {
                if (std::fabs(a[i][lead]) > std::fabs(a[pivot][lead])) {
                    pivot = i;
                }
            }
        }
        else {
            while (pivot < rows && rref_is_zero(a[pivot][lead])) {
                pivot++;
            }
        }
        if (pivot == rows || rref_is_zero(a[pivot][lead])) {
            continue;
        }
        std::swap(a[pivot], a[r]);
        auto lv = a[r][lead];
        for (int j = 0; j < cols; j++) {
            a[r][j] /= lv;
        }
        for (int i = 0; i < rows; i++) {
            if (i != r) {
                lv = a[i][lead];
                for (int j = 0; j < cols; j++) {
                    a[i][j] -= lv * a[r][j];
                }
            }
        }
        std::fill(a[r].begin(), a[r].begin() + lead, 0);
        a[r][lead] = 1;
        r++;
    }
    return a;
}

template<typename Matrix>
int matrix_rank(Matrix a) {
    row_reduce(a);
    int rank = 0;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        for (int j = 0; j < static_cast<int>(a[i].size()); j++) {
            if (!rref_is_zero(a[i][j])) {
                rank++;
                break;
            }
        }
    }
    return rank;
}

```

```

template<typename Matrix, typename T>
int solve_system(const Matrix &a, const std::vector<T> &b, std::vector<T> *x) {
    if (x == nullptr || a.empty() || a.size() != b.size()) {
        return -1;
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    Matrix m(a);
    for (int i = 0; i < rows; i++) {
        m[i].push_back(b[i]);
    }
    row_reduce(m);
    int rank = 0;
    for (int i = 0; i < rows; i++) {
        int lead = -1;
        for (int j = 0; j < cols && lead < 0; j++) {
            if (!rref_is_zero(m[i][j])) {
                lead = j;
            }
        }
        if (lead < 0) {
            // A zero coefficient row with a nonzero constant means 0 = nonzero (no solution).
            if (!rref_is_zero(m[i][cols])) {
                return -1;
            }
        } else {
            rank++;
        }
    }
    // A unique solution requires a pivot in every column; fewer means free variables remain. This
    // also catches trailing free columns, which a per-row "lead > i" test alone would miss.
    if (rank < cols) {
        return -2;
    }
    x->resize(cols);
    for (int i = 0; i < cols; i++) {
        (*x)[i] = m[i][cols];
    }
    return 0;
}

```

## Example Usage

```

using namespace std;

int main() {
    vector<vector<double>> a{{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
    vector<double> b{3, 1, -2};
    vector<double> x;
    assert(solve_system(a, b, &x) == 0);
    assert(matrix_rank(a) == 3);
    assert(matrix_rank(vector<vector<double>>{{1, 2, 3}, {2, 4, 6}, {0, 1, 1}}) == 2);
    assert(solve_system(vector<vector<double>>{{0, 0}}, vector<double>{1}, &x) == -1);
    assert(solve_system(vector<vector<double>>{{1, 0}}, vector<double>{1}, &x) == -2);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        double sum = 0;
        for (int j = 0; j < static_cast<int>(a[i].size()); j++) {
            sum += a[i][j] * x[j];
        }
        assert(fabs(sum - b[i]) < EPS);
    }
    return 0;
}

```

### 6.5.3 Determinant and Inverse

Computes the determinant and inverse of a square matrix using Gaussian elimination. The inverse of a matrix  $a$  is another matrix  $b$  such that  $ab$  equals the identity matrix. The inverse of  $a$  exists if and only if the determinant of  $a$  is nonzero. In this case,  $a$  is called invertible or non-singular. The determinant falls out of elimination as the product of the pivots, negated once per row swap. The inverse is found by appending the identity matrix and row-reducing the combined matrix: the operations that turn  $a$  into the identity turn the identity into the inverse.

Floating-point matrices use the largest-magnitude pivot in each column, although Gaussian elimination can still suffer from rounding error; use LU decomposition for greater numerical stability. Exact field types such as `Modular<MOD>` use any nonzero pivot and perform exact arithmetic. These types must support construction from 0 and 1, equality, addition, subtraction, multiplication, and division. For an integer matrix whose determinant is wanted exactly, the Bareiss algorithm runs a fraction-free elimination: every intermediate value is itself a determinant of a submatrix, so the divisions are always exact and the arithmetic stays in integers.

- `det_naive(a)` returns the determinant of an  $n$  by  $n$  matrix `a`, using the classic divide-and-conquer algorithm by Laplace expansions. It is division-free and computes in the matrix's element type, so an integer matrix yields an exact integer determinant; but at  $O(n!)$  it is only practical for tiny  $n$ , so prefer `det_bareiss` for exact integer determinants.
- `det(a, eps = 1e-10)` returns the determinant of an  $n$  by  $n$  floating-point or exact-field matrix `a` using Gaussian elimination. Floating-point pivots within `eps` of zero are treated as singular; exact fields use equality with zero.
- `det_bareiss(a)` returns the exact determinant of an integer matrix `a` using fraction-free elimination, with no rounding error.
- `inverse(a, eps = 1e-10)` returns the inverse of square matrix `a` with floating-point or exact-field elements, or `std::nullopt` if the matrix is singular. Floating-point pivots within `eps` of zero are treated as singular.

Overflow warning: In `det_bareiss()`, stored entries are minors of `a`, but the products in each update can be larger. They use 128-bit intermediates when available; otherwise those products must fit in `int64_t`.

#### Time Complexity:

- $O(n!)$  per call to `det_naive()`, where  $n$  is the dimension of the matrix.
- $O(n^3)$  per call to `det()`, `det_bareiss()`, and `inverse()` where  $n$  is the dimension of the matrix.

#### Space Complexity:

- $O(n)$  auxiliary stack space and  $O(n^3)$  auxiliary heap space for `det_naive()`, where  $n$  is the dimension of the matrix.
- $O(n^2)$  auxiliary heap space for `det()`, `det_bareiss()`, and `inverse()`.

```
#include <cmath>
#include <cstdint>
#include <optional>
#include <type_traits>
#include <vector>

template<typename SquareMatrix>
auto det_naive(const SquareMatrix &a) {
    using T = std::decay_t<decltype(a[0][0])>;
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return T{1};
    }
    if (n == 1) {
        return a[0][0];
    }
    if (n == 2) {
        return a[0][0] * a[1][1] - a[0][1] * a[1][0];
    }
    T res = 0;
    SquareMatrix temp(n - 1, typename SquareMatrix::value_type(n - 1));
    for (int p = 0; p < n; p++) {
```

```

    int h = 0, k = 0;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (j == p) {
                continue;
            }
            temp[h][k++] = a[i][j];
            if (k == n - 1) {
                h++;
                k = 0;
            }
        }
    }
    res += (p % 2 == 0 ? 1 : -1) * a[0][p] * det_naive(temp);
}
return res;
}

template<typename SquareMatrix>
int elimination_pivot(const SquareMatrix &a, int row, int col, double eps) {
    using T = std::decay_t<decltype(a[0][0])>;
    int n = static_cast<int>(a.size());
    if constexpr (std::is_floating_point_v<T>) {
        int pivot = row;
        for (int r = row + 1; r < n; r++) {
            if (std::fabs(a[r][col]) > std::fabs(a[pivot][col])) {
                pivot = r;
            }
        }
        return std::fabs(a[pivot][col]) < eps ? -1 : pivot;
    }
    for (int r = row; r < n; r++) {
        if (a[r][col] != T{0}) {
            return r;
        }
    }
    return -1;
}

template<typename SquareMatrix>
auto det(const SquareMatrix &a, double eps = 1e-10) {
    using T = std::decay_t<decltype(a[0][0])>;
    int n = static_cast<int>(a.size());
    SquareMatrix b(a);
    T res = 1;
    for (int i = 0; i < n; i++) {
        int p = elimination_pivot(b, i, i, eps);
        if (p == -1) {
            return T{0};
        }
        if (p != i) {
            std::swap(b[p], b[i]);
            res = T{0} - res;
        }
        res *= b[i][i];
        for (int j = i + 1; j < n; j++) {
            T z = b[j][i] / b[i][i];
            for (int k = i; k < n; k++) {
                b[j][k] -= z * b[i][k];
            }
        }
    }
    return res;
}

int64_t det_bareiss(std::vector<std::vector<int64_t>> a) {
    int n = static_cast<int>(a.size());
    int64_t prev = 1, sign = 1;

```

```

for (int k = 0; k < n; k++) {
    if (a[k][k] == 0) { // Swap in a nonzero pivot; each swap flips the sign.
        int p = -1;
        for (int r = k + 1; r < n; r++) {
            if (a[r][k] != 0) {
                p = r;
                break;
            }
        }
        if (p == -1) {
            return 0;
        }
        std::swap(a[k], a[p]);
        sign = -sign;
    }
    for (int i = k + 1; i < n; i++) {
        for (int j = k + 1; j < n; j++) {
#ifdef defined(__SIZEOF_INT128__)
            __extension__ typedef __int128 int128_t;
            int128_t numerator =
                static_cast<int128_t>(a[i][j]) * a[k][k] - static_cast<int128_t>(a[i][k]) * a[k][j];
            a[i][j] = static_cast<int64_t>(numerator / prev);
#else
            a[i][j] = (a[i][j] * a[k][k] - a[i][k] * a[k][j]) / prev; // Overflow warning.
#endif
        }
    }
    prev = a[k][k];
}
return n == 0 ? 1 : sign * a[n - 1][n - 1];
}

template<typename SquareMatrix>
std::optional<SquareMatrix> inverse(SquareMatrix a, double eps = 1e-10) {
    using T = std::decay_t<decltype(a[0][0])>;
    int n = static_cast<int>(a.size());
    for (int i = 0; i < n; i++) {
        a[i].resize(2 * n);
        for (int j = n; j < n * 2; j++) {
            a[i][j] = (i == j - n ? 1 : 0);
        }
    }
    for (int i = 0; i < n; i++) {
        int p = elimination_pivot(a, i, i, eps);
        if (p == -1) {
            return std::nullopt;
        }
        std::swap(a[p], a[i]);
        T pivot = a[i][i];
        for (int j = i; j < n * 2; j++) {
            a[i][j] /= pivot;
        }
        for (int j = 0; j < n; j++) {
            if (i != j) {
                T factor = a[j][i];
                for (int k = 0; k < n * 2; k++) {
                    a[j][k] -= factor * a[i][k];
                }
            }
        }
    }
    for (int i = 0; i < n; i++) {
        a[i].erase(a[i].begin(), a[i].begin() + n);
    }
    return a;
}

```



## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<vector<double>> a{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    int n = static_cast<int>(a.size());
    vector<vector<double>> res(n, vector<double>(n));
    assert(fabs(det(a) - det_naive(a)) < 1e-10);

    // Bareiss gives the determinant of an integer matrix exactly, with no rounding.
    vector<vector<int64_t>> ai{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    assert(det_naive(ai) == -306); // Division-free, so also exact in the int64_t element type.
    assert(det_naive(vector<vector<int64_t>>{}) == 1);
    assert(det_bareiss(ai) == -306);
    assert(det_bareiss({{2, 0, 0}, {0, 3, 0}, {0, 0, 5}}) == 30);
    assert(det_bareiss({{1, 2}, {2, 4}}) == 0); // Singular.
    auto inv = inverse(a);
    assert(inv);
    assert(!inverse(vector<vector<double>>{{1, 2}, {2, 4}}));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                res[i][j] += a[i][k] * (*inv)[k][j];
            }
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            assert(fabs(res[i][j] - (i == j ? 1 : 0)) < 1e-10);
        }
    }
    return 0;
}
```

### 6.5.4 LU Decomposition

The LU decomposition of a matrix  $A$  with row-partial pivoting is a factorization of  $A$  (after some rows are possibly permuted by a permutation matrix  $p$ ) as a product of a lower triangular matrix  $L$  and an upper triangular matrix  $U$ . This factorization can be used to tackle many common problems in linear algebra such as solving systems of linear equations and computing determinants. An improvement on basic row reduction, LU decomposition by row-partial pivoting keeps the relative magnitude of matrix values small, thus reducing the relative error due to rounding in computed solutions. These routines require floating-point matrix elements; use 6.5.3 for exact-field or integer determinant calculations.

- `lu_decompose(a, &perm, eps = 1e-10)` assigns floating-point matrix `a` to merged LU decomposition matrix `lu`; it returns 0 for even row-swap parity, 1 for odd parity, or  $-1$  for a degenerate matrix (i.e. singular for square matrices). The merged matrix stores `l[i][j]` in `lu[i][j]` when `i > j`, and `u[i][j]` otherwise. The diagonal entries `l[i][i]` are always 1, so they are not explicitly stored. Access general entries of the lower and upper triangular matrices via `getl(lu, i, j)` and `getu(lu, i, j)`. Optional output vector `perm` represents  $p$ : `perm[i]` is the only column containing 1 in row `i`. Left-multiplying `a` by this permutation matrix gives the product of the separate lower and upper triangular matrices, not the merged storage matrix `lu` itself.
- `solve_system(a, b, &x, eps = 1e-10)` solves the system of linear equations  $Ax = b$  given an  $m$  by  $n$  floating-point matrix `a` and a length  $m$  vector `b`, returning 0 if there is one solution or  $-1$  if there are zero or infinite solutions. If there is exactly one solution, then the output vector `x` is populated with the solution of length  $n$ ; otherwise, `x` is unchanged.
- `det(a)` returns the determinant of an  $n$  by  $n$  floating-point matrix `a` using LU decomposition.
- `inverse(a)` returns the inverse of the  $n$  by  $n$  floating-point matrix `a`, or `std::nullopt` if `a` is singular.

**Time Complexity:**

- $O(m \cdot n \cdot \min(m, n))$  per call to `lu_decompose(a)`, where  $m$  and  $n$  are the numbers of rows and columns of `a`, respectively.
- $O(m \cdot n^2)$  per call to `solve_system()`, which requires  $m \geq n$ .
- $O(n^3)$  per call to `det(a)` and `inverse(a)`, where  $n$  is the length of square matrix `a`.

**Space Complexity:**

- $O(1)$  auxiliary for `lu_decompose()`.
- $O(n^2)$  auxiliary for `det()` and `inverse()`.
- $O(m \cdot n)$  auxiliary for `solve_system()`.

```
#include <algorithm>
#include <cmath>
#include <numeric>
#include <optional>
#include <type_traits>
#include <vector>

template<typename Matrix>
int lu_decompose(Matrix &a, std::vector<int> *perm = nullptr, double eps = 1e-10) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    int parity = 0;
    if (perm != nullptr) {
        perm->resize(rows);
        std::iota(perm->begin(), perm->end(), 0);
    }
    for (int i = 0; i < rows && i < cols; i++) {
        int pi = i;
        for (int k = i + 1; k < rows; k++) {
            if (std::fabs(a[k][i]) > std::fabs(a[pi][i])) {
                pi = k;
            }
        }
        if (std::fabs(a[pi][i]) < eps) {
            return -1;
        }
        if (pi != i) {
            if (perm != nullptr) {
                std::iter_swap(perm->begin() + i, perm->begin() + pi);
            }
            std::iter_swap(a.begin() + i, a.begin() + pi);
            parity = 1 - parity;
        }
        for (int j = i + 1; j < rows; j++) {
            a[j][i] /= a[i][i];
            for (int k = i + 1; k < cols; k++) {
                a[j][k] -= a[j][i] * a[i][k];
            }
        }
    }
    return parity;
}

template<typename Matrix>
auto getl(const Matrix &lu, int i, int j) {
    using T = std::decay_t<decltype(lu[0][0])>;
    return i > j ? lu[i][j] : T(i == j);
}

template<typename Matrix>
auto getu(const Matrix &lu, int i, int j) {
    using T = std::decay_t<decltype(lu[0][0])>;
    return i <= j ? lu[i][j] : T{0};
}
```

```

template<typename Matrix, typename T>
int solve_system(const Matrix &a, const std::vector<T> &b, std::vector<T> *x, double eps = 1e-10) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    if (x == nullptr || a.empty() || a.size() != b.size() || rows < cols) {
        return -1;
    }
    Matrix lu(a);
    std::vector<int> perm;
    int status = lu_decompose(lu, &perm, eps);
    if (status < 0) {
        return status;
    }
    std::vector<T> solution(cols);
    for (int i = 0; i < cols; i++) {
        solution[i] = b[perm[i]];
        for (int k = 0; k < i; k++) {
            solution[i] -= getl(lu, i, k) * solution[k];
        }
    }
    for (int i = cols - 1; i >= 0; i--) {
        for (int k = i + 1; k < cols; k++) {
            solution[i] -= getu(lu, i, k) * solution[k];
        }
        solution[i] /= getu(lu, i, i);
    }
    for (int i = 0; i < rows; i++) {
        T val = 0;
        for (int j = 0; j < cols; j++) {
            val += a[i][j] * solution[j];
        }
        // Mixed absolute/relative tolerance: dividing by b[i] alone would skip the check for negative
        // b[i] (ratio goes negative) and divide by zero when b[i] == 0.
        if (std::fabs(val - b[i]) > eps * (1.0 + std::fabs(b[i]))) {
            return -1;
        }
    }
    x->swap(solution);
    return 0;
}

template<typename Matrix>
auto det(const Matrix &a) {
    using T = std::decay_t<decltype(a[0][0])>;
    int n = static_cast<int>(a.size());
    Matrix lu(a);
    int status = lu_decompose(lu);
    if (status < 0) {
        return T{0};
    }
    T res = 1;
    for (int i = 0; i < n; i++) {
        res *= lu[i][i];
    }
    return status == 0 ? res : -res;
}

template<typename SquareMatrix>
std::optional<SquareMatrix> inverse(SquareMatrix a) {
    int n = static_cast<int>(a.size());
    std::vector<int> perm;
    int status = lu_decompose(a, &perm);
    if (status < 0) {
        return std::nullopt;
    }
    SquareMatrix ia(n, typename SquareMatrix::value_type(n, 0));
    for (int j = 0; j < n; j++) {

```

```

for (int i = 0; i < n; i++) {
    if (perm[i] == j) {
        ia[i][j] = 1.0;
    } else {
        ia[i][j] = 0.0;
    }
    for (int k = 0; k < i; k++) {
        ia[i][j] -= getl(a, i, k) * ia[k][j];
    }
}
for (int i = n - 1; i >= 0; i--) {
    for (int k = i + 1; k < n; k++) {
        ia[i][j] -= getu(a, i, k) * ia[k][j];
    }
    ia[i][j] /= getu(a, i, i);
}
}
return ia;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    { // Solve a system.
        vector<vector<double>> a{{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
        vector<double> b{3, 1, -2};
        vector<double> x;
        assert(solve_system(a, b, &x) == 0);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            double sum = 0;
            for (int j = 0; j < static_cast<int>(a[0].size()); j++) {
                sum += a[i][j] * x[j];
            }
            assert(EQ(sum, b[i]));
        }
    }
    { // Find the determinant.
        vector<vector<double>> a{{1, 3, 5}, {2, 4, 7}, {1, 1, 0}};
        assert(EQ(det(a), 4));
    }
    { // Find the inverse.
        vector<vector<double>> a{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
        auto inv = inverse(a);
        int n = static_cast<int>(a.size());
        vector<vector<double>> res(n, vector<double>(n));
        assert(inv);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n; k++) {
                    res[i][j] += a[i][k] * (*inv)[k][j];
                }
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                assert(EQ(res[i][j], i == j ? 1 : 0));
            }
        }
    }
}

```

```

{ // Singular matrices have no inverse.
  vector<vector<double>> singular{{1, 2}, {2, 4}};
  assert(!inverse(singular));
  // Failed operations leave their output arguments unchanged.
  vector<vector<double>> inconsistent{{1}, {1}};
  vector<double> b{1, 2}, x{99, 100};
  assert(solve_system(inconsistent, b, &x) == -1);
  assert((x == vector<double>{99, 100}));
}
return 0;
}

```

### 6.5.5 QR Decomposition and Least Squares

The QR decomposition factors an  $m$  by  $n$  matrix  $A$  with  $m \geq n$  into  $A = QR$ , where  $Q$  has orthonormal columns and  $R$  is upper triangular. Orthonormal columns are what make the factorization useful:  $Q$  preserves lengths, so multiplying by  $Q^T$  rewrites a problem in a new basis without distorting distances, and a triangular system is solved by back substitution.

This implementation uses Householder reflections. A Householder reflection is the mirror across the hyperplane orthogonal to a chosen vector  $v$ , written  $I - 2vv^T/(v^T v)$ , and  $v$  is chosen so that the reflection maps the part of a column below the diagonal onto the axis, zeroing it in one step. Applying one reflection per column leaves  $R$ , and accumulating the reflections gives  $Q$ . Reflecting is stable because it never scales anything: the alternative of orthogonalizing column by column with Gram-Schmidt loses orthogonality as the columns approach linear dependence, whereas reflections keep it to within rounding error.

The overdetermined system  $Ax = b$  usually has no solution, so least squares asks for the  $x$  whose residual is shortest. Because  $Q$  preserves lengths,  $\|Ax - b\|$  equals  $\|Rx - Q^T b\|$ , and the entries of  $Q^T b$  below row  $n$  cannot be changed by any  $x$ ; minimizing therefore means solving the square triangular system formed by the first  $n$  rows, which back substitution does directly.

- `qr_decompose(a, &q, &r)` factors the  $m$  by  $n$  matrix `a` with  $m \geq n$  into an  $m$  by  $m$  orthogonal matrix `q` and an  $m$  by  $n$  upper triangular matrix `r`, whose product is `a`.
- `least_squares(a, b, eps = 1e-10)` returns the vector  $x$  minimizing the Euclidean norm of  $\|ax - b\|$ , or `std::nullopt` if the columns of `a` are linearly dependent to within `eps`. The matrix `a` must have at least as many rows as columns, and `b` must have one entry per row.

Fitting any linear model reduces to this: one row per data point, one column per basis function. Solving the normal equations  $A^T Ax = A^T b$  by the row reduction of section 6.5.2 gives the same answer but squares the condition number, losing roughly twice as many digits.

#### Time Complexity:

- $O(m^2 \cdot n)$  per call to `qr_decompose()`, where  $m$  and  $n$  are the numbers of rows and columns.
- $O(m \cdot n^2)$  per call to `least_squares()`, since it never forms `q` explicitly.

#### Space Complexity:

- $O(m^2 + m \cdot n)$  for the matrices returned by `qr_decompose()`.
- $O(m \cdot n)$  auxiliary and  $O(n)$  for the vector returned by `least_squares()`.

```

#include <cassert>
#include <cmath>
#include <optional>
#include <vector>

using Matrix = std::vector<std::vector<double>>>;

// Reflects the trailing rows of the given column onto the axis, returning the reflection vector.
std::vector<double> householder_vector(const Matrix &a, int col) {
    int m = static_cast<int>(a.size());

```

```

std::vector<double> v(m);
double norm = 0;
for (int i = col; i < m; i++) {
    v[i] = a[i][col];
    norm += v[i] * v[i];
}
norm = std::sqrt(norm);
// Reflect away from a[col][col] so that the subtraction below never cancels.
v[col] += a[col][col] >= 0 ? norm : -norm;
return v;
}

void qr_decompose(const Matrix &a, Matrix *q, Matrix *r) {
    int m = static_cast<int>(a.size()), n = a.empty() ? 0 : static_cast<int>(a[0].size());
    assert(m >= n && q != nullptr && r != nullptr);
    *r = a;
    q->assign(m, std::vector<double>(m));
    for (int i = 0; i < m; i++) {
        (*q)[i][i] = 1;
    }
    for (int col = 0; col < n; col++) {
        std::vector<double> v = householder_vector(*r, col);
        double vv = 0;
        for (int i = col; i < m; i++) {
            vv += v[i] * v[i];
        }
        if (vv == 0) {
            continue; // The column is already zero below the diagonal.
        }
        for (int j = 0; j < n; j++) { // Apply the reflection to R from the left.
            double dot = 0;
            for (int i = col; i < m; i++) {
                dot += v[i] * (*r)[i][j];
            }
            for (int i = col; i < m; i++) {
                (*r)[i][j] -= 2 * dot / vv * v[i];
            }
        }
        for (int j = 0; j < m; j++) { // Accumulate the reflection into Q from the right.
            double dot = 0;
            for (int i = col; i < m; i++) {
                dot += v[i] * (*q)[j][i];
            }
            for (int i = col; i < m; i++) {
                (*q)[j][i] -= 2 * dot / vv * v[i];
            }
        }
    }
}

std::optional<std::vector<double>> least_squares(
    const Matrix &a, const std::vector<double> &b, double eps = 1e-10
) {
    int m = static_cast<int>(a.size()), n = a.empty() ? 0 : static_cast<int>(a[0].size());
    assert(m >= n && static_cast<int>(b.size()) == m);
    Matrix r = a;
    std::vector<double> rhs = b;
    for (int col = 0; col < n; col++) {
        std::vector<double> v = householder_vector(r, col);
        double vv = 0;
        for (int i = col; i < m; i++) {
            vv += v[i] * v[i];
        }
        if (vv == 0) {
            continue;
        }
        for (int j = col; j < n; j++) {
            double dot = 0;

```

```

    for (int i = col; i < m; i++) {
        dot += v[i] * r[i][j];
    }
    for (int i = col; i < m; i++) {
        r[i][j] -= 2 * dot / vv * v[i];
    }
}
double dot = 0; // The same reflection applied to the right-hand side.
for (int i = col; i < m; i++) {
    dot += v[i] * rhs[i];
}
for (int i = col; i < m; i++) {
    rhs[i] -= 2 * dot / vv * v[i];
}
}
std::vector<double> x(n);
for (int i = n - 1; i >= 0; i--) { // Back substitution over the triangular top block.
    if (std::fabs(r[i][i]) < eps) {
        return std::nullopt;
    }
    double sum = rhs[i];
    for (int j = i + 1; j < n; j++) {
        sum -= r[i][j] * x[j];
    }
    x[i] = sum / r[i][i];
}
return x;
}

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    Matrix a{{12, -51, 4}, {6, 167, -68}, {-4, 24, -41}};
    Matrix q, r;
    qr_decompose(a, &q, &r);
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            double product = 0, orthogonal = 0;
            for (int k = 0; k < 3; k++) {
                product += q[i][k] * r[k][j];
                orthogonal += q[k][i] * q[k][j];
            }
            assert(EQ(product, a[i][j])); // The factors multiply back to a.
            assert(EQ(orthogonal, i == j ? 1.0 : 0.0)); // The columns of q are orthonormal.
            assert(i <= j || EQ(r[i][j], 0.0)); // r is upper triangular.
        }
    }

    // An exactly solvable square system.
    auto exact = least_squares(Matrix{{2, 1}, {1, 3}}, vector<double>{5, 10});
    assert(exact.has_value() && EQ((*exact)[0], 1.0) && EQ((*exact)[1], 3.0));

    // Fit y = c0 + c1*x to four collinear points, then to points with one outlier.
    Matrix design{{1, 0}, {1, 1}, {1, 2}, {1, 3}};
    auto line = least_squares(design, vector<double>{1, 3, 5, 7});
    assert(line.has_value() && EQ((*line)[0], 1.0) && EQ((*line)[1], 2.0));
    auto noisy = least_squares(design, vector<double>{1, 3, 5, 8});
    assert(noisy.has_value() && EQ((*noisy)[0], 0.8) && EQ((*noisy)[1], 2.3));
}

```

```
// Dependent columns have no unique least-squares solution.
assert(!least_squares(Matrix{{1, 2}, {2, 4}, {3, 6}}, vector<double>{1, 2, 3}).has_value());
return 0;
}
```

### 6.5.6 Characteristic Polynomial

Computes the characteristic polynomial of a square matrix over a prime field: the coefficients of  $\det(xI - A)$ , a monic degree- $n$  polynomial whose roots are the eigenvalues of  $A$ . Its constant term recovers the determinant (up to sign), its second-highest coefficient is the negated trace, and evaluating it at a scalar gives  $\det(xI - A)$  for that  $x$ . It is the standard tool behind eigenvalue counting, matrix-power recurrences via Cayley-Hamilton, and problems that reduce to a determinant in an unknown.

A direct expansion would cost a determinant per coefficient. Instead the matrix is first reduced to upper Hessenberg form, where every entry more than one place below the diagonal is zero. This reduction uses similarity transforms (a row operation paired with the inverse column operation), which leave the characteristic polynomial unchanged. The characteristic polynomial of a Hessenberg matrix then follows from a short recurrence over its leading principal submatrices, adding one row and column at a time, for an overall cost of  $O(n^3)$ . The arithmetic is exact modulo a prime, since the Hessenberg reduction divides by pivots and is not numerically stable over the reals.

- `characteristic_polynomial(a)` returns a vector  $c$  of length  $n + 1$  with  $\det(xI - \mathbf{a}) = c_0 + c_1x + \dots + c_nx^n$  modulo `MOD`, where  $c_n = 1$ . The matrix `a` must be square; its entries are reduced modulo `MOD` internally.

**Time Complexity:**  $O(n^3)$  per call, where  $n$  is the dimension of the matrix.

**Space Complexity:**  $O(n^2)$  auxiliary.

```
#include <stdint>
#include <utility>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

void to_hessenberg(std::vector<std::vector<int64_t>> &m) {
    int n = static_cast<int>(m.size());
    for (int j = 0; j + 2 < n; j++) {
        int pivot = -1;
        for (int i = j + 1; i < n; i++) {
            if (m[i][j] != 0) {
                pivot = i;
                break;
            }
        }
        if (pivot == -1) {
            continue;
        }
        if (pivot != j + 1) { // Move the pivot to the subdiagonal via a similarity swap.
            std::swap(m[pivot], m[j + 1]);
            for (int r = 0; r < n; r++) {
                std::swap(m[r][pivot], m[r][j + 1]);
            }
        }
    }
}
```



```

    }
}
int64_t pinv = powmod(m[j + 1][j], MOD - 2, MOD);
for (int k = j + 2; k < n; k++) {
    int64_t u = m[k][j] * pinv % MOD;
    if (u == 0) {
        continue;
    }
    for (int c = 0; c < n; c++) { // Row k -= u * row (j + 1).
        m[k][c] = (m[k][c] - u * m[j + 1][c] % MOD + MOD) % MOD;
    }
    for (int r = 0; r < n; r++) { // Inverse column operation keeps it a similarity.
        m[r][j + 1] = (m[r][j + 1] + u * m[r][k]) % MOD;
    }
}
}
}
}

std::vector<int64_t> characteristic_polynomial(std::vector<std::vector<int64_t>> a) {
    int n = static_cast<int>(a.size());
    for (auto &row : a) {
        for (int64_t &x : row) {
            x = (x % MOD + MOD) % MOD;
        }
    }
    to_hessenberg(a);
    std::vector<std::vector<int64_t>> p(n + 1);
    p[0] = {1};
    for (int k = 0; k < n; k++) {
        p[k + 1].assign(k + 2, 0);
        for (int i = 0; i < static_cast<int>(p[k].size()); i++) {
            p[k + 1][i + 1] = (p[k + 1][i + 1] + p[k][i]) % MOD; // x * p[k].
            p[k + 1][i] = (p[k + 1][i] - a[k][k] * p[k][i] % MOD + MOD) % MOD; // -a[k][k] * p[k].
        }
        int64_t t = 1;
        for (int i = 0; i < k; i++) {
            t = t * a[k - i][k - i - 1] % MOD; // Running product of subdiagonal entries.
            int64_t coeff = t * a[k - i - 1][k] % MOD;
            for (int j = 0; j < static_cast<int>(p[k - i - 1].size()); j++) {
                p[k + 1][j] = (p[k + 1][j] - coeff * p[k - i - 1][j] % MOD + MOD) % MOD;
            }
        }
    }
    return p[n];
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // [[1, 2], [3, 4]]: det(xI - A) = x^2 - 5x - 2.
    vector<vector<int64_t>> a{{1, 2}, {3, 4}};
    assert((characteristic_polynomial(a) == vector<int64_t>(MOD - 2, MOD - 5, 1))); // -2, -5, 1.

    // det(xI - B) = (x - 3)(x^2 - 7x + 6) = x^3 - 10x^2 + 27x - 18.
    vector<vector<int64_t>> b{{2, 0, 1}, {0, 3, 0}, {4, 1, 5}};
    assert((characteristic_polynomial(b) == vector<int64_t>(MOD - 18, 27, MOD - 10, 1)));
    return 0;
}

```

### 6.5.7 Eigenvalues (Jacobi)

An eigenvector of a square matrix is a direction the matrix does not rotate, only rescales, and the scale factor is its eigenvalue. For a real symmetric matrix, the spectral theorem guarantees that a full set of real eigenvalues exists and that eigenvectors for distinct eigenvalues are orthogonal, so the matrix is diagonalized by an orthogonal change of basis.

The Jacobi eigenvalue algorithm builds that basis one rotation at a time. It repeatedly picks the largest off-diagonal entry and applies a rotation in the plane of that entry's row and column, choosing the angle that sets the entry to zero. A rotation is a similarity transform, so it preserves the eigenvalues, and it also preserves the sum of squares of all entries; since zeroing an entry removes its square from the off-diagonal total and moves it onto the diagonal, every sweep strictly reduces the off-diagonal weight. Earlier zeros are partially refilled by later rotations, but the total decays geometrically, and the matrix converges to a diagonal one whose entries are the eigenvalues. Accumulating the rotations gives the eigenvectors.

- `jacobi_eigen(a, eps = 1e-12, iterations = 100)` returns the pair `(values, vectors)` for a real symmetric  $n$  by  $n$  matrix `a`. The eigenvalues in `values` are sorted in decreasing order, and `vectors[i]` is a unit eigenvector for `values[i]`, so the eigenvectors are rows rather than columns. Iteration stops once every off-diagonal entry is smaller than `eps`. The matrix must be symmetric, which is not checked; an unsymmetric one silently produces nonsense.

A general matrix instead needs the roots of the characteristic polynomial of section 6.5.6, found including complex ones by the Ehrlich-Aberth iteration of section 5.4.5, though accuracy degrades with degree since polynomial roots are far more sensitive to coefficients than eigenvalues are to the matrix. When only the dominant eigenvalue is wanted, power iteration finds it with no decomposition at all, by repeatedly multiplying a random vector and renormalizing.

**Time Complexity:**  $O(n^3)$  per sweep and  $O(n^3 \log(1/\text{eps}))$  per call in practice, where  $n$  is the dimension of the matrix. Each rotation costs  $O(n)$  and the search for the largest off-diagonal entry costs  $O(n^2)$ .

**Space Complexity:**  $O(n^2)$  auxiliary, and  $O(n^2)$  for the returned eigenvectors.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <numeric>
#include <utility>
#include <vector>

using Matrix = std::vector<std::vector<double>>>;

std::pair<std::vector<double>, Matrix> jacobi_eigen(
    Matrix a, double eps = 1e-12, int iterations = 100
) {
    int n = static_cast<int>(a.size());
    Matrix vectors(n, std::vector<double>(n));
    for (int i = 0; i < n; i++) {
        vectors[i][i] = 1;
    }
    for (int sweep = 0; sweep < iterations; sweep++) {
        int p = 0, q = 1;
        double largest = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (std::fabs(a[i][j]) > largest) {
                    largest = std::fabs(a[i][j]);
                    p = i;
                    q = j;
                }
            }
        }
        if (n < 2 || largest < eps) {
            break;
        }
    }
}
```

```

// Choose the rotation angle that zeroes a[p][q], using the numerically safe form of
// tan(theta) rather than computing the angle itself.
double theta = (a[q][q] - a[p][p]) / (2 * a[p][q]);
double t = (theta >= 0 ? 1 : -1) / (std::fabs(theta) + std::sqrt(theta * theta + 1));
double c = 1 / std::sqrt(t * t + 1), s = t * c;
for (int k = 0; k < n; k++) {
    double akp = a[k][p], akq = a[k][q];
    a[k][p] = c * akp - s * akq;
    a[k][q] = s * akp + c * akq;
}
for (int k = 0; k < n; k++) {
    double apk = a[p][k], aqk = a[q][k];
    a[p][k] = c * apk - s * aqk;
    a[q][k] = s * apk + c * aqk;
    double vpk = vectors[p][k], vqk = vectors[q][k];
    vectors[p][k] = c * vpk - s * vqk;
    vectors[q][k] = s * vpk + c * vqk;
}
}
std::vector<int> order(n);
std::iota(order.begin(), order.end(), 0);
std::sort(order.begin(), order.end(), [&](int x, int y) { return a[x][x] > a[y][y]; });
std::vector<double> values(n);
Matrix sorted(n);
for (int i = 0; i < n; i++) {
    values[i] = a[order[i]][order[i]];
    sorted[i] = vectors[order[i]];
}
return {values, sorted};
}

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    // A symmetric matrix with eigenvalues 3, 2, and 1.
    Matrix a{{2, 0, -1}, {0, 2, 0}, {-1, 0, 2}};
    auto [values, vectors] = jacobi_eigen(a);
    assert(EQ(values[0], 3) && EQ(values[1], 2) && EQ(values[2], 1));

    for (int i = 0; i < 3; i++) {
        double norm = 0;
        for (int k = 0; k < 3; k++) {
            norm += vectors[i][k] * vectors[i][k];
        }
        assert(EQ(norm, 1)); // Eigenvectors come back normalized.
        for (int r = 0; r < 3; r++) {
            double image = 0;
            for (int k = 0; k < 3; k++) {
                image += a[r][k] * vectors[i][k];
            }
            assert(EQ(image, values[i] * vectors[i][r])); // The defining equation a*v = lambda*v.
        }
    }

    // A diagonal matrix is already solved, and a 1-by-1 matrix is its own eigenvalue.
    auto diagonal = jacobi_eigen(Matrix{{5, 0}, {0, -4}});
    assert(EQ(diagonal.first[0], 5) && EQ(diagonal.first[1], -4));
    assert(EQ(jacobi_eigen(Matrix{{7}}).first[0], 7));
    return 0;
}

```

```
}
```

### 6.5.8 Sparse Matrix

Provides a sparse matrix type together with arithmetic and elimination routines. A sparse matrix stores only nonzero entries, which is useful when the matrix is large but most values are zero.

The matrix keeps both row maps and column maps in sync: `row(i)` can iterate all nonzeros in row `i`, while `col(j)` can iterate all nonzeros in column `j`. That bidirectional storage is especially useful for sparse elimination or graph-like matrix operations, where both row updates and column pivot lookups are common.

The class treats `T{}` as additive zero and requires `value == T{}` to be a valid zero test.

- `SparseMatrix<T>(rows, cols)` constructs a `rows` by `cols` sparse matrix.
- `num_rows()` returns the number of rows.
- `num_cols()` returns the number of columns.
- `nonzeros()` returns the number of stored nonzero entries.
- `get(i, j)` returns the value at row `i`, column `j`, or 0 if the entry is not stored.
- `set(i, j, value)` assigns an entry. Assigning zero erases it from both maps.
- `add(i, j, delta)` adds `delta` to an entry. If the result becomes zero, the entry is erased.
- `row(i)` returns a map from column index to value for the nonzero entries in row `i`.
- `col(j)` returns a map from row index to value for the nonzero entries in column `j`.
- `swap_rows(i, j)` swaps two rows while keeping the column maps synchronized.
- `transpose()` transposes the matrix in place.
- `a * x` returns the matrix-vector product with vector `x`.
- `a + b` and `a - b` return the sum and difference of two matrices with identical dimensions.
- `a * b` returns the matrix product, requiring `a.num_cols()` to equal `b.num_rows()`. Only pairs of stored entries are ever multiplied.
- `a * k` returns `a` scaled by the value `k`. Scaling by zero yields an empty matrix.
- Operators `+=`, `-=`, and `*=` assign the corresponding result back into the left operand. In all of these operators, entries that cancel to zero are not stored.

#### Time Complexity:

- $O(\log d)$  per call to `get()`, `set()`, and `add()`, where  $d$  is the number of nonzeros in the touched row or column.
- $O(1)$  per call to `transpose()` and  $O(z)$  per matrix-vector product, where  $z$  is the number of stored nonzero entries.
- $O((r_i + r_j) \cdot \log d)$  per call to `swap_rows(i, j)`, where  $r_i$  and  $r_j$  are the sizes of the two rows.
- $O((z_a + z_b) \cdot \log d)$  per call to `operator+` and `operator-`, where  $z_a$  and  $z_b$  are the operand nonzero counts.
- $O(z \cdot \log d)$  per call to scalar `operator*`, where  $z$  is the number of stored nonzero entries.
- $O(f \cdot \log d)$  per call to `operator*`, where  $f$  is the number of scalar products accumulated, that is, the sum of `b.row(k).size()` over every stored entry `a[i][k]`.
- $O(1)$  per call to `num_rows()`, `num_cols()`, `nonzeros()`, `row()`, and `col()`.

#### Space Complexity:

- $O(z)$  storage.
- $O(1)$  auxiliary for `get()`, `set()`, `add()`, `row()`, `col()`, and `transpose()`.
- $O(r_i + r_j)$  auxiliary for `swap_rows(i, j)`.
- $O(m)$  for the vector returned by matrix-vector multiplication, where  $m$  is the number of rows.
- $O(c)$  auxiliary for `operator*`, where  $c$  is the largest number of nonzeros in one result row.

```
#include <cassert>
#include <stdint>
#include <map>
#include <unordered_set>
```

```

#include <utility>
#include <vector>

template<typename T>
class SparseMatrix {
    int rows = 0, cols = 0, stored = 0;
    std::vector<std::map<int, T>> rvals, cvals;

    bool is_zero(const T &value) const { return value == T{}; }

public:
    SparseMatrix(int rows, int cols) : rows(rows), cols(cols), rvals(rows), cvals(cols) {}

    int num_rows() const { return rows; }
    int num_cols() const { return cols; }
    int nonzeros() const { return stored; }

    T get(int i, int j) const {
        auto it = rvals[i].find(j);
        return it == rvals[i].end() ? T{} : it->second;
    }

    void set(int i, int j, const T &value) {
        auto it = rvals[i].find(j);
        if (is_zero(value)) {
            if (it != rvals[i].end()) {
                rvals[i].erase(it);
                cvals[j].erase(i);
                stored--;
            }
            return;
        }
        if (it == rvals[i].end()) {
            stored++;
        }
        rvals[i][j] = value;
        cvals[j][i] = value;
    }

    void add(int i, int j, const T &delta) {
        if (is_zero(delta)) {
            return;
        }
        set(i, j, get(i, j) + delta);
    }

    const std::map<int, T> &row(int i) const { return rvals[i]; }
    const std::map<int, T> &col(int j) const { return cvals[j]; }

    void swap_rows(int i, int j) {
        if (i == j) {
            return;
        }
        std::unordered_set<int> touched;
        for (const auto &[col, value] : rvals[i]) {
            touched.insert(col);
        }
        for (const auto &[col, value] : rvals[j]) {
            touched.insert(col);
        }
        for (int col : touched) {
            T x = get(i, col), y = get(j, col);
            set(i, col, y);
            set(j, col, x);
        }
    }

    void transpose() {

```

```

    std::swap(rows, cols);
    std::swap(rvals, cvals);
}

std::vector<T> operator*(const std::vector<T> &x) const {
    assert(static_cast<int>(x.size()) == cols);
    std::vector<T> res(rows);
    for (int i = 0; i < rows; i++) {
        for (const auto &[j, value] : rvals[i]) {
            res[i] += value * x[j];
        }
    }
    return res;
}

SparseMatrix operator+(const SparseMatrix &b) const {
    assert(rows == b.rows && cols == b.cols);
    SparseMatrix res(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (const auto &[j, value] : rvals[i]) {
            res.set(i, j, value);
        }
        for (const auto &[j, value] : b.rvals[i]) {
            res.add(i, j, value); // Cancellation to zero erases the entry.
        }
    }
    return res;
}

SparseMatrix operator-(const SparseMatrix &b) const {
    assert(rows == b.rows && cols == b.cols);
    SparseMatrix res(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (const auto &[j, value] : rvals[i]) {
            res.set(i, j, value);
        }
        for (const auto &[j, value] : b.rvals[i]) {
            res.add(i, j, T{} - value);
        }
    }
    return res;
}

// Scaling by zero cancels every entry, so the result stores nothing at all.
SparseMatrix operator*(const T &k) const {
    SparseMatrix res(rows, cols);
    if (is_zero(k)) {
        return res;
    }
    for (int i = 0; i < rows; i++) {
        for (const auto &[j, value] : rvals[i]) {
            res.set(i, j, value * k);
        }
    }
    return res;
}

// Row-by-row product: each stored a[i][k] scatters row k of b into an accumulator for row i, so
// only nonzero-by-nonzero pairs are ever multiplied.
SparseMatrix operator*(const SparseMatrix &b) const {
    assert(cols == b.rows);
    SparseMatrix res(rows, b.cols);
    for (int i = 0; i < rows; i++) {
        std::map<int, T> acc;
        for (const auto &[k, aik] : rvals[i]) {
            for (const auto &[j, bkj] : b.rvals[k]) {
                acc[j] = acc[j] + aik * bkj;
            }
        }
    }
}

```

```

    }
    for (const auto &[j, value] : acc) {
        res.set(i, j, value);
    }
}
return res;
}

SparseMatrix &operator+=(const SparseMatrix &b) { return *this = *this + b; }
SparseMatrix &operator-=(const SparseMatrix &b) { return *this = *this - b; }
SparseMatrix &operator*=(const SparseMatrix &b) { return *this = *this * b; }
SparseMatrix &operator*=(const T &k) { return *this = *this * k; }
};

```

The elimination routines require field-like arithmetic because they divide by pivots, so use types such as `double`, rational numbers, or modular integers. For floating-point matrices, replace `is_zero()` with an epsilon comparison if small roundoff values should be erased. At each pivot column, sparse elimination chooses the available row with the fewest stored entries to reduce fill-in, although an unfriendly matrix can still become dense.

- `row_reduce(a, limit)` converts columns  $[0, \text{limit})$  of `a` to sparse row echelon form, returning the rank found in those columns.
- `det(a)` returns the determinant of a square sparse matrix.
- `sparse_rank(a)` returns the rank of a sparse matrix.
- `solve_system(a, b, &x)` solves the system  $ax = b$ , returning 0 for one solution,  $-1$  for no solution, or  $-2$  for infinitely many solutions. When one solution exists, `x` is populated.

#### Time Complexity:

- $O(f \cdot \log d)$  for sparse elimination, where  $f$  is the number of entry updates performed after fill-in and  $d$  is a touched row or column size. In the worst case this is still cubic.
- $O(f \cdot \log d)$  per call to `det()`, `sparse_rank()`, and `solve_system()`.

**Space Complexity:**  $O(z + m)$  auxiliary for elimination, in addition to the copied matrix used by `det()` and `sparse_rank()` or the augmented matrix used by `solve_system()`.

```

template<typename T>
int row_reduce(SparseMatrix<T> &a, int limit) {
    assert(0 <= limit && limit <= a.num_cols());
    int r = 0;
    for (int c = 0; c < limit && r < a.num_rows(); c++) {
        int pivot = -1;
        for (const auto &[i, value] : a.col(c)) {
            if (i >= r && (pivot == -1 || a.row(i).size() < a.row(pivot).size())) {
                pivot = i;
            }
        }
        if (pivot == -1) {
            continue;
        }
        a.swap_rows(r, pivot);
        T inv_pivot = T{1} / a.get(r, c);
        std::vector<std::pair<int, T>> pivot_row(a.row(r).begin(), a.row(r).end());
        std::vector<int> touched_rows;
        for (const auto &[i, value] : a.col(c)) {
            if (i > r) {
                touched_rows.push_back(i);
            }
        }
        for (int i : touched_rows) {
            T coeff = -a.get(i, c) * inv_pivot;
            for (const auto &[j, value] : pivot_row) {
                if (j != c) {
                    a.add(i, j, coeff * value);
                }
            }
        }
    }
}

```

```

    }
    a.set(i, c, T{});
  }
  r++;
}
return r;
}

template<typename T>
T det(SparseMatrix<T> a) {
  assert(a.num_rows() == a.num_cols());
  int swaps = 0, r = 0, n = a.num_rows();
  T det = 1;
  for (int c = 0; c < n && r < n; c++) {
    int pivot = -1;
    for (const auto &[i, value] : a.col(c)) {
      if (i >= r && (pivot == -1 || a.row(i).size() < a.row(pivot).size())) {
        pivot = i;
      }
    }
    if (pivot == -1) {
      return T{};
    }
    if (pivot != r) {
      a.swap_rows(r, pivot);
      swaps++;
    }
    T pivot_value = a.get(r, c);
    det *= pivot_value;
    T inv_pivot = T{1} / pivot_value;
    std::vector<std::pair<int, T>> pivot_row(a.row(r).begin(), a.row(r).end());
    std::vector<int> touched_rows;
    for (const auto &[i, value] : a.col(c)) {
      if (i > r) {
        touched_rows.push_back(i);
      }
    }
    for (int i : touched_rows) {
      T coeff = -a.get(i, c) * inv_pivot;
      for (const auto &[j, value] : pivot_row) {
        if (j != c) {
          a.add(i, j, coeff * value);
        }
      }
      a.set(i, c, T{});
    }
    r++;
  }
  return (r == n ? (swaps % 2 == 0 ? det : -det) : T{});
}

template<typename T>
int sparse_rank(SparseMatrix<T> a) {
  return row_reduce(a, a.num_cols());
}

template<typename T>
int solve_system(const SparseMatrix<T> &a, const std::vector<T> &b, std::vector<T> *x) {
  if (x == nullptr || a.num_rows() != static_cast<int>(b.size())) {
    return -1;
  }
  int rows = a.num_rows(), cols = a.num_cols();
  SparseMatrix<T> aug(rows, cols + 1);
  for (int i = 0; i < rows; i++) {
    for (const auto &[j, value] : a.row(i)) {
      aug.set(i, j, value);
    }
    aug.set(i, cols, b[i]);
  }

```



```

}
row_reduce(aug, cols);
std::vector<int> pivot_col;
for (int i = 0; i < rows; i++) {
    int lead = -1;
    for (const auto &[j, value] : aug.row(i)) {
        if (j < cols) {
            lead = j;
            break;
        }
    }
    if (lead == -1) {
        if (aug.get(i, cols) != T{}) {
            return -1;
        }
    } else {
        pivot_col.push_back(lead);
    }
}
if (static_cast<int>(pivot_col.size()) < cols) {
    return -2;
}
x->assign(cols, T{});
for (int row = static_cast<int>(pivot_col.size()) - 1; row >= 0; row--) {
    int c = pivot_col[row];
    T rhs = aug.get(row, cols);
    for (const auto &[j, value] : aug.row(row)) {
        if (j > c && j < cols) {
            rhs -= value * (*x)[j];
        }
    }
    (*x)[c] = rhs / aug.get(row, c);
}
return 0;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SparseMatrix<int64_t> a(3, 4);
    a.set(0, 1, 5);
    a.set(2, 3, 7);
    a.add(0, 1, -2);
    a.add(1, 2, 4);

    assert(a.num_rows() == 3 && a.num_cols() == 4);
    assert(a.nonzeros() == 3);
    assert(a.get(0, 1) == 3);
    assert(a.get(0, 0) == 0);
    assert(a.row(0).begin()->first == 1);
    assert(a.col(2).begin()->first == 1);

    vector<int64_t> x{10, 20, 30, 40};
    assert((a * x == vector<int64_t>{60, 120, 280}));

    a.add(0, 1, -3); // Entries that become zero are removed from both row and column maps.
    assert(a.get(0, 1) == 0);
    assert(a.nonzeros() == 2);
    assert(a.row(0).empty());
    assert(a.col(1).empty());

    a.transpose();
    assert(a.num_rows() == 4 && a.num_cols() == 3);
}

```

```

assert(a.get(3, 2) == 7);
assert(a.get(2, 1) == 4);

// p = [1 2]   q = [0 3]   p + q = [1 5]   p * q = [4 3]
//      [0 1]   [2 0]       [2 1]       [2 0]
SparseMatrix<int64_t> p(2, 2), q(2, 2);
p.set(0, 0, 1);
p.set(0, 1, 2);
p.set(1, 1, 1);
q.set(0, 1, 3);
q.set(1, 0, 2);
SparseMatrix<int64_t> sum = p + q;
assert(sum.get(0, 0) == 1 && sum.get(0, 1) == 5);
assert(sum.get(1, 0) == 2 && sum.get(1, 1) == 1);
SparseMatrix<int64_t> product = p * q;
assert(product.get(0, 0) == 4 && product.get(0, 1) == 3);
assert(product.get(1, 0) == 2 && product.get(1, 1) == 0);
assert(product.nonzeros() == 3); // The zero entry is never stored.

// Cancelling entries are dropped rather than stored as explicit zeros.
SparseMatrix<int64_t> negated(2, 2);
negated.set(0, 0, -1);
negated.set(0, 1, -2);
negated.set(1, 1, -1);
assert((p + negated).nonzeros() == 0);
assert((p - p).nonzeros() == 0);

SparseMatrix<int64_t> difference = p - q;
assert(difference.get(0, 0) == 1 && difference.get(0, 1) == -1);
assert(difference.get(1, 0) == -2 && difference.get(1, 1) == 1);

SparseMatrix<int64_t> scaled = p * 3;
assert(scaled.get(0, 0) == 3 && scaled.get(0, 1) == 6 && scaled.get(1, 1) == 3);
assert((p * 0LL).nonzeros() == 0);

SparseMatrix<int64_t> acc = p;
acc += q;
assert(acc.get(0, 1) == 5 && acc.get(1, 0) == 2);
acc -= q;
acc *= 2;
assert(acc.get(0, 0) == 2 && acc.get(0, 1) == 4 && acc.get(1, 1) == 2);

SparseMatrix<double> d(3, 3);
d.set(0, 0, 2);
d.set(0, 1, 1);
d.set(1, 1, 3);
d.set(2, 0, 1);
d.set(2, 2, 4);
double determinant = det(d);
assert(determinant > 24 - 1e-9 && determinant < 24 + 1e-9);
assert(sparse_rank(d) == 3);

SparseMatrix<double> sys(3, 3);
sys.set(0, 0, -1);
sys.set(0, 1, 2);
sys.set(0, 2, 5);
sys.set(1, 0, 1);
sys.set(1, 2, -6);
sys.set(2, 0, -4);
sys.set(2, 1, 2);
sys.set(2, 2, 2);
vector<double> rhs{3, 1, -2}, sol;
assert(solve_system(sys, rhs, &sol) == 0);
for (int i = 0; i < sys.num_rows(); i++) {
    double sum = 0;
    for (const auto &[j, value] : sys.row(i)) {
        sum += value * sol[j];
    }
}

```

```

    assert(sum > rhs[i] - 1e-9 && sum < rhs[i] + 1e-9);
}
return 0;
}

```

### 6.5.9 Matrix-Tree Theorem

Kirchhoff's matrix-tree theorem counts the spanning trees of an undirected graph. Form the Laplacian matrix  $L = D - A$ , where  $D$  is the diagonal matrix of degrees and  $A$  is the adjacency matrix (with multiplicities for parallel edges). Every cofactor of  $L$  is equal, and that common value is the number of spanning trees. Equivalently, delete any one row and the matching column from  $L$  and take the determinant of what remains.

The determinant is evaluated modulo a prime by Gaussian elimination, so the count is returned modulo `MOD`. Spanning-tree counts grow astronomically, so a modular count is the usual contest form; run the routine under a few different primes and combine with the Chinese remainder theorem to recover a large exact value. The same construction handles weighted graphs (sum edge weights into the Laplacian to get the weighted tree count) and, with the directed Laplacian and a fixed deleted row, counts rooted spanning arborescences.

- `count_spanning_trees(n, edges)` returns the number of spanning trees of the undirected graph on nodes  $[0, n)$  with the given edge list, modulo `MOD`. Parallel edges are honored; self-loops do not affect the count. The result is 0 when the graph is disconnected, and 1 when `n` is 1.

**Time Complexity:**  $O(n^3 + m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(n^2)$  auxiliary.

```

#include <stdint>
#include <utility>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

int64_t count_spanning_trees(int n, const std::vector<std::pair<int, int>> &edges) {
    if (n <= 1) {
        return n == 1 ? 1 : 0;
    }
    // Build the Laplacian, then drop the last row and column to form an (n-1) by (n-1) cofactor.
    int m = n - 1;
    std::vector<std::vector<int64_t>> lap(m, std::vector<int64_t>(m));
    for (const auto &e : edges) {
        int u = e.first, v = e.second;
        if (u == v) {
            continue; // Self-loops contribute nothing to the Laplacian.
        }
        if (u < m) {
            lap[u][u] = (lap[u][u] + 1) % MOD;
        }
        if (v < m) {
            lap[v][v] = (lap[v][v] + 1) % MOD;
        }
        if (u < m && v < m) {
            lap[u][v] = (lap[u][v] - 1 + MOD) % MOD;
        }
    }
}

```

```

    lap[v][u] = (lap[v][u] - 1 + MOD) % MOD;
}
}
// Determinant modulo MOD by Gaussian elimination.
int64_t det = 1;
for (int col = 0; col < m; col++) {
    int pivot = -1;
    for (int r = col; r < m; r++) {
        if (lap[r][col] != 0) {
            pivot = r;
            break;
        }
    }
    if (pivot == -1) {
        return 0; // Singular cofactor: the graph is disconnected.
    }
    if (pivot != col) {
        std::swap(lap[pivot], lap[col]);
        det = (MOD - det) % MOD;
    }
    det = det * lap[col][col] % MOD;
    int64_t pinv = powmod(lap[col][col], MOD - 2, MOD);
    for (int r = col + 1; r < m; r++) {
        int64_t factor = lap[r][col] * pinv % MOD;
        if (factor == 0) {
            continue;
        }
        for (int k = col; k < m; k++) {
            lap[r][k] = (lap[r][k] - factor * lap[col][k]) % MOD;
            if (lap[r][k] < 0) {
                lap[r][k] += MOD;
            }
        }
    }
}
return det;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // A path 0-1-2 and a single triangle each have a known number of spanning trees.
    assert(count_spanning_trees(3, {{0, 1}, {1, 2}}) == 1); // A tree has exactly one.
    assert(count_spanning_trees(3, {{0, 1}, {1, 2}, {2, 0}}) == 3); // Triangle: drop any one edge.

    // Complete graphs: Cayley's formula gives  $n^{n-2}$  spanning trees.
    auto complete = [](int n) {
        vector<pair<int, int>> e;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                e.emplace_back(i, j);
            }
        }
        return count_spanning_trees(n, e);
    };
    assert(complete(4) == 16); //  $4^2$ .
    assert(complete(5) == 125); //  $5^3$ .

    // Disconnected graphs have no spanning tree.
    assert(count_spanning_trees(4, {{0, 1}, {2, 3}}) == 0);
    return 0;
}

```

## 6.6 Polynomials and Linear Recurrences

### 6.6.1 Number Theoretic Transform

The number theoretic transform (NTT) is the discrete Fourier transform carried out in modular arithmetic instead of over the complex numbers. It multiplies two polynomials (equivalently, computes the convolution of two sequences) modulo a prime in  $O(n \log n)$  time, with no floating point error. The modulus must be of the form  $c \cdot 2^k + 1$  so that the ring of integers modulo it contains a  $2^k$ -th root of unity, which limits transform sizes to powers of two up to  $2^k$ . The code below uses the common prime  $998244353 = 119 \cdot 2^{23} + 1$  with primitive root 3, supporting transforms of any power-of-two length up to  $2^{23}$ .

The transform replaces a coefficient vector with its evaluations at the powers of a root of unity; pointwise multiplying two such evaluation vectors and transforming back gives their convolution. This is the same divide-and-conquer butterfly as the fast Fourier transform, with the root of unity and all arithmetic taken modulo the prime.

- `ntt(a, invert = false)` transforms the vector `a` in place, whose length must be a power of two. The forward transform uses `invert = false`; the inverse uses `invert = true`.
- `convolve(a, b)` returns the convolution of `a` and `b` modulo the prime, that is, the coefficients of the product of the two polynomials whose coefficients are the inputs. The result has length `a.size() + b.size() - 1`, or is empty if either input is empty. Input values are assumed to lie in  $[0, \text{MOD})$ . Small inputs use direct multiplication to avoid NTT overhead.

#### Time Complexity:

- $O(n \log n)$  per call to `ntt()`, where  $n$  is the length of the vector.
- $O(|a||b|)$  per call to `convolve()` on small inputs and  $O(n \log n)$  otherwise, where  $n$  is the padded transform length.

#### Space Complexity:

- $O(1)$  auxiliary for `ntt()`, transforming in place.
- $O(n)$  auxiliary for `convolve()`.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

const int64_t MOD = 998244353;
const int64_t ROOT = 3;
const int MAX_POWER_OF_TWO = 23;
const int NAIVE_CUTOFF = 150;

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

void ntt(std::vector<int64_t> &a, bool invert = false) {
    int n = static_cast<int>(a.size());
    assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << MAX_POWER_OF_TWO));
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
    }
}
```

```

    }
    j ^= bit;
    if (i < j) {
        std::swap(a[i], a[j]);
    }
}
for (int len = 2; len <= n; len <<= 1) {
    int64_t root = powmod(ROOT, (MOD - 1) / len, MOD);
    if (invert) {
        root = powmod(root, MOD - 2, MOD);
    }
    for (int i = 0; i < n; i += len) {
        int64_t w = 1;
        for (int k = 0; k < len / 2; k++) {
            int64_t u = a[i + k], v = a[i + k + len / 2] * w % MOD;
            a[i + k] = (u + v) % MOD;
            a[i + k + len / 2] = (u - v + MOD) % MOD;
            w = w * root % MOD;
        }
    }
}
if (invert) {
    int64_t n_inv = powmod(n, MOD - 2, MOD);
    for (int64_t &x : a) {
        x = x * n_inv % MOD;
    }
}
}

std::vector<int64_t> convolve(std::vector<int64_t> a, std::vector<int64_t> b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int result_size = static_cast<int>(a.size() + b.size()) - 1;
    if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
        std::vector<int64_t> res(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
            }
        }
        return res;
    }
    int n = 1;
    while (n < result_size) {
        n <<= 1;
    }
    assert(n <= (1 << MAX_POWER_OF_TWO));
    a.resize(n);
    b.resize(n);
    ntt(a);
    ntt(b);
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * b[i] % MOD;
    }
    ntt(a, true);
    a.resize(result_size);
    return a;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {

```

```

// (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
vector<int64_t> a{1, 2, 3}, b{4, 5, 6};
assert((convolve(a, b) == vector<int64_t>{4, 13, 28, 27, 18}));
assert(convolve({}, b).empty());
assert((convolve(vector<int64_t>{5}, vector<int64_t>{7}) == vector<int64_t>{35}));
vector<int64_t> ones(160, 1);
vector<int64_t> d = convolve(ones, ones);
assert(d[0] == 1 && d[159] == 160 && d[318] == 1);
return 0;
}

```

## 6.6.2 Fast Fourier Transform

The fast Fourier transform (FFT) multiplies two polynomials, or equivalently convolves two sequences, in  $O(n \log n)$  time by evaluating them at the complex roots of unity. Like the number theoretic transform of the previous section, it replaces a coefficient vector with its evaluations at the powers of a root of unity, multiplies two such evaluation vectors pointwise, and transforms back. The difference is the arithmetic: the FFT works over the complex numbers with  $e^{2\pi i/n}$  as the root of unity, so it convolves arbitrary real coefficients rather than residues modulo a prime.

The trade-off is precision. Complex arithmetic in double precision accumulates rounding error, so the result is recovered by rounding each output to the nearest integer; this is reliable as long as the true coefficients stay well within the roughly 15 significant decimal digits a `double` can hold. When exact results modulo a prime are required, or coefficients are large, prefer the number theoretic transform; use the FFT for real-valued convolution or big-integer multiplication.

- `fft(a, invert = false)` transforms the complex vector `a` in place, whose length must be a power of two. The forward transform uses `invert = false`; the inverse uses `invert = true`.
- `convolve(a, b)` returns the convolution of two integer sequences `a` and `b`, that is, the coefficients of the product of the two polynomials whose coefficients are the inputs. The result has length `a.size() + b.size() - 1`, or is empty if either input is empty, with each entry rounded to the nearest integer. Small inputs use direct multiplication to avoid FFT overhead.

Overflow warning: The exact coefficients and the direct branch's intermediate products and sums must fit in `int64_t`.

### Time Complexity:

- $O(n \log n)$  per call to `fft()`, where  $n$  is the length of the vector.
- $O(|a||b|)$  per call to `convolve()` on small inputs and  $O(n \log n)$  otherwise, where  $n$  is the padded transform length.

### Space Complexity:

- $O(1)$  auxiliary for `fft()`, transforming in place.
- $O(n)$  auxiliary for `convolve()`.

```

#include <algorithm>
#include <cassert>
#include <cmath>
#include <complex>
#include <cstdint>
#include <utility>
#include <vector>

using cdbl = std::complex<double>;
const double PI = std::acos(-1.0);
const int NAIVE_CUTOFF = 150;

void fft(std::vector<cdbl> &a, bool invert = false) {
    int n = static_cast<int>(a.size());

```

```

assert(n > 0 && (n & (n - 1)) == 0);
for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) {
        j ^= bit;
    }
    j ^= bit;
    if (i < j) {
        std::swap(a[i], a[j]);
    }
}
for (int len = 2; len <= n; len <<= 1) {
    double angle = 2 * PI / len * (invert ? -1 : 1);
    cdbl root(std::cos(angle), std::sin(angle));
    for (int i = 0; i < n; i += len) {
        cdbl w(1);
        for (int k = 0; k < len / 2; k++) {
            cdbl u = a[i + k], v = a[i + k + len / 2] * w;
            a[i + k] = u + v;
            a[i + k + len / 2] = u - v;
            w *= root;
        }
    }
}
if (invert) {
    for (cdbl &x : a) {
        x /= n;
    }
}
}

std::vector<int64_t> convolve(const std::vector<int64_t> &a, const std::vector<int64_t> &b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int result_size = static_cast<int>(a.size() + b.size() - 1);
    if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
        std::vector<int64_t> res(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                res[i + j] += a[i] * b[j]; // Overflow warning.
            }
        }
        return res;
    }
    int n = 1;
    while (n < result_size) {
        n <<= 1;
    }
    std::vector<cdbl> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    fa.resize(n);
    fb.resize(n);
    fft(fa);
    fft(fb);
    for (int i = 0; i < n; i++) {
        fa[i] *= fb[i];
    }
    fft(fa, true);
    std::vector<int64_t> res(result_size);
    for (int i = 0; i < result_size; i++) {
        res[i] = std::llround(fa[i].real());
    }
    return res;
}
}

```



## Example Usage

```
using namespace std;

int main() {
    // (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
    vector<int64_t> a{1, 2, 3}, b{4, 5, 6};
    assert((convolve(a, b) == vector<int64_t>{4, 13, 28, 27, 18}));
    assert(convolve({}, b).empty());
    assert((convolve(vector<int64_t>{5}, vector<int64_t>{7}) == vector<int64_t>{35}));
    vector<int64_t> ones(160, 1);
    vector<int64_t> d = convolve(ones, ones);
    assert(d[0] == 1 && d[159] == 160 && d[318] == 1);
    return 0;
}
```

### 6.6.3 Polynomial Operations

Polynomial operations treat a vector `a` as coefficients of  $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$  over a field. For formal power series operations, vectors represent coefficients modulo  $x^n$ ; no notion of analytic convergence is involved. This section implements the core algebra needed for formal power series and polynomial division modulo the prime 998244353. Multiplication uses the number theoretic transform, so inverse and division are fast enough for large polynomials while still keeping the API close to the underlying coefficient vectors.

The code aliases one modular field element as `Coeff` and one coefficient vector as `Poly`. Input coefficients must lie in  $[0, \text{MOD})$ .

- `eval(a, x)` returns the value of `a` at `x` modulo `MOD`; `x` is reduced modulo `MOD`.
- `add(a, b)` returns `a + b`.
- `subtract(a, b)` returns `a - b`.
- `multiply(a, b)` returns `a * b`.
- `derivative(a)` returns the formal derivative of `a`.
- `antiderivative(a)` returns the formal antiderivative of `a` with constant term zero.

For formal power series, the inverse modulo  $x^n$  is a series  $b$  satisfying  $a \cdot b \equiv 1 \pmod{x^n}$ . Series division uses this inverse, and logarithm follows from  $(\log a)' = a'/a$ . Exponential and square root use Newton iteration, doubling the number of correct coefficients each round. Power factors  $a = x^t cb$ , where  $b(0) = 1$ , and applies  $a^k = x^{tk} c^k \exp(k \log b)$ . To extend the normalized square-root implementation, factor out the lowest power of  $x$ , normalize its constant term, then multiply back its modular square root (see 6.3.5); an odd lowest nonzero power has no square root.

- `inverse(a, n)` returns the first `n` coefficients of  $1/a$ , requiring `a[0]` to be nonzero.
- `series_divide(a, b, n)` returns the first `n` coefficients of the formal power series  $a/b$ , requiring `b[0]` to be nonzero.
- `log(a, n)` returns the first `n` coefficients of  $\log(a)$ , requiring `a[0]` to be 1.
- `exp(a, n)` returns the first `n` coefficients of  $\exp(a)$ , requiring `a[0]` to be 0.
- `sqr(a, n)` returns the first `n` coefficients of the square root of `a` whose constant term is 1, requiring `a[0]` to be 1.
- `power(a, k, n)` returns the first `n` coefficients of  $a^k$  for a nonnegative `uint64_t` exponent `k`; `k = 0` returns the constant series 1.

Euclidean polynomial division uses the reversal trick: reverse both polynomials, compute a truncated series inverse of the reversed divisor, multiply, truncate, and reverse back.

- `div(a, b)` returns the polynomial quotient of  $a/b$ , requiring `b` to be nonzero.
- `mod(a, b)` returns the polynomial remainder of  $a/b$ , requiring `b` to be nonzero.

**Time Complexity:**

- $O(n)$  per call to `eval()`, `add()`, `subtract()`, `derivative()`, and `antiderivative()`.
- $O(|a||b|)$  per call to `multiply()` on small inputs and  $O(n \log n)$  otherwise, where  $n$  is the padded transform length.
- $O(n \log n)$  per call to `inverse()` and `series_divide()`, where  $n$  is the requested length.
- $O(n \log n)$  per call to `log()`, `exp()`, and `sqrt()`, where  $n$  is the requested length. The exponential carries the largest constant, since every Newton round computes a logarithm.
- $O(n \log n + \log k)$  per call to `power()`, where  $n$  is the requested length.
- $O(n \log n)$  per call to `div()` and `mod()`, where  $n$  is the padded multiplication length.

**Space Complexity:**  $O(1)$  auxiliary per call to `eval()` and  $O(n)$  auxiliary for all other operations.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

using Coeff = int64_t;
using Poly = std::vector<Coeff>;

const Coeff MOD = 998244353;
const Coeff ROOT = 3;
const int MAX_POWER_OF_TWO = 23;
const int NAIVE_CUTOFF = 150;

Coeff powmod(Coeff x, uint64_t n, Coeff m = MOD) {
    Coeff res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

void ntt(Poly &a, bool invert) {
    int n = static_cast<int>(a.size());
    assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << MAX_POWER_OF_TWO));
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
            std::swap(a[i], a[j]);
        }
    }
    for (int len = 2; len <= n; len <<= 1) {
        Coeff root = powmod(ROOT, (MOD - 1) / len);
        if (invert) {
            root = powmod(root, MOD - 2);
        }
        for (int i = 0; i < n; i += len) {
            Coeff w = 1;
            for (int k = 0; k < len / 2; k++) {
                Coeff u = a[i + k], v = a[i + k + len / 2] * w % MOD;
                a[i + k] = (u + v) % MOD;
                a[i + k + len / 2] = (u - v + MOD) % MOD;
                w = w * root % MOD;
            }
        }
    }
}
```

```

    if (invert) {
        Coeff n_inv = powmod(n, MOD - 2);
        for (Coeff &x : a) {
            x = x * n_inv % MOD;
        }
    }
}

void trim(Poly &a) {
    while (!a.empty() && a.back() == 0) {
        a.pop_back();
    }
}

Coeff eval(const Poly &a, Coeff x) {
    x = (x % MOD + MOD) % MOD;
    Coeff res = 0;
    for (auto it = a.rbegin(); it != a.rend(); ++it) {
        res = (res * x + *it) % MOD;
    }
    return res;
}

Poly add(const Poly &a, const Poly &b) {
    Poly res(std::max(a.size(), b.size()));
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        Coeff x = i < static_cast<int>(a.size()) ? a[i] : 0;
        Coeff y = i < static_cast<int>(b.size()) ? b[i] : 0;
        res[i] = (x + y) % MOD;
    }
    trim(res);
    return res;
}

Poly subtract(const Poly &a, const Poly &b) {
    Poly res(std::max(a.size(), b.size()));
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        Coeff x = i < static_cast<int>(a.size()) ? a[i] : 0;
        Coeff y = i < static_cast<int>(b.size()) ? b[i] : 0;
        res[i] = (x - y + MOD) % MOD;
    }
    trim(res);
    return res;
}

Poly multiply(Poly a, Poly b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int result_size = static_cast<int>(a.size() + b.size() - 1);
    if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
        Poly res(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
            }
        }
        trim(res);
        return res;
    }
    int n = 1;
    while (n < result_size) {
        n <<= 1;
    }
    assert(n <= (1 << MAX_POWER_OF_TWO));
    a.resize(n);
    b.resize(n);
    ntt(a, false);

```

```

    ntt(b, false);
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * b[i] % MOD;
    }
    ntt(a, true);
    a.resize(result_size);
    trim(a);
    return a;
}

Poly derivative(const Poly &a) {
    if (a.size() <= 1) {
        return {};
    }
    Poly res(a.size() - 1);
    for (int i = 1; i < static_cast<int>(a.size()); i++) {
        res[i - 1] = a[i] * i % MOD;
    }
    trim(res);
    return res;
}

Poly antiderivative(const Poly &a) {
    Poly res(a.size() + 1);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        res[i + 1] = a[i] * powmod(i + 1, MOD - 2) % MOD;
    }
    return res;
}

Poly inverse(const Poly &a, int n) {
    assert(n >= 0 && !a.empty() && a[0] != 0);
    if (n == 0) {
        return {};
    }
    Poly res{powmod(a[0], MOD - 2)};
    while (static_cast<int>(res.size()) < n) {
        int len = static_cast<int>(res.size()) << 1;
        Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), len));
        Poly prod = multiply(multiply(res, res), cut);
        res.resize(len);
        for (int i = len / 2; i < std::min(len, static_cast<int>(prod.size())); i++) {
            res[i] = (MOD - prod[i]) % MOD;
        }
    }
    res.resize(n);
    return res;
}

Poly series_divide(const Poly &a, const Poly &b, int n) {
    assert(n >= 0 && !b.empty() && b[0] != 0);
    if (n == 0) {
        return {};
    }
    Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), n));
    Poly res = multiply(cut, inverse(b, n));
    res.resize(n);
    return res;
}

Poly log(const Poly &a, int n) {
    assert(n >= 0 && !a.empty() && a[0] == 1);
    if (n == 0) {
        return {};
    }
    // Since (log a)' = a'/a, integrating the truncated quotient recovers log a.
    Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), n));
    Poly res = multiply(derivative(cut), inverse(cut, n));

```

```

    res.resize(n - 1);
    res = antiderivative(res);
    res.resize(n);
    return res;
}

Poly exp(const Poly &a, int n) {
    assert(n >= 0 && (a.empty() || a[0] == 0));
    if (n == 0) {
        return {};
    }
    // Newton iteration on  $\log(f) - a = 0$  gives  $f \leftarrow f \cdot (1 - \log(f) + a)$ , doubling the number of
    // correct coefficients each round.
    Poly res{1};
    while (static_cast<int>(res.size()) < n) {
        int len = static_cast<int>(res.size()) << 1;
        Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), len));
        Poly correction = subtract(cut, log(res, len));
        correction.resize(std::max(1, static_cast<int>(correction.size())));
        correction[0] = (correction[0] + 1) % MOD;
        res = multiply(res, correction);
        res.resize(len);
    }
    res.resize(n);
    return res;
}

Poly sqrt(const Poly &a, int n) {
    assert(n >= 0 && !a.empty() && a[0] == 1);
    if (n == 0) {
        return {};
    }
    // Newton iteration on  $f^2 - a = 0$  gives  $f \leftarrow (f + a/f) / 2$ .
    const Coeff INV2 = powmod(2, MOD - 2);
    Poly res{1};
    while (static_cast<int>(res.size()) < n) {
        int len = static_cast<int>(res.size()) << 1;
        Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), len));
        res = add(res, multiply(cut, inverse(res, len)));
        res.resize(len);
        for (Coeff &c : res) {
            c = c * INV2 % MOD;
        }
    }
    res.resize(n);
    return res;
}

Poly power(const Poly &a, uint64_t k, int n) {
    assert(n >= 0);
    if (n == 0) {
        return {};
    }
    Poly res(n);
    if (k == 0) {
        res[0] = 1;
        return res;
    }
    int limit = std::min(static_cast<int>(a.size()), n);
    int first = 0;
    while (first < limit && a[first] == 0) {
        first++;
    }
    if (first == limit || (first > 0 && k > static_cast<uint64_t>((n - 1) / first))) {
        return res;
    }
    int shift = static_cast<int>(static_cast<uint64_t>(first) * k);
    int len = n - shift;

```

```

    int terms = std::min(static_cast<int>(a.size()) - first, len);
    Poly normalized(a.begin() + first, a.begin() + first + terms);
    Coeff leading = normalized[0];
    Coeff leading_inv = powmod(leading, MOD - 2);
    for (Coeff &c : normalized) {
        c = c * leading_inv % MOD;
    }
    Poly exponent = log(normalized, len);
    Coeff k_mod = static_cast<Coeff>(k % static_cast<uint64_t>(MOD));
    for (Coeff &c : exponent) {
        c = c * k_mod % MOD;
    }
    Poly body = exp(exponent, len);
    Coeff leading_power = powmod(leading, k);
    for (int i = 0; i < len; i++) {
        res[i + shift] = body[i] * leading_power % MOD;
    }
    return res;
}

Poly div(Poly a, Poly b) {
    trim(a);
    trim(b);
    assert(!b.empty());
    if (a.size() < b.size()) {
        return {};
    }
    int quotient_size = static_cast<int>(a.size() - b.size() + 1);
    std::reverse(a.begin(), a.end());
    std::reverse(b.begin(), b.end());
    b.resize(quotient_size);
    Poly q = multiply(a, inverse(b, quotient_size));
    q.resize(quotient_size);
    std::reverse(q.begin(), q.end());
    trim(q);
    return q;
}

Poly mod(const Poly &a, const Poly &b) {
    Poly q = div(a, b);
    Poly res = subtract(a, multiply(q, b));
    if (res.size() >= b.size()) {
        res.resize(b.size() - 1);
    }
    trim(res);
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 5 + 7x + 11x^2 evaluates to 63 at x = 2 and 9 at x = -1.
    Poly p{5, 7, 11};
    assert(eval(p, 2) == 63 && eval(p, -1) == 9);

    // (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
    Poly a{1, 2, 3}, b{4, 5, 6};
    assert((add(a, b) == Poly{5, 7, 9}));
    assert((subtract(b, a) == Poly{3, 3, 3}));
    assert((multiply(a, b) == Poly{4, 13, 28, 27, 18}));

    // d/dx (5 + 7x + 11x^2) = 7 + 22x; integrating back chooses constant term 0.
    assert((derivative(p) == Poly{7, 22}));
}

```

```

assert((antiderivative(derivative(p)) == Poly{0, 7, 11}));

// (1 - x)^-1 = 1 + x + x^2 + x^3 + ... modulo x^6.
assert((inverse({1, MOD - 1}, 6) == Poly{1, 1, 1, 1, 1, 1}));

// (1 + x) / (1 - x) = 1 + 2x + 2x^2 + ... modulo x^5.
assert((series_divide({1, 1}, {1, MOD - 1}, 5) == Poly{1, 2, 2, 2, 2}));

// (x^3 - 1) / (x - 1) = x^2 + x + 1, remainder 0.
Poly dividend{MOD - 1, 0, 0, 1};
Poly divisor{MOD - 1, 1};
assert((div(dividend, divisor) == Poly{1, 1, 1}));
assert(mod(dividend, divisor).empty());

// exp(x) = 1 + x + x^2/2 + x^3/6 + ..., checked by clearing each denominator.
Poly exp_x = exp({0, 1}, 4);
assert(exp_x[0] == 1 && exp_x[1] == 1);
assert(exp_x[2] * 2 % MOD == 1 && exp_x[3] * 6 % MOD == 1);

// log(1 + x) = x - x^2/2 + x^3/3 - ...
Poly log_1x = log({1, 1}, 4);
assert(log_1x[0] == 0 && log_1x[1] == 1);
assert((log_1x[2] * 2 + 1) % MOD == 0 && log_1x[3] * 3 % MOD == 1);

// Truncated to the same length, exp and log invert each other.
Poly series{1, 5, 9, 2, 7};
Poly logged = log(series, 5);
assert(logged[0] == 0);
assert(exp(logged, 5) == series);

// A series square root squares back to the original, modulo x^5.
Poly root = sqrt(series, 5);
Poly root_squared = multiply(root, root);
root_squared.resize(5);
assert(root_squared == series);

// (2x + x^2)^3 = 8x^3 + 12x^4 + 6x^5 + x^6.
assert((power({0, 2, 1}, 3, 7) == Poly{0, 0, 0, 8, 12, 6, 1}));
assert((power({}, 0, 4) == Poly{1, 0, 0, 0}));

Poly ones(160, 1);
Poly square = multiply(ones, ones);
assert(square[0] == 1 && square[159] == 160 && square[318] == 1);
return 0;
}

```

## 6.6.4 Polynomial Interpolation

Recovers the unique polynomial of degree less than  $n$  that passes through  $n$  given points, working modulo a prime. The  $x$  coordinates must be distinct; the  $y$  coordinates may be anything. Two operations are provided: reconstructing the full coefficient vector, and evaluating the interpolating polynomial at a single point without ever forming the coefficients.

Coefficient recovery uses Newton's divided differences. It first replaces the sample values by their successive divided differences in place, then expands the Newton basis  $1, (x - x_0), (x - x_0)(x - x_1), \dots$  into ordinary coefficients by multiplying through one factor at a time. Point evaluation instead applies the Lagrange formula directly, summing each sample weighted by the product of  $(t - x_j)/(x_k - x_j)$  over the other nodes. Coefficient recovery performs  $O(n^2)$  modular inverses, while point evaluation performs  $n$ .

- `interpolate(x, y)` returns the coefficients  $a_0, \dots, a_{n-1}$  (constant term first) of the polynomial  $P$  with  $P(x[i]) = y[i]$ , modulo `MOD`. The entries of `x` must be distinct modulo `MOD`.
- `interpolate_at(x, y, t)` returns  $P(t)$  modulo `MOD` for that same polynomial, without building its coefficients.

**Time Complexity:**  $O(n^2 \log p)$  per call to `interpolate()` and  $O(n^2 + n \log p)$  per call to `interpolate_at()`, where  $n$  is the number of points and  $p$  is `MOD`.

**Space Complexity:**  $O(n)$  auxiliary.

```
#include <cassert>
#include <cstdint>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = res * x % m;
        }
        x = x * x % m;
    }
    return res;
}

int64_t modnorm(int64_t a) { return (a %= MOD) < 0 ? a + MOD : a; }
int64_t addmod(int64_t a, int64_t b) { return modnorm(modnorm(a) + modnorm(b)); }
int64_t submod(int64_t a, int64_t b) { return modnorm(modnorm(a) - modnorm(b)); }
int64_t inv(int64_t a) { return powmod(modnorm(a), MOD - 2, MOD); }

std::vector<int64_t> interpolate(std::vector<int64_t> x, std::vector<int64_t> y) {
    assert(x.size() == y.size());
    int n = static_cast<int>(x.size());
    for (int i = 0; i < n; i++) {
        x[i] = modnorm(x[i]);
        y[i] = modnorm(y[i]);
    }
    // Replace y by its divided differences in place.
    for (int k = 0; k < n; k++) {
        for (int i = k + 1; i < n; i++) {
            y[i] = submod(y[i], y[k]) * inv(submod(x[i], x[k])) % MOD;
        }
    }
    // Expand the Newton basis into ordinary coefficients.
    std::vector<int64_t> res(n), temp(n);
    temp[0] = 1;
    for (int k = 0; k < n; k++) {
        int64_t last = 0;
        for (int i = 0; i < n; i++) {
            res[i] = addmod(res[i], y[k] * temp[i] % MOD);
            int64_t old = temp[i];
            temp[i] = last;
            last = old;
            temp[i] = submod(temp[i], last * x[k] % MOD);
        }
    }
    return res;
}

int64_t interpolate_at(const std::vector<int64_t> &x, const std::vector<int64_t> &y, int64_t t) {
    assert(x.size() == y.size());
    int n = static_cast<int>(x.size());
    t = modnorm(t);
    int64_t res = 0;
    for (int k = 0; k < n; k++) {
        int64_t num = 1, den = 1;
        for (int j = 0; j < n; j++) {
            if (j != k) {
                num = num * submod(t, x[j]) % MOD;
                den = den * submod(x[k], x[j]) % MOD;
            }
        }
    }
}
```



```

    }
    res = addmod(res, modnorm(y[k]) * num % MOD * inv(den) % MOD);
}
return res;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // P(x) = x^2 + 2x + 3 sampled at x = 0, 1, 2 gives y = 3, 6, 11.
    vector<int64_t> x{0, 1, 2}, y{3, 6, 11};
    assert((interpolate(x, y) == vector<int64_t>{3, 2, 1}));
    assert(interpolate_at(x, y, 5) == 38); // 25 + 10 + 3.

    // Non-consecutive nodes for P(x) = 2x^3 - x + 4.
    auto poly = [](int64_t t) { return modnorm(2 * t * t * t - t + 4); };
    vector<int64_t> xs{2, 5, 7, 10}, ys;
    for (int64_t v : xs) {
        ys.push_back(poly(v));
    }
    assert((interpolate(xs, ys) == vector<int64_t>{4, MOD - 1, 0, 2})); // 4 - x + 2x^3.
    assert(interpolate_at(xs, ys, 13) == poly(13));
    return 0;
}

```

### 6.6.5 Linear Recurrence (Berlekamp-Massey)

The Berlekamp-Massey algorithm finds the shortest linear recurrence that generates a given sequence over a field. Given the first terms of a sequence taken modulo a prime, it returns coefficients  $\Lambda_0, \dots, \Lambda_{L-1}$  of the minimal recurrence  $s_i = \Lambda_0 s_{i-1} + \Lambda_1 s_{i-2} + \dots + \Lambda_{L-1} s_{i-L} \pmod{p}$ , holding for every  $i \geq L$ . It scans the sequence once, maintaining a current candidate recurrence and correcting it whenever a term fails to match the prediction, using the last position where a correction was needed. The order  $L$  never decreases, and  $2L$  terms suffice to determine a recurrence of order  $L$ . This is the standard tool for guessing a linear recurrence from sampled values, after which the sequence can be extended or its  $n$ -th term found by fast methods.

An order- $L$  recurrence generally needs at least  $2L$  terms to be determined, so the result is most trustworthy when  $s$  is comfortably longer than twice the returned length. If the returned  $L$  is close to  $n/2$ , the sequence is under-determined and the recurrence may be a spurious overfit.

The code operates modulo the prime 998244353; change `MOD` to use a different prime field.

- `berlekamp_massey(s)` returns the coefficients of the shortest recurrence that the sequence `s` satisfies, as described above. The returned length is the order  $L$  of the recurrence. An empty vector is returned when `s` is entirely zero (no recurrence is needed). Entries are reduced modulo `MOD` internally.

**Time Complexity:**  $O(n^2 + L \log p)$  per call, where  $n$  is the length of `s`,  $L$  is the order of the recurrence found, and  $p$  is the modulus.

**Space Complexity:**  $O(n)$  auxiliary.

```

#include <cstdint>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (x %= m; n > 0; n >>= 1) {

```

```

    if (n & 1) {
        res = res * x % m;
    }
    x = x * x % m;
}
return res;
}

std::vector<int64_t> berlekamp_massey(const std::vector<int64_t> &s) {
    int n = static_cast<int>(s.size()), len = 0, m = 0;
    if (n == 0) {
        return {};
    }
    std::vector<int64_t> cur(n), last(n), prev;
    cur[0] = last[0] = 1;
    int64_t inv_last_delta = 1;
    for (int i = 0; i < n; i++) {
        m++;
        int64_t delta = (s[i] % MOD + MOD) % MOD;
        for (int j = 1; j <= len; j++) {
            int64_t value = (s[i - j] % MOD + MOD) % MOD;
            delta = (delta + cur[j] * value) % MOD;
        }
        if (delta == 0) {
            continue;
        }
        prev = cur;
        int64_t coeff = delta * inv_last_delta % MOD;
        for (int j = m; j < n; j++) {
            cur[j] = (cur[j] - coeff * last[j - m] % MOD + MOD) % MOD;
        }
        if (2 * len > i) {
            continue;
        }
        len = i + 1 - len;
        last = prev;
        inv_last_delta = powmod(delta, MOD - 2, MOD); // Refresh only when the length increases.
        m = 0;
    }
    cur.resize(len + 1);
    cur.erase(cur.begin());
    for (int64_t &x : cur) {
        x = (MOD - x) % MOD;
    }
    return cur;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(berlekamp_massey({}).empty());
    // Fibonacci: s[i] = s[i-1] + s[i-2].
    vector<int64_t> fib{1, 1, 2, 3, 5, 8, 13, 21};
    assert((berlekamp_massey(fib) == vector<int64_t>{1, 1}));

    // Geometric: s[i] = 2 * s[i-1].
    vector<int64_t> geo{1, 2, 4, 8, 16, 32};
    assert((berlekamp_massey(geo) == vector<int64_t>{2}));
    assert((berlekamp_massey(vector<int64_t>{1, -1, 1, -1, 1, -1}) == vector<int64_t>{MOD - 1}));

    // Tribonacci: s[i] = s[i-1] + s[i-2] + s[i-3].
    vector<int64_t> trib{0, 1, 1, 2, 4, 7, 13, 24, 44};
    assert((berlekamp_massey(trib) == vector<int64_t>{1, 1, 1}));
}

```

```

    return 0;
}

```

### 6.6.6 Linear Recurrence (Kitamasa)

Computes the  $n$ -th term of a linear recurrence modulo a prime in logarithmic time. Given a recurrence of order  $L$ ,  $s_i = c_0 s_{i-1} + c_1 s_{i-2} + \dots + c_{L-1} s_{i-L}$ , together with the first  $L$  terms, the naive approach unrolls  $n$  steps. Kitamasa's method instead jumps directly to index  $n$  by working with the characteristic polynomial. The coefficient layout matches the output of Berlekamp-Massey, so the two compose directly: guess the recurrence from sampled values, then jump to any index.

The key identity is that the  $n$ -th term is a fixed linear combination of the first  $L$  terms, with weights given by  $x^n$  reduced modulo the characteristic polynomial  $f(x) = x^L - c_0 x^{L-1} - \dots - c_{L-1}$ . The reduction  $x^L = c_0 x^{L-1} + \dots + c_{L-1}$  lets any product of two degree- $< L$  polynomials be folded back to degree  $< L$ , so  $x^n \bmod f$  is obtained by binary exponentiation, and the  $n$ -th term is the dot product of its coefficients with the initial terms.

Each doubling step is one polynomial multiplication followed by one remainder, both taken from section 6.6.3, where the multiplication is a number theoretic transform and the remainder is a Newton-iterated division. Squaring and reducing by hand instead costs  $O(L^2)$  per step, which is simpler to write but asymptotically worse; `MOD` is inherited from that section and must stay transform-friendly.

- `kth_term(rec, init, n)` returns  $s_n$  modulo `MOD`, where `rec` holds the coefficients  $c_0, \dots, c_{L-1}$  and `init` holds at least  $L$  initial terms  $s_0, \dots, s_{L-1}$ . Indexing is 0-based and `n`  $\geq 0$ . Values are reduced modulo `MOD` internally.

**Time Complexity:**  $O(L \log L \log n)$  per call.

**Space Complexity:**  $O(L)$  auxiliary.

```

#include <cassert>
#include <cstdint>
#include <vector>

#define main polynomial_example
#include "6.6.3_Polynomial_Operations.cpp"
#undef main

int64_t kth_term(const std::vector<int64_t> &rec, const std::vector<int64_t> &init, int64_t n) {
    assert(n >= 0 && init.size() >= rec.size());
    int L = static_cast<int>(rec.size());
    if (n < static_cast<int64_t>(init.size())) {
        return ((init[n] % MOD) + MOD) % MOD;
    }
    if (L == 0) {
        return 0;
    }
    // The characteristic polynomial f(x) = x^L - c_0 x^(L-1) - ... - c_(L-1), indexed by degree.
    Poly f(L + 1);
    f[L] = 1;
    for (int j = 0; j < L; j++) {
        f[L - 1 - j] = ((-rec[j]) % MOD + MOD) % MOD;
    }
    Poly result{1}, base = mod(Poly{0, 1}, f); // The polynomials x^0 and x, each modulo f.
    for (int64_t e = n; e > 0; e >>= 1) {
        if (e & 1) {
            result = mod(multiply(result, base), f);
        }
        base = mod(multiply(base, base), f);
    }
    int64_t ans = 0;
    for (int j = 0; j < L && j < static_cast<int>(result.size()); j++) {

```

```

    ans = (ans + result[j] * (((init[j] % MOD) + MOD) % MOD)) % MOD;
}
return ans;
}

```

## Example Usage

```

using namespace std;

int main() {
    // Fibonacci s_i = s_{i-1} + s_{i-2} with s_0 = s_1 = 1: 1, 1, 2, 3, 5, 8, ...
    vector<int64_t> fib_rec{1, 1}, fib_init{1, 1};
    assert(kth_term(fib_rec, fib_init, 0) == 1);
    assert(kth_term(fib_rec, fib_init, 9) == 55);
    assert(kth_term(fib_rec, fib_init, 10) == 89);

    // Tribonacci s_i = s_{i-1} + s_{i-2} + s_{i-3} with s_0,s_1,s_2 = 0,1,1.
    vector<int64_t> trib_rec{1, 1, 1}, trib_init{0, 1, 1};
    assert(kth_term(trib_rec, trib_init, 8) == 44);

    // Geometric s_i = 3 s_{i-1}, s_0 = 1: matches 3^n modulo MOD.
    vector<int64_t> geo_rec{3}, geo_init{1};
    int64_t pw = 1;
    for (int i = 0; i < 50; i++) {
        assert(kth_term(geo_rec, geo_init, i) == pw);
        pw = pw * 3 % MOD;
    }
    assert(kth_term({-1}, {1}, 5) == MOD - 1);
    return 0;
}

```

## 6.7 Probability and Statistics

---

### 6.7.1 Probability Distributions

The following functions evaluate the standard probability distributions. For a discrete distribution, the probability mass function gives the probability of an exact outcome and the cumulative distribution function gives the probability of that outcome or any smaller one. For a continuous distribution, the density integrates to those probabilities rather than being one, so only the cumulative function is a probability. Counts must be nonnegative, and every probability argument must lie in  $[0, 1]$ .

Every mass function here is evaluated in logarithms and exponentiated once at the end. Computing  $\binom{n}{k} p^k (1-p)^{n-k}$  directly overflows the binomial coefficient and underflows the powers long before the product itself becomes small, whereas the logarithms stay near the magnitude of the answer. `std::lgamma` supplies  $\ln n!$  as  $\ln \Gamma(n+1)$  without forming  $n!$ , so the factorials never exist, and outcomes are ordered so that no cancellation occurs.

Each cumulative function sums its mass function term by term, which keeps the code short and the error small but costs one term per outcome. For a single tail probability at a very large count the normal approximation with a continuity correction is the usual substitute, while `quantile()` inverts any of these by the bisection of section 5.1.1.

- `log_factorial(n)` returns  $\ln(n!)$ , and `log_choose(n, k)` returns  $\ln \binom{n}{k}$ , which is  $-\infty$  when  $k$  lies outside  $[0, n]$ .
- `binomial_pmf(n, k, p)` and `binomial_cdf(n, k, p)` evaluate the number of successes in  $n$  independent trials that each succeed with probability  $p$ .
- `geometric_pmf(k, p)` and `geometric_cdf(k, p)` evaluate the number of trials up to and including the first success, so  $k$  is at least 1 and  $p$  must be positive.

- `negative_binomial_pmf(k, r, p)` and `negative_binomial_cdf(k, r, p)` evaluate the number of trials up to and including the `r`-th success, so `k` is at least `r`, which is at least 1.
- `poisson_pmf(k, lambda)` and `poisson_cdf(k, lambda)` evaluate the number of events in one unit of time when events arrive independently at nonnegative mean rate `lambda`.
- `hypergeometric_pmf(n, k, draws, hits)` and `hypergeometric_cdf(n, k, draws, hits)` evaluate the number of marked items among `draws` items taken without replacement from a population of `n` items of which `k` are marked.
- `normal_pdf(x, mu = 0, sigma = 1)` and `normal_cdf(x, mu = 0, sigma = 1)` evaluate the normal distribution of mean `mu` and positive standard deviation `sigma`, defaulting to the standard normal distribution.
- `exponential_pdf(x, lambda)` and `exponential_cdf(x, lambda)` evaluate the waiting time until the next event when events arrive at positive mean rate `lambda`.
- `quantile(cdf, p, lo, hi, iterations = 100)` returns the smallest  $x$  in  $[lo, hi]$  whose cumulative probability `cdf(x)` reaches `p`, for any nondecreasing `cdf`.

Sampling is left to `<random>`, whose `std::binomial_distribution`, `std::poisson_distribution`, `std::normal_distribution`, and their siblings draw from exactly these families. What the standard library does not provide, and this section supplies, is the ability to evaluate and invert them.

Overflow warning: `log_choose()` is exact only while `std::lgamma` resolves neighboring integers, which holds well past the range where a `double` probability retains any precision.

#### Time Complexity:

- $O(1)$  per call to every mass and density function, and to `log_factorial()` and `log_choose()`.
- $O(k)$  per call to `binomial_cdf()`, `poisson_cdf()`, `negative_binomial_cdf()`, and `hypergeometric_cdf()`, where  $k$  is the outcome whose tail is summed.
- $O(1)$  per call to `geometric_cdf()`, `normal_cdf()`, and `exponential_cdf()`, which have closed forms.
- $O(n)$  calls to `cdf()` per call to `quantile()`, where  $n$  is the number of iterations.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <limits>

double log_factorial(int n) {
    return std::lgamma(n + 1.0);
}

double log_choose(int n, int k) {
    if (k < 0 || k > n) {
        return -std::numeric_limits<double>::infinity();
    }
    return log_factorial(n) - log_factorial(k) - log_factorial(n - k);
}

double binomial_pmf(int n, int k, double p) {
    if (k < 0 || k > n) {
        return 0;
    }
    if (p <= 0) {
        return k == 0 ? 1 : 0;
    }
    if (p >= 1) {
        return k == n ? 1 : 0;
    }
    return std::exp(log_choose(n, k) + k * std::log(p) + (n - k) * std::log1p(-p));
}

double binomial_cdf(int n, int k, double p) {
    double res = 0;
    for (int i = 0; i <= std::min(k, n); i++) {
        res += binomial_pmf(n, i, p);
    }
}
```

```

    return std::min(res, 1.0);
}

double geometric_pmf(int k, double p) {
    return k < 1 ? 0 : p * std::pow(1 - p, k - 1);
}

double geometric_cdf(int k, double p) {
    return k < 1 ? 0 : -std::expm1(k * std::log1p(-p));
}

double negative_binomial_pmf(int k, int r, double p) {
    if (k < r || r < 1) {
        return 0;
    }
    if (p >= 1) {
        return k == r ? 1 : 0;
    }
    return std::exp(log_choose(k - 1, r - 1) + r * std::log(p) + (k - r) * std::log1p(-p));
}

double negative_binomial_cdf(int k, int r, double p) {
    double res = 0;
    for (int i = r; i <= k; i++) {
        res += negative_binomial_pmf(i, r, p);
    }
    return std::min(res, 1.0);
}

double poisson_pmf(int k, double lambda) {
    if (k < 0) {
        return 0;
    }
    if (lambda <= 0) {
        return k == 0 ? 1 : 0;
    }
    return std::exp(-lambda + k * std::log(lambda) - log_factorial(k));
}

double poisson_cdf(int k, double lambda) {
    double res = 0;
    for (int i = 0; i <= k; i++) {
        res += poisson_pmf(i, lambda);
    }
    return std::min(res, 1.0);
}

double hypergeometric_pmf(int n, int k, int draws, int hits) {
    if (hits < 0 || hits > k || draws - hits < 0 || draws - hits > n - k) {
        return 0;
    }
    return std::exp(log_choose(k, hits) + log_choose(n - k, draws - hits) - log_choose(n, draws));
}

double hypergeometric_cdf(int n, int k, int draws, int hits) {
    double res = 0;
    for (int i = 0; i <= hits; i++) {
        res += hypergeometric_pmf(n, k, draws, i);
    }
    return std::min(res, 1.0);
}

double normal_pdf(double x, double mu = 0, double sigma = 1) {
    static const double INV_SQRT_2PI = 0.3989422804014327;
    double z = (x - mu) / sigma;
    return INV_SQRT_2PI / sigma * std::exp(-0.5 * z * z);
}

```

```
double normal_cdf(double x, double mu = 0, double sigma = 1) {
    static const double INV_SQRT_2 = 0.7071067811865476;
    return 0.5 * std::erfc(-(x - mu) * INV_SQRT_2 / sigma);
}

double exponential_pdf(double x, double lambda) {
    return x < 0 ? 0 : lambda * std::exp(-lambda * x);
}

double exponential_cdf(double x, double lambda) {
    return x < 0 ? 0 : -std::expm1(-lambda * x);
}

template<typename Cdf>
double quantile(Cdf cdf, double p, double lo, double hi, int iterations = 100) {
    for (int i = 0; i < iterations; i++) {
        double mid = lo + (hi - lo) / 2;
        if (cdf(mid) >= p) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    return hi;
}
```

## Example Usage

```
#include <cassert>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    // Three fair coin flips give two heads with probability 3/8.
    assert(EQ(binomial_pmf(3, 2, 0.5), 0.375));
    assert(EQ(binomial_cdf(3, 3, 0.5), 1));
    // Large counts stay finite where the binomial coefficient alone would not.
    assert(EQ(binomial_cdf(1000, 1000, 0.5), 1));
    assert(binomial_pmf(1000, 500, 0.5) > 0.02);

    // A fair die shows its first six on the third roll with probability 25/216.
    assert(EQ(geometric_pmf(3, 1.0 / 6), 25.0 / 216));
    assert(EQ(geometric_cdf(1, 0.25), 0.25));
    assert(EQ(negative_binomial_pmf(3, 2, 0.5), 0.25));

    assert(EQ(poisson_pmf(0, 2), exp(-2.0)));
    assert(EQ(poisson_cdf(1, 1), 2 * exp(-1.0)));

    // Two marked items among five drawn from ten, of which four are marked.
    assert(EQ(hypergeometric_pmf(10, 4, 5, 2), 10.0 / 21));
    assert(EQ(hypergeometric_cdf(10, 4, 5, 4), 1));

    assert(EQ(normal_cdf(0), 0.5));
    assert(fabs(normal_cdf(1.959963984540054) - 0.975) < 1e-9);
    assert(EQ(exponential_cdf(0, 3), 0));

    // Inverting a cumulative function recovers the value that produced it.
    assert(
        fabs(quantile([](double x) { return normal_cdf(x); }, 0.975, -10, 10) - 1.959963984540054) <
        1e-9
    );
    return 0;
}
```

### 6.7.2 Expected Value DP

A random process that moves between states accumulates an expected cost that satisfies one linear equation per state. If leaving state  $u$  costs  $c_u$  and sends the process to state  $v$  with probability  $p_{uv}$ , then linearity of expectation gives  $E[u] = c_u + \sum_v p_{uv} E[v]$ , where the expected cost of a state is the immediate cost plus the average of the expected costs of its successors. Counting steps is the special case  $c_u = 1$  for every non-absorbing state, and an absorbing state is one with no outgoing transitions, so that  $E = c$  there.

When the transition graph is acyclic, the equations can be evaluated instead of solved: process the states in reverse topological order and every successor on the right-hand side is already known. This is ordinary backward induction, and it is the reason expected-value problems are usually solved from the goal backwards rather than from the start forwards.

A self-loop is the one cycle this method still handles. If the process stays in  $u$  with probability  $q$ , then  $E[u] = c_u + qE[u] + \sum_{v \neq u} p_{uv} E[v]$ , and isolating  $E[u]$  gives  $E[u] = (c_u + \sum_{v \neq u} p_{uv} E[v]) / (1 - q)$ . Equivalently, the process retries until it leaves, so the expected number of attempts is the geometric mean  $1/(1 - q)$ . Self-loops appear whenever a move can be rejected and repeated, such as a die roll that would overshoot the goal or a draw that repeats a coupon already held.

- `expected_cost(trans, cost)` returns a vector `e` such that `e[u]` is the expected total cost accumulated from state `u` until the process reaches a state with no outgoing transitions. States are the indices of `trans`, each entry `trans[u]` lists the outgoing transitions of `u` as pairs of `(next_state, probability)`, and `cost[u]` is the cost charged on leaving `u`. Transition probabilities out of a state must sum to at most 1, with any missing probability treated as ending the process.

Aside from self-loops the transition graph must be acyclic, since a genuine cycle leaves the equations mutually dependent and they must then be solved as a linear system; two states that reach each other call for the absorbing Markov chains of section 6.7.3 instead.

**Time Complexity:**  $O(n + m)$  per call, where  $n$  is the number of states and  $m$  is the total number of transitions.

**Space Complexity:**

- $O(n + m)$  auxiliary.
- $O(n)$  for the returned vector.

```
#include <cassert>
#include <utility>
#include <vector>

std::vector<double> expected_cost(
    const std::vector<std::vector<std::pair<int, double>>> &trans, const std::vector<double> &cost
) {
    int n = static_cast<int>(trans.size());
    assert(static_cast<int>(cost.size()) == n);
    std::vector<int> indeg(n);
    for (int u = 0; u < n; u++) {
        for (auto [v, p] : trans[u]) {
            if (v != u) { // Self-loops are resolved algebraically, so they do not order the states.
                indeg[v]++;
            }
        }
    }
    std::vector<int> order, ready;
    order.reserve(n);
    for (int u = 0; u < n; u++) {
        if (indeg[u] == 0) {
            ready.push_back(u);
        }
    }
    while (!ready.empty()) {
        int u = ready.back();
        ready.pop_back();
        order.push_back(u);
    }
}
```



```

    for (auto [v, p] : trans[u]) {
        if (v != u && --indeg[v] == 0) {
            ready.push_back(v);
        }
    }
}
assert(static_cast<int>(order.size()) == n); // Cycles other than self-loops are unsupported.
std::vector<double> res(n);
for (int i = n - 1; i >= 0; i--) {
    int u = order[i];
    double stay = 0, total = cost[u];
    for (auto [v, p] : trans[u]) {
        if (v == u) {
            stay += p;
        } else {
            total += p * res[v];
        }
    }
    assert(stay < 1); // A state that never leaves itself has infinite expected cost.
    res[u] = total / (1 - stay);
}
return res;
}

```

## Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    // Roll a fair die to advance along squares [0, n], where a roll that would pass the last square
    // is rerolled. State n is absorbing, and every roll costs one step.
    const int n = 2;
    vector<vector<pair<int, double>>> board(n + 1);
    for (int i = 0; i < n; i++) {
        for (int face = 1; face <= 6; face++) {
            board[i].emplace_back(i + face <= n ? i + face : i, 1.0 / 6);
        }
    }
    vector<double> steps(n + 1, 1);
    steps[n] = 0;
    auto rolls = expected_cost(board, steps);
    assert(EQ(rolls[n], 0));
    assert(EQ(rolls[1], 6.0)); // Only one of six faces advances from square 1.
    assert(EQ(rolls[0], 6.0));
    // The same equation before the self-loop is eliminated: four of six faces reroll square 0.
    assert(EQ(rolls[0], 1 + rolls[1] / 6 + rolls[2] / 6 + 4.0 / 6 * rolls[0]));

    // The coupon collector, where state k is the number of distinct coupons already held. A draw
    // repeats a held coupon with probability k/m, which is a self-loop.
    const int m = 8;
    vector<vector<pair<int, double>>> coupons(m + 1);
    for (int k = 0; k < m; k++) {
        coupons[k].emplace_back(k, static_cast<double>(k) / m);
        coupons[k].emplace_back(k + 1, static_cast<double>(m - k) / m);
    }
    vector<double> draws(m + 1, 1);
    draws[m] = 0;
    auto collected = expected_cost(coupons, draws);
    double harmonic = 0;
    for (int i = 1; i <= m; i++) {
        harmonic += 1.0 / i;
    }
}

```

```

}
assert(EQ(collected[0], m * harmonic)); // The classic m*H_m coupon collector bound.
return 0;
}

```

### 6.7.3 Absorbing Markov Chains

An absorbing Markov chain splits its states into absorbing states, which are never left once entered, and transient states, which are left forever with probability 1. Ordering the transient states first splits the transition matrix into four blocks:  $Q$  holds the probabilities between transient states,  $R$  holds the probabilities from transient states into absorbing ones, and the two remaining blocks are a zero matrix and an identity matrix, since an absorbing state only ever moves to itself.

The expected number of visits to transient state  $j$  starting from transient state  $i$  is the entry  $(i, j)$  of  $I + Q + Q^2 + \dots$ , since the entry  $(i, j)$  of  $Q^k$  is the probability of being at  $j$  after exactly  $k$  steps. Because absorption is certain, the powers of  $Q$  tend to zero and the series converges to the fundamental matrix  $N = (I - Q)^{-1}$ . Every other quantity follows from  $N$ : summing a row counts the expected visits to all transient states, which is the expected number of steps before absorption, and multiplying by  $R$  takes one final step into an absorbing state.

- `fundamental_matrix(q)` returns  $N = (I - \mathbf{q})^{-1}$  for the transient-to-transient matrix  $\mathbf{q}$ , as a matrix whose entry in row  $i$  and column  $j$  is the expected number of visits to transient state  $j$  when starting at transient state  $i$ , counting the starting visit itself.
- `absorption_steps(q)` returns a vector  $t$  where  $t_i$  is the expected number of steps taken from transient state  $i$  until absorption.
- `absorption_probabilities(q, r)` returns a matrix  $B$  where  $B_{i,k}$  is the probability that a chain starting at transient state  $i$  is eventually absorbed at absorbing state  $k$ , given the transient-to-absorbing matrix  $\mathbf{r}$ .

Every transient state must reach an absorbing one, which is exactly what makes  $I - Q$  invertible; a state that cannot is not transient but part of a closed group to be modeled as one absorbing state, so the zero pivot is asserted rather than reported. Unlike the backward induction of section 6.7.2, cycles are no obstacle here, since the system is solved rather than evaluated.

#### Time Complexity:

- $O(n^3)$  per call to `fundamental_matrix()` and `absorption_steps()`, where  $n$  is the number of transient states.
- $O(n^3 + n^2 \cdot k)$  per call to `absorption_probabilities()`, where  $k$  is the number of absorbing states.

#### Space Complexity:

- $O(n^2)$  auxiliary for `fundamental_matrix()` and `absorption_steps()`, and  $O(n^2 + n \cdot k)$  for `absorption_probabilities()`.
- $O(n^2)$  for the returned matrix from `fundamental_matrix()`,  $O(n)$  for the returned vector from `absorption_steps()`, and  $O(n \cdot k)$  for the returned matrix from `absorption_probabilities()`.

```

#include <cassert>
#include <cmath>
#include <vector>

const double EPS = 1e-9;

std::vector<std::vector<double>> fundamental_matrix(const std::vector<std::vector<double>> &q) {
    int n = static_cast<int>(q.size());
    // Reduce the augmented matrix [I - Q | I] to [I | N] by Gauss-Jordan elimination.
    std::vector<std::vector<double>> a(n, std::vector<double>(2 * n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            a[i][j] = (i == j ? 1.0 : 0.0) - q[i][j];
        }
        a[i][n + i] = 1;
    }
}

```

```

}
for (int col = 0; col < n; col++) {
    int pivot = col;
    for (int i = col + 1; i < n; i++) {
        if (fabs(a[i][col]) > fabs(a[pivot][col])) {
            pivot = i;
        }
    }
    assert(fabs(a[pivot][col]) > EPS); // Some transient state never reaches an absorbing state.
    std::swap(a[col], a[pivot]);
    double scale = a[col][col];
    for (int j = col; j < 2 * n; j++) {
        a[col][j] /= scale;
    }
    for (int i = 0; i < n; i++) {
        if (i != col && a[i][col] != 0) {
            double factor = a[i][col];
            for (int j = col; j < 2 * n; j++) {
                a[i][j] -= factor * a[col][j];
            }
        }
    }
}
std::vector<std::vector<double>> res(n, std::vector<double>(n));
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        res[i][j] = a[i][n + j];
    }
}
return res;
}

std::vector<double> absorption_steps(const std::vector<std::vector<double>> &q) {
    auto visits = fundamental_matrix(q);
    int n = static_cast<int>(visits.size());
    std::vector<double> res(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res[i] += visits[i][j];
        }
    }
    return res;
}

std::vector<std::vector<double>> absorption_probabilities(
    const std::vector<std::vector<double>> &q, const std::vector<std::vector<double>> &r
) {
    assert(q.size() == r.size());
    auto visits = fundamental_matrix(q);
    int n = static_cast<int>(visits.size()), k = n == 0 ? 0 : static_cast<int>(r[0].size());
    std::vector<std::vector<double>> res(n, std::vector<double>(k));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int c = 0; c < k; c++) {
                res[i][c] += visits[i][j] * r[j][c];
            }
        }
    }
    return res;
}

```

## Example Usage

```

#include <cmath>
using namespace std;

```

```

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    // A gambler with i of 4 dollars bets a dollar on a fair coin until going broke or reaching 4.
    // Transient states 0, 1, and 2 stand for holding 1, 2, and 3 dollars, and the two absorbing
    // states are ruin and victory. States 0 and 1 can each reach the other, so the transition graph
    // is cyclic and backward induction alone does not apply.
    vector<vector<double>> q{{0, 0.5, 0}, {0.5, 0, 0.5}, {0, 0.5, 0}};
    vector<vector<double>> r{{0.5, 0}, {0, 0}, {0, 0.5}};

    auto visits = fundamental_matrix(q);
    assert(EQ(visits[0][0], 1.5) && EQ(visits[0][1], 1.0) && EQ(visits[0][2], 0.5));
    assert(EQ(visits[1][0], 1.0) && EQ(visits[1][1], 2.0) && EQ(visits[1][2], 1.0));
    assert(EQ(visits[2][0], 0.5) && EQ(visits[2][1], 1.0) && EQ(visits[2][2], 1.5));

    // Starting with i dollars, the game lasts i*(4 - i) rounds on average.
    auto steps = absorption_steps(q);
    assert(EQ(steps[0], 3.0) && EQ(steps[1], 4.0) && EQ(steps[2], 3.0));

    // A fair game is won with probability i/4.
    auto odds = absorption_probabilities(q, r);
    assert(EQ(odds[0][1], 0.25) && EQ(odds[1][1], 0.5) && EQ(odds[2][1], 0.75));
    for (int i = 0; i < 3; i++) {
        assert(EQ(odds[i][0] + odds[i][1], 1.0)); // Absorption is certain.
    }
    return 0;
}

```

## 6.7.4 Stationary Distribution

A Markov chain with no absorbing states never settles on a single state, but the fraction of time it spends in each state does settle. That limit is the stationary distribution: the row vector  $\pi$  with  $\pi P = \pi$  and  $\sum_i \pi_i = 1$ , which is the one distribution left unchanged by a step of the chain. It exists and is unique whenever the chain is irreducible, meaning every state can reach every other, and it is the limit of the state distribution from any starting point once the chain is also aperiodic.

The defining equations are one short of determining  $\pi$ . Each column of  $\pi P = \pi$  says that the probability flowing into a state equals the probability flowing out, and those  $n$  equations sum to the trivial identity  $1 = 1$ , leaving the system rank  $n - 1$  and any scalar multiple of a solution also a solution. Substituting the normalization  $\sum_i \pi_i = 1$  for one of them pins down the scale and makes the system uniquely solvable by ordinary elimination.

- `stationary_distribution(p, eps = 1e-10)` returns the stationary distribution of the  $n$  by  $n$  row-stochastic transition matrix `p`, where `p[i][j]` is the probability of stepping from state `i` to state `j`. It returns `std::nullopt` when the system is singular to within `eps`, which happens exactly when the chain is reducible and therefore has no unique stationary distribution.

Two quantities follow at once: the expected number of steps to return to state  $i$  is  $1/\pi_i$ , and on an undirected graph walked by a uniformly random incident edge,  $\pi_v$  is proportional to the degree of  $v$ , which is why such a walk is uniform in the limit on a regular graph. Where the absorbing chains of section 6.7.3 ask what happens before the process stops, this asks what happens when it never does.

**Time Complexity:**  $O(n^3)$  per call, where  $n$  is the number of states.

**Space Complexity:**  $O(n^2)$  auxiliary and  $O(n)$  for the returned distribution.

```

#include <cassert>
#include <cmath>
#include <optional>
#include <vector>

```

```

std::optional<std::vector<double>> stationary_distribution(
    const std::vector<std::vector<double>> &p, double eps = 1e-10
) {
    int n = static_cast<int>(p.size());
    if (n == 0) {
        return std::vector<double>{};
    }
    // Build the transpose of P - I, then replace its last row with the normalization equation.
    std::vector<std::vector<double>> a(n, std::vector<double>(n + 1));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            a[i][j] = p[j][i] - (i == j ? 1.0 : 0.0);
        }
    }
    for (int j = 0; j < n; j++) {
        a[n - 1][j] = 1;
    }
    a[n - 1][n] = 1;
    for (int col = 0; col < n; col++) {
        int pivot = col;
        for (int i = col + 1; i < n; i++) {
            if (fabs(a[i][col]) > fabs(a[pivot][col])) {
                pivot = i;
            }
        }
        if (fabs(a[pivot][col]) < eps) {
            return std::nullopt; // Reducible, so the stationary distribution is not unique.
        }
        std::swap(a[col], a[pivot]);
        double scale = a[col][col]; // Saved, since the loop below overwrites it on its first step.
        for (int j = col; j <= n; j++) {
            a[col][j] /= scale;
        }
        for (int i = 0; i < n; i++) {
            if (i != col && a[i][col] != 0) {
                double factor = a[i][col];
                for (int j = col; j <= n; j++) {
                    a[i][j] -= factor * a[col][j];
                }
            }
        }
    }
    std::vector<double> res(n);
    for (int i = 0; i < n; i++) {
        res[i] = a[i][n];
    }
    return res;
}

```

### Example Usage

```

#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    // A two-state chain that leaves state 0 with probability 1/4 and state 1 with probability 1/2,
    // so it rests in state 0 twice as often.
    auto two = stationary_distribution({{0.75, 0.25}, {0.5, 0.5}});
    assert(two.has_value());
    assert(EQ((*two)[0], 2.0 / 3) && EQ((*two)[1], 1.0 / 3));

    // A random walk on a path of three vertices, where the middle vertex has twice the degree and so

```

```

// twice the stationary probability. Its expected return time is the reciprocal, 2.
auto path = stationary_distribution({{0, 1, 0}, {0.5, 0, 0.5}, {0, 1, 0}});
assert(path.has_value());
assert(EQ((*path)[0], 0.25) && EQ((*path)[1], 0.5) && EQ((*path)[2], 0.25));
assert(EQ(1 / (*path)[1], 2.0));

// A doubly stochastic chain is uniform in the limit no matter the transition probabilities.
auto cycle = stationary_distribution({{0, 0.3, 0.7}, {0.7, 0, 0.3}, {0.3, 0.7, 0}});
assert(cycle.has_value());
for (int i = 0; i < 3; i++) {
    assert(EQ((*cycle)[i], 1.0 / 3));
}

// Two states that never reach each other admit no unique distribution.
assert(!stationary_distribution({{1, 0}, {0, 1}}).has_value());
return 0;
}

```

## 6.7.5 Statistical Learning

The following routines fit a model to observed data and use it to describe or predict unobserved data. Each takes its input as a `Points` matrix whose rows are samples and whose columns are features, so a row is one observation of the same measurements taken in the same order. Supervised routines additionally take a `labels` vector, parallel to the rows, holding each sample's class as an integer in `[0, classes)`.

Features measured on different scales distort every distance here, so call `standardize()` before `kmeans()`, `knn_classify()`, or `logistic_regression()` and apply the returned pair to later queries. Naive Bayes is exempt, since it fits a separate variance per feature. None of these choose `k` or `12` for you: hold out part of the data and keep the value predicting it best.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <numeric>
#include <random>
#include <utility>
#include <vector>

using Points = std::vector<std::vector<double>>>;

double sqdist(const std::vector<double> &a, const std::vector<double> &b) {
    double res = 0;
    for (size_t i = 0; i < a.size(); i++) {
        res += (a[i] - b[i]) * (a[i] - b[i]);
    }
    return res;
}

```

Standardizing subtracts each column's mean and divides by its standard deviation, which is what makes one Euclidean distance meaningful across features recorded in different units. The population standard deviation is used, dividing by  $n$  rather than  $n - 1$ , since these columns are the data being fit and not a sample standing in for a larger population. A column that never varies would give a zero divisor, so it reports 1 instead and standardizes to all zeros, contributing nothing to any distance, which is exactly what a constant feature deserves.

- `column_mean(points)` and `column_stddev(points, mean)` return the per-column mean and population standard deviation, the latter reporting 1 for a column that never varies.
- `standardize(points)` rescales every column in place to zero mean and unit variance and returns the `(mean, stddev)` it used, while `standardize(x, mean, stddev)` applies a returned pair to one later sample.

**Time Complexity:**  $O(n \cdot d)$  per call over  $n$  rows of  $d$  columns, and  $O(d)$  for one sample.

**Space Complexity:**  $O(d)$  auxiliary.

```
std::vector<double> column_mean(const Points &points) {
    int n = static_cast<int>(points.size()), d = static_cast<int>(points[0].size());
    std::vector<double> res(d);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < d; j++) {
            res[j] += points[i][j];
        }
    }
    for (int j = 0; j < d; j++) {
        res[j] /= n;
    }
    return res;
}

std::vector<double> column_stddev(const Points &points, const std::vector<double> &mean) {
    int n = static_cast<int>(points.size()), d = static_cast<int>(points[0].size());
    std::vector<double> res(d);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < d; j++) {
            res[j] += (points[i][j] - mean[j]) * (points[i][j] - mean[j]);
        }
    }
    for (int j = 0; j < d; j++) {
        res[j] = std::sqrt(res[j] / n);
        if (res[j] < 1e-12) {
            res[j] = 1;
        }
    }
    return res;
}

void standardize(
    std::vector<double> &x, const std::vector<double> &mean, const std::vector<double> &stddev
) {
    for (size_t j = 0; j < x.size(); j++) {
        x[j] = (x[j] - mean[j]) / stddev[j];
    }
}

std::pair<std::vector<double>, std::vector<double>> standardize(Points &points) {
    std::vector<double> mean = column_mean(points), stddev = column_stddev(points, mean);
    for (std::vector<double> &row : points) {
        standardize(row, mean, stddev);
    }
    return std::make_pair(mean, stddev);
}
```

$k$ -means partitions the rows into  $k$  clusters so that each row lies nearer to its own cluster's mean than to any other. Lloyd's algorithm alternates the two halves of that condition: assign every row to the nearest centroid, then move every centroid to the mean of the rows assigned to it. Each half can only lower the total squared distance, so the process converges, but only to a local optimum that the starting centroids decide.

Choosing those starting centroids is called seeding, and  $k$ -means++ is the standard rule: take the first uniformly at random, then draw each remaining one with probability proportional to its squared distance from the nearest already chosen. That spreads them out instead of letting several land in one dense region, bounding the expected cost at  $O(\log k)$  times optimal, which uniform choice cannot promise. The loop stops once an assignment pass changes nothing, and a cluster that loses all its rows keeps its previous centroid.

- `kmeans(points, k, rng, iterations = 100)` returns `(centroids, assignment)`, partitioning the rows into `k` clusters, where `assignment[i]` is the cluster of row `i` and `centroids[c]` is the mean of cluster `c`. The number of rows must be at least `k`, which must be positive.

**Time Complexity:**  $O(i \cdot n \cdot k \cdot d)$  per call for `iterations`  $i$ ,  $n$  rows, and  $d$  columns.

**Space Complexity:**  $O(n + k \cdot d)$  auxiliary.

```
std::pair<Points, std::vector<int>> kmeans(
    const Points &points, int k, std::mt19937 &rng, int iterations = 100
) {
    int n = static_cast<int>(points.size());
    Points centroids;
    centroids.push_back(points[std::uniform_int_distribution<int>(0, n - 1)(rng)]);
    std::vector<double> best(n);
    for (int i = 0; i < n; i++) {
        best[i] = sqdist(points[i], centroids[0]);
    }
    while (static_cast<int>(centroids.size()) < k) {
        std::discrete_distribution<int> pick(best.begin(), best.end());
        centroids.push_back(points[pick(rng)]);
        for (int i = 0; i < n; i++) {
            best[i] = std::min(best[i], sqdist(points[i], centroids.back()));
        }
    }
    std::vector<int> assignment(n, -1);
    for (int it = 0; it < iterations; it++) {
        bool changed = false;
        for (int i = 0; i < n; i++) {
            int nearest = 0;
            double nearest_dist = sqdist(points[i], centroids[0]);
            for (int c = 1; c < k; c++) {
                double dist = sqdist(points[i], centroids[c]);
                if (dist < nearest_dist) {
                    nearest = c;
                    nearest_dist = dist;
                }
            }
            if (assignment[i] != nearest) {
                assignment[i] = nearest;
                changed = true;
            }
        }
        if (!changed) {
            break;
        }
        Points sums(k, std::vector<double>(points[0].size()));
        std::vector<int> counts(k);
        for (int i = 0; i < n; i++) {
            for (size_t j = 0; j < points[i].size(); j++) {
                sums[assignment[i]][j] += points[i][j];
            }
            counts[assignment[i]]++;
        }
        for (int c = 0; c < k; c++) {
            if (counts[c] > 0) {
                for (size_t j = 0; j < sums[c].size(); j++) {
                    centroids[c][j] = sums[c][j] / counts[c];
                }
            }
        }
    }
    return std::make_pair(centroids, assignment);
}
```

$k$ -nearest neighbors classifies a query by letting the  $k$  nearest training rows vote, which fits no model at all: the training data is the model, and all the work happens at query time. Small  $k$  follows the data closely and is swayed by noise, while large  $k$  smooths across genuine boundaries, so  $k$  trades variance against bias. Averaging the neighbors' values rather than counting their votes turns the same routine into regression.



Selecting the  $k$  nearest with `std::nth_element` costs linear time rather than the  $O(n \log n)$  of a full sort, since the neighbors' relative order is irrelevant to the vote. Scanning every row makes each query linear in the data, which the k-d tree of section 2.6.5 improves to logarithmic on low-dimensional data; above roughly ten features the two converge and the scan below is preferable.

- `knn_classify(points, labels, query, k)` returns the majority label among the `k` rows nearest to `query`, breaking a tie toward the smaller label. `k` must lie in  $[1, n]$  for `n` rows.

**Time Complexity:**  $O(n \cdot d)$  per call, for  $n$  rows of  $d$  columns.

**Space Complexity:**  $O(n)$  auxiliary.

```
int knn_classify(
    const Points &points, const std::vector<int> &labels, const std::vector<double> &query, int k
) {
    int n = static_cast<int>(points.size());
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::vector<double> dist(n);
    for (int i = 0; i < n; i++) {
        dist[i] = sqdist(points[i], query);
    }
    std::nth_element(order.begin(), order.begin() + (k - 1), order.end(), [&](int a, int b) {
        return dist[a] < dist[b];
    });
    std::vector<int> votes(*std::max_element(labels.begin(), labels.end()) + 1);
    for (int i = 0; i < k; i++) {
        votes[labels[order[i]]]++;
    }
    return static_cast<int>(std::max_element(votes.begin(), votes.end()) - votes.begin());
}
```

Naive Bayes picks the class maximizing  $P(c) \prod_j P(x_j | c)$ , which is Bayes' rule with the denominator dropped because it does not depend on the class. The independence it assumes among features given the class is almost never true, and yet the classifier is accurate anyway: the products it computes are poorly calibrated as probabilities, but the class that maximizes them is usually still the right one, since the errors tend to move every class the same way.

The Gaussian variant below models each feature within each class by a normal distribution, so fitting is one pass accumulating a mean and variance per (class, feature) pair. Scores accumulate as logarithms, both because a product of hundreds of densities underflows and because the logarithm of a normal density is a plain quadratic. A feature that is constant within a class would give a zero variance and an infinite score, so variances are floored at a small fraction of the overall spread.

- `NaiveBayes(points, labels, classes)` fits a Gaussian naive Bayes classifier, `predict(x)` returns its most probable class for a sample `x`, and `log_likelihood(x, c)` returns the unnormalized log probability that `x` belongs to class `c`. A class carried by no row is given zero prior, so it scores  $-\infty$  and is never predicted.

**Time Complexity:**

- $O(n \cdot d)$  per construction.
- $O(c \cdot d)$  per call to `predict()` and `log_likelihood()` for  $n$  rows of  $d$  columns and  $c$  classes.

**Space Complexity:**  $O(c \cdot d)$  for storage.

```
class NaiveBayes {
    std::vector<double> log_prior;
    Points mean, var;

public:
    NaiveBayes(const Points &points, const std::vector<int> &labels, int classes) {
        int n = static_cast<int>(points.size()), d = static_cast<int>(points[0].size());
        std::vector<int> counts(classes);
        mean.assign(classes, std::vector<double>(d));
```

```

var.assign(classes, std::vector<double>(d));
log_prior.assign(classes, 0);
for (int i = 0; i < n; i++) {
    counts[labels[i]]++;
    for (int j = 0; j < d; j++) {
        mean[labels[i]][j] += points[i][j];
    }
}
for (int c = 0; c < classes; c++) {
    if (counts[c] == 0) { // A class with no rows has zero prior, so it is never predicted.
        log_prior[c] = -std::numeric_limits<double>::infinity();
        continue;
    }
    log_prior[c] = std::log(static_cast<double>(counts[c]) / n);
    for (int j = 0; j < d; j++) {
        mean[c][j] /= counts[c];
    }
}
double spread = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < d; j++) {
        double diff = points[i][j] - mean[labels[i]][j];
        var[labels[i]][j] += diff * diff;
        spread += diff * diff;
    }
}
double floor_var = 1e-9 * spread / n + 1e-12;
for (int c = 0; c < classes; c++) {
    for (int j = 0; j < d; j++) {
        var[c][j] = counts[c] == 0 ? floor_var : std::max(var[c][j] / counts[c], floor_var);
    }
}

double log_likelihood(const std::vector<double> &x, int c) const {
    static const double LOG_2PI = 1.8378770664093453;
    double res = log_prior[c];
    for (size_t j = 0; j < x.size(); j++) {
        double diff = x[j] - mean[c][j];
        res -= 0.5 * (LOG_2PI + std::log(var[c][j]) + diff * diff / var[c][j]);
    }
    return res;
}

int predict(const std::vector<double> &x) const {
    int best = 0;
    double best_score = log_likelihood(x, 0);
    for (size_t c = 1; c < log_prior.size(); c++) {
        double score = log_likelihood(x, static_cast<int>(c));
        if (score > best_score) {
            best = static_cast<int>(c);
            best_score = score;
        }
    }
    return best;
}
};

```

Logistic regression models the probability of class 1 as  $\sigma(w \cdot x)$  for the logistic function  $\sigma(z) = 1/(1 + e^{-z})$ , which maps any real score into  $(0, 1)$ . Fitting minimizes the average negative log-likelihood, a convex function whose gradient is  $X^\top(\sigma(Xw) - y)/n$ : each row pushes the weights by its own features, scaled by how wrong the current prediction is on it. Convexity makes every local minimum global, so plain gradient descent suffices.

Unlike the least squares of section 6.5.5 there is no closed form, and on perfectly separable data the weights diverge as every predicted probability is driven to 0 or 1. The `l2` penalty adds  $\lambda\|w\|^2$  to keep them finite,

excluding the intercept so that shifting all labels does not distort it. Section 5.2.4 supplies the adaptive steps worth substituting when convergence is slow.

- `logistic_predict(w, x)` returns the probability that sample `x` belongs to class 1 under a weight vector `w` whose index 0 is the intercept.
- `logistic_regression(points, labels, ...)` returns such a weight vector, fit by gradient descent over labels that are 0 or 1. Optional parameters default to `rate = 0.1`, `iterations = 1000`, and `l2 = 0`.

**Time Complexity:**  $O(i \cdot n \cdot d)$  per call for `iterations`  $i$ ,  $n$  rows, and  $d$  columns.

**Space Complexity:**  $O(d)$  auxiliary.

```
double sigmoid(double z) {
    return z >= 0 ? 1 / (1 + std::exp(-z)) : std::exp(z) / (1 + std::exp(z));
}

double logistic_predict(const std::vector<double> &w, const std::vector<double> &x) {
    double z = w[0];
    for (size_t j = 0; j < x.size(); j++) {
        z += w[j + 1] * x[j];
    }
    return sigmoid(z);
}

std::vector<double> logistic_regression(
    const Points &points, const std::vector<int> &labels, double rate = 0.1, int iterations = 1000,
    double l2 = 0
) {
    int n = static_cast<int>(points.size()), d = static_cast<int>(points[0].size());
    std::vector<double> w(d + 1), grad(d + 1);
    for (int it = 0; it < iterations; it++) {
        std::fill(grad.begin(), grad.end(), 0.0);
        for (int i = 0; i < n; i++) {
            double error = logistic_predict(w, points[i]) - labels[i];
            grad[0] += error;
            for (int j = 0; j < d; j++) {
                grad[j + 1] += error * points[i][j];
            }
        }
        w[0] -= rate * grad[0] / n;
        for (int j = 1; j <= d; j++) {
            w[j] -= rate * (grad[j] / n + l2 * w[j]);
        }
    }
    return w;
}
```

## Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Two well-separated blobs: the first three rows lie near (0, 0) and the last three near
    // (10, 10).
    Points points{{0, 0}, {1, 0}, {0, 1}, {10, 10}, {11, 10}, {10, 11}};
    vector<int> labels{0, 0, 0, 1, 1, 1};

    // Standardizing puts both columns on one scale; the same pair transforms later queries.
    Points scaled(points);
    pair<vector<double>, vector<double>> shift = standardize(scaled);
    vector<double> query{9, 9};
    standardize(query, shift.first, shift.second);
    assert(knn_classify(scaled, labels, query, 3) == 1);
}
```

```

mt19937 rng(1234567);
pair<Points, vector<int>> clustered = kmeans(points, 2, rng);
assert(clustered.first.size() == 2);
assert(clustered.second[0] == clustered.second[1] && clustered.second[1] == clustered.second[2]);
assert(clustered.second[3] == clustered.second[4] && clustered.second[4] == clustered.second[5]);
assert(clustered.second[0] != clustered.second[3]);

assert(knn_classify(points, labels, {0.5, 0.5}, 3) == 0);
assert(knn_classify(points, labels, {9, 9}, 3) == 1);

NaiveBayes nb(points, labels, 2);
assert(nb.predict({0.5, 0.5}) == 0);
assert(nb.predict({9, 9}) == 1);
// Declaring a third class that no row carries leaves it with zero prior, never predicted.
NaiveBayes sparse(points, labels, 3);
assert(isinf(sparse.log_likelihood({0.5, 0.5}, 2)));
assert(sparse.predict({0.5, 0.5}) == 0 && sparse.predict({9, 9}) == 1);

vector<double> w = logistic_regression(points, labels, 0.5, 2000);
assert(logistic_predict(w, {0.5, 0.5}) < 0.5);
assert(logistic_predict(w, {9, 9}) > 0.5);
return 0;
}

```

## 6.8 Game Theory

---

### 6.8.1 Nim

Solves normal-play nim and related impartial games where each position is the disjoint sum of independent piles. In normal play, the player who makes the last move wins. A nim position is losing exactly when the XOR of all pile sizes is 0.

This is the base case of the Sprague-Grundy theorem: every finite impartial game position is equivalent to a Nim pile of size equal to its Grundy number, and sums of games combine by XOR.

- `nim_sum(piles)` returns the XOR of all pile sizes.
- `first_player_wins(piles)` returns whether the player to move has a winning strategy.
- `winning_move(piles)` returns a move (`pile`, `new_size`) that moves to XOR 0, or  $(-1, -1)$  if the position is losing.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the number of piles.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <utility>
#include <vector>

int nim_sum(const std::vector<int> &piles) {
    int x = 0;
    for (int p : piles) {
        x ^= p;
    }
    return x;
}

bool first_player_wins(const std::vector<int> &piles) {
    return nim_sum(piles) != 0;
}

std::pair<int, int> winning_move(const std::vector<int> &piles) {
    int x = nim_sum(piles);

```

```

if (x == 0) {
    return {-1, -1};
}
for (int i = 0; i < static_cast<int>(piles.size()); i++) {
    if (int target = piles[i] ^ x; target < piles[i]) {
        return {i, target};
    }
}
return {-1, -1};
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> piles{3, 4, 5};
    // 3 ^ 4 ^ 5 = 2, so the position is winning.
    assert(nim_sum(piles) == 2);
    assert(first_player_wins(piles));
    auto [pile_idx, new_size] = winning_move(piles);
    piles[pile_idx] = new_size;
    // A winning move always leaves XOR 0 for the opponent.
    assert(nim_sum(piles) == 0);

    vector<int> losing{1, 2, 3};
    assert(!first_player_wins(losing));
    assert((winning_move(losing) == pair<int, int>{-1, -1}));
    return 0;
}

```

## 6.8.2 Nim Product

Nimbers are the values of impartial games, added by XOR. Under the additional operation of nim multiplication they form a field: the nonnegative integers below  $2^{2^k}$  are closed under both operations, with XOR as addition and nim product as multiplication. Writing  $\otimes$  for the nim product and  $\oplus$  for XOR, nim multiplication is commutative, associative, and distributive over  $\oplus$ , and is defined recursively by  $a \otimes b = \text{mex}\{(a' \otimes b) \oplus (a \otimes b') \oplus (a' \otimes b') : 0 \leq a' < a, 0 \leq b' < b\}$ . Its main use is multiplying the Grundy values of independent “coin-turning” games whose move sets combine as a product, such as two-dimensional turning games built from one-dimensional ones.

Rather than evaluate the recursive definition directly, this implementation precomputes the nim product of every pair of single bits  $2^i \otimes 2^j$  for  $i, j < 64$ . Each such product follows from the Fermat-like rule  $2^{2^k} \otimes 2^{2^k} = 2^{2^k} \oplus 2^{2^k-1}$  and the fact that distinct Fermat powers multiply like ordinary powers of two. A full product then XORs together the bit-pair products selected by the set bits of the operands. Because every 64-bit value lies in the field of size  $2^{64}$ , the operation is total on `uint64_t` and the nonzero values have multiplicative inverses.

- `nim_product(x, y)` returns the nim product of `x` and `y`.
- `nim_pow(b, e)` returns `b` raised to the `e`-th power under nim multiplication.
- `nim_inverse(b)` returns the multiplicative inverse of a nonzero `b`, so that `nim_product(b, nim_inverse(b)) == 1`.

#### Time Complexity:

- $O(1)$  table construction on first use (a fixed 64 by 64 table).
- $O(64^2)$  per call to `nim_product()`;  $O(64^2 \log e)$  per call to `nim_pow()` and `nim_inverse()`.

**Space Complexity:**  $O(1)$  for a fixed 64 by 64 table of 64-bit products.

```

#include <cassert>

```

```

#include <cstdint>

uint64_t bit_prod[64][64];

// Build nim products for pairs of single-bit values. Each entry depends only on entries with a
// smaller first index, so a single ascending pass fills the table.
const bool nim_product_init = [] {
    for (int i = 0; i < 64; i++) {
        for (int j = 0; j < 64; j++) {
            if ((i & j) == 0) {
                bit_prod[i][j] = 1ULL << (i | j); // Distinct Fermat powers multiply like 2^(i|j).
            } else {
                int a = (i & j) & ~(i & j); // Lowest Fermat power shared by both exponents.
                bit_prod[i][j] = bit_prod[i ^ a][j] ^ bit_prod[(i ^ a) | (a - 1)][(j ^ a) | (i & (a - 1))];
            }
        }
    }
    return true;
}();

uint64_t nim_product(uint64_t x, uint64_t y) {
    uint64_t res = 0;
    for (int i = 0; i < 64 && (x >> i) != 0; i++) {
        if ((x >> i) & 1) {
            for (int j = 0; j < 64 && (y >> j) != 0; j++) {
                if ((y >> j) & 1) {
                    res ^= bit_prod[i][j];
                }
            }
        }
    }
    return res;
}

uint64_t nim_pow(uint64_t b, uint64_t e) {
    uint64_t res = 1;
    for (; e > 0; e >>= 1) {
        if (e & 1) {
            res = nim_product(res, b);
        }
        b = nim_product(b, b);
    }
    return res;
}

uint64_t nim_inverse(uint64_t b) {
    assert(b != 0);
    // The multiplicative group of the size-2^64 field has order 2^64 - 1, so b^(2^64 - 2) = b^{-1}.
    return nim_pow(b, static_cast<uint64_t>(-2));
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Smallest nontrivial products: {0,1,2,3} form the field GF(4) under XOR and nim product.
    assert(nim_product(2, 2) == 3);
    assert(nim_product(2, 3) == 1);
    assert(nim_product(3, 3) == 2);
    assert(nim_product(1, 12345) == 12345); // 1 is the multiplicative identity.
    assert(nim_product(0, 12345) == 0);

    // Field axioms on a range of values: commutativity, distributivity over XOR, and inverses.
    for (uint64_t a = 0; a < 64; a++) {

```

```

for (uint64_t b = 0; b < 64; b++) {
    assert(nim_product(a, b) == nim_product(b, a));
    for (uint64_t c = 0; c < 8; c++) {
        assert(nim_product(a, b ^ c) == (nim_product(a, b) ^ nim_product(a, c)));
    }
}
if (a != 0) {
    assert(nim_product(a, nim_inverse(a)) == 1);
}
}
assert(nim_product(0x1234567890abcdefULL, nim_inverse(0x1234567890abcdefULL)) == 1);
return 0;
}

```

### 6.8.3 Sprague-Grundy Theorem

Computes Grundy numbers for finite impartial games. The Grundy number of a game position is the minimum excluded nonnegative integer (MEX) of the Grundy numbers reachable in one move. A position is losing exactly when its Grundy number is 0.

For a sum of independent impartial games, the combined Grundy number is the XOR of the component Grundy numbers. This is the Sprague-Grundy theorem.

- `subtraction_game_grundy(max_stones, moves)` computes Grundy numbers for a game where a move subtracts one positive value in `moves` from the pile. It returns a vector of length `max_stones + 1`, whose element `grundy[stones]` is the Grundy number for a pile of that size. Nonpositive moves are ignored.
- `sum_grundy(grundies)` returns the XOR of component Grundy numbers.

#### Time Complexity:

- $O(n \cdot m)$  per call to `subtraction_game_grundy(max_stones, moves)`, where  $n$  is `max_stones` and  $m$  is the number of allowed moves.
- $O(k)$  per call to `sum_grundy()`, where  $k$  is the input length.

#### Space Complexity:

- $O(n)$  output and  $O(m)$  auxiliary for `subtraction_game_grundy(max_stones, moves)`.
- $O(1)$  auxiliary for `sum_grundy(grundies)`.

```

#include <vector>

int mex(const std::vector<int> &values) {
    std::vector<char> seen(values.size() + 1);
    for (int v : values) {
        if (0 <= v && v < static_cast<int>(seen.size())) {
            seen[v] = true;
        }
    }
    for (int i = 0; i < static_cast<int>(seen.size()); i++) {
        if (!seen[i]) {
            return i;
        }
    }
    return static_cast<int>(seen.size());
}

std::vector<int> subtraction_game_grundy(int max_stones, const std::vector<int> &moves) {
    std::vector<int> grundy(max_stones + 1);
    for (int stones = 1; stones <= max_stones; stones++) {
        std::vector<int> reachable;
        for (int m : moves) {
            if (0 < m && m <= stones) {
                reachable.push_back(grundy[stones - m]);
            }
        }
    }
}

```

```

    }
}
grundy[stones] = mex(reachable);
}
return grundy;
}

int sum_grundy(const std::vector<int> &grundies) {
    int x = 0;
    for (int g : grundies) {
        x ^= g;
    }
    return x;
}

```

### Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> moves{1, 3, 4};
    auto g = subtraction_game_grundy(10, moves);
    // With moves 1, 3, 4, pile size 2 is losing and pile size 4 has mex {0, 1} = 2.
    assert((g == vector<int>{0, 1, 0, 1, 2, 3, 2, 0, 1, 0, 1}));

    vector<int> heaps{g[5], g[7], g[10]};
    // Independent games combine by xor of their Grundy numbers.
    assert(sum_grundy(heaps) == (g[5] ^ g[7] ^ g[10]));
    return 0;
}

```

## 6.8.4 Grundy Numbers on DAG

Computes Grundy numbers for impartial games whose positions form a directed acyclic graph. Each node is a game position, and each outgoing edge is a legal move. Terminal nodes have Grundy number 0; all other nodes take the MEX of their successors' Grundy numbers.

The implementation uses memoized DFS. It assumes the graph is acyclic; if cycles are present, the usual finite impartial-game Grundy recurrence is not directly valid without additional analysis.

- `grundy_on_dag(g)` returns the Grundy number of every node in graph `g`, where `g[u]` contains all positions reachable from position `u` in one move.

**Time Complexity:**  $O(n + m)$  per call, where  $n$  is the number of nodes and  $m$  is the number of edges.

**Space Complexity:**  $O(d)$  auxiliary stack space and  $O(n + m)$  auxiliary heap space, where  $d$  is DFS depth.

```

#include <vector>

std::vector<int> grundy_on_dag(const std::vector<std::vector<int>> &g) {
    std::vector<int> memo(g.size(), -1);
    auto dfs = [&](auto &&dfs, int u) {
        if (memo[u] != -1) {
            return;
        }
        std::vector<char> seen(g[u].size() + 1);
        for (int v : g[u]) {
            dfs(dfs, v);
            int x = memo[v];
            if (x < static_cast<int>(seen.size())) {
                seen[x] = true;
            }
        }
        int mex = 0;
        while (seen[mex]) mex++;
        memo[u] = mex;
    };
    dfs(dfs, 0);
    return memo;
}

```



```

    }
}
int res = 0;
while (res < static_cast<int>(seen.size()) && seen[res]) {
    res++;
}
memo[u] = res;
};
for (int u = 0; u < static_cast<int>(g.size()); u++) {
    dfs(dfs, u);
}
return memo;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>> g{{1, 2}, {3}, {3, 4}, {}, {}};
    // Terminal nodes have Grundy 0, their predecessors have mex {0} = 1, and node 0 has mex {1} = 0.
    assert((grundy_on_dag(g) == vector<int>{0, 1, 1, 0, 0}));
    return 0;
}

```

### 6.8.5 Minimax and Alpha-Beta Pruning

Searches a finite two-player, zero-sum, perfect-information game tree using minimax and alpha-beta pruning.

Minimax assumes both players play optimally: the maximizing player chooses the child with largest value, and the minimizing player chooses the child with smallest value.

Alpha-beta pruning computes the same value as minimax, but avoids exploring branches that cannot affect the final answer. It is most effective when good moves are searched first.

The example game is “take 1 or 2 stones”: starting with `stones` stones, players alternate taking 1 or 2 stones, and the player who takes the last stone wins. The evaluation returns 1 when the maximizing player wins and  $-1$  when the minimizing player wins.

- `minimax(stones, maximizing)` returns the exact game-tree value, where `maximizing` indicates whose turn it is.
- `alpha_beta(stones, maximizing)` returns the same value using alpha-beta pruning.
- `best_take(stones)` returns an optimal first move using alpha-beta pruning, either 1 or 2, for positive `stones`.

**Time Complexity:**  $O(b^d)$  per call in the worst case, where  $b$  is branching factor and  $d$  is search depth.

Alpha-beta can achieve  $O(b^{d/2})$  with optimal move ordering.

**Space Complexity:**  $O(d)$  auxiliary recursion stack space.

```

#include <algorithm>

int minimax(int stones, bool maximizing) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
    int best = maximizing ? -2 : 2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        int child = minimax(stones - take, !maximizing);
        best = maximizing ? std::max(best, child) : std::min(best, child);
    }
    return best;
}

```

```

int alpha_beta(int stones, bool maximizing, int alpha = -2, int beta = 2) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
    int value = maximizing ? -2 : 2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        int child = alpha_beta(stones - take, !maximizing, alpha, beta);
        if (maximizing) {
            value = std::max(value, child);
            alpha = std::max(alpha, value);
        } else {
            value = std::min(value, child);
            beta = std::min(beta, value);
        }
        if (alpha >= beta) {
            break;
        }
    }
    return value;
}

int best_take(int stones) {
    int best_move = 1, best_value = -2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        int value = alpha_beta(stones - take, false);
        if (value > best_value) {
            best_value = value;
            best_move = take;
        }
    }
    return best_move;
}

```

## Example Usage

```

#include <cassert>

int main() {
    assert(minimax(1, true) == 1);
    assert(minimax(3, true) == -1);
    for (int stones = 1; stones <= 10; stones++) {
        assert(minimax(stones, true) == alpha_beta(stones, true));
    }
    assert(best_take(4) == 1); // Move to losing state with 3 stones.
    assert(best_take(5) == 2); // Move to losing state with 3 stones.
    return 0;
}

```

# Chapter 7

## Geometry

### 7.1 Geometry Primitives

---

#### 7.1.1 Point

A 2D point class templated on the coordinate type `T`. Algebraic operations preserve `T`, while metric operations use `fp_t`, which is `T` for floating-point coordinates and `double` otherwise.

Non-floating-point coordinate types such as `Modular` or `Rational` compose for exact operations (arithmetic, `dot`, `cross`, `sqnorm`, comparisons, and `EQ`) using the coordinate type's own operators. Their metric operations convert coordinates explicitly to `double`; for `Modular`, this uses the stored representative and is not finite-field geometry.

- `TPoint<T>()` constructs the origin, while `TPoint<T>(x, y)` constructs point  $(x, y)$ . A pair of values can also be converted explicitly to a point.
- Operators `+`, `-`, `*`, `/`, and their compound forms act element-wise on points or apply a scalar. Comparisons are exact and lexicographic, as required by standard algorithms and containers. `EQ(p, q)` instead uses `EPS` for floating-point coordinates and remains exact for other coordinate types.
- `sqnorm()`, `dot(p)`, and `cross(p)` return the exact algebraic results in coordinate type `T`.
- `norm()`, `arg()`, `proj(p)`, and `normalize()` return metric results in `fp_t`; division also promotes to `fp_t` to avoid integer truncation.
- `rotate90()`, `rotate180()`, and `rotate270()` rotate by the corresponding counter-clockwise cardinal angle; overloads taking `p` rotate about point `p`.
- `rotate_cw(t)` and `rotate_ccw(t)` rotate by an arbitrary angle `t` in radians; overloads taking `(p, t)` rotate about point `p`.
- `reflect(p)` reflects across point `p`, while `reflect(p, q)` reflects across the line through points `p` and `q`.
- `to_double()` and `to_ldouble()` explicitly convert the coordinate type. Integral points also convert implicitly to floating-point point types, so `PointI` can be passed where `PointD` is expected.
- `PointI`, `PointL`, `PointD`, and `PointLD` use `int`, `long long`, `double`, and `long double` coordinates, respectively. `Point` aliases `PointD`.

Overflow warning: the exact products `dot()`, `cross()`, and `sqnorm()` grow like the squared coordinate magnitude. With `PointI` these overflow a 32-bit `int` once coordinates exceed roughly a few tens of thousands, so use `PointL` for larger integer coordinates.

**Time Complexity:**  $O(1)$  per call to the constructor and all other operations.

**Space Complexity:**

- $O(1)$  for storage of the point.
- $O(1)$  auxiliary for all operations.

```

#include <cmath>
#include <ostream>
#include <tuple>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

// Epsilon-aware for floating-point coordinates; exact for int and for coordinate types like Modular
// or Rational, which therefore compose for all of the predicates below.
template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T>
struct TPoint {
    // Metric operations preserve floating-point coordinates and otherwise promote to double.
    using fp_t = std::conditional_t<std::is_floating_point<T>::value, T, double>;

    T x, y;

    TPoint() : x(0), y(0) {}
    TPoint(T x, T y) : x(x), y(y) {}
    explicit TPoint(const std::pair<T, T> &p) : x(p.first), y(p.second) {}

    // Implicit conversion from integral to floating-point point (optional, can just use to_double()).
    template<
        typename U,
        typename = std::enable_if_t<std::is_integral<U>::value && std::is_floating_point<T>::value>>
    TPoint(const TPoint<U> &p) : x(static_cast<T>(p.x)), y(static_cast<T>(p.y)) {}

    friend bool EQ(const TPoint &a, const TPoint &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }

    bool operator==(const TPoint &p) const { return x == p.x && y == p.y; }
    bool operator!=(const TPoint &p) const { return !(*this == p); }
    bool operator<(const TPoint &p) const { return std::tie(x, y) < std::tie(p.x, p.y); }
    bool operator>(const TPoint &p) const { return p < *this; }
    bool operator<=(const TPoint &p) const { return !(*this > p); }
    bool operator>=(const TPoint &p) const { return !(*this < p); }
    TPoint operator+(const TPoint &p) const { return {x + p.x, y + p.y}; }
    TPoint operator-(const TPoint &p) const { return {x - p.x, y - p.y}; }
    TPoint operator+(T v) const { return {x + v, y + v}; }
    TPoint operator-(T v) const { return {x - v, y - v}; }
    TPoint operator*(T v) const { return {x * v, y * v}; }

    // Division always promotes to fp_t to avoid integer truncation.
    TPoint<fp_t> operator/(fp_t v) const { return {fp_t(x) / v, fp_t(y) / v}; }

    TPoint &operator+=(const TPoint &p) { x += p.x; y += p.y; return *this; }
    TPoint &operator-=(const TPoint &p) { x -= p.x; y -= p.y; return *this; }
    TPoint &operator+=(T v) { x += v; y += v; return *this; }
    TPoint &operator-=(T v) { x -= v; y -= v; return *this; }
    TPoint &operator*=(T v) { x *= v; y *= v; return *this; }
    friend TPoint operator+(T v, const TPoint &p) { return p + v; }
    friend TPoint operator*(T v, const TPoint &p) { return p * v; }

    // --- Exact operations: return T or TPoint<T>, work for any coordinate type ---

    T sqnorm() const { return x * x + y * y; } // Overflow warning.
    T dot(const TPoint &p) const { return x * p.x + y * p.y; } // Overflow warning.

```

```

T cross(const TPoint &p) const { return x * p.y - y * p.x; } // Overflow warning.

// --- Floating-point operations: return fp_t or TPoint<fp_t> ---

fp_t norm() const { return std::hypot(fp_t(x), fp_t(y)); }
fp_t arg() const { return std::atan2(fp_t(y), fp_t(x)); }
fp_t proj(const TPoint &p) const { return (fp_t(x) * p.x + fp_t(y) * p.y) / p.norm(); }

TPoint<fp_t> normalize() const {
    fp_t n = norm();
    if (n < 1e-30) { // guard against dividing by a near-zero norm, not a
        return {fp_t{0}, fp_t{0}}; // geometric tolerance (EPS is too coarse here)
    }
    return {fp_t(x) / n, fp_t(y) / n};
}

// --- Cardinal rotations: exact for all coordinate types including int ---

// Returns (x, y) rotated 90/180/270 degrees counter-clockwise about the origin.
TPoint rotate90() const { return {-y, x}; }
TPoint rotate180() const { return {-x, -y}; }
TPoint rotate270() const { return {y, -x}; }

// Returns (x, y) rotated 90/180/270 degrees counter-clockwise about point p.
TPoint rotate90(const TPoint &p) const { return (*this - p).rotate90() + p; }
TPoint rotate180(const TPoint &p) const { return (*this - p).rotate180() + p; }
TPoint rotate270(const TPoint &p) const { return (*this - p).rotate270() + p; }

// --- Arbitrary-angle rotations: always return floating-point ---

// Returns (x, y) rotated t radians clockwise about the origin.
TPoint<fp_t> rotate_cw(fp_t t) const {
    fp_t fx(x), fy(y);
    return {fx * std::cos(t) + fy * std::sin(t), fy * std::cos(t) - fx * std::sin(t)};
}

// Returns (x, y) rotated t radians counter-clockwise about the origin.
TPoint<fp_t> rotate_ccw(fp_t t) const {
    fp_t fx(x), fy(y);
    return {fx * std::cos(t) - fy * std::sin(t), fx * std::sin(t) + fy * std::cos(t)};
}

// Returns (x, y) rotated t radians clockwise about point p.
TPoint<fp_t> rotate_cw(const TPoint &p, fp_t t) const {
    return TPoint<fp_t>{fp_t(x) - p.x, fp_t(y) - p.y}.rotate_cw(t) +
        TPoint<fp_t>{fp_t(p.x), fp_t(p.y)};
}

// Returns (x, y) rotated t radians counter-clockwise about point p.
TPoint<fp_t> rotate_ccw(const TPoint &p, fp_t t) const {
    return TPoint<fp_t>{fp_t(x) - p.x, fp_t(y) - p.y}.rotate_ccw(t) +
        TPoint<fp_t>{fp_t(p.x), fp_t(p.y)};
}

// --- Reflections ---

// Returns (x, y) reflected across point p. Exact for any coordinate type.
TPoint reflect(const TPoint &p) const { return {2 * p.x - x, 2 * p.y - y}; }

// Returns (x, y) reflected across the line containing points p and q.
// Always returns floating-point coordinates.
TPoint<fp_t> reflect(const TPoint &p, const TPoint &q) const {
    TPoint<fp_t> fp{fp_t(p.x), fp_t(p.y)};
    if (EQ(p, q)) {
        return TPoint<fp_t>{fp_t(x), fp_t(y)}.reflect(fp);
    }
    TPoint<fp_t> r{fp_t(x) - p.x, fp_t(y) - p.y};
    TPoint<fp_t> s{fp_t(q.x) - p.x, fp_t(q.y) - p.y};

```

```

    fp_t ssq = s.sqnorm();
    r = TPoint<fp_t>{(r.x * s.x + r.y * s.y) / ssq, (r.x * s.y - r.y * s.x) / ssq};
    return TPoint<fp_t>{r.x * s.x - r.y * s.y + fp.x, r.x * s.y + r.y * s.x + fp.y};
}

// --- Explicit type conversions ---

TPoint<double> to_double() const { return {static_cast<double>(x), static_cast<double>(y)}; }

TPoint<long double> to_ldouble() const {
    return {static_cast<long double>(x), static_cast<long double>(y)};
}

// --- Friend free-function versions ---

friend T sqnorm(const TPoint &p) { return p.sqnorm(); }
friend fp_t norm(const TPoint &p) { return p.norm(); }
friend fp_t arg(const TPoint &p) { return p.arg(); }
friend T dot(const TPoint &p, const TPoint &q) { return p.dot(q); }
friend T cross(const TPoint &p, const TPoint &q) { return p.cross(q); }
friend fp_t proj(const TPoint &p, const TPoint &q) { return p.proj(q); }
friend TPoint<fp_t> normalize(const TPoint &p) { return p.normalize(); }
friend TPoint rotate90(const TPoint &p) { return p.rotate90(); }
friend TPoint rotate180(const TPoint &p) { return p.rotate180(); }
friend TPoint rotate270(const TPoint &p) { return p.rotate270(); }
friend TPoint rotate90(const TPoint &p, const TPoint &q) { return p.rotate90(q); }
friend TPoint rotate180(const TPoint &p, const TPoint &q) { return p.rotate180(q); }
friend TPoint rotate270(const TPoint &p, const TPoint &q) { return p.rotate270(q); }
friend TPoint<fp_t> rotate_cw(const TPoint &p, fp_t t) { return p.rotate_cw(t); }
friend TPoint<fp_t> rotate_ccw(const TPoint &p, fp_t t) { return p.rotate_ccw(t); }
friend TPoint<fp_t> rotate_cw(const TPoint &p, const TPoint &q, fp_t t) { return p.rotate_cw(q, t); }
friend TPoint<fp_t> rotate_ccw(const TPoint &p, const TPoint &q, fp_t t) { return p.rotate_ccw(q, t); }
friend TPoint reflect(const TPoint &p, const TPoint &q) { return p.reflect(q); }
friend TPoint<fp_t> reflect(const TPoint &p, const TPoint &a, const TPoint &b) { return p.reflect(a, b); }

friend std::ostream &operator<<(std::ostream &out, const TPoint &p) {
    if constexpr (std::is_floating_point<T>::value) {
        return out << "(" << (std::fabs(p.x) < EPS ? 0 : p.x) << ", "
            << (std::fabs(p.y) < EPS ? 0 : p.y) << ")";
    }
    return out << "(" << p.x << ", " << p.y << ")";
}
};

using PointI = TPoint<int>;
using PointL = TPoint<long long>;
using PointD = TPoint<double>;
using PointLD = TPoint<long double>;
using Point = PointD; // Default point type is double.

// Can compose with numerical types from chapter 6:
// using PointB = TPoint<BigInt>;
// using PointR = TPoint<Rational<int64_t>>;
// using PointM = TPoint<Modular<1000000007>>;

```

## Example Usage

```

#include <cassert>
using namespace std;

const double PI = acos(-1.0);

int main() {
    Point p(-10, 3);
    assert(EQ(Point(-18, 29), p + Point(-3, 9) * 6.0 / 2.0 - Point(-1, 1)));
    assert(EQ(109, p.sqnorm()));
}

```

```

assert(EQ(10.44030650891, p.norm()));
assert(EQ(2.850135859112, p.arg()));
assert(EQ(0, p.dot(Point(3, 10))));
assert(EQ(0, p.cross(Point(10, -3))));
assert(EQ(10, p.proj(Point(-10, 0))));
assert(EQ(1, p.normalize().norm()));
assert(EQ(Point(-3, -10), p.rotate90()));
assert(EQ(Point(10, -3), p.rotate180()));
assert(EQ(Point(3, 10), p.rotate270()));
assert(EQ(Point(3, 12), p.rotate_cw(Point(1, 1), PI / 2)));
assert(EQ(Point(1, -10), p.rotate_ccw(Point(2, 2), PI / 2)));
assert(EQ(Point(10, -3), p.reflect(Point(0, 0))));
assert(EQ(Point(-10, -3), p.reflect(Point(-2, 0), Point(5, 0))));

// Integer point - exact arithmetic, float-only ops return PointD.
PointI a(3, 4), b(1, 0);
assert(a + b == PointI(4, 4));
assert(a - b == PointI(2, 4));
assert(a * 2 == PointI(6, 8));
assert(a.sqnorm() == 25);
assert(a.dot(b) == 3);
assert(a.cross(b) == -4);
assert(EQ(a.norm(), 5.0));
assert(a.rotate90() == PointI(-4, 3));
assert(a.rotate180() == PointI(-3, -4));
assert(a.rotate270() == PointI(4, -3));
PointD anorm = a.normalize(); // returns PointD
assert(EQ(anorm.norm(), 1.0));
PointD adiv = a / 2.0; // returns PointD (division always promotes)
assert(EQ(adiv.x, 1.5) && EQ(adiv.y, 2.0));
auto rotated = PointI(1, 0).rotate_ccw(PI / 4); // returns PointD (arbitrary rotation promotes)
double root2over2 = sqrt(2.0) / 2.0;
assert(EQ(rotated.x, root2over2) && EQ(rotated.y, root2over2));

// Implicit conversion PointI -> PointD.
PointD promoted = PointI(1, 2);
assert(promoted == PointD(1, 2));
return 0;
}

```

## 7.1.2 Line

A straight line in two dimensions, represented by coefficients of  $ax + by + c = 0$  and templated on their type `T`. Coefficients are canonicalized by default so proportional valid coefficient vectors have the same representation, up to floating-point rounding.

The template flag `CANONICALIZE` controls whether construction normalizes the coefficients and defaults to `true`. When enabled, `EXACT = true` divides coefficients by their Euclidean GCD and fixes their sign, while `EXACT = false` scales a nonvertical line to  $b = 1$  and a vertical line to  $a = 1$ . `EXACT` defaults to `true` for built-in integral types; set it explicitly for custom exact types such as `BigInt` and `Rational`. Exact canonicalization requires `%`, comparisons, and basic arithmetic, and predicates remain exact for integral `T`. Floating-point and `Modular` coefficients use the scaling form.

- `TLine<T>(a, b, c)` constructs a line from its coefficients, while `TLine<T>(p, q)` constructs the line through distinct points `p` and `q`. For floating-point `T`, `TLine<T>(slope, p)` constructs the line of the given slope through `p`. The default constructor produces an invalid line.
- `canonicalized()` returns a canonical copy. `operator==` compares stored coefficients exactly, while `EQ(l1, l2)` canonicalizes raw lines if necessary and uses `EPS` for floating-point coefficients.
- `valid()`, `horizontal()`, and `vertical()` classify the line.
- `slope()`, `x(y)`, and `y(x)` return `fp_t`, which is `T` for floating-point coefficients and `double` otherwise. An undefined result is returned as `NaN`.

- `contains(p)`, `is_parallel(l)`, and `is_perpendicular(l)` test geometric relationships.
- `parallel(p)` and `perpendicular(p)` return the corresponding line through point `p`.
- `operator<<` outputs the line in the form `ax+by+c=0`.
- `LineI`, `LineL`, `LineD`, and `LineLD` use `int`, `int64_t`, `double`, and `long double` coefficients, respectively. `Line` aliases `LineD`.

Overflow warning: the exact predicates form products of coefficients, which grow like the squared coordinate magnitude. With `LineI` these may overflow once coordinates reach a few tens of thousands, so use `LineL` for larger integer coordinates.

#### Time Complexity:

- $O(\log C)$  per integral canonicalizing constructor or call to `canonicalize()`, where  $C$  is the largest absolute coefficient.
- $O(1)$  per call to all other operations.

#### Space Complexity:

- $O(1)$  for storage of the line.
- $O(1)$  auxiliary for all operations.

```
#include <cmath>
#include <cstdint>
#include <limits>
#include <ostream>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;
const double M_NAN = std::numeric_limits<double>::quiet_NaN();

// Epsilon-aware for floating-point coordinates; exact for int and for coordinate types like Modular
// or Rational, which therefore compose for all of the predicates below.
template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

// SFINAE guard: valid only when Pt exposes numeric .x/.y members, so templated point constructors
// don't hijack scalar-argument calls.
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

template<typename T, bool CANONICALIZE = true, bool EXACT = std::is_integral<T>::value>
struct TLine {
    // Slope and solve operations preserve floating-point coordinates and otherwise promote to double.
    using fp_t = std::conditional_t<std::is_floating_point<T>::value, T, double>;

    T a, b, c;

    void canonicalize() {
        if constexpr (EXACT) {
            auto gcd = [](T x, T y) {
                if (x < T{0}) x = -x;
                if (y < T{0}) y = -y;
                while (y != T{0}) {
                    x %= y;
                    std::swap(x, y);
                }
                return x;
            };
            T g = gcd(a, gcd(b, c));
            if (g != 0) {
                a /= g;
                b /= g;
            }
        }
    }
};
```



```

    c /= g;
}
if (a < 0 || (a == 0 && b < 0)) {
    a = -a;
    b = -b;
    c = -c;
}
} else if (!EQ(b, T{0})) {
    a /= b;
    c /= b;
    b = T{1};
} else if (!EQ(a, T{0})) {
    c /= a;
    a = T{1};
    b = T{0};
}
}

TLine() : a(0), b(0), c(0) {} // Invalid or uninitialized line.

TLine(T a, T b, T c) : a(a), b(b), c(c) {
    if constexpr (CANONICALIZE) canonicalize();
}

// Line through two points. Coefficients are exact for integral points (overflow warning for c).
template<typename Pt, typename = if_point<Pt>>
TLine(const Pt &p, const Pt &q) : a(q.y - p.y), b(p.x - q.x), c(-(a * p.x + b * p.y)) {
    if constexpr (CANONICALIZE) canonicalize();
}

// Line of a given slope through a point. Intended for floating-point T (a generic slope
// produces non-integer coefficients), so this constructor is disabled for integer T.
template<
    class Pt, typename = if_point<Pt>, typename U = T,
    class = std::enable_if_t<std::is_floating_point<U>::value>>
TLine(fp_t slope, const Pt &p) : a(-slope), b(1), c(slope * p.x - p.y) {
    if constexpr (CANONICALIZE) canonicalize();
}

TLine<T, true, EXACT> canonicalized() const { return TLine<T, true, EXACT>(a, b, c); }

bool operator==(const TLine &l) const { return a == l.a && b == l.b && c == l.c; }
bool operator!=(const TLine &l) const { return !(*this == l); }

friend bool EQ(const TLine &l1, const TLine &l2) {
    if constexpr (CANONICALIZE) {
        return EQ(l1.a, l2.a) && EQ(l1.b, l2.b) && EQ(l1.c, l2.c);
    }
    return EQ(l1.canonicalized(), l2.canonicalized());
}

// Returns whether the line is a valid line (not both a and b zero).
bool valid() const { return !(EQ(a, 0) && EQ(b, 0)); }
bool horizontal() const { return valid() && EQ(a, 0); }
bool vertical() const { return valid() && EQ(b, 0); }

// Slope -a/b, or NaN if the line is vertical or invalid.
fp_t slope() const { return (!valid() || EQ(b, 0)) ? M_NAN : -fp_t(a) / b; }

// Solve for x at a given y (NaN if horizontal or invalid).
fp_t x(fp_t y) const { return (!valid() || EQ(a, 0)) ? M_NAN : -(fp_t(b) * y + c) / a; }

// Solve for y at a given x (NaN if vertical or invalid).
fp_t y(fp_t x) const { return (!valid() || EQ(b, 0)) ? M_NAN : -(fp_t(a) * x + c) / b; }

// Whether point p lies on the line. Exact for integer T and integer p.
template<typename Pt>
bool contains(const Pt &p) const {

```

```

    return valid() && EQ(a * p.x + b * p.y + c, 0); // Overflow warning.
}

// Parallel iff the normals are parallel (cross == 0); perpendicular iff normals are perpendicular
// (dot == 0). Both exact for integer T. Overflow warning.
bool is_parallel(const TLine &l) const { return valid() && l.valid() && EQ(a * l.b, l.a * b); }

bool is_perpendicular(const TLine &l) const {
    return valid() && l.valid() && EQ(a * l.a, -(b * l.b));
}

// Parallel/perpendicular line through point p. Exact for integer T and integer p.
template<typename Pt, typename = if_point<Pt>>
TLine parallel(const Pt &p) const {
    return TLine(a, b, -(a * p.x + b * p.y)); // Overflow warning.
}

template<typename Pt, typename = if_point<Pt>>
TLine perpendicular(const Pt &p) const {
    return TLine(-b, a, b * p.x - a * p.y); // Overflow warning.
}

friend std::ostream &operator<<(std::ostream &out, const TLine &l) {
    auto clean = [](T v) {
        if constexpr (std::is_floating_point_v<T>) {
            return std::fabs(v) < EPS ? T{0} : v;
        }
        return v;
    };
    auto flags = out.flags();
    out << clean(l.a) << "x" << std::showpos << clean(l.b) << "y" << clean(l.c) << "=0";
    out.flags(flags);
    return out;
}
};

using LineI = TLine<int>;
using LineL = TLine<int64_t>;
using LineD = TLine<double>;
using LineLD = TLine<long double>;
using Line = LineD; // Default line type is double.

// Can compose with numerical types from chapter 6:
// using LineB = TLine<BigInt, true, true>;
// using LineR = TLine<Rational<int64_t>, true, true>;

```

## Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    Line invalid;
    assert(!invalid.contains(Point(0, 0)) && !invalid.is_parallel(invalid));

    // Floating-point line.
    Line l(2, -5, -8);
}

```

```

Line para = Line(2, -5, -8).parallel(Point(-6, -2));
Line perp = Line(2, -5, -8).perpendicular(Point(-6, -2));
assert(l.is_parallel(para) && l.is_perpendicular(perp));
assert(l.slope() == 0.4);
assert(EQ(para, Line(-0.4, 1, -0.4)));
assert(EQ(perp, Line(2.5, 1, 17)));
assert(l.contains(Point(1.5, -1))); // 2(1.5) - 5(-1) - 8 = 0.

// Integer line through integer points - exact canonical coefficients and predicates.
LineI il(PointI(0, 0), PointI(2, 1)); // a=1, b=-2, c=0 -> x - 2y = 0
assert(il.a == 1 && il.b == -2 && il.c == 0);
assert(il.contains(PointI(4, 2))); // exact integer check
assert(!il.contains(PointI(4, 3)));
assert(il.is_parallel(LineI(PointI(1, 1), PointI(3, 2)))); // both slope 1/2
assert(il.is_perpendicular(LineI(PointI(0, 0), PointI(1, -2)))); // dot of normals = 0
assert(il.slope() == 0.5); // fp_t result

LineI reduced(2, -4, -6);
assert(reduced == LineI(1, -2, -3));

using RawLine = TLine<double, false>;
RawLine raw(2, -5, -8), scaled(-0.4, 1, 1.6);
assert(raw != scaled); // Stored coefficients differ.
assert(EQ(raw, scaled)); // Canonicalized copies represent the same line.
RawLine canonical = raw;
canonical.canonicalize();
assert(canonical == scaled);
assert(EQ(raw.canonicalized(), Line(-0.4, 1, 1.6)));
return 0;
}

```

### 7.1.3 Circle

A circle in two dimensions with epsilon-aware operations. The circle centered at  $(h, k)$  is represented by the relation  $(x - h)^2 + (y - k)^2 = r^2$ , where the radius  $r$  is normalized to a nonnegative number.

This implementation stores and computes circle parameters in `double`. Point-accepting constructors and predicates are templated on the point type `Pt` and only read `.x`/`.y`, so they accept `Point`, `PointD`, or `PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields.

- `Circle()` constructs the zero-radius circle centered at the origin.
- `Circle(r)` constructs a circle of radius `abs(r)` centered at the origin.
- `Circle(h, k, r)` constructs a circle of radius `abs(r)` centered at  $(h, k)$ .
- `Circle(o, r)` constructs a circle of radius `abs(r)` centered at point `o`.
- `Circle(a, b)` constructs the circle whose diameter is segment `a-b`.
- `Circle(a, b, c)` constructs the circumcircle through non-collinear points `a`, `b`, and `c`, throwing if the points are collinear.
- `Circle(a, b, r)` constructs one circle of radius `abs(r)` passing through points `a` and `b`, throwing if no finite circle is determined by the inputs.
- `operator==` and `operator!=` compare stored centers and radii exactly; `EQ(a, b)` compares them using `EPS`.
- `contains(p)` returns whether point `p` lies inside or on the circle.
- `on_boundary(p)` returns whether point `p` lies on the circle boundary.
- `incircle(a, b, c)` returns the circle inscribed in triangle `abc`.
- `in_circumcircle(a, b, c, d)` returns 1 if point `d` lies strictly inside the circle through points `a`, `b`, and `c`, 0 if it lies on the circle, or -1 if it lies outside. This has exact reasoning for integral points by evaluating the determinant directly without constructing a `Circle` or taking square roots. Points `a`, `b`, and `c` must not be collinear. For integer inputs, the sign test is exact provided the chosen intermediate type does not overflow.

Overflow warning: the determinant in `in_circumcircle()` grows like the fourth power of the coordinate magnitude. For large integer coordinates, use a point or intermediate type wide enough for that determinant.

**Time Complexity:**  $O(1)$  per call to the constructors and all other operations.

**Space Complexity:**

- $O(1)$  for storage of the circle.
- $O(1)$  auxiliary for all operations.

```
#include <cmath>
#include <cstdint>
#include <ostream>
#include <stdexcept>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

// SFINAE guard: valid only when Pt exposes numeric .x/.y members. This keeps the templated point
// constructors from hijacking calls like Circle(double, double, double).
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

struct Circle {
    double h, k, r;

    Circle() : h(0), k(0), r(0) {}
    explicit Circle(double r) : h(0), k(0), r(fabs(r)) {}
    Circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &o, double r) : h(o.x), k(o.y), r(fabs(r)) {}

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = std::hypot(a.x - h, a.y - k);
    }

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b, const Pt &c) {
        double ax = a.x, ay = a.y, bx = b.x, by = b.y, cx = c.x, cy = c.y;
        double asq = ax * ax + ay * ay, bsq = bx * bx + by * by, csq = cx * cx + cy * cy;
        double d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by));
        if (EQ(d, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (asq * (by - cy) + bsq * (cy - ay) + csq * (ay - by)) / d;
        k = (asq * (cx - bx) + bsq * (ax - cx) + csq * (bx - ax)) / d;
        r = std::hypot(ax - h, ay - k);
    }

    template<typename Pt, typename = if_point<Pt>>
```

```

Circle(const Pt &a, const Pt &b, double r) : r(fabs(r)) {
    if (EQ(this->r, 0) && EQ(a.x, b.x) && EQ(a.y, b.y)) {
        h = a.x;
        k = a.y;
        return;
    }
    double d = std::hypot(b.x - a.x, b.y - a.y);
    if (EQ(d, 0)) {
        throw std::runtime_error("Identical points, infinite circles.");
    }
    if (LT(this->r * 2.0, d)) {
        throw std::runtime_error("Points too far away to make Circle.");
    }
    double z = this->r * this->r - d * d / 4.0;
    double v = std::sqrt(z < 0 ? 0 : z) / d;
    double mx = (a.x + b.x) / 2.0, my = (a.y + b.y) / 2.0;
    h = mx + v * (a.y - b.y);
    k = my + v * (b.x - a.x);
    // The other answer is (h, k) = (mx - v*(a.y - b.y), my - v*(b.x - a.x)).
}

bool operator==(const Circle &c) const { return h == c.h && k == c.k && r == c.r; }
bool operator!=(const Circle &c) const { return !(*this == c); }

friend bool EQ(const Circle &a, const Circle &b) {
    return EQ(a.h, b.h) && EQ(a.k, b.k) && EQ(a.r, b.r);
}

template<typename Pt>
bool contains(const Pt &p) const {
    return LE(std::hypot(p.x - h, p.y - k), r);
}

template<typename Pt>
bool on_boundary(const Pt &p) const {
    return EQ(std::hypot(p.x - h, p.y - k), r);
}

friend std::ostream &operator<<(std::ostream &out, const Circle &c) {
    auto flags = out.flags();
    out << std::showpos << "(x" << -(fabs(c.h) < EPS ? 0 : c.h) << ")^2+"
        << "(y" << -(fabs(c.k) < EPS ? 0 : c.k) << ")^2" << std::noshowpos << "="
        << (fabs(c.r) < EPS ? 0 : c.r * c.r);
    out.flags(flags);
    return out;
}
};

template<typename Pt>
Circle incircle(const Pt &a, const Pt &b, const Pt &c) {
    double al = std::hypot(b.x - c.x, b.y - c.y);
    double bl = std::hypot(a.x - c.x, a.y - c.y);
    double cl = std::hypot(a.x - b.x, a.y - b.y);
    double l = al + bl + cl;
    double px = a.x - c.x, py = a.y - c.y, qx = b.x - c.x, qy = b.y - c.y;
    return EQ(l, 0) ? Circle(a.x, a.y, 0)
        : Circle(
            (al * a.x + bl * b.x + cl * c.x) / l, (al * a.y + bl * b.y + cl * c.y) / l,
            fabs(px * qy - py * qx) / l
        );
}

template<typename Pt>
int in_circumcircle(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    using T = decltype(a.x + a.x);
    using W = std::conditional_t<
        std::is_floating_point<T>::value, long double,
        std::conditional_t<std::is_integral<T>::value, int64_t, T>>;

```

```

W adx = (W)a.x - d.x, ady = (W)a.y - d.y;
W bdx = (W)b.x - d.x, bdy = (W)b.y - d.y;
W cdx = (W)c.x - d.x, cdy = (W)c.y - d.y;
// Overflow warning.
W det = (adx * adx + ady * ady) * (bdx * cdy - cdx * bdy) +
        (bdx * bdx + bdy * bdy) * (cdx * ady - adx * cdy) +
        (cdx * cdx + cdy * cdy) * (adx * bdy - bdx * ady);
W orient = ((W)b.x - a.x) * ((W)c.y - a.y) - ((W)b.y - a.y) * ((W)c.x - a.x);
W val = W{0} < orient ? det : -det;
return EQ(val, W{0}) ? 0 : (W{0} < val ? 1 : -1);
}

```

## Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    Circle c(-2, 5, sqrt(10));
    assert(EQ(c, Circle(Point(-2, 5), sqrt(10))));
    assert(EQ(c, Circle(Point(1, 6), Point(-5, 4))));
    assert(EQ(c, Circle(Point(-3, 2), Point(-3, 8), Point(-1, 8))));
    assert(EQ(Circle(Point(0, 0), Point(0, 2), -1), Circle(0, 1, 1)));
    assert(EQ(c, incircle(Point(-12, 5), Point(3, 0), Point(0, 9))));
    assert(c.contains(Point(-2, 8)) && !c.contains(Point(-2, 9)));
    assert(c.on_boundary(Point(-1, 2)) && !c.on_boundary(Point(-1.01, 2)));

    // Integer-coordinate points are accepted; the Circle is computed in double.
    assert(EQ(c, Circle(PointI(1, 6), PointI(-5, 4))));
    assert(EQ(c, Circle(PointI(-3, 2), PointI(-3, 8), PointI(-1, 8))));
    assert(EQ(c, incircle(PointI(-12, 5), PointI(3, 0), PointI(0, 9))));
    assert(c.contains(PointI(-2, 8)) && !c.contains(PointI(-2, 9)));
    assert(c.on_boundary(PointI(-1, 2)));

    // Exact integer in-circle predicate: unit circle through (1,0), (0,1), (-1,0).
    assert(in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(0, 0)) == 1); // inside
    assert(in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(2, 0)) == -1); // out
    assert(in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(0, -1)) == 0); // edge

    // Orientation-independent: reversing a, b, c gives the same answer.
    assert(in_circumcircle(PointI(-1, 0), PointI(0, 1), PointI(1, 0), PointI(0, 0)) == 1);
    return 0;
}

```

### 7.1.4 Triangle

Common triangle calculations in two dimensions. The functions are templated on the point type `Pt`, which should accept `Point`, `PointD`, or `PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields. `triangle_area()` returns `double` regardless of input type (the area is fractional via the `/2`). `same_side()` and `point_in_triangle()` do all arithmetic in the point's own coordinate type and are exact for integer points: they reduce each cross product to a sign before combining, so no precision is lost and cross products are never multiplied together.

- `triangle_area(a, b, c)` returns the area of the triangle with vertices `a`, `b`, and `c`.

- `triangle_area_sides(s1, s2, s3)` returns the area of a triangle with side lengths `s1`, `s2`, and `s3`. The given lengths must be nonnegative and form a valid triangle.
- `triangle_area_medians(m1, m2, m3)` returns the area of a triangle with medians of lengths `m1`, `m2`, and `m3`. The median of a triangle is a line segment joining a vertex to the midpoint of the opposing edge.
- `triangle_area_altitudes(h1, h2, h3)` returns the area of a triangle with altitudes `h1`, `h2`, and `h3`. An altitude of a triangle is the shortest line between a vertex and the infinite line that is extended from its opposite edge.
- `same_side(p1, p2, a, b, include_boundary = true)` returns whether points `p1` and `p2` lie on the same side of the line containing points `a` and `b`. If one or both points lie exactly on the line, then the result will depend on `include_boundary`.
- `point_in_triangle(p, a, b, c, include_boundary = true)` returns whether point `p` lies within the triangle with vertices `a`, `b`, and `c`. If the point lies on or close to an edge (by roughly `EPS`), then the result will depend on `include_boundary`.

Use an integral point type for integer coordinates, and a floating-point type for fractional coordinates.

Overflow warning: each individual cross product is still on the order of the squared coordinate magnitude, so for integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) once coordinates reach a few tens of thousands.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename Pt>
double triangle_area(const Pt &a, const Pt &b, const Pt &c) {
    double acx = static_cast<double>(a.x) - c.x, acy = static_cast<double>(a.y) - c.y;
    double bcx = static_cast<double>(b.x) - c.x, bcy = static_cast<double>(b.y) - c.y;
    return fabs(acx * bcy - acy * bcx) / 2.0;
}

double triangle_area_sides(double s1, double s2, double s3) {
    double s = (s1 + s2 + s3) / 2.0;
    return std::sqrt(s * (s - s1) * (s - s2) * (s - s3));
}

double triangle_area_medians(double m1, double m2, double m3) {
    return 4.0 * triangle_area_sides(m1, m2, m3) / 3.0;
}

double triangle_area_altitudes(double h1, double h2, double h3) {
    if (EQ(h1, 0) || EQ(h2, 0) || EQ(h3, 0)) {
        return 0;
    }
    double x = h1 * h1, y = h2 * h2, z = h3 * h3;
    double v = 2.0 / (x * y) + 2.0 / (x * z) + 2.0 / (y * z);
    return 1.0 / std::sqrt(v - 1.0 / (x * x) - 1.0 / (y * y) - 1.0 / (z * z));
}
```

```

template<typename Pt>
bool same_side(const Pt &p1, const Pt &p2, const Pt &a, const Pt &b, bool include_boundary = true) {
    // Cross products in the point's native coordinate type (exact for integer points).
    auto c1 = (b.x - a.x) * (p1.y - a.y) - (b.y - a.y) * (p1.x - a.x);
    auto c2 = (b.x - a.x) * (p2.y - a.y) - (b.y - a.y) * (p2.x - a.x);
    // Compare the signs, not the product c1 * c2, to avoid integer overflow.
    int s1 = LT(0, c1) ? 1 : (LT(c1, 0) ? -1 : 0);
    int s2 = LT(0, c2) ? 1 : (LT(c2, 0) ? -1 : 0);
    return include_boundary ? (s1 * s2 >= 0) : (s1 * s2 > 0);
}

template<typename Pt>
bool point_in_triangle(
    const Pt &p, const Pt &a, const Pt &b, const Pt &c, bool include_boundary = true
) {
    return same_side(p, a, b, c, include_boundary) && same_side(p, b, a, c, include_boundary) &&
        same_side(p, c, a, b, include_boundary);
}

```

## Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    assert(EQ(6, triangle_area(Point(0, -1), Point(4, -1), Point(0, -4))));
    assert(EQ(6, triangle_area_sides(3, 4, 5)));
    assert(EQ(6, triangle_area_medians(3.605551275, 2.5, 4.272001873)));
    assert(EQ(6, triangle_area_altitudes(3, 4, 2.4)));

    // Fractional coordinates require a floating-point point type.
    assert(point_in_triangle(Point(0, 0), Point(-1, 0), Point(0, -2), Point(4, 0)));
    assert(!point_in_triangle(Point(0, 1), Point(-1, 0), Point(0, -2), Point(4, 0)));
    assert(point_in_triangle(Point(-2.44, 0.82), Point(-1, 0), Point(-3, 1), Point(4, 0)));
    assert(!point_in_triangle(Point(-2.44, 0.7), Point(-1, 0), Point(-3, 1), Point(4, 0)));

    // Integer coordinates: same_side / point_in_triangle are computed exactly in int.
    assert(EQ(8, triangle_area(PointI(0, 0), PointI(4, 0), PointI(0, 4))));
    assert(point_in_triangle(PointI(1, 1), PointI(0, 0), PointI(4, 0), PointI(0, 4)));
    assert(!point_in_triangle(PointI(5, 5), PointI(0, 0), PointI(4, 0), PointI(0, 4)));
    assert(point_in_triangle(PointI(2, 2), PointI(0, 0), PointI(4, 0), PointI(0, 4))); // on edge
    assert(!point_in_triangle(PointI(2, 2), PointI(0, 0), PointI(4, 0), PointI(0, 4), false));
    return 0;
}

```

### 7.1.5 Rectangle

Common axis-aligned rectangle calculations in two dimensions. The functions are templated on the point type `Pt`, which should accept `Point`, `PointD`, or `PointI` from 7.1.1, or any struct with numeric `x` and `y` fields.

- `point_in_rect(p, v, w, h, include_boundary = true)` returns whether `p` is inside the axis-aligned rectangle whose opposite corners are `v` and `(v.x + w, v.y + h)`. Negative widths and heights are supported. The `include_boundary` flag controls whether boundary points count.



- `point_in_rect(p, a, b, include_boundary = true)` returns whether `p` is inside the axis-aligned rectangle with opposite corners `a` and `b`.
- `rect_intersection(a1, b1, a2, b2, &p, &q, include_boundary = true)` finds the intersection of two axis-aligned rectangles, where `a1/b1` and `a2/b2` are opposite-corner pairs. Returns `-1` if the rectangles are disjoint, `0` if they partially intersect, `1` if the first rectangle is completely inside the second, and `2` if the second rectangle is completely inside the first. If an intersection exists, its lower-left and upper-right corners are stored in `p` and `q` when those pointers are non-null. The `include_boundary` flag controls whether boundary-only contact counts as intersecting.

Overflow warning: For integer-coordinate inputs, comparisons are exact as long as intermediate coordinate additions, subtractions, and min/max values do not overflow.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U> bool GT(T a, U b) { return LT(b, a); }
template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

template<typename Pt, typename T>
bool point_in_rect(const Pt &p, const Pt &v, const T &w, const T &h, bool include_boundary = true) {
    if (w < 0) {
        return point_in_rect(p, Pt(v.x + w, v.y), -w, h, include_boundary);
    }
    if (h < 0) {
        return point_in_rect(p, Pt(v.x, v.y + h), w, -h, include_boundary);
    }
    return include_boundary ? (GE(p.x, v.x) && LE(p.x, v.x + w) && GE(p.y, v.y) && LE(p.y, v.y + h))
        : (GT(p.x, v.x) && LT(p.x, v.x + w) && GT(p.y, v.y) && LT(p.y, v.y + h));
}

template<typename Pt>
bool point_in_rect(const Pt &p, const Pt &a, const Pt &b, bool include_boundary = true) {
    auto x1 = std::min(a.x, b.x), y1 = std::min(a.y, b.y);
    auto xh = std::max(a.x, b.x), yh = std::max(a.y, b.y);
    return point_in_rect(p, Pt(x1, y1), xh - x1, yh - y1, include_boundary);
}

template<typename Pt>
int rect_intersection(
    const Pt &a1, const Pt &b1, const Pt &a2, const Pt &b2, Pt *p = nullptr, Pt *q = nullptr,
    const bool include_boundary = true
) {
    // Normalize each rectangle to [x1, xh] x [y1, yh] with coordinate-wise min/max only.
    // (Using auto keeps the native coordinate type, so integer rectangles stay exact.)
    auto x11 = std::min(a1.x, b1.x), xh1 = std::max(a1.x, b1.x);
    auto y11 = std::min(a1.y, b1.y), yh1 = std::max(a1.y, b1.y);
```

```

auto x12 = std::min(a2.x, b2.x), xh2 = std::max(a2.x, b2.x);
auto y12 = std::min(a2.y, b2.y), yh2 = std::max(a2.y, b2.y);
// The intersection is the overlap of the two coordinate intervals.
auto ix1 = std::max(x11, x12), ixh = std::min(xh1, xh2);
auto iy1 = std::max(y11, y12), iyh = std::min(yh1, yh2);
if (include_boundary ? (GT(ix1, ixh) || GT(iy1, iyh)) : (GE(ix1, ixh) || GE(iy1, iyh))) {
    return -1; // Completely disjoint.
}
// Opposing corners of the overlap: bottom-left in p, top-right in q.
if (p != nullptr) {
    *p = Pt(ix1, iy1);
}
if (q != nullptr) {
    *q = Pt(ixh, iyh);
}
// The overlap equals a rectangle exactly when that rectangle is contained in the other.
if (EQ(ix1, x11) && EQ(ixh, xh1) && EQ(iy1, y11) && EQ(iyh, yh1)) {
    return 1; // Rectangle 1 completely inside 2.
}
if (EQ(ix1, x12) && EQ(ixh, xh2) && EQ(iy1, y12) && EQ(iyh, yh2)) {
    return 2; // Rectangle 2 completely inside 1.
}
return 0; // Partial intersection.
}

```

## Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
};

int main() {
    assert(point_in_rect(Point(0, -1), Point(0, -3), 3, 2));
    assert(!point_in_rect(Point(0, -1), Point(0, -3), 3, 2, false));
    assert(point_in_rect(Point(2, -2), Point(3, -3), -3, 2));
    assert(!point_in_rect(Point(0, 0), Point(3, -1), -3, -2));
    assert(point_in_rect(Point(2, -2), Point(3, -3), Point(0, -1)));
    assert(!point_in_rect(Point(0, -1), Point(3, -3), Point(0, -1), false));
    assert(!point_in_rect(Point(-1, -2), Point(3, -3), Point(0, -1)));

    Point p, q;
    // Output coordinates come directly from input coordinates via min/max, so exact equality is safe.
    assert(-1 == rect_intersection(Point(0, 0), Point(1, 1), Point(2, 2), Point(3, 3)));
    assert(0 == rect_intersection(Point(1, 1), Point(7, 7), Point(5, 5), Point(0, 0), &p, &q));
    assert(p == Point(1, 1) && q == Point(5, 5));
    assert(1 == rect_intersection(Point(1, 1), Point(0, 0), Point(0, 0), Point(1, 10), &p, &q));
    assert(p == Point(0, 0) && q == Point(1, 1));
    assert(2 == rect_intersection(Point(0, 5), Point(5, 7), Point(1, 6), Point(2, 5), &p, &q));
    assert(p == Point(1, 5) && q == Point(2, 6));
    assert(
        -1 == rect_intersection(Point(0, 0), Point(1, 1), Point(1, 1), Point(2, 2), &p, &q, false)
    );
    return 0;
}

```

## 7.2 Angles, Distances, and Intersections

### 7.2.1 Angles

Angle calculations in two dimensions. All operations are inherently floating-point (`atan2`, `acos`, trigonometry), so these functions use the local double-coordinate `PointD` type. Convert integer points to `PointD` when asking for angles. The constants `DEG` and `RAD` may be used as multipliers to convert between degrees and radians. For example, if `t` is a value in degrees, then `t * DEG` is the equivalent angle in radians; if `t` is in radians, then `t * RAD` is the equivalent angle in degrees.

- `reduce_deg(t)` takes an angle `t` degrees and returns an equivalent angle in the range  $[0, 360)$  degrees (e.g.  $-630$  becomes  $90$ ).
- `reduce_rad(t)` takes an angle `t` radians and returns an equivalent angle in the range  $[0, 2\pi)$  radians (e.g.  $720.5$  becomes  $0.5$ ).
- `polar_x(r, t)` and `polar_y(r, t)` return the Cartesian coordinates of the point at radius `r` and angle `t` radians in polar coordinates (see `std::polar()`).
- `polar_angle(p)` returns the angle in radians of the line segment from  $(0, 0)$  to point `p`, relative counterclockwise to the positive  $x$ -axis.
- `angle(a, o, b)` returns the smallest angle in radians formed by the points `a`, `o`, `b` with vertex at point `o`. The points `a` and `b` must differ from `o`.
- `angle_between(a, b)` returns the directed counter-clockwise angle in  $[0, 2\pi)$  from vector `a` to vector `b`, treating both points as vectors from the origin. Both vectors must be nonzero.
- `angle_between(a1, b1, a2, b2)` returns the smaller angle in radians between two lines whose normal vectors are  $(a1, b1)$  and  $(a2, b2)$ , limited to  $[0, \pi/2]$ . Both coefficient pairs must represent valid lines.
- `cross(a, b, o = Pt(0, 0))` returns the signed  $z$ -component of the three-dimensional cross product between points `a` and `b`, where the  $z$ -coordinates are implicitly zero and the origin is shifted to point `o`. This is also double the signed area of the triangle from these three points.
- `turn(a, o, b)` returns  $1$  if the path `a`  $\rightarrow$  `o`  $\rightarrow$  `b` forms a left turn on the plane,  $0$  if the path forms a straight line segment, or  $-1$  if it forms a right turn.

Overflow warning: `cross()` and `turn()` preserve the point's coordinate type, so their products may overflow for integral points. Use a 64-bit coordinate type for large integer coordinates.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;
const double PI = std::acos(-1.0);
const double DEG = PI / 180;
const double RAD = 180 / PI;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

double reduce_deg(double t) {
    if (t < -360) {
```

```

    return reduce_deg(std::fmod(t, 360));
}
if (t < 0) {
    return t + 360;
}
return (t >= 360) ? std::fmod(t, 360) : t;
}

double reduce_rad(double t) {
    if (t < -2 * PI) {
        return reduce_rad(std::fmod(t, 2 * PI));
    }
    if (t < 0) {
        return t + 2 * PI;
    }
    return (t >= 2 * PI) ? std::fmod(t, 2 * PI) : t;
}

double polar_x(double r, double t) {
    return r * std::cos(t);
}

double polar_y(double r, double t) {
    return r * std::sin(t);
}

template<typename Pt>
double polar_angle(const Pt &p) {
    double t = std::atan2(static_cast<double>(p.y), static_cast<double>(p.x));
    return (t < 0) ? (t + 2 * PI) : t;
}

template<typename Pt>
double angle(const Pt &a, const Pt &o, const Pt &b) {
    double ux = o.x - a.x, uy = o.y - a.y;
    double vx = o.x - b.x, vy = o.y - b.y;
    double cosine = (ux * vx + uy * vy) / (std::hypot(ux, uy) * std::hypot(vx, vy));
    return std::acos(std::clamp(cosine, -1.0, 1.0));
}

template<typename Pt>
double angle_between(const Pt &a, const Pt &b) {
    double t = std::atan2(a.x * b.y - a.y * b.x, a.x * b.x + a.y * b.y);
    return (t < 0) ? (t + 2 * PI) : t;
}

double angle_between(double a1, double b1, double a2, double b2) {
    double t = std::atan2(a1 * b2 - a2 * b1, a1 * a2 + b1 * b2);
    if (t < 0) {
        t += PI;
    }
    return LT(PI / 2, t) ? (PI - t) : t;
}

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o = Pt(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<typename Pt>
int turn(const Pt &a, const Pt &o, const Pt &b) {
    auto c = cross(a, b, o);
    return LT(c, 0) ? 1 : (LT(0, c) ? -1 : 0);
}

```

## Example Usage

```
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
};

#include <cassert>

int main() {
    assert(EQ(123, reduce_deg(-8 * 360 + 123)));
    assert(EQ(1.2345, reduce_rad(2 * PI * 8 + 1.2345)));
    assert(EQ(-4, polar_x(4, PI)) && EQ(0, polar_y(4, PI)));
    assert(EQ(0, polar_x(4, -PI / 2)) && EQ(-4, polar_y(4, -PI / 2)));
    assert(EQ(45, polar_angle(Point(5, 5)) * RAD));
    assert(EQ(135 * DEG, polar_angle(Point(-4, 4))));
    assert(EQ(90 * DEG, angle(Point(5, 0), Point(0, 5), Point(-5, 0))));
    assert(EQ(225 * DEG, angle_between(Point(0, 5), Point(5, -5))));
    assert(-1 == cross(Point(0, 1), Point(1, 0), Point(0, 0)));
    assert(-1 == turn(Point(0, 1), Point(0, 0), Point(-5, -5)));
    return 0;
}
```

### 7.2.2 Distances

Distance calculations in two dimensions for points, lines, and line segments. The functions are templated on the point type `Pt`: pass any struct with numeric `.x` and `.y` fields (for example `PointD`/`PointI` from 7.1.1, or the local example struct). `sqdist` preserves the coordinate type and is exact for integer points. Functions that return an actual distance use `double` for the final division/square root. `closest_point` returns a point of the same type as its inputs, so use a floating-point `Pt` there for a meaningful (non-truncated) result.

- `dist(a, b)` and `sqdist(a, b)` respectively return the distance and squared distance between points `a` and `b`.
- `line_dist(p, a, b, c)` returns the distance from point `p` to the valid line  $ax + by + c = 0$ .
- `line_dist(p, a, b)` returns the distance from point `p` to the infinite line containing points `a` and `b`. If `a = b`, the distance from `p` to `a` is returned.
- `line_dist(a1, b1, c1, a2, b2, c2)` returns the distance between two valid lines.
- `seg_dist(p, a, b)` returns the distance from point `p` to the line segment `a-b`.
- `seg_dist(a, b, c, d)` returns the minimum distance between line segments `a-b` and `c-d`, or 0 if the segments touch or intersect.
- `closest_point(a, b, p)` returns the point on segment `a-b` closest to point `p`, using the input point type for the result. Use a floating-point point type when the closest point may have fractional coordinates.

Overflow warning: squared distances, dot products, and cross products are formed in the point's coordinate arithmetic type. For integer point types, use a 64-bit coordinate type when coordinates may exceed a few tens of thousands.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
```

```

    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

// sqdist returns the coordinate type (exact for integer points).
template<typename Pt>
auto sqdist(const Pt &a, const Pt &b) {
    auto dx = b.x - a.x, dy = b.y - a.y;
    return dx * dx + dy * dy; // Overflow warning.
}

template<typename Pt>
double dist(const Pt &a, const Pt &b) {
    return std::sqrt(static_cast<double>(sqdist(a, b)));
}

template<typename Pt, typename T>
double line_dist(const Pt &p, const T &a, const T &b, const T &c) {
    return fabs(static_cast<double>(a) * p.x + static_cast<double>(b) * p.y + c) /
        std::sqrt(static_cast<double>(a) * a + static_cast<double>(b) * b);
}

template<typename Pt>
double line_dist(const Pt &p, const Pt &a, const Pt &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y); // Overflow warning.
    double u = static_cast<double>(d) / n;
    double dx = a.x + u * (b.x - a.x) - p.x, dy = a.y + u * (b.y - a.y) - p.y;
    return std::sqrt(dx * dx + dy * dy);
}

template<typename T>
double line_dist(const T &a1, const T &b1, const T &c1, const T &a2, const T &b2, const T &c2) {
    if (EQ(static_cast<double>(a1) * b2, static_cast<double>(a2) * b1)) {
        double factor = EQ(b1, 0) ? (static_cast<double>(a1) / a2) : (static_cast<double>(b1) / b2);
        return EQ(c1, c2 * factor)
            ? 0
            : fabs(c2 * factor - c1) /
                std::sqrt(static_cast<double>(a1) * a1 + static_cast<double>(b1) * b1);
    }
    return 0;
}

template<typename Pt>
double seg_dist(const Pt &p, const Pt &a, const Pt &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y); // Overflow warning.
    if (LE(d, 0) || EQ(n, 0)) {
        return dist(p, a);
    }
    if (GE(d, n)) {
        return dist(p, b);
    }
    double t = static_cast<double>(d) / n;

```

```

    double dx = a.x + t * (b.x - a.x) - p.x, dy = a.y + t * (b.y - a.y) - p.y;
    return std::sqrt(dx * dx + dy * dy);
}

template<typename Pt>
double seg_dist(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return seg_dist(a, c, d);
    }
    if (EQ(c.x, d.x) && EQ(c.y, d.y)) {
        return seg_dist(c, a, b);
    }
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto c1 = ab_x * cd_y - ab_y * cd_x; // Overflow warning.
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) {
        auto less = [](const Pt &u, const Pt &v) { return u.x != v.x ? u.x < v.x : u.y < v.y; };
        auto minp = [](const Pt &u, const Pt &v) -> const Pt & { return less(v, u) ? v : u; };
        auto maxp = [](const Pt &u, const Pt &v) -> const Pt & { return less(u, v) ? v : u; };
        const Pt &res1 = maxp(minp(a, b), minp(c, d));
        const Pt &res2 = minp(maxp(a, b), maxp(c, d));
        if (!less(res2, res1)) {
            return 0;
        }
    }
    else if (!EQ(c1, 0)) {
        auto t_num = ac_x * cd_y - ac_y * cd_x;
        bool c1_pos = c1 > 0;
        bool t_between_01 = c1_pos ? (LE(0, t_num) && LE(t_num, c1)) : (LE(t_num, 0) && LE(c1, t_num));
        bool u_between_01 = c1_pos ? (LE(0, c2) && LE(c2, c1)) : (LE(c2, 0) && LE(c1, c2));
        if (t_between_01 && u_between_01) {
            return 0;
        }
    }
    return std::min(
        std::min(seg_dist(a, c, d), seg_dist(b, c, d)), std::min(seg_dist(c, a, b), seg_dist(d, a, b))
    );
}

template<typename Pt>
Pt closest_point(const Pt &a, const Pt &b, const Pt &p) {
    Pt res{};
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        res.x = a.x;
        res.y = a.y;
        return res;
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y); // Overflow warning.
    double t = static_cast<double>(d) / n;
    if (t <= 0) {
        res.x = a.x;
        res.y = a.y;
    }
    else if (t >= 1) {
        res.x = b.x;
        res.y = b.y;
    }
    else {
        res.x = a.x + t * (b.x - a.x);
        res.y = a.y + t * (b.y - a.y);
    }
    return res;
}

```

## Example Usage

```
#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    assert(EQ(5, dist(Point(-1, -1), Point(2, 3))));
    assert(EQ(25, sqdist(Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, line_dist(Point(2, 1), -4, 3, -1)));
    assert(EQ(0.8, line_dist(Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, line_dist(Point(2, 1), Point(-1, -1), Point(2, 3))));
    assert(EQ(0, line_dist(-4, 3, -1, 8, 6, 2)));
    assert(EQ(0.8, line_dist(-4, 3, -1, -8, 6, -10)));
    assert(EQ(1.0, seg_dist(Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, seg_dist(Point(2, 1), Point(-1, -1), Point(2, 3))));
    assert(EQ(0, seg_dist(Point(0, 2), Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(0.6, seg_dist(Point(-1, 0), Point(-2, 2), Point(-1, -1), Point(2, 3))));

    // Integer points: sqdist is exact, dist returns double.
    PointI ia{-1, -1}, ib{2, 3};
    assert(sqdist(ia, ib) == 25); // int result
    assert(EQ(dist(ia, ib), 5.0)); // double result
    return 0;
}
```

## 7.2.3 Line Intersection

Intersection calculations in two dimensions for straight lines and line segments. The functions are templated on the point type `Pt`: pass any struct with numeric `.x` and `.y` fields (for example `PointD` / `PointI` from 7.1.1, or the local example struct) for which `operator<` orders points lexicographically using exact coordinate comparisons. Point output types are deduced from the caller-supplied arguments, so they may differ from the input type, e.g. integer endpoints with a floating-point output point.

- `line_intersection(a1, b1, c1, a2, b2, c2, &p)` intersects lines  $a_1x + b_1y + c_1 = 0$  and  $a_2x + b_2y + c_2 = 0$ , returning `-1` (parallel), `0` (one point, stored into `p` if it is not `nullptr`), or `1` (identical). Both coefficient triples must represent valid lines.
- `line_intersection(p1, p2, p3, p4, &p)` intersects the infinite lines determined by points `p1`, `p2`, `p3`, and `p4`, returning `-1` (parallel), `0` (one point, stored into `p` if it is not `nullptr`), or `1` (identical). The points in each pair must differ.
- `seg_intersection(a, b, c, d, &p, &q, include_boundary = true)` intersects segments `a-b` and `c-d`, returning `-1` (none), `0` (one point), or `1` (overlapping segment). The `include_boundary` flag controls whether segments that meet only at an endpoint count as intersecting. If the intersection is a point (and `p` is not `nullptr`), it will be stored into `p`. If the intersection is an overlapping segment (and `p` and `q` are not `nullptr`) then their endpoints will be stored into pointers `p` and `q`, which must reference a floating-point point type.
- `seg_intersection(a, b, c, d, include_boundary = true)` is a simplified, detection-only version of the above. Both versions are exact for detection if the input point type is integral.
- `closest_point(a, b, c, p)` returns the closest point on the valid line  $ax + by + c = 0$  to point `p`.

Overflow warning: the exact integer paths multiply coordinate differences (cross products and squared lengths), which grow like the squared coordinate magnitude. With 32-bit `int` coordinates these overflow once coordinates



exceed roughly a few tens of thousands, so for larger integer inputs use a point type with 64-bit (`int64_t`) coordinates.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

// Guard to check outputs are real coordinates, since intersections are generally not integral.
template<typename OutPt>
constexpr bool is_float_pt =
    std::is_floating_point_v<decltype(OutPt::x)> && std::is_floating_point_v<decltype(OutPt::y)>;

template<typename OutPt>
int line_intersection(double a1, double b1, double c1, double a2, double b2, double c2, OutPt *p) {
    static_assert(is_float_pt<OutPt>, "output point coordinates must be floating-point");
    double det = a1 * b2 - a2 * b1; // Cramer's rule for a1*x + b1*y = -c1, a2*x + b2*y = -c2.
    if (EQ(det, 0)) { // Parallel (lines need not be given in normalized form).
        // Identical iff the other two 2x2 determinants also vanish.
        return (EQ(a1 * c2 - a2 * c1, 0) && EQ(b1 * c2 - b2 * c1, 0)) ? 1 : -1;
    }
    if (p != nullptr) {
        double x = (b1 * c2 - b2 * c1) / det; // Local x to ensure y is derived from full-precision x.
        p->x = x;
        p->y = !EQ(b1, 0) ? -(a1 * x + c1) / b1 : -(a2 * x + c2) / b2;
    }
    return 0;
}

template<typename Pt, typename OutPt>
int line_intersection(const Pt &p1, const Pt &p2, const Pt &p3, const Pt &p4, OutPt *p) {
    static_assert(is_float_pt<OutPt>, "output point coordinates must be floating-point");
    auto a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    auto c1 = -(p1.x * p2.y - p2.x * p1.y);
    auto a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    auto c2 = -(p3.x * p4.y - p4.x * p3.y);
    auto x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    auto det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (p != nullptr) {
        p->x = static_cast<double>(x) / det;
        p->y = static_cast<double>(y) / det;
    }
    return 0;
}
```

```

}

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return EQ((p.x - a.x) * (b.y - a.y) - (p.y - a.y) * (b.x - a.x), 0) &&
        LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

// Detection is exact for integer Pt. Intersection point output may use a separate float OutPt.
template<typename Pt, typename OutPt>
int seg_intersection(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, OutPt *p = nullptr, OutPt *q = nullptr,
    const bool include_boundary = true
) {
    static_assert(is_float_pt<OutPt>, "output point coordinates must be floating-point");
    auto set_out = [](auto *out, auto x, auto y) {
        if (out != nullptr) {
            out->x = x;
            out->y = y;
        }
    };
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
    auto cd2 = cd_x * cd_x + cd_y * cd_y;
    if (EQ(ab2, 0)) {
        if (include_boundary && point_on_segment(a, c, d)) {
            set_out(p, a.x, a.y);
            return 0; // Degenerate: first segment is a point intersecting the second segment.
        }
        return -1; // Degenerate: first segment is a point not intersecting the second segment.
    }
    if (EQ(cd2, 0)) {
        if (include_boundary && point_on_segment(c, a, b)) {
            set_out(p, c.x, c.y);
            return 0; // Degenerate: second segment is a point intersecting the first segment.
        }
        return -1; // Degenerate: second segment is a point not intersecting the first segment.
    }
    auto c1 = ab_x * cd_y - ab_y * cd_x;
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        Pt res1 = std::max(std::min(a, b), std::min(c, d));
        Pt res2 = std::min(std::max(a, b), std::max(c, d));
        if (include_boundary ? !(res2 < res1) : (res1 < res2)) {
            if (res1 == res2) {
                set_out(p, res1.x, res1.y);
                return 0; // Collinear and overlapping: touch at one endpoint.
            }
            if (p != nullptr && q != nullptr) { // Both ends are reported, or neither is.
                set_out(p, res1.x, res1.y);
                set_out(q, res2.x, res2.y);
            }
            return 1; // Collinear and overlapping: overlap is a segment.
        }
        return -1; // Collinear and disjoint.
    }
    if (EQ(c1, 0)) {
        return -1; // Parallel and disjoint.
    }
    auto t_num = ac_x * cd_y - ac_y * cd_x;
    bool c1_pos = c1 > 0;
    bool t_ok =
        c1_pos
        ? (include_boundary ? (LE(0, t_num) && LE(t_num, c1)) : (LT(0, t_num) && LT(t_num, c1)))

```

```

        : (include_boundary ? (LE(c1, t_num) && LE(t_num, 0)) : (LT(c1, t_num) && LT(t_num, 0)));
bool u_ok = c1_pos ? (include_boundary ? (LE(0, c2) && LE(c2, c1)) : (LT(0, c2) && LT(c2, c1)))
                : (include_boundary ? (LE(c1, c2) && LE(c2, 0)) : (LT(c1, c2) && LT(c2, 0)));
if (t_ok && u_ok) {
    double t = static_cast<double>(t_num) / c1;
    set_out(p, a.x + t * ab_x, a.y + t * ab_y);
    return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersection.
}

// Simplified detection-only version (no output points needed); exact for integer Pt.
// Alternatively, just call the version above and pass static_cast<Pt *>(nullptr) for p and q.
template<typename Pt>
int seg_intersection(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, bool include_boundary = true
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
    auto cd2 = cd_x * cd_x + cd_y * cd_y;
    if (EQ(ab2, 0)) {
        return (include_boundary && point_on_segment(a, c, d)) ? 0 : -1;
    }
    if (EQ(cd2, 0)) {
        return (include_boundary && point_on_segment(c, a, b)) ? 0 : -1;
    }
    auto c1 = ab_x * cd_y - ab_y * cd_x;
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        Pt res1 = std::max(std::min(a, b), std::min(c, d));
        Pt res2 = std::min(std::max(a, b), std::max(c, d));
        if (include_boundary ? !(res2 < res1) : (res1 < res2)) {
            return (res1 == res2) ? 0 : 1;
        }
        return -1;
    }
    if (EQ(c1, 0)) {
        return -1;
    }
    auto t_num = ac_x * cd_y - ac_y * cd_x;
    bool c1_pos = c1 > 0;
    bool t_ok =
        c1_pos
            ? (include_boundary ? (LE(0, t_num) && LE(t_num, c1)) : (LT(0, t_num) && LT(t_num, c1)))
            : (include_boundary ? (LE(c1, t_num) && LE(t_num, 0)) : (LT(c1, t_num) && LT(t_num, 0)));
    bool u_ok = c1_pos ? (include_boundary ? (LE(0, c2) && LE(c2, c1)) : (LT(0, c2) && LT(c2, c1)))
                    : (include_boundary ? (LE(c1, c2) && LE(c2, 0)) : (LT(c1, c2) && LT(c2, 0)));
    return (t_ok && u_ok) ? 0 : -1;
}

template<typename Pt>
Pt closest_point(double a, double b, double c, const Pt &p) {
    Pt res{};
    if (EQ(a, 0)) {
        res.x = p.x;
        res.y = -c / b;
    } else if (EQ(b, 0)) {
        res.x = -c / a;
        res.y = p.y;
    } else {
        line_intersection(a, b, c, -b, a, b * p.x - a * p.y, &res);
    }
    return res;
}

```

## Example Usage

```
#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const Point &p) const { return p < *this; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const PointI &p) const { return p < *this; }
};

int main() {
    Point p, q;
    assert(line_intersection(-1.0, 1.0, 0.0, 1.0, 1.0, -3.0, &p) == 0);
    assert(EQ(p, Point(1.5, 1.5)));
    assert(line_intersection(Point(0, 0), Point(1, 1), Point(0, 4), Point(4, 0), &p) == 0);
    assert(EQ(p, Point(2, 2)));

    auto test = [&](double a, double b, double c, double d, double e, double f, double g, double h) {
        return seg_intersection(Point(a, b), Point(c, d), Point(e, f), Point(g, h), &p, &q);
    };
    assert(0 == test(-4, 0, 4, 0, 0, -4, 0, 4) && EQ(p, Point(0, 0)));
    assert(0 == test(0, 0, 10, 10, 2, 2, 16, 4) && EQ(p, Point(2, 2)));
    assert(0 == test(-2, 2, -2, -2, -2, 0, 0, 0) && EQ(p, Point(-2, 0)));
    assert(0 == test(0, 4, 4, 4, 4, 0, 4, 8) && EQ(p, Point(4, 4)));
    assert(1 == test(10, 10, 0, 0, 2, 2, 6, 6));
    assert(EQ(p, Point(2, 2)) && EQ(q, Point(6, 6)));
    assert(-1 == test(6, 8, 8, 10, 12, 12, 4, 4));
    assert(-1 == test(-4, 2, -8, 8, 0, 0, -4, 6));

    assert(EQ(Point(2.5, 2.5), closest_point(-1.0, -1.0, 5.0, Point(0, 0))));
    assert(EQ(Point(3, 0), closest_point(1.0, 0.0, -3.0, Point(0, 0))));

    // Detection-only with integer coordinates (exact, no float needed).
    assert(seg_intersection(PointI(0, 0), PointI(4, 4), PointI(0, 4), PointI(4, 0)) == 0);
    assert(seg_intersection(PointI(0, 0), PointI(1, 0), PointI(2, 0), PointI(3, 0)) == -1);
    assert(seg_intersection(PointI(0, 0), PointI(4, 4), PointI(0, 4), PointI(4, 0), &p) == 0);
    assert(EQ(p, Point(2, 2)));

    // include_boundary = false treats endpoint-only contact as non-intersecting. The T-junction
    // and shared-endpoint cases below intersect by default but are rejected when the flag is off.
    assert(0 == seg_intersection(Point(0, 4), Point(4, 4), Point(4, 0), Point(4, 8), &p, &q));
    assert(-1 == seg_intersection(Point(0, 4), Point(4, 4), Point(4, 0), Point(4, 8), &p, &q, false));
    assert(-1 == seg_intersection(PointI(0, 0), PointI(2, 2), PointI(2, 2), PointI(6, 6), false));
    return 0;
}
```

## 7.2.4 Circle Intersection

Circle tangent and intersection calculations in two dimensions. Such calculations are floating-point by nature, so this section mostly computes in `double`. The input point `p` to `tangent()` can be any type with numeric `.x` and `.y` fields, and intersection point outputs are written through a caller-supplied point type.

Circle radii are normalized to be nonnegative.

- `tangent(c, p, &l1, &l2)` determines the line(s) tangent to circle `c` that pass through point `p`, returning `-1` if there is no tangent line because `p` is strictly inside `c`, `0` if there is exactly one tangent line because `p` is on the boundary of `c` (in which case the line will be stored into pointer `l1` if it's not `nullptr`), or `1` if there are two tangent lines because `p` is strictly outside of `c` (in which case the lines will be stored into pointers `l1` and `l2` if they are not `nullptr`). The circle must have positive radius.
- `intersection(c, l, &p, &q)` determines the intersection between the circle `c` and line `l`, returning `-1` if there is no intersection, `0` if the line has one intersection point because the line is tangent (in which case it will be stored into pointer `p` if it's not `nullptr`), or `1` if there are two intersection points because the line crosses through the circle (stored independently into `p` and `q` when those pointers are non-null). The line must be valid.
- `intersection(c1, c2, &p, &q)` determines the intersection points between two circles `c1` and `c2`, returning `-2` if circle `c2` completely encloses circle `c1`, `-1` if circle `c1` completely encloses circle `c2`, `0` if the circles are completely disjoint, `1` if the circles are tangent with one intersection (stored in `p` if it's not `nullptr`), `2` if the circles intersect at two points (stored independently in `p` and `q` when those pointers are non-null), or `3` if the circles are equal and intersect at infinite points.
- `intersection_area(c1, c2)` returns the intersection area of circles `c1` and `c2`.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9, PI = std::acos(-1.0);

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

struct Circle {
    double h, k, r;

    Circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}
};

struct Line {
    double a, b, c;

    Line() : a(0), b(0), c(0) {}

    Line(double a, double b, double c) {
        if (!EQ(b, 0)) {
            this->a = a / b;
            this->c = c / b;
            this->b = 1;
        } else {
            this->c = c / a;
        }
    }
};
```

```

        this->a = 1;
        this->b = 0;
    }
}

Line(double px, double py, double qx, double qy) : a(0), b(0), c(0) {
    if (EQ(px, qx)) {
        if (!EQ(py, qy)) { // Vertical line.
            a = 1;
            b = 0;
            c = -px;
        } // Else, invalid line.
    } else {
        a = -static_cast<double>(py - qy) / (px - qx);
        b = 1;
        c = -(a * px) - (b * py);
    }
}

bool operator==(const Line &l1) const { return a == l1.a && b == l1.b && c == l1.c; }

friend bool EQ(const Line &l1, const Line &l2) {
    return EQ(l1.a, l2.a) && EQ(l1.b, l2.b) && EQ(l1.c, l2.c);
}
};

template<typename Pt>
int tangent(const Circle &c, const Pt &p, Line *l1 = nullptr, Line *l2 = nullptr) {
    auto sqnorm = [](double x, double y) { return x * x + y * y; };
    double vopx = p.x - c.h, vopy = p.y - c.k;
    if (EQ(sqnorm(vopx, vopy), c.r * c.r)) { // Point on an edge.
        if (l1 != nullptr) { // Tangent through p is perpendicular to the radius cp.
            *l1 = Line(vopx, vopy, -(vopx * p.x + vopy * p.y));
        }
        return 0;
    }
    if (LE(sqnorm(vopx, vopy), c.r * c.r)) {
        return -1; // Point inside Circle, no intersection.
    }
    double qx = vopx / c.r, qy = vopy / c.r;
    double n = sqnorm(qx, qy), d = qy * std::sqrt(n - 1.0);
    double t1x = (qx - d) / n, t1y = c.k;
    double t2x = (qx + d) / n, t2y = c.k;
    if (!EQ(qy, 0)) { // Common case.
        t1y += c.r * (1.0 - t1x * qx) / qy;
        t2y += c.r * (1.0 - t2x * qx) / qy;
    } else { // Point at center horizontal, y = 0.
        d = c.r * std::sqrt(1.0 - t1x * t1x);
        t1y += d;
        t2y -= d;
    }
    t1x = t1x * c.r + c.h;
    t2x = t2x * c.r + c.h;
    // Note: here, t1 and t2 are the two points of tangencies
    if (l1 != nullptr) {
        *l1 = Line(p.x, p.y, t1x, t1y);
    }
    if (l2 != nullptr) {
        *l2 = Line(p.x, p.y, t2x, t2y);
    }
    return 1;
}

// Guard to check outputs are real coordinates, since intersections are generally not integral.
template<typename OutPt>
constexpr bool is_float_pt =
    std::is_floating_point_v<decltype(OutPt::x)> && std::is_floating_point_v<decltype(OutPt::y)>;

```

```

template<typename OutPt>
int intersection(const Circle &c, const Line &l, OutPt *p = nullptr, OutPt *q = nullptr) {
    static_assert(is_float_pt<OutPt>, "output point coordinates must be floating-point");
    auto set_out = [](auto *out, auto x, auto y) {
        if (out != nullptr) {
            out->x = x;
            out->y = y;
        }
    };
    double v = c.h * l.a + c.k * l.b + l.c;
    double aabb = l.a * l.a + l.b * l.b;
    double disc = v * v / aabb - c.r * c.r;
    if (disc > EPS) {
        return -1;
    }
    double x0 = -l.a * v / aabb, y0 = -l.b * v / aabb;
    if (disc > -EPS) {
        set_out(p, x0 + c.h, y0 + c.k);
        return 0;
    }
    double k = std::sqrt(std::max(0.0, disc / -aabb));
    set_out(p, x0 + k * l.b + c.h, y0 - k * l.a + c.k);
    set_out(q, x0 - k * l.b + c.h, y0 + k * l.a + c.k);
    return 1;
}

template<typename OutPt>
int intersection(const Circle &c1, const Circle &c2, OutPt *p = nullptr, OutPt *q = nullptr) {
    static_assert(is_float_pt<OutPt>, "output point coordinates must be floating-point");
    auto set_out = [](auto *out, auto x, auto y) {
        if (out != nullptr) {
            out->x = x;
            out->y = y;
        }
    };
    if (EQ(c1.h, c2.h) && EQ(c1.k, c2.k)) {
        return EQ(c1.r, c2.r) ? 3 : (c1.r > c2.r ? -1 : -2);
    }
    double d12x = c2.h - c1.h, d12y = c2.k - c1.k;
    double d = std::hypot(d12x, d12y);
    if (LT(c1.r + c2.r, d)) {
        return 0;
    }
    if (LT(d, fabs(c1.r - c2.r))) {
        return c1.r > c2.r ? -1 : -2;
    }
    double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d);
    double x0 = c1.h + (d12x * a / d), y0 = c1.k + (d12y * a / d);
    double s = std::sqrt(std::max(0.0, c1.r * c1.r - a * a));
    double rx = -d12y * s / d, ry = d12x * s / d;
    if (EQ(rx, 0) && EQ(ry, 0)) {
        set_out(p, x0, y0);
        return 1;
    }
    set_out(p, x0 - rx, y0 - ry);
    set_out(q, x0 + rx, y0 + ry);
    return 2;
}

double intersection_area(const Circle &c1, const Circle &c2) {
    double r = std::min(c1.r, c2.r), R = std::max(c1.r, c2.r);
    double d = std::hypot(c2.h - c1.h, c2.k - c1.k);
    if (LE(d, R - r)) {
        return PI * r * r;
    }
    if (LE(R + r, d)) {
        return 0;
    }
}

```

```

double alpha = std::clamp((d * d + r * r - R * R) / 2 / d / r, -1.0, 1.0);
double beta = std::clamp((d * d + R * R - r * r) / 2 / d / R, -1.0, 1.0);
return r * r * std::acos(alpha) + R * R * std::acos(beta) -
    0.5 * std::sqrt((-d + r + R) * (d + r - R) * (d - r + R) * (d + r + R));
}

```

## Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
};

int main() {
    Line l1, l2;
    assert(-1 == tangent(Circle(0, 0, 4), Point(1, 1), &l1, &l2));
    assert(0 == tangent(Circle(0, 0, sqrt(2)), Point(1, 1), &l1, &l2));
    assert(EQ(l1, Line(-1, -1, 2)));
    assert(1 == tangent(Circle(0, 0, 2), Point(2, 2), &l1, &l2));
    assert(EQ(l1, Line(0, -2, 4)));
    assert(EQ(l2, Line(2, 0, -4)));

    Point p, q;
    assert(-1 == intersection(Circle(1, 1, 3), Line(5, 3, -30), &p, &q));
    assert(0 == intersection(Circle(1, 1, 3), Line(0, 1, -4), &p, &q));
    assert(EQ(p, Point(1, 4)));
    assert(1 == intersection(Circle(1, 1, 3), Line(0, 1, -1), &p, &q));
    assert(EQ(p, Point(4, 1)));
    assert(EQ(q, Point(-2, 1)));
    assert(1 == intersection(Circle(1, 1, 3), Line(0, 1, -1), &p));
    assert(EQ(p, Point(4, 1)));

    assert(-2 == intersection(Circle(1, 1, 1), Circle(0, 0, 3), &p, &q));
    assert(-1 == intersection(Circle(0, 0, 3), Circle(1, 1, 1), &p, &q));
    assert(0 == intersection(Circle(5, 0, 4), Circle(-5, 0, 4), &p, &q));
    assert(1 == intersection(Circle(-5, 0, 5), Circle(5, 0, 5), &p, &q));
    assert(EQ(p, Point(0, 0)));
    assert(2 == intersection(Circle(-0.5, 0, 1), Circle(0.5, 0, 1), &p, &q));
    assert(EQ(p, Point(0, -sqrt(3) / 2)));
    assert(EQ(q, Point(0, sqrt(3) / 2)));
    assert(
        2 == intersection(Circle(-0.5, 0, 1), Circle(0.5, 0, 1), static_cast<Point *>(nullptr), &q)
    );
    assert(EQ(q, Point(0, sqrt(3) / 2)));

    // Each circle passes through the other's center.
    double r = 3, a = intersection_area(Circle(-r / 2, 0, r), Circle(r / 2, 0, r));
    assert(EQ(a, r * r * (2 * PI / 3 - sqrt(3) / 2)));
    return 0;
}

```

## 7.2.5 Circle Tangents

Given two circles, compute their common tangent lines by returning the two points of tangency on each circle. External tangents touch both circles on the same side of the line joining their centers; internal tangents cross between the circles. A point can be treated as a zero-radius circle, so this also covers tangents from a point to a circle.



- `circle_tangents(c1, r1, c2, r2, internal = false)` returns 0, 1, or 2 tangent point pairs. Each pair  $(p, q)$  means the tangent line touches the first circle at  $p$  and the second circle at  $q$ . Set `internal = true` to get internal tangents; otherwise external tangents are returned.
- `all_circle_tangents(c1, r1, c2, r2)` returns all external and internal tangents.

If the two circles are identical, there are infinitely many tangents and the functions return an empty vector. If one circle strictly contains the other, there are no external tangents; if the circles overlap, there are no internal tangents.

**Time Complexity:**  $O(1)$  per call.

**Space Complexity:**  $O(1)$  auxiliary, excluding the returned vector.

```
#include <algorithm>
#include <cmath>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    Point operator/(double k) const { return {x / k, y / k}; }
    double sqnorm() const { return x * x + y * y; }
    Point rotate90() const { return {-y, x}; }
};

template<typename PtA, typename PtB>
std::vector<std::pair<Point, Point>> circle_tangents(
    const PtA &c1, double r1, const PtB &c2, double r2, bool internal = false
) {
    r1 = fabs(r1);
    r2 = fabs(r2);
    if (internal) {
        r2 = -r2;
    }
    Point d(c2.x - c1.x, c2.y - c1.y);
    double dr = r1 - r2, d2 = d.sqnorm(), h2 = d2 - dr * dr;
    if (EQ(d2, 0) || LT(h2, 0)) {
        return {};
    }
    std::vector<std::pair<Point, Point>> res;
    for (double s : {-1.0, 1.0}) {
        Point v = (d * dr + d.rotate90() * (std::sqrt(std::max(0.0, h2)) * s)) / d2;
        res.emplace_back(Point(c1.x, c1.y) + v * r1, Point(c2.x, c2.y) + v * r2);
    }
    if (EQ(h2, 0)) {
        res.pop_back();
    }
}
```

```

    }
    return res;
}

template<typename PtA, typename PtB>
std::vector<std::pair<Point, Point>> all_circle_tangents(
    const PtA &c1, double r1, const PtB &c2, double r2
) {
    auto res = circle_tangents(c1, r1, c2, r2);
    auto internal = circle_tangents(c1, r1, c2, r2, true);
    res.insert(res.end(), internal.begin(), internal.end());
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Point a(0, 0), b(4, 0);
    auto ext = circle_tangents(a, 1, b, 1);
    assert(ext.size() == 2);
    assert(EQ(ext[0].first, Point(0, -1)) && EQ(ext[0].second, Point(4, -1)));
    assert(EQ(ext[1].first, Point(0, 1)) && EQ(ext[1].second, Point(4, 1)));

    auto in = circle_tangents(a, 1, b, 1, true);
    assert(in.size() == 2);
    assert(
        EQ(in[0].first, Point(0.5, -sqrt(3.0) / 2)) && EQ(in[0].second, Point(3.5, sqrt(3.0) / 2))
    );
    assert(
        EQ(in[1].first, Point(0.5, sqrt(3.0) / 2)) && EQ(in[1].second, Point(3.5, -sqrt(3.0) / 2))
    );
    auto all = ext;
    all.insert(all.end(), in.begin(), in.end());
    assert(all_circle_tangents(a, 1, b, 1) == all);
    auto point_to_circle = circle_tangents(Point(0, 0), 0, Point(2, 0), 1);
    assert(point_to_circle.size() == 2);
    assert(
        EQ(point_to_circle[0].first, Point(0, 0)) &&
        EQ(point_to_circle[0].second, Point(1.5, -sqrt(3.0) / 2))
    );
    assert(
        EQ(point_to_circle[1].first, Point(0, 0)) &&
        EQ(point_to_circle[1].second, Point(1.5, sqrt(3.0) / 2))
    );
    assert(circle_tangents(Point(0, 0), 5, Point(1, 0), 1).empty()); // contained
    assert(circle_tangents(Point(0, 0), 2, Point(3, 0), 2, true).empty()); // overlapping
    return 0;
}

```

## 7.3 Polygons and Point Sets

### 7.3.1 Polygon Sorting and Area

Given a list of distinct points in two dimensions, order them into a valid polygon and determine the area. The sorting comparator orders two points by the sign of their cross product around a chosen center, that is, by angle. To form a simple polygon, choose a center inside the points' convex hull that differs from every input point; the arithmetic mean used in the example is usually convenient. The area functions use the shoelace formula, summing

the cross products of consecutive vertex pairs. All functions accept `Point`, `PointI`, or `PointL` from 7.1.1, or any compatible point type with numeric `.x` and `.y` fields and the comparison operators used by the requested operation.

- `cw_comp(a, b, c)` returns whether point `a` compares clockwise before `b` about `c`.
- `polygon_area_2x(lo, hi)` returns exactly double the area of the polygon with vertices specified by the range `[lo, hi)` of points in either clockwise or counter-clockwise order. The return value is integral or floating-point, depending on the input point type. For integer vertices, divide by 2 in the caller if the exact area is needed.
- `polygon_area(lo, hi)` returns the area as `double`.
- `polygon_centroid(lo, hi)` returns the centroid (center of mass) of a non-degenerate simple polygon with vertices in boundary order as an `std::pair<double, double>`. It uses signed shoelace weights, so either clockwise or counter-clockwise inputs will work.

Overflow warning: `cw_comp` and `polygon_area_2x` form cross products that grow like the squared coordinate magnitude (and the shoelace sum accumulates over all vertices). For integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) for large or numerous coordinates.

#### Time Complexity:

- $O(n)$  per call to `polygon_area_2x()`, `polygon_area()`, and `polygon_centroid()`, where  $n$  is the number of points.
- $O(1)$  per call to `cw_comp()` and the comparators.

**Space Complexity:**  $O(1)$  auxiliary for all operations.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <random>
#include <stdexcept>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

// Comparator (clockwise angular order about c). Exact: comparisons are pure sign tests on
// coordinate differences and the cross product, so it is a valid strict weak ordering and is
// exact for integer-coordinate points.
template<typename Pt>
bool cw_comp(const Pt &a, const Pt &b, const Pt &c) {
    if (a.x - c.x >= 0 && b.x - c.x < 0) {
        return true;
    }
    if (a.x - c.x < 0 && b.x - c.x >= 0) {
        return false;
    }
    if (a.x - c.x == 0 && b.x - c.x == 0) {
        if (a.y - c.y >= 0 || b.y - c.y >= 0) {
            return a.y > b.y;
        }
        return b.y > a.y;
    }
}

auto acx = a.x - c.x, acy = a.y - c.y;
auto bcx = b.x - c.x, bcy = b.y - c.y;
auto det = acx * bcy - acy * bcx; // Overflow warning.
if (det == 0) {
    auto acnorm = acx * acx + acy * acy; // Overflow warning.
    auto bcnorm = bcx * bcx + bcy * bcy;
    return acnorm > bcnorm;
}
```

```

    }
    return det < 0;
}

// Returns 2 * area. Result is exact (integer) for integer-coordinate points.
template<typename It>
auto polygon_area_2x(It lo, It hi) {
    using T = decltype(lo->x * lo->y);
    if (lo == hi) {
        return T{0};
    }
    T area = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        area += static_cast<T>(j->x - i->x) * static_cast<T>(j->y + i->y); // Overflow warning.
    }
    return area < T{0} ? -area : area;
}

template<typename It>
double polygon_area(It lo, It hi) {
    return static_cast<double>(polygon_area_2x(lo, hi)) / 2.0;
}

template<typename It>
std::pair<double, double> polygon_centroid(It lo, It hi) {
    assert(hi - lo >= 3);
    double cx = 0, cy = 0, area2 = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        double cross = static_cast<double>(j->x) * i->y - static_cast<double>(i->x) * j->y;
        cx += (static_cast<double>(j->x) + i->x) * cross;
        cy += (static_cast<double>(j->y) + i->y) * cross;
        area2 += cross;
    }
    if (EQ(area2, 0)) {
        throw std::runtime_error("Cannot compute centroid of zero-area polygon.");
    }
    return {cx / (3 * area2), cy / (3 * area2)};
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const Point &p) const { return p < *this; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const PointI &p) const { return p < *this; }
};

template<typename It>
Point mean_center(It lo, It hi) {

```

```

assert(lo != hi);
double x_sum = 0, y_sum = 0, n = hi - lo;
for (It it = lo; it != hi; ++it) {
    x_sum += it->x;
    y_sum += it->y;
}
return Point(x_sum / n, y_sum / n);
}

int main() {
    vector<Point> points{Point(1, 3), Point(1, 2), Point(2, 1), Point(0, 0), Point(-1, 3)};
    vector<Point> v(points);
    mt19937 rng(1234567); // Fixed seed for reproducibility.
    shuffle(v.begin(), v.end(), rng);
    Point c = mean_center(v.begin(), v.end());
    assert(EQ(c, Point(0.6, 1.8)));
    sort(v.begin(), v.end(), [c](const Point &a, const Point &b) { return cw_comp(a, b, c); });
    assert((v == vector<Point>{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}}));
    assert(EQ(polygon_area(v.begin(), v.end()), 5));

    sort(v.begin(), v.end(), [c](const Point &a, const Point &b) { return cw_comp(b, a, c); });
    assert((v == vector<Point>{{-1, 3}, {0, 0}, {2, 1}, {1, 2}, {1, 3}}));
    assert(EQ(polygon_area(v.begin(), v.end()), 5));

    // Integer points: polygon_area_2x is exact (no float arithmetic).
    vector<PointI> iv{{0, 0}, {4, 0}, {0, 3}}; // right triangle, area = 6
    assert(polygon_area_2x(iv.begin(), iv.end()) == 12); // exact int
    assert(EQ(polygon_area(iv.begin(), iv.end()), 6.0));
    auto centroid = polygon_centroid(iv.begin(), iv.end());
    assert(EQ(centroid.first, 4.0 / 3.0) && EQ(centroid.second, 1.0));
    return 0;
}

```

### 7.3.2 Point-in-Polygon

Given a point `p` and a polygon, determines whether `p` lies inside using ray casting. A ray cast from `p` must cross the polygon's boundary an odd number of times exactly when `p` is inside, so the function counts crossings between a horizontal ray and each polygon edge. The function is templated on the point type `Pt` and accepts `Point`, `PointI`, or `PointL` from 7.1.1, or any compatible type with numeric `x` and `y` fields. All comparisons are exact (no epsilon): the ray-crossing parity needs a consistent comparison to be correct, and the result is exact for integer-coordinate points.

- `point_in_polygon(p, lo, hi, include_boundary = true)` returns whether `p` lies within the polygon with vertices specified by the range `[lo, hi)` of points in either clockwise or counter-clockwise order. The `include_boundary` flag controls whether points exactly on an edge or vertex count as inside.

Overflow warning: the edge orientation test forms a cross product that grows like the squared coordinate magnitude. For integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) once coordinates exceed roughly a few tens of thousands.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the distance between `lo` and `hi`.

**Space Complexity:**  $O(1)$  auxiliary.

```

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// Detection is exact for integer-coordinate Pt.
template<typename Pt, typename It>
bool point_in_polygon(const Pt &p, It lo, It hi, bool include_boundary = true) {

```

```

    if (lo == hi) {
        return false;
    }
    bool res = false;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        if (i->y == p.y && (i->x == p.x || (j->y == p.y && (i->x <= p.x || j->x <= p.x)))) {
            return include_boundary;
        }
        if ((p.y < i->y) != (p.y < j->y)) {
            auto det = cross(*i, *j, p);
            if (det == 0) {
                return include_boundary;
            }
            if ((0 < det) != (i->y < j->y)) {
                res = !res;
            }
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    vector<Point> poly{Point(-1, 3), Point(1, 3), Point(2, 1), Point(0, 0)};
    assert(point_in_polygon(Point(1, 2), poly.begin(), poly.end()));
    assert(point_in_polygon(Point(0, 3), poly.begin(), poly.end()));
    assert(!point_in_polygon(Point(0, 3), poly.begin(), poly.end(), false));
    assert(!point_in_polygon(Point(0, 3.01), poly.begin(), poly.end()));
    assert(!point_in_polygon(Point(2, 2), poly.begin(), poly.end()));

    // Integer-coordinate points: exact detection.
    vector<PointI> ipoly{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
    assert(point_in_polygon(PointI(2, 2), ipoly.begin(), ipoly.end()));
    assert(!point_in_polygon(PointI(4, 2), ipoly.begin(), ipoly.end(), false));
    assert(!point_in_polygon(PointI(5, 2), ipoly.begin(), ipoly.end()));
    return 0;
}

```

### 7.3.3 Convex Hull and Diametral Pair

The convex hull of points in two dimensions is the smallest convex set containing them. A diametral pair is a pair of input points at maximum distance. Monotone chain computes the hull by sorting the points lexicographically and building the lower and upper boundaries in one pass each, popping any point that would fail to create a counter-clockwise turn. Rotating calipers then walks two antipodal pointers around the hull, advancing whichever increases the separation and visiting every candidate diametral pair in linear time. Both functions accept either floating-point or integral coordinates, and use exact comparisons for integral points.

- `convex_hull(p)` returns the convex hull of points `p` in counter-clockwise order. Duplicate input points and collinear points along hull edges are omitted. To instead return the hull points in clockwise order, replace the cross product comparisons `<= 0` with `>= 0`.
- `diametral_pair(p)` returns the maximum-distance pair of points in `p`.

Overflow warning: `cross()` and the squared distance in `diametral_pair()` grow like the squared coordinate magnitude. With 32-bit `int` coordinates they overflow once coordinates exceed a few tens of thousands; use a 64-bit (`int64_t`) coordinate type for larger integer inputs.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of points.

**Space Complexity:**  $O(n)$  auxiliary for storage of the convex hull.

```
#include <algorithm>
#include <cmath>
#include <random>
#include <utility>
#include <vector>

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// Convex hull: exact for integer-coordinate points.
template<typename Pt>
std::vector<Pt> convex_hull(std::vector<Pt> p) {
    std::sort(p.begin(), p.end());
    auto same_point = [](const Pt &a, const Pt &b) { return !(a < b) && !(b < a); };
    p.erase(std::unique(p.begin(), p.end(), same_point), p.end());
    int n = static_cast<int>(p.size());
    if (n <= 1) {
        return p;
    }
    int k = 0;
    std::vector<Pt> res(2 * n);
    for (const Pt &q : p) {
        while (k >= 2 && cross(res[k - 1], q, res[k - 2]) <= 0) {
            k--;
        }
        res[k++] = q;
    }
    int t = k + 1;
    for (int i = n - 2; i >= 0; i--) {
        while (k >= t && cross(res[k - 1], p[i], res[k - 2]) <= 0) {
            k--;
        }
        res[k++] = p[i];
    }
    res.resize(k - 1);
    return res;
}

// Diametral pair: squared-distance comparisons are exact for integer-coordinate points.
template<typename Pt>
std::pair<Pt, Pt> diametral_pair(const std::vector<Pt> &p) {
    auto h = convex_hull(p);
    int m = static_cast<int>(h.size());
    if (m == 0) {
        return std::pair<Pt, Pt>{};
    }
    if (m == 1) {
        return std::pair<Pt, Pt>{h[0], h[0]};
    }
    if (m == 2) {
        return std::pair<Pt, Pt>{h[0], h[1]};
    }
}
```

```

}
int k = 1;
while (std::abs(cross(h[0], h[(k + 1) % m], h[m - 1])) > std::abs(cross(h[0], h[k], h[m - 1]))) {
    k++;
}
auto sqdist = [](const Pt &a, const Pt &b) {
    // Overflow risk for integer Pt: ~0(max_coord^2); use int64_t if necessary.
    auto dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
};
auto maxsq = sqdist(h[0], h[0]);
std::pair<Pt, Pt> res{h[0], h[0]};
for (int i = 0, j = k; i <= k && j < m; i++) {
    auto d = sqdist(h[i], h[j]);
    if (d > maxsq) {
        maxsq = d;
        res = {h[i], h[j]};
    }
    while (j < m && std::abs(cross(h[(i + 1) % m], h[(j + 1) % m], h[i])) >
           std::abs(cross(h[(i + 1) % m], h[j], h[i]))) {
        d = sqdist(h[i], h[(j + 1) % m]);
        if (d > maxsq) {
            maxsq = d;
            res = {h[i], h[(j + 1) % m]};
        }
        j++;
    }
}
return res;
}

```

## Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &o) const { return x == o.x && y == o.y; }
    bool operator!=(const PointI &o) const { return !(*this == o); }
    bool operator<(const PointI &o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator>(const PointI &o) const { return o < *this; }
};

int main() {
    {
        vector<PointI> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
        mt19937 rng(1234567); // Fixed seed for reproducibility.
        shuffle(v.begin(), v.end(), rng);
        vector<PointI> h{{-1, 3}, {0, 0}, {2, 1}, {1, 3}};
        assert(convex_hull(v) == h);
    }
    {
        auto [p1, p2] = diametral_pair(vector<PointI>{{0, 0}, {3, 0}, {0, 3}, {1, 1}, {4, 4}});
        assert(p1 == PointI(0, 0) && p2 == PointI(4, 4));
    }
    {
        vector<PointI> v{{0, 0}, {4, 0}, {4, 4}, {0, 4}, {2, 2}};
        auto h = convex_hull(v);
        assert(h.size() == 4); // interior point (2,2) excluded
        auto [p1, p2] = diametral_pair(v);
        // diametral pair is exact
        assert(
            (p1 == PointI(0, 0) && p2 == PointI(4, 4)) || (p1 == PointI(4, 4) && p2 == PointI(0, 0)) ||

```



```

    (p1 == PointI(4, 0) && p2 == PointI(0, 4)) || (p1 == PointI(0, 4) && p2 == PointI(4, 0))
    );
}
{
    assert((convex_hull(vector<PointI>{{2, 3}, {2, 3}, {2, 3}}) == vector<PointI>{{2, 3}}));
}
return 0;
}

```

### 7.3.4 Convex Polygon Queries

Given a convex polygon in counter-clockwise order, answer containment and tangent/intersection queries. Point containment is logarithmic by splitting the polygon into triangles sharing `poly[0]`. The line and tangent helpers below are written as simple linear scans; this is shorter and less assumption-heavy than the classic logarithmic versions, while still documenting the exact query semantics in one place.

- `point_in_convex_polygon(poly, p, include_boundary = true)` returns whether point `p` lies inside the convex polygon `poly`. The polygon must be in counter-clockwise order, with no repeated final vertex. Points on the boundary count as inside unless `include_boundary` is set to `false`.
- `line_convex_polygon_intersection(poly, a, b)` describes where the infinite directed line through `a`  $\rightarrow$  `b` hits the polygon:  $(-1, -1)$  for no intersection,  $(i, -1)$  for touching vertex  $i$ ,  $(i, i)$  for containing side  $i \rightarrow i + 1$ , or  $(i, j)$  for crossing sides  $i \rightarrow i + 1$  and  $j \rightarrow j + 1$ , with side indices taken modulo the number of vertices. The polygon must have at least three vertices, and `a` and `b` must differ. When the line crosses two sides, their order is determined using a floating-point line parameter.
- `convex_polygon_tangents(poly, p)` returns the two tangent vertex indices from an outside point `p` as `(left, right)`. The left tangent has all polygon vertices on or to the left of the directed line `p`  $\rightarrow$  `poly[left]`; the right tangent is analogous for the right side.

Overflow warning: For integer-coordinate inputs, all orientation tests are exact provided the cross products do not overflow.

#### Time Complexity:

- $O(\log n)$  per call to `point_in_convex_polygon()`, where  $n$  is the number of polygon vertices.
- $O(n)$  per call to `line_convex_polygon_intersection` and `convex_polygon_tangents`.

Space Complexity:  $O(1)$  auxiliary.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <utility>
#include <vector>

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<typename T>
int sgn(const T &x) {
    return (T{0} < x) - (x < T{0});
}

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    return cross(a, b, p) == 0 && std::min(a.x, b.x) <= p.x && p.x <= std::max(a.x, b.x) &&
        std::min(a.y, b.y) <= p.y && p.y <= std::max(a.y, b.y);
}

```

```

template<typename Pt>
bool point_in_convex_polygon(
    const std::vector<Pt> &poly, const Pt &p, bool include_boundary = true
) {
    int n = static_cast<int>(poly.size());
    if (n == 0) {
        return false;
    }
    if (n == 1) {
        return include_boundary && p.x == poly[0].x && p.y == poly[0].y;
    }
    if (n == 2) {
        return include_boundary && point_on_segment(p, poly[0], poly[1]);
    }
    int left = sgn(cross(poly[1], p, poly[0]));
    int right = sgn(cross(poly[n - 1], p, poly[0]));
    if (left < 0 || right > 0) {
        return false;
    }
    if (left == 0) {
        return include_boundary && point_on_segment(p, poly[0], poly[1]);
    }
    if (right == 0) {
        return include_boundary && point_on_segment(p, poly[0], poly[n - 1]);
    }
    int lo = 1, hi = n - 1;
    while (hi - lo > 1) {
        int mid = lo + (hi - lo) / 2;
        if (cross(poly[mid], p, poly[0]) >= 0) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    int s = sgn(cross(poly[lo + 1], p, poly[lo]));
    return include_boundary ? s >= 0 : s > 0;
}

template<typename Pt>
std::pair<int, int> line_convex_polygon_intersection(
    const std::vector<Pt> &poly, const Pt &a, const Pt &b
) {
    int n = static_cast<int>(poly.size());
    assert(n >= 3 && (a.x != b.x || a.y != b.y));
    std::vector<int> on, cross_edges;
    bool has_pos = false, has_neg = false;
    for (int i = 0; i < n; i++) {
        int s = sgn(cross(b, poly[i], a));
        has_pos |= s > 0;
        has_neg |= s < 0;
        if (s == 0) {
            on.push_back(i);
        }
    }
    if (!has_pos && !has_neg) {
        return n >= 2 ? std::make_pair(0, 0) : std::make_pair(-1, -1);
    }
    if (!(has_pos && has_neg)) {
        for (int i = 0; i < static_cast<int>(on.size()); i++) {
            int j = (i + 1) % static_cast<int>(on.size());
            if ((on[i] + 1) % n == on[j]) {
                return {on[i], on[j]};
            }
        }
    }
    return on.empty() ? std::make_pair(-1, -1) : std::make_pair(on[0], -1);
}

for (int i = 0; i < n; i++) {
    int j = (i + 1) % n;

```

```

    int si = sgn(cross(b, poly[i], a)), sj = sgn(cross(b, poly[j], a));
    if (si == 0 || si * sj < 0) {
        cross_edges.push_back(i);
    }
}
if (cross_edges.size() == 1) {
    return {cross_edges[0], -1};
}
auto line_parameter = [&](int i) {
    int j = (i + 1) % n;
    auto ex = poly[j].x - poly[i].x, ey = poly[j].y - poly[i].y;
    auto ax = a.x, ay = a.y, dx = b.x - a.x, dy = b.y - a.y;
    auto det = dx * ey - dy * ex;
    return static_cast<double>((poly[i].x - ax) * ey - (poly[i].y - ay) * ex) / det;
};
if (line_parameter(cross_edges[1]) < line_parameter(cross_edges[0])) {
    std::swap(cross_edges[0], cross_edges[1]);
}
return {cross_edges[0], cross_edges[1]};
}

template<typename Pt>
std::pair<int, int> convex_polygon_tangents(const std::vector<Pt> &poly, const Pt &p) {
    int n = static_cast<int>(poly.size());
    std::pair<int, int> res{-1, -1};
    for (int i = 0; i < n; i++) {
        int prev = (i + n - 1) % n, next = (i + 1) % n;
        int s1 = sgn(cross(poly[i], poly[prev], p));
        int s2 = sgn(cross(poly[i], poly[next], p));
        bool left = s1 >= 0 && s2 >= 0;
        bool right = s1 <= 0 && s2 <= 0;
        if (left) {
            res.first = i;
        }
        if (right) {
            res.second = i;
        }
    }
    return res;
}

```

## Example Usage

```

#include <algorithm>
using namespace std;

struct PointI {
    int64_t x, y;
    PointI(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
};

int main() {
    vector<PointI> square{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
    assert(point_in_convex_polygon(square, PointI(2, 2)));
    assert(point_in_convex_polygon(square, PointI(4, 2)));
    assert(!point_in_convex_polygon(square, PointI(4, 2), false));
    assert(!point_in_convex_polygon(square, PointI(5, 2)));

    vector<PointI> pentagon{{0, 0}, {3, 0}, {5, 2}, {2, 5}, {-1, 3}};
    assert(point_in_convex_polygon(pentagon, PointI(2, 3)));
    assert(point_in_convex_polygon(pentagon, PointI(0, 0)));
    assert(!point_in_convex_polygon(pentagon, PointI(0, 0), false));
    assert(!point_in_convex_polygon(pentagon, PointI(5, 4)));

    assert(line_convex_polygon_intersection(square, PointI(-1, 2), PointI(5, 2)) == make_pair(3, 1));
    assert(line_convex_polygon_intersection(square, PointI(0, 0), PointI(4, 0)) == make_pair(0, 0));
}

```

```

assert(line_convex_polygon_intersection(square, PointI(5, 0), PointI(5, 4)) == make_pair(-1, -1));

// From the point to the right of the square, the upper-right and lower-right vertices are
// tangent.
assert(convex_polygon_tangents(square, PointI(6, 2)) == make_pair(2, 1));
return 0;
}

```

### 7.3.5 Minimum Enclosing Circle

Given a set of points in two dimensions, finds the unique circle of minimum radius (equivalently, minimum area) containing all points.

This implementation uses Welzl's randomized incremental algorithm. The points are shuffled and processed one at a time while maintaining the current minimum enclosing circle. Whenever a point lies outside the current circle, the circle is rebuilt with that point constrained to lie on the boundary. The final circle is determined by at most three boundary points.

- `min_enclosing_circle(p)` returns the minimum enclosing circle for the points in `p`. The point type may be any type exposing numeric `.x` and `.y` members. The returned circle always uses `double` coordinates, so integer-coordinate inputs are accepted.

**Time Complexity:**  $O(n)$  expected time per call, where  $n$  is the number of points, or  $O(n^3)$  in the worst case for a particular shuffle order.

**Space Complexity:**  $O(n)$  auxiliary for the shuffled copy of the points.

```

#include <algorithm>
#include <chrono>
#include <cmath>
#include <random>
#include <stdexcept>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

double sqnorm(double x, double y) {
    return x * x + y * y;
}

// SFINAE guard: valid only when Pt exposes numeric .x/.y members. This keeps the templated point
// constructors from hijacking calls like Circle(double, double, double).
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

```

```

struct Circle {
    double h, k, r;

    Circle() : h(0), k(0), r(0) {}
    Circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = std::hypot(a.x - h, a.y - k);
    }

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b, const Pt &c) {
        double an = sqnorm(b.x - c.x, b.y - c.y);
        double bn = sqnorm(a.x - c.x, a.y - c.y);
        double cn = sqnorm(a.x - b.x, a.y - b.y);
        double wa = an * (bn + cn - an), wb = bn * (an + cn - bn), wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (wa * a.x + wb * b.x + wc * c.x) / w;
        k = (wa * a.y + wb * b.y + wc * c.y) / w;
        r = std::hypot(a.x - h, a.y - k);
    }

    template<typename Pt, typename = if_point<Pt>>
    bool contains(const Pt &p) const {
        return LE(sqnorm(p.x - h, p.y - k), r * r);
    }
};

// Input points can be any type with .x and .y fields; Circle output is always double.
template<typename Pt>
Circle min_enclosing_circle(std::vector<Pt> p) {
    if (p.empty()) {
        return Circle(0, 0, 0);
    }
    if (p.size() == 1) {
        return Circle(p[0].x, p[0].y, 0);
    }
    static std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
    std::shuffle(p.begin(), p.end(), rng);
    Circle res(p[0], p[1]);
    for (int i = 2; i < static_cast<int>(p.size()); i++) {
        if (res.contains(p[i])) {
            continue;
        }
        res = Circle(p[0], p[i]);
        for (int j = 1; j < i; j++) {
            if (res.contains(p[j])) {
                continue;
            }
            res = Circle(p[i], p[j]);
            for (int k = 0; k < j; k++) {
                if (!res.contains(p[k])) {
                    res = Circle(p[i], p[j], p[k]);
                }
            }
        }
    }
    return res;
}

```

## Example Usage

```
#include <cassert>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    vector<Point> empty;
    Circle none = min_enclosing_circle(empty);
    assert(EQ(none.h, 0) && EQ(none.k, 0) && EQ(none.r, 0));

    vector<Point> one{{2, 3}};
    Circle single = min_enclosing_circle(one);
    assert(EQ(single.h, 2) && EQ(single.k, 3) && EQ(single.r, 0));

    vector<Point> two{{0, 0}, {4, 0}};
    Circle diameter = min_enclosing_circle(two);
    assert(EQ(diameter.h, 2) && EQ(diameter.k, 0) && EQ(diameter.r, 2));

    vector<Point> v{{0, 0}, {0, 1}, {1, 0}, {1, 1}};
    Circle res = min_enclosing_circle(v);
    assert(EQ(res.h, 0.5) && EQ(res.k, 0.5) && EQ(res.r, 1 / sqrt(2)));

    // Integer-coordinate input: Circle output is always double.
    vector<PointI> iv{{0, 0}, {0, 2}, {2, 0}, {2, 2}};
    Circle ic = min_enclosing_circle(iv);
    assert(EQ(ic.h, 1.0) && EQ(ic.k, 1.0));
    return 0;
}
```

### 7.3.6 Closest Pair

Given a list of points in two dimensions, finds the closest pair using a divide-and-conquer algorithm. The points are split in half by  $x$ -coordinate and each half is solved recursively; the combining step then only needs to examine points within the best distance so far of the dividing line, where each point in this strip is compared to a constant number of  $y$ -ordered neighbors.

- `closest_pair(p, &res)` returns the minimum squared distance between any two points in `p`. If `res` is non-null, one closest pair is stored there in lexicographic order. With fewer than two points, the maximum value of the squared-distance type is returned and `res` is unchanged. The point type may be any type exposing numeric `.x` and `.y` members and a lexicographic `operator<`. The distance preserves the coordinate arithmetic type, and the returned pair preserves the point type. For integer coordinates, the distance is exact provided intermediate products do not overflow.

Overflow warning: squared distances grow like the square of the coordinate magnitude. For integer point types, use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) when coordinates may exceed a few tens of thousands.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of points.

**Space Complexity:**

- $O(n)$  auxiliary heap space.
- $O(\log n)$  auxiliary stack space.

```

#include <algorithm>
#include <cmath>
#include <iterator>
#include <limits>
#include <utility>
#include <vector>

template<typename Pt>
auto sqdist(const Pt &a, const Pt &b) {
    auto dx = b.x - a.x, dy = b.y - a.y;
    return dx * dx + dy * dy; // Overflow warning.
}

template<typename It, typename T, typename Pt = typename std::iterator_traits<It>::value_type>
void closest_pair_rec(It lo, It hi, std::vector<Pt> &tmp, T &best, std::pair<Pt, Pt> *res) {
    int n = hi - lo;
    auto by_y = [](const Pt &a, const Pt &b) { return a.y != b.y ? a.y < b.y : a.x < b.x; };
    if (n <= 3) {
        for (It i = lo; i != hi; ++i) {
            for (It j = i + 1; j != hi; ++j) {
                T d = sqdist(*i, *j);
                if (d < best) {
                    best = d;
                    if (res != nullptr) {
                        *res = std::minmax(*i, *j);
                    }
                }
            }
        }
        std::sort(lo, hi, by_y);
        return;
    }
    It mid = lo + n / 2;
    auto midx = mid->x;
    closest_pair_rec(lo, mid, tmp, best, res);
    closest_pair_rec(mid, hi, tmp, best, res);
    // Each half is now y-sorted, so merge them before examining the center strip.
    tmp.clear();
    std::merge(lo, mid, mid, hi, std::back_inserter(tmp), by_y);
    std::move(tmp.begin(), tmp.end(), lo);
    tmp.clear();
    for (It it = lo; it != hi; ++it) {
        auto dx = it->x - midx;
        if (dx * dx < best) {
            tmp.push_back(*it);
        }
    }
    for (int i = 0; i < static_cast<int>(tmp.size()); i++) {
        for (int j = i + 1; j < static_cast<int>(tmp.size()); j++) {
            auto dy = tmp[j].y - tmp[i].y;
            if (dy * dy >= best) {
                break;
            }
            T d = sqdist(tmp[i], tmp[j]);
            if (d < best) {
                best = d;
                if (res != nullptr) {
                    *res = std::minmax(tmp[i], tmp[j]);
                }
            }
        }
    }
    tmp.clear();
}

template<typename Pt>
auto closest_pair(std::vector<Pt> p, std::pair<Pt, Pt> *res = nullptr) {
    using T = decltype(sqdist(p[0], p[0]));

```

```

T best = std::numeric_limits<T>::max();
std::sort(p.begin(), p.end(), [](const Pt &a, const Pt &b) {
    return a.x != b.x ? a.x < b.x : a.y < b.y;
});
std::vector<Pt> tmp;
tmp.reserve(p.size());
closest_pair_rec(p.begin(), p.end(), tmp, best, res);
return best;
}

```

### Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator<(const PointI &o) const { return x != o.x ? x < o.x : y < o.y; }
};

int main() {
    vector<Point> v{{2, 3}, {12, 30}, {40, 50}, {5, 1}, {12, 10}, {3, 4}};
    pair<Point, Point> res;
    assert(EQ(closest_pair(v, &res), 2));
    auto [p1, p2] = res;
    assert(p1 == Point(2, 3) && p2 == Point(3, 4));

    // Integer-coordinate input: exact pair selection, int squared distance returned.
    vector<PointI> iv{{0, 0}, {10, 10}, {3, 4}};
    pair<PointI, PointI> ires;
    assert(closest_pair(iv, &ires) == 25);
    auto [i1, i2] = ires;
    assert(i1 == PointI(0, 0) && i2 == PointI(3, 4));
    return 0;
}

```

### 7.3.7 Maximum Collinear Points

Given a set of points with integer coordinates, compute the maximum number of points that lie on one line. For every anchor point, normalize the direction vector to every other point by dividing by `gcd(abs(dx), abs(dy))` and choosing a canonical sign; equal normalized directions from the same anchor are collinear with it.

- `max_collinear_points(p)` returns the largest number of collinear points. Duplicate points are counted as separate input points.

Overflow warning: coordinate differences and their absolute values must fit in `int64_t`.

**Time Complexity:**  $O(n^2 \log n)$  per call, where  $n$  is the number of points.



**Space Complexity:**  $O(n)$  auxiliary.

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdint>
#include <map>
#include <numeric>
#include <utility>
#include <vector>

struct Point {
    int64_t x, y;
    Point(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
};

int max_collinear_points(const std::vector<Point> &p) {
    int n = static_cast<int>(p.size()), ans = std::min(n, 1);
    for (int i = 0; i < n; i++) {
        std::map<std::pair<int64_t, int64_t>, int> cnt;
        int duplicates = 0, best = 0;
        for (int j = 0; j < n; j++) {
            if (i == j) {
                continue;
            }
            int64_t dx = p[j].x - p[i].x, dy = p[j].y - p[i].y; // Overflow warning.
            if (dx == 0 && dy == 0) {
                duplicates++;
                continue;
            }
            int64_t g = std::gcd(std::abs(dx), std::abs(dy));
            dx /= g;
            dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) {
                dx = -dx;
                dy = -dy;
            }
            best = std::max(best, ++cnt[{dx, dy}]);
        }
        ans = std::max(ans, best + duplicates + 1);
    }
    return ans;
}
```

### Example Usage

```
using namespace std;

int main() {
    vector<Point> p{{0, 0}, {1, 1}, {2, 2}, {3, 5}, {4, 6}, {3, 3}};
    assert(max_collinear_points(p) == 4);
    p.emplace_back(1, 1);
    assert(max_collinear_points(p) == 5);
    assert(max_collinear_points({}) == 0);
    assert(max_collinear_points({Point(2, 3)}) == 1);
    return 0;
}
```

### 7.3.8 Rectangle Coverage

Given weighted axis-aligned rectangles, iterate over every elementary cell induced by their coordinate-compressed boundaries. Within each such cell, the accumulated weight is constant, so the callback can compute rectangle union area, exactly- $k$  coverage area, threshold area, maximum overlap, weighted integrals, coverage histograms, or other offline coverage statistics.

Rectangles are half-open,  $[x_1, x_2) \times [y_1, y_2)$ , so adjacent rectangles sharing only an edge do not double-count any area. This also matches the difference-array update: add at the lower boundaries and subtract at the first coordinate after the rectangle.

- `for_each_rect_cell(rects, visit)` calls `visit(x1, y1, x2, y2, weight)` once for each positive-area compressed cell inside the bounding box of the rectangles, where `weight` is the sum of weights covering that cell. Cells are visited in increasing `x1`, then increasing `y1`. Each rectangle must satisfy `x1 < x2` and `y1 < y2`.

Enumerating cells costs  $O(n^2)$  time and memory, which is the price of answering any statistic at all rather than one chosen in advance. When only the union area or perimeter is wanted, the sweep of section 7.3.9 computes it in  $O(n \log n)$  time and  $O(n)$  memory, which is the only option once the rectangle count reaches the thousands.

Overflow warning: Coordinate differences, cell areas, and accumulated weights must fit in `int64_t`.

**Time Complexity:**  $O(n^2 + n \log n)$  per call, where  $n$  is the number of rectangles.

**Space Complexity:**  $O(n^2)$  auxiliary.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

struct Rect {
    int64_t x1, y1, x2, y2, w = 1;
};

template<typename Fn>
void for_each_rect_cell(const std::vector<Rect> &rects, Fn visit) {
    std::vector<int64_t> xs, ys;
    for (const Rect &r : rects) {
        assert(r.x1 < r.x2 && r.y1 < r.y2);
        xs.push_back(r.x1);
        xs.push_back(r.x2);
        ys.push_back(r.y1);
        ys.push_back(r.y2);
    }
    std::sort(xs.begin(), xs.end());
    std::sort(ys.begin(), ys.end());
    xs.erase(std::unique(xs.begin(), xs.end()), xs.end());
    ys.erase(std::unique(ys.begin(), ys.end()), ys.end());
    int X = static_cast<int>(xs.size()), Y = static_cast<int>(ys.size());
    std::vector<std::vector<int64_t>> diff(X + 1, std::vector<int64_t>(Y + 1));
    auto index = [](const std::vector<int64_t> &v, int64_t x) {
        return static_cast<int>(std::lower_bound(v.begin(), v.end(), x) - v.begin());
    };
    for (const Rect &r : rects) {
        int x1 = index(xs, r.x1), x2 = index(xs, r.x2);
        int y1 = index(ys, r.y1), y2 = index(ys, r.y2);
        diff[x1][y1] += r.w; // Overflow warning.
        diff[x2][y1] -= r.w;
        diff[x1][y2] -= r.w;
        diff[x2][y2] += r.w;
    }
    for (int x = 0; x < X; x++) {
        for (int y = 0; y < Y; y++) {
            diff[x][y] += (x > 0) ? diff[x - 1][y] : 0;
            diff[x][y] += (y > 0) ? diff[x][y - 1] : 0;
            diff[x][y] -= (x > 0 && y > 0) ? diff[x - 1][y - 1] : 0;
        }
    }
}
```

```

        if (x + 1 < X && y + 1 < Y) {
            visit(xs[x], ys[y], xs[x + 1], ys[y + 1], diff[x][y]);
        }
    }
}
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    auto rect_area_if = [](const vector<Rect> &rects, auto pred) {
        int64_t area = 0;
        for_each_rect_cell(rects, [&](int64_t x1, int64_t y1, int64_t x2, int64_t y2, int64_t w) {
            if (pred(w)) {
                area += (x2 - x1) * (y2 - y1); // Overflow warning.
            }
        });
        return area;
    };

    // Rectangles A=[0,2)x[0,2), B=[1,3)x[1,3), and C=[2,4)x[0,1) all with default weight 1.
    //
    // y=3 +---+---+---+---+
    //      |   | B | B |   |
    // y=2 +---+---+---+---+
    //      | A |A/B| B |   |
    // y=1 +---+---+---+---+
    //      | A | A | C | C |
    // y=0 +---+---+---+---+
    //      x=0  1  2  3  4
    vector<Rect> rects{{0, 0, 2, 2}, {1, 1, 3, 3}, {2, 0, 4, 1}};
    assert(rect_area_if(rects, [](int64_t w) { return w > 0; }) == 9);
    assert(rect_area_if(rects, [](int64_t w) { return w == 2; }) == 1);

    // Rectangles A=[0,4)x[0,3) of w=2, B=[2,6)x[1,4) of w=3, and C=[1,5)x[2,5) of w=1.
    //
    // y=5 +---+---+---+---+---+
    //      |   | 1 | 1 | 1 | 1 |   |
    // y=4 +---+---+---+---+---+
    //      |   | 1 | 4 | 4 | 4 | 3 |
    // y=3 +---+---+---+---+---+
    //      | 2 | 3 | 6 | 6 | 4 | 3 |
    // y=2 +---+---+---+---+---+
    //      | 2 | 2 | 5 | 5 | 3 | 3 |
    // y=1 +---+---+---+---+---+
    //      | 2 | 2 | 2 | 2 | 2 |   |
    // y=0 +---+---+---+---+---+
    //      x=0  1  2  3  4  5  6
    vector<Rect> weighted{{0, 0, 4, 3, 2}, {2, 1, 6, 4, 3}, {1, 2, 5, 5, 1}};
    assert(rect_area_if(weighted, [](int64_t w) { return w >= 3; }) == 13);
    assert(rect_area_if(weighted, [](int64_t w) { return w >= 5; }) == 4);

    int64_t max_weight = 0, integral = 0;
    for_each_rect_cell(weighted, [&](int64_t x1, int64_t y1, int64_t x2, int64_t y2, int64_t w) {
        max_weight = max(max_weight, w);
        integral += w * (x2 - x1) * (y2 - y1); // Overflow warning.
    });
    assert(max_weight == 6);
    assert(integral == 12 * 2 + 12 * 3 + 12);
    return 0;
}

```

### 7.3.9 Rectangle Union

Given axis-aligned rectangles that may overlap arbitrarily, measure their union. Klee's algorithm sweeps a vertical line left to right, maintaining how much of that line the rectangles currently cover. That covered length is constant between consecutive events, so each strip contributes the length times its width, and every other measure below is another aggregate of the same cross section.

A segment tree over the compressed  $y$  coordinates holds the state, storing at each node the number of intervals covering its whole span and the length of that span which is covered. It never pushes anything down: a node's count refers only to intervals covering it entirely, so a node with a positive count reports its full span while one with a zero count defers to its children. Every addition is matched by a later removal, so counts never go negative and no lazy tag is needed.

Three further aggregates ride along on the same nodes. Doubly covered length shifts the recurrence by one, a node covered twice reporting its whole span and a node covered once reporting whatever its children cover at all. The weighted maximum is the largest root-to-leaf sum of weights, so each node reports its own weight plus the larger of its children's; left at the default weight of one per rectangle, that is simply the greatest number of rectangles covering a point. The perimeter needs the number of maximal covered runs, plus whether a node's first and last units are covered so that a run crossing a boundary is not counted twice: vertical edges are then the change in covered length at each event, and horizontal edges are twice the run count times the strip width.

Rectangles are half-open,  $[x_1, x_2) \times [y_1, y_2)$ , so adjacent rectangles sharing only an edge neither double-count any area nor leave a seam.

- `rect_union(rects)` returns a `UnionMeasure` holding the `area` covered at least once, the `overlap_area` covered at least twice, the `perimeter` of the outline including any enclosed hole, and `max_weight`, the greatest weighted sum at any point. Subtracting the second from the first leaves the area covered exactly once. Each rectangle must satisfy `x1 < x2` and `y1 < y2`.

Weights change only the maximum, since the three geometric measures count coverage rather than payload, and weighted area needs no sweep at all: it is  $\sum w_i$  times area, one loop over the input. Area covered at least  $k$  times instead needs a vector of  $k$  lengths per node; for that or any aggregate that does not decompose this way, the compression of section 7.3.8 enumerates every elementary cell at  $O(n^2)$ .

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of rectangles.

**Space Complexity:**  $O(n)$  auxiliary.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <tuple>
#include <utility>
#include <vector>

struct Rect {
    int64_t x1, y1, x2, y2, w = 1;
};

struct UnionMeasure {
    int64_t area = 0, overlap_area = 0, perimeter = 0, max_weight = 0;
};

// The cross section of the sweep: over the currently covered y intervals, the length covered at
// least once and at least twice, the number of maximal covered runs, and the greatest weight at any
// point. Each is its own recurrence, so each is pulled up separately.
class CoverTree {
    std::vector<int64_t> ys; // Compressed coordinates; leaf i spans [ys[i], ys[i + 1]).
    int len;
    std::vector<int> count, runs;
    std::vector<int64_t> covered, covered_twice, weight, heaviest;
    std::vector<char> covers_lo, covers_hi;
```

```

// Length covered at least once and at least twice. The second reads the first one level down, so
// covering at least k times extends this chain by one array per level.
void pull_coverage(int i, int lo, int hi) {
    int left = 2 * i + 1, right = left + 1;
    if (count[i] > 0) {
        covered[i] = ys[hi + 1] - ys[lo];
        // One cover here turns anything already covered below into a double cover.
        covered_twice[i] =
            count[i] > 1 ? covered[i] : (lo == hi ? 0 : covered[left] + covered[right]);
    } else if (lo == hi) {
        covered[i] = covered_twice[i] = 0;
    } else {
        covered[i] = covered[left] + covered[right];
        covered_twice[i] = covered_twice[left] + covered_twice[right];
    }
}

// Number of maximal covered runs, with the flags that keep a run spanning a boundary from being
// counted in both children.
void pull_runs(int i, int lo, int hi) {
    int left = 2 * i + 1, right = left + 1;
    if (count[i] > 0) {
        runs[i] = covers_lo[i] = covers_hi[i] = 1;
    } else if (lo == hi) {
        runs[i] = covers_lo[i] = covers_hi[i] = 0;
    } else {
        runs[i] = runs[left] + runs[right] - (covers_hi[left] && covers_lo[right] ? 1 : 0);
        covers_lo[i] = covers_lo[left];
        covers_hi[i] = covers_hi[right];
    }
}

// The weight over an interval is the sum of the weights above it, so the heaviest point is the
// largest root-to-leaf sum. This one ignores the cover count entirely.
void pull_weight(int i, int lo, int hi) {
    heaviest[i] = weight[i] + (lo == hi ? 0 : std::max(heaviest[2 * i + 1], heaviest[2 * i + 2]));
}

public:
explicit CoverTree(std::vector<int64_t> coords)
    : ys(std::move(coords)),
      len(static_cast<int>(ys.size()) - 1),
      count(4 * len),
      runs(4 * len),
      covered(4 * len),
      covered_twice(4 * len),
      weight(4 * len),
      heaviest(4 * len),
      covers_lo(4 * len),
      covers_hi(4 * len) {
    assert(len > 0);
}

// Adds delta covers of weight w to every elementary interval in [lo, hi]; a removal negates both.
void update(int lo, int hi, int delta, int64_t w) {
    auto rec = [&](auto &&rec, int i, int l, int h) {
        if (hi < l || h < lo) {
            return;
        }
        if (lo <= l && h <= hi) {
            count[i] += delta;
            weight[i] += delta * w;
        } else {
            int mid = l + (h - l) / 2;
            rec(rec, 2 * i + 1, l, mid);
            rec(rec, 2 * i + 2, mid + 1, h);
        }
    };
    pull_coverage(i, l, h);
}

```

```

    pull_runs(i, l, h);
    pull_weight(i, l, h);
};
rec(rec, 0, 0, len - 1);
}

int64_t covered_length() const { return covered[0]; }
int64_t twice_covered_length() const { return covered_twice[0]; }
int run_count() const { return runs[0]; }
int64_t heaviest_weight() const { return heaviest[0]; }
};

UnionMeasure rect_union(const std::vector<Rect> &rects) {
    if (rects.empty()) {
        return {};
    }
    std::vector<int64_t> ys;
    for (const Rect &r : rects) {
        assert(r.x1 < r.x2 && r.y1 < r.y2);
        ys.push_back(r.y1);
        ys.push_back(r.y2);
    }
    std::sort(ys.begin(), ys.end());
    ys.erase(std::unique(ys.begin(), ys.end()), ys.end());
    auto index = [&](int64_t y) {
        return static_cast<int>(std::lower_bound(ys.begin(), ys.end(), y) - ys.begin());
    };
    std::vector<std::tuple<int64_t, int, int, int, int64_t>> events; // (x, lo, hi, delta, weight)
    for (const Rect &r : rects) {
        events.emplace_back(r.x1, index(r.y1), index(r.y2) - 1, 1, r.w);
        events.emplace_back(r.x2, index(r.y1), index(r.y2) - 1, -1, r.w);
    }
    std::sort(events.begin(), events.end());
    CoverTree tree(std::move(ys));
    UnionMeasure res;
    int64_t width = 0, doubled = 0, strips = 0, prev_x = 0;
    for (size_t i = 0; i < events.size(); i) {
        int64_t x = std::get<0>(events[i]);
        if (i > 0) { // The strip since the previous event has a constant cross-section.
            res.area += width * (x - prev_x);
            res.overlap_area += doubled * (x - prev_x);
            res.perimeter += 2 * strips * (x - prev_x);
        }
        while (i < events.size() && std::get<0>(events[i]) == x) {
            auto [ex, tgt_lo, tgt_hi, delta, w] = events[i++];
            tree.update(tgt_lo, tgt_hi, delta, w);
        }
        // Length that appeared or vanished at this x must be bounded by vertical edges.
        res.perimeter += std::llabs(tree.covered_length() - width);
        res.max_weight = std::max(res.max_weight, tree.heaviest_weight());
        width = tree.covered_length();
        doubled = tree.twice_covered_length();
        strips = tree.run_count();
        prev_x = x;
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(rect_union({}).area == 0 && rect_union({}).perimeter == 0);
}

```

```

UnionMeasure one = rect_union({{0, 0, 2, 3}});
assert(one.area == 6 && one.perimeter == 10);
assert(one.overlap_area == 0 && one.max_weight == 1);

// Disjoint rectangles simply add up, and nothing is covered twice.
UnionMeasure apart = rect_union({{0, 0, 1, 1}, {2, 0, 3, 1}});
assert(apart.area == 2 && apart.perimeter == 8);
assert(apart.overlap_area == 0 && apart.max_weight == 1);

// Two squares meeting in a unit square, whose union outlines an L-shaped hexagon.
UnionMeasure pair = rect_union({{0, 0, 2, 2}, {1, 1, 3, 3}});
assert(pair.area == 7 && pair.perimeter == 12);
assert(pair.overlap_area == 1 && pair.max_weight == 2);
assert(pair.area - pair.overlap_area == 6); // Area belonging to exactly one square.

// A rectangle nested inside another adds no area, but doubles the cover where it sits.
UnionMeasure nested = rect_union({{0, 0, 10, 10}, {2, 2, 4, 4}});
assert(nested.area == 100 && nested.perimeter == 40);
assert(nested.overlap_area == 4 && nested.max_weight == 2);

// Four rectangles enclosing a hole, overlapping only at the four corners.
UnionMeasure ring = rect_union({{0, 0, 3, 1}, {0, 2, 3, 3}, {0, 0, 1, 3}, {2, 0, 3, 3}});
assert(ring.area == 8);
assert(ring.perimeter == 16); // 12 around the outside and 4 around the unit hole.
assert(ring.overlap_area == 4 && ring.max_weight == 2);

// Three rectangles stacked over one cell.
UnionMeasure stack = rect_union({{0, 0, 2, 2}, {1, 0, 3, 2}, {1, 1, 3, 3}});
assert(stack.max_weight == 3 && stack.overlap_area == 3);
return 0;
}

```

### 7.3.10 Multiple Segment Intersection

Given a list of line segments in two dimensions, determine whether any pair of segments intersect using a sweep line algorithm. Endpoint events are processed from left to right while an ordered set maintains the segments currently crossing the sweep line, sorted by  $y$ ; only segments that become adjacent in this set need to be tested, since any leftmost intersection must involve a pair that is adjacent just before it occurs. The input type `Segment<Pt>` is templated on the point type, so endpoints may be integer (`PointI`) or floating-point (`Point` or `PointD`), but must support `operator<` which orders points lexicographically. The cross-product sign tests in `seg_intersection` are exact for integer endpoints, so intersection detection is exact.

This implementation counts endpoint contacts as intersections. See 7.2.3 for pairwise intersection predicates when that distinction matters.

- `find_intersecting_pair(segs)` returns the indices of one pair of intersecting segments, or `std::nullopt` if none exist. Constructing a `Segment` places its lexicographically smaller endpoint first, as required by the sweep.

Overflow warning: `seg_intersection` forms the usual quadratic cross products, but the  $y$ -ordering cross-multiplication (`ay * bdx`) is cubic in the coordinate magnitude. Use `int64_t` coordinates for modest integer inputs and a wider intermediate type when these cubic products may exceed it.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of segments.

**Space Complexity:**  $O(n)$  auxiliary, where  $n$  is the number of segments.

```

#include <algorithm>
#include <cassert>
#include <optional>
#include <set>
#include <tuple>
#include <utility>
#include <vector>

```

```

// Simplified detection-only version of seg_intersection() from 7.2.3 (exact for integral Pt).
template<typename Pt>
bool seg_intersection(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    auto cross = [](const Pt &p, const Pt &q, const Pt &r) {
        return (q.x - p.x) * (r.y - p.y) - (q.y - p.y) * (r.x - p.x);
    };
    auto c1 = cross(a, b, c), c2 = cross(a, b, d), c3 = cross(c, d, a), c4 = cross(c, d, b);
    if (c1 == 0 && c2 == 0 && c3 == 0 && c4 == 0) {
        Pt p = std::max(std::min(a, b), std::min(c, d));
        Pt q = std::min(std::max(a, b), std::max(c, d));
        return !(q < p);
    }
    auto straddles = [](auto x, auto y) { return (x <= 0 && 0 <= y) || (y <= 0 && 0 <= x); };
    return straddles(c1, c2) && straddles(c3, c4);
}

// The point type Pt must support operator< (used to canonicalize endpoint order).
template<typename Pt>
struct Segment {
    Pt p, q;
    Segment(const Pt &p, const Pt &q) : p(std::min(p, q)), q(std::max(p, q)) {}
};

template<typename Pt>
bool intersects(const Segment<Pt> &s1, const Segment<Pt> &s2) {
    return seg_intersection(s1.p, s1.q, s2.p, s2.q);
}

template<typename Pt>
std::optional<std::pair<int, int>> find_intersecting_pair(const std::vector<Segment<Pt>> &segs) {
    struct Event {
        Pt p;
        bool is_end;
        int seg;
    };
    std::vector<Event> e;
    for (int i = 0; i < static_cast<int>(segs.size()); i++) {
        e.push_back(Event{segs[i].p, false, i});
        e.push_back(Event{segs[i].q, true, i});
    }
    std::sort(e.begin(), e.end(), [](const Event &a, const Event &b) {
        return std::tie(a.p.x, a.is_end, a.p.y, a.seg) < std::tie(b.p.x, b.is_end, b.p.y, b.seg);
    });
    // Compare y-values at sweep coordinate x without division: y = (p.y*dx + dy*(x - p.x)) / dx.
    // Vertical segments use their lower endpoint.
    auto ycmp = [](const auto &a, const auto &b, auto x) {
        // Overflow risk for integer Pt: the final cross-multiply is ~O(max_coord^3).
        auto adx = a.q.x - a.p.x, bdx = b.q.x - b.p.x;
        auto ay = a.p.y * adx + (a.q.y - a.p.y) * (x - a.p.x);
        auto by = b.p.y * bdx + (b.q.y - b.p.y) * (x - b.p.x);
        if (adx == 0) {
            ay = a.p.y;
            adx = 1;
        }
        if (bdx == 0) {
            by = b.p.y;
            bdx = 1;
        }
        auto lhs = ay * bdx, rhs = by * adx;
        return (lhs < rhs) ? -1 : ((rhs < lhs) ? 1 : 0);
    };
    auto cmp = [&segs, ycmp](int a, int b) {
        if (a == b) {
            return false;
        }
        auto x = std::max(segs[a].p.x, segs[b].p.x);
        int order = ycmp(segs[a], segs[b], x);
    };
}

```



```

    return (order == 0) ? (a < b) : (order < 0);
};
using ActiveSet = std::set<int, decltype(cmp)>;
ActiveSet s(cmp);
std::vector<typename ActiveSet::iterator> position(segs.size());
auto result = [](int i, int j) { return std::pair{std::min(i, j), std::max(i, j)}; };
for (const auto &ev : e) {
    int seg = ev.seg;
    if (!ev.is_end) {
        auto it = s.insert(seg).first;
        position[seg] = it;
        auto next = it;
        if (++next != s.end() && intersects(segs[*next], segs[seg])) {
            return result(*next, seg);
        }
        if (it != s.begin() && intersects(segs[*--it], segs[seg])) {
            return result(*it, seg);
        }
    } else {
        auto it = position[seg];
        auto next = it;
        if (++next != s.end() && it != s.begin()) {
            auto prev = it;
            if (intersects(segs[*next], segs[*--prev])) {
                return result(*next, *prev);
            }
        }
        s.erase(it);
    }
}
return std::nullopt;
}

```

## Example Usage

```

#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
};

int main() {
    { // Double-coordinate segments.
        vector<Segment<Point>> v{
            Segment<Point>(Point(0, 0), Point(2, 2)), Segment<Point>(Point(3, 0), Point(0, -1)),
            Segment<Point>(Point(0, 2), Point(2, -2)), Segment<Point>(Point(0, 3), Point(9, 0))
        };
        assert((find_intersecting_pair(v) == pair{0, 2}));
    }
    { // Integer-coordinate segments: detection is exact.
        vector<Segment<PointI>> v{
            Segment<PointI>({0, 0}, {2, 2}), Segment<PointI>({3, 0}, {0, -1}),
            Segment<PointI>({0, 2}, {2, -2}), Segment<PointI>({0, 3}, {9, 0})
        };
        assert((find_intersecting_pair(v) == pair{0, 2}));

        const vector<Segment<PointI>> disjoint{

```

```

    Segment<PointI>({0, 0}, {1, 0}), Segment<PointI>({0, 5}, {1, 5})
};
assert(!find_intersecting_pair(disjoint));
const vector<Segment<PointI>> duplicate{
    Segment<PointI>({0, 0}, {2, 2}), Segment<PointI>({0, 0}, {2, 2})
};
assert((find_intersecting_pair(duplicate) == pair{0, 1}));
}
{ // Shared endpoints count as intersections.
    vector<Segment<PointI>> shared{
        Segment<PointI>({0, 0}, {2, 2}), Segment<PointI>({2, 2}, {4, 0})
    };
    assert((find_intersecting_pair(shared) == pair{0, 1}));
}
return 0;
}

```

### 7.3.11 Manhattan MST

Given points in the plane, produce a small set of candidate edges guaranteed to contain a minimum spanning tree under Manhattan distance  $|x_1 - x_2| + |y_1 - y_2|$ . The plane is swept in four rotations/reflections; in each sweep, dominance by  $x + y$  identifies the only nearby candidates that can matter. Run Kruskal on the returned edges to obtain the MST.

- `manhattan_mst_candidates(p)` returns  $O(n)$  candidate edges as tuples  $(\text{weight}, \text{u}, \text{v})$ , where  $n = \text{p.size()}$  and `u` and `v` are indices into `p`.
- `manhattan_mst_weight(p)` returns the MST weight by running Kruskal's algorithm on those candidate edges.

Overflow warning: the sweeps form coordinate sums, negations, and differences such as `x + y` and `dx + dy`. With `int64_t` coordinates, those intermediate values must fit in `int64_t`.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of points.

**Space Complexity:**  $O(n)$  auxiliary.

```

#include <algorithm>
#include <cassert>
#include <stdint>
#include <stdlib>
#include <map>
#include <numeric>
#include <tuple>
#include <utility>
#include <vector>

struct Point {
    int64_t x, y;
    Point(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
};

std::vector<std::tuple<int64_t, int, int>> manhattan_mst_candidates(std::vector<Point> p) {
    int n = static_cast<int>(p.size());
    std::vector<int> id(n);
    std::iota(id.begin(), id.end(), 0);
    std::vector<std::tuple<int64_t, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) {
        std::sort(id.begin(), id.end(), [&](int i, int j) {
            return p[i].x + p[i].y < p[j].x + p[j].y;
        });
        std::map<int64_t, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-p[i].y); it != sweep.end(); it++) {
                int j = it->second;
                int64_t dx = p[i].x - p[j].x, dy = p[i].y - p[j].y;
            }
        }
    }
}

```

```

        if (dy > dx) {
            break;
        }
        edges.emplace_back(dx + dy, i, j);
        it = sweep.erase(it);
    }
    sweep[-p[i].y] = i;
}
for (Point &q : p) {
    if (rot & 1) {
        q.x = -q.x;
    } else {
        std::swap(q.x, q.y);
    }
}
}
return edges;
}

struct DSU {
    std::vector<int> root, rank;

    explicit DSU(int n) : root(n), rank(n, 0) { std::iota(root.begin(), root.end(), 0); }
    int find(int u) { return root[u] == u ? u : root[u] = find(root[u]); }

    bool unite(int u, int v) {
        u = find(u);
        v = find(v);
        if (u == v) {
            return false;
        }
        if (rank[u] < rank[v]) {
            std::swap(u, v);
        }
        root[v] = u;
        if (rank[u] == rank[v]) {
            rank[u]++;
        }
        return true;
    }
};

int64_t manhattan_mst_weight(const std::vector<Point> &p) {
    auto edges = manhattan_mst_candidates(p);
    std::sort(edges.begin(), edges.end());
    DSU dsu(static_cast<int>(p.size()));
    int64_t res = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(u, v)) {
            res += w;
        }
    }
    return res;
}

```

## Example Usage

```

using namespace std;

int main() {
    assert(manhattan_mst_weight({Point(5, -7)}) == 0);
    assert(manhattan_mst_weight({Point(0, 0), Point(3, -4)}) == 7);

    vector<Point> p{{0, 0}, {2, 1}, {3, 4}, {-1, 2}, {5, 0}};
    assert(manhattan_mst_weight(p) == 14);
    auto edges = manhattan_mst_candidates(p);
}

```

```

assert(edges.size() <= 4 * p.size());
for (auto [w, u, v] : edges) {
    assert(0 <= u && u < static_cast<int>(p.size()));
    assert(0 <= v && v < static_cast<int>(p.size()));
    assert(w == abs(p[u].x - p[v].x) + abs(p[u].y - p[v].y));
}
return 0;
}

```

## 7.4 Advanced Planar Geometry

### 7.4.1 Convex Polygon Cut

Given a convex polygon and a directed line from `p` to `q`, clips the polygon to the closed left half-plane of that line. The polygon is walked edge by edge: vertices on the left side of the line are kept, and whenever an edge crosses the line, the intersection point is appended to the output.

- `convex_cut(lo, hi, p, q)` returns the portion of the polygon lying on or to the left of the directed line from `p` to `q`. The input range `[lo, hi)` must contain the vertices of a convex polygon in boundary order, either clockwise or counterclockwise. The returned polygon preserves that boundary order and uses `Point` with `double` coordinates because edge-line intersections are computed in floating point. The points `p` and `q` must differ.

Overflow warning: For integer-coordinate inputs, classification is exact only if the intermediate products do not overflow.

**Time Complexity:**  $O(n)$  per call, where  $n$  is the distance between `lo` and `hi`.

**Space Complexity:**  $O(n)$  for the returned polygon and  $O(1)$  auxiliary.

```

#include <cassert>
#include <cmath>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
};

template<typename PtA, typename PtB, typename PtO>
auto cross(const PtA &a, const PtB &b, const PtO &o) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

```

```

template<typename PtA, typename PtO, typename PtB>
int turn(const PtA &a, const PtO &o, const PtB &b) {
    auto c = cross(a, b, o);
    return LT(c, 0) ? 1 : (LT(0, c) ? -1 : 0);
}

template<typename PtA, typename PtB>
int line_intersection(
    const PtA &p1, const PtA &p2, const PtB &p3, const PtB &p4, Point *p = nullptr
) {
    double a1 = static_cast<double>(p2.y) - p1.y, b1 = static_cast<double>(p1.x) - p2.x;
    double c1 = -(static_cast<double>(p1.x) * p2.y - static_cast<double>(p2.x) * p1.y);
    double a2 = static_cast<double>(p4.y) - p3.y, b2 = static_cast<double>(p3.x) - p4.x;
    double c2 = -(static_cast<double>(p3.x) * p4.y - static_cast<double>(p4.x) * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    double det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (p != nullptr) {
        *p = Point(x / det, y / det);
    }
    return 0;
}

template<typename It, typename Pt>
std::vector<Point> convex_cut(It lo, It hi, const Pt &p, const Pt &q) {
    assert(!EQ(p.x, q.x) || !EQ(p.y, q.y));
    if (lo == hi) {
        return {};
    }
    std::vector<Point> res;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        Point pj(static_cast<double>(j->x), static_cast<double>(j->y));
        Point pi(static_cast<double>(i->x), static_cast<double>(i->y));
        int d1 = turn(q, p, *j), d2 = turn(q, p, *i);
        if (d1 <= 0) {
            res.push_back(pj);
        }
        if (d1 * d2 < 0) {
            Point r;
            line_intersection(p, q, *j, *i, &r);
            res.push_back(r);
        }
    }
    return res;
}

```

## Example Usage

```

using namespace std;

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

bool EQ(const vector<Point> &a, const vector<Point> &b) {
    return a.size() == b.size() &&
        equal(a.begin(), a.end(), b.begin(), [](const Point &p, const Point &q) {
            return EQ(p, q);
        });
}

int main() {
    {

```

```

vector<Point> v{{1, 3}, {2, 2}, {2, 1}, {0, 0}, {-1, 3}};
// Cut using the vertical line through (0, 0).
vector<Point> c{{-1, 3}, {0, 3}, {0, 0}};
assert(EQ(convex_cut(v.begin(), v.end(), Point(0, 0), Point(0, 1)), c));
}
{ // On a non-convex input, the result may be multiple disjoint polygons!
vector<Point> v{{0, 0}, {2, 2}, {0, 4}, {3, 4}, {3, 0}};
vector<Point> c{{1, 0}, {0, 0}, {1, 1}, {1, 3}, {0, 4}, {1, 4}};
assert(EQ(convex_cut(v.begin(), v.end(), Point(1, 0), Point(1, 4)), c));
}
{
vector<PointI> v{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
vector<Point> c{{0, 4}, {0, 0}, {2, 0}, {2, 4}};
assert(EQ(convex_cut(v.begin(), v.end(), PointI(2, 0), PointI(2, 4)), c));
}
return 0;
}

```

## 7.4.2 Half-Plane Intersection

Given a list of directed lines, compute the convex polygon contained in every left half-plane. Half-plane intersection sorts the lines by direction, keeps only the tightest representative among parallel lines, and maintains a deque of lines whose pairwise intersections form the current boundary. Directions are ordered by half-plane and cross-product signs, without epsilon comparisons, to satisfy `std::sort`'s strict ordering requirement. `EPS` is then used outside the comparator to group parallel directions and retain only the tightest half-plane in each group.

- `half_plane_intersection(planes)` returns the polygon cut out by the half-planes in counter-clockwise order. Each half-plane is represented by `HalfPlane(p, q)`, meaning the closed region to the left of the directed line  $p \rightarrow q$ . This implementation is intended for intersections that form a bounded polygon with positive area; empty, unbounded, or degenerate inputs may return an empty vector. Add explicit bounding-box half-planes when a bounded polygon is required. The two points defining each half-plane must differ.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of half-planes.

**Space Complexity:**  $O(n)$  auxiliary.

```

#include <algorithm>
#include <cmath>
#include <deque>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
}

```

```

    Point operator*(double k) const { return {x * k, y * k}; }
    double cross(const Point &p) const { return x * p.y - y * p.x; }
};

struct HalfPlane {
    Point p, dir;

    HalfPlane() = default;
    HalfPlane(Point p, Point q) : p(p), dir(q - p) {}

    bool out(const Point &q) const { return LT(dir.cross(q - p), 0); }

    Point intersect(const HalfPlane &h) const {
        double t = h.dir.cross(p - h.p) / dir.cross(h.dir);
        return p + dir * t;
    }

    bool operator<(const HalfPlane &h) const {
        auto half = [](const Point &d) { return d.y < 0 || (d.y == 0 && d.x < 0); };
        if (half(dir) != half(h.dir)) {
            return half(dir) < half(h.dir);
        }
        return dir.cross(h.dir) > 0;
    }
};

std::vector<Point> half_plane_intersection(std::vector<HalfPlane> planes) {
    std::sort(planes.begin(), planes.end());
    std::vector<HalfPlane> unique;
    for (const HalfPlane &h : planes) {
        if (!unique.empty() && EQ(unique.back().dir.cross(h.dir), 0)) {
            if (unique.back().dir.cross(h.p - unique.back().p) > 0) {
                unique.back() = h;
            }
            continue;
        }
        unique.push_back(h);
    }
    std::deque<HalfPlane> dq;
    auto bad_back = [](const HalfPlane &h) {
        return dq.size() >= 2 && h.out(dq[dq.size() - 2].intersect(dq.back()));
    };
    auto bad_front = [](const HalfPlane &h) {
        return dq.size() >= 2 && h.out(dq[0].intersect(dq[1]));
    };
    for (const HalfPlane &h : unique) {
        while (bad_back(h)) {
            dq.pop_back();
        }
        while (bad_front(h)) {
            dq.pop_front();
        }
        dq.push_back(h);
    }
    while (dq.size() >= 3 && dq.front().out(dq[dq.size() - 2].intersect(dq.back()))) {
        dq.pop_back();
    }
    while (dq.size() >= 3 && dq.back().out(dq[0].intersect(dq[1]))) {
        dq.pop_front();
    }
    if (dq.size() < 3) {
        return {};
    }
    std::vector<Point> res;
    for (int i = 0; i < static_cast<int>(dq.size()); i++) {
        if (EQ(dq[i].dir.cross(dq[(i + 1) % dq.size()].dir), 0)) {
            return {};
        }
    }
}

```

```

    res.push_back(dq[i].intersect(dq[(i + 1) % dq.size()]));
}
return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

double polygon_area(const vector<Point> &p) {
    double area = 0;
    for (int i = 0, j = static_cast<int>(p.size()) - 1; i < static_cast<int>(p.size()); j = i++) {
        area += p[j].cross(p[i]);
    }
    return fabs(area) / 2.0;
}

int main() {
    vector<HalfPlane> box{
        HalfPlane(Point(0, 0), Point(4, 0)),
        HalfPlane(Point(4, 0), Point(4, 4)),
        HalfPlane(Point(4, 4), Point(0, 4)),
        HalfPlane(Point(0, 4), Point(0, 0)),
    };
    auto square = half_plane_intersection(box);
    assert(square.size() == 4 && EQ(polygon_area(square), 16));

    auto tighter = box;
    tighter.emplace_back(Point(0, 1), Point(4, 1)); // y >= 1
    assert(EQ(polygon_area(half_plane_intersection(tighter)), 12));

    box.emplace_back(Point(2, 0), Point(2, 4)); // x <= 2
    auto rect = half_plane_intersection(box);
    assert(rect.size() == 4 && EQ(polygon_area(rect), 8));

    vector<HalfPlane> empty{
        HalfPlane(Point(0, 0), Point(1, 0)), // y >= 0
        HalfPlane(Point(1, 1), Point(0, 1)), // y <= 1
        HalfPlane(Point(0, 2), Point(1, 2)), // y >= 2
    };
    assert(half_plane_intersection(empty).empty());
    return 0;
}

```

### 7.4.3 Minkowski Sum

Given two convex polygons, compute their Minkowski sum: the set of all points  $a + b$  where  $a$  comes from the first polygon and  $b$  comes from the second. After rotating both polygons to start at their lowest-leftmost vertex, the boundary edges of the sum are obtained by merging the two cyclic edge-angle lists. Minkowski sums are useful for collision/translation problems, convex polygon distance checks, and diameter computations via  $P + (-P)$ .

- `minkowski_sum(a, b)` returns the convex polygon representing the sum. Inputs must be convex polygons in boundary order, clockwise or counter-clockwise, with no repeated final vertex. Collinear consecutive output vertices are removed.

**Time Complexity:**  $O(n + m)$  per call, where  $n$  and  $m$  are the polygon sizes.

**Space Complexity:**  $O(n + m)$  auxiliary.

```

#include <algorithm>
#include <cmath>

```



```

#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
    double cross(const Point &p) const { return x * p.y - y * p.x; }
};

std::vector<Point> normalize(std::vector<Point> p) {
    while (p.size() > 1 && EQ(p.front(), p.back())) {
        p.pop_back();
    }
    if (p.size() <= 1) {
        return p;
    }
    double area = 0;
    for (int i = 0, j = static_cast<int>(p.size()) - 1; i < static_cast<int>(p.size()); j = i++) {
        area += p[j].cross(p[i]);
    }
    if (area < 0) {
        std::reverse(p.begin(), p.end());
    }
    int start = 0;
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        if (p[i].y < p[start].y || (EQ(p[i].y, p[start].y) && p[i].x < p[start].x)) {
            start = i;
        }
    }
    std::rotate(p.begin(), p.begin() + start, p.end());
    return p;
}

std::vector<Point> remove_collinear(const std::vector<Point> &p) {
    std::vector<Point> res;
    for (const Point &q : p) {
        while (res.size() >= 2 && EQ((res.back() - res[res.size() - 2]).cross(q - res.back(), 0)) {
            res.pop_back();
        }
        if (res.empty() || !EQ(q, res.back())) {
            res.push_back(q);
        }
    }
    while (res.size() >= 3 && EQ((res.back() - res[res.size() - 2]).cross(res[0] - res.back(), 0)) {
        res.pop_back();
    }
    while (res.size() >= 3 && EQ((res[0] - res.back()).cross(res[1] - res[0], 0)) {
        res.erase(res.begin());
    }
    return res;
}

std::vector<Point> minkowski_sum(std::vector<Point> a, std::vector<Point> b) {
    a = normalize(std::move(a));
    b = normalize(std::move(b));

```

```

    if (a.empty() || b.empty()) {
        return {};
    }
    if (a.size() == 1) {
        for (Point &p : b) {
            p = p + a[0];
        }
        return b;
    }
    if (b.size() == 1) {
        for (Point &p : a) {
            p = p + b[0];
        }
        return a;
    }
    int n = static_cast<int>(a.size()), m = static_cast<int>(b.size());
    a.push_back(a[0]);
    a.push_back(a[1]);
    b.push_back(b[0]);
    b.push_back(b[1]);
    std::vector<Point> res;
    int i = 0, j = 0;
    while (i < n || j < m) {
        res.push_back(a[i] + b[j]);
        if (i == n) {
            j++;
            continue;
        }
        if (j == m) {
            i++;
            continue;
        }
        double cr = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
        if (cr > -EPS) {
            i++;
        }
        if (cr < EPS) {
            j++;
        }
    }
    return remove_collinear(res);
}

```

## Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Point> empty;
    assert(minkowski_sum(empty, {Point(1, 2)}).empty());

    vector<Point> square{{0, 0}, {1, 0}, {1, 1}, {0, 1}};
    vector<Point> tri{{0, 0}, {2, 0}, {0, 2}};
    // The square thickens the triangle by one unit in the positive x/y directions.
    assert((minkowski_sum(square, tri) == vector<Point>{{0, 0}, {3, 0}, {3, 1}, {1, 3}, {0, 3}}));

    // Adding a single point translates the polygon.
    assert((minkowski_sum(square, {Point(2, 3)}) == vector<Point>{{2, 3}, {3, 3}, {3, 4}, {2, 4}}));
    assert((minkowski_sum({Point(1, 2)}, {Point(3, 4)}) == vector<Point>{{4, 6}}));
    return 0;
}

```

### 7.4.4 Polygon Intersection and Union

Given two simple (non-self-intersecting) polygons, determine the areas of their intersection and union using a sweep line algorithm and the inclusion-exclusion principle. Every vertex and every pairwise edge intersection contributes its  $x$ -coordinate as a sweep boundary, so within each vertical slab between consecutive boundaries the two borders cannot cross, and the slab's overlap is accumulated exactly from trapezoid areas. The union then follows by inclusion-exclusion as the sum of the individual areas minus the intersection.

- `intersection_area(a, b)` returns the intersection area of polygons `a` and `b`, whose vertices are listed in boundary order.
- `union_area(a, b)` returns the union area of polygons `a` and `b`.

Overflow warning: the segment-intersection tests form cross products in the input point's coordinate type, which grow like the squared coordinate magnitude. For integral point types, use 64-bit coordinates (e.g. `PointL` from 7.1.1) if necessary.

**Time Complexity:**  $O(n \cdot m \cdot (n + m) \cdot \log(n + m))$  per call to `intersection_area()` and `union_area()`, where  $n$  is the number of vertices in the first polygon and  $m$  is the number of vertices in the second polygon (or  $O(N^3 \log N)$  for  $N = n + m$ ).

**Space Complexity:**  $O(n \cdot m)$  auxiliary for `intersection_area()` and `union_area()`, where  $n$  and  $m$  are the respective polygon sizes.

```
#include <algorithm>
#include <cmath>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return EQ((p.x - a.x) * (b.y - a.y) - (p.y - a.y) * (b.x - a.x), 0) &&
        LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

// Specialized version of seg_intersection() from 7.2.3, simplified for include_boundary = true,
// returning -1 for no intersection, 0 for one intersection point, 1 for positive length overlap.
template<typename Pt>
int seg_intersection1(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, double *outx, double *outy
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
```

```

auto cd2 = cd_x * cd_x + cd_y * cd_y;
if (EQ(ab2, 0)) {
    if (point_on_segment(a, c, d)) {
        *outx = a.x;
        *outy = a.y;
        return 0;
    }
    return -1;
}
if (EQ(cd2, 0)) {
    if (point_on_segment(c, a, b)) {
        *outx = c.x;
        *outy = c.y;
        return 0;
    }
    return -1;
}
auto c1 = ab_x * cd_y - ab_y * cd_x;
auto c2 = ac_x * ab_y - ac_y * ab_x;
if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
    Pt res1 = std::max(std::min(a, b), std::min(c, d));
    Pt res2 = std::min(std::max(a, b), std::max(c, d));
    if (!res2 < res1) {
        if (res1 == res2) {
            *outx = static_cast<double>(res1.x);
            *outy = static_cast<double>(res1.y);
            return 0;
        }
        return 1;
    }
    return -1;
}
if (EQ(c1, 0)) {
    return -1;
}
auto t_num = ac_x * cd_y - ac_y * cd_x;
bool c1_pos = c1 > 0;
bool t_ok = c1_pos ? (LE(0, t_num) && LE(t_num, c1)) : (LE(c1, t_num) && LE(t_num, 0));
bool u_ok = c1_pos ? (LE(0, c2) && LE(c2, c1)) : (LE(c1, c2) && LE(c2, 0));
if (t_ok && u_ok) {
    double t = static_cast<double>(t_num) / c1;
    *outx = static_cast<double>(a.x + t * ab_x);
    *outy = static_cast<double>(a.y + t * ab_y);
    return 0;
}
return -1;
}

// The two segments may use different point types (e.g. polygon edge vs. sweep line).
template<typename PtA, typename PtB>
int line_intersection(
    const PtA &p1, const PtA &p2, const PtB &p3, const PtB &p4, double *outx, double *outy
) {
    double a1 = static_cast<double>(p2.y) - p1.y, b1 = static_cast<double>(p1.x) - p2.x;
    double c1 = -(a1 * p1.x + b1 * p1.y);
    double a2 = static_cast<double>(p4.y) - p3.y, b2 = static_cast<double>(p3.x) - p4.x;
    double c2 = -(a2 * p3.x + b2 * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    double det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (outx != nullptr && outy != nullptr) {
        *outx = x / det;
        *outy = y / det;
    }
    return 0;
}

```

```

template<typename Pt>
double intersection_area(const std::vector<Pt> &a, const std::vector<Pt> &b) {
    if (a.empty() || b.empty()) {
        return 0;
    }
    struct SweepPoint {
        double x, y;
    }; // For line intersection with the sweep line.
    const std::vector<Pt> *polys[2] = {&a, &b};
    int orientation[2];
    for (int id = 0; id < 2; id++) {
        const auto &p = *polys[id];
        int n = static_cast<int>(p.size());
        double area = 0;
        for (int i = 0, j = n - 1; i < n; j = i++) {
            area += (static_cast<double>(p[j].x) - p[i].x) * (static_cast<double>(p[j].y) + p[i].y);
        }
        orientation[id] = (area < 0 ? 1 : (area > 0 ? -1 : 0));
    }
    std::vector<double> x_coords;
    for (const Pt &p : a) {
        x_coords.push_back(p.x);
    }
    for (const Pt &p : b) {
        x_coords.push_back(p.x);
    }
    int n = static_cast<int>(a.size()), m = static_cast<int>(b.size());
    for (int i1 = 0, j1 = n - 1; i1 < n; j1 = i1++) {
        for (int i2 = 0, j2 = m - 1; i2 < m; j2 = i2++) {
            double outx, outy;
            if (seg_intersection1(a[i1], a[j1], b[i2], b[j2], &outx, &outy) == 0) {
                x_coords.push_back(outx);
            }
        }
    }
    std::sort(x_coords.begin(), x_coords.end());
    x_coords.erase(
        std::unique(x_coords.begin(), x_coords.end(), [](double a, double b) { return EQ(a, b); }),
        x_coords.end()
    );
    double res = 0;
    for (int k = 0; k + 1 < static_cast<int>(x_coords.size()); k++) {
        double x = (x_coords[k] + x_coords[k + 1]) / 2;
        SweepPoint sweep0{x, 0}, sweep1{x, 1};
        std::vector<std::pair<double, int>> events; // (y, mask delta)
        for (int id = 0; id < 2; id++) {
            const auto &p = *polys[id];
            int size = static_cast<int>(p.size());
            for (int j = 0, i = size - 1; j < size; i = j++) {
                double px, py;
                if (line_intersection1(p[j], p[i], sweep0, sweep1, &px, &py) == 0) {
                    double y = py, x0 = p[i].x, x1 = p[j].x;
                    if (x0 < x && x1 > x) {
                        events.emplace_back(y, orientation[id] * (1 << id));
                    } else if (x0 > x && x1 < x) {
                        events.emplace_back(y, -orientation[id] * (1 << id));
                    }
                }
            }
        }
        std::sort(events.begin(), events.end());
        double height = 0;
        int coverage[2] = {0, 0};
        for (int j = 0; j < static_cast<int>(events.size()); j++) {
            auto [y, mask_delta] = events[j];
            if (coverage[0] != 0 && coverage[1] != 0) {
                height += y - events[j - 1].first;
            }
        }
    }
}

```

```

    }
    int bit = std::abs(mask_delta);
    int poly = (bit == 1 ? 0 : 1);
    coverage[poly] += mask_delta / bit;
}
res += height * (x_coords[k + 1] - x_coords[k]);
}
return res;
}

template<typename Pt>
double polygon_area(const std::vector<Pt> &p) {
    if (p.empty()) {
        return 0;
    }
    int n = static_cast<int>(p.size());
    double area = 0;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        area += (static_cast<double>(p[j].x) - p[i].x) * (static_cast<double>(p[j].y) + p[i].y);
    }
    return fabs(area / 2.0);
}

template<typename Pt>
double union_area(const std::vector<Pt> &a, const std::vector<Pt> &b) {
    return polygon_area(a) + polygon_area(b) - intersection_area(a, b);
}

```

## Example Usage

```

#include <cassert>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &o) const { return x == o.x && y == o.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
};

int main() {
    vector<Point> p, s;
    // Irregular pentagon overlapping the square below in a triangle of area 1.5.
    p.emplace_back(1, 3);
    p.emplace_back(1, 2);
    p.emplace_back(2, 1);
    p.emplace_back(0, 0);
    p.emplace_back(-1, 3);
    // Square of area 12.5 in quadrant 2.
    s.emplace_back(0, 0);
    s.emplace_back(0, 3);
    s.emplace_back(-3, 3);
    s.emplace_back(-3, 0);
    assert(EQ(1.5, intersection_area(p, s)));
    assert(EQ(12.5, union_area(p, s)));

    // Integer-coordinate polygons are accepted (computation proceeds in double).
}

```

```

vector<PointI> ip{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
vector<PointI> is{{0, 0}, {0, 3}, {-3, 3}, {-3, 0}};
assert(EQ(1.5, intersection_area(ip, is)));
assert(EQ(12.5, union_area(ip, is)));
return 0;
}

```

### 7.4.5 Delaunay Triangulation (Simple)

Given a set  $P$  of distinct two-dimensional points, a Delaunay triangulation of  $P$  is a triangulation of the convex hull of  $P$  such that no point of  $P$  lies strictly inside the circumcircle of any triangle in the triangulation. The Delaunay triangulation is not necessarily unique when four or more points are cocircular.

This implementation produces one valid triangulation using a simple brute-force algorithm. Each candidate triangle is tested against the empty-circumcircle condition, and candidates whose edges would properly cross already accepted triangles are rejected.

All arithmetic uses the point's own coordinate type, so integer inputs yield an exact triangulation.

- `delaunay_triangulation(p)` returns a Delaunay triangulation of the distinct points in `p` as a vector of clockwise-oriented vertex triples, or an empty vector if no triangulation exists (e.g. fewer than three points, or all points collinear).

Overflow warning: the lifted-paraboloid test grows like the fourth power of the coordinate magnitude. With 64-bit integer coordinates, it overflows once coordinates exceed roughly 17600. For floating-point coordinates the test is subject to the usual rounding error.

**Time Complexity:**  $O(n^6)$  per call, where  $n$  is the number of points. There are  $O(n^3)$  candidate triangles; each scans all points and, conservatively, up to  $O(n^3)$  previously accepted candidates.

**Space Complexity:**  $O(n)$  auxiliary, excluding the returned triangulation.

```

#include <cmath>
#include <tuple>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T, typename U>
bool LE(T a, U b) {
    return !LT(b, a);
}

// Specialized version of seg_intersection() from 7.2.3, simplified for include_boundary = false,
// i.e. we're detecting proper crossings of two non-parallel segments whose interiors intersect.
template<typename Pt>
bool proper_seg_intersection(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    auto cross = [](const Pt &o, const Pt &p, const Pt &q) {
        return (p.x - o.x) * (q.y - o.y) - (p.y - o.y) * (q.x - o.x); // Overflow warning.
    };
};

```

```

    auto c1 = cross(a, b, c), c2 = cross(a, b, d), c3 = cross(c, d, a), c4 = cross(c, d, b);
    return ((LT(c1, 0) && LT(0, c2)) || (LT(c2, 0) && LT(0, c1))) &&
           ((LT(c3, 0) && LT(0, c4)) || (LT(c4, 0) && LT(0, c3)));
}

template<typename Pt>
std::vector<std::tuple<Pt, Pt, Pt>> delaunay_triangulation(const std::vector<Pt> &p) {
    int n = static_cast<int>(p.size());
    std::vector<decltype(Pt::x)> z;
    for (const auto &[px, py] : p) {
        z.emplace_back(px * px + py * py); // Overflow warning.
    }
    std::vector<std::tuple<Pt, Pt, Pt>> res;
    std::vector<std::tuple<int, int, int>> res_idx;
    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = i + 1; k < n; k++) {
                if (j == k) {
                    continue;
                }
                // Overflow warning.
                auto nx = (p[j].y - p[i].y) * (z[k] - z[i]) - (p[k].y - p[i].y) * (z[j] - z[i]);
                auto ny = (p[k].x - p[i].x) * (z[j] - z[i]) - (p[j].x - p[i].x) * (z[k] - z[i]);
                auto nz = (p[j].x - p[i].x) * (p[k].y - p[i].y) - (p[k].x - p[i].x) * (p[j].y - p[i].y);
                if (LE(0, nz)) {
                    continue;
                }
                std::vector<Pt> s1{p[i], p[j], p[k], p[i]};
                for (int m = 0; m < n; m++) {
                    if (LT(0, nx * (p[m].x - p[i].x) + ny * (p[m].y - p[i].y) + nz * (z[m] - z[i]))) {
                        goto skip;
                    }
                }
                // Reject candidates whose edges properly cross already accepted triangles.
                for (auto [t0, t1, t2] : res_idx) {
                    std::vector<Pt> s2{p[t0], p[t1], p[t2], p[t0]};
                    for (int u = 0; u < 3; u++) {
                        for (int v = 0; v < 3; v++) {
                            if (proper_seg_intersection(s1[u], s1[u + 1], s2[v], s2[v + 1])) {
                                goto skip;
                            }
                        }
                    }
                }
                res.emplace_back(p[i], p[j], p[k]);
                res_idx.emplace_back(i, j, k);
            skip:;
        }
    }
    return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {

```



```

int x, y;
PointI(int x = 0, int y = 0) : x(x), y(y) {}
bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
};

int main() {
    vector<Point> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<Point, Point, Point>> t{
        {Point{1, 3}, Point{1, 2}, Point{-1, 3}},
        {Point{1, 3}, Point{2, 1}, Point{1, 2}},
        {Point{1, 2}, Point{2, 1}, Point{0, 0}},
        {Point{1, 2}, Point{0, 0}, Point{-1, 3}}
    };
    assert(delaunay_triangulation(v) == t);

    // Integer-coordinate inputs are triangulated using exact integer arithmetic.
    vector<PointI> iv{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointI, PointI, PointI>> ti{
        {PointI{1, 3}, PointI{1, 2}, PointI{-1, 3}},
        {PointI{1, 3}, PointI{2, 1}, PointI{1, 2}},
        {PointI{1, 2}, PointI{2, 1}, PointI{0, 0}},
        {PointI{1, 2}, PointI{0, 0}, PointI{-1, 3}}
    };
    assert(delaunay_triangulation(iv) == ti);
    return 0;
}

```

### 7.4.6 Delaunay Triangulation (Guibas-Stolfi)

Given a set  $P$  of distinct two-dimensional points, a Delaunay triangulation of  $P$  is a triangulation of the convex hull of  $P$  such that no point of  $P$  lies strictly inside the circumcircle of any triangle in the triangulation. The Delaunay triangulation is not necessarily unique when four or more points are cocircular.

This implementation produces one valid triangulation using the Guibas-Stolfi divide-and-conquer algorithm with a quad-edge data structure. The point set is split in half, each half is triangulated recursively, and the halves are stitched together from the bottom up by a sequence of cross edges, with circumcircle tests deciding each connecting edge and deleting invalidated ones.

The point type must provide exact lexicographic `operator<`, since sorting and duplicate removal require a strict ordering rather than epsilon equality. All arithmetic uses the point's coordinate type, so integer inputs yield an exact triangulation.

- `delaunay_triangulation(p)` returns the triangles of one Delaunay triangulation as a vector of counterclockwise-oriented `Point<T>` triples (`std::tuple`). Duplicate points are ignored. If fewer than three non-collinear unique points remain, the result is empty.

Overflow warning: the in-circle test grows like the fourth power of the coordinate magnitude and dominates the overflow budget. With 64-bit integer coordinates, it overflows once coordinates exceed roughly 14000. For floating-point coordinates, the predicates are subject to the usual rounding error.

**Time Complexity:**  $O(n \log n)$  per call, where  $n$  is the number of input points.

**Space Complexity:**

- $O(n \log n)$  for allocated quad-edges in the worst case because deleted edges remain in the pool.
- $O(n)$  for the returned triangulation.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <tuple>
#include <type_traits>

```

```

#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) < C(b) - EPS;
    return C(a) < C(b);
}

template<typename T>
struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    friend bool EQ(const Point &a, const Point &b) { return EQ(a.x, b.x) && EQ(a.y, b.y); }
    bool operator<(const Point &p) const { return std::tie(x, y) < std::tie(p.x, p.y); }
};

template<typename T>
T cross(const Point<T> &a, const Point<T> &b, const Point<T> &o) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x); // Overflow warning.
}

template<typename T>
struct Edge {
    Point<T> origin;
    Edge *rot, *onext;
    bool primal, deleted, used;

    Edge *rev() const { return rot->rot; }
    Edge *lnext() const { return rot->rev()->onext->rot; }
    Edge *oprev() const { return rot->onext->rot; }
    Point<T> dest() const { return rev()->origin; }
};

template<typename T>
struct QuadEdgePool {
    std::vector<Edge<T>*> edges;

    QuadEdgePool() = default;
    QuadEdgePool(const QuadEdgePool &) = delete;
    QuadEdgePool &operator=(const QuadEdgePool &) = delete;

    ~QuadEdgePool() {
        for (Edge<T> *e : edges) {
            delete e;
        }
    }

    Edge<T> *make_edge(const Point<T> &u, const Point<T> &v) {
        Edge<T> *e1 = new Edge<T>, *e2 = new Edge<T>, *e3 = new Edge<T>, *e4 = new Edge<T>;
        edges.push_back(e1);
        edges.push_back(e2);
        edges.push_back(e3);
        edges.push_back(e4);
        e1->origin = u;
        e2->origin = v;
        e1->rot = e3;
        e2->rot = e4;
        e3->rot = e2;
    }
}

```

```

    e4->rot = e1;
    e1->onext = e1;
    e2->onext = e2;
    e3->onext = e4;
    e4->onext = e3;
    e1->primal = e2->primal = true;
    e3->primal = e4->primal = false;
    e1->deleted = e2->deleted = e3->deleted = e4->deleted = false;
    e1->used = e2->used = e3->used = e4->used = false;
    return e1;
}

void delete_edge(Edge<T> *e) {
    splice(e, e->oprev());
    splice(e->rev(), e->rev()->oprev());
    e->deleted = e->rot->deleted = e->rev()->deleted = e->rev()->rot->deleted = true;
}

Edge<T> *connect(Edge<T> *a, Edge<T> *b) {
    Edge<T> *e = make_edge(a->dest(), b->origin);
    splice(e, a->lnext());
    splice(e->rev(), b);
    return e;
}

static void splice(Edge<T> *a, Edge<T> *b) {
    std::swap(a->onext->rot->onext, b->onext->rot->onext);
    std::swap(a->onext, b->onext);
}
};

template<typename T>
bool left_of(const Point<T> &p, Edge<T> *e) {
    return LT(0, cross(e->origin, e->dest(), p));
}

template<typename T>
bool right_of(const Point<T> &p, Edge<T> *e) {
    return LT(cross(e->origin, e->dest(), p), 0);
}

template<typename T>
bool in_circle(const Point<T> &a, const Point<T> &b, const Point<T> &c, const Point<T> &d) {
    T adx = a.x - d.x, ady = a.y - d.y;
    T bdx = b.x - d.x, bdy = b.y - d.y;
    T cdx = c.x - d.x, cdy = c.y - d.y;
    T abdet = adx * bdy - bdx * ady; // Overflow warning.
    T bcdet = bdx * cdy - cdx * bdy;
    T cadet = cdx * ady - adx * cdy;
    T alift = adx * adx + ady * ady;
    T blift = bdx * bdx + bdy * bdy;
    T clift = cdx * cdx + cdy * cdy;
    return LT(0, alift * bcdet + blift * cadet + clift * abdet);
}

template<typename T>
std::pair<Edge<T> *, Edge<T> *> build_triangulation(
    const std::vector<Point<T>> &p, int lo, int hi, QuadEdgePool<T> &pool
) {
    if (hi - lo == 1) {
        Edge<T> *a = pool.make_edge(p[lo], p[hi]);
        return {a, a->rev()};
    }
    if (hi - lo == 2) {
        Edge<T> *a = pool.make_edge(p[lo], p[lo + 1]);
        Edge<T> *b = pool.make_edge(p[lo + 1], p[hi]);
        pool.splice(a->rev(), b);
        int side = LT(0, cross(p[lo], p[lo + 1], p[hi]))

```

```

        ? 1
        : (LT(cross(p[lo], p[lo + 1], p[hi]), 0) ? -1 : 0);
    if (side == 0) {
        return {a, b->rev()};
    }
    Edge<T> *c = pool.connect(b, a);
    if (side == 1) {
        return {a, b->rev()};
    }
    return {c->rev(), c};
}
int mid = lo + (hi - lo) / 2;
auto [ldo, ldi] = build_triangulation(p, lo, mid, pool);
auto [rdi, rdo] = build_triangulation(p, mid + 1, hi, pool);
while (true) {
    if (left_of(rdi->origin, ldi)) {
        ldi = ldi->lnext();
    } else if (right_of(ldi->origin, rdi)) {
        rdi = rdi->rev()->onext;
    } else {
        break;
    }
}
Edge<T> *base = pool.connect(rdi->rev(), ldi);
if (ldi->origin == ldo->origin) {
    ldo = base->rev();
}
if (rdi->origin == rdo->origin) {
    rdo = base;
}
while (true) {
    Edge<T> *lcand = base->rev()->onext;
    if (right_of(lcand->dest(), base)) {
        while (in_circle(base->dest(), base->origin, lcand->dest(), lcand->onext->dest())) {
            Edge<T> *next = lcand->onext;
            pool.delete_edge(lcand);
            lcand = next;
        }
    }
    Edge<T> *rcand = base->oprev();
    if (right_of(rcand->dest(), base)) {
        while (in_circle(base->dest(), base->origin, rcand->dest(), rcand->oprev()->dest())) {
            Edge<T> *prev = rcand->oprev();
            pool.delete_edge(rcand);
            rcand = prev;
        }
    }
    bool lvalid = right_of(lcand->dest(), base), rvalid = right_of(rcand->dest(), base);
    if (!lvalid && !rvalid) {
        break;
    }
    if (!lvalid ||
        (rvalid && in_circle(lcand->dest(), lcand->origin, rcand->origin, rcand->dest()))) {
        base = pool.connect(rcand, base->rev());
    } else {
        base = pool.connect(base->rev(), lcand->rev());
    }
}
return {ldo, rdo};
}

template<typename T>
auto delaunay_triangulation(std::vector<Point<T>> p) {
    using Pt = Point<T>;
    using Triangle = std::tuple<Pt, Pt, Pt>;
    // Rotates triangle vertices so that the lexicographically smallest comes first, preserving the
    // cyclic (counterclockwise) order, to give each triangle a canonical, comparable representation.
    auto canonical = [](const Pt &a, const Pt &b, const Pt &c) -> Triangle {

```

```

    if (b < a && b < c) {
        return {b, c, a};
    }
    if (c < a && c < b) {
        return {c, a, b};
    }
    return {a, b, c};
};
std::sort(p.begin(), p.end());
auto same_point = [](const Pt &a, const Pt &b) { return !(a < b) && !(b < a); };
p.erase(std::unique(p.begin(), p.end(), same_point), p.end());
std::vector<Triangle> res;
if (p.size() < 3) {
    return res;
}
QuadEdgePool<T> pool;
build_triangulation(p, 0, static_cast<int>(p.size()) - 1, pool);
for (Edge<T> *start : pool.edges) {
    if (!start->primal || start->deleted || start->used) {
        continue;
    }
    std::vector<Pt> face;
    Edge<T> *curr = start;
    do {
        curr->used = true;
        face.push_back(curr->origin);
        curr = curr->lnext();
    } while (curr != start);
    if (face.size() == 3 && LT(0, cross(face[0], face[1], face[2]))) {
        res.push_back(canonical(face[0], face[1], face[2]));
    }
}
std::sort(res.begin(), res.end());
return res;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

using PointL = Point<int64_t>;
using PointD = Point<double>;

int main() {
    vector<PointD> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointD, PointD, PointD>> t{
        {PointD{-1, 3}, PointD{0, 0}, PointD{1, 2}},
        {PointD{-1, 3}, PointD{1, 2}, PointD{1, 3}},
        {PointD{0, 0}, PointD{2, 1}, PointD{1, 2}},
        {PointD{1, 2}, PointD{2, 1}, PointD{1, 3}}
    };
    assert(delaunay_triangulation(v) == t);

    // Integer-coordinate inputs are triangulated using exact integer arithmetic.
    vector<PointL> iv{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointL, PointL, PointL>> ti{
        {PointL{-1, 3}, PointL{0, 0}, PointL{1, 2}},
        {PointL{-1, 3}, PointL{1, 2}, PointL{1, 3}},
        {PointL{0, 0}, PointL{2, 1}, PointL{1, 2}},
        {PointL{1, 2}, PointL{2, 1}, PointL{1, 3}}
    };
    assert(delaunay_triangulation(iv) == ti);
    return 0;
}

```

## 7.5 3D Geometry

### 7.5.1 3D Point

A 3D point class templated on the coordinate type `T`. Algebraic operations preserve `T`, while metric operations use `fp_t`, which is `T` for floating-point coordinates and `double` otherwise.

- `TPoint3<T>(x = 0, y = 0, z = 0)` constructs point  $(x, y, z)$ .
- Operators `+`, `-`, `*`, and `/` preserve the coordinate type except that division promotes to `fp_t`. Comparisons are exact and lexicographic, while `EQ(p, q)` uses `EPS` for floating-point coordinates and remains exact for other types.
- `sqnorm()`, `dot(p)`, and `cross(p)` return exact algebraic results in coordinate type `T`.
- `norm()`, `phi()`, and `theta()` return the length, azimuth, and polar angle in `fp_t`.
- `unit()` returns the unit vector and requires a nonzero point; `normal(p)` returns a unit normal and requires nonparallel vectors.
- `rotate(angle, axis)` rotates the point about a nonzero `axis` using Rodrigues' formula.
- `Point3I`, `Point3L`, `Point3D`, and `Point3LD` use `int`, `int64_t`, `double`, and `long double` coordinates, respectively.

Overflow warning: the exact products `dot()`, `cross()`, and `sqnorm()` grow like the squared coordinate magnitude. With `TPoint3<int>` these overflow a 32-bit `int` once coordinates exceed a few tens of thousands, so use `Point3L` (`TPoint3<int64_t>`) for larger integer coordinates.

**Time Complexity:**  $O(1)$  per operation.

**Space Complexity:**  $O(1)$  for storage and auxiliary.

```
#include <cassert>
#include <cmath>
#include <cstdint>
#include <tuple>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    if constexpr (std::is_floating_point_v<C>) return C(a) == C(b) || std::fabs(C(a) - C(b)) <= EPS;
    return C(a) == C(b);
}

template<typename T>
struct TPoint3 {
    using fp_t = std::conditional_t<std::is_floating_point<T>::value, T, double>;

    T x, y, z;

    TPoint3(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
    bool operator==(const TPoint3 &p) const { return std::tie(x, y, z) == std::tie(p.x, p.y, p.z); }

    friend bool EQ(const TPoint3 &a, const TPoint3 &b) {
        return EQ(a.x, b.x) && EQ(a.y, b.y) && EQ(a.z, b.z);
    }

    bool operator!=(const TPoint3 &p) const { return !(*this == p); }
    bool operator<(const TPoint3 &p) const { return std::tie(x, y, z) < std::tie(p.x, p.y, p.z); }
    TPoint3 operator+(const TPoint3 &p) const { return {x + p.x, y + p.y, z + p.z}; }
    TPoint3 operator-(const TPoint3 &p) const { return {x - p.x, y - p.y, z - p.z}; }
    TPoint3 operator*(T k) const { return {x * k, y * k, z * k}; }
    TPoint3 operator/(fp_t k) const { return {fp_t(x) / k, fp_t(y) / k, fp_t(z) / k}; }
    T dot(const TPoint3 &p) const { return x * p.x + y * p.y + z * p.z; } // Overflow warning.
    T sqnorm() const { return x * x + y * y + z * z; } // Overflow warning.
    fp_t norm() const { return std::hypot(std::hypot(fp_t(x), fp_t(y)), fp_t(z)); }
```

```

fp_t phi() const { return std::atan2(fp_t(y), fp_t(x)); }
fp_t theta() const { return std::atan2(std::hypot(fp_t(x), fp_t(y)), fp_t(z)); }
TPoint3<fp_t> unit() const { return *this / norm(); }

TPoint3 cross(const TPoint3 &p) const {
    return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x}; // Overflow warning.
}

TPoint3<fp_t> normal(const TPoint3 &p) const { return cross(p).unit(); }

TPoint3<fp_t> rotate(fp_t angle, const TPoint3 &axis) const {
    fp_t s = std::sin(angle), c = std::cos(angle);
    TPoint3<fp_t> u = axis.unit(), v{fp_t(x), fp_t(y), fp_t(z)};
    return u * v.dot(u) * (1 - c) + v * c - v.cross(u) * s;
}
};

using Point3I = TPoint3<int>;
using Point3L = TPoint3<int64_t>;
using Point3D = TPoint3<double>;
using Point3LD = TPoint3<long double>;

```

### Example Usage

```

using namespace std;

int main() {
    Point3I a(1, 0, 0), b(0, 1, 0);
    assert(a.dot(b) == 0);
    assert(a.cross(b) == Point3I(0, 0, 1));
    assert(a.sqnorm() == 1);

    Point3D p(1, 0, 0);
    assert(EQ(p.rotate(acos(-1.0) / 2, Point3D(0, 0, 1)), Point3D(0, 1, 0)));
    return 0;
}

```

## 7.5.2 Polyhedron Volume

Given an oriented triangular mesh of a closed polyhedron, compute its signed volume. Each triangle contributes the signed volume of the tetrahedron formed by the triangle  $abc$  and the origin  $((a \times b) \cdot c)/6$ . If faces are oriented outward, the result is positive for the orientation used by `convex_hull_3d()` in 7.5.3; take `abs()` if only the magnitude is needed.

- `signed_polyhedron_volume(p, faces)` returns the signed volume. Each face triplet  $(a, b, c)$  represents the triangle with vertices `p[a]`, `p[b]`, and `p[c]`.

Overflow warning: for integral point types, each scalar triple product must fit in the point's coordinate arithmetic type before it is converted to `double`.

**Time Complexity:**  $O(f)$  per call, where  $f$  is the number of triangular faces.

**Space Complexity:**  $O(1)$  auxiliary.

```

#include <cmath>
#include <tuple>
#include <vector>

struct Point3D {
    double x, y, z;

    Point3D(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
}

```

```

double dot(const Point3D &p) const { return x * p.x + y * p.y + z * p.z; }

Point3D cross(const Point3D &p) const {
    return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x};
}
};

template<typename Pt, typename F>
double signed_polyhedron_volume(const std::vector<Pt> &p, const std::vector<F> &faces) {
    double volume = 0;
    for (const auto &[a, b, c] : faces) {
        volume += p[a].cross(p[b]).dot(p[c]); // Overflow warning.
    }
    return volume / 6.0;
}

```

## Example Usage

```

#include <cassert>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    using Face = tuple<int, int, int>; // (a, b, c)
    vector<Point3D> p{{0, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    vector<Face> faces{
        Face(0, 2, 1),
        Face(0, 1, 3),
        Face(0, 3, 2),
        Face(1, 2, 3),
    };
    assert(EQ(signed_polyhedron_volume(p, faces), 1.0 / 6.0));
    return 0;
}

```

### 7.5.3 3D Convex Hull

The convex hull of points in 3D is the smallest convex polyhedron containing them. This incremental algorithm computes its triangular faces: start with a tetrahedron, delete every visible face for each new point, and stitch the horizon edges to that point. Returned faces are oriented outward.

- `convex_hull_3d(p)` returns triangular faces as `Face{a, b, c}` triples, where each face has vertices `p[a]`, `p[b]`, and `p[c]`. The input must contain at least four points, with no four coplanar. For coplanar or highly degenerate inputs, perturb the points slightly or use a more robust library.

**Time Complexity:**  $O(n^2)$  per call, where  $n$  is the number of points.

**Space Complexity:**  $O(n^2)$  auxiliary.

```

#include <array>
#include <cassert>
#include <utility>
#include <vector>

const double EPS = 1e-9;

struct Point3D {
    double x, y, z;

```



```

Point3D(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
Point3D operator-(const Point3D &p) const { return {x - p.x, y - p.y, z - p.z}; }
Point3D operator*(double k) const { return {x * k, y * k, z * k}; }
double dot(const Point3D &p) const { return x * p.x + y * p.y + z * p.z; }

Point3D cross(const Point3D &p) const {
    return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x};
}

};

struct Face {
    int a, b, c;
    Face(int a = 0, int b = 0, int c = 0) : a(a), b(b), c(c) {}
};

std::vector<Face> convex_hull_3d(const std::vector<Point3D> &p) {
    int n = static_cast<int>(p.size());
    assert(n >= 4);
    std::vector<std::vector<std::array<int, 2>>> edge(
        n, std::vector<std::array<int, 2>>(n, {-1, -1})
    );
    std::vector<Face> faces;
    auto normal = [&](const Face &f) { return (p[f.b] - p[f.a]).cross(p[f.c] - p[f.a]); };
    auto visible = [&](const Face &f, int i) { return normal(f).dot(p[i] - p[f.a]) > EPS; };
    auto add_edge = [&](int a, int b, int c) {
        if (a > b) {
            std::swap(a, b);
        }
        auto &e = edge[a][b];
        (e[0] == -1 ? e[0] : e[1]) = c;
    };
    auto remove_edge = [&](int a, int b, int c) {
        if (a > b) {
            std::swap(a, b);
        }
        auto &e = edge[a][b];
        (e[0] == c ? e[0] : e[1]) = -1;
    };
    auto edge_count = [&](int a, int b) {
        if (a > b) {
            std::swap(a, b);
        }
        return (edge[a][b][0] != -1) + (edge[a][b][1] != -1);
    };
    auto make_face = [&](int a, int b, int c, int inside) {
        Face f(a, b, c);
        if (normal(f).dot(p[inside] - p[a]) > 0) {
            std::swap(f.b, f.c);
        }
        add_edge(f.a, f.b, f.c);
        add_edge(f.b, f.c, f.a);
        add_edge(f.c, f.a, f.b);
        faces.push_back(f);
    };
    for (int i = 0; i < 4; i++) {
        for (int j = i + 1; j < 4; j++) {
            for (int k = j + 1; k < 4; k++) {
                make_face(i, j, k, 6 - i - j - k);
            }
        }
    }
    for (int i = 4; i < n; i++) {
        for (int j = 0; j < static_cast<int>(faces.size()); j++) {
            Face f = faces[j];
            if (visible(f, i)) {
                remove_edge(f.a, f.b, f.c);
                remove_edge(f.b, f.c, f.a);
                remove_edge(f.c, f.a, f.b);
            }
        }
    }
}

```

```

        std::swap(faces[j--], faces.back());
        faces.pop_back();
    }
}
int old_faces = static_cast<int>(faces.size());
for (int j = 0; j < old_faces; j++) {
    Face f = faces[j];
    if (edge_count(f.a, f.b) != 2) {
        make_face(f.a, f.b, i, f.c);
    }
    if (edge_count(f.b, f.c) != 2) {
        make_face(f.b, f.c, i, f.a);
    }
    if (edge_count(f.c, f.a) != 2) {
        make_face(f.c, f.a, i, f.b);
    }
}
}
return faces;
}

```

## Example Usage

```

using namespace std;

void assert_outward(const vector<Point3D> &p, const vector<Face> &faces) {
    for (const Face &f : faces) {
        assert(0 <= f.a && f.a < static_cast<int>(p.size()));
        assert(0 <= f.b && f.b < static_cast<int>(p.size()));
        assert(0 <= f.c && f.c < static_cast<int>(p.size()));
        assert(f.a != f.b && f.b != f.c && f.c != f.a);
        Point3D normal = (p[f.b] - p[f.a]).cross(p[f.c] - p[f.a]);
        for (const Point3D &q : p) {
            assert(normal.dot(q - p[f.a]) <= EPS);
        }
    }
}

int main() {
    vector<Point3D> tetra{0, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    auto faces = convex_hull_3d(tetra);
    // A tetrahedron has four triangular faces.
    assert(faces.size() == 4);
    assert_outward(tetra, faces);

    tetra.emplace_back(1, 1, 1);
    faces = convex_hull_3d(tetra);
    // Adding the opposite point makes a triangular bipyramid with six triangular faces.
    assert(faces.size() == 6);
    assert_outward(tetra, faces);
    return 0;
}

```

## Chapter 8

# Miscellany

### 8.1 Contest Utilities

---

Small contest convenience helpers that are useful across many algorithms. These snippets avoid algorithm-specific policy and are meant to be pasted near the top of a solution file.

A personal template may also collect shortened versions of utilities elsewhere in the anthology, such as fast I/O and debugging, custom hashing, coordinate compression, bit operations, modular arithmetic, binary search, and GNU policy-based data structures. They remain in their dedicated sections here to avoid duplication and let each template be assembled to taste.

- `Rep(i,N)`, `For(i,L,H)`, `Rev(i,N)`, `Dwn(i,H,L)`, and `Each(x,C)` are compact loop macros.
- `All(c)` and `Rall(c)` expand to the forward or reverse iterator pair of a container or array.
- `sz(c)` returns the signed size of a container.
- `ckmin(a, b)` assigns `a = b` only when `b < a`, and returns whether the assignment happened.
- `ckmax(a, b)` assigns `a = b` only when `a < b`, and returns whether the assignment happened.
- `floor_div(a, b)` and `ceil_div(a, b)` divide signed integers with mathematical rounding toward negative or positive infinity. Requires nonzero `b` and the resulting quotient to be representable by the result type; in particular, the minimum value cannot be divided by `-1`.
- `make_vec(n1, ..., nk, v)` returns a `k`-dimensional nested vector with the given sizes and every element initialized to `v`, whose type is deduced from `v`. The last argument is always the fill value, so `make_vec(3, 5)` is a vector of three fives rather than a 3 by 5 grid.
- `indices(n)` returns the vector `[0, 1, ..., n - 1]`.
- `lb(a, x, comp = std::less<>)` and `ub(a, x, comp = std::less<>)` return lower and upper bound indices in random-access sequence `a`, which must be sorted according to `comp`.
- `sort_unique(v, comp = std::less<>)` sorts a vector and removes duplicates, which are the elements that `comp` deems equivalent.
- `argsort(a, comp = std::less<>)` returns the indices of `a` ordered by their values according to `comp`; the order of indices whose values compare equal is unspecified.
- `find_ptr(m, k)` returns a pointer to the value that key `k` maps to in the associative container `m`, or `nullptr` if the key is absent. The pointer is const only when `m` is, so `if (auto *p = find_ptr(m, k))` both tests for the key and updates it in a single lookup.
- `get_or(m, k, def = {})` returns the value that key `k` maps to in the associative container `m`, or `def` if the key is absent. Unlike `operator[]`, it never inserts and works on a const `m`.
- `erase_one(c, x)` erases one copy of `x` from associative container `c`, asserting that it is present.
- `min_heap<T>` is a min-heap alias.
- `y_combinator(f)` wraps a recursive lambda so that it can call itself as its first argument.

```
#include <algorithm>
```

```

#include <cassert>
#include <functional>
#include <iterator>
#include <numeric>
#include <queue>
#include <type_traits>
#include <utility>
#include <vector>

#define Rep(i, N)    for (int i = 0, _##i = (N); i < _##i; ++i)    // 0 to N-1
#define For(i, L, H) for (int i = (L), _##i = (H); i <= _##i; ++i) // L to H
#define Rev(i, N)    for (int i = (N); --i >= 0;)                // N-1 to 0
#define Dwn(i, H, L) for (int i = (H), _##i = (L); i >= _##i; --i) // H to L
#define Each(x, C)   for (auto &x : (C))
#define All(C)       std::begin(C), std::end(C)
#define Rall(C)      std::rbegin(C), std::rend(C)

template<typename C> int sz(const C &c) { return static_cast<int>(c.size()); }
template<typename T, typename U> bool ckmin(T &a, const U &b) { return b < a && (a = b, true); }
template<typename T, typename U> bool ckmax(T &a, const U &b) { return a < b && (a = b, true); }

template<typename T>
T floor_div(T a, T b) {
    assert(b != 0);
    T q = a / b, r = a % b;
    return q - (r != 0 && ((r < 0) != (b < 0)));
}

template<typename T>
T ceil_div(T a, T b) {
    assert(b != 0);
    T q = a / b, r = a % b;
    return q + (r != 0 && ((r < 0) == (b < 0)));
}

template<typename T>
std::vector<T> make_vec(int n, const T &v) {
    return std::vector<T>(n, v);
}

template<typename... Args>
auto make_vec(int n, Args... args) {
    auto inner = make_vec(args...);
    return std::vector<decltype(inner)>(n, inner);
}

std::vector<int> indices(int n) {
    std::vector<int> v(n);
    std::iota(v.begin(), v.end(), 0);
    return v;
}

template<typename Seq, typename T, typename Compare = std::less<>>
int lb(const Seq &a, const T &x, Compare comp = Compare{}) {
    return static_cast<int>(std::lower_bound(a.begin(), a.end(), x, comp) - a.begin());
}

template<typename Seq, typename T, typename Compare = std::less<>>
int ub(const Seq &a, const T &x, Compare comp = Compare{}) {
    return static_cast<int>(std::upper_bound(a.begin(), a.end(), x, comp) - a.begin());
}

template<typename T, typename Compare = std::less<>>
void sort_unique(std::vector<T> &v, Compare comp = Compare{}) {
    std::sort(v.begin(), v.end(), comp);
    auto same = [&](const T &a, const T &b) { return !comp(a, b) && !comp(b, a); };
    v.erase(std::unique(v.begin(), v.end(), same), v.end());
}

```

```

template<typename Seq, typename Compare = std::less<>>
std::vector<int> argsort(const Seq &a, Compare comp = Compare{}) {
    std::vector<int> p = indices(static_cast<int>(a.size()));
    std::sort(p.begin(), p.end(), [&](int i, int j) { return comp(a[i], a[j]); });
    return p;
}

template<typename M>
auto find_ptr(M &m, const typename M::key_type &k) {
    auto it = m.find(k);
    return it == m.end() ? nullptr : &it->second;
}

template<typename M>
typename M::mapped_type get_or(
    const M &m, const typename M::key_type &k, const typename M::mapped_type &def = {}
) {
    auto it = m.find(k);
    return it == m.end() ? def : it->second;
}

template<typename C, typename T>
void erase_one(C &c, const T &x) {
    auto it = c.find(x);
    assert(it != c.end());
    c.erase(it);
}

template<typename T>
using min_heap = std::priority_queue<T, std::vector<T>, std::greater<T>>;

template<typename Fn>
class y_combinator_result {
    Fn fn;

public:
    template<typename T>
    explicit y_combinator_result(T &&f) : fn(std::forward<T>(f)) {}

    template<typename... Args>
    decltype(auto) operator()(Args &&...args) {
        return fn(std::ref(*this), std::forward<Args>(args)...);
    }
};

template<typename Fn>
decltype(auto) y_combinator(Fn &&f) {
    return y_combinator_result<std::decay_t<Fn>>(std::forward<Fn>(f));
}

```

## Example Usage

```

#include <map>
#include <set>
using namespace std;

int main() {
    int tot = 0;
    Rep(i, 4) tot += i;    // 0 + 1 + 2 + 3 = 6
    For(i, 2, 5) tot += i; // 6 + (2 + 3 + 4 + 5) = 20
    Rev(i, 3) tot += i;    // 20 + (2 + 1 + 0) = 23
    Dwn(i, 8, 6) tot += i; // 23 + (8 + 7 + 6) = 44
    assert(tot == 44);

    int x = 10;
}

```

```

assert(ckmin(x, 7) && x == 7);
assert(!ckmin(x, 9) && x == 7);
assert(ckmax(x, 11) && x == 11);

vector<int> v{1, 2, 3};
assert(sz(v) == 3);
sort(Rall(v));
assert((v == vector<int>{3, 2, 1}));
assert(lb(v, 2, greater<>{}) == 1 && ub(v, 2, greater<>{}) == 2);
sort(All(v));
assert(lb(v, 2) == 1 && ub(v, 2) == 2);
assert(floor_div(7, -3) == -3);
assert(floor_div(-7, 3) == -3);
assert(ceil_div(7, -3) == -2);
assert(ceil_div(-7, 3) == -2);

auto grid = make_vec(2, 3, -1);
grid[0][0] = 7;
assert(sz(grid) == 2 && sz(grid[1]) == 3 && grid[1][0] == -1);
v = {3, 1, 3, 2, 1};
sort_unique(v);
assert((v == vector<int>{1, 2, 3}));
v = {3, 1, 3, 2, 1};
sort_unique(v, greater<>{});
assert((v == vector<int>{3, 2, 1}));
assert((indices(4) == vector<int>{0, 1, 2, 3}));
assert((argsort(vector<int>{30, 10, 20}) == vector<int>{1, 2, 0}));
assert((argsort(vector<int>{30, 10, 20}, greater<>{}) == vector<int>{0, 2, 1}));

map<int, int> cnt{{2, 5}};
if (int *p = find_ptr(cnt, 2)) {
    *p += 10; // A single lookup both tests for the key and updates it.
}
const map<int, int> &view = cnt; // A const container yields a pointer to const.
assert(*find_ptr(view, 2) == 15 && find_ptr(view, 7) == nullptr);
assert(get_or(cnt, 2) == 15 && get_or(cnt, 7, -1) == -1 && sz(cnt) == 1);
min_heap<int> pq;
pq.push(3);
pq.push(1);
assert(pq.top() == 1);

multiset<int> ms{1, 2, 2, 3};
erase_one(ms, 2);
assert(ms.count(2) == 1);

// Recursive lambda with y_combinator.
auto fib =
    y_combinator([](auto fib, int n) -> int { return n < 2 ? n : fib(n - 1) + fib(n - 2); });
assert(fib(10) == 55);

// Recursive lambda without y_combinator: pass the lambda to itself as the first argument.
vector<vector<int>> adj{{1, 2}, {0}, {0}};
int seen = 0;
auto dfs = [&](auto &&dfs, int u, int p) -> void {
    seen++;
    Each(v, adj[u]) if (v != p) dfs(dfs, v, u);
};
dfs(dfs, 0, -1);
assert(seen == 3);

// Recursive lambdas are automatically supported in C++23.
#if __cplusplus >= 202302L
auto fact = [&](this auto fact, int n) -> int { return n ? n * fact(n - 1) : 1; };
assert(fact(5) == 120);
#endif
return 0;
}

```

## 8.2 Fast IO

---

Fast input and output helpers. For most cases, it is enough to speed up standard I/O simply with `ios::sync_with_stdio(false); cin.tie(nullptr);`. The following classes only come into play when input is genuinely huge or `iostream` overhead is measurable.

- `set_in(name)`, `set_out(name)`, and `set_io(iname, oname)` redirect standard input/output to files. For example, `set_io("task.in", "task.out")` is convenient for USACO-style problems. An empty name leaves that stream untouched, so `set_io("", "")` is a no-op that can stay in place when the same solution is submitted to a judge that pipes its input.

File-redirectation failures throw `std::runtime_error`.

```
#include <cassert>
#include <cctype>
#include <cstdint>
#include <cstdio>
#include <limits>
#include <stdexcept>
#include <string>
#include <type_traits>

void set_in(const std::string &name) {
    if (!name.empty() && !freopen(name.c_str(), "r", stdin)) {
        throw std::runtime_error("Failed to open input file: " + name);
    }
}

void set_out(const std::string &name) {
    if (!name.empty() && !freopen(name.c_str(), "w", stdout)) {
        throw std::runtime_error("Failed to open output file: " + name);
    }
}

void set_io(const std::string &iname, const std::string &oname) {
    set_in(iname);
    set_out(oname);
}
```

When only integers are read, one loop over a character reader is the whole technique and is short enough to copy or retype: skip anything that starts no number, then fold digits in with  $x * 10 + d$ . Which character reader it uses decides everything, since the per-character mutex that the locked `getchar()` takes costs more than the parsing does. The example below times the alternatives against each other on the same input.

- `read_int(x)` reads one integer from `stdin` into `x`, skipping any leading character that cannot start a number. A leading `-` is honored, so `Int` must be signed for negative input. Behavior is undefined at end of file.

```
// The unlocked reader is where nearly all of the speed comes from. It is POSIX, not standard C++.
inline int read_char() {
#ifdef WIN32
    return _getchar_nolock();
#elif defined(__unix__) || defined(__APPLE__)
    return getchar_unlocked();
#else
    return getchar();
#endif
}

template<typename Int>
void read_int(Int &x) {
    int c, s = 1;
    while ((c = read_char()) != '-' && (c < '0' || c > '9'));
    if (c == '-') s = -1, c = read_char();
```

```

for (x = 0; '0' <= c && c <= '9'; c = read_char()) x = x * 10 + c - '0';
x *= s;
}

```

`FastInput` reads large byte blocks with `fread()` and parses tokens from an internal buffer.

- `FastInput in(file = stdin)` constructs a faster reader from a `FILE*`.
- `in >> x` reads a non-whitespace token into `char`, `std::string`, integral types, or floating-point types.

Detected read failures throw `std::runtime_error`. End-of-file is not an error, and the parser otherwise assumes valid input. Floating-point input is provided for convenience, not as the main performance path.

```

class FastInput {
    static constexpr int BUF_SIZE = 1 << 20;
    FILE *file;
    char buf[BUF_SIZE];
    int pos = 0, len = 0;

    char get_char() {
        if (pos == len) {
            len = static_cast<int>(fread(buf, 1, BUF_SIZE, file));
            pos = 0;
            if (len == 0) {
                if (ferror(file)) {
                    throw std::runtime_error("Failed to read input.");
                }
                return 0;
            }
        }
        return buf[pos++];
    }

    void skip_blanks() {
        while (true) {
            char c = get_char();
            if (!c) {
                return;
            }
            if (!std::isspace(static_cast<unsigned char>(c))) {
                --pos;
                return;
            }
        }
    }

public:
    explicit FastInput(FILE *file_ = stdin) : file(file_) {}

    FastInput(const FastInput &) = delete;
    FastInput &operator=(const FastInput &) = delete;

    FastInput &operator>>(char &c) {
        skip_blanks();
        c = get_char();
        return *this;
    }

    FastInput &operator>>(std::string &s) {
        skip_blanks();
        s.clear();
        while (true) {
            char c = get_char();
            if (!c || std::isspace(static_cast<unsigned char>(c))) {
                break;
            }
            s += c;
        }
    }
}

```



```

    }
    return *this;
}

FastInput &operator>>(bool &x) {
    int val;
    *this >> val;
    x = val != 0;
    return *this;
}

template<typename T>
typename std::enable_if<
    std::is_integral<T>::value && !std::is_same<T, bool>::value, FastInput &>::type
operator>>(T &x) {
    skip_blanks();
    char c = get_char();
    bool neg = false;
    if (c == '-') {
        neg = true;
        c = get_char();
    }
    using U = typename std::make_unsigned<T>::type;
    U val = 0;
    while (c && !std::isspace(static_cast<unsigned char>(c))) {
        val = U{10} * val + U(c - '0');
        c = get_char();
    }
    if constexpr (std::is_signed<T>::value) {
        x = neg ? T(U{0} - val) : T(val);
    } else {
        x = T(val);
    }
    return *this;
}

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, FastInput &>::type operator>>(T &x) {
    std::string s;
    *this >> s;
    x = static_cast<T>(std::strtold(s.c_str(), nullptr));
    return *this;
}
};

```

`FastOutput` formats values into an internal buffer and writes them in large blocks with `fwrite()`.

- `FastOutput out(file = stdout)` constructs a faster writer to a `FILE*`.
- `out << x` writes `char`, C strings, `std::string`, integral types, or floating-point types.
- `out.flush()` writes any buffered output immediately.

Detected write failures throw `std::runtime_error`. Destruction flushes on a best-effort basis; call `flush()` explicitly to detect a final write failure. Floating-point output is provided for convenience, not as the main performance path.

```

class FastOutput {
    static constexpr int BUF_SIZE = 1 << 20;
    FILE *file;
    char buf[BUF_SIZE];
    int pos = 0;

    bool flush_buf() {
        int n = pos;
        pos = 0;
        return fwrite(buf, 1, n, file) == static_cast<std::size_t>(n);
    }
}

```

```

void put_char(char c) {
    if (pos == BUF_SIZE) {
        flush();
    }
    buf[pos++] = c;
}

public:
explicit FastOutput(FILE *file_ = stdout) : file(file_) {}

~FastOutput() { flush_buf(); }
FastOutput(const FastOutput &) = delete;
FastOutput &operator=(const FastOutput &) = delete;

void flush() {
    if (!flush_buf()) {
        throw std::runtime_error("Failed to write output.");
    }
}

FastOutput &operator<<(char c) {
    put_char(c);
    return *this;
}

FastOutput &operator<<(const char *s) {
    while (*s) {
        put_char(*s++);
    }
    return *this;
}

FastOutput &operator<<(const std::string &s) {
    for (char c : s) {
        put_char(c);
    }
    return *this;
}

FastOutput &operator<<(bool x) {
    put_char(x ? '1' : '0');
    return *this;
}

template<typename T>
typename std::enable_if<
    std::is_integral<T>::value && !std::is_same<T, bool>::value, FastOutput &>::type
operator<<(T x) {
    using U = typename std::make_unsigned<T>::type;
    U val = U(x);
    if constexpr (std::is_signed<T>::value) {
        if (x < 0) {
            put_char('-');
            val = U{0} - val;
        }
    }
    char s[std::numeric_limits<U>::digits10 + 1];
    int n = 0;
    do {
        s[n++] = char('0' + val % 10);
        val /= 10;
    } while (val);
    while (n-- > 0) {
        put_char(s[n]);
    }
    return *this;
}

```

```

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, FastOutput &>::type operator<<(T x) {
    char s[64];
    std::snprintf(
        s, sizeof(s), "%.Lg", std::numeric_limits<T>::max_digits10, static_cast<long double>(x)
    );
    return *this << s;
}
};

```

## Example Usage

```

#include <ctime>
#include <filesystem>
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    // set_io("file.in", "file.out");

    FILE *input = tmpfile();
    fputs("42 hello 3.5 -2147483648 1 Z 1234567890123\n", input);
    rewind(input);

    FastInput in(input);
    int x;
    string s;
    double y;
    int z;
    bool flag;
    char ch;
    long long big;
    in >> x >> s >> y >> z >> flag >> ch >> big;
    assert(x == 42 && s == "hello" && y == 3.5 && z == -2147483648);
    assert(flag && ch == 'Z' && big == 1234567890123LL);
    fclose(input);

    FILE *output = tmpfile();
    {
        FastOutput out(output);
        out << x << ' ' << s << ' ' << y << ' ' << z << ' ' << flag << ' ' << ch << ' ' << big << '\n';
        out.flush();
    }
    rewind(output);
    char buf[96] = {};
    assert(fgets(buf, sizeof(buf), output));
    assert(string(buf) == "42 hello 3.5 -2147483648 1 Z 1234567890123\n");
    fclose(output);

    // Benchmark various ways of reading the same million integers from a scratch file.
    const int n = 1000000;
    string text;
    for (int i = 0; i < n; i++) {
        text += to_string(i) + ' ';
    }
    string path = (filesystem::temp_directory_path() / "fast_io.tmp").string();
    ofstream(path) << text;
    auto time_reads = [&](const char *name, auto read_one) {
        set_in(path);
        clock_t start = clock();
        for (int i = 0, v = 0; i < n; i++) {
            read_one(v);
        }
        printf("%-22s %6.3f s\n", name, double(clock() - start) / CLOCKS_PER_SEC);
    };
}

```

```

};
printf("Reading %d integers:\n", n);
time_reads("cin", [](int &v) { cin >> v; });
time_reads("scanf", [](int &v) { scanf("%d", &v); });
time_reads("read_int (unlocked)", [](int &v) { read_int(v); });
time_reads("FastInput", [](int &v) {
    static FastInput in;
    in >> v;
});
remove(path.c_str());
return 0;
}

```

### Example Output

```

Reading 1000000 integers:
cin          1.039 s
scanf        0.051 s
read_int (unlocked) 0.011 s
FastInput    0.026 s

```

## 8.3 Debugging

Local-only debug printing for contest code. `dbg`, `pr`, `dbg_rows`, and `pr_rows` expand to no-ops unless `LOCAL` is defined (e.g. via the compiler flag `-DLOCAL`), so they can remain in submitted code without producing output.

- `dbg_repr(x)` returns a string representation of arithmetic types, strings, pairs, tuples, iterable containers, and custom types that support insertion into `std::ostream`.
- `dbg(a, b, c)` prints the function name, line number, argument names, and values to `std::cerr`.
- `pr(a, b, c)` prints only the values to `std::cerr`; `pr()` prints a blank line.
- `dbg_rows(rows, col_width, limit = -1)` prints the function name, line number, argument name, and each iterable row on a separate indexed line. A positive `col_width` right-aligns each element in a field of that width. Pass 0 for ordinary container formatting. A nonnegative `limit` prints at most that many rows and elements per row, marking omitted entries with `...`.
- `pr_rows(rows, col_width, limit = -1)` prints only the indexed rows with the same formatting.

Avoid naming these helpers `to_string()` unless you want them mixed into overload resolution with `std::to_string()` and other user-defined `to_string()` functions. A separate name keeps debug formatting local to this snippet.

```

#include <cstdint>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <type_traits>
#include <utility>

template<typename T, typename = void>
struct is_iterable : std::false_type {};

template<typename T>
struct is_iterable<
    T, std::void_t<decltype(std::begin(std::declval<T>())), decltype(std::end(std::declval<T>()))>>
    : std::true_type {};

template<typename T, typename = void>
struct is_streamable : std::false_type {};

template<typename T>

```

```

struct is_streamable<
    T, std::void_t<decltype(std::declval<std::ostream &>() << std::declval<const T &>())>>
    : std::true_type {};
```

```

std::string dbg_repr(const std::string &s) { return '"' + s + '"'; }
std::string dbg_repr(const char *s) { return dbg_repr(std::string(s)); }
std::string dbg_repr(char c) { return std::string("'") + c + "'"; }
std::string dbg_repr(bool b) { return b ? "true" : "false"; }
```

```

template<typename T>
typename std::enable_if<
    std::is_arithmetic<T>::value && !std::is_same<T, char>::value &&
    !std::is_same<T, bool>::value && !std::is_floating_point<T>::value,
    std::string>::type
dbg_repr(T x) {
    std::ostringstream out;
    out << x;
    return out.str();
}
```

```

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, std::string>::type dbg_repr(T x) {
    std::ostringstream out;
    out << std::setprecision(10) << x; // Set floating point dbg precision here.
    return out.str();
}
```

```

template<typename T>
typename std::enable_if<
    is_streamable<T>::value && !is_iterable<T>::value && !std::is_arithmetic<T>::value &&
    !std::is_convertible<T, std::string>::value,
    std::string>::type
dbg_repr(const T &x) {
    std::ostringstream out;
    out << x;
    return out.str();
}
```

```

template<typename A, typename B>
std::string dbg_repr(const std::pair<A, B> &p) {
    return "(" + dbg_repr(p.first) + ", " + dbg_repr(p.second) + ")";
}
```

```

template<typename Tuple, std::size_t... Is>
std::string dbg_tuple_repr(const Tuple &t, std::index_sequence<Is...>) {
    std::string res = "(";
    bool first = true;
    ((res += (first ? (first = false, "") : ", ") + dbg_repr(std::get<Is>(t))), ...);
    return res + ")";
}
```

```

template<typename... Ts>
std::string dbg_repr(const std::tuple<Ts...> &t) {
    return dbg_tuple_repr(t, std::index_sequence_for<Ts...>{});
}
```

```

template<typename T>
typename std::enable_if<
    is_iterable<T>::value && !std::is_convertible<T, std::string>::value, std::string>::type
dbg_repr(const T &v) {
    std::string res = "{";
    bool first = true;
    for (const auto &x : v) {
        if (!first) {
            res += ", ";
        }
        first = false;
        res += dbg_repr(x);
    }
    return res + "}";
}
```

```

    }
    return res + "}";
}

template<typename Head, typename... Tail>
void dbg_out(bool leading_space, const Head &head, const Tail &...tail) {
    if (leading_space) {
        std::cerr << ' ';
    }
    std::cerr << dbg_repr(head);
    ((std::cerr << ' ' << dbg_repr(tail)), ...);
    std::cerr << '\n';
}

void pr_out() {
    std::cerr << '\n';
}

template<typename Head, typename... Tail>
void pr_out(const Head &head, const Tail &...tail) {
    dbg_out(false, head, tail...);
}

template<typename Rows>
void dbg_rows_out(const Rows &rows, int col_width = 0, int limit = -1) {
    int i = 0;
    for (const auto &row : rows) {
        if (i == limit) {
            std::cerr << "...\\n";
            break;
        }
        std::cerr << i++ << ':' << (col_width > 0 ? "" : " {");
        int j = 0;
        for (const auto &x : row) {
            if (j == limit) {
                std::cerr << (col_width > 0 ? " ..." : (j == 0 ? "... : ", "..."));
                break;
            }
            std::cerr << (col_width > 0 ? " " : (j == 0 ? "" : ", ")) << std::setw(col_width)
                << dbg_repr(x);
            j++;
        }
        std::cerr << (col_width > 0 ? "\\n" : "}\\n");
    }
}

#ifdef LOCAL
#define dbg(...) \
    std::cerr << "[" << __func__ << ":" << __LINE__ << "]" " " << #__VA_ARGS__ << ":", \
    dbg_out(true, __VA_ARGS__)
#define pr(...) pr_out(__VA_ARGS__)
#define dbg_rows(rows, ...) \
    std::cerr << "[" << __func__ << ":" << __LINE__ << "]" " " << #rows << ":\n", \
    dbg_rows_out(rows, __VA_ARGS__)
#define pr_rows(rows, ...) dbg_rows_out(rows, __VA_ARGS__)
#else
#define dbg(...) ((void)0)
#define pr(...) ((void)0)
#define dbg_rows(...) ((void)0)
#define pr_rows(...) ((void)0)
#endif

```

## Example Usage

```

#include <cassert>
#include <map>

```

```

#include <vector>
using namespace std;

struct Point {
    int x, y;
};

// Can overload dbg_repr to dbg your custom structs.
string dbg_repr(const Point &p) {
    return "Point(" + dbg_repr(p.x) + ", " + dbg_repr(p.y) + ")";
}

struct Edge {
    int u, v, w;
};

string dbg_repr(const Edge &e) {
    return "Edge{" + dbg_repr(e.u) + "->" + dbg_repr(e.v) + ", w=" + dbg_repr(e.w) + "}";
}

struct Streamable {
    int value;

    friend ostream &operator<<(ostream &out, const Streamable &x) {
        return out << "Streamable(" << x.value << ")";
    }
};

int main() {
    vector<int> v{1, 2, 3};
    auto p = make_pair(4, string("x"));
    auto t = make_tuple(1, true, string("ok"));
    map<string, int> mp{{"a", 1}, {"b", 2}};
    Point pt{2, 3};
    Edge e{0, 1, 5};
    double d = 3.141592653589793;

    assert(dbg_repr(v) == "{1, 2, 3}");
    assert(dbg_repr(p) == "(4, \"x\")");
    assert(dbg_repr(t) == "(1, true, \"ok\")");
    assert(dbg_repr(mp) == "{(\"a\", 1), (\"b\", 2)}");
    assert(dbg_repr(pt) == "Point(2, 3)");
    assert(dbg_repr(e) == "Edge{0->1, w=5}");
    assert(dbg_repr(Streamable{6}) == "Streamable(6)");
    assert(dbg_repr(vector<Streamable>{{7}, {8}}) == "{Streamable(7), Streamable(8)}");
    assert(dbg_repr(1.0 / 3.0) == "0.3333333333");
    assert(dbg_repr(d) == "3.141592654");

    dbg(v);
    dbg(p, t, mp);
    dbg(pt, e);
    pr("plain output", d);

    vector<vector<int>> adj{{1, 2}, {0}, {0}};
    pr();
    dbg_rows(adj, 0);

    vector<vector<int>> grid{{1, 20, -3}, {400, 5, 6}, {7, 8, 9}};
    pr();
    dbg_rows(grid, 3, 2);
    return 0;
}

```

**Example Output**

```
[main:250] v: {1, 2, 3}
[main:251] p, t, mp: (4, "x") (1, true, "ok") {("a", 1), ("b", 2)}
[main:252] pt, e: Point(2, 3) Edge{0->1, w=5}
"plain output" 3.141592654

[main:257] adj:
0: {1, 2}
1: {0}
2: {0}

[main:261] grid:
0:  1  20 ...
1: 400  5 ...
...
```

## 8.4 Stress-Testing

---

Provides random input generators and a harness that compares an optimized solution against a trusted brute force implementation. Small tests keep brute force practical and produce manageable counterexamples. The generators create common contest inputs such as distinct samples, permutations, graphs, trees, and integer compositions for stress tests, randomized tests, and benchmarks.

- `rng` is a global 64-bit Mersenne Twister. Call `rng.seed(seed)` before a stress test to make it reproducible. Use an explicit seed so generated failures are reproducible.
- `rand_int(lo, hi)` returns a uniformly random integer in range `[lo, hi]`.
- `rand_biased(lo, hi, bias)` returns an integer in `[lo, hi]`. Positive `bias` favors larger values by taking the maximum of `bias + 1` samples, negative `bias` similarly favors smaller values, and zero is uniform.
- `rand_real(lo = 0.0, hi = 1.0)` returns a uniformly random real number in the half-open range `[lo, hi)`.
- `rand_choice(values)` returns a uniformly random element of the nonempty vector `values`.
- `rand_weighted(weights)` returns a sampled 0-based index with probability proportional to its nonnegative weight.
- `rand_vec(n, lo, hi)` returns `n` independent uniformly random integers in `[lo, hi]`.
- `rand_str(n, alphabet)` returns a length-`n` string whose characters are sampled independently and uniformly from the nonempty string `alphabet`.
- `rand_perm(n, first = 0)` returns a random permutation of the `n` consecutive integers beginning at `first`.
- `rand_distinct(k, lo, hi)` returns `k` distinct integers sampled uniformly from `[lo, hi]`.
- `rand_tree(n)` returns a uniformly random labeled tree on nodes `[0, n)` as an edge list using a Prüfer code.
- `rand_graph(n, m, directed = false, connected = false)` returns a random simple graph with `n` nodes numbered `[0, n)` and exactly `m` edges. Without connectivity it is uniform; with connectivity it starts from `rand_tree(n)` and is not uniform over connected graphs. A connected directed graph is weakly connected.
- `rand_composition(parts, sum, min_part)` uniformly splits `sum` into `parts` ordered integers, each at least `min_part`.

### Time Complexity:

- $O(1)$  per call to `rand_int()`, `rand_real()`, and `rand_choice()`.
- $O(|b| + 1)$  per call to `rand_biased()` where `b` is the bias.
- $O(n)$  per call to `rand_weighted()`, `rand_vec()`, `rand_str()`, and `rand_perm()`, where `n` is the number of weights or generated elements.
- $O(k)$  expected per call to `rand_distinct()`.
- $O(n \log n)$  per call to `rand_tree()`.
- $O(m)$  expected per call to `rand_graph()` without connectivity, or  $O(n \log n + m)$  with it.
- $O(k \log k)$  per call to `rand_composition()`, where `k` is `parts`.



**Space Complexity:**

- $O(1)$  auxiliary for the scalar generators and `rand_choice()`.
- $O(n)$  auxiliary for `rand_weighted()`.
- $O(n)$  for the values returned by `rand_vec()`, `rand_str()`, `rand_perm()`, and `rand_tree()`.
- $O(k)$  auxiliary and output space for `rand_distinct()` and `rand_composition()`.
- $O(m)$  auxiliary and output space for `rand_graph()`.

```
#include <algorithm>
#include <cassert>
#include <climits>
#include <cmath>
#include <cstdint>
#include <functional>
#include <initializer_list>
#include <numeric>
#include <optional>
#include <queue>
#include <random>
#include <string>
#include <type_traits>
#include <unordered_set>
#include <utility>
#include <vector>

std::mt19937_64 rng;

template<typename Int>
Int rand_int(Int lo, Int hi) {
    static_assert(std::is_integral<Int>::value, "rand_int() requires an integral type");
    assert(lo <= hi);
    return std::uniform_int_distribution<Int>(lo, hi)(rng);
}

template<typename Int>
Int rand_biased(Int lo, Int hi, int bias) {
    Int value = rand_int(lo, hi);
    while (bias > 0) {
        value = std::max(value, rand_int(lo, hi));
        bias--;
    }
    while (bias < 0) {
        value = std::min(value, rand_int(lo, hi));
        bias++;
    }
    return value;
}

double rand_real(double lo = 0.0, double hi = 1.0) {
    assert(lo < hi);
    return std::uniform_real_distribution<double>(lo, hi)(rng);
}

template<typename T>
T rand_choice(const std::vector<T> &values) {
    assert(!values.empty() && values.size() <= INT_MAX);
    return values[rand_int(0, static_cast<int>(values.size()) - 1)];
}

template<typename W>
int rand_weighted(const std::vector<W> &weights) {
    static_assert(std::is_arithmetic<W>::value, "weights must be numeric");
    assert(!weights.empty() && weights.size() <= INT_MAX);
    std::vector<double> probabilities;
    probabilities.reserve(weights.size());
    double total = 0;
    for (const W &value : weights) {
```

```

    double weight = static_cast<double>(value);
    assert(std::isfinite(weight) && weight >= 0);
    probabilities.push_back(weight);
    total += weight;
}
assert(std::isfinite(total) && total > 0);
return std::discrete_distribution<int>(probabilities.begin(), probabilities.end())(rng);
}

template<typename W>
int rand_weighted(std::initializer_list<W> weights) {
    return rand_weighted(std::vector<W>(weights));
}

template<typename Int>
std::vector<Int> rand_vec(int n, Int lo, Int hi) {
    assert(n >= 0 && lo <= hi);
    std::vector<Int> values(n);
    for (Int &x : values) {
        x = rand_int(lo, hi);
    }
    return values;
}

std::string rand_str(int n, const std::string &alphabet) {
    assert(n >= 0 && !alphabet.empty() && alphabet.size() <= INT_MAX);
    std::string s(n, ' ');
    for (char &c : s) {
        c = alphabet[rand_int(0, static_cast<int>(alphabet.size()) - 1)];
    }
    return s;
}

std::vector<int> rand_perm(int n, int first = 0) {
    assert(n >= 0 && (n == 0 || static_cast<int64_t>(first) + n - 1 <= INT_MAX));
    std::vector<int> p(n);
    std::iota(p.begin(), p.end(), first);
    std::shuffle(p.begin(), p.end(), rng);
    return p;
}

std::vector<int> rand_distinct(int k, int lo, int hi) {
    int64_t count = static_cast<int64_t>(hi) - lo + 1;
    assert(lo <= hi && 0 <= k && k <= count);
    std::unordered_set<int> used;
    used.reserve(k);
    std::vector<int> values;
    values.reserve(k);
    // Floyd's algorithm avoids materializing the entire interval.
    for (int64_t j = static_cast<int64_t>(hi) - k + 1; j <= hi; ++j) {
        int x = rand_int(lo, static_cast<int>(j));
        if (!used.insert(x).second) {
            x = static_cast<int>(j);
            used.insert(x);
        }
        values.push_back(x);
    }
    std::shuffle(values.begin(), values.end(), rng);
    return values;
}

std::vector<std::pair<int, int>> rand_tree(int n) {
    assert(n >= 1);
    if (n == 1) {
        return {};
    }
    std::vector<int> degree(n, 1), code(n - 2);
    for (int &u : code) {

```

```

    degree[u = rand_int(0, n - 1)]++;
}
std::priority_queue<int, std::vector<int>, std::greater<int>> leaves;
for (int u = 0; u < n; ++u) {
    if (degree[u] == 1) {
        leaves.push(u);
    }
}
std::vector<std::pair<int, int>> edges;
for (int u : code) {
    int leaf = leaves.top();
    leaves.pop();
    edges.emplace_back(leaf, u);
    if (--degree[u] == 1) {
        leaves.push(u);
    }
}
int u = leaves.top();
leaves.pop();
edges.emplace_back(u, leaves.top());
std::shuffle(edges.begin(), edges.end(), rng);
return edges;
}

std::vector<std::pair<int, int>> rand_graph(
    int n, int m, bool directed = false, bool connected = false
) {
    assert(n >= 0 && m >= 0 && (!connected || n >= 1));
    int64_t total =
        directed ? static_cast<int64_t>(n) * (n - 1) : static_cast<int64_t>(n) * (n - 1) / 2;
    int required = connected ? n - 1 : 0;
    assert(required <= m && m <= total);
    std::vector<std::pair<int, int>> edges;
    if (connected) {
        edges = rand_tree(n);
    }
    std::unordered_set<int64_t> required_edges;
    required_edges.reserve(required);
    for (auto &[u, v] : edges) {
        if ((directed && rand_int(0, 1)) || (!directed && u > v)) {
            std::swap(u, v);
        }
        required_edges.insert(static_cast<int64_t>(u) * n + v);
    }
    int64_t available = total - required;
    int extra = m - required;
    bool complement = extra > available / 2;
    int sample = complement ? static_cast<int>(total - m) : extra;
    // For dense graphs, sample the omitted non-tree edges instead of the included ones.
    std::unordered_set<int64_t> sampled;
    sampled.reserve(sample);
    while (static_cast<int>(sampled.size()) < sample) {
        int u = rand_int(0, n - 1), v = rand_int(0, n - 2);
        if (v >= u) {
            v++;
        }
        if (!directed && u > v) {
            std::swap(u, v);
        }
        int64_t key = static_cast<int64_t>(u) * n + v;
        if (!required_edges.count(key)) {
            sampled.insert(key);
        }
    }
    edges.reserve(m);
    if (!complement) {
        for (int64_t key : sampled) {
            edges.emplace_back(static_cast<int>(key / n), static_cast<int>(key % n));
        }
    }
}

```

```

    }
} else {
    edges.clear();
    for (int u = 0; u < n; u++) {
        for (int v = directed ? 0 : u + 1; v < n; v++) {
            if (u != v && !sampled.count(static_cast<int64_t>(u) * n + v)) {
                edges.emplace_back(u, v);
            }
        }
    }
}
std::shuffle(edges.begin(), edges.end(), rng);
return edges;
}

std::vector<int> rand_composition(int parts, int sum, int min_part) {
    int64_t remaining = static_cast<int64_t>(sum) - static_cast<int64_t>(parts) * min_part;
    assert(parts >= 1 && min_part >= 0 && remaining >= 0 && remaining + parts - 2 <= INT_MAX);
    if (parts == 1) {
        return {sum};
    }
    auto cuts = rand_distinct(parts - 1, 0, static_cast<int>(remaining) + parts - 2);
    std::sort(cuts.begin(), cuts.end());
    std::vector<int> values;
    int prev = -1;
    for (int cut : cuts) {
        values.push_back(min_part + cut - prev - 1);
        prev = cut;
    }
    values.push_back(min_part + static_cast<int>(remaining) + parts - 2 - prev);
    return values;
}

```

A stress test compares an optimized implementation against a trusted brute force implementation on many generated inputs. Small tests make brute force practical and produce manageable counterexamples.

- `stress(trials, gen, solve, brute, equal = std::equal_to<>())` runs `trials` tests, calling `gen(trial)` with each 1-based trial number and comparing `solve(test)` with `brute(test)` using `equal`. It returns a `StressFailure` containing the number, first failing test, and actual and expected results. The generated test type must be copyable so each solution receives an independent input, and `equal` must accept the respective results of `solve(test)` and `brute(test)`. If all generated tests pass, it returns `std::nullopt`.

**Time Complexity:**  $O(t \cdot (G + X + S + B + C))$  per call in the worst case, where  $t$  is `trials`,  $X$  is the cost of copying a generated test, and  $G$ ,  $S$ ,  $B$ , and  $C$  are the costs of `gen`, `solve`, `brute`, and `equal`, respectively.

**Space Complexity:**  $O(x + a + e)$  for the test copies and results, where  $x$ ,  $a$ , and  $e$  are their respective sizes, in addition to the working space used by the callbacks.

```

template<typename Test, typename Actual, typename Expected>
struct StressFailure {
    int trial;
    Test test;
    Actual actual;
    Expected expected;
};

template<typename Generate, typename Solve, typename Brute, typename Equal = std::equal_to<>>
auto stress(int trials, Generate gen, Solve solve, Brute brute, Equal equal = {}) {
    assert(trials >= 0);
    using Test = std::decay_t<decltype(gen(1))>;
    using Actual = std::decay_t<decltype(solve(std::declval<Test &>()))>;
    using Expected = std::decay_t<decltype(brute(std::declval<Test &>()))>;
    using Failure = StressFailure<Test, Actual, Expected>;
    for (int trial = 1; trial <= trials; ++trial) {
        auto test = gen(trial);

```

```

    auto solve_test = test, brute_test = test;
    auto actual = solve(solve_test);
    auto expected = brute(brute_test);
    if (!equal(actual, expected)) {
        return std::optional<Failure>{
            Failure{trial, std::move(test), std::move(actual), std::move(expected)}
        };
    }
}
return std::optional<Failure>{};
}

```

## Example Usage

```

using namespace std;

bool EQ(double a, double b) {
    return a == b || fabs(a - b) <= 1e-9;
}

int main() {
    rng.seed(1234567); // Fixed seed for reproducibility.

    double real = rand_real(-1.0, 1.0);
    assert(-1.0 <= real && real < 1.0);
    int biased = rand_biased(1, 100000, 3);
    assert(1 <= biased && biased <= 100000);
    assert(rand_weighted({20, 20, 30}) < 3);
    auto values = rand_vec(8, -5, 5);
    assert(values.size() == 8 && all_of(values.begin(), values.end(), [](int x) {
        return -5 <= x && x <= 5;
    }));
    assert(rand_str(12, "abc").size() == 12);
    auto graph = rand_graph(5, 7);
    assert(graph.size() == 7 && all_of(graph.begin(), graph.end(), [](auto edge) {
        return edge.first < edge.second;
    }));
    assert(rand_tree(6).size() == 5);
    assert(rand_graph(6, 8, false, true).size() == 8);
    auto composition = rand_composition(4, 20, 2);
    assert(accumulate(composition.begin(), composition.end(), 0) == 20);
    assert(*min_element(composition.begin(), composition.end()) >= 2);

    auto generate = [](int trial) {
        int n = rand_int(1, 8), kind = trial <= 3 ? trial - 1 : rand_int(0, 2);
        switch (kind) {
            case 0:
                return rand_perm(n, 1);
            case 1:
                return rand_distinct(n, -5, 5);
            default:
                break;
        }
        static const vector<int> boundary_values{-5, 0, 5};
        bernoulli_distribution use_boundary(0.25);
        vector<int> a(n);
        for (int &x : a) {
            x = use_boundary(rng) ? rand_choice(boundary_values) : rand_int(-5, 5);
        }
        return a;
    };
    auto solve = [](vector<int> a) {
        sort(a.begin(), a.end());
        return a;
    };
    auto brute = [](vector<int> a) {

```

```

do {
    if (is_sorted(a.begin(), a.end())) {
        return a;
    }
} while (next_permutation(a.begin(), a.end()));
sort(a.begin(), a.end());
return a;
};

assert(!stress(100, generate, solve, brute));
assert(!stress(
    1, [](int) { return 0; }, [](int) { return 1.0; }, [](int) { return 1.0 + 1e-12; }, EQ
));

// Mutating solvers still receive independent copies of the generated test.
auto mutate = [](vector<int> &a) { return ++a[0]; };
auto unchanged = [](const vector<int> &a) { return a[0]; };
auto failure = stress(1, [](int) { return vector<int>{1}; }, mutate, unchanged);
assert(failure && failure->trial == 1 && failure->test == vector<int>{1});
assert(failure->actual == 2 && failure->expected == 1);
return 0;
}

```

## 8.5 Benchmarking

---

Small timing helpers for local profiling and codebook examples. Use `std::chrono::steady_clock` instead of wall-clock time so elapsed durations are monotonic.

- `Timer()` starts a timer immediately.
- `timer.reset()` restarts the timer.
- `timer.elapsed()` returns the elapsed time in milliseconds as a double.
- `benchmark(iterations, f)` runs `f()` `iterations` times and returns the average elapsed milliseconds per call. `iterations` must be positive.

Benchmarks should be treated as local measurements, not proofs of asymptotic performance. Warm-up, CPU frequency scaling, compiler optimization, and input choice can dominate tiny timings.

```

#include <cassert>
#include <chrono>

class Timer {
    using Clock = std::chrono::steady_clock;
    Clock::time_point start_time;

public:
    Timer() { reset(); }

    void reset() { start_time = Clock::now(); }

    double elapsed() const {
        return std::chrono::duration<double, std::milli>(Clock::now() - start_time).count();
    }
};

template<typename Fn>
double benchmark(int iterations, Fn f) {
    assert(iterations > 0);
    Timer timer;
    for (int i = 0; i < iterations; ++i) {
        f();
    }
    return timer.elapsed() / iterations;
}

```

```
}
```

## Example Usage

```
#include <cassert>
#include <numeric>
#include <vector>
using namespace std;

int main() {
    Timer timer;
    vector<int> v(1000);
    iota(v.begin(), v.end(), 0);
    assert(timer.elapsed() >= 0.0);

    volatile long long sink = 0;
    double avg_time = benchmark(5, [&] { sink += accumulate(v.begin(), v.end(), 0LL); });
    assert(avg_time >= 0.0);
    assert(sink > 0);
    return 0;
}
```

## 8.6 GNU Policy-Based Data Structures

---

GNU policy-based data structures (PBDS) provide non-standard associative containers and priority queues, including order-statistic trees and fast hash tables. They are not part of the C++ standard library, but they are available on many GNU C++ contest judges.

- `HashMap<K, V>` and `HashSet<K>` behave like faster, non-standard alternatives to `std::unordered_map<K, V>` and `std::unordered_set<K>`, using `gp_hash_table`. The default `SplitMix64Hasher` is for integer-like keys; pass a custom hash for other key types. It randomizes the hash seed to avoid weak adversarial integer hashes.
- `OrderedSet<T>` and `OrderedMap<K, V>` behave like `std::set<T>` and `std::map<K, V>` but also support `find_by_order(k)` and `order_of_key(x)`.
- `OrderedMultiset<T>` behaves like `std::multiset<T>` by storing duplicates as (value, unique ID) pairs.
- `OrderedMultimap<K, V>` behaves like `std::multimap<K, V>` by storing duplicate keys as (key, unique ID) pairs.
- `PairingHeap<T>` is a meldable priority queue; `push()` returns an iterator that can be passed to `modify()` or `erase()`.
- `BinaryHeap<T>`, `BinomialHeap<T>`, and `RcBinomialHeap<T>` are alternative GNU priority queue tags with the same interface.

The order-statistic operations use GNU PBDS conventions: `find_by_order(k)` is 0-based and `order_of_key(x)` returns the number of keys strictly less than `x`, even if `x` is absent. Raw PBDS ordered trees return iterators from `find_by_order(k)`; the multiset and multimap wrappers below unwrap that iterator into a value. The priority queue `join()` operation melds another heap into the current one. Since this depends on `__gnu_pbds`, keep a standard library fallback in mind when portability matters.

### Time Complexity:

- `HashMap` / `HashSet`: same expected bounds as `std::unordered_map` / `std::unordered_set` (expected  $O(1)$ , worst-case  $O(n)$ ).
- `OrderedSet` / `OrderedMap`: same as `std::set` / `std::map`, with  $O(\log n)$  for `find_by_order` and `order_of_key`.
- `OrderedMultiset` / `OrderedMultimap`: same as `std::multiset` / `std::multimap`, with  $O(\log n)$  for `find_by_order` and `order_of_key`.
- $O(1)$  amortized per call to `push()` and `join()` for `PairingHeap`.
- $O(\log n)$  amortized per call to `pop()`, `modify()`, and `erase()` for `PairingHeap`.

**Space Complexity:**  $O(n)$  for all containers.

```

#include <cassert>
#include <chrono>
#include <cstdint>
#include <cstddef>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <functional>
#include <utility>

struct SplitMix64Hasher {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    std::size_t operator()(uint64_t x) const {
        static const uint64_t RAND_SEED = std::chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + RAND_SEED);
    }
};

template<typename K, typename Hash = SplitMix64Hasher>
using HashSet = __gnu_pbds::gp_hash_table<K, __gnu_pbds::null_type, Hash>;

template<typename K, typename V, typename Hash = SplitMix64Hasher>
using HashMap = __gnu_pbds::gp_hash_table<K, V, Hash>;

template<typename K, typename V, typename Compare = std::less<K>>
using OrderedMap = __gnu_pbds::tree<
    K, V, Compare, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>;

template<typename T, typename Compare = std::less<T>>
using OrderedSet = OrderedMap<T, __gnu_pbds::null_type, Compare>;

template<typename T>
class OrderedMultiset {
    OrderedSet<std::pair<T, int>> tree;
    int timer = 0;

public:
    void insert(const T &x) { tree.insert({x, timer++}); }

    bool erase_one(const T &x) {
        auto it = tree.lower_bound({x, -1});
        if (it == tree.end() || it->first != x) {
            return false;
        }
        tree.erase(it);
        return true;
    }

    T find_by_order(int k) const {
        assert(0 <= k && k < size());
        return tree.find_by_order(k)->first;
    }

    int order_of_key(const T &x) const { return tree.order_of_key({x, -1}); }
    int size() const { return static_cast<int>(tree.size()); }
};

template<typename K, typename V>
class OrderedMultimap {
    OrderedMap<std::pair<K, int>, V> tree;
    int timer = 0;

```



```

public:
    void insert(const K &key, const V &value) { tree.insert({key, timer++}, value); }

    bool erase_one(const K &key) {
        auto it = tree.lower_bound({key, -1});
        if (it == tree.end() || it->first.first != key) {
            return false;
        }
        tree.erase(it);
        return true;
    }

    std::pair<K, V> find_by_order(int k) const {
        assert(0 <= k && k < size());
        auto it = tree.find_by_order(k);
        return {it->first.first, it->second};
    }

    int order_of_key(const K &x) const { return tree.order_of_key({x, -1}); }
    int size() const { return static_cast<int>(tree.size()); }
};

template<typename T, typename Compare = std::less<T>>
using PairingHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::pairing_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using BinaryHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::binary_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using BinomialHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::binomial_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using RcBinomialHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::rc_binomial_heap_tag>;

```

## Example Usage

```

int main() {
    OrderedSet<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(30);
    s.insert(20); // Duplicate ignored, like std::set.
    // Net new over std::set: order-statistic queries.
    assert(s.size() == 3);
    assert(*s.find_by_order(0) == 10);
    assert(*s.find_by_order(1) == 20);
    assert(s.order_of_key(25) == 2);
    assert(s.order_of_key(100) == 3);

    HashMap<int, int> hm;
    hm[10] = 1;
    hm[20] = 2;
    assert(hm[10] == 1);
    HashSet<int> hs;
    hs.insert(7);
    assert(hs.find(7) != hs.end());

    OrderedMultiset<int> ms;
    ms.insert(5);
    ms.insert(5);
    ms.insert(7);
    assert(ms.size() == 3);
    // Net new over std::multiset: order statistics with duplicates.
    assert(ms.order_of_key(7) == 2);
    assert(ms.find_by_order(1) == 5);
    assert(ms.erase_one(5));
}

```

```

assert(ms.order_of_key(7) == 1);
assert(ms.erase_one(5));
assert(!ms.erase_one(5));
assert(ms.find_by_order(0) == 7);

OrderedMultimap<int, char> mp;
mp.insert(5, 'a');
mp.insert(5, 'b');
mp.insert(7, 'c');
// Net new over std::multimap: order statistics with duplicate keys.
assert(mp.order_of_key(7) == 2);
assert(mp.find_by_order(1).first == 5);
assert(mp.erase_one(5));
assert(mp.order_of_key(7) == 1);
assert(mp.erase_one(5));
assert(!mp.erase_one(5));
assert(mp.find_by_order(0).first == 7);

PairingHeap<int> h1, h2;
auto it = h1.push(10);
auto erased = h1.push(5);
h1.modify(it, 20); // Net new over std::priority_queue: update through a handle.
h1.erase(erased); // Also net new: remove an arbitrary heap item through a handle.
h2.push(7);
h1.join(h2); // Also net new: meld another heap into this one.
assert(h2.empty());
assert(h1.top() == 20);
h1.pop();
assert(h1.top() == 7);
return 0;
}

```

## 8.7 Multithreading

---

Basic multithreading can speed up independent local work such as stress tests, batch experiments, or offline preprocessing. It is rarely useful in standard contest submissions, and many judges restrict threads, so use it only when the environment allows it.

- `parallel_for(n, threads, f)` runs `f(i)` once for each  $0 \leq i < n$ , splitting the indices among at most `threads` worker threads.
- `parallel_cases(cases, threads, solve)` solves independent test cases in parallel and returns the outputs in the original case order, using at most `threads` workers and calling `solve(case, output)` for each case.

Each call to `f(i)` must be safe to run concurrently with the others. If it writes shared state, use separate output slots or synchronization.

```

#include <algorithm>
#include <cassert>
#include <numeric>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

template<typename Fn>
void parallel_for(int n, int threads, Fn f) {
    threads = std::max(1, std::min(threads, n));
    std::vector<std::thread> workers;
    workers.reserve(threads);
    for (int id = 0; id < threads; ++id) {
        workers.emplace_back([=, &f] {
            for (int i = id; i < n; i += threads) {

```

```

        f(i);
    }
    });
}
for (auto &worker : workers) {
    worker.join();
}
}

template<typename Case, typename Solve>
std::vector<std::string> parallel_cases(const std::vector<Case> &cases, int threads, Solve solve) {
    std::vector<std::string> out(cases.size());
    parallel_for(static_cast<int>(cases.size()), threads, [&](int i) {
        std::ostringstream os;
        solve(cases[i], os);
        out[i] = os.str();
    });
    return out;
}

```

### Example Usage

```

using namespace std;

int main() {
    vector<long long> squares(1000);
    parallel_for(static_cast<int>(squares.size()), 4, [&](int i) { squares[i] = 1LL * i * i; });
    assert(squares[17] == 289);
    assert(accumulate(squares.begin(), squares.end(), 0LL) == 332833500);

    vector<int> cases{3, 1, 4, 2};
    auto out =
        parallel_cases(cases, 2, [](int x, ostream &os) { os << x << "^2 = " << x * x; });
    assert((out == vector<string>{"3^2 = 9", "1^2 = 1", "4^2 = 16", "2^2 = 4"}));
    return 0;
}

```

## 8.8 Bump Allocation

---

A bump allocator serves allocations consecutively from one fixed buffer and never reclaims individual objects. This is useful in local contest code that creates many small objects whose lifetimes all extend to the end of the program. Replacing global `operator new` preserves ordinary allocation call sites while avoiding the time and per-allocation metadata of a general-purpose allocator.

- `bump_alloc(size, align)` allocates `size` bytes with the requested alignment.
- `operator new(size)` allocates an aligned block of `size` bytes from the static buffer.
- `operator delete(ptr)` is a no-op; memory is reclaimed only when the program exits.
- `BumpAllocator<T>` provides the same monotonic allocation strategy to STL containers.

When only one object type is allocated, a typed pool such as `Node nodes[MAX_NODES]` with an index into the next unused node is simpler and less invasive. The global override is useful when existing code allocates multiple types, while `BumpAllocator<T>` can be attached to selected containers. The two interfaces below share one buffer; a solution may keep only the interface it uses. This contest-oriented snippet does not replace other allocation forms such as `new[]`, nothrow `new`, or over-aligned allocation.

**Time Complexity:**  $O(1)$  per allocation.

**Space Complexity:**  $O(\text{BUFFER\_SIZE})$  static memory.

```

#include <cassert>
#include <cstddef>
#include <vector>

static constexpr std::size_t BUFFER_SIZE = 64 << 20; // 64 MiB; adjust to the memory limit.
alignas(std::max_align_t) static unsigned char buffer[BUFFER_SIZE];
static std::size_t buffer_pos = BUFFER_SIZE;

void *bump_alloc(std::size_t size, std::size_t align) {
    if (size == 0) {
        size = 1;
    }
    assert(size <= buffer_pos);
    buffer_pos = (buffer_pos - size) & ~(align - 1);
    return buffer + buffer_pos;
}

void *operator new(std::size_t size) { return bump_alloc(size, alignof(std::max_align_t)); }
void operator delete(void *) noexcept {}
void operator delete(void *, std::size_t) noexcept {}

template<typename T>
struct BumpAllocator {
    using value_type = T;

    BumpAllocator() = default;

    template<typename U>
    BumpAllocator(const BumpAllocator<U> &) {}

    T *allocate(std::size_t n) {
        static_assert(alignof(T) <= alignof(std::max_align_t), "over-aligned types are not supported");
        assert(n <= BUFFER_SIZE / sizeof(T));
        return reinterpret_cast<T *>(bump_alloc(n * sizeof(T), alignof(T)));
    }

    void deallocate(T *, std::size_t) {}
    template<typename U> bool operator==(const BumpAllocator<U> &) const { return true; }
    template<typename U> bool operator!=(const BumpAllocator<U> &) const { return false; }
};

```

## Example Usage

```

using namespace std;

struct Node {
    int value;
    Node *next;
};

int main() {
    Node *tail = new Node{2, nullptr};
    Node *head = new Node{1, tail};
    assert(head->value == 1 && head->next->value == 2);

    vector<int, BumpAllocator<int>> values{1, 2, 3, 4};
    assert((values == vector<int, BumpAllocator<int>>{1, 2, 3, 4}));
    return 0;
}

```